Landmark XVI-P — Combined release

(p23: O.1 requirement recorded, one-loop power counting, P.6 entry inventory (kappa^3 three-point, not evaluated); P.5 calculation renormalized at one loop kappa^2 for the two-point functions with QdS external legs; bulk symmetry not claimed; O.1 NOT_CLOSED)

XVI-P p23

2026-10-10

Contents

I Science **2**

1 p23: the O.1 requirement, one-loop power counting and the P.6 entry inventory **2**

2 p22: evanescent components (EV-M2), evanescent counterterms (EV-CT) and the run-time d guard **2**

3 Small release (p21): absolute D11 pin **4**

4 Correction release (p20): erratum to p19 **5**

5 P.5 identity step (p19): the projected one-loop κ^2 ch-identity defect **6**
5.1 Assembly . . . 6
5.2 Counterterm domain and solve . . . 7

6 P.5 calculation, step 4 (p18): M2 discharged, D11 evaluated, the full hh slot **9**
6.1 M2 (certificate `P18_D11_M2.json`) . . . 10
6.2 D11 (certificate `P18_D11.json`) and the full slot (`P18_hh_slot.json`) . . . 10

7 Scope of revision p17: the hh evanescent poles and the ch slot **11**
7.1 Where the hh self-energy enters the ch projection . . . 12
7.2 Decoupling (proof, exact in d) . . . 12
7.3 4D poles: coordinates and the transversality constraint . . . 12
7.4 Phase convention (G6) . . . 13
7.5 Review fixes G1–G4 . . . 13

8 Scope of revision p16: E2 analysis and Batch C **13**
8.1 E2: decoupling from the ch-projected slot (proof) . . . 13
8.2 Batch C: D01, D03 (covariant route) . . . 14
8.3 Review fixes F3–F5 and rebuild flags . . . 15

9 Scope of revision p15: first P.5 calculation **16**
9.1 Integral library and the scaleless rule . . . 16
9.2 Batch A: D02, D12, D14a–c . . . 16
9.3 Batch B: D13 (κ^2 K$'\cdot c$) . . . 16
9.4 Disclosed p14r contract revision . . . 17
9.5 Reproducibility . . . 17

10 Scope of revision p14 (repaired in p14r) **18**

11 P.3/P.4 frozen as P.5 inputs **18**

12 Milestones (p14 status) **19**

13 The quantum object **20**

14 Prescription and the D11 matching note **20**
14.1 Operational DRED rules (p14r) 21

15 Counterterm domain **22**

16 Executable amplitude rules **23**

17 P.5 entry dossier **24**

18 Reproducibility contract (p14r) **28**

19 Frozen classical record (P.3/P.4, unchanged since p13) **29**

20 Conventions **29**

21 C3 repaired: the boundary identity and its orientation **29**

22 Quartic generator: deterministic acceptance **30**

23 BRST from finite geometric transformations **30**

24 Source action **31**

25 Gauge-fixing map in transformed variables **31**

26 Master equation through $O(\kappa^2)$ by antifield number **31**

27 ch dependency audit and the enumeration lemma **32**

28 Acceptance propagation and release gate **32**

29 Inherited D11 context (o70-sync, read-only) **33**

30 Massless scalar matter-loop subsector (o70-sync) **33**
30.1 Gate 1: mixed scalar Noether (retained) 33
30.2 Gate 2: normalized D11 — prefactor Constructed; tensor Constructed 33
30.3 Gate 3 / Deliverables 1–3 33
30.4 Gate 4 / Deliverable 4: contact moments 34

31 Fixed scalar extension u_μ^R (o70-sync) **34**

32 Track II: insertion status freeze **35**

II Appendix 36

33 Appendix overview **36**

34 Release status / acceptance certificate (p23) **36**

35 Milestone ledger (p23) **69**

36 Test results (p23) **70**

37 O.1 requirement, one-loop power counting, P.6 entry inventory (p23) **74**

38 p22 layer record (release gate p22, revised in p23) **82**

39 Evanescent components: EV-M2 statement and evanescent counterterms (p22) **112**

40 p21 layer record (release gate p21, revised in p22) **115**

41 D11 pin: ratio (p20) and absolute sign and magnitude (derived in p21) **141**

42 p20 layer record (release gate p20, revised in p21) **144**

43 P.5 identity: projected ch-identity defect, counterterm solve, full closure (p19 layer, corrected in p20) **167**

44 p19 layer record (release gate p19, revised in p20) **178**

45 Milestone ledger (p19 layer) **197**

46 p18 layer record (release gate p18) **198**

47 Milestone ledger (p18 layer) **212**

48 P.5 calculation: D11 matching premise M2 (discharge; p18) **213**

49 P.5 calculation: D11 evaluated under the declared prescription (p18) **215**

50 P.5 calculation: full hh slot D01+D02+D03+D11+D12 vs the ch projection (p18) **218**

51 p17 layer record (release gate p17) **222**

52 Milestone ledger (p17 layer) **234**

53 P.5 calculation: D01/D03 evanescent poles vs the ch-projected slot (decoupling proof; p17) **235**

54 p16 layer record (release gate p16) **240**

55 Milestone ledger (p16 layer) **250**

56 P.5 calculation: E2 evanescent decoupling proof (D13; p16) **251**

57 P.5 calculation: Batch C certificate (D01, D03; p16) **254**

58 p15 layer release record (Batch A, D13; p15 gate) **344**

59 Milestone ledger (p15) **352**

60 P.5 calculation: integral library certificate (p15) **353**

61 P.5 calculation: Batch A certificate (D02, D12, D14a-c; p15) **354**

62 P.5 calculation: Batch B certificate (D13; p15) **360**

63 p14r contract record (P.5 entry; contract gate) **373**

64 Closure release artifact of the p14r contract **379**

65 P.5 entry dossier (machine-readable) **380**

66 Milestone ledger (p14) **403**

67 P.5 entry: quantum object certificate **404**

68 P.5 entry: prescription and D11 matching certificate **408**

69 P.5 entry: counterterm domain certificate **412**

70 P.5 entry: amplitude rules certificate **415**

71 P.5 entry: operational DRED rules certificate (p14r rules; p15 R5 correction; p16 F3 hardening) **425**

72 Symbolic integrand D01 (p14r export; evaluated in p16) **427**

73 Symbolic integrand D03 (p14r export; evaluated in p16) **441**

74 Symbolic integrand D13 (p14r export; evaluated in p15) **451**

75 Symbolic integrand D14a (p14r export; evaluated in p15) **453**

76 Symbolic integrand D14b (p14r export; evaluated in p15) **453**

77 Symbolic integrand D14c (p14r export; evaluated in p15) **453**

78 Symbolic integrand D02 (p14r export; evaluated in p15) **453**

79 Symbolic integrand D12 (p14r export; evaluated in p15) **454**

80 Boundary identity certificate (C3) **455**

81 Quartic verification certificate **456**

82 BRST derivation certificate **460**

83 Source action certificate **464**

84 Gauge-fixing map certificate **465**

85 Master equation certificate **466**

86 ch source audit certificate **473**

87 ch dependency rows ledger **476**

88 Generated graph inventory **477**

89 V4_grav export **478**

90 ell4 conventions export **478**

91 Source: conventions.py **479**

92 Source: boundary_identity.py (C3) **481**

93 Source: jetpoly.py **484**

94 Source: certio.py (certificate serialization + invariants) **485**

95 Source: grav_noether.py **487**

96 Source: graded_jets.py **488**

97 Source: bv_model.py **493**

98 Source: brst_from_geometry.py **495**

99 Source: source_action.py **498**

100 Source: gauge_fixing_map.py **500**

101 Source: planewave_exact.py **505**

102 Source: series_eval.py **506**

103 Source: quartic.py **508**

104 Source: ell4_full_polynomial.py (builder library) **513**

105 Source: inverse_metric_series.py **517**

106 Source: master_kappa2.py **518**

107 Source: ch_source_audit.py **522**

108 Source: release.py (frozen p13 classical gate; forked verbatim by release_p14) **527**

109 Source: dossier.py (frozen p13 dossier library) **532**

110 Source: inherited.py (pinned inherited inputs) **536**

111 Source: tools/source_manifest.py (source set, source-only zip) **537**

112 Source: tools/rebuild_from_source.py (rebuild entrypoint) **540**

113 REBUILD_INSTRUCTIONS.md **547**

114 Source: tools/build.py **551**

115 Source: tools/manifest.py **572**

116 Source: p14_conventions.py (P.5 entry contract constants) **573**

117 Source: freeze_p13.py (p13 freeze F1-F3) **574**

118 Source: amplitude_rules.py **577**

119 Source: counterterm_domain.py **588**

120 Source: quantum_object.py **593**

121 Source: prescription_d11.py **596**

122 Source: dred.py (operational DRED rules, p14r; disclosed revisions: p15 structural R5 safeguard, p16 F3 hardening, p17 G2 guarded loader parse_d, p18 H3/H5 documentation only, p22 runtime d guard – additive block) 602

123 Source: hessian_route.py (W6 independent regeneration, p14r) 617

124 Source: release_p14.py (p14 release gate) 621

125 Source: dossier_p14.py (P.5 entry dossier) 627

126 Source: tests/test_p14.py 631

127 Source: p15_conventions.py (p15 constants, contract pins) 645

128 Source: integrals.py (p15 integral library, S0) 647

129 Source: batchA.py (p15 Batch A) 652

130 Source: d13.py (p15 Batch B: D13) 656

131 Source: release_p15.py (p15 release gate) 665

132 Source: tests/test_p15.py 669

133 Source: p16_conventions.py (p16 constants, version label, pins) 672

134 Source: covariant.py (p16 covariant route: exact covariant integrand, FP and PV loop routes) 674

135 Source: e2_evanescent.py (p16 E2 decoupling proof) 679

136 Source: batchC.py (p16 Batch C: D01, D03) 682

137 Source: release_p16.py (p16 release gate) 689

138 Source: tests/test_p16.py 694

139 Source: p17_conventions.py (p17 constants, version label, pins, disclosures) 699

140 Source: hh_slot.py (p17 D01/D03 evanescent poles vs the ch slot: proof, controls, mutations) 702

141 Source: release_p17.py (p17 release gate) 707

142 Source: tests/test_p17.py 712

143 Source: p18_conventions.py (p18 constants, version label, pins, disclosures) 717

144 Source: d11.py (p18 M2 discharge, D11 evaluation, full hh slot) 720

145 Source: release_p18.py (p18 release gate) 727

146 Source: tests/test_p18.py 733

147 Source: p19_conventions.py (p19 constants, version label, pins, disclosures) 740

148 Source: defect.py (p19 layer, disclosed p20 revision: K1 orientation fix, full sigma-0 closure, restoration redefined) 742

149 Source: release_p19.py (p19 layer gate, disclosed p20 revision) 756

150 Source: tests/test_p19.py 763

151 Source: p20_conventions.py (p20 constants, version label, pins, disclosures) 771

152 Source: d11_pin.py (p20 D11 pin; disclosed p21 revision: absolute pin derivation) 772

153 Source: release_p20.py (p20 layer gate, disclosed p21 revision) 777

154 Source: tests/test_p20.py 783

155 Source: p21_conventions.py (p21 constants, version label, pins, disclosures, scan allow-list) 789

156 Source: release_p21.py (p21 layer gate, disclosed p22 revision) 791

157 Source: tests/test_p21.py 795

158 Source: p22_conventions.py (p22 constants, version label, disclosures, guard plants) 802

159 Source: evanescent.py (p22 EV-M2 and evanescent counterterms) 805

160 Source: contract_registry.py (p22 registry of the post-p18 contract revision) 809

161 Source: release_p22.py (p22 layer gate, disclosed p23 revision) 811

162 Source: tests/test_p22.py 817

163 Source: p23_conventions.py (p23 constants, disclosures, pinned EV values, poly plants) 822

164 Source: power_counting.py (p23 O.1 requirement, power counting, P.6 entry inventory) 824

165 Source: release_p23.py (p23 release gate) 827

166 Source: tests/test_p23.py 832

167 Archival material (p22): evanescent M2/CT and runtime d guard, superseded by p23 836

168 Archival material (p21): small release, superseded by p22 837

169 Archival material (p20): correction of p19, superseded by p21 837

170 Archival material (p19): P.5 identity step, superseded by the p20 correction 838

171 Archival material (p18): M2 discharged, D11 evaluated, full hh slot, superseded by p19 839

172 Archival material (p17): D01/D03 evanescent poles vs the ch slot, superseded by p18 840

173 Archival material (p16): E2 decoupling proof and Batch C (D01, D03), superseded by p17 840

174 Archival material (p15): first P.5 calculation (Batch A, D13), superseded by p16 841

175 Archival material (p14r): the P.5-entry contract release (contract active in p15, p16) 842

176 Archival material (p14): superseded by the p14r repair 843

177 Archival material (p13): frozen classical P.3/P.4 inputs 843

178 Archival material (p12): NOT p14 evidence **844**

179 Archival material (p11): NOT p14 evidence **845**

180 Archival material (p10): NOT p14 evidence **845**

181 Archival material (p02–p09): NOT p14 evidence; superseded milestone wording **846**
181.1 P.1 — Freeze of the D11 distributional kernel . . . 846
181.1.1 Four prerequisites (conjunction) . . . 846
181.1.2 Reproducibility . . . 847
181.2 XVI-P supplemental proofs for P.1 premises . . . 847
181.2.1 Holomorphy of $F(\zeta) = (Q - i0)^{\zeta}$ near $\zeta = 0$, including the origin . . . 847
181.2.2 Common-prescription rules $(D)/(Q)/(P)$ on U . . . 848
181.3 Antibracket generator correspondence for s_0 . . . 848
181.3.1 Inherited bracket . . . 849
181.3.2 Which summand contributes . . . 849
181.3.3 Index convention . . . 849
181.4 P.2 — D11 linearized functional identity . . . 849
181.4.1 Definitions . . . 849
181.4.2 General variation formula . . . 850
181.4.3 Cancellation . . . 850
181.4.4 One-point / D12 obligation . . . 850
181.5 P-next.1 — $P_{\text{in}}/P_{\text{out}}$ and `P3_hh_identity` . . . 850
181.5.1 Split projections . . . 850
181.5.2 Named target `P3_hh_identity` . . . 851
181.5.3 Three bracket rows (by functional differentiation) . . . 851
181.5.4 Statuses . . . 851
181.5.5 P.3.3 interface stub . . . 851
181.5.6 Non-claims . . . 851
181.6 P.3 — Identity-driven domain (provisional) . . . 851
181.6.1 Projection P_{ret} by retained monomials . . . 852
181.6.2 Order-κ^2 expansion . . . 852
181.6.3 Repaired topology (vertices + propagator pairings) . . . 852
181.6.4 Compatibility and acceptance . . . 852
181.7 P-next.2 — Gauge-fixed interface and named graphs . . . 852
181.7.1 P.3.3 — Gauge-fixed / minimal-BV interface . . . 853
181.7.2 P.3.4 — Graphs for named *ch*-row coefficients only . . . 853
181.7.3 Blocked for P.4 . . . 853
181.7.4 Statuses . . . 853
181.8 P-next.3 — Classical prerequisites . . . 854
181.8.1 P.4.1 Conventions . . . 854
181.8.2 Exported vertices . . . 854
181.8.3 ℓ_4 indexed generator . . . 854
181.8.4 P.4.3 BRST / antifield order . . . 854
181.8.5 P.4.4 Master check (lite) . . . 854
181.8.6 P.4.5 Dependency rerun (lite) . . . 854
181.8.7 Statuses . . . 854
181.9 P-next.4 — Full ℓ_4 and classical master . . . 854
181.9.1 Full indexed ℓ_4 . . . 855
181.9.2 D02 availability . . . 855
181.9.3 Master table . . . 855
181.9.4 Statuses . . . 855
181.10 p09 — Evidence-and-closure release . . . 855
181.10.1 Quartic verification . . . 855

181.10.2 Source-dependent *ch* audit 855
181.10.3 Gauge-fixing map 856
181.10.4 Classical BV 856
181.10.5 Package 856
181.11 Scope, baseline, and four verdicts 856

Part I
Science

1 p23: the O.1 requirement, one-loop power counting and the P.6 entry inventory

O.1, exactly. The XVI-O ledger `O_quantum_target.json` (o70-sync; verbatim copy in `provenance/XVI-O`, sha pinned, read only) states the O.1 target: the one-loop κ^2 bulk two-point sector on S_{inv} (graviton, scalar, ghost, including the $\text{gh} = 0$ BV source seeds; frozen gauge fermion; $m = 0$). O.1 closes iff this sector closes under a *verified* retained quantum identity. Every checklist item must be earned: O.2 (classical κ^2 master, including grav ℓ_4), O.4 (inventory), O.5 (graphs evaluated), O.6 (identity verified) and O.7 (remainder cancellation / counterterm solution). The target must first be enlarged if the identity constrains more legs or orders at κ^2. *In XVI-P scope:* O.4, O.5, the projected part of O.6/O.7, and the enlarge rule. *Out of scope:* O.2 and the bulk ST4b identity, which belong to XVI-O/XVI-N and stay untouched. Hence O.1 is **not closable inside XVI-P**, and it stays NOT_CLOSED.

Exact one-loop power counting. Every interaction vertex of the frozen inventory carries $\kappa^{\text{legs}-2}$, with antifield sources counted as legs. This is checked vertex by vertex. A one-loop graph has $I = V$, so it is κ^E with E external legs. The one-loop identity at κ^2 is $(S^{(0)}, \Gamma_1^{(2)}) + (S^{(1)}, \Gamma_1^{(1)})$; the term $(S^{(2)}, \Gamma_1^{(0)})$ vanishes because $\Gamma_1^{(0)}$ is field independent. Only $E = 2$ and $E = 1$ enter. A mechanical enumeration of all one-loop 1PI graphs (declared propagators hh, $\phi\phi$, $c\bar{c}$) gives the following:

- for $E = 2$ it reproduces the frozen κ^2 inventory exactly (D01–D06, D11, D12, D13: 9/9);
- for $E = 1$ it gives the 3 graphs of the one-point row;
- it detects four mutations (a wrong κ power, two dropped vertices, a spurious propagator).

The enlarge rule is **not triggered**: there is no one-loop three-point sector at κ^2.

P.6 entry (honest partial). Three-point functions first appear at κ^3. The one-loop κ^3 three-point inventory has 15 graphs: hhh 5, $h\phi\phi$ 4, $c\bar{c}h$ 2, Kch 2, c^*cc 1, $\phi^*c\phi$ 1. This is a topology-level inventory only. The amplitude rules, the counterterm domain over $\mathbb{Q}(i)$ (including evanescent operators), the evaluation, the three-point defect and σ_0 closure are **not done** in p23, so no defect outcome is claimed.

Fixes from the p22 review. (i) Polynomial-domain refusal (p23 contract revision, one block appended to `dred.py`). From outside sympy, polynomial algebra that can evaluate d raises: `rem`, `prem`, `div`, `resultant`, `compose`, `Poly.shift`, `Poly.rem`, and similar. Internal renames of d are admitted only to a Dummy, so `Matrix.subs(d→y)` raises. A live probe runs 9 forms on 4 expressions, and rational-function algebra is unchanged. (ii) The gate now checks the values $\rho = 1/2$, E_2 coefficient $-1/16$, SE coefficient $+1/8$, and the summed h.h coefficients against both pinned values and the archived p22 certificate. (iii) The README title line carries "two-point functions with QdS external legs".

2 p22: evanescent components (EV-M2), evanescent counterterms (EV-CT) and the runtime d guard

Contract revision (disclosed). p22 carries one disclosed contract revision, listed in `p22_conventions.CONTRACT_REVISION`: (i) evanescent counterterms are admitted outside the p14r 4D domain, and (ii) `code/p5_entry/dred.py` gains one delimited additive block (the runtime d guard). Outside the block the file is byte-identical to the p18 release; `contract_registry.dred_revision` checks this, and the p22 gate pins the exact sha256 of the revised file.

EV-M2: the matching statement. XVI-O has no evanescent counterpart, so the eight evanescent ($\tilde{g}$) structures of D11 cannot be matched to XVI-O. We tested three candidate statements exactly, with d opaque: (a) *match a d-dimensional continuation of the XVI-O kernel* is refuted for both natural continuations ($g_4 \to \hat{g}$ and g_4 kept), because the exact D11 tensor differs from either continuation of its own 4D part on ˜external legs. XVI-O defines no other continuation, so (a) is not a usable statement. (b) *Pure local evanescent operators*: proven. Every evanescent pole (D01, D03, D11) is a polynomial in p and independent of d after pole extraction. (c) *Vanishing in the renormalised result*: proven in a stronger form. After subtraction, every evanescent component is identically zero on 4D (hatted) external legs at every d, not merely at $O(\epsilon)$. The scope is `D11_matching_verified_scope` = "4D components: M2 discharged (P18_D11_M2, unchanged); evanescent components: EV-M2 proven; NOT matched to XVI-O".

EV-CT: the counterterms. The coefficient is $-$(UV pole) in the $N_\epsilon \kappa^2/\epsilon$ normalisation of P14. There are three pieces:

- h.h operators for the eight $\tilde{g}$ structures, summed over D01+D03+D11: $g_4 \tilde{g}\,(p^2)^2$: $-7i/120$ (twice); $g_{4,mr}\tilde{g}_{ns}(p^2)^2$: $-i/160$; $\tilde{g}\tilde{g}\,(p^2)^2$: $i/160$ (twice); $\tilde{g}\,pp\,p^2$: $7i/120$ (twice), $i/160$. Per-graph values are in `certificates/P22_evanescent.json`.
- the K′.c operator for D13 E_2, with coefficient $-1/16$;
- its ghost-equation partner $\bar{c}\,\Sigma_{\rm SE}\,c$, with coefficient $+1/8$.

Closure. σ_0 of the counterterms is zero in every sector, so full closure holds with the evanescent counterterms included. In h.c, the per-graph $p \cdot T$ of D01 and D03 cancel in the sum, D11 is transverse, and $E \cdot E_2 = 0$. In c.b, the ghost equation with $\rho = 1/2$ (calibrated on the 4D pair) holds exactly in d for E_2 and $\Sigma_{\rm SE}$. c.cbs is unreachable. **Identity.** The h.h slot values of the counterterm sum are zero in all six external modes, and the K.c slot values are zero, so the projected identity is unchanged. **Remnants.** The finite evanescent parts are nonzero, but they vanish identically on hatted external legs. With QdS external legs there is no $O(\epsilon)\times$pole effect. If external traces are taken with g_4 (Q4S), a finite term $7i\,p^2(\hat{a}\cdot\hat{b}\,p^2 - \hat{a}\cdot p\,\hat{b}\cdot p)/60$ arises. It is disclosed as a scheme choice. **Mutations.** Each of the following is detected: dropping the D01 or D03 counterterm; doubling D03; dropping the SE partner; flipping the sign of the E_2 counterterm; and putting a trace operator in place of $s_0 h$. **Status.** P.5 calculation status is *renormalized* (one loop, κ^2, the two-point functions hh / K′.c / $\bar{c}$c, MS-bar in DRED with the p22 evanescent counterterms, external legs in QdS). It does not cover higher points, two loops, bulk symmetry or O.1.

Runtime d guard. The guard intercepts the following calls whenever the caller is outside sympy/mpmath/dred: `subs`, `xreplace` (including on a bare d), `replace`, `evalf/n`, the Poly value and coefficient accessors, `coeff`, `as_independent`, `as_coeff_exponent`, `series`, `collect`, `limit`, `Lambda`, and $d == d$. Any of them that would give d a value raises `PrematureD4Substitution`. Authorised paths (`substitute_d`, `authorised_d`) still work. A live probe in the gate runs 30 forms on 4 expressions: every reviewer form from p21, the power-base hole, alias and free_symbols lookups, `Symbol.__new__` and `Function` calls. All of them raise, and ordinary algebra is unchanged. Some bypasses cannot be intercepted at runtime: identity (`is`) tests, string round-trips and `Poly.rep`. These are disclosed and covered only by the secondary static allow-list scan.

Gate. The tools/ and notes prefix exemptions are removed. Every changed, added or removed file relative to the p21 zip must be listed in `DISCLOSED_REVISIONS_P22`. The only exceptions are the generated reference values and the pinned archival/p21 records. Earlier-layer gates accept the registered p22 revisions only through `contract_registry`.

3 Small release (p21): absolute D11 pin

Scope.
p21 is a small release from p20 with no scope expansion. It follows the findings of the p20 independent rebuild (PASS; report sha256 e8f358f9…, bound to tree 2c0aa153… and zip ca32ce01…; same box, not a second machine). No contract revision; the p15–p19 layers are as in p20; the disclosed p21 revisions of the p20 layer are `d11_pin.py` and `release_p20.py`. bulk_symmetry_restored is false regardless. O.1 stays NOT_CLOSED.

Absolute D11 pin (p20: OPEN). The sign and magnitude of the D11 UV pole are derived from $\frac{1}{2}\mathrm{Tr}\ln$ of the minimal scalar loop under the package's declared conventions, each checked verbatim in the code:

- $G = \Gamma^{(2)}$: the amplitude rules declare $i\Gamma_1 = \int \mathrm{pref}\, N/\prod(q^2 + i0)$ and G6 declares $G = \kappa^2 N_\epsilon \hat{G}$, $\hat{G} = -i\,\mathrm{pref}/\kappa^2 \int N/(D_1 D_2)$; hence $G = -i \cdot i\Gamma = \Gamma$.
- Wick direction: $N_\epsilon = iS_\epsilon/(16\pi^2)$ and the certified bubble pole $\hat{I}(1,1) = +1/\epsilon$ give $d^d l_M = +i\, d^d l_E$; with $\eta = (+,-,-,-)$ and $S_\phi = \frac{1}{2}\int \sqrt{-g}\, g^{\mu\nu}\partial\phi\partial\phi$ this gives $iS_M = -S_E$, so $L_M^{\mathrm{eff}} = -L_E^{\mathrm{eff}}$.
- Heat kernel (Euclidean, $d = 4 - 2\epsilon$): $L_{E,\mathrm{div}} = -a_2/(32\pi^2\epsilon)$, so $L_{M,\mathrm{div}} = +a_2/(32\pi^2\epsilon)$ with $a_2 = C^2/120 + R^2/72$ ($\xi = 0$).
- With $g = \eta + \kappa h$ the Minkowski quadratic forms are $C^2 \to \frac{1}{2}p^4 h P_2 h$ and $R^2 \to 3p^4 h P_0 h$.

Result: $\Gamma^{(2)}|_{\mathrm{pole}} = \kappa^2/(16\pi^2\epsilon)\,[p^4 P_2/240 + p^4 P_0/24]$. This equals the package pole $\kappa^2 N_\epsilon \hat{G}/\epsilon$ with $c_2 = -i/240$, $c_0 = -i/24$ exactly (ratio 1 for both spins). No convention choice is external; the external inputs are mathematics only (the a_2 coefficients and the heat-kernel pole formula). A $\times 2$ normalisation error and a sign flip keep the 10:1 ratio but are detected by the absolute pin.

Gate. The top-level `absolute_pin_status` and `ratio_pin_status` (the keys typeset in this PDF) must equal the nested statuses. Run standalone, the p20 gate re-checks the p18 layer (quantum inputs earned) and the graph inventory instead of trusting the P19 layer record; the p21 gate re-checks all lower layers.

Scan. The enumerated pattern list is complemented by an allow-list scan (`tests/test_p21.py`): expression-tree decomposition (`.args`, `make_args`, `as_ordered_terms`, …), sympy callable aliases, computed symbol names and value calls mentioning d are refused everywhere except at reviewed sites. All eight forms reported by the reviewer are caught. A runtime check confirms that the stored exact-d D11 amplitudes re-parse with the opaque d and reproduce their poles only through `substitute_d`. It also found a limitation: `expr.subs(d, v)` is not intercepted when d occurs only as a power base. The contract is not changed in p21; no package value path uses it, and the static rule refuses it. This is a check, not a proof.

Disclosure. The p20 edit of `manuscript/notes/P19_defect.tex` is now listed in `DISCLOSED_MANUSCRIPT_REVISIONS`. The reference key `/p19/change_log_vs_p18`, which was recomputed on the current tree, is renamed `p19_gate_change_log_vs_p18_recomputed_on_current_tree`. The log as released stays in the pinned `archival/p19/reference_values_p19.json`.

4 Correction release (p20): erratum to p19

Scope.
p20 is a correction release of p19 with no scope expansion. It follows the findings of the p19 independent rebuild (PASS with findings; report sha256 1b01f1d8…, bound to tree fcf90354… and zip da924275…; same box, not a second machine). There is no contract revision, and the p15–p18 layers are byte-identical. The disclosed p20 revisions of the p19 layer are `defect.py` and `release_p19.py`. bulk_symmetry_restored is false regardless. O.1 stays NOT_CLOSED; the insertion stays frozen; XV-B and XIV-J are unopened; Route B is not discharged; no $c\,\delta$ is added to u_μ^R.

Erratum K1 (the p19 text is retracted). p19 stated that full σ_0 closure of the MS-bar counterterm fails with the derived ghost-equation factor $+1$, that it needs -1, and that this was unresolved. The statement is wrong; it came from a Grassmann-orientation bookkeeping error.

- The route-G densities are generated in the order (c,cb). The P15 ghost self-energy coordinates Σ use the (cb,c) orientation, and $V[\text{c.cb}\#k] = -V[(\text{cb},\text{c})\#k]$. p20 measures this ratio (via `pw_vertex`) as -1 for $k = 0, 1$.
- In route-G (c,cb) coordinates, Σ has pole $(+3/8, -1/4)$ and local part $(2/3, -5/12)$.
- The derived factor is $+1$ (the ghost equation $\Sigma = 2iD[G_{Kc}]$ reproduces the P15 pole). With it, the *complete* MS-bar counterterm $S_{ct} = -(\text{h.h} + \text{c.K} + \text{c.cb poles})$ is σ_0-closed in every ghost $+1$ sector (h.c, c.b, c.cbs).
- Without the c.cb term, or with Σ unconverted (the p19 error), $\sigma_0(S_{ct})$ has 88 entries in the c.b sector.
- Tree-level controls: $V_{S_0}[\text{cb},\text{c}] = 2iD\,V_{S_0}[K, c]$, and $\sigma_0(S_0^{Kc} + S_0^{\bar{c}c}) = 0$ (the S_0^{Kc} part alone has a nonzero image).
- In quotient coordinates, the c.K-only data have c.cb components $(-3/8, 1/4)$, which cancel against $\Sigma = (+3/8, -1/4)$. The full data including Σ have quotient coordinates 0.

Restoration redefined. The p19 flag (renormalised $P_{ch}\sigma_0(S_{ct}) = 0$, $x = 0$) follows from the identity and locality, so it is not independent. In p20, `projected_restoration_verified` means full σ_0 closure of the complete one-loop κ^2 MS-bar S_{ct} (including c.cb, Σ held fixed) in all ghost $+1$ sectors, together with the projected identity. Scope: that counterterm only; not bulk symmetry, not O.1. The corrected computation closes, so the flag is earned. The p19 check is kept as a labelled route-consistency control, and it holds.

Blind spots. The full-closure test acts as a mutation detector, with Σ held fixed:

- drop D13 and flip K.c are detected (c.b, 88 entries);
- drop D01, drop D03, flip D03 and D03 $\times 2$ are detected (h.c, 280 entries);
- the p19 orientation error is detected;
- drop D11 is not detected: D11 is transverse, so its pole is closed by itself.

D11 normalisation pin (`P20_D11_pin.json`).

- *Derived from package data.* The 4D UV pole of D11 (`P18_D11.json`, package orbit convention; transverse; only spin-2 and spin-0s parts) has Barnes–Rivers coefficients $c_2 = -i/240$ and $c_0 = -i/24$, so $c_0 : c_2 = 10$. The quadratic forms on $g = \eta + h$ are derived here from linearised curvature: $C^2 \to \frac{1}{2}p^4 h P_2 h$, $R^2 \to 3p^4 h P_0^s h$, and Gauss–Bonnet $\to 0$.

- *External literature.* The a_2 coefficient of a minimally coupled real scalar is $\frac{1}{120}C^2+\frac{1}{72}R^2$ (Birrell–Davies §6.2; 't Hooft–Veltman 1974). This predicts $\frac{1}{240} : \frac{1}{24} = 1 : 10$.
- *Ratio pin: PASS.* Drop D11 gives $c_2 = c_0 = 0$ and is detected, and so is a swap of the two gg structures.
- *Absolute pin: OPEN* [**superseded in p21: the absolute pin was derived and verified, sign and magnitude (P21, section 1; certificates/P20_D11_pin.json, absolute_pin_status PASS); the text below is the p20 statement, kept for the record**]. The magnitudes agree up to a common factor $-i$, but the map from $\hat{G}$ to a_2 (phase, N_ϵ, κ normalisation, sign of Γ) is not derived. An overall $\times 2$ error is caught only by this open pin and by M2 (P18).

J1 wording. The $\mathbb{Q}(i)$ counts are the real dimensions under a compatible real structure. The earlier wording, "the physical field", is replaced.

Scan and gate. The static scan is still a pattern check, not a proof. It now also flags:

- Add(*listcomp over e.args);
- an aliased Poly import;
- Poly.___call___;
- as_coefficients_dict;
- a functools.reduce rebuild;
- Symbol(name=...);
- an explicit-constructor rebuild.

A standalone reject of the p19 or p20 gate no longer leaves a stale RELEASE_STATUS.json.

Status. independent_rebuild_passed_p19 is true (bound to the p19 tree and zip; same box). The p20 independent rebuild is pending.

5 P.5 identity step (p19): the projected one-loop κ^2 ch-identity defect

Scope.
p19 keeps the p14r contract and the p15–p18 calculation layers byte-identical (`release_p19.py` recomputes their sha256). It assembles the ch projection (P.3, frozen) of the $\hbar\kappa^2$ part of (S_Ψ, Γ_Ψ) from every evaluated input, exactly in d. It then solves for a symmetry-restoring counterterm in the declared domain. Only the ch projection is covered (one c, one h; one loop; κ^2; off shell). bulk_symmetry_restored is false. O.1 stays NOT_CLOSED; the insertion stays frozen; XV-B and XIV-J are unopened; Route B is not discharged; no $c\,\delta$ is added to u^R_μ.

5.1 Assembly

The ledger `P13_ch_dependency_rows.json` has three surviving rows:

- R_{hh}, $(S_0, \Gamma_1^{[2]})$ $s_2K{:}K.c$, inputs D01+D02+D03+D11+D12, with $X_{hh} = \int (s_0 h)\, G_{hh}\, h$;
- R_{Kc}, $(S_0, \Gamma_1^{[2]})$ $s_1h{:}h^2$, input D13, with $X_{Kc} = \int G_{Kc}\, c\, E\, h$, where E is the h^2-monomial operator of S_0;
- R_{1pt}, $(S_1, \Gamma_1^{[1]})$, inputs D14a–c.

Every input is evaluated; none is Missing, and a Missing input is refused rather than set to zero (control). The defect is computed for seven external assignments, including "c hat, h tilde". For each it is given as the exact d-dependent value times MB, its pole, its coefficient of L and its MS-bar local part.

Result: the defect is zero, exactly in d, in every part and every assignment. Why it is zero. Each surviving row vanishes separately in the projected slot. No counterterm cancellation is involved, and no cancellation between rows.

(i) hh row: full-row transverse decoupling. D01 and D03 are each nonzero in the slot and cancel each other (the p17/p18 transverse sum). D11 is transverse by itself, and D02 = D12 = 0.

(ii) K.c row: D13 reaches the ch slot only through T_1 and E_1 (T_2, T_3 and E_2 are longitudinal; E is transverse), and $A_{T_1} = A_{E_1} = 0$ exactly in d.

(iii) One-point row: the tadpoles D14a–c vanish under the declared prescription.

This rests on:

- M2 / the D11 matching, verified on the *4D components only* (the D11 matching for the evanescent components is not applicable and not verified);
- the evanescent decoupling proofs (P16 E2; P17/P18 hh evanescent and full-row decoupling);
- the frozen P.3 projection and the P13 ledger;
- the declared prescription.

Evanescent decoupling in the assembly. The evanescent parts of both rows were assembled explicitly and are zero for all seven assignments. The evanescent poles themselves are nonzero; they are regulated and not subtracted.

5.2 Counterterm domain and solve

Route G applies $\sigma_0 = (S_0, \cdot)$ to the 11 p14r domain densities: h.h#0–4, c.cb#0–1, c.K#0–2, $\phi.\phi$. Route C computes the covariant ch images exactly in d. Both give rank $P_{ch}\sigma_0 = 3$.

- Over $\mathbb{Q}(i)$ (these counts are the real dimensions under a compatible real structure; wording corrected in p20): rank $\sigma_0|_D = 5$, $\dim D \cap \ker \sigma_0 = 6 = \dim D \cap \operatorname{Im} \sigma_0$, cohomology 0, and the quotient has dimension 5 (complement h.h#0–2, c.cb#0–1).
- The h.h + c.K Γ_1 data, without the ghost self-energy and in route-G (c,cb) labels, have quotient coordinates pole $(0, 0, 0, -3/8, 1/4)$, coefficient of L $(0, 0, 0, 3/8, -1/4)$ and local part $(0, 0, 0, -2/3, 5/12)$. The h.h pole is transverse, i.e. σ_0-exact. These coordinates equal $-[\Sigma]$: Σ itself has $(+3/8, -1/4)$, and the data including Σ have quotient 0 [clarified in p20].
- The linear solve $\Delta + P_{ch}\sigma_0(x) = 0$ gives $x = 0$ (outcome ZERO); no residual and no obstruction.
- The MS-bar counterterm has zero ch image. The h.h + c.K part alone has a nonzero full σ_0 image in the c.b sector; adding the c.cb ghost self-energy counterterm in the correct orientation closes it [corrected in p20, see the p20 erratum note].

Flags and their definitions.

- `projected_defect_computed`: Δ is assembled from all evaluated inputs, exactly in d, in all parts and all seven assignments.
- `projected_identity_verified`: $\Delta = 0$ for the *regulated* Γ_1.

- `projected_restoration_verified` [REDEFINED in p20]: full σ_0 closure, in every ghost +1 sector (h.c, c.b, c.cbs), of the *complete* one-loop κ^2 MS-bar counterterm S_{ct} (h.h + c.K + c.cb, ghost self-energy held fixed, derived relative factor +1, orientation converted). The p19 definition ($P_{ch}\sigma_0(S_{ct}) = 0$, $x = 0$) follows from the identity and locality; it is kept only as a labelled route-consistency control.

Scope of the restoration flag: the one-loop κ^2 counterterm only; not bulk symmetry, not O.1. The release gate re-checks each definition from the certificate content. bulk_symmetry_restored is false regardless.

K1 [corrected in p20; the p19 text is retracted]. p19 reported that full closure of the MS-bar counterterm fails with the derived ghost-equation factor +1 and needs −1. That was a Grassmann-orientation bookkeeping error: the route-G densities are built in the order (c,cb), whereas the P15 Σ coordinates are (cb,c)-oriented, and $V[\text{c.cb}\#k] = -V[(\text{cb},\text{c})\#k]$. With the measured ratio −1 applied, the derived factor +1 gives full closure (p20 erratum note). The relations below are stated in the (cb,c) orientation:

- The ghost-equation operator $2iD$, verified by P15 as $\Sigma = 2iD[G_{Kc}]$, gives $2iD\,V[\text{c.K}\#1] = -V[(\text{cb},\text{c})\#0]$ and $2iD\,V[\text{c.K}\#2] = -V[(\text{cb},\text{c})\#1]$.
- Applied to the c.K pole, this reproduces the P15 Σ pole $(-3/8, 1/4)$ in (cb,c) coordinates, i.e. $(+3/8, -1/4)$ in the route-G (c,cb) coordinates.

J1: finding about the p14r contract (contract not modified). The p14r counterterm-domain certificate counts Im σ_0 3, quotient 8, $D \cap \ker$ 5 and rank $P_{ch}\sigma_0$ 4. These are artefacts of a real/imaginary split over $\mathbb{Q}$: the densities carry an explicit i for odd derivative counts, so an i-multiple counts as independent. Over $\mathbb{Q}(i)$ the counts are 6, 5, 6 and 3 (the real dimensions under a compatible real structure), and Route C agrees. With the split, p19 reproduces the p14r counts (control).

Controls and mutations. All controls are true. Eight mutations each change the outcome:

- drop D03; drop D01; flip the D03 phase; double D03; trace operator instead of $s_0 h$; de Donder $q^2 P$ instead of E; drop the D03 evanescent part (naive 4D). Each gives OBSTRUCTION (nonlocal residual).
- D13 with G_t dropped gives NONZERO but CANCELLABLE (rank 3).

Recorded blind spots of the projected identity: dropping D11 (transverse by itself), dropping D13 ($A_{T_1} = A_{E_1} = 0$) and flipping the K.c row sign (each row vanishes separately). In p20, the full-closure test detects the last two, and the D11 normalisation pin covers the first. Removing D11 from the inventory makes the inventory incomplete, and projected_defect_computed is then not earned.

assignment (c, h)	defect pole	coeff. of L	local finite
`Q4S (declared, R2)`	0	0	0
`QdS (hat)`	0	0	0
`c tilde, h hat`	0	0	0
`c tilde, h mixed`	0	0	0
`c hat, h mixed`	0	0	0
`all tilde`	0	0	0
`c hat, h tilde`	0	0	0

Route G over `Q(i)`: rank $\sigma_0|_D = 5$, dim $D \cap \ker\sigma_0 = 6$, dim $D \cap \mathrm{Im}\,\sigma_0 = 6$, cohomology 0, quotient 5 (complement `h.h#0, h.h#1, h.h#2, c.cb#0, c.cb#1`); rank $P_{ch}\sigma_0 = 3$ (Route C: 3). Over Q with the re/im split (p14r convention): dim $D \cap \mathrm{Im} = 3$, quotient 8, dim $D \cap \ker = 5$.
Quotient coordinates of the Γ_1 local data: pole `['0', '0', '0', '-3/8', '1/4']`, coeff. of L `['0', '0', '0', '3/8', '-1/4']`, local `['0', '0', '0', '-2/3', '5/12']`.

mutation	outcome	changes
`drop_graph_D03`	`OBSTRUCTION (nonlocal residual: the L-coeffi`	yes
`drop_graph_D01`	`OBSTRUCTION (nonlocal residual: the L-coeffi`	yes
`flip_phase_D03`	`OBSTRUCTION (nonlocal residual: the L-coeffi`	yes
`symmetry_factor_D03_x2`	`OBSTRUCTION (nonlocal residual: the L-coeffi`	yes
`wrong_projection_trace_operator_hh`	`OBSTRUCTION (nonlocal residual: the L-coeffi`	yes
`wrong_projection_gauge_fixed_operator_KK`	`OBSTRUCTION (nonlocal residual: the L-coeffi`	yes
`naive_4D_D03_evanescent_dropped`	`OBSTRUCTION (nonlocal residual: the L-coeffi`	yes
`naive_4D_D13_Gt_dropped`	`NONZERO, CANCELLABLE by a counterterm in the`	yes

Counterterm solve: ZERO; x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite renormalisations are ker(P_ch sigma_0) cap D (dim 8), of which 2 modulo D cap Im sigma_0.
Why zero. each surviving ch row vanishes separately in the projected slot; no counterterm cancellation and no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupling - D01 and D03 are individually nonzero and cancel against each other in the slot (p17/p18 transverse sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only through T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-point row: the tadpoles D14a-c vanish under the declared prescription. The defect is therefore zero without any counterterm (solve x = 0).
Flag definitions. **projected_defect_computed**: Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from every evaluated input of the three surviving rows (none Missing), exact in d, pole + nonlocal + MS-bar local parts, all seven external assignments. **projected_identity_verified**: Delta = 0 for the REGULATED one-loop functional Gamma1: pole, nonlocal and local parts vanish for all seven assignments, exactly in d, with all controls true and every mutation changing the outcome. **projected_restoration_verified**: p20 definition (replaces p19's, which follows from the identity + locality): FULL sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm S_ct = -(pole) in the declared domain, h.h + c.K + c.cb (ghost self-energy Sigma held fixed at its P15 value, orientation converted, derived relative factor +1), in every ghost +1 sector (h.c, c.b, c.cbs), together with projected_identity_verified. Scope: the one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1. **bulk_symmetry_restored**: hard stop: False regardless (not claimed; O.1 NOT_CLOSED).
K1 (corrected in p20; erratum to p19). derived relative factor 1; orientation ratio (route-G (c,cb) over P15 (cb,c)) ['-1', '-1']; Sigma pole in (c,cb) coordinates ['3/8', '-1/4']; full sigma_0 image of the complete MS-bar counterterm: {'closed': True, 'entries': 0, 'sectors': []}. ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolved' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots). Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild).
J1 (finding about the p14r contract). FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the p14r counterterm-domain counts (Im sigma_0 3, quotient 8, D cap ker 5, P_ch sigma_0 rank 4) are artefacts of a re/im split over Q (the densities carry explicit i for odd derivative counts, so i-multiples count as independent). The counts over Q(i) - sigma_0 rank 5, D cap ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch sigma_0 rank 3 (= Route C) - equal the real dimensions under a compatible real structure. The p14r certificate stays as released; both are reported

6 P.5 calculation, step 4 (p18): M2 discharged, D11 evaluated, the full hh slot

Scope.

p18 keeps the p14r contract and the p15, p16 and p17 calculation layers byte-identical (release_p18.py recomputes their sha256). It discharges the D11 matching premise M2 on the 4D components, evaluates D11 under the declared prescription, and rechecks the ch-projected hh slot with D11 included. quantum_inputs_evaluated is earned (every graph of the inventory is evaluated in one prescription). projected_identity_verified, projected_restoration_verified and bulk_symmetry_restored are NOT earned. O.1 stays NOT_CLOSED; the insertion stays frozen; XV-B and XIV-J are unopened; Route B is not discharged; no $c\,\delta$ is added to u^R_μ.

6.1 M2 (certificate `P18_D11_M2.json`)

M2 states that on $U = \mathbb{R}^4 \setminus \{0\}$ the inverse Fourier transform of P^{DR}_{AB} equals $R_A R_B \, \Pi^R_{\rm tr}$, with $\Pi^R_{\rm tr} = (i\kappa^2/4)\, K u^R_\mu$, $K = \alpha D_2 + \beta D_0$, $\alpha = A^2/60$, $\beta = A^2/18$ and $A = 1/4\pi^2$.

- *Route B (exact; primary).* Write $P^{DR} = \kappa^2 N_\varepsilon \hat{G}$ with $\hat{G} = \sum_s A_s(d) S_s \, \mathrm{MB}$ and $\mathrm{MB} = 1/\varepsilon + 2 - L$. At $d \to 4$ its nonlocal part is $-(i\kappa^2/16\pi^2) L \sum_s A_s(4) S_s(p)$; everything else is polynomial. On the other side, $\mathcal{F}[u^R] = c_{\rm FT} \log(-p^2 - i0) + \text{const}$, and $K \to \hat{K}(p)$ with $\hat{P}_{mn} = p_m p_n - \eta_{mn} p^2$. This operator is fourth order, so the sign convention of the transform drops out. M2 then holds iff, for each of the 100 component pairs, $X_{AB} = \sum_s A_s(4) S_s|_{AB} = -4\pi^2 c_{\rm FT} A^2 \, Y_{AB} = -(i/4)\, Y_{AB}$, where $Y = R_A R_B (\hat{D}_2/60 + \hat{D}_0/18)$. This holds exactly on all 100 pairs. The difference then has a polynomial Fourier transform, so it is supported at $\{0\}$, and μ enters only through polynomials.

- $c_{\rm FT}$*, calibrated twice.* (a) From the library DR bubble, $\mathcal{F}[D_F^2] = -N_\varepsilon \mathrm{MB}$ and $D_F^2 = A^2 u$. (b) As $-r$, with the XVI-O residue $r = -i\pi^2$. Both give $c_{\rm FT} = i\pi^2$.

- *Route A (structural corroboration).* M1 is re-run: the P integrand equals the Fourier convolution of the XVI-O product $T = VV(CC + CC)$ (100/100). The frozen XVI-O code, imported read-only, re-derives α, β and verifies $T = Ku$ on U for all 256 components. A regularised convolution agrees with the pointwise product on U (cited: XVI-O lem:BV_rules_on_U, Constructed).

- *Controls and mutations.* D11 is transverse exactly in d; the FP and PV loop routes agree. Six mutations are each rejected: $\alpha \leftrightarrow \beta$, off-diagonal weight 1, flipped FT sign, doubled symmetry factor, D01 in place of D11, and $\beta = 0$.

- *Scope.* The flag is qualified as D11_matching_verified_scope = "4D components". M2 is shown on the 4D components only. The evanescent components are the 8 $\tilde{g}$-carrying structures, i.e. components with an index pair in the ε-scalar directions. Their status is not_applicable_not_verified, because XVI-O is strictly 4D and has no counterpart. The evanescent structures of D11 have nonzero L-coefficients (8, recorded); they are ε-scalar components outside the 4D statement.

- *Consequence (M3/M4).* M3 is now unconditional on the 4D components: $L_{\rm match}$ is supported at 0. It is transverse and homogeneous of degree four, so it lies in the 2-dimensional transverse h·h subspace. Its coordinates need the XVI-O finite constant of u^R and are not computed.

6.2 D11 (certificate `P18_D11.json`) and the full slot (`P18_hh_slot.json`)

- *Prescription.* Operational DRED R1–R6 with opaque d, MS-bar, Feynman $+i0$, N4 contact exclusions, M3 bounds 4/8/4, exactly-degree-four normalisation. Both points are checked per structure: every structure has momentum degree 4, so the order-0/2 contacts excluded by N4 do not arise.

- *Inventory.* quantum_inputs_evaluated is earned. Every graph of the three surviving ch rows is evaluated, and none is Missing: hh D01, D02, D03, D11, D12; K·c D13; one-point D14a–c.

- *Evaluation.* 14 structures, 6 of them 4D. The covariant form is exact and unique; the FP and PV routes agree exactly in d; the result is exchange symmetric. The nonlocal coefficient equals minus the pole. Premature $d = 4$ is refused and would shift the local parts. Each of the four mutations is rejected.

- *4D pole.* $z_{D11} = i(-1/160, -1/480, 1/80, 1/240, -1/120) = i(\frac{1}{240} k_1 - \frac{1}{120} k_2)$, which is transverse.

- *Full row.* $p_m G^{(mn)(rs)}_{D11} = 0$ holds identically in d, for the 4D and the evanescent parts. D11 therefore decouples from the slot by itself; D01 and D03 decouple only as a sum.

  - $X_{D01+D03+D11} = 0$ for all seven external assignments. This includes $\hat{c}, \tilde{h}$ (H2), where $X_{D01} = -X_{D03} = p^4 \, \tilde{a}\cdot\tilde{b} \, \hat{c}\cdot p/16$.
  - No evanescent structure survives, so D11 does not reopen the evanescent question and no counterterm revision is needed.

– The summed 4D pole is $i(\frac{41}{80}k_1 - \frac{43}{120}k_2)$, which is transverse.

– Caveat: D11 phase and symmetry-factor mutations are invisible to the slot. They are tested by M2 instead.

– The summed evanescent poles of the hh Green function (8 structures) are nonzero but transverse. Renormalising that Green function itself is outside the projection and not claimed.

M2: `DISCHARGED (4D components)`; route B: 100 component pairs, 0 mismatches, constant `-I/4`; c_FT = `I*pi^2` (DR bubble) = `I*pi^2` (-r).

D11 structure	evan.	UV pole	coeff. of L	local (MS-bar)	shift (d=4 first)
g4_mn g4_rs (p.p)^2	no	-I/80	I/80	-I/75	7*I/600
g4_mn gt_rs (p.p)^2	yes	-I/120	I/120	-77*I/3600	-17*I/3600
g4_mn p_r p_s (p.p)^1	no	I/80	-I/80	I/75	-7*I/600
g4_mr g4_ns (p.p)^2	no	-I/480	I/480	-23*I/3600	-I/450
g4_mr gt_ns (p.p)^2	yes	I/480	-I/480	23*I/3600	I/450
g4_mr p_n p_s (p.p)^1	no	I/480	-I/480	23*I/3600	I/450
g4_rs gt_mn (p.p)^2	yes	-I/120	I/120	-77*I/3600	-17*I/3600
g4_rs p_m p_n (p.p)^1	no	I/80	-I/80	I/75	-7*I/600
gt_mn gt_rs (p.p)^2	yes	-I/480	I/480	-23*I/3600	-I/450
gt_mn p_r p_s (p.p)^1	yes	I/120	-I/120	77*I/3600	17*I/3600
gt_mr gt_ns (p.p)^2	yes	-I/480	I/480	-23*I/3600	-I/450
gt_mr p_n p_s (p.p)^1	yes	-I/480	I/480	-23*I/3600	-I/450
gt_rs p_m p_n (p.p)^1	yes	I/120	-I/120	77*I/3600	17*I/3600
p_m p_n p_r p_s	no	-I/60	I/60	-47*I/1800	13*I/1800

4D pole	z_1	z_2	z_3	z_4	z_5	transverse
D11	-I/160	-I/480	I/80	I/240	-I/120	yes
D01+D03	-23*I/240	-61*I/240	23*I/120	61*I/120	-7*I/20	yes
D01+D03+D11	-49*I/480	-41*I/160	49*I/240	41*I/80	-43*I/120	yes

assignment (c, h)	X_{D01}	X_{D03}	X_{D11}	sum
Q4S (declared, R2)	p.p^2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b .c + 2*a .b *c^.p - a .c *b^.p)/(16*(d - 1))	-p.p^2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b .c + 2*a .b *c^.p - a .c *b^.p)/(16*(d - 1))	0	0
QdS (hat)	p.p^2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))	-p.p^2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))	0	0
c tilde, h hat	0	0	0	0
c tilde, h mixed	0	0	0	0
c hat, h mixed	0	0	0	0
all tilde	0	0	0	0
c hat, h tilde	a .b *c^.p*p.p^2/16	-a .b *c^.p*p.p^2/16	0	0

Full-row verdict: `DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a sum; no evanescent structure survives in the slot.`

7 Scope of revision p17: the hh evanescent poles and the ch slot

p17 = P.5 calculation, third step; D11 is NOT evaluated; no slot is complete.

p17 keeps the p14r contract and the p15 and p16 calculation layers (byte-identical) and adds: (1) a deterministic analysis of whether the evanescent ($\tilde{g}$) UV poles of D01 and D03 reach the ch-projected slot, with controls and mutations; (2) the explicit coordinates of the 4D poles in the p14r $h \cdot h$ basis and the constraint they satisfy; (3) the fixes G1–G4 and G6 of the p16 independent review (one disclosed dred.py revision: the guarded loader `parse_d`). D11 stays NOT evaluated (M2 undischarged), `D11_matching_verified`=false and `quantum_inputs_evaluated` stays not earned. O.1 is NOT_CLOSED, `symmetry_restored`=`bulk_symmetry_restored`=false, the insertion stays frozen, XV-B and XIV-J stay closed, Route B is not discharged, XVI-P is unregistered, and no $c\delta$ term is added to u^R_μ.

7.1 Where the hh self-energy enters the ch projection

Reading the ledger `ledgers/P13_ch_dependency_rows.json` (a computed control, not an assertion), the graphs D01, D03 occur in exactly one row: $(S_0, \Gamma_1^{[2]})\, s_2\, K{:}K\cdot c$, with Γ_1 pattern h^2. The same row also holds D02, D12 (zero, p15) and D11 (not evaluated). The slot functional is

$$X_{hh}[c,h] = \int (s\,h)_{(mn)}(p;c)\; G^{(mn)(rs)}(p)\; h_{(rs)}(-p).$$

The operator $s\,h$ is read off the $K\cdot c$ monomial of the frozen $S_\Psi^{(0)}$ (16 nonzero components). Its covariant fit is exact and unique: $(s\,h)_{(mn)\nu} = \frac{i}{2}(\eta_{m\nu}p_n + \eta_{n\nu}p_m)$. It is therefore LONGITUDINAL in (mn).

7.2 Decoupling (proof, exact in d)

- **H1.** $s\,h$ is longitudinal (above), so X_{hh} depends on G only through $p_m G^{(mn)(rs)}$. Because $p \in$ QdS, $p\cdot\tilde{g} = 0$, so $\tilde{g}$ on the pair (mn) drops out.
- **H2/H3.** $p_m\,(G_{D01} + G_{D03})^{(mn)(rs)} = 0$ identically in d. This holds for the 4D part and for the evanescent part separately: each of the six structures of $p_m G$ (for example $\tilde{g}_{rs}p_n$: $-\frac{i\,p^4}{8(d-1)}$ from D01 and $+\frac{i\,p^4}{8(d-1)}$ from D03) cancels exactly. Hence $X_{D01+D03} = 0$ for all six external assignments (Q4S, QdS, mixed $\hat{}$/). With $D02 = D12 = 0$, the hh slot receives nothing from the evaluated graphs.
- **H4 (controls).** X_{D01} and X_{D03} are each nonzero, and so are their evanescent parts alone. The cancellation is therefore a property of the SUM (the ghost loop cancels the longitudinal part of the bubble), not of each graph. The map is not trivially zero.
- **Order.** At one loop the ch projection is linear in Γ_1, so no product of evanescent poles occurs. The cancellation is exact in d and not merely $O(\epsilon)$.

Mutations, each rejected (summed slot becomes nonzero): wrong relative sign of D03; D03 doubled (symmetry factor); D03 with its evanescent part dropped (naive 4D); the trace operator $\eta_{mn}p_\nu$ in place of $s\,h$.

Verdict: DECOUPLED (proof), as the sum D01 + D03 (+ D02 = D12 = 0), and not graph by graph. No evanescent structure survives in the slot. No evanescent $h\cdot h$ counterterm is added, so the counterterm quotient is unchanged and no contract file changes for this. The summed evanescent poles of the hh Green function are nonzero (eight structures, recorded in the certificate) but transverse. Renormalizing the hh Green function itself would need evanescent $h\cdot h$ counterterms; that matters only outside this projection and is not claimed. D11, in the same row, is open.

7.3 4D poles: coordinates and the transversality constraint

The five p14r $h\cdot h$ directions span the five pair-symmetric 4-derivative 4D structures (rank 5), so span membership is automatic. The sharper statement is a constraint. Longitudinality $p_m G^{(mn)(rs)} = 0$ imposes rank-3 linear conditions on the coordinates z, which leaves a 2-dimensional transverse subspace with basis $k_1 = (\frac{1}{2}, -\frac{1}{2}, -1, 1, 0)$ and $k_2 = (1, 0, -2, 0, 1)$. The individual pole coordinates are not transverse:

$$z_{D01} = i\,(-\tfrac{27}{160}, -\tfrac{71}{240}, \tfrac{19}{60}, \tfrac{11}{20}, -\tfrac{13}{30}), \qquad z_{D03} = i\,(\tfrac{7}{96}, \tfrac{1}{24}, -\tfrac{1}{8}, -\tfrac{1}{24}, \tfrac{1}{12}).$$

Their sum is transverse:

$$z_{D01+D03} = i\,(-\tfrac{23}{240}, -\tfrac{61}{240}, \tfrac{23}{120}, \tfrac{61}{120}, -\tfrac{7}{20}) = i\,(\tfrac{61}{120}\,k_1 - \tfrac{7}{20}\,k_2).$$

This is the Ward-type statement the ch identity needs at this order. It is incomplete without D11.

7.4 Phase convention (G6)

Every graph is exported as $G = \kappa^2 N_\epsilon \hat{G}$ with $N_\epsilon = iS_\epsilon/16\pi^2$. For D01 and D03, $\hat{G} = -i\,\mathrm{pref}/\kappa^2 \int$ with real export prefactors ($\kappa^2/2$, $-\kappa^2$), so $A(d)$ carries an overall i. D13's exported prefactor is $i\kappa^2$, so its $A(d)$ is real. Because $s\,h = i(\dots)$, X_{hh} is real like the D13 slot. The D01/D03 relative phase is tested by H3 (mutation M1 breaks it). The D13-versus-hh relative phase is not tested by a nonzero identity, since both slot contributions vanish. No physical overall phase is asserted.

7.5 Review fixes G1–G4

G2: `dred.parse_d` (locals $\{d$: opaque $d\}$) and `dred.assert_opaque_d` close the plain-`Symbol('d')` hazard of re-parsed strings (disclosed contract revision; no value affected). G1/G3: the guard docstring and the disclosure now state exactly what is refused at runtime and what is excluded by the extended static AST scan: replace with any function query, sympify/S/parse_expr of non-sympy data without the guarded locals, builtin eval/exec, srepr round-trips, `.func(*args)` rebuilds, `Poly(..).eval`, lambdify. The scan is a pattern check, not a proof. G4: the change log lists the p17 reference-values file as added.

structure of $G^{(mn)(rs)}$	evanescent	D01 reaches slot	D03 reaches slot
`g4_mn g4_rs (p.p)^2`	no	yes	yes
`g4_mn gt_rs (p.p)^2`	yes	yes	yes
`g4_mn p_r p_s (p.p)^1`	no	yes	yes
`g4_mr g4_ns (p.p)^2`	no	yes	yes
`g4_mr gt_ns (p.p)^2`	yes	yes	yes
`g4_mr p_n p_s (p.p)^1`	no	yes	yes
`g4_rs gt_mn (p.p)^2`	yes	no	no
`g4_rs p_m p_n (p.p)^1`	no	yes	yes
`gt_mn gt_rs (p.p)^2`	yes	no	–
`gt_mn p_r p_s (p.p)^1`	yes	no	no
`gt_mr gt_ns (p.p)^2`	yes	no	–
`gt_mr p_n p_s (p.p)^1`	yes	yes	yes
`gt_rs p_m p_n (p.p)^1`	yes	yes	yes
`p_m p_n p_r p_s`	no	yes	yes

4D pole	z_1	z_2	z_3	z_4	z_5	transverse
D01	`-27*I/160`	`-71*I/240`	`19*I/60`	`11*I/20`	`-13*I/30`	no
D03	`7*I/96`	`I/24`	`-I/8`	`-I/24`	`I/12`	no
D01+D03	`-23*I/240`	`-61*I/240`	`23*I/120`	`61*I/120`	`-7*I/20`	yes

Verdict: `DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph`. Surviving evanescent structures in the slot: none. Transverse kernel (p14r coordinates): `(1/2, -1/2, -1, 1, 0); (1, 0, -2, 0, 1)`. D11: NOT evaluated.

8 Scope of revision p16: E2 analysis and Batch C

p16 = P.5 calculation, second step; D11 is NOT evaluated; no slot is complete.

p16 keeps the p14r contract and the p15 calculation layer (byte-identical) and adds: (1) a deterministic proof that the evanescent D13 pole (structure E2) decouples from the ch-projected slot, with mutation controls; (2) Batch C, the graphs D01 and D03 of the $\kappa^2\,hh$ slot, under the same frozen prescription (DRED R1–R6, Feynman $+i0$, MS-bar, S0, d opaque); (3) the fixes F3–F5 of the p15 independent review. D11 stays NOT evaluated (M2 undischarged), `D11_matching_verified`=false and `quantum_inputs_evaluated` stays not earned. O.1 is NOT_CLOSED, `symmetry_restored`=`bulk_symmetry_restored`=false, the insertion stays frozen, XV-B and XIV-J stay closed, Route B is not discharged, XVI-P is unregistered, and no $c\delta$ term is added to u^R_μ.

8.1 E2: decoupling from the ch-projected slot (proof)

In p15 D13 has an evanescent structure $E2_{(mn)\nu} = (\tilde{g}_{m\nu}p_n + \tilde{g}_{n\nu}p_m)p^2$ with $A_{E2}(d) = 3/(16(d-1))$ and UV pole $1/16$; p14r declares no evanescent counterterm. D13 enters the ch projection only through the row

$(S_0, \Gamma_1^{[2]})\, s_1\, h{:}h^2$:

$$X_{D13}[c,h] = \int G_{(mn)\nu}(p)\, c^{\nu}(p)\, E^{(mn)(rs)}(p)\, h_{(rs)}(-p),$$

where E is the operator of the h^2 monomials of $S_\Psi^{(0)}$ (linearized Einstein–Hilbert). Its exact covariant form is fitted from the frozen density and verified on all components; the fit is unique.

- **L1.** $p_m E^{(mn)(rs)} = 0$ as a covariant identity. Only $g_4 \cdot p = p$ is used; no trace is taken, so the identity holds exactly in d.
- **L2.** $E2$ is longitudinal in (mn).
- **L3.** Therefore $E2 \cdot E = 0$ identically. The slot value is 0 for every external assignment (Q4S, QdS, mixed $\hat{\ }/\tilde{\ }$ components), exactly in d and not merely $O(\epsilon)$.
- **L4 (control).** The non-longitudinal $T1$ and $E1$ give nonzero slot values, so the map is not trivially zero. $A_{T1} = A_{E1} = 0$ for D13, so D13's whole ch-slot contribution is 0.

Mutation controls: the gauge-fixed operator $q^2 P$ in place of E, $E1$ in place of $E2$, and a wrong index slot each make the $E2$ slot nonzero and are rejected. At one loop the ch projection is linear in Γ_1, so no product of evanescent poles arises.

Outcome: DECOUPLED (proof). No evanescent counterterm is added and no contract file changes for E2. The $E2$ pole remains a UV pole of the ϵ-scalar component of the $K' \cdot c$ Green function (R6: retained and recorded). Its renormalization matters only outside this projection, and it is not claimed here. D13 stays *regulated*, with a ch-slot contribution of 0.

8.2 Batch C: D01, D03 (covariant route)

Finding C1. For rank-4 numerators, the p14r per-component R3 lift produces component sums quadratic in $\tilde{g}$ (D03: 10 of 100 components). After the component sums, their abstract value cannot be recovered, because the linear realization $\tilde{g} = (4-d)M$ is not faithful there. A finite component basis cannot carry a projector of non-integer trace.

Batch C therefore uses a *covariant route* that applies the same rules R1–R6 with free external indices only:

1. The exact covariant 4D numerator is fitted from the frozen export and verified as a polynomial identity on all 100 components. It is unique (upper external indices, off-diagonal weight 2).
2. The lift $\eta \to g_4$ is made.
3. Two independent loop routes are used: Feynman parameters with symmetric integration, and the frozen `dred.reduce_tensor` with the J library and S0. They agree exactly in d.
4. Then $\hat{g} = g_4 - \tilde{g}$ on the free indices.

D13 is reproduced exactly by this route (control), and the rules are unchanged (no contract change).

Further controls:

- The integrands are regenerated from S_Ψ (D01 from the ℓ_3 density, D03 from the ghost vertices) and equal the export on 100 of 100 components.
- The results are exchange symmetric.
- The nonlocal log coefficient equals minus the pole.
- The 4D poles lie in the span of the five p14r $h.h$ counterterm directions.

Seven mutations are each rejected: ghost-loop sign, symmetry factor, premature $d = 4$, dropped $\tilde{g}$, external index position, off-diagonal weight, and the component route with the linear realization.

graph	structure	$A(d)$ (coefficient of MB)	UV pole	MS-bar finite
D01	g4_mn g4_rs (p.p)^2	I*(-2*d^2 - 10*d - 9)/(16*(d^2 - 1))	-27*I/80	I*(405*L - 982)/1200
D01	g4_mn gt_rs (p.p)^2	I*(5*d + 6)/(16*(d^2 - 1))	13*I/120	I*(523 - 195*L)/1800
D01	g4_mn p_r p_s (p.p)^1	I*(d^2 + 12*d + 12)/(16*(d^2 - 1))	19*I/60	I*(724 - 285*L)/900
D01	g4_mr g4_ns (p.p)^2	I*(-6*d^2 + 4*d + 9)/(16*(d^2 - 1))	-71*I/240	I*(1065*L - 1946)/3600
D01	g4_mr gt_ns (p.p)^2	I*(-2*d - 1)/(16*(d^2 - 1))	-3*I/80	I*(45*L - 118)/1200
D01	g4_mr p_n p_s (p.p)^1	I*(6*d^2 - 5*d - 10)/(16*(d^2 - 1))	11*I/40	I*(97 - 55*L)/200
D01	g4_rs gt_mn (p.p)^2	I*(5*d + 6)/(16*(d^2 - 1))	13*I/120	I*(523 - 195*L)/1800
D01	g4_rs p_m p_n (p.p)^1	I*(d^2 + 12*d + 12)/(16*(d^2 - 1))	19*I/60	I*(724 - 285*L)/900
D01	gt_mn gt_rs (p.p)^2	-I/(16*d^2 - 16)	-I/240	I*(15*L - 46)/3600
D01	gt_mn p_r p_s (p.p)^1	I*(-7*d - 8)/(16*(d^2 - 1))	-3*I/20	I*(90*L - 241)/600
D01	gt_mr gt_ns (p.p)^2	-I/(16*d^2 - 16)	-I/240	I*(15*L - 46)/3600
D01	gt_mr p_n p_s (p.p)^1	I*(3*d + 2)/(16*(d^2 - 1))	7*I/120	I*(277 - 105*L)/1800
D01	gt_rs p_m p_n (p.p)^1	I*(-7*d - 8)/(16*(d^2 - 1))	-3*I/20	I*(90*L - 241)/600
D01	p_m p_n p_r p_s	I*(-13*d^2 - 2*d + 8)/(16*(d^2 - 1))	-13*I/15	I*(780*L - 1597)/900
D03	g4_mn g4_rs (p.p)^2	I*(d + 3)/(16*(d - 1))	7*I/48	I*(50 - 21*L)/144
D03	g4_mn gt_rs (p.p)^2	-I/(8*d - 8)	-I/24	I*(3*L - 8)/72
D03	g4_mn p_r p_s (p.p)^1	-3*I/(8*d - 8)	-I/8	I*(3*L - 8)/24
D03	g4_mr g4_ns (p.p)^2	I*(d - 3)/(8*(d - 1))	I/24	I*(2 - 3*L)/72
D03	g4_mr gt_ns (p.p)^2	I/(8*(d - 1))	I/24	I*(8 - 3*L)/72
D03	g4_mr p_n p_s (p.p)^1	I*(7 - 2*d)/(16*(d - 1))	-I/48	I*(3*L + 4)/144
D03	g4_rs gt_mn (p.p)^2	-I/(8*d - 8)	-I/24	I*(3*L - 8)/72
D03	g4_rs p_m p_n (p.p)^1	-3*I/(8*d - 8)	-I/8	I*(3*L - 8)/24
D03	gt_mn p_r p_s (p.p)^1	I/(4*(d - 1))	I/12	I*(8 - 3*L)/36
D03	gt_mr p_n p_s (p.p)^1	-3*I/(16*d - 16)	-I/16	I*(3*L - 8)/48
D03	gt_rs p_m p_n (p.p)^1	I/(4*(d - 1))	I/12	I*(8 - 3*L)/36
D03	p_m p_n p_r p_s	I*(d - 2)/(4*(d - 1))	I/6	I*(5 - 3*L)/18

Status: D01: regulated. D03: regulated. E2 (D13): DECOUPLED (proof).

With $A(d)$ the coefficient of MB $= 1/\epsilon + 2 - L$, $L = \log(-p^2/\mu^2 - i0)$, in units of N_ϵ: the UV pole is $A(4)$, and the MS-bar finite part is $A(4)(2-L) - 2A'(4)$. *Status:* D01 and D03 are *regulated*. Their 4D sectors are renormalized by poles in the p14r $h.h$ span. Their evanescent ($\tilde{g}$) poles are NONZERO and retained, and no evanescent $h.h$ counterterm is declared. The hh slot (D01+D02+D03+D11+D12) stays incomplete because D11 is not evaluated.

8.3 Review fixes F3–F5 and rebuild flags

- **F3 (one disclosed contract revision, `dred.py`).** The opaque d now refuses `expr.replace(d,·)`, pattern matching on d and the lambdify/code-printer paths outside `substitute_d`. Paths without a hook (callable-query replace, `Poly.eval`, string round trips) are excluded by a static AST scan. The P13–P15 certificate values are unchanged against the p15 reference values.
- **F4.** The page footer comes from one version variable. The PDF readback also covers the D13, Batch A, Batch C and E2 value blocks.
- **F5.** Every changed file, contract and non-contract, is listed in `change_log_vs_p15` of the release status.

`independent_rebuild_passed_p15`=true is bound to the p15 tree 79f53752… and zip 54325c1f… (same box). The p16 tree's own independent rebuild is pending.

9 Scope of revision p15: first P.5 calculation

p15 = P.5 calculation, first batch; D01, D03, D11 are NOT evaluated; no slot is complete.

p15 evaluates the Batch A graphs D02, D12, D14a, D14b, D14c and the Batch B graph D13 under the frozen p14r prescription: DRED R1–R6 with the $\hat{g}/\tilde{g}$ algebra, contractions before the Laurent expansion, d kept symbolic and evanescent parts retained; Feynman $+i0$; MS-bar with $N_\epsilon = iS_\epsilon/(16\pi^2)$, $S_\epsilon = (4\pi)^\epsilon e^{-\gamma_E \epsilon}$. The declared action is massless (no ϕ mass is declared). `quantum_inputs_evaluated`, `projected_defect_computed`, `projected_identity_verified` and `projected_restoration_verified` remain not earned, and `bulk_symmetry_restored`=false. O.1 is NOT_CLOSED, the insertion stays frozen, XV-B and XIV-J stay closed, and Route B is not discharged. The new flags `batchA_evaluated` and `D13_evaluated` each carry an execution, a claim and a scope part.

9.1 Integral library and the scaleless rule

The library (`P15_integral_library.json`) provides A_0, B_0 and the tensor reduction in $\hat{g}$. It is certified exactly in d against Feynman parameters, and numerically at $d = 2.6, 3.3, 3.7$.

The scaleless rule is *stated*, not assumed:

$$\text{S0:} \quad \int d^d l\, (l^2)^k/(l^2)^\nu := 0, \qquad \omega = 4 + 2k - 2\nu.$$

Its UV/IR reading is as follows. For $\omega = 0$ the zero is the cancellation $1/\epsilon_{\rm UV} - 1/\epsilon_{\rm IR}$. For $\omega \neq 0$ it discards a pure power divergence, and neither the UV part nor the IR part has a pole at $d = 4$. Applying S0 to an integral that carries a scale is an error, and Missing is never replaced by zero by convention.

9.2 Batch A: D02, D12, D14a–c

All five graphs are *vanishing under the declared prescription*: there is no UV pole, the finite part is 0, and no p14r counterterm is needed. Every scaleless member is a pure power divergence ($\omega = 2, 4$), so the zero is *not* a $1/\epsilon_{\rm UV} - 1/\epsilon_{\rm IR}$ cancellation.

- **IR-regulator mass** (analysis only). With a mass the poles are proportional to $\kappa^2 m^4 hh$ and $\kappa^2 m^2 p^2 hh$, which are order-0/2 contacts excluded by N4, or to $\kappa m^4 h$, for which the κ^1 basis is empty. All of them vanish as $m \to 0$.
- **Analytic regulator.** It does not separate the UV from the IR here.
- **Cross-check.** Two routes agree: the frozen dred lift with S0, and an independent symmetric tensor integration.
- **D12.** D12 ($\kappa^2\, hh$) is kept separate from the order-κ one-point slot (D14a–c). `D12_status` moves from Missing to *vanishing_under_declared_prescription*. This is *derived* under P's own prescription; XVI-O's value is not imported.

9.3 Batch B: D13 (κ^2 $\mathbf{K}' \cdot c$)

Write $G_{(mn)\nu}(p) = V_{\Gamma_1}[K'_{(mn)}(-p), c^\nu(p)] = \kappa^2 N_\epsilon \hat{G}$, with $\hat{G} = \sum_i A_i(d)\, T_i\, B_0$, $B_0 = 1/\epsilon + 2 - L$ and $L = \log(-p^2/\mu^2 - i0)$. The structures are

$$T_1 = \eta_{mn} p_\nu p^2, \quad T_2 = (\eta_{m\nu} p_n + \eta_{n\nu} p_m) p^2, \quad T_3 = p_m p_n p_\nu,$$

together with the evanescent $E_1 = \tilde{G}_{mn} p_\nu p^2$ and $E_2 = (\tilde{G}_{m\nu} p_n + \tilde{G}_{n\nu} p_m) p^2$. The coefficients are

$$A_{T_2} = \frac{4d-7}{16(d-1)}, \qquad A_{T_3} = \frac{3(2-d)}{8(d-1)}, \qquad A_{E_2} = \frac{3}{16(d-1)}, \qquad A_{T_1} = A_{E_1} = 0.$$

This gives:

- **UV poles:** T_2: 3/16, T_3: $-1/4$, E_2: 1/16.
- **MS-bar finite parts:** T_2: $1/3 - \frac{3}{16}L$, T_3: $-5/12 + \frac{1}{4}L$, E_2: $1/6 - \frac{1}{16}L$. The nonlocal part is $-(\text{pole}) \cdot L$, and the local part is a cubic polynomial.
- **Counterterm map.** The 4D pole lies in the span of the three p14r $c \cdot K'$ directions, with coordinates $(0, 3/8, -1/4)$. The local ambiguity is a finite renormalization along the same directions.
- **d-dependence.** The $(d-4)\times$pole terms shift the local part (T_2: $-1/24$, T_3: $+1/12$). Setting $d = 4$ first gives a different MS-bar finite part.
- **Evanescent pole and status.** The evanescent pole (E_2) is nonzero, and the p14r contract declares no evanescent counterterm. D13's status is therefore *regulated*. Its 4D sector is *renormalized*; the evanescent pole is recorded but not absorbed, and stays an open contract item.

Controls.

- **Two routes, exact in d.** Route 1 is the sha-pinned export through the dred Pipeline. Route 2 is the integrand regenerated from the rule tables and evaluated by Feynman parameters with d symbolic. They agree exactly.
- **Exact covariant fit.**
- **BRST sanity (ghost equation).** $2i\,D^\rho[G]$ equals the evaluated one-loop ghost self-energy. The ghost self-energy's 4D pole lies in the p14r $c \cdot \bar{c}$ span, at $(-3/8, 1/4)$; its evanescent pole is $-1/8$.
- **Structural R5 safeguard.** It is active.

Mutations. Each mutation changes the result and is rejected: wrong ghost-loop sign, premature $d = 4$ (refused at the substitution), dropped evanescent part, wrong symmetry factor, and wrong K'/c ordering.

graph	status	UV pole (units $\kappa^2 N_\epsilon/\epsilon$)	finite part
D02	vanishing_under_declared_prescription	0	0
D12	vanishing_under_declared_prescription	0	0
D14a	vanishing_under_declared_prescription	0	0
D14b	vanishing_under_declared_prescription	0	0
D14c	vanishing_under_declared_prescription	0	0
D13	regulated (4D: renormalized)	T1: 0, T2: 3/16, T3: -1/4, E1: 0, E2: 1/16	local T1: 0, T2: 1/3, T3: -5/12, E1: 0, E2: 1/6

9.4 Disclosed p14r contract revision

Finding F1 of the p14r independent rebuild concerns the p14r `PrematureD4Substitution` guard in `dred.py`, which was a heuristic.

- **What went wrong.** It gave a false positive on $\eta^{rs}\int l_r l_s/(l^2 - m^2) = J$, a correct d-free result. It gave a false negative under the premature-$d = 4$ mutation on the same integral: the result was $dJ/4$, a finite error of $-m^4/2$.
- **What p15 does.** p15 replaces the guard by a structural, provenance-based safeguard. d is opaque and can be substituted only through `dred.substitute_d`, which records each use. Numeric traces of $\hat{g}/\tilde{g}$ are refused. The R5 docstring is corrected, and the reviewer's toys are regression tests.
- **What does not change.** R1–R6 are unchanged, and every other contract file is byte-identical (checked against pinned sha256). Released p14r values are unaffected, since p14r applied the Laurent step only to toys.

9.5 Reproducibility

The independent rebuild of p14r *passed*: 143/143 values equal and 60 tests. It is bound to the p14r tree `a64c0ae3...` and zip `19e8bf59...`. It ran on the same box, in a fresh directory and venv, and was not run on a second machine. It is not inherited by p15, whose own independent flag is pending.

10 Scope of revision p14 (repaired in p14r)

p14r = a narrow repair of p14; P.3/P.4 code byte-identical; no loop evaluated.

- The prescription is made operational: executable DRED index rules, tests against premature $d = 4$, and the withdrawn claim that no evanescent remainder arises (Section 14.1).
- The D11 M3 labels are corrected, and M3 is made conditional on M2, with an explicit normalization condition.
- The counterterm quotient is worded as candidate directions before the identity constraints.
- The regeneration coverage is stated, and the independent Hessian-route regeneration W6 is added for D01, D02, D03 and D12.
- The isolated-copy gate tests contain the frozen inputs.

p14 is archived in `archival/p14/`.

p14 = P.5 entry: the contract is fixed; no loop is evaluated.

p14 fixes what P.5 will compute and how: the quantum object Γ_Ψ in transformed variables (Section 13); one prescription together with a D11 matching note against XVI-O's frozen D11 (Section 14); the one-loop counterterm domain at κ^1, κ^2 (Section 15); and executable amplitude rules with symbolic integrands for D01, D02, D03, D11 (reference only), D12, D13, D14a–c (Section 16). No integral is evaluated: the Batch A/B/C integrals belong to p15 and later. Opening P.5 entry earns none of the loop or restoration flags. `quantum_inputs_evaluated`, `projected_defect_computed`, `projected_identity_verified` and `projected_restoration_verified` are not earned; `bulk_symmetry_restored`=false. D12 is Missing, never zero. O.1 is NOT_CLOSED. The insertion stays frozen, XV-B and XIV-J are not opened, Route B is not discharged, and no $c\,\delta$ is added to u^R_μ. XVI-P stays *unregistered*: the package is evidence in the sense of XVI-O `conv:package`, not a series certificate, and XVI-A–XVI-O are not edited.

11 P.3/P.4 frozen as P.5 inputs

An independent rebuild of p13 passed. A reviewer unpacked `XVI-P-source-only.zip` (sha256 `2f85a566...`) in a fresh environment and ran the source-only entrypoint. All reference values matched (112/112, no mismatch), 77 values were recomputed by separate code, and a hand-written spot check of the conformal boundary identity agreed. The report is archived as `archival/p13/reproducibility/independent_rebuild_report_p13.json` (sha256 pinned). **Caveat:** the rebuild ran on the same box (shared kernel, TeX Live and poppler) in a fresh virtual environment from the source-only zip; it was not on a second machine.

The flag is *bound to the p13 source tree* `2b1baf33...` and to the p13 zip; p14 changes cannot inherit it, and p14 has its own (pending) independent flag. `P3_P4_frozen_as_P5_inputs` is recomputed in every build by `freeze_p13.py` as the conjunction of three conditions: F1, the archived independent report has its pinned hash, `pass`=true, no mismatch, and is bound to the p13 tree and zip; F2, the 54 frozen files (classical P.3/P.4 code, the inherited XVI-O code and notes, pins and inherited data; `archival/p13/frozen_p13_inputs.json`) are byte-identical; F3, the regenerated classical values (export hashes, the seven classical certificates and two ledgers, residual records, pins) equal `archival/p13/reference_values_p13.json`. The classical code is therefore unchanged in p14. Its certificates keep their `P13_*` names, and the p14 gate recomputes the p13 acceptance from them (Section 19). **P.5 entry is stated against this freeze:** if any of F1–F3 fails, P.5 entry is not earned and publication is rejected.

12 Milestones (p14 status)

milestone	definition (unchanged since p13, `conventions.MILESTONES`)	status (p14)
P.3	fixed output projection + exhaustive coefficient dependencies + graph topology inventory	complete (frozen)
P.4	required classical action, transformations, source terms, gauge map and master verification. **P.4 means classical readiness, NOT coupled one-loop restoration.**	earned (frozen)
P.5 entry	quantum prescription, amplitude rules and counterterm domain fixed	earned iff the five contract flags hold and P.3/P.4 are frozen (see summary)
P.5 calculation	coefficients evaluated and renormalized	open (p15+)
P.5 identity	projected defect computed and, if possible, cancelled	open

P.5 entry := `quantum_action_defined` ∧ `quantum_prescription_defined` ∧ `D11_matching_defined` ∧ `counterterm_domain_defined` ∧ `amplitude_rules_verified`, each recomputed from its certificate, together with `P3_P4_frozen_as_P5_inputs`. The values of this build:

contract item	value (from the certificates of this build)
integrated fields	h_(mn) (10 independent components), cˆmu, cbar_mu, phi, bˆmu (retained)
external antifields	K'_(mn): h*' (10 comps, sigma_h = +1), c*'_mu: c*', cbar*'ˆmu: cbar*', phi*': phi*', b*': absent (s b = 0)
b	RETAINED and integrated; det M_bb = -16; eliminating b gives 4 terms quadratic in cbar*'; Gamma_1 independent of bs, cbs
antifield shift	eps = -1 (derived)
prescription	P14_DRED_MSbar_FeynmanI0: MS-bar: subtract the pole in the combination 1/eps - gamma_E + log 4 pi; scale mu
i0	Feynman: every propagator denominator qˆ2 + i0 (eta = +,-,-,-); Wick rotation l0 = i l4 implied
DRED rules	verified = true; T1 finite a/8 + b/4 (premature d=4 would give b/4); T2 finite -a/2 ((d-4) x pole); mutations: premature_d4 rejected, trace_ghat_eq_4 rejected, contract_after_laurent rejected
D11 matching	defined = true, verified = false; M1: 100 component pairs, 0 mismatches; M3 (conditional on M2): kernel sd <= 8, contact order <= 4; contact space order_0 2, order_2 4, order_4 5; under N4 5 (transverse 2); M2 defined_not_verified
counterterm domain	kappaˆ1: 0; kappaˆ2: 11 (h.h 5, c.cb 2, c.K 3, phi.phi 1); sigma_0-equivalent: 3 (h.h 2, c.cb 0, c.K 0, phi.phi 1); 8 candidate directions modulo the computed exact equivalences, before the identity constraints (D cap ker sigma_0 not computed)
amplitude controls	W1_wick_berezin_shadow: pass, W2_multiplicity: pass, W3_closed_form_vertices: pass, W5_regeneration: pass, W6_hessian_route: pass, W4_ward_bose: pass; W5 regenerates D14a-c, D11, D13; W6 (Hessian route) regenerates D01, D02, D03, D12 == exports: true
mutations (must be rejected)	wrong_tadpole_factor: rejected, wrong_ghost_loop_sign: rejected, offdiag_multiplicity: rejected, reversed_source_ghost_order: rejected, omit_ghost_transport_term: rejected
P.5 entry flags	quantum_action_defined = true, quantum_prescription_defined = true, D11_matching_defined = true, counterterm_domain_defined = true, amplitude_rules_verified = true
P3_P4_frozen_as_P5_inputs	true (the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it was NOT run on a second machine)
P.5 entry	earned (contract fixed; stated against the P.3/P.4 freeze)

graph	prefactor	denominators	comps	monomials	deg	status
D01	`kappa**2/2`	l^2+i0, (l-p)^2+i0	100	3888	4	integrand_available
D02	`-kappa**2/2`	l^2+i0	94	220	2	integrand_available
D03	`-kappa**2`	(l-p)^2+i0, l^2+i0	100	2728	4	integrand_available
D11	`kappa**2/2`	l^2+i0, (l-p)^2+i0	100	1722	4	reference
D12	`-kappa**2/2`	l^2+i0	94	160	2	Missing
D13	`I*kappa**2`	l^2+i0, (p-l)^2+i0	40	396	3	integrand_available
D14a	`-kappa/2`	l^2+i0	10	22	2	integrand_available
D14b	`kappa`	l^2+i0	4	16	2	integrand_available
D14c	`-kappa/2`	l^2+i0	10	22	2	integrand_available

13 The quantum object

Builder/output: `quantum_object.py`, `P14_quantum_object.json`.

Action. The action is computed directly in transformed variables:

$$S_\Psi(\Phi,\Phi^{*\prime}) = \sum_k \kappa^{k-2}\int \ell_k + S_\phi + X\big(\Phi,\Phi^{*\prime}+\varepsilon\,\partial\Psi/\partial\Phi\big), \qquad \varepsilon=-1 \text{ (derived)},$$

with $\Psi = -i\int \bar c_\mu(2D^\mu[h]-b^\mu)$. By order, X contains $K'c$, $\bar c c$, b^2, bh, $b\bar c^{*\prime}$ at κ^0; $K'ch$, $\bar c ch$, $c^{*\prime}cc$, $\phi^{*\prime}c\phi$ at κ^1; and nothing at κ^2. S_ϕ contributes $\phi\phi$, $h\phi\phi$, $hh\phi\phi$. There are no switch or insertion fields: the baseline is $S_{\rm inv}$, not S_D.

Fields and sources. The integrated fields are $h_{(mn)}$ (10 independent components), c^μ, $\bar c_\mu$, ϕ and b^μ. The external antifields are $K'_{(mn)}$ ($\sigma_h=+1$), $c^{*\prime}_\mu$, $\bar c^{*\prime\mu}$ and $\phi^{*\prime}$; $b^{*\prime}$ is absent because $sb=0$.

b is retained and integrated. Four reasons:

- b occurs only in κ^0 bilinears, so it has no interaction vertex;
- M_{bb} is constant and invertible (det $=-16$);
- the full (h,b) inverse is verified, with Schur complement q^2P;
- eliminating b (computed) would make S_Ψ *quadratic* in $\bar c^{*\prime}$, whereas retaining it keeps S_Ψ linear in every external antifield.

$\bar c^{*\prime}$ couples only through $\bar c^{*\prime}b$, which has one quantum leg and is tree-level and one-particle reducible. So the 1PI Γ_1 is independent of $\bar c^{*\prime}$ and $b^{*\prime}$, while K', $c^{*\prime}$ and $\phi^{*\prime}$ enter through $K'ch$, $c^{*\prime}cc$ and $\phi^{*\prime}c\phi$.

1PI convention. The weight is $e^{iS_\Psi/\hbar}$ and $\Gamma_\Psi = S_\Psi + \hbar\Gamma_1 + O(\hbar^2)$, with the Legendre transform taken in the fields only (antifields are spectators). $\Gamma_1 = \frac{i}{2}\mathrm{STr}\log S^{(2)}_\Psi = -i\log Z|_{\rm 1PI}$, so that $i\Gamma_1$ is the sum of 1PI one-loop graphs (Gate 0; the Berezin/Wick shadow below).

Vacuum and one-point. The vacuum is normalised by $\Gamma_1[0]=0$. The one-point function (D14a–c) is retained: no tadpole cancellation and no background shift are imposed, and no value is assigned.

Fourier, propagators and i0. We use $\partial\to ik$ with incoming momenta. The propagators are $\langle hh\rangle = i(P^{-1})_{ab}/(q^2+i0)$ (de Donder; independent components), $\langle c^\mu\bar c_\rho\rangle = i(M_0^{-1})_{\mu\rho} = -\delta_{\mu\rho}/(q^2+i0)$ and $\langle\phi\phi\rangle = i/(q^2+i0)$, all of them inverses of the verified Hessians; the $i0$ is Feynman throughout.

Bookkeeping. Each n-leg vertex is $O(\kappa^{n-2})$. At one loop, Γ_1 at κ^k has exactly k external legs ($\sum_v(n_v-2)=E$), and this is checked against the prefactor of every exported integrand.

Pullback, not theorem. $\Gamma_\Psi(\Phi,\Phi^{*\prime}) = \Gamma(\Phi,\Phi^{*\prime}-\partial\Psi/\partial\Phi)$ is used only as the classical pullback (verified at tree level). No quantum gauge-change theorem is asserted.

14 Prescription and the D11 matching note

Builder/output: `prescription_d11.py`, `P14_prescription_D11_matching.json`.

One prescription. Dimensional reduction (DRED), made operational in p14r (Section 14.1). Subtraction is $\overline{\rm MS}$ at scale μ, and the $i0$ is Feynman. The rules for scaleless integrals belong to the definition but

are *not applied*: no tadpole (D02, D12, D14a–c) receives a value. D12 stays Missing. XVI-O fixed D12 and the one-point function by `dimensional_regularization_m0`; P's prescription belongs to the same family, but P does not import that value.

14.1 Operational DRED rules (p14r)

Builder/output: `dred.py`, `P14_dred_rules.json`; `quantum_prescription_defined` requires `dred_rules_verified`, and the release gate recomputes it.

R1, spaces. The quasi-4-dimensional space Q4S splits as QdS $\oplus$ Q$(4-d)$S: $g^{(4)} = \hat{g} + \tilde{g}$, with $\mathrm{tr}\, g^{(4)} = 4$, $\mathrm{tr}\,\hat{g} = d$ and $\mathrm{tr}\,\tilde{g} = 4 - d$. The metrics are projectors: $\hat{g}\hat{g} = \hat{g}$, $\tilde{g}\tilde{g} = \tilde{g}$, $\hat{g}\tilde{g} = 0$, $g^{(4)}X = X$.

R2, momenta. The loop momentum l *and* the external momenta p lie in QdS ($p \cdot \tilde{g} = 0$; a stated choice). Field, vertex and polarisation indices are Q4S indices.

R3, lifting. The exported numerators are polynomials in the lower components l_r, p_r ($r = 0..3$) at fixed external components. They are lifted as $l_r := l \cdot e_{(r)} = l \cdot \hat{e}_{(r)}$ and $p_r = p \cdot e_{(r)}$, with $e_{(r)}$ the η-orthonormal basis of Q4S. The vertex tensors are lifted with $\eta \to g^{(4)}$, so nothing is truncated.

R4, reduction. $\int l^{\mu_1} \cdots l^{\mu_n} F(l^2, l \cdot p) = \sum_k C_k B_k$, with B_k the symmetric tensors built from $\hat{g}$ and p, and C_k given by the exact Gram (Passarino–Veltman) projection in symbolic d. Contracting with the lifted basis gives $\hat{g}_{rs} = \eta_{rs} - \tilde{G}_{rs}$, where the evanescent $\tilde{G}_{rs}$ satisfy $\eta^{rs}\tilde{G}_{rs} = 4 - d$ and $\tilde{G}_{rs}p^s = 0$. The checked closed forms are: the rank-2 tadpole $\hat{g}\, J/d$; the rank-4 tadpole $J/(d(d+2))$; $B_{00} = (J_{10}p^2 - J_{02})/((d-1)p^2)$; and $B_{11} = (dJ_{02} - p^2 J_{10})/((d-1)p^4)$.

R5, order. *Every* index contraction, including the external 4D projection, is done *before* the Laurent expansion. d stays symbolic through the $\overline{\mathrm{MS}}$ subtraction and the finite-part extraction, so $(d-4)\times$pole terms are retained.

R6, evanescent. Evanescent remainders *do* arise in general. The lift of the exported integrands has $\tilde{G}$ parts in every graph, and contraction produces $(4-d)\times$pole finite terms. They are kept. The p14 sentence claiming that none arise is withdrawn.

Tests (generic placeholders $J = a/\epsilon + b + c\epsilon$, $d = 4 - 2\epsilon$; no integral of this theory is evaluated).

- T1, the η_{00} part of $\int l_0 l_0 F$, has finite part $a/8 + b/4$. Substituting $d = 4$ before the expansion would give $b/4$; the guarded pipeline raises `PrematureD4Substitution`.
- T2, $\eta^{rs}\times$ the $\tilde{G}$ part, equals $-(4-d)J/d$ with finite part $-a/2$, a $(d-4)\times$pole term.

The mutations $\mathrm{tr}\,\hat{g} = 4$ (rules check fails; T1 becomes $b/4$ and T2 becomes 0) and expand-before-contract (`UncontractedObjects`) are both rejected. Background literature (nothing is imported from it): W. Siegel, Phys. Lett. B84 (1979) 193 and B94 (1980) 37; D. Stöckinger, JHEP 0503 (2005) 076, hep-ph/0503129; A. Signer and D. Stöckinger, Nucl. Phys. B808 (2009) 88, arXiv:0807.4424.

D11 matching (defined, not verified).

M0, baseline. S_{inv}, not S_D. $a_1 = \frac{1}{2}\tau\eta - h$ is XVI-O's vertex (`lem:d11-prefactor`: $\Pi = (i\kappa^2/4)T$).

M1, exact integrand identity (computed). For all 100 pairs (A, B) of independent components,

$$-i\,\mathcal{P}\,\frac{N^P_{AB}(l,p)}{(l^2+i0)((l-p)^2+i0)} = \frac{i\kappa^2}{4}\,R^{\mu\nu}_A R^{\rho\sigma}_B V^{ab}_{\mu\nu} V^{cd}_{\rho\sigma}\big[\tilde{C}_{ac}(l)\tilde{C}_{bd}(l-p) + \tilde{C}_{ad}(l)\tilde{C}_{bc}(l-p)\big],$$

where:

- $\tilde{C}_{ab}(k) = k_a k_b\, i/(k^2 + i0)$ is the Fourier transform of $C = -\partial\partial D_F$;
- V is XVI-O's own `V_tensor`, imported from `code/matter_loop/d11_tensor.py`;
- R_A is the η-raising map $h^{\mu\nu} = \eta^{\mu a}\eta^{\nu b} h_{ab}$, with orbit multiplicity 2 for off-diagonal A;

- $\mathcal{P} = \kappa^2/2$ is P's prefactor, which carries the symmetry factor $\frac{1}{2}$.

The identity matches $\Gamma_1^{(2)} = \frac{1}{2}\int h^{\mu\nu}\Pi_{\mu\nu\rho\sigma}h^{\rho\sigma}$ (XVI-O) with P's ordered extraction, and it is bound to the exported D11 integrand by its sha256.

M2, nonlocal agreement on $U = \mathbb{R}^4 \setminus \{0\}$ (defined only). $\langle \mathcal{F}^{-1}[P^{\mathrm{DR}}_{AB}], f\rangle = \langle R_A R_B \Pi^R_{\mathrm{tr}}, f\rangle$ for $f \in \mathcal{D}(U)$, with $\Pi^R_{\mathrm{tr}} = (i\kappa^2/4)\mathcal{K}u^R_\mu$, $\alpha = A^2/60$ and $\beta = A^2/18$ (`lem:BV_rules_on_U`, all 256 components). This needs the evaluated amplitude, which is p15+ work.

M3, local difference (conditional on M2). $L_{\mathrm{match}} = \mathcal{F}^{-1}[P^{\mathrm{DR}}] - RR\,\Pi^R_{\mathrm{tr}}$. The degrees are as follows. The scalar extension u^R_μ has scaling degree 4. The kernel $\mathcal{K}u^R_\mu$ (with $\mathcal{K}$ of order 4) has scaling degree at most 8. A contact term has $\mathrm{sd}(\partial^\alpha\delta) = 4 + |\alpha|$. *Kernel scaling degree at most eight; degree of divergence at most four; hence the local ambiguity has derivative order at most four.*

If M2 holds, then L_{match} is supported at $\{0\}$ and lies in the bounded contact space of Lorentz-covariant even hh structures of derivative order ≤ 4. This space is enumerated: order 0 (μ^4) has dimension 2 (transverse 0); order 2 ($\mu^2 p^2$) has dimension 4 (transverse 1, the linearised Einstein–Hilbert structure); order 4 has dimension 5 (transverse 2). Odd orders cannot occur with four indices built from p and η.

Exactly degree four is an *explicit normalization condition* N4: no μ^4 and no $\mu^2 p^2$ contact. N4 is *not* implied by "no other scale", since μ is a scale. Under N4 the space is the 5-dimensional order-4 span, equal to the hh sector of the counterterm domain. XVI-O's contact ansatz is a candidate list (`NOT_proved_spanning`) with three named order-4 entries, one of them the collective `L4_mixed`; its lower-order entries are excluded by N4. P's list is the explicit one.

M4, finite adjustments. Under M2 and N4, these are booked only as P-side counterterms in the hh sector (without N4 the order-0/2 contacts would have to be added there). u^R_μ is not modified (no $c\,\delta$), so the scalar-extension prescription `analytic_FP_u_mu_R_o70` is preserved.

M5, residue. $r = -i\pi^2$ is an XVI-O-side statement, valid only *after* the constructive holomorphy of $F(\zeta) = (Q - i0)^\zeta$ (`lem:analytic_FP_u_mu_R_o70`). P's $1/\epsilon$ pole is P-side, and no pole–residue identification is asserted.

M6/M7. The XVI-O o70 files are byte-pinned. XVI-O's kernel Ward identity $\partial(\Pi_w + L) = 0$ is *not* functional ST4b, and the projected identity (XVI-N `lem:field-only`) is distinct from bulk restoration.

Hence `D11_matching_defined`=true and `D11_matching_verified`=false.

15 Counterterm domain

Builder/output: `counterterm_domain.py`, `P14_counterterm_domain.json`.

This is an *order-bounded EFT constraint list, not an admission* (XIV-J and XV-B are not opened). The one-loop κ^n part has n external legs and local dimension $4 + n$, with $[\kappa] = -1$; $[h] = [\phi] = [c] = [\bar{c}] = 1$; and $[K'] = [c^{*\prime}] = [\phi^{*\prime}] = [\bar{c}^{*\prime}] = [b] = 2$. The functionals have ghost number 0, are even under $\phi \to -\phi$ (with $\phi^{*\prime} \to -\phi^{*\prime}$), and are Lorentz covariant (parity-odd structures vanish for two-leg terms). The permitted species are the legs of loop-capable vertices; b, $\bar{c}^{*\prime}$ and $b^{*\prime}$ are excluded. The structures are *generated* mechanically from η and the leg momentum and turned into densities, and their ranks are exact. Total derivatives vanish identically in the two-point representation.

- κ^1: empty (a one-leg term with derivatives is a total derivative, and there is no dimension-5 field).
- κ^2: 11 structures.
  - hh, 5: $h\Box^2 h$, $\tau\Box^2\tau$, $(\partial\partial h)\Box\tau$, $(\partial h)\Box(\partial h)$, $(\partial\partial h)^2$;
  - $\bar{c}c$, 2: $\bar{c}_\mu\Box^2 c^\mu$, $\bar{c}_\mu\partial^\mu\Box\partial_\nu c^\nu$;
  - $K'c$, 3: $K'^{mn}\partial_m\partial_n\partial_r c^r$, $K'^m{}_m\Box\partial_r c^r$, $K'^{mn}\Box\partial_m c_n$;
  - $\phi\phi$, 1: $\phi\Box^2\phi$.

Source dependence. K' enters only via $K'c$. $c^{*\prime}$ and $\phi^{*\prime}$ appear first in three-leg terms at κ^3, outside the order bound.

Equivalences. Apply $\sigma_0 = (S_\Psi^{(0)}, \cdot)$ (frozen antibracket) to every generated ghost-number -1, dimension-5 bilinear Ξ. Antifield-linear Ξ are field redefinitions (canonical transformations); field-only Ξ are s-exact. The domain $\cap\,\mathrm{Im}\,\sigma_0$ is given in the summary; the $S_\Psi^{(0)}$ density used for this is checked against the verified Hessians. *Eight candidate directions modulo the computed exact equivalences, before imposing the relevant identity constraints.* This is not a cohomological classification, which needs $D \cap \ker \sigma_0$ (the σ_0-closedness constraints) and the quotient $(D \cap \ker \sigma_0)/(D \cap \mathrm{Im}\,\sigma_0)$. $D \cap \ker \sigma_0$ has not been computed, so the count 8 is an upper bound only. The enumeration itself is unchanged.

Normalisation. $\overline{\mathrm{MS}}$ (pure poles at μ). There is no κ^2 two-derivative counterterm, so neither κ nor the h normalisation is renormalised at this order. This is the *imposed* normalization condition N4 of M3, not a consequence of masslessness. There is no κ^1 counterterm. Finite adjustments stay inside the domain.

16 Executable amplitude rules

Builder/output: `feynman.py`, `shadow.py`, `amplitude_rules.py`, `P14_amplitude_rules.json`, `exports/P14_vertex_tensors.json`, `exports/P14_integrands/*.json`.

Rules.

- *Extraction:* ordered, $V[L_1..L_n] = \partial^L_{L_n} \cdots \partial^L_{L_1} S$, with leg 1 leftmost; the Feynman vertex is $i\kappa^n V$.
- *Graviton legs:* independent components; the multiplicity (1 diagonal, 2 off-diagonal) enters only the tensor form.
- *Ghost:* $\bar{c}$ is on the left in $\bar{c}ch$ and in M_0; a closed ghost loop carries -1.
- *Source:* K' sits left of c in $K'ch$ and in the Γ_1 coefficient $K'_{a'}G_{a'\nu}c^\nu$.
- *Symmetry factors:* $S_f = \frac{1}{2}$ for D01, D02, D11, D12, D14a, D14c and $S_f = 1$ for D03, D13, D14b.
- *Ghost-loop sign:* -1 for D03 and D14b.

The vertex tensors include every derivative term of the ghost-graviton vertex ($\bar{c}\,c\,h = 2ia\,D^\rho[\mathcal{L}_c h]$, including the transport term $c^r\partial_r h$) and the K' vertex.

Controls.

W1. Berezin/Wick shadow theory: the symmetry factors and Grassmann signs are fixed independently.

W2. A tensor-route recomputation of the multiplicities.

W3. Closed forms of the vertices.

W4. The primary evidence is the generic symbolic *off-shell* Ward identities at κ^1 and κ^2. A supplementary check is the "Ward contraction with two free on-shell graviton legs and an off-shell gauge leg". W4 also checks Bose symmetry.

W5. Independent sympy regeneration of D14a, D14b, D14c, D11 and D13 *only*, plus a Bose-symmetry check of D01.

W6 (p14r). Independent regeneration of D01, D02, D03 and D12 from the one-loop expansion of the functional (super)trace log of the field-dependent Hessian of S_Ψ (`hessian_route.py`), not from the Feynman-rule assembler:

$$i\Gamma^{AB} = -\tfrac{1}{2}\,\mathrm{Tr}(GU_2^{AB}) + \tfrac{1}{2}\,\mathrm{Tr}(GU_1^A GU_1^B)\ \text{(bosons)}, \qquad i\Gamma^{AB} = -\,\mathrm{Tr}(GU_1^A GU_1^B)\ \text{(ghosts)}.$$

The ingredients are an own monomial differentiation of ℓ_3, ℓ_4, an own inverse of the full 14×14 (h, b) Hessian, and the ghost and scalar Hessians from the S_Ψ densities. Structural checks: $X^{(1)}$ has no field-only term and $X^{(2)} = 0$, so there is no $\bar{c}chh$ vertex. The route equals the exported integrands exactly, component by component (D01 100, D02 94, D03 100, D12 94; 0 mismatches), and it also rejects the tadpole-factor, ghost-sign, multiplicity and transport-term mutations.

Mutations that must be rejected (all rejected): wrong tadpole factor, wrong ghost-loop sign, off-diagonal multiplicity, reversed source/ghost order, and omitted ghost transport term. D13 has priority; its integrand is exported with prefactor $i\kappa^2$ and order (K', c). Integrands are symbolic (not integrated); the per-graph table is in the summary above.

Isolated-copy gate tests (p14r). The test copies contain every frozen input (the top-level directories of `frozen_p13_inputs.json`, including `tests/`); the p14 copies lacked `tests/`, so every copy was unfrozen. The expected outcomes are:

- the unmutated copy has freeze true, and publication is accepted;
- each P.5-only mutation (DRED mutation not rejected, counterterm flag, amplitude mutation) keeps the freeze *true*, turns the named contract flag false, and is rejected;
- the hard-stop mutations keep the freeze true and are rejected;
- the classical mutations make the freeze false and are rejected.

17 P.5 entry dossier

One row per claim, each resolved mechanically by `dossier_p14.py`: the function exists, the certificate key holds the expected value, and the test exists. The five P.3/P.4 chains are carried over unchanged as frozen inputs; the P.5 entry chain is new.

id	claim	file :: function	certificate key = value	test
P.3: projection -> dependency rows -> graphs [frozen P.3/P.4 input]				
P3.1	ch output projection fixed (c^1 h^1, ghost number 1, kappa^2, one loop, transformed variables)	`ch_source_audit.py :: main`	`ch_source_audit/ch_projection_defined = true` `ch_source_audit/ch_pattern/ghost_number = 1`	`ch_rows_and_quantum_input_slots`
P3.2	vertex inventory generated from S_Psi (not copied)	`ch_source_audit.py :: S_psi_inventory, vertex_options`	`ch_source_audit/vertex_inventory = {"kappa0": ["K.c",...` `ch_source_audit/checks/b_has_no_interaction_vertex = true`	`ch_rows_and_quantum_input_slots`
P3.3	dependency rows exhaustive: regenerated rows equal the published rows	`ch_source_audit.py :: audit, regenerate_and_compare`	`ch_source_audit/checks/regenerated_rows_match_published = true` `ch_source_audit/regeneration/complete = true`	`e2e_dropped_hstar_c_row`

P3.4	6 rows: 3 surviving, 3 derived zero (S2, cbar*, b* rows)	`ch_source_audit.py :: audit`	`ch_source_audit/rows_total = 6`
			`ch_source_audit/rows_surviving = 3`
			`ch_source_audit/rows_derived_zero = 3`
				`ch_rows_and_quantum_input_slots`

P3.5	enumeration lemma (justifies max_len = 2) checked mechanically	`ch_source_audit.py :: enumeration_lemma`	`ch_source_audit/enumeration_lemma/holds = true`	`enumeration_lemma`

P3.6	graph topologies generated and named; no unnamed graph	`ch_source_audit.py :: enumerate_graphs, name_graph`	`graph_inventory/names_TBD = []`
			`graph_inventory/graphs = {"D03_hh_ghost_loo...`
			`ch_source_audit/checks/names_TBD_empty = true`
				`ch_rows_and_quantum_input_slots`

P3.7	P.3 flags	`release_p14.py :: classical_block`	`RELEASE_STATUS/classical/flags/ch_projection_defined = true` `RELEASE_STATUS/classical/flags/ch_dependency_inventory_complete = true`	`milestones_consistent`
P.4: classical coefficient -> export -> verification [frozen P.3/P.4 input]				
P4.A1	C3 boundary identity sqrt(-g)R - L_GG = -d_mu B^mu on generic jets	`boundary_identity.py :: generic_jet_proof`	`boundary_identity/C3_proved = true`	`C3_boundary_identity_generic_and_conformal`
P4.A2	ell_2, ell_3, ell_4 built and exported; export hash == quartic cert == master input	`quartic.py :: build_and_export`	`RELEASE_STATUS/classical/hash_consistency/consistent = true` `quartic_verified/ell4_sha256 = "33ac9e49c4d1644ee...`	`stale_ell4_hash_blocks_master`
P4.A3	C1 inverse, C2 det/trace-log (10 symbolic components)	`quartic.py :: inverse_control, det_tracelog_generic`	`quartic_verified/acceptance_controls_deterministic/C1 = true` `quartic_verified/acceptance_controls_deterministic/C2 = true`	`quartic_acceptance_all_deterministic`
P4.A4	C4 export == independent evaluator (exact polynomial identity)	`quartic.py :: export_identity`	`quartic_verified/acceptance_controls_deterministic/C4 = true`	`quartic_acceptance_all_deterministic`
P4.A5	C5/C6 ell_3 = XII-C, ell_2 = XI-A Fierz-Pauli modulo divergences (pinned XVI-N grav_density)	`quartic.py :: regression_deterministic, xvin_symbolic`	`quartic_verified/acceptance_controls_deterministic/C5 = true` `quartic_verified/acceptance_controls_deterministic/C6 = true`	`quartic_acceptance_all_deterministic`

id	claim	file :: function	certificate key = value	test
P4.A6	C7 gravitational Noether identities on the exports, kappa^0..kappa^2	grav_noether.py :: noether	quartic_verified/controls/C7_accepted_orders = *all entries true*	grav_noether_exact_and_mutation
P4.A7	quartic mutations rejected; quartic flag	quartic.py :: main	quartic_verified/mutations_all_rejected = true RELEASE_STATUS/classical/flags/quartic_generator_verified = true	e2e_corrupted_ell4_coefficient
P.4: transformations -> source terms [frozen P.3/P.4 input]				
P4.B1	ghost law derived from s^2 = 0 (unique lambda = 1)	brst_from_geometry.py :: solve_ghost_law	brst_derivation/ghost_law_derivation/lam_solutions = ["1"]	brst_and_source_terms_derived
P4.B2	s_2 derived zero in every sector; s nilpotent at all orders	brst_from_geometry.py :: per_sector_expansion, nilpotency	brst_derivation/s2_per_sector = {"h": "derived_zer... brst_derivation/nilpotency_exact_all_orders = true	brst_and_source_terms_derived
P4.B3	sigma_A derived from the generator rule (unique); antifield-linear, no kappa^2 X terms	source_action.py :: derive_sigma, build_X_inv	source_action/sigma_unique = true source_action/X_kappa2_terms = 0 source_action/antifield_linear = true	brst_and_source_terms_derived e2e_flipped_odd_sign e2e_omitted_cstar_term
P.4: gauge map -> Hessians -> source convention [frozen P.3/P.4 input]				
P4.C1	antifield shift orientation derived (eps = -1); S_Psi = S + s Psi mod d	gauge_fixing_map.py :: build, shift_map	gauge_fixing_map/derived_eps = -1 gauge_fixing_map/S_Psi_equals_S_plus_sPsi_mod_d = true	e2e_altered_gauge_shift
P4.C2	canonicity of the map on all 299 (anti)field pairs	gauge_fixing_map.py :: canonicity	gauge_fixing_map/canonicity/pass = true gauge_fixing_map/canonicity/pairs_tested = 299	gauge_hessian_generic_and_full_canonicity
P4.C3	graviton Hessian at generic symbolic k: Schur complement k^2 P, propagator (k^2 P)^-1	gauge_fixing_map.py :: hessian_check	gauge_fixing_map/propagators/graviton/schur_equals_k2P = true gauge_fixing_map/propagators/graviton/propagator_hh_equals_inv_k2P = true	gauge_hessian_generic_and_full_canonicity
P4.C4	ghost (i cbar Box c) and scalar (-Box) kinetic terms; propagators match transformed Hessians	gauge_fixing_map.py :: ghost_kinetic_check	gauge_fixing_map/propagators/ghost/match = true gauge_fixing_map/propagators/scalar/match = true gauge_fixing_map/propagators_match_transformed_Hessians = true	gauge_hessian_generic_and_full_canonicity
P4.C5	source convention Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/d-Phi): classical pullback, NOT a quantum theorem	gauge_fixing_map.py :: build	gauge_fixing_map/source_identity_relation_defined = true gauge_fixing_map/source_identity_relation/status = "DEFINED classical..	gauge_hessian_generic_and_full_canonicity
P4.C6	gauge-map flag	release_p14.py :: classical_block	RELEASE_STATUS/classical/flags/gauge_fixing_map_constructed = true	milestones_consistent
P.4: master component -> residual record -> acceptance gate [frozen P.3/P.4 input]				
P4.D1	master equation at kappa^0: every antifield-number component; grav af0 record; gate	master_kappa2.py :: run, XX_by_order, grav_deterministic	master_kappa2/table_by_kappa_and_antifield_number/kappa0 = *all entries true* master_kappa2/details/grav_af0_deterministic/kappa0/Q_terms = [0, 0, 0, 0] master_kappa2/details/grav_af0_deterministic/kappa0/Q_identically_zero = true RELEASE_STATUS/classical/residual_records/master_grav_per_order/kappa0/accepted = true	residual_records_regenerated_and_agree
P4.D2	master equation at kappa^1: every antifield-number component; grav af0 record; gate	master_kappa2.py :: run, XX_by_order, grav_deterministic	master_kappa2/table_by_kappa_and_antifield_number/kappa1 = *all entries true* master_kappa2/details/grav_af0_deterministic/kappa1/Q_terms = [0, 0, 0, 0] master_kappa2/details/grav_af0_deterministic/kappa1/Q_identically_zero = true RELEASE_STATUS/classical/residual_records/master_grav_per_order/kappa1/accepted = true	residual_records_regenerated_and_agree

id	claim	file :: function	certificate key = value	test
P4.D3	master equation at kappa^2: every antifield-number component; grav af0 record; gate	master_kappa2.py :: run, XX_by_order, grav_deterministic	master_kappa2/table_by_kappa_and_antifield_number/kappa2 = *all entries true* master_kappa2/details/grav_af0_deterministic/kappa2/Q_terms = [0, 0, 0, 0] master_kappa2/details/grav_af0_deterministic/kappa2/Q_identically_zero = true RELEASE_STATUS/classical/residual_records/master_grav_per_order/kappa2/accepted = true	residual_records_regenerated_and_agree
P4.D4	no required master residual; no unconditional pass field	master_kappa2.py :: master_residuals	master_kappa2/required_master_residuals = []	master_no_unconditional_pass_and_deterministic
P4.D5	count/flag invariant on every record; quartic and master records agree	certio.py :: invariant_violations, cross_check	RELEASE_STATUS/classical/deterministic_prerequisites/residual_record_invariants_all_certificates = true RELEASE_STATUS/classical/residual_records/cross_certificate/agree = true	certificate_mutation_rejects_publication certio_rejects_inconsistent_record_before_serialization
P4.D6	acceptance gate: release_claim_accepted and classical readiness (P.4)	release_p14.py :: classical_block	RELEASE_STATUS/classical/release_claim_accepted = true RELEASE_STATUS/classical/classical_readiness_for_ch_identity = true RELEASE_STATUS/classical/flags/required_classical_BV_components_verified = true	release_accepted_and_hard_stops
P4.D7	publication gate re-invoked before every pdf/zip/manifest stage, also on resumed builds	build.py :: publication_gate, pdf_readback	(build-time gate)	resumed_stage_build_blocked
P.5_entry: quantum object -> prescription/D11 matching -> counterterm domain -> amplitude rules -> gate				
P5E.1	quantum object in transformed variables: S_Psi parts, eps = -1, b retained (elimination makes S quadratic in cbar*'), Gamma_1 independent of cbar*', b*'; pullback != quantum theorem	quantum_object.py :: main, patterns, b_elimination	quantum_object/quantum_action_defined = true quantum_object/Q1_transformed_variables/eps_derived = -1 quantum_object/Q2_fields_antifields_b/b_decision = "RETAINED and integrated" quantum_object/Q2_fields_antifields_b/Gamma1_independent_of = ["bs", "cbs"] quantum_object/Q5_kappa_loop_bookkeeping/ok = true quantum_object/Q6_pullback_vs_theorem/quantum_gauge_change_theorem_asserted = false	quantum_object
P5E.2	ONE prescription: operational DRED (rules R1-R6), MS-bar, Feynman i0; scaleless rules defined but not applied; quantum_prescription_defined requires the verified DRED rules	prescription_d11.py :: main	prescription_D11_matching/quantum_prescription_defined = true prescription_D11_matching/prescription/id = "DRED_MSbar_FeynmanI0" prescription_D11_matching/prescription_checks = *all entries true* prescription_D11_matching/prescription_checks/operational_DRED_rules_verified = true prescription_D11_matching/D12_status = "Missing"	prescription_and_D11_matching
P5E.2b	operational DRED: g4 = ghat + gtilde (tr d, 4-d), l and p in QdS, l_r = l.e_(r) lifting, ghat-projected Gram/PV reduction, contractions before Laurent with d symbolic; evanescent parts retained; premature d=4, tr ghat=4 and expand-before-contract are rejected	dred.py :: reduce_tensor, eta_trace_Gt, toy_four_d_part, toy_evanescent_trace, known_reductions, lift_exported, main	dred_rules/dred_rules_verified = true dred_rules/mutations_all_rejected = true dred_rules/mutations/premature_d4/rejected = true dred_rules/mutations/trace_ghat_eq_4/rejected = true dred_rules/mutations/contract_after_laurent/rejected = true dred_rules/toy_checks/T1_four_d_part/finite = "a/8 + b/4" dred_rules/toy_checks/T2_evanescent_trace/finite = "-a/2" dred_rules/known_reductions/pass = true dred_rules/evanescent_parts_present_in_exports = true dred_rules/loop_evaluations_performed = false	dred_rules_certificate dred_premature_d4_changes_finite_part_and_is_caught dred_evanescent_trace_is_d_minus_4_times_pole dred_trace_ghat_eq_4_rejected dred_known_reductions_and_contraction_engine

id	claim	file :: function	certificate key = value	test
P5E.3	D11 matching DEFINED (M1 exact integrand identity vs XVI-O V_tensor, 100 pairs; M3 CONDITIONAL on M2: sd(u^R) 4, kernel sd <= 8, contact derivative order <= 4, contact space orders 0/2/4 = 2/4/5, degree 4 imposed by normalization condition N4; no c delta; r = -i pi^2 XVI-O side only); NOT verified	prescription_d11.py :: M1_identity, M3_local_space, M3_contact_analysis	prescription_D11_matching/D11_matching_defined = true prescription_D11_matching/D11_matching_verified = false prescription_D11_matching/matching/M1_integrand_identity/mismatches = [] prescription_D11_matching/matching/M1_integrand_identity/component_pairs_checked = 100 prescription_D11_matching/matching/M1_integrand_identity/bound_to_export = true prescription_D11_matching/matching/M3_local_difference/labels/scalar_extension_scaling_degree = 4 prescription_D11_matching/matching/M3_local_difference/labels/kernel_scaling_degree_bound = 8 prescription_D11_matching/matching/M3_local_difference/labels/local_contact_derivative_order_bound = 4 prescription_D11_matching/matching/M3_local_difference/lower_degree_contacts_excluded_by_N4/order_2_mu2 = 4 prescription_D11_matching/matching/M3_local_difference/dimension = 5 prescription_D11_matching/matching/M3_local_difference/transverse_subspace_dimension = 2 prescription_D11_matching/matching/M4_finite_adjustments/u_R_c_delta_added_by_P = false prescription_D11_matching/matching/M5_residue/r = "-i * pi**2"	prescription_and_D11_matching
P5E.4	counterterm domain: kappa^1 empty, kappa^2 basis 11 (hh 5, cbar c 2, K'c 3, phi phi 1) generated mechanically; eight candidate directions modulo the computed exact equivalences, before imposing the identity constraints (no cohomological classification: D cap ker sigma_0 not computed); constraint list, not an admission	counterterm_domain.py :: structures, admissible_pairs, rank, main	counterterm_domain/counterterm_domain_defined = true counterterm_domain/kappa1/basis_size = 0 counterterm_domain/kappa2/basis_size = 11 counterterm_domain/equivalences/quotient_dimension = 8 counterterm_domain/equivalences/D_cap_ker_sigma0_computed = false counterterm_domain/equivalences/S0_density_check/ok = true counterterm_domain/XIV_J_opened = false counterterm_domain/XV_B_opened = false	counterterm_domain
P5E.5	amplitude rules: vertices (incl. ghost-graviton derivative terms, K' vertex), propagators, ordering, multiplicities, symmetry factors, signs; symbolic integrands; W1-W6 controls (W5 regenerates only D14a-c, D11, D13; W6 regenerates D01, D02, D03, D12)	amplitude_rules.py :: build_integrands, verify, W2, W3, W4, W5	amplitude_rules/amplitude_rules_verified = true amplitude_rules/controls/W1_wick_berezin_shadow/pass = true amplitude_rules/controls/W4_ward_bose/pass = true amplitude_rules/integrands_only = true amplitude_rules/loop_evaluations_performed = false amplitude_rules/graphs/D12/D12_status = "Missing"	amplitude_rules_and_mutations
P5E.5b	independent Hessian/Tr-log regeneration of D01, D02, D03, D12 from the field-dependent Hessian of S_Psi (own (h,b) inverse; not the Feynman-rule assembler) == exported integrands	hessian_route.py :: ell_table, graviton_inverse, ghost_inverse, reference, W6, compare_exports	amplitude_rules/controls/W6_hessian_route/pass = true amplitude_rules/W6_exports/all_agree = true amplitude_rules/W6_exports/D01/n_mismatches = 0 amplitude_rules/W6_exports/D03/n_mismatches = 0	hessian_route_regenerates_D01_D02_D03_D12
P5E.6	mutations rejected: tadpole factor, ghost-loop sign, off-diagonal multiplicity, source/ghost order, ghost transport term	amplitude_rules.py :: apply_mutations, main	amplitude_rules/mutations_all_rejected = true amplitude_rules/mutations/omit_ghost_transport_term/rejected = true amplitude_rules/mutations/reversed_source_ghost_order/rejected = true	amplitude_rules_and_mutations
P5E.7	Berezin/Wick shadow fixes symmetry factors and Grassmann signs independently	shadow.py :: expect, gamma1, run_shadows	amplitude_rules/controls/W1_wick_berezin_shadow/pass = true	amplitude_rules_and_mutations

id	claim	file :: function	certificate key = value	test
P5E.8	P.3/P.4 freeze bound to the p13 tree (independent rebuild passed; same-box caveat); frozen code and values unchanged	`freeze_p13.py :: freeze, F1_independent, F2_frozen_code, F3_frozen_values`	`RELEASE_STATUS/P3_P4_frozen_as_P5_inputs` = true `RELEASE_STATUS/p13_independent_rebuild/bound_to_source_tree_sha256` = "2b1baf335468944aa... `RELEASE_STATUS/p13_independent_rebuild/caveat` = "the p13 independe... `RELEASE_STATUS/freeze/F2/changed_or_missing` = [] `RELEASE_STATUS/freeze/F3/differences` = []	`freeze_binding`
P5E.9	P.5 entry gate: earned iff all five contract flags hold; loop/restoration flags NOT earned	`release_p14.py :: _main, p5_entry_flags, milestones_p14`	`RELEASE_STATUS/P5_entry_flags` = *all entries true* `RELEASE_STATUS/P5_entry_earned` = true `RELEASE_STATUS/hard_stops_hold` = true `RELEASE_STATUS/hard_stops/quantum_inputs_evaluated` = "not_earned" `RELEASE_STATUS/hard_stops/projected_identity_verified` = "not_earned" `RELEASE_STATUS/hard_stops/bulk_symmetry_restored` = false `RELEASE_STATUS/D11_matching_verified` = false `RELEASE_STATUS/D12_status` = "Missing" `RELEASE_STATUS/hard_stops/O1_closure` = "NOT_CLOSED"	`p5_entry_gate_and_hard_stops`
P5E.10	publication gate (p14) re-invoked before pdf/zip/manifest stages	`build.py :: publication_gate, pdf_readback`	(build-time gate)	`resumed_stage_build_blocked`
Quantum input slots (named dependencies, NOT evaluated coefficients)				
slot	hh at kappa^2 (Γ_1 pattern `h^2`)	`ch_source_audit.py :: audit`	`ch_source_audit/surviving_rows/0` = D01, D02, D03, D11, D12	`ch_rows_and_quantum_input_slots`
slot	K'c at kappa^2 (Γ_1 pattern `K.c`)	`ch_source_audit.py :: audit`	`ch_source_audit/surviving_rows/1` = D13	`ch_rows_and_quantum_input_slots`
slot	h one-point at kappa (Γ_1 pattern `h`)	`ch_source_audit.py :: audit`	`ch_source_audit/surviving_rows/2` = D14a, D14b, D14c	`ch_rows_and_quantum_input_slots`

Reproducibility flags (package evidence, read from RELEASE_STATUS at build time; p14 flags bound to the p14 tree): `author_rebuild_passed` = false; `independent_rebuild_passed` = false; `P3_P4_frozen_as_P5_inputs` = true; p13 `independent_rebuild_passed` = true (bound to the p13 source tree `2b1baf335468944a...`; caveat: the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it was NOT run on a second machine).

18 Reproducibility contract (p14r)

- `author_rebuild_passed` is bound to the *p14r* source tree (`reproducibility/author_rebuild_report_p14r.json`).
- The p14r `independent_rebuild_passed` is `pending`.
- The p13 independent result is recorded separately and bound to the p13 tree; this is the same-box caveat above.
- The source-only archive is `XVI-P-p14r-source-only.zip`, with reference values in `reproducibility/reference_values_p14r.json` and entrypoint `python3 tools/rebuild_from_source.py`.
- The p13 source-only zip, its reference values and the independent report are archived unchanged in `archival/p13/`. The p14 release (zip, reference values, author report) is archived in `archival/p14/`.

Non-claim 1. No loop is evaluated, no one-loop coefficient is fixed, and no symmetry is restored. P.5 entry fixes a contract: an action, a prescription, a matching *definition*, a counterterm *domain* and integrands. P.5 calculation and P.5 identity are open. The prepared S_D keeps $(S_D, S_D) \neq 0$ (XVI-N prop:broken-square), and no completion contradicting XVI-N thm:massless-obstruction is claimed. En01, Em01, Em02-B/D, En02, VII-E and VIII-B are not premises.

19 Frozen classical record (P.3/P.4, unchanged since p13)

Frozen P.5 inputs.
The sections below are the p13 classical record, carried over verbatim (only the release-gate and test file names are updated). The code that produces it is byte-identical to the p13 source tree `2b1baf33...`; its certificates keep the names `P13_*` and are regenerated in every p14 build. The regenerated values are compared with `archival/p13/reference_values_p13.json` (`freeze_p13.F3_frozen_values`).

20 Conventions

Declared once in `conventions.py`.

$\eta = \mathrm{diag}(+,-,-,-)$; $g_{\mu\nu} = \eta_{\mu\nu} + \kappa h_{\mu\nu}$ (affine in h); $S_{\rm EH} = -(2/\kappa^2)\int \sqrt{-g}R = \sum_k \kappa^{k-2}\int \ell_k$ with $\ell_k = -2[\varepsilon^k]L_{\Gamma\Gamma}$, $L_{\Gamma\Gamma} = \sqrt{-g}\, g^{\mu\nu}(\Gamma^\lambda_{\mu\sigma}\Gamma^\sigma_{\nu\lambda} - \Gamma^\lambda_{\mu\nu}\Gamma^\sigma_{\lambda\sigma})$; $R^\rho{}_{\sigma\mu\nu} = \partial_\mu\Gamma^\rho_{\nu\sigma} - \partial_\nu\Gamma^\rho_{\mu\sigma} + \Gamma\Gamma - \Gamma\Gamma$, $R_{\sigma\nu} = R^\rho{}_{\sigma\rho\nu}$. Massless scalar $S_\phi = \frac{1}{2}\int \sqrt{-g}g^{\mu\nu}\partial_\mu\phi\partial_\nu\phi$ with $\sqrt{-g}g^{\mu\nu} = \eta + \kappa a_1 + \kappa^2 a_2$ (XI-A, re-derived in `P13_source_action`). Ghost scaling $\xi^\mu = \kappa a c^\mu$, $a = \frac{1}{2}$. Antibracket (XVI-N def:antibracket)

$$(F,G) = \sum_A \int \Big[\frac{\delta^R F}{\delta\Phi^A}\frac{\delta^L G}{\delta\Phi^*_A} - \frac{\delta^R F}{\delta\Phi^*_A}\frac{\delta^L G}{\delta\Phi^A}\Big], \qquad s\Phi = (S,\Phi),\ s\Phi^* = (S,\Phi^*).$$

Canonical pair for the graviton: (h_{mn}, K^{mn}), $m \le n$. Divergence test (exact): a local density without constant term is a total divergence iff all its Euler derivatives vanish (contractible jet space).

21 C3 repaired: the boundary identity and its orientation

Builder/output: `boundary_identity.py`, `P13_boundary_identity.json`.

Defect. The p09/p10 control C3 evaluated the identity on $g = \eta + A(x^0)\,e_{00}$. This metric is locally flat ($x^0 \mapsto \int\sqrt{1+A}\,dx^0$ removes A); its only nonzero symbol is $\Gamma^0{}_{00}$, and $\sqrt{-g}R = L_{\Gamma\Gamma} = B^\mu = \partial_\mu B^\mu = 0$ identically. The check was therefore vacuous and could not detect a sign; the source comment "keeps ∂B nontrivial" was false. p09/p10 also stated the orientation as $\sqrt{-g}R - L_{\Gamma\Gamma} = +\partial_\mu B^\mu$. That orientation is wrong.

Proposition 2 (Boundary identity, corrected orientation)**.** *With* $B^\mu = \sqrt{-g}\,(g^{\mu\nu}\Gamma^\lambda_{\nu\lambda} - g^{\lambda\nu}\Gamma^\mu_{\lambda\nu})$,

$$\sqrt{-g}\,R - L_{\Gamma\Gamma} = -\,\partial_\mu B^\mu.$$

Deterministic proof (three parts). (i) *Orientation.* The statement above, with $-\partial_\mu B^\mu$, replaces the p09/p10 statement; it is used in `conventions.BOUNDARY_IDENTITY`, the exports and all certificates. (ii) *Nontrivial conformal control.* For $g = f\eta$ with f a generic function of all four coordinates, sympy gives exactly $\sqrt{-g}R = -3\Box f + \frac{3}{2f}(\partial f)^2$, $L_{\Gamma\Gamma} = \frac{3}{2f}(\partial f)^2$, $B^\mu = 3\partial^\mu f$. The identity holds with residual 0; the p10 orientation leaves $-6\Box f \neq 0$. (iii) *Generic jet proof.* Let G^{ab} be ten *free* symmetric symbols (the inverse metric), and let the 40 first jets $g_{ab,c}$ and 100 second jets $g_{ab,cd}$ be free. Use $\partial_c G^{ab} = -G^{ar}g_{rs,c}G^{sb}$ and $\partial_c\sqrt{-g} = \frac{1}{2}\sqrt{-g}\,G^{ab}g_{ab,c}$ (Jacobi; both checked exactly via $\partial\det = \mathrm{tr}(\mathrm{adj}\,\partial g)$ and $\mathrm{adj}\,g = \det g\,\mathbf{1}$). Then $(\sqrt{-g}R - L_{\Gamma\Gamma} + \partial_\mu B^\mu)/\sqrt{-g}$ is a polynomial in 150 variables over $\mathbb{Q}$ (2406, 1764 and 2442 terms in the three pieces) and is *identically zero* for all ten components, first and second jets; with the p10 orientation it has 2736 nonzero terms. Because the identity holds for free G, it holds a fortiori for $G = g^{-1}$. □

Integrated-action equivalence is unaffected. For compactly supported h, $\int\partial_\mu B^\mu = 0$ with either sign, so $S_{\rm EH} = -(2/\kappa^2)\int L_{\Gamma\Gamma}$ and $\ell_k = -2[\varepsilon^k]L_{\Gamma\Gamma}$ are unchanged. The error concerned the stated local identity, not the action.

22 Quartic generator: deterministic acceptance

Builder/output: `quartic.py`, `grav_noether.py`, `series_eval.py`, `jetpoly.py`; `P13_quartic_verified.json`.

ℓ_2, ℓ_3, ℓ_4 are built from $L_{\Gamma\Gamma}$ and exported (`exports/ell{2,3,4}_generating_polynomial.srepr`; ℓ_4 sha256 `33ac9e49...e123`, unchanged from p10; 3360 monomials, a reproducibility count only). *Acceptance* consists only of exact symbolic identities:

control	identity (exact)	status
C1	$q\,q^{-1} = 1 + O(\varepsilon^5)$, all 10 symbolic components	deterministic, holds
C2	$\log(-\det q) = \mathrm{tr}\log(\eta^{-1}q)$, $[\sqrt{\cdot}]^2_{\mathrm{series}} = -\det q$ to ε^4, 10 comp.	deterministic, holds
C3	$\sqrt{-g}R - L_{\Gamma\Gamma} = -\partial_\mu B^\mu$ (generic jets + conformal control)	deterministic, holds
C4	export ℓ_k = independent evaluator in $\mathbb{Q}$[50 jet symbols], $k = 2,3,4$	polynomial identity
C5	$E_{h_{mn}}(\ell_3 - \ell_3^{\text{XII-C}}) = 0$, all 10 components	deterministic, holds
C6	$E_{h_{mn}}(\ell_2 - \ell_2^{\text{FP}}) = 0$, all 10 components	deterministic, holds
C7	$E_{c^\mu}(s_0\ell_{n+2} + s_1\ell_{n+1}) = 0$, $n = 0,1,2$, on the exports	deterministic, holds
reg.	numeric evaluator at random jets; plane-wave cubic regression	*sampled regression control only*

Identification of the exported coefficients. The independent evaluator computes $L_{\Gamma\Gamma}(\eta + \varepsilon h)$ from its definition with the series inverse (C1) and the series $\sqrt{-q}$ (C2), truncated at ε^4 (exact for $k \le 4$). C4 proves that the exports equal $-2[\varepsilon^k]L_{\Gamma\Gamma}$ as polynomials, so no sampling is involved. C3 gives $L_{\Gamma\Gamma} = \sqrt{-g}R + \partial_\mu B^\mu$, hence $\int \ell_k = [\kappa^{k-2}]S_{\mathrm{EH}}$ for compact support. C5 compares, with the same sign and normalisation, against XVI-N `grav_density.ell3_and_curvature` (XII-C $\ell_3 = -2(r_3 + b_1 r_2 + b_2 r_1)$), evaluated on *symbolic* second jets; the opposite sign is not a divergence. This is not a claim that XII-C is the quartic, and not a Ward identity. C7 is exact because W_n is linear in c: $W_n \simeq c^\mu E_{c^\mu}(W_n)$, so W_n is a divergence iff $E_{c^\mu}(W_n) \equiv 0$. At κ^1 and κ^2 the s_0 piece alone leaves 159 and 2088 terms per component, so the cancellation is genuinely mixed. Mutations (each rejected by a deterministic test): M1 one ℓ_4 coefficient $\times\frac{7}{5}$ and M2 sign of ℓ_4 (C7 at κ^2); M3 swapped contraction (C4 for all k, C5, C6, C7 at all orders); M4 missing ε^2 coefficient of $\sqrt{-q}$ (C4 for ℓ_4; C7 at κ^2); M5 wrong inverse sign (C1); M6 stale ℓ_4 hash; M7 the p10 orientation $+\partial_\mu B^\mu$ (C3 generic and conformal).

Polarization (one convention throughout). $V_4[h_1,h_2,h_3,h_4] := [t_1t_2t_3t_4]\,\ell_4(\sum_i t_i h_i) = D^4\ell_4(0)[h_1,h_2,h_3,h_4]$, the *unnormalised* fourth variation; the normalised symmetric form would be $V_4/4!$ and is not used. Where the factors enter D02: $1/4!$ appears only in $\ell_4(h) = \frac{1}{4!}V_4[h,h,h,h]$ and is cancelled by the 4! assignments of fields to legs; the vertex rule is $i\kappa^2 V_4[h_1,..,h_4]$ (i from e^{iS}, κ^2 from $S = \sum \kappa^{k-2}\int \ell_k$); the symmetry factor $\frac{1}{2}$ (two legs contracted) enters only at graph level. The same statement appears in `P13_quartic_verified.json`, `exports/V4_grav.json` and `exports/ell4_conventions.json`. `D02_vertex_available`=true, `D02_quantum_evaluated`=false.

23 BRST from finite geometric transformations

Builder/output: `brst_from_geometry.py`.

From $\delta g = \mathcal{L}_\xi g$ with $\xi = \kappa a c$ and the affine metric:

$$sh_{\mu\nu} = \tfrac{1}{\kappa}(\mathcal{L}_\xi g)_{\mu\nu} = a(\partial_\mu c_\nu + \partial_\nu c_\mu) + \kappa a(\mathcal{L}_c h)_{\mu\nu},$$
$$sc^\mu = \kappa a\, c^\rho\partial_\rho c^\mu, \qquad s\phi = \kappa a\, c^\rho\partial_\rho\phi, \qquad s\bar{c}_\mu = ib_\mu, \qquad sb_\mu = 0.$$

The ghost law $s\xi^\mu = \lambda\,\xi^\rho\partial_\rho\xi^\mu$ is an ansatz whose coefficient is *derived*: $s^2h = s^2\phi = 0$ on all jets (115 independent conditions) has the unique solution $\lambda = 1$. **Derived s_2 per sector:** $s_2h = 0$, $s_2c = 0$, $s_2\phi = 0$, $s_2\bar{c} = 0$, $s_2b = 0$. Reason (a derivation, not the p09 policy "no invented s_2"): g is affine in h and $\mathcal{L}_\xi$ is linear in g, so $\kappa^{-1}\mathcal{L}_{\kappa ac}g$ has κ-degree exactly 1; $s\xi$ is quadratic in ξ, so sc has degree exactly 1; $s\phi$ is linear in ξ; the pair $(\bar{c}, b)$ is κ-independent. $s^2 = 0$ holds exactly to all orders. Discriminating control: for $g = \eta + \kappa h + \frac{\kappa^2}{2}h\eta^{-1}h$ the same construction gives $s_2h \neq 0$, and dropping it breaks $s^2 = 0$ at $O(\kappa^2)$.

24 Source action

Builder/output: `source_action.py` (unchanged since p10).

$S_{\rm af} = \sum_A \sigma_A \int \Phi^*_A\, s\Phi^A$ (antifield on the left). Requiring the generator rule $(S, \Phi^A) = s\Phi^A$ with the XVI-N bracket *derives* $\sigma_A = (-1)^{|\Phi^A|}$: $\sigma_h = \sigma_\phi = +1$, $\sigma_c = \sigma_{\bar c} = -1$. The nonminimal coupling $-\int \bar c^{*\mu}\, ib_\mu$ is included; b^* does not occur ($sb = 0$). $X = S_{\rm af}$ has ghost number 0, is antifield-linear, and has 17 / 124 / 0 terms at $\kappa^0/\kappa^1/\kappa^2$. Because X is linear in antifields, $(S,S) = 2(S_{\rm field}, X) + (X,X)$ with $2(S_{\rm field}, X) = 2\int E_\Phi(S_{\rm field})\, \sigma s\Phi$ of antifield number 0 and $(X,X) = -2\sum \Phi^* s^2\Phi \pmod d$ of antifield number 1.

25 Gauge-fixing map in transformed variables

Builder/output: `gauge_fixing_map.py`.

$\Psi = -i\int \bar c_\mu (2D^\mu[h] - b^\mu)$. The orientation $\varepsilon = -1$ of $\Phi^*_A = \Phi^{*\prime}_A + \varepsilon\, \delta\Psi/\delta\Phi^A$ is derived from the Hessians, and $S_\Psi(\Phi, \Phi^{*\prime}) := S(\Phi, \Phi^{*\prime} - \delta\Psi/\delta\Phi) \simeq S + s\Psi$. **Hessians at generic momentum (since p11).** The (h,b) Hessian M is computed for a *symbolic* $k = (k_0, k_1, k_2, k_3)$. With $M = \left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)$, $\det D = -16 \neq 0$, and the Schur complement satisfies $A - BD^{-1}C = k^2 P$ identically in $\mathbb{Q}(k)$, with $\det(k^2 P) = 64(k^2)^{10} \neq 0$. The block-inverse formula then gives $(M^{-1})_{hh} = (k^2 P)^{-1}$ (graviton kernel $K = -\Box P$). The ghost kinetic term $i\bar c \Box c$ and the scalar $-\Box$ are exact as well. No momentum sample is used. **Canonicity** is checked on all 23 antifield components (276 antifield/antifield pairs and 23 antifield/field pairs). **Interface:** $\Gamma_\Psi(\Phi, \Phi^{*\prime}) := \Gamma(\Phi, \Phi^{*\prime} - \delta\Psi/\delta\Phi)$ is a *defined classical pullback/interface* along the canonical antifield shift. It is *not* an independently established quantum gauge-change theorem; no quantum Γ is constructed here.

26 Master equation through $O(\kappa^2)$ by antifield number

Builder/output: `master_kappa2.py`, `P13_master_kappa2.json`.

The κ^2 equation is $2(S_0, S_2) + (S_1, S_1) = 0$. The engine parts (antifield, ghost and scalar) are exact symbolic, as in p10. **Since p11,** the gravitational Noether rows are now *deterministic*: $W_n = s_0\ell_{n+2} + s_1\ell_{n+1}$ is built as an exact jet polynomial from the sha-checked exports, and $E_{c^\mu}(W_n) \equiv 0$ is proved for $n = 0, 1, 2$. The plane-wave evaluation is kept only as a regression control.

Lemma 3 (Multidegree, corrected)**.** *Let P_d project onto field multidegree d and let E_u be the Euler operator of field u. Then $E_u P_d = P_{d-e_u} E_u$. (E_u lowers the u-degree by one and preserves the others; total derivatives preserve multidegree.) Consequently, if $E_u F = 0$ for all u, then $E_u(P_d F) = P_{d-e_u}(E_u F) = 0$ for all d and u, so every homogeneous component $P_d F$ ($|d| \geq 1$) is a divergence; conversely a sum of divergences is a divergence.*

(p10 stated $E_u P_d = P_d E_u$, which is false; the conclusion survives with the corrected intertwining. It is checked on a test polynomial in `tests/test_p14.py`.) The divergence test is applied to the summed component; per-bracket-pair pieces are reported only to exhibit cancellations.

order	component	terms	divergence	per-pair (individually non-divergent)
κ^0	af0 grav Noether ($s_0\ell_2$)	exact	yes	—
	af0 (X_Ψ, X_Ψ) transformed	32	yes	$h, \bar c$
	af1, af2 (both variable sets)	0	yes	—
κ^1	af0 grav ($s_0\ell_3 + s_1\ell_2$)	2148	yes	s_0 alone: 159 per μ
	af0 scalar $2(L_0, X_1) + 2(L_1, X_0)$	20	yes	each piece
	af0 (X_Ψ, X_Ψ) transformed	372	yes	$h, c, \bar c$
	af1 invariant / transformed	180 / 180	yes	h, c
κ^2	af0 grav ($s_0\ell_4 + s_1\ell_3$, $s_2 = 0$)	15428	yes	s_0 alone: 2088 per μ
	af0 scalar $2(L_1, X_1) + 2(L_2, X_0)$	20	yes	each piece
	af0 (X_Ψ, X_Ψ) transformed	2392	yes	h, c
	af1 invariant / transformed	1356 / 1356	yes	h, c, ϕ
	af2 (both)	0	yes	—

Proposition 4 (Classical master equation of $S_{\rm inv}$ through $O(\kappa^2)$)**.** *Under the declared conventions every component of (S,S) through $O(\kappa^2)$ is a total divergence, in invariant and in Ψ-transformed variables, with all mixed cancellations retained:* `required_master_residuals=[]`.

Evidence (deterministic). Engine parts: exact Euler-operator divergence test. Gravitational af0: exact polynomial identity $E_{c^\mu}(W_n) \equiv 0$ on the exports. Structurally, $\sqrt{-g}R$ is a scalar density, so $\delta_\xi(\sqrt{-g}R) = \partial_\mu(\xi^\mu\sqrt{-g}R)$; $L_{\Gamma\Gamma} = \sqrt{-g}R + \partial_\mu B^\mu$ (C3, corrected orientation), and $s\,\partial_\mu B^\mu = \partial_\mu s B^\mu$. af1 rows are $-2\sum \Phi^* s^2\Phi$ and vanish by the derived nilpotency. □

The master certificate has no unconditional `pass` field. `execution_pass` (it ran), `claim_status` (`verified` only if there are no residuals, the hashes match and the quartic prerequisite is verified) and `scope` are separate.

27 *ch* dependency audit and the enumeration lemma

Builder/output: `ch_source_audit.py`.

Pattern: one c, one h; no $\phi, \bar{c}, b$ or antifields; ghost number 1; κ^2; one loop; off-shell. The p10 audit is unchanged. Six rows are possible and three survive; no coefficient is evaluated in this release: the h^*c row (D13); the $K'R_0(c)$ row (hh: D01, D02, D03, D11, D12; D11 is the inherited O5 kernel constructed under prescription, not re-evaluated in this release); and the order-κ one-point row (D14a–c, distinct from D12). The $\bar{c}^{*\prime}$, $b^{*\prime}$, ϕ and $(S_2,\cdot)$ rows are derived zero; `names_TBD`=[].

Lemma 5 (One-loop enumeration, `max_len`=2)**.** *(i) A connected one-loop graph is 1PI iff every internal line lies on its unique cycle, since a line off the cycle is a bridge. So after the quadratic (κ^0, two-field) terms are absorbed into propagators, a one-loop 1PI graph is a cycle of interaction vertices, each with exactly two internal legs. (ii) Every retained interaction vertex has coupling order κ^n with $n \geq 1$. Mechanically, the κ^0 inventory contains only two-field monomials. (iii) Hence a $\kappa^{\leq 2}$ contribution has at most two vertices, and* `max_len`*=2 is exhaustive. (iv) The auxiliary b occurs only in κ^0 bilinears (b^2, bh, $b\bar{c}^*$; checked). Its Gaussian elimination produces only bilinear (kinetic or source-bilinear) terms, never an omitted interaction. (v) Source-linear monomials (one quantum leg) cannot sit on the cycle. Attached off it, they create a bridge, i.e. a one-particle-reducible graph, so they cannot create extra 1PI coefficients.*

The relation $\Gamma_\Psi \leftrightarrow \Gamma$ used to read off rows is the defined classical pullback/interface above, not a quantum gauge-change theorem.

28 Acceptance propagation and release gate

Builder/output: `release_p14.py` (`classical_block`: verbatim fork of the p13 `release.py`, which is frozen and no longer run), `RELEASE_STATUS.json` (key `classical`), `P14_release.json`, `P14_closure_release.json`.

`release_claim_accepted` := `execution_pass` ∧ readiness ∧ (every deterministic prerequisite: C1–C7, quartic mutations rejected, deterministic grav Noether in the master, generic-k Hessian, full canonicity, enumeration lemma) ∧ hash consistency (ℓ_k export file = quartic certificate = master input). The closure artifacts are published only if `release_claim_accepted` and readiness both hold. Otherwise the release deletes any stale closure artifact, writes `P14_release_rejected.json` and exits nonzero, and the build stops before any PDF, zip or sync. End-to-end mutation tests (`tests/test_p14.py`, each in an isolated copy of the package) run mutation → stage residual or failed prerequisite → flag false → readiness false → publication rejected. They cover a flipped odd-derivative sign, a corrupted ℓ_4 coefficient in the export file, a dropped h^*c row, an altered gauge shift, an omitted c^* term, and a stale ℓ_4 hash (the recorded hash is actually mutated).

29 Inherited D11 context (o70-sync, read-only)

The following inherited notes are reproduced unchanged for context; they are not modified by p13 or p14. **o70-sync.** Four mathematical prerequisites Constructed: fixed u_μ^R (analytic-family residue $r = -i\pi^2$, proof `lem:analytic_FP_u_mu_R_o70`); global $T = \mathcal{K}u$ with α, β *derived* ($A^2/60$, $A^2/18$), all 256 components symbolic; W_w jet-pullback as the *only* active reference (`W_w_jet_pullback_via_Pi_tr_o68`); contact moments for L with `kernel_two_point_Ward` (not functional ST4b). Kernel counterterm earned under prescription after prerequisites pass. UV subtraction vs finite repair kept separate. Corpus $\mathcal{P}$ (trace reversal) not reused for $P_{\rm tr}$. Bulk `symmetry_restored=false`; O.1 NOT_CLOSED. $\Pi^R_{\rm tr}|_U = (i\kappa^2/4)T$ (prefactor retained). $s_0 C_L$ not evaluated (minimal release).

30 Massless scalar matter-loop subsector (o70-sync)

Subsector verdict: Constructed under prescription (kernel).

Four prerequisites Constructed; Gate 3 spanning still Partial (candidate_list). Bulk O.1 NOT_CLOSED; insertion freeze unchanged; Em01 not a premise. *D11_kernel_counterterm* = earned only after math prerequisites pass (development mark: Conditional).

30.1 Gate 1: mixed scalar Noether (retained)

Coupling-stripped $S_{\phi,i} = \frac{1}{2}\int a_i^{\mu\nu}\partial_\mu\phi\partial_\nu\phi$ ($a_0 = \eta$). With BRST ghost normalization $a = \frac{1}{2}$ (distinct from D11 coefficients α, β) and $sq = \kappa a\,\mathcal{L}_c q$, the densitized identities

$$s_0 a_1 = a\,\mathcal{D}_c\eta, \qquad s_0 a_2 + s_1 a_1 = a\,\mathcal{D}_c a_1$$

are verified by jet-polynomial substitution (executable). The boxed density identity

$$\boxed{s_0\mathcal{L}_2 + s_1\mathcal{L}_1 = a\,\partial_\rho(c^\rho\mathcal{L}_1)}$$

follows and integrates under compact support. This is the mixed metric–scalar Noether cancellation (Gate 1), not an independent scalar BV master equation. Separate: (i) invariant baseline $S_{\rm inv}$; (ii) prepared-action defect (S_D, S_D); (iii) insertion-class obstruction (XVI-N thm:massless-obstruction).

30.2 Gate 2: normalized D11 — prefactor Constructed; tensor Constructed

Route A (Wick) independently yields

$$\log Z\big|_{\rm bubble} = -\frac{\kappa^2}{8}\int h\,T\,h, \qquad \Gamma_{1,\rm matter} = -i\log Z, \qquad \Pi_{\rm bubble} = \frac{i\kappa^2}{4}\,VV(CC + CC).$$

Route B expands $\log Z = -\frac{1}{2}\operatorname{Tr}\log H$ to $(-\kappa^2/4)\operatorname{Tr}(D_F H_1 D_F H_1)$. Symmetry of a_1 gives $\mathrm{comb}_B = \frac{1}{2}$ (Constructed — not unresolved) and $\Pi_B = i\kappa^2/4\,T$. Tensor witness $T_{0000} = 24A^2/r^8$ at $z = (0, r, 0, 0)$.

30.3 Gate 3 / Deliverables 1–3

Scalar $u_\mu^R = \mathrm{FP}_{\zeta=0}[\mu^{2\zeta}(Q - i0)^{-2+\zeta}]$ with $r = -i\pi^2$ from analytic-family recurrence + Gate 0 ($\Box(Q - i0)^{-1} = 4\pi^2 i\delta$), not Wick-Jacobian alone. Global $T = \mathcal{K}u$ on U ($\mathcal{K} = \alpha\mathcal{D}^{(2)} + \beta\mathcal{D}^{(0)}$; α, β derived; all 256 components; lemma `lem:BV_rules_on_U`). Active W_w reference only: $\langle\Pi^R_w, f\rangle = \langle\Pi^R_{\rm tr}, W_w f\rangle$ (prescription `W_w_jet_pullback_via_Pi_tr_o68`). Old cutoff ARCHIVED; equivalence NOT claimed. False pointwise $|T| \le C|z|_E^{-8}$ hypothesis removed (annular seminorms). Spanning set still `candidate_list` ⇒ Gate 3 Partial overall.

30.4 Gate 4 / Deliverable 4: contact moments

$$M_\gamma = \langle \Pi_{\rm tr}, w z^\gamma/\gamma! \rangle, \qquad L = \sum_{|\gamma|\le 4} (-1)^{|\gamma|} M_\gamma \, \partial^\gamma \delta.$$

Odd moments vanish by evenness. $\Pi_w + L = \Pi_{\rm tr}$; $\partial(\Pi_w + L) = 0$ both pairs (`kernel_two_point_Ward`; not functional ST4b). L order ≤ 4; kernel-divergence contact bound on ∂L is five. Formal transpose (not L^2 adjoint). L not claimed Lorentz covariant. $C_L = \frac{1}{2}\int h\, L\, h$ (gh= 0, even); $s_0 C_L$ not evaluated. Kernel CT $\neq$ bulk BV restore.

31 Fixed scalar extension u_μ^R (o70-sync)

Prescription. One analytic finite-part scheme:

$$u_\zeta(z) = \mu^{2\zeta}(Q - i0)^{-2+\zeta}, \qquad u_\mu^R = \mathrm{FP}_{\zeta=0}\, u_\zeta.$$

Frozen: $\eta = \mathrm{diag}(+---)$; $Q - i0$; principal logarithm; $\mu > 0$ (inverse length); finite part = Laurent constant term; **no** additional finite $c\delta$. Proof identifier: `lem:analytic_FP_u_mu_R_o70` (equivalent FP prescription to o68/o69; constructive holomorphy proof added).

Lemma (distributional analytic continuation).

1. u_ζ admits a distributional meromorphic continuation near $\zeta = 0$ against arbitrary compact smooth tests, with a simple pole.
2. Holomorphy on tests supported in U: the pairing equals $\langle (Q - i0)^{-2}, f\rangle$ at $\zeta = 0$.
3. The pole part is local: $\dfrac{r\, f(0)}{\zeta}$ with $r = -i\pi^2$, derived below — not by naming analytic regularization.
4. u_μ^R is Lorentz covariant and even under $z \mapsto -z$.
5. $\mathrm{sd}(u_\mu^R) \le 4$; residual $c\delta$ freedom frozen by excluding any added contact (XVI-J `prop:extension` = freedom only).

Ordinary integrability near the cone. Near a nonzero null point with local coordinate $t = Q$, ordinary absolute local integrability of $(Q - i0)^{-2+\zeta}$ requires $\int_{-\varepsilon}^{\varepsilon} |t|^{-2+\Re\zeta}\, dt < \infty$, hence $\Re\zeta > 1$. At $\zeta = 0$ the U-supported pairing is obtained by boundary-value / analytic-family continuation, not by an ordinary integral.

Proof that $\phi_{-1+\zeta}$ is holomorphic near $\zeta = 0$. Do *not* argue from the zero of $4\zeta(\zeta - 1)$ alone.

1. Let $F(\zeta) = (Q - i0)^\zeta$. This is a holomorphic family of distributions near $\zeta = 0$: instantiate Dütsch–Fredenhagen–Keller–Rejzner, arXiv:1311.5424, Definition 4.2 (analytic regularization) and §4.1–4.2 (complex powers / meromorphic two-point families), with hypotheses $Q - i0$, principal logarithm, tests in $\mathcal{D}(\mathbb{R}^4)$, and Gate 0 normalization. (Classical support: Gelfand–Shilov / Hörmander complex powers $(x \pm i0)^\lambda$ are entire in λ.)
2. Const-coeff differentiation under the pairing gives the recurrence $\Box\phi_\lambda = 4\lambda(\lambda + 1)\phi_{\lambda-1}$, hence
$$\Box F(\zeta) = 4\zeta(\zeta + 1)\, \phi_{-1+\zeta}.$$
3. $F(0) = (Q - i0)^0 = 1$, so $\Box F(0) = 0$. Holomorphy of F yields $\Box F(\zeta) = \zeta\, G(\zeta)$ with G holomorphic in $\mathcal{D}'$.
4. Therefore
$$\phi_{-1+\zeta} = \frac{\Box F(\zeta)}{4\zeta(\zeta + 1)} = \frac{G(\zeta)}{4(\zeta + 1)}$$
is holomorphic near $\zeta = 0$.

Residue (only after the holomorphy premise). Gate 0 gives $D_F = -A(Q - i0)^{-1}$ with $A = 1/(4\pi^2)$ and $(-\Box)D_F = i\delta$, hence

$$\Box(Q - i0)^{-1} = 4\pi^2 i\,\delta.$$

The second recurrence

$$\Box\phi_{-1+\zeta} = 4\zeta(-1+\zeta)\,\phi_{-2+\zeta}$$

and the holomorphy of $\phi_{-1+\zeta}$ yield

$$\langle \phi_{-2+\zeta}, f\rangle = \frac{r\,f(0)}{\zeta} + \mathrm{FP} + O(\zeta), \qquad \boxed{r = -i\pi^2}.$$

Scale $\mu^{2\zeta}$ affects only the finite part. Operational Laurent identity:

$$\langle u_\zeta, f\rangle = \frac{r\,f(0)}{\zeta} + \langle u_\mu^R, f\rangle + O(\zeta).$$

Euclidean radial integral ($|S^3| = 2\pi^2$) is an *independent sign check*, not the sole justification. Not momentum-space MS; not XVI-J's value-jet extension of $d(z)^2$.

32 Track II: insertion status freeze

Retained objects. Seed $F_t = \langle t_D, h \otimes h \otimes \phi\rangle$ (XVI-I; no extra factorial) and the regular, seed-preserving, antifield-linear class $I_t = F_t + A_0 + \kappa A_1$ with $A_0 = \int(h^*U + \phi^*V + c^*W)$.

Frozen negative / conditional result. Under XVI-N massless-obstruction hypotheses, on the constructed massless special-family witness: no such I_t satisfies $Q_{\mathrm{inv}} I_t = O(\kappa^2)$. Route A is sufficient and discharged; Route B is not claimed discharged. Certified small-mass interval $0 \le m < 1.3715 \times 10^{-4}$ at frozen geometry; arbitrary prescribed $m > 0$ unresolved.

Hard rule. No quantum completion is claimed whose classical leading term contradicts this obstruction. Extra antifields or nonminimal fields do not automatically remove the order-κ^0 field-only contradiction ($P_0 Q_0 A_0 = 0$ on the free shell).

O.9 separation. Existence of a renormalized product of the bare F_t is not closure of F_t in the BV/BRST sense. N-Ins.5 remains structural only after invariant completion. Success on the bulk quantum sector (verdict 1) does not complete F_t (verdict 3).

Part II
Appendix

33 Appendix overview

All listings below are generated by `tools/build.py` from exactly the source files executed and the certificate files written in this build (ASCII-sanitised copies in `manuscript/listing_src`; the build verifies byte-identity after typesetting and again before zipping). Lists of scalars are printed on one line, so a page break cannot split them; page numbers are printed as "XVI-P p23 | page N" (one version variable, `p22_conventions.VERSION_LABEL`, written to `manuscript/version.tex`). The external manifest (with PDF/zip hashes) is written after the PDFs and is deliberately not reproduced inside any PDF.

34 Release status / acceptance certificate (p23)

`[listing 01/133] certificates/P23_release.json`


```
{
  "landmark": "XVI-P",
  "revision": "p23",
  "contract_revision": "p23 (p14r contract + disclosed p15/p16/p17/p18 revisions + the p22 contract revision +
  +>  the DISCLOSED p23 contract revision: polynomial-domain refusal for d and Dummy-only internal renames, on
  +> e delimited block appended to code/p5_entry/dred.py; p18 core and p22 block byte-identical)",
  "date": "2026-10-10",
  "version_label": "XVI-P p23",
  "deliverable": "p23 (stage release on the O.1 direction): O.1 exact requirement and scope; one-loop power co
  +> unting and enlarge-rule check; P.6 ENTRY inventory (kappa^3 three-point, topology level, NOT evaluated);
  +> polynomial-domain refusal for d (p23 contract revision); EV value check; title qualifier",
  "execution_pass": true,
  "claim_status": "p23. O.1 requires the one-loop kappa^2 bulk two-point sector on S_inv to close under a VERI
  +> FIED retained identity with O.2 (classical kappa^2 master incl. grav ell_4), O.4, O.5, O.6, O.7 all earne
  +> d. Exact power counting: every vertex is kappa^(legs-2), so one-loop graphs are kappa^E; the kappa^2 iden
  +> tity involves only E=2 and E=1, the enumeration reproduces the frozen kappa^2 inventory exactly, and the
  +> enlarge rule is NOT triggered (no one-loop three-point sector at kappa^2). Three-point first appears at k
  +> appa^3: P.6 ENTRY inventory = 15 one-loop graphs, NOT evaluated. O.1 is not closable inside XVI-P (O.2 an
  +> d the bulk ST4b identity are XVI-O/XVI-N items, untouched). P.5 renormalized at one loop kappa^2 for the
  +> two-point functions with QdS external legs. Bulk symmetry NOT restored/claimed; O.1 NOT_CLOSED.",
  "flag_definitions": {
    "projected_defect_computed": "Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from every evaluat
  +> ed input of the three surviving rows (none Missing), exact in d, pole + nonlocal + MS-bar local parts, al
  +> l seven external assignments",
    "projected_identity_verified": "Delta = 0 for the REGULATED one-loop functional Gamma1: pole, nonlocal and
  +>  local parts vanish for all seven assignments, exactly in d, with all controls true and every mutation ch
  +> anging the outcome",
    "projected_restoration_verified": "p20 definition (replaces p19's, which follows from the identity + local
  +> ity): FULL sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm S_ct = -(pole) in the decl
  +> ared domain, h.h + c.K + c.cb (ghost self-energy Sigma held fixed at its P15 value, orientation converted
  +> , derived relative factor +1), in every ghost +1 sector (h.c, c.b, c.cbs), together with projected_identi
  +> ty_verified.  Scope: the one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1",
    "bulk_symmetry_restored": "hard stop: False regardless (not claimed; O.1 NOT_CLOSED)"
  },
  "projected_restoration_scope": "p20 definition: full sigma_0 closure (h.c, c.b, c.cbs) of the complete one-l
  +> oop kappa^2 MS-bar counterterm incl. the c.cb ghost self-energy counterterm (held fixed, derived factor +
  +> 1); NOT bulk symmetry, NOT O.1",
  "route_consistency_control_p19_criterion": {
    "label": "route-consistency control (the p19 restoration criterion, NOT a flag): P_ch sigma_0(S_ct) = 0 fo
  +> r the h.h + c.K MS-bar counterterm and the solve x = 0; implied by the identity + locality",
    "holds": true
  },
  "full_closure_mutation_detector": {
    "test": "full sigma_0 closure (all ghost +1 sectors) of the complete MS-bar counterterm, Sigma held fixed"
  +> ,
    "cases": {
      "drop_graph_D11": {
        "closed": true,
        "entries": 0,
        "sectors": [],
        "detected": false
      },
      "drop_graph_D13 (c.K data = 0, Sigma held fixed)": {
        "closed": false,
        "entries": 88,
```

```
      "sectors": ["c.b"],
      "detected": true
    },
    "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "drop_graph_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "drop_graph_D01": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "flip_phase_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "symmetry_factor_D03_x2": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "Sigma_orientation_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    }
  },
  "reading": "detects drop D13 and flip K.c (the p19 blind spots) through the c.b sector, and the hh-row mut
+> ations through h.c; drop D11 stays undetected (D11 transverse => its pole is sigma_0-closed by itself): c
+> overed by M2 (P18) and the p20 D11 normalisation pin (P20_D11_pin.json)"
},
"p20_controls": {
  "orientation_ratio_uniform_minus_one": true,
  "tree_ghost_equation_V_cbc_eq_2iD_V_Kc": true,
  "tree_sigma0_SOKc_plus_SOcbc_zero": true,
  "derived_ghost_relation_reproduces_P15_sigma": true,
  "full_closure_detects_drop_D13_and_flip_Kc": true,
  "full_closure_detects_p19_orientation_error": true,
  "S_ct_without_ccb_not_closed (c.cb counterterm is needed)": true
},
"why_zero": "each surviving ch row vanishes separately in the projected slot; no counterterm cancellation an
+> d no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupl
+> ing -- D01 and D03 are individually nonzero and cancel against each other in the slot (p17/p18 transverse
+>  sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only throu
+> gh T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-poin
+> t row: the tadpoles D14a-c vanish under the declared prescription.  The defect is therefore zero without
+> any counterterm (solve x = 0).",
"rests_on": ["M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D compon
+> ents'; evanescent components of the D11 matching: not applicable, not verified)", "the evanescent decoupl
+> ing proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row decoupling)", "the frozen P.3 projection
+> and the P13 ch dependency ledger (three surviving rows)", "the declared prescription (DRED R1-R6, MS-bar,
+>  P14_prescription_D11_matching)"],
"findings": {
  "J1_p14r_counterterm_domain_field": "FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the p14r cou
+> nterterm-domain counts (Im sigma_0 3, quotient 8, D cap ker 5, P_ch sigma_0 rank 4) are artefacts of a re
+> /im split over Q (the densities carry explicit i for odd derivative counts, so i-multiples count as indep
+> endent).  The counts over Q(i) -- sigma_0 rank 5, D cap ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch
+>  sigma_0 rank 3 (= Route C) -- equal the real dimensions under a compatible real structure.  The p14r cer
+> tificate stays as released; both are reported",
  "K1_ghost_closure_sign": "p20 CORRECTION (erratum to p19): resolved -- an orientation bookkeeping error. W
+> ith the P15 Sigma coordinates converted to the route-G (c,cb) orientation (ratio -1), the derived factor
+> +1 gives full sigma_0 closure of the complete MS-bar counterterm in h.c, c.b, c.cbs.  The p19 text 'closu
+> re fails / needs -1 / unresolved' is retracted"
},
"K1_ghost_closure_convention": {
  "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
  "M_2iD_cK_to_cbc": [
    ["0", "2"],
```

```
    ["-1", "0"],
    ["0", "-1"]
  ],
  "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
  "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
  "derived_equals_P15": true,
  "relative_factor_derived": "1",
  "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
  "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
  "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
  "tree_controls": {
    "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
    "S0_Kc_terms": 16,
    "S0_cbc_terms": 40,
    "sigma0_S0Kc_plus_S0cbc_entries": 0,
    "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
    "ok": true
  },
  "full_closure_complete_S_ct": {
    "closed": true,
    "entries": 0,
    "sectors": []
  },
  "full_closure_S_ct_without_ccb (p19 package S_ct)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"]
  },
  "full_closure_unconverted (p19 error)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"],
    "detected": true
  },
  "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolved
+> ' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots).
+>  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
},
"scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the hh an
+> d K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement abou
+> t higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests ful
+> l sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c.b,
+> c.cbs) -- a statement about that counterterm only, not bulk symmetry, not 0.1",
"release_claim_accepted": true,
"release_claim_accepted_definition": "contract = registered p22 revision only (pinned) AND lower layers re-c
+> hecked live AND p20/p21 layers AND absolute pin AND EV-M2 + EV-CT certificate AND live runtime guard prob
+> e AND every added/modified/removed file disclosed (no prefix exemption) AND the pinned p21 independent re
+> cord AND the p22 hard stops",
"publication": "accepted",
"publication_gated_on": ["execution_pass", "quantum_inputs_evaluated (p18 layer + recheck)", "projected_defe
+> ct_computed", "flag implications restoration => identity => computed"],
"flags": {
  "library_certified": true,
  "batchA_evaluated": {
    "execution": true,
    "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
+> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
+> V pole, finite part 0, no p14r counterterm required",
    "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
+>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
+> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
+> e; NOT quantum_inputs_evaluated"
  },
  "D13_evaluated": {
    "execution": true,
    "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
+> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
+> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
+> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
    "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
+> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
+> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
+> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
  },
  "batchC_evaluated": {
    "execution": true,
    "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
+> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
+> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
+> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
    "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
```

```
+>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
+> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
  },
  "E2_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof)",
    "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
+>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
  },
  "hh_evanescent_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
    "scope": "the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of D01
+>  + D03 (D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green function
+>  outside this projection; D11 (same row) NOT evaluated"
  },
  "D11_matching_M2": {
    "execution": true,
    "claim": "DISCHARGED (4D components)",
    "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the
+>  4D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0
+> is IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns
+>  the classical insertion and is not touched"
  },
  "hh_slot_full_row_decoupled": {
    "execution": true,
    "claim": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
    "scope": "row (S0,Gamma1^[2]) s2 K:K.c; D01 + D02 + D03 + D11 + D12; the summed evanescent poles remain
+> (transverse) UV poles of the hh Green function outside this projection"
  },
  "projected_ch_identity": {
    "execution": true,
    "claim": "ZERO",
    "scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the h
+> h and K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement
+> about higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests
+>  full sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c
+> .b, c.cbs) -- a statement about that counterterm only, not bulk symmetry, not O.1"
  }
},
"projected_defect_computed": "earned",
"projected_identity_verified": "earned",
"projected_restoration_verified": "earned",
"projected_defect_summary": {
  "defect_zero_exact_in_d": true,
  "evanescent_in_assembly": "verified explicitly: the evanescent (gt) parts of the hh row (D01+D03+D11) and
+> of the K.c row (D13 E1, E2) give ZERO for all seven external assignments, exactly in d; the evanescent po
+> les themselves are nonzero (P18_hh_slot, P15_D13) -- they drop from the ch slots by the decoupling proofs
+> , not by convention",
  "quotient_coordinates": {
    "complement_directions": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"],
    "pole": ["0", "0", "0", "-3/8", "1/4"],
    "nonlocal_L": ["0", "0", "0", "3/8", "-1/4"],
    "local": ["0", "0", "0", "-2/3", "5/12"],
    "reading": "coordinates modulo D cap Im sigma_0 (field redefinitions / s-exact); the h.h pole is transve
+> rse, i.e. in D cap Im sigma_0, so its quotient coordinates vanish"
  },
  "D_cap_ker_sigma0": {
    "computed": true,
    "dim": 6,
    "dim_D_cap_Im_sigma0": 6,
    "cohomology_dim": 0,
    "sector_ranks": {
      "h.h": {
        "dim": 5,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 3
      },
      "c.cb": {
        "dim": 2,
        "sigma0_rank": 2,
        "P_ch_sigma0_rank": 0
      },
      "c.K": {
        "dim": 3,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 1
      },
      "phi.phi": {
        "dim": 1,
```

```
        "sigma0_rank": 0,
        "P_ch_sigma0_rank": 0
      }
    }
  },
  "field_note": "Route G is computed over Q(i) (primary) and, for comparison, over Q with the re/im split us
+> ed by the p14r domain certificate.  The p14r densities carry explicit factors i for odd derivative counts
+>  (c.K: 3 derivatives), so over Q an i-multiple counts as independent: e.g. the ch image of c.K#0 is i tim
+> es an h.h image.  The counts over Q(i) equal the real dimensions under a compatible real structure (one r
+> eal coefficient per density, the i fixed by reality of the action); ranks and solvability are quoted over
+>  Q(i)."
},
"counterterm_solve": {
  "system": "Delta + P_ch sigma_0(x) = 0, x in the declared domain D (11 directions), part by part",
  "rank_P_ch_sigma0_routeG": 3,
  "rank_routeC": 3,
  "routeC_image_relations": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "outcome": "ZERO",
  "residual": null,
  "solution": "x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite renorma
+> lisations are ker(P_ch sigma_0) cap D (dim 8), of which 2 modulo D cap Im sigma_0",
  "MSbar_counterterm_ch_image_zero": true,
  "MSbar_counterterm_full_sigma0_image_sectors": ["c.b"],
  "full_closure_relative_factor_table_UNCONVERTED_p19_error": {
    "1": false,
    "-1": true,
    "I": false,
    "-I": false
  },
  "full_closure_reading": "p20 (corrects p19): with the P15 Sigma coordinates converted from the (cb,c) to t
+> he route-G (c,cb) orientation (measured ratio -1), the DERIVED factor +1 gives full sigma_0 closure of th
+> e complete MS-bar counterterm (h.h + c.K + c.cb) in every ghost +1 sector; the p19 table above (factors 1
+> , -1, i, -i on UNCONVERTED coordinates) is the bookkeeping error, kept for the erratum",
  "K1_ghost_closure_convention": {
    "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
    "M_2iD_cK_to_cbc": [
      ["0", "2"],
      ["-1", "0"],
      ["0", "-1"]
    ],
    "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
    "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
    "derived_equals_P15": true,
    "relative_factor_derived": "1",
    "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
    "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
    "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
    "tree_controls": {
      "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
      "S0_Kc_terms": 16,
      "S0_cbc_terms": 40,
      "sigma0_S0Kc_plus_S0cbc_entries": 0,
      "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
      "ok": true
    },
    "full_closure_complete_S_ct": {
      "closed": true,
      "entries": 0,
      "sectors": []
    },
    "full_closure_S_ct_without_ccb (p19 package S_ct)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"]
    },
    "full_closure_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolv
+> ed' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots)
+> .  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
  },
```

```
  "note_full_sigma0": "the h.h + c.K MS-bar counterterm ALONE has a nonzero (c, b) image (delta S0/delta h c
+> arries b); adding the c.cb ghost self-energy counterterm (correct orientation, derived factor +1) closes
+> it: see K1_ghost_closure_convention.full_closure_complete_S_ct"
},
"defect_controls": {
  "inventory_complete_no_missing": true,
  "missing_input_refused_not_zero": true,
  "s0h_longitudinal": true,
  "E_transverse": true,
  "hh_row_zero_exact_in_d_all_7": true,
  "hh_row_evanescent_part_zero_all_7": true,
  "hh_row_D01_D03_individually_nonzero": true,
  "Kc_row_zero_exact_in_d_all_7": true,
  "Kc_row_evanescent_part_zero_all_7": true,
  "Kc_slot_map_nontrivial_T1_E1": true,
  "E2_alone_zero_all_7": true,
  "D13_A_T1_A_E1_zero_exact": true,
  "onepoint_row_D14abc_vanishing": true,
  "defect_zero_pole_nonlocal_local_all_7": true,
  "defect_independent_of_row_signs": true,
  "routeG_S0_check": true,
  "routeG_defect_zero_by_sector": true,
  "routeG_hh_local_data_in_span": true,
  "routes_agree_on_ch_image_rank": true,
  "routeG_hh_kernel_equals_p18_transverse_subspace": true,
  "D_cap_Im_inside_D_cap_ker_sigma0": true,
  "D_cap_Im_inside_ker_P_ch_sigma0": true,
  "ghost_equation_relation_reproduces_P15_sigma_pole": true,
  "MSbar_counterterm_in_ker_P_ch_sigma0": true,
  "mutations_all_change_outcome": true,
  "routeG_Q_split_reproduces_p14r_counts": true
},
"defect_mutations": {
  "drop_graph_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/48",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the ghost loop D03"
  },
  "drop_graph_D01": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the graviton bubble D01"
  },
  "flip_phase_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/24",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/24",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
```

```
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/8"
      }
    },
    "note": "relative sign of D03 flipped"
  },
  "symmetry_factor_D03_x2": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "D03 doubled"
  },
  "wrong_projection_trace_operator_hh": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-3*c^.p*p.p*(6*a.b*p.p - 6*a^.p*b^.p - a~.b~*p.p)/16",
        "QdS (hat)": "-9*c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/8",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-15*a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "eta_mn p_v in place of s0 h (not the P.3 projection)"
  },
  "wrong_projection_gauge_fixed_operator_Kc": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p*(a.b*c^.p*p.p - 3*a.c*b^.p*p.p - 3*a^.p*b.c*p.p + 4*a^.p*b^.p*c^.p - a^.p
+> *b~.c~*p.p - a~.c~*b^.p*p.p)/16",
        "QdS (hat)": "p.p*(a^.b^*c^.p*p.p - 3*a^.c^*b^.p*p.p - 3*a^.p*b^.c^*p.p + 4*a^.p*b^.p*c^.p)/16",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/4",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "de Donder q^2 P in place of the h^2-monomial operator E (not the P.3 projection)"
  },
  "naive_4D_D03_evanescent_dropped": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/24"
      }
    },
    "note": "D03 Gt parts dropped"
  },
  "naive_4D_D13_Gt_dropped": {
    "outcome": "NONZERO, CANCELLABLE by a counterterm in the declared domain",
    "changes_outcome": true,
    "residual": {
      "local": {
```

```
        "Q4S (declared, R2)": "-c^.p*p.p*(a.b*p.p - a^.p*b^.p)/12",
        "QdS (hat)": "-c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/12",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/12"
      }
    },
    "note": "D13 evaluated with Gt dropped (P15_D13 mutation fit: T1 = -(d-4)/(16(d-1)))",
    "finite_counterterm": {
      "rank": 3,
      "solvable": true,
      "solution": ["(6*x3 + 12*x4 - 24*I*x5 - I)/12", "-x3/2", "-(12*x3 + 24*x4 - 24*I*x5 - I)/12", "x3", "x
+> 4", "x5", "x6", "x7"]
    }
  }
},
"defect_insensitive_variations": {
  "drop_graph_D11": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "D11 is transverse by itself (p_m G_D11 = 0 exactly in d): the slot cannot see it; covered
+> by M2 (P18) and by the inventory refusal below"
  },
  "drop_graph_D13": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "A_T1 = A_E1 = 0 and T2, T3, E2 are longitudinal: D13 reaches the slot with zero; covered b
+> y the D13 certificate (two routes, mutations)"
  },
  "flip_row_sign_Kc": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "each row vanishes separately"
  }
},
"quantum_inputs_evaluated": "earned",
"quantum_inputs_inventory": [
  {
    "graph": "D01",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D02",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D03",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D11",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"
  },
  {
    "graph": "D12",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D13",
    "slot_row": "(S0,Gamma1^[2]) s1 h:h^2",
    "evaluated": true,
    "status": "evaluated"
  },
  {
    "graph": "D14a",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
```

```
  {
    "graph": "D14b",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14c",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  }
],
"D11_matching_verified": true,
"D11_matching_verified_scope": "4D components: M2 discharged (P18_D11_M2, unchanged); evanescent components:
+>  EV-M2 proven (local poles + exact 4D decoupling of the renormalised remainder); NOT matched to XVI-O (no
+>  counterpart; both natural d-dim continuations refuted)",
"D11_evanescent_components_matching": "not_applicable_not_verified",
"M3_status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
"D11_evaluated": true,
"hh_full_row_decoupling_proven": true,
"evanescent_hh_counterterm_added": true,
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"hh_evanescent_status": "decoupled_from_ch_slot_proven_as_sum",
"hh_evanescent_decoupling_proven": true,
"D01_evaluated": true,
"D03_evaluated": true,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
```

```
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
```

```
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D11": {
    "status": "evaluated",
    "certificate": "certificates/P18_D11.json",
    "sector_status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
    "M2": "DISCHARGED (4D components)",
    "ch_slot": "decoupled by itself (p_m G_D11 = 0 exactly in d)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_byte_identical_to_p18": true,
  "dred_registry_state_p22": "p18 + p22 guard block",
  "contract_revision_in_p19": false,
  "disclosed_revisions_p19": {}
},
"p18_layer": {
  "p18_layer_files_byte_identical": true,
  "P18_release_accepted": true,
  "P18_quantum_inputs_evaluated": "earned",
  "I1_gate": "the p19 gate requires the p18 layer record's quantum_inputs_evaluated == 'earned'"
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
    "C7_grav_noether_k0_k1_k2_on_exports": true,
    "quartic_mutations_all_rejected": true,
    "master_grav_noether_deterministic_k0_k1_k2": true,
    "master_engine_parts_exact": true,
    "graviton_hessian_generic_symbolic_k": true,
```

```
  "canonicity_all_23_antifield_components": true,
  "ch_enumeration_lemma_mechanical": true,
  "residual_record_invariants_all_certificates": true,
  "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
  "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
  "cross_certificate_quartic_vs_master_agree": true,
  "stored_C7_flag_equals_recomputation": true
},
"failed_prerequisites": [],
"hash_consistency": {
  "consistent": true,
  "ell": {
    "ell2": {
      "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
    },
    "ell3": {
      "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
    },
    "ell4": {
      "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
    "quartic_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    },
```

```
      "master_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    }
  },
  "hard_stops_p13_hold": true,
  "hard_stops_p13": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"hard_stops": {
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-bar in DRE
+> D with the p22 evanescent counterterms; external legs in QdS)",
    "P.5_identity": "projected defect computed (zero exactly in d, all parts, all seven assignments; each ro
+> w vanishes separately); projected identity verified; projected restoration verified (p20 definition: full
+>  sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm incl. c.cb, h.c/c.b/c.cbs); K1 corre
+> cted in p20 (orientation erratum); D11 ratio pin verified; D11 absolute pin verified (p21); bulk symmetry
+>  NOT restored/claimed; O.1 NOT_CLOSED [p21: D11 absolute pin derived and verified] [p22: evanescent count
+> erterms closed; identity unchanged]",
    "P.6_entry": "inventory (one loop kappa^3 three-point, 15 graphs, topology level); not evaluated",
    "O.1": "NOT_CLOSED (requirement recorded; not closable inside XVI-P)"
  },
  "definition_vs_status_note": "I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficients evalua
+> ted and renormalized'; the STATUS states exactly what holds: 4D sector renormalized, evanescent poles reg
+> ulated and decoupled from the ch slots (not subtracted)",
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)"
},
"author_rebuild_passed": true,
"independent_rebuild_passed": "pending",
"independent_rebuild_passed_p18": true,
"independent_rebuild_passed_p17": true,
"independent_rebuild_passed_p16": true,
```

```
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author/independent flags refer to the CURRENT (p23) tree only (independent: pending u
+> ntil a reviewer report exists); independent_rebuild_passed_p22 refers to the p22 tree 1bc6edff6ca0788bb0f
+> ee4e8bf259bf8a0c80791f6dd08be77f79e56b492a50c / zip ade2fa5c2dac6614c41ae38dc97d7c485975015a134650734d466
+> ea6ffdef76f (same-box caveat), not inherited",
"p18_independent_rebuild": {
  "report": "archival/p18/reproducibility/independent_rebuild_report_p18.json",
  "present": true,
  "report_sha256": "893f167ef7124d981c0569ca386035aed06e0dbb0c90a2362a6ceeafd1930cb2",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094",
  "bound_to_source_zip_sha256": "7c7d5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f",
  "tree_matches_p18": true,
  "zip_matches_p18": true,
  "mismatches": 0,
  "findings": ["I1_release_p18_publication_not_gated_on_quantum_inputs_evaluated", "I2_scan_pattern_gaps", "
+> I3_scope_of_M2_and_evaluated", "I4_table_row_count", "I5_slot_blind_to_D11_normalisation", "I6_test_p16_v
+> ictim_restriction", "I7_pdftotext_dehyphenation", "H1_H5_resolution", "bug_fixes_disclosed"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p19": "bound to the p18 tree/zip; the p19 tree has its own (pending) independent flag",
  "archived_p18_records_pinned": {
    "archival/p18/reference_values_p18.json": true,
    "archival/p18/reproducibility/author_rebuild_report_p18.json": true,
    "archival/p18/reproducibility/independent_rebuild_report_p18.json": true,
    "archival/p18/p18_source_files.json": true
  },
  "pins_ok": true
},
"p17_independent_rebuild": {
  "report": "archival/p17/reproducibility/independent_rebuild_report_p17.json",
  "present": true,
  "report_sha256": "92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
  "bound_to_source_zip_sha256": "7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416",
  "tree_matches_p17": true,
  "zip_matches_p17": true,
  "mismatches": 0,
  "findings": ["H1_static_scan_pattern_gaps", "H2_assignment_table_omits_c_hat_h_tilde", "H3_stale_comments"
+> , "H4_curly_apostrophes_in_typeset_listings", "H5_parse_d_srepr", "H6_scope", "G1_G2_G3_G4_resolution"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p18": "bound to the p17 tree/zip; the p18 tree has its own (pending) independent flag",
  "archived_p17_records_pinned": {
    "archival/p17/reference_values_p17.json": true,
    "archival/p17/reproducibility/author_rebuild_report_p17.json": true,
    "archival/p17/reproducibility/independent_rebuild_report_p17.json": true,
    "archival/p17/p17_source_files.json": true
  },
  "pins_ok": true
},
"p16_independent_rebuild": {
  "report": "archival/p16/reproducibility/independent_rebuild_report_p16.json",
  "present": true,
  "report_sha256": "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
  "bound_to_source_zip_sha256": "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57",
  "tree_matches_p16": true,
  "zip_matches_p16": true,
  "mismatches": 0,
  "findings": ["G1_static_scan_scope_narrower_than_disclosure", "G2_plain_symbol_d_from_stored_strings", "G3
+> _hook_coverage_detail", "G4_change_log_omits_new_reference_file", "G5_test_p15_assertion_relaxed", "G6_im
+> aginary_normalisation", "G7_scope", "F3_F4_F5_from_p15"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
```

```
  "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent flag",
  "archived_p16_records_pinned": {
    "archival/p16/reference_values_p16.json": true,
    "archival/p16/reproducibility/author_rebuild_report_p16.json": true,
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": true,
    "archival/p16/p16_source_files.json": true
  },
  "pins_ok": true
},
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"M2_summary": {
  "verdict": "DISCHARGED (4D components)",
  "D11_matching_verified_scope": "4D components",
  "evanescent_components_matching": {
    "status": "not_applicable_not_verified",
```

```
    "what_they_are": "the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt): compo
+> nents of the hh two-point function with at least one index pair in the epsilon-scalar (4-d) directions of
+>  DRED; they vanish on hatted 4D components",
    "why_not_matched": "XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent counte
+> rpart; M2 is a 4D statement, so there is nothing to match them to. Their L-coefficients are nonzero (reco
+> rded below); they are retained (R6) and drop out of the ch-projected slot (p_m G_D11 = 0 for the evanesce
+> nt part separately)"
  },
  "M3_status": {
    "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
    "statement": "L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree <= 8 i
+> t is a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)",
    "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching)"
  },
  "statement": "M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR_AB],
+>  f> = <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18",
  "route_B": {
    "equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair",
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 0,
    "first": [],
    "agree": true,
    "constant": "-I/4"
  },
  "FT_constant": {
    "master_pole": "1",
    "master_finite": "2 - L",
    "pole_is_1": true,
    "L_coefficient_of_MB": "-1",
    "c_FT_from_DR_bubble": "I*pi**2",
    "c_FT_from_minus_r": "I*pi**2",
    "r_certificate": "-i * pi**2",
    "r_certificate_as_expected": true,
    "agree": true
  },
  "controls": {
    "route_B_all_100_pairs_agree": true,
    "FT_constant_two_calibrations_agree": true,
    "route_A_M1_integrand_identity": true,
    "route_A_XVI_O_K_u_equals_T": true,
    "D11_transverse_exact_in_d": true,
    "routes_FP_PV_agree": true,
    "covariant_fit_exact_unique": true,
    "mutations_all_rejected": true
  },
  "analytic_inputs_cited": ["Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, resi
+> due at zeta = -2 is r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bub
+> ble)", "a polynomial in p is the Fourier transform of a distribution supported at {0}", "Route A only: tw
+> o extensions of the product V V (C C + C C) from U differ by a distribution supported at 0 (XVI-O lem:BV_
+> rules_on_U, Constructed)"],
  "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the 4
+> D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0 is
+>  IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns t
+> he classical insertion and is not touched",
  "M3_consequence": "with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both sides a
+> re transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contacts, consistent with
+> N4), so L_match lies in the 2-dim transverse subspace of the p14r h.h span; its coordinates need the XVI-
+> O finite constant of u^R and are NOT computed (M4: booked as P-side hh counterterm; u^R_mu unmodified, no
+>  c delta)"
},
"D11_summary": {
  "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
  "prescription": "operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions, M3 bounds
+>  4/8/4, exactly-degree-four normalization (massless: every local term is a homogeneous quartic in p)",
  "prescription_usage": {
    "exactly_degree_four_normalization": "every structure S_s (p.p)^e of Ghat has total momentum degree 4 (c
+> omputed per structure: g4_mn g4_rs (p.p)^2: 4, g4_mn gt_rs (p.p)^2: 4, g4_mn p_r p_s (p.p)^1: 4, g4_mr g4
+> _ns (p.p)^2: 4, g4_mr gt_ns (p.p)^2: 4, g4_mr p_n p_s (p.p)^1: 4, g4_rs gt_mn (p.p)^2: 4, g4_rs p_m p_n (
+> p.p)^1: 4, gt_mn gt_rs (p.p)^2: 4, gt_mn p_r p_s (p.p)^1: 4, gt_mr gt_ns (p.p)^2: 4, gt_mr p_n p_s (p.p)^
+> 1: 4, gt_rs p_m p_n (p.p)^1: 4, p_m p_n p_r p_s: 4)",
    "N4_contact_exclusions": "used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by N4 do not
+> arise (massless; every pole and local MS-bar term is a homogeneous quartic in p), so N4 removes nothing h
+> ere and no order-0/2 counterterm is needed",
    "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"
  },
  "UV_pole_in_p14r_hh_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
  "evanescent_pole_nonzero": true,
  "controls": {
    "exactly_degree_four_every_structure": true,
    "N4_no_order_0_or_2_contact_arises": true,
```

```
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "integrand_equals_XVI_O_convolution_M1": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true,
    "M2_discharged": true,
    "mutations_all_rejected": true
  }
},
"hh_slot_summary": {
  "verdict": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
  "row": "(S0,Gamma1^[2]) s2 K:K.c",
  "graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
  "how_D11_decouples": "graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse by i
+> tself, 4D and evanescent parts separately; its evanescent poles are nonzero but transverse. Hence D11 mut
+> ations of phase or symmetry factor are NOT detectable by the slot (they are tested by M2 instead); the sl
+> ot sees D11 only through a non-longitudinal operator (trace operator control: nonzero)",
  "surviving_evanescent_structures_in_slot": [],
  "consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop (D11 in
+> cluded), so NO contract revision is made for it.  The summed evanescent poles of the hh Green function ar
+> e nonzero but transverse; renormalising the hh Green function itself would need evanescent h.h counterter
+> ms -- outside this projection and NOT claimed.",
  "controls": {
    "operator_longitudinal": true,
    "p_dot_G_D11_zero_exact_in_d": true,
    "p_dot_G_sum_zero_exact_in_d": true,
    "p_dot_G_evanescent_sum_zero": true,
    "slot_sum_zero_all_7_assignments": true,
    "evanescent_slot_sum_zero_all_7_assignments": true,
    "D01_D03_individually_nonzero_incl_c_hat_h_tilde": true,
    "D11_reaches_slot_under_trace_operator": true,
    "4D_pole_sum_D01_D03_D11_transverse": true,
    "4D_pole_D11_transverse": true,
    "mutations_all_rejected": true
  }
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
```

```
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
```

```
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p23.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": true,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "auto",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p23.json",
    "present": false,
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4"
  },
  "p22_archive": "archival/p22",
  "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
},
"XVI_P_registered": false,
"XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
"XVI_M_pin_ok": true,
"symmetry_restored": false,
"bulk_symmetry_restored": false,
"O1_closure": "NOT_CLOSED",
"insertion_status": "Frozen_negative_obstruction_preserved",
"XV_B_opened": false,
"XIV_J_opened": false,
"Route_B_discharged": false,
"u_R_c_delta_added": false,
"inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
  "P22_release.json": "58877fceb7702beb30140f59d1e9573cc17ac2514ae16d1602470ac73b7bd96d",
  "P23_O1_scope.json": "50c9cff03bfddc0280bff9bb6a5643296b8b8eca7b38c71dd6be3f504fc6e2ca"
},
"p19_layer_disclosed_test_revisions_p19": {
  "tests/test_p14.py": "names p18 -> p19 (reference file, source zip, author report, status revision); gener
+> ated prefixes certificates/ledgers P19_, manuscript/body/P19-",
  "tests/test_p18.py": "rs() reads the p18 LAYER record certificates/P18_release.json; the live p18 independ
+> ent report is bound to the p18 tree (pending or False here); test-file change log accepts p18 or disclose
+> d p19 revisions; build label is the current revision"
},
"D11_ratio_pin_verified": "earned",
"D11_absolute_pin_verified": "earned",
"p18_layer_byte_identical": true,
"p20_layer_change_log_vs_p19": {
  "base": "archival/p19/p19_source_files.json (pinned; files of the p19 zip sha256 da9242752d2e3753d8b2ce7fb
+> b21e7cdb28487cc7966c2ca52d1ef1376658875)",
  "base_pinned_ok": true,
```

```
  "added": ["archival/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p
+> 19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/in
+> dependent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "arch
+> ival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "arch
+> ival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p2
+> 1_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebu
+> ild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/R
+> EADME.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archiv
+> al/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archiv
+> al/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p
+> 5_calc/d11_pin.py", "code/p5_calc/evanescent.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_co
+> nventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_
+> counting.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py"
+> , "code/p5_calc/release_p23.py", "manuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pi
+> n.tex", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_q
+> uantum_target.json", "reproducibility/reference_values_p23.json", "tests/test_p20.py", "tests/test_p21.py
+> ", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p19.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/defect.py", "code/p5_calc/release_p18.p
+> y", "code/p5_calc/release_p19.py", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.tex", "manuscript
+> /notes/P19_defect.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "tests/test_p18.py
+> ", "tests/test_p19.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": [],
  "counts": {
    "added": 43,
    "removed": 1,
    "modified": 16
  }
},
"p20_layer_disclosed_test_revisions_p20": {
  "tests/test_p14.py": "names p19 -> p20; generated prefixes P20_",
  "tests/test_p19.py": "K1 tests corrected (orientation, full closure with derived +1); restoration on the p
+> 20 definition; J1 wording; p19 independent report bound to the p19 tree; scan sites = p19 + p20 reviewed
+> lists"
},
"p20_layer_disclosed_p19_layer_revisions_p20": {
  "code/p5_calc/defect.py": "K1 orientation fix (measured ratio, Sigma converted to route-G (c,cb)); tree co
+> ntrols; full sigma_0 closure of the complete MS-bar S_ct; restoration REDEFINED; full-closure mutation de
+> tector; p19 criterion relabelled route-consistency control; J1 wording; K1 erratum texts",
  "code/p5_calc/release_p19.py": "claim texts: K1 erratum (p19 'closure fails / needs -1' retracted); restor
+> ation content checks on the p20 definition; reject removes stale RELEASE_STATUS.json"
},
"p20_layer_p19_layer": {
  "p19_conventions_byte_identical": true,
  "disclosed_p20_revisions": {
    "code/p5_calc/defect.py": {
      "sha256": "675c1fa812c51b9955dd48a4149d309ff2e99ec7797fad753eb57b37e908ce6f",
      "differs_from_p19": true,
      "disclosed": true
    },
    "code/p5_calc/release_p19.py": {
      "sha256": "4d048345270a39a77cc30f71d3bdcbef17fffec6c44c1ac89487135ea3555970",
      "differs_from_p19": true,
      "disclosed": true
    }
  },
  "P19_release_accepted": true,
  "lower_layers_rechecked_p21": {
    "p18_layer_ok": true,
    "P18_quantum_inputs_evaluated": "earned",
    "inventory_recheck_ok": true
  }
},
"p20_layer_erratum_p19": {
  "K1": "p19 stated that full sigma_0 closure of the MS-bar counterterm fails with the derived ghost-equatio
+> n factor +1 and needs -1 (unresolved). RETRACTED: Grassmann-orientation bookkeeping error -- the P15 Sigm
+> a coordinates are (cb,c)-oriented, the route-G c.cb slots (c,cb), V[c.cb#k] = -V[(cb,c)#k]. Corrected: Si
+> gma = (+3/8, -1/4) in (c,cb) coordinates, derived +1, full closure in h.c, c.b, c.cbs.",
  "restoration": "p19's projected_restoration_verified (P_ch sigma_0(S_ct) = 0, x = 0) followed from the ide
+> ntity + locality; REDEFINED in p20 as full sigma_0 closure of the complete MS-bar S_ct (incl. c.cb), kept
+>  as a labelled route-consistency control",
  "J1": "'over Q(i), the physical field' replaced by: the Q(i) counts are the real dimensions under a compat
+> ible real structure",
  "found_by": "p19 independent rebuild (report sha256 1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf00
+> 8dd2811f45)"
},
"p20_layer_K1_correction": {
  "content_revision_p20": true,
  "orientation_ratio_minus_one": true,
  "Sigma_pole_routeG_c_cb": true,
  "derived_factor_plus_one": true,
```

```
  "derived_relation_reproduces_P15": true,
  "full_closure_complete_S_ct": true,
  "without_ccb_not_closed": true,
  "tree_controls_ok": true,
  "p20_controls_all_true": true,
  "detector_drop_D13": true,
  "detector_flip_Kc": true,
  "detector_drop_D11_recorded_blind": true,
  "route_consistency_control_holds": true,
  "quotient_incl_Sigma_zero": true,
  "quotient_Sigma_alone_c_cb_plus_3_8_minus_1_4": true,
  "J1_wording": true,
  "restoration_definition_p20": true,
  "bulk_false": true
},
"p20_layer_retraction_check": {
  "phrases": ["full c.b / ghost-sector closure FAILS", "full c.b / ghost-sector closure of the MS-bar counte
+> rterm FAILS", "for which full closure fails", "with +1 full closure fails", "full closure \\emph{fails}",
+>  "This is unresolved", "This is not resolved in p19", "needs relative factor -1; the derived ghost-equati
+> on relation gives +1; not resolved", "factor +1 (K1) full c.b closure FAILS", "full c.b closure NOT estab
+> lished (K1)", "the physical field"],
  "scope": ["README.md", "manuscript/notes/P19_defect.tex", "certificates/P19_defect.json", "certificates/P1
+> 9_release.json", "manuscript/body/P19-results-table.tex"],
  "hits": {}
},
"p20_layer_D11_pin": {
  "check": {
    "ratio_pin_PASS": true,
    "ratio_10": true,
    "transverse": true,
    "C2_quadratic_form_half_P2": true,
    "GaussBonnet_zero": true,
    "absolute_pin_PASS": true,
    "absolute_pin_flag": true,
    "top_level_absolute_status_consistent": true,
    "top_level_ratio_status_consistent": true,
    "absolute_ratios_one": true,
    "no_external_convention_choice": true,
    "drop_D11_detected": true,
    "x2_detected_by_absolute": true,
    "sign_flip_detected_by_absolute": true,
    "corrected_in_p21": true,
    "literature_source_stated": true,
    "revision": true
  },
  "ratio_pin": {
    "status": "PASS",
    "statement": "spin-0 : spin-2 of the D11 4D UV pole = 10 : 1, derived from the certificate; equals the m
+> inimal-scalar a_2 prediction (1/72)(3) : (1/120)(1/2)"
  },
  "absolute_pin": {
    "status": "PASS",
    "observed_common_factor_c2_vs_bare_a2": "-I",
    "observed_common_factor_c0_vs_bare_a2": "-I",
    "derivation_p21": {
      "package_declarations_checked": {
        "N_eps = i S_eps/(16 pi^2)": true,
        "Gamma_1 = -i * graph sum": true,
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
        "g = eta + kappa h": true,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
        "eta = (+,-,-,-)": true,
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
      },
      "steps": {
        "1_G_equals_Gamma2": "1",
        "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
        "3_i_S_M_over_S_E (must be -1)": "-1",
        "4_L_M_eff_over_L_E_eff": "-1",
        "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
        "6_minkowski_quadratic_forms": {
          "Riem2": {
            "P2": "1",
            "P0s": "1"
          },
          "Ric2": {
            "P2": "1/4",
            "P0s": "1"
          },
          "R2": {
```

```
            "P2": "0",
            "P0s": "3"
          },
          "C2": {
            "P2": "1/2",
            "P0s": "0"
          },
          "E_GaussBonnet": {
            "P2": "0",
            "P0s": "0"
          }
        }
      },
      "package_Gamma2_pole_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "heat_kernel_prediction_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "ratio_package_over_prediction": {
        "spin2": "1",
        "spin0s": "1"
      },
      "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2
+>  (Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps
+> )) int a_2 (Euclidean, d = 4-2eps)"],
      "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, s
+> ign of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
      "status": "PASS"
    },
    "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under th
+> e package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the W
+> ick direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(
+> 16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1
+> , both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no ext
+> ernal convention choice.  p20 had left this OPEN."
  },
  "spin_decomposition": {
    "c2_spin2": "-I/240",
    "c0_spin0s": "-I/24",
    "projector_norms": ["5", "1"],
    "transverse": true,
    "only_spin2_and_spin0s": true,
    "ratio_c0_over_c2": "10"
  },
  "mutations": {
    "drop_graph_D11": {
      "c2": "0",
      "c0": "0",
      "ratio": "None",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_normalisation_x2": {
      "c2": "-I/120",
      "c0": "-I/12",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_sign_flip": {
      "c2": "I/240",
      "c0": "I/24",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "swap_g4g4_structures": {
      "c2": "-I/40",
      "c0": "-I/32",
      "ratio": "5/4",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
```

```
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    }
  }
},
"p20_layer_open_pins": [],
"p20_layer_open_pins_closed_in_p21": ["D11_absolute_normalisation (Ghat -> a_2 map not derived)"],
"p20_layer_p19_independent_rebuild": {
  "report": "archival/p19/reproducibility/independent_rebuild_report_p19.json",
  "present": true,
  "report_sha256": "1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf008dd2811f45",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36",
  "bound_to_source_zip_sha256": "da9242752d2e3753d8b2ce7fbb21e7cdb28487cc7966c2ca52d1ef1376658875",
  "tree_matches_p19": true,
  "zip_matches_p19": true,
  "mismatches": 0,
  "findings": ["K1_ghost_closure_sign_is_orientation_bookkeeping", "quotient_coordinates_c.cb_orientation",
+> "restoration_flag_not_independent", "mutation_criterion_relabelled", "J1_wording", "I2_scan_misses", "gat
+> e_stale_RELEASE_STATUS"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p20": "bound to the p19 tree/zip; the p20 tree has its own (pending) independent flag",
  "archived_p19_records_pinned": {
    "archival/p19/reference_values_p19.json": true,
    "archival/p19/reproducibility/author_rebuild_report_p19.json": true,
    "archival/p19/reproducibility/independent_rebuild_report_p19.json": true,
    "archival/p19/p19_source_files.json": true
  },
  "pins_ok": true
},
"p20_layer_independent_rebuild_passed_p19": true,
"p19_gate_change_log_vs_p18_recomputed_on_current_tree": {
  "base": "archival/p18/p18_source_files.json (pinned; files of the p18 zip sha256 7c7d5451adb459092edc2606e
+> 1b3322909325000d47bab89c14b7f7a6d25c92f)",
  "base_pinned_ok": true,
  "reference_file": "reproducibility/reference_values_p23.json listed as added by declaration (written after
+>  the author build)",
  "added": ["archival/p18/README.md", "archival/p18/p18_source_files.json", "archival/p18/reference_values_p
+> 18.json", "archival/p18/reproducibility/author_rebuild_report_p18.json", "archival/p18/reproducibility/in
+> dependent_rebuild_report_p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.json", "arch
+> ival/p19/reference_values_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "arch
+> ival/p19/reproducibility/independent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p2
+> 0_source_files.json", "archival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebu
+> ild_report_p20.json", "archival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/R
+> EADME.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/
+> reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report
+> _p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_
+> source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuil
+> d_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/con
+> tract_registry.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "c
+> ode/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "c
+> ode/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "co
+> de/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/r
+> elease_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_c
+> orrection.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscri
+> pt/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values_p
+> 23.json", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_p22.py", "tests/test
+> _p23.py"],
  "removed": ["reproducibility/reference_values_p18.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/release_p18.py", "code/p5_entry/dred.py
+> ", "manuscript/body/XVI-P-body.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "test
+> s/test_p18.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p18/README.md", "archival
+> /p18/p18_source_files.json", "archival/p18/reference_values_p18.json", "archival/p18/reproducibility/auth
+> or_rebuild_report_p18.json", "archival/p18/reproducibility/independent_rebuild_report_p18.json", "archiva
+> l/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p19.json", "archiv
+> al/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/independent_rebuild
+> _report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/referenc
+> e_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproduc
+> ibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.js
+> on", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.js
+> on", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archi
+> val/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_
+> values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducib
+> ility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/d11_pin.py
```

```
+> ", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "code/p5_calc/p19_conventions.py", "code/p5_ca
+> lc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_ca
+> lc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p18.py", "code/p5_calc/re
+> lease_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py
+> ", "code/p5_calc/release_p23.py", "manuscript/body/XVI-P-body.tex", "manuscript/notes/P19_defect.tex", "m
+> anuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evane
+> scent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibili
+> ty/reference_values_p18.json", "reproducibility/reference_values_p23.json", "tests/test_p14.py", "tests/t
+> est_p16.py", "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/t
+> est_p21.py", "tests/test_p22.py", "tests/test_p23.py", "tools/build.py", "tools/rebuild_from_source.py",
+> "tools/source_manifest.py"],
  "counts": {
    "added": 53,
    "removed": 1,
    "modified": 12
  }
},
"p21_layer_absolute_pin": {
  "check": {
    "ratio_pin_PASS": true,
    "ratio_10": true,
    "transverse": true,
    "C2_quadratic_form_half_P2": true,
    "GaussBonnet_zero": true,
    "absolute_pin_PASS": true,
    "absolute_pin_flag": true,
    "top_level_absolute_status_consistent": true,
    "top_level_ratio_status_consistent": true,
    "absolute_ratios_one": true,
    "no_external_convention_choice": true,
    "drop_D11_detected": true,
    "x2_detected_by_absolute": true,
    "sign_flip_detected_by_absolute": true,
    "corrected_in_p21": true,
    "literature_source_stated": true,
    "revision": true
  },
  "absolute_pin": {
    "status": "PASS",
    "observed_common_factor_c2_vs_bare_a2": "-I",
    "observed_common_factor_c0_vs_bare_a2": "-I",
    "derivation_p21": {
      "package_declarations_checked": {
        "N_eps = i S_eps/(16 pi^2)": true,
        "Gamma_1 = -i * graph sum": true,
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
        "g = eta + kappa h": true,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
        "eta = (+,-,-,-)": true,
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
      },
      "steps": {
        "1_G_equals_Gamma2": "1",
        "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
        "3_i_S_M_over_S_E (must be -1)": "-1",
        "4_L_M_eff_over_L_E_eff": "-1",
        "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
        "6_minkowski_quadratic_forms": {
          "Riem2": {
            "P2": "1",
            "P0s": "1"
          },
          "Ric2": {
            "P2": "1/4",
            "P0s": "1"
          },
          "R2": {
            "P2": "0",
            "P0s": "3"
          },
          "C2": {
            "P2": "1/2",
            "P0s": "0"
          },
          "E_GaussBonnet": {
            "P2": "0",
            "P0s": "0"
          }
        }
      },
```

```
      "package_Gamma2_pole_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "heat_kernel_prediction_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "ratio_package_over_prediction": {
        "spin2": "1",
        "spin0s": "1"
      },
      "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2
+>  (Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps
+> )) int a_2 (Euclidean, d = 4-2eps)"],
      "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, s
+> ign of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
      "status": "PASS"
    },
    "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under th
+> e package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the W
+> ick direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(
+> 16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1
+> , both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no ext
+> ernal convention choice.  p20 had left this OPEN."
  },
  "absolute_pin_status": "PASS",
  "mutations": {
    "drop_graph_D11": {
      "c2": "0",
      "c0": "0",
      "ratio": "None",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_normalisation_x2": {
      "c2": "-I/120",
      "c0": "-I/12",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_sign_flip": {
      "c2": "I/240",
      "c0": "I/24",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "swap_g4g4_structures": {
      "c2": "-I/40",
      "c0": "-I/32",
      "ratio": "5/4",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    }
  }
},
"p21_layer_lower_layers": {
  "contract_identity": true,
  "p18_layer": true,
  "P18_quantum_inputs_evaluated": "earned",
  "inventory": true,
  "p19_layer": true,
  "k1_correction": true,
  "retraction": true,
  "retraction_hits": {},
  "contract_detail": {
    "p14r_contract_files": 12,
    "changed_vs_p14r": ["code/p5_entry/dred.py"],
    "added": [],
    "dred_byte_identical_to_p18": true,
    "dred_registry_state_p22": "p18 + p22 guard block",
```

```
    "contract_revision_in_p19": false,
    "disclosed_revisions_p19": {}
  },
  "rechecked_live": true
},
"p21_layer_p20_layer": {
  "p20_conventions_byte_identical": true,
  "disclosed_p21_revisions": {
    "code/p5_calc/d11_pin.py": {
      "sha256": "a2e77eecc84263e254d3b7cd7b441731324e8d15b5d15c71dc05d61ff5fe54f8",
      "differs_from_p20": true,
      "disclosed": true
    },
    "code/p5_calc/release_p20.py": {
      "sha256": "a1afba60c21c923886e682b960dda8b2619353712517f078c9967ad9c57e4d5f",
      "differs_from_p20": true,
      "disclosed": true
    }
  },
  "P20_release_accepted": true,
  "P20_D11_absolute_pin_verified": "earned"
},
"p21_layer_manuscript_disclosure": {
  "manuscript/notes/P19_defect.tex": {
    "current_equals_disclosed_p20": true,
    "differs_from_p19": true,
    "revised_in": "p20"
  }
},
"p21_layer_change_log_vs_p20": {
  "base": "archival/p20/p20_source_files.json (pinned; files of the p20 zip sha256 ca32ce018933f12d1a623217d
+> 6a98d647817360dd12211e2e9469cf850a63bb2)",
  "base_pinned_ok": true,
  "added": ["archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/reference_values_p
+> 20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproducibility/in
+> dependent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.json", "arch
+> ival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "arch
+> ival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22/ce
+> rtificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_values_p22
+> .json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/inde
+> pendent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/evanescent.py", "cod
+> e/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "cod
+> e/p5_calc/power_counting.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py", "code/p5_calc
+> /release_p23.py", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manusc
+> ript/notes/P23_01_scope.tex", "provenance/XVI-O/O_quantum_target.json", "reproducibility/reference_values
+> _p23.json", "tests/test_p21.py", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p20.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/d11_pin.py", "code/p5_calc/release_p18.
+> py", "code/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_entry/dred.py", "manuscript/b
+> ody/XVI-P-body.tex", "manuscript/notes/P20_correction.tex", "tests/test_p14.py", "tests/test_p16.py", "te
+> sts/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tools/build.py", "tools
+> /rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": [],
  "undisclosed_package_changes": [],
  "counts": {
    "added": 33,
    "removed": 1,
    "modified": 18
  }
},
"p21_layer_open_pins": [],
"p21_layer_disclosed_p20_layer_revisions_p21": {
  "code/p5_calc/d11_pin.py": "absolute_pin(): sign and magnitude derived from 1/2 Tr ln under the declared c
+> onventions (Minkowski quadratic forms curvature_forms_eta); absolute pin PASS; mutations x2 and sign flip
+> ; top-level absolute_pin_status from the derivation",
  "code/p5_calc/release_p20.py": "absolute pin PASS required; top-level / nested status consistency; standal
+> one run re-checks the p18 layer (q earned) and the graph inventory; claim texts"
},
"p21_layer_disclosed_test_revisions_p21": {
  "tests/test_p14.py": "names p20 -> p21; generated prefixes P21_",
  "tests/test_p19.py": "reviewed coefficient sites read from p21_conventions (adds the d11_pin Minkowski for
+> m site)",
  "tests/test_p20.py": "D11 pin tests: absolute pin PASS (p21) instead of OPEN; gate mutations on the p21 ch
+> ecks; reviewed sites from p21_conventions; status/version expectations p21"
},
"p21_layer_disclosed_manuscript_revisions": {
  "manuscript/notes/P19_defect.tex": {
    "p19_sha256": "04b974b5ea742c6885f3d694ccad6bda7908105c0a23bf483e96bc113d90bbc8",
    "p20_sha256": "1c71a71a48ce9c1749bd2164a0f720a3589f2d679877cfd04309d69670e5dfa8",
    "revised_in": "p20",
    "change": "K1 paragraph retracted/rewritten (Grassmann orientation erratum), restoration definition REDE
```

```
+> FINED, J1 wording, blind-spot text; marked inline [corrected in p20] / [REDEFINED in p20]; unchanged in p
+> 21"
  }
},
"p21_layer_independent_rebuild_passed_p20": true,
"p21_layer_p20_independent_rebuild": {
  "report": "archival/p20/reproducibility/independent_rebuild_report_p20.json",
  "present": true,
  "report_sha256": "e8f358f9c4f13c76693968211b49e04d546a0bb3a4c0ab4b1a0e485f62595389",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700",
  "bound_to_source_zip_sha256": "ca32ce018933f12d1a623217d6a98d647817360dd12211e2e9469cf850a63bb2",
  "tree_matches_p20": true,
  "zip_matches_p20": true,
  "mismatches": 0,
  "findings": ["p19_quotient_finding_withdrawn", "D11_absolute_pin_left_OPEN_but_derivable", "scan_misses_p2
+> 0", "gate_top_level_absolute_pin_status_unchecked", "p20_gate_trusts_P19_layer_record", "P19_note_edit_di
+> sclosure", "restoration_scope", "p19_change_log_key_recomputed_on_p20_tree"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p21": "bound to the p20 tree/zip; the p21 tree has its own (pending) independent flag",
  "archived_p20_records_pinned": {
    "archival/p20/reference_values_p20.json": true,
    "archival/p20/reproducibility/author_rebuild_report_p20.json": true,
    "archival/p20/reproducibility/independent_rebuild_report_p20.json": true,
    "archival/p20/p20_source_files.json": true
  },
  "pins_ok": true
},
"evanescent": {
  "check": {
    "EV_M2_proven": true,
    "EV_CT_earned": true,
    "controls_all_true": true,
    "hc_closed": true,
    "cb_closed": true,
    "identity_unchanged": true,
    "mutations_detected": true,
    "continuations_refuted_recorded": true,
    "Q4S_trace_remnant_disclosed": true,
    "P5_status": true,
    "contract_revision_named": true
  },
  "EV_M2": "EV-M2: the evanescent D11 poles are local evanescent operators (removed by the p22 evanescent co
+> unterterm), and the renormalised evanescent remainder vanishes identically on 4D external configurations,
+>  exactly in d.  No matching to XVI-O is asserted.",
  "EV_M2_verdict": "PROVEN (exact, opaque d)",
  "coefficients_hh_sum": {
    "g4_mn gt_rs (p.p)^2": "-7*I/120",
    "g4_mr gt_ns (p.p)^2": "-I/160",
    "g4_rs gt_mn (p.p)^2": "-7*I/120",
    "gt_mn gt_rs (p.p)^2": "I/160",
    "gt_mn p_r p_s (p.p)^1": "7*I/120",
    "gt_mr gt_ns (p.p)^2": "I/160",
    "gt_mr p_n p_s (p.p)^1": "I/160",
    "gt_rs p_m p_n (p.p)^1": "7*I/120"
  },
  "coefficient_Kc_E2": "-1/16",
  "coefficient_ccb_SE": "1/8",
  "remnants": {
    "finite_evanescent_remainder": "MS-bar finite parts (local + L) x gt-structures: recorded per structure
+> in P18_D11 / P16_batchC / P15_D13; NONZERO; identically zero on 4D (hatted) external legs, exactly in d (
+> four_d_decoupling below)",
    "O_eps_times_pole_at_this_order": "none with QdS (hatted) external legs: all four indices are external,
+> so tr gt = 4 - d never arises and the evanescent operators are ZERO on hatted legs at every d; see the Q4
+> S trace entry for the one place it can arise",
    "trace_of_pole_tensor_g4^{mn}T_mn,rs (hatted r,s)": {
      "per_graph": {
        "D01": "-I*p.p*(d - 4)*(13*a^.b^*p.p - 18*a^.p*b^.p)/120",
        "D03": "I*p.p*(d - 4)*(a^.b^*p.p - 2*a^.p*b^.p)/24",
        "D11": "I*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)/120"
      },
      "sum": "-7*I*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)/120"
    },
    "O_eps_times_pole_Q4S_trace_external": {
      "finite_term_if_external_trace_mode_uses_g4": "7*I*p.p*(a^.b^*p.p - a^.p*b^.p)/60",
      "rule": "(d - 4) x (1/eps) -> -2 (d = 4 - 2 eps); times the residue above",
```

```
      "status": "NONZERO: a finite local transverse term arises iff the external trace mode is defined with
+> the quasi-4D g4 (it then carries eps-scalar components); with QdS (hatted) external legs it is zero.  Thi
+> s is a scheme / external-state convention (DRED eps-scalar sector), disclosed: the p22 renormalised resul
+> t is stated for QdS external legs"
    },
    "where_0_eps_x_pole_would_enter": "insertions of the evanescent counterterms into loops (two loops) or i
+> nto composite operators with traces: (4 - d) x (1/eps) = finite local terms -- beyond this order; not eva
+> luated, not claimed"
  }
},
"P5_calculation_status": "renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-bar in D
+> RED with the p22 evanescent counterterms; external legs in QdS)",
"d_runtime_guard": {
  "ok": true,
  "n_plants": 30,
  "not_raised": {},
  "other_outcomes": {},
  "algebra_ok": "(d**2 + 30*d - 34)/(32*(d - 1)*(d + 1))",
  "entry_points": ["OpaqueDimension.__eq__ (d == d)", "Atom.xreplace (bare d)", "Basic.subs", "Basic.xreplac
+> e", "Basic.replace", "evalf/n(subs=...)", "Poly.eval", "Poly.__call__", "Poly.all_coeffs", "Poly.coeffs",
+>  "Poly.terms", "Poly.all_terms", "Poly.as_dict", "Poly.as_list", "Poly.coeff_monomial", "Poly.nth", "Poly
+> .LC", "Poly.TC", "Poly.EC", "Poly.as_expr", "Expr.coeff", "Expr.as_independent", "Expr.as_coeff_exponent"
+> , "Expr.series", "collect", "limit", "Lambda(d, ...)", "polytools.rem (d in domain)", "polytools.prem (d
+> in domain)", "polytools.pdiv (d in domain)", "polytools.pquo (d in domain)", "polytools.pexquo (d in doma
+> in)", "polytools.div (d in domain)", "polytools.quo (d in domain)", "polytools.exquo (d in domain)", "pol
+> ytools.gcdex (d in domain)", "polytools.half_gcdex (d in domain)", "polytools.invert (d in domain)", "pol
+> ytools.subresultants (d in domain)", "polytools.resultant (d in domain)", "polytools.discriminant (d in d
+> omain)", "polytools.compose (d in domain)", "polytools.decompose (d in domain)", "polytools.sturm (d in d
+> omain)", "polytools.nroots (d in domain)", "polytools.real_roots (d in domain)", "polytools.count_roots (
+> d in domain)", "polytools.ground_roots (d in domain)", "polytools.intervals (d in domain)", "polytools.re
+> fine_root (d in domain)", "polytools.nth_power_roots_poly (d in domain)", "Poly.rem (d in domain)", "Poly
+> .prem (d in domain)", "Poly.pdiv (d in domain)", "Poly.pquo (d in domain)", "Poly.pexquo (d in domain)",
+> "Poly.div (d in domain)", "Poly.quo (d in domain)", "Poly.exquo (d in domain)", "Poly.gcdex (d in domain)
+> ", "Poly.half_gcdex (d in domain)", "Poly.invert (d in domain)", "Poly.subresultants (d in domain)", "Pol
+> y.resultant (d in domain)", "Poly.discriminant (d in domain)", "Poly.compose (d in domain)", "Poly.decomp
+> ose (d in domain)", "Poly.sturm (d in domain)", "Poly.nroots (d in domain)", "Poly.real_roots (d in domai
+> n)", "Poly.count_roots (d in domain)", "Poly.ground_roots (d in domain)", "Poly.intervals (d in domain)",
+>  "Poly.refine_root (d in domain)", "Poly.nth_power_roots_poly (d in domain)", "Poly.shift (d in domain)",
+>  "Poly.transform (d in domain)", "Poly.shift_list (d in domain)", "internal rename of d to a plain Symbol
+>  (Matrix.subs)"],
  "residue_not_interceptable": "NOT runtime-interceptable (code-review matters; the static allow-list scan i
+> s the secondary barrier): identity / name / type tests in hand-written expression-tree rebuilds (`x is d`
+> , x.name == 'd', isinstance), string round-trips (str(e).replace('d', '4')), low-level polynomial represe
+> ntations (Poly.rep, DMP, polyutils dict builders), and code that disguises its module name as sympy.  Sym
+> bol.__new__(Symbol, 'd') yields a PLAIN d that does not match the opaque d (no value is produced); parse_
+> d / assert_opaque_d refuse plain d in loaded data.",
  "static_scan": "secondary barrier (tests/test_p21.py scan4 allow-list, p22 sites reviewed)"
},
"contract": {
  "dred_registry": {
    "state": "p18 + p22 guard block",
    "ok": true,
    "sha256": "24112d9a6584613474e96f538840a9ff6aecbf9ebd588cd890fd38d2e295de5a",
    "p18_core_byte_identical": true,
    "registered_blocks": ["p22 runtime d guard", "p23 polynomial-domain refusal"]
  },
  "p22_block_unchanged": true,
  "p23_registered": true,
  "dred_sha256": "24112d9a6584613474e96f538840a9ff6aecbf9ebd588cd890fd38d2e295de5a",
  "dred_pinned": "24112d9a6584613474e96f538840a9ff6aecbf9ebd588cd890fd38d2e295de5a",
  "contract_revision_p23": "p23 (p14r contract + disclosed p15/p16/p17/p18 revisions + the p22 contract revi
+> sion + the DISCLOSED p23 contract revision: polynomial-domain refusal for d and Dummy-only internal renam
+> es, one delimited block appended to code/p5_entry/dred.py; p18 core and p22 block byte-identical)",
  "other_contract_files_changed": []
},
"lower_layers": {
  "contract_identity": true,
  "p18_layer": true,
  "P18_quantum_inputs_evaluated": "earned",
  "inventory": true,
  "p19_layer": true,
  "k1_correction": true,
  "retraction": true,
  "retraction_hits": {},
  "contract_detail": {
    "p14r_contract_files": 12,
    "changed_vs_p14r": ["code/p5_entry/dred.py"],
    "added": [],
    "dred_byte_identical_to_p18": true,
    "dred_registry_state_p22": "p18 + p22 guard block",
    "contract_revision_in_p19": false,
```

```
    "disclosed_revisions_p19": {}
  },
  "rechecked_live": true
},
"p20_layer": {
  "p20_conventions_byte_identical": true,
  "disclosed_p21_revisions": {
    "code/p5_calc/d11_pin.py": {
      "sha256": "a2e77eecc84263e254d3b7cd7b441731324e8d15b5d15c71dc05d61ff5fe54f8",
      "differs_from_p20": true,
      "disclosed": true
    },
    "code/p5_calc/release_p20.py": {
      "sha256": "a1afba60c21c923886e682b960dda8b2619353712517f078c9967ad9c57e4d5f",
      "differs_from_p20": true,
      "disclosed": true
    }
  },
  "P20_release_accepted": true,
  "P20_D11_absolute_pin_verified": "earned"
},
"p21_layer": {
  "p21_conventions_byte_identical": true,
  "P21_release_accepted": true,
  "P21_earned": {
    "projected_defect_computed": "earned",
    "projected_identity_verified": "earned",
    "projected_restoration_verified": "earned",
    "D11_ratio_pin_verified": "earned",
    "D11_absolute_pin_verified": "earned"
  }
},
"absolute_pin_check": {
  "ratio_pin_PASS": true,
  "ratio_10": true,
  "transverse": true,
  "C2_quadratic_form_half_P2": true,
  "GaussBonnet_zero": true,
  "absolute_pin_PASS": true,
  "absolute_pin_flag": true,
  "top_level_absolute_status_consistent": true,
  "top_level_ratio_status_consistent": true,
  "absolute_ratios_one": true,
  "no_external_convention_choice": true,
  "drop_D11_detected": true,
  "x2_detected_by_absolute": true,
  "sign_flip_detected_by_absolute": true,
  "corrected_in_p21": true,
  "literature_source_stated": true,
  "revision": true
},
"manuscript_disclosure": {
  "manuscript/notes/P20_correction.tex": {
    "differs_from_p21": true,
    "superseded_marker_present": true,
    "revised_in": "p22"
  }
},
"change_log_vs_p21": {
  "base": "archival/p21/p21_source_files.json (pinned; files of the p21 zip sha256 a71d6b5203bc3914833db9fd1
+> 3b9d186bef93d1ae3007b2e4c99fdfce11e0678)",
  "base_pinned_ok": true,
  "added": ["archival/p21/README.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p
+> 21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/in
+> dependent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.j
+> son", "archival/p22/p22_source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/repro
+> ducibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.
+> json", "code/p5_calc/contract_registry.py", "code/p5_calc/evanescent.py", "code/p5_calc/p22_conventions.p
+> y", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p22.py", "
+> code/p5_calc/release_p23.py", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P23_01_scope.tex",
+>  "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values_p23.json", "tests/test_p22.p
+> y", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p21.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/release_p18.py", "code/p5_calc/release_
+> p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_entry/dred.py", "manuscri
+> pt/body/XVI-P-body.tex", "manuscript/notes/P20_correction.tex", "tests/test_p14.py", "tests/test_p16.py",
+>  "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py",
+>  "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "contract_changes_registered": true,
  "undisclosed_changes": [],
```

```
  "pinned_sha_mismatches": {},
  "disclosed_but_unchanged": [],
  "exemptions": {
    "generated": ["reproducibility/reference_values_p22.json", "reproducibility/reference_values_p21.json"],
    "added_archive_prefixes": ["archival/p21/"]
  },
  "counts": {
    "added": 24,
    "removed": 1,
    "modified": 19
  }
},
"open_pins": [],
"evanescent_M2_verified": "earned",
"evanescent_counterterms_verified": "earned",
"d_runtime_guard_verified": "earned",
"disclosed_revisions_p22": {
  "code/p5_entry/dred.py": "CONTRACT REVISION p22: runtime d guard (one delimited additive block; p18 text u
+> nchanged)",
  "code/p5_calc/p22_conventions.py": "new: p22 conventions",
  "code/p5_calc/evanescent.py": "new: EV-M2 and EV-CT (priorities 1, 2)",
  "code/p5_calc/contract_registry.py": "new: registry of the post-p18 contract revision (consulted by layer
+> gates)",
  "code/p5_calc/release_p22.py": "new: p22 release gate",
  "code/p5_calc/release_p18.py": "p22: accept the registered dred.py revision in place of the p17 AST identi
+> ty",
  "code/p5_calc/release_p19.py": "p22: contract identity accepts the registered dred.py revision; p18-layer
+> pin accepts the registered release_p18.py revision",
  "code/p5_calc/release_p20.py": "p22: change log ignores the registered contract revision; p18-layer pin ac
+> cepts the registered release_p18.py revision",
  "code/p5_calc/release_p21.py": "p22: change log ignores the registered contract revision and treats the p2
+> 2 revised files (contract_registry.P22_REVISED_FILES) as disclosed",
  "tests/test_p14.py": "names p21 -> p22",
  "tests/test_p16.py": "the replace(lambda x: x == d, ..) demonstration now RAISES (p22 runtime guard)",
  "tests/test_p17.py": "g1 scan reviews the p18 contract core of the registered dred revision (contract_regi
+> stry.scan_view; the p22 block is covered by the runtime probe + scan4)",
  "tests/test_p18.py": "h1 scan via contract_registry.scan_view; h3/h5 accept the registered p18+p22-block d
+> red state",
  "tests/test_p19.py": "i2 scan via contract_registry.scan_view; p18 calc layer accepts the registered p22 r
+> evision of release_p18.py (contract_registry.layer_file_ok)",
  "tests/test_p20.py": "status / version expectations p22; p21 tree no longer current; scan3 via scan_view;
+> p18 calc layer via contract_registry.layer_file_ok",
  "tests/test_p21.py": "scan allow-lists read from p22_conventions; the power-base subs limitation test now
+> asserts the p22 fix; status/version expectations p22; p21 layer independent flag not current",
  "tests/test_p22.py": "new: p22 tests",
  "manuscript/notes/P20_correction.tex": "inline [superseded in p21] marker on the 'Absolute pin: OPEN' item
+> ",
  "manuscript/notes/P22_evanescent.tex": "new: p22 note",
  "manuscript/body/XVI-P-body.tex": "input of the p22 note",
  "tools/build.py": "p22 build: release_p22 stage, p22 listings, value blocks, generated files",
  "tools/source_manifest.py": "reference file / zip name p22; p21 archive records",
  "tools/rebuild_from_source.py": "p22 reference values, release criteria, archival pins",
  "README.md": "p22 section",
  "REBUILD_INSTRUCTIONS.md": "p22 instructions"
},
"disclosed_manuscript_revisions_p22": {
  "manuscript/notes/P20_correction.tex": {
    "p21_sha256": "138c858852906d4daf3ed86d2e48a8d5ee8ef48717514bde00d4ca7151d3e7fb",
    "revised_in": "p22",
    "change": "inline [superseded in p21] marker on the 'Absolute pin: OPEN' item (reviewer stale_OPEN_text)
+> "
  }
},
"independent_rebuild_passed_p21": true,
"p21_independent_rebuild": {
  "report": "archival/p21/reproducibility/independent_rebuild_report_p21.json",
  "present": true,
  "report_sha256": "8a582b332cd242689c54b1f962dc88270b9592040c0ba432576a505f0e36a60d",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "b835353ffe16a8ff159e2916fe3efdf2b9d89a4d512e80e9e6da587cecd0fe5e",
  "bound_to_source_zip_sha256": "a71d6b5203bc3914833db9fd13b9d186bef93d1ae3007b2e4c99fdfce11e0678",
  "tree_matches_p21": true,
  "zip_matches_p21": true,
  "mismatches": 0,
  "findings": ["p21_gate_disclosure_prefix_exemptions", "scan_misses_p21", "d_guard_power_base_hole", "stale
+> _OPEN_text_in_p20_section", "p20_gate_standalone_scope", "absolute_pin_step1_hardcoded", "reviewer_rule_n
+> ote"],
  "caveat": "same box (disclosed)",
  "live_report_identical_to_archived": true,
```

```
  "NOT_inherited_by_p22": "bound to the p21 tree/zip; the p22 tree has its own (pending) independent flag",
  "archived_p21_records_pinned": {
    "archival/p21/reference_values_p21.json": true,
    "archival/p21/reproducibility/author_rebuild_report_p21.json": true,
    "archival/p21/reproducibility/independent_rebuild_report_p21.json": true,
    "archival/p21/p21_source_files.json": true
  },
  "pins_ok": true
},
"O1": {
  "requirement": "O.1 closes iff the one-loop kappa^2 bulk two-point sector on S_inv closes under a VERIFIED
+>  retained quantum identity: every checklist item (O.2 classical kappa^2 master Constructed incl. grav ell
+> _4; O.4 inventory; O.5 all graphs evaluated; O.6 identity verified; O.7 remainder cancellation / CT solut
+> ion) earned, and the target enlarged first if the identity constrains more legs/orders at kappa^2.",
  "items_scope": {
    "State retained quantum identity (O.6)": {
      "scope": "PARTLY in XVI-P",
      "evidence": "projected ch identity at one loop kappa^2 stated and verified (P19/P20), incl. evanescent
+>  CTs (P22); the BULK retained identity (ST4b functional) is XVI-O's statement, not changed"
    },
    "List amplitudes constrained by identity (O.4)": {
      "scope": "IN XVI-P (this record)",
      "evidence": "one-loop kappa^2: exactly the E=2 two-point and E=1 one-point graphs; enumeration reprodu
+> ces the frozen inventory; completeness at topology level only"
    },
    "S_inv^{[2]} classical consistency (O.2)": {
      "scope": "OUT OF SCOPE",
      "evidence": "classical kappa^2 master 2(S0,S2)+(S1,S1): grav ell_4 Incomplete in XVI-O/XVI-N; XVI-A..O
+>  untouched -> O.1 cannot close inside XVI-P"
    },
    "Evaluate inventory graphs (O.5)": {
      "scope": "IN XVI-P",
      "evidence": "all E=2 kappa^2 graphs evaluated (P15..P18, D12 Missing never zero per P14 -> see open it
+> ems)"
    },
    "Demonstrate identity remainder cancellation / CT solution (O.6/O.7)": {
      "scope": "PARTLY in XVI-P",
      "evidence": "projected: closure + restoration earned (P20, P22); bulk ST4a/ST4b split: XVI-O"
    },
    "Enlarge target if identity requires more legs/orders (O.1)": {
      "scope": "IN XVI-P (this record)",
      "evidence": "NOT triggered at kappa^2 at one loop (three-point one-loop sector empty at kappa^2); next
+>  order kappa^3 three-point = P.6 entry"
    }
  },
  "closable_in_XVI_P": false,
  "reason": "O.2 (classical kappa^2 master, grav ell_4 Incomplete) and the bulk ST4b identity are owned by X
+> VI-O/XVI-N, which stay untouched",
  "ledger": "provenance/XVI-O/O_quantum_target.json",
  "ledger_sha256": "3cd3d1ba491381ba0a73d1677bfed10d3743e0575f61c71c325e626e4a78a645"
},
"power_counting": {
  "vertex_rule": true,
  "kappa_equals_E": true,
  "identity_at_kappa2": {
    "terms": ["(S^(0), Gamma_1^(2))  [E=2]", "(S^(1), Gamma_1^(1))  [E=1]", "(S^(2), Gamma_1^(0)) = 0  [fiel
+> d independent]"],
    "orders_entering": ["E=2 two-point (hh, phiphi, c.cb, K.c)", "E=1 one-point"],
    "three_point_one_loop_at_kappa2": "EMPTY (kappa^E with E=3 is kappa^3)",
    "enlarge_rule_triggered": false
  },
  "frozen_E2_match": {
    "enumerated": 9,
    "frozen": 9,
    "missing_from_enumeration": [],
    "not_in_frozen_inventory": [],
    "equal": true
  }
},
"P6_entry": {
  "sector": "one loop, kappa^3, three-point (incl. antifield K,c,h and cs,c,c / phis,c,phi)",
  "n_graphs": 15,
  "by_external": {
    "K,c,h": 2,
    "c,c,cs": 1,
    "c,cb,h": 2,
    "c,phi,phis": 1,
    "h,h,h": 5,
    "h,phi,phi": 4
  },
  "status": "inventory only (topology level); amplitude rules, counterterm domain, evaluation, defect and cl
```

```
+> osure NOT done",
  "evaluated": false
},
"three_point_defect": "not computed (no graph evaluated)",
"title_qualifier": {
  "title_line": "# XVI-P \u2014 revision p23 (stage release on the 0.1 direction: 0.1 requirement recorded,
+> one-loop power counting and enlarge-rule check, P.6 entry inventory (\u03ba\u00b3 three-point, not evalua
+> ted), polynomial-domain refusal for d; P.5 renormalized at one loop \u03ba\u00b2 for the two-point functi
+> ons with QdS external legs; bulk symmetry not claimed; 0.1 NOT_CLOSED) \u2014 2026-10-10",
  "pdf_title_lines_with_qualifier": 2,
  "required": "P.5 renormalized at one loop kappa^2 for the two-point functions with QdS external legs"
},
"d_poly_domain_refusal": {
  "ok": true,
  "n_plants": 9,
  "not_raised": {},
  "algebra": {
    "cancel_factor": "(d**2 + 30*d - 34)/(32*(d - 1)*(d + 1))",
    "apart": "1/32 - 1/(64*(d + 1)) - 3/(64*(d - 1))",
    "together": "(d**2 - 2*d - 2)/(32*(d**2 - 1))",
    "authorised_value": "1/80"
  },
  "algebra_ok": true
},
"EV_values": {
  "archived_cert_pinned": true,
  "values": {
    "calibration_rho_from_4D_pair": {
      "expected": "1/2",
      "current": "1/2",
      "archived_p22": "1/2",
      "ok": true
    },
    "coefficient_Kc_E2": {
      "expected": "-1/16",
      "current": "-1/16",
      "archived_p22": "-1/16",
      "ok": true
    },
    "coefficient_ccb_SE": {
      "expected": "1/8",
      "current": "1/8",
      "archived_p22": "1/8",
      "ok": true
    },
    "coefficients_hh_counterterm_sum": {
      "expected": {
        "g4_mn gt_rs (p.p)^2": "-7*I/120",
        "g4_mr gt_ns (p.p)^2": "-I/160",
        "g4_rs gt_mn (p.p)^2": "-7*I/120",
        "gt_mn gt_rs (p.p)^2": "I/160",
        "gt_mn p_r p_s (p.p)^1": "7*I/120",
        "gt_mr gt_ns (p.p)^2": "I/160",
        "gt_mr p_n p_s (p.p)^1": "I/160",
        "gt_rs p_m p_n (p.p)^1": "7*I/120"
      },
      "current": {
        "g4_mn gt_rs (p.p)^2": "-7*I/120",
        "g4_mr gt_ns (p.p)^2": "-I/160",
        "g4_rs gt_mn (p.p)^2": "-7*I/120",
        "gt_mn gt_rs (p.p)^2": "I/160",
        "gt_mn p_r p_s (p.p)^1": "7*I/120",
        "gt_mr gt_ns (p.p)^2": "I/160",
        "gt_mr p_n p_s (p.p)^1": "I/160",
        "gt_rs p_m p_n (p.p)^1": "7*I/120"
      },
      "archived_p22": {
        "g4_mn gt_rs (p.p)^2": "-7*I/120",
        "g4_mr gt_ns (p.p)^2": "-I/160",
        "g4_rs gt_mn (p.p)^2": "-7*I/120",
        "gt_mn gt_rs (p.p)^2": "I/160",
        "gt_mn p_r p_s (p.p)^1": "7*I/120",
        "gt_mr gt_ns (p.p)^2": "I/160",
        "gt_mr p_n p_s (p.p)^1": "I/160",
        "gt_rs p_m p_n (p.p)^1": "7*I/120"
      },
      "ok": true
    }
  }
},
"p22_layer": {
```

```
  "publication": "accepted",
  "earned": {
    "projected_defect_computed": true,
    "projected_identity_verified": true,
    "projected_restoration_verified": true,
    "D11_ratio_pin_verified": true,
    "D11_absolute_pin_verified": true,
    "evanescent_M2_verified": true,
    "evanescent_counterterms_verified": true,
    "d_runtime_guard_verified": true
  },
  "p22_conventions_byte_identical": true,
  "live_contract_check": true,
  "live_guard_probe": true,
  "guard_not_raised": {},
  "live_evanescent_check": true,
  "hard_stops_hold": true,
  "bulk_symmetry_restored": false,
  "O1_closure": "NOT_CLOSED"
},
"o1_scope_check": {
  "present": true,
  "ledger_sha_pinned": true,
  "cert_ledger_sha": true,
  "ledger_outcome_NOT_CLOSED": true,
  "vertex_rule_holds": true,
  "one_loop_kappa_equals_E": true,
  "frozen_E2_match": true,
  "enlarge_rule_not_triggered": true,
  "O1_not_closable_in_XVI_P": true,
  "E1_count": true,
  "E3_count": true,
  "E3_by_external": true,
  "mutations_detected": true,
  "pass": true
},
"change_log_vs_p22": {
  "base": "archival/p22/p22_source_files.json (pinned; files of the p22 zip sha256 ade2fa5c2dac6614c41ae38dc
+> 97d7c485975015a134650734d466ea6ffdef76f)",
  "base_pinned_ok": true,
  "added": ["archival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_sou
+> rce_files.json", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuild_r
+> eport_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/p23_co
+> nventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p23.py", "manuscript/notes/P23_O1_
+> scope.tex", "provenance/XVI-O/O_quantum_target.json", "reproducibility/reference_values_p23.json", "tests
+> /test_p23.py"],
  "removed": ["reproducibility/reference_values_p22.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/contract_registry.py", "code/p5_calc/re
+> lease_p21.py", "code/p5_calc/release_p22.py", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.tex",
+> "tests/test_p14.py", "tests/test_p21.py", "tests/test_p22.py", "tools/build.py", "tools/rebuild_from_sour
+> ce.py", "tools/source_manifest.py"],
  "undisclosed_changes": [],
  "pinned_sha_mismatches": {},
  "disclosed_but_unpinned": [],
  "disclosed_but_unchanged": [],
  "registry_lists_match_disclosure": true,
  "counts": {
    "added": 13,
    "removed": 1,
    "modified": 13
  }
},
"O1_requirement_recorded": "earned",
"one_loop_power_counting_verified": "earned",
"P6_entry_inventory": "earned",
"d_poly_domain_refusal_verified": "earned",
"EV_values_verified": "earned",
"disclosed_revisions_p23": {
  "code/p5_entry/dred.py": "p23 contract revision: polynomial-domain refusal + Dummy-only internal renames (
+> appended block)",
  "code/p5_calc/contract_registry.py": "p23 block recognised; p22 block pinned byte-identical; p23 revised/a
+> dded lists",
  "code/p5_calc/release_p21.py": "accepts the p23 registered revisions (contract_registry.P23_REVISED_FILES)
+> ",
  "code/p5_calc/release_p22.py": "accepts the registered p23 dred state and p23-disclosed files (pinned by t
+> he p23 gate)",
  "code/p5_calc/power_counting.py": "new: O.1 requirement, one-loop power counting, enlarge rule, P.6 entry
+> inventory",
  "code/p5_calc/p23_conventions.py": "new",
  "code/p5_calc/release_p23.py": "new: p23 gate",
  "tests/test_p14.py": "p23 names / status expectations",
```

```
    "tests/test_p21.py": "allow-list scan reads p23 sites; status expectations",
    "tests/test_p22.py": "p22 layer record read from certificates/P22_release.json; version label current",
    "tests/test_p23.py": "new",
    "tools/build.py": "p23 stages (power_counting, release_p23), listings, value blocks, archival p22",
    "tools/source_manifest.py": "p23 zip / reference; p22 archive records",
    "tools/rebuild_from_source.py": "p23 reference / certificates / ledgers",
    "README.md": "p23 section; title line with the two-point / QdS qualifier",
    "REBUILD_INSTRUCTIONS.md": "p23",
    "manuscript/body/XVI-P-body.tex": "input of the p23 note",
    "manuscript/notes/P23_01_scope.tex": "new: p23 note",
    "provenance/XVI-O/O_quantum_target.json": "new: verbatim read-only copy of the XVI-O 0.1 ledger (sha pinne
  +> d)"
  },
  "independent_rebuild_passed_p22": true,
  "p22_independent_rebuild": {
    "report": "archival/p22/reproducibility/independent_rebuild_report_p22.json",
    "report_sha256_pinned": true,
    "pass": true,
    "independent_rebuild_passed": true,
    "tree_matches_p22": true,
    "zip_matches_p22": true,
    "mismatches": 0,
    "findings": ["runtime_guard_bypass_poly_algebra", "p22_gate_does_not_check_EV_values", "pins_live_in_unpin
  +> ned_conventions", "earlier_layer_gates_accept_p22_cert_edits", "ev_decoupling_structural", "rho_single_eq
  +> uation_test", "renormalized_scope", "not_recomputed"],
    "caveat": "same box (disclosed)",
    "NOT_inherited_by_p23": "bound to the p22 tree/zip; the p23 tree has its own (pending) independent flag",
    "archived_p22_records_pinned": {
      "archival/p22/p22_source_files.json": true,
      "archival/p22/reference_values_p22.json": true,
      "archival/p22/reproducibility/author_rebuild_report_p22.json": true,
      "archival/p22/reproducibility/independent_rebuild_report_p22.json": true,
      "archival/p22/certificates/P22_evanescent.json": true
    },
    "pins_ok": true
  }
}
```


35 Milestone ledger (p23)

[listing 02/133] ledgers/P23_milestones.json


```
{
  "landmark": "XVI-P",
  "revision": "p23",
  "ledger": "milestones (authoritative definitions + status)",
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-bar in DRED
  +> with the p22 evanescent counterterms; external legs in QdS)",
    "P.5_identity": "projected defect computed (zero exactly in d, all parts, all seven assignments; each row
  +> vanishes separately); projected identity verified; projected restoration verified (p20 definition: full s
  +> igma_0 closure of the complete one-loop kappa^2 MS-bar counterterm incl. c.cb, h.c/c.b/c.cbs); K1 correct
  +> ed in p20 (orientation erratum); D11 ratio pin verified; D11 absolute pin verified (p21); bulk symmetry N
  +> OT restored/claimed; 0.1 NOT_CLOSED [p21: D11 absolute pin derived and verified] [p22: evanescent counter
  +> terms closed; identity unchanged]",
    "P.6_entry": "inventory (one loop kappa^3 three-point, 15 graphs, topology level); not evaluated",
    "O.1": "NOT_CLOSED (requirement recorded; not closable inside XVI-P)"
  },
  "definition_vs_status_note": "I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficients evaluate
  +> d and renormalized'; the STATUS states exactly what holds: 4D sector renormalized, evanescent poles regul
  +> ated and decoupled from the ch slots (not subtracted)",
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)",
  "author_rebuild_passed": true,
```

```
  "independent_rebuild_passed": "pending",
  "independent_rebuild_passed_p22": true,
  "hard_stops": {
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "u_R_c_delta_added": false,
    "XVI_P_registered": false
  },
  "flags": {
    "projected_defect_computed": "earned",
    "projected_identity_verified": "earned",
    "projected_restoration_verified": "earned",
    "D11_ratio_pin_verified": "earned",
    "D11_absolute_pin_verified": "earned",
    "evanescent_M2_verified": "earned",
    "evanescent_counterterms_verified": "earned",
    "d_runtime_guard_verified": "earned",
    "O1_requirement_recorded": "earned",
    "one_loop_power_counting_verified": "earned",
    "P6_entry_inventory": "earned",
    "d_poly_domain_refusal_verified": "earned",
    "EV_values_verified": "earned",
    "quantum_inputs_evaluated": "earned",
    "bulk_symmetry_restored": false
  }
}
```


36 Test results (p23)

[listing 03/133] certificates/P23_test_results.json


```
{
  "revision": "p23",
  "summary": "562 passed in 322.09s (0:05:22)",
  "groups": {
    "baseline": {
      "passed": 474,
      "failed": 0,
      "skipped": 0
    },
    "certificate_mutation": {
      "passed": 84,
      "failed": 0,
      "skipped": 0
    },
    "resumed_build": {
      "passed": 4,
      "failed": 0,
      "skipped": 0
    }
  },
  "results": {
    "certificate_mutations": {
      "master_count_flag_true": {
        "rc": 1,
        "publication": "REJECTED",
        "release_claim_accepted": false,
        "failed_prerequisites": ["master_grav_noether_deterministic_k0_k1_k2", "residual_record_invariants_all
  +> _certificates", "master_grav_Q_terms_zero_and_flag_true_all_orders", "cross_certificate_quartic_vs_master
  +> _agree"],
        "closure_artifact_exists": false
      },
      "quartic_count_flag_true": {
        "rc": 1,
        "publication": "REJECTED",
        "release_claim_accepted": false,
        "failed_prerequisites": ["C7_grav_noether_k0_k1_k2_on_exports", "residual_record_invariants_all_certif
  +> icates", "quartic_C7_Q_terms_zero_and_flag_true_all_orders", "cross_certificate_quartic_vs_master_agree",
  +>  "stored_C7_flag_equals_recomputation"],
        "closure_artifact_exists": false
      },
      "cross_certificate_mismatch": {
        "rc": 1,
```

```
      "publication": "REJECTED",
      "release_claim_accepted": false,
      "failed_prerequisites": ["cross_certificate_quartic_vs_master_agree"],
      "closure_artifact_exists": false
    }
  },
  "e2e_mutations": {
    "flipped_odd_sign": {
      "stage_codes": {
        "master": 0,
        "release": 1
      },
      "flag": "required_classical_BV_components_verified",
      "flag_value": false,
      "readiness": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_earned": false,
      "release_claim_accepted": false,
      "publication": "REJECTED",
      "closure_artifact_exists": false,
      "master_residuals": ["invariant_kappa1_af1:K(0, 0),K(0, 1),K(0, 2),K(0, 3)", "transformed_kappa1_af0:c
+> b0,cb1,cb2,cb3", "transformed_kappa1_af1:K(0, 0),K(0, 1),K(0, 2),K(0, 3)", "invariant_kappa2_af1:K(0, 0),
+> K(0, 1),K(0, 2),K(0, 3)", "transformed_kappa2_af0:cb0,cb1,cb2,cb3", "transformed_kappa2_af1:K(0, 0),K(0,
+> 1),K(0, 2),K(0, 3)"],
      "failed_prerequisites": ["master_engine_parts_exact"]
    },
    "corrupted_ell4_coefficient": {
      "stage_codes": {
        "master": 0,
        "release": 1
      },
      "flag": "required_classical_BV_components_verified",
      "flag_value": false,
      "readiness": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_earned": false,
      "release_claim_accepted": false,
      "publication": "REJECTED",
      "closure_artifact_exists": false,
      "master_residuals": ["grav_noether_kappa2_af0:Q_terms=[2, 7, 5, 0]", "ell4_sha256_mismatch_vs_P13_quar
+> tic_verified"],
      "failed_prerequisites": ["master_grav_noether_deterministic_k0_k1_k2", "master_engine_parts_exact", "m
+> aster_grav_Q_terms_zero_and_flag_true_all_orders", "cross_certificate_quartic_vs_master_agree", "hash_con
+> sistency"]
    },
    "dropped_hstar_c_row": {
      "stage_codes": {
        "ch": 0,
        "release": 1
      },
      "flag": "ch_dependency_inventory_complete",
      "flag_value": false,
      "readiness": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_earned": false,
      "release_claim_accepted": false,
      "publication": "REJECTED",
      "closure_artifact_exists": false,
      "master_residuals": [],
      "failed_prerequisites": []
    },
    "altered_gauge_shift": {
      "stage_codes": {
        "gauge": 0,
        "master": 0,
        "release": 1
      },
      "flag": "gauge_fixing_map_constructed",
      "flag_value": false,
      "readiness": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_earned": false,
      "release_claim_accepted": false,
      "publication": "REJECTED",
      "closure_artifact_exists": false,
      "master_residuals": ["transformed_kappa0_af0:b0,b1,b2,b3", "transformed_kappa1_af0:b0,b1,b2,b3"],
      "failed_prerequisites": ["master_engine_parts_exact", "canonicity_all_23_antifield_components"]
    },
    "omitted_cstar_term": {
      "stage_codes": {
        "master": 0,
```

```
        "release": 1
      },
      "flag": "required_classical_BV_components_verified",
      "flag_value": false,
      "readiness": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_earned": false,
      "release_claim_accepted": false,
      "publication": "REJECTED",
      "closure_artifact_exists": false,
      "master_residuals": ["invariant_kappa1_af1:K(0, 0),K(0, 1),K(0, 2),K(0, 3)", "transformed_kappa1_af0:c
+> 0,c1,c2,c3", "transformed_kappa1_af1:K(0, 0),K(0, 1),K(0, 2),K(0, 3)", "invariant_kappa2_af1:K(0, 0),K(0,
+>  1),K(0, 2),K(0, 3)", "transformed_kappa2_af0:c0,c1,c2,c3", "transformed_kappa2_af1:K(0, 0),K(0, 1),K(0,
+> 2),K(0, 3)"],
      "failed_prerequisites": ["master_engine_parts_exact"]
    },
    "stale_hash": {
      "stage_codes": {
        "master": 0,
        "release": 1
      },
      "flag": "required_classical_BV_components_verified",
      "flag_value": false,
      "readiness": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_earned": false,
      "release_claim_accepted": false,
      "publication": "REJECTED",
      "closure_artifact_exists": false,
      "master_residuals": ["ell4_sha256_mismatch_vs_P13_quartic_verified"],
      "failed_prerequisites": ["master_engine_parts_exact", "hash_consistency"]
    }
  },
  "matter_loop_controls": {
    "rc": 0,
    "tail": ["check_certs: PASS", "ALL test_matter_loop_gates: PASS"]
  },
  "p5_entry_gate_mutations": {
    "baseline_unmutated": {
      "rc": 0,
      "publication": "accepted",
      "P5_entry_earned": true,
      "P3_P4_frozen_as_P5_inputs": true,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": true,
        "D11_matching_defined": true,
        "counterterm_domain_defined": true,
        "amplitude_rules_verified": true
      },
      "hard_stops_hold": true,
      "closure_artifact_exists": true
    },
    "dred_mutation_not_rejected": {
      "rc": 1,
      "publication": "REJECTED",
      "P5_entry_earned": false,
      "P3_P4_frozen_as_P5_inputs": true,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": false,
        "D11_matching_defined": true,
        "counterterm_domain_defined": true,
        "amplitude_rules_verified": true
      },
      "hard_stops_hold": true,
      "closure_artifact_exists": false
    },
    "counterterm_flag_false": {
      "rc": 1,
      "publication": "REJECTED",
      "P5_entry_earned": false,
      "P3_P4_frozen_as_P5_inputs": true,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": true,
        "D11_matching_defined": true,
        "counterterm_domain_defined": false,
        "amplitude_rules_verified": true
      },
      "hard_stops_hold": true,
```

```
      "closure_artifact_exists": false
    },
    "D11_matching_verified_true": {
      "rc": 1,
      "publication": "REJECTED",
      "P5_entry_earned": true,
      "P3_P4_frozen_as_P5_inputs": true,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": true,
        "D11_matching_defined": true,
        "counterterm_domain_defined": true,
        "amplitude_rules_verified": true
      },
      "hard_stops_hold": false,
      "closure_artifact_exists": false
    },
    "D12_value_zero": {
      "rc": 1,
      "publication": "REJECTED",
      "P5_entry_earned": true,
      "P3_P4_frozen_as_P5_inputs": true,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": true,
        "D11_matching_defined": true,
        "counterterm_domain_defined": true,
        "amplitude_rules_verified": true
      },
      "hard_stops_hold": false,
      "closure_artifact_exists": false
    },
    "amplitude_mutation_not_rejected": {
      "rc": 1,
      "publication": "REJECTED",
      "P5_entry_earned": false,
      "P3_P4_frozen_as_P5_inputs": true,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": true,
        "D11_matching_defined": true,
        "counterterm_domain_defined": true,
        "amplitude_rules_verified": false
      },
      "hard_stops_hold": true,
      "closure_artifact_exists": false
    },
    "frozen_classical_file_changed": {
      "rc": 1,
      "publication": "REJECTED",
      "P5_entry_earned": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": true,
        "D11_matching_defined": true,
        "counterterm_domain_defined": true,
        "amplitude_rules_verified": true
      },
      "hard_stops_hold": true,
      "closure_artifact_exists": false
    },
    "p13_independent_report_tampered": {
      "rc": 1,
      "publication": "REJECTED",
      "P5_entry_earned": false,
      "P3_P4_frozen_as_P5_inputs": false,
      "P5_entry_flags": {
        "quantum_action_defined": true,
        "quantum_prescription_defined": true,
        "D11_matching_defined": true,
        "counterterm_domain_defined": true,
        "amplitude_rules_verified": true
      },
      "hard_stops_hold": true,
      "closure_artifact_exists": false
    }
  },
  "resumed_stage_gate": {
    "pdf": {
      "rc": 3,
```

```
      "stage": "pdf",
      "pdfs_left": [],
      "zips_left": [],
      "manifest_left": false,
      "stale": ["REJECTED.json", "REJECTED__RELEASE_MANIFEST_EXTERNAL.txt", "REJECTED__dist__XVI-P-full.zip"
 +> , "REJECTED__dist__XVI-P-p23-source-only.zip", "REJECTED__dist__XVI-P-review-focus.zip", "REJECTED__manus
 +> cript__XVI-P-appendix.pdf", "REJECTED__manuscript__XVI-P-science.pdf", "REJECTED__manuscript__XVI-P.pdf"]
 +> ,
      "marker": true,
      "synced": [],
      "reasons": ["release_p14.py exit 1", "P14_release (contract record) unreadable: [Errno 2] No such file
 +>  or directory: '/tmp/p14resume_trm2nly9/certificates/P14_release.json'", "release_claim_accepted/readines
 +> s not true", "P5 entry not earned (all five contract flags required)", "p13 freeze (P3_P4_frozen_as_P5_in
 +> puts) not true", "hard stop violated: quantum_inputs_evaluated=None"]
    },
    "zip": {
      "rc": 3,
      "stage": "zip",
      "pdfs_left": [],
      "zips_left": [],
      "manifest_left": false,
      "stale": ["REJECTED.json", "REJECTED__RELEASE_MANIFEST_EXTERNAL.txt", "REJECTED__dist__XVI-P-full.zip"
 +> , "REJECTED__dist__XVI-P-p23-source-only.zip", "REJECTED__dist__XVI-P-review-focus.zip", "REJECTED__manus
 +> cript__XVI-P-appendix.pdf", "REJECTED__manuscript__XVI-P-science.pdf", "REJECTED__manuscript__XVI-P.pdf"]
 +> ,
      "marker": true,
      "synced": [],
      "reasons": ["release_p14.py exit 1", "P14_release (contract record) unreadable: [Errno 2] No such file
 +>  or directory: '/tmp/p14resume_woefqcb1/certificates/P14_release.json'", "release_claim_accepted/readines
 +> s not true", "P5 entry not earned (all five contract flags required)", "p13 freeze (P3_P4_frozen_as_P5_in
 +> puts) not true", "hard stop violated: quantum_inputs_evaluated=None"]
    },
    "manifest": {
      "rc": 3,
      "stage": "manifest",
      "pdfs_left": [],
      "zips_left": [],
      "manifest_left": false,
      "stale": ["REJECTED.json", "REJECTED__RELEASE_MANIFEST_EXTERNAL.txt", "REJECTED__dist__XVI-P-full.zip"
 +> , "REJECTED__dist__XVI-P-p23-source-only.zip", "REJECTED__dist__XVI-P-review-focus.zip", "REJECTED__manus
 +> cript__XVI-P-appendix.pdf", "REJECTED__manuscript__XVI-P-science.pdf", "REJECTED__manuscript__XVI-P.pdf"]
 +> ,
      "marker": true,
      "synced": [],
      "reasons": ["release_p14.py exit 1", "P14_release (contract record) unreadable: [Errno 2] No such file
 +>  or directory: '/tmp/p14resume_epn38w9i/certificates/P14_release.json'", "release_claim_accepted/readines
 +> s not true", "P5 entry not earned (all five contract flags required)", "p13 freeze (P3_P4_frozen_as_P5_in
 +> puts) not true", "hard stop violated: quantum_inputs_evaluated=None"]
    },
    "manifest_unverified_pdfs": {
      "rc": 3,
      "stage": "manifest",
      "pdfs_left": [],
      "zips_left": [],
      "manifest_left": false,
      "stale": ["REJECTED.json", "REJECTED__RELEASE_MANIFEST_EXTERNAL.txt", "REJECTED__dist__XVI-P-full.zip"
 +> , "REJECTED__dist__XVI-P-p23-source-only.zip", "REJECTED__dist__XVI-P-review-focus.zip", "REJECTED__manus
 +> cript__XVI-P-appendix.pdf", "REJECTED__manuscript__XVI-P-science.pdf", "REJECTED__manuscript__XVI-P.pdf"]
 +> ,
      "marker": true,
      "synced": [],
      "reasons": ["pdf_inputs.json absent (PDFs not built by this pipeline)", "pdf readback absent or failed
 +> ", "zip_not_current:XVI-P-review-focus.zip: not a valid zip", "zip_not_current:XVI-P-full.zip: not a vali
 +> d zip", "zip_not_current:XVI-P-p23-source-only.zip:XVI-P-p23-source-only.zip: BadZipFile", "author_rebuil
 +> d_passed refers to a different source-only zip"]
    }
  }
 }
}
```

37 O.1 requirement, one-loop power counting, P.6 entry inventory (p23)

[listing 04/133] certificates/P23_01_scope.json

```
{
 "landmark": "XVI-P",
```

```
"revision": "p23",
"deliverable": "p23 stage 1: O.1 exact requirement, one-loop power counting, enlarge-rule check, P.6 entry in
 +> ventory",
"O1_source": {
 "file": "provenance/XVI-O/O_quantum_target.json",
 "sha256": "3cd3d1ba491381ba0a73d1677bfed10d3743e0575f61c71c325e626e4a78a645",
 "read_only": true,
 "gate": "O.1",
 "target": {
  "name": "one-loop_order_kappa2_bulk_two_point_on_S_inv",
  "loop_order": 1,
  "coupling_order": "kappa^2",
  "external": "bulk two-point (graviton, scalar, ghost) including Einstein-scalar hh channels + vertex-derive
 +> d gh=0 BV/source tree seeds; bulk mass frozen m=0",
  "baseline_action": "S_inv (NOT S_D)",
  "gauge": "frozen gauge fermion Psi=-i int cbar_mu (2 D^mu[h]-b^mu); b off shell",
  "bulk_mass": "m=0 (Track II massless-obstruction alignment)"
 },
 "closure_question": "Does this sector close under the retained quantum identity?",
 "outcome": "NOT_CLOSED",
 "checklist": [
  {
   "item": "State retained quantum identity",
   "status": "Constructed_as_statement",
   "owner": "O.6"
  },
  {
   "item": "List amplitudes constrained by identity",
   "status": "manually_specified_partially_checked_O.4",
   "owner": "O.4"
  },
  {
   "item": "S_inv^{[2]} classical consistency",
   "status": "Provisional_foundation_repaired_master_Partial",
   "owner": "O.2"
  },
  {
   "item": "Evaluate inventory graphs",
   "status": "D11_kernel_Constructed_subsector_under_prescription_o70-sync_bulk_incomplete",
   "owner": "O.5"
  },
  {
   "item": "Demonstrate identity remainder cancellation / CT solution",
   "status": "Unresolved_ST4a_ST4b_split",
   "owner": "O.6/O.7"
  },
  {
   "item": "Enlarge target if identity requires more legs/orders",
   "status": "rule_active_not_triggered",
   "owner": "O.1"
  }
 ],
 "enlarge_rules": [
  "If the stated identity constrains n-point or higher-loop pieces at the same coupling order, enlarge the ta
 +> rget before Constructed.",
  "If classical kappa^2 consistency fails, downgrade or repair O.2 before quantum contractions.",
  "Never close the sector by silently identifying XVI-M selected graph with an action graph via ell_3."
 ]
},
"O1_requirement": "O.1 closes iff the one-loop kappa^2 bulk two-point sector on S_inv closes under a VERIFIED
 +>  retained quantum identity: every checklist item (O.2 classical kappa^2 master Constructed incl. grav ell
 +> _4; O.4 inventory; O.5 all graphs evaluated; O.6 identity verified; O.7 remainder cancellation / CT solut
 +> ion) earned, and the target enlarged first if the identity constrains more legs/orders at kappa^2.",
"O1_items_scope": {
 "State retained quantum identity (O.6)": {
  "scope": "PARTLY in XVI-P",
  "evidence": "projected ch identity at one loop kappa^2 stated and verified (P19/P20), incl. evanescent CTs
 +> (P22); the BULK retained identity (ST4b functional) is XVI-O's statement, not changed"
 },
 "List amplitudes constrained by identity (O.4)": {
  "scope": "IN XVI-P (this record)",
  "evidence": "one-loop kappa^2: exactly the E=2 two-point and E=1 one-point graphs; enumeration reproduces t
 +> he frozen inventory; completeness at topology level only"
 },
 "S_inv^{[2]} classical consistency (O.2)": {
  "scope": "OUT OF SCOPE",
  "evidence": "classical kappa^2 master 2(S0,S2)+(S1,S1): grav ell_4 Incomplete in XVI-O/XVI-N; XVI-A..O unto
 +> uched -> O.1 cannot close inside XVI-P"
 },
 "Evaluate inventory graphs (O.5)": {
  "scope": "IN XVI-P",
```

```
  "evidence": "all E=2 kappa^2 graphs evaluated (P15..P18, D12 Missing never zero per P14 -> see open items)"
 },
 "Demonstrate identity remainder cancellation / CT solution (O.6/O.7)": {
  "scope": "PARTLY in XVI-P",
  "evidence": "projected: closure + restoration earned (P20, P22); bulk ST4a/ST4b split: XVI-O"
 },
 "Enlarge target if identity requires more legs/orders (O.1)": {
  "scope": "IN XVI-P (this record)",
  "evidence": "NOT triggered at kappa^2 at one loop (three-point one-loop sector empty at kappa^2); next orde
 +> r kappa^3 three-point = P.6 entry"
 }
},
"O1_closable_in_XVI_P": false,
"O1_closure_status": "NOT_CLOSED",
"P6_entry_n_graphs": 15,
"one_point_n_graphs": 3,
"O1_closable_reason": "O.2 (classical kappa^2 master, grav ell_4 Incomplete) and the bulk ST4b identity are o
 +> wned by XVI-O/XVI-N, which stay untouched",
"vertex_kappa_power_equals_legs_minus_2": {
 "V2_grav": true,
 "V3_grav": true,
 "V4_grav": true,
 "MO_ghost": true,
 "V_cb_c_h": true,
 "V_K_c": true,
 "V_K_c_h": true,
 "V_cs_c_c": true,
 "V_phis_c_phi": true,
 "V_b_h": true,
 "V_b_b": true,
 "V_cbs_b": true,
 "M_phi": true,
 "V_h_phi_phi": true,
 "V_hh_phi_phi": true
},
"vertex_rule_holds": true,
"one_loop_kappa_equals_E": true,
"identity_at_kappa2": {
 "terms": [
  "(S^(0), Gamma_1^(2))  [E=2]",
  "(S^(1), Gamma_1^(1))  [E=1]",
  "(S^(2), Gamma_1^(0)) = 0  [field independent]"
 ],
 "orders_entering": [
  "E=2 two-point (hh, phiphi, c.cb, K.c)",
  "E=1 one-point"
 ],
 "three_point_one_loop_at_kappa2": "EMPTY (kappa^E with E=3 is kappa^3)",
 "enlarge_rule_triggered": false
},
"graphs_E1": [
 {
  "external": "h",
  "vertices": [
   "V3_grav"
  ]
 },
 {
  "external": "h",
  "vertices": [
   "V_cb_c_h"
  ]
 },
 {
  "external": "h",
  "vertices": [
   "V_h_phi_phi"
  ]
 }
],
"graphs_E2": [
 {
  "vertices": [
   "V3_grav",
   "V3_grav"
  ],
  "loop_lines": [
   "h-h",
   "h-h"
  ],
  "external": "h,h",
```

```
 "kappa_power": 2,
 "V": 2,
 "I": 2,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V4_grav"
 ],
 "loop_lines": [
  "h-h"
 ],
 "external": "h,h",
 "kappa_power": 2,
 "V": 1,
 "I": 1,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_K_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "h-h",
  "cb-c"
 ],
 "external": "K,c",
 "kappa_power": 2,
 "V": 2,
 "I": 2,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "cb-c",
  "cb-c"
 ],
 "external": "h,h",
 "kappa_power": 2,
 "V": 2,
 "I": 2,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "h-h",
  "cb-c"
 ],
 "external": "c,cb",
 "kappa_power": 2,
 "V": 2,
 "I": 2,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "loop_lines": [
  "phi-phi",
  "h-h"
 ],
 "external": "phi,phi",
 "kappa_power": 2,
 "V": 2,
 "I": 2,
```

```
  "L": 1,
  "kappa_equals_E": true
 },
 {
  "vertices": [
   "V_h_phi_phi",
   "V_h_phi_phi"
  ],
  "loop_lines": [
   "phi-phi",
   "phi-phi"
  ],
  "external": "h,h",
  "kappa_power": 2,
  "V": 2,
  "I": 2,
  "L": 1,
  "kappa_equals_E": true
 },
 {
  "vertices": [
   "V_hh_phi_phi"
  ],
  "loop_lines": [
   "h-h"
  ],
  "external": "phi,phi",
  "kappa_power": 2,
  "V": 1,
  "I": 1,
  "L": 1,
  "kappa_equals_E": true
 },
 {
  "vertices": [
   "V_hh_phi_phi"
  ],
  "loop_lines": [
   "phi-phi"
  ],
  "external": "h,h",
  "kappa_power": 2,
  "V": 1,
  "I": 1,
  "L": 1,
  "kappa_equals_E": true
 }
],
"frozen_E2_match": {
 "enumerated": 9,
 "frozen": 9,
 "missing_from_enumeration": [],
 "not_in_frozen_inventory": [],
 "equal": true
},
"P6_entry_kappa3_three_point": {
 "n_graphs": 15,
 "by_external": {
  "K,c,h": 2,
  "c,c,cs": 1,
  "c,cb,h": 2,
  "c,phi,phis": 1,
  "h,h,h": 5,
  "h,phi,phi": 4
 },
 "graphs": [
  {
   "vertices": [
    "V3_grav",
    "V3_grav",
    "V3_grav"
   ],
   "loop_lines": [
    "h-h",
    "h-h",
    "h-h"
   ],
   "external": "h,h,h",
   "kappa_power": 3,
   "V": 3,
   "I": 3,
```

```
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V3_grav",
  "V4_grav"
 ],
 "loop_lines": [
  "h-h",
  "h-h"
 ],
 "external": "h,h,h",
 "kappa_power": 3,
 "V": 2,
 "I": 2,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V3_grav",
  "V_K_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "h-h",
  "c-cb",
  "h-h"
 ],
 "external": "K,c,h",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V3_grav",
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "h-h",
  "c-cb",
  "h-h"
 ],
 "external": "c,cb,h",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V3_grav",
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "loop_lines": [
  "h-h",
  "phi-phi",
  "h-h"
 ],
 "external": "h,phi,phi",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V3_grav",
  "V_hh_phi_phi"
 ],
 "loop_lines": [
  "h-h",
  "h-h"
```

```
 ],
 "external": "h,phi,phi",
 "kappa_power": 3,
 "V": 2,
 "I": 2,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_K_c_h",
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "h-h",
  "cb-c",
  "cb-c"
 ],
 "external": "K,c,h",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "cb-c",
  "cb-c",
  "cb-c"
 ],
 "external": "h,h,h",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "loop_lines": [
  "cb-c",
  "h-h",
  "cb-c"
 ],
 "external": "c,cb,h",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h",
  "V_cs_c_c"
 ],
 "loop_lines": [
  "h-h",
  "cb-c",
  "c-cb"
 ],
 "external": "c,c,cs",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
```

```
 "vertices": [
  "V_cb_c_h",
  "V_h_phi_phi",
  "V_phis_c_phi"
 ],
 "loop_lines": [
  "h-h",
  "phi-phi",
  "c-cb"
 ],
 "external": "c,phi,phis",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_h_phi_phi",
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "loop_lines": [
  "phi-phi",
  "phi-phi",
  "h-h"
 ],
 "external": "h,phi,phi",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_h_phi_phi",
  "V_hh_phi_phi"
 ],
 "loop_lines": [
  "phi-phi",
  "h-h"
 ],
 "external": "h,phi,phi",
 "kappa_power": 3,
 "V": 2,
 "I": 2,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_h_phi_phi",
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "loop_lines": [
  "phi-phi",
  "phi-phi",
  "phi-phi"
 ],
 "external": "h,h,h",
 "kappa_power": 3,
 "V": 3,
 "I": 3,
 "L": 1,
 "kappa_equals_E": true
},
{
 "vertices": [
  "V_h_phi_phi",
  "V_hh_phi_phi"
 ],
 "loop_lines": [
  "phi-phi",
  "phi-phi"
 ],
 "external": "h,h,h",
 "kappa_power": 3,
 "V": 2,
```

```
    "I": 2,
    "L": 1,
    "kappa_equals_E": true
   }
  ],
  "status": "inventory only (topology level). NOT evaluated: amplitude rules, counterterm domain and evaluatio
  +> n are the next release"
 },
 "mutations": {
  "V3_grav_kappa_power_+1": true,
  "drop_V_cb_c_h": true,
  "drop_V_hh_phi_phi": true,
  "spurious_c_h_propagator": true
 },
 "mutations_detected": true,
 "seconds": 9.33,
 "pass": true
}
```


38 p22 layer record (release gate p22, revised in p23)

[listing 05/133] certificates/P22_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p22",
  "contract_revision": "p22 (p14r contract + disclosed p15/p16/p17/p18 revisions + the DISCLOSED p22 contract
  +> revision: evanescent counterterms (EV-CT) and the runtime d guard in code/p5_entry/dred.py)",
  "date": "2026-10-10",
  "version_label": "XVI-P p22",
  "deliverable": "p22: evanescent M2 statement, evanescent counterterms (p22 contract revision), runtime d gua
  +> rd (p22 contract revision), gate without prefix exemptions",
  "execution_pass": true,
  "claim_status": "p22. EV-M2: the 8 evanescent D11 structures have local UV poles and their renormalised rema
  +> inder vanishes identically on QdS external legs (exact in d); they are NOT matched to XVI-O (no counterpa
  +> rt; both natural d-dim continuations refuted). EV-CT: the declared evanescent counterterms (h.h gt-operat
  +> ors, K'.c E2, cb.c SE; coefficient = -pole) are sigma_O-closed together with the 4D MS-bar counterterm an
  +> d leave the projected identity unchanged; P.5 calculation status: renormalized (one loop, kappa^2, two-po
  +> int functions hh / K'.c / cb.c; MS-bar in DRED with the p22 evanescent counterterms; external legs in QdS
  +> ). Runtime d guard at the sympy entry points (contract revision); static scan secondary. Projected (ch-on
  +> ly) results as in p20/p21; bulk symmetry NOT restored/claimed; O.1 NOT_CLOSED.",
  "flag_definitions": {
    "projected_defect_computed": "Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from every evaluat
  +> ed input of the three surviving rows (none Missing), exact in d, pole + nonlocal + MS-bar local parts, al
  +> l seven external assignments",
    "projected_identity_verified": "Delta = 0 for the REGULATED one-loop functional Gamma1: pole, nonlocal and
  +>  local parts vanish for all seven assignments, exactly in d, with all controls true and every mutation ch
  +> anging the outcome",
    "projected_restoration_verified": "p20 definition (replaces p19's, which follows from the identity + local
  +> ity): FULL sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm S_ct = -(pole) in the decl
  +> ared domain, h.h + c.K + c.cb (ghost self-energy Sigma held fixed at its P15 value, orientation converted
  +> , derived relative factor +1), in every ghost +1 sector (h.c, c.b, c.cbs), together with projected_identi
  +> ty_verified.  Scope: the one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1",
    "bulk_symmetry_restored": "hard stop: False regardless (not claimed; O.1 NOT_CLOSED)"
  },
  "projected_restoration_scope": "p20 definition: full sigma_0 closure (h.c, c.b, c.cbs) of the complete one-l
  +> oop kappa^2 MS-bar counterterm incl. the c.cb ghost self-energy counterterm (held fixed, derived factor +
  +> 1); NOT bulk symmetry, NOT O.1",
  "route_consistency_control_p19_criterion": {
    "label": "route-consistency control (the p19 restoration criterion, NOT a flag): P_ch sigma_O(S_ct) = 0 fo
  +> r the h.h + c.K MS-bar counterterm and the solve x = 0; implied by the identity + locality",
    "holds": true
  },
  "full_closure_mutation_detector": {
    "test": "full sigma_0 closure (all ghost +1 sectors) of the complete MS-bar counterterm, Sigma held fixed"
  +> ,
    "cases": {
      "drop_graph_D11": {
        "closed": true,
        "entries": 0,
        "sectors": [],
        "detected": false
      },
      "drop_graph_D13 (c.K data = 0, Sigma held fixed)": {
        "closed": false,
        "entries": 88,
        "sectors": ["c.b"],
        "detected": true
```

```
    },
    "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "drop_graph_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "drop_graph_D01": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "flip_phase_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "symmetry_factor_D03_x2": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "Sigma_orientation_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    }
  },
  "reading": "detects drop D13 and flip K.c (the p19 blind spots) through the c.b sector, and the hh-row mut
+> ations through h.c; drop D11 stays undetected (D11 transverse => its pole is sigma_0-closed by itself): c
+> overed by M2 (P18) and the p20 D11 normalisation pin (P20_D11_pin.json)"
},
"p20_controls": {
  "orientation_ratio_uniform_minus_one": true,
  "tree_ghost_equation_V_cbc_eq_2iD_V_Kc": true,
  "tree_sigma0_SOKc_plus_SOcbc_zero": true,
  "derived_ghost_relation_reproduces_P15_sigma": true,
  "full_closure_detects_drop_D13_and_flip_Kc": true,
  "full_closure_detects_p19_orientation_error": true,
  "S_ct_without_ccb_not_closed (c.cb counterterm is needed)": true
},
"why_zero": "each surviving ch row vanishes separately in the projected slot; no counterterm cancellation an
+> d no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupl
+> ing -- D01 and D03 are individually nonzero and cancel against each other in the slot (p17/p18 transverse
+>  sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only throu
+> gh T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-poin
+> t row: the tadpoles D14a-c vanish under the declared prescription.  The defect is therefore zero without
+> any counterterm (solve x = 0).",
"rests_on": ["M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D compon
+> ents'; evanescent components of the D11 matching: not applicable, not verified)", "the evanescent decoupl
+> ing proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row decoupling)", "the frozen P.3 projection
+> and the P13 ch dependency ledger (three surviving rows)", "the declared prescription (DRED R1-R6, MS-bar,
+>  P14_prescription_D11_matching)"],
"findings": {
  "J1_p14r_counterterm_domain_field": "FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the p14r cou
+> nterterm-domain counts (Im sigma_0 3, quotient 8, D cap ker 5, P_ch sigma_0 rank 4) are artefacts of a re
+> /im split over Q (the densities carry explicit i for odd derivative counts, so i-multiples count as indep
+> endent).  The counts over Q(i) -- sigma_0 rank 5, D cap ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch
+>  sigma_0 rank 3 (= Route C) -- equal the real dimensions under a compatible real structure.  The p14r cer
+> tificate stays as released; both are reported",
  "K1_ghost_closure_sign": "p20 CORRECTION (erratum to p19): resolved -- an orientation bookkeeping error. W
+> ith the P15 Sigma coordinates converted to the route-G (c,cb) orientation (ratio -1), the derived factor
+> +1 gives full sigma_0 closure of the complete MS-bar counterterm in h.c, c.b, c.cbs.  The p19 text 'closu
+> re fails / needs -1 / unresolved' is retracted"
},
"K1_ghost_closure_convention": {
  "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
  "M_2iD_cK_to_cbc": [
    ["0", "2"],
    ["-1", "0"],
    ["0", "-1"]
```

```
  ],
  "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
  "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
  "derived_equals_P15": true,
  "relative_factor_derived": "1",
  "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
  "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
  "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
  "tree_controls": {
    "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
    "S0_Kc_terms": 16,
    "S0_cbc_terms": 40,
    "sigma0_S0Kc_plus_S0cbc_entries": 0,
    "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
    "ok": true
  },
  "full_closure_complete_S_ct": {
    "closed": true,
    "entries": 0,
    "sectors": []
  },
  "full_closure_S_ct_without_ccb (p19 package S_ct)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"]
  },
  "full_closure_unconverted (p19 error)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"],
    "detected": true
  },
  "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolved
+> ' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots).
+>  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
},
"scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the hh an
+> d K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement abou
+> t higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests ful
+> l sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c.b,
+> c.cbs) -- a statement about that counterterm only, not bulk symmetry, not O.1",
"release_claim_accepted": true,
"release_claim_accepted_definition": "contract = registered p22 revision only (pinned) AND lower layers re-c
+> hecked live AND p20/p21 layers AND absolute pin AND EV-M2 + EV-CT certificate AND live runtime guard prob
+> e AND every added/modified/removed file disclosed (no prefix exemption) AND the pinned p21 independent re
+> cord AND the p22 hard stops",
"publication": "accepted",
"publication_gated_on": ["execution_pass", "quantum_inputs_evaluated (p18 layer + recheck)", "projected_defe
+> ct_computed", "flag implications restoration => identity => computed"],
"flags": {
  "library_certified": true,
  "batchA_evaluated": {
    "execution": true,
    "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
+> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
+> V pole, finite part 0, no p14r counterterm required",
    "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
+>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
+> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
+> e; NOT quantum_inputs_evaluated"
  },
  "D13_evaluated": {
    "execution": true,
    "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
+> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
+> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
+> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
    "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
+> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
+> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
+> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
  },
  "batchC_evaluated": {
    "execution": true,
    "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
+> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
+> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
+> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
    "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
+>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
+> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
```

```
  },
  "E2_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof)",
    "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
+>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
  },
  "hh_evanescent_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
    "scope": "the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of D01
+>  + D03 (D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green function
+>  outside this projection; D11 (same row) NOT evaluated"
  },
  "D11_matching_M2": {
    "execution": true,
    "claim": "DISCHARGED (4D components)",
    "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the
+>  4D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0
+> is IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns
+>  the classical insertion and is not touched"
  },
  "hh_slot_full_row_decoupled": {
    "execution": true,
    "claim": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
    "scope": "row (S0,Gamma1^[2]) s2 K:K.c; D01 + D02 + D03 + D11 + D12; the summed evanescent poles remain
+> (transverse) UV poles of the hh Green function outside this projection"
  },
  "projected_ch_identity": {
    "execution": true,
    "claim": "ZERO",
    "scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the h
+> h and K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement
+> about higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests
+>  full sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c
+> .b, c.cbs) -- a statement about that counterterm only, not bulk symmetry, not 0.1"
  }
},
"projected_defect_computed": "earned",
"projected_identity_verified": "earned",
"projected_restoration_verified": "earned",
"projected_defect_summary": {
  "defect_zero_exact_in_d": true,
  "evanescent_in_assembly": "verified explicitly: the evanescent (gt) parts of the hh row (D01+D03+D11) and
+> of the K.c row (D13 E1, E2) give ZERO for all seven external assignments, exactly in d; the evanescent po
+> les themselves are nonzero (P18_hh_slot, P15_D13) -- they drop from the ch slots by the decoupling proofs
+> , not by convention",
  "quotient_coordinates": {
    "complement_directions": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"],
    "pole": ["0", "0", "0", "-3/8", "1/4"],
    "nonlocal_L": ["0", "0", "0", "3/8", "-1/4"],
    "local": ["0", "0", "0", "-2/3", "5/12"],
    "reading": "coordinates modulo D cap Im sigma_0 (field redefinitions / s-exact); the h.h pole is transve
+> rse, i.e. in D cap Im sigma_0, so its quotient coordinates vanish"
  },
  "D_cap_ker_sigma0": {
    "computed": true,
    "dim": 6,
    "dim_D_cap_Im_sigma0": 6,
    "cohomology_dim": 0,
    "sector_ranks": {
      "h.h": {
        "dim": 5,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 3
      },
      "c.cb": {
        "dim": 2,
        "sigma0_rank": 2,
        "P_ch_sigma0_rank": 0
      },
      "c.K": {
        "dim": 3,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 1
      },
      "phi.phi": {
        "dim": 1,
        "sigma0_rank": 0,
        "P_ch_sigma0_rank": 0
```

```
      }
    }
  },
  "field_note": "Route G is computed over Q(i) (primary) and, for comparison, over Q with the re/im split us
+> ed by the p14r domain certificate.  The p14r densities carry explicit factors i for odd derivative counts
+>  (c.K: 3 derivatives), so over Q an i-multiple counts as independent: e.g. the ch image of c.K#0 is i tim
+> es an h.h image.  The counts over Q(i) equal the real dimensions under a compatible real structure (one r
+> eal coefficient per density, the i fixed by reality of the action); ranks and solvability are quoted over
+>  Q(i)."
},
"counterterm_solve": {
  "system": "Delta + P_ch sigma_0(x) = 0, x in the declared domain D (11 directions), part by part",
  "rank_P_ch_sigma0_routeG": 3,
  "rank_routeC": 3,
  "routeC_image_relations": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "outcome": "ZERO",
  "residual": null,
  "solution": "x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite renorma
+> lisations are ker(P_ch sigma_0) cap D (dim 8), of which 2 modulo D cap Im sigma_0",
  "MSbar_counterterm_ch_image_zero": true,
  "MSbar_counterterm_full_sigma0_image_sectors": ["c.b"],
  "full_closure_relative_factor_table_UNCONVERTED_p19_error": {
    "1": false,
    "-1": true,
    "I": false,
    "-I": false
  },
  "full_closure_reading": "p20 (corrects p19): with the P15 Sigma coordinates converted from the (cb,c) to t
+> he route-G (c,cb) orientation (measured ratio -1), the DERIVED factor +1 gives full sigma_0 closure of th
+> e complete MS-bar counterterm (h.h + c.K + c.cb) in every ghost +1 sector; the p19 table above (factors 1
+> , -1, i, -i on UNCONVERTED coordinates) is the bookkeeping error, kept for the erratum",
  "K1_ghost_closure_convention": {
    "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
    "M_2iD_cK_to_cbc": [
      ["0", "2"],
      ["-1", "0"],
      ["0", "-1"]
    ],
    "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
    "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
    "derived_equals_P15": true,
    "relative_factor_derived": "1",
    "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
    "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
    "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
    "tree_controls": {
      "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
      "S0_Kc_terms": 16,
      "S0_cbc_terms": 40,
      "sigma0_S0Kc_plus_S0cbc_entries": 0,
      "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
      "ok": true
    },
    "full_closure_complete_S_ct": {
      "closed": true,
      "entries": 0,
      "sectors": []
    },
    "full_closure_S_ct_without_ccb (p19 package S_ct)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"]
    },
    "full_closure_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolv
+> ed' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots)
+> .  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
  },
  "note_full_sigma0": "the h.h + c.K MS-bar counterterm ALONE has a nonzero (c, b) image (delta S0/delta h c
+> arries b); adding the c.cb ghost self-energy counterterm (correct orientation, derived factor +1) closes
```

```
+> it: see K1_ghost_closure_convention.full_closure_complete_S_ct"
},
"defect_controls": {
  "inventory_complete_no_missing": true,
  "missing_input_refused_not_zero": true,
  "s0h_longitudinal": true,
  "E_transverse": true,
  "hh_row_zero_exact_in_d_all_7": true,
  "hh_row_evanescent_part_zero_all_7": true,
  "hh_row_D01_D03_individually_nonzero": true,
  "Kc_row_zero_exact_in_d_all_7": true,
  "Kc_row_evanescent_part_zero_all_7": true,
  "Kc_slot_map_nontrivial_T1_E1": true,
  "E2_alone_zero_all_7": true,
  "D13_A_T1_A_E1_zero_exact": true,
  "onepoint_row_D14abc_vanishing": true,
  "defect_zero_pole_nonlocal_local_all_7": true,
  "defect_independent_of_row_signs": true,
  "routeG_S0_check": true,
  "routeG_defect_zero_by_sector": true,
  "routeG_hh_local_data_in_span": true,
  "routes_agree_on_ch_image_rank": true,
  "routeG_hh_kernel_equals_p18_transverse_subspace": true,
  "D_cap_Im_inside_D_cap_ker_sigma0": true,
  "D_cap_Im_inside_ker_P_ch_sigma0": true,
  "ghost_equation_relation_reproduces_P15_sigma_pole": true,
  "MSbar_counterterm_in_ker_P_ch_sigma0": true,
  "mutations_all_change_outcome": true,
  "routeG_Q_split_reproduces_p14r_counts": true
},
"defect_mutations": {
  "drop_graph_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/48",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the ghost loop D03"
  },
  "drop_graph_D01": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the graviton bubble D01"
  },
  "flip_phase_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/24",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/24",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/8"
```

```
      }
    },
    "note": "relative sign of D03 flipped"
  },
  "symmetry_factor_D03_x2": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "D03 doubled"
  },
  "wrong_projection_trace_operator_hh": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-3*c^.p*p.p*(6*a.b*p.p - 6*a^.p*b^.p - a~.b~*p.p)/16",
        "QdS (hat)": "-9*c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/8",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-15*a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "eta_mn p_v in place of s0 h (not the P.3 projection)"
  },
  "wrong_projection_gauge_fixed_operator_Kc": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p*(a.b*c^.p*p.p - 3*a.c*b^.p*p.p - 3*a^.p*b.c*p.p + 4*a^.p*b^.p*c^.p - a^.p
+> *b~.c~*p.p - a~.c~*b^.p*p.p)/16",
        "QdS (hat)": "p.p*(a^.b^*c^.p*p.p - 3*a^.c^*b^.p*p.p - 3*a^.p*b^.c^*p.p + 4*a^.p*b^.p*c^.p)/16",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/4",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "de Donder q^2 P in place of the h^2-monomial operator E (not the P.3 projection)"
  },
  "naive_4D_D03_evanescent_dropped": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/24"
      }
    },
    "note": "D03 Gt parts dropped"
  },
  "naive_4D_D13_Gt_dropped": {
    "outcome": "NONZERO, CANCELLABLE by a counterterm in the declared domain",
    "changes_outcome": true,
    "residual": {
      "local": {
        "Q4S (declared, R2)": "-c^.p*p.p*(a.b*p.p - a^.p*b^.p)/12",
        "QdS (hat)": "-c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/12",
```

```
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/12"
      }
    },
    "note": "D13 evaluated with Gt dropped (P15_D13 mutation fit: T1 = -(d-4)/(16(d-1)))",
    "finite_counterterm": {
      "rank": 3,
      "solvable": true,
      "solution": ["(6*x3 + 12*x4 - 24*I*x5 - I)/12", "-x3/2", "-(12*x3 + 24*x4 - 24*I*x5 - I)/12", "x3", "x
+> 4", "x5", "x6", "x7"]
    }
  }
},
"defect_insensitive_variations": {
  "drop_graph_D11": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "D11 is transverse by itself (p_m G_D11 = 0 exactly in d): the slot cannot see it; covered
+> by M2 (P18) and by the inventory refusal below"
  },
  "drop_graph_D13": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "A_T1 = A_E1 = 0 and T2, T3, E2 are longitudinal: D13 reaches the slot with zero; covered b
+> y the D13 certificate (two routes, mutations)"
  },
  "flip_row_sign_Kc": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "each row vanishes separately"
  }
},
"quantum_inputs_evaluated": "earned",
"quantum_inputs_inventory": [
  {
    "graph": "D01",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D02",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D03",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D11",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"
  },
  {
    "graph": "D12",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D13",
    "slot_row": "(S0,Gamma1^[2]) s1 h:h^2",
    "evaluated": true,
    "status": "evaluated"
  },
  {
    "graph": "D14a",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14b",
```

```
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14c",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  }
],
"D11_matching_verified": true,
"D11_matching_verified_scope": "4D components: M2 discharged (P18_D11_M2, unchanged); evanescent components:
+>  EV-M2 proven (local poles + exact 4D decoupling of the renormalised remainder); NOT matched to XVI-O (no
+>  counterpart; both natural d-dim continuations refuted)",
"D11_evanescent_components_matching": "not_applicable_not_verified",
"M3_status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
"D11_evaluated": true,
"hh_full_row_decoupling_proven": true,
"evanescent_hh_counterterm_added": true,
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"hh_evanescent_status": "decoupled_from_ch_slot_proven_as_sum",
"hh_evanescent_decoupling_proven": true,
"D01_evaluated": true,
"D03_evaluated": true,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
```

```
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
```

```
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D11": {
    "status": "evaluated",
    "certificate": "certificates/P18_D11.json",
    "sector_status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
    "M2": "DISCHARGED (4D components)",
    "ch_slot": "decoupled by itself (p_m G_D11 = 0 exactly in d)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_byte_identical_to_p18": true,
  "dred_registry_state_p22": "p18 + p22 guard block",
  "contract_revision_in_p19": false,
  "disclosed_revisions_p19": {}
},
"p18_layer": {
  "p18_layer_files_byte_identical": true,
  "P18_release_accepted": true,
  "P18_quantum_inputs_evaluated": "earned",
  "I1_gate": "the p19 gate requires the p18 layer record's quantum_inputs_evaluated == 'earned'"
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
    "C7_grav_noether_k0_k1_k2_on_exports": true,
    "quartic_mutations_all_rejected": true,
    "master_grav_noether_deterministic_k0_k1_k2": true,
    "master_engine_parts_exact": true,
    "graviton_hessian_generic_symbolic_k": true,
    "canonicity_all_23_antifield_components": true,
    "ch_enumeration_lemma_mechanical": true,
```

```
  "residual_record_invariants_all_certificates": true,
  "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
  "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
  "cross_certificate_quartic_vs_master_agree": true,
  "stored_C7_flag_equals_recomputation": true
},
"failed_prerequisites": [],
"hash_consistency": {
  "consistent": true,
  "ell": {
    "ell2": {
      "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
    },
    "ell3": {
      "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
    },
    "ell4": {
      "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
    "quartic_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    },
    "master_parameters": {
      "a_ghost": "1/2",
```

```
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    }
  },
  "hard_stops_p13_hold": true,
  "hard_stops_p13": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"hard_stops": {
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-bar in DRE
+> D with the p22 evanescent counterterms; external legs in QdS)",
    "P.5_identity": "projected defect computed (zero exactly in d, all parts, all seven assignments; each ro
+> w vanishes separately); projected identity verified; projected restoration verified (p20 definition: full
+>  sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm incl. c.cb, h.c/c.b/c.cbs); K1 corre
+> cted in p20 (orientation erratum); D11 ratio pin verified; D11 absolute pin verified (p21); bulk symmetry
+>  NOT restored/claimed; O.1 NOT_CLOSED [p21: D11 absolute pin derived and verified] [p22: evanescent count
+> erterms closed; identity unchanged]"
  },
  "definition_vs_status_note": "I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficients evalua
+> ted and renormalized'; the STATUS states exactly what holds: 4D sector renormalized, evanescent poles reg
+> ulated and decoupled from the ch slots (not subtracted)",
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)"
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p18": true,
"independent_rebuild_passed_p17": true,
"independent_rebuild_passed_p16": true,
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p22) sourc
```

```
+> e tree only (independent: pending until a reviewer report for this tree exists); independent_rebuild_pass
+> ed_p21 refers to the p21 tree b835353ffe16a8ff159e2916fe3efdf2b9d89a4d512e80e9e6da587cecd0fe5e / zip a71d
+> 6b5203bc3914833db9fd13b9d186bef93d1ae3007b2e4c99fdfce11e0678 (same-box caveat) and is not inherited by p2
+> 2",
"p18_independent_rebuild": {
  "report": "archival/p18/reproducibility/independent_rebuild_report_p18.json",
  "present": true,
  "report_sha256": "893f167ef7124d981c0569ca386035aed06e0dbb0c90a2362a6ceeafd1930cb2",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094",
  "bound_to_source_zip_sha256": "7c7d5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f",
  "tree_matches_p18": true,
  "zip_matches_p18": true,
  "mismatches": 0,
  "findings": ["I1_release_p18_publication_not_gated_on_quantum_inputs_evaluated", "I2_scan_pattern_gaps", "
+> I3_scope_of_M2_and_evaluated", "I4_table_row_count", "I5_slot_blind_to_D11_normalisation", "I6_test_p16_v
+> ictim_restriction", "I7_pdftotext_dehyphenation", "H1_H5_resolution", "bug_fixes_disclosed"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p19": "bound to the p18 tree/zip; the p19 tree has its own (pending) independent flag",
  "archived_p18_records_pinned": {
    "archival/p18/reference_values_p18.json": true,
    "archival/p18/reproducibility/author_rebuild_report_p18.json": true,
    "archival/p18/reproducibility/independent_rebuild_report_p18.json": true,
    "archival/p18/p18_source_files.json": true
  },
  "pins_ok": true
},
"p17_independent_rebuild": {
  "report": "archival/p17/reproducibility/independent_rebuild_report_p17.json",
  "present": true,
  "report_sha256": "92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
  "bound_to_source_zip_sha256": "7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416",
  "tree_matches_p17": true,
  "zip_matches_p17": true,
  "mismatches": 0,
  "findings": ["H1_static_scan_pattern_gaps", "H2_assignment_table_omits_c_hat_h_tilde", "H3_stale_comments"
+> , "H4_curly_apostrophes_in_typeset_listings", "H5_parse_d_srepr", "H6_scope", "G1_G2_G3_G4_resolution"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p18": "bound to the p17 tree/zip; the p18 tree has its own (pending) independent flag",
  "archived_p17_records_pinned": {
    "archival/p17/reference_values_p17.json": true,
    "archival/p17/reproducibility/author_rebuild_report_p17.json": true,
    "archival/p17/reproducibility/independent_rebuild_report_p17.json": true,
    "archival/p17/p17_source_files.json": true
  },
  "pins_ok": true
},
"p16_independent_rebuild": {
  "report": "archival/p16/reproducibility/independent_rebuild_report_p16.json",
  "present": true,
  "report_sha256": "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
  "bound_to_source_zip_sha256": "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57",
  "tree_matches_p16": true,
  "zip_matches_p16": true,
  "mismatches": 0,
  "findings": ["G1_static_scan_scope_narrower_than_disclosure", "G2_plain_symbol_d_from_stored_strings", "G3
+> _hook_coverage_detail", "G4_change_log_omits_new_reference_file", "G5_test_p15_assertion_relaxed", "G6_im
+> aginary_normalisation", "G7_scope", "F3_F4_F5_from_p15"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent flag",
  "archived_p16_records_pinned": {
    "archival/p16/reference_values_p16.json": true,
```

```
    "archival/p16/reproducibility/author_rebuild_report_p16.json": true,
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": true,
    "archival/p16/p16_source_files.json": true
  },
  "pins_ok": true
},
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"M2_summary": {
  "verdict": "DISCHARGED (4D components)",
  "D11_matching_verified_scope": "4D components",
  "evanescent_components_matching": {
    "status": "not_applicable_not_verified",
    "what_they_are": "the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt): compo
+> nents of the hh two-point function with at least one index pair in the epsilon-scalar (4-d) directions of
+>  DRED; they vanish on hatted 4D components",
```

```
    "why_not_matched": "XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent counte
+> rpart; M2 is a 4D statement, so there is nothing to match them to. Their L-coefficients are nonzero (reco
+> rded below); they are retained (R6) and drop out of the ch-projected slot (p_m G_D11 = 0 for the evanesce
+> nt part separately)"
  },
  "M3_status": {
    "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
    "statement": "L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree <= 8 i
+> t is a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)",
    "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching)"
  },
  "statement": "M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR_AB],
+>  f> = <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18",
  "route_B": {
    "equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair",
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 0,
    "first": [],
    "agree": true,
    "constant": "-I/4"
  },
  "FT_constant": {
    "master_pole": "1",
    "master_finite": "2 - L",
    "pole_is_1": true,
    "L_coefficient_of_MB": "-1",
    "c_FT_from_DR_bubble": "I*pi**2",
    "c_FT_from_minus_r": "I*pi**2",
    "r_certificate": "-i * pi**2",
    "r_certificate_as_expected": true,
    "agree": true
  },
  "controls": {
    "route_B_all_100_pairs_agree": true,
    "FT_constant_two_calibrations_agree": true,
    "route_A_M1_integrand_identity": true,
    "route_A_XVI_O_K_u_equals_T": true,
    "D11_transverse_exact_in_d": true,
    "routes_FP_PV_agree": true,
    "covariant_fit_exact_unique": true,
    "mutations_all_rejected": true
  },
  "analytic_inputs_cited": ["Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, resi
+> due at zeta = -2 is r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bub
+> ble)", "a polynomial in p is the Fourier transform of a distribution supported at {0}", "Route A only: tw
+> o extensions of the product V V (C C + C C) from U differ by a distribution supported at 0 (XVI-O lem:BV_
+> rules_on_U, Constructed)"],
  "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the 4
+> D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0 is
+>  IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns t
+> he classical insertion and is not touched",
  "M3_consequence": "with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both sides a
+> re transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contacts, consistent with
+> N4), so L_match lies in the 2-dim transverse subspace of the p14r h.h span; its coordinates need the XVI-
+> O finite constant of u^R and are NOT computed (M4: booked as P-side hh counterterm; u^R_mu unmodified, no
+>  c delta)"
},
"D11_summary": {
  "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
  "prescription": "operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions, M3 bounds
+>  4/8/4, exactly-degree-four normalization (massless: every local term is a homogeneous quartic in p)",
  "prescription_usage": {
    "exactly_degree_four_normalization": "every structure S_s (p.p)^e of Ghat has total momentum degree 4 (c
+> omputed per structure: g4_mn g4_rs (p.p)^2: 4, g4_mn gt_rs (p.p)^2: 4, g4_mn p_r p_s (p.p)^1: 4, g4_mr g4
+> _ns (p.p)^2: 4, g4_mr gt_ns (p.p)^2: 4, g4_mr p_n p_s (p.p)^1: 4, g4_rs gt_mn (p.p)^2: 4, g4_rs p_m p_n (
+> p.p)^1: 4, gt_mn gt_rs (p.p)^2: 4, gt_mn p_r p_s (p.p)^1: 4, gt_mr gt_ns (p.p)^2: 4, gt_mr p_n p_s (p.p)^
+> 1: 4, gt_rs p_m p_n (p.p)^1: 4, p_m p_n p_r p_s: 4)",
    "N4_contact_exclusions": "used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by N4 do not
+> arise (massless; every pole and local MS-bar term is a homogeneous quartic in p), so N4 removes nothing h
+> ere and no order-0/2 counterterm is needed",
    "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"
  },
  "UV_pole_in_p14r_hh_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
  "evanescent_pole_nonzero": true,
  "controls": {
    "exactly_degree_four_every_structure": true,
    "N4_no_order_0_or_2_contact_arises": true,
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
```

```
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "integrand_equals_XVI_O_convolution_M1": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true,
    "M2_discharged": true,
    "mutations_all_rejected": true
  }
},
"hh_slot_summary": {
  "verdict": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
  "row": "(SO,Gamma1^[2]) s2 K:K.c",
  "graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
  "how_D11_decouples": "graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse by i
+> tself, 4D and evanescent parts separately; its evanescent poles are nonzero but transverse. Hence D11 mut
+> ations of phase or symmetry factor are NOT detectable by the slot (they are tested by M2 instead); the sl
+> ot sees D11 only through a non-longitudinal operator (trace operator control: nonzero)",
  "surviving_evanescent_structures_in_slot": [],
  "consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop (D11 in
+> cluded), so NO contract revision is made for it.  The summed evanescent poles of the hh Green function ar
+> e nonzero but transverse; renormalising the hh Green function itself would need evanescent h.h counterter
+> ms -- outside this projection and NOT claimed.",
  "controls": {
    "operator_longitudinal": true,
    "p_dot_G_D11_zero_exact_in_d": true,
    "p_dot_G_sum_zero_exact_in_d": true,
    "p_dot_G_evanescent_sum_zero": true,
    "slot_sum_zero_all_7_assignments": true,
    "evanescent_slot_sum_zero_all_7_assignments": true,
    "D01_D03_individually_nonzero_incl_c_hat_h_tilde": true,
    "D11_reaches_slot_under_trace_operator": true,
    "4D_pole_sum_D01_D03_D11_transverse": true,
    "4D_pole_D11_transverse": true,
    "mutations_all_rejected": true
  }
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
```

```
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
```

```
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p22.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "1bc6edff6ca0788bb0fee4e8bf259bf8a0c80791f6dd08be77f79e56b492a50c",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "auto",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p22.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "1bc6edff6ca0788bb0fee4e8bf259bf8a0c80791f6dd08be77f79e56b492a50c",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": false,
    "inherited_mode": null,
    "environment": {
      "python": "3.13.5",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
      "pip_freeze": ["iniconfig==2.3.1", "mpmath==1.3.0", "packaging==26.3", "pluggy==1.6.0", "Pygments==2.2
+> 1.0", "pytest==9.1.1", "sympy==1.14.0"]
    }
  },
  "p21_archive": "archival/p21",
  "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
},
"XVI_P_registered": false,
"XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
"XVI_M_pin_ok": true,
"symmetry_restored": false,
"bulk_symmetry_restored": false,
"O1_closure": "NOT_CLOSED",
"insertion_status": "Frozen_negative_obstruction_preserved",
"XV_B_opened": false,
"XIV_J_opened": false,
"Route_B_discharged": false,
"u_R_c_delta_added": false,
"inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
  "P21_release.json": "f7d5856226f4808d391d7b5a602ac29171ebb30bdeed61d2bb898e95dc13d2c6",
  "P22_evanescent.json": "07e792f209957c53f99aa39fe4daa53cc04a39d3a4f67d43132a847a7e13431f"
},
"p19_layer_disclosed_test_revisions_p19": {
  "tests/test_p14.py": "names p18 -> p19 (reference file, source zip, author report, status revision); gener
+> ated prefixes certificates/ledgers P19_, manuscript/body/P19-",
  "tests/test_p18.py": "rs() reads the p18 LAYER record certificates/P18_release.json; the live p18 independ
+> ent report is bound to the p18 tree (pending or False here); test-file change log accepts p18 or disclose
+> d p19 revisions; build label is the current revision"
```

```
},
"D11_ratio_pin_verified": "earned",
"D11_absolute_pin_verified": "earned",
"p18_layer_byte_identical": true,
"p20_layer_change_log_vs_p19": {
  "base": "archival/p19/p19_source_files.json (pinned; files of the p19 zip sha256 da9242752d2e3753d8b2ce7fb
+> b21e7cdb28487cc7966c2ca52d1ef1376658875)",
  "base_pinned_ok": true,
  "added": ["archival/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p
+> 19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/in
+> dependent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "arch
+> ival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "arch
+> ival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p2
+> 1_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebu
+> ild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/R
+> EADME.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archiv
+> al/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archiv
+> al/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p
+> 5_calc/d11_pin.py", "code/p5_calc/evanescent.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_co
+> nventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_
+> counting.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py"
+> , "code/p5_calc/release_p23.py", "manuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pi
+> n.tex", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P23_O1_scope.tex", "provenance/XVI-O/O_q
+> uantum_target.json", "reproducibility/reference_values_p23.json", "tests/test_p20.py", "tests/test_p21.py
+> ", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p19.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/defect.py", "code/p5_calc/release_p18.p
+> y", "code/p5_calc/release_p19.py", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.tex", "manuscript
+> /notes/P19_defect.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "tests/test_p18.py
+> ", "tests/test_p19.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": [],
  "counts": {
    "added": 43,
    "removed": 1,
    "modified": 16
  }
},
"p20_layer_disclosed_test_revisions_p20": {
  "tests/test_p14.py": "names p19 -> p20; generated prefixes P20_",
  "tests/test_p19.py": "K1 tests corrected (orientation, full closure with derived +1); restoration on the p
+> 20 definition; J1 wording; p19 independent report bound to the p19 tree; scan sites = p19 + p20 reviewed
+> lists"
},
"p20_layer_disclosed_p19_layer_revisions_p20": {
  "code/p5_calc/defect.py": "K1 orientation fix (measured ratio, Sigma converted to route-G (c,cb)); tree co
+> ntrols; full sigma_0 closure of the complete MS-bar S_ct; restoration REDEFINED; full-closure mutation de
+> tector; p19 criterion relabelled route-consistency control; J1 wording; K1 erratum texts",
  "code/p5_calc/release_p19.py": "claim texts: K1 erratum (p19 'closure fails / needs -1' retracted); restor
+> ation content checks on the p20 definition; reject removes stale RELEASE_STATUS.json"
},
"p20_layer_p19_layer": {
  "p19_conventions_byte_identical": true,
  "disclosed_p20_revisions": {
    "code/p5_calc/defect.py": {
      "sha256": "675c1fa812c51b9955dd48a4149d309ff2e99ec7797fad753eb57b37e908ce6f",
      "differs_from_p19": true,
      "disclosed": true
    },
    "code/p5_calc/release_p19.py": {
      "sha256": "4d048345270a39a77cc30f71d3bdcbef17fffec6c44c1ac89487135ea3555970",
      "differs_from_p19": true,
      "disclosed": true
    }
  },
  "P19_release_accepted": true,
  "lower_layers_rechecked_p21": {
    "p18_layer_ok": true,
    "P18_quantum_inputs_evaluated": "earned",
    "inventory_recheck_ok": true
  }
},
"p20_layer_erratum_p19": {
  "K1": "p19 stated that full sigma_0 closure of the MS-bar counterterm fails with the derived ghost-equatio
+> n factor +1 and needs -1 (unresolved). RETRACTED: Grassmann-orientation bookkeeping error -- the P15 Sigm
+> a coordinates are (cb,c)-oriented, the route-G c.cb slots (c,cb), V[c.cb#k] = -V[(cb,c)#k]. Corrected: Si
+> gma = (+3/8, -1/4) in (c,cb) coordinates, derived +1, full closure in h.c, c.b, c.cbs.",
  "restoration": "p19's projected_restoration_verified (P_ch sigma_0(S_ct) = 0, x = 0) followed from the ide
+> ntity + locality; REDEFINED in p20 as full sigma_0 closure of the complete MS-bar S_ct (incl. c.cb), kept
+>  as a labelled route-consistency control",
  "J1": "'over Q(i), the physical field' replaced by: the Q(i) counts are the real dimensions under a compat
+> ible real structure",
```

```
  "found_by": "p19 independent rebuild (report sha256 1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf00
+> 8dd2811f45)"
},
"p20_layer_K1_correction": {
  "content_revision_p20": true,
  "orientation_ratio_minus_one": true,
  "Sigma_pole_routeG_c_cb": true,
  "derived_factor_plus_one": true,
  "derived_relation_reproduces_P15": true,
  "full_closure_complete_S_ct": true,
  "without_ccb_not_closed": true,
  "tree_controls_ok": true,
  "p20_controls_all_true": true,
  "detector_drop_D13": true,
  "detector_flip_Kc": true,
  "detector_drop_D11_recorded_blind": true,
  "route_consistency_control_holds": true,
  "quotient_incl_Sigma_zero": true,
  "quotient_Sigma_alone_c_cb_plus_3_8_minus_1_4": true,
  "J1_wording": true,
  "restoration_definition_p20": true,
  "bulk_false": true
},
"p20_layer_retraction_check": {
  "phrases": ["full c.b / ghost-sector closure FAILS", "full c.b / ghost-sector closure of the MS-bar counte
+> rterm FAILS", "for which full closure fails", "with +1 full closure fails", "full closure \\emph{fails}",
+>  "This is unresolved", "This is not resolved in p19", "needs relative factor -1; the derived ghost-equati
+> on relation gives +1; not resolved", "factor +1 (K1) full c.b closure FAILS", "full c.b closure NOT estab
+> lished (K1)", "the physical field"],
  "scope": ["README.md", "manuscript/notes/P19_defect.tex", "certificates/P19_defect.json", "certificates/P1
+> 9_release.json", "manuscript/body/P19-results-table.tex"],
  "hits": {}
},
"p20_layer_D11_pin": {
  "check": {
    "ratio_pin_PASS": true,
    "ratio_10": true,
    "transverse": true,
    "C2_quadratic_form_half_P2": true,
    "GaussBonnet_zero": true,
    "absolute_pin_PASS": true,
    "absolute_pin_flag": true,
    "top_level_absolute_status_consistent": true,
    "top_level_ratio_status_consistent": true,
    "absolute_ratios_one": true,
    "no_external_convention_choice": true,
    "drop_D11_detected": true,
    "x2_detected_by_absolute": true,
    "sign_flip_detected_by_absolute": true,
    "corrected_in_p21": true,
    "literature_source_stated": true,
    "revision": true
  },
  "ratio_pin": {
    "status": "PASS",
    "statement": "spin-0 : spin-2 of the D11 4D UV pole = 10 : 1, derived from the certificate; equals the m
+> inimal-scalar a_2 prediction (1/72)(3) : (1/120)(1/2)"
  },
  "absolute_pin": {
    "status": "PASS",
    "observed_common_factor_c2_vs_bare_a2": "-I",
    "observed_common_factor_c0_vs_bare_a2": "-I",
    "derivation_p21": {
      "package_declarations_checked": {
        "N_eps = i S_eps/(16 pi^2)": true,
        "Gamma_1 = -i * graph sum": true,
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
        "g = eta + kappa h": true,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
        "eta = (+,-,-,-)": true,
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
      },
      "steps": {
        "1_G_equals_Gamma2": "1",
        "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
        "3_i_S_M_over_S_E (must be -1)": "-1",
        "4_L_M_eff_over_L_E_eff": "-1",
        "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
        "6_minkowski_quadratic_forms": {
          "Riem2": {
```

```
            "P2": "1",
            "P0s": "1"
          },
          "Ric2": {
            "P2": "1/4",
            "P0s": "1"
          },
          "R2": {
            "P2": "0",
            "P0s": "3"
          },
          "C2": {
            "P2": "1/2",
            "P0s": "0"
          },
          "E_GaussBonnet": {
            "P2": "0",
            "P0s": "0"
          }
        }
      },
      "package_Gamma2_pole_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "heat_kernel_prediction_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "ratio_package_over_prediction": {
        "spin2": "1",
        "spin0s": "1"
      },
      "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2
+>  (Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps
+> )) int a_2 (Euclidean, d = 4-2eps)"],
      "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, s
+> ign of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
      "status": "PASS"
    },
    "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under th
+> e package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the W
+> ick direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(
+> 16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1
+> , both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no ext
+> ernal convention choice.  p20 had left this OPEN."
  },
  "spin_decomposition": {
    "c2_spin2": "-I/240",
    "c0_spin0s": "-I/24",
    "projector_norms": ["5", "1"],
    "transverse": true,
    "only_spin2_and_spin0s": true,
    "ratio_c0_over_c2": "10"
  },
  "mutations": {
    "drop_graph_D11": {
      "c2": "0",
      "c0": "0",
      "ratio": "None",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_normalisation_x2": {
      "c2": "-I/120",
      "c0": "-I/12",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_sign_flip": {
      "c2": "I/240",
      "c0": "I/24",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
```

```
      "detected_by_absolute_pin": true
    },
    "swap_g4g4_structures": {
      "c2": "-I/40",
      "c0": "-I/32",
      "ratio": "5/4",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    }
  }
},
"p20_layer_open_pins": [],
"p20_layer_open_pins_closed_in_p21": ["D11_absolute_normalisation (Ghat -> a_2 map not derived)"],
"p20_layer_p19_independent_rebuild": {
  "report": "archival/p19/reproducibility/independent_rebuild_report_p19.json",
  "present": true,
  "report_sha256": "1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf008dd2811f45",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36",
  "bound_to_source_zip_sha256": "da9242752d2e3753d8b2ce7fbb21e7cdb28487cc7966c2ca52d1ef1376658875",
  "tree_matches_p19": true,
  "zip_matches_p19": true,
  "mismatches": 0,
  "findings": ["K1_ghost_closure_sign_is_orientation_bookkeeping", "quotient_coordinates_c.cb_orientation",
+> "restoration_flag_not_independent", "mutation_criterion_relabelled", "J1_wording", "I2_scan_misses", "gat
+> e_stale_RELEASE_STATUS"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p20": "bound to the p19 tree/zip; the p20 tree has its own (pending) independent flag",
  "archived_p19_records_pinned": {
    "archival/p19/reference_values_p19.json": true,
    "archival/p19/reproducibility/author_rebuild_report_p19.json": true,
    "archival/p19/reproducibility/independent_rebuild_report_p19.json": true,
    "archival/p19/p19_source_files.json": true
  },
  "pins_ok": true
},
"p20_layer_independent_rebuild_passed_p19": true,
"p19_gate_change_log_vs_p18_recomputed_on_current_tree": {
  "base": "archival/p18/p18_source_files.json (pinned; files of the p18 zip sha256 7c7d5451adb459092edc2606e
+> 1b3322909325000d47bab89c14b7f7a6d25c92f)",
  "base_pinned_ok": true,
  "reference_file": "reproducibility/reference_values_p23.json listed as added by declaration (written after
+>  the author build)",
  "added": ["archival/p18/README.md", "archival/p18/p18_source_files.json", "archival/p18/reference_values_p
+> 18.json", "archival/p18/reproducibility/author_rebuild_report_p18.json", "archival/p18/reproducibility/in
+> dependent_rebuild_report_p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.json", "arch
+> ival/p19/reference_values_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "arch
+> ival/p19/reproducibility/independent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p2
+> 0_source_files.json", "archival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebu
+> ild_report_p20.json", "archival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/R
+> EADME.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/
+> reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report
+> _p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_
+> source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuil
+> d_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/con
+> tract_registry.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "c
+> ode/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "c
+> ode/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "co
+> de/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/r
+> elease_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_c
+> orrection.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscri
+> pt/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values_p
+> 23.json", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_p22.py", "tests/test
+> _p23.py"],
  "removed": ["reproducibility/reference_values_p18.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/release_p18.py", "code/p5_entry/dred.py
+> ", "manuscript/body/XVI-P-body.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "test
+> s/test_p18.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p18/README.md", "archival
+> /p18/p18_source_files.json", "archival/p18/reference_values_p18.json", "archival/p18/reproducibility/auth
+> or_rebuild_report_p18.json", "archival/p18/reproducibility/independent_rebuild_report_p18.json", "archiva
+> l/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p19.json", "archiv
+> al/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/independent_rebuild
```

```
+> _report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/referenc
+> e_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproduc
+> ibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.js
+> on", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.js
+> on", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archi
+> val/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_
+> values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducib
+> ility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/d11_pin.py
+> ", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "code/p5_calc/p19_conventions.py", "code/p5_ca
+> lc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_ca
+> lc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p18.py", "code/p5_calc/re
+> lease_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py
+> ", "code/p5_calc/release_p23.py", "manuscript/body/XVI-P-body.tex", "manuscript/notes/P19_defect.tex", "m
+> anuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evane
+> scent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibili
+> ty/reference_values_p18.json", "reproducibility/reference_values_p23.json", "tests/test_p14.py", "tests/t
+> est_p16.py", "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/t
+> est_p21.py", "tests/test_p22.py", "tests/test_p23.py", "tools/build.py", "tools/rebuild_from_source.py",
+> "tools/source_manifest.py"],
  "counts": {
    "added": 53,
    "removed": 1,
    "modified": 12
  }
},
"p21_layer_absolute_pin": {
  "check": {
    "ratio_pin_PASS": true,
    "ratio_10": true,
    "transverse": true,
    "C2_quadratic_form_half_P2": true,
    "GaussBonnet_zero": true,
    "absolute_pin_PASS": true,
    "absolute_pin_flag": true,
    "top_level_absolute_status_consistent": true,
    "top_level_ratio_status_consistent": true,
    "absolute_ratios_one": true,
    "no_external_convention_choice": true,
    "drop_D11_detected": true,
    "x2_detected_by_absolute": true,
    "sign_flip_detected_by_absolute": true,
    "corrected_in_p21": true,
    "literature_source_stated": true,
    "revision": true
  },
  "absolute_pin": {
    "status": "PASS",
    "observed_common_factor_c2_vs_bare_a2": "-I",
    "observed_common_factor_c0_vs_bare_a2": "-I",
    "derivation_p21": {
      "package_declarations_checked": {
        "N_eps = i S_eps/(16 pi^2)": true,
        "Gamma_1 = -i * graph sum": true,
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
        "g = eta + kappa h": true,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
        "eta = (+,-,-,-)": true,
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
      },
      "steps": {
        "1_G_equals_Gamma2": "1",
        "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
        "3_i_S_M_over_S_E (must be -1)": "-1",
        "4_L_M_eff_over_L_E_eff": "-1",
        "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
        "6_minkowski_quadratic_forms": {
          "Riem2": {
            "P2": "1",
            "P0s": "1"
          },
          "Ric2": {
            "P2": "1/4",
            "P0s": "1"
          },
          "R2": {
            "P2": "0",
            "P0s": "3"
          },
          "C2": {
            "P2": "1/2",
```

```
            "P0s": "0"
          },
          "E_GaussBonnet": {
            "P2": "0",
            "P0s": "0"
          }
        }
      },
      "package_Gamma2_pole_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "heat_kernel_prediction_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "ratio_package_over_prediction": {
        "spin2": "1",
        "spin0s": "1"
      },
      "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2
+>  (Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps
+> )) int a_2 (Euclidean, d = 4-2eps)"],
      "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, s
+> ign of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
      "status": "PASS"
    },
    "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under th
+> e package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the W
+> ick direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(
+> 16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1
+> , both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no ext
+> ernal convention choice.  p20 had left this OPEN."
  },
  "absolute_pin_status": "PASS",
  "mutations": {
    "drop_graph_D11": {
      "c2": "0",
      "c0": "0",
      "ratio": "None",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_normalisation_x2": {
      "c2": "-I/120",
      "c0": "-I/12",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_sign_flip": {
      "c2": "I/240",
      "c0": "I/24",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "swap_g4g4_structures": {
      "c2": "-I/40",
      "c0": "-I/32",
      "ratio": "5/4",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    }
  }
},
"p21_layer_lower_layers": {
  "contract_identity": true,
  "p18_layer": true,
  "P18_quantum_inputs_evaluated": "earned",
  "inventory": true,
  "p19_layer": true,
  "k1_correction": true,
```

```
  "retraction": true,
  "retraction_hits": {},
  "contract_detail": {
    "p14r_contract_files": 12,
    "changed_vs_p14r": ["code/p5_entry/dred.py"],
    "added": [],
    "dred_byte_identical_to_p18": true,
    "dred_registry_state_p22": "p18 + p22 guard block",
    "contract_revision_in_p19": false,
    "disclosed_revisions_p19": {}
  },
  "rechecked_live": true
},
"p21_layer_p20_layer": {
  "p20_conventions_byte_identical": true,
  "disclosed_p21_revisions": {
    "code/p5_calc/d11_pin.py": {
      "sha256": "a2e77eecc84263e254d3b7cd7b441731324e8d15b5d15c71dc05d61ff5fe54f8",
      "differs_from_p20": true,
      "disclosed": true
    },
    "code/p5_calc/release_p20.py": {
      "sha256": "a1afba60c21c923886e682b960dda8b2619353712517f078c9967ad9c57e4d5f",
      "differs_from_p20": true,
      "disclosed": true
    }
  },
  "P20_release_accepted": true,
  "P20_D11_absolute_pin_verified": "earned"
},
"p21_layer_manuscript_disclosure": {
  "manuscript/notes/P19_defect.tex": {
    "current_equals_disclosed_p20": true,
    "differs_from_p19": true,
    "revised_in": "p20"
  }
},
"p21_layer_change_log_vs_p20": {
  "base": "archival/p20/p20_source_files.json (pinned; files of the p20 zip sha256 ca32ce018933f12d1a623217d
+> 6a98d647817360dd12211e2e9469cf850a63bb2)",
  "base_pinned_ok": true,
  "added": ["archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/reference_values_p
+> 20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproducibility/in
+> dependent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.json", "arch
+> ival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "arch
+> ival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22/ce
+> rtificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_values_p22
+> .json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/inde
+> pendent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/evanescent.py", "cod
+> e/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "cod
+> e/p5_calc/power_counting.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py", "code/p5_calc
+> /release_p23.py", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manusc
+> ript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values
+> _p23.json", "tests/test_p21.py", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p20.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/d11_pin.py", "code/p5_calc/release_p18.
+> py", "code/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_entry/dred.py", "manuscript/b
+> ody/XVI-P-body.tex", "manuscript/notes/P20_correction.tex", "tests/test_p14.py", "tests/test_p16.py", "te
+> sts/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tools/build.py", "tools
+> /rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": [],
  "undisclosed_package_changes": [],
  "counts": {
    "added": 33,
    "removed": 1,
    "modified": 18
  }
},
"p21_layer_open_pins": [],
"p21_layer_disclosed_p20_layer_revisions_p21": {
  "code/p5_calc/d11_pin.py": "absolute_pin(): sign and magnitude derived from 1/2 Tr ln under the declared c
+> onventions (Minkowski quadratic forms curvature_forms_eta); absolute pin PASS; mutations x2 and sign flip
+> ; top-level absolute_pin_status from the derivation",
  "code/p5_calc/release_p20.py": "absolute pin PASS required; top-level / nested status consistency; standal
+> one run re-checks the p18 layer (q earned) and the graph inventory; claim texts"
},
"p21_layer_disclosed_test_revisions_p21": {
  "tests/test_p14.py": "names p20 -> p21; generated prefixes P21_",
  "tests/test_p19.py": "reviewed coefficient sites read from p21_conventions (adds the d11_pin Minkowski for
+> m site)",
  "tests/test_p20.py": "D11 pin tests: absolute pin PASS (p21) instead of OPEN; gate mutations on the p21 ch
```

```
+> ecks; reviewed sites from p21_conventions; status/version expectations p21"
},
"p21_layer_disclosed_manuscript_revisions": {
  "manuscript/notes/P19_defect.tex": {
    "p19_sha256": "04b974b5ea742c6885f3d694ccad6bda7908105c0a23bf483e96bc113d90bbc8",
    "p20_sha256": "1c71a71a48ce9c1749bd2164a0f720a3589f2d679877cfd04309d69670e5dfa8",
    "revised_in": "p20",
    "change": "K1 paragraph retracted/rewritten (Grassmann orientation erratum), restoration definition REDE
+> FINED, J1 wording, blind-spot text; marked inline [corrected in p20] / [REDEFINED in p20]; unchanged in p
+> 21"
  }
},
"p21_layer_independent_rebuild_passed_p20": true,
"p21_layer_p20_independent_rebuild": {
  "report": "archival/p20/reproducibility/independent_rebuild_report_p20.json",
  "present": true,
  "report_sha256": "e8f358f9c4f13c76693968211b49e04d546a0bb3a4c0ab4b1a0e485f62595389",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700",
  "bound_to_source_zip_sha256": "ca32ce018933f12d1a623217d6a98d647817360dd12211e2e9469cf850a63bb2",
  "tree_matches_p20": true,
  "zip_matches_p20": true,
  "mismatches": 0,
  "findings": ["p19_quotient_finding_withdrawn", "D11_absolute_pin_left_OPEN_but_derivable", "scan_misses_p2
+> 0", "gate_top_level_absolute_pin_status_unchecked", "p20_gate_trusts_P19_layer_record", "P19_note_edit_di
+> sclosure", "restoration_scope", "p19_change_log_key_recomputed_on_p20_tree"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p21": "bound to the p20 tree/zip; the p21 tree has its own (pending) independent flag",
  "archived_p20_records_pinned": {
    "archival/p20/reference_values_p20.json": true,
    "archival/p20/reproducibility/author_rebuild_report_p20.json": true,
    "archival/p20/reproducibility/independent_rebuild_report_p20.json": true,
    "archival/p20/p20_source_files.json": true
  },
  "pins_ok": true
},
"evanescent": {
  "check": {
    "EV_M2_proven": true,
    "EV_CT_earned": true,
    "controls_all_true": true,
    "hc_closed": true,
    "cb_closed": true,
    "identity_unchanged": true,
    "mutations_detected": true,
    "continuations_refuted_recorded": true,
    "Q4S_trace_remnant_disclosed": true,
    "P5_status": true,
    "contract_revision_named": true
  },
  "EV_M2": "EV-M2: the evanescent D11 poles are local evanescent operators (removed by the p22 evanescent co
+> unterterm), and the renormalised evanescent remainder vanishes identically on 4D external configurations,
+>  exactly in d.  No matching to XVI-O is asserted.",
  "EV_M2_verdict": "PROVEN (exact, opaque d)",
  "coefficients_hh_sum": {
    "g4_mn gt_rs (p.p)^2": "-7*I/120",
    "g4_mr gt_ns (p.p)^2": "-I/160",
    "g4_rs gt_mn (p.p)^2": "-7*I/120",
    "gt_mn gt_rs (p.p)^2": "I/160",
    "gt_mn p_r p_s (p.p)^1": "7*I/120",
    "gt_mr gt_ns (p.p)^2": "I/160",
    "gt_mr p_n p_s (p.p)^1": "I/160",
    "gt_rs p_m p_n (p.p)^1": "7*I/120"
  },
  "coefficient_Kc_E2": "-1/16",
  "coefficient_ccb_SE": "1/8",
  "remnants": {
    "finite_evanescent_remainder": "MS-bar finite parts (local + L) x gt-structures: recorded per structure
+> in P18_D11 / P16_batchC / P15_D13; NONZERO; identically zero on 4D (hatted) external legs, exactly in d (
+> four_d_decoupling below)",
    "O_eps_times_pole_at_this_order": "none with QdS (hatted) external legs: all four indices are external,
+> so tr gt = 4 - d never arises and the evanescent operators are ZERO on hatted legs at every d; see the Q4
+> S trace entry for the one place it can arise",
    "trace_of_pole_tensor_g4^{mn}T_mn,rs (hatted r,s)": {
      "per_graph": {
        "D01": "-I*p.p*(d - 4)*(13*a^.b^*p.p - 18*a^.p*b^.p)/120",
```

```
        "D03": "I*p.p*(d - 4)*(a^.b^*p.p - 2*a^.p*b^.p)/24",
        "D11": "I*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)/120"
      },
      "sum": "-7*I*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)/120"
    },
    "O_eps_times_pole_Q4S_trace_external": {
      "finite_term_if_external_trace_mode_uses_g4": "7*I*p.p*(a^.b^*p.p - a^.p*b^.p)/60",
      "rule": "(d - 4) x (1/eps) -> -2 (d = 4 - 2 eps); times the residue above",
      "status": "NONZERO: a finite local transverse term arises iff the external trace mode is defined with
+> the quasi-4D g4 (it then carries eps-scalar components); with QdS (hatted) external legs it is zero.  Thi
+> s is a scheme / external-state convention (DRED eps-scalar sector), disclosed: the p22 renormalised resul
+> t is stated for QdS external legs"
    },
    "where_O_eps_x_pole_would_enter": "insertions of the evanescent counterterms into loops (two loops) or i
+> nto composite operators with traces: (4 - d) x (1/eps) = finite local terms -- beyond this order; not eva
+> luated, not claimed"
  }
},
"P5_calculation_status": "renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-bar in D
+> RED with the p22 evanescent counterterms; external legs in QdS)",
"d_runtime_guard": {
  "ok": true,
  "n_plants": 30,
  "not_raised": {},
  "other_outcomes": {},
  "algebra_ok": "(d**2 + 30*d - 34)/(32*(d - 1)*(d + 1))",
  "entry_points": ["OpaqueDimension.__eq__ (d == d)", "Atom.xreplace (bare d)", "Basic.subs", "Basic.xreplac
+> e", "Basic.replace", "evalf/n(subs=...)", "Poly.eval", "Poly.__call__", "Poly.all_coeffs", "Poly.coeffs",
+>  "Poly.terms", "Poly.all_terms", "Poly.as_dict", "Poly.as_list", "Poly.coeff_monomial", "Poly.nth", "Poly
+> .LC", "Poly.TC", "Poly.EC", "Poly.as_expr", "Expr.coeff", "Expr.as_independent", "Expr.as_coeff_exponent"
+> , "Expr.series", "collect", "limit", "Lambda(d, ...)", "polytools.rem (d in domain)", "polytools.prem (d
+> in domain)", "polytools.pdiv (d in domain)", "polytools.pquo (d in domain)", "polytools.pexquo (d in doma
+> in)", "polytools.div (d in domain)", "polytools.quo (d in domain)", "polytools.exquo (d in domain)", "pol
+> ytools.gcdex (d in domain)", "polytools.half_gcdex (d in domain)", "polytools.invert (d in domain)", "pol
+> ytools.subresultants (d in domain)", "polytools.resultant (d in domain)", "polytools.discriminant (d in d
+> omain)", "polytools.compose (d in domain)", "polytools.decompose (d in domain)", "polytools.sturm (d in d
+> omain)", "polytools.nroots (d in domain)", "polytools.real_roots (d in domain)", "polytools.count_roots (
+> d in domain)", "polytools.ground_roots (d in domain)", "polytools.intervals (d in domain)", "polytools.re
+> fine_root (d in domain)", "polytools.nth_power_roots_poly (d in domain)", "Poly.rem (d in domain)", "Poly
+> .prem (d in domain)", "Poly.pdiv (d in domain)", "Poly.pquo (d in domain)", "Poly.pexquo (d in domain)",
+> "Poly.div (d in domain)", "Poly.quo (d in domain)", "Poly.exquo (d in domain)", "Poly.gcdex (d in domain)
+> ", "Poly.half_gcdex (d in domain)", "Poly.invert (d in domain)", "Poly.subresultants (d in domain)", "Pol
+> y.resultant (d in domain)", "Poly.discriminant (d in domain)", "Poly.compose (d in domain)", "Poly.decomp
+> ose (d in domain)", "Poly.sturm (d in domain)", "Poly.nroots (d in domain)", "Poly.real_roots (d in domai
+> n)", "Poly.count_roots (d in domain)", "Poly.ground_roots (d in domain)", "Poly.intervals (d in domain)",
+>  "Poly.refine_root (d in domain)", "Poly.nth_power_roots_poly (d in domain)", "Poly.shift (d in domain)",
+>  "Poly.transform (d in domain)", "Poly.shift_list (d in domain)", "internal rename of d to a plain Symbol
+>  (Matrix.subs)"],
  "residue_not_interceptable": "NOT runtime-interceptable (code-review matters; the static allow-list scan i
+> s the secondary barrier): identity / name / type tests in hand-written expression-tree rebuilds (`x is d`
+> , x.name == 'd', isinstance), string round-trips (str(e).replace('d', '4')), low-level polynomial represe
+> ntations (Poly.rep, DMP, polyutils dict builders), and code that disguises its module name as sympy.  Sym
+> bol.__new__(Symbol, 'd') yields a PLAIN d that does not match the opaque d (no value is produced); parse_
+> d / assert_opaque_d refuse plain d in loaded data.",
  "static_scan": "secondary barrier (tests/test_p21.py scan4 allow-list, p22 sites reviewed)"
},
"contract": {
  "dred_registry": {
    "state": "p18 + p22 guard block",
    "ok": true,
    "sha256": "24112d9a6584613474e96f538840a9ff6aecbf9ebd588cd890fd38d2e295de5a",
    "p18_core_byte_identical": true,
    "registered_blocks": ["p22 runtime d guard", "p23 polynomial-domain refusal"]
  },
  "dred_sha256": "24112d9a6584613474e96f538840a9ff6aecbf9ebd588cd890fd38d2e295de5a",
  "dred_pinned": "1f994759452b9d76eea3cc3a90e02bbb1ed237f9c0865a99d1bc5a2cbe0b08bc",
  "dred_pin_ok": false,
  "contract_revision_p22": "p22 (p14r contract + disclosed p15/p16/p17/p18 revisions + the DISCLOSED p22 con
+> tract revision: evanescent counterterms (EV-CT) and the runtime d guard in code/p5_entry/dred.py)"
},
"lower_layers": {
  "contract_identity": true,
  "p18_layer": true,
  "P18_quantum_inputs_evaluated": "earned",
  "inventory": true,
  "p19_layer": true,
  "k1_correction": true,
  "retraction": true,
  "retraction_hits": {},
  "contract_detail": {
    "p14r_contract_files": 12,
```

```
    "changed_vs_p14r": ["code/p5_entry/dred.py"],
    "added": [],
    "dred_byte_identical_to_p18": true,
    "dred_registry_state_p22": "p18 + p22 guard block",
    "contract_revision_in_p19": false,
    "disclosed_revisions_p19": {}
  },
  "rechecked_live": true
},
"p20_layer": {
  "p20_conventions_byte_identical": true,
  "disclosed_p21_revisions": {
    "code/p5_calc/d11_pin.py": {
      "sha256": "a2e77eecc84263e254d3b7cd7b441731324e8d15b5d15c71dc05d61ff5fe54f8",
      "differs_from_p20": true,
      "disclosed": true
    },
    "code/p5_calc/release_p20.py": {
      "sha256": "a1afba60c21c923886e682b960dda8b2619353712517f078c9967ad9c57e4d5f",
      "differs_from_p20": true,
      "disclosed": true
    }
  },
  "P20_release_accepted": true,
  "P20_D11_absolute_pin_verified": "earned"
},
"p21_layer": {
  "p21_conventions_byte_identical": true,
  "P21_release_accepted": true,
  "P21_earned": {
    "projected_defect_computed": "earned",
    "projected_identity_verified": "earned",
    "projected_restoration_verified": "earned",
    "D11_ratio_pin_verified": "earned",
    "D11_absolute_pin_verified": "earned"
  }
},
"absolute_pin_check": {
  "ratio_pin_PASS": true,
  "ratio_10": true,
  "transverse": true,
  "C2_quadratic_form_half_P2": true,
  "GaussBonnet_zero": true,
  "absolute_pin_PASS": true,
  "absolute_pin_flag": true,
  "top_level_absolute_status_consistent": true,
  "top_level_ratio_status_consistent": true,
  "absolute_ratios_one": true,
  "no_external_convention_choice": true,
  "drop_D11_detected": true,
  "x2_detected_by_absolute": true,
  "sign_flip_detected_by_absolute": true,
  "corrected_in_p21": true,
  "literature_source_stated": true,
  "revision": true
},
"manuscript_disclosure": {
  "manuscript/notes/P20_correction.tex": {
    "differs_from_p21": true,
    "superseded_marker_present": true,
    "revised_in": "p22"
  }
},
"change_log_vs_p21": {
  "base": "archival/p21/p21_source_files.json (pinned; files of the p21 zip sha256 a71d6b5203bc3914833db9fd1
+> 3b9d186bef93d1ae3007b2e4c99fdfce11e0678)",
  "base_pinned_ok": true,
  "added": ["archival/p21/README.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p
+> 21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/in
+> dependent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.j
+> son", "archival/p22/p22_source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/repro
+> ducibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.
+> json", "code/p5_calc/contract_registry.py", "code/p5_calc/evanescent.py", "code/p5_calc/p22_conventions.p
+> y", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p22.py", "
+> code/p5_calc/release_p23.py", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P23_01_scope.tex",
+>  "provenance/XVI-O/O_quantum_target.json", "reproducibility/reference_values_p23.json", "tests/test_p22.p
+> y", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p21.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/release_p18.py", "code/p5_calc/release_
+> p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_entry/dred.py", "manuscri
+> pt/body/XVI-P-body.tex", "manuscript/notes/P20_correction.tex", "tests/test_p14.py", "tests/test_p16.py",
```

```
+>  "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py",
+>  "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "contract_changes_registered": true,
  "undisclosed_changes": [],
  "pinned_sha_mismatches": {},
  "disclosed_but_unchanged": [],
  "exemptions": {
    "generated": ["reproducibility/reference_values_p22.json", "reproducibility/reference_values_p21.json"],
    "added_archive_prefixes": ["archival/p21/"]
  },
  "counts": {
    "added": 24,
    "removed": 1,
    "modified": 19
  }
},
"open_pins": [],
"evanescent_M2_verified": "earned",
"evanescent_counterterms_verified": "earned",
"d_runtime_guard_verified": "earned",
"disclosed_revisions_p22": {
  "code/p5_entry/dred.py": "CONTRACT REVISION p22: runtime d guard (one delimited additive block; p18 text u
+> nchanged)",
  "code/p5_calc/p22_conventions.py": "new: p22 conventions",
  "code/p5_calc/evanescent.py": "new: EV-M2 and EV-CT (priorities 1, 2)",
  "code/p5_calc/contract_registry.py": "new: registry of the post-p18 contract revision (consulted by layer
+> gates)",
  "code/p5_calc/release_p22.py": "new: p22 release gate",
  "code/p5_calc/release_p18.py": "p22: accept the registered dred.py revision in place of the p17 AST identi
+> ty",
  "code/p5_calc/release_p19.py": "p22: contract identity accepts the registered dred.py revision; p18-layer
+> pin accepts the registered release_p18.py revision",
  "code/p5_calc/release_p20.py": "p22: change log ignores the registered contract revision; p18-layer pin ac
+> cepts the registered release_p18.py revision",
  "code/p5_calc/release_p21.py": "p22: change log ignores the registered contract revision and treats the p2
+> 2 revised files (contract_registry.P22_REVISED_FILES) as disclosed",
  "tests/test_p14.py": "names p21 -> p22",
  "tests/test_p16.py": "the replace(lambda x: x == d, ..) demonstration now RAISES (p22 runtime guard)",
  "tests/test_p17.py": "g1 scan reviews the p18 contract core of the registered dred revision (contract_regi
+> stry.scan_view; the p22 block is covered by the runtime probe + scan4)",
  "tests/test_p18.py": "h1 scan via contract_registry.scan_view; h3/h5 accept the registered p18+p22-block d
+> red state",
  "tests/test_p19.py": "i2 scan via contract_registry.scan_view; p18 calc layer accepts the registered p22 r
+> evision of release_p18.py (contract_registry.layer_file_ok)",
  "tests/test_p20.py": "status / version expectations p22; p21 tree no longer current; scan3 via scan_view;
+> p18 calc layer via contract_registry.layer_file_ok",
  "tests/test_p21.py": "scan allow-lists read from p22_conventions; the power-base subs limitation test now
+> asserts the p22 fix; status/version expectations p22; p21 layer independent flag not current",
  "tests/test_p22.py": "new: p22 tests",
  "manuscript/notes/P20_correction.tex": "inline [superseded in p21] marker on the 'Absolute pin: OPEN' item
+> ",
  "manuscript/notes/P22_evanescent.tex": "new: p22 note",
  "manuscript/body/XVI-P-body.tex": "input of the p22 note",
  "tools/build.py": "p22 build: release_p22 stage, p22 listings, value blocks, generated files",
  "tools/source_manifest.py": "reference file / zip name p22; p21 archive records",
  "tools/rebuild_from_source.py": "p22 reference values, release criteria, archival pins",
  "README.md": "p22 section",
  "REBUILD_INSTRUCTIONS.md": "p22 instructions"
},
"disclosed_manuscript_revisions_p22": {
  "manuscript/notes/P20_correction.tex": {
    "p21_sha256": "138c858852906d4daf3ed86d2e48a8d5ee8ef48717514bde00d4ca7151d3e7fb",
    "revised_in": "p22",
    "change": "inline [superseded in p21] marker on the 'Absolute pin: OPEN' item (reviewer stale_OPEN_text)
+> "
  }
},
"independent_rebuild_passed_p21": true,
"p21_independent_rebuild": {
  "report": "archival/p21/reproducibility/independent_rebuild_report_p21.json",
  "present": true,
  "report_sha256": "8a582b332cd242689c54b1f962dc88270b9592040c0ba432576a505f0e36a60d",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "b835353ffe16a8ff159e2916fe3efdf2b9d89a4d512e80e9e6da587cecd0fe5e",
  "bound_to_source_zip_sha256": "a71d6b5203bc3914833db9fd13b9d186bef93d1ae3007b2e4c99fdfce11e0678",
  "tree_matches_p21": true,
  "zip_matches_p21": true,
  "mismatches": 0,
```

```
    "findings": ["p21_gate_disclosure_prefix_exemptions", "scan_misses_p21", "d_guard_power_base_hole", "stale
  +> _OPEN_text_in_p20_section", "p20_gate_standalone_scope", "absolute_pin_step1_hardcoded", "reviewer_rule_n
  +> ote"],
    "caveat": "same box (disclosed)",
    "live_report_identical_to_archived": true,
    "NOT_inherited_by_p22": "bound to the p21 tree/zip; the p22 tree has its own (pending) independent flag",
    "archived_p21_records_pinned": {
      "archival/p21/reference_values_p21.json": true,
      "archival/p21/reproducibility/author_rebuild_report_p21.json": true,
      "archival/p21/reproducibility/independent_rebuild_report_p21.json": true,
      "archival/p21/p21_source_files.json": true
    },
    "pins_ok": true
  }
}
```


39 Evanescent components: EV-M2 statement and evanescent counterterms (p22)

[listing 06/133] certificates/P22_evanescent.json


```
{
  "landmark": "XVI-P",
  "revision": "p22",
  "contract_revision": "p22 (p14r contract + disclosed p15/p16/p17/p18 revisions + the DISCLOSED p22 contract
  +> revision: evanescent counterterms (EV-CT) and the runtime d guard in code/p5_entry/dred.py)",
  "deliverable": "p22 priorities 1-2: evanescent matching statement (EV-M2) and evanescent counterterms (EV-CT
  +> )",
  "EV_M2": {
    "question": "what must the 8 evanescent D11 structures satisfy, given XVI-O has no evanescent counterpart?
  +> ",
    "candidates": {
      "match a d-dim continuation of the XVI-O kernel": "REFUTED for both natural continuations (g4 -> ghat; g
  +> 4 kept): the exact D11 tensor differs from either continuation of its own 4D part on tilde external legs;
  +>  XVI-O defines no other continuation -- not a usable statement",
      "vanish as eps -> 0 in the renormalised result": "TRUE in the stronger form: after subtraction, every ev
  +> anescent component is identically ZERO on 4D external legs at every d (not merely O(eps))",
      "be pure local evanescent operators": "TRUE for the UV poles (constants x polynomial gt-structures); FAL
  +> SE for the finite parts (they carry L = log(-p^2/mu^2)): the finite remainder is nonlocal but evanescent"
    },
    "statement": "EV-M2: the evanescent D11 poles are local evanescent operators (removed by the p22 evanescen
  +> t counterterm), and the renormalised evanescent remainder vanishes identically on 4D external configurati
  +> ons, exactly in d.  No matching to XVI-O is asserted.",
    "verdict": "PROVEN (exact, opaque d)",
    "locality": true,
    "four_D_decoupling": {
      "D01: g4_mn gt_rs (p.p)^2": "0",
      "D01: g4_rs gt_mn (p.p)^2": "0",
      "D01: gt_mn gt_rs (p.p)^2": "0",
      "D01: g4_mr gt_ns (p.p)^2": "0",
      "D01: gt_mr gt_ns (p.p)^2": "0",
      "D01: gt_mn p_r p_s (p.p)^1": "0",
      "D01: gt_mr p_n p_s (p.p)^1": "0",
      "D01: gt_rs p_m p_n (p.p)^1": "0",
      "D03: gt_mn p_r p_s (p.p)^1": "0",
      "D03: gt_mr p_n p_s (p.p)^1": "0",
      "D03: gt_rs p_m p_n (p.p)^1": "0",
      "D03: g4_mn gt_rs (p.p)^2": "0",
      "D03: g4_mr gt_ns (p.p)^2": "0",
      "D03: g4_rs gt_mn (p.p)^2": "0",
      "D11: g4_mn gt_rs (p.p)^2": "0",
      "D11: g4_rs gt_mn (p.p)^2": "0",
      "D11: gt_mn gt_rs (p.p)^2": "0",
      "D11: g4_mr gt_ns (p.p)^2": "0",
      "D11: gt_mr gt_ns (p.p)^2": "0",
      "D11: gt_mn p_r p_s (p.p)^1": "0",
      "D11: gt_mr p_n p_s (p.p)^1": "0",
      "D11: gt_rs p_m p_n (p.p)^1": "0"
    },
    "continuation_tests": {
      "D11": {
        "ghat_continuation_mismatches": {
          "a^b^c~e~": "-I*c~.e~*p.p*(d - 2)*(a^.b^*p.p - a^.p*b^.p)/(32*(d - 1))",
          "a~b~c^e^": "-I*a~.b~*p.p*(d - 2)*(c^.e^*p.p - c^.p*e^.p)/(32*(d - 1))",
          "a~b~c~e~": "-I*a~.b~*c~.e~*p.p**2/32"
        },
        "g4_kept_continuation_mismatches": {
```

```
        "a^b^c~e~": "-I*d*c~.e~*p.p*(a^.b^*p.p - a^.p*b^.p)/(32*(d - 1)*(d + 1))",
        "a^b~c^e~": "I*b~.e~*p.p*(a^.c^*p.p - a^.p*c^.p)/(32*(d - 1)*(d + 1))",
        "a^b~c~e^": "I*b~.c~*p.p*(a^.e^*p.p - a^.p*e^.p)/(32*(d - 1)*(d + 1))",
        "a~b^c^e~": "I*a~.e~*p.p*(b^.c^*p.p - b^.p*c^.p)/(32*(d - 1)*(d + 1))",
        "a~b^c~e^": "I*a~.c~*p.p*(b^.e^*p.p - b^.p*e^.p)/(32*(d - 1)*(d + 1))",
        "a~b~c^e^": "-I*d*a~.b~*p.p*(c^.e^*p.p - c^.p*e^.p)/(32*(d - 1)*(d + 1))",
        "a~b~c~e~": "-I*p.p**2*(2*d*a~.b~*c~.e~ + a~.b~*c~.e~ - a~.c~*b~.e~ - a~.e~*b~.c~)/(32*(d - 1)*(d +
+> 1))"
      },
      "ghat_continuation_holds": false,
      "g4_kept_continuation_holds": false,
      "hatted_externals_agree_both": true
    }
  }
},
"D11_matching_verified_scope": "4D components: M2 discharged (P18_D11_M2, unchanged); evanescent components:
+>  EV-M2 proven (local poles + exact 4D decoupling of the renormalised remainder); NOT matched to XVI-O (no
+>  counterpart; both natural d-dim continuations refuted)",
"EV_CT": {
  "contract_revision": "p22: evanescent counterterms admitted outside the p14r 4D domain -- h.h operators O_
+> k = 1/2 h_mn S_k^{(mn)(rs)}(gt, g4, p) h_rs for the 8 gt-structures of the hh two-point function (D01 + D
+> 03 + D11), the K'.c operator K'^{mn} E2_{(mn)v} c^v (D13) and its ghost-equation partner cb_m SE^m_v c^v
+> (Sigma, gt p^4); coefficient = -(UV pole) (MS-bar, N_eps kappa^2 / eps normalisation of P14)",
  "coefficients_hh_counterterm_per_graph": {
    "D01": {
      "g4_mn gt_rs (p.p)^2": "-13*I/120",
      "g4_rs gt_mn (p.p)^2": "-13*I/120",
      "gt_mn gt_rs (p.p)^2": "I/240",
      "g4_mr gt_ns (p.p)^2": "3*I/80",
      "gt_mr gt_ns (p.p)^2": "I/240",
      "gt_mn p_r p_s (p.p)^1": "3*I/20",
      "gt_mr p_n p_s (p.p)^1": "-7*I/120",
      "gt_rs p_m p_n (p.p)^1": "3*I/20"
    },
    "D03": {
      "gt_mn p_r p_s (p.p)^1": "-I/12",
      "gt_mr p_n p_s (p.p)^1": "I/16",
      "gt_rs p_m p_n (p.p)^1": "-I/12",
      "g4_mn gt_rs (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "-I/24",
      "g4_rs gt_mn (p.p)^2": "I/24"
    },
    "D11": {
      "g4_mn gt_rs (p.p)^2": "I/120",
      "g4_rs gt_mn (p.p)^2": "I/120",
      "gt_mn gt_rs (p.p)^2": "I/480",
      "g4_mr gt_ns (p.p)^2": "-I/480",
      "gt_mr gt_ns (p.p)^2": "I/480",
      "gt_mn p_r p_s (p.p)^1": "-I/120",
      "gt_mr p_n p_s (p.p)^1": "I/480",
      "gt_rs p_m p_n (p.p)^1": "-I/120"
    }
  },
  "coefficients_hh_counterterm_sum": {
    "g4_mn gt_rs (p.p)^2": "-7*I/120",
    "g4_mr gt_ns (p.p)^2": "-I/160",
    "g4_rs gt_mn (p.p)^2": "-7*I/120",
    "gt_mn gt_rs (p.p)^2": "I/160",
    "gt_mn p_r p_s (p.p)^1": "7*I/120",
    "gt_mr gt_ns (p.p)^2": "I/160",
    "gt_mr p_n p_s (p.p)^1": "I/160",
    "gt_rs p_m p_n (p.p)^1": "7*I/120"
  },
  "coefficient_Kc_E2": "-1/16",
  "coefficient_ccb_SE": "1/8",
  "a_closure": {
    "h.c": {
      "p_dot_T_ev_per_graph": {
        "D01": {
          "(('g', 't', 'n', 'r'), ('v', 'p', 's'))": "I*p.p**2/48",
          "(('g', 't', 'n', 's'), ('v', 'p', 'r'))": "I*p.p**2/48",
          "(('g', 't', 'r', 's'), ('v', 'p', 'n'))": "-I*p.p**2/24"
        },
        "D03": {
          "(('g', 't', 'n', 'r'), ('v', 'p', 's'))": "-I*p.p**2/48",
          "(('g', 't', 'n', 's'), ('v', 'p', 'r'))": "-I*p.p**2/48",
          "(('g', 't', 'r', 's'), ('v', 'p', 'n'))": "I*p.p**2/24"
        },
        "D11": {}
      },
      "p_dot_T_ev_sum": {},
```

```
      "E_dot_E2": 0,
      "closed": true
    },
    "c.b": {
      "calibration_rho_from_4D_pair": "1/2",
      "image_ev_pair": {},
      "closed": true,
      "ghost_equation_exact_in_d_gt_sector": true
    },
    "c.cbs": "unreachable: S0's quadratic part is h.h, h.b, b.b, b.cbs (+ K.c, cb.c); sigma_0 of h.h, K'.c,
+> cb.c bilinears lands in h.c and c.b only",
    "full_closure_with_4D_counterterm": "closed: the 4D MS-bar counterterm is closed (P20_defect, unchanged)
+>  and sigma_0 is linear"
  },
  "b_projected_identity": {
    "hh_slot_values_sum": {
      "Q4S (declared, R2)": "0",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0"
    },
    "hh_slot_values_per_graph": {
      "D01": {
        "Q4S (declared, R2)": "-p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0"
      },
      "D03": {
        "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0"
      },
      "D11": {
        "Q4S (declared, R2)": "0",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0"
      }
    },
    "Kc_slot_values": {
      "Q4S (declared, R2)": "0",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0"
    },
    "unchanged": true
  },
  "c_remnants": {
    "finite_evanescent_remainder": "MS-bar finite parts (local + L) x gt-structures: recorded per structure
+> in P18_D11 / P16_batchC / P15_D13; NONZERO; identically zero on 4D (hatted) external legs, exactly in d (
+> four_d_decoupling below)",
    "O_eps_times_pole_at_this_order": "none with QdS (hatted) external legs: all four indices are external,
+> so tr gt = 4 - d never arises and the evanescent operators are ZERO on hatted legs at every d; see the Q4
+> S trace entry for the one place it can arise",
    "trace_of_pole_tensor_g4^{mn}T_mn,rs (hatted r,s)": {
      "per_graph": {
        "D01": "-I*p.p*(d - 4)*(13*a^.b^*p.p - 18*a^.p*b^.p)/120",
        "D03": "I*p.p*(d - 4)*(a^.b^*p.p - 2*a^.p*b^.p)/24",
        "D11": "I*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)/120"
      },
      "sum": "-7*I*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)/120"
    },
    "O_eps_times_pole_Q4S_trace_external": {
      "finite_term_if_external_trace_mode_uses_g4": "7*I*p.p*(a^.b^*p.p - a^.p*b^.p)/60",
      "rule": "(d - 4) x (1/eps) -> -2 (d = 4 - 2 eps); times the residue above",
      "status": "NONZERO: a finite local transverse term arises iff the external trace mode is defined with
+> the quasi-4D g4 (it then carries eps-scalar components); with QdS (hatted) external legs it is zero.  Thi
+> s is a scheme / external-state convention (DRED eps-scalar sector), disclosed: the p22 renormalised resul
+> t is stated for QdS external legs"
```

```
    },
    "where_O_eps_x_pole_would_enter": "insertions of the evanescent counterterms into loops (two loops) or i
  +> nto composite operators with traces: (4 - d) x (1/eps) = finite local terms -- beyond this order; not eva
  +> luated, not claimed"
   }
  },
  "mutations": {
   "drop_D03_evanescent_ct (h.c)": true,
   "drop_D01_evanescent_ct (h.c)": true,
   "D03_evanescent_ct_x2 (h.c)": true,
   "drop_SE_partner (c.b)": true,
   "flip_sign_E2_ct (c.b)": true,
   "trace_operator_in_place_of_s0h (slot)": true
  },
  "insensitive_variations": {
   "D11_evanescent_ct_x2 (h.c)": "p_m T_D11,ev = 0 by itself: the D11 evanescent counterterm is closed alone;
  +>  its normalisation is pinned by the D11 pins, not by closure",
   "detected": false
  },
  "controls": {
   "poles_equal_stored_D13_and_Sigma": true,
   "D11_evanescent_poles_equal_stored": true,
   "hh_evanescent_pole_sum_equals_P18_hh_slot": true,
   "ev_poles_local": true,
   "ev_four_D_decoupling_exact": true,
   "ev_D11_hatted_externals_agree": true,
   "hc_p_dot_T_ev_sum_zero": true,
   "hc_E_dot_E2_zero": true,
   "hc_D01_D03_ev_sum_transverse": true,
   "hc_D01_alone_not_transverse": true,
   "cb_calibration_unique_on_4D_pair": true,
   "cb_ev_pair_closed": true,
   "ghost_equation_ev_exact_in_d": true,
   "identity_unchanged_hh_slot_zero_all_ext": true,
   "identity_unchanged_kc_slot_zero_all_ext": true,
   "D01_ev_ct_alone_reaches_slot": true,
   "mutations_all_detected": true
  },
  "evanescent_counterterm_earned": true,
  "EV_M2_status": "proven",
  "EV_CT_status": "earned",
  "P5_calculation_status": "renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-bar in D
  +> RED with the p22 evanescent counterterms; external legs in QdS)",
  "scope": "one loop, kappa^2, the two-point functions entering the ch rows; NOT the one-point row beyond kapp
  +> a^1, not higher points, not two loops; not bulk symmetry, not O.1",
  "authorised_d_substitutions": {
   "evanescent: UV pole A(4) of an evanescent structure (after exact-d algebra)": 59
  },
  "seconds": 19.9
}
```


40 p21 layer record (release gate p21, revised in p22)

[listing 07/133] certificates/P21_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p21",
  "contract_revision": "p21 (p14r contract + disclosed p15/p16/p17/p18 revisions; none in p19, p20 or p21)",
  "date": "2026-10-09",
  "version_label": "XVI-P p21",
  "deliverable": "p21 small release (absolute D11 pin derived; gate, scan, disclosure fixes)",
  "execution_pass": true,
  "claim_status": "p21 small release from p20 (no scope expansion). D11 absolute pin: the sign and magnitude o
  +> f the D11 UV pole are derived from 1/2 Tr ln of the minimal scalar loop under the package's declared conv
  +> entions and equal the heat-kernel a_2 = C^2/120 + R^2/72 exactly (Gamma^(2)|_pole = kappa^2/(16 pi^2 eps)
  +>  [p^4 P2/240 + p^4 POs/24]); external inputs are mathematics only (a_2 coefficients, heat-kernel pole for
  +> mula), no external convention choice. Projected (ch-only) results as in p20; bulk symmetry NOT restored/c
  +> laimed; O.1 NOT_CLOSED.",
  "flag_definitions": {
   "projected_defect_computed": "Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from every evaluat
  +> ed input of the three surviving rows (none Missing), exact in d, pole + nonlocal + MS-bar local parts, al
  +> l seven external assignments",
   "projected_identity_verified": "Delta = 0 for the REGULATED one-loop functional Gamma1: pole, nonlocal and
  +>  local parts vanish for all seven assignments, exactly in d, with all controls true and every mutation ch
  +> anging the outcome",
   "projected_restoration_verified": "p20 definition (replaces p19's, which follows from the identity + local
  +> ity): FULL sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm S_ct = -(pole) in the decl
```

```
+> ared domain, h.h + c.K + c.cb (ghost self-energy Sigma held fixed at its P15 value, orientation converted
+> , derived relative factor +1), in every ghost +1 sector (h.c, c.b, c.cbs), together with projected_identi
+> ty_verified.  Scope: the one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1",
  "bulk_symmetry_restored": "hard stop: False regardless (not claimed; O.1 NOT_CLOSED)"
},
"projected_restoration_scope": "p20 definition: full sigma_0 closure (h.c, c.b, c.cbs) of the complete one-l
+> oop kappa^2 MS-bar counterterm incl. the c.cb ghost self-energy counterterm (held fixed, derived factor +
+> 1); NOT bulk symmetry, NOT O.1",
"route_consistency_control_p19_criterion": {
  "label": "route-consistency control (the p19 restoration criterion, NOT a flag): P_ch sigma_0(S_ct) = 0 fo
+> r the h.h + c.K MS-bar counterterm and the solve x = 0; implied by the identity + locality",
  "holds": true
},
"full_closure_mutation_detector": {
  "test": "full sigma_0 closure (all ghost +1 sectors) of the complete MS-bar counterterm, Sigma held fixed"
+> ,
  "cases": {
    "drop_graph_D11": {
      "closed": true,
      "entries": 0,
      "sectors": [],
      "detected": false
    },
    "drop_graph_D13 (c.K data = 0, Sigma held fixed)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "drop_graph_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "drop_graph_D01": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "flip_phase_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "symmetry_factor_D03_x2": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "Sigma_orientation_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    }
  },
  "reading": "detects drop D13 and flip K.c (the p19 blind spots) through the c.b sector, and the hh-row mut
+> ations through h.c; drop D11 stays undetected (D11 transverse => its pole is sigma_0-closed by itself): c
+> overed by M2 (P18) and the p20 D11 normalisation pin (P20_D11_pin.json)"
},
"p20_controls": {
  "orientation_ratio_uniform_minus_one": true,
  "tree_ghost_equation_V_cbc_eq_2iD_V_Kc": true,
  "tree_sigma0_SOKc_plus_SOcbc_zero": true,
  "derived_ghost_relation_reproduces_P15_sigma": true,
  "full_closure_detects_drop_D13_and_flip_Kc": true,
  "full_closure_detects_p19_orientation_error": true,
  "S_ct_without_ccb_not_closed (c.cb counterterm is needed)": true
},
"why_zero": "each surviving ch row vanishes separately in the projected slot; no counterterm cancellation an
+> d no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupl
```

```
+> ing -- D01 and D03 are individually nonzero and cancel against each other in the slot (p17/p18 transverse
+>  sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only throu
+> gh T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-poin
+> t row: the tadpoles D14a-c vanish under the declared prescription.  The defect is therefore zero without
+> any counterterm (solve x = 0).",
"rests_on": ["M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D compon
+> ents'; evanescent components of the D11 matching: not applicable, not verified)", "the evanescent decoupl
+> ing proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row decoupling)", "the frozen P.3 projection
+> and the P13 ch dependency ledger (three surviving rows)", "the declared prescription (DRED R1-R6, MS-bar,
+>  P14_prescription_D11_matching)"],
"findings": {
  "J1_p14r_counterterm_domain_field": "FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the p14r cou
+> nterterm-domain counts (Im sigma_0 3, quotient 8, D cap ker 5, P_ch sigma_0 rank 4) are artefacts of a re
+> /im split over Q (the densities carry explicit i for odd derivative counts, so i-multiples count as indep
+> endent).  The counts over Q(i) -- sigma_0 rank 5, D cap ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch
+>  sigma_0 rank 3 (= Route C) -- equal the real dimensions under a compatible real structure.  The p14r cer
+> tificate stays as released; both are reported",
  "K1_ghost_closure_sign": "p20 CORRECTION (erratum to p19): resolved -- an orientation bookkeeping error. W
+> ith the P15 Sigma coordinates converted to the route-G (c,cb) orientation (ratio -1), the derived factor
+> +1 gives full sigma_0 closure of the complete MS-bar counterterm in h.c, c.b, c.cbs.  The p19 text 'closu
+> re fails / needs -1 / unresolved' is retracted"
},
"K1_ghost_closure_convention": {
  "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
  "M_2iD_cK_to_cbc": [
    ["0", "2"],
    ["-1", "0"],
    ["0", "-1"]
  ],
  "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
  "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
  "derived_equals_P15": true,
  "relative_factor_derived": "1",
  "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
  "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
  "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
  "tree_controls": {
    "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
    "S0_Kc_terms": 16,
    "S0_cbc_terms": 40,
    "sigma0_S0Kc_plus_S0cbc_entries": 0,
    "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
    "ok": true
  },
  "full_closure_complete_S_ct": {
    "closed": true,
    "entries": 0,
    "sectors": []
  },
  "full_closure_S_ct_without_ccb (p19 package S_ct)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"]
  },
  "full_closure_unconverted (p19 error)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"],
    "detected": true
  },
  "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolved
+> ' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots).
+>  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
},
"scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the hh an
+> d K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement abou
+> t higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests ful
+> l sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c.b,
+> c.cbs) -- a statement about that counterterm only, not bulk symmetry, not O.1",
"release_claim_accepted": true,
"release_claim_accepted_definition": "lower layers re-checked live (contract, p18 incl. q earned, inventory,
+>  p19 layer, K1, retraction) AND p20 layer accepted (disclosed p21 revisions of d11_pin.py and release_p20
+> .py only) AND the absolute D11 pin AND every package change disclosed AND the pinned p20 independent reco
+> rd AND the p21 hard stops",
"publication": "accepted",
"publication_gated_on": ["execution_pass", "quantum_inputs_evaluated (p18 layer + recheck)", "projected_defe
+> ct_computed", "flag implications restoration => identity => computed"],
"flags": {
  "library_certified": true,
  "batchA_evaluated": {
    "execution": true,
```

```
    "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
+> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
+> V pole, finite part 0, no p14r counterterm required",
    "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
+>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
+> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
+> e; NOT quantum_inputs_evaluated"
  },
  "D13_evaluated": {
    "execution": true,
    "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
+> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
+> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
+> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
    "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
+> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
+> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
+> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
  },
  "batchC_evaluated": {
    "execution": true,
    "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
+> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
+> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
+> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
    "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
+>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
+> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
  },
  "E2_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof)",
    "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
+>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
  },
  "hh_evanescent_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
    "scope": "the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of D01
+>  + D03 (D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green function
+>  outside this projection; D11 (same row) NOT evaluated"
  },
  "D11_matching_M2": {
    "execution": true,
    "claim": "DISCHARGED (4D components)",
    "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the
+>  4D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0
+> is IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns
+>  the classical insertion and is not touched"
  },
  "hh_slot_full_row_decoupled": {
    "execution": true,
    "claim": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
    "scope": "row (S0,Gamma1^[2]) s2 K:K.c; D01 + D02 + D03 + D11 + D12; the summed evanescent poles remain
+> (transverse) UV poles of the hh Green function outside this projection"
  },
  "projected_ch_identity": {
    "execution": true,
    "claim": "ZERO",
    "scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the h
+> h and K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement
+> about higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests
+>  full sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c
+> .b, c.cbs) -- a statement about that counterterm only, not bulk symmetry, not 0.1"
  }
},
"projected_defect_computed": "earned",
"projected_identity_verified": "earned",
"projected_restoration_verified": "earned",
"projected_defect_summary": {
  "defect_zero_exact_in_d": true,
  "evanescent_in_assembly": "verified explicitly: the evanescent (gt) parts of the hh row (D01+D03+D11) and
+> of the K.c row (D13 E1, E2) give ZERO for all seven external assignments, exactly in d; the evanescent po
+> les themselves are nonzero (P18_hh_slot, P15_D13) -- they drop from the ch slots by the decoupling proofs
+> , not by convention",
  "quotient_coordinates": {
    "complement_directions": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"],
    "pole": ["0", "0", "0", "-3/8", "1/4"],
    "nonlocal_L": ["0", "0", "0", "3/8", "-1/4"],
    "local": ["0", "0", "0", "-2/3", "5/12"],
```

```
    "reading": "coordinates modulo D cap Im sigma_0 (field redefinitions / s-exact); the h.h pole is transve
+> rse, i.e. in D cap Im sigma_0, so its quotient coordinates vanish"
  },
  "D_cap_ker_sigma0": {
    "computed": true,
    "dim": 6,
    "dim_D_cap_Im_sigma0": 6,
    "cohomology_dim": 0,
    "sector_ranks": {
      "h.h": {
        "dim": 5,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 3
      },
      "c.cb": {
        "dim": 2,
        "sigma0_rank": 2,
        "P_ch_sigma0_rank": 0
      },
      "c.K": {
        "dim": 3,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 1
      },
      "phi.phi": {
        "dim": 1,
        "sigma0_rank": 0,
        "P_ch_sigma0_rank": 0
      }
    }
  },
  "field_note": "Route G is computed over Q(i) (primary) and, for comparison, over Q with the re/im split us
+> ed by the p14r domain certificate.  The p14r densities carry explicit factors i for odd derivative counts
+>  (c.K: 3 derivatives), so over Q an i-multiple counts as independent: e.g. the ch image of c.K#0 is i tim
+> es an h.h image.  The counts over Q(i) equal the real dimensions under a compatible real structure (one r
+> eal coefficient per density, the i fixed by reality of the action); ranks and solvability are quoted over
+>  Q(i)."
},
"counterterm_solve": {
  "system": "Delta + P_ch sigma_0(x) = 0, x in the declared domain D (11 directions), part by part",
  "rank_P_ch_sigma0_routeG": 3,
  "rank_routeC": 3,
  "routeC_image_relations": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "outcome": "ZERO",
  "residual": null,
  "solution": "x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite renorma
+> lisations are ker(P_ch sigma_0) cap D (dim 8), of which 2 modulo D cap Im sigma_0",
  "MSbar_counterterm_ch_image_zero": true,
  "MSbar_counterterm_full_sigma0_image_sectors": ["c.b"],
  "full_closure_relative_factor_table_UNCONVERTED_p19_error": {
    "1": false,
    "-1": true,
    "I": false,
    "-I": false
  },
  "full_closure_reading": "p20 (corrects p19): with the P15 Sigma coordinates converted from the (cb,c) to t
+> he route-G (c,cb) orientation (measured ratio -1), the DERIVED factor +1 gives full sigma_0 closure of th
+> e complete MS-bar counterterm (h.h + c.K + c.cb) in every ghost +1 sector; the p19 table above (factors 1
+> , -1, i, -i on UNCONVERTED coordinates) is the bookkeeping error, kept for the erratum",
  "K1_ghost_closure_convention": {
    "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
    "M_2iD_cK_to_cbc": [
      ["0", "2"],
      ["-1", "0"],
      ["0", "-1"]
    ],
    "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
    "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
    "derived_equals_P15": true,
    "relative_factor_derived": "1",
    "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
    "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
    "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
    "tree_controls": {
      "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
```

```
      "S0_Kc_terms": 16,
      "S0_cbc_terms": 40,
      "sigma0_S0Kc_plus_S0cbc_entries": 0,
      "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
      "ok": true
    },
    "full_closure_complete_S_ct": {
      "closed": true,
      "entries": 0,
      "sectors": []
    },
    "full_closure_S_ct_without_ccb (p19 package S_ct)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"]
    },
    "full_closure_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolv
+> ed' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots)
+> .  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
  },
  "note_full_sigma0": "the h.h + c.K MS-bar counterterm ALONE has a nonzero (c, b) image (delta S0/delta h c
+> arries b); adding the c.cb ghost self-energy counterterm (correct orientation, derived factor +1) closes
+> it: see K1_ghost_closure_convention.full_closure_complete_S_ct"
},
"defect_controls": {
  "inventory_complete_no_missing": true,
  "missing_input_refused_not_zero": true,
  "s0h_longitudinal": true,
  "E_transverse": true,
  "hh_row_zero_exact_in_d_all_7": true,
  "hh_row_evanescent_part_zero_all_7": true,
  "hh_row_D01_D03_individually_nonzero": true,
  "Kc_row_zero_exact_in_d_all_7": true,
  "Kc_row_evanescent_part_zero_all_7": true,
  "Kc_slot_map_nontrivial_T1_E1": true,
  "E2_alone_zero_all_7": true,
  "D13_A_T1_A_E1_zero_exact": true,
  "onepoint_row_D14abc_vanishing": true,
  "defect_zero_pole_nonlocal_local_all_7": true,
  "defect_independent_of_row_signs": true,
  "routeG_S0_check": true,
  "routeG_defect_zero_by_sector": true,
  "routeG_hh_local_data_in_span": true,
  "routes_agree_on_ch_image_rank": true,
  "routeG_hh_kernel_equals_p18_transverse_subspace": true,
  "D_cap_Im_inside_D_cap_ker_sigma0": true,
  "D_cap_Im_inside_ker_P_ch_sigma0": true,
  "ghost_equation_relation_reproduces_P15_sigma_pole": true,
  "MSbar_counterterm_in_ker_P_ch_sigma0": true,
  "mutations_all_change_outcome": true,
  "routeG_Q_split_reproduces_p14r_counts": true
},
"defect_mutations": {
  "drop_graph_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/48",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the ghost loop D03"
  },
  "drop_graph_D01": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
```

```
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the graviton bubble D01"
  },
  "flip_phase_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/24",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/24",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/8"
      }
    },
    "note": "relative sign of D03 flipped"
  },
  "symmetry_factor_D03_x2": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "D03 doubled"
  },
  "wrong_projection_trace_operator_hh": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-3*c^.p*p.p*(6*a.b*p.p - 6*a^.p*b^.p - a~.b~*p.p)/16",
        "QdS (hat)": "-9*c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/8",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-15*a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "eta_mn p_v in place of s0 h (not the P.3 projection)"
  },
  "wrong_projection_gauge_fixed_operator_Kc": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p*(a.b*c^.p*p.p - 3*a.c*b^.p*p.p - 3*a^.p*b.c*p.p + 4*a^.p*b^.p*c^.p - a^.p
+> *b~.c~*p.p - a~.c~*b^.p*p.p)/16",
        "QdS (hat)": "p.p*(a^.b^*c^.p*p.p - 3*a^.c^*b^.p*p.p - 3*a^.p*b^.c^*p.p + 4*a^.p*b^.p*c^.p)/16",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/4",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
```

```
      }
    },
    "note": "de Donder q^2 P in place of the h^2-monomial operator E (not the P.3 projection)"
  },
  "naive_4D_D03_evanescent_dropped": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/24"
      }
    },
    "note": "D03 Gt parts dropped"
  },
  "naive_4D_D13_Gt_dropped": {
    "outcome": "NONZERO, CANCELLABLE by a counterterm in the declared domain",
    "changes_outcome": true,
    "residual": {
      "local": {
        "Q4S (declared, R2)": "-c^.p*p.p*(a.b*p.p - a^.p*b^.p)/12",
        "QdS (hat)": "-c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/12",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/12"
      }
    },
    "note": "D13 evaluated with Gt dropped (P15_D13 mutation fit: T1 = -(d-4)/(16(d-1)))",
    "finite_counterterm": {
      "rank": 3,
      "solvable": true,
      "solution": ["(6*x3 + 12*x4 - 24*I*x5 - I)/12", "-x3/2", "-(12*x3 + 24*x4 - 24*I*x5 - I)/12", "x3", "x
+> 4", "x5", "x6", "x7"]
    }
  }
},
"defect_insensitive_variations": {
  "drop_graph_D11": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "D11 is transverse by itself (p_m G_D11 = 0 exactly in d): the slot cannot see it; covered
+> by M2 (P18) and by the inventory refusal below"
  },
  "drop_graph_D13": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "A_T1 = A_E1 = 0 and T2, T3, E2 are longitudinal: D13 reaches the slot with zero; covered b
+> y the D13 certificate (two routes, mutations)"
  },
  "flip_row_sign_Kc": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "each row vanishes separately"
  }
},
"quantum_inputs_evaluated": "earned",
"quantum_inputs_inventory": [
  {
    "graph": "D01",
    "slot_row": "(SO,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D02",
    "slot_row": "(SO,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D03",
    "slot_row": "(SO,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
```

```
    "status": "regulated"
  },
  {
    "graph": "D11",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"
  },
  {
    "graph": "D12",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D13",
    "slot_row": "(S0,Gamma1^[2]) s1 h:h^2",
    "evaluated": true,
    "status": "evaluated"
  },
  {
    "graph": "D14a",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14b",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14c",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  }
],
"D11_matching_verified": true,
"D11_matching_verified_scope": "4D components",
"D11_evanescent_components_matching": "not_applicable_not_verified",
"M3_status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
"D11_evaluated": true,
"hh_full_row_decoupling_proven": true,
"evanescent_hh_counterterm_added": false,
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"hh_evanescent_status": "decoupled_from_ch_slot_proven_as_sum",
"hh_evanescent_decoupling_proven": true,
"D01_evaluated": true,
"D03_evaluated": true,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
```

```
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
```

```
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D11": {
    "status": "evaluated",
    "certificate": "certificates/P18_D11.json",
    "sector_status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
    "M2": "DISCHARGED (4D components)",
    "ch_slot": "decoupled by itself (p_m G_D11 = 0 exactly in d)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_byte_identical_to_p18": true,
  "dred_registry_state_p22": "p18 + p22 guard block",
  "contract_revision_in_p19": false,
  "disclosed_revisions_p19": {}
},
"p18_layer": {
  "p18_layer_files_byte_identical": true,
  "P18_release_accepted": true,
  "P18_quantum_inputs_evaluated": "earned",
  "I1_gate": "the p19 gate requires the p18 layer record's quantum_inputs_evaluated == 'earned'"
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
```

```
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
    "C7_grav_noether_k0_k1_k2_on_exports": true,
    "quartic_mutations_all_rejected": true,
    "master_grav_noether_deterministic_k0_k1_k2": true,
    "master_engine_parts_exact": true,
    "graviton_hessian_generic_symbolic_k": true,
    "canonicity_all_23_antifield_components": true,
    "ch_enumeration_lemma_mechanical": true,
    "residual_record_invariants_all_certificates": true,
    "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
    "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
    "cross_certificate_quartic_vs_master_agree": true,
    "stored_C7_flag_equals_recomputation": true
  },
  "failed_prerequisites": [],
  "hash_consistency": {
    "consistent": true,
    "ell": {
      "ell2": {
        "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
      },
      "ell3": {
        "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
      },
      "ell4": {
        "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
        "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
        "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    }
  },
  "required_master_residuals": [],
  "residual_records": {
    "invariant_violations": {},
    "quartic_C7_per_order": {
      "kappa0": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      },
      "kappa1": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      },
      "kappa2": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      }
    },
    "master_grav_per_order": {
      "kappa0": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      },
      "kappa1": {
        "residual_counts": [0, 0, 0, 0],
```

```
        "zero_flag": true,
        "accepted": true
      },
      "kappa2": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      }
    },
    "cross_certificate": {
      "same_inputs": true,
      "differences": [],
      "agree": true,
      "quartic_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      },
      "master_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    }
  },
  "hard_stops_p13_hold": true,
  "hard_stops_p13": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"hard_stops": {
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
```

```
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "evaluated; 4D sector renormalized (MS-bar, declared domain); evanescent regulated (p
+> oles retained, not subtracted), decoupled from the ch slots (p15-p18 inputs D01, D02, D03, D11, D12, D13,
+>  D14a-c)",
    "P.5_identity": "projected defect computed (zero exactly in d, all parts, all seven assignments; each ro
+> w vanishes separately); projected identity verified; projected restoration verified (p20 definition: full
+>  sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm incl. c.cb, h.c/c.b/c.cbs); K1 corre
+> cted in p20 (orientation erratum); D11 ratio pin verified; D11 absolute pin verified (p21); bulk symmetry
+>  NOT restored/claimed; O.1 NOT_CLOSED [p21: D11 absolute pin derived and verified]"
  },
  "definition_vs_status_note": "I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficients evalua
+> ted and renormalized'; the STATUS states exactly what holds: 4D sector renormalized, evanescent poles reg
+> ulated and decoupled from the ch slots (not subtracted)",
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)"
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p18": true,
"independent_rebuild_passed_p17": true,
"independent_rebuild_passed_p16": true,
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p21) sourc
+> e tree only (independent: pending until a reviewer report for this tree exists); independent_rebuild_pass
+> ed_p20 refers to the p20 tree 2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700 / zip ca32
+> ce018933f12d1a623217d6a98d647817360dd12211e2e9469cf850a63bb2 (same-box caveat) and is not inherited by p2
+> 1",
"p18_independent_rebuild": {
  "report": "archival/p18/reproducibility/independent_rebuild_report_p18.json",
  "present": true,
  "report_sha256": "893f167ef7124d981c0569ca386035aed06e0dbb0c90a2362a6ceeafd1930cb2",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094",
  "bound_to_source_zip_sha256": "7c7d5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f",
  "tree_matches_p18": true,
  "zip_matches_p18": true,
  "mismatches": 0,
  "findings": ["I1_release_p18_publication_not_gated_on_quantum_inputs_evaluated", "I2_scan_pattern_gaps", "
+> I3_scope_of_M2_and_evaluated", "I4_table_row_count", "I5_slot_blind_to_D11_normalisation", "I6_test_p16_v
+> ictim_restriction", "I7_pdftotext_dehyphenation", "H1_H5_resolution", "bug_fixes_disclosed"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p19": "bound to the p18 tree/zip; the p19 tree has its own (pending) independent flag",
  "archived_p18_records_pinned": {
    "archival/p18/reference_values_p18.json": true,
    "archival/p18/reproducibility/author_rebuild_report_p18.json": true,
    "archival/p18/reproducibility/independent_rebuild_report_p18.json": true,
    "archival/p18/p18_source_files.json": true
  },
  "pins_ok": true
},
"p17_independent_rebuild": {
  "report": "archival/p17/reproducibility/independent_rebuild_report_p17.json",
  "present": true,
  "report_sha256": "92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
  "bound_to_source_zip_sha256": "7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416",
  "tree_matches_p17": true,
  "zip_matches_p17": true,
  "mismatches": 0,
  "findings": ["H1_static_scan_pattern_gaps", "H2_assignment_table_omits_c_hat_h_tilde", "H3_stale_comments"
+> , "H4_curly_apostrophes_in_typeset_listings", "H5_parse_d_srepr", "H6_scope", "G1_G2_G3_G4_resolution"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p18": "bound to the p17 tree/zip; the p18 tree has its own (pending) independent flag",
  "archived_p17_records_pinned": {
    "archival/p17/reference_values_p17.json": true,
    "archival/p17/reproducibility/author_rebuild_report_p17.json": true,
    "archival/p17/reproducibility/independent_rebuild_report_p17.json": true,
```

```
    "archival/p17/p17_source_files.json": true
  },
  "pins_ok": true
},
"p16_independent_rebuild": {
  "report": "archival/p16/reproducibility/independent_rebuild_report_p16.json",
  "present": true,
  "report_sha256": "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
  "bound_to_source_zip_sha256": "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57",
  "tree_matches_p16": true,
  "zip_matches_p16": true,
  "mismatches": 0,
  "findings": ["G1_static_scan_scope_narrower_than_disclosure", "G2_plain_symbol_d_from_stored_strings", "G3
+> _hook_coverage_detail", "G4_change_log_omits_new_reference_file", "G5_test_p15_assertion_relaxed", "G6_im
+> aginary_normalisation", "G7_scope", "F3_F4_F5_from_p15"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent flag",
  "archived_p16_records_pinned": {
    "archival/p16/reference_values_p16.json": true,
    "archival/p16/reproducibility/author_rebuild_report_p16.json": true,
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": true,
    "archival/p16/p16_source_files.json": true
  },
  "pins_ok": true
},
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
```

```
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"M2_summary": {
  "verdict": "DISCHARGED (4D components)",
  "D11_matching_verified_scope": "4D components",
  "evanescent_components_matching": {
    "status": "not_applicable_not_verified",
    "what_they_are": "the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt): compo
+> nents of the hh two-point function with at least one index pair in the epsilon-scalar (4-d) directions of
+>  DRED; they vanish on hatted 4D components",
    "why_not_matched": "XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent counte
+> rpart; M2 is a 4D statement, so there is nothing to match them to. Their L-coefficients are nonzero (reco
+> rded below); they are retained (R6) and drop out of the ch-projected slot (p_m G_D11 = 0 for the evanesce
+> nt part separately)"
  },
  "M3_status": {
    "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
    "statement": "L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree <= 8 i
+> t is a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)",
    "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching)"
  },
  "statement": "M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR_AB],
+>  f> = <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18",
  "route_B": {
    "equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair",
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 0,
    "first": [],
    "agree": true,
    "constant": "-I/4"
  },
  "FT_constant": {
    "master_pole": "1",
    "master_finite": "2 - L",
    "pole_is_1": true,
    "L_coefficient_of_MB": "-1",
    "c_FT_from_DR_bubble": "I*pi**2",
    "c_FT_from_minus_r": "I*pi**2",
    "r_certificate": "-i * pi**2",
    "r_certificate_as_expected": true,
    "agree": true
  },
  "controls": {
    "route_B_all_100_pairs_agree": true,
    "FT_constant_two_calibrations_agree": true,
    "route_A_M1_integrand_identity": true,
    "route_A_XVI_O_K_u_equals_T": true,
    "D11_transverse_exact_in_d": true,
    "routes_FP_PV_agree": true,
    "covariant_fit_exact_unique": true,
    "mutations_all_rejected": true
  },
  "analytic_inputs_cited": ["Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, resi
+> due at zeta = -2 is r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bub
+> ble)", "a polynomial in p is the Fourier transform of a distribution supported at {0}", "Route A only: tw
+> o extensions of the product V V (C C + C C) from U differ by a distribution supported at 0 (XVI-O lem:BV_
+> rules_on_U, Constructed)"],
  "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the 4
+> D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0 is
+>  IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns t
+> he classical insertion and is not touched",
  "M3_consequence": "with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both sides a
+> re transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contacts, consistent with
+> N4), so L_match lies in the 2-dim transverse subspace of the p14r h.h span; its coordinates need the XVI-
```

```
+> 0 finite constant of u^R and are NOT computed (M4: booked as P-side hh counterterm; u^R_mu unmodified, no
+>  c delta)"
},
"D11_summary": {
  "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
  "prescription": "operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions, M3 bounds
+>  4/8/4, exactly-degree-four normalization (massless: every local term is a homogeneous quartic in p)",
  "prescription_usage": {
    "exactly_degree_four_normalization": "every structure S_s (p.p)^e of Ghat has total momentum degree 4 (c
+> omputed per structure: g4_mn g4_rs (p.p)^2: 4, g4_mn gt_rs (p.p)^2: 4, g4_mn p_r p_s (p.p)^1: 4, g4_mr g4
+> _ns (p.p)^2: 4, g4_mr gt_ns (p.p)^2: 4, g4_mr p_n p_s (p.p)^1: 4, g4_rs gt_mn (p.p)^2: 4, g4_rs p_m p_n (
+> p.p)^1: 4, gt_mn gt_rs (p.p)^2: 4, gt_mn p_r p_s (p.p)^1: 4, gt_mr gt_ns (p.p)^2: 4, gt_mr p_n p_s (p.p)^
+> 1: 4, gt_rs p_m p_n (p.p)^1: 4, p_m p_n p_r p_s: 4)",
    "N4_contact_exclusions": "used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by N4 do not
+> arise (massless; every pole and local MS-bar term is a homogeneous quartic in p), so N4 removes nothing h
+> ere and no order-0/2 counterterm is needed",
    "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"
  },
  "UV_pole_in_p14r_hh_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
  "evanescent_pole_nonzero": true,
  "controls": {
    "exactly_degree_four_every_structure": true,
    "N4_no_order_0_or_2_contact_arises": true,
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "integrand_equals_XVI_0_convolution_M1": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true,
    "M2_discharged": true,
    "mutations_all_rejected": true
  }
},
"hh_slot_summary": {
  "verdict": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
  "row": "(S0,Gamma1^[2]) s2 K:K.c",
  "graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
  "how_D11_decouples": "graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse by i
+> tself, 4D and evanescent parts separately; its evanescent poles are nonzero but transverse. Hence D11 mut
+> ations of phase or symmetry factor are NOT detectable by the slot (they are tested by M2 instead); the sl
+> ot sees D11 only through a non-longitudinal operator (trace operator control: nonzero)",
  "surviving_evanescent_structures_in_slot": [],
  "consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop (D11 in
+> cluded), so NO contract revision is made for it.  The summed evanescent poles of the hh Green function ar
+> e nonzero but transverse; renormalising the hh Green function itself would need evanescent h.h counterter
+> ms -- outside this projection and NOT claimed.",
  "controls": {
    "operator_longitudinal": true,
    "p_dot_G_D11_zero_exact_in_d": true,
    "p_dot_G_sum_zero_exact_in_d": true,
    "p_dot_G_evanescent_sum_zero": true,
    "slot_sum_zero_all_7_assignments": true,
    "evanescent_slot_sum_zero_all_7_assignments": true,
    "D01_D03_individually_nonzero_incl_c_hat_h_tilde": true,
    "D11_reaches_slot_under_trace_operator": true,
    "4D_pole_sum_D01_D03_D11_transverse": true,
    "4D_pole_D11_transverse": true,
    "mutations_all_rejected": true
  }
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
```

```
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
```

```
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p21.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "b835353ffe16a8ff159e2916fe3efdf2b9d89a4d512e80e9e6da587cecd0fe5e",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "vendored",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p21.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "b835353ffe16a8ff159e2916fe3efdf2b9d89a4d512e80e9e6da587cecd0fe5e",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": false,
    "inherited_mode": null,
    "environment": {
      "python": "3.13.5",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
      "pip_freeze": ["iniconfig==2.3.1", "mpmath==1.3.0", "packaging==26.3", "pluggy==1.6.0", "Pygments==2.2
+> 1.0", "pytest==9.1.1", "sympy==1.14.0"]
```

```
    }
  },
  "p20_archive": "archival/p20",
  "kind": "package evidence (XVI-0 conv:package); not a mathematical certificate"
},
"XVI_P_registered": false,
"XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
"XVI_M_pin_ok": true,
"symmetry_restored": false,
"bulk_symmetry_restored": false,
"01_closure": "NOT_CLOSED",
"insertion_status": "Frozen_negative_obstruction_preserved",
"XV_B_opened": false,
"XIV_J_opened": false,
"Route_B_discharged": false,
"u_R_c_delta_added": false,
"inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
  "P20_release.json": "8511e602d2f72f3a64deb22ffa60bc1ccf1c876bb06d68b6c91a9d9ac6e296e2",
  "P20_D11_pin.json": "d4966cca9fc96634b235ee99f34086bbd1d555e5781fd0127fafa55d18533ad6"
},
"p19_layer_disclosed_test_revisions_p19": {
  "tests/test_p14.py": "names p18 -> p19 (reference file, source zip, author report, status revision); gener
+> ated prefixes certificates/ledgers P19_, manuscript/body/P19-",
  "tests/test_p18.py": "rs() reads the p18 LAYER record certificates/P18_release.json; the live p18 independ
+> ent report is bound to the p18 tree (pending or False here); test-file change log accepts p18 or disclose
+> d p19 revisions; build label is the current revision"
},
"D11_ratio_pin_verified": "earned",
"D11_absolute_pin_verified": "earned",
"p18_layer_byte_identical": true,
"p20_layer_change_log_vs_p19": {
  "base": "archival/p19/p19_source_files.json (pinned; files of the p19 zip sha256 da9242752d2e3753d8b2ce7fb
+> b21e7cdb28487cc7966c2ca52d1ef1376658875)",
  "base_pinned_ok": true,
  "added": ["archival/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p
+> 19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/in
+> dependent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "arch
+> ival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "arch
+> ival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p2
+> 1_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebu
+> ild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/R
+> EADME.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archiv
+> al/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archiv
+> al/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p
+> 5_calc/d11_pin.py", "code/p5_calc/evanescent.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_co
+> nventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_
+> counting.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py"
+> , "code/p5_calc/release_p23.py", "manuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pi
+> n.tex", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_q
+> uantum_target.json", "reproducibility/reference_values_p23.json", "tests/test_p20.py", "tests/test_p21.py
+> ", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p19.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/defect.py", "code/p5_calc/release_p18.p
+> y", "code/p5_calc/release_p19.py", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.tex", "manuscript
+> /notes/P19_defect.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "tests/test_p18.py
+> ", "tests/test_p19.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": [],
  "counts": {
    "added": 43,
    "removed": 1,
    "modified": 16
  }
},
"p20_layer_disclosed_test_revisions_p20": {
  "tests/test_p14.py": "names p19 -> p20; generated prefixes P20_",
  "tests/test_p19.py": "K1 tests corrected (orientation, full closure with derived +1); restoration on the p
+> 20 definition; J1 wording; p19 independent report bound to the p19 tree; scan sites = p19 + p20 reviewed
+> lists"
},
"p20_layer_disclosed_p19_layer_revisions_p20": {
  "code/p5_calc/defect.py": "K1 orientation fix (measured ratio, Sigma converted to route-G (c,cb)); tree co
+> ntrols; full sigma_0 closure of the complete MS-bar S_ct; restoration REDEFINED; full-closure mutation de
+> tector; p19 criterion relabelled route-consistency control; J1 wording; K1 erratum texts",
  "code/p5_calc/release_p19.py": "claim texts: K1 erratum (p19 'closure fails / needs -1' retracted); restor
+> ation content checks on the p20 definition; reject removes stale RELEASE_STATUS.json"
},
"p20_layer_p19_layer": {
  "p19_conventions_byte_identical": true,
  "disclosed_p20_revisions": {
    "code/p5_calc/defect.py": {
      "sha256": "675c1fa812c51b9955dd48a4149d309ff2e99ec7797fad753eb57b37e908ce6f",
```

```
      "differs_from_p19": true,
      "disclosed": true
    },
    "code/p5_calc/release_p19.py": {
      "sha256": "4d048345270a39a77cc30f71d3bdcbef17fffec6c44c1ac89487135ea3555970",
      "differs_from_p19": true,
      "disclosed": true
    }
  },
  "P19_release_accepted": true,
  "lower_layers_rechecked_p21": {
    "p18_layer_ok": true,
    "P18_quantum_inputs_evaluated": "earned",
    "inventory_recheck_ok": true
  }
},
"p20_layer_erratum_p19": {
  "K1": "p19 stated that full sigma_0 closure of the MS-bar counterterm fails with the derived ghost-equatio
+> n factor +1 and needs -1 (unresolved). RETRACTED: Grassmann-orientation bookkeeping error -- the P15 Sigm
+> a coordinates are (cb,c)-oriented, the route-G c.cb slots (c,cb), V[c.cb#k] = -V[(cb,c)#k]. Corrected: Si
+> gma = (+3/8, -1/4) in (c,cb) coordinates, derived +1, full closure in h.c, c.b, c.cbs.",
  "restoration": "p19's projected_restoration_verified (P_ch sigma_0(S_ct) = 0, x = 0) followed from the ide
+> ntity + locality; REDEFINED in p20 as full sigma_0 closure of the complete MS-bar S_ct (incl. c.cb), kept
+>  as a labelled route-consistency control",
  "J1": "'over Q(i), the physical field' replaced by: the Q(i) counts are the real dimensions under a compat
+> ible real structure",
  "found_by": "p19 independent rebuild (report sha256 1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf00
+> 8dd2811f45)"
},
"p20_layer_K1_correction": {
  "content_revision_p20": true,
  "orientation_ratio_minus_one": true,
  "Sigma_pole_routeG_c_cb": true,
  "derived_factor_plus_one": true,
  "derived_relation_reproduces_P15": true,
  "full_closure_complete_S_ct": true,
  "without_ccb_not_closed": true,
  "tree_controls_ok": true,
  "p20_controls_all_true": true,
  "detector_drop_D13": true,
  "detector_flip_Kc": true,
  "detector_drop_D11_recorded_blind": true,
  "route_consistency_control_holds": true,
  "quotient_incl_Sigma_zero": true,
  "quotient_Sigma_alone_c_cb_plus_3_8_minus_1_4": true,
  "J1_wording": true,
  "restoration_definition_p20": true,
  "bulk_false": true
},
"p20_layer_retraction_check": {
  "phrases": ["full c.b / ghost-sector closure FAILS", "full c.b / ghost-sector closure of the MS-bar counte
+> rterm FAILS", "for which full closure fails", "with +1 full closure fails", "full closure \\emph{fails}",
+>  "This is unresolved", "This is not resolved in p19", "needs relative factor -1; the derived ghost-equati
+> on relation gives +1; not resolved", "factor +1 (K1) full c.b closure FAILS", "full c.b closure NOT estab
+> lished (K1)", "the physical field"],
  "scope": ["README.md", "manuscript/notes/P19_defect.tex", "certificates/P19_defect.json", "certificates/P1
+> 9_release.json", "manuscript/body/P19-results-table.tex"],
  "hits": {}
},
"p20_layer_D11_pin": {
  "check": {
    "ratio_pin_PASS": true,
    "ratio_10": true,
    "transverse": true,
    "C2_quadratic_form_half_P2": true,
    "GaussBonnet_zero": true,
    "absolute_pin_PASS": true,
    "absolute_pin_flag": true,
    "top_level_absolute_status_consistent": true,
    "top_level_ratio_status_consistent": true,
    "absolute_ratios_one": true,
    "no_external_convention_choice": true,
    "drop_D11_detected": true,
    "x2_detected_by_absolute": true,
    "sign_flip_detected_by_absolute": true,
    "corrected_in_p21": true,
    "literature_source_stated": true,
    "revision": true
  },
  "ratio_pin": {
    "status": "PASS",
```

```
    "statement": "spin-0 : spin-2 of the D11 4D UV pole = 10 : 1, derived from the certificate; equals the m
+> inimal-scalar a_2 prediction (1/72)(3) : (1/120)(1/2)"
  },
  "absolute_pin": {
    "status": "PASS",
    "observed_common_factor_c2_vs_bare_a2": "-I",
    "observed_common_factor_c0_vs_bare_a2": "-I",
    "derivation_p21": {
      "package_declarations_checked": {
        "N_eps = i S_eps/(16 pi^2)": true,
        "Gamma_1 = -i * graph sum": true,
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
        "g = eta + kappa h": true,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
        "eta = (+,-,-,-)": true,
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
      },
      "steps": {
        "1_G_equals_Gamma2": "1",
        "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
        "3_i_S_M_over_S_E (must be -1)": "-1",
        "4_L_M_eff_over_L_E_eff": "-1",
        "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
        "6_minkowski_quadratic_forms": {
          "Riem2": {
            "P2": "1",
            "P0s": "1"
          },
          "Ric2": {
            "P2": "1/4",
            "P0s": "1"
          },
          "R2": {
            "P2": "0",
            "P0s": "3"
          },
          "C2": {
            "P2": "1/2",
            "P0s": "0"
          },
          "E_GaussBonnet": {
            "P2": "0",
            "P0s": "0"
          }
        }
      },
      "package_Gamma2_pole_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "heat_kernel_prediction_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "ratio_package_over_prediction": {
        "spin2": "1",
        "spin0s": "1"
      },
      "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2
+>  (Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps
+> )) int a_2 (Euclidean, d = 4-2eps)"],
      "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, s
+> ign of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
      "status": "PASS"
    },
    "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under th
+> e package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the W
+> ick direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(
+> 16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1
+> , both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no ext
+> ernal convention choice.  p20 had left this OPEN."
  },
  "spin_decomposition": {
    "c2_spin2": "-I/240",
    "c0_spin0s": "-I/24",
    "projector_norms": ["5", "1"],
    "transverse": true,
    "only_spin2_and_spin0s": true,
    "ratio_c0_over_c2": "10"
  },
```

```
  "mutations": {
    "drop_graph_D11": {
      "c2": "0",
      "c0": "0",
      "ratio": "None",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_normalisation_x2": {
      "c2": "-I/120",
      "c0": "-I/12",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_sign_flip": {
      "c2": "I/240",
      "c0": "I/24",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "swap_g4g4_structures": {
      "c2": "-I/40",
      "c0": "-I/32",
      "ratio": "5/4",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    }
  }
},
"p20_layer_open_pins": [],
"p20_layer_open_pins_closed_in_p21": ["D11_absolute_normalisation (Ghat -> a_2 map not derived)"],
"p20_layer_p19_independent_rebuild": {
  "report": "archival/p19/reproducibility/independent_rebuild_report_p19.json",
  "present": true,
  "report_sha256": "1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf008dd2811f45",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36",
  "bound_to_source_zip_sha256": "da9242752d2e3753d8b2ce7fbb21e7cdb28487cc7966c2ca52d1ef1376658875",
  "tree_matches_p19": true,
  "zip_matches_p19": true,
  "mismatches": 0,
  "findings": ["K1_ghost_closure_sign_is_orientation_bookkeeping", "quotient_coordinates_c.cb_orientation",
+> "restoration_flag_not_independent", "mutation_criterion_relabelled", "J1_wording", "I2_scan_misses", "gat
+> e_stale_RELEASE_STATUS"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p20": "bound to the p19 tree/zip; the p20 tree has its own (pending) independent flag",
  "archived_p19_records_pinned": {
    "archival/p19/reference_values_p19.json": true,
    "archival/p19/reproducibility/author_rebuild_report_p19.json": true,
    "archival/p19/reproducibility/independent_rebuild_report_p19.json": true,
    "archival/p19/p19_source_files.json": true
  },
  "pins_ok": true
},
"p20_layer_independent_rebuild_passed_p19": true,
"p19_gate_change_log_vs_p18_recomputed_on_current_tree": {
  "base": "archival/p18/p18_source_files.json (pinned; files of the p18 zip sha256 7c7d5451adb459092edc2606e
+> 1b33229093250O0d47bab89c14b7f7a6d25c92f)",
  "base_pinned_ok": true,
  "reference_file": "reproducibility/reference_values_p23.json listed as added by declaration (written after
+>  the author build)",
  "added": ["archival/p18/README.md", "archival/p18/p18_source_files.json", "archival/p18/reference_values_p
+> 18.json", "archival/p18/reproducibility/author_rebuild_report_p18.json", "archival/p18/reproducibility/in
+> dependent_rebuild_report_p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.json", "arch
+> ival/p19/reference_values_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "arch
+> ival/p19/reproducibility/independent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p2
```

```
+> 0_source_files.json", "archival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebu
+> ild_report_p20.json", "archival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/R
+> EADME.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/
+> reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report
+> _p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_
+> source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuil
+> d_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/con
+> tract_registry.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "c
+> ode/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "c
+> ode/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "co
+> de/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/r
+> elease_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_c
+> orrection.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscri
+> pt/notes/P23_O1_scope.tex", "provenance/XVI-O/O_quantum_target.json", "reproducibility/reference_values_p
+> 23.json", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_p22.py", "tests/test
+> _p23.py"],
  "removed": ["reproducibility/reference_values_p18.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/release_p18.py", "code/p5_entry/dred.py
+> ", "manuscript/body/XVI-P-body.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "test
+> s/test_p18.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p18/README.md", "archival
+> /p18/p18_source_files.json", "archival/p18/reference_values_p18.json", "archival/p18/reproducibility/auth
+> or_rebuild_report_p18.json", "archival/p18/reproducibility/independent_rebuild_report_p18.json", "archiva
+> l/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p19.json", "archiv
+> al/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/independent_rebuild
+> _report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/referenc
+> e_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproduc
+> ibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.js
+> on", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.js
+> on", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archi
+> val/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_
+> values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducib
+> ility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/d11_pin.py
+> ", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "code/p5_calc/p19_conventions.py", "code/p5_ca
+> lc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_ca
+> lc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p18.py", "code/p5_calc/re
+> lease_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py
+> ", "code/p5_calc/release_p23.py", "manuscript/body/XVI-P-body.tex", "manuscript/notes/P19_defect.tex", "m
+> anuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evane
+> scent.tex", "manuscript/notes/P23_O1_scope.tex", "provenance/XVI-O/O_quantum_target.json", "reproducibili
+> ty/reference_values_p18.json", "reproducibility/reference_values_p23.json", "tests/test_p14.py", "tests/t
+> est_p16.py", "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/t
+> est_p21.py", "tests/test_p22.py", "tests/test_p23.py", "tools/build.py", "tools/rebuild_from_source.py",
+> "tools/source_manifest.py"],
  "counts": {
    "added": 53,
    "removed": 1,
    "modified": 12
  }
},
"absolute_pin": {
  "check": {
    "ratio_pin_PASS": true,
    "ratio_1O": true,
    "transverse": true,
    "C2_quadratic_form_half_P2": true,
    "GaussBonnet_zero": true,
    "absolute_pin_PASS": true,
    "absolute_pin_flag": true,
    "top_level_absolute_status_consistent": true,
    "top_level_ratio_status_consistent": true,
    "absolute_ratios_one": true,
    "no_external_convention_choice": true,
    "drop_D11_detected": true,
    "x2_detected_by_absolute": true,
    "sign_flip_detected_by_absolute": true,
    "corrected_in_p21": true,
    "literature_source_stated": true,
    "revision": true
  },
  "absolute_pin": {
    "status": "PASS",
    "observed_common_factor_c2_vs_bare_a2": "-I",
    "observed_common_factor_c0_vs_bare_a2": "-I",
    "derivation_p21": {
      "package_declarations_checked": {
        "N_eps = i S_eps/(16 pi^2)": true,
        "Gamma_1 = -i * graph sum": true,
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
```

```
        "g = eta + kappa h": true,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
        "eta = (+,-,-,-)": true,
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
      },
      "steps": {
        "1_G_equals_Gamma2": "1",
        "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
        "3_i_S_M_over_S_E (must be -1)": "-1",
        "4_L_M_eff_over_L_E_eff": "-1",
        "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
        "6_minkowski_quadratic_forms": {
          "Riem2": {
            "P2": "1",
            "P0s": "1"
          },
          "Ric2": {
            "P2": "1/4",
            "P0s": "1"
          },
          "R2": {
            "P2": "0",
            "P0s": "3"
          },
          "C2": {
            "P2": "1/2",
            "P0s": "0"
          },
          "E_GaussBonnet": {
            "P2": "0",
            "P0s": "0"
          }
        }
      },
      "package_Gamma2_pole_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "heat_kernel_prediction_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "ratio_package_over_prediction": {
        "spin2": "1",
        "spin0s": "1"
      },
      "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2
+>  (Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps
+> )) int a_2 (Euclidean, d = 4-2eps)"],
      "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, s
+> ign of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
      "status": "PASS"
    },
    "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under th
+> e package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the W
+> ick direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(
+> 16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1
+> , both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no ext
+> ernal convention choice.  p20 had left this OPEN."
  },
  "absolute_pin_status": "PASS",
  "mutations": {
    "drop_graph_D11": {
      "c2": "0",
      "c0": "0",
      "ratio": "None",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_normalisation_x2": {
      "c2": "-I/120",
      "c0": "-I/12",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_sign_flip": {
```

```
      "c2": "I/240",
      "c0": "I/24",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "swap_g4g4_structures": {
      "c2": "-I/40",
      "c0": "-I/32",
      "ratio": "5/4",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    }
  }
},
"lower_layers": {
  "contract_identity": true,
  "p18_layer": true,
  "P18_quantum_inputs_evaluated": "earned",
  "inventory": true,
  "p19_layer": true,
  "k1_correction": true,
  "retraction": true,
  "retraction_hits": {},
  "contract_detail": {
    "p14r_contract_files": 12,
    "changed_vs_p14r": ["code/p5_entry/dred.py"],
    "added": [],
    "dred_byte_identical_to_p18": true,
    "dred_registry_state_p22": "p18 + p22 guard block",
    "contract_revision_in_p19": false,
    "disclosed_revisions_p19": {}
  },
  "rechecked_live": true
},
"p20_layer": {
  "p20_conventions_byte_identical": true,
  "disclosed_p21_revisions": {
    "code/p5_calc/d11_pin.py": {
      "sha256": "a2e77eecc84263e254d3b7cd7b441731324e8d15b5d15c71dc05d61ff5fe54f8",
      "differs_from_p20": true,
      "disclosed": true
    },
    "code/p5_calc/release_p20.py": {
      "sha256": "a1afba60c21c923886e682b960dda8b2619353712517f078c9967ad9c57e4d5f",
      "differs_from_p20": true,
      "disclosed": true
    }
  },
  "P20_release_accepted": true,
  "P20_D11_absolute_pin_verified": "earned"
},
"manuscript_disclosure": {
  "manuscript/notes/P19_defect.tex": {
    "current_equals_disclosed_p20": true,
    "differs_from_p19": true,
    "revised_in": "p20"
  }
},
"change_log_vs_p20": {
  "base": "archival/p20/p20_source_files.json (pinned; files of the p20 zip sha256 ca32ce018933f12d1a623217d
+> 6a98d647817360dd12211e2e9469cf850a63bb2)",
  "base_pinned_ok": true,
  "added": ["archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/reference_values_p
+> 20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproducibility/in
+> dependent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.json", "arch
+> ival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "arch
+> ival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22/ce
+> rtificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_values_p22
+> .json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/inde
+> pendent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/evanescent.py", "cod
+> e/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "cod
+> e/p5_calc/power_counting.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py", "code/p5_calc
+> /release_p23.py", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manusc
+> ript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values
+> _p23.json", "tests/test_p21.py", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p20.json"],
```

```
    "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/d11_pin.py", "code/p5_calc/release_p18.
  +> py", "code/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_entry/dred.py", "manuscript/b
  +> ody/XVI-P-body.tex", "manuscript/notes/P20_correction.tex", "tests/test_p14.py", "tests/test_p16.py", "te
  +> sts/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tools/build.py", "tools
  +> /rebuild_from_source.py", "tools/source_manifest.py"],
    "contract_files_changed": [],
    "undisclosed_package_changes": [],
    "counts": {
      "added": 33,
      "removed": 1,
      "modified": 18
    }
  },
  "open_pins": [],
  "disclosed_p20_layer_revisions_p21": {
    "code/p5_calc/d11_pin.py": "absolute_pin(): sign and magnitude derived from 1/2 Tr ln under the declared c
  +> onventions (Minkowski quadratic forms curvature_forms_eta); absolute pin PASS; mutations x2 and sign flip
  +> ; top-level absolute_pin_status from the derivation",
    "code/p5_calc/release_p20.py": "absolute pin PASS required; top-level / nested status consistency; standal
  +> one run re-checks the p18 layer (q earned) and the graph inventory; claim texts"
  },
  "disclosed_test_revisions_p21": {
    "tests/test_p14.py": "names p20 -> p21; generated prefixes P21_",
    "tests/test_p19.py": "reviewed coefficient sites read from p21_conventions (adds the d11_pin Minkowski for
  +> m site)",
    "tests/test_p20.py": "D11 pin tests: absolute pin PASS (p21) instead of OPEN; gate mutations on the p21 ch
  +> ecks; reviewed sites from p21_conventions; status/version expectations p21"
  },
  "disclosed_manuscript_revisions": {
    "manuscript/notes/P19_defect.tex": {
      "p19_sha256": "04b974b5ea742c6885f3d694ccad6bda7908105c0a23bf483e96bc113d90bbc8",
      "p20_sha256": "1c71a71a48ce9c1749bd2164a0f720a3589f2d679877cfd04309d69670e5dfa8",
      "revised_in": "p20",
      "change": "K1 paragraph retracted/rewritten (Grassmann orientation erratum), restoration definition REDE
  +> FINED, J1 wording, blind-spot text; marked inline [corrected in p20] / [REDEFINED in p20]; unchanged in p
  +> 21"
    }
  },
  "independent_rebuild_passed_p20": true,
  "p20_independent_rebuild": {
    "report": "archival/p20/reproducibility/independent_rebuild_report_p20.json",
    "present": true,
    "report_sha256": "e8f358f9c4f13c76693968211b49e04d546a0bb3a4c0ab4b1a0e485f62595389",
    "report_sha256_pinned": true,
    "pass": true,
    "independent_rebuild_passed": true,
    "bound_to_source_tree_sha256": "2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700",
    "bound_to_source_zip_sha256": "ca32ce018933f12d1a623217d6a98d647817360dd12211e2e9469cf850a63bb2",
    "tree_matches_p20": true,
    "zip_matches_p20": true,
    "mismatches": 0,
    "findings": ["p19_quotient_finding_withdrawn", "D11_absolute_pin_left_OPEN_but_derivable", "scan_misses_p2
  +> 0", "gate_top_level_absolute_pin_status_unchecked", "p20_gate_trusts_P19_layer_record", "P19_note_edit_di
  +> sclosure", "restoration_scope", "p19_change_log_key_recomputed_on_p20_tree"],
    "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
  +> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
  +> only zip; it was NOT run on a second machine.",
    "live_report_identical_to_archived": true,
    "NOT_inherited_by_p21": "bound to the p20 tree/zip; the p21 tree has its own (pending) independent flag",
    "archived_p20_records_pinned": {
      "archival/p20/reference_values_p20.json": true,
      "archival/p20/reproducibility/author_rebuild_report_p20.json": true,
      "archival/p20/reproducibility/independent_rebuild_report_p20.json": true,
      "archival/p20/p20_source_files.json": true
    },
    "pins_ok": true
  }
}
```


41 D11 pin: ratio (p20) and absolute sign and magnitude (derived in p21)

[listing 08/133] certificates/P20_D11_pin.json


```
{
  "landmark": "XVI-P",
  "revision": "p20",
```

```
"deliverable": "D11 normalisation pin via the trace-anomaly / C^2 coefficient (p19 reviewer blind spot)",
"source_certificate": "certificates/P18_D11.json",
"graph": "D11 (hh scalar loop, minimal coupling h.phi^2)",
"input_4D_UV_pole": {
  "g4_mn g4_rs (p.p)^2": "-I/80",
  "g4_mn p_r p_s (p.p)^1": "I/80",
  "g4_mr g4_ns (p.p)^2": "-I/480",
  "g4_mr p_n p_s (p.p)^1": "I/480",
  "g4_rs p_m p_n (p.p)^1": "I/80",
  "p_m p_n p_r p_s": "-I/60"
},
"orbit_convention": "covariant.orbit over the symmetric pairs (m,n), (r,s) (as in the D11 fit)",
"spin_decomposition": {
  "c2_spin2": "-I/240",
  "c0_spin0s": "-I/24",
  "projector_norms": ["5", "1"],
  "transverse": true,
  "only_spin2_and_spin0s": true,
  "ratio_c0_over_c2": "10"
},
"curvature_quadratic_forms_derived": {
  "Riem2": {
    "P2": "1",
    "P0s": "1"
  },
  "Ric2": {
    "P2": "1/4",
    "P0s": "1"
  },
  "R2": {
    "P2": "0",
    "P0s": "3"
  },
  "C2": {
    "P2": "1/2",
    "P0s": "0"
  },
  "E_GaussBonnet": {
    "P2": "0",
    "P0s": "0"
  }
},
"literature_a2_minimal_scalar": {
  "C2": "1/120",
  "R2_xi0": "1/72",
  "source": "Birrell & Davies, Quantum Fields in Curved Space (CUP 1982), sec. 6.2 (a_2 of a scalar field);
+> 't Hooft & Veltman, Ann. Inst. H. Poincare A20 (1974) 69 -- EXTERNAL, not package-derived",
  "predicted_spin2": "1/240",
  "predicted_spin0s": "1/24",
  "predicted_ratio": "10"
},
"ratio_pin": {
  "status": "PASS",
  "statement": "spin-0 : spin-2 of the D11 4D UV pole = 10 : 1, derived from the certificate; equals the min
+> imal-scalar a_2 prediction (1/72)(3) : (1/120)(1/2)"
},
"absolute_pin": {
  "status": "PASS",
  "observed_common_factor_c2_vs_bare_a2": "-I",
  "observed_common_factor_c0_vs_bare_a2": "-I",
  "derivation_p21": {
    "package_declarations_checked": {
      "N_eps = i S_eps/(16 pi^2)": true,
      "Gamma_1 = -i * graph sum": true,
      "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
      "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
      "g = eta + kappa h": true,
      "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
      "eta = (+,-,-,-)": true,
      "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
    },
    "steps": {
      "1_G_equals_Gamma2": "1",
      "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
      "3_i_S_M_over_S_E (must be -1)": "-1",
      "4_L_M_eff_over_L_E_eff": "-1",
      "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
      "6_minkowski_quadratic_forms": {
        "Riem2": {
          "P2": "1",
          "P0s": "1"
```

```
        },
        "Ric2": {
          "P2": "1/4",
          "P0s": "1"
        },
        "R2": {
          "P2": "0",
          "P0s": "3"
        },
        "C2": {
          "P2": "1/2",
          "P0s": "0"
        },
        "E_GaussBonnet": {
          "P2": "0",
          "P0s": "0"
        }
      }
    },
    "package_Gamma2_pole_per_kappa2_over_eps": {
      "spin2": "1/(3840*pi**2)",
      "spin0s": "1/(384*pi**2)"
    },
    "heat_kernel_prediction_per_kappa2_over_eps": {
      "spin2": "1/(3840*pi**2)",
      "spin0s": "1/(384*pi**2)"
    },
    "ratio_package_over_prediction": {
      "spin2": "1",
      "spin0s": "1"
    },
    "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2 (
+> Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps))
+>  int a_2 (Euclidean, d = 4-2eps)"],
    "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, sig
+> n of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
    "status": "PASS"
  },
  "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under the
+> package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the Wic
+> k direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(16
+>  pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1,
+> both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no exter
+> nal convention choice.  p20 had left this OPEN."
},
"mutations": {
  "drop_graph_D11": {
    "c2": "0",
    "c0": "0",
    "ratio": "None",
    "ratio_pin_passes": false,
    "detected_by_ratio_pin": true,
    "absolute_pin_passes": false,
    "detected_by_absolute_pin": true
  },
  "D11_normalisation_x2": {
    "c2": "-I/120",
    "c0": "-I/12",
    "ratio": "10",
    "ratio_pin_passes": true,
    "detected_by_ratio_pin": false,
    "absolute_pin_passes": false,
    "detected_by_absolute_pin": true
  },
  "D11_sign_flip": {
    "c2": "I/240",
    "c0": "I/24",
    "ratio": "10",
    "ratio_pin_passes": true,
    "detected_by_ratio_pin": false,
    "absolute_pin_passes": false,
    "detected_by_absolute_pin": true
  },
  "swap_g4g4_structures": {
    "c2": "-I/40",
    "c0": "-I/32",
    "ratio": "5/4",
    "ratio_pin_passes": false,
    "detected_by_ratio_pin": true,
    "absolute_pin_passes": false,
    "detected_by_absolute_pin": true
```

```
    }
  },
  "reading": "drop D11 and a structure swap are detected by the ratio pin; an overall x2 or a sign flip keeps
  +> the ratio and is detected by the absolute pin (p21) and, for x2, by M2 (P18)",
  "corrected_in_revision": "p21 (absolute pin derived; p20 had it OPEN)",
  "D11_absolute_pin_verified": true,
  "D11_ratio_pin_verified": true,
  "c2_spin2": "-I/240",
  "c0_spin0s": "-I/24",
  "ratio_c0_over_c2": "10",
  "ratio_pin_status": "PASS",
  "absolute_pin_status": "PASS",
  "seconds": 5.3
}
```


42 p20 layer record (release gate p20, revised in p21)

[listing 09/133] certificates/P20_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p20",
  "contract_revision": "p20 (p14r contract + disclosed p15/p16/p17/p18 revisions; none in p19 or p20)",
  "date": "2026-10-09",
  "version_label": "XVI-P p20",
  "deliverable": "p20 CORRECTION of p19 (K1 orientation; restoration redefined; blind-spot pins; J1 wording; s
  +> can)",
  "execution_pass": true,
  "claim_status": "p20 CORRECTION release (no scope expansion). P.5_identity: projected defect assembled from
  +> every evaluated input (D01, D02, D03, D11, D12, D13, D14a-c); outcome ZERO. WHY ZERO: each surviving ch r
  +> ow vanishes separately in the projected slot; no counterterm cancellation and no cancellation between row
  +> s is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupling -- D01 and D03 are indivi
  +> dually nonzero and cancel against each other in the slot (p17/p18 transverse sum), D11 is transverse by i
  +> tself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only through T1 and E1 (T2, T3, E2 are
  +> longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-point row: the tadpoles D14a-c va
  +> nish under the declared prescription.  The defect is therefore zero without any counterterm (solve x = 0)
  +> . RESTS ON: M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D compo
  +> nents'; evanescent components of the D11 matching: not applicable, not verified); the evanescent decoupli
  +> ng proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row decoupling); the frozen P.3 projection and
  +>  the P13 ch dependency ledger (three surviving rows); the declared prescription (DRED R1-R6, MS-bar, P14_
  +> prescription_D11_matching). FLAGS: projected_defect_computed=earned, projected_identity_verified=earned,
  +> projected_restoration_verified=earned (definitions: flag_definitions). projected_restoration_verified (p2
  +> 0 definition) = full sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm (h.h + c.K + c.c
  +> b; ghost self-energy held fixed; derived relative factor +1; orientation converted) in every ghost +1 sec
  +> tor (h.c, c.b, c.cbs): {'closed': True, 'entries': 0, 'sectors': []}. Scope: that counterterm only -- NOT
  +>  bulk symmetry (bulk_symmetry_restored=False), O.1 NOT_CLOSED. The p19 criterion (P_ch sigma_0(S_ct) = 0,
  +>  x = 0) follows from the identity + locality and is kept only as a route-consistency control. ERRATUM (K1
  +> ): the p19 text 'full closure fails with the derived factor, needs -1, unresolved' was a Grassmann-orient
  +> ation bookkeeping error and is RETRACTED. J1: the p14r counterterm-domain counts are Q re/im-split artefa
  +> cts; the Q(i) counts are the real dimensions under a compatible real structure (finding about the p14r co
  +> ntract; contract not modified) D11: ratio pin verified (spin-0 : spin-2 = 10 : 1, derived from P18_D11 an
  +> d equal to the minimal-scalar a_2 prediction, literature source stated); absolute pin (sign and magnitude
  +> ) derived in p21 from 1/2 Tr ln under the declared conventions and equal to the heat-kernel a_2 (p20: OPE
  +> N).",
  "flag_definitions": {
    "projected_defect_computed": "Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from every evaluat
  +> ed input of the three surviving rows (none Missing), exact in d, pole + nonlocal + MS-bar local parts, al
  +> l seven external assignments",
    "projected_identity_verified": "Delta = 0 for the REGULATED one-loop functional Gamma1: pole, nonlocal and
  +>  local parts vanish for all seven assignments, exactly in d, with all controls true and every mutation ch
  +> anging the outcome",
    "projected_restoration_verified": "p20 definition (replaces p19's, which follows from the identity + local
  +> ity): FULL sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm S_ct = -(pole) in the decl
  +> ared domain, h.h + c.K + c.cb (ghost self-energy Sigma held fixed at its P15 value, orientation converted
  +> , derived relative factor +1), in every ghost +1 sector (h.c, c.b, c.cbs), together with projected_identi
  +> ty_verified.  Scope: the one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1",
    "bulk_symmetry_restored": "hard stop: False regardless (not claimed; O.1 NOT_CLOSED)"
  },
  "projected_restoration_scope": "p20 definition: full sigma_0 closure (h.c, c.b, c.cbs) of the complete one-l
  +> oop kappa^2 MS-bar counterterm incl. the c.cb ghost self-energy counterterm (held fixed, derived factor +
  +> 1); NOT bulk symmetry, NOT O.1",
  "route_consistency_control_p19_criterion": {
    "label": "route-consistency control (the p19 restoration criterion, NOT a flag): P_ch sigma_0(S_ct) = 0 fo
  +> r the h.h + c.K MS-bar counterterm and the solve x = 0; implied by the identity + locality",
    "holds": true
  },
  "full_closure_mutation_detector": {
    "test": "full sigma_0 closure (all ghost +1 sectors) of the complete MS-bar counterterm, Sigma held fixed"
```

```
+> ,
  "cases": {
    "drop_graph_D11": {
      "closed": true,
      "entries": 0,
      "sectors": [],
      "detected": false
    },
    "drop_graph_D13 (c.K data = 0, Sigma held fixed)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "drop_graph_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "drop_graph_D01": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "flip_phase_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "symmetry_factor_D03_x2": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "Sigma_orientation_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    }
  },
  "reading": "detects drop D13 and flip K.c (the p19 blind spots) through the c.b sector, and the hh-row mut
+> ations through h.c; drop D11 stays undetected (D11 transverse => its pole is sigma_0-closed by itself): c
+> overed by M2 (P18) and the p20 D11 normalisation pin (P20_D11_pin.json)"
},
"p20_controls": {
  "orientation_ratio_uniform_minus_one": true,
  "tree_ghost_equation_V_cbc_eq_2iD_V_Kc": true,
  "tree_sigma0_S0Kc_plus_S0cbc_zero": true,
  "derived_ghost_relation_reproduces_P15_sigma": true,
  "full_closure_detects_drop_D13_and_flip_Kc": true,
  "full_closure_detects_p19_orientation_error": true,
  "S_ct_without_ccb_not_closed (c.cb counterterm is needed)": true
},
"why_zero": "each surviving ch row vanishes separately in the projected slot; no counterterm cancellation an
+> d no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupl
+> ing -- D01 and D03 are individually nonzero and cancel against each other in the slot (p17/p18 transverse
+>  sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only throu
+> gh T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-poin
+> t row: the tadpoles D14a-c vanish under the declared prescription.  The defect is therefore zero without
+> any counterterm (solve x = 0).",
"rests_on": ["M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D compon
+> ents'; evanescent components of the D11 matching: not applicable, not verified)", "the evanescent decoupl
+> ing proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row decoupling)", "the frozen P.3 projection
+> and the P13 ch dependency ledger (three surviving rows)", "the declared prescription (DRED R1-R6, MS-bar,
+>  P14_prescription_D11_matching)"],
"findings": {
  "J1_p14r_counterterm_domain_field": "FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the p14r cou
+> nterterm-domain counts (Im sigma_0 3, quotient 8, D cap ker 5, P_ch sigma_0 rank 4) are artefacts of a re
+> /im split over Q (the densities carry explicit i for odd derivative counts, so i-multiples count as indep
+> endent).  The counts over Q(i) -- sigma_0 rank 5, D cap ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch
```

```
+>  sigma_0 rank 3 (= Route C) -- equal the real dimensions under a compatible real structure.  The p14r cer
+> tificate stays as released; both are reported",
  "K1_ghost_closure_sign": "p20 CORRECTION (erratum to p19): resolved -- an orientation bookkeeping error. W
+> ith the P15 Sigma coordinates converted to the route-G (c,cb) orientation (ratio -1), the derived factor
+> +1 gives full sigma_0 closure of the complete MS-bar counterterm in h.c, c.b, c.cbs.  The p19 text 'closu
+> re fails / needs -1 / unresolved' is retracted"
},
"K1_ghost_closure_convention": {
  "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
  "M_2iD_cK_to_cbc": [
    ["0", "2"],
    ["-1", "0"],
    ["0", "-1"]
  ],
  "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
  "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
  "derived_equals_P15": true,
  "relative_factor_derived": "1",
  "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
  "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
  "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
  "tree_controls": {
    "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
    "S0_Kc_terms": 16,
    "S0_cbc_terms": 40,
    "sigma0_S0Kc_plus_S0cbc_entries": 0,
    "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
    "ok": true
  },
  "full_closure_complete_S_ct": {
    "closed": true,
    "entries": 0,
    "sectors": []
  },
  "full_closure_S_ct_without_ccb (p19 package S_ct)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"]
  },
  "full_closure_unconverted (p19 error)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"],
    "detected": true
  },
  "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolved
+> ' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots).
+>  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
},
"scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the hh an
+> d K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement abou
+> t higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests ful
+> l sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c.b,
+> c.cbs) -- a statement about that counterterm only, not bulk symmetry, not 0.1",
"release_claim_accepted": true,
"release_claim_accepted_definition": "p19 layer accepted (with the disclosed p20 revisions of defect.py and
+> release_p19.py only) AND p18 layer + contract byte-identical AND the K1 correction content AND the retrac
+> tion check AND the D11 ratio pin AND the pinned p19 independent record AND the p20 hard stops",
"publication": "accepted",
"publication_gated_on": ["execution_pass", "quantum_inputs_evaluated (p18 layer + recheck)", "projected_defe
+> ct_computed", "flag implications restoration => identity => computed"],
"flags": {
  "library_certified": true,
  "batchA_evaluated": {
    "execution": true,
    "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
+> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
+> V pole, finite part 0, no p14r counterterm required",
    "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
+>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
+> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
+> e; NOT quantum_inputs_evaluated"
  },
  "D13_evaluated": {
    "execution": true,
    "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
+> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
+> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
+> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
    "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
+> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
```

```
+> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
+> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
  },
  "batchC_evaluated": {
    "execution": true,
    "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
+> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
+> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
+> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
    "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
+>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
+> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
  },
  "E2_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof)",
    "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
+>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
  },
  "hh_evanescent_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
    "scope": "the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of D01
+>  + D03 (D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green function
+>  outside this projection; D11 (same row) NOT evaluated"
  },
  "D11_matching_M2": {
    "execution": true,
    "claim": "DISCHARGED (4D components)",
    "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the
+>  4D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0
+> is IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns
+>  the classical insertion and is not touched"
  },
  "hh_slot_full_row_decoupled": {
    "execution": true,
    "claim": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
    "scope": "row (S0,Gamma1^[2]) s2 K:K.c; D01 + D02 + D03 + D11 + D12; the summed evanescent poles remain
+> (transverse) UV poles of the hh Green function outside this projection"
  },
  "projected_ch_identity": {
    "execution": true,
    "claim": "ZERO",
    "scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the h
+> h and K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement
+> about higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests
+>  full sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c
+> .b, c.cbs) -- a statement about that counterterm only, not bulk symmetry, not O.1"
  }
},
"projected_defect_computed": "earned",
"projected_identity_verified": "earned",
"projected_restoration_verified": "earned",
"projected_defect_summary": {
  "defect_zero_exact_in_d": true,
  "evanescent_in_assembly": "verified explicitly: the evanescent (gt) parts of the hh row (D01+D03+D11) and
+> of the K.c row (D13 E1, E2) give ZERO for all seven external assignments, exactly in d; the evanescent po
+> les themselves are nonzero (P18_hh_slot, P15_D13) -- they drop from the ch slots by the decoupling proofs
+> , not by convention",
  "quotient_coordinates": {
    "complement_directions": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"],
    "pole": ["0", "0", "0", "-3/8", "1/4"],
    "nonlocal_L": ["0", "0", "0", "3/8", "-1/4"],
    "local": ["0", "0", "0", "-2/3", "5/12"],
    "reading": "coordinates modulo D cap Im sigma_0 (field redefinitions / s-exact); the h.h pole is transve
+> rse, i.e. in D cap Im sigma_0, so its quotient coordinates vanish"
  },
  "D_cap_ker_sigma0": {
    "computed": true,
    "dim": 6,
    "dim_D_cap_Im_sigma0": 6,
    "cohomology_dim": 0,
    "sector_ranks": {
      "h.h": {
        "dim": 5,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 3
      },
      "c.cb": {
        "dim": 2,
```

```
        "sigma0_rank": 2,
        "P_ch_sigma0_rank": 0
      },
      "c.K": {
        "dim": 3,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 1
      },
      "phi.phi": {
        "dim": 1,
        "sigma0_rank": 0,
        "P_ch_sigma0_rank": 0
      }
    }
  },
  "field_note": "Route G is computed over Q(i) (primary) and, for comparison, over Q with the re/im split us
+> ed by the p14r domain certificate.  The p14r densities carry explicit factors i for odd derivative counts
+>  (c.K: 3 derivatives), so over Q an i-multiple counts as independent: e.g. the ch image of c.K#0 is i tim
+> es an h.h image.  The counts over Q(i) equal the real dimensions under a compatible real structure (one r
+> eal coefficient per density, the i fixed by reality of the action); ranks and solvability are quoted over
+>  Q(i)."
},
"counterterm_solve": {
  "system": "Delta + P_ch sigma_0(x) = 0, x in the declared domain D (11 directions), part by part",
  "rank_P_ch_sigma0_routeG": 3,
  "rank_routeC": 3,
  "routeC_image_relations": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "outcome": "ZERO",
  "residual": null,
  "solution": "x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite renorma
+> lisations are ker(P_ch sigma_0) cap D (dim 8), of which 2 modulo D cap Im sigma_0",
  "MSbar_counterterm_ch_image_zero": true,
  "MSbar_counterterm_full_sigma0_image_sectors": ["c.b"],
  "full_closure_relative_factor_table_UNCONVERTED_p19_error": {
    "1": false,
    "-1": true,
    "I": false,
    "-I": false
  },
  "full_closure_reading": "p20 (corrects p19): with the P15 Sigma coordinates converted from the (cb,c) to t
+> he route-G (c,cb) orientation (measured ratio -1), the DERIVED factor +1 gives full sigma_0 closure of th
+> e complete MS-bar counterterm (h.h + c.K + c.cb) in every ghost +1 sector; the p19 table above (factors 1
+> , -1, i, -i on UNCONVERTED coordinates) is the bookkeeping error, kept for the erratum",
  "K1_ghost_closure_convention": {
    "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
    "M_2iD_cK_to_cbc": [
      ["0", "2"],
      ["-1", "0"],
      ["0", "-1"]
    ],
    "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
    "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
    "derived_equals_P15": true,
    "relative_factor_derived": "1",
    "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
    "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
    "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
    "tree_controls": {
      "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
      "SO_Kc_terms": 16,
      "SO_cbc_terms": 40,
      "sigma0_SOKc_plus_SOcbc_entries": 0,
      "sigma0_SOKc_alone_entries (control, nonzero)": 16,
      "ok": true
    },
    "full_closure_complete_S_ct": {
      "closed": true,
      "entries": 0,
      "sectors": []
    },
    "full_closure_S_ct_without_ccb (p19 package S_ct)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"]
    },
```

```
    "full_closure_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolv
+> ed' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots)
+> .  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
  },
  "note_full_sigma0": "the h.h + c.K MS-bar counterterm ALONE has a nonzero (c, b) image (delta S0/delta h c
+> arries b); adding the c.cb ghost self-energy counterterm (correct orientation, derived factor +1) closes
+> it: see K1_ghost_closure_convention.full_closure_complete_S_ct"
},
"defect_controls": {
  "inventory_complete_no_missing": true,
  "missing_input_refused_not_zero": true,
  "s0h_longitudinal": true,
  "E_transverse": true,
  "hh_row_zero_exact_in_d_all_7": true,
  "hh_row_evanescent_part_zero_all_7": true,
  "hh_row_D01_D03_individually_nonzero": true,
  "Kc_row_zero_exact_in_d_all_7": true,
  "Kc_row_evanescent_part_zero_all_7": true,
  "Kc_slot_map_nontrivial_T1_E1": true,
  "E2_alone_zero_all_7": true,
  "D13_A_T1_A_E1_zero_exact": true,
  "onepoint_row_D14abc_vanishing": true,
  "defect_zero_pole_nonlocal_local_all_7": true,
  "defect_independent_of_row_signs": true,
  "routeG_S0_check": true,
  "routeG_defect_zero_by_sector": true,
  "routeG_hh_local_data_in_span": true,
  "routes_agree_on_ch_image_rank": true,
  "routeG_hh_kernel_equals_p18_transverse_subspace": true,
  "D_cap_Im_inside_D_cap_ker_sigma0": true,
  "D_cap_Im_inside_ker_P_ch_sigma0": true,
  "ghost_equation_relation_reproduces_P15_sigma_pole": true,
  "MSbar_counterterm_in_ker_P_ch_sigma0": true,
  "mutations_all_change_outcome": true,
  "routeG_Q_split_reproduces_p14r_counts": true
},
"defect_mutations": {
  "drop_graph_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/48",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the ghost loop D03"
  },
  "drop_graph_D01": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the graviton bubble D01"
  },
  "flip_phase_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
```

```
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/24",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/24",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/8"
      }
    },
    "note": "relative sign of D03 flipped"
  },
  "symmetry_factor_D03_x2": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "D03 doubled"
  },
  "wrong_projection_trace_operator_hh": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-3*c^.p*p.p*(6*a.b*p.p - 6*a^.p*b^.p - a~.b~*p.p)/16",
        "QdS (hat)": "-9*c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/8",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-15*a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "eta_mn p_v in place of s0 h (not the P.3 projection)"
  },
  "wrong_projection_gauge_fixed_operator_Kc": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p*(a.b*c^.p*p.p - 3*a.c*b^.p*p.p - 3*a^.p*b.c*p.p + 4*a^.p*b^.p*c^.p - a^.p
+> *b~.c~*p.p - a~.c~*b^.p*p.p)/16",
        "QdS (hat)": "p.p*(a^.b^*c^.p*p.p - 3*a^.c^*b^.p*p.p - 3*a^.p*b^.c^*p.p + 4*a^.p*b^.p*c^.p)/16",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/4",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "de Donder q^2 P in place of the h^2-monomial operator E (not the P.3 projection)"
  },
  "naive_4D_D03_evanescent_dropped": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0",
```

```
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/24"
      }
    },
    "note": "D03 Gt parts dropped"
  },
  "naive_4D_D13_Gt_dropped": {
    "outcome": "NONZERO, CANCELLABLE by a counterterm in the declared domain",
    "changes_outcome": true,
    "residual": {
      "local": {
        "Q4S (declared, R2)": "-c^.p*p.p*(a.b*p.p - a^.p*b^.p)/12",
        "QdS (hat)": "-c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/12",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/12"
      }
    },
    "note": "D13 evaluated with Gt dropped (P15_D13 mutation fit: T1 = -(d-4)/(16(d-1)))",
    "finite_counterterm": {
      "rank": 3,
      "solvable": true,
      "solution": ["(6*x3 + 12*x4 - 24*I*x5 - I)/12", "-x3/2", "-(12*x3 + 24*x4 - 24*I*x5 - I)/12", "x3", "x
+> 4", "x5", "x6", "x7"]
    }
  }
},
"defect_insensitive_variations": {
  "drop_graph_D11": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "D11 is transverse by itself (p_m G_D11 = 0 exactly in d): the slot cannot see it; covered
+> by M2 (P18) and by the inventory refusal below"
  },
  "drop_graph_D13": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "A_T1 = A_E1 = 0 and T2, T3, E2 are longitudinal: D13 reaches the slot with zero; covered b
+> y the D13 certificate (two routes, mutations)"
  },
  "flip_row_sign_Kc": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "each row vanishes separately"
  }
},
"quantum_inputs_evaluated": "earned",
"quantum_inputs_inventory": [
  {
    "graph": "D01",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D02",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D03",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D11",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"
  },
  {
    "graph": "D12",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D13",
```

```
    "slot_row": "(S0,Gamma1^[2]) s1 h:h^2",
    "evaluated": true,
    "status": "evaluated"
  },
  {
    "graph": "D14a",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14b",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14c",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  }
],
"D11_matching_verified": true,
"D11_matching_verified_scope": "4D components",
"D11_evanescent_components_matching": "not_applicable_not_verified",
"M3_status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
"D11_evaluated": true,
"hh_full_row_decoupling_proven": true,
"evanescent_hh_counterterm_added": false,
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"hh_evanescent_status": "decoupled_from_ch_slot_proven_as_sum",
"hh_evanescent_decoupling_proven": true,
"D01_evaluated": true,
"D03_evaluated": true,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
```

```
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
```

```
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D11": {
    "status": "evaluated",
    "certificate": "certificates/P18_D11.json",
    "sector_status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
    "M2": "DISCHARGED (4D components)",
    "ch_slot": "decoupled by itself (p_m G_D11 = 0 exactly in d)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_byte_identical_to_p18": true,
  "dred_registry_state_p22": "p18 + p22 guard block",
  "contract_revision_in_p19": false,
  "disclosed_revisions_p19": {}
},
"p18_layer": {
  "p18_layer_files_byte_identical": true,
  "P18_release_accepted": true,
  "P18_quantum_inputs_evaluated": "earned",
  "I1_gate": "the p19 gate requires the p18 layer record's quantum_inputs_evaluated == 'earned'"
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
```

```
  "C4_export_equals_independent_evaluator": true,
  "C5_cubic_regression_Euler": true,
  "C6_quadratic_regression_Euler": true,
  "C7_grav_noether_k0_k1_k2_on_exports": true,
  "quartic_mutations_all_rejected": true,
  "master_grav_noether_deterministic_k0_k1_k2": true,
  "master_engine_parts_exact": true,
  "graviton_hessian_generic_symbolic_k": true,
  "canonicity_all_23_antifield_components": true,
  "ch_enumeration_lemma_mechanical": true,
  "residual_record_invariants_all_certificates": true,
  "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
  "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
  "cross_certificate_quartic_vs_master_agree": true,
  "stored_C7_flag_equals_recomputation": true
},
"failed_prerequisites": [],
"hash_consistency": {
  "consistent": true,
  "ell": {
    "ell2": {
      "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
    },
    "ell3": {
      "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
    },
    "ell4": {
      "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
    "quartic_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
```

```
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      },
      "master_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    }
  },
  "hard_stops_p13_hold": true,
  "hard_stops_p13": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"hard_stops": {
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "evaluated; 4D sector renormalized (MS-bar, declared domain); evanescent regulated (p
+> oles retained, not subtracted), decoupled from the ch slots (p15-p18 inputs D01, D02, D03, D11, D12, D13,
+>  D14a-c)",
    "P.5_identity": "projected defect computed (zero exactly in d, all parts, all seven assignments; each ro
+> w vanishes separately); projected identity verified; projected restoration verified (p20 definition: full
+>  sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm incl. c.cb, h.c/c.b/c.cbs); K1 corre
+> cted in p20 (orientation erratum); D11 ratio pin verified; D11 absolute pin verified (p21); bulk symmetry
+>  NOT restored/claimed; O.1 NOT_CLOSED"
  },
  "definition_vs_status_note": "I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficients evalua
+> ted and renormalized'; the STATUS states exactly what holds: 4D sector renormalized, evanescent poles reg
+> ulated and decoupled from the ch slots (not subtracted)",
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)"
```

```
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p18": true,
"independent_rebuild_passed_p17": true,
"independent_rebuild_passed_p16": true,
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p20) sourc
+> e tree only (independent: pending until a reviewer report for this tree exists); independent_rebuild_pass
+> ed_p19 refers to the p19 tree fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36 / zip da92
+> 42752d2e3753d8b2ce7fbb21e7cdb28487cc7966c2ca52d1ef1376658875 (same-box caveat) and is not inherited by p2
+> 0",
"p18_independent_rebuild": {
  "report": "archival/p18/reproducibility/independent_rebuild_report_p18.json",
  "present": true,
  "report_sha256": "893f167ef7124d981c0569ca386035aed06e0dbb0c90a2362a6ceeafd1930cb2",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094",
  "bound_to_source_zip_sha256": "7c7d5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f",
  "tree_matches_p18": true,
  "zip_matches_p18": true,
  "mismatches": 0,
  "findings": ["I1_release_p18_publication_not_gated_on_quantum_inputs_evaluated", "I2_scan_pattern_gaps", "
+> I3_scope_of_M2_and_evaluated", "I4_table_row_count", "I5_slot_blind_to_D11_normalisation", "I6_test_p16_v
+> ictim_restriction", "I7_pdftotext_dehyphenation", "H1_H5_resolution", "bug_fixes_disclosed"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p19": "bound to the p18 tree/zip; the p19 tree has its own (pending) independent flag",
  "archived_p18_records_pinned": {
    "archival/p18/reference_values_p18.json": true,
    "archival/p18/reproducibility/author_rebuild_report_p18.json": true,
    "archival/p18/reproducibility/independent_rebuild_report_p18.json": true,
    "archival/p18/p18_source_files.json": true
  },
  "pins_ok": true
},
"p17_independent_rebuild": {
  "report": "archival/p17/reproducibility/independent_rebuild_report_p17.json",
  "present": true,
  "report_sha256": "92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
  "bound_to_source_zip_sha256": "7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416",
  "tree_matches_p17": true,
  "zip_matches_p17": true,
  "mismatches": 0,
  "findings": ["H1_static_scan_pattern_gaps", "H2_assignment_table_omits_c_hat_h_tilde", "H3_stale_comments"
+> , "H4_curly_apostrophes_in_typeset_listings", "H5_parse_d_srepr", "H6_scope", "G1_G2_G3_G4_resolution"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p18": "bound to the p17 tree/zip; the p18 tree has its own (pending) independent flag",
  "archived_p17_records_pinned": {
    "archival/p17/reference_values_p17.json": true,
    "archival/p17/reproducibility/author_rebuild_report_p17.json": true,
    "archival/p17/reproducibility/independent_rebuild_report_p17.json": true,
    "archival/p17/p17_source_files.json": true
  },
  "pins_ok": true
},
"p16_independent_rebuild": {
  "report": "archival/p16/reproducibility/independent_rebuild_report_p16.json",
  "present": true,
  "report_sha256": "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
  "bound_to_source_zip_sha256": "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57",
  "tree_matches_p16": true,
  "zip_matches_p16": true,
  "mismatches": 0,
```

```
  "findings": ["G1_static_scan_scope_narrower_than_disclosure", "G2_plain_symbol_d_from_stored_strings", "G3
+> _hook_coverage_detail", "G4_change_log_omits_new_reference_file", "G5_test_p15_assertion_relaxed", "G6_im
+> aginary_normalisation", "G7_scope", "F3_F4_F5_from_p15"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent flag",
  "archived_p16_records_pinned": {
    "archival/p16/reference_values_p16.json": true,
    "archival/p16/reproducibility/author_rebuild_report_p16.json": true,
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": true,
    "archival/p16/p16_source_files.json": true
  },
  "pins_ok": true
},
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
```

```
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"M2_summary": {
  "verdict": "DISCHARGED (4D components)",
  "D11_matching_verified_scope": "4D components",
  "evanescent_components_matching": {
    "status": "not_applicable_not_verified",
    "what_they_are": "the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt): compo
+> nents of the hh two-point function with at least one index pair in the epsilon-scalar (4-d) directions of
+>  DRED; they vanish on hatted 4D components",
    "why_not_matched": "XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent counte
+> rpart; M2 is a 4D statement, so there is nothing to match them to. Their L-coefficients are nonzero (reco
+> rded below); they are retained (R6) and drop out of the ch-projected slot (p_m G_D11 = 0 for the evanesce
+> nt part separately)"
  },
  "M3_status": {
    "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
    "statement": "L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree <= 8 i
+> t is a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)",
    "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching)"
  },
  "statement": "M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR_AB],
+>  f> = <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18",
  "route_B": {
    "equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair",
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 0,
    "first": [],
    "agree": true,
    "constant": "-I/4"
  },
  "FT_constant": {
    "master_pole": "1",
    "master_finite": "2 - L",
    "pole_is_1": true,
    "L_coefficient_of_MB": "-1",
    "c_FT_from_DR_bubble": "I*pi**2",
    "c_FT_from_minus_r": "I*pi**2",
    "r_certificate": "-i * pi**2",
    "r_certificate_as_expected": true,
    "agree": true
  },
  "controls": {
    "route_B_all_100_pairs_agree": true,
    "FT_constant_two_calibrations_agree": true,
    "route_A_M1_integrand_identity": true,
    "route_A_XVI_O_K_u_equals_T": true,
    "D11_transverse_exact_in_d": true,
    "routes_FP_PV_agree": true,
    "covariant_fit_exact_unique": true,
    "mutations_all_rejected": true
  },
  "analytic_inputs_cited": ["Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, resi
+> due at zeta = -2 is r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bub
+> ble)", "a polynomial in p is the Fourier transform of a distribution supported at {0}", "Route A only: tw
+> o extensions of the product V V (C C + C C) from U differ by a distribution supported at 0 (XVI-O lem:BV_
+> rules_on_U, Constructed)"],
  "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the 4
+> D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0 is
+>  IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns t
+> he classical insertion and is not touched",
  "M3_consequence": "with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both sides a
+> re transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contacts, consistent with
+> N4), so L_match lies in the 2-dim transverse subspace of the p14r h.h span; its coordinates need the XVI-
+> O finite constant of u^R and are NOT computed (M4: booked as P-side hh counterterm; u^R_mu unmodified, no
+>  c delta)"
},
"D11_summary": {
  "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
  "prescription": "operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions, M3 bounds
+>  4/8/4, exactly-degree-four normalization (massless: every local term is a homogeneous quartic in p)",
  "prescription_usage": {
    "exactly_degree_four_normalization": "every structure S_s (p.p)^e of Ghat has total momentum degree 4 (c
+> omputed per structure: g4_mn g4_rs (p.p)^2: 4, g4_mn gt_rs (p.p)^2: 4, g4_mn p_r p_s (p.p)^1: 4, g4_mr g4
+> _ns (p.p)^2: 4, g4_mr gt_ns (p.p)^2: 4, g4_mr p_n p_s (p.p)^1: 4, g4_rs gt_mn (p.p)^2: 4, g4_rs p_m p_n (
+> p.p)^1: 4, gt_mn gt_rs (p.p)^2: 4, gt_mn p_r p_s (p.p)^1: 4, gt_mr gt_ns (p.p)^2: 4, gt_mr p_n p_s (p.p)^
+> 1: 4, gt_rs p_m p_n (p.p)^1: 4, p_m p_n p_r p_s: 4)",
    "N4_contact_exclusions": "used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by N4 do not
+> arise (massless; every pole and local MS-bar term is a homogeneous quartic in p), so N4 removes nothing h
+> ere and no order-0/2 counterterm is needed",
```

```
    "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"
  },
  "UV_pole_in_p14r_hh_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
  "evanescent_pole_nonzero": true,
  "controls": {
    "exactly_degree_four_every_structure": true,
    "N4_no_order_0_or_2_contact_arises": true,
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "integrand_equals_XVI_0_convolution_M1": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true,
    "M2_discharged": true,
    "mutations_all_rejected": true
  }
},
"hh_slot_summary": {
  "verdict": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
  "row": "(S0,Gamma1^[2]) s2 K:K.c",
  "graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
  "how_D11_decouples": "graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse by i
+> tself, 4D and evanescent parts separately; its evanescent poles are nonzero but transverse. Hence D11 mut
+> ations of phase or symmetry factor are NOT detectable by the slot (they are tested by M2 instead); the sl
+> ot sees D11 only through a non-longitudinal operator (trace operator control: nonzero)",
  "surviving_evanescent_structures_in_slot": [],
  "consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop (D11 in
+> cluded), so NO contract revision is made for it.  The summed evanescent poles of the hh Green function ar
+> e nonzero but transverse; renormalising the hh Green function itself would need evanescent h.h counterter
+> ms -- outside this projection and NOT claimed.",
  "controls": {
    "operator_longitudinal": true,
    "p_dot_G_D11_zero_exact_in_d": true,
    "p_dot_G_sum_zero_exact_in_d": true,
    "p_dot_G_evanescent_sum_zero": true,
    "slot_sum_zero_all_7_assignments": true,
    "evanescent_slot_sum_zero_all_7_assignments": true,
    "D01_D03_individually_nonzero_incl_c_hat_h_tilde": true,
    "D11_reaches_slot_under_trace_operator": true,
    "4D_pole_sum_D01_D03_D11_transverse": true,
    "4D_pole_D11_transverse": true,
    "mutations_all_rejected": true
  }
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
```

```
+> ),",
       "T_coordinates": ["0", "0", "1"]
     }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
```

```
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p20.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "vendored",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p20.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": false,
    "inherited_mode": null,
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
      "venv": "/workspace/indep_rebuild_p20/venv"
    }
  },
  "p19_archive": "archival/p19",
  "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
},
"XVI_P_registered": false,
"XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
"XVI_M_pin_ok": true,
"symmetry_restored": false,
"bulk_symmetry_restored": false,
"O1_closure": "NOT_CLOSED",
"insertion_status": "Frozen_negative_obstruction_preserved",
```

```
"XV_B_opened": false,
"XIV_J_opened": false,
"Route_B_discharged": false,
"u_R_c_delta_added": false,
"inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
  "P19_release.json": "8391fa8580cc08c0a732c98cb19d9bfac0589f2729ea8d4a308476091f8f359e",
  "P19_defect.json": "8ce344b285f7a5c4c81776659b690f78f464e1b33d909f2c72b8300ff1dc754e",
  "P20_D11_pin.json": "d4966cca9fc96634b235ee99f34086bbd1d555e5781fd0127fafa55d18533ad6"
},
"p19_layer_change_log_vs_p18": {
  "base": "archival/p18/p18_source_files.json (pinned; files of the p18 zip sha256 7c7d5451adb459092edc2606e
+> 1b3322909325000d47bab89c14b7f7a6d25c92f)",
  "base_pinned_ok": true,
  "reference_file": "reproducibility/reference_values_p23.json listed as added by declaration (written after
+>  the author build)",
  "added": ["archival/p18/README.md", "archival/p18/p18_source_files.json", "archival/p18/reference_values_p
+> 18.json", "archival/p18/reproducibility/author_rebuild_report_p18.json", "archival/p18/reproducibility/in
+> dependent_rebuild_report_p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.json", "arch
+> ival/p19/reference_values_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "arch
+> ival/p19/reproducibility/independent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p2
+> 0_source_files.json", "archival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebu
+> ild_report_p20.json", "archival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/R
+> EADME.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/
+> reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report
+> _p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_
+> source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuil
+> d_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/con
+> tract_registry.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "c
+> ode/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "c
+> ode/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "co
+> de/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/r
+> elease_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_c
+> orrection.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscri
+> pt/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values_p
+> 23.json", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_p22.py", "tests/test
+> _p23.py"],
  "removed": ["reproducibility/reference_values_p18.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/release_p18.py", "code/p5_entry/dred.py
+> ", "manuscript/body/XVI-P-body.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "test
+> s/test_p18.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p18/README.md", "archival
+> /p18/p18_source_files.json", "archival/p18/reference_values_p18.json", "archival/p18/reproducibility/auth
+> or_rebuild_report_p18.json", "archival/p18/reproducibility/independent_rebuild_report_p18.json", "archiva
+> l/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p19.json", "archiv
+> al/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/independent_rebuild
+> _report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/referenc
+> e_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproduc
+> ibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.js
+> on", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.js
+> on", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archi
+> val/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_
+> values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducib
+> ility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/d11_pin.py
+> ", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "code/p5_calc/p19_conventions.py", "code/p5_ca
+> lc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_ca
+> lc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p18.py", "code/p5_calc/re
+> lease_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py
+> ", "code/p5_calc/release_p23.py", "manuscript/body/XVI-P-body.tex", "manuscript/notes/P19_defect.tex", "m
+> anuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evane
+> scent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibili
+> ty/reference_values_p18.json", "reproducibility/reference_values_p23.json", "tests/test_p14.py", "tests/t
+> est_p16.py", "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/t
+> est_p21.py", "tests/test_p22.py", "tests/test_p23.py", "tools/build.py", "tools/rebuild_from_source.py",
+> "tools/source_manifest.py"],
  "counts": {
    "added": 53,
    "removed": 1,
    "modified": 12
  }
},
"p19_layer_disclosed_test_revisions_p19": {
  "tests/test_p14.py": "names p18 -> p19 (reference file, source zip, author report, status revision); gener
+> ated prefixes certificates/ledgers P19_, manuscript/body/P19-",
  "tests/test_p18.py": "rs() reads the p18 LAYER record certificates/P18_release.json; the live p18 independ
+> ent report is bound to the p18 tree (pending or False here); test-file change log accepts p18 or disclose
+> d p19 revisions; build label is the current revision"
},
"erratum_p19": {
  "K1": "p19 stated that full sigma_0 closure of the MS-bar counterterm fails with the derived ghost-equatio
+> n factor +1 and needs -1 (unresolved). RETRACTED: Grassmann-orientation bookkeeping error -- the P15 Sigm
```

```
+> a coordinates are (cb,c)-oriented, the route-G c.cb slots (c,cb), V[c.cb#k] = -V[(cb,c)#k]. Corrected: Si
+> gma = (+3/8, -1/4) in (c,cb) coordinates, derived +1, full closure in h.c, c.b, c.cbs.",
  "restoration": "p19's projected_restoration_verified (P_ch sigma_0(S_ct) = 0, x = 0) followed from the ide
+> ntity + locality; REDEFINED in p20 as full sigma_0 closure of the complete MS-bar S_ct (incl. c.cb), kept
+>  as a labelled route-consistency control",
  "J1": "'over Q(i), the physical field' replaced by: the Q(i) counts are the real dimensions under a compat
+> ible real structure",
  "found_by": "p19 independent rebuild (report sha256 1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf00
+> 8dd2811f45)"
},
"K1_correction": {
  "content_revision_p20": true,
  "orientation_ratio_minus_one": true,
  "Sigma_pole_routeG_c_cb": true,
  "derived_factor_plus_one": true,
  "derived_relation_reproduces_P15": true,
  "full_closure_complete_S_ct": true,
  "without_ccb_not_closed": true,
  "tree_controls_ok": true,
  "p20_controls_all_true": true,
  "detector_drop_D13": true,
  "detector_flip_Kc": true,
  "detector_drop_D11_recorded_blind": true,
  "route_consistency_control_holds": true,
  "quotient_incl_Sigma_zero": true,
  "quotient_Sigma_alone_c_cb_plus_3_8_minus_1_4": true,
  "J1_wording": true,
  "restoration_definition_p20": true,
  "bulk_false": true
},
"retraction_check": {
  "phrases": ["full c.b / ghost-sector closure FAILS", "full c.b / ghost-sector closure of the MS-bar counte
+> rterm FAILS", "for which full closure fails", "with +1 full closure fails", "full closure \\emph{fails}",
+>  "This is unresolved", "This is not resolved in p19", "needs relative factor -1; the derived ghost-equati
+> on relation gives +1; not resolved", "factor +1 (K1) full c.b closure FAILS", "full c.b closure NOT estab
+> lished (K1)", "the physical field"],
  "scope": ["README.md", "manuscript/notes/P19_defect.tex", "certificates/P19_defect.json", "certificates/P1
+> 9_release.json", "manuscript/body/P19-results-table.tex"],
  "hits": {}
},
"D11_pin": {
  "check": {
    "ratio_pin_PASS": true,
    "ratio_10": true,
    "transverse": true,
    "C2_quadratic_form_half_P2": true,
    "GaussBonnet_zero": true,
    "absolute_pin_PASS": true,
    "absolute_pin_flag": true,
    "top_level_absolute_status_consistent": true,
    "top_level_ratio_status_consistent": true,
    "absolute_ratios_one": true,
    "no_external_convention_choice": true,
    "drop_D11_detected": true,
    "x2_detected_by_absolute": true,
    "sign_flip_detected_by_absolute": true,
    "corrected_in_p21": true,
    "literature_source_stated": true,
    "revision": true
  },
  "ratio_pin": {
    "status": "PASS",
    "statement": "spin-0 : spin-2 of the D11 4D UV pole = 10 : 1, derived from the certificate; equals the m
+> inimal-scalar a_2 prediction (1/72)(3) : (1/120)(1/2)"
  },
  "absolute_pin": {
    "status": "PASS",
    "observed_common_factor_c2_vs_bare_a2": "-I",
    "observed_common_factor_c0_vs_bare_a2": "-I",
    "derivation_p21": {
      "package_declarations_checked": {
        "N_eps = i S_eps/(16 pi^2)": true,
        "Gamma_1 = -i * graph sum": true,
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": true,
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": true,
        "g = eta + kappa h": true,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": true,
        "eta = (+,-,-,-)": true,
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": true
      },
      "steps": {
```

```
        "1_G_equals_Gamma2": "1",
        "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": "1",
        "3_i_S_M_over_S_E (must be -1)": "-1",
        "4_L_M_eff_over_L_E_eff": "-1",
        "5_L_M_div_per_a2_over_eps": "1/(32*pi**2)",
        "6_minkowski_quadratic_forms": {
          "Riem2": {
            "P2": "1",
            "P0s": "1"
          },
          "Ric2": {
            "P2": "1/4",
            "P0s": "1"
          },
          "R2": {
            "P2": "0",
            "P0s": "3"
          },
          "C2": {
            "P2": "1/2",
            "P0s": "0"
          },
          "E_GaussBonnet": {
            "P2": "0",
            "P0s": "0"
          }
        }
      },
      "package_Gamma2_pole_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "heat_kernel_prediction_per_kappa2_over_eps": {
        "spin2": "1/(3840*pi**2)",
        "spin0s": "1/(384*pi**2)"
      },
      "ratio_package_over_prediction": {
        "spin2": "1",
        "spin0s": "1"
      },
      "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2 R^2
+>  (Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)", "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps
+> )) int a_2 (Euclidean, d = 4-2eps)"],
      "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction, s
+> ign of W_div, N_eps, kappa normalisation, signature) is fixed by the package declarations checked above",
      "status": "PASS"
    },
    "statement": "p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minimal scalar loop under th
+> e package's declared conventions (G = Gamma^(2); N_eps = i/(16 pi^2) and the +1/eps bubble pole fix the W
+> ick direction, i S_M = -S_E, L_M = -L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(
+> 16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N_eps Ghat/eps exactly (ratio 1
+> , both spins).  External inputs are mathematics only (a_2 coefficients; heat-kernel pole formula); no ext
+> ernal convention choice.  p20 had left this OPEN."
  },
  "spin_decomposition": {
    "c2_spin2": "-I/240",
    "c0_spin0s": "-I/24",
    "projector_norms": ["5", "1"],
    "transverse": true,
    "only_spin2_and_spin0s": true,
    "ratio_c0_over_c2": "10"
  },
  "mutations": {
    "drop_graph_D11": {
      "c2": "0",
      "c0": "0",
      "ratio": "None",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "D11_normalisation_x2": {
      "c2": "-I/120",
      "c0": "-I/12",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
```

```
    "D11_sign_flip": {
      "c2": "I/240",
      "c0": "I/24",
      "ratio": "10",
      "ratio_pin_passes": true,
      "detected_by_ratio_pin": false,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    },
    "swap_g4g4_structures": {
      "c2": "-I/40",
      "c0": "-I/32",
      "ratio": "5/4",
      "ratio_pin_passes": false,
      "detected_by_ratio_pin": true,
      "absolute_pin_passes": false,
      "detected_by_absolute_pin": true
    }
  }
},
"open_pins": [],
"open_pins_closed_in_p21": ["D11_absolute_normalisation (Ghat -> a_2 map not derived)"],
"D11_ratio_pin_verified": "earned",
"D11_absolute_pin_verified": "earned",
"p19_layer": {
  "p19_conventions_byte_identical": true,
  "disclosed_p20_revisions": {
    "code/p5_calc/defect.py": {
      "sha256": "675c1fa812c51b9955dd48a4149d309ff2e99ec7797fad753eb57b37e908ce6f",
      "differs_from_p19": true,
      "disclosed": true
    },
    "code/p5_calc/release_p19.py": {
      "sha256": "4d048345270a39a77cc30f71d3bdcbef17fffec6c44c1ac89487135ea3555970",
      "differs_from_p19": true,
      "disclosed": true
    }
  },
  "P19_release_accepted": true,
  "lower_layers_rechecked_p21": {
    "p18_layer_ok": true,
    "P18_quantum_inputs_evaluated": "earned",
    "inventory_recheck_ok": true
  }
},
"p18_layer_byte_identical": true,
"change_log_vs_p19": {
  "base": "archival/p19/p19_source_files.json (pinned; files of the p19 zip sha256 da9242752d2e3753d8b2ce7fb
+> b21e7cdb28487cc7966c2ca52d1ef1376658875)",
  "base_pinned_ok": true,
  "added": ["archival/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p
+> 19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/in
+> dependent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "arch
+> ival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "arch
+> ival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p2
+> 1_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebu
+> ild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/R
+> EADME.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archiv
+> al/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archiv
+> al/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p
+> 5_calc/d11_pin.py", "code/p5_calc/evanescent.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_co
+> nventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_
+> counting.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py"
+> , "code/p5_calc/release_p23.py", "manuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pi
+> n.tex", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_q
+> uantum_target.json", "reproducibility/reference_values_p23.json", "tests/test_p20.py", "tests/test_p21.py
+> ", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p19.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/defect.py", "code/p5_calc/release_p18.p
+> y", "code/p5_calc/release_p19.py", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.tex", "manuscript
+> /notes/P19_defect.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "tests/test_p18.py
+> ", "tests/test_p19.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": [],
  "counts": {
    "added": 43,
    "removed": 1,
    "modified": 16
  }
},
"disclosed_p19_layer_revisions_p20": {
  "code/p5_calc/defect.py": "K1 orientation fix (measured ratio, Sigma converted to route-G (c,cb)); tree co
```

```
  +> ntrols; full sigma_0 closure of the complete MS-bar S_ct; restoration REDEFINED; full-closure mutation de
  +> tector; p19 criterion relabelled route-consistency control; J1 wording; K1 erratum texts",
    "code/p5_calc/release_p19.py": "claim texts: K1 erratum (p19 'closure fails / needs -1' retracted); restor
  +> ation content checks on the p20 definition; reject removes stale RELEASE_STATUS.json"
  },
  "disclosed_test_revisions_p20": {
    "tests/test_p14.py": "names p19 -> p20; generated prefixes P20_",
    "tests/test_p19.py": "K1 tests corrected (orientation, full closure with derived +1); restoration on the p
  +> 20 definition; J1 wording; p19 independent report bound to the p19 tree; scan sites = p19 + p20 reviewed
  +> lists"
  },
  "independent_rebuild_passed_p19": true,
  "p19_independent_rebuild": {
    "report": "archival/p19/reproducibility/independent_rebuild_report_p19.json",
    "present": true,
    "report_sha256": "1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf008dd2811f45",
    "report_sha256_pinned": true,
    "pass": true,
    "independent_rebuild_passed": true,
    "bound_to_source_tree_sha256": "fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36",
    "bound_to_source_zip_sha256": "da9242752d2e3753d8b2ce7fbb21e7cdb28487cc7966c2ca52d1ef1376658875",
    "tree_matches_p19": true,
    "zip_matches_p19": true,
    "mismatches": 0,
    "findings": ["K1_ghost_closure_sign_is_orientation_bookkeeping", "quotient_coordinates_c.cb_orientation",
  +> "restoration_flag_not_independent", "mutation_criterion_relabelled", "J1_wording", "I2_scan_misses", "gat
  +> e_stale_RELEASE_STATUS"],
    "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
  +> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
  +> only zip; it was NOT run on a second machine.",
    "live_report_identical_to_archived": true,
    "NOT_inherited_by_p20": "bound to the p19 tree/zip; the p20 tree has its own (pending) independent flag",
    "archived_p19_records_pinned": {
      "archival/p19/reference_values_p19.json": true,
      "archival/p19/reproducibility/author_rebuild_report_p19.json": true,
      "archival/p19/reproducibility/independent_rebuild_report_p19.json": true,
      "archival/p19/p19_source_files.json": true
    },
    "pins_ok": true
  }
}
```


43 P.5 identity: projected ch-identity defect, counterterm solve, full closure (p19 layer, corrected in p20)

[listing 10/133] certificates/P19_defect.json


```
{
  "landmark": "XVI-P",
  "revision": "p19",
  "deliverable": "P.5 identity: projected one-loop kappa^2 ch-identity defect, counterterm solve",
  "object": "Delta[c,h] = ch projection (P.3, frozen) of the hbar kappa^2 part of (S_Psi, Gamma_Psi) = s_hh X_
  +> hh + s_Kc X_Kc + X_1pt (rows of ledgers/P13_ch_dependency_rows.json)",
  "prescription": "declared (P14_prescription_D11_matching; DRED R1-R6; MS-bar; MB = 1/eps + 2 - L)",
  "inventory": [
    {
      "row": "(S0,Gamma1^[2]) s2 K:K.c",
      "graph": "D01",
      "present": true,
      "evaluated": true,
      "status": "regulated"
    },
    {
      "row": "(S0,Gamma1^[2]) s2 K:K.c",
      "graph": "D02",
      "present": true,
      "evaluated": true,
      "status": "vanishing_under_declared_prescription"
    },
    {
      "row": "(S0,Gamma1^[2]) s2 K:K.c",
      "graph": "D03",
      "present": true,
      "evaluated": true,
      "status": "regulated"
    },
    {
```

```
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graph": "D11",
    "present": true,
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"
  },
  {
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graph": "D12",
    "present": true,
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "row": "(S0,Gamma1^[2]) s1 h:h^2",
    "graph": "D13",
    "present": true,
    "evaluated": true,
    "status": "regulated"
  },
  {
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "graph": "D14a",
    "present": true,
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "graph": "D14b",
    "present": true,
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "graph": "D14c",
    "present": true,
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  }
],
"rows": {
  "(S0,Gamma1^[2]) s2 K:K.c": {
    "inputs": ["D01", "D02", "D03", "D11", "D12"],
    "functional": "X_hh = int (s0 h)_(mn) G_hh^{(mn)(rs)} h_(rs)",
    "value_exact_d_times_MB": {
      "Q4S (declared, R2)": "0",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0",
      "c hat, h tilde": "0"
    },
    "evanescent_part": {
      "Q4S (declared, R2)": "0",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0",
      "c hat, h tilde": "0"
    },
    "per_graph": {
      "D01": {
        "Q4S (declared, R2)": "p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*
+> c^.p - a~.c~*b^.p)/(16*(d - 1))",
        "QdS (hat)": "p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      },
      "D03": {
        "Q4S (declared, R2)": "-p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~
+> *c^.p - a~.c~*b^.p)/(16*(d - 1))",
        "QdS (hat)": "-p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
```

```
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
      },
      "D11": {
        "Q4S (declared, R2)": "0",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "0"
      }
    }
  },
  "(S0,Gamma1^[2]) s1 h:h^2": {
    "inputs": ["D13"],
    "functional": "X_Kc = int G_Kc_{(mn) nu} c^nu E^{(mn)(rs)} h_(rs)",
    "value_exact_d_times_MB": {
      "Q4S (declared, R2)": "0",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0",
      "c hat, h tilde": "0"
    },
    "evanescent_part": {
      "Q4S (declared, R2)": "0",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0",
      "c hat, h tilde": "0"
    },
    "D13_coefficients": {
      "T1": "0",
      "T2": "(4*d - 7)/(16*(d - 1))",
      "T3": "-3*(d - 2)/(8*(d - 1))",
      "E1": "0",
      "E2": "3/(16*(d - 1))"
    }
  },
  "(S1,Gamma1^[1]) s2 K:K.c.h": {
    "inputs": ["D14a", "D14b", "D14c"],
    "value": "0 (each one-point tadpole vanishes under the declared prescription, P15_batchA: evaluated, not
+>  Missing)"
  }
},
"external_assignments": {
  "Q4S (declared, R2)": ["c", "a", "b"],
  "QdS (hat)": ["c^", "a^", "b^"],
  "c tilde, h hat": ["c~", "a^", "b^"],
  "c tilde, h mixed": ["c~", "a~", "b^"],
  "c hat, h mixed": ["c^", "a~", "b^"],
  "all tilde": ["c~", "a~", "b~"],
  "c hat, h tilde": ["c^", "a~", "b~"]
},
"defect": {
  "Q4S (declared, R2)": {
    "exact_d_times_MB": "0",
    "pole": "0",
    "nonlocal_L": "0",
    "local": "0"
  },
  "QdS (hat)": {
    "exact_d_times_MB": "0",
    "pole": "0",
    "nonlocal_L": "0",
    "local": "0"
  },
  "c tilde, h hat": {
    "exact_d_times_MB": "0",
    "pole": "0",
    "nonlocal_L": "0",
    "local": "0"
  },
  "c tilde, h mixed": {
    "exact_d_times_MB": "0",
    "pole": "0",
```

```
    "nonlocal_L": "0",
    "local": "0"
  },
  "c hat, h mixed": {
    "exact_d_times_MB": "0",
    "pole": "0",
    "nonlocal_L": "0",
    "local": "0"
  },
  "all tilde": {
    "exact_d_times_MB": "0",
    "pole": "0",
    "nonlocal_L": "0",
    "local": "0"
  },
  "c hat, h tilde": {
    "exact_d_times_MB": "0",
    "pole": "0",
    "nonlocal_L": "0",
    "local": "0"
  }
},
"defect_zero_exact_in_d": true,
"defect_row_sign_independence": {
  "s_hh=1,s_Kc=1": true,
  "s_hh=1,s_Kc=-1": true,
  "s_hh=-1,s_Kc=1": true,
  "s_hh=-1,s_Kc=-1": true
},
"evanescent_in_assembly": "verified explicitly: the evanescent (gt) parts of the hh row (D01+D03+D11) and of
+>  the K.c row (D13 E1, E2) give ZERO for all seven external assignments, exactly in d; the evanescent pole
+> s themselves are nonzero (P18_hh_slot, P15_D13) -- they drop from the ch slots by the decoupling proofs,
+> not by convention",
"route_G": {
  "field": "Q(i)",
  "labels": ["h.h#0", "h.h#1", "h.h#2", "h.h#3", "h.h#4", "c.cb#0", "c.cb#1", "c.K#0", "c.K#1", "c.K#2", "ph
+> i.phi#0"],
  "n": 11,
  "S0_check": {
    "hh_vs_V2_grav_mismatches": 0,
    "ghost_M0_ok": true,
    "ok": true
  },
  "gh1_sectors": ["h.c", "c.b", "c.cbs"],
  "dim_D": 11,
  "image_rank_sigma0_on_Xi": 22,
  "sigma0_rank_on_D": 5,
  "dim_D_cap_ker_sigma0": 6,
  "D_cap_ker_sigma0_basis": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "0", "-2", "1", "0", "0", "0"],
    ["0", "0", "0", "0", "0", "1", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "P_ch_sigma0_rank_on_D": 3,
  "dim_D_cap_ker_P_ch_sigma0": 8,
  "ker_P_ch_basis": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0", "0", "0", "0"],
    ["0", "0", "0", "0", "0", "1", "0", "0", "0", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "0", "0", "1", "0", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "dim_D_cap_Im_sigma0": 6,
  "D_cap_Im_sigma0_basis": [
    ["2", "0", "-2", "0", "0", "0", "-2*I", "I", "0", "0", "0"],
    ["1", "-1", "-2", "2", "0", "I", "0", "0", "I", "0", "0"],
    ["0", "0", "2", "0", "-2", "0", "-2*I", "I", "0", "0", "0"],
    ["0", "0", "0", "0", "0", "I/2", "I/2", "0", "I/2", "I/2", "0"],
    ["0", "0", "0", "0", "0", "0", "I", "0", "0", "I", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "-1"]
  ],
  "Im_in_ker_sigma0": true,
  "Im_in_ker_P_ch_sigma0": true,
  "cohomology_dim": 0,
  "quotient_complement": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"],
```

```
  "quotient_dim": 5,
  "sector_ranks": {
    "h.h": {
      "dim": 5,
      "sigma0_rank": 3,
      "P_ch_sigma0_rank": 3
    },
    "c.cb": {
      "dim": 2,
      "sigma0_rank": 2,
      "P_ch_sigma0_rank": 0
    },
    "c.K": {
      "dim": 3,
      "sigma0_rank": 3,
      "P_ch_sigma0_rank": 1
    },
    "phi.phi": {
      "dim": 1,
      "sigma0_rank": 0,
      "P_ch_sigma0_rank": 0
    }
  }
},
"route_G_Q_split_p14r_convention": {
  "field": "Q re/im split (p14r convention)",
  "labels": ["h.h#0", "h.h#1", "h.h#2", "h.h#3", "h.h#4", "c.cb#0", "c.cb#1", "c.K#0", "c.K#1", "c.K#2", "ph
+> i.phi#0"],
  "n": 11,
  "S0_check": {
    "hh_vs_V2_grav_mismatches": 0,
    "ghost_M0_ok": true,
    "ok": true
  },
  "gh1_sectors": ["h.c", "c.b", "c.cbs"],
  "dim_D": 11,
  "image_rank_sigma0_on_Xi": 22,
  "sigma0_rank_on_D": 6,
  "dim_D_cap_ker_sigma0": 5,
  "P_ch_sigma0_rank_on_D": 4,
  "dim_D_cap_ker_P_ch_sigma0": 7,
  "dim_D_cap_Im_sigma0": 3,
  "Im_in_ker_sigma0": true,
  "Im_in_ker_P_ch_sigma0": true,
  "cohomology_dim": 2,
  "quotient_complement": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1", "c.K#0", "c.K#1", "c.K#2"],
  "quotient_dim": 8,
  "sector_ranks": {
    "h.h": {
      "dim": 5,
      "sigma0_rank": 3,
      "P_ch_sigma0_rank": 3
    },
    "c.cb": {
      "dim": 2,
      "sigma0_rank": 2,
      "P_ch_sigma0_rank": 0
    },
    "c.K": {
      "dim": 3,
      "sigma0_rank": 3,
      "P_ch_sigma0_rank": 1
    },
    "phi.phi": {
      "dim": 1,
      "sigma0_rank": 0,
      "P_ch_sigma0_rank": 0
    }
  }
},
"field_note": "Route G is computed over Q(i) (primary) and, for comparison, over Q with the re/im split used
+>  by the p14r domain certificate.  The p14r densities carry explicit factors i for odd derivative counts (
+> c.K: 3 derivatives), so over Q an i-multiple counts as independent: e.g. the ch image of c.K#0 is i times
+>  an h.h image.  The counts over Q(i) equal the real dimensions under a compatible real structure (one rea
+> l coefficient per density, the i fixed by reality of the action); ranks and solvability are quoted over Q
+> (i).",
"gamma1_local_data_domain_coordinates": {
  "labels": ["h.h#0", "h.h#1", "h.h#2", "h.h#3", "h.h#4", "c.cb#0", "c.cb#1", "c.K#0", "c.K#1", "c.K#2", "ph
+> i.phi#0"],
  "convention": "Ghat coordinates (G = kappa^2 N_eps Ghat MB) on the p14r densities; h.h from D01+D03+D11 (4
+> D structures; D02 = D12 = 0), c.K from P15_D13; c.cb (ghost self-energy, P15_D13) recorded separately: no
```

```
+> t a ch input (P_ch sigma_0 = 0 on c.cb); phi.phi not an input (sigma_0 = 0 on it)",
  "pole": ["-49*I/480", "-41*I/160", "49*I/240", "41*I/80", "-43*I/120", "0", "0", "0", "3/8", "-1/4", "0"],
  "nonlocal_L": ["49*I/480", "41*I/160", "-49*I/240", "-41*I/80", "43*I/120", "0", "0", "0", "-3/8", "1/4",
+> "0"],
  "local": ["-109*I/450", "-623*I/1200", "109*I/225", "623*I/600", "-2741*I/3600", "0", "0", "0", "2/3", "-5
+> /12", "0"],
  "c_cb_ghost_self_energy (P15 (cb,c) orientation)": {
    "pole": ["-3/8", "1/4"],
    "local": ["-2/3", "5/12"]
  },
  "c_cb_ghost_self_energy (route-G (c,cb) orientation, p20)": {
    "pole": ["3/8", "-1/4"],
    "local": ["2/3", "-5/12"]
  }
},
"quotient_coordinates": {
  "complement_directions": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"],
  "pole": ["0", "0", "0", "-3/8", "1/4"],
  "nonlocal_L": ["0", "0", "0", "3/8", "-1/4"],
  "local": ["0", "0", "0", "-2/3", "5/12"],
  "reading": "coordinates modulo D cap Im sigma_0 (field redefinitions / s-exact); the h.h pole is transvers
+> e, i.e. in D cap Im sigma_0, so its quotient coordinates vanish"
},
"quotient_coordinates_including_Sigma": {
  "pole": ["0", "0", "0", "0", "0"],
  "local": ["0", "0", "0", "0", "0"]
},
"quotient_coordinates_Sigma_alone_routeG": {
  "pole": ["0", "0", "0", "3/8", "-1/4"],
  "local": ["0", "0", "0", "2/3", "-5/12"]
},
"quotient_reading_p20": "all in route-G (c,cb) labels: 'quotient_coordinates' is the class of the h.h + c.K
+> Gamma1 data WITHOUT the ghost self-energy, = -[Sigma] (c.cb entries (-3/8, 1/4)); Sigma itself has c.cb c
+> oordinates (+3/8, -1/4) (P15's (cb,c) numbers carry the opposite sign); the complete data incl. Sigma hav
+> e quotient 0, consistent with full closure and cohomology 0.  (The p19 reviewer read the p19 entries as S
+> igma in the wrong orientation; numerically they coincide with P15's (cb,c) Sigma.)",
"routeG_defect_by_sector": {
  "pole": {
    "h.h": 0,
    "c.K": 0
  },
  "nonlocal_L": {
    "h.h": 0,
    "c.K": 0
  },
  "local": {
    "h.h": 0,
    "c.K": 0
  }
},
"counterterm_solve": {
  "system": "Delta + P_ch sigma_0(x) = 0, x in the declared domain D (11 directions), part by part",
  "rank_P_ch_sigma0_routeG": 3,
  "rank_routeC": 3,
  "routeC_image_relations": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "outcome": "ZERO",
  "residual": null,
  "solution": "x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite renorma
+> lisations are ker(P_ch sigma_0) cap D (dim 8), of which 2 modulo D cap Im sigma_0",
  "MSbar_counterterm_ch_image_zero": true,
  "MSbar_counterterm_full_sigma0_image_sectors": ["c.b"],
  "full_closure_relative_factor_table_UNCONVERTED_p19_error": {
    "1": false,
    "-1": true,
    "I": false,
    "-I": false
  },
  "full_closure_reading": "p20 (corrects p19): with the P15 Sigma coordinates converted from the (cb,c) to t
+> he route-G (c,cb) orientation (measured ratio -1), the DERIVED factor +1 gives full sigma_0 closure of th
+> e complete MS-bar counterterm (h.h + c.K + c.cb) in every ghost +1 sector; the p19 table above (factors 1
+> , -1, i, -i on UNCONVERTED coordinates) is the bookkeeping error, kept for the erratum",
  "K1_ghost_closure_convention": {
    "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
    "M_2iD_cK_to_cbc": [
      ["0", "2"],
```

```
      ["-1", "0"],
      ["0", "-1"]
    ],
    "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
    "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
    "derived_equals_P15": true,
    "relative_factor_derived": "1",
    "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
    "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
    "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
    "tree_controls": {
      "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
      "S0_Kc_terms": 16,
      "S0_cbc_terms": 40,
      "sigma0_S0Kc_plus_S0cbc_entries": 0,
      "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
      "ok": true
    },
    "full_closure_complete_S_ct": {
      "closed": true,
      "entries": 0,
      "sectors": []
    },
    "full_closure_S_ct_without_ccb (p19 package S_ct)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"]
    },
    "full_closure_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolv
+> ed' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots)
+> .  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
  },
  "note_full_sigma0": "the h.h + c.K MS-bar counterterm ALONE has a nonzero (c, b) image (delta S0/delta h c
+> arries b); adding the c.cb ghost self-energy counterterm (correct orientation, derived factor +1) closes
+> it: see K1_ghost_closure_convention.full_closure_complete_S_ct"
},
"D_cap_ker_sigma0": {
  "computed": true,
  "dim": 6,
  "dim_D_cap_Im_sigma0": 6,
  "cohomology_dim": 0,
  "sector_ranks": {
    "h.h": {
      "dim": 5,
      "sigma0_rank": 3,
      "P_ch_sigma0_rank": 3
    },
    "c.cb": {
      "dim": 2,
      "sigma0_rank": 2,
      "P_ch_sigma0_rank": 0
    },
    "c.K": {
      "dim": 3,
      "sigma0_rank": 3,
      "P_ch_sigma0_rank": 1
    },
    "phi.phi": {
      "dim": 1,
      "sigma0_rank": 0,
      "P_ch_sigma0_rank": 0
    }
  }
},
"mutations": {
  "drop_graph_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/48",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
```

```
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the ghost loop D03"
  },
  "drop_graph_D01": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the graviton bubble D01"
  },
  "flip_phase_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/24",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/24",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/8"
      }
    },
    "note": "relative sign of D03 flipped"
  },
  "symmetry_factor_D03_x2": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "D03 doubled"
  },
  "wrong_projection_trace_operator_hh": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-3*c^.p*p.p*(6*a.b*p.p - 6*a^.p*b^.p - a~.b~*p.p)/16",
        "QdS (hat)": "-9*c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/8",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-15*a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "eta_mn p_v in place of s0 h (not the P.3 projection)"
  },
  "wrong_projection_gauge_fixed_operator_Kc": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
```

```
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p*(a.b*c^.p*p.p - 3*a.c*b^.p*p.p - 3*a^.p*b.c*p.p + 4*a^.p*b^.p*c^.p - a^.p
+> *b~.c~*p.p - a~.c~*b^.p*p.p)/16",
        "QdS (hat)": "p.p*(a^.b^*c^.p*p.p - 3*a^.c^*b^.p*p.p - 3*a^.p*b^.c^*p.p + 4*a^.p*b^.p*c^.p)/16",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/4",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "de Donder q^2 P in place of the h^2-monomial operator E (not the P.3 projection)"
  },
  "naive_4D_D03_evanescent_dropped": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/24"
      }
    },
    "note": "D03 Gt parts dropped"
  },
  "naive_4D_D13_Gt_dropped": {
    "outcome": "NONZERO, CANCELLABLE by a counterterm in the declared domain",
    "changes_outcome": true,
    "residual": {
      "local": {
        "Q4S (declared, R2)": "-c^.p*p.p*(a.b*p.p - a^.p*b^.p)/12",
        "QdS (hat)": "-c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/12",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/12"
      }
    },
    "note": "D13 evaluated with Gt dropped (P15_D13 mutation fit: T1 = -(d-4)/(16(d-1)))",
    "finite_counterterm": {
      "rank": 3,
      "solvable": true,
      "solution": ["(6*x3 + 12*x4 - 24*I*x5 - I)/12", "-x3/2", "-(12*x3 + 24*x4 - 24*I*x5 - I)/12", "x3", "x
+> 4", "x5", "x6", "x7"]
    }
  }
},
"insensitive_variations": {
  "drop_graph_D11": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "D11 is transverse by itself (p_m G_D11 = 0 exactly in d): the slot cannot see it; covered
+> by M2 (P18) and by the inventory refusal below"
  },
  "drop_graph_D13": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "A_T1 = A_E1 = 0 and T2, T3, E2 are longitudinal: D13 reaches the slot with zero; covered b
+> y the D13 certificate (two routes, mutations)"
  },
  "flip_row_sign_Kc": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "each row vanishes separately"
  }
},
"missing_control": {
  "D11_removed_from_inventory": {
    "inventory_ok": false,
    "projected_defect_computed_if_removed": "not_earned",
    "changes_outcome": true
  }
```

```
},
"controls": {
  "inventory_complete_no_missing": true,
  "missing_input_refused_not_zero": true,
  "s0h_longitudinal": true,
  "E_transverse": true,
  "hh_row_zero_exact_in_d_all_7": true,
  "hh_row_evanescent_part_zero_all_7": true,
  "hh_row_D01_D03_individually_nonzero": true,
  "Kc_row_zero_exact_in_d_all_7": true,
  "Kc_row_evanescent_part_zero_all_7": true,
  "Kc_slot_map_nontrivial_T1_E1": true,
  "E2_alone_zero_all_7": true,
  "D13_A_T1_A_E1_zero_exact": true,
  "onepoint_row_D14abc_vanishing": true,
  "defect_zero_pole_nonlocal_local_all_7": true,
  "defect_independent_of_row_signs": true,
  "routeG_S0_check": true,
  "routeG_defect_zero_by_sector": true,
  "routeG_hh_local_data_in_span": true,
  "routes_agree_on_ch_image_rank": true,
  "routeG_hh_kernel_equals_p18_transverse_subspace": true,
  "D_cap_Im_inside_D_cap_ker_sigma0": true,
  "D_cap_Im_inside_ker_P_ch_sigma0": true,
  "ghost_equation_relation_reproduces_P15_sigma_pole": true,
  "MSbar_counterterm_in_ker_P_ch_sigma0": true,
  "mutations_all_change_outcome": true,
  "routeG_Q_split_reproduces_p14r_counts": true
},
"why_zero": "each surviving ch row vanishes separately in the projected slot; no counterterm cancellation an
+> d no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupl
+> ing -- D01 and D03 are individually nonzero and cancel against each other in the slot (p17/p18 transverse
+>  sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only throu
+> gh T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-poin
+> t row: the tadpoles D14a-c vanish under the declared prescription.  The defect is therefore zero without
+> any counterterm (solve x = 0).",
"rests_on": ["M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D compon
+> ents'; evanescent components of the D11 matching: not applicable, not verified)", "the evanescent decoupl
+> ing proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row decoupling)", "the frozen P.3 projection
+> and the P13 ch dependency ledger (three surviving rows)", "the declared prescription (DRED R1-R6, MS-bar,
+>  P14_prescription_D11_matching)"],
"flag_definitions": {
  "projected_defect_computed": "Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from every evaluat
+> ed input of the three surviving rows (none Missing), exact in d, pole + nonlocal + MS-bar local parts, al
+> l seven external assignments",
  "projected_identity_verified": "Delta = 0 for the REGULATED one-loop functional Gamma1: pole, nonlocal and
+>  local parts vanish for all seven assignments, exactly in d, with all controls true and every mutation ch
+> anging the outcome",
  "projected_restoration_verified": "p20 definition (replaces p19's, which follows from the identity + local
+> ity): FULL sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm S_ct = -(pole) in the decl
+> ared domain, h.h + c.K + c.cb (ghost self-energy Sigma held fixed at its P15 value, orientation converted
+> , derived relative factor +1), in every ghost +1 sector (h.c, c.b, c.cbs), together with projected_identi
+> ty_verified.  Scope: the one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1",
  "bulk_symmetry_restored": "hard stop: False regardless (not claimed; O.1 NOT_CLOSED)"
},
"renormalised_defect_zero": true,
"route_consistency_control_p19_criterion": {
  "label": "route-consistency control (the p19 restoration criterion, NOT a flag): P_ch sigma_0(S_ct) = 0 fo
+> r the h.h + c.K MS-bar counterterm and the solve x = 0; implied by the identity + locality",
  "holds": true
},
"p20_controls": {
  "orientation_ratio_uniform_minus_one": true,
  "tree_ghost_equation_V_cbc_eq_2iD_V_Kc": true,
  "tree_sigma0_S0Kc_plus_S0cbc_zero": true,
  "derived_ghost_relation_reproduces_P15_sigma": true,
  "full_closure_detects_drop_D13_and_flip_Kc": true,
  "full_closure_detects_p19_orientation_error": true,
  "S_ct_without_ccb_not_closed (c.cb counterterm is needed)": true
},
"full_closure_mutation_detector": {
  "test": "full sigma_0 closure (all ghost +1 sectors) of the complete MS-bar counterterm, Sigma held fixed"
+> ,
  "cases": {
    "drop_graph_D11": {
      "closed": true,
      "entries": 0,
      "sectors": [],
      "detected": false
    },
    "drop_graph_D13 (c.K data = 0, Sigma held fixed)": {
```

```
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "drop_graph_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "drop_graph_D01": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "flip_phase_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "symmetry_factor_D03_x2": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "Sigma_orientation_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    }
  },
  "reading": "detects drop D13 and flip K.c (the p19 blind spots) through the c.b sector, and the hh-row mut
+> ations through h.c; drop D11 stays undetected (D11 transverse => its pole is sigma_0-closed by itself): c
+> overed by M2 (P18) and the p20 D11 normalisation pin (P20_D11_pin.json)"
},
"findings": {
  "J1_p14r_counterterm_domain_field": "FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the p14r cou
+> nterterm-domain counts (Im sigma_0 3, quotient 8, D cap ker 5, P_ch sigma_0 rank 4) are artefacts of a re
+> /im split over Q (the densities carry explicit i for odd derivative counts, so i-multiples count as indep
+> endent).  The counts over Q(i) -- sigma_0 rank 5, D cap ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch
+>  sigma_0 rank 3 (= Route C) -- equal the real dimensions under a compatible real structure.  The p14r cer
+> tificate stays as released; both are reported",
  "K1_ghost_closure_sign": "p20 CORRECTION (erratum to p19): resolved -- an orientation bookkeeping error. W
+> ith the P15 Sigma coordinates converted to the route-G (c,cb) orientation (ratio -1), the derived factor
+> +1 gives full sigma_0 closure of the complete MS-bar counterterm in h.c, c.b, c.cbs.  The p19 text 'closu
+> re fails / needs -1 / unresolved' is retracted"
},
"projected_defect_computed": "earned",
"projected_identity_verified": "earned",
"projected_restoration_verified": "earned",
"bulk_symmetry_restored": false,
"corrected_in_revision": "p20 (K1 orientation fix; restoration redefined; see the p20 erratum)",
"scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the hh an
+> d K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement abou
+> t higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests ful
+> l sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c.b,
+> c.cbs) -- a statement about that counterterm only, not bulk symmetry, not 0.1",
"authorised_d_substitutions": {
  "d13.expand_coeff: Laurent expansion after all contractions and the fit": 61,
  "d13.expand_coeff: diagnostic A'(4)": 61,
  "d13.expand_coeff: diagnostic A(4) (d-dependence proof only)": 61,
  "defect: 4D domain direction image (local, d-free after contraction)": 14,
  "integrals.laurent: d = 4 - 2 eps after all contractions": 1
},
"seconds": 234.9
}
```

44 p19 layer record (release gate p19, revised in p20)

[listing 11/133] certificates/P19_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p19",
  "contract_revision": "p19 (p14r contract + disclosed p15/p16/p17/p18 revisions; none in p19)",
  "date": "2026-10-09",
  "version_label": "XVI-P p19",
  "deliverable": "P.5 identity step (projected one-loop kappa^2 ch-identity defect; counterterm solve; I1-I3)"
  +> ,
  "execution_pass": true,
  "claim_status": "P.5_identity: projected defect assembled from every evaluated input (D01, D02, D03, D11, D1
  +> 2, D13, D14a-c); outcome ZERO. WHY ZERO: each surviving ch row vanishes separately in the projected slot;
  +>  no counterterm cancellation and no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D1
  +> 2): full-row transverse decoupling -- D01 and D03 are individually nonzero and cancel against each other
  +> in the slot (p17/p18 transverse sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13
  +>  reaches the ch slot only through T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 =
  +>  0 exactly in d; (iii) one-point row: the tadpoles D14a-c vanish under the declared prescription.  The de
  +> fect is therefore zero without any counterterm (solve x = 0). RESTS ON: M2 / D11 matching verified on the
  +>  4D components only (D11_matching_verified_scope = '4D components'; evanescent components of the D11 matc
  +> hing: not applicable, not verified); the evanescent decoupling proofs (P16 E2 decoupling, P17/P18 hh evan
  +> escent / full-row decoupling); the frozen P.3 projection and the P13 ch dependency ledger (three survivin
  +> g rows); the declared prescription (DRED R1-R6, MS-bar, P14_prescription_D11_matching). FLAGS: projected_
  +> defect_computed=earned, projected_identity_verified=earned, projected_restoration_verified=earned (defini
  +> tions: flag_definitions). projected_restoration_verified (p20 definition) = full sigma_0 closure of the C
  +> OMPLETE one-loop kappa^2 MS-bar counterterm (h.h + c.K + c.cb; ghost self-energy held fixed; derived rela
  +> tive factor +1; orientation converted) in every ghost +1 sector (h.c, c.b, c.cbs): {'closed': True, 'entr
  +> ies': 0, 'sectors': []}. Scope: that counterterm only -- NOT bulk symmetry (bulk_symmetry_restored=False)
  +> , O.1 NOT_CLOSED. The p19 criterion (P_ch sigma_0(S_ct) = 0, x = 0) follows from the identity + locality
  +> and is kept only as a route-consistency control. ERRATUM (K1): the p19 text 'full closure fails with the
  +> derived factor, needs -1, unresolved' was a Grassmann-orientation bookkeeping error and is RETRACTED. J1:
  +>  the p14r counterterm-domain counts are Q re/im-split artefacts; the Q(i) counts are the real dimensions
  +> under a compatible real structure (finding about the p14r contract; contract not modified)",
  "flag_definitions": {
    "projected_defect_computed": "Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from every evaluat
  +> ed input of the three surviving rows (none Missing), exact in d, pole + nonlocal + MS-bar local parts, al
  +> l seven external assignments",
    "projected_identity_verified": "Delta = 0 for the REGULATED one-loop functional Gamma1: pole, nonlocal and
  +>  local parts vanish for all seven assignments, exactly in d, with all controls true and every mutation ch
  +> anging the outcome",
    "projected_restoration_verified": "p20 definition (replaces p19's, which follows from the identity + local
  +> ity): FULL sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm S_ct = -(pole) in the decl
  +> ared domain, h.h + c.K + c.cb (ghost self-energy Sigma held fixed at its P15 value, orientation converted
  +> , derived relative factor +1), in every ghost +1 sector (h.c, c.b, c.cbs), together with projected_identi
  +> ty_verified.  Scope: the one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1",
    "bulk_symmetry_restored": "hard stop: False regardless (not claimed; O.1 NOT_CLOSED)"
  },
  "projected_restoration_scope": "p20 definition: full sigma_0 closure (h.c, c.b, c.cbs) of the complete one-l
  +> oop kappa^2 MS-bar counterterm incl. the c.cb ghost self-energy counterterm (held fixed, derived factor +
  +> 1); NOT bulk symmetry, NOT O.1",
  "route_consistency_control_p19_criterion": {
    "label": "route-consistency control (the p19 restoration criterion, NOT a flag): P_ch sigma_0(S_ct) = 0 fo
  +> r the h.h + c.K MS-bar counterterm and the solve x = 0; implied by the identity + locality",
    "holds": true
  },
  "full_closure_mutation_detector": {
    "test": "full sigma_0 closure (all ghost +1 sectors) of the complete MS-bar counterterm, Sigma held fixed"
  +> ,
    "cases": {
      "drop_graph_D11": {
        "closed": true,
        "entries": 0,
        "sectors": [],
        "detected": false
      },
      "drop_graph_D13 (c.K data = 0, Sigma held fixed)": {
        "closed": false,
        "entries": 88,
        "sectors": ["c.b"],
        "detected": true
      },
      "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": {
        "closed": false,
        "entries": 88,
        "sectors": ["c.b"],
        "detected": true
      },
      "drop_graph_D03": {
```

```
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "drop_graph_D01": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "flip_phase_D03": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "symmetry_factor_D03_x2": {
      "closed": false,
      "entries": 280,
      "sectors": ["h.c"],
      "detected": true
    },
    "Sigma_orientation_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    }
  },
  "reading": "detects drop D13 and flip K.c (the p19 blind spots) through the c.b sector, and the hh-row mut
+> ations through h.c; drop D11 stays undetected (D11 transverse => its pole is sigma_0-closed by itself): c
+> overed by M2 (P18) and the p20 D11 normalisation pin (P20_D11_pin.json)"
},
"p20_controls": {
  "orientation_ratio_uniform_minus_one": true,
  "tree_ghost_equation_V_cbc_eq_2iD_V_Kc": true,
  "tree_sigma0_SOKc_plus_SOcbc_zero": true,
  "derived_ghost_relation_reproduces_P15_sigma": true,
  "full_closure_detects_drop_D13_and_flip_Kc": true,
  "full_closure_detects_p19_orientation_error": true,
  "S_ct_without_ccb_not_closed (c.cb counterterm is needed)": true
},
"why_zero": "each surviving ch row vanishes separately in the projected slot; no counterterm cancellation an
+> d no cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transverse decoupl
+> ing -- D01 and D03 are individually nonzero and cancel against each other in the slot (p17/p18 transverse
+>  sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13): D13 reaches the ch slot only throu
+> gh T1 and E1 (T2, T3, E2 are longitudinal, E transverse) and A_T1 = A_E1 = 0 exactly in d; (iii) one-poin
+> t row: the tadpoles D14a-c vanish under the declared prescription.  The defect is therefore zero without
+> any counterterm (solve x = 0).",
"rests_on": ["M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D compon
+> ents'; evanescent components of the D11 matching: not applicable, not verified)", "the evanescent decoupl
+> ing proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row decoupling)", "the frozen P.3 projection
+> and the P13 ch dependency ledger (three surviving rows)", "the declared prescription (DRED R1-R6, MS-bar,
+>  P14_prescription_D11_matching)"],
"findings": {
  "J1_p14r_counterterm_domain_field": "FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the p14r cou
+> nterterm-domain counts (Im sigma_0 3, quotient 8, D cap ker 5, P_ch sigma_0 rank 4) are artefacts of a re
+> /im split over Q (the densities carry explicit i for odd derivative counts, so i-multiples count as indep
+> endent).  The counts over Q(i) -- sigma_0 rank 5, D cap ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch
+>  sigma_0 rank 3 (= Route C) -- equal the real dimensions under a compatible real structure.  The p14r cer
+> tificate stays as released; both are reported",
  "K1_ghost_closure_sign": "p20 CORRECTION (erratum to p19): resolved -- an orientation bookkeeping error. W
+> ith the P15 Sigma coordinates converted to the route-G (c,cb) orientation (ratio -1), the derived factor
+> +1 gives full sigma_0 closure of the complete MS-bar counterterm in h.c, c.b, c.cbs.  The p19 text 'closu
+> re fails / needs -1 / unresolved' is retracted"
},
"K1_ghost_closure_convention": {
  "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
  "M_2iD_cK_to_cbc": [
    ["0", "2"],
    ["-1", "0"],
    ["0", "-1"]
  ],
  "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
  "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
  "derived_equals_P15": true,
  "relative_factor_derived": "1",
  "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
  "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
  "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
```

```
  "tree_controls": {
    "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
    "S0_Kc_terms": 16,
    "S0_cbc_terms": 40,
    "sigma0_S0Kc_plus_S0cbc_entries": 0,
    "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
    "ok": true
  },
  "full_closure_complete_S_ct": {
    "closed": true,
    "entries": 0,
    "sectors": []
  },
  "full_closure_S_ct_without_ccb (p19 package S_ct)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"]
  },
  "full_closure_unconverted (p19 error)": {
    "closed": false,
    "entries": 88,
    "sectors": ["c.b"],
    "detected": true
  },
  "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolved
+> ' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots).
+>  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
},
"scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the hh an
+> d K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement abou
+> t higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests ful
+> l sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c.b,
+> c.cbs) -- a statement about that counterterm only, not bulk symmetry, not O.1",
"release_claim_accepted": true,
"release_claim_accepted_definition": "p14r contract accepted AND p15..p18 layers accepted and byte-identical
+>  (p18 layer with quantum_inputs_evaluated earned) AND contract identity (dred.py byte-identical to p18; n
+> o new contract file) AND the inventory recheck AND the defect certificate (all controls, all mutations ch
+> ange the outcome, flag implications) AND p19 hard stops",
"publication": "accepted",
"publication_gated_on": ["execution_pass", "quantum_inputs_evaluated (p18 layer + recheck)", "projected_defe
+> ct_computed", "flag implications restoration => identity => computed"],
"flags": {
  "library_certified": true,
  "batchA_evaluated": {
    "execution": true,
    "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
+> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
+> V pole, finite part 0, no p14r counterterm required",
    "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
+>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
+> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
+> e; NOT quantum_inputs_evaluated"
  },
  "D13_evaluated": {
    "execution": true,
    "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
+> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
+> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
+> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
    "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
+> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
+> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
+> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
  },
  "batchC_evaluated": {
    "execution": true,
    "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
+> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
+> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
+> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
    "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
+>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
+> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
  },
  "E2_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof)",
    "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
+>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
  },
  "hh_evanescent_decoupled_from_ch_slot": {
```

```
    "execution": true,
    "claim": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
    "scope": "the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of D01
+>  + D03 (D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green function
+>  outside this projection; D11 (same row) NOT evaluated"
  },
  "D11_matching_M2": {
    "execution": true,
    "claim": "DISCHARGED (4D components)",
    "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the
+>  4D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0
+> is IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns
+>  the classical insertion and is not touched"
  },
  "hh_slot_full_row_decoupled": {
    "execution": true,
    "claim": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
    "scope": "row (S0,Gamma1^[2]) s2 K:K.c; D01 + D02 + D03 + D11 + D12; the summed evanescent poles remain
+> (transverse) UV poles of the hh Green function outside this projection"
  },
  "projected_ch_identity": {
    "execution": true,
    "claim": "ZERO",
    "scope": "ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of the h
+> h and K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no statement
+> about higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) additionally tests
+>  full sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost +1 sectors (h.c, c
+> .b, c.cbs) -- a statement about that counterterm only, not bulk symmetry, not O.1"
  }
},
"projected_defect_computed": "earned",
"projected_identity_verified": "earned",
"projected_restoration_verified": "earned",
"projected_defect_summary": {
  "defect_zero_exact_in_d": true,
  "evanescent_in_assembly": "verified explicitly: the evanescent (gt) parts of the hh row (D01+D03+D11) and
+> of the K.c row (D13 E1, E2) give ZERO for all seven external assignments, exactly in d; the evanescent po
+> les themselves are nonzero (P18_hh_slot, P15_D13) -- they drop from the ch slots by the decoupling proofs
+> , not by convention",
  "quotient_coordinates": {
    "complement_directions": ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"],
    "pole": ["0", "0", "0", "-3/8", "1/4"],
    "nonlocal_L": ["0", "0", "0", "3/8", "-1/4"],
    "local": ["0", "0", "0", "-2/3", "5/12"],
    "reading": "coordinates modulo D cap Im sigma_0 (field redefinitions / s-exact); the h.h pole is transve
+> rse, i.e. in D cap Im sigma_0, so its quotient coordinates vanish"
  },
  "D_cap_ker_sigma0": {
    "computed": true,
    "dim": 6,
    "dim_D_cap_Im_sigma0": 6,
    "cohomology_dim": 0,
    "sector_ranks": {
      "h.h": {
        "dim": 5,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 3
      },
      "c.cb": {
        "dim": 2,
        "sigma0_rank": 2,
        "P_ch_sigma0_rank": 0
      },
      "c.K": {
        "dim": 3,
        "sigma0_rank": 3,
        "P_ch_sigma0_rank": 1
      },
      "phi.phi": {
        "dim": 1,
        "sigma0_rank": 0,
        "P_ch_sigma0_rank": 0
      }
    }
  },
  "field_note": "Route G is computed over Q(i) (primary) and, for comparison, over Q with the re/im split us
+> ed by the p14r domain certificate.  The p14r densities carry explicit factors i for odd derivative counts
+>  (c.K: 3 derivatives), so over Q an i-multiple counts as independent: e.g. the ch image of c.K#0 is i tim
+> es an h.h image.  The counts over Q(i) equal the real dimensions under a compatible real structure (one r
+> eal coefficient per density, the i fixed by reality of the action); ranks and solvability are quoted over
```

```
+>  Q(i)."
},
"counterterm_solve": {
  "system": "Delta + P_ch sigma_0(x) = 0, x in the declared domain D (11 directions), part by part",
  "rank_P_ch_sigma0_routeG": 3,
  "rank_routeC": 3,
  "routeC_image_relations": [
    ["1/2", "-1/2", "-1", "1", "0", "0", "0", "0"],
    ["1", "0", "-2", "0", "1", "0", "0", "0"],
    ["-2*I", "0", "2*I", "0", "0", "1", "0", "0"],
    ["0", "0", "0", "0", "0", "0", "1", "0"],
    ["0", "0", "0", "0", "0", "0", "0", "1"]
  ],
  "outcome": "ZERO",
  "residual": null,
  "solution": "x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite renorma
+> lisations are ker(P_ch sigma_0) cap D (dim 8), of which 2 modulo D cap Im sigma_0",
  "MSbar_counterterm_ch_image_zero": true,
  "MSbar_counterterm_full_sigma0_image_sectors": ["c.b"],
  "full_closure_relative_factor_table_UNCONVERTED_p19_error": {
    "1": false,
    "-1": true,
    "I": false,
    "-I": false
  },
  "full_closure_reading": "p20 (corrects p19): with the P15 Sigma coordinates converted from the (cb,c) to t
+> he route-G (c,cb) orientation (measured ratio -1), the DERIVED factor +1 gives full sigma_0 closure of th
+> e complete MS-bar counterterm (h.h + c.K + c.cb) in every ghost +1 sector; the p19 table above (factors 1
+> , -1, i, -i on UNCONVERTED coordinates) is the bookkeeping error, kept for the erratum",
  "K1_ghost_closure_convention": {
    "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_Kc]",
    "M_2iD_cK_to_cbc": [
      ["0", "2"],
      ["-1", "0"],
      ["0", "-1"]
    ],
    "z_cb_derived_from_cK_pole_cbc_orientation": ["-3/8", "1/4"],
    "z_cb_P15_sigma_fit_cbc_orientation": ["-3/8", "1/4"],
    "derived_equals_P15": true,
    "relative_factor_derived": "1",
    "orientation_ratio_routeG_over_cbc": ["-1", "-1"],
    "Sigma_pole_routeG_c_cb_coordinates": ["3/8", "-1/4"],
    "Sigma_local_routeG_c_cb_coordinates": ["2/3", "-5/12"],
    "tree_controls": {
      "ghost_equation_tree_V_cbc_eq_2iD_V_Kc": true,
      "S0_Kc_terms": 16,
      "S0_cbc_terms": 40,
      "sigma0_S0Kc_plus_S0cbc_entries": 0,
      "sigma0_S0Kc_alone_entries (control, nonzero)": 16,
      "ok": true
    },
    "full_closure_complete_S_ct": {
      "closed": true,
      "entries": 0,
      "sectors": []
    },
    "full_closure_S_ct_without_ccb (p19 package S_ct)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"]
    },
    "full_closure_unconverted (p19 error)": {
      "closed": false,
      "entries": 88,
      "sectors": ["c.b"],
      "detected": true
    },
    "reading": "ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -1, unresolv
+> ed' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates placed in route-G (c,cb) slots)
+> .  Corrected: derived +1, full closure holds (finding K1 of the p19 independent rebuild)."
  },
  "note_full_sigma0": "the h.h + c.K MS-bar counterterm ALONE has a nonzero (c, b) image (delta S0/delta h c
+> arries b); adding the c.cb ghost self-energy counterterm (correct orientation, derived factor +1) closes
+> it: see K1_ghost_closure_convention.full_closure_complete_S_ct"
},
"defect_controls": {
  "inventory_complete_no_missing": true,
  "missing_input_refused_not_zero": true,
  "s0h_longitudinal": true,
  "E_transverse": true,
  "hh_row_zero_exact_in_d_all_7": true,
```

```
  "hh_row_evanescent_part_zero_all_7": true,
  "hh_row_D01_D03_individually_nonzero": true,
  "Kc_row_zero_exact_in_d_all_7": true,
  "Kc_row_evanescent_part_zero_all_7": true,
  "Kc_slot_map_nontrivial_T1_E1": true,
  "E2_alone_zero_all_7": true,
  "D13_A_T1_A_E1_zero_exact": true,
  "onepoint_row_D14abc_vanishing": true,
  "defect_zero_pole_nonlocal_local_all_7": true,
  "defect_independent_of_row_signs": true,
  "routeG_SO_check": true,
  "routeG_defect_zero_by_sector": true,
  "routeG_hh_local_data_in_span": true,
  "routes_agree_on_ch_image_rank": true,
  "routeG_hh_kernel_equals_p18_transverse_subspace": true,
  "D_cap_Im_inside_D_cap_ker_sigma0": true,
  "D_cap_Im_inside_ker_P_ch_sigma0": true,
  "ghost_equation_relation_reproduces_P15_sigma_pole": true,
  "MSbar_counterterm_in_ker_P_ch_sigma0": true,
  "mutations_all_change_outcome": true,
  "routeG_Q_split_reproduces_p14r_counts": true
},
"defect_mutations": {
  "drop_graph_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/48",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the ghost loop D03"
  },
  "drop_graph_D01": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "hh row without the graviton bubble D01"
  },
  "flip_phase_D03": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b
+> ^.p)/24",
        "QdS (hat)": "-p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/24",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/8"
      }
    },
    "note": "relative sign of D03 flipped"
  },
  "symmetry_factor_D03_x2": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
```

```
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p - a~.c~*b^
+> .p)/48",
        "QdS (hat)": "p.p**2*(a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/48",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "D03 doubled"
  },
  "wrong_projection_trace_operator_hh": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "-3*c^.p*p.p*(6*a.b*p.p - 6*a^.p*b^.p - a~.b~*p.p)/16",
        "QdS (hat)": "-9*c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/8",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-15*a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "eta_mn p_v in place of s0 h (not the P.3 projection)"
  },
  "wrong_projection_gauge_fixed_operator_Kc": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p*(a.b*c^.p*p.p - 3*a.c*b^.p*p.p - 3*a^.p*b.c*p.p + 4*a^.p*b^.p*c^.p - a^.p
+> *b~.c~*p.p - a~.c~*b^.p*p.p)/16",
        "QdS (hat)": "p.p*(a^.b^*c^.p*p.p - 3*a^.c^*b^.p*p.p - 3*a^.p*b^.c^*p.p + 4*a^.p*b^.p*c^.p)/16",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/4",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
      }
    },
    "note": "de Donder q^2 P in place of the h^2-monomial operator E (not the P.3 projection)"
  },
  "naive_4D_D03_evanescent_dropped": {
    "outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no local counter
+> term can cancel it)",
    "changes_outcome": true,
    "residual": {
      "nonlocal_L": {
        "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/48",
        "QdS (hat)": "0",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "a~.c~*b^.p*p.p**2/48",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/24"
      }
    },
    "note": "D03 Gt parts dropped"
  },
  "naive_4D_D13_Gt_dropped": {
    "outcome": "NONZERO, CANCELLABLE by a counterterm in the declared domain",
    "changes_outcome": true,
    "residual": {
      "local": {
        "Q4S (declared, R2)": "-c^.p*p.p*(a.b*p.p - a^.p*b^.p)/12",
        "QdS (hat)": "-c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)/12",
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0",
        "c hat, h tilde": "-a~.b~*c^.p*p.p**2/12"
      }
    },
    "note": "D13 evaluated with Gt dropped (P15_D13 mutation fit: T1 = -(d-4)/(16(d-1)))",
```

```
    "finite_counterterm": {
      "rank": 3,
      "solvable": true,
      "solution": ["(6*x3 + 12*x4 - 24*I*x5 - I)/12", "-x3/2", "-(12*x3 + 24*x4 - 24*I*x5 - I)/12", "x3", "x
+> 4", "x5", "x6", "x7"]
    }
  }
},
"defect_insensitive_variations": {
  "drop_graph_D11": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "D11 is transverse by itself (p_m G_D11 = 0 exactly in d): the slot cannot see it; covered
+> by M2 (P18) and by the inventory refusal below"
  },
  "drop_graph_D13": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "A_T1 = A_E1 = 0 and T2, T3, E2 are longitudinal: D13 reaches the slot with zero; covered b
+> y the D13 certificate (two routes, mutations)"
  },
  "flip_row_sign_Kc": {
    "outcome": "ZERO",
    "changes_outcome": false,
    "why_blind": "each row vanishes separately"
  }
},
"quantum_inputs_evaluated": "earned",
"quantum_inputs_inventory": [
  {
    "graph": "D01",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D02",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D03",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated"
  },
  {
    "graph": "D11",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"
  },
  {
    "graph": "D12",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D13",
    "slot_row": "(S0,Gamma1^[2]) s1 h:h^2",
    "evaluated": true,
    "status": "evaluated"
  },
  {
    "graph": "D14a",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14b",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription"
  },
  {
    "graph": "D14c",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
```

```
    "status": "vanishing_under_declared_prescription"
  }
],
"D11_matching_verified": true,
"D11_matching_verified_scope": "4D components",
"D11_evanescent_components_matching": "not_applicable_not_verified",
"M3_status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
"D11_evaluated": true,
"hh_full_row_decoupling_proven": true,
"evanescent_hh_counterterm_added": false,
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"hh_evanescent_status": "decoupled_from_ch_slot_proven_as_sum",
"hh_evanescent_decoupling_proven": true,
"D01_evaluated": true,
"D03_evaluated": true,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
```

```
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
```

```
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D11": {
    "status": "evaluated",
    "certificate": "certificates/P18_D11.json",
    "sector_status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
    "M2": "DISCHARGED (4D components)",
    "ch_slot": "decoupled by itself (p_m G_D11 = 0 exactly in d)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_byte_identical_to_p18": true,
  "dred_registry_state_p22": "p18 + p22 guard block",
  "contract_revision_in_p19": false,
  "disclosed_revisions_p19": {}
},
"p18_layer": {
  "p18_layer_files_byte_identical": true,
  "P18_release_accepted": true,
  "P18_quantum_inputs_evaluated": "earned",
  "I1_gate": "the p19 gate requires the p18 layer record's quantum_inputs_evaluated == 'earned'"
},
"change_log_vs_p18": {
  "base": "archival/p18/p18_source_files.json (pinned; files of the p18 zip sha256 7c7d5451adb459092edc2606e
+> 1b3322909325000d47bab89c14b7f7a6d25c92f)",
  "base_pinned_ok": true,
  "reference_file": "reproducibility/reference_values_p23.json listed as added by declaration (written after
+>  the author build)",
  "added": ["archival/p18/README.md", "archival/p18/p18_source_files.json", "archival/p18/reference_values_p
+> 18.json", "archival/p18/reproducibility/author_rebuild_report_p18.json", "archival/p18/reproducibility/in
+> dependent_rebuild_report_p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.json", "arch
+> ival/p19/reference_values_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "arch
+> ival/p19/reproducibility/independent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p2
+> 0_source_files.json", "archival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebu
+> ild_report_p20.json", "archival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/R
+> EADME.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/
+> reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report
+> _p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_
+> source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuil
+> d_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/con
+> tract_registry.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "c
+> ode/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "c
+> ode/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "co
+> de/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/r
+> elease_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_c
+> orrection.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscri
+> pt/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values_p
+> 23.json", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_p22.py", "tests/test
+> _p23.py"],
  "removed": ["reproducibility/reference_values_p18.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_calc/release_p18.py", "code/p5_entry/dred.py
+> ", "manuscript/body/XVI-P-body.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "test
+> s/test_p18.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p18/README.md", "archival
+> /p18/p18_source_files.json", "archival/p18/reference_values_p18.json", "archival/p18/reproducibility/auth
+> or_rebuild_report_p18.json", "archival/p18/reproducibility/independent_rebuild_report_p18.json", "archiva
+> l/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p19.json", "archiv
+> al/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/independent_rebuild
+> _report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/referenc
+> e_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproduc
+> ibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.js
+> on", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.js
+> on", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archi
+> val/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_
```

```
+> values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducib
+> ility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/d11_pin.py
+> ", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "code/p5_calc/p19_conventions.py", "code/p5_ca
+> lc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_ca
+> lc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p18.py", "code/p5_calc/re
+> lease_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py
+> ", "code/p5_calc/release_p23.py", "manuscript/body/XVI-P-body.tex", "manuscript/notes/P19_defect.tex", "m
+> anuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evane
+> scent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibili
+> ty/reference_values_p18.json", "reproducibility/reference_values_p23.json", "tests/test_p14.py", "tests/t
+> est_p16.py", "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/t
+> est_p21.py", "tests/test_p22.py", "tests/test_p23.py", "tools/build.py", "tools/rebuild_from_source.py",
+> "tools/source_manifest.py"],
  "counts": {
    "added": 53,
    "removed": 1,
    "modified": 12
  }
},
"disclosed_test_revisions_p19": {
  "tests/test_p14.py": "names p18 -> p19 (reference file, source zip, author report, status revision); gener
+> ated prefixes certificates/ledgers P19_, manuscript/body/P19-",
  "tests/test_p18.py": "rs() reads the p18 LAYER record certificates/P18_release.json; the live p18 independ
+> ent report is bound to the p18 tree (pending or False here); test-file change log accepts p18 or disclose
+> d p19 revisions; build label is the current revision"
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
    "C7_grav_noether_k0_k1_k2_on_exports": true,
    "quartic_mutations_all_rejected": true,
    "master_grav_noether_deterministic_k0_k1_k2": true,
    "master_engine_parts_exact": true,
    "graviton_hessian_generic_symbolic_k": true,
    "canonicity_all_23_antifield_components": true,
    "ch_enumeration_lemma_mechanical": true,
    "residual_record_invariants_all_certificates": true,
    "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
    "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
    "cross_certificate_quartic_vs_master_agree": true,
    "stored_C7_flag_equals_recomputation": true
  },
  "failed_prerequisites": [],
  "hash_consistency": {
    "consistent": true,
    "ell": {
      "ell2": {
        "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
      },
      "ell3": {
        "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
      },
      "ell4": {
        "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
```

```
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
    "quartic_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    },
    "master_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    }
  }
},
"hard_stops_p13_hold": true,
"hard_stops_p13": {
  "quantum_inputs_evaluated": "not_earned",
  "projected_identity_verified": "not_earned",
  "D02_quantum_evaluated": false,
  "D12_quantum_evaluated": false,
  "D12_status": "Missing",
  "symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
```

```
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"hard_stops": {
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "evaluated; 4D sector renormalized (MS-bar, declared domain); evanescent regulated (p
+> oles retained, not subtracted), decoupled from the ch slots (p15-p18 inputs D01, D02, D03, D11, D12, D13,
+>  D14a-c)",
    "P.5_identity": "projected defect computed (zero exactly in d: pole, nonlocal and local parts, all seven
+>  external assignments); projected identity verified; projected restoration verified for the renormalised
+> Gamma_R = Gamma1 + S_ct (ch projection, one loop, kappa^2 only; zero because each row vanishes separately
+> , no counterterm cancellation); projected restoration (p20 definition): full sigma_0 closure of the compl
+> ete one-loop kappa^2 MS-bar counterterm incl. c.cb (K1 orientation corrected in p20; p19 'closure fails'
+> retracted); bulk symmetry NOT restored/claimed; O.1 NOT_CLOSED"
  },
  "definition_vs_status_note": "I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficients evalua
+> ted and renormalized'; the STATUS states exactly what holds: 4D sector renormalized, evanescent poles reg
+> ulated and decoupled from the ch slots (not subtracted)",
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)"
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p18": true,
"independent_rebuild_passed_p17": true,
"independent_rebuild_passed_p16": true,
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p19) sourc
+> e tree only (independent: pending until a reviewer report for this tree exists); independent_rebuild_pass
+> ed_p18 refers to the p18 tree bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094 / zip 7c7d
+> 5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f (same-box caveat) and is not inherited by p1
+> 9",
"p18_independent_rebuild": {
  "report": "archival/p18/reproducibility/independent_rebuild_report_p18.json",
  "present": true,
  "report_sha256": "893f167ef7124d981c0569ca386035aed06e0dbb0c90a2362a6ceeafd1930cb2",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094",
  "bound_to_source_zip_sha256": "7c7d5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f",
  "tree_matches_p18": true,
  "zip_matches_p18": true,
  "mismatches": 0,
  "findings": ["I1_release_p18_publication_not_gated_on_quantum_inputs_evaluated", "I2_scan_pattern_gaps", "
+> I3_scope_of_M2_and_evaluated", "I4_table_row_count", "I5_slot_blind_to_D11_normalisation", "I6_test_p16_v
+> ictim_restriction", "I7_pdftotext_dehyphenation", "H1_H5_resolution", "bug_fixes_disclosed"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
```

```
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p19": "bound to the p18 tree/zip; the p19 tree has its own (pending) independent flag",
  "archived_p18_records_pinned": {
    "archival/p18/reference_values_p18.json": true,
    "archival/p18/reproducibility/author_rebuild_report_p18.json": true,
    "archival/p18/reproducibility/independent_rebuild_report_p18.json": true,
    "archival/p18/p18_source_files.json": true
  },
  "pins_ok": true
},
"p17_independent_rebuild": {
  "report": "archival/p17/reproducibility/independent_rebuild_report_p17.json",
  "present": true,
  "report_sha256": "92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
  "bound_to_source_zip_sha256": "7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416",
  "tree_matches_p17": true,
  "zip_matches_p17": true,
  "mismatches": 0,
  "findings": ["H1_static_scan_pattern_gaps", "H2_assignment_table_omits_c_hat_h_tilde", "H3_stale_comments"
+> , "H4_curly_apostrophes_in_typeset_listings", "H5_parse_d_srepr", "H6_scope", "G1_G2_G3_G4_resolution"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p18": "bound to the p17 tree/zip; the p18 tree has its own (pending) independent flag",
  "archived_p17_records_pinned": {
    "archival/p17/reference_values_p17.json": true,
    "archival/p17/reproducibility/author_rebuild_report_p17.json": true,
    "archival/p17/reproducibility/independent_rebuild_report_p17.json": true,
    "archival/p17/p17_source_files.json": true
  },
  "pins_ok": true
},
"p16_independent_rebuild": {
  "report": "archival/p16/reproducibility/independent_rebuild_report_p16.json",
  "present": true,
  "report_sha256": "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
  "bound_to_source_zip_sha256": "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57",
  "tree_matches_p16": true,
  "zip_matches_p16": true,
  "mismatches": 0,
  "findings": ["G1_static_scan_scope_narrower_than_disclosure", "G2_plain_symbol_d_from_stored_strings", "G3
+> _hook_coverage_detail", "G4_change_log_omits_new_reference_file", "G5_test_p15_assertion_relaxed", "G6_im
+> aginary_normalisation", "G7_scope", "F3_F4_F5_from_p15"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent flag",
  "archived_p16_records_pinned": {
    "archival/p16/reference_values_p16.json": true,
    "archival/p16/reproducibility/author_rebuild_report_p16.json": true,
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": true,
    "archival/p16/p16_source_files.json": true
  },
  "pins_ok": true
},
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
```

```
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"M2_summary": {
  "verdict": "DISCHARGED (4D components)",
  "D11_matching_verified_scope": "4D components",
  "evanescent_components_matching": {
    "status": "not_applicable_not_verified",
    "what_they_are": "the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt): compo
+> nents of the hh two-point function with at least one index pair in the epsilon-scalar (4-d) directions of
+>  DRED; they vanish on hatted 4D components",
    "why_not_matched": "XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent counte
+> rpart; M2 is a 4D statement, so there is nothing to match them to. Their L-coefficients are nonzero (reco
+> rded below); they are retained (R6) and drop out of the ch-projected slot (p_m G_D11 = 0 for the evanesce
+> nt part separately)"
  },
  "M3_status": {
    "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
    "statement": "L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree <= 8 i
+> t is a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)",
    "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching)"
  },
  "statement": "M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR_AB],
+>  f> = <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18",
  "route_B": {
    "equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair",
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 0,
    "first": [],
    "agree": true,
    "constant": "-I/4"
```

```
  },
  "FT_constant": {
    "master_pole": "1",
    "master_finite": "2 - L",
    "pole_is_1": true,
    "L_coefficient_of_MB": "-1",
    "c_FT_from_DR_bubble": "I*pi**2",
    "c_FT_from_minus_r": "I*pi**2",
    "r_certificate": "-i * pi**2",
    "r_certificate_as_expected": true,
    "agree": true
  },
  "controls": {
    "route_B_all_100_pairs_agree": true,
    "FT_constant_two_calibrations_agree": true,
    "route_A_M1_integrand_identity": true,
    "route_A_XVI_O_K_u_equals_T": true,
    "D11_transverse_exact_in_d": true,
    "routes_FP_PV_agree": true,
    "covariant_fit_exact_unique": true,
    "mutations_all_rejected": true
  },
  "analytic_inputs_cited": ["Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, resi
+> due at zeta = -2 is r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bub
+> ble)", "a polynomial in p is the Fourier transform of a distribution supported at {0}", "Route A only: tw
+> o extensions of the product V V (C C + C C) from U differ by a distribution supported at 0 (XVI-O lem:BV_
+> rules_on_U, Constructed)"],
  "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the 4
+> D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0 is
+>  IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns t
+> he classical insertion and is not touched",
  "M3_consequence": "with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both sides a
+> re transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contacts, consistent with
+> N4), so L_match lies in the 2-dim transverse subspace of the p14r h.h span; its coordinates need the XVI-
+> O finite constant of u^R and are NOT computed (M4: booked as P-side hh counterterm; u^R_mu unmodified, no
+>  c delta)"
},
"D11_summary": {
  "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
  "prescription": "operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions, M3 bounds
+>  4/8/4, exactly-degree-four normalization (massless: every local term is a homogeneous quartic in p)",
  "prescription_usage": {
    "exactly_degree_four_normalization": "every structure S_s (p.p)^e of Ghat has total momentum degree 4 (c
+> omputed per structure: g4_mn g4_rs (p.p)^2: 4, g4_mn gt_rs (p.p)^2: 4, g4_mn p_r p_s (p.p)^1: 4, g4_mr g4
+> _ns (p.p)^2: 4, g4_mr gt_ns (p.p)^2: 4, g4_mr p_n p_s (p.p)^1: 4, g4_rs gt_mn (p.p)^2: 4, g4_rs p_m p_n (
+> p.p)^1: 4, gt_mn gt_rs (p.p)^2: 4, gt_mn p_r p_s (p.p)^1: 4, gt_mr gt_ns (p.p)^2: 4, gt_mr p_n p_s (p.p)^
+> 1: 4, gt_rs p_m p_n (p.p)^1: 4, p_m p_n p_r p_s: 4)",
    "N4_contact_exclusions": "used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by N4 do not
+> arise (massless; every pole and local MS-bar term is a homogeneous quartic in p), so N4 removes nothing h
+> ere and no order-0/2 counterterm is needed",
    "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"
  },
  "UV_pole_in_p14r_hh_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
  "evanescent_pole_nonzero": true,
  "controls": {
    "exactly_degree_four_every_structure": true,
    "N4_no_order_0_or_2_contact_arises": true,
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "integrand_equals_XVI_O_convolution_M1": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true,
    "M2_discharged": true,
    "mutations_all_rejected": true
  }
},
"hh_slot_summary": {
  "verdict": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
  "row": "(SO,Gamma1^[2]) s2 K:K.c",
  "graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
  "how_D11_decouples": "graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse by i
+> tself, 4D and evanescent parts separately; its evanescent poles are nonzero but transverse. Hence D11 mut
+> ations of phase or symmetry factor are NOT detectable by the slot (they are tested by M2 instead); the sl
+> ot sees D11 only through a non-longitudinal operator (trace operator control: nonzero)",
  "surviving_evanescent_structures_in_slot": [],
```

```
  "consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop (D11 in
+> cluded), so NO contract revision is made for it.  The summed evanescent poles of the hh Green function ar
+> e nonzero but transverse; renormalising the hh Green function itself would need evanescent h.h counterter
+> ms -- outside this projection and NOT claimed.",
  "controls": {
    "operator_longitudinal": true,
    "p_dot_G_D11_zero_exact_in_d": true,
    "p_dot_G_sum_zero_exact_in_d": true,
    "p_dot_G_evanescent_sum_zero": true,
    "slot_sum_zero_all_7_assignments": true,
    "evanescent_slot_sum_zero_all_7_assignments": true,
    "D01_D03_individually_nonzero_incl_c_hat_h_tilde": true,
    "D11_reaches_slot_under_trace_operator": true,
    "4D_pole_sum_D01_D03_D11_transverse": true,
    "4D_pole_D11_transverse": true,
    "mutations_all_rejected": true
  }
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
```

```
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
```

```
    "reference_values": "reproducibility/reference_values_p23.json",
    "author": {
      "report": "reproducibility/author_rebuild_report_p19.json",
      "present": true,
      "report_pass": true,
      "report_source_tree_sha256": "fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36",
      "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
      "bound_to_current_source": false,
      "report_records_source_zip_sha256": true,
      "inherited_mode": "vendored",
      "environment": {
        "python": "3.13.5",
        "sympy": "1.14.0",
        "mpmath": "1.3.0",
        "pytest": "9.1.1",
        "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
        "pdftotext": "pdftotext version 25.03.0",
        "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
      }
    },
    "independent": {
      "report": "reproducibility/independent_rebuild_report_p19.json",
      "present": true,
      "report_pass": true,
      "report_source_tree_sha256": "fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36",
      "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
      "bound_to_current_source": false,
      "report_records_source_zip_sha256": false,
      "inherited_mode": null,
      "environment": {
        "python": "3.13.5",
        "sympy": "1.14.0",
        "mpmath": "1.3.0",
        "pytest": "9.1.1",
        "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
        "pdftotext": "pdftotext version 25.03.0",
        "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
        "venv": "/workspace/indep_rebuild_p19/venv"
      }
    },
    "p18_archive": "archival/p18",
    "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
  },
  "XVI_P_registered": false,
  "XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
  "XVI_M_pin_ok": true,
  "symmetry_restored": false,
  "bulk_symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
    "P14_release.json": "23b5b7bd32da5e33b794827ae40dc2de0a73a547c9ec26d7a69b08b87e727b63",
    "P15_release.json": "6010e0f56336e10d594a2834dc16395aebe89690224b08f293b82fa5db4797de",
    "P16_release.json": "f81db7fdaf2e28a4451cfabb97db61b46b7f472e4b45ed911b8326b87d2893d6",
    "P17_release.json": "97a548b4d29495bf648d1cb93b00f886980128a9d2eb5d8f7ddbd50f09ab13f1",
    "P18_release.json": "93543ccdd7c0b3f4f95d0c3f6d12afcd078e5f04c4abebb35a7bd22ddeb5f938",
    "P19_defect.json": "8ce344b285f7a5c4c81776659b690f78f464e1b33d909f2c72b8300ff1dc754e"
  }
}
```


45 Milestone ledger (p19 layer)

[listing 12/133] ledgers/P19_milestones.json


```
{
  "landmark": "XVI-P",
  "revision": "p19",
  "ledger": "milestones (authoritative definitions + status)",
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
```

```
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "evaluated; 4D sector renormalized (MS-bar, declared domain); evanescent regulated (pol
  +> es retained, not subtracted), decoupled from the ch slots (p15-p18 inputs D01, D02, D03, D11, D12, D13, D
  +> 14a-c)",
    "P.5_identity": "projected defect computed (zero exactly in d: pole, nonlocal and local parts, all seven e
  +> xternal assignments); projected identity verified; projected restoration verified for the renormalised Ga
  +> mma_R = Gamma1 + S_ct (ch projection, one loop, kappa^2 only; zero because each row vanishes separately,
  +> no counterterm cancellation); projected restoration (p20 definition): full sigma_0 closure of the complet
  +> e one-loop kappa^2 MS-bar counterterm incl. c.cb (K1 orientation corrected in p20; p19 'closure fails' re
  +> tracted); bulk symmetry NOT restored/claimed; O.1 NOT_CLOSED"
  },
  "definition_vs_status_note": "I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficients evaluate
  +> d and renormalized'; the STATUS states exactly what holds: 4D sector renormalized, evanescent poles regul
  +> ated and decoupled from the ch slots (not subtracted)",
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)",
  "author_rebuild_passed": false,
  "independent_rebuild_passed": false,
  "independent_rebuild_passed_p18": true,
  "hard_stops": {
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "u_R_c_delta_added": false,
    "XVI_P_registered": false
  },
  "flags": {
    "projected_defect_computed": "earned",
    "projected_identity_verified": "earned",
    "projected_restoration_verified": "earned",
    "quantum_inputs_evaluated": "earned",
    "bulk_symmetry_restored": false
  }
}
```


46 p18 layer record (release gate p18)

[listing 13/133] certificates/P18_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p18",
  "contract_revision": "p18 (p14r contract + disclosed p15/p16/p17/p18 revisions)",
  "date": "2026-10-09",
  "version_label": "XVI-P p18",
  "deliverable": "P.5 calculation, fourth step (M2 discharge, D11 evaluation, full hh slot; H1-H5)",
  "execution_pass": true,
  "claim_status": "P.5_calculation: every graph of the inventory (D01, D02, D03, D11, D12, D13, D14a-c) evalua
  +> ted under the one declared prescription; M2 discharged (4D components); the hh row decoupled from the ch-
  +> projected slot (D11 by itself, D01 + D03 as the sum); P.5_identity open",
  "scope": "quantum inputs = the evaluated one-loop graphs of the three ch slots (hh, K.c, one-point) under DR
  +> ED R1-R6 / Feynman +i0 / MS-bar / S0, d opaque. NOT the projected defect/identity, NOT projected restorat
  +> ion, NOT bulk restoration, NOT the insertion. Evanescent poles of the hh (D01, D03, D11) and K.c (D13 E2)
  +>  Green functions are retained and nonzero; they decouple from the ch projection, and renormalising those
  +> Green functions themselves would need evanescent counterterms (not claimed).",
  "release_claim_accepted": true,
  "release_claim_accepted_definition": "p14r contract accepted AND p15, p16, p17 layers accepted and byte-iden
  +> tical AND contract identity (only dred.py changed vs p14r; vs p17 comments/docstrings only; no new contra
  +> ct file) AND M2 discharged AND D11 evaluated AND the full-row slot decoupling AND p18 hard stops",
  "publication": "accepted",
  "flags": {
    "library_certified": true,
    "batchA_evaluated": {
      "execution": true,
      "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
  +> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
```

```
+> V pole, finite part 0, no p14r counterterm required",
    "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
+>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
+> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
+> e; NOT quantum_inputs_evaluated"
  },
  "D13_evaluated": {
    "execution": true,
    "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
+> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
+> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
+> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
    "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
+> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
+> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
+> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
  },
  "batchC_evaluated": {
    "execution": true,
    "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
+> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
+> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
+> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
    "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
+>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
+> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
  },
  "E2_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof)",
    "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
+>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
  },
  "hh_evanescent_decoupled_from_ch_slot": {
    "execution": true,
    "claim": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
    "scope": "the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of D01
+>  + D03 (D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green function
+>  outside this projection; D11 (same row) NOT evaluated"
  },
  "D11_matching_M2": {
    "execution": true,
    "claim": "DISCHARGED (4D components)",
    "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the
+>  4D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0
+> is IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns
+>  the classical insertion and is not touched"
  },
  "hh_slot_full_row_decoupled": {
    "execution": true,
    "claim": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
    "scope": "row (S0,Gamma1^[2]) s2 K:K.c; D01 + D02 + D03 + D11 + D12; the summed evanescent poles remain
+> (transverse) UV poles of the hh Green function outside this projection"
  }
},
"D11_matching_verified": true,
"D11_matching_verified_scope": "4D components",
"D11_evanescent_components_matching": "not_applicable_not_verified",
"M3_status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
"D11_evaluated": true,
"quantum_inputs_evaluated": "earned",
"quantum_inputs_inventory": [
  {
    "graph": "D01",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "regulated",
    "certificate": "certificates/P16_batchC.json"
  },
  {
    "graph": "D02",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription",
    "certificate": "certificates/P15_batchA.json"
  },
  {
    "graph": "D03",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
```

```
    "status": "regulated",
    "certificate": "certificates/P16_batchC.json"
  },
  {
    "graph": "D11",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
    "certificate": "certificates/P18_D11.json"
  },
  {
    "graph": "D12",
    "slot_row": "(S0,Gamma1^[2]) s2 K:K.c",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription",
    "certificate": "certificates/P15_batchA.json"
  },
  {
    "graph": "D13",
    "slot_row": "(S0,Gamma1^[2]) s1 h:h^2",
    "evaluated": true,
    "status": "evaluated",
    "certificate": "certificates/P15_D13.json"
  },
  {
    "graph": "D14a",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription",
    "certificate": "certificates/P15_batchA.json"
  },
  {
    "graph": "D14b",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription",
    "certificate": "certificates/P15_batchA.json"
  },
  {
    "graph": "D14c",
    "slot_row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "evaluated": true,
    "status": "vanishing_under_declared_prescription",
    "certificate": "certificates/P15_batchA.json"
  }
],
"projected_defect_computed": "not_earned",
"projected_identity_verified": "not_earned",
"projected_restoration_verified": "not_earned",
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"hh_evanescent_status": "decoupled_from_ch_slot_proven_as_sum",
"hh_evanescent_decoupling_proven": true,
"hh_full_row_decoupling_proven": true,
"evanescent_hh_counterterm_added": false,
"D01_evaluated": true,
"D03_evaluated": true,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
```

```
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
```

```
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D11": {
    "status": "evaluated",
    "certificate": "certificates/P18_D11.json",
    "sector_status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
    "M2": "DISCHARGED (4D components)",
    "ch_slot": "decoupled by itself (p_m G_D11 = 0 exactly in d)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"D12_status_derivation": "certificates/P15_batchA.json graphs.D12 (route 1 frozen-dred lift + S0, route 2 sy
+> mmetric integration, IR-mass and cutoff analysis)",
"M2_summary": {
  "verdict": "DISCHARGED (4D components)",
  "D11_matching_verified_scope": "4D components",
  "evanescent_components_matching": {
    "status": "not_applicable_not_verified",
    "what_they_are": "the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt): compo
+> nents of the hh two-point function with at least one index pair in the epsilon-scalar (4-d) directions of
+>  DRED; they vanish on hatted 4D components",
    "why_not_matched": "XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent counte
+> rpart; M2 is a 4D statement, so there is nothing to match them to. Their L-coefficients are nonzero (reco
+> rded below); they are retained (R6) and drop out of the ch-projected slot (p_m G_D11 = 0 for the evanesce
+> nt part separately)"
  },
  "M3_status": {
    "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
```

```
    "statement": "L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree <= 8 i
+> t is a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)",
    "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching)"
  },
  "statement": "M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR_AB],
+>  f> = <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18",
  "route_B": {
    "equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair",
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 0,
    "first": [],
    "agree": true,
    "constant": "-I/4"
  },
  "FT_constant": {
    "master_pole": "1",
    "master_finite": "2 - L",
    "pole_is_1": true,
    "L_coefficient_of_MB": "-1",
    "c_FT_from_DR_bubble": "I*pi**2",
    "c_FT_from_minus_r": "I*pi**2",
    "r_certificate": "-i * pi**2",
    "r_certificate_as_expected": true,
    "agree": true
  },
  "controls": {
    "route_B_all_100_pairs_agree": true,
    "FT_constant_two_calibrations_agree": true,
    "route_A_M1_integrand_identity": true,
    "route_A_XVI_O_K_u_equals_T": true,
    "D11_transverse_exact_in_d": true,
    "routes_FP_PV_agree": true,
    "covariant_fit_exact_unique": true,
    "mutations_all_rejected": true
  },
  "analytic_inputs_cited": ["Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, resi
+> due at zeta = -2 is r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bub
+> ble)", "a polynomial in p is the Fourier transform of a distribution supported at {0}", "Route A only: tw
+> o extensions of the product V V (C C + C C) from U differ by a distribution supported at 0 (XVI-O lem:BV_
+> rules_on_U, Constructed)"],
  "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the 4
+> D statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0 is
+>  IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns t
+> he classical insertion and is not touched",
  "M3_consequence": "with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both sides a
+> re transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contacts, consistent with
+> N4), so L_match lies in the 2-dim transverse subspace of the p14r h.h span; its coordinates need the XVI-
+> O finite constant of u^R and are NOT computed (M4: booked as P-side hh counterterm; u^R_mu unmodified, no
+>  c delta)"
},
"D11_summary": {
  "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
  "prescription": "operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions, M3 bounds
+>  4/8/4, exactly-degree-four normalization (massless: every local term is a homogeneous quartic in p)",
  "prescription_usage": {
    "exactly_degree_four_normalization": "every structure S_s (p.p)^e of Ghat has total momentum degree 4 (c
+> omputed per structure: g4_mn g4_rs (p.p)^2: 4, g4_mn gt_rs (p.p)^2: 4, g4_mn p_r p_s (p.p)^1: 4, g4_mr g4
+> _ns (p.p)^2: 4, g4_mr gt_ns (p.p)^2: 4, g4_mr p_n p_s (p.p)^1: 4, g4_rs gt_mn (p.p)^2: 4, g4_rs p_m p_n (
+> p.p)^1: 4, gt_mn gt_rs (p.p)^2: 4, gt_mn p_r p_s (p.p)^1: 4, gt_mr gt_ns (p.p)^2: 4, gt_mr p_n p_s (p.p)^
+> 1: 4, gt_rs p_m p_n (p.p)^1: 4, p_m p_n p_r p_s: 4)",
    "N4_contact_exclusions": "used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by N4 do not
+> arise (massless; every pole and local MS-bar term is a homogeneous quartic in p), so N4 removes nothing h
+> ere and no order-0/2 counterterm is needed",
    "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"
  },
  "UV_pole_in_p14r_hh_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
  "evanescent_pole_nonzero": true,
  "controls": {
    "exactly_degree_four_every_structure": true,
    "N4_no_order_0_or_2_contact_arises": true,
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "integrand_equals_XVI_O_convolution_M1": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true,
```

```
    "M2_discharged": true,
    "mutations_all_rejected": true
  }
},
"hh_slot_summary": {
  "verdict": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a s
+> um; no evanescent structure survives in the slot",
  "row": "(S0,Gamma1^[2]) s2 K:K.c",
  "graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
  "how_D11_decouples": "graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse by i
+> tself, 4D and evanescent parts separately; its evanescent poles are nonzero but transverse. Hence D11 mut
+> ations of phase or symmetry factor are NOT detectable by the slot (they are tested by M2 instead); the sl
+> ot sees D11 only through a non-longitudinal operator (trace operator control: nonzero)",
  "surviving_evanescent_structures_in_slot": [],
  "consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop (D11 in
+> cluded), so NO contract revision is made for it.  The summed evanescent poles of the hh Green function ar
+> e nonzero but transverse; renormalising the hh Green function itself would need evanescent h.h counterter
+> ms -- outside this projection and NOT claimed.",
  "controls": {
    "operator_longitudinal": true,
    "p_dot_G_D11_zero_exact_in_d": true,
    "p_dot_G_sum_zero_exact_in_d": true,
    "p_dot_G_evanescent_sum_zero": true,
    "slot_sum_zero_all_7_assignments": true,
    "evanescent_slot_sum_zero_all_7_assignments": true,
    "D01_D03_individually_nonzero_incl_c_hat_h_tilde": true,
    "D11_reaches_slot_under_trace_operator": true,
    "4D_pole_sum_D01_D03_D11_transverse": true,
    "4D_pole_D11_transverse": true,
    "mutations_all_rejected": true
  }
},
"phase_convention_G6": "Overall phase (G6).  Every graph is exported as G = kappa^2 N_eps Ghat with N_eps =
+> i S_eps/(16 pi^2) and Ghat = -i * prefactor/kappa^2 * int N/(D1 D2).  For D01 (export prefactor kappa^2/2
+> ) and D03 (prefactor -kappa^2, closed ghost loop) the prefactor is real, so A(d) carries an overall facto
+> r I; for D13 the exported prefactor is I*kappa^2, so A(d) is real.  The relative phase between graphs of
+> the same slot is fixed by the common -i and the real export prefactors (D01 vs D03: real ratio); it is ch
+> ecked by the exact cancellation of the summed hh slot and by the rejected mutation M1 (relative sign).  P
+> hysical overall phase: not asserted.",
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_changed_vs_p17": true,
  "dred_executable_code_identical_to_p17_ast_without_docstrings": true,
  "disclosed_revisions_p18": {
    "code/p5_entry/dred.py": {
      "kind": "contract revision (disclosed), p18: COMMENTS/DOCSTRING ONLY (on top of p15 F1, p16 F3, p17 G1
+> -G3)",
      "finding": "H3/H5, reproducibility/independent_rebuild_report_p17.json",
      "why": "H3: OpaqueDimension.matches comment named tests/test_p16.py as the scan (now tests/test_p18.py
+> ); H5: parse_d(srepr(e)) raises PlainDimensionSymbol (fails closed) but this was not documented",
      "change": "comment/docstring text only; no statement executed by Python changed (AST-equal, tests/test
+> _p18.py)",
      "rules_R1_R6_changed": false,
      "released_values_affected": false,
      "why_unaffected": "the module AST without docstrings equals that of the p17 dred.py (tests/test_p18.py
+> ); the P14..P17 certificate content hashes recomputed in p18 equal archival/p17/reference_values_p17.json
+> "
    }
  },
  "evanescent_hh_counterterm_added": false,
  "why_no_evanescent_hh_counterterm": "full row decoupled: p_m G_D11 = 0 exactly in d and p_m (G_D01 + G_D03
+> ) = 0 exactly in d (certificates/P18_hh_slot.json)"
},
"contract_revision_disclosed": true,
"disclosed_test_revisions_p18": {
  "tests/test_p16.py": "test_f4_value_block_readback_detects_a_changed_value: the planted mutation is applie
+> d to a snippet that occurs VERBATIM in the typeset page (the p18 full-slot verdict is longer than the 110
+> -column wrap, so str.replace on the wrapped page was a no-op and the test could not plant its mutation).
+> The readback itself was correct (a mutation inside the wrapped verdict is detected; tests/test_p18.py::te
+> st_readback_detects_mutation_inside_wrapped_value). One line; no assertion weakened.",
  "tests/test_p14.py": "source-set / zip / reference / author-report names p17 -> p18; generated-prefix list
+>  extended by P18_* certificates/ledgers and the P18 results table; top-level status revision p18. (Its qu
+> antum_inputs_evaluated assertions read the p14r CONTRACT record certificates/P14_release.json and are unc
+> hanged.) No assertion weakened.",
  "tests/test_p17.py": "rs() reads the p17 LAYER record certificates/P17_release.json (the top-level RELEASE
+> _STATUS.json is now the p18 status); the version-label test checks that the build's CURRENT label is 'XVI
+> -P <current revision>'; the p17 layer's own independent flag (recomputed by release_p17.py against the CU
+> RRENT tree) may be False (the reviewer's live p17 report is bound to the p17 tree) or pending (source-onl
+> y rebuild); the p17 PASS itself is recorded by release_p18 as independent_rebuild_passed_p17. Three lines
```

```
+> ; all other p17 assertions unchanged."
},
"p17_layer": {
  "p17_layer_files_byte_identical": true,
  "P17_release_accepted": true,
  "hh_evanescent_decoupling_proven_p17": true
},
"change_log_vs_p17": {
  "base": "archival/p17/p17_source_files.json (pinned; files of the p17 zip sha256 7f4f61df9ce9c6483d76fa7b7
+> b024591d6abfbae80ddc3dc0bee387245d25416)",
  "base_pinned_ok": true,
  "reference_file": "reproducibility/reference_values_p23.json listed as added by declaration (written after
+>  the author build)",
  "added": ["archival/p17/README.md", "archival/p17/p17_source_files.json", "archival/p17/reference_values_p
+> 17.json", "archival/p17/reproducibility/author_rebuild_report_p17.json", "archival/p17/reproducibility/in
+> dependent_rebuild_report_p17.json", "archival/p18/README.md", "archival/p18/p18_source_files.json", "arch
+> ival/p18/reference_values_p18.json", "archival/p18/reproducibility/author_rebuild_report_p18.json", "arch
+> ival/p18/reproducibility/independent_rebuild_report_p18.json", "archival/p19/README.md", "archival/p19/p1
+> 9_source_files.json", "archival/p19/reference_values_p19.json", "archival/p19/reproducibility/author_rebu
+> ild_report_p19.json", "archival/p19/reproducibility/independent_rebuild_report_p19.json", "archival/p20/R
+> EADME.md", "archival/p20/p20_source_files.json", "archival/p20/reference_values_p20.json", "archival/p20/
+> reproducibility/author_rebuild_report_p20.json", "archival/p20/reproducibility/independent_rebuild_report
+> _p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.json", "archival/p21/reference_value
+> s_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility
+> /independent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescen
+> t.json", "archival/p22/p22_source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/re
+> producibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p
+> 22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/d11.py", "code/p5_calc/d11_pin.py", "code/p5
+> _calc/defect.py", "code/p5_calc/evanescent.py", "code/p5_calc/p18_conventions.py", "code/p5_calc/p19_conv
+> entions.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conv
+> entions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p
+> 18.py", "code/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "cod
+> e/p5_calc/release_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P18_D11.tex", "manuscript/not
+> es/P19_defect.tex", "manuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manu
+> script/notes/P22_evanescent.tex", "manuscript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target
+> .json", "reproducibility/reference_values_p23.json", "tests/test_p18.py", "tests/test_p19.py", "tests/tes
+> t_p20.py", "tests/test_p21.py", "tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p17.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.
+> tex", "manuscript/preamble.tex", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "tools/bu
+> ild.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p17/README.md", "archival
+> /p17/p17_source_files.json", "archival/p17/reference_values_p17.json", "archival/p17/reproducibility/auth
+> or_rebuild_report_p17.json", "archival/p17/reproducibility/independent_rebuild_report_p17.json", "archiva
+> l/p18/README.md", "archival/p18/p18_source_files.json", "archival/p18/reference_values_p18.json", "archiv
+> al/p18/reproducibility/author_rebuild_report_p18.json", "archival/p18/reproducibility/independent_rebuild
+> _report_p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/referenc
+> e_values_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproduc
+> ibility/independent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.js
+> on", "archival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.js
+> on", "archival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archi
+> val/p21/p21_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/a
+> uthor_rebuild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "arch
+> ival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json
+> ", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json
+> ", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py
+> ", "code/p5_calc/d11.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/evanescent.p
+> y", "code/p5_calc/p18_conventions.py", "code/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.p
+> y", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.p
+> y", "code/p5_calc/power_counting.py", "code/p5_calc/release_p18.py", "code/p5_calc/release_p19.py", "code
+> /p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py", "code/p5_calc/rel
+> ease_p23.py", "manuscript/body/XVI-P-body.tex", "manuscript/notes/P18_D11.tex", "manuscript/notes/P19_def
+> ect.tex", "manuscript/notes/P20_correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/not
+> es/P22_evanescent.tex", "manuscript/notes/P23_01_scope.tex", "manuscript/preamble.tex", "provenance/XVI-0
+> /0_quantum_target.json", "reproducibility/reference_values_p17.json", "reproducibility/reference_values_p
+> 23.json", "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "tests/test_p18.py", "tests/test
+> _p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_p22.py", "tests/test_p23.py", "tools/buil
+> d.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"],
  "counts": {
    "added": 63,
    "removed": 1,
    "modified": 11
  }
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
```

```
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
    "C7_grav_noether_k0_k1_k2_on_exports": true,
    "quartic_mutations_all_rejected": true,
    "master_grav_noether_deterministic_k0_k1_k2": true,
    "master_engine_parts_exact": true,
    "graviton_hessian_generic_symbolic_k": true,
    "canonicity_all_23_antifield_components": true,
    "ch_enumeration_lemma_mechanical": true,
    "residual_record_invariants_all_certificates": true,
    "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
    "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
    "cross_certificate_quartic_vs_master_agree": true,
    "stored_C7_flag_equals_recomputation": true
  },
  "failed_prerequisites": [],
  "hash_consistency": {
    "consistent": true,
    "ell": {
      "ell2": {
        "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
      },
      "ell3": {
        "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
      },
      "ell4": {
        "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
        "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
        "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    }
  },
  "required_master_residuals": [],
  "residual_records": {
    "invariant_violations": {},
    "quartic_C7_per_order": {
      "kappa0": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      },
      "kappa1": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      },
      "kappa2": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      }
    },
    "master_grav_per_order": {
      "kappa0": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      },
      "kappa1": {
```

```
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      },
      "kappa2": {
        "residual_counts": [0, 0, 0, 0],
        "zero_flag": true,
        "accepted": true
      }
    },
    "cross_certificate": {
      "same_inputs": true,
      "differences": [],
      "agree": true,
      "quartic_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      },
      "master_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    }
  },
  "hard_stops_p13_hold": true,
  "hard_stops_p13": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"hard_stops": {
  "projected_defect_computed": "not_earned",
  "projected_identity_verified": "not_earned",
  "projected_restoration_verified": "not_earned",
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
```

```
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "quantum inputs evaluated (p15: Batch A D02/D12/D14a-c, Batch B D13; p16: E2 decouple
+> d, Batch C D01/D03; p17: D01/D03 evanescent poles decoupled as the sum; p18: M2 discharged, D11 evaluated
+> , full hh row decoupled); the projected identity is NOT assembled",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (projected_identity_verified / projected_restoration_verified n
+> ot earned)"
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p17": true,
"independent_rebuild_passed_p16": true,
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p18) sourc
+> e tree only (independent: pending until a reviewer report for this tree exists); independent_rebuild_pass
+> ed_p17 refers to the p17 tree 2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92 / zip 7f4f
+> 61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416 (same-box caveat) and is not inherited by p1
+> 8; _p16 / _p15 / _p14r / _p13 as recorded by the p17 layer",
"p17_independent_rebuild": {
  "report": "archival/p17/reproducibility/independent_rebuild_report_p17.json",
  "present": true,
  "report_sha256": "92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
  "bound_to_source_zip_sha256": "7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416",
  "tree_matches_p17": true,
  "zip_matches_p17": true,
  "mismatches": 0,
  "findings": ["H1_static_scan_pattern_gaps", "H2_assignment_table_omits_c_hat_h_tilde", "H3_stale_comments"
+> , "H4_curly_apostrophes_in_typeset_listings", "H5_parse_d_srepr", "H6_scope", "G1_G2_G3_G4_resolution"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p18": "bound to the p17 tree/zip; the p18 tree has its own (pending) independent flag",
  "archived_p17_records_pinned": {
    "archival/p17/reference_values_p17.json": true,
    "archival/p17/reproducibility/author_rebuild_report_p17.json": true,
    "archival/p17/reproducibility/independent_rebuild_report_p17.json": true,
    "archival/p17/p17_source_files.json": true
  },
  "pins_ok": true
},
"p16_independent_rebuild": {
  "report": "archival/p16/reproducibility/independent_rebuild_report_p16.json",
  "present": true,
  "report_sha256": "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
  "bound_to_source_zip_sha256": "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57",
  "tree_matches_p16": true,
  "zip_matches_p16": true,
  "mismatches": 0,
  "findings": ["G1_static_scan_scope_narrower_than_disclosure", "G2_plain_symbol_d_from_stored_strings", "G3
+> _hook_coverage_detail", "G4_change_log_omits_new_reference_file", "G5_test_p15_assertion_relaxed", "G6_im
+> aginary_normalisation", "G7_scope", "F3_F4_F5_from_p15"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent flag",
  "archived_p16_records_pinned": {
    "archival/p16/reference_values_p16.json": true,
    "archival/p16/reproducibility/author_rebuild_report_p16.json": true,
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": true,
    "archival/p16/p16_source_files.json": true
  },
  "pins_ok": true
```

```
},
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
```

```
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
```

```
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p18.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "vendored",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p18.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": false,
    "inherited_mode": null,
    "environment": {
      "python": "3.13.5",
```

```
        "sympy": "1.14.0",
        "mpmath": "1.3.0",
        "pytest": "9.1.1",
        "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
        "pdftotext": "pdftotext version 25.03.0",
        "pdfinfo": "pdfinfo version 25.03.0",
        "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
        "venv": "/workspace/indep_rebuild_p18/venv",
        "pip_installed": "sympy==1.14.0 mpmath==1.3.0 pytest==9.1.1 (exact versions via pip)"
      }
    },
    "p17_archive": "archival/p17",
    "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
  },
  "XVI_P_registered": false,
  "XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
  "XVI_M_pin_ok": true,
  "symmetry_restored": false,
  "bulk_symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
    "P14_release.json": "23b5b7bd32da5e33b794827ae40dc2de0a73a547c9ec26d7a69b08b87e727b63",
    "P15_release.json": "6010e0f56336e10d594a2834dc16395aebe89690224b08f293b82fa5db4797de",
    "P16_release.json": "f81db7fdaf2e28a4451cfabb97db61b46b7f472e4b45ed911b8326b87d2893d6",
    "P17_release.json": "97a548b4d29495bf648d1cb93b00f886980128a9d2eb5d8f7ddbd50f09ab13f1",
    "P18_D11_M2.json": "2c63a6e8fe0e3ca6faae720baa3e653634c70f44f9acce3fe33b1f7e2d566634",
    "P18_D11.json": "8cfb4f3884621349328c3b6c5dcc76411a6ad6c9b1821490941740a652659613",
    "P18_hh_slot.json": "fe85ab9ff6b048e9584c47488c287aa31de57fb9b3df44cae3a57e23cfcdab12"
  }
}
```


47 Milestone ledger (p18 layer)

[listing 14/133] ledgers/P18_milestones.json


```
{
  "landmark": "XVI-P",
  "revision": "p18",
  "ledger": "milestones (authoritative definitions + status)",
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "quantum inputs evaluated (p15: Batch A D02/D12/D14a-c, Batch B D13; p16: E2 decoupled,
  +>  Batch C D01/D03; p17: D01/D03 evanescent poles decoupled as the sum; p18: M2 discharged, D11 evaluated,
  +> full hh row decoupled); the projected identity is NOT assembled",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (projected_identity_verified / projected_restoration_verified not
  +>  earned)",
  "author_rebuild_passed": false,
  "independent_rebuild_passed": false,
  "independent_rebuild_passed_p17": true,
  "hard_stops": {
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
```

```
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "u_R_c_delta_added": false,
    "XVI_P_registered": false
  },
  "flags": {
    "D11_matching_verified": true,
    "D11_evaluated": true,
    "quantum_inputs_evaluated": "earned",
    "hh_full_row_decoupling_proven": true
  }
}
```


48 P.5 calculation: D11 matching premise M2 (discharge; p18)

[listing 15/133] certificates/P18_D11_M2.json


```
{
  "landmark": "XVI-P",
  "revision": "p18",
  "deliverable": "D11 matching premise M2",
  "statement": "M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR_AB], f
  +> > = <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18",
  "verdict": "DISCHARGED (4D components)",
  "D11_matching_verified": true,
  "D11_matching_verified_scope": "4D components",
  "evanescent_components_matching": {
    "status": "not_applicable_not_verified",
    "what_they_are": "the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt): compone
  +> nts of the hh two-point function with at least one index pair in the epsilon-scalar (4-d) directions of D
  +> RED; they vanish on hatted 4D components",
    "why_not_matched": "XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent counterp
  +> art; M2 is a 4D statement, so there is nothing to match them to. Their L-coefficients are nonzero (record
  +> ed below); they are retained (R6) and drop out of the ch-projected slot (p_m G_D11 = 0 for the evanescent
  +>  part separately)"
  },
  "route_B": {
    "equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair",
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 0,
    "first": [],
    "agree": true,
    "constant": "-I/4"
  },
  "FT_constant": {
    "master_pole": "1",
    "master_finite": "2 - L",
    "pole_is_1": true,
    "L_coefficient_of_MB": "-1",
    "c_FT_from_DR_bubble": "I*pi**2",
    "c_FT_from_minus_r": "I*pi**2",
    "r_certificate": "-i * pi**2",
    "r_certificate_as_expected": true,
    "agree": true
  },
  "route_A": {
    "M1": {
      "component_pairs_checked": 100,
      "nonzero_pairs": 100,
      "mismatches": [],
      "pass": true,
      "recomputed_integrand_sha256": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609",
      "exported_integrand_sha256": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609"
    },
    "XVI_O": {
      "alpha_beta_rederived": {
        "alpha": "A**2/60",
        "beta": "A**2/18",
        "match": true
      },
      "K_u_equals_T_all_256": {
        "status": "Constructed",
        "pass": true,
        "components": 256,
        "failures": 0
      },
```

```
    "ok": true
  }
},
"transversality": {
  "p_dot_G_4D_part": {},
  "p_dot_G_evanescent_part": {},
  "transverse_exact_in_d": true
},
"mutations": {
  "alpha_beta_swapped": {
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 100,
    "first": [
      [0, 0],
      [0, 1],
      [0, 2]
    ],
    "agree": false,
    "constant": "-I/4",
    "rejected": true,
    "accepted_as_rejection": true
  },
  "offdiagonal_weight_1": {
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 84,
    "first": [
      [0, 1],
      [0, 2],
      [0, 3]
    ],
    "agree": false,
    "constant": "-I/4",
    "rejected": true,
    "accepted_as_rejection": true
  },
  "FT_log_constant_sign_flipped": {
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 100,
    "first": [
      [0, 0],
      [0, 1],
      [0, 2]
    ],
    "agree": false,
    "constant": "I/4",
    "rejected": true,
    "accepted_as_rejection": true
  },
  "symmetry_factor_doubled": {
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 100,
    "first": [
      [0, 0],
      [0, 1],
      [0, 2]
    ],
    "agree": false,
    "constant": "-I/4",
    "rejected": true,
    "accepted_as_rejection": true
  },
  "D01_in_place_of_D11": {
    "pairs": 100,
    "nonzero_pairs": 100,
    "mismatches": 100,
    "first": [
      [0, 0],
      [0, 1],
      [0, 2]
    ],
    "agree": false,
    "constant": "-I/4",
    "rejected": true,
    "accepted_as_rejection": true
  },
  "D0_dropped_beta_0": {
    "pairs": 100,
```

```
      "nonzero_pairs": 100,
      "mismatches": 100,
      "first": [
        [0, 0],
        [0, 1],
        [0, 2]
      ],
      "agree": false,
      "constant": "-I/4",
      "rejected": true,
      "accepted_as_rejection": true
    }
  },
  "controls": {
    "route_B_all_100_pairs_agree": true,
    "FT_constant_two_calibrations_agree": true,
    "route_A_M1_integrand_identity": true,
    "route_A_XVI_O_K_u_equals_T": true,
    "D11_transverse_exact_in_d": true,
    "routes_FP_PV_agree": true,
    "covariant_fit_exact_unique": true,
    "mutations_all_rejected": true
  },
  "analytic_inputs_cited": ["Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, residu
  +> e at zeta = -2 is r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bubbl
  +> e)", "a polynomial in p is the Fourier transform of a distribution supported at {0}", "Route A only: two
  +> extensions of the product V V (C C + C C) from U differ by a distribution supported at 0 (XVI-O lem:BV_ru
  +> les_on_U, Constructed)"],
  "scope": "4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside the 4D
  +> statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at p^2 != 0 is I
  +> R finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massless obstruction concerns the
  +>  classical insertion and is not touched",
  "evanescent_L_coefficients": {
    "g4_mn gt_rs (p.p)^2": "I/120",
    "g4_mr gt_ns (p.p)^2": "-I/480",
    "g4_rs gt_mn (p.p)^2": "I/120",
    "gt_mn gt_rs (p.p)^2": "I/480",
    "gt_mn p_r p_s (p.p)^1": "-I/120",
    "gt_mr gt_ns (p.p)^2": "I/480",
    "gt_mr p_n p_s (p.p)^1": "I/480",
    "gt_rs p_m p_n (p.p)^1": "-I/120"
  },
  "M3_status": {
    "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
    "statement": "L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree <= 8 it
  +> is a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)",
    "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching)"
  },
  "M3_consequence": "with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both sides are
  +>  transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contacts, consistent with N4
  +> ), so L_match lies in the 2-dim transverse subspace of the p14r h.h span; its coordinates need the XVI-O
  +> finite constant of u^R and are NOT computed (M4: booked as P-side hh counterterm; u^R_mu unmodified, no c
  +>  delta)"
}
```


49 P.5 calculation: D11 evaluated under the declared prescription (p18)

[listing 16/133] certificates/P18_D11.json


```
{
  "landmark": "XVI-P",
  "revision": "p18",
  "deliverable": "D11 evaluation under the declared prescription",
  "graph": "D11",
  "export": {
    "file": "exports/P14_integrands/D11.json",
    "sha256_recomputed": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609",
    "sha256_stored": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609",
    "sha256_p14r_certificate": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609",
    "pinned": true
  },
  "prefactor_export": "kappa**2/2",
  "normalization": "Ghat = -I/2 * int N/(D1 D2) / N_eps",
  "prescription": "operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions, M3 bounds 4
  +> /8/4, exactly-degree-four normalization (massless: every local term is a homogeneous quartic in p)",
  "covariant_form": {
```

```
  "n_ansatz": 51,
  "n_used": 15,
  "exact": true,
  "bad_components": [],
  "rank": 51,
  "unique": true,
  "offdiag_weight": "2",
  "upper": true
},
"prescription_usage": {
  "exactly_degree_four_normalization": "every structure S_s (p.p)^e of Ghat has total momentum degree 4 (com
+> puted per structure: g4_mn g4_rs (p.p)^2: 4, g4_mn gt_rs (p.p)^2: 4, g4_mn p_r p_s (p.p)^1: 4, g4_mr g4_n
+> s (p.p)^2: 4, g4_mr gt_ns (p.p)^2: 4, g4_mr p_n p_s (p.p)^1: 4, g4_rs gt_mn (p.p)^2: 4, g4_rs p_m p_n (p.
+> p)^1: 4, gt_mn gt_rs (p.p)^2: 4, gt_mn p_r p_s (p.p)^1: 4, gt_mr gt_ns (p.p)^2: 4, gt_mr p_n p_s (p.p)^1:
+>  4, gt_rs p_m p_n (p.p)^1: 4, p_m p_n p_r p_s: 4)",
  "N4_contact_exclusions": "used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by N4 do not ar
+> ise (massless; every pole and local MS-bar term is a homogeneous quartic in p), so N4 removes nothing her
+> e and no order-0/2 counterterm is needed",
  "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"
},
"result": {
  "g4_mn g4_rs (p.p)^2": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*(-d**2 + 2*d + 2)/(32*(d**2 - 1))",
    "UV_pole": "-I/80",
    "MSbar_finite": "I*(15*L - 16)/1200",
    "nonlocal_coefficient_of_L": "I/80",
    "local_polynomial": "-I/75",
    "finite_if_d_set_to_4_first": "I*(L - 2)/80",
    "local_shift_from_d_dependence": "7*I/600"
  },
  "g4_mn gt_rs (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "-I*d/(32*d**2 - 32)",
    "UV_pole": "-I/120",
    "MSbar_finite": "I*(30*L - 77)/3600",
    "nonlocal_coefficient_of_L": "I/120",
    "local_polynomial": "-77*I/3600",
    "finite_if_d_set_to_4_first": "I*(L - 2)/120",
    "local_shift_from_d_dependence": "-17*I/3600"
  },
  "g4_mn p_r p_s (p.p)^1": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*(d**2 - 2*d - 2)/(32*(d**2 - 1))",
    "UV_pole": "I/80",
    "MSbar_finite": "I*(16 - 15*L)/1200",
    "nonlocal_coefficient_of_L": "-I/80",
    "local_polynomial": "I/75",
    "finite_if_d_set_to_4_first": "I*(2 - L)/80",
    "local_shift_from_d_dependence": "-7*I/600"
  },
  "g4_mr g4_ns (p.p)^2": {
    "evanescent": false,
    "A_exact_d_times_MB": "-I/(32*d**2 - 32)",
    "UV_pole": "-I/480",
    "MSbar_finite": "I*(15*L - 46)/7200",
    "nonlocal_coefficient_of_L": "I/480",
    "local_polynomial": "-23*I/3600",
    "finite_if_d_set_to_4_first": "I*(L - 2)/480",
    "local_shift_from_d_dependence": "-I/450"
  },
  "g4_mr gt_ns (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "I/(32*(d**2 - 1))",
    "UV_pole": "I/480",
    "MSbar_finite": "I*(46 - 15*L)/7200",
    "nonlocal_coefficient_of_L": "-I/480",
    "local_polynomial": "23*I/3600",
    "finite_if_d_set_to_4_first": "I*(2 - L)/480",
    "local_shift_from_d_dependence": "I/450"
  },
  "g4_mr p_n p_s (p.p)^1": {
    "evanescent": false,
    "A_exact_d_times_MB": "I/(32*(d**2 - 1))",
    "UV_pole": "I/480",
    "MSbar_finite": "I*(46 - 15*L)/7200",
    "nonlocal_coefficient_of_L": "-I/480",
    "local_polynomial": "23*I/3600",
    "finite_if_d_set_to_4_first": "I*(2 - L)/480",
    "local_shift_from_d_dependence": "I/450"
  },
```

```
  "g4_rs gt_mn (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "-I*d/(32*d**2 - 32)",
    "UV_pole": "-I/120",
    "MSbar_finite": "I*(30*L - 77)/3600",
    "nonlocal_coefficient_of_L": "I/120",
    "local_polynomial": "-77*I/3600",
    "finite_if_d_set_to_4_first": "I*(L - 2)/120",
    "local_shift_from_d_dependence": "-17*I/3600"
  },
  "g4_rs p_m p_n (p.p)^1": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*(d**2 - 2*d - 2)/(32*(d**2 - 1))",
    "UV_pole": "I/80",
    "MSbar_finite": "I*(16 - 15*L)/1200",
    "nonlocal_coefficient_of_L": "-I/80",
    "local_polynomial": "I/75",
    "finite_if_d_set_to_4_first": "I*(2 - L)/80",
    "local_shift_from_d_dependence": "-7*I/600"
  },
  "gt_mn gt_rs (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "-I/(32*d**2 - 32)",
    "UV_pole": "-I/480",
    "MSbar_finite": "I*(15*L - 46)/7200",
    "nonlocal_coefficient_of_L": "I/480",
    "local_polynomial": "-23*I/3600",
    "finite_if_d_set_to_4_first": "I*(L - 2)/480",
    "local_shift_from_d_dependence": "-I/450"
  },
  "gt_mn p_r p_s (p.p)^1": {
    "evanescent": true,
    "A_exact_d_times_MB": "I*d/(32*(d**2 - 1))",
    "UV_pole": "I/120",
    "MSbar_finite": "I*(77 - 30*L)/3600",
    "nonlocal_coefficient_of_L": "-I/120",
    "local_polynomial": "77*I/3600",
    "finite_if_d_set_to_4_first": "I*(2 - L)/120",
    "local_shift_from_d_dependence": "17*I/3600"
  },
  "gt_mr gt_ns (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "-I/(32*d**2 - 32)",
    "UV_pole": "-I/480",
    "MSbar_finite": "I*(15*L - 46)/7200",
    "nonlocal_coefficient_of_L": "I/480",
    "local_polynomial": "-23*I/3600",
    "finite_if_d_set_to_4_first": "I*(L - 2)/480",
    "local_shift_from_d_dependence": "-I/450"
  },
  "gt_mr p_n p_s (p.p)^1": {
    "evanescent": true,
    "A_exact_d_times_MB": "-I/(32*d**2 - 32)",
    "UV_pole": "-I/480",
    "MSbar_finite": "I*(15*L - 46)/7200",
    "nonlocal_coefficient_of_L": "I/480",
    "local_polynomial": "-23*I/3600",
    "finite_if_d_set_to_4_first": "I*(L - 2)/480",
    "local_shift_from_d_dependence": "-I/450"
  },
  "gt_rs p_m p_n (p.p)^1": {
    "evanescent": true,
    "A_exact_d_times_MB": "I*d/(32*(d**2 - 1))",
    "UV_pole": "I/120",
    "MSbar_finite": "I*(77 - 30*L)/3600",
    "nonlocal_coefficient_of_L": "-I/120",
    "local_polynomial": "77*I/3600",
    "finite_if_d_set_to_4_first": "I*(2 - L)/120",
    "local_shift_from_d_dependence": "17*I/3600"
  },
  "p_m p_n p_r p_s": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*d*(2 - d)/(32*(d**2 - 1))",
    "UV_pole": "-I/60",
    "MSbar_finite": "I*(30*L - 47)/1800",
    "nonlocal_coefficient_of_L": "I/60",
    "local_polynomial": "-47*I/1800",
    "finite_if_d_set_to_4_first": "I*(L - 2)/60",
    "local_shift_from_d_dependence": "13*I/1800"
  }
},
```

```
  "evaluated": true,
  "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)",
  "UV_pole_in_p14r_hh_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
  "evanescent_poles": {
    "g4_mn gt_rs (p.p)^2": "-I/120",
    "g4_rs gt_mn (p.p)^2": "-I/120",
    "gt_mn gt_rs (p.p)^2": "-I/480",
    "g4_mr gt_ns (p.p)^2": "I/480",
    "gt_mr gt_ns (p.p)^2": "-I/480",
    "gt_mn p_r p_s (p.p)^1": "I/120",
    "gt_mr p_n p_s (p.p)^1": "-I/480",
    "gt_rs p_m p_n (p.p)^1": "I/120"
  },
  "evanescent_pole_nonzero": true,
  "longitudinal_projection_p_m_G": {},
  "controls": {
    "exactly_degree_four_every_structure": true,
    "N4_no_order_0_or_2_contact_arises": true,
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "integrand_equals_XVI_0_convolution_M1": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true,
    "M2_discharged": true,
    "mutations_all_rejected": true
  },
  "mutations": {
    "premature_d4": {
      "rejected": true,
      "changes_result": true,
      "reason": "PrematureD4Substitution: d -> 4 via subs outside substitute_d (R5)",
      "accepted_as_rejection": true
    },
    "drop_evanescent_gt": {
      "changes_result": true,
      "rejected": true,
      "reason": "FP route with gt dropped != PV route (R6 retention check)",
      "accepted_as_rejection": true
    },
    "wrong_external_index_position": {
      "changes_result": true,
      "rejected": true,
      "accepted_as_rejection": true
    },
    "wrong_offdiagonal_weight": {
      "changes_result": true,
      "rejected": true,
      "accepted_as_rejection": true
    }
  },
  "D11_evaluated": true,
  "D11_matching_verified": true,
  "authorised_d_substitutions": {
    "d11: L-coefficient A(4) of D11 (after exact-d algebra; M2 nonlocal comparison)": 14,
    "d11: M2 mutation: D01 in place of D11 (A(4))": 14,
    "d11: evanescent UV pole A(4) of the summed hh function (record only)": 22,
    "d13.expand_coeff: Laurent expansion after all contractions and the fit": 28,
    "d13.expand_coeff: diagnostic A'(4)": 28,
    "d13.expand_coeff: diagnostic A(4) (d-dependence proof only)": 28,
    "hh_slot: UV pole A(4) of a 4D structure (after exact-d algebra)": 18,
    "integrals.laurent: d = 4 - 2 eps after all contractions": 1
  }
}
```


50 P.5 calculation: full hh slot D01+D02+D03+D11+D12 vs the ch projection (p18)

[listing 17/133] certificates/P18_hh_slot.json


```
{
  "landmark": "XVI-P",
  "revision": "p18",
```

```
"deliverable": "full hh slot (D01+D02+D03+D11+D12) vs the ch projection",
"row": "(S0,Gamma1^[2]) s2 K:K.c",
"graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
"verdict": "DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 as a sum
+> ; no evanescent structure survives in the slot",
"decoupling_proven_full_row": true,
"surviving_evanescent_structures_in_slot": [],
"D11_reopens_evanescent_question": false,
"how_D11_decouples": "graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse by its
+> elf, 4D and evanescent parts separately; its evanescent poles are nonzero but transverse. Hence D11 mutat
+> ions of phase or symmetry factor are NOT detectable by the slot (they are tested by M2 instead); the slot
+>  sees D11 only through a non-longitudinal operator (trace operator control: nonzero)",
"operator_s_h": {
  "exact": true,
  "unique": true,
  "offdiag_weight": 1,
  "upper": false,
  "n_components": 16,
  "rank": 4,
  "n_ansatz": 4,
  "covariant": [
    [
      "I/2",
      [
        ["g4", "m", "v"],
        ["p", "n"]
      ]
    ]
  ]
},
"slot_values": {
  "D01": {
    "Q4S (declared, R2)": "p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p
+>  - a~.c~*b^.p)/(16*(d - 1))",
    "QdS (hat)": "p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0",
    "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
  },
  "D03": {
    "Q4S (declared, R2)": "-p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.
+> p - a~.c~*b^.p)/(16*(d - 1))",
    "QdS (hat)": "-p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0",
    "c hat, h tilde": "-a~.b~*c^.p*p.p**2/16"
  },
  "D11": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0",
    "c hat, h tilde": "0"
  },
  "D01+D03+D11": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0",
    "c hat, h tilde": "0"
  },
  "evanescent_parts_D01+D03+D11": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0",
    "c hat, h tilde": "0"
  },
  "evanescent_part_D11": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
```

```
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0",
    "c hat, h tilde": "0"
  },
  "D11_under_trace_operator": {
    "Q4S (declared, R2)": "c^.p*p.p*(d*a.b*p.p - d*a^.p*b^.p - 2*a.b*p.p + 2*a^.p*b^.p + a~.b~*p.p)/(16*(d -
+>  1))",
    "QdS (hat)": "c^.p*p.p*(d - 2)*(a^.b^*p.p - a^.p*b^.p)/(16*(d - 1))",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0",
    "c hat, h tilde": "a~.b~*c^.p*p.p**2/16"
  }
},
"p_dot_G": {
  "D11": {},
  "D01+D03+D11": {},
  "evanescent_D01+D03+D11": {}
},
"four_D_poles": {
  "basis_dimension": 5,
  "span_is_all_4D_structures": true,
  "longitudinal_constraint_rank": 3,
  "transverse_subspace_dimension": 2,
  "transverse_basis_in_p14r_coordinates": [
    ["1/2", "-1/2", "-1", "1", "0"],
    ["1", "0", "-2", "0", "1"]
  ],
  "graphs": {
    "D11": {
      "coordinates_p14r_basis": ["-I/160", "-I/480", "I/80", "I/240", "-I/120"],
      "in_span": true,
      "transverse": true
    },
    "D01+D03": {
      "coordinates_p14r_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
      "in_span": true,
      "transverse": true
    },
    "D01+D03+D11": {
      "coordinates_p14r_basis": ["-49*I/480", "-41*I/160", "49*I/240", "41*I/80", "-43*I/120"],
      "in_span": true,
      "transverse": true
    }
  },
  "n_4D_orbit_labels": 6,
  "dim_pair_symmetric_4D_structures": 5,
  "basis_rank": 5
},
"evanescent_poles_D01_D03_D11_in_hh_green_function": {
  "g4_mn gt_rs (p.p)^2": "7*I/120",
  "g4_mr gt_ns (p.p)^2": "I/160",
  "g4_rs gt_mn (p.p)^2": "7*I/120",
  "gt_mn gt_rs (p.p)^2": "-I/160",
  "gt_mn p_r p_s (p.p)^1": "-7*I/120",
  "gt_mr gt_ns (p.p)^2": "-I/160",
  "gt_mr p_n p_s (p.p)^1": "-I/160",
  "gt_rs p_m p_n (p.p)^1": "-7*I/120"
},
"consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop (D11 incl
+> uded), so NO contract revision is made for it.  The summed evanescent poles of the hh Green function are
+> nonzero but transverse; renormalising the hh Green function itself would need evanescent h.h counterterms
+>  -- outside this projection and NOT claimed.",
"controls": {
  "operator_longitudinal": true,
  "p_dot_G_D11_zero_exact_in_d": true,
  "p_dot_G_sum_zero_exact_in_d": true,
  "p_dot_G_evanescent_sum_zero": true,
  "slot_sum_zero_all_7_assignments": true,
  "evanescent_slot_sum_zero_all_7_assignments": true,
  "D01_D03_individually_nonzero_incl_c_hat_h_tilde": true,
  "D11_reaches_slot_under_trace_operator": true,
  "4D_pole_sum_D01_D03_D11_transverse": true,
  "4D_pole_D11_transverse": true,
  "mutations_all_rejected": true
},
"mutations": {
  "M1_wrong_relative_sign_D03": {
```

```
    "rejected": true,
    "accepted_as_rejection": true
  },
  "M3_D03_evanescent_dropped_naive_4D": {
    "rejected": true,
    "accepted_as_rejection": true
  },
  "M4_trace_operator_in_place_of_s_h": {
    "rejected": true,
    "accepted_as_rejection": true
  }
},
"inventory": {
  "D01": {
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graph_id": "D01_hh_bubble_cubic",
    "certificate": "certificates/P16_batchC.json",
    "present": true,
    "status": "regulated",
    "evaluated": true
  },
  "D02": {
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graph_id": "D02_hh_tadpole_quartic",
    "certificate": "certificates/P15_batchA.json",
    "present": true,
    "status": "vanishing_under_declared_prescription",
    "evaluated": true
  },
  "D03": {
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graph_id": "D03_hh_ghost_loop",
    "certificate": "certificates/P16_batchC.json",
    "present": true,
    "status": "regulated",
    "evaluated": true
  },
  "D11": {
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graph_id": "D11_hh_scalar_loop",
    "certificate": "certificates/P18_D11.json",
    "present": true,
    "evaluated": true,
    "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"
  },
  "D12": {
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graph_id": "D12_hh_scalar_tadpole",
    "certificate": "certificates/P15_batchA.json",
    "present": true,
    "status": "vanishing_under_declared_prescription",
    "evaluated": true
  },
  "D13": {
    "row": "(S0,Gamma1^[2]) s1 h:h^2",
    "graph_id": "D13_Kc_ghost_graviton_loop",
    "certificate": "certificates/P15_D13.json",
    "present": true,
    "status": "evaluated (P15_D13.json)",
    "evaluated": true
  },
  "D14a": {
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "graph_id": "D14a_h_onepoint_grav_tadpole",
    "certificate": "certificates/P15_batchA.json",
    "present": true,
    "status": "vanishing_under_declared_prescription",
    "evaluated": true
  },
  "D14b": {
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "graph_id": "D14b_h_onepoint_ghost_tadpole",
    "certificate": "certificates/P15_batchA.json",
    "present": true,
    "status": "vanishing_under_declared_prescription",
    "evaluated": true
  },
  "D14c": {
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "graph_id": "D14c_h_onepoint_scalar_tadpole",
    "certificate": "certificates/P15_batchA.json",
```

```
      "present": true,
      "status": "vanishing_under_declared_prescription",
      "evaluated": true
    }
  },
  "quantum_inputs_evaluated": "earned",
  "quantum_inputs_scope": "every graph of the P13 inventory (three ch slots: hh, K.c, one-point) is evaluated
  +> under the one frozen prescription; this is NOT projected_identity_verified (the projected identity is not
  +>  assembled)",
  "seconds": 288.5
}
```


51 p17 layer record (release gate p17)

[listing 18/133] certificates/P17_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p17",
  "contract_revision": "p17 (p14r contract + disclosed p15/p16/p17 revisions)",
  "date": "2026-10-08",
  "version_label": "XVI-P p17",
  "deliverable": "P.5 calculation, third step (D01/D03 evanescent poles vs the ch-projected slot; G1-G6)",
  "execution_pass": true,
  "claim_status": "P.5_calculation: Batch A, D13 (p15), Batch C D01, D03 (p16) evaluated under the declared pr
  +> escription; E2 (p16) and the D01/D03 evanescent poles (p17, as the sum) decoupled from the ch-projected s
  +> lot (proofs); P.5_entry: earned (p14r contract); classical_readiness (frozen P.4): earned",
  "scope": "P.5 calculation: Batch A (D02, D12, D14a-c), Batch B (D13) and Batch C (D01, D03) evaluated under
  +> the frozen prescription (DRED R1-R6, Feynman +i0, MS-bar, S0; d opaque); D01 + D03 contribute zero to the
  +>  ch-projected slot (exact in d); D11 NOT evaluated (M2 undischarged), so the hh slot is incomplete and no
  +>  slot of the identity is complete; NOT quantum inputs, NOT the projected defect/identity, NOT bulk restor
  +> ation, NOT the insertion",
  "release_claim_accepted": true,
  "release_claim_accepted_definition": "p14r contract accepted AND p15 and p16 layers accepted and byte-identi
  +> cal AND contract identity (only dred.py changed vs p14r; changed vs p16 by the disclosed G2 loader; guard
  +>  probes; no new contract file) AND the hh-slot decoupling proof AND p17 hard stops",
  "publication": "accepted",
  "flags": {
    "library_certified": true,
    "batchA_evaluated": {
      "execution": true,
      "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
  +> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
  +> V pole, finite part 0, no p14r counterterm required",
      "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
  +>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
  +> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
  +> e; NOT quantum_inputs_evaluated"
    },
    "D13_evaluated": {
      "execution": true,
      "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
  +> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
  +> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
  +> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
      "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
  +> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
  +> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
  +> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
    },
    "batchC_evaluated": {
      "execution": true,
      "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
  +> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
  +> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
  +> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
      "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
  +>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
  +> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
    },
    "E2_decoupled_from_ch_slot": {
      "execution": true,
      "claim": "DECOUPLED (proof)",
      "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
  +>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
    },
    "hh_evanescent_decoupled_from_ch_slot": {
      "execution": true,
```

```
    "claim": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
    "scope": "the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of D01
+>  + D03 (D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green function
+>  outside this projection; D11 (same row) NOT evaluated"
  }
},
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"hh_evanescent_status": "decoupled_from_ch_slot_proven_as_sum",
"hh_evanescent_decoupling_proven": true,
"evanescent_hh_counterterm_added": false,
"D01_evaluated": true,
"D03_evaluated": true,
"D11_evaluated": false,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
```

```
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
```

```
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true,
    "ch_slot": "decoupled as the sum D01 + D03 (evanescent and 4D parts; certificates/P17_hh_slot.json); NOT
+>  graph by graph"
  },
  "D11": {
    "status": "unevaluated",
    "reason": "M2 undischarged (D11 matching not verified)",
    "ch_slot": "open (same row (S0,Gamma1^[2]) s2 K:K.c)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"D12_status_derivation": "certificates/P15_batchA.json graphs.D12 (route 1 frozen-dred lift + S0, route 2 sy
+> mmetric integration, IR-mass and cutoff analysis)",
"hh_slot_summary": {
  "verdict": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
  "status": "decoupled_from_ch_slot_proven_as_sum",
  "row": "(S0, Gamma1^[2]) s2 K:K.c (ledgers/P13_ch_dependency_rows.json; graphs D01, D02, D03, D11, D12); t
+> he hh two-point function enters no other ch row (computed from the ledger: rows_with_D01_D03)",
  "operator_s_h": {
    "exact": true,
    "unique": true,
    "offdiag_weight": 1,
    "upper": false,
    "n_components": 16,
    "rank": 4,
    "n_ansatz": 4,
    "covariant": [
      [
        "I/2",
        [
          ["g4", "m", "v"],
          ["p", "n"]
        ]
      ]
    ]
  },
  "proof": {
    "H1": "s h read off S_Psi^(0) (K.c monomial): exact unique covariant form, longitudinal in (mn)",
    "H2": "X_g depends on G only through p_m G^{(mn)(rs)}; p in QdS so p.gt = 0 removes gt on (mn)",
    "H3": "p_m (G_D01 + G_D03)^{(mn)(rs)} = 0 identically in d (4D and evanescent parts separately); X_{D01+
+> D03} = 0 for every external assignment; D02 = D12 = 0 (P15_batchA.json)",
    "H4": "controls: X_D01, X_D03 individually nonzero (also their evanescent parts); 4D poles individually
+> not transverse",
    "order": "one loop: the ch projection is linear in Gamma1; the cancellation is exact in d (not O(eps));
+> no product of evanescent poles arises"
  },
  "surviving_evanescent_structures_in_slot": [],
  "consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop, so NO
+> contract revision is made for it.  The summed evanescent poles of the hh Green function are nonzero (list
+>  above) but TRANSVERSE; renormalising the hh Green function itself would need evanescent h.h counterterms
+>  -- relevant only outside this projection and NOT claimed.  D11 (same row) is NOT evaluated: its slot con
+> tribution is open.",
  "four_D_poles": {
    "basis_dimension": 5,
    "span_is_all_4D_structures": true,
    "longitudinal_constraint_rank": 3,
    "transverse_subspace_dimension": 2,
    "transverse_basis_in_p14r_coordinates": [
      ["1/2", "-1/2", "-1", "1", "0"],
      ["1", "0", "-2", "0", "1"]
    ],
    "graphs": {
      "D01": {
        "coordinates_p14r_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
        "in_span": true,
        "transverse": false
      },
      "D03": {
        "coordinates_p14r_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
        "in_span": true,
        "transverse": false
      },
      "D01+D03": {
```

```
        "coordinates_p14r_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
        "in_span": true,
        "transverse": true
      }
    },
    "n_4D_orbit_labels": 6,
    "dim_pair_symmetric_4D_structures": 5,
    "basis_rank": 5,
    "note": "the five p14r h.h directions span all five symmetric 4-derivative 4D structures, so span member
+> ship is automatic; the sharper, non-automatic statement is transversality: the summed pole lies in the 2-
+> dim transverse subspace, the individual poles do not"
  },
  "evanescent_poles_D01_plus_D03_in_hh_green_function": {
    "g4_mn gt_rs (p.p)^2": "I/15",
    "g4_mr gt_ns (p.p)^2": "I/240",
    "g4_rs gt_mn (p.p)^2": "I/15",
    "gt_mn gt_rs (p.p)^2": "-I/240",
    "gt_mn p_r p_s (p.p)^1": "-I/15",
    "gt_mr gt_ns (p.p)^2": "-I/240",
    "gt_mr p_n p_s (p.p)^1": "-I/240",
    "gt_rs p_m p_n (p.p)^1": "-I/15"
  },
  "phase_convention_G6": "G = kappa^2 N_eps Ghat, N_eps = i S_eps/(16 pi^2) for every graph; D01/D03 Ghat =
+> -i pref/kappa^2 int (real export prefactors kappa^2/2, -kappa^2) carry an overall I; D13 Ghat (prefactor
+> I kappa^2) is real; s h = i(...) so X_hh is real like the D13 slot; the D01/D03 relative phase is tested
+> by H3 (mutation M1 breaks it); the D13 vs hh relative phase is not tested by a nonzero identity (both slo
+> t contributions vanish)",
  "controls": {
    "ledger_hh_graphs_enter_exactly_one_row": true,
    "H1_operator_exact_unique": true,
    "H1_operator_longitudinal": true,
    "vertex_equals_P16_certificate": true,
    "H3_p_dot_G_sum_zero_exact_in_d": true,
    "H3_evanescent_part_p_dot_G_sum_zero": true,
    "H3_4D_part_p_dot_G_sum_zero": true,
    "H3_slot_sum_zero_all_external_assignments": true,
    "H4_D01_slot_nonzero": true,
    "H4_D03_slot_nonzero": true,
    "H4_evanescent_parts_individually_reach_slot": true,
    "H4_4D_poles_individually_not_transverse": true,
    "4D_pole_sum_transverse": true,
    "transverse_subspace_dimension_2": true,
    "hh_basis_spans_all_pair_symmetric_4D_structures": true,
    "D01_plus_D03_coordinates_in_transverse_kernel": true,
    "mutations_all_rejected": true
  }
},
"phase_convention_G6": "Overall phase (G6).  Every graph is exported as G = kappa^2 N_eps Ghat with N_eps =
+> i S_eps/(16 pi^2) and Ghat = -i * prefactor/kappa^2 * int N/(D1 D2).  For D01 (export prefactor kappa^2/2
+> ) and D03 (prefactor -kappa^2, closed ghost loop) the prefactor is real, so A(d) carries an overall facto
+> r I; for D13 the exported prefactor is I*kappa^2, so A(d) is real.  The relative phase between graphs of
+> the same slot is fixed by the common -i and the real export prefactors (D01 vs D03: real ratio); it is ch
+> ecked by the exact cancellation of the summed hh slot and by the rejected mutation M1 (relative sign).  P
+> hysical overall phase: not asserted.",
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_changed_vs_p16": true,
  "disclosed_revisions_p17": {
    "code/p5_entry/dred.py": {
      "kind": "contract revision (disclosed), p17 (on top of the p15 F1 and p16 F3 revisions)",
      "finding": "G1/G2/G3, reproducibility/independent_rebuild_report_p16.json",
      "why": "G2: sympify of a stored string yields a plain Symbol('d') that is not the opaque d (subs/has a
+> gainst dred.d then silently do nothing); G1/G3: the p16 docstring/disclosure overstated the coverage of t
+> he guard and of the static scan",
      "change": "new guarded loader dred.parse_d(text) (locals {'d': opaque d}; refuses a result with a plai
+> n symbol named d), dred.assert_opaque_d(expr) and exception PlainDimensionSymbol; docstring now states ex
+> actly what the runtime guard refuses and what it does not (Wild-pattern match, C/Fortran printers); unhoo
+> kable value paths (replace with ANY function query, string round-trips via sympify/S/parse_expr/eval/exec
+> , srepr+eval, Poly(..).eval, e.func(*args) rebuilds) are excluded by the extended static AST scan (tests/
+> test_p17.py), a pattern check, not a proof",
      "rules_R1_R6_changed": false,
      "released_values_affected": false,
      "why_unaffected": "only functions were added (no existing code path changed); the P14/P15/P16 certific
+> ate content hashes recomputed in p17 equal those of archival/p16/reference_values_p16.json (tests/test_p1
+> 7.py)"
    }
  },
  "guard_probes": {
    "probes": {
```

```
      "expr.replace(d, 4)": "refused",
      "sympy.lambdify(d, expr)": "refused",
      "expr.match(d)": "refused",
      "expr.subs(d, 4)": "refused",
      "expr.xreplace({d: 4})": "refused",
      "dred.parse_d(str(expr)).subs(d, 4)": "refused"
    },
    "all_refused": true,
    "G2": {
      "parse_d_round_trip_equals_original": true,
      "parse_d_restores_opaque_d": true,
      "plain_sympify_detected_by_assert_opaque_d": true,
      "extra_locals_rebinding_d_refused": true
    },
    "G2_ok": true,
    "authorised_path_works": true
  },
  "evanescent_hh_counterterm_added": false,
  "why_no_evanescent_hh_counterterm": "D01/D03 evanescent poles decouple from the ch-projected slot as the s
+> um D01 + D03 (proof, certificates/P17_hh_slot.json)"
},
"contract_revision_disclosed": true,
"disclosed_test_revisions_p17": {
  "tests/test_p14.py": "source-set / zip / reference names p16 -> p17 (SM.ZIP_NAME, SM.REFERENCE_FILE, autho
+> r report path, resumed-build placeholder zip); generated-prefix list extended by P17_* certificates/ledge
+> rs and the P17 results table; top-level status revision p17. No assertion weakened.",
  "tests/test_p16.py": "rs() reads the p16 LAYER record certificates/P16_release.json (the top-level RELEASE
+> _STATUS.json is now the p17 status); the footer test checks that the build's CURRENT label is 'XVI-P <cur
+> rent revision>' (no longer equal to the p16 label); the p16 layer's own independent flag may be False (th
+> e live p16 report is bound to the p16 tree) or pending. Three lines; all other p16 assertions unchanged."
},
"p16_layer": {
  "p16_layer_files_byte_identical": true,
  "P16_release_accepted": true,
  "batchC_evaluated": true,
  "E2_status": "decoupled_from_ch_slot_proven"
},
"change_log_vs_p16": {
  "base": "archival/p16/p16_source_files.json (pinned; files of the p16 zip sha256 56d5887daa7a96dcc7cf2f70c
+> 6735e802125c9b19a33ac17a3ef975e330fef57)",
  "base_pinned_ok": true,
  "reference_file": "reproducibility/reference_values_p23.json listed as added by declaration (written after
+>  the author build)",
  "added": ["archival/p16/README.md", "archival/p16/p16_source_files.json", "archival/p16/reference_values_p
+> 16.json", "archival/p16/reproducibility/author_rebuild_report_p16.json", "archival/p16/reproducibility/in
+> dependent_rebuild_report_p16.json", "archival/p17/README.md", "archival/p17/p17_source_files.json", "arch
+> ival/p17/reference_values_p17.json", "archival/p17/reproducibility/author_rebuild_report_p17.json", "arch
+> ival/p17/reproducibility/independent_rebuild_report_p17.json", "archival/p18/README.md", "archival/p18/p1
+> 8_source_files.json", "archival/p18/reference_values_p18.json", "archival/p18/reproducibility/author_rebu
+> ild_report_p18.json", "archival/p18/reproducibility/independent_rebuild_report_p18.json", "archival/p19/R
+> EADME.md", "archival/p19/p19_source_files.json", "archival/p19/reference_values_p19.json", "archival/p19/
+> reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility/independent_rebuild_report
+> _p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/reference_value
+> s_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproducibility
+> /independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.json", "a
+> rchival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "a
+> rchival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22
+> /certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_values_
+> p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/i
+> ndependent_rebuild_report_p22.json", "code/p5_calc/contract_registry.py", "code/p5_calc/d11.py", "code/p5
+> _calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/evanescent.py", "code/p5_calc/hh_slot.py", "co
+> de/p5_calc/p17_conventions.py", "code/p5_calc/p18_conventions.py", "code/p5_calc/p19_conventions.py", "co
+> de/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "co
+> de/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p17.py", "code/p5
+> _calc/release_p18.py", "code/p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/releas
+> e_p21.py", "code/p5_calc/release_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P17_hh_slot.te
+> x", "manuscript/notes/P18_D11.tex", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_correction.t
+> ex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P2
+> 3_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_values_p23.json", "
+> tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "
+> tests/test_p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p16.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.
+> tex", "manuscript/preamble.tex", "tests/test_p14.py", "tests/test_p16.py", "tools/build.py", "tools/rebui
+> ld_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p16/README.md", "archival
+> /p16/p16_source_files.json", "archival/p16/reference_values_p16.json", "archival/p16/reproducibility/auth
+> or_rebuild_report_p16.json", "archival/p16/reproducibility/independent_rebuild_report_p16.json", "archiva
+> l/p17/README.md", "archival/p17/p17_source_files.json", "archival/p17/reference_values_p17.json", "archiv
+> al/p17/reproducibility/author_rebuild_report_p17.json", "archival/p17/reproducibility/independent_rebuild
```

```
+> _report_p17.json", "archival/p18/README.md", "archival/p18/p18_source_files.json", "archival/p18/referenc
+> e_values_p18.json", "archival/p18/reproducibility/author_rebuild_report_p18.json", "archival/p18/reproduc
+> ibility/independent_rebuild_report_p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.js
+> on", "archival/p19/reference_values_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.js
+> on", "archival/p19/reproducibility/independent_rebuild_report_p19.json", "archival/p20/README.md", "archi
+> val/p20/p20_source_files.json", "archival/p20/reference_values_p20.json", "archival/p20/reproducibility/a
+> uthor_rebuild_report_p20.json", "archival/p20/reproducibility/independent_rebuild_report_p20.json", "arch
+> ival/p21/README.md", "archival/p21/p21_source_files.json", "archival/p21/reference_values_p21.json", "arc
+> hival/p21/reproducibility/author_rebuild_report_p21.json", "archival/p21/reproducibility/independent_rebu
+> ild_report_p21.json", "archival/p22/README.md", "archival/p22/certificates/P22_evanescent.json", "archiva
+> l/p22/p22_source_files.json", "archival/p22/reference_values_p22.json", "archival/p22/reproducibility/aut
+> hor_rebuild_report_p22.json", "archival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p
+> 5_calc/contract_registry.py", "code/p5_calc/d11.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py",
+>  "code/p5_calc/evanescent.py", "code/p5_calc/hh_slot.py", "code/p5_calc/p17_conventions.py", "code/p5_cal
+> c/p18_conventions.py", "code/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.py", "code/p5_cal
+> c/p21_conventions.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_cal
+> c/power_counting.py", "code/p5_calc/release_p17.py", "code/p5_calc/release_p18.py", "code/p5_calc/release
+> _p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py", "c
+> ode/p5_calc/release_p23.py", "manuscript/body/XVI-P-body.tex", "manuscript/notes/P17_hh_slot.tex", "manus
+> cript/notes/P18_D11.tex", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_correction.tex", "manu
+> script/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscript/notes/P23_01_scope
+> .tex", "manuscript/preamble.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility/reference_va
+> lues_p16.json", "reproducibility/reference_values_p23.json", "tests/test_p14.py", "tests/test_p16.py", "t
+> ests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "t
+> ests/test_p22.py", "tests/test_p23.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_m
+> anifest.py"],
  "counts": {
    "added": 73,
    "removed": 1,
    "modified": 10
  }
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
    "C7_grav_noether_k0_k1_k2_on_exports": true,
    "quartic_mutations_all_rejected": true,
    "master_grav_noether_deterministic_k0_k1_k2": true,
    "master_engine_parts_exact": true,
    "graviton_hessian_generic_symbolic_k": true,
    "canonicity_all_23_antifield_components": true,
    "ch_enumeration_lemma_mechanical": true,
    "residual_record_invariants_all_certificates": true,
    "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
    "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
    "cross_certificate_quartic_vs_master_agree": true,
    "stored_C7_flag_equals_recomputation": true
  },
  "failed_prerequisites": [],
  "hash_consistency": {
    "consistent": true,
    "ell": {
      "ell2": {
        "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
      },
      "ell3": {
```

```
      "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
    },
    "ell4": {
      "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
    "quartic_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    },
    "master_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    }
  }
},
"hard_stops_p13_hold": true,
"hard_stops_p13": {
  "quantum_inputs_evaluated": "not_earned",
  "projected_identity_verified": "not_earned",
  "D02_quantum_evaluated": false,
```

```
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"D11_matching_verified": false,
"quantum_inputs_evaluated": "not_earned",
"hard_stops": {
  "quantum_inputs_evaluated": "not_earned",
  "projected_defect_computed": "not_earned",
  "projected_identity_verified": "not_earned",
  "projected_restoration_verified": "not_earned",
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "D11_matching_verified": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false,
  "D11_evaluated": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "in progress (p15: Batch A D02/D12/D14a-c and Batch B D13; p16: E2 decoupled from the
+>  ch slot (proof), Batch C D01/D03 evaluated; p17: D01/D03 evanescent poles decoupled from the ch slot as
+> the sum D01 + D03 (proof); D11 matching M2 NOT evaluated)",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not ear
+> ned)"
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p16": true,
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p17) sourc
+> e tree only (independent: pending until a reviewer report for this tree exists); independent_rebuild_pass
+> ed_p16 refers to the p16 tree 90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e / zip 56d5
+> 887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57 (same-box caveat) and is not inherited by p1
+> 7; _p15 / _p14r / _p13 as recorded by the p16 layer",
"p16_independent_rebuild": {
  "report": "archival/p16/reproducibility/independent_rebuild_report_p16.json",
  "present": true,
  "report_sha256": "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
  "bound_to_source_zip_sha256": "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57",
  "tree_matches_p16": true,
  "zip_matches_p16": true,
  "mismatches": 0,
```

```
  "findings": ["G1_static_scan_scope_narrower_than_disclosure", "G2_plain_symbol_d_from_stored_strings", "G3
+> _hook_coverage_detail", "G4_change_log_omits_new_reference_file", "G5_test_p15_assertion_relaxed", "G6_im
+> aginary_normalisation", "G7_scope", "F3_F4_F5_from_p15"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent flag",
  "archived_p16_records_pinned": {
    "archival/p16/reference_values_p16.json": true,
    "archival/p16/reproducibility/author_rebuild_report_p16.json": true,
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": true,
    "archival/p16/p16_source_files.json": true
  },
  "pins_ok": true
},
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
```

```
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
```

```
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p17.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "vendored",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
```

```
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p17.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": false,
    "inherited_mode": null,
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "pdfinfo": "pdfinfo version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
      "venv": "/workspace/indep_rebuild_p17/venv",
      "pip_installed": "sympy==1.14.0 mpmath==1.3.0 pytest==9.1.1 (exact versions via pip)"
    }
  },
  "p16_archive": "archival/p16",
  "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
},
"XVI_P_registered": false,
"XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
"XVI_M_pin_ok": true,
"symmetry_restored": false,
"bulk_symmetry_restored": false,
"O1_closure": "NOT_CLOSED",
"insertion_status": "Frozen_negative_obstruction_preserved",
"XV_B_opened": false,
"XIV_J_opened": false,
"Route_B_discharged": false,
"u_R_c_delta_added": false,
"inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
  "P14_release.json": "23b5b7bd32da5e33b794827ae40dc2de0a73a547c9ec26d7a69b08b87e727b63",
  "P15_release.json": "6010e0f56336e10d594a2834dc16395aebe89690224b08f293b82fa5db4797de",
  "P16_release.json": "f81db7fdaf2e28a4451cfabb97db61b46b7f472e4b45ed911b8326b87d2893d6",
  "P17_hh_slot.json": "e848b57061da2eb5dd1801a5d32f1e184639a813295272f24066a8cc2849c1e5"
}
}
```


52 Milestone ledger (p17 layer)

[listing 19/133] ledgers/P17_milestones.json


```
{
  "landmark": "XVI-P",
  "revision": "p17",
  "ledger": "milestones (authoritative definitions + status)",
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "in progress (p15: Batch A D02/D12/D14a-c and Batch B D13; p16: E2 decoupled from the c
  +> h slot (proof), Batch C D01/D03 evaluated; p17: D01/D03 evanescent poles decoupled from the ch slot as th
  +> e sum D01 + D03 (proof); D11 matching M2 NOT evaluated)",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not earne
```

```
  +> d)",
  "author_rebuild_passed": false,
  "independent_rebuild_passed": false,
  "independent_rebuild_passed_p16": true,
  "hard_stops": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "D11_matching_verified": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "u_R_c_delta_added": false,
    "XVI_P_registered": false,
    "D11_evaluated": false
  },
  "flags": {
    "batchA_evaluated": true,
    "D13_evaluated": true,
    "batchC_evaluated": true,
    "E2_decoupling_proven": true,
    "hh_evanescent_decoupling_proven": true
  }
}
```


53 P.5 calculation: D01/D03 evanescent poles vs the ch-projected slot (decoupling proof; p17)

[listing 20/133] certificates/P17_hh_slot.json


```
{
  "landmark": "XVI-P",
  "revision": "p17",
  "deliverable": "D01/D03 evanescent poles vs the ch-projected slot (row (S0,Gamma1^[2]) s2 K:K.c)",
  "question": "do the evanescent (gt) UV poles of D01/D03 enter the ch-projected one-loop identity?",
  "verdict": "DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph",
  "status": "decoupled_from_ch_slot_proven_as_sum",
  "surviving_evanescent_structures_in_slot": [],
  "row": "(S0, Gamma1^[2]) s2 K:K.c (ledgers/P13_ch_dependency_rows.json; graphs D01, D02, D03, D11, D12); the
  +>  hh two-point function enters no other ch row (computed from the ledger: rows_with_D01_D03)",
  "rows_with_D01_D03": {
    "(S0,Gamma1^[2]) s2 K:K.c": {
      "Gamma1_pattern": "h^2",
      "S_monomial": "K.c",
      "differentiated": "K",
      "graphs": ["D01_hh_bubble_cubic", "D02_hh_tadpole_quartic", "D03_hh_ghost_loop", "D11_hh_scalar_loop", "
  +> D12_hh_scalar_tadpole"],
      "status": "surviving_unevaluated"
    }
  },
  "functional": "X_hh[c,h] = int (s h)_{(mn)}(p; c) G^{(mn)(rs)}(p) h_(rs)(-p)",
  "operator_s_h": {
    "exact": true,
    "unique": true,
    "offdiag_weight": 1,
    "upper": false,
    "n_components": 16,
    "rank": 4,
    "n_ansatz": 4,
    "covariant": [
      [
        "I/2",
        [
          ["g4", "m", "v"],
          ["p", "n"]
        ]
      ]
    ]
  },
  "proof": {
    "H1": "s h read off S_Psi^(0) (K.c monomial): exact unique covariant form, longitudinal in (mn)",
    "H2": "X_g depends on G only through p_m G^{(mn)(rs)}; p in QdS so p.gt = 0 removes gt on (mn)",
```

```
  "H3": "p_m (G_D01 + G_D03)^{(mn)(rs)} = 0 identically in d (4D and evanescent parts separately); X_{D01+D0
+> 3} = 0 for every external assignment; D02 = D12 = 0 (P15_batchA.json)",
  "H4": "controls: X_D01, X_D03 individually nonzero (also their evanescent parts); 4D poles individually no
+> t transverse",
  "order": "one loop: the ch projection is linear in Gamma1; the cancellation is exact in d (not O(eps)); no
+>  product of evanescent poles arises"
},
"p_dot_G": {
  "D01": {
    "(('g', '4', 'n', 'r'), ('v', 'p', 's'))": "-I*p.p**2/(16*(d - 1))",
    "(('g', '4', 'n', 's'), ('v', 'p', 'r'))": "-I*p.p**2/(16*(d - 1))",
    "(('g', '4', 'r', 's'), ('v', 'p', 'n'))": "-I*p.p**2*(d - 3)/(16*(d - 1))",
    "(('g', 't', 'n', 'r'), ('v', 'p', 's'))": "I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'n', 's'), ('v', 'p', 'r'))": "I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'r', 's'), ('v', 'p', 'n'))": "-I*p.p**2/(8*(d - 1))"
  },
  "D03": {
    "(('g', '4', 'n', 'r'), ('v', 'p', 's'))": "I*p.p**2/(16*(d - 1))",
    "(('g', '4', 'n', 's'), ('v', 'p', 'r'))": "I*p.p**2/(16*(d - 1))",
    "(('g', '4', 'r', 's'), ('v', 'p', 'n'))": "I*p.p**2*(d - 3)/(16*(d - 1))",
    "(('g', 't', 'n', 'r'), ('v', 'p', 's'))": "-I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'n', 's'), ('v', 'p', 'r'))": "-I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'r', 's'), ('v', 'p', 'n'))": "I*p.p**2/(8*(d - 1))"
  },
  "D01+D03": {},
  "evanescent_D01": {
    "(('g', 't', 'n', 'r'), ('v', 'p', 's'))": "I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'n', 's'), ('v', 'p', 'r'))": "I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'r', 's'), ('v', 'p', 'n'))": "-I*p.p**2/(8*(d - 1))"
  },
  "evanescent_D03": {
    "(('g', 't', 'n', 'r'), ('v', 'p', 's'))": "-I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'n', 's'), ('v', 'p', 'r'))": "-I*p.p**2/(16*(d - 1))",
    "(('g', 't', 'r', 's'), ('v', 'p', 'n'))": "I*p.p**2/(8*(d - 1))"
  },
  "evanescent_D01+D03": {}
},
"slot_values": {
  "D01": {
    "Q4S (declared, R2)": "p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.p
+>  - a~.c~*b^.p)/(16*(d - 1))",
    "QdS (hat)": "p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "D03": {
    "Q4S (declared, R2)": "-p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^.
+> p - a~.c~*b^.p)/(16*(d - 1))",
    "QdS (hat)": "-p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "D01+D03": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  }
},
"per_structure": {
  "D01": {
    "g4_mn g4_rs (p.p)^2": {
      "evanescent": false,
      "reaches_slot": true,
      "gt_on_differentiated_pair": false
    },
    "g4_mn gt_rs (p.p)^2": {
      "evanescent": true,
      "reaches_slot": true,
      "gt_on_differentiated_pair": false
    },
    "g4_mn p_r p_s (p.p)^1": {
      "evanescent": false,
      "reaches_slot": true,
      "gt_on_differentiated_pair": false
```

```
  },
  "g4_mr g4_ns (p.p)^2": {
    "evanescent": false,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "g4_mr gt_ns (p.p)^2": {
    "evanescent": true,
    "reaches_slot": true,
    "gt_on_differentiated_pair": true
  },
  "g4_mr p_n p_s (p.p)^1": {
    "evanescent": false,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "g4_rs gt_mn (p.p)^2": {
    "evanescent": true,
    "reaches_slot": false,
    "gt_on_differentiated_pair": true
  },
  "g4_rs p_m p_n (p.p)^1": {
    "evanescent": false,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "gt_mn gt_rs (p.p)^2": {
    "evanescent": true,
    "reaches_slot": false,
    "gt_on_differentiated_pair": true
  },
  "gt_mn p_r p_s (p.p)^1": {
    "evanescent": true,
    "reaches_slot": false,
    "gt_on_differentiated_pair": true
  },
  "gt_mr gt_ns (p.p)^2": {
    "evanescent": true,
    "reaches_slot": false,
    "gt_on_differentiated_pair": true
  },
  "gt_mr p_n p_s (p.p)^1": {
    "evanescent": true,
    "reaches_slot": true,
    "gt_on_differentiated_pair": true
  },
  "gt_rs p_m p_n (p.p)^1": {
    "evanescent": true,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "p_m p_n p_r p_s": {
    "evanescent": false,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  }
},
"D03": {
  "g4_mn g4_rs (p.p)^2": {
    "evanescent": false,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "g4_mn gt_rs (p.p)^2": {
    "evanescent": true,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "g4_mn p_r p_s (p.p)^1": {
    "evanescent": false,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "g4_mr g4_ns (p.p)^2": {
    "evanescent": false,
    "reaches_slot": true,
    "gt_on_differentiated_pair": false
  },
  "g4_mr gt_ns (p.p)^2": {
    "evanescent": true,
    "reaches_slot": true,
```

```
      "gt_on_differentiated_pair": true
    },
    "g4_mr p_n p_s (p.p)^1": {
      "evanescent": false,
      "reaches_slot": true,
      "gt_on_differentiated_pair": false
    },
    "g4_rs gt_mn (p.p)^2": {
      "evanescent": true,
      "reaches_slot": false,
      "gt_on_differentiated_pair": true
    },
    "g4_rs p_m p_n (p.p)^1": {
      "evanescent": false,
      "reaches_slot": true,
      "gt_on_differentiated_pair": false
    },
    "gt_mn p_r p_s (p.p)^1": {
      "evanescent": true,
      "reaches_slot": false,
      "gt_on_differentiated_pair": true
    },
    "gt_mr p_n p_s (p.p)^1": {
      "evanescent": true,
      "reaches_slot": true,
      "gt_on_differentiated_pair": true
    },
    "gt_rs p_m p_n (p.p)^1": {
      "evanescent": true,
      "reaches_slot": true,
      "gt_on_differentiated_pair": false
    },
    "p_m p_n p_r p_s": {
      "evanescent": false,
      "reaches_slot": true,
      "gt_on_differentiated_pair": false
    }
  }
},
"evanescent_poles_D01_plus_D03_in_hh_green_function": {
  "g4_mn gt_rs (p.p)^2": "I/15",
  "g4_mr gt_ns (p.p)^2": "I/240",
  "g4_rs gt_mn (p.p)^2": "I/15",
  "gt_mn gt_rs (p.p)^2": "-I/240",
  "gt_mn p_r p_s (p.p)^1": "-I/15",
  "gt_mr gt_ns (p.p)^2": "-I/240",
  "gt_mr p_n p_s (p.p)^1": "-I/240",
  "gt_rs p_m p_n (p.p)^1": "-I/15"
},
"consequence": "no evanescent h.h counterterm is required by the ch-projected identity at one loop, so NO co
+> ntract revision is made for it.  The summed evanescent poles of the hh Green function are nonzero (list a
+> bove) but TRANSVERSE; renormalising the hh Green function itself would need evanescent h.h counterterms -
+> - relevant only outside this projection and NOT claimed.  D11 (same row) is NOT evaluated: its slot contr
+> ibution is open.",
"four_D_poles": {
  "basis_dimension": 5,
  "span_is_all_4D_structures": true,
  "longitudinal_constraint_rank": 3,
  "transverse_subspace_dimension": 2,
  "transverse_basis_in_p14r_coordinates": [
    ["1/2", "-1/2", "-1", "1", "0"],
    ["1", "0", "-2", "0", "1"]
  ],
  "graphs": {
    "D01": {
      "coordinates_p14r_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
      "in_span": true,
      "transverse": false
    },
    "D03": {
      "coordinates_p14r_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
      "in_span": true,
      "transverse": false
    },
    "D01+D03": {
      "coordinates_p14r_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
      "in_span": true,
      "transverse": true
    }
  },
  "n_4D_orbit_labels": 6,
```

```
  "dim_pair_symmetric_4D_structures": 5,
  "basis_rank": 5,
  "note": "the five p14r h.h directions span all five symmetric 4-derivative 4D structures, so span membersh
+> ip is automatic; the sharper, non-automatic statement is transversality: the summed pole lies in the 2-di
+> m transverse subspace, the individual poles do not"
},
"phase_convention_G6": "G = kappa^2 N_eps Ghat, N_eps = i S_eps/(16 pi^2) for every graph; D01/D03 Ghat = -i
+>  pref/kappa^2 int (real export prefactors kappa^2/2, -kappa^2) carry an overall I; D13 Ghat (prefactor I
+> kappa^2) is real; s h = i(...) so X_hh is real like the D13 slot; the D01/D03 relative phase is tested by
+>  H3 (mutation M1 breaks it); the D13 vs hh relative phase is not tested by a nonzero identity (both slot
+> contributions vanish)",
"D11": "NOT evaluated (M2 undischarged); its contribution to this row is open",
"controls": {
  "ledger_hh_graphs_enter_exactly_one_row": true,
  "H1_operator_exact_unique": true,
  "H1_operator_longitudinal": true,
  "vertex_equals_P16_certificate": true,
  "H3_p_dot_G_sum_zero_exact_in_d": true,
  "H3_evanescent_part_p_dot_G_sum_zero": true,
  "H3_4D_part_p_dot_G_sum_zero": true,
  "H3_slot_sum_zero_all_external_assignments": true,
  "H4_D01_slot_nonzero": true,
  "H4_D03_slot_nonzero": true,
  "H4_evanescent_parts_individually_reach_slot": true,
  "H4_4D_poles_individually_not_transverse": true,
  "4D_pole_sum_transverse": true,
  "transverse_subspace_dimension_2": true,
  "hh_basis_spans_all_pair_symmetric_4D_structures": true,
  "D01_plus_D03_coordinates_in_transverse_kernel": true,
  "mutations_all_rejected": true
},
"mutations": {
  "M1_wrong_relative_sign_D03": {
    "summed_slot_nonzero": true,
    "rejected": true,
    "accepted_as_rejection": true,
    "slot_values": {
      "Q4S (declared, R2)": "p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c^
+> .p - a~.c~*b^.p)/(8*(d - 1))",
      "QdS (hat)": "p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(8*(d - 1))",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0"
    }
  },
  "M2_D03_times_2_symmetry_factor": {
    "summed_slot_nonzero": true,
    "rejected": true,
    "accepted_as_rejection": true,
    "slot_values": {
      "Q4S (declared, R2)": "-p.p**2*(d*a.b*c^.p - 3*a.b*c^.p + a.c*b^.p + a^.p*b.c - a^.p*b~.c~ + 2*a~.b~*c
+> ^.p - a~.c~*b^.p)/(16*(d - 1))",
      "QdS (hat)": "-p.p**2*(d*a^.b^*c^.p - 3*a^.b^*c^.p + a^.c^*b^.p + a^.p*b^.c^)/(16*(d - 1))",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0"
    }
  },
  "M3_D03_evanescent_dropped_naive_4D": {
    "summed_slot_nonzero": true,
    "rejected": true,
    "accepted_as_rejection": true,
    "slot_values": {
      "Q4S (declared, R2)": "-p.p**2*(a^.p*b~.c~ - 2*a~.b~*c^.p + a~.c~*b^.p)/(16*(d - 1))",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "-a~.c~*b^.p*p.p**2/(16*(d - 1))",
      "c hat, h mixed": "0",
      "all tilde": "0"
    }
  },
  "M4_trace_operator_in_place_of_s_h": {
    "summed_slot_nonzero": true,
    "rejected": true,
    "accepted_as_rejection": true,
    "slot_values": {
      "Q4S (declared, R2)": "c^.p*p.p*(7*d*a.b*p.p - 7*d*a^.p*b^.p - 2*a.b*p.p + 2*a^.p*b^.p - 5*a~.b~*p.p)/
+> (8*(d - 1))",
      "QdS (hat)": "c^.p*p.p*(7*d - 2)*(a^.b^*p.p - a^.p*b^.p)/(8*(d - 1))",
```

```
        "c tilde, h hat": "0",
        "c tilde, h mixed": "0",
        "c hat, h mixed": "0",
        "all tilde": "0"
      }
    }
  },
  "decoupling_proven": true,
  "authorised_d_substitutions": {
    "hh_slot: UV pole A(4) of a 4D structure (after exact-d algebra)": 12,
    "hh_slot: evanescent UV pole A(4) (record only)": 14
  },
  "quantum_inputs_evaluated": "not_earned",
  "seconds": 49.8
}
```


54 p16 layer record (release gate p16)

[listing 21/133] certificates/P16_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p16",
  "contract_revision": "p16 (p14r contract + disclosed p15/p16 revisions)",
  "date": "2026-10-08",
  "version_label": "XVI-P p16",
  "deliverable": "P.5 calculation, second step (E2 evanescent analysis; Batch C: D01, D03)",
  "execution_pass": true,
  "claim_status": "P.5_calculation: Batch A, D13 (p15) and Batch C D01, D03 (p16) evaluated under the declared
  +>  prescription; E2 pole decoupled from the ch-projected slot (proof); P.5_entry: earned (p14r contract); c
  +> lassical_readiness (frozen P.4): earned",
  "scope": "P.5 calculation: Batch A (D02, D12, D14a-c), Batch B (D13) and Batch C (D01, D03) evaluated under
  +> the frozen prescription (DRED R1-R6, Feynman +i0, MS-bar, S0; d opaque); D11 NOT evaluated (M2 undischarg
  +> ed), so the hh slot is incomplete and no slot of the identity is complete; NOT quantum inputs, NOT the pr
  +> ojected defect/identity, NOT bulk restoration, NOT the insertion",
  "release_claim_accepted": true,
  "release_claim_accepted_definition": "p14r contract accepted AND p15 layer accepted and byte-identical AND c
  +> ontract identity (only dred.py changed vs p14r; F3 hardening active; no new contract file) AND E2 decoupl
  +> ing proof AND batchC_evaluated AND p16 hard stops",
  "publication": "accepted",
  "flags": {
    "library_certified": true,
    "batchA_evaluated": {
      "execution": true,
      "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
  +> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
  +> V pole, finite part 0, no p14r counterterm required",
      "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
  +>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
  +> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
  +> e; NOT quantum_inputs_evaluated"
    },
    "D13_evaluated": {
      "execution": true,
      "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
  +> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
  +> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
  +> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
      "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
  +> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
  +> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
  +> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
    },
    "batchC_evaluated": {
      "execution": true,
      "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact
  +> covariant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integ
  +> ration routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonlo
  +> cal log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
      "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS;
  +>  evanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm
  +> is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
    },
    "E2_decoupled_from_ch_slot": {
      "execution": true,
      "claim": "DECOUPLED (proof)",
      "scope": "the ch-projected one-loop kappa^2 identity (row (S0,Gamma1^[2]) s1 h:h^2); the E2 pole remains
  +>  a UV pole of the epsilon-scalar K'.c Green function outside this projection"
```

```
  }
},
"batchA_evaluated": true,
"D13_evaluated": true,
"batchC_evaluated": true,
"library_certified": true,
"E2_status": "decoupled_from_ch_slot_proven",
"E2_decoupling_proven": true,
"D01_evaluated": true,
"D03_evaluated": true,
"D11_evaluated": false,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true,
```

```
    "E2_ch_slot": "decoupled_from_ch_slot_proven"
  },
  "D01": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "-27*I/80",
      "g4_mn gt_rs (p.p)^2": "13*I/120",
      "g4_mn p_r p_s (p.p)^1": "19*I/60",
      "g4_mr g4_ns (p.p)^2": "-71*I/240",
      "g4_mr gt_ns (p.p)^2": "-3*I/80",
      "g4_mr p_n p_s (p.p)^1": "11*I/40",
      "g4_rs gt_mn (p.p)^2": "13*I/120",
      "g4_rs p_m p_n (p.p)^1": "19*I/60",
      "gt_mn gt_rs (p.p)^2": "-I/240",
      "gt_mn p_r p_s (p.p)^1": "-3*I/20",
      "gt_mr gt_ns (p.p)^2": "-I/240",
      "gt_mr p_n p_s (p.p)^1": "7*I/120",
      "gt_rs p_m p_n (p.p)^1": "-3*I/20",
      "p_m p_n p_r p_s": "-13*I/15"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(405*L - 982)/1200",
      "g4_mn gt_rs (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_mn p_r p_s (p.p)^1": "I*(724 - 285*L)/900",
      "g4_mr g4_ns (p.p)^2": "I*(1065*L - 1946)/3600",
      "g4_mr gt_ns (p.p)^2": "I*(45*L - 118)/1200",
      "g4_mr p_n p_s (p.p)^1": "I*(97 - 55*L)/200",
      "g4_rs gt_mn (p.p)^2": "I*(523 - 195*L)/1800",
      "g4_rs p_m p_n (p.p)^1": "I*(724 - 285*L)/900",
      "gt_mn gt_rs (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mn p_r p_s (p.p)^1": "I*(90*L - 241)/600",
      "gt_mr gt_ns (p.p)^2": "I*(15*L - 46)/3600",
      "gt_mr p_n p_s (p.p)^1": "I*(277 - 105*L)/1800",
      "gt_rs p_m p_n (p.p)^1": "I*(90*L - 241)/600",
      "p_m p_n p_r p_s": "I*(780*L - 1597)/900"
    },
    "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
    "evanescent_pole_nonzero": true
  },
  "D03": {
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole": {
      "g4_mn g4_rs (p.p)^2": "7*I/48",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mn p_r p_s (p.p)^1": "-I/8",
      "g4_mr g4_ns (p.p)^2": "I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_mr p_n p_s (p.p)^1": "-I/48",
      "g4_rs gt_mn (p.p)^2": "-I/24",
      "g4_rs p_m p_n (p.p)^1": "-I/8",
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "p_m p_n p_r p_s": "I/6"
    },
    "MSbar_finite": {
      "g4_mn g4_rs (p.p)^2": "I*(50 - 21*L)/144",
      "g4_mn gt_rs (p.p)^2": "I*(3*L - 8)/72",
      "g4_mn p_r p_s (p.p)^1": "I*(3*L - 8)/24",
      "g4_mr g4_ns (p.p)^2": "I*(2 - 3*L)/72",
      "g4_mr gt_ns (p.p)^2": "I*(8 - 3*L)/72",
      "g4_mr p_n p_s (p.p)^1": "I*(3*L + 4)/144",
      "g4_rs gt_mn (p.p)^2": "I*(3*L - 8)/72",
      "g4_rs p_m p_n (p.p)^1": "I*(3*L - 8)/24",
      "gt_mn p_r p_s (p.p)^1": "I*(8 - 3*L)/36",
      "gt_mr p_n p_s (p.p)^1": "I*(3*L - 8)/48",
      "gt_rs p_m p_n (p.p)^1": "I*(8 - 3*L)/36",
      "p_m p_n p_r p_s": "I*(5 - 3*L)/18"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_pole_nonzero": true
```

```
  },
  "D11": {
    "status": "unevaluated",
    "reason": "M2 undischarged (D11 matching not verified)"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"D12_status_derivation": "certificates/P15_batchA.json graphs.D12 (route 1 frozen-dred lift + S0, route 2 sy
+> mmetric integration, IR-mass and cutoff analysis)",
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"E2_summary": {
  "outcome": "DECOUPLED (proof)",
```

```
  "E2_status": "decoupled_from_ch_slot_proven",
  "proof": {
    "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0,
+> 'transverse': True}",
    "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
    "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
    "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
    "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution
+> is exactly zero (not O(eps)); no product of evanescent poles arises at this order"
  },
  "consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed
+> for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scala
+> r (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is re
+> levant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
  "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot
+>  status: zero contribution (proven)",
  "D13_slot_contribution": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "controls": {
    "E_covariant_form_exact": true,
    "E_covariant_form_unique": true,
    "L1_E_transverse": true,
    "L2_E2_longitudinal": true,
    "L3_E2_slot_zero_all_externals": true,
    "L4_slot_map_nontrivial": true,
    "D13_A_T1_and_A_E1_zero": true,
    "D13_slot_contribution_zero": true,
    "mutations_all_rejected": true
  }
},
"batchC_summary": {
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators
+> (D01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt =
+> (4-d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the sam
+> e rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/3537691962
+> 25", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
  "controls": {
    "D13_reproduced_by_covariant_route": true,
    "hh_basis_fits_exact": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true,
    "D01_evaluated": true,
    "D03_evaluated": true
  },
  "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)"
},
"contract_identity": {
  "p14r_contract_files": 12,
  "changed_vs_p14r": ["code/p5_entry/dred.py"],
  "added": [],
  "dred_changed_vs_p15": true,
  "disclosed_revisions_p16": {
    "code/p5_entry/dred.py": {
      "kind": "contract revision (disclosed), p16 (on top of the p15 F1 correction)",
      "finding": "F3_opaque_d_bypass, reproducibility/independent_rebuild_report_p15.json",
      "why": "OpaqueDimension guarded subs/xreplace/evalf(subs)/limit, but expr.replace(d, 4) and sympy.lamb
+> dify bypassed it (the reviewer probe returned 0); no package code used those paths",
      "change": "OpaqueDimension.matches raises outside substitute_d (closes expr.replace(d, value) and patt
+> ern queries for d); the code-printer hooks used by lambdify (_lambdacode, _numpycode, _scipycode, _mpmath
```

```
+> code, _pythoncode, _cupycode, _jaxcode, _tensorflowcode) raise outside substitute_d; remaining value path
+> s that the guard CANNOT hook -- replace with a callable query (expr.replace(lambda x: x == d, ...)), Poly
+> (.., d).eval, string round-trips -- are covered by a static AST scan of code/p5_calc and code/p5_entry (t
+> ests/test_p16.py) -- none is used",
      "rules_R1_R6_changed": false,
      "released_values_affected": false,
      "why_unaffected": "no package code used replace/lambdify/Poly.eval on d (static scan); the P14/P15 cer
+> tificate content hashes recomputed in p16 equal those of archival/p15/reference_values_p15.json (tests/te
+> st_p16.py)"
    }
  },
  "disclosed_corrections_p15": {
    "code/p5_entry/dred.py": {
      "kind": "p14r contract revision (disclosed), applied inside p15",
      "finding": "F1_premature_d4_guard_heuristic, reproducibility/independent_rebuild_report_p14r.json",
      "why": "the p14r PrematureD4Substitution guard was a heuristic on the shape of the expression before t
+> he Laurent expansion: FALSE POSITIVE on eta^{rs} int l_r l_s/(l^2-m^2) = J (correct, d-free) and FALSE NE
+> GATIVE under premature_d4 on the same integral (eta_trace_Gt reintroduces d via tr gtilde = 4-d: d J/4, f
+> inite error -m^4/2); the R5 docstring claim 'is detected' did not hold in general",
      "change": "structural, provenance-based safeguard: d is an OpaqueDimension (subs/xreplace raise Premat
+> ureD4Substitution outside dred.substitute_d, which records every authorised use); numeric traces of ghat/
+> gtilde refused (NumericTraceForbidden) unless display_only; the heuristic is removed; R5 docstring correc
+> ted; reviewer toys added as regression checks (reviewer_toys)",
      "rules_R1_R6_changed": false,
      "released_p14r_values_affected": false,
      "why_unaffected": "p14r applied Pipeline.laurent only to toy placeholders; no loop integral was evalua
+> ted"
    }
  },
  "F3_probe": {
    "probes": {
      "expr.replace(d, 4)": "refused",
      "sympy.lambdify(d, expr)": "refused",
      "expr.match(d)": "refused",
      "expr.subs(d, 4)": "refused",
      "expr.xreplace({d: 4})": "refused",
      "expr.evalf(subs={d: 4})": "refused"
    },
    "all_refused": true,
    "authorised_path_works": true
  },
  "evanescent_counterterm_added": false,
  "why_no_evanescent_counterterm": "E2 decoupling from the ch-projected slot proven (P16_E2_evanescent.json)
+> "
},
"contract_revision_disclosed": true,
"p15_layer": {
  "p15_calc_files_byte_identical": true,
  "P15_release_accepted": true,
  "batchA_evaluated": true,
  "D13_evaluated": true,
  "D12_status": "vanishing_under_declared_prescription"
},
"change_log_vs_p15": {
  "base": "archival/p15/p15_source_files.json (pinned; files of the p15 zip sha256 54325c1ff6c079e0389f96d9c
+> 0875524c73f6a706e079fd1734c1b1701c707d3)",
  "base_pinned_ok": true,
  "excluded": "reproducibility/reference_values_p23.json (new in p16; written after the author build)",
  "added": ["archival/p15/README.md", "archival/p15/p15_source_files.json", "archival/p15/reference_values_p
+> 15.json", "archival/p15/reproducibility/author_rebuild_report_p15.json", "archival/p15/reproducibility/in
+> dependent_rebuild_report_p15.json", "archival/p16/README.md", "archival/p16/p16_source_files.json", "arch
+> ival/p16/reference_values_p16.json", "archival/p16/reproducibility/author_rebuild_report_p16.json", "arch
+> ival/p16/reproducibility/independent_rebuild_report_p16.json", "archival/p17/README.md", "archival/p17/p1
+> 7_source_files.json", "archival/p17/reference_values_p17.json", "archival/p17/reproducibility/author_rebu
+> ild_report_p17.json", "archival/p17/reproducibility/independent_rebuild_report_p17.json", "archival/p18/R
+> EADME.md", "archival/p18/p18_source_files.json", "archival/p18/reference_values_p18.json", "archival/p18/
+> reproducibility/author_rebuild_report_p18.json", "archival/p18/reproducibility/independent_rebuild_report
+> _p18.json", "archival/p19/README.md", "archival/p19/p19_source_files.json", "archival/p19/reference_value
+> s_p19.json", "archival/p19/reproducibility/author_rebuild_report_p19.json", "archival/p19/reproducibility
+> /independent_rebuild_report_p19.json", "archival/p20/README.md", "archival/p20/p20_source_files.json", "a
+> rchival/p20/reference_values_p20.json", "archival/p20/reproducibility/author_rebuild_report_p20.json", "a
+> rchival/p20/reproducibility/independent_rebuild_report_p20.json", "archival/p21/README.md", "archival/p21
+> /p21_source_files.json", "archival/p21/reference_values_p21.json", "archival/p21/reproducibility/author_r
+> ebuild_report_p21.json", "archival/p21/reproducibility/independent_rebuild_report_p21.json", "archival/p2
+> 2/README.md", "archival/p22/certificates/P22_evanescent.json", "archival/p22/p22_source_files.json", "arc
+> hival/p22/reference_values_p22.json", "archival/p22/reproducibility/author_rebuild_report_p22.json", "arc
+> hival/p22/reproducibility/independent_rebuild_report_p22.json", "code/p5_calc/batchC.py", "code/p5_calc/c
+> ontract_registry.py", "code/p5_calc/covariant.py", "code/p5_calc/d11.py", "code/p5_calc/d11_pin.py", "cod
+> e/p5_calc/defect.py", "code/p5_calc/e2_evanescent.py", "code/p5_calc/evanescent.py", "code/p5_calc/hh_slo
+> t.py", "code/p5_calc/p16_conventions.py", "code/p5_calc/p17_conventions.py", "code/p5_calc/p18_convention
+> s.py", "code/p5_calc/p19_conventions.py", "code/p5_calc/p20_conventions.py", "code/p5_calc/p21_convention
```

```
+> s.py", "code/p5_calc/p22_conventions.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/power_counting
+> .py", "code/p5_calc/release_p16.py", "code/p5_calc/release_p17.py", "code/p5_calc/release_p18.py", "code/
+> p5_calc/release_p19.py", "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/rele
+> ase_p22.py", "code/p5_calc/release_p23.py", "manuscript/notes/P16_E2_and_batchC.tex", "manuscript/notes/P
+> 17_hh_slot.tex", "manuscript/notes/P18_D11.tex", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20
+> _correction.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manusc
+> ript/notes/P23_01_scope.tex", "provenance/XVI-0/0_quantum_target.json", "tests/test_p16.py", "tests/test_
+> p17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_
+> p22.py", "tests/test_p23.py"],
  "removed": ["reproducibility/reference_values_p15.json"],
  "modified": ["README.md", "REBUILD_INSTRUCTIONS.md", "code/p5_entry/dred.py", "manuscript/body/XVI-P-body.
+> tex", "manuscript/preamble.tex", "tests/test_p14.py", "tests/test_p15.py", "tools/build.py", "tools/rebui
+> ld_from_source.py", "tools/source_manifest.py"],
  "contract_files_changed": ["code/p5_entry/dred.py"],
  "non_contract_files_changed": ["README.md", "REBUILD_INSTRUCTIONS.md", "archival/p15/README.md", "archival
+> /p15/p15_source_files.json", "archival/p15/reference_values_p15.json", "archival/p15/reproducibility/auth
+> or_rebuild_report_p15.json", "archival/p15/reproducibility/independent_rebuild_report_p15.json", "archiva
+> l/p16/README.md", "archival/p16/p16_source_files.json", "archival/p16/reference_values_p16.json", "archiv
+> al/p16/reproducibility/author_rebuild_report_p16.json", "archival/p16/reproducibility/independent_rebuild
+> _report_p16.json", "archival/p17/README.md", "archival/p17/p17_source_files.json", "archival/p17/referenc
+> e_values_p17.json", "archival/p17/reproducibility/author_rebuild_report_p17.json", "archival/p17/reproduc
+> ibility/independent_rebuild_report_p17.json", "archival/p18/README.md", "archival/p18/p18_source_files.js
+> on", "archival/p18/reference_values_p18.json", "archival/p18/reproducibility/author_rebuild_report_p18.js
+> on", "archival/p18/reproducibility/independent_rebuild_report_p18.json", "archival/p19/README.md", "archi
+> val/p19/p19_source_files.json", "archival/p19/reference_values_p19.json", "archival/p19/reproducibility/a
+> uthor_rebuild_report_p19.json", "archival/p19/reproducibility/independent_rebuild_report_p19.json", "arch
+> ival/p20/README.md", "archival/p20/p20_source_files.json", "archival/p20/reference_values_p20.json", "arc
+> hival/p20/reproducibility/author_rebuild_report_p20.json", "archival/p20/reproducibility/independent_rebu
+> ild_report_p20.json", "archival/p21/README.md", "archival/p21/p21_source_files.json", "archival/p21/refer
+> ence_values_p21.json", "archival/p21/reproducibility/author_rebuild_report_p21.json", "archival/p21/repro
+> ducibility/independent_rebuild_report_p21.json", "archival/p22/README.md", "archival/p22/certificates/P22
+> _evanescent.json", "archival/p22/p22_source_files.json", "archival/p22/reference_values_p22.json", "archi
+> val/p22/reproducibility/author_rebuild_report_p22.json", "archival/p22/reproducibility/independent_rebuil
+> d_report_p22.json", "code/p5_calc/batchC.py", "code/p5_calc/contract_registry.py", "code/p5_calc/covarian
+> t.py", "code/p5_calc/d11.py", "code/p5_calc/d11_pin.py", "code/p5_calc/defect.py", "code/p5_calc/e2_evane
+> scent.py", "code/p5_calc/evanescent.py", "code/p5_calc/hh_slot.py", "code/p5_calc/p16_conventions.py", "c
+> ode/p5_calc/p17_conventions.py", "code/p5_calc/p18_conventions.py", "code/p5_calc/p19_conventions.py", "c
+> ode/p5_calc/p20_conventions.py", "code/p5_calc/p21_conventions.py", "code/p5_calc/p22_conventions.py", "c
+> ode/p5_calc/p23_conventions.py", "code/p5_calc/power_counting.py", "code/p5_calc/release_p16.py", "code/p
+> 5_calc/release_p17.py", "code/p5_calc/release_p18.py", "code/p5_calc/release_p19.py", "code/p5_calc/relea
+> se_p20.py", "code/p5_calc/release_p21.py", "code/p5_calc/release_p22.py", "code/p5_calc/release_p23.py",
+> "manuscript/body/XVI-P-body.tex", "manuscript/notes/P16_E2_and_batchC.tex", "manuscript/notes/P17_hh_slot
+> .tex", "manuscript/notes/P18_D11.tex", "manuscript/notes/P19_defect.tex", "manuscript/notes/P20_correctio
+> n.tex", "manuscript/notes/P21_absolute_pin.tex", "manuscript/notes/P22_evanescent.tex", "manuscript/notes
+> /P23_01_scope.tex", "manuscript/preamble.tex", "provenance/XVI-0/0_quantum_target.json", "reproducibility
+> /reference_values_p15.json", "tests/test_p14.py", "tests/test_p15.py", "tests/test_p16.py", "tests/test_p
+> 17.py", "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py", "tests/test_p21.py", "tests/test_p
+> 22.py", "tests/test_p23.py", "tools/build.py", "tools/rebuild_from_source.py", "tools/source_manifest.py"
+> ],
  "counts": {
    "added": 84,
    "removed": 1,
    "modified": 10
  }
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
```

```
  "C7_grav_noether_k0_k1_k2_on_exports": true,
  "quartic_mutations_all_rejected": true,
  "master_grav_noether_deterministic_k0_k1_k2": true,
  "master_engine_parts_exact": true,
  "graviton_hessian_generic_symbolic_k": true,
  "canonicity_all_23_antifield_components": true,
  "ch_enumeration_lemma_mechanical": true,
  "residual_record_invariants_all_certificates": true,
  "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
  "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
  "cross_certificate_quartic_vs_master_agree": true,
  "stored_C7_flag_equals_recomputation": true
},
"failed_prerequisites": [],
"hash_consistency": {
  "consistent": true,
  "ell": {
    "ell2": {
      "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
    },
    "ell3": {
      "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
    },
    "ell4": {
      "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
    "quartic_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
```

```
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      },
      "master_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    }
  },
  "hard_stops_p13_hold": true,
  "hard_stops_p13": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"D11_matching_verified": false,
"quantum_inputs_evaluated": "not_earned",
"hard_stops": {
  "quantum_inputs_evaluated": "not_earned",
  "projected_defect_computed": "not_earned",
  "projected_identity_verified": "not_earned",
  "projected_restoration_verified": "not_earned",
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "D11_matching_verified": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "XVI_P_registered": false,
  "D11_evaluated": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "in progress (p15: Batch A D02/D12/D14a-c and Batch B D13; p16: E2 decoupled from the
+>  ch slot (proof), Batch C D01/D03 evaluated; D11 matching M2 NOT evaluated)",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not ear
+> ned)"
},
"author_rebuild_passed": false,
```

```
"independent_rebuild_passed": false,
"independent_rebuild_passed_p15": true,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p16) sourc
+> e tree only (independent: pending until a reviewer report for this tree exists); independent_rebuild_pass
+> ed_p15 refers to the p15 tree 79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f / zip 5432
+> 5c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3 (same-box caveat) and is not inherited by p1
+> 6; independent_rebuild_passed_p14r / _p13 as recorded by the p15 layer",
"p15_independent_rebuild": {
  "report": "archival/p15/reproducibility/independent_rebuild_report_p15.json",
  "present": true,
  "report_sha256": "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
  "bound_to_source_zip_sha256": "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3",
  "tree_matches_p15": true,
  "zip_matches_p15": true,
  "mismatches": 0,
  "findings": ["F1_F2_resolved", "F3_opaque_d_bypass", "F4_stale_page_footer_label", "F5_non_contract_change
+> s", "F6_D13_spot_check_scope", "F7_evanescent_E2_pole"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent flag",
  "archived_p15_records_pinned": {
    "archival/p15/reference_values_p15.json": true,
    "archival/p15/reproducibility/author_rebuild_report_p15.json": true,
    "archival/p15/p15_source_files.json": true
  },
  "pins_ok": true
},
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
```

```
    "author": {
      "report": "reproducibility/author_rebuild_report_p16.json",
      "present": true,
      "report_pass": true,
      "report_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
      "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
      "bound_to_current_source": false,
      "report_records_source_zip_sha256": true,
      "inherited_mode": "vendored",
      "environment": {
        "python": "3.13.5",
        "sympy": "1.14.0",
        "mpmath": "1.3.0",
        "pytest": "9.1.1",
        "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
        "pdftotext": "pdftotext version 25.03.0",
        "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
      }
    },
    "independent": {
      "report": "reproducibility/independent_rebuild_report_p16.json",
      "present": true,
      "report_pass": true,
      "report_source_tree_sha256": "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e",
      "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
      "bound_to_current_source": false,
      "report_records_source_zip_sha256": false,
      "inherited_mode": null,
      "environment": {
        "python": "3.13.5",
        "sympy": "1.14.0",
        "mpmath": "1.3.0",
        "pytest": "9.1.1",
        "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
        "pdftotext": "pdftotext version 25.03.0",
        "pdfinfo": "pdfinfo version 25.03.0",
        "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
        "venv": "/workspace/indep_rebuild_p16/venv",
        "pip_installed": "sympy==1.14.0 mpmath==1.3.0 pytest==9.1.1 (exact versions via pip)"
      }
    },
    "p15_archive": "archival/p15",
    "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
  },
  "XVI_P_registered": false,
  "XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
  "XVI_M_pin_ok": true,
  "symmetry_restored": false,
  "bulk_symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
    "P14_release.json": "23b5b7bd32da5e33b794827ae40dc2de0a73a547c9ec26d7a69b08b87e727b63",
    "P14_dred_rules.json": "0fff45df2dd1e9cec6e4808c9ac7d5da5fa526c5e9a0971498ea9ad57442b24e",
    "P15_release.json": "6010e0f56336e10d594a2834dc16395aebe89690224b08f293b82fa5db4797de",
    "P16_E2_evanescent.json": "ad81f29cd831f407c39b0d150f7a0c039fc4b56a236de37342bb2a7a07ca681d",
    "P16_batchC.json": "cd2a5d0db4f3f2553ff9ed0817f9dc6dc27c42a5a73f63c3f97c11df2b0b77f3"
  }
}
```


55 Milestone ledger (p16 layer)

[listing 22/133] ledgers/P16_milestones.json


```
{
  "landmark": "XVI-P",
  "revision": "p16",
  "ledger": "milestones (authoritative definitions + status)",
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
```

```
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "in progress (p15: Batch A D02/D12/D14a-c and Batch B D13; p16: E2 decoupled from the c
  +> h slot (proof), Batch C D01/D03 evaluated; D11 matching M2 NOT evaluated)",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not earne
  +> d)",
  "author_rebuild_passed": false,
  "independent_rebuild_passed": false,
  "independent_rebuild_passed_p15": true,
  "hard_stops": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "D11_matching_verified": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "u_R_c_delta_added": false,
    "XVI_P_registered": false,
    "D11_evaluated": false
  },
  "flags": {
    "batchA_evaluated": true,
    "D13_evaluated": true,
    "batchC_evaluated": true,
    "E2_decoupling_proven": true
  }
}
```


56 P.5 calculation: E2 evanescent decoupling proof (D13; p16)

[listing 23/133] certificates/P16_E2_evanescent.json


```
{
  "landmark": "XVI-P",
  "revision": "p16",
  "deliverable": "p16 priority 1: E2 evanescent pole vs the ch slot",
  "question": "does the D13 E2 pole (1/16; A_E2 = 3/(16(d-1))) decouple from the ch-projected slot?",
  "outcome": "DECOUPLED (proof)",
  "E2_status": "decoupled_from_ch_slot_proven",
  "slot": {
    "row": "(S0, Gamma1^[2]) s1 h:h^2 (ledgers/P13_ch_dependency_rows.json; the only row containing D13)",
    "functional": "X_D13[c,h] = int G_{(mn) nu}(p) c^nu(p) E^{(mn)(rs)}(p) h_(rs)(-p)",
    "E": "operator of the h^2 monomials of S_Psi^(0) (counterterm_domain.S0_density, pw_vertex extraction; exa
  +> ct covariant form recovered, unique)",
    "E_fit": {
      "n_ansatz": 16,
      "n_used": 5,
      "exact": true,
      "bad_components": [],
      "rank": 16,
      "unique": true,
      "offdiag_weight": "2",
      "upper": true
    },
    "E_covariant": [
      [
        "-1/2",
        [
          ["g4", "m", "r"],
          ["p", "n"],
          ["p", "s"]
        ]
```

```
    ],
    [
      "-1/2",
      [
        ["g4", "m", "s"],
        ["p", "n"],
        ["p", "r"]
      ]
    ],
    [
      "-1/2",
      [
        ["g4", "n", "r"],
        ["p", "m"],
        ["p", "s"]
      ]
    ],
    [
      "-1/2",
      [
        ["g4", "n", "s"],
        ["p", "m"],
        ["p", "r"]
      ]
    ],
    [
      "-p.p",
      [
        ["g4", "m", "n"],
        ["g4", "r", "s"]
      ]
    ],
    [
      "1",
      [
        ["g4", "m", "n"],
        ["p", "r"],
        ["p", "s"]
      ]
    ],
    [
      "1",
      [
        ["g4", "r", "s"],
        ["p", "m"],
        ["p", "n"]
      ]
    ],
    [
      "p.p/2",
      [
        ["g4", "m", "r"],
        ["g4", "n", "s"]
      ]
    ],
    [
      "p.p/2",
      [
        ["g4", "m", "s"],
        ["g4", "n", "r"]
      ]
    ]
  ]
},
"proof": {
  "L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> {'residual_terms': 0, 't
+> ransverse': True}",
  "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
  "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
  "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
  "order": "one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the E2 contribution is
+>  exactly zero (not O(eps)); no product of evanescent poles arises at this order"
},
"slot_table": {
  "T1": {
    "Q4S (declared, R2)": "-2*c^.p*p.p*(a.b*p.p - a^.p*b^.p)",
    "QdS (hat)": "-2*c^.p*p.p*(a^.b^*p.p - a^.p*b^.p)",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
```

```
  },
  "T2": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "T3": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "E1": {
    "Q4S (declared, R2)": "c^.p*p.p*(d*a.b*p.p - d*a^.p*b^.p - 4*a.b*p.p + 4*a^.p*b^.p + a~.b~*p.p)",
    "QdS (hat)": "c^.p*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  },
  "E2": {
    "Q4S (declared, R2)": "0",
    "QdS (hat)": "0",
    "c tilde, h hat": "0",
    "c tilde, h mixed": "0",
    "c hat, h mixed": "0",
    "all tilde": "0"
  }
},
"external_assignments": {
  "Q4S (declared, R2)": ["c", "a", "b"],
  "QdS (hat)": ["c^", "a^", "b^"],
  "c tilde, h hat": ["c~", "a^", "b^"],
  "c tilde, h mixed": ["c~", "a~", "b^"],
  "c hat, h mixed": ["c^", "a~", "b^"],
  "all tilde": ["c~", "a~", "b~"]
},
"D13_slot_contribution": {
  "Q4S (declared, R2)": "0",
  "QdS (hat)": "0",
  "c tilde, h hat": "0",
  "c tilde, h mixed": "0",
  "c hat, h mixed": "0",
  "all tilde": "0"
},
"consequence": "the E2 pole does not enter the ch-projected identity; no evanescent counterterm is needed fo
+> r the slot, so NO contract revision is made for E2.  The E2 pole remains a UV pole of the epsilon-scalar
+> (Gt) component of the K'.c Green function of Gamma1 (R6: retained, recorded); its renormalization is rele
+> vant only outside the ch projection (other projections, higher loops) and is NOT claimed here",
"D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm); ch-slot s
+> tatus: zero contribution (proven)",
"mutations": {
  "M1_gauge_fixed_operator_qq2P": {
    "E2_slot_nonzero": true,
    "rejected": true,
    "slot_values": {
      "Q4S (declared, R2)": "p.p**2*(a^.p*b~.c~ + a~.c~*b^.p)",
      "QdS (hat)": "0",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "a~.c~*b^.p*p.p**2",
      "c hat, h mixed": "0",
      "all tilde": "0"
    },
    "reason": "L1 fails: p_m (q^2 P)^(mn)(rs) has 3 residual terms (q^2 P is not the h^2-monomial operator o
+> f the slot and is not transverse)",
    "accepted_as_rejection": true
  },
  "M2_E1_in_place_of_E2": {
    "E2_slot_nonzero": true,
    "rejected": true,
    "slot_values": {
      "Q4S (declared, R2)": "c^.p*p.p*(d*a.b*p.p - d*a^.p*b^.p - 4*a.b*p.p + 4*a^.p*b^.p + a~.b~*p.p)",
      "QdS (hat)": "c^.p*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
```

```
      "all tilde": "0"
    },
    "reason": "L2 fails: gt_mn p_nu p^2 is not longitudinal in (mn)",
    "accepted_as_rejection": true
  },
  "M3_wrong_index_slot": {
    "E2_slot_nonzero": true,
    "rejected": true,
    "slot_values": {
      "Q4S (declared, R2)": "c^.p*p.p*(d*a.b*p.p - d*a^.p*b^.p - 4*a.b*p.p + 4*a^.p*b^.p + a~.b~*p.p)",
      "QdS (hat)": "c^.p*p.p*(d - 4)*(a^.b^*p.p - a^.p*b^.p)",
      "c tilde, h hat": "0",
      "c tilde, h mixed": "0",
      "c hat, h mixed": "0",
      "all tilde": "0"
    },
    "reason": "L2 fails for the mis-slotted structure (gt_mn p_nu term)",
    "accepted_as_rejection": true
  }
},
"controls": {
  "E_covariant_form_exact": true,
  "E_covariant_form_unique": true,
  "L1_E_transverse": true,
  "L2_E2_longitudinal": true,
  "L3_E2_slot_zero_all_externals": true,
  "L4_slot_map_nontrivial": true,
  "D13_A_T1_and_A_E1_zero": true,
  "D13_slot_contribution_zero": true,
  "mutations_all_rejected": true
},
"E2_decoupling_proven": true,
"authorised_d_substitutions": {},
"seconds": 5.6
}
```


57 P.5 calculation: Batch C certificate (D01, D03; p16)

[listing 24/133] certificates/P16_batchC.json


```
{
  "landmark": "XVI-P",
  "revision": "p16",
  "deliverable": "P.5 calculation Batch C: D01, D03 (kappa^2 hh)",
  "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 + S0 (P15_integral_library.json); covariant route (cov
  +> ariant.py, finding C1)",
  "convention": "G_{(mn)(rs)}(p) = coefficient tensor of the ordered two-point vertex V_Gamma1[h_(mn)(p), h_(r
  +> s)(-p)] with\nUPPER external indices (dual to the lower field components h_(mn); exported component (a, b
  +> ) = weight(a) weight(b)\nx tensor component, weight 2 for an off-diagonal field component), G = kappa^2 N
  +> _eps Ghat, N_eps = i S_eps/(16 pi^2),\nGhat = -i * prefactor/kappa^2 * int N/(D1 D2) = sum_s A_s(d) s MB,
  +>   MB = 1/eps + 2 - L,  L = log(-p^2/mu^2 - i0).",
  "finding_C1_component_route": {
    "statement": "the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 numerators (D
  +> 01/D03) whose abstract value cannot be recovered after the component sums; the linear realization Gt = (4
  +> -d)M is not faithful there; Batch C therefore uses the covariant route (C1-C5), which evaluates the same
  +> rules R1-R6 with free external indices only",
    "components_with_Gt_quadratic_terms": {
      "D01": 100,
      "D03": 10
    },
    "demonstration": {
      "expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs",
      "abstract_value": "4 - d",
      "realized_values": ["163215913117273*(d - 4)**2/154303081175887", "5518289693389*(d - 4)**2/353769196225
  +> ", "2454341204199841*(d - 4)**2/2188546635429025", "42946156473193*(d - 4)**2/18012647945161"],
      "realization_dependent": true,
      "equals_abstract": false
    },
    "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
    "contract_change": false
  },
  "graphs": {
    "D01": {
      "slot": "kappa^2 hh two-point (row (S0,Gamma1^[2]) s2 K:K.c with D02, D11, D12; D11 NOT evaluated)",
      "export": {
        "file": "exports/P14_integrands/D01.json",
        "sha256_recomputed": "869a5db72f275c25a68b9b0295e3381fdf57b9c71de3c632172d70dce2fabce8",
        "sha256_stored": "869a5db72f275c25a68b9b0295e3381fdf57b9c71de3c632172d70dce2fabce8",
```

```
  "sha256_p14r_certificate": "869a5db72f275c25a68b9b0295e3381fdf57b9c71de3c632172d70dce2fabce8",
  "pinned": true
},
"prefactor_export": "kappa**2/2",
"normalization": "Ghat = -I/2 * int N/(D1 D2) / N_eps",
"covariant_form": {
  "n_ansatz": 51,
  "n_used": 45,
  "exact": true,
  "bad_components": [],
  "rank": 51,
  "unique": true,
  "offdiag_weight": "2",
  "upper": true
},
"result": {
  "g4_mn g4_rs (p.p)^2": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*(-2*d**2 - 10*d - 9)/(16*(d**2 - 1))",
    "UV_pole": "-27*I/80",
    "MSbar_finite": "I*(405*L - 982)/1200",
    "nonlocal_coefficient_of_L": "27*I/80",
    "local_polynomial": "-491*I/600",
    "finite_if_d_set_to_4_first": "27*I*(L - 2)/80",
    "local_shift_from_d_dependence": "-43*I/300"
  },
  "g4_mn gt_rs (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "I*(5*d + 6)/(16*(d**2 - 1))",
    "UV_pole": "13*I/120",
    "MSbar_finite": "I*(523 - 195*L)/1800",
    "nonlocal_coefficient_of_L": "-13*I/120",
    "local_polynomial": "523*I/1800",
    "finite_if_d_set_to_4_first": "13*I*(2 - L)/120",
    "local_shift_from_d_dependence": "133*I/1800"
  },
  "g4_mn p_r p_s (p.p)^1": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*(d**2 + 12*d + 12)/(16*(d**2 - 1))",
    "UV_pole": "19*I/60",
    "MSbar_finite": "I*(724 - 285*L)/900",
    "nonlocal_coefficient_of_L": "-19*I/60",
    "local_polynomial": "181*I/225",
    "finite_if_d_set_to_4_first": "19*I*(2 - L)/60",
    "local_shift_from_d_dependence": "77*I/450"
  },
  "g4_mr g4_ns (p.p)^2": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*(-6*d**2 + 4*d + 9)/(16*(d**2 - 1))",
    "UV_pole": "-71*I/240",
    "MSbar_finite": "I*(1065*L - 1946)/3600",
    "nonlocal_coefficient_of_L": "71*I/240",
    "local_polynomial": "-973*I/1800",
    "finite_if_d_set_to_4_first": "71*I*(L - 2)/240",
    "local_shift_from_d_dependence": "23*I/450"
  },
  "g4_mr gt_ns (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "I*(-2*d - 1)/(16*(d**2 - 1))",
    "UV_pole": "-3*I/80",
    "MSbar_finite": "I*(45*L - 118)/1200",
    "nonlocal_coefficient_of_L": "3*I/80",
    "local_polynomial": "-59*I/600",
    "finite_if_d_set_to_4_first": "3*I*(L - 2)/80",
    "local_shift_from_d_dependence": "-7*I/300"
  },
  "g4_mr p_n p_s (p.p)^1": {
    "evanescent": false,
    "A_exact_d_times_MB": "I*(6*d**2 - 5*d - 10)/(16*(d**2 - 1))",
    "UV_pole": "11*I/40",
    "MSbar_finite": "I*(97 - 55*L)/200",
    "nonlocal_coefficient_of_L": "-11*I/40",
    "local_polynomial": "97*I/200",
    "finite_if_d_set_to_4_first": "11*I*(2 - L)/40",
    "local_shift_from_d_dependence": "-13*I/200"
  },
  "g4_rs gt_mn (p.p)^2": {
    "evanescent": true,
    "A_exact_d_times_MB": "I*(5*d + 6)/(16*(d**2 - 1))",
    "UV_pole": "13*I/120",
    "MSbar_finite": "I*(523 - 195*L)/1800",
```

```
      "nonlocal_coefficient_of_L": "-13*I/120",
      "local_polynomial": "523*I/1800",
      "finite_if_d_set_to_4_first": "13*I*(2 - L)/120",
      "local_shift_from_d_dependence": "133*I/1800"
    },
    "g4_rs p_m p_n (p.p)^1": {
      "evanescent": false,
      "A_exact_d_times_MB": "I*(d**2 + 12*d + 12)/(16*(d**2 - 1))",
      "UV_pole": "19*I/60",
      "MSbar_finite": "I*(724 - 285*L)/900",
      "nonlocal_coefficient_of_L": "-19*I/60",
      "local_polynomial": "181*I/225",
      "finite_if_d_set_to_4_first": "19*I*(2 - L)/60",
      "local_shift_from_d_dependence": "77*I/450"
    },
    "gt_mn gt_rs (p.p)^2": {
      "evanescent": true,
      "A_exact_d_times_MB": "-I/(16*d**2 - 16)",
      "UV_pole": "-I/240",
      "MSbar_finite": "I*(15*L - 46)/3600",
      "nonlocal_coefficient_of_L": "I/240",
      "local_polynomial": "-23*I/1800",
      "finite_if_d_set_to_4_first": "I*(L - 2)/240",
      "local_shift_from_d_dependence": "-I/225"
    },
    "gt_mn p_r p_s (p.p)^1": {
      "evanescent": true,
      "A_exact_d_times_MB": "I*(-7*d - 8)/(16*(d**2 - 1))",
      "UV_pole": "-3*I/20",
      "MSbar_finite": "I*(90*L - 241)/600",
      "nonlocal_coefficient_of_L": "3*I/20",
      "local_polynomial": "-241*I/600",
      "finite_if_d_set_to_4_first": "3*I*(L - 2)/20",
      "local_shift_from_d_dependence": "-61*I/600"
    },
    "gt_mr gt_ns (p.p)^2": {
      "evanescent": true,
      "A_exact_d_times_MB": "-I/(16*d**2 - 16)",
      "UV_pole": "-I/240",
      "MSbar_finite": "I*(15*L - 46)/3600",
      "nonlocal_coefficient_of_L": "I/240",
      "local_polynomial": "-23*I/1800",
      "finite_if_d_set_to_4_first": "I*(L - 2)/240",
      "local_shift_from_d_dependence": "-I/225"
    },
    "gt_mr p_n p_s (p.p)^1": {
      "evanescent": true,
      "A_exact_d_times_MB": "I*(3*d + 2)/(16*(d**2 - 1))",
      "UV_pole": "7*I/120",
      "MSbar_finite": "I*(277 - 105*L)/1800",
      "nonlocal_coefficient_of_L": "-7*I/120",
      "local_polynomial": "277*I/1800",
      "finite_if_d_set_to_4_first": "7*I*(2 - L)/120",
      "local_shift_from_d_dependence": "67*I/1800"
    },
    "gt_rs p_m p_n (p.p)^1": {
      "evanescent": true,
      "A_exact_d_times_MB": "I*(-7*d - 8)/(16*(d**2 - 1))",
      "UV_pole": "-3*I/20",
      "MSbar_finite": "I*(90*L - 241)/600",
      "nonlocal_coefficient_of_L": "3*I/20",
      "local_polynomial": "-241*I/600",
      "finite_if_d_set_to_4_first": "3*I*(L - 2)/20",
      "local_shift_from_d_dependence": "-61*I/600"
    },
    "p_m p_n p_r p_s": {
      "evanescent": false,
      "A_exact_d_times_MB": "I*(-13*d**2 - 2*d + 8)/(16*(d**2 - 1))",
      "UV_pole": "-13*I/15",
      "MSbar_finite": "I*(780*L - 1597)/900",
      "nonlocal_coefficient_of_L": "13*I/15",
      "local_polynomial": "-1597*I/900",
      "finite_if_d_set_to_4_first": "13*I*(L - 2)/15",
      "local_shift_from_d_dependence": "-37*I/900"
    }
  },
  "status": "regulated",
  "sector_status": {
    "4D": "renormalized (pole in the p14r h.h span)",
    "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
```

```
  },
  "UV_pole_in_p14r_hh_basis": ["-27*I/160", "-71*I/240", "19*I/60", "11*I/20", "-13*I/30"],
  "evanescent_poles": {
    "g4_mn gt_rs (p.p)^2": "13*I/120",
    "g4_rs gt_mn (p.p)^2": "13*I/120",
    "gt_mn gt_rs (p.p)^2": "-I/240",
    "g4_mr gt_ns (p.p)^2": "-3*I/80",
    "gt_mr gt_ns (p.p)^2": "-I/240",
    "gt_mn p_r p_s (p.p)^1": "-3*I/20",
    "gt_mr p_n p_s (p.p)^1": "7*I/120",
    "gt_rs p_m p_n (p.p)^1": "-3*I/20"
  },
  "evanescent_pole_nonzero": true,
  "longitudinal_projection_p_m_G": {
    "4_nr p_s": "-I*p.p**2/(16*(d - 1))",
    "4_ns p_r": "-I*p.p**2/(16*(d - 1))",
    "4_rs p_n": "-I*p.p**2*(d - 3)/(16*(d - 1))",
    "t_nr p_s": "I*p.p**2/(16*(d - 1))",
    "t_ns p_r": "I*p.p**2/(16*(d - 1))",
    "t_rs p_n": "-I*p.p**2/(8*(d - 1))"
  },
  "regeneration": {
    "components": 100,
    "mismatches": 0,
    "agree": true
  },
  "component_route_Gt_quadratic_components": 100,
  "controls": {
    "export_sha_pinned": true,
    "covariant_form_exact": true,
    "covariant_form_unique": true,
    "orbit_consistent": true,
    "route_FP_equals_route_PV_exact_in_d": true,
    "regenerated_integrand_equals_export": true,
    "exchange_symmetric_mn_rs": true,
    "nonlocal_equals_minus_pole": true,
    "pole_4D_in_p14r_hh_span": true,
    "R5_d_retained": true
  },
  "evaluated": true
},
"D03": {
  "slot": "kappa^2 hh two-point (row (S0,Gamma1^[2]) s2 K:K.c with D02, D11, D12; D11 NOT evaluated)",
  "export": {
    "file": "exports/P14_integrands/D03.json",
    "sha256_recomputed": "728cd2564ff9024d77e950c07642f4a287d59c0d9c6ff56f4de03d1ed7a978bc",
    "sha256_stored": "728cd2564ff9024d77e950c07642f4a287d59c0d9c6ff56f4de03d1ed7a978bc",
    "sha256_p14r_certificate": "728cd2564ff9024d77e950c07642f4a287d59c0d9c6ff56f4de03d1ed7a978bc",
    "pinned": true
  },
  "prefactor_export": "-kappa**2",
  "normalization": "Ghat = I * int N/(D1 D2) / N_eps",
  "covariant_form": {
    "n_ansatz": 51,
    "n_used": 23,
    "exact": true,
    "bad_components": [],
    "rank": 51,
    "unique": true,
    "offdiag_weight": "2",
    "upper": true
  },
  "result": {
    "g4_mn g4_rs (p.p)^2": {
      "evanescent": false,
      "A_exact_d_times_MB": "I*(d + 3)/(16*(d - 1))",
      "UV_pole": "7*I/48",
      "MSbar_finite": "I*(50 - 21*L)/144",
      "nonlocal_coefficient_of_L": "-7*I/48",
      "local_polynomial": "25*I/72",
      "finite_if_d_set_to_4_first": "7*I*(2 - L)/48",
      "local_shift_from_d_dependence": "I/18"
    },
    "g4_mn gt_rs (p.p)^2": {
      "evanescent": true,
      "A_exact_d_times_MB": "-I/(8*d - 8)",
      "UV_pole": "-I/24",
      "MSbar_finite": "I*(3*L - 8)/72",
      "nonlocal_coefficient_of_L": "I/24",
      "local_polynomial": "-I/9",
      "finite_if_d_set_to_4_first": "I*(L - 2)/24",
```

```
  "local_shift_from_d_dependence": "-I/36"
},
"g4_mn p_r p_s (p.p)^1": {
  "evanescent": false,
  "A_exact_d_times_MB": "-3*I/(8*d - 8)",
  "UV_pole": "-I/8",
  "MSbar_finite": "I*(3*L - 8)/24",
  "nonlocal_coefficient_of_L": "I/8",
  "local_polynomial": "-I/3",
  "finite_if_d_set_to_4_first": "I*(L - 2)/8",
  "local_shift_from_d_dependence": "-I/12"
},
"g4_mr g4_ns (p.p)^2": {
  "evanescent": false,
  "A_exact_d_times_MB": "I*(d - 3)/(8*(d - 1))",
  "UV_pole": "I/24",
  "MSbar_finite": "I*(2 - 3*L)/72",
  "nonlocal_coefficient_of_L": "-I/24",
  "local_polynomial": "I/36",
  "finite_if_d_set_to_4_first": "I*(2 - L)/24",
  "local_shift_from_d_dependence": "-I/18"
},
"g4_mr gt_ns (p.p)^2": {
  "evanescent": true,
  "A_exact_d_times_MB": "I/(8*(d - 1))",
  "UV_pole": "I/24",
  "MSbar_finite": "I*(8 - 3*L)/72",
  "nonlocal_coefficient_of_L": "-I/24",
  "local_polynomial": "I/9",
  "finite_if_d_set_to_4_first": "I*(2 - L)/24",
  "local_shift_from_d_dependence": "I/36"
},
"g4_mr p_n p_s (p.p)^1": {
  "evanescent": false,
  "A_exact_d_times_MB": "I*(7 - 2*d)/(16*(d - 1))",
  "UV_pole": "-I/48",
  "MSbar_finite": "I*(3*L + 4)/144",
  "nonlocal_coefficient_of_L": "I/48",
  "local_polynomial": "I/36",
  "finite_if_d_set_to_4_first": "I*(L - 2)/48",
  "local_shift_from_d_dependence": "5*I/72"
},
"g4_rs gt_mn (p.p)^2": {
  "evanescent": true,
  "A_exact_d_times_MB": "-I/(8*d - 8)",
  "UV_pole": "-I/24",
  "MSbar_finite": "I*(3*L - 8)/72",
  "nonlocal_coefficient_of_L": "I/24",
  "local_polynomial": "-I/9",
  "finite_if_d_set_to_4_first": "I*(L - 2)/24",
  "local_shift_from_d_dependence": "-I/36"
},
"g4_rs p_m p_n (p.p)^1": {
  "evanescent": false,
  "A_exact_d_times_MB": "-3*I/(8*d - 8)",
  "UV_pole": "-I/8",
  "MSbar_finite": "I*(3*L - 8)/24",
  "nonlocal_coefficient_of_L": "I/8",
  "local_polynomial": "-I/3",
  "finite_if_d_set_to_4_first": "I*(L - 2)/8",
  "local_shift_from_d_dependence": "-I/12"
},
"gt_mn p_r p_s (p.p)^1": {
  "evanescent": true,
  "A_exact_d_times_MB": "I/(4*(d - 1))",
  "UV_pole": "I/12",
  "MSbar_finite": "I*(8 - 3*L)/36",
  "nonlocal_coefficient_of_L": "-I/12",
  "local_polynomial": "2*I/9",
  "finite_if_d_set_to_4_first": "I*(2 - L)/12",
  "local_shift_from_d_dependence": "I/18"
},
"gt_mr p_n p_s (p.p)^1": {
  "evanescent": true,
  "A_exact_d_times_MB": "-3*I/(16*d - 16)",
  "UV_pole": "-I/16",
  "MSbar_finite": "I*(3*L - 8)/48",
  "nonlocal_coefficient_of_L": "I/16",
  "local_polynomial": "-I/6",
  "finite_if_d_set_to_4_first": "I*(L - 2)/16",
  "local_shift_from_d_dependence": "-I/24"
```

```
      },
      "gt_rs p_m p_n (p.p)^1": {
        "evanescent": true,
        "A_exact_d_times_MB": "I/(4*(d - 1))",
        "UV_pole": "I/12",
        "MSbar_finite": "I*(8 - 3*L)/36",
        "nonlocal_coefficient_of_L": "-I/12",
        "local_polynomial": "2*I/9",
        "finite_if_d_set_to_4_first": "I*(2 - L)/12",
        "local_shift_from_d_dependence": "I/18"
      },
      "p_m p_n p_r p_s": {
        "evanescent": false,
        "A_exact_d_times_MB": "I*(d - 2)/(4*(d - 1))",
        "UV_pole": "I/6",
        "MSbar_finite": "I*(5 - 3*L)/18",
        "nonlocal_coefficient_of_L": "-I/6",
        "local_polynomial": "5*I/18",
        "finite_if_d_set_to_4_first": "I*(2 - L)/6",
        "local_shift_from_d_dependence": "-I/18"
      }
    },
    "status": "regulated",
    "sector_status": {
      "4D": "renormalized (pole in the p14r h.h span)",
      "evanescent": "regulated: evanescent poles NONZERO, retained (R6); no evanescent h.h counterterm is de
+> clared"
    },
    "UV_pole_in_p14r_hh_basis": ["7*I/96", "I/24", "-I/8", "-I/24", "I/12"],
    "evanescent_poles": {
      "gt_mn p_r p_s (p.p)^1": "I/12",
      "gt_mr p_n p_s (p.p)^1": "-I/16",
      "gt_rs p_m p_n (p.p)^1": "I/12",
      "g4_mn gt_rs (p.p)^2": "-I/24",
      "g4_mr gt_ns (p.p)^2": "I/24",
      "g4_rs gt_mn (p.p)^2": "-I/24"
    },
    "evanescent_pole_nonzero": true,
    "longitudinal_projection_p_m_G": {
      "4_nr p_s": "I*p.p**2/(16*(d - 1))",
      "4_ns p_r": "I*p.p**2/(16*(d - 1))",
      "4_rs p_n": "I*p.p**2*(d - 3)/(16*(d - 1))",
      "p_n p_r p_s": "0",
      "t_nr p_s": "-I*p.p**2/(16*(d - 1))",
      "t_ns p_r": "-I*p.p**2/(16*(d - 1))",
      "t_rs p_n": "I*p.p**2/(8*(d - 1))"
    },
    "regeneration": {
      "components": 100,
      "mismatches": 0,
      "agree": true
    },
    "component_route_Gt_quadratic_components": 10,
    "controls": {
      "export_sha_pinned": true,
      "covariant_form_exact": true,
      "covariant_form_unique": true,
      "orbit_consistent": true,
      "route_FP_equals_route_PV_exact_in_d": true,
      "regenerated_integrand_equals_export": true,
      "exchange_symmetric_mn_rs": true,
      "nonlocal_equals_minus_pole": true,
      "pole_4D_in_p14r_hh_span": true,
      "R5_d_retained": true
    },
    "evaluated": true
  }
},
"D01_plus_D03_4D_pole_in_p14r_hh_basis": ["-23*I/240", "-61*I/240", "23*I/120", "61*I/120", "-7*I/20"],
"hh_basis": [
  {
    "density": "(h00*h00_d0000 - 2*h00*h00_d0011 - 2*h00*h00_d0022 - 2*h00*h00_d0033 + h00*h00_d1111 + 2*h00
+> *h00_d1122 + 2*h00*h00_d1133 + h00*h00_d2222 + 2*h00*h00_d2233 + h00*h00_d3333 - h00*h11_d0000 + 2*h00*h1
+> 1_d",
    "coords": {
      "g4_mn g4_rs (p.p)^2": "2"
    }
  },
  {
    "density": "(h00*h00_d0000 - 2*h00*h00_d0011 - 2*h00*h00_d0022 - 2*h00*h00_d0033 + h00*h00_d1111 + 2*h00
+> *h00_d1122 + 2*h00*h00_d1133 + h00*h00_d2222 + 2*h00*h00_d2233 + h00*h00_d3333 - 2*h01*h01_d0000 + 4*h01*
```

```
+> h01",
    "coords": {
      "g4_mr g4_ns (p.p)^2": "1"
    }
  },
  {
    "density": "(h00*h00_d0000 - h00*h00_d0011 - h00*h00_d0022 - h00*h00_d0033 - h00*h11_d0000 + h00*h11_d00
+> 11 + h00*h11_d0022 + h00*h11_d0033 - h00*h22_d0000 + h00*h22_d0011 + h00*h22_d0022 + h00*h22_d0033 - h00*
+> h33",
    "coords": {
      "g4_mn p_r p_s (p.p)^1": "1",
      "g4_rs p_m p_n (p.p)^1": "1"
    }
  },
  {
    "density": "(h00*h00_d0000 - h00*h00_d0011 - h00*h00_d0022 - h00*h00_d0033 - h00*h01_d0001 + h00*h01_d01
+> 11 + h00*h01_d0122 + h00*h01_d0133 - h00*h02_d0002 + h00*h02_d0112 + h00*h02_d0222 + h00*h02_d0233 - h00*
+> h03",
    "coords": {
      "g4_mr p_n p_s (p.p)^1": "1/2"
    }
  },
  {
    "density": "(h00*h00_d0000 - 2*h00*h01_d0001 - 2*h00*h02_d0002 - 2*h00*h03_d0003 + h00*h11_d0011 + 2*h00
+> *h12_d0012 + 2*h00*h13_d0013 + h00*h22_d0022 + 2*h00*h23_d0023 + h00*h33_d0033 - 2*h00_d0001*h01 - 2*h00_
+> d00",
    "coords": {
      "p_m p_n p_r p_s": "2"
    }
  }
],
"D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)",
"controls": {
  "D13_reproduced_by_covariant_route": true,
  "hh_basis_fits_exact": true,
  "R5_structural_safeguard_active": true,
  "mutations_all_rejected": true,
  "D01_evaluated": true,
  "D03_evaluated": true
},
"mutations": {
  "wrong_ghost_loop_sign_D03": {
    "changes_result": true,
    "rejected": true,
    "reason": "regenerated (loop sign +1) != frozen export: 100 components (the result flips sign: A -> -A)"
+> ,
    "accepted_as_rejection": true
  },
  "wrong_symmetry_factor_D01": {
    "changes_result": true,
    "rejected": true,
    "reason": "S_f = 1 regeneration != export: 100 components",
    "accepted_as_rejection": true
  },
  "premature_d4": {
    "rejected": true,
    "changes_result": true,
    "reason": "PrematureD4Substitution at the substitution site: d -> 4 via subs outside substitute_d (R5)",
    "example": {
      "structure": "g4_mn g4_rs (p.p)^2",
      "local_if_d4_first": "27*I*L/80 - 27*I/40",
      "certified_finite": "27*I*L/80 - 491*I/600"
    },
    "accepted_as_rejection": true
  },
  "drop_evanescent_gt": {
    "changes_result": true,
    "rejected": true,
    "reason": "Feynman-parameter route with gt dropped != PV route (ghat kept): R6 retention check fails",
    "accepted_as_rejection": true
  },
  "wrong_external_index_position": {
    "changes_result": true,
    "rejected": true,
    "reason": "covariant form with lower external indices: exact = False (['inconsistent'])",
    "accepted_as_rejection": true
  },
  "wrong_offdiagonal_weight": {
    "changes_result": true,
    "rejected": true,
    "reason": "off-diagonal weight 1: exact = False",
```

```
    "accepted_as_rejection": true
  },
  "component_route_linear_realization": {
    "changes_result": true,
    "rejected": true,
    "reason": "Gt-quadratic component sums are realization-dependent under Gt = (4-d) M: ['163215913117273*(
+> d - 4)**2/154303081175887', '5518289693389*(d - 4)**2/353769196225'] vs 4 - d",
    "accepted_as_rejection": true
  }
},
"batchC_evaluated": {
  "execution": true,
  "claim": "D01 and D03 evaluated in closed form under the declared prescription (covariant route): exact co
+> variant integrands (verified on all 100 components, regenerated from S_Psi), two independent loop-integra
+> tion routes agree exactly in d, 4D UV poles local and in the p14r h.h span, MS-bar finite parts = nonloca
+> l log + local quartic polynomial, evanescent poles recorded (R6), mutations rejected",
  "scope": "two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momenta in QdS; e
+> vanescent structures retained (their poles are NONZERO and not absorbed: no evanescent h.h counterterm is
+>  declared); D11 NOT evaluated, so the hh slot is incomplete; NOT quantum_inputs_evaluated"
},
"scaleless_rule_applications": [
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D01[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[0,4] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,4] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,-3) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,-3) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,-3) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D01[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,1] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D03[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
```

```
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
```

```
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO3[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
```

```
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
```

```
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
```

```
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
```

```
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
```

```
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
```

```
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D01[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
```

```
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
```

```
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
```

```
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
```

```
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
```

```
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO3[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
```

```
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D01[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
```

```
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[0,4] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
```

```
},
{
  "context": "J_DO1[0,4] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_DO1[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
```

```
      "nu": -1,
      "omega": 6,
      "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
    },
    {
      "context": "J_D01[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
      "k": 0,
      "nu": -1,
      "omega": 6,
      "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
    },
    {
      "context": "J_D01[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
      "k": 0,
      "nu": -1,
      "omega": 6,
      "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
    },
    {
      "context": "J_D01[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
      "k": 0,
      "nu": -1,
      "omega": 6,
      "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
    },
    {
      "context": "J_D01[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
      "k": 0,
      "nu": -1,
      "omega": 6,
      "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
    },
    {
      "context": "J_D01[0,4] [I(0,-2) -> polynomial/(l^2)^-2]",
      "k": 0,
      "nu": -2,
      "omega": 8,
      "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
    },
    {
      "context": "J_D01[0,4] [I(0,-2) -> polynomial/(l^2)^-2]",
      "k": 0,
      "nu": -2,
      "omega": 8,
      "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
    },
    {
      "context": "J_D01[0,4] [I(0,-2) -> polynomial/(l^2)^-2]",
      "k": 0,
      "nu": -2,
      "omega": 8,
      "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
    }
  ],
  "loop_evaluations_performed": true,
  "seconds": 182.8
}
```


58 p15 layer release record (Batch A, D13; p15 gate)

[listing 25/133] certificates/P15_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p15",
  "contract_revision": "p14r",
  "date": "2026-10-08",
  "deliverable": "P.5 calculation, first batch (Batch A: D02, D12, D14a-c; Batch B: D13)",
  "execution_pass": true,
  "claim_status": "P.5_calculation: Batch A and D13 evaluated under the p14r prescription; P.5_entry: earned (
  +> p14r contract); classical_readiness (frozen P.4): earned",
  "scope": "first P.5 calculation: the Batch A graphs (D02, D12, D14a-c) and the Batch B graph D13 evaluated u
  +> nder the frozen p14r prescription (DRED R1-R6, Feynman +i0, MS-bar) with the explicit scaleless rule S0;
  +> D01, D03, D11 NOT evaluated; no slot of the identity is complete; NOT quantum inputs, NOT the projected d
  +> efect/identity, NOT bulk restoration, NOT the insertion",
  "release_claim_accepted": true,
  "release_claim_accepted_definition": "p14r contract accepted (P14_release.json) AND contract byte-identity e
  +> xcept the disclosed dred.py correction (regression toys pass) AND library certified AND batchA_evaluated
```

```
+> AND D13_evaluated AND p15 hard stops",
"publication": "accepted",
"flags": {
  "library_certified": true,
  "batchA_evaluated": {
    "execution": true,
    "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-ba
+> r + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no U
+> V pole, finite part 0, no p14r counterterm required",
    "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles
+>  carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4)
+> and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complet
+> e; NOT quantum_inputs_evaluated"
  },
  "D13_evaluated": {
    "execution": true,
    "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of th
+> e 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is rec
+> orded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two ind
+> ependent routes agree exactly in d, ghost equation verified, mutations rejected",
    "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); exte
+> rnal momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c co
+> unterterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluate
+> d; the slot's identity role is not evaluated (Batch C, D11 M2)"
  }
},
"batchA_evaluated": true,
"D13_evaluated": true,
"library_certified": true,
"graphs": {
  "D02": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)"
  },
  "D12": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 2,
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)"
  },
  "D14a": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14b": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D14c": {
    "status": "vanishing_under_declared_prescription",
    "UV_pole": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar counterterm
+>  0",
    "finite_part": "0 (all components)",
    "routes_agree": true,
    "kappa_order": 1,
    "slot": "order-kappa one-point slot"
  },
  "D13": {
    "status": "regulated",
    "sector_status": {
      "4D_structures_T1_T3": "renormalized",
      "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its countert
+> erm is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent cou
```

```
+> nterterm: open contract item, not absorbed)"
    },
    "UV_pole": {
      "T1": "0",
      "T2": "3/16",
      "T3": "-1/4",
      "E1": "0",
      "E2": "1/16"
    },
    "MSbar_finite": {
      "T1": "0",
      "T2": "1/3 - 3*L/16",
      "T3": "L/4 - 5/12",
      "E1": "0",
      "E2": "1/6 - L/16"
    },
    "UV_pole_in_p14r_cK_basis": ["0", "3/8", "-1/4"],
    "evanescent_pole_nonzero": true
  },
  "D01": {
    "status": "unevaluated"
  },
  "D03": {
    "status": "unevaluated"
  },
  "D11": {
    "status": "unevaluated"
  }
},
"D12_status": "vanishing_under_declared_prescription",
"D12_status_derivation": "certificates/P15_batchA.json graphs.D12 (route 1 frozen-dred lift + S0, route 2 sy
+> mmetric integration, IR-mass and cutoff analysis)",
"D12_previous_status": "Missing (p14r)",
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"D13_summary": {
  "status": "regulated",
  "sector_status": {
    "4D_structures_T1_T3": "renormalized",
    "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterter
+> m is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent count
+> erterm: open contract item, not absorbed)"
  },
  "convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Gha
+> t = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower
+> (K' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
  "UV_pole_in_p14r_cK_basis": {
    "coordinates": ["0", "3/8", "-1/4"],
    "basis": [
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-
+> I)*[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0
+> , ",
        "T_coordinates": ["1", "0", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1,
+> 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I
+> /2)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[
+> ('",
        "T_coordinates": ["0", "1/2", "0"]
      },
      {
        "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1
+> ))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[
+> ('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()
+> ),",
        "T_coordinates": ["0", "0", "1"]
      }
    ]
  },
  "local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
  "MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K'
+> densities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_m
+> n p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
  "evanescent_pole_nonzero": true,
```

```
  "evanescent_counterterm_declared_in_p14r": false,
  "local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(
+> 16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu
+> ^2) along the UV-pole direction",
  "renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no l
+> ower-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O)
+> not modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i
+> (4) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
  "controls": {
    "export_sha_pinned": true,
    "route1_equals_route2_exact_in_d": true,
    "covariant_fit_exact": true,
    "fits_agree": true,
    "pole_local_in_p14r_cK_span": true,
    "ghost_equation_2iD_G_equals_ghost_self_energy": true,
    "ghost_self_energy_pole_in_p14r_c_cb_span": true,
    "nonlocal_equals_minus_pole": true,
    "R5_d_retained": true,
    "R5_structural_safeguard_active": true,
    "mutations_all_rejected": true
  }
},
"contract_identity": {
  "p14r_contract_files": 12,
  "changed": ["code/p5_entry/dred.py"],
  "added": [],
  "disclosed_corrections": {
    "code/p5_entry/dred.py": {
      "kind": "p14r contract revision (disclosed), applied inside p15",
      "finding": "F1_premature_d4_guard_heuristic, reproducibility/independent_rebuild_report_p14r.json",
      "why": "the p14r PrematureD4Substitution guard was a heuristic on the shape of the expression before t
+> he Laurent expansion: FALSE POSITIVE on eta^{rs} int l_r l_s/(l^2-m^2) = J (correct, d-free) and FALSE NE
+> GATIVE under premature_d4 on the same integral (eta_trace_Gt reintroduces d via tr gtilde = 4-d: d J/4, f
+> inite error -m^4/2); the R5 docstring claim 'is detected' did not hold in general",
      "change": "structural, provenance-based safeguard: d is an OpaqueDimension (subs/xreplace raise Premat
+> ureD4Substitution outside dred.substitute_d, which records every authorised use); numeric traces of ghat/
+> gtilde refused (NumericTraceForbidden) unless display_only; the heuristic is removed; R5 docstring correc
+> ted; reviewer toys added as regression checks (reviewer_toys)",
      "rules_R1_R6_changed": false,
      "released_p14r_values_affected": false,
      "why_unaffected": "p14r applied Pipeline.laurent only to toy placeholders; no loop integral was evalua
+> ted"
    }
  },
  "byte_identical_except_disclosed": true,
  "regression_toys_pass": true,
  "dred_certificate_ok": true,
  "regression_toys": {
    "eta_trace_correct": {
      "finite": "a1*m**4",
      "raised": false,
      "ok": true
    },
    "eta_trace_premature_d4": {
      "raised": true,
      "reason": "d -> 4 outside substitute_d (R5: d is opaque before the Laurent expansion)",
      "finite_if_forced": "m**4*(a1 - 1/2)",
      "ok": true
    },
    "A0_mixing": {
      "correct_finite": "a1*m**4",
      "mixed_refused": true,
      "finite_if_forced": "m**4*(a1 + 1/2)",
      "shift": "m**4/2",
      "ok": true
    },
    "B0_mixing": {
      "correct_finite": "0",
      "mixed_refused": true,
      "finite_if_forced": "-p2/6",
      "shift": "-p2/6",
      "ok": true
    },
    "pass": true
  },
  "test_revisions_disclosed": {
    "tests/test_p14.py": "test_dred_trace_ghat_eq_4_rejected: a numeric trace is now refused (NumericTraceFo
+> rbidden); the tr ghat = 4 display uses Rules(trace_hat=4, display_only=True); infrastructure assertions r
+> ead the p14r contract record from certificates/P14_release.json (the top-level RELEASE_STATUS.json is the
+>  p15 status)"
  },
```

```
  "P3_P4_code": "frozen p13 classical code byte-identical (freeze F2, checked by the p14r contract gate)"
},
"contract_correction_disclosed": true,
"contract_layer": {
  "certificate": "certificates/P14_release.json",
  "accepted": true,
  "revision": "p14r",
  "P5_entry_earned": true,
  "P5_entry_flags": {
    "quantum_action_defined": true,
    "quantum_prescription_defined": true,
    "D11_matching_defined": true,
    "counterterm_domain_defined": true,
    "amplitude_rules_verified": true
  },
  "P3_P4_frozen_as_P5_inputs": true,
  "contract_hard_stops": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "D11_matching_verified": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "P5_entry_opened": true,
    "P5_calculation_opened": false,
    "P5_identity_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "note": "the contract-layer statements (e.g. D12_status Missing, P5_calculation_opened false) describe the
+>  p14r CONTRACT and are superseded for p15 by this status; the contract record's rebuild flags refer to th
+> e current tree through the p14r report paths and are superseded by the p15 flags below"
},
"P5_entry_earned": true,
"P5_entry_flags": {
  "quantum_action_defined": true,
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "counterterm_domain_defined": true,
  "amplitude_rules_verified": true
},
"P3_P4_frozen_as_P5_inputs": true,
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
    "C1_inverse_10comp": true,
    "C2_det_tracelog_10comp": true,
    "C3_boundary_identity_generic_and_conformal": true,
    "C4_export_equals_independent_evaluator": true,
    "C5_cubic_regression_Euler": true,
    "C6_quadratic_regression_Euler": true,
    "C7_grav_noether_k0_k1_k2_on_exports": true,
    "quartic_mutations_all_rejected": true,
    "master_grav_noether_deterministic_k0_k1_k2": true,
    "master_engine_parts_exact": true,
    "graviton_hessian_generic_symbolic_k": true,
    "canonicity_all_23_antifield_components": true,
    "ch_enumeration_lemma_mechanical": true,
    "residual_record_invariants_all_certificates": true,
    "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
    "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
    "cross_certificate_quartic_vs_master_agree": true,
    "stored_C7_flag_equals_recomputation": true
  },
```

```
"failed_prerequisites": [],
"hash_consistency": {
  "consistent": true,
  "ell": {
    "ell2": {
      "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
    },
    "ell3": {
      "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
    },
    "ell4": {
      "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
    "quartic_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
      }
    },
    "master_parameters": {
      "a_ghost": "1/2",
      "transport_term": 1,
      "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
      "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
      "ell_sha256": {
        "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
        "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
```

```
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    }
  },
  "hard_stops_p13_hold": true,
  "hard_stops_p13": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "symmetry_restored": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "Route_B_discharged": false,
    "P5_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
},
"D11_matching_verified": false,
"hard_stops": {
  "quantum_inputs_evaluated": "not_earned",
  "projected_defect_computed": "not_earned",
  "projected_identity_verified": "not_earned",
  "projected_restoration_verified": "not_earned",
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "D11_matching_verified": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "u_R_c_delta_added": false,
  "D01_evaluated": false,
  "D03_evaluated": false,
  "D11_evaluated": false
},
"hard_stops_hold": true,
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "opened (p15: Batch A D02/D12/D14a-c and Batch B D13 evaluated under the p14r prescri
+> ption; Batch C D01/D03 and the D11 matching M2 NOT evaluated)",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not ear
+> ned)"
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p14r": true,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p15) sourc
+> e tree only; independent_rebuild_passed_p14r refers to the p14r tree a64c0ae365980fa6b077f991df4316347527
+> 613075e87f17fecfe422db6e3894 / zip 19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3 (same
+> -box caveat) and is not inherited by p15; independent_rebuild_passed_p13 carries the P.3/P.4 freeze",
"p14r_independent_rebuild": {
  "report": "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
  "present": true,
  "report_sha256": "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b",
  "report_sha256_pinned": true,
  "pass": true,
```

```
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
  "bound_to_source_zip_sha256": "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3",
  "tree_matches_p14r": true,
  "zip_matches_p14r": true,
  "mismatches": 0,
  "findings": ["F1_premature_d4_guard_heuristic", "F2_trace_ghat_4_mutation_scope"],
  "caveat": "SAME-BOX caveat: this independent rebuild ran on the same machine as the author build (shared k
+> ernel, TeX Live, poppler, platform string identical) in a fresh directory and fresh venv from the source-
+> only zip; it was NOT run on a second machine.",
  "live_report_identical_to_archived": true,
  "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent flag",
  "archived_p14r_records_pinned": {
    "archival/p14r/reference_values_p14r.json": true,
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json": true
  }
},
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p15.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "vendored",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p15.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": false,
    "inherited_mode": null,
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "pdfinfo": "pdfinfo version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
      "venv": "/workspace/indep_rebuild_p15/venv",
      "pip_installed": "sympy==1.14.0 mpmath==1.3.0 pytest==9.1.1 (exact versions via pip)"
    }
  },
```

```
    "p14r_archive": "archival/p14r",
    "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
  },
  "XVI_P_registered": false,
  "XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
  "XVI_M_pin_ok": true,
  "symmetry_restored": false,
  "bulk_symmetry_restored": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
    "P14_release.json": "23b5b7bd32da5e33b794827ae40dc2de0a73a547c9ec26d7a69b08b87e727b63",
    "P14_dred_rules.json": "0fff45df2dd1e9cec6e4808c9ac7d5da5fa526c5e9a0971498ea9ad57442b24e",
    "P15_integral_library.json": "8d9524a19ec6aaf287fb6dcfc9735a54bd56ae0111eb5902e263e6551a4f59dd",
    "P15_batchA.json": "123d82b9664e802caabc33f10fcef97694f16c631ff687e121340e3c09d95bae",
    "P15_D13.json": "dbf71b3165c320ede1840b35573499d39c9d9497e6b9cff040e4b9f0bcc897d2"
  }
}
```


59 Milestone ledger (p15)

[listing 26/133] ledgers/P15_milestones.json


```
{
  "landmark": "XVI-P",
  "revision": "p15",
  "ledger": "milestones (authoritative definitions + status)",
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "opened (p15: Batch A D02/D12/D14a-c and Batch B D13 evaluated under the p14r prescript
  +> ion; Batch C D01/D03 and the D11 matching M2 NOT evaluated)",
    "P.5_identity": "open"
  },
  "P5_entry_earned": true,
  "P3_P4_frozen_as_P5_inputs": true,
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not earne
  +> d)",
  "author_rebuild_passed": false,
  "independent_rebuild_passed": false,
  "independent_rebuild_passed_p14r": true,
  "hard_stops": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "D11_matching_verified": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "u_R_c_delta_added": false,
    "D01_evaluated": false,
    "D03_evaluated": false,
    "D11_evaluated": false
  },
  "flags": {
    "batchA_evaluated": true,
    "D13_evaluated": true,
    "library_certified": true
  }
```

}

60 P.5 calculation: integral library certificate (p15)

[listing 27/133] certificates/P15_integral_library.json


```
{
  "landmark": "XVI-P",
  "revision": "p15",
  "deliverable": "P.5 calculation: one-loop integral library",
  "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 (DRED R1-R6, Feynman +i0, MS-bar); S0 stated explicitl
  +> y",
  "normalisation": "I = N_eps * Ihat, N_eps = i S_eps/(16 pi^2), S_eps = (4 pi)^eps e^{-gamma_E eps}, mu^{2 ep
  +> s}; MS-bar = drop the poles of Ihat",
  "library_certified": true,
  "B0_massless_laurent": {
    "pole": "1",
    "finite": "2 - L",
    "order_eps": "L**2/2 - 2*L - pi**2/12 + 4",
    "ok": true
  },
  "A0_laurent": {
    "pole": "m2",
    "finite": "m2*(1 - Lm)",
    "ok": true
  },
  "B0_exact_d_numeric": {
    "d_values": [2.6, 3.3, 3.7],
    "numeric_vs_formula": [
      [0.29051439745794433, 0.29051439745794433],
      [0.08354836485354544, 0.08354836485354544],
      [0.07842730369003323, 0.07842730373608314]
    ],
    "ok": true
  },
  "T_exact_d_numeric": {
    "cases": ["d=0.7,k=0", "d=1.5,k=0", "d=-0.6,k=1"],
    "numeric_vs_formula": [
      [0.5710341301506534, 0.5710341301506534],
      [0.5432174056063133, 0.543217405606654],
      [-1.9177113399413097, -1.9177113399413115]
    ],
    "ok": true
  },
  "tensor_reduction_vs_feynman_parameters": {
    "ranks": [0, 1, 2, 3],
    "checks": 140,
    "mismatches": [],
    "n_mismatches": 0,
    "ok": true,
    "note": "route 1 uses S0 for every cancelled propagator; route 2 uses none (Delta != 0): agreement certifi
  +> es S0 on this family"
  },
  "J_table": {
    "J_LIB[0,0]": "MB",
    "J_LIB[0,1]": "MB*p.p/2",
    "J_LIB[0,2]": "MB*p.p**2/4",
    "J_LIB[0,3]": "MB*p.p**3/8",
    "J_LIB[0,4]": "MB*p.p**4/16"
  },
  "scaleless_rule": {
    "statement": "S0: int d^d l (l^2)^k/(l^2)^nu := 0 when the integrand carries no scale",
    "lambda_split": {
      "omega=4": {
        "omega": 4,
        "s_at_d4": 4,
        "sum_zero": true,
        "IR_part": "Lambda**d/d",
        "UV_part": "-Lambda**d/d",
        "IR_pole_at_d4": "0 (no pole)",
        "UV_pole_at_d4": "0 (no pole)"
      },
      "omega=2": {
        "omega": 2,
        "s_at_d4": 2,
        "sum_zero": true,
        "IR_part": "Lambda**(d - 2)/(d - 2)",
        "UV_part": "-Lambda**(d - 2)/(d - 2)",
```

```
        "IR_pole_at_d4": "0 (no pole)",
        "UV_pole_at_d4": "0 (no pole)"
      },
      "omega=0": {
        "omega": 0,
        "s_at_d4": 0,
        "sum_zero": true,
        "IR_part": "Lambda**(d - 4)/(d - 4)",
        "UV_part": "-Lambda**(d - 4)/(d - 4)",
        "IR_pole_at_d4": "-1/2",
        "UV_pole_at_d4": "1/2"
      },
      "omega=-2": {
        "omega": -2,
        "s_at_d4": -2,
        "sum_zero": true,
        "IR_part": "Lambda**(d - 6)/(d - 6)",
        "UV_part": "-Lambda**(d - 6)/(d - 6)",
        "IR_pole_at_d4": "0 (no pole)",
        "UV_pole_at_d4": "0 (no pole)"
      }
    },
    "misuse_guard": {
      "rejected": true,
      "reason": "S0 applied to an integral with a scale (A0 with m != 0 (mutation: S0 on a massive tadpole))"
    }
  },
  "mass_regulator_limit": {
    "k=0": {
      "exact": "-m2*exp(EulerGamma*(2 - d/2))*exp(Lm*(d/2 - 2))*gamma(1 - d/2)",
      "m2_power": "d/2 - 1",
      "vanishes_as_m_to_0_for": "Re d > 2"
    },
    "k=1": {
      "exact": "m2**2*exp(EulerGamma*(2 - d/2))*exp(Lm*(d/2 - 2))*gamma(-d/2)*gamma(d/2 + 1)/gamma(d/2)",
      "m2_power": "d/2",
      "vanishes_as_m_to_0_for": "Re d > 0"
    },
    "k=2": {
      "exact": "-m2**3*exp(EulerGamma*(2 - d/2))*exp(Lm*(d/2 - 2))*gamma(-d/2 - 1)*gamma(d/2 + 2)/gamma(d/2)",
      "m2_power": "d/2 + 1",
      "vanishes_as_m_to_0_for": "Re d > -2"
    }
  },
  "i0": {
    "L": "log(-p^2/mu^2 - i0) = log|p^2/mu^2| - i pi theta(p^2)  (Feynman +i0)",
    "Lm": "log(m^2/mu^2) (m^2 - i0)"
  },
  "ok": true,
  "seconds": 38.6,
  "S0_ledger_size": 100
}
```


61 P.5 calculation: Batch A certificate (D02, D12, D14a-c; p15)

[listing 28/133] certificates/P15_batchA.json


```
{
  "landmark": "XVI-P",
  "revision": "p15",
  "deliverable": "P.5 calculation Batch A (tadpoles)",
  "contract_revision": "p14r",
  "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 + S0 (certificates/P15_integral_library.json)",
  "graphs": {
    "D02": {
      "graph": "D02",
      "slot": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)",
      "kappa_order": 2,
      "export": {
        "file": "exports/P14_integrands/D02.json",
        "sha256_recomputed": "e37cb4152690018ec8ce2ac1a0539a29864528e65e06a9efe09441b14fb6a063",
        "sha256_stored": "e37cb4152690018ec8ce2ac1a0539a29864528e65e06a9efe09441b14fb6a063",
        "sha256_p14r_certificate": "e37cb4152690018ec8ce2ac1a0539a29864528e65e06a9efe09441b14fb6a063",
        "pinned": true
      },
      "prefactor": "-kappa**2/2",
      "denominators": ["l^2+i0"],
      "massless_declared": true,
```

```
    "status": "vanishing_under_declared_prescription",
    "regulated_expression": {
      "form": "iGamma_1 = N_eps * (-kappa**2/2) * sum_comp [ sum_k c_k(d, p, Gt) J_D02[k,0] ],  J_D02[k,0] =
+>  int (l^2)^k / (l^2 + i0)",
      "components": 94,
      "J_members_k": [0, 1],
      "components_with_evanescent_Gt_part": 70,
      "examples": {
        "0,0": "-9*Gt_00*J_D02[1,0]/(2*d) + 5*Gt_11*J_D02[1,0]/(2*d) + 5*Gt_22*J_D02[1,0]/(2*d) + 5*Gt_33*J_
+> D02[1,0]/(2*d) + 12*J_D02[1,0]/d",
        "0,1": "2*Gt_01*J_D02[1,0]/d",
        "0,2": "2*Gt_02*J_D02[1,0]/d"
      }
    },
    "UV_subtraction": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar coun
+> terterm 0",
    "finite_part": "0 (all components)",
    "local_ambiguity": "none generated in the declared prescription; the kappa^2 hh slot's local freedom is
+> the p14r h.h counterterm directions (degree 4 in p) and is NOT touched by D02/D12 (no pole, no finite loc
+> al term)",
    "renormalization_conditions": "MS-bar at mu (P14 normalization condition 0) + N4 (no mu^4, mu^2 p^2 cont
+> acts); no counterterm is required",
    "scaleless_analysis": {
      "S0_applications": [
        {
          "context": "D02: int (l^2)^0/l^2",
          "k": 0,
          "nu": 1,
          "omega": 2,
          "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
        },
        {
          "context": "D02: int (l^2)^1/l^2",
          "k": 1,
          "nu": 1,
          "omega": 4,
          "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
        }
      ],
      "logarithmic_members": [],
      "UV_IR_reading": "no member is logarithmic (omega = 0): the DR zero is NOT a 1/eps_UV - 1/eps_IR cance
+> llation here; it discards pure power divergences (omega = 2, 4); the IR region converges at d = 4 and nei
+> ther the UV nor the IR part has a pole at d = 4"
    },
    "prescription_dependence": {
      "IR_regulator_mass": {
        "rule": "1/(l^2 + i0) -> 1/(l^2 - m^2 + i0) on the loop line (analysis only; m is not in the action)
+> ",
        "pole_powers_of_m2": [1, 2],
        "pole_examples": {
          "0,0": "-9*Gt_00*m2**2/8 + 5*Gt_11*m2**2/8 + 5*Gt_22*m2**2/8 + 5*Gt_33*m2**2/8 + 3*m2**2",
          "0,1": "Gt_01*m2**2/2",
          "0,2": "Gt_02*m2**2/2"
        },
        "finite_examples": {
          "0,0": "9*Gt_00*Lm*m2**2/8 - 27*Gt_00*m2**2/16 - 5*Gt_11*Lm*m2**2/8 + 15*Gt_11*m2**2/16 - 5*Gt_22*
+> Lm*m2**2/8 + 15*Gt_22*m2**2/16 - 5*Gt_33*Lm*m2**2/8 + 15*Gt_33*m2**2/16 - 3*Lm*m2**2 + 9*m2**2/2",
          "0,1": "-Gt_01*Lm*m2**2/2 + 3*Gt_01*m2**2/4",
          "0,2": "-Gt_02*Lm*m2**2/2 + 3*Gt_02*m2**2/4"
        },
        "limit_m_to_0": "0 (pole ~ m^{2j}, finite ~ m^{2j}(a + b log m^2), j >= 1)",
        "counterterm_needed_at_finite_m": "poles ~ kappa^2 m^4 hh and kappa^2 m^2 p^2 hh: order-0/order-2 co
+> ntacts, OUTSIDE the p14r h.h basis (excluded by N4); they vanish as m -> 0"
      },
      "analytic_regulator": "(l^2)^{-1-nu}: the integrand stays scaleless for every nu, the value stays 0; a
+> n analytic regulator alone does not separate UV from IR here",
      "cutoff_split": "each member splits into IR (+Lambda^omega/omega...) and UV (-Lambda^omega/omega...) p
+> ieces with no pole at d = 4; with a hard cutoff the would-be counterterms are kappa^2 Lambda^4 hh and kap
+> pa^2 Lambda^2 p^2 hh (order 0/2: excluded by N4, not p14r directions)",
      "p14r_counterterm_required": false
    },
    "verification_controls": {
      "export_sha_pinned_to_p14r_certificate": true,
      "independent_integrand_route_p14r": "W6 Hessian/Tr-log route (P14_amplitude_rules W6_exports)",
      "route1_dred_lift_equals_route2_symmetric_integration": true,
      "components_compared": 94,
      "all_components_vanish": true
    },
    "value_imported_from_XVI_0": false,
    "XVI_0_comparison": "n/a"
  },
```

```
  "D12": {
    "graph": "D12",
    "slot": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)",
    "kappa_order": 2,
    "export": {
      "file": "exports/P14_integrands/D12.json",
      "sha256_recomputed": "044080e17dc3acdfcbd2640995d831b69ab2d637712f27148fa60a6a2a99b6fa",
      "sha256_stored": "044080e17dc3acdfcbd2640995d831b69ab2d637712f27148fa60a6a2a99b6fa",
      "sha256_p14r_certificate": "044080e17dc3acdfcbd2640995d831b69ab2d637712f27148fa60a6a2a99b6fa",
      "pinned": true
    },
    "prefactor": "-kappa**2/2",
    "denominators": ["l^2+i0"],
    "massless_declared": true,
    "status": "vanishing_under_declared_prescription",
    "regulated_expression": {
      "form": "iGamma_1 = N_eps * (-kappa**2/2) * sum_comp [ sum_k c_k(d, p, Gt) J_D12[k,0] ],  J_D12[k,0] =
+>  int (l^2)^k / (l^2 + i0)",
      "components": 94,
      "J_members_k": [1],
      "components_with_evanescent_Gt_part": 94,
      "examples": {
        "0,0": "-3*Gt_00*J_D12[1,0]/(4*d) - Gt_11*J_D12[1,0]/(4*d) - Gt_22*J_D12[1,0]/(4*d) - Gt_33*J_D12[1,
+> 0]/(4*d)",
        "0,1": "Gt_01*J_D12[1,0]/d",
        "0,2": "Gt_02*J_D12[1,0]/d"
      }
    },
    "UV_subtraction": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar coun
+> terterm 0",
    "finite_part": "0 (all components)",
    "local_ambiguity": "none generated in the declared prescription; the kappa^2 hh slot's local freedom is
+> the p14r h.h counterterm directions (degree 4 in p) and is NOT touched by D02/D12 (no pole, no finite loc
+> al term)",
    "renormalization_conditions": "MS-bar at mu (P14 normalization condition 0) + N4 (no mu^4, mu^2 p^2 cont
+> acts); no counterterm is required",
    "scaleless_analysis": {
      "SO_applications": [
        {
          "context": "D12: int (l^2)^1/l^2",
          "k": 1,
          "nu": 1,
          "omega": 4,
          "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
        }
      ],
      "logarithmic_members": [],
      "UV_IR_reading": "no member is logarithmic (omega = 0): the DR zero is NOT a 1/eps_UV - 1/eps_IR cance
+> llation here; it discards pure power divergences (omega = 4); the IR region converges at d = 4 and neithe
+> r the UV nor the IR part has a pole at d = 4"
    },
    "prescription_dependence": {
      "IR_regulator_mass": {
        "rule": "1/(l^2 + i0) -> 1/(l^2 - m^2 + i0) on the loop line (analysis only; m is not in the action)
+> ",
        "pole_powers_of_m2": [2],
        "pole_examples": {
          "0,0": "-3*Gt_00*m2**2/16 - Gt_11*m2**2/16 - Gt_22*m2**2/16 - Gt_33*m2**2/16",
          "0,1": "Gt_01*m2**2/4",
          "0,2": "Gt_02*m2**2/4"
        },
        "finite_examples": {
          "0,0": "3*Gt_00*Lm*m2**2/16 - 9*Gt_00*m2**2/32 + Gt_11*Lm*m2**2/16 - 3*Gt_11*m2**2/32 + Gt_22*Lm*m
+> 2**2/16 - 3*Gt_22*m2**2/32 + Gt_33*Lm*m2**2/16 - 3*Gt_33*m2**2/32",
          "0,1": "-Gt_01*Lm*m2**2/4 + 3*Gt_01*m2**2/8",
          "0,2": "-Gt_02*Lm*m2**2/4 + 3*Gt_02*m2**2/8"
        },
        "limit_m_to_0": "0 (pole ~ m^{2j}, finite ~ m^{2j}(a + b log m^2), j >= 1)",
        "counterterm_needed_at_finite_m": "poles ~ kappa^2 m^4 hh and kappa^2 m^2 p^2 hh: order-0/order-2 co
+> ntacts, OUTSIDE the p14r h.h basis (excluded by N4); they vanish as m -> 0"
      },
      "analytic_regulator": "(l^2)^{-1-nu}: the integrand stays scaleless for every nu, the value stays 0; a
+> n analytic regulator alone does not separate UV from IR here",
      "cutoff_split": "each member splits into IR (+Lambda^omega/omega...) and UV (-Lambda^omega/omega...) p
+> ieces with no pole at d = 4; with a hard cutoff the would-be counterterms are kappa^2 Lambda^4 hh and kap
+> pa^2 Lambda^2 p^2 hh (order 0/2: excluded by N4, not p14r directions)",
      "p14r_counterterm_required": false
    },
    "verification_controls": {
      "export_sha_pinned_to_p14r_certificate": true,
      "independent_integrand_route_p14r": "W6 Hessian/Tr-log route (P14_amplitude_rules W6_exports)",
```

```
      "route1_dred_lift_equals_route2_symmetric_integration": true,
      "components_compared": 94,
      "all_components_vanish": true
    },
    "value_imported_from_XVI_O": false,
    "XVI_O_comparison": "XVI-O declares the D12/one-point value 0 by dimensional_regularization_m0 (coincide
+> nt prescription); p15 DERIVES the same vanishing under P's own declared prescription; not imported"
  },
  "D14a": {
    "graph": "D14a",
    "slot": "order-kappa one-point slot",
    "kappa_order": 1,
    "export": {
      "file": "exports/P14_integrands/D14a.json",
      "sha256_recomputed": "e141db64f2ed3ef1cb09709b03b8bbaf6c55d29604cb6873e09d05387473e606",
      "sha256_stored": "e141db64f2ed3ef1cb09709b03b8bbaf6c55d29604cb6873e09d05387473e606",
      "sha256_p14r_certificate": "e141db64f2ed3ef1cb09709b03b8bbaf6c55d29604cb6873e09d05387473e606",
      "pinned": true
    },
    "prefactor": "-kappa/2",
    "denominators": ["l^2+i0"],
    "massless_declared": true,
    "status": "vanishing_under_declared_prescription",
    "regulated_expression": {
      "form": "iGamma_1 = N_eps * (-kappa/2) * sum_comp [ sum_k c_k(d, p, Gt) J_D14a[k,0] ],  J_D14a[k,0] =
+> int (l^2)^k / (l^2 + i0)",
      "components": 10,
      "J_members_k": [1],
      "components_with_evanescent_Gt_part": 10,
      "examples": {
        "0": "3*Gt_00*J_D14a[1,0]/d - Gt_11*J_D14a[1,0]/d - Gt_22*J_D14a[1,0]/d - Gt_33*J_D14a[1,0]/d - 6*J_
+> D14a[1,0]/d",
        "1": "-4*Gt_01*J_D14a[1,0]/d",
        "2": "-4*Gt_02*J_D14a[1,0]/d"
      }
    },
    "UV_subtraction": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar coun
+> terterm 0",
    "finite_part": "0 (all components)",
    "local_ambiguity": "none: the p14r kappa^1 counterterm basis is empty (one-leg local functionals are tot
+> al derivatives or need a dimension-5 field)",
    "renormalization_conditions": "MS-bar at mu (P14 normalization condition 0) + N4 (no mu^4, mu^2 p^2 cont
+> acts); no counterterm is required",
    "scaleless_analysis": {
      "SO_applications": [
        {
          "context": "D14a: int (l^2)^1/l^2",
          "k": 1,
          "nu": 1,
          "omega": 4,
          "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
        }
      ],
      "logarithmic_members": [],
      "UV_IR_reading": "no member is logarithmic (omega = 0): the DR zero is NOT a 1/eps_UV - 1/eps_IR cance
+> llation here; it discards pure power divergences (omega = 4); the IR region converges at d = 4 and neithe
+> r the UV nor the IR part has a pole at d = 4"
    },
    "prescription_dependence": {
      "IR_regulator_mass": {
        "rule": "1/(l^2 + i0) -> 1/(l^2 - m^2 + i0) on the loop line (analysis only; m is not in the action)
+> ",
        "pole_powers_of_m2": [2],
        "pole_examples": {
          "0": "3*Gt_00*m2**2/4 - Gt_11*m2**2/4 - Gt_22*m2**2/4 - Gt_33*m2**2/4 - 3*m2**2/2",
          "1": "-Gt_01*m2**2",
          "2": "-Gt_02*m2**2"
        },
        "finite_examples": {
          "0": "-3*Gt_00*Lm*m2**2/4 + 9*Gt_00*m2**2/8 + Gt_11*Lm*m2**2/4 - 3*Gt_11*m2**2/8 + Gt_22*Lm*m2**2/
+> 4 - 3*Gt_22*m2**2/8 + Gt_33*Lm*m2**2/4 - 3*Gt_33*m2**2/8 + 3*Lm*m2**2/2 - 9*m2**2/4",
          "1": "Gt_01*Lm*m2**2 - 3*Gt_01*m2**2/2",
          "2": "Gt_02*Lm*m2**2 - 3*Gt_02*m2**2/2"
        },
        "limit_m_to_0": "0 (pole ~ m^{2j}, finite ~ m^{2j}(a + b log m^2), j >= 1)",
        "counterterm_needed_at_finite_m": "pole ~ kappa m^4 h: a one-point (cosmological-type) contact, OUTS
+> IDE the p14r basis (kappa^1 basis empty); it vanishes as m -> 0"
      },
      "analytic_regulator": "(l^2)^{-1-nu}: the integrand stays scaleless for every nu, the value stays 0; a
+> n analytic regulator alone does not separate UV from IR here",
      "cutoff_split": "each member splits into IR (+Lambda^omega/omega...) and UV (-Lambda^omega/omega...) p
```

```
+> ieces with no pole at d = 4; with a hard cutoff the would-be counterterms are kappa Lambda^4 h (one-point
+> ; no p14r kappa^1 direction)",
      "p14r_counterterm_required": false
    },
    "verification_controls": {
      "export_sha_pinned_to_p14r_certificate": true,
      "independent_integrand_route_p14r": "W5 independent sympy regeneration",
      "route1_dred_lift_equals_route2_symmetric_integration": true,
      "components_compared": 10,
      "all_components_vanish": true
    },
    "value_imported_from_XVI_O": false,
    "XVI_O_comparison": "n/a"
  },
  "D14b": {
    "graph": "D14b",
    "slot": "order-kappa one-point slot",
    "kappa_order": 1,
    "export": {
      "file": "exports/P14_integrands/D14b.json",
      "sha256_recomputed": "630ff664eec930d2c3eab60f5782dc9a3d4151c85304549f315cd94ad68d94de",
      "sha256_stored": "630ff664eec930d2c3eab60f5782dc9a3d4151c85304549f315cd94ad68d94de",
      "sha256_p14r_certificate": "630ff664eec930d2c3eab60f5782dc9a3d4151c85304549f315cd94ad68d94de",
      "pinned": true
    },
    "prefactor": "kappa",
    "denominators": ["l^2+i0"],
    "massless_declared": true,
    "status": "vanishing_under_declared_prescription",
    "regulated_expression": {
      "form": "iGamma_1 = N_eps * (kappa) * sum_comp [ sum_k c_k(d, p, Gt) J_D14b[k,0] ],  J_D14b[k,0] = int
+>  (l^2)^k / (l^2 + i0)",
      "components": 4,
      "J_members_k": [1],
      "components_with_evanescent_Gt_part": 4,
      "examples": {
        "0": "-Gt_00*J_D14b[1,0]/d + Gt_11*J_D14b[1,0]/d + Gt_22*J_D14b[1,0]/d + Gt_33*J_D14b[1,0]/d + 4*J_D
+> 14b[1,0]/d",
        "4": "Gt_00*J_D14b[1,0]/d - Gt_11*J_D14b[1,0]/d - Gt_22*J_D14b[1,0]/d - Gt_33*J_D14b[1,0]/d - 4*J_D1
+> 4b[1,0]/d",
        "7": "Gt_00*J_D14b[1,0]/d - Gt_11*J_D14b[1,0]/d - Gt_22*J_D14b[1,0]/d - Gt_33*J_D14b[1,0]/d - 4*J_D1
+> 4b[1,0]/d"
      }
    },
    "UV_subtraction": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar coun
+> terterm 0",
    "finite_part": "0 (all components)",
    "local_ambiguity": "none: the p14r kappa^1 counterterm basis is empty (one-leg local functionals are tot
+> al derivatives or need a dimension-5 field)",
    "renormalization_conditions": "MS-bar at mu (P14 normalization condition 0) + N4 (no mu^4, mu^2 p^2 cont
+> acts); no counterterm is required",
    "scaleless_analysis": {
      "S0_applications": [
        {
          "context": "D14b: int (l^2)^1/l^2",
          "k": 1,
          "nu": 1,
          "omega": 4,
          "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
        }
      ],
      "logarithmic_members": [],
      "UV_IR_reading": "no member is logarithmic (omega = 0): the DR zero is NOT a 1/eps_UV - 1/eps_IR cance
+> llation here; it discards pure power divergences (omega = 4); the IR region converges at d = 4 and neithe
+> r the UV nor the IR part has a pole at d = 4"
    },
    "prescription_dependence": {
      "IR_regulator_mass": {
        "rule": "1/(l^2 + i0) -> 1/(l^2 - m^2 + i0) on the loop line (analysis only; m is not in the action)
+> ",
        "pole_powers_of_m2": [2],
        "pole_examples": {
          "0": "-Gt_00*m2**2/4 + Gt_11*m2**2/4 + Gt_22*m2**2/4 + Gt_33*m2**2/4 + m2**2",
          "4": "Gt_00*m2**2/4 - Gt_11*m2**2/4 - Gt_22*m2**2/4 - Gt_33*m2**2/4 - m2**2",
          "7": "Gt_00*m2**2/4 - Gt_11*m2**2/4 - Gt_22*m2**2/4 - Gt_33*m2**2/4 - m2**2"
        },
        "finite_examples": {
          "0": "Gt_00*Lm*m2**2/4 - 3*Gt_00*m2**2/8 - Gt_11*Lm*m2**2/4 + 3*Gt_11*m2**2/8 - Gt_22*Lm*m2**2/4 +
+>  3*Gt_22*m2**2/8 - Gt_33*Lm*m2**2/4 + 3*Gt_33*m2**2/8 - Lm*m2**2 + 3*m2**2/2",
          "4": "-Gt_00*Lm*m2**2/4 + 3*Gt_00*m2**2/8 + Gt_11*Lm*m2**2/4 - 3*Gt_11*m2**2/8 + Gt_22*Lm*m2**2/4
+> - 3*Gt_22*m2**2/8 + Gt_33*Lm*m2**2/4 - 3*Gt_33*m2**2/8 + Lm*m2**2 - 3*m2**2/2",
```

```
        "7": "-Gt_00*Lm*m2**2/4 + 3*Gt_00*m2**2/8 + Gt_11*Lm*m2**2/4 - 3*Gt_11*m2**2/8 + Gt_22*Lm*m2**2/4
+> - 3*Gt_22*m2**2/8 + Gt_33*Lm*m2**2/4 - 3*Gt_33*m2**2/8 + Lm*m2**2 - 3*m2**2/2"
      },
      "limit_m_to_0": "0 (pole ~ m^{2j}, finite ~ m^{2j}(a + b log m^2), j >= 1)",
      "counterterm_needed_at_finite_m": "pole ~ kappa m^4 h: a one-point (cosmological-type) contact, OUTS
+> IDE the p14r basis (kappa^1 basis empty); it vanishes as m -> 0"
    },
    "analytic_regulator": "(l^2)^{-1-nu}: the integrand stays scaleless for every nu, the value stays 0; a
+> n analytic regulator alone does not separate UV from IR here",
    "cutoff_split": "each member splits into IR (+Lambda^omega/omega...) and UV (-Lambda^omega/omega...) p
+> ieces with no pole at d = 4; with a hard cutoff the would-be counterterms are kappa Lambda^4 h (one-point
+> ; no p14r kappa^1 direction)",
    "p14r_counterterm_required": false
  },
  "verification_controls": {
    "export_sha_pinned_to_p14r_certificate": true,
    "independent_integrand_route_p14r": "W5 independent sympy regeneration",
    "route1_dred_lift_equals_route2_symmetric_integration": true,
    "components_compared": 4,
    "all_components_vanish": true
  },
  "value_imported_from_XVI_O": false,
  "XVI_O_comparison": "n/a"
},
"D14c": {
  "graph": "D14c",
  "slot": "order-kappa one-point slot",
  "kappa_order": 1,
  "export": {
    "file": "exports/P14_integrands/D14c.json",
    "sha256_recomputed": "26d8a239783f101b997843a91ba11acaea453cab04a9fa1266974ad0e2a7f120",
    "sha256_stored": "26d8a239783f101b997843a91ba11acaea453cab04a9fa1266974ad0e2a7f120",
    "sha256_p14r_certificate": "26d8a239783f101b997843a91ba11acaea453cab04a9fa1266974ad0e2a7f120",
    "pinned": true
  },
  "prefactor": "-kappa/2",
  "denominators": ["l^2+i0"],
  "massless_declared": true,
  "status": "vanishing_under_declared_prescription",
  "regulated_expression": {
    "form": "iGamma_1 = N_eps * (-kappa/2) * sum_comp [ sum_k c_k(d, p, Gt) J_D14c[k,0] ],  J_D14c[k,0] =
+> int (l^2)^k / (l^2 + i0)",
    "components": 10,
    "J_members_k": [1],
    "components_with_evanescent_Gt_part": 10,
    "examples": {
      "0": "Gt_00*J_D14c[1,0]/(2*d) + Gt_11*J_D14c[1,0]/(2*d) + Gt_22*J_D14c[1,0]/(2*d) + Gt_33*J_D14c[1,0
+> ]/(2*d) + J_D14c[1,0]/d",
      "1": "-2*Gt_01*J_D14c[1,0]/d",
      "2": "-2*Gt_02*J_D14c[1,0]/d"
    }
  },
  "UV_subtraction": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar coun
+> terterm 0",
  "finite_part": "0 (all components)",
  "local_ambiguity": "none: the p14r kappa^1 counterterm basis is empty (one-leg local functionals are tot
+> al derivatives or need a dimension-5 field)",
  "renormalization_conditions": "MS-bar at mu (P14 normalization condition 0) + N4 (no mu^4, mu^2 p^2 cont
+> acts); no counterterm is required",
  "scaleless_analysis": {
    "SO_applications": [
      {
        "context": "D14c: int (l^2)^1/l^2",
        "k": 1,
        "nu": 1,
        "omega": 4,
        "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
      }
    ],
    "logarithmic_members": [],
    "UV_IR_reading": "no member is logarithmic (omega = 0): the DR zero is NOT a 1/eps_UV - 1/eps_IR cance
+> llation here; it discards pure power divergences (omega = 4); the IR region converges at d = 4 and neithe
+> r the UV nor the IR part has a pole at d = 4"
  },
  "prescription_dependence": {
    "IR_regulator_mass": {
      "rule": "1/(l^2 + i0) -> 1/(l^2 - m^2 + i0) on the loop line (analysis only; m is not in the action)
+> ",
      "pole_powers_of_m2": [2],
      "pole_examples": {
        "0": "Gt_00*m2**2/8 + Gt_11*m2**2/8 + Gt_22*m2**2/8 + Gt_33*m2**2/8 + m2**2/4",
```

```
          "1": "-Gt_01*m2**2/2",
          "2": "-Gt_02*m2**2/2"
        },
        "finite_examples": {
          "0": "-Gt_00*Lm*m2**2/8 + 3*Gt_00*m2**2/16 - Gt_11*Lm*m2**2/8 + 3*Gt_11*m2**2/16 - Gt_22*Lm*m2**2/
  +> 8 + 3*Gt_22*m2**2/16 - Gt_33*Lm*m2**2/8 + 3*Gt_33*m2**2/16 - Lm*m2**2/4 + 3*m2**2/8",
          "1": "Gt_01*Lm*m2**2/2 - 3*Gt_01*m2**2/4",
          "2": "Gt_02*Lm*m2**2/2 - 3*Gt_02*m2**2/4"
        },
        "limit_m_to_0": "0 (pole ~ m^{2j}, finite ~ m^{2j}(a + b log m^2), j >= 1)",
        "counterterm_needed_at_finite_m": "pole ~ kappa m^4 h: a one-point (cosmological-type) contact, OUTS
  +> IDE the p14r basis (kappa^1 basis empty); it vanishes as m -> 0"
      },
      "analytic_regulator": "(l^2)^{-1-nu}: the integrand stays scaleless for every nu, the value stays 0; a
  +> n analytic regulator alone does not separate UV from IR here",
      "cutoff_split": "each member splits into IR (+Lambda^omega/omega...) and UV (-Lambda^omega/omega...) p
  +> ieces with no pole at d = 4; with a hard cutoff the would-be counterterms are kappa Lambda^4 h (one-point
  +> ; no p14r kappa^1 direction)",
      "p14r_counterterm_required": false
    },
    "verification_controls": {
      "export_sha_pinned_to_p14r_certificate": true,
      "independent_integrand_route_p14r": "W5 independent sympy regeneration",
      "route1_dred_lift_equals_route2_symmetric_integration": true,
      "components_compared": 10,
      "all_components_vanish": true
    },
    "value_imported_from_XVI_O": false,
    "XVI_O_comparison": "XVI-O declares the D12/one-point value 0 by dimensional_regularization_m0 (coincide
  +> nt prescription); p15 DERIVES the same vanishing under P's own declared prescription; not imported"
  }
},
"D12_slot": {
  "graph": "D12",
  "kappa_order": 2,
  "status": "vanishing_under_declared_prescription",
  "derivation": "certificates/P15_batchA.json graphs.D12 (route 1 frozen-dred lift + S0, route 2 symmetric i
  +> ntegration, IR-mass and cutoff analysis)",
  "previous_status": "Missing (p14r)"
},
"one_point_slot_kappa1": {
  "graphs": ["D14a", "D14b", "D14c"],
  "kappa_order": 1,
  "status": "vanishing_under_declared_prescription",
  "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"
},
"gate_checks": {
  "D02": true,
  "D12": true,
  "D14a": true,
  "D14b": true,
  "D14c": true,
  "D12_separate_from_one_point_slot": true
},
"batchA_evaluated": {
  "execution": true,
  "claim": "D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS-bar
  +> + Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription with no UV
  +> pole, finite part 0, no p14r counterterm required",
  "scope": "massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpoles c
  +> arry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14r basis (N4) an
  +> d vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated); no slot is complete;
  +>  NOT quantum_inputs_evaluated"
},
"loop_evaluations_performed": true,
"seconds": 1216.9
}
```


62 P.5 calculation: Batch B certificate (D13; p15)

[listing 29/133] certificates/P15_D13.json


```
{
  "landmark": "XVI-P",
  "revision": "p15",
  "deliverable": "P.5 calculation Batch B: D13 (K'.c, kappa^2)",
  "contract_revision": "p14r",
  "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 + S0 (certificates/P15_integral_library.json)",
```

```
"convention": "G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * Ghat, Ghat
+> = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), L = log(-p^2/mu^2 - i0); indices lower (K
+> ' slot dual to h_(mn), c^nu -> lower nu); p_r lower components; p^2 = eta^{rs} p_r p_s",
"export": {
  "file": "exports/P14_integrands/D13.json",
  "sha256_recomputed": "42d999fbc4eb3aa01b57050fbe5ff97525976e56dca0e748bae6ca9f03c67026",
  "sha256_stored": "42d999fbc4eb3aa01b57050fbe5ff97525976e56dca0e748bae6ca9f03c67026",
  "sha256_p14r_certificate": "42d999fbc4eb3aa01b57050fbe5ff97525976e56dca0e748bae6ca9f03c67026",
  "pinned": true
},
"prefactor_export": "I*kappa**2",
"status": "regulated",
"sector_status": {
  "4D_structures_T1_T3": "renormalized",
  "evanescent_structures_E1_E2": "regulated: pole nonzero (E2), recorded and retained (R6); its counterterm
+> is an evanescent K'.c operator NOT in the p14r 4D basis (the p14r contract declares no evanescent counter
+> term: open contract item, not absorbed)"
},
"evanescent_counterterm_declared_in_p14r": false,
"result": {
  "T1": {
    "structure": "eta_mn p_nu p^2",
    "A_exact_d_times_MB": "0",
    "UV_pole": "0",
    "MSbar_finite": "0",
    "nonlocal_coefficient_of_L": "0",
    "local_polynomial": "0",
    "finite_if_d_set_to_4_first": "0",
    "local_shift_from_d_dependence": "0"
  },
  "T2": {
    "structure": "(eta_m nu p_n + eta_n nu p_m) p^2",
    "A_exact_d_times_MB": "(4*d - 7)/(16*(d - 1))",
    "UV_pole": "3/16",
    "MSbar_finite": "1/3 - 3*L/16",
    "nonlocal_coefficient_of_L": "-3/16",
    "local_polynomial": "1/3",
    "finite_if_d_set_to_4_first": "3/8 - 3*L/16",
    "local_shift_from_d_dependence": "-1/24"
  },
  "T3": {
    "structure": "p_m p_n p_nu",
    "A_exact_d_times_MB": "3*(2 - d)/(8*(d - 1))",
    "UV_pole": "-1/4",
    "MSbar_finite": "L/4 - 5/12",
    "nonlocal_coefficient_of_L": "1/4",
    "local_polynomial": "-5/12",
    "finite_if_d_set_to_4_first": "L/4 - 1/2",
    "local_shift_from_d_dependence": "1/12"
  },
  "E1": {
    "structure": "Gt_mn p_nu p^2",
    "A_exact_d_times_MB": "0",
    "UV_pole": "0",
    "MSbar_finite": "0",
    "nonlocal_coefficient_of_L": "0",
    "local_polynomial": "0",
    "finite_if_d_set_to_4_first": "0",
    "local_shift_from_d_dependence": "0"
  },
  "E2": {
    "structure": "(Gt_m nu p_n + Gt_n nu p_m) p^2",
    "A_exact_d_times_MB": "3/(16*(d - 1))",
    "UV_pole": "1/16",
    "MSbar_finite": "1/6 - L/16",
    "nonlocal_coefficient_of_L": "-1/16",
    "local_polynomial": "1/6",
    "finite_if_d_set_to_4_first": "1/8 - L/16",
    "local_shift_from_d_dependence": "1/24"
  }
},
"UV_pole_in_p14r_cK_basis": {
  "coordinates": ["0", "3/8", "-1/4"],
  "basis": [
    {
      "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1, 1)
+> )] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (-I)
+> *[('c', 0, ()), ('K', (1, 1), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0,
+> ",
      "T_coordinates": ["1", "0", "0"]
    },
```

```
    {
      "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1, 1)
+> )] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 3, 3))] + (I/2
+> )*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))] + (-I/2)*[('c', 0, ()), ('K', (0, 1), (1, 1, 1))] + (-I/2)*[('
+> ",
      "T_coordinates": ["0", "1/2", "0"]
    },
    {
      "density": "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (0, 1), (0, 0, 1))
+> ] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), ('K', (0, 3), (0, 0, 3))] + (I)*[('
+> c', 0, ()), ('K', (1, 1), (0, 1, 1))] + (I)*[('c', 0, ()), ('K', (1, 2), (0, 1, 2))] + (I)*[('c', 0, ()),
+> ",
      "T_coordinates": ["0", "0", "1"]
    }
  ]
},
"local_polynomial_in_p14r_cK_basis": ["0", "2/3", "-5/12"],
"nonlocal_in_p14r_cK_basis": ["0", "-3/8", "1/4"],
"MSbar_counterterm": "delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p14r c.K' de
+> nsities, z = UV_pole coordinates); plus, for the evanescent structures, an evanescent counterterm (Gt_mn
+> p_nu p^2, (Gt p + Gt p) p^2) NOT in the p14r basis",
"evanescent_pole_nonzero": true,
"local_ambiguity": "finite renormalization along the three p14r c.K' directions: G^R -> G^R + kappa^2 (i/(16
+>  pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local polynomial by + A_i(4) log(mu'^2/mu^2
+> ) along the UV-pole direction",
"renormalization_conditions": "MS-bar at mu: counterterm = pure pole (P14 normalization condition 0); no low
+> er-degree K'.c contact (mu^2 K' dc) is generated (no scale but mu; pole is cubic in p); u^R_mu (XVI-O) no
+> t modified; an alternative momentum subtraction at p^2 = -M^2 differs by the finite renormalization A_i(4
+> ) log(M^2/mu^2) - (local MS-bar polynomial) along the same directions",
"absorptive_part": "for timelike p^2 > 0: L = log(p^2/mu^2) - i pi, Im Ghat = + pi * sum A_i(4) T_i",
"d_dependence": {
  "evanescent_d_terms_matter": true,
  "local_shift_from_d_dependence_T": {
    "T1": "0",
    "T2": "-1/24",
    "T3": "1/12"
  },
  "evanescent_structure_poles": {
    "E1": "0",
    "E2": "1/16"
  },
  "statement": "the (d-4) x pole terms change the local finite polynomial (premature d = 4 gives a different
+>  MS-bar finite part); the difference is local and lies in the p14r c.K' span (a scheme-type finite renorm
+> alization), so it matters for the finite part in the declared scheme"
},
"authorised_d_substitutions": {
  "d13.exact_fit: numerical rank test for row selection only (the solve and the verification of every row ar
+> e exact in d)": 324,
  "integrals.laurent: d = 4 - 2 eps after all contractions": 1,
  "d13.expand_coeff: Laurent expansion after all contractions and the fit": 13,
  "d13.expand_coeff: diagnostic A(4) (d-dependence proof only)": 13,
  "d13.expand_coeff: diagnostic A'(4)": 13,
  "d13.mutation display: pole": 3,
  "mutation display only (enforce=False): premature d = 4": 396
},
"independent_cross_check_route": "route 2: integrand regenerated from the rule tables (feynman.pw_vertex; no
+>  exported data) and evaluated by Feynman parameters with d symbolic (integrals.fp_two_point; no Gram/PV r
+> eduction, no S0); agrees with route 1 (frozen export -> dred Pipeline) exactly in d",
"scaleless_rule_applications": [
  {
    "context": "J_D13[0,1] [I(0,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D13[0,1] [I(1,0) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
    "context": "J_D13[0,2] [I(-1,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  },
  {
```

```
  "context": "J_D13[0,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,2] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,2] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
```

```
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,3] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(0,-2) -> polynomial/(l^2)^-2]",
  "k": 0,
  "nu": -2,
  "omega": 8,
  "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(1,-3) -> polynomial/(l^2)^1]",
```

```
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(1,-2) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(1,-1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[0,4] [I(1,0) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,0] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,1] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,1] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,2] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
```

```
},
{
  "context": "J_D13[1,2] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,2] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(0,-2) -> polynomial/(l^2)^-2]",
  "k": 0,
  "nu": -2,
  "omega": 8,
  "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,3] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
```

```
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-4,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-3,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-2,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-1,-2) -> polynomial/(l^2)^-2]",
  "k": 0,
  "nu": -2,
  "omega": 8,
  "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(0,-3) -> polynomial/(l^2)^-3]",
  "k": 0,
  "nu": -3,
  "omega": 10,
  "class": "UV power divergence Lambda^10: no pole on either side at d = 4"
},
```

```
{
  "context": "J_D13[1,4] [I(0,-2) -> polynomial/(l^2)^-2]",
  "k": 0,
  "nu": -2,
  "omega": 8,
  "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(0,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(0,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[1,4] [I(0,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,0] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,1] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,1] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,1] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,2] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,2] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,2] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,2] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
```

```
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,2] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,2] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-4,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-3,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-2,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-1,-2) -> polynomial/(l^2)^-2]",
  "k": 0,
  "nu": -2,
  "omega": 8,
  "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-1,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,3] [I(-1,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
```

```
  "context": "J_D13[2,3] [I(-1,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-5,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-4,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-4,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-3,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-3,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-3,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-2,-2) -> polynomial/(l^2)^-2]",
  "k": 0,
  "nu": -2,
  "omega": 8,
  "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-2,-1) -> polynomial/(l^2)^-1]",
  "k": 0,
  "nu": -1,
  "omega": 6,
  "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-2,0) -> polynomial/(l^2)^0]",
  "k": 0,
  "nu": 0,
  "omega": 4,
  "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-2,1) -> polynomial/(l^2)^1]",
  "k": 0,
  "nu": 1,
  "omega": 2,
  "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
},
{
  "context": "J_D13[2,4] [I(-1,-3) -> polynomial/(l^2)^-3]",
  "k": 0,
  "nu": -3,
  "omega": 10,
```

```
    "class": "UV power divergence Lambda^10: no pole on either side at d = 4"
  },
  {
    "context": "J_D13[2,4] [I(-1,-2) -> polynomial/(l^2)^-2]",
    "k": 0,
    "nu": -2,
    "omega": 8,
    "class": "UV power divergence Lambda^8: no pole on either side at d = 4"
  },
  {
    "context": "J_D13[2,4] [I(-1,-1) -> polynomial/(l^2)^-1]",
    "k": 0,
    "nu": -1,
    "omega": 6,
    "class": "UV power divergence Lambda^6: no pole on either side at d = 4"
  },
  {
    "context": "J_D13[2,4] [I(-1,0) -> polynomial/(l^2)^0]",
    "k": 0,
    "nu": 0,
    "omega": 4,
    "class": "UV power divergence Lambda^4: no pole on either side at d = 4"
  },
  {
    "context": "J_D13[2,4] [I(-1,1) -> polynomial/(l^2)^1]",
    "k": 0,
    "nu": 1,
    "omega": 2,
    "class": "UV power divergence Lambda^2: no pole on either side at d = 4"
  }
],
"dred_ledger_reductions_introducing_d": 248,
"controls": {
  "export_sha_pinned": true,
  "route1_equals_route2_exact_in_d": true,
  "covariant_fit_exact": true,
  "fits_agree": true,
  "pole_local_in_p14r_cK_span": true,
  "ghost_equation_2iD_G_equals_ghost_self_energy": true,
  "ghost_self_energy_pole_in_p14r_c_cb_span": true,
  "nonlocal_equals_minus_pole": true,
  "R5_d_retained": true,
  "R5_structural_safeguard_active": true,
  "mutations_all_rejected": true
},
"route_comparison": {
  "compared": 160,
  "mismatches": [],
  "n_mismatches": 0,
  "agree": true
},
"fit_equations": 240,
"ghost_equation": {
  "compared": 64,
  "mismatches": [],
  "n_mismatches": 0,
  "agree": true,
  "statement": "Sigma_{rho nu}(p) [one-loop ghost self-energy, same topology, evaluated by route 2 from the
+> cbar.c.h vertex] == 2 i D^rho_p[G_{.. nu}] with D^rho T = d_s T^{rho s} - 1/2 d^rho tr T (linear de Donde
+> r gauge: ghost equation); exact in d on the faithful Gt realization"
},
"ghost_self_energy_pole": {
  "fit_exact": true,
  "structures": {
    "S1": "delta^rho_nu p^4",
    "S2": "p^rho p_nu p^2",
    "SE": "Gt^rho_nu p^4 (evanescent)"
  },
  "coefficients_exact_d": {
    "S1": "-(4*d - 7)/(8*(d - 1))",
    "S2": "3*(d - 2)/(8*(d - 1))",
    "SE": "-3/(8*(d - 1))"
  },
  "pole": {
    "S1": "-3/8",
    "S2": "1/4",
    "SE": "-1/8"
  },
  "local": {
    "S1": "-2/3",
    "S2": "5/12",
```

```
    "SE": "-1/3"
  },
  "p14r_c_cb_basis": [
    {
      "density": "(-1)*[('c', 0, (0, 0, 0, 0)), ('cb', 0, ())] + (2)*[('c', 0, (0, 0, 1, 1)), ('cb', 0, ())]
+>  + (2)*[('c', 0, (0, 0, 2, 2)), ('cb', 0, ())] + (2)*[('c', 0, (0, 0, 3, 3)), ('cb', 0, ())] + (-1)*[('c'
+> , 0, (1, 1, 1, 1)), ('cb', 0, ())] + (-2)*[('c', 0, (1, 1, 2, 2)), ('cb', 0, ())] + (-2)*[('c', 0, (1, 1,
+> ",
      "S_coordinates": ["1", "0"]
    },
    {
      "density": "(-1)*[('c', 0, (0, 0, 0, 0)), ('cb', 0, ())] + (1)*[('c', 0, (0, 0, 1, 1)), ('cb', 0, ())]
+>  + (1)*[('c', 0, (0, 0, 2, 2)), ('cb', 0, ())] + (1)*[('c', 0, (0, 0, 3, 3)), ('cb', 0, ())] + (-1)*[('c'
+> , 1, (0, 0, 0, 1)), ('cb', 0, ())] + (1)*[('c', 1, (0, 1, 1, 1)), ('cb', 0, ())] + (1)*[('c', 1, (0, 1, 2
+> ",
      "S_coordinates": ["0", "1"]
    }
  ],
  "pole_in_p14r_c_cb_coordinates": ["-3/8", "1/4"],
  "evanescent_pole_SE": "-1/8",
  "pole_local_and_in_p14r_c_cb_span": true
},
"mutations": {
  "wrong_ghost_loop_sign": {
    "changes_result": true,
    "rejected": true,
    "reason": "route 1 (frozen export) != regenerated route 2: 160 mismatches; W1 shadow: D13 has no closed
+> ghost loop",
    "pole_T_if_accepted": ["0", "-3/16", "1/4"],
    "accepted_as_rejection": true
  },
  "premature_d4": {
    "rejected": true,
    "changes_result": true,
    "reason": "PrematureD4Substitution (structural, at the substitution site): d -> 4 outside substitute_d (
+> R5: d is opaque before the Laurent expansion)",
    "fit_d_free_if_not_caught": false,
    "local_part_if_not_caught_T": ["-1/24", "5/12", "-1/2"],
    "local_part_certified_T": ["0", "1/3", "-5/12"],
    "note": "the Gt realization Gt = (4-d) M reintroduces d after the premature substitution (the false-nega
+> tive mechanism of finding F1); only the structural safeguard catches it",
    "pole_T_if_not_caught": ["0", "3/16", "-1/4", "0", "1/16"],
    "accepted_as_rejection": true
  },
  "drop_evanescent_Gt": {
    "changes_result": true,
    "rejected": true,
    "reason": "route 1 with Gt dropped != route 2 (ghat kept): 160 mismatches; R6 retention check fails",
    "fit_if_accepted": {
      "T1": "-(d - 4)/(16*(d - 1))",
      "T2": "(5*d - 11)/(16*(d - 1))",
      "T3": "-3*(d - 2)/(8*(d - 1))",
      "E1": "0",
      "E2": "0"
    },
    "accepted_as_rejection": true
  },
  "wrong_symmetry_factor": {
    "changes_result": true,
    "rejected": true,
    "reason": "route 1 != route 2 with S_f = 1/2: 160 mismatches",
    "accepted_as_rejection": true
  },
  "wrong_K_c_ordering": {
    "changes_result": true,
    "rejected": true,
    "reason": "route 1 != route 2 with leg order (c, K', h): 160 mismatches",
    "accepted_as_rejection": true
  }
},
"mutations_all_rejected": true,
"D13_evaluated": {
  "execution": true,
  "claim": "D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of the
+> 4D structures is local and in the span of the three p14r c.K' directions (the evanescent E2 pole is recor
+> ded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + local cubic polynomial, two indep
+> endent routes agree exactly in d, ghost equation verified, mutations rejected",
  "scope": "one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); extern
+> al momenta in QdS (R2); evanescent Gt structures retained (their pole is NONZERO: an evanescent K'.c coun
+> terterm outside the p14r 4D basis is needed for full DRED renormalization); NOT quantum_inputs_evaluated;
+>  the slot's identity role is not evaluated (Batch C, D11 M2)"
```

```
  },
  "loop_evaluations_performed": true,
  "seconds": 64.4
}
```


63 p14r contract record (P.5 entry; contract gate)

[listing 30/133] certificates/P14_release.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "date": "2026-10-08",
  "deliverable": "P.5 entry contract",
  "execution_pass": true,
  "claim_status": "P.5_entry: earned (contract fixed); classical_readiness (frozen P.4): earned",
  "scope": "P.5 entry: the quantum object, ONE prescription with the D11 matching note, the one-loop counterte
  +> rm domain and executable amplitude rules for the fixed one-loop kappa^2 ch identity on the S_inv baseline
  +> ; integrands only; NO loop evaluation; NOT quantum inputs, NOT the projected identity, NOT bulk restorati
  +> on, NOT the prepared S_D, NOT the insertion",
  "release_claim_accepted": true,
  "release_claim_accepted_definition": "p13 classical release claim (recomputed from the frozen inputs) AND p1
  +> 4/p14r hard stops AND P5_entry_earned (all five contract flags AND P3_P4_frozen_as_P5_inputs)",
  "P5_entry_earned": true,
  "P5_entry_flags": {
    "quantum_action_defined": true,
    "quantum_prescription_defined": true,
    "D11_matching_defined": true,
    "counterterm_domain_defined": true,
    "amplitude_rules_verified": true
  },
  "P5_entry_records": {
    "quantum_action_defined": {
      "certificate": "P14_quantum_object.json",
      "present": true,
      "value": true,
      "revision": "p14r",
      "content_sha256_ignoring_timings": "d1edb02102eaa396e7dedc82b8dbca6ffb28c67b280b62c93639c611b7a4fde2"
    },
    "quantum_prescription_defined": {
      "certificate": "P14_prescription_D11_matching.json",
      "present": true,
      "value": true,
      "revision": "p14r",
      "content_sha256_ignoring_timings": "93889d4304b5a01f4ecfe00e5858a6ff99aa97ba1bb41262e3b8a306186cad45",
      "dred_certificate_present": true,
      "dred_rules_ok": true
    },
    "D11_matching_defined": {
      "certificate": "P14_prescription_D11_matching.json",
      "present": true,
      "value": true,
      "revision": "p14r",
      "content_sha256_ignoring_timings": "93889d4304b5a01f4ecfe00e5858a6ff99aa97ba1bb41262e3b8a306186cad45",
      "M1_pass": true,
      "D11_matching_verified": false
    },
    "counterterm_domain_defined": {
      "certificate": "P14_counterterm_domain.json",
      "present": true,
      "value": true,
      "revision": "p14r",
      "content_sha256_ignoring_timings": "e6431c0ab8c301beba0c80546e49e10970438b3e3893735acfff7cd125001215",
      "kappa1": 0,
      "kappa2": 11,
      "by_sector": {
        "h.h": 5,
        "c.cb": 2,
        "c.K": 3,
        "phi.phi": 1
      }
    },
    "amplitude_rules_verified": {
      "certificate": "P14_amplitude_rules.json",
      "present": true,
      "value": true,
      "revision": "p14r",
      "content_sha256_ignoring_timings": "fbd79f7a506ef6d3ac2f1f5c99890126b234cb2f7c2f281d76f369c7ac33621b",
```

```
    "controls_all_pass": true,
    "mutations_all_rejected": true,
    "W6_hessian_route_and_exports": true
  }
},
"P5_entry_qualification": "P.5 entry = the contract is fixed, nothing is evaluated. Qualifications (p14r): t
+> he DRED prescription is operational at the level of index algebra, lifting, ghat-projected tensor reducti
+> on and the order contraction -> Laurent (tested on generic placeholders and applied to the lifted exporte
+> d integrands), but no scalar integral is evaluated and the scaleless-integral rules are not applied; D11
+> M3 is conditional on M2 and its degree-4 form on the imposed normalization condition N4; the counterterm
+> count 8 is a count of candidate directions modulo the computed exact equivalences, before the identity co
+> nstraints (D cap ker sigma_0 not computed); D11_matching_verified = false; D12 Missing.",
"P5_entry_dependency": "P.5 entry is stated against the P.3/P.4 freeze: P3_P4_frozen_as_P5_inputs = True (p1
+> 3 independent rebuild passed, bound to the p13 tree; the p13 independent rebuild ran on the SAME box (sha
+> red kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it was NOT run on a second machi
+> ne)",
"P3_P4_frozen_as_P5_inputs": true,
"freeze": {
  "P3_P4_frozen_as_P5_inputs": true,
  "definition": "F1 independent_rebuild_passed(p13 tree) AND F2 frozen code byte-identical AND F3 frozen val
+> ues equal",
  "independent_rebuild_passed_p13": true,
  "frozen_code_unchanged": true,
  "frozen_values_unchanged": true,
  "bound_to": {
    "p13_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
    "p13_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266"
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "F1": {
    "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
    "present": true,
    "sha256_ok": true,
    "report_pass": true,
    "report_independent_rebuild_passed": true,
    "mismatches": 0,
    "independent_comparisons": 77,
    "compared_with_reference": {
      "compared": 112,
      "equal": 112
    },
    "bound_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
    "bound_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
    "bound_to_p13_tree": true,
    "bound_to_p13_zip": true,
    "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh
+>  venv from the source-only zip; it was NOT run on a second machine",
    "live_report_present": true,
    "live_report_identical_to_archived": true
  },
  "F2": {
    "files": 54,
    "changed_or_missing": [],
    "list_sha256_ok": true,
    "archived_zip_sha256_ok": true,
    "list_equals_archived_zip": true
  },
  "F3": {
    "compared": 70,
    "equal": 70,
    "differences": [],
    "reference_sha256_ok": true
  }
},
"D11_matching_verified": false,
"D12_status": "Missing",
"projected_vs_bulk": "projected (XVI-N lem:field-only; the ch projection of the fixed one-loop kappa^2 ident
+> ity) is distinct from bulk BV restoration; the XVI-O kernel two-point Ward identity d(Pi_w + L) = 0 is NO
+> T functional ST4b",
"classical": {
  "flags": {
    "quartic_generator_verified": true,
    "ch_projection_defined": true,
    "ch_dependency_inventory_complete": true,
    "gauge_fixing_map_constructed": true,
    "required_classical_BV_components_verified": true
  },
  "classical_readiness_for_ch_identity": true,
  "release_claim_accepted": true,
  "deterministic_prerequisites": {
```

```
  "C1_inverse_10comp": true,
  "C2_det_tracelog_10comp": true,
  "C3_boundary_identity_generic_and_conformal": true,
  "C4_export_equals_independent_evaluator": true,
  "C5_cubic_regression_Euler": true,
  "C6_quadratic_regression_Euler": true,
  "C7_grav_noether_k0_k1_k2_on_exports": true,
  "quartic_mutations_all_rejected": true,
  "master_grav_noether_deterministic_k0_k1_k2": true,
  "master_engine_parts_exact": true,
  "graviton_hessian_generic_symbolic_k": true,
  "canonicity_all_23_antifield_components": true,
  "ch_enumeration_lemma_mechanical": true,
  "residual_record_invariants_all_certificates": true,
  "quartic_C7_Q_terms_zero_and_flag_true_all_orders": true,
  "master_grav_Q_terms_zero_and_flag_true_all_orders": true,
  "cross_certificate_quartic_vs_master_agree": true,
  "stored_C7_flag_equals_recomputation": true
},
"failed_prerequisites": [],
"hash_consistency": {
  "consistent": true,
  "ell": {
    "ell2": {
      "export_file": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "quartic_cert": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "master_input": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0"
    },
    "ell3": {
      "export_file": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "quartic_cert": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "master_input": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1"
    },
    "ell4": {
      "export_file": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "quartic_cert": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "master_input": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  }
},
"required_master_residuals": [],
"residual_records": {
  "invariant_violations": {},
  "quartic_C7_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "master_grav_per_order": {
    "kappa0": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa1": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    },
    "kappa2": {
      "residual_counts": [0, 0, 0, 0],
      "zero_flag": true,
      "accepted": true
    }
  },
  "cross_certificate": {
    "same_inputs": true,
    "differences": [],
    "agree": true,
```

```
      "quartic_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      },
      "master_parameters": {
        "a_ghost": "1/2",
        "transport_term": 1,
        "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
        "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
        "ell_sha256": {
          "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
          "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
          "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
        }
      }
    },
    "hard_stops_p13_hold": true,
    "hard_stops_p13": {
      "quantum_inputs_evaluated": "not_earned",
      "projected_identity_verified": "not_earned",
      "D02_quantum_evaluated": false,
      "D12_quantum_evaluated": false,
      "D12_status": "Missing",
      "symmetry_restored": false,
      "O1_closure": "NOT_CLOSED",
      "insertion_status": "Frozen_negative_obstruction_preserved",
      "XV_B_opened": false,
      "Route_B_discharged": false,
      "P5_opened": false,
      "loop_evaluations_performed": false,
      "u_R_c_delta_added": false
    },
    "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"
  },
  "statuses": {
    "quartic_generator_verified": "earned",
    "ch_projection_defined": "earned",
    "ch_dependency_inventory_complete": "earned",
    "gauge_fixing_map_constructed": "earned",
    "required_classical_BV_components_verified": "earned",
    "classical_readiness_for_ch_identity": "earned",
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "propagators_match_transformed_Hessians": true,
    "source_identity_relation_defined": true,
    "source_identity_relation": "defined classical pullback/interface (related, not identical): Gamma_Psi(Phi,
  +> Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi); not a quantum theorem",
    "D02_vertex_available": true,
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "quantum_action_defined": "earned",
    "quantum_prescription_defined": "earned",
    "D11_matching_defined": "earned",
    "counterterm_domain_defined": "earned",
    "amplitude_rules_verified": "earned",
    "D11_matching_verified": false,
    "P5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)"
  },
  "ch_surviving_rows_unevaluated": [
    {
      "row": "(S0,Gamma1^[2]) s2 K:K.c",
      "Gamma1_pattern": "h^2",
      "graphs": ["D01_hh_bubble_cubic", "D02_hh_tadpole_quartic", "D03_hh_ghost_loop", "D11_hh_scalar_loop", "
  +> D12_hh_scalar_tadpole"],
      "value": "not evaluated (P.5 calculation is p15+); integrands exported where available"
    },
    {
      "row": "(S0,Gamma1^[2]) s1 h:h^2",
      "Gamma1_pattern": "K.c",
      "graphs": ["D13_Kc_ghost_graviton_loop"],
      "value": "not evaluated (P.5 calculation is p15+); integrands exported where available"
    },
```

```
  {
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "Gamma1_pattern": "h",
    "graphs": ["D14a_h_onepoint_grav_tadpole", "D14b_h_onepoint_ghost_tadpole", "D14c_h_onepoint_scalar_tadp
+> ole"],
    "value": "not evaluated (P.5 calculation is p15+); integrands exported where available"
  }
],
"remaining_blockers": [
  {
    "id": "Batch A/B/C one-loop integrals (D01, D02, D03, D12, D13, D14a-c; D11 matching M2/L_match)",
    "status": "integrands exported; not evaluated (p15+)",
    "blocks": "quantum_inputs_evaluated"
  },
  {
    "id": "D12_quantum_evaluation",
    "status": "Missing",
    "blocks": "quantum_inputs_evaluated"
  },
  {
    "id": "D11_matching_verification (M2 on U, L_match)",
    "status": "defined, not verified",
    "blocks": "D11_matching_verified"
  }
],
"hard_stops": {
  "quantum_inputs_evaluated": "not_earned",
  "projected_defect_computed": "not_earned",
  "projected_identity_verified": "not_earned",
  "projected_restoration_verified": "not_earned",
  "bulk_symmetry_restored": false,
  "symmetry_restored": false,
  "D02_quantum_evaluated": false,
  "D12_quantum_evaluated": false,
  "D12_status": "Missing",
  "D11_matching_verified": false,
  "O1_closure": "NOT_CLOSED",
  "insertion_status": "Frozen_negative_obstruction_preserved",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "P5_entry_opened": true,
  "P5_calculation_opened": false,
  "P5_identity_opened": false,
  "loop_evaluations_performed": false,
  "u_R_c_delta_added": false
},
"hard_stops_hold": true,
"certificate_hard_stops": {
  "D11_matching_verified_false": true,
  "D12_Missing_in_prescription": true,
  "D12_Missing_in_amplitudes": true,
  "no_graph_value": true,
  "no_loop_evaluation": true
},
"milestones": {
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
+> te for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
+> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "open (not started; Batch A/B/C integrals are p15+)",
    "P.5_identity": "open"
  },
  "P3_complete": true,
  "P4_classical_readiness_earned": true,
  "P5_entry_earned": true,
  "P5_entry_rule": "P.5 entry is earned iff all five contract flags hold (and is stated against the P.3/P.4
+> freeze)",
  "P5_entry_flags": {
    "quantum_action_defined": true,
    "quantum_prescription_defined": true,
```

```
    "D11_matching_defined": true,
    "counterterm_domain_defined": true,
    "amplitude_rules_verified": true
  },
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not ear
+> ned)",
  "P3_P4_frozen_as_P5_inputs": true,
  "frozen_rule": "P.3/P.4 are frozen P.5 inputs iff the p13 independent rebuild passed (bound to the p13 tre
+> e) AND the frozen code/values are unchanged",
  "P5_entry_depends_on": "P3_P4_frozen_as_P5_inputs (p13 independent rebuild; caveat: the p13 independent re
+> build ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it
+>  was NOT run on a second machine)",
  "history": "archival/p02_p09/notes/P_scope.tex (p06-p08 'P-next.N' labels; its table row 'P4 coupled one-l
+> oop restoration' is SUPERSEDED: P.4 = classical readiness); archival/p12/ (p12 used 'classical readiness'
+>  without milestone labels); archival/p13/ (p13 release: P.3 complete, P.4 earned, P.5 entry next)",
  "XVI_O_unaffected": "XVI-P is unregistered package evidence; XVI-O prop:kappa2-master stays Partial"
},
"author_rebuild_passed": false,
"independent_rebuild_passed": false,
"independent_rebuild_passed_p13": true,
"rebuild_flag_scope": "author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p14r) sour
+> ce tree only; independent_rebuild_passed_p13 refers to the p13 source tree 2b1baf335468944aa800c05eaa8b0b
+> 4df67ee893491079fcc944eb8fb0e69a70 / zip 2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266
+>  and is not inherited by p14r changes; it carries the P3_P4_frozen_as_P5_inputs freeze. Caveat: the p13 i
+> ndependent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh venv from the source
+> -only zip; it was NOT run on a second machine",
"p13_independent_rebuild": {
  "independent_rebuild_passed": true,
  "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
  "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
  "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
  "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
  "comparisons": {
    "reference_values_matched": {
      "compared": 112,
      "equal": 112
    },
    "independent_comparisons": 77,
    "mismatches": 0
  },
  "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
+> env from the source-only zip; it was NOT run on a second machine",
  "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
},
"reproducibility": {
  "contract": "REBUILD_INSTRUCTIONS.md",
  "entrypoint": "python3 tools/rebuild_from_source.py",
  "source_zip": "XVI-P-p23-source-only.zip",
  "reference_values": "reproducibility/reference_values_p23.json",
  "author": {
    "report": "reproducibility/author_rebuild_report_p14r.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": true,
    "inherited_mode": "vendored",
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41"
    }
  },
  "independent": {
    "report": "reproducibility/independent_rebuild_report_p14r.json",
    "present": true,
    "report_pass": true,
    "report_source_tree_sha256": "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894",
    "current_source_tree_sha256": "262f1e46ecddcede2279e838299302669c80cd2f6d7644aee7cb54cb4c6a4fe4",
    "bound_to_current_source": false,
    "report_records_source_zip_sha256": false,
    "inherited_mode": null,
    "environment": {
      "python": "3.13.5",
      "sympy": "1.14.0",
      "mpmath": "1.3.0",
```

```
      "pytest": "9.1.1",
      "pdftex": "pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)",
      "pdftotext": "pdftotext version 25.03.0",
      "pdfinfo": "pdfinfo version 25.03.0",
      "platform": "Linux-6.12.94+-x86_64-with-glibc2.41",
      "venv": "/workspace/indep_rebuild_p14r/venv",
      "pip_installed": "sympy==1.14.0 mpmath==1.3.0 pytest==9.1.1 (exact versions via pip)"
    }
   },
   "p13_archive": "archival/p13",
   "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"
 },
 "XVI_P_registered": false,
 "symmetry_restored": false,
 "bulk_symmetry_restored": false,
 "O1_closure": "NOT_CLOSED",
 "insertion_status": "Frozen_negative_obstruction_preserved",
 "XV_B_opened": false,
 "XIV_J_opened": false,
 "Route_B_discharged": false,
 "obstruction": "XVI-N thm:massless-obstruction respected; no completion of the prepared/inserted functional;
 +>  no c delta added to u^R_mu (XVI-J prop:extension); counterterm domain is a constraint list, not an admis
 +> sion",
 "not_premises": ["En01-A", "En01-B", "En01-C", "Em01 family", "Em02-B", "Em02-D", "En02-A", "En02-B", "VII-E
 +> ", "VIII-B"],
 "baseline": "S_inv (unprepared, sigma_eps == 1, no insertion I_t); prepared S_D keeps (S_D,S_D) != 0 (XVI-N
 +> prop:broken-square)",
 "ghost_normalization_a": "1/2",
 "XVI_M_sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
 "XVI_M_pin_ok": true,
 "XVI_A_to_O_edited": false,
 "XVI_O_o70_sync": "read-only",
 "inputs_content_sha256_ignoring_timings": {
   "P13_quartic_verified.json": "9ea00e3fac0551c01076311a0fe5f169a2c8952a1ca3521ced0df6d073e3eca4",
   "P13_boundary_identity.json": "fc0d07742b7bb4895f34b978738d1056a1b2859ba70ad909b8a35f116c7acf27",
   "P13_brst_derivation.json": "e9a9a3027b8c36b27d42101af6d4e5c032094a28e10183710d95c0c3955dfdc5",
   "P13_source_action.json": "d98816405eb974e19e989f7b9a779fd9993205f6f33fc5cf0d004f69008e4ca8",
   "P13_gauge_fixing_map.json": "0484aa54f81bc7c61701a7801dc86ef76363e5f909255c40ca9d20bfa0603454",
   "P13_master_kappa2.json": "e82b4975ccc20b6423ab0b2ce098ff91681e1e33daa74016913ee2560cb07418",
   "P13_ch_source_audit.json": "b8a5a2b2ee13b7cd9264fec7de6f7246bb0e129b9d4d96a5d218545670c7212c",
   "P14_quantum_object.json": "d1edb02102eaa396e7dedc82b8dbca6ffb28c67b280b62c93639c611b7a4fde2",
   "P14_prescription_D11_matching.json": "93889d4304b5a01f4ecfe00e5858a6ff99aa97ba1bb41262e3b8a306186cad45",
   "P14_counterterm_domain.json": "e6431c0ab8c301beba0c80546e49e10970438b3e3893735acfff7cd125001215",
   "P14_amplitude_rules.json": "fbd79f7a506ef6d3ac2f1f5c99890126b234cb2f7c2f281d76f369c7ac33621b"
 },
 "publication": "accepted"
}
```


64 Closure release artifact of the p14r contract

[listing 31/133] certificates/P14_closure_release.json


```
{
 "landmark": "XVI-P",
 "revision": "p14r",
 "P5_entry_earned": true,
 "P5_entry_flags": {
   "quantum_action_defined": true,
   "quantum_prescription_defined": true,
   "D11_matching_defined": true,
   "counterterm_domain_defined": true,
   "amplitude_rules_verified": true
 },
 "P3_P4_frozen_as_P5_inputs": true,
 "P5_entry_dependency": "P.5 entry is stated against the P.3/P.4 freeze: P3_P4_frozen_as_P5_inputs = True (p1
 +> 3 independent rebuild passed, bound to the p13 tree; the p13 independent rebuild ran on the SAME box (sha
 +> red kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it was NOT run on a second machi
 +> ne)",
 "release_claim_accepted": true,
 "P5_entry_qualification": "P.5 entry = the contract is fixed, nothing is evaluated. Qualifications (p14r): t
 +> he DRED prescription is operational at the level of index algebra, lifting, ghat-projected tensor reducti
 +> on and the order contraction -> Laurent (tested on generic placeholders and applied to the lifted exporte
 +> d integrands), but no scalar integral is evaluated and the scaleless-integral rules are not applied; D11
 +> M3 is conditional on M2 and its degree-4 form on the imposed normalization condition N4; the counterterm
 +> count 8 is a count of candidate directions modulo the computed exact equivalences, before the identity co
 +> nstraints (D cap ker sigma_0 not computed); D11_matching_verified = false; D12 Missing.",
 "classical_readiness_for_ch_identity": true,
 "scope": "P.5 entry: the quantum object, ONE prescription with the D11 matching note, the one-loop counterte
```

```
  +> rm domain and executable amplitude rules for the fixed one-loop kappa^2 ch identity on the S_inv baseline
  +> ; integrands only; NO loop evaluation; NOT quantum inputs, NOT the projected identity, NOT bulk restorati
  +> on, NOT the prepared S_D, NOT the insertion",
  "quantum_inputs_evaluated": "not_earned",
  "projected_defect_computed": "not_earned",
  "projected_identity_verified": "not_earned",
  "projected_restoration_verified": "not_earned",
  "bulk_symmetry_restored": false,
  "D11_matching_verified": false,
  "D12_status": "Missing",
  "O1_closure": "NOT_CLOSED",
  "release_certificate": "certificates/P14_release.json",
  "hashes_of_release_files": "external manifest only (RELEASE_MANIFEST_EXTERNAL.txt)"
}
```


65 P.5 entry dossier (machine-readable)

[listing 32/133] certificates/P14_dossier.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "document": "P.5 entry dossier (with the frozen P.3/P.4 chains)",
  "status_of_this_document": "package evidence for XVI-P (unregistered; XVI-O conv:package): an archive, a lis
  +> ting and a manifest are not a series certificate. Does not update XVI-O (prop:kappa2-master stays Partial
  +>  there; symmetry_restored false; O.1 NOT_CLOSED).",
  "milestone_definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "milestone_status": {
    "definitions": {
      "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (comple
  +> te for the declared ch target)",
      "P.4": "required classical action, transformations, source terms, gauge map and master verification (cla
  +> ssical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
      "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
      "P.5_calculation": "coefficients evaluated and renormalized",
      "P.5_identity": "projected defect computed and, if possible, cancelled"
    },
    "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
    "status": {
      "P.3": "complete (declared ch target)",
      "P.4": "earned (classical readiness)",
      "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
      "P.5_calculation": "open (not started; Batch A/B/C integrals are p15+)",
      "P.5_identity": "open"
    },
    "P3_complete": true,
    "P4_classical_readiness_earned": true,
    "P5_entry_earned": true,
    "P5_entry_rule": "P.5 entry is earned iff all five contract flags hold (and is stated against the P.3/P.4
  +> freeze)",
    "P5_entry_flags": {
      "quantum_action_defined": true,
      "quantum_prescription_defined": true,
      "D11_matching_defined": true,
      "counterterm_domain_defined": true,
      "amplitude_rules_verified": true
    },
    "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not ear
  +> ned)",
    "P3_P4_frozen_as_P5_inputs": true,
    "frozen_rule": "P.3/P.4 are frozen P.5 inputs iff the p13 independent rebuild passed (bound to the p13 tre
  +> e) AND the frozen code/values are unchanged",
    "P5_entry_depends_on": "P3_P4_frozen_as_P5_inputs (p13 independent rebuild; caveat: the p13 independent re
  +> build ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it
  +>  was NOT run on a second machine)",
    "history": "archival/p02_p09/notes/P_scope.tex (p06-p08 'P-next.N' labels; its table row 'P4 coupled one-l
  +> oop restoration' is SUPERSEDED: P.4 = classical readiness); archival/p12/ (p12 used 'classical readiness'
  +>  without milestone labels); archival/p13/ (p13 release: P.3 complete, P.4 earned, P.5 entry next)",
    "XVI_O_unaffected": "XVI-P is unregistered package evidence; XVI-O prop:kappa2-master stays Partial"
  },
```

```
"chains": [
  {
    "milestone": "P.3",
    "chain": "projection -> dependency rows -> graphs [frozen P.3/P.4 input]",
    "rows": [
      {
        "id": "P3.1",
        "claim": "ch output projection fixed (c^1 h^1, ghost number 1, kappa^2, one loop, transformed variab
+> les)",
        "file": "code/p4_classical/ch_source_audit.py",
        "functions": ["main"],
        "checks": [
          {
            "source": "certificates/P13_ch_source_audit.json",
            "key": "/ch_projection_defined",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_ch_source_audit.json",
            "key": "/ch_pattern/ghost_number",
            "expect": 1,
            "value": "1",
            "ok": true
          }
        ],
        "tests": ["test_ch_rows_and_quantum_input_slots"],
        "functions_found": {
          "main": true
        },
        "tests_found": {
          "test_ch_rows_and_quantum_input_slots": true
        },
        "resolved": true
      },
      {
        "id": "P3.2",
        "claim": "vertex inventory generated from S_Psi (not copied)",
        "file": "code/p4_classical/ch_source_audit.py",
        "functions": ["S_psi_inventory", "vertex_options"],
        "checks": [
          {
            "source": "certificates/P13_ch_source_audit.json",
            "key": "/vertex_inventory",
            "expect": "NONEMPTY",
            "value": "{\"kappa0\": [\"K.c\", \"b.cbs\", \"b.h\", \"b^2\", \"c....",
            "ok": true
          },
          {
            "source": "certificates/P13_ch_source_audit.json",
            "key": "/checks/b_has_no_interaction_vertex",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_ch_rows_and_quantum_input_slots"],
        "functions_found": {
          "S_psi_inventory": true,
          "vertex_options": true
        },
        "tests_found": {
          "test_ch_rows_and_quantum_input_slots": true
        },
        "resolved": true
      },
      {
        "id": "P3.3",
        "claim": "dependency rows exhaustive: regenerated rows equal the published rows",
        "file": "code/p4_classical/ch_source_audit.py",
        "functions": ["audit", "regenerate_and_compare"],
        "checks": [
          {
            "source": "certificates/P13_ch_source_audit.json",
            "key": "/checks/regenerated_rows_match_published",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
```

```
      "source": "certificates/P13_ch_source_audit.json",
      "key": "/regeneration/complete",
      "expect": true,
      "value": "true",
      "ok": true
    }
  ],
  "tests": ["test_e2e_dropped_hstar_c_row"],
  "functions_found": {
    "audit": true,
    "regenerate_and_compare": true
  },
  "tests_found": {
    "test_e2e_dropped_hstar_c_row": true
  },
  "resolved": true
},
{
  "id": "P3.4",
  "claim": "6 rows: 3 surviving, 3 derived zero (S2, cbar*, b* rows)",
  "file": "code/p4_classical/ch_source_audit.py",
  "functions": ["audit"],
  "checks": [
    {
      "source": "certificates/P13_ch_source_audit.json",
      "key": "/rows_total",
      "expect": 6,
      "value": "6",
      "ok": true
    },
    {
      "source": "certificates/P13_ch_source_audit.json",
      "key": "/rows_surviving",
      "expect": 3,
      "value": "3",
      "ok": true
    },
    {
      "source": "certificates/P13_ch_source_audit.json",
      "key": "/rows_derived_zero",
      "expect": 3,
      "value": "3",
      "ok": true
    }
  ],
  "tests": ["test_ch_rows_and_quantum_input_slots"],
  "functions_found": {
    "audit": true
  },
  "tests_found": {
    "test_ch_rows_and_quantum_input_slots": true
  },
  "resolved": true
},
{
  "id": "P3.5",
  "claim": "enumeration lemma (justifies max_len = 2) checked mechanically",
  "file": "code/p4_classical/ch_source_audit.py",
  "functions": ["enumeration_lemma"],
  "checks": [
    {
      "source": "certificates/P13_ch_source_audit.json",
      "key": "/enumeration_lemma/holds",
      "expect": true,
      "value": "true",
      "ok": true
    }
  ],
  "tests": ["test_enumeration_lemma"],
  "functions_found": {
    "enumeration_lemma": true
  },
  "tests_found": {
    "test_enumeration_lemma": true
  },
  "resolved": true
},
{
  "id": "P3.6",
  "claim": "graph topologies generated and named; no unnamed graph",
  "file": "code/p4_classical/ch_source_audit.py",
```

```
      "functions": ["enumerate_graphs", "name_graph"],
      "checks": [
        {
          "source": "ledgers/P13_graph_inventory.json",
          "key": "/names_TBD",
          "expect": [],
          "value": "[]",
          "ok": true
        },
        {
          "source": "ledgers/P13_graph_inventory.json",
          "key": "/graphs",
          "expect": "NONEMPTY",
          "value": "{\"D03_hh_ghost_loop\": {\"external\": \"h^2\", \"ka...",
          "ok": true
        },
        {
          "source": "certificates/P13_ch_source_audit.json",
          "key": "/checks/names_TBD_empty",
          "expect": true,
          "value": "true",
          "ok": true
        }
      ],
      "tests": ["test_ch_rows_and_quantum_input_slots"],
      "functions_found": {
        "enumerate_graphs": true,
        "name_graph": true
      },
      "tests_found": {
        "test_ch_rows_and_quantum_input_slots": true
      },
      "resolved": true
    },
    {
      "id": "P3.7",
      "claim": "P.3 flags",
      "file": "code/p5_entry/release_p14.py",
      "functions": ["classical_block"],
      "checks": [
        {
          "source": "RELEASE_STATUS.json",
          "key": "/classical/flags/ch_projection_defined",
          "expect": true,
          "value": "true",
          "ok": true
        },
        {
          "source": "RELEASE_STATUS.json",
          "key": "/classical/flags/ch_dependency_inventory_complete",
          "expect": true,
          "value": "true",
          "ok": true
        }
      ],
      "tests": ["test_milestones_consistent"],
      "functions_found": {
        "classical_block": true
      },
      "tests_found": {
        "test_milestones_consistent": true
      },
      "resolved": true
    }
  ]
},
{
  "milestone": "P.4",
  "chain": "classical coefficient -> export -> verification [frozen P.3/P.4 input]",
  "rows": [
    {
      "id": "P4.A1",
      "claim": "C3 boundary identity sqrt(-g)R - L_GG = -d_mu B^mu on generic jets",
      "file": "code/classical/boundary_identity.py",
      "functions": ["generic_jet_proof"],
      "checks": [
        {
          "source": "certificates/P13_boundary_identity.json",
          "key": "/C3_proved",
          "expect": true,
          "value": "true",
```

```
      "ok": true
    }
  ],
  "tests": ["test_C3_boundary_identity_generic_and_conformal"],
  "functions_found": {
    "generic_jet_proof": true
  },
  "tests_found": {
    "test_C3_boundary_identity_generic_and_conformal": true
  },
  "resolved": true
},
{
  "id": "P4.A2",
  "claim": "ell_2, ell_3, ell_4 built and exported; export hash == quartic cert == master input",
  "file": "code/classical/quartic.py",
  "functions": ["build_and_export"],
  "checks": [
    {
      "source": "RELEASE_STATUS.json",
      "key": "/classical/hash_consistency/consistent",
      "expect": true,
      "value": "true",
      "ok": true
    },
    {
      "source": "certificates/P13_quartic_verified.json",
      "key": "/ell4_sha256",
      "expect": "NONEMPTY",
      "value": "\"33ac9e49c4d1644ee47f09b80983f4ea29827d33af93...",
      "ok": true
    }
  ],
  "tests": ["test_stale_ell4_hash_blocks_master"],
  "functions_found": {
    "build_and_export": true
  },
  "tests_found": {
    "test_stale_ell4_hash_blocks_master": true
  },
  "resolved": true
},
{
  "id": "P4.A3",
  "claim": "C1 inverse, C2 det/trace-log (10 symbolic components)",
  "file": "code/classical/quartic.py",
  "functions": ["inverse_control", "det_tracelog_generic"],
  "checks": [
    {
      "source": "certificates/P13_quartic_verified.json",
      "key": "/acceptance_controls_deterministic/C1",
      "expect": true,
      "value": "true",
      "ok": true
    },
    {
      "source": "certificates/P13_quartic_verified.json",
      "key": "/acceptance_controls_deterministic/C2",
      "expect": true,
      "value": "true",
      "ok": true
    }
  ],
  "tests": ["test_quartic_acceptance_all_deterministic"],
  "functions_found": {
    "inverse_control": true,
    "det_tracelog_generic": true
  },
  "tests_found": {
    "test_quartic_acceptance_all_deterministic": true
  },
  "resolved": true
},
{
  "id": "P4.A4",
  "claim": "C4 export == independent evaluator (exact polynomial identity)",
  "file": "code/classical/quartic.py",
  "functions": ["export_identity"],
  "checks": [
    {
      "source": "certificates/P13_quartic_verified.json",
```

```
            "key": "/acceptance_controls_deterministic/C4",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_quartic_acceptance_all_deterministic"],
        "functions_found": {
          "export_identity": true
        },
        "tests_found": {
          "test_quartic_acceptance_all_deterministic": true
        },
        "resolved": true
      },
      {
        "id": "P4.A5",
        "claim": "C5/C6 ell_3 = XII-C, ell_2 = XI-A Fierz-Pauli modulo divergences (pinned XVI-N grav_densit
+> y)",
        "file": "code/classical/quartic.py",
        "functions": ["regression_deterministic", "xvin_symbolic"],
        "checks": [
          {
            "source": "certificates/P13_quartic_verified.json",
            "key": "/acceptance_controls_deterministic/C5",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_quartic_verified.json",
            "key": "/acceptance_controls_deterministic/C6",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_quartic_acceptance_all_deterministic"],
        "functions_found": {
          "regression_deterministic": true,
          "xvin_symbolic": true
        },
        "tests_found": {
          "test_quartic_acceptance_all_deterministic": true
        },
        "resolved": true
      },
      {
        "id": "P4.A6",
        "claim": "C7 gravitational Noether identities on the exports, kappa^0..kappa^2",
        "file": "code/classical/grav_noether.py",
        "functions": ["noether"],
        "checks": [
          {
            "source": "certificates/P13_quartic_verified.json",
            "key": "/controls/C7_accepted_orders",
            "expect": "ALL_TRUE",
            "value": "{\"kappa0\": true, \"kappa1\": true, \"kappa2\": true}",
            "ok": true
          }
        ],
        "tests": ["test_grav_noether_exact_and_mutation"],
        "functions_found": {
          "noether": true
        },
        "tests_found": {
          "test_grav_noether_exact_and_mutation": true
        },
        "resolved": true
      },
      {
        "id": "P4.A7",
        "claim": "quartic mutations rejected; quartic flag",
        "file": "code/classical/quartic.py",
        "functions": ["main"],
        "checks": [
          {
            "source": "certificates/P13_quartic_verified.json",
            "key": "/mutations_all_rejected",
            "expect": true,
            "value": "true",
```

```
          "ok": true
        },
        {
          "source": "RELEASE_STATUS.json",
          "key": "/classical/flags/quartic_generator_verified",
          "expect": true,
          "value": "true",
          "ok": true
        }
      ],
      "tests": ["test_e2e_corrupted_ell4_coefficient"],
      "functions_found": {
        "main": true
      },
      "tests_found": {
        "test_e2e_corrupted_ell4_coefficient": true
      },
      "resolved": true
    }
  ]
},
{
  "milestone": "P.4",
  "chain": "transformations -> source terms [frozen P.3/P.4 input]",
  "rows": [
    {
      "id": "P4.B1",
      "claim": "ghost law derived from s^2 = 0 (unique lambda = 1)",
      "file": "code/classical/brst_from_geometry.py",
      "functions": ["solve_ghost_law"],
      "checks": [
        {
          "source": "certificates/P13_brst_derivation.json",
          "key": "/ghost_law_derivation/lam_solutions",
          "expect": ["1"],
          "value": "[\"1\"]",
          "ok": true
        }
      ],
      "tests": ["test_brst_and_source_terms_derived"],
      "functions_found": {
        "solve_ghost_law": true
      },
      "tests_found": {
        "test_brst_and_source_terms_derived": true
      },
      "resolved": true
    },
    {
      "id": "P4.B2",
      "claim": "s_2 derived zero in every sector; s nilpotent at all orders",
      "file": "code/classical/brst_from_geometry.py",
      "functions": ["per_sector_expansion", "nilpotency"],
      "checks": [
        {
          "source": "certificates/P13_brst_derivation.json",
          "key": "/s2_per_sector",
          "expect": "ALL_EQ:derived_zero",
          "value": "{\"h\": \"derived_zero\", \"c\": \"derived_zero\", \"p...",
          "ok": true
        },
        {
          "source": "certificates/P13_brst_derivation.json",
          "key": "/nilpotency_exact_all_orders",
          "expect": true,
          "value": "true",
          "ok": true
        }
      ],
      "tests": ["test_brst_and_source_terms_derived"],
      "functions_found": {
        "per_sector_expansion": true,
        "nilpotency": true
      },
      "tests_found": {
        "test_brst_and_source_terms_derived": true
      },
      "resolved": true
    },
    {
      "id": "P4.B3",
```

```
        "claim": "sigma_A derived from the generator rule (unique); antifield-linear, no kappa^2 X terms",
        "file": "code/classical/source_action.py",
        "functions": ["derive_sigma", "build_X_inv"],
        "checks": [
          {
            "source": "certificates/P13_source_action.json",
            "key": "/sigma_unique",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_source_action.json",
            "key": "/X_kappa2_terms",
            "expect": 0,
            "value": "0",
            "ok": true
          },
          {
            "source": "certificates/P13_source_action.json",
            "key": "/antifield_linear",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_brst_and_source_terms_derived", "test_e2e_flipped_odd_sign", "test_e2e_omitted_cstar
+> _term"],
        "functions_found": {
          "derive_sigma": true,
          "build_X_inv": true
        },
        "tests_found": {
          "test_brst_and_source_terms_derived": true,
          "test_e2e_flipped_odd_sign": true,
          "test_e2e_omitted_cstar_term": true
        },
        "resolved": true
      }
    ]
  },
  {
    "milestone": "P.4",
    "chain": "gauge map -> Hessians -> source convention [frozen P.3/P.4 input]",
    "rows": [
      {
        "id": "P4.C1",
        "claim": "antifield shift orientation derived (eps = -1); S_Psi = S + s Psi mod d",
        "file": "code/classical/gauge_fixing_map.py",
        "functions": ["build", "shift_map"],
        "checks": [
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/derived_eps",
            "expect": -1,
            "value": "-1",
            "ok": true
          },
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/S_Psi_equals_S_plus_sPsi_mod_d",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_e2e_altered_gauge_shift"],
        "functions_found": {
          "build": true,
          "shift_map": true
        },
        "tests_found": {
          "test_e2e_altered_gauge_shift": true
        },
        "resolved": true
      },
      {
        "id": "P4.C2",
        "claim": "canonicity of the map on all 299 (anti)field pairs",
        "file": "code/classical/gauge_fixing_map.py",
        "functions": ["canonicity"],
```

```
        "checks": [
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/canonicity/pass",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/canonicity/pairs_tested",
            "expect": 299,
            "value": "299",
            "ok": true
          }
        ],
        "tests": ["test_gauge_hessian_generic_and_full_canonicity"],
        "functions_found": {
          "canonicity": true
        },
        "tests_found": {
          "test_gauge_hessian_generic_and_full_canonicity": true
        },
        "resolved": true
      },
      {
        "id": "P4.C3",
        "claim": "graviton Hessian at generic symbolic k: Schur complement k^2 P, propagator (k^2 P)^-1",
        "file": "code/classical/gauge_fixing_map.py",
        "functions": ["hessian_check"],
        "checks": [
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/propagators/graviton/schur_equals_k2P",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/propagators/graviton/propagator_hh_equals_inv_k2P",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_gauge_hessian_generic_and_full_canonicity"],
        "functions_found": {
          "hessian_check": true
        },
        "tests_found": {
          "test_gauge_hessian_generic_and_full_canonicity": true
        },
        "resolved": true
      },
      {
        "id": "P4.C4",
        "claim": "ghost (i cbar Box c) and scalar (-Box) kinetic terms; propagators match transformed Hessia
+> ns",
        "file": "code/classical/gauge_fixing_map.py",
        "functions": ["ghost_kinetic_check"],
        "checks": [
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/propagators/ghost/match",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/propagators/scalar/match",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/propagators_match_transformed_Hessians",
            "expect": true,
            "value": "true",
```

```
            "ok": true
          }
        ],
        "tests": ["test_gauge_hessian_generic_and_full_canonicity"],
        "functions_found": {
          "ghost_kinetic_check": true
        },
        "tests_found": {
          "test_gauge_hessian_generic_and_full_canonicity": true
        },
        "resolved": true
      },
      {
        "id": "P4.C5",
        "claim": "source convention Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi): classical pullbac
+> k, NOT a quantum theorem",
        "file": "code/classical/gauge_fixing_map.py",
        "functions": ["build"],
        "checks": [
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/source_identity_relation_defined",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P13_gauge_fixing_map.json",
            "key": "/source_identity_relation/status",
            "expect": "CONTAINS:NOT an independently established quantum",
            "value": "\"DEFINED classical pullback/interface along t...",
            "ok": true
          }
        ],
        "tests": ["test_gauge_hessian_generic_and_full_canonicity"],
        "functions_found": {
          "build": true
        },
        "tests_found": {
          "test_gauge_hessian_generic_and_full_canonicity": true
        },
        "resolved": true
      },
      {
        "id": "P4.C6",
        "claim": "gauge-map flag",
        "file": "code/p5_entry/release_p14.py",
        "functions": ["classical_block"],
        "checks": [
          {
            "source": "RELEASE_STATUS.json",
            "key": "/classical/flags/gauge_fixing_map_constructed",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_milestones_consistent"],
        "functions_found": {
          "classical_block": true
        },
        "tests_found": {
          "test_milestones_consistent": true
        },
        "resolved": true
      }
    ]
  },
  {
    "milestone": "P.4",
    "chain": "master component -> residual record -> acceptance gate [frozen P.3/P.4 input]",
    "rows": [
      {
        "id": "P4.D1",
        "claim": "master equation at kappa^0: every antifield-number component; grav af0 record; gate",
        "file": "code/classical/master_kappa2.py",
        "functions": ["run", "XX_by_order", "grav_deterministic"],
        "checks": [
          {
            "source": "certificates/P13_master_kappa2.json",
            "key": "/table_by_kappa_and_antifield_number/kappa0",
```

```
      "expect": "ALL_TRUE",
      "value": "{\"af0_grav_Noether\": true, \"af0_scalar_Noethe...",
      "ok": true
    },
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/details/grav_af0_deterministic/kappa0/Q_terms",
      "expect": [0, 0, 0, 0],
      "value": "[0, 0, 0, 0]",
      "ok": true
    },
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/details/grav_af0_deterministic/kappa0/Q_identically_zero",
      "expect": true,
      "value": "true",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/classical/residual_records/master_grav_per_order/kappa0/accepted",
      "expect": true,
      "value": "true",
      "ok": true
    }
  ],
  "tests": ["test_residual_records_regenerated_and_agree"],
  "functions_found": {
    "run": true,
    "XX_by_order": true,
    "grav_deterministic": true
  },
  "tests_found": {
    "test_residual_records_regenerated_and_agree": true
  },
  "resolved": true
},
{
  "id": "P4.D2",
  "claim": "master equation at kappa^1: every antifield-number component; grav af0 record; gate",
  "file": "code/classical/master_kappa2.py",
  "functions": ["run", "XX_by_order", "grav_deterministic"],
  "checks": [
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/table_by_kappa_and_antifield_number/kappa1",
      "expect": "ALL_TRUE",
      "value": "{\"af0_grav_Noether\": true, \"af0_scalar_Noethe...",
      "ok": true
    },
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/details/grav_af0_deterministic/kappa1/Q_terms",
      "expect": [0, 0, 0, 0],
      "value": "[0, 0, 0, 0]",
      "ok": true
    },
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/details/grav_af0_deterministic/kappa1/Q_identically_zero",
      "expect": true,
      "value": "true",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/classical/residual_records/master_grav_per_order/kappa1/accepted",
      "expect": true,
      "value": "true",
      "ok": true
    }
  ],
  "tests": ["test_residual_records_regenerated_and_agree"],
  "functions_found": {
    "run": true,
    "XX_by_order": true,
    "grav_deterministic": true
  },
  "tests_found": {
    "test_residual_records_regenerated_and_agree": true
  },
```

```
  "resolved": true
},
{
  "id": "P4.D3",
  "claim": "master equation at kappa^2: every antifield-number component; grav af0 record; gate",
  "file": "code/classical/master_kappa2.py",
  "functions": ["run", "XX_by_order", "grav_deterministic"],
  "checks": [
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/table_by_kappa_and_antifield_number/kappa2",
      "expect": "ALL_TRUE",
      "value": "{\"af0_grav_Noether\": true, \"af0_scalar_Noethe...",
      "ok": true
    },
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/details/grav_af0_deterministic/kappa2/Q_terms",
      "expect": [0, 0, 0, 0],
      "value": "[0, 0, 0, 0]",
      "ok": true
    },
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/details/grav_af0_deterministic/kappa2/Q_identically_zero",
      "expect": true,
      "value": "true",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/classical/residual_records/master_grav_per_order/kappa2/accepted",
      "expect": true,
      "value": "true",
      "ok": true
    }
  ],
  "tests": ["test_residual_records_regenerated_and_agree"],
  "functions_found": {
    "run": true,
    "XX_by_order": true,
    "grav_deterministic": true
  },
  "tests_found": {
    "test_residual_records_regenerated_and_agree": true
  },
  "resolved": true
},
{
  "id": "P4.D4",
  "claim": "no required master residual; no unconditional pass field",
  "file": "code/classical/master_kappa2.py",
  "functions": ["master_residuals"],
  "checks": [
    {
      "source": "certificates/P13_master_kappa2.json",
      "key": "/required_master_residuals",
      "expect": [],
      "value": "[]",
      "ok": true
    }
  ],
  "tests": ["test_master_no_unconditional_pass_and_deterministic"],
  "functions_found": {
    "master_residuals": true
  },
  "tests_found": {
    "test_master_no_unconditional_pass_and_deterministic": true
  },
  "resolved": true
},
{
  "id": "P4.D5",
  "claim": "count/flag invariant on every record; quartic and master records agree",
  "file": "code/classical/certio.py",
  "functions": ["invariant_violations", "cross_check"],
  "checks": [
    {
      "source": "RELEASE_STATUS.json",
      "key": "/classical/deterministic_prerequisites/residual_record_invariants_all_certificates",
      "expect": true,
```

```
            "value": "true",
            "ok": true
          },
          {
            "source": "RELEASE_STATUS.json",
            "key": "/classical/residual_records/cross_certificate/agree",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_certificate_mutation_rejects_publication", "test_certio_rejects_inconsistent_record_
+> before_serialization"],
        "functions_found": {
          "invariant_violations": true,
          "cross_check": true
        },
        "tests_found": {
          "test_certificate_mutation_rejects_publication": true,
          "test_certio_rejects_inconsistent_record_before_serialization": true
        },
        "resolved": true
      },
      {
        "id": "P4.D6",
        "claim": "acceptance gate: release_claim_accepted and classical readiness (P.4)",
        "file": "code/p5_entry/release_p14.py",
        "functions": ["classical_block"],
        "checks": [
          {
            "source": "RELEASE_STATUS.json",
            "key": "/classical/release_claim_accepted",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "RELEASE_STATUS.json",
            "key": "/classical/classical_readiness_for_ch_identity",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "RELEASE_STATUS.json",
            "key": "/classical/flags/required_classical_BV_components_verified",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_release_accepted_and_hard_stops"],
        "functions_found": {
          "classical_block": true
        },
        "tests_found": {
          "test_release_accepted_and_hard_stops": true
        },
        "resolved": true
      },
      {
        "id": "P4.D7",
        "claim": "publication gate re-invoked before every pdf/zip/manifest stage, also on resumed builds",
        "file": "tools/build.py",
        "functions": ["publication_gate", "pdf_readback"],
        "checks": [],
        "tests": ["test_resumed_stage_build_blocked"],
        "functions_found": {
          "publication_gate": true,
          "pdf_readback": true
        },
        "tests_found": {
          "test_resumed_stage_build_blocked": true
        },
        "resolved": true
      }
    ]
  },
  {
    "milestone": "P.5_entry",
    "chain": "quantum object -> prescription/D11 matching -> counterterm domain -> amplitude rules -> gate",
```

```
    "rows": [
      {
        "id": "P5E.1",
        "claim": "quantum object in transformed variables: S_Psi parts, eps = -1, b retained (elimination ma
+> kes S quadratic in cbar*'), Gamma_1 independent of cbar*', b*'; pullback != quantum theorem",
        "file": "code/p5_entry/quantum_object.py",
        "functions": ["main", "patterns", "b_elimination"],
        "checks": [
          {
            "source": "certificates/P14_quantum_object.json",
            "key": "/quantum_action_defined",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_quantum_object.json",
            "key": "/Q1_transformed_variables/eps_derived",
            "expect": -1,
            "value": "-1",
            "ok": true
          },
          {
            "source": "certificates/P14_quantum_object.json",
            "key": "/Q2_fields_antifields_b/b_decision",
            "expect": "RETAINED and integrated",
            "value": "\"RETAINED and integrated\"",
            "ok": true
          },
          {
            "source": "certificates/P14_quantum_object.json",
            "key": "/Q2_fields_antifields_b/Gamma1_independent_of",
            "expect": ["bs", "cbs"],
            "value": "[\"bs\", \"cbs\"]",
            "ok": true
          },
          {
            "source": "certificates/P14_quantum_object.json",
            "key": "/Q5_kappa_loop_bookkeeping/ok",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_quantum_object.json",
            "key": "/Q6_pullback_vs_theorem/quantum_gauge_change_theorem_asserted",
            "expect": false,
            "value": "false",
            "ok": true
          }
        ],
        "tests": ["test_quantum_object"],
        "functions_found": {
          "main": true,
          "patterns": true,
          "b_elimination": true
        },
        "tests_found": {
          "test_quantum_object": true
        },
        "resolved": true
      },
      {
        "id": "P5E.2",
        "claim": "ONE prescription: operational DRED (rules R1-R6), MS-bar, Feynman i0; scaleless rules defi
+> ned but not applied; quantum_prescription_defined requires the verified DRED rules",
        "file": "code/p5_entry/prescription_d11.py",
        "functions": ["main"],
        "checks": [
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/quantum_prescription_defined",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/prescription/id",
            "expect": "P14_DRED_MSbar_FeynmanI0",
            "value": "\"P14_DRED_MSbar_FeynmanI0\"",
```

```
            "ok": true
          },
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/prescription_checks",
            "expect": "ALL_TRUE",
            "value": "{\"propagators_are_verified_Hessian_inverses\":...",
            "ok": true
          },
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/prescription_checks/operational_DRED_rules_verified",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/D12_status",
            "expect": "Missing",
            "value": "\"Missing\"",
            "ok": true
          }
        ],
        "tests": ["test_prescription_and_D11_matching"],
        "functions_found": {
          "main": true
        },
        "tests_found": {
          "test_prescription_and_D11_matching": true
        },
        "resolved": true
      },
      {
        "id": "P5E.2b",
        "claim": "operational DRED: g4 = ghat + gtilde (tr d, 4-d), l and p in QdS, l_r = l.e_(r) lifting, g
+> hat-projected Gram/PV reduction, contractions before Laurent with d symbolic; evanescent parts retained;
+> premature d=4, tr ghat=4 and expand-before-contract are rejected",
        "file": "code/p5_entry/dred.py",
        "functions": ["reduce_tensor", "eta_trace_Gt", "toy_four_d_part", "toy_evanescent_trace", "known_red
+> uctions", "lift_exported", "main"],
        "checks": [
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/dred_rules_verified",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/mutations_all_rejected",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/mutations/premature_d4/rejected",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/mutations/trace_ghat_eq_4/rejected",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/mutations/contract_after_laurent/rejected",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/toy_checks/T1_four_d_part/finite",
            "expect": "a/8 + b/4",
```

```
            "value": "\"a/8 + b/4\"",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/toy_checks/T2_evanescent_trace/finite",
            "expect": "-a/2",
            "value": "\"-a/2\"",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/known_reductions/pass",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/evanescent_parts_present_in_exports",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_dred_rules.json",
            "key": "/loop_evaluations_performed",
            "expect": false,
            "value": "false",
            "ok": true
          }
        ],
        "tests": ["test_dred_rules_certificate", "test_dred_premature_d4_changes_finite_part_and_is_caught",
+>  "test_dred_evanescent_trace_is_d_minus_4_times_pole", "test_dred_trace_ghat_eq_4_rejected", "test_dred_k
+> nown_reductions_and_contraction_engine"],
        "functions_found": {
          "reduce_tensor": true,
          "eta_trace_Gt": true,
          "toy_four_d_part": true,
          "toy_evanescent_trace": true,
          "known_reductions": true,
          "lift_exported": true,
          "main": true
        },
        "tests_found": {
          "test_dred_rules_certificate": true,
          "test_dred_premature_d4_changes_finite_part_and_is_caught": true,
          "test_dred_evanescent_trace_is_d_minus_4_times_pole": true,
          "test_dred_trace_ghat_eq_4_rejected": true,
          "test_dred_known_reductions_and_contraction_engine": true
        },
        "resolved": true
      },
      {
        "id": "P5E.3",
        "claim": "D11 matching DEFINED (M1 exact integrand identity vs XVI-O V_tensor, 100 pairs; M3 CONDITI
+> ONAL on M2: sd(u^R) 4, kernel sd <= 8, contact derivative order <= 4, contact space orders 0/2/4 = 2/4/5,
+>  degree 4 imposed by normalization condition N4; no c delta; r = -i pi^2 XVI-O side only); NOT verified",
        "file": "code/p5_entry/prescription_d11.py",
        "functions": ["M1_identity", "M3_local_space", "M3_contact_analysis"],
        "checks": [
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/D11_matching_defined",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/D11_matching_verified",
            "expect": false,
            "value": "false",
            "ok": true
          },
          {
            "source": "certificates/P14_prescription_D11_matching.json",
            "key": "/matching/M1_integrand_identity/mismatches",
            "expect": [],
            "value": "[]",
            "ok": true
```

```
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M1_integrand_identity/component_pairs_checked",
    "expect": 100,
    "value": "100",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M1_integrand_identity/bound_to_export",
    "expect": true,
    "value": "true",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M3_local_difference/labels/scalar_extension_scaling_degree",
    "expect": 4,
    "value": "4",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M3_local_difference/labels/kernel_scaling_degree_bound",
    "expect": 8,
    "value": "8",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M3_local_difference/labels/local_contact_derivative_order_bound",
    "expect": 4,
    "value": "4",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M3_local_difference/lower_degree_contacts_excluded_by_N4/order_2_mu2",
    "expect": 4,
    "value": "4",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M3_local_difference/dimension",
    "expect": 5,
    "value": "5",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M3_local_difference/transverse_subspace_dimension",
    "expect": 2,
    "value": "2",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M4_finite_adjustments/u_R_c_delta_added_by_P",
    "expect": false,
    "value": "false",
    "ok": true
  },
  {
    "source": "certificates/P14_prescription_D11_matching.json",
    "key": "/matching/M5_residue/r",
    "expect": "-i * pi**2",
    "value": "\"-i * pi**2\"",
    "ok": true
  }
],
"tests": ["test_prescription_and_D11_matching"],
"functions_found": {
  "M1_identity": true,
  "M3_local_space": true,
  "M3_contact_analysis": true
},
"tests_found": {
  "test_prescription_and_D11_matching": true
},
```

```
        "resolved": true
      },
      {
        "id": "P5E.4",
        "claim": "counterterm domain: kappa^1 empty, kappa^2 basis 11 (hh 5, cbar c 2, K'c 3, phi phi 1) gen
+> erated mechanically; eight candidate directions modulo the computed exact equivalences, before imposing t
+> he identity constraints (no cohomological classification: D cap ker sigma_0 not computed); constraint lis
+> t, not an admission",
        "file": "code/p5_entry/counterterm_domain.py",
        "functions": ["structures", "admissible_pairs", "rank", "main"],
        "checks": [
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/counterterm_domain_defined",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/kappa1/basis_size",
            "expect": 0,
            "value": "0",
            "ok": true
          },
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/kappa2/basis_size",
            "expect": 11,
            "value": "11",
            "ok": true
          },
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/equivalences/quotient_dimension",
            "expect": 8,
            "value": "8",
            "ok": true
          },
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/equivalences/D_cap_ker_sigma0_computed",
            "expect": false,
            "value": "false",
            "ok": true
          },
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/equivalences/S0_density_check/ok",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/XIV_J_opened",
            "expect": false,
            "value": "false",
            "ok": true
          },
          {
            "source": "certificates/P14_counterterm_domain.json",
            "key": "/XV_B_opened",
            "expect": false,
            "value": "false",
            "ok": true
          }
        ],
        "tests": ["test_counterterm_domain"],
        "functions_found": {
          "structures": true,
          "admissible_pairs": true,
          "rank": true,
          "main": true
        },
        "tests_found": {
          "test_counterterm_domain": true
        },
        "resolved": true
      },
      {
```

```
        "id": "P5E.5",
        "claim": "amplitude rules: vertices (incl. ghost-graviton derivative terms, K' vertex), propagators,
+>  ordering, multiplicities, symmetry factors, signs; symbolic integrands; W1-W6 controls (W5 regenerates o
+> nly D14a-c, D11, D13; W6 regenerates D01, D02, D03, D12)",
        "file": "code/p5_entry/amplitude_rules.py",
        "functions": ["build_integrands", "verify", "W2", "W3", "W4", "W5"],
        "checks": [
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/amplitude_rules_verified",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/controls/W1_wick_berezin_shadow/pass",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/controls/W4_ward_bose/pass",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/integrands_only",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/loop_evaluations_performed",
            "expect": false,
            "value": "false",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/graphs/D12/D12_status",
            "expect": "Missing",
            "value": "\"Missing\"",
            "ok": true
          }
        ],
        "tests": ["test_amplitude_rules_and_mutations"],
        "functions_found": {
          "build_integrands": true,
          "verify": true,
          "W2": true,
          "W3": true,
          "W4": true,
          "W5": true
        },
        "tests_found": {
          "test_amplitude_rules_and_mutations": true
        },
        "resolved": true
      },
      {
        "id": "P5E.5b",
        "claim": "independent Hessian/Tr-log regeneration of D01, D02, D03, D12 from the field-dependent Hes
+> sian of S_Psi (own (h,b) inverse; not the Feynman-rule assembler) == exported integrands",
        "file": "code/p5_entry/hessian_route.py",
        "functions": ["ell_table", "graviton_inverse", "ghost_inverse", "reference", "W6", "compare_exports"
+> ],
        "checks": [
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/controls/W6_hessian_route/pass",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
```

```
            "key": "/W6_exports/all_agree",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/W6_exports/D01/n_mismatches",
            "expect": 0,
            "value": "0",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/W6_exports/D03/n_mismatches",
            "expect": 0,
            "value": "0",
            "ok": true
          }
        ],
        "tests": ["test_hessian_route_regenerates_D01_D02_D03_D12"],
        "functions_found": {
          "ell_table": true,
          "graviton_inverse": true,
          "ghost_inverse": true,
          "reference": true,
          "W6": true,
          "compare_exports": true
        },
        "tests_found": {
          "test_hessian_route_regenerates_D01_D02_D03_D12": true
        },
        "resolved": true
      },
      {
        "id": "P5E.6",
        "claim": "mutations rejected: tadpole factor, ghost-loop sign, off-diagonal multiplicity, source/gho
+> st order, ghost transport term",
        "file": "code/p5_entry/amplitude_rules.py",
        "functions": ["apply_mutations", "main"],
        "checks": [
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/mutations_all_rejected",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/mutations/omit_ghost_transport_term/rejected",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "certificates/P14_amplitude_rules.json",
            "key": "/mutations/reversed_source_ghost_order/rejected",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_amplitude_rules_and_mutations"],
        "functions_found": {
          "apply_mutations": true,
          "main": true
        },
        "tests_found": {
          "test_amplitude_rules_and_mutations": true
        },
        "resolved": true
      },
      {
        "id": "P5E.7",
        "claim": "Berezin/Wick shadow fixes symmetry factors and Grassmann signs independently",
        "file": "code/p5_entry/shadow.py",
        "functions": ["expect", "gamma1", "run_shadows"],
        "checks": [
          {
            "source": "certificates/P14_amplitude_rules.json",
```

```
            "key": "/controls/W1_wick_berezin_shadow/pass",
            "expect": true,
            "value": "true",
            "ok": true
          }
        ],
        "tests": ["test_amplitude_rules_and_mutations"],
        "functions_found": {
          "expect": true,
          "gamma1": true,
          "run_shadows": true
        },
        "tests_found": {
          "test_amplitude_rules_and_mutations": true
        },
        "resolved": true
      },
      {
        "id": "P5E.8",
        "claim": "P.3/P.4 freeze bound to the p13 tree (independent rebuild passed; same-box caveat); frozen
+>  code and values unchanged",
        "file": "code/p5_entry/freeze_p13.py",
        "functions": ["freeze", "F1_independent", "F2_frozen_code", "F3_frozen_values"],
        "checks": [
          {
            "source": "RELEASE_STATUS.json",
            "key": "/P3_P4_frozen_as_P5_inputs",
            "expect": true,
            "value": "true",
            "ok": true
          },
          {
            "source": "RELEASE_STATUS.json",
            "key": "/p13_independent_rebuild/bound_to_source_tree_sha256",
            "expect": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
            "value": "\"2b1baf335468944aa800c05eaa8b0b4df67ee8934910...",
            "ok": true
          },
          {
            "source": "RELEASE_STATUS.json",
            "key": "/p13_independent_rebuild/caveat",
            "expect": "CONTAINS:SAME box",
            "value": "\"the p13 independent rebuild ran on the SAME ...",
            "ok": true
          },
          {
            "source": "RELEASE_STATUS.json",
            "key": "/freeze/F2/changed_or_missing",
            "expect": [],
            "value": "[]",
            "ok": true
          },
          {
            "source": "RELEASE_STATUS.json",
            "key": "/freeze/F3/differences",
            "expect": [],
            "value": "[]",
            "ok": true
          }
        ],
        "tests": ["test_freeze_binding"],
        "functions_found": {
          "freeze": true,
          "F1_independent": true,
          "F2_frozen_code": true,
          "F3_frozen_values": true
        },
        "tests_found": {
          "test_freeze_binding": true
        },
        "resolved": true
      },
      {
        "id": "P5E.9",
        "claim": "P.5 entry gate: earned iff all five contract flags hold; loop/restoration flags NOT earned
+> ",
        "file": "code/p5_entry/release_p14.py",
        "functions": ["_main", "p5_entry_flags", "milestones_p14"],
        "checks": [
          {
            "source": "RELEASE_STATUS.json",
```

```
      "key": "/P5_entry_flags",
      "expect": "ALL_TRUE",
      "value": "{\"quantum_action_defined\": true, \"quantum_pre...",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/P5_entry_earned",
      "expect": true,
      "value": "true",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/hard_stops_hold",
      "expect": true,
      "value": "true",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/hard_stops/quantum_inputs_evaluated",
      "expect": "not_earned",
      "value": "\"not_earned\"",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/hard_stops/projected_identity_verified",
      "expect": "not_earned",
      "value": "\"not_earned\"",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/hard_stops/bulk_symmetry_restored",
      "expect": false,
      "value": "false",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/D11_matching_verified",
      "expect": false,
      "value": "false",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/D12_status",
      "expect": "Missing",
      "value": "\"Missing\"",
      "ok": true
    },
    {
      "source": "RELEASE_STATUS.json",
      "key": "/hard_stops/O1_closure",
      "expect": "NOT_CLOSED",
      "value": "\"NOT_CLOSED\"",
      "ok": true
    }
  ],
  "tests": ["test_p5_entry_gate_and_hard_stops"],
  "functions_found": {
    "_main": true,
    "p5_entry_flags": true,
    "milestones_p14": true
  },
  "tests_found": {
    "test_p5_entry_gate_and_hard_stops": true
  },
  "resolved": true
},
{
  "id": "P5E.10",
  "claim": "publication gate (p14) re-invoked before pdf/zip/manifest stages",
  "file": "tools/build.py",
  "functions": ["publication_gate", "pdf_readback"],
  "checks": [],
  "tests": ["test_resumed_stage_build_blocked"],
  "functions_found": {
```

```
          "publication_gate": true,
          "pdf_readback": true
        },
        "tests_found": {
          "test_resumed_stage_build_blocked": true
        },
        "resolved": true
      }
    ]
  }
],
"quantum_input_slots": [
  {
    "slot": "hh at kappa^2",
    "Gamma1_pattern": "h^2",
    "row": "(S0,Gamma1^[2]) s2 K:K.c",
    "graphs": ["D01", "D02", "D03", "D11", "D12"],
    "graph_names": ["D01_hh_bubble_cubic", "D02_hh_tadpole_quartic", "D03_hh_ghost_loop", "D11_hh_scalar_loo
+> p", "D12_hh_scalar_tadpole"],
    "status": "named dependency; NOT an evaluated coefficient",
    "graph_status": {
      "D01_hh_bubble_cubic": "listed_unevaluated",
      "D02_hh_tadpole_quartic": "vertex_available_quantum_unevaluated",
      "D03_hh_ghost_loop": "listed_unevaluated",
      "D11_hh_scalar_loop": "Constructed_kernel_under_prescription (inherited O5; not re-evaluated)",
      "D12_hh_scalar_tadpole": "Missing (D12_quantum_evaluated=false)"
    },
    "evidence": {
      "file": "code/p4_classical/ch_source_audit.py",
      "function": "audit",
      "certificate": "certificates/P13_ch_source_audit.json",
      "key": "/surviving_rows/0",
      "test": "test_ch_rows_and_quantum_input_slots"
    },
    "resolved": true
  },
  {
    "slot": "K'c at kappa^2",
    "Gamma1_pattern": "K.c",
    "row": "(S0,Gamma1^[2]) s1 h:h^2",
    "graphs": ["D13"],
    "graph_names": ["D13_Kc_ghost_graviton_loop"],
    "status": "named dependency; NOT an evaluated coefficient",
    "graph_status": {
      "D13_Kc_ghost_graviton_loop": "listed_unevaluated (generated from S_Psi; h* c row; coefficient not eva
+> luated)"
    },
    "evidence": {
      "file": "code/p4_classical/ch_source_audit.py",
      "function": "audit",
      "certificate": "certificates/P13_ch_source_audit.json",
      "key": "/surviving_rows/1",
      "test": "test_ch_rows_and_quantum_input_slots"
    },
    "resolved": true
  },
  {
    "slot": "h one-point at kappa",
    "Gamma1_pattern": "h",
    "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "graphs": ["D14a", "D14b", "D14c"],
    "graph_names": ["D14a_h_onepoint_grav_tadpole", "D14b_h_onepoint_ghost_tadpole", "D14c_h_onepoint_scalar
+> _tadpole"],
    "status": "named dependency; NOT an evaluated coefficient",
    "graph_status": {
      "D14a_h_onepoint_grav_tadpole": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12",
      "D14b_h_onepoint_ghost_tadpole": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12",
      "D14c_h_onepoint_scalar_tadpole": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12"
    },
    "evidence": {
      "file": "code/p4_classical/ch_source_audit.py",
      "function": "audit",
      "certificate": "certificates/P13_ch_source_audit.json",
      "key": "/surviving_rows/2",
      "test": "test_ch_rows_and_quantum_input_slots"
    },
    "resolved": true
  }
],
"hard_stops": {
  "quantum_inputs_evaluated": "not_earned",
```

```
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "D11_matching_verified": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "P5_entry_opened": true,
    "P5_calculation_opened": false,
    "P5_identity_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "reproducibility": {
    "author_rebuild_passed": false,
    "independent_rebuild_passed": false
  },
  "p13_independent_rebuild": {
    "independent_rebuild_passed": true,
    "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
    "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
    "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
    "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
    "comparisons": {
      "reference_values_matched": {
        "compared": 112,
        "equal": 112
      },
      "independent_comparisons": 77,
      "mismatches": 0
    },
    "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
  +> env from the source-only zip; it was NOT run on a second machine",
    "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
  },
  "P3_P4_frozen_as_P5_inputs": true,
  "all_links_resolved": true,
  "n_rows": 42,
  "n_slots": 3
}
```


66 Milestone ledger (p14)

[listing 33/133] ledgers/P14_milestones.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "ledger": "milestones (authoritative definitions + status)",
  "definitions": {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory (complete
  +>  for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification (class
  +> ical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled"
  },
  "order": ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity"],
  "status": {
    "P.3": "complete (declared ch target)",
    "P.4": "earned (classical readiness)",
    "P.5_entry": "earned (contract fixed; stated against the P.3/P.4 freeze)",
    "P.5_calculation": "open (not started; Batch A/B/C integrals are p15+)",
    "P.5_identity": "open"
  },
  "P3_complete": true,
  "P4_classical_readiness_earned": true,
  "P5_entry_earned": true,
  "P5_entry_rule": "P.5 entry is earned iff all five contract flags hold (and is stated against the P.3/P.4 fr
  +> eeze)",
```

```
  "P5_entry_flags": {
    "quantum_action_defined": true,
    "quantum_prescription_defined": true,
    "D11_matching_defined": true,
    "counterterm_domain_defined": true,
    "amplitude_rules_verified": true
  },
  "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified not earne
  +> d)",
  "P3_P4_frozen_as_P5_inputs": true,
  "frozen_rule": "P.3/P.4 are frozen P.5 inputs iff the p13 independent rebuild passed (bound to the p13 tree)
  +>  AND the frozen code/values are unchanged",
  "P5_entry_depends_on": "P3_P4_frozen_as_P5_inputs (p13 independent rebuild; caveat: the p13 independent rebu
  +> ild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it w
  +> as NOT run on a second machine)",
  "history": "archival/p02_p09/notes/P_scope.tex (p06-p08 'P-next.N' labels; its table row 'P4 coupled one-loo
  +> p restoration' is SUPERSEDED: P.4 = classical readiness); archival/p12/ (p12 used 'classical readiness' w
  +> ithout milestone labels); archival/p13/ (p13 release: P.3 complete, P.4 earned, P.5 entry next)",
  "XVI_O_unaffected": "XVI-P is unregistered package evidence; XVI-O prop:kappa2-master stays Partial",
  "author_rebuild_passed": false,
  "independent_rebuild_passed": false,
  "p13_independent_rebuild": {
    "independent_rebuild_passed": true,
    "bound_to_source_tree_sha256": "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70",
    "bound_to_source_zip_sha256": "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266",
    "report": "archival/p13/reproducibility/independent_rebuild_report_p13.json",
    "report_sha256": "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4",
    "comparisons": {
      "reference_values_matched": {
        "compared": 112,
        "equal": 112
      },
      "independent_comparisons": 77,
      "mismatches": 0
    },
    "caveat": "the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a fresh v
  +> env from the source-only zip; it was NOT run on a second machine",
    "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independent flag"
  },
  "hard_stops": {
    "quantum_inputs_evaluated": "not_earned",
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": false,
    "symmetry_restored": false,
    "D02_quantum_evaluated": false,
    "D12_quantum_evaluated": false,
    "D12_status": "Missing",
    "D11_matching_verified": false,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": false,
    "XIV_J_opened": false,
    "Route_B_discharged": false,
    "P5_entry_opened": true,
    "P5_calculation_opened": false,
    "P5_identity_opened": false,
    "loop_evaluations_performed": false,
    "u_R_c_delta_added": false
  },
  "dossier": "certificates/P14_dossier.json"
}
```


67 P.5 entry: quantum object certificate

[listing 34/133] certificates/P14_quantum_object.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "deliverable": "P.5 entry D1: quantum object",
  "quantum_action_defined": true,
  "definition": "Q1..Q6 all ok",
  "S_Psi": "sum_k kappa^{k-2} ell_k + S_phi + X(Phi, Phi*' + eps dPsi/dPhi), eps = -1",
  "Q1_transformed_variables": {
    "eps_derived": -1,
    "gauge_fixing_cert_eps": -1,
```

```
  "S_Psi_equals_S_plus_sPsi_mod_d": true,
  "NOT_S_D": true,
  "Psi": "Psi = -i int cbar_mu (2 D^mu[h] - b^mu),  D^mu[h] = d_nu h^{mu nu} - (1/2) d^mu tr h",
  "X_patterns_by_kappa": {
    "0": ["b.b", "b.cbs", "c.K", "c.cb", "h.b"],
    "1": ["c.c.cs", "c.phi.phis", "h.c.K", "h.c.cb"],
    "2": []
  },
  "S_phi_patterns_by_kappa": {
    "0": ["phi.phi"],
    "1": ["h.phi.phi"],
    "2": ["h.h.phi.phi"]
  },
  "X_terms_by_kappa_P13": {
    "0": 17,
    "1": 124,
    "2": 0,
    "3": 0
  },
  "no_switch_or_insertion_fields": true,
  "ok": true
},
"Q2_fields_antifields_b": {
  "integrated": ["h_(mn) (10 independent components)", "c^mu", "cbar_mu", "phi", "b^mu (retained)"],
  "external_antifields": {
    "K'_(mn)": "h*' (10 comps, sigma_h = +1)",
    "c*'_mu": "c*'",
    "cbar*'^mu": "cbar*'",
    "phi*'": "phi*'",
    "b*'": "absent (s b = 0)"
  },
  "external_antifields_retained_P13": ["K", "cbs", "cs0", "cs1", "cs2", "cs3", "phis"],
  "b_decision": "RETAINED and integrated",
  "b_justification": {
    "b_occurs_only_at_kappa_orders": [0],
    "b_patterns": ["b.b", "b.cbs", "h.b"],
    "no_b_interaction_vertex": true,
    "Mbb_constant_invertible": true,
    "full_h_b_inverse_verified": true,
    "schur_complement_equals_q2_P": true,
    "elimination": {
      "b_components": ["b0", "b1", "b2", "b3"],
      "b_hessian_constant": true,
      "b_hessian_det": "-16",
      "solution_b0": "-I*cbs0/2 + h00_d0/2 - h01_d1 - h02_d2 - h03_d3 + h11_d0/2 + h22_d0/2 + h33_d0/2",
      "after_elimination_terms": 144,
      "cbar_star_quadratic_terms": 4,
      "cbar_star_linear_terms": 28,
      "quadratic_example": "-cbs0**2/4",
      "reading": "eliminating b makes S_Psi QUADRATIC in cbar*' (and shifts the cbar*'-linear terms into h-d
+> ependent ones); retaining b keeps S_Psi linear in all external antifields"
    }
  },
  "loop_capable_species": ["K", "c", "cb", "cs", "h", "phi", "phis"],
  "Gamma1_independent_of": ["bs", "cbs"],
  "source_spectators": "cbar*' couples only via cbar*'.b (one quantum leg; tree, one-particle reducible thro
+> ugh the b-h mixing) and b*' is absent => Gamma_1 (1PI) is independent of cbar*', b*'; K', c*', phi*' ente
+> r Gamma_1 through K'.c.h, c*'.c.c, phi*'.c.phi",
  "ok": true
},
"Q3_1PI_vacuum_onepoint": {
  "weight": "e^{i S_Psi/hbar}",
  "Gamma": "Gamma_Psi[Phi, Phi*'] = S_Psi + hbar Gamma_1 + O(hbar^2); Legendre transform in the fields Phi =
+>  (h, c, cbar, b, phi) only; antifields Phi*' are spectators (not transformed)",
  "Gamma1": "Gamma_1 = (i/2) STr log S_Psi^{(2)} (fields only) = -i log Z|_1PI; i Gamma_1 = sum of 1PI one-l
+> oop graphs (graph i's from vertices i kappa^n V and propagators i M^{-1})",
  "gate0_consistency": {
    "Gamma_1 = -i log Z in convention sheet": true,
    "e^{iS/hbar} in convention sheet": true,
    "W1 Berezin/Wick shadow pass": true
  },
  "vacuum_normalization": "Z normalised so that Gamma_1[0] = 0 (field-independent vacuum graphs dropped)",
  "one_point_normalization": "the one-point function Gamma_1^{(1)} (graphs D14a, D14b, D14c) is RETAINED as
+> part of the object: no tadpole-cancellation condition and no background shift imposed; its value is not a
+> ssigned here (no zero is substituted)",
  "background": "fields of Gamma are the classical (mean) fields; <h> is not shifted",
  "ok": true
},
"Q4_fourier_propagators_i0": {
  "fourier": "f(x) = int d^4k/(2pi)^4 e^{ikx} f~(k), d_mu -> i k_mu, incoming momenta (Gate 0: d -> ik)",
  "eta": "diag(+,-,-,-)",
```

```
  "graviton": "<h_a h_b>(q) = i (P^{-1})_{ab} / (q^2 + i0)  (independent components; de Donder)",
  "ghost": "<c^mu(q) cbar_rho(-q)> = i (M0^{-1})_{mu rho} = -delta_{mu rho}/(q^2 + i0), M0 = -i q^2 delta",
  "scalar": "<phi phi>(q) = i/(q^2 + i0)",
  "b": "b retained: mixed (h,b) inverse verified; b-lines never enter a loop (no b vertex)",
  "i0": "Feynman: every propagator denominator q^2 + i0 (D_F = i/(k^2+i0), Gate 0)",
  "checks": {
    "Pinv_equals_deDonder": true,
    "schur_q2P": true,
    "full_h_b_inverse": true,
    "ghost_M0_q2_const": true,
    "D_F convention sheet": true,
    "d -> ik convention sheet": true,
    "propagators_match_transformed_Hessians_P13": true
  },
  "ok": true
},
"Q5_kappa_loop_bookkeeping": {
  "rule": "S_Psi = sum_n kappa^n S^(n); each vertex with n legs is O(kappa^{n-2}); one loop: Gamma_1 at kapp
+> a^k has exactly k external legs (sum over vertices of (n_v - 2) = E); hbar counts loops (one loop = hbar^
+> 1)",
  "orders": "P.5 scope: one loop, kappa^1 (one-point) and kappa^2 (two-point incl. K'c)",
  "vertices": {
    "V2_grav": {
      "legs": 2,
      "kappa_power": 0,
      "kappa_eq_legs_minus_2": true
    },
    "V3_grav": {
      "legs": 3,
      "kappa_power": 1,
      "kappa_eq_legs_minus_2": true
    },
    "V4_grav": {
      "legs": 4,
      "kappa_power": 2,
      "kappa_eq_legs_minus_2": true
    },
    "M0_ghost": {
      "legs": 2,
      "kappa_power": 0,
      "kappa_eq_legs_minus_2": true
    },
    "V_cb_c_h": {
      "legs": 3,
      "kappa_power": 1,
      "kappa_eq_legs_minus_2": true
    },
    "V_K_c": {
      "legs": 2,
      "kappa_power": 0,
      "kappa_eq_legs_minus_2": true
    },
    "V_K_c_h": {
      "legs": 3,
      "kappa_power": 1,
      "kappa_eq_legs_minus_2": true
    },
    "V_cs_c_c": {
      "legs": 3,
      "kappa_power": 1,
      "kappa_eq_legs_minus_2": true
    },
    "V_phis_c_phi": {
      "legs": 3,
      "kappa_power": 1,
      "kappa_eq_legs_minus_2": true
    },
    "V_b_h": {
      "legs": 2,
      "kappa_power": 0,
      "kappa_eq_legs_minus_2": true
    },
    "V_b_b": {
      "legs": 2,
      "kappa_power": 0,
      "kappa_eq_legs_minus_2": true
    },
    "V_cbs_b": {
      "legs": 2,
      "kappa_power": 0,
      "kappa_eq_legs_minus_2": true
```

```
  },
  "M_phi": {
    "legs": 2,
    "kappa_power": 0,
    "kappa_eq_legs_minus_2": true
  },
  "V_h_phi_phi": {
    "legs": 3,
    "kappa_power": 1,
    "kappa_eq_legs_minus_2": true
  },
  "V_hh_phi_phi": {
    "legs": 4,
    "kappa_power": 2,
    "kappa_eq_legs_minus_2": true
  }
},
"graphs": {
  "D01": {
    "vertices": ["V3_grav", "V3_grav"],
    "sum_n_minus_2": 2,
    "external_legs": 2,
    "integrand_prefactor": "kappa**2/2",
    "integrand_kappa_power": 2,
    "ok": true
  },
  "D02": {
    "vertices": ["V4_grav"],
    "sum_n_minus_2": 2,
    "external_legs": 2,
    "integrand_prefactor": "-kappa**2/2",
    "integrand_kappa_power": 2,
    "ok": true
  },
  "D03": {
    "vertices": ["V_cb_c_h", "V_cb_c_h"],
    "sum_n_minus_2": 2,
    "external_legs": 2,
    "integrand_prefactor": "-kappa**2",
    "integrand_kappa_power": 2,
    "ok": true
  },
  "D11": {
    "vertices": ["V_h_phi_phi", "V_h_phi_phi"],
    "sum_n_minus_2": 2,
    "external_legs": 2,
    "integrand_prefactor": "kappa**2/2",
    "integrand_kappa_power": 2,
    "ok": true
  },
  "D12": {
    "vertices": ["V_hh_phi_phi"],
    "sum_n_minus_2": 2,
    "external_legs": 2,
    "integrand_prefactor": "-kappa**2/2",
    "integrand_kappa_power": 2,
    "ok": true
  },
  "D13": {
    "vertices": ["V_K_c_h", "V_cb_c_h"],
    "sum_n_minus_2": 2,
    "external_legs": 2,
    "integrand_prefactor": "I*kappa**2",
    "integrand_kappa_power": 2,
    "ok": true
  },
  "D14a": {
    "vertices": ["V3_grav"],
    "sum_n_minus_2": 1,
    "external_legs": 1,
    "integrand_prefactor": "-kappa/2",
    "integrand_kappa_power": 1,
    "ok": true
  },
  "D14b": {
    "vertices": ["V_cb_c_h"],
    "sum_n_minus_2": 1,
    "external_legs": 1,
    "integrand_prefactor": "kappa",
    "integrand_kappa_power": 1,
    "ok": true
```

```
      },
      "D14c": {
        "vertices": ["V_h_phi_phi"],
        "sum_n_minus_2": 1,
        "external_legs": 1,
        "integrand_prefactor": "-kappa/2",
        "integrand_kappa_power": 1,
        "ok": true
      }
    },
    "amplitude_cert_present": true,
    "ok": true
  },
  "Q6_pullback_vs_theorem": {
    "classical_pullback": "Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' + (-1) dPsi/dPhi)",
    "classical_instance_verified": true,
    "status_P13": "DEFINED classical pullback/interface along the canonical antifield shift; NOT an independen
  +> tly established quantum gauge-change theorem (no quantum Gamma is constructed here)",
    "statement": "Gamma_Psi is DEFINED here directly in transformed variables. The relation Gamma_Psi(Phi,Phi*
  +> ') = Gamma(Phi, Phi*' - dPsi/dPhi) is used ONLY as the classical pullback (verified at tree level, P13).
  +> No quantum gauge-(in)dependence theorem is asserted or used; any such statement is a separate (p15+) obje
  +> ct.",
    "quantum_gauge_change_theorem_asserted": false,
    "ok": true
  },
  "loop_evaluations_performed": false,
  "seconds": 9.0
}
```


68 P.5 entry: prescription and D11 matching certificate

[listing 35/133] certificates/P14_prescription_D11_matching.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "deliverable": "P.5 entry D2: quantum prescription and D11 matching note",
  "quantum_prescription_defined": true,
  "D11_matching_defined": true,
  "D11_matching_verified": false,
  "D11_matching_verified_reason": "M2 (nonlocal agreement on U) and L_match require the evaluated DR D11 ampli
  +> tude (p15+); only M0, M1, M3-space, M4-M7 are established",
  "prescription": {
    "id": "P14_DRED_MSbar_FeynmanI0",
    "regularization": "dimensional reduction (DRED), operational rules R1-R6 (certificates/P14_dred_rules.json
  +> , code/p5_entry/dred.py): quasi-4D metric g4 = ghat + gtilde, tr g4 = 4, tr ghat = d, tr gtilde = 4 - d;
  +> loop momentum l and external momenta p in the d-dimensional space QdS (p.gtilde = 0); field/vertex/polari
  +> sation indices quasi-4D; int d^d l/(2 pi)^d with mu^{2 eps}",
    "lifting": "exported numerators are polynomials in lower components l_r, p_r at fixed external indices; l_
  +> r := l.e_(r) (e_(r) the eta-orthonormal basis of Q4S), p_r = p.e_(r); vertex tensors lifted with eta -> g
  +> 4",
    "subtraction": "MS-bar: subtract the pole in the combination 1/eps - gamma_E + log 4 pi; scale mu",
    "iO": "Feynman: every propagator denominator q^2 + i0 (eta = +,-,-,-); Wick rotation l0 = i l4 implied",
    "tensor_reduction": "Gram/Passarino-Veltman projection on symmetric ghat/p tensors with symbolic d; free q
  +> uasi-4D indices are projected with ghat_rs = eta_rs - Gt_rs (Gt_rs evanescent, eta^{rs} Gt_rs = 4-d, Gt_r
  +> s p^s = 0)",
    "order_of_operations": "ALL index contractions (incl. the external 4D projection) BEFORE the Laurent expan
  +> sion; d symbolic through MS-bar subtraction and finite-part extraction; (d-4) x pole terms retained; prem
  +> ature d = 4 is detected (PrematureD4Substitution)",
    "evanescent": "evanescent remainders DO arise (Gt structures in every exported graph; (4-d) x pole finite
  +> terms after contraction); they are retained, never dropped (the p14 'none arise' statement is withdrawn)"
  +> ,
    "literature_background_only": "Siegel PLB 84 (1979) 193, PLB 94 (1980) 37; Stockinger hep-ph/0503129; Sign
  +> er-Stockinger arXiv:0807.4424",
    "scaleless_integrals": "PART of the definition (DR value of a scaleless massless integral) but NOT APPLIED
  +>  in p14: no tadpole value is assigned; D12 stays Missing; never replaced by zero",
    "massless": true,
    "IR": "no IR regulator needed at the integrand level; IR/UV separation of scaleless integrals is deferred
  +> together with their evaluation (p15+)",
    "applies_to": ["DO1", "DO2", "DO3", "D11 (P side, matching only)", "D12", "D13", "D14a", "D14b", "D14c"],
    "consistency_with_XVI_O": "XVI-O fixes D12/one-point by dimensional_regularization_m0 (coincident prescrip
  +> tion); P's DR is the same family; P does NOT import XVI-O's prescription VALUE 0 (D12 Missing). XVI-O's D
  +> 11 extension is analytic FP in position space (NOT momentum MS); the two are related only through matchin
  +> g (M2-M4), not identified."
  },
  "prescription_checks": {
    "propagators_are_verified_Hessian_inverses": true,
    "scalar_i0_matches_Gate0": true,
```

```
  "XVI_O_coincident_prescription_is_DR_m0": true,
  "XVI_O_D11_is_NOT_momentum_MS": true,
  "no_value_imported_D12": true,
  "quantum_object_defined": true,
  "operational_DRED_rules_verified": true,
  "evanescent_remainders_retained": true,
  "all_denominators_feynman_i0": true
},
"power_counting": {
  "D01": {
    "denominators": ["l^2+i0", "(l-p)^2+i0"],
    "numerator_degree": 4,
    "superficial_degree": 4,
    "all_feynman_i0": true
  },
  "D02": {
    "denominators": ["l^2+i0"],
    "numerator_degree": 2,
    "superficial_degree": 4,
    "all_feynman_i0": true
  },
  "D03": {
    "denominators": ["(l-p)^2+i0", "l^2+i0"],
    "numerator_degree": 4,
    "superficial_degree": 4,
    "all_feynman_i0": true
  },
  "D11": {
    "denominators": ["l^2+i0", "(l-p)^2+i0"],
    "numerator_degree": 4,
    "superficial_degree": 4,
    "all_feynman_i0": true
  },
  "D12": {
    "denominators": ["l^2+i0"],
    "numerator_degree": 2,
    "superficial_degree": 4,
    "all_feynman_i0": true
  },
  "D13": {
    "denominators": ["l^2+i0", "(p-l)^2+i0"],
    "numerator_degree": 3,
    "superficial_degree": 3,
    "all_feynman_i0": true
  },
  "D14a": {
    "denominators": ["l^2+i0"],
    "numerator_degree": 2,
    "superficial_degree": 4,
    "all_feynman_i0": true
  },
  "D14b": {
    "denominators": ["l^2+i0"],
    "numerator_degree": 2,
    "superficial_degree": 4,
    "all_feynman_i0": true
  },
  "D14c": {
    "denominators": ["l^2+i0"],
    "numerator_degree": 2,
    "superficial_degree": 4,
    "all_feynman_i0": true
  }
},
"D12_status": "Missing",
"D12_value_imported": false,
"matching": {
  "baseline_M0": {
    "baseline": "S_inv (XVI-N invariant extended action), NOT S_D",
    "no_switch_fields": true,
    "a1_vertex_XVI_O": "a1 = (1/2) tau eta - h",
    "a1_P": "a1 = (1/2) tau eta - h (scalar_B_series, XI-A)",
    "NOT_S_D_P13": true,
    "pass": true
  },
  "lemmas_referenced": {
    "lem:d11-prefactor": {
      "XVI_O": "Pi = (i kappa^2/4) T",
      "status": "Constructed",
      "pass": true,
      "used_in": "M1 (prefactor i kappa^2/4 imported, compared exactly)"
```

```
    },
    "lem:analytic_FP_u_mu_R_o70": {
      "prescription_id": "analytic_FP_u_mu_R_o70",
      "pass": true,
      "u_mu_R": {
        "definition": "FP_{zeta=0}[ mu^{2 zeta} (Q-i0)^{-2+zeta} ]",
        "restricts_to": "(Q-i0)^{-2} on U",
        "sd_max": 4,
        "Lorentz_covariant": true,
        "even": true,
        "extra_c_delta": false
      },
      "used_in": "M2/M5 (XVI-O side only)"
    },
    "lem:BV_rules_on_U": {
      "status": "Constructed",
      "pass": true,
      "all_256_symbolic": true,
      "alpha": "A**2/60",
      "beta": "A**2/18",
      "used_in": "M2 (XVI-O's Pi_tr^R on U is the matching target)"
    }
  },
  "M1_integrand_identity": {
    "identity": "-i * prefactor_P * N^P_{AB}(l,p)/(D1 D2)  ==  (i kappa^2/4) R_A R_B V V [C~(l)C~(l-p) + C~(
+> l)C~(l-p)]_{AB}",
    "D1": "l^2 + i0",
    "D2": "(l-p)^2 + i0",
    "P_prefactor": "kappa**2/2",
    "P_routing": {
      "A": ["h_a: p", "phi: -l", "phi: l-p"],
      "B": ["h_b: -p", "phi: l", "phi: p-l"],
      "propagators": ["<phi phi>(l)", "<phi phi>(l-p)"]
    },
    "XVI_O_V_tensor": "code/matter_loop/d11_tensor.py::V_tensor (frozen, imported)",
    "XVI_O_prefactor": "i kappa^2/4 (O5_Gate2_normalized_D11)",
    "C_tilde": "C~_ab(k) = k_a k_b * i/(k^2+i0)  (FT of C = -d_a d_b D_F, d -> ik)",
    "raising_map": "h^{mu nu} = eta^{mu a} eta^{nu b} h_ab; d/dh_(ab) carries orbit multiplicity 2 for a !=
+> b",
    "normalisation": "Gamma_1^(2) = (1/2) int h^{mu nu} Pi_{mu nu rho sigma} h^{rho sigma} (XVI-O) <-> Gamma
+> _1^{AB} = d^2 Gamma_1/dh_A dh_B (P ordered extraction); i Gamma_1 = graph sum",
    "component_pairs_checked": 100,
    "nonzero_pairs": 100,
    "mismatches": [],
    "recomputed_integrand_sha256": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609",
    "exported_integrand_sha256": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609",
    "bound_to_export": true,
    "pass": true,
    "seconds": 1.8
  },
  "M2_nonlocal_agreement_on_U": {
    "definition": "for all test functions f in D(U), U = R^4 \\ {0}: <FT^{-1}[P^DR_{AB}](z), f> = <R_A R_B P
+> i_tr^R(z), f>, Pi_tr^R = (i kappa^2/4) K u^R_mu (alpha = A^2/60, beta = A^2/18, A = 1/(4 pi^2)); mu-indep
+> endent on U",
    "status": "defined_not_verified",
    "requires": "evaluated DR D11 amplitude (Batch, p15+)",
    "integrand_level_evidence": "M1 (exact integrand identity) -- not a proof of M2"
  },
  "M3_local_difference": {
    "definition": "L_match := FT^{-1}[P^DR_{AB}] - R_A R_B Pi_tr^R, a distribution supported at z = 0 IF M2
+> holds",
    "labels": {
      "scalar_extension_scaling_degree": 4,
      "kernel_differential_order": 4,
      "kernel_scaling_degree_bound": 8,
      "degree_of_divergence_bound": 4,
      "local_contact_derivative_order_bound": 4,
      "contact_scaling_degree_rule": "sd(d^alpha delta) = 4 + |alpha|  (in 4 dimensions)",
      "labels_consistent": true
    },
    "wording": "Kernel scaling degree at most eight; degree of divergence at most four; hence the local ambi
+> guity has derivative order at most four.",
    "conditional_on": "M2_nonlocal_agreement_on_U (defined, NOT verified)",
    "conditional_statement": "IF M2 holds, then L_match vanishes on U = R^4 \\ {0}, hence is supported at {0
+> }; with kernel scaling degree <= 8 it is a finite sum of d^alpha delta with 4 + |alpha| <= 8, i.e. it lie
+> s in the bounded contact space of derivative order <= 4",
    "bounded_contact_space": {
      "order_0": {
        "mu_power": 4,
        "dimension": 2,
        "transverse_subspace_dimension": 0
```

```
      },
      "order_2": {
        "mu_power": 2,
        "dimension": 4,
        "transverse_subspace_dimension": 1
      },
      "order_4": {
        "mu_power": 0,
        "dimension": 5,
        "transverse_subspace_dimension": 2
      }
    },
    "odd_orders": "absent: four tensor indices built from p and eta need an even number of p's",
    "normalization_condition": "normalization condition N4 (EXPLICIT, imposed): L_match is homogeneous of de
+> gree exactly four in p, i.e. it contains no mu^4 (order-0) and no mu^2 p^2 (order-2) contact terms.  This
+>  is NOT implied by 'no other scale': mu itself is a scale, so mu^{4-|alpha|} d^alpha delta with |alpha| =
+>  0, 2 are dimensionally allowed",
    "lower_degree_contacts_excluded_by_N4": {
      "order_0_mu4": 2,
      "order_2_mu2": 4
    },
    "under_N4": "L_match lies in the order-4 (degree-4) hh span",
    "dimension": 5,
    "transverse_subspace_dimension": 2,
    "matches_counterterm_hh_sector": true,
    "XVI_O_contact_ansatz_ids": ["L0_hh", "L0_tr", "L2_Box", "L2_trBox", "L2_divdiv", "L2_divtr", "L4_Box2",
+>  "L4_trBox2", "L4_mixed"],
    "XVI_O_order4_elements": 3,
    "XVI_O_ansatz_NOT_proved_spanning": true,
    "comparison_with_XVI_O_ansatz": "XVI-O's contact ansatz is a candidate list (NOT_proved_spanning) with 3
+>  named order-4 entries, one of them the collective 'L4_mixed'; P enumerates the order-4 space explicitly:
+>  5 structures (transverse subspace 2). No contradiction; P's list is the explicit one used for M3/M4",
    "XVI_O_order_lt_4_elements_excluded_by_N4": ["L0_hh", "L0_tr", "L2_Box", "L2_trBox", "L2_divdiv", "L2_di
+> vtr"],
    "pass": true,
    "status": "conditional on M2; space enumerated; degree 4 imposed by N4; L_match not computed"
  },
  "M4_finite_adjustments": {
    "rule": "under M2 and N4, L_match (and any finite renormalization) is booked ONLY as P-side counterterms
+>  in the hh sector of P14_counterterm_domain (5 elements; transverse subspace 2 if the P amplitude is tran
+> sverse); u^R_mu is NOT modified; without N4 the order-0/2 contacts (mu^4, mu^2 p^2) would have to be adde
+> d to that sector",
    "u_R_extra_c_delta_XVI_O": false,
    "u_R_c_delta_added_by_P": false,
    "scalar_extension_prescription_preserved": true,
    "XVI_O_D11_kernel_counterterm": "earned"
  },
  "M5_residue": {
    "r": "-i * pi**2",
    "derived_from": "constructive_F_holomorphy_THEN_second_recurrence_plus_Gate0",
    "holomorphy_proof_pass": true,
    "statement": "r = -i pi^2 is XVI-O-side, valid only AFTER constructive holomorphy of F(zeta) = (Q-i0)^ze
+> ta; P's MS-bar 1/eps pole is P-side; no identification of the P pole with r is asserted in p14"
  },
  "M6_provenance": {
    "files": 24,
    "all_pinned_ok": true,
    "pins": {
      "code/matter_loop/d11_Ww_reference.py": "445b0fed527cedb2af36aab30ce08c87a88f7b2654f523dd427c993c8c590
+> f71",
      "code/matter_loop/d11_contact_moments.py": "2b6d3f28853d091e87a07a2a6ede52e9cd0da0dfdb90812006952fb2e8
+> d2f5c0",
      "code/matter_loop/d11_divergences.py": "a2cf807e6f76b2bec13acfce8a6949080261e32958fa278a6eeeb1fadc222b
+> 26",
      "code/matter_loop/d11_extension.py": "e92082154d3f68470001d9bfb4ad334eddd59d3b6021c2eb2b3c46f19f5159d5
+> ",
      "code/matter_loop/d11_global_representation.py": "fabe63f6b455d21fa697858a005af16578f94405f5673559f361
+> 3609484e99e6",
      "code/matter_loop/d11_normalized.py": "c02f97ae926269c1765c5410b4b897aa664a141978224a95af4572b4b2caeeb
+> 0",
      "code/matter_loop/d11_route1_transverse.py": "7d02ca8fa2c9fc2354a732ba9c1121e64968580bf8c393047da5d7a8
+> e85fde86",
      "code/matter_loop/d11_scalar_extension.py": "21a8836b69c2c9599450e1369e2e5d7cfce1ca9f61a6ec5a2ac87b2d7
+> 12e21d0",
      "code/matter_loop/d11_subsector_summary.py": "0e3a735a3f33d26fd5a1a0e56b3317c8ef7145f23ffe4916e7b92858
+> e60087b1",
      "code/matter_loop/d11_tensor.py": "6e916fa960449f4f187415de8b4a9a2c0aa211717de1237b5951987fe85efe77",
      "code/matter_loop/d11_ward_d12.py": "0b4fdbf1f4f5296d056a34636b9005bc34a505f35786c43d29996796ccfff21a"
+> ,
      "notes/O_scope_and_dependencies.tex": "24aa7e9e6b23edbfa9705559bcbb0e4d037bfd328a8427995d2cd74f0253b54
+> 8",
```

```
      "notes/S_inv_order2_scaffold.tex": "0a1fb17a3160fc26f411280dd30749a7322f377680f1b79e03647bbfe4ab1d41",
      "notes/counterterm_ledgers.tex": "54681d431f114c2d58e74dab3c0f8535ebe0fce8ed7611bad3447b809c59b094",
      "notes/d11_scalar_extension.tex": "d7ca98981e4e5f1c0ac0a89c35cf6a577965ac4766867f40d6a8dcdc5d852d61",
      "notes/diagram_inventory_note.tex": "5044c224d2825c88c93e830273f613d99fbdef4815d465d9a372fd79acd1ced6"
+> ,
      "notes/insertion_status_freeze.tex": "88cdebb51e068efdc1057f66d6f578bb1e4b6ac9fdd13ca0500de5f36b7a4403
+> ",
      "notes/matter_loop_subsector.tex": "3f1c74e798a5fac9f600f17480dbb6f4dc0d7adacb15872c411df1aeeb7c3e85",
      "notes/quantum_convention_sheet.tex": "044b9410ef54d478b18225585835bf57fff8f7b290f7913620e7b8c5be93e1b
+> 9",
      "notes/renormalized_amplitudes.tex": "107bc032d8b9c4d8e06f49c0427d661b5341099cd8779c1650147b9c20c61324
+> ",
      "notes/retained_quantum_identity.tex": "e89109ad9a1c619153348cfd5e5b117507fe987a021c8857f466eb949ace36
+> 56",
      "notes/symmetry_defect_note.tex": "e123af1ca785edc294a84bf3e6425e2ba46d508eeba5c4de1ba3defe0702c67a"
    }
  },
  "M7_projected_vs_bulk": {
    "XVI_O_kernel_Ward_NOT_functional_ST4b": true,
    "symmetry_restored_bulk_XVI_O": false,
    "statement": "projected (XVI-N lem:field-only; the ch projection of the fixed one-loop kappa^2 identity)
+>  is distinct from bulk BV restoration; the XVI-O kernel two-point Ward identity d(Pi_w + L) = 0 is NOT fu
+> nctional ST4b"
  },
  "conventions": {
    "XVI_O_index_convention": {
      "calculation_convention": "raised h^{mu nu}; lower kernel indices T_{mu nu rho sigma}",
      "science_volume_lowered_h": "h_{mu nu} = eta_{mu a} eta_{nu b} h^{ab}",
      "inverse": "h^{mu nu} = eta^{mu a} eta^{nu b} h_{ab}",
      "Ward_operators": "partial^mu T_{mu nu rho sigma},  partial^rho T_{mu nu rho sigma}",
      "explicit_eta_map": true
    },
    "XVI_O_Pi": "Pi = (i kappa^2/4) T,  T = V V (CC+CC)",
    "P_components": "independent lower h_(mn), m<=n",
    "map": "h^{mu nu} = eta^{mu a} eta^{nu b} h_ab; d/dh_(ab) carries orbit multiplicity 2 for a != b",
    "W_w_reference_id": "W_w_jet_pullback_via_Pi_tr_o68"
  }
},
"loop_evaluations_performed": false,
"seconds": 11.6
}
```


69 P.5 entry: counterterm domain certificate

[listing 36/133] certificates/P14_counterterm_domain.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "deliverable": "P.5 entry D3: counterterm domain",
  "kind": "order-bounded EFT constraint list (one loop, kappa^1 and kappa^2); NOT an admission",
  "XIV_J_opened": false,
  "XV_B_opened": false,
  "dimensions": {
    "h": 1,
    "c": 1,
    "cb": 1,
    "b": 2,
    "phi": 1,
    "K": 2,
    "cs": 2,
    "cbs": 2,
    "bs": 1,
    "phis": 2,
    "kappa": -1
  },
  "ghost_numbers": {
    "h": 0,
    "c": 1,
    "cb": -1,
    "b": 0,
    "phi": 0,
    "K": -1,
    "cs": -2,
    "cbs": 0,
    "bs": -1,
    "phis": -1
  },
```

```
"phi_parity": {
  "h": 0,
  "c": 0,
  "cb": 0,
  "b": 0,
  "phi": 1,
  "K": 0,
  "cs": 0,
  "cbs": 0,
  "bs": 0,
  "phis": 1
},
"power_counting": "every interaction vertex is kappa^{legs-2} (amplitude_rules vertex inventory) => one-loop
+>  kappa^n terms have n external legs and local operator dimension 4+n; massless, no other scale",
"permitted_species": ["h", "c", "cb", "phi", "K", "cs", "phis"],
"excluded_species": ["b", "cbs", "bs"],
"exclusion_reason": "not a leg of any loop-capable vertex (b only in kappa^0 bilinears; cbar*' only in cbar*
+> '.b; b*' absent since s b = 0)",
"loop_capable_vertices": {
  "V3_grav": ["h", "h", "h"],
  "V4_grav": ["h", "h", "h", "h"],
  "V_cb_c_h": ["cb", "c", "h"],
  "V_K_c_h": ["K", "c", "h"],
  "V_cs_c_c": ["cs", "c", "c"],
  "V_phis_c_phi": ["phis", "c", "phi"],
  "V_h_phi_phi": ["h", "phi", "phi"],
  "V_hh_phi_phi": ["h", "h", "phi", "phi"]
},
"permitted_source_dependence": "K' only in the K'.c sector at kappa^2; c*' and phi*' enter first in three-le
+> g terms (c*'.c.c, phi*'.c.phi) at kappa^3 (outside the order bound); cbar*', b*' never",
"symmetries": ["Lorentz covariance (structures generated from eta and p; parity-odd eps structures vanish fo
+> r two-leg terms with one momentum)", "phi -> -phi (with phi*' -> -phi*')", "ghost number 0", "Bose/Fermi
+> symmetry of identical legs (ordered two-point representation)"],
"kappa1": {
  "candidates": [
    {
      "species": "h",
      "derivatives": 4,
      "dimension_ok": true,
      "surviving_mod_total_derivatives": false,
      "reason": "a one-leg local functional with k>0 derivatives is a total derivative; k = 0 needs a dimens
+> ion-5 field (none)"
    }
  ],
  "basis": [],
  "basis_size": 0
},
"kappa2": {
  "sectors": {
    "h.h": {
      "derivatives": 4,
      "dimension": 5,
      "structures": ["(h00*h00_d0000 - 2*h00*h00_d0011 - 2*h00*h00_d0022 - 2*h00*h00_d0033 + h00*h00_d1111 +
+>  2*h00*h00_d1122 + 2*h00*h00_d1133 + h00*h00_d2222 + 2*h00*h00_d2233 + h00", "(h00*h00_d0000 - 2*h00*h00_
+> d0011 - 2*h00*h00_d0022 - 2*h00*h00_d0033 + h00*h00_d1111 + 2*h00*h00_d1122 + 2*h00*h00_d1133 + h00*h00_d
+> 2222 + 2*h00*h00_d2233 + h00", "(h00*h00_d0000 - h00*h00_d0011 - h00*h00_d0022 - h00*h00_d0033 - h00*h11_
+> d0000 + h00*h11_d0011 + h00*h11_d0022 + h00*h11_d0033 - h00*h22_d0000 + h00*h22_d0011 +", "(h00*h00_d0000
+>  - h00*h00_d0011 - h00*h00_d0022 - h00*h00_d0033 - h00*h01_d0001 + h00*h01_d0111 + h00*h01_d0122 + h00*h0
+> 1_d0133 - h00*h02_d0002 + h00*h02_d0112 +", "(h00*h00_d0000 - 2*h00*h01_d0001 - 2*h00*h02_d0002 - 2*h00*h
+> 03_d0003 + h00*h11_d0011 + 2*h00*h12_d0012 + 2*h00*h13_d0013 + h00*h22_d0022 + 2*h00*h23_d0023 + h00"]
    },
    "c.cb": {
      "derivatives": 4,
      "dimension": 2,
      "structures": ["(1)*[('c', 0, ()), ('cb', 0, (0, 0, 0, 0))] + (-2)*[('c', 0, ()), ('cb', 0, (0, 0, 1,
+> 1))] + (-2)*[('c', 0, ()), ('cb', 0, (0, 0, 2, 2))] + (-2)*[('c', 0, ()), ", "(1)*[('c', 0, ()), ('cb', 0
+> , (0, 0, 0, 0))] + (-1)*[('c', 0, ()), ('cb', 0, (0, 0, 1, 1))] + (-1)*[('c', 0, ()), ('cb', 0, (0, 0, 2,
+>  2))] + (-1)*[('c', 0, ()), "]
    },
    "c.K": {
      "derivatives": 3,
      "dimension": 3,
      "structures": ["(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1
+> , 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 2, 2))] + (-I)*[('c', 0, ()", "(I)*[('c', 0, ()), ('K', (0
+> , 0), (0, 0, 0))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0, 1, 1))] + (-I)*[('c', 0, ()), ('K', (0, 0), (0,
+>  2, 2))] + (-I)*[('c', 0, ()", "(I)*[('c', 0, ()), ('K', (0, 0), (0, 0, 0))] + (I)*[('c', 0, ()), ('K', (
+> 0, 1), (0, 0, 1))] + (I)*[('c', 0, ()), ('K', (0, 2), (0, 0, 2))] + (I)*[('c', 0, ()), "]
    },
    "phi.phi": {
      "derivatives": 4,
      "dimension": 1,
```

```
      "structures": ["(phi*phi_d0000 - 2*phi*phi_d0011 - 2*phi*phi_d0022 - 2*phi*phi_d0033 + phi*phi_d1111 +
+>  2*phi*phi_d1122 + 2*phi*phi_d1133 + phi*phi_d2222 + 2*phi*phi_d2233 + phi"]
    }
  },
  "basis_size": 11,
  "basis_size_by_sector": {
    "h.h": 5,
    "c.cb": 2,
    "c.K": 3,
    "phi.phi": 1
  }
},
"equivalences": {
  "total_derivatives": "built in: two-point momentum representation with momentum conservation",
  "sigma0": "sigma_0 = (S_Psi^(0), .) with the frozen XVI-N antibracket (graded_jets.bracket)",
  "S0_density_check": {
    "hh_vs_V2_grav_mismatches": 0,
    "ghost_M0_ok": true,
    "ok": true
  },
  "Xi_classes": {
    "h.cb": {
      "derivatives": 3,
      "candidates": 4,
      "image_rank": 3,
      "kind": "field-only (s-exact gauge-fixing direction)"
    },
    "h.K": {
      "derivatives": 2,
      "candidates": 9,
      "image_rank": 5,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
    "h.bs": {
      "derivatives": 3,
      "candidates": 4,
      "image_rank": 3,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
    "c.cs": {
      "derivatives": 2,
      "candidates": 2,
      "image_rank": 2,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
    "cb.b": {
      "derivatives": 2,
      "candidates": 2,
      "image_rank": 2,
      "kind": "field-only (s-exact gauge-fixing direction)"
    },
    "cb.cbs": {
      "derivatives": 2,
      "candidates": 2,
      "image_rank": 2,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
    "b.K": {
      "derivatives": 1,
      "candidates": 3,
      "image_rank": 2,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
    "b.bs": {
      "derivatives": 2,
      "candidates": 2,
      "image_rank": 2,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
    "phi.phis": {
      "derivatives": 2,
      "candidates": 1,
      "image_rank": 1,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
    "K.cbs": {
      "derivatives": 1,
      "candidates": 3,
      "image_rank": 2,
      "kind": "antifield-linear (field redefinition / canonical transformation)"
    },
```

```
      "cbs.bs": {
        "derivatives": 2,
        "candidates": 2,
        "image_rank": 2,
        "kind": "antifield-linear (field redefinition / canonical transformation)"
      }
    },
    "image_rank": 22,
    "domain_cap_image": 3,
    "sector_cap_image": {
      "h.h": 2,
      "c.cb": 0,
      "c.K": 0,
      "phi.phi": 1
    },
    "candidate_directions_by_sector": {
      "h.h": 3,
      "c.cb": 2,
      "c.K": 3,
      "phi.phi": 0
    },
    "quotient_dimension": 8,
    "reading": "domain elements in Im sigma_0 are removable by field redefinitions/canonical transformations (
  +> antifield-linear Xi) or are s-exact (field-only Xi). Eight candidate directions modulo the computed exact
  +>  equivalences, before imposing the relevant identity constraints.",
    "cohomology_caveat": "NOT a cohomological classification: that needs D cap ker sigma_0 (the identity const
  +> raints, i.e. sigma_0-closedness of the counterterm) and the quotient (D cap ker sigma_0)/(D cap Im sigma_
  +> 0); D cap ker sigma_0 has NOT been computed. The count above is D/(D cap Im sigma_0) only, an upper bound
  +>  for the cohomological count",
    "D_cap_ker_sigma0_computed": false
  },
  "normalization_conditions": ["MS-bar in the declared prescription (P14_prescription_D11_matching): counterte
  +> rms are pure poles at scale mu", "no dimension-4 (two-derivative) counterterm at kappa^2: kappa and the h
  +>  normalisation are not renormalised at this order -- an IMPOSED normalization condition (N4 of P14_prescr
  +> iption_D11_matching M3: no mu^2 p^2 or mu^4 contacts), not a consequence of masslessness, since mu is a s
  +> cale; no kappa^1 counterterm (empty domain)", "finite adjustments only inside this domain; the D11 matchi
  +> ng difference L_match is booked in the hh sector", "u^R_mu (XVI-O) is never modified: no c delta"],
  "counterterm_domain_defined": true,
  "seconds": 148.0
}
```


70 P.5 entry: amplitude rules certificate

[listing 37/133] certificates/P14_amplitude_rules.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "deliverable": "P.5 entry D4: executable amplitude rules",
  "amplitude_rules_verified": true,
  "verified_definition": "W1 AND W2 AND W3 AND W4 AND W5 AND W6 pass AND the W6 Hessian route equals the EXPOR
  +> TED D01/D02/D03/D12 integrands AND every listed mutation rejected (no env mutation)",
  "mutations_from_env": [],
  "integrands_only": true,
  "loop_evaluations_performed": false,
  "conventions": {
    "weight": "e^{iS}",
    "fourier": "f(x) = int d^4k/(2pi)^4 e^{ikx} f~(k); d -> i k; incoming momenta",
    "extraction": "ordered: V[L1..Ln] = d^L_{Ln}...d^L_{L1} S (leg 1 leftmost)",
    "vertex_factor": "i kappa^n V (V unnormalised n-th variation; frozen V4 convention)",
    "Gamma1": "i Gamma_1^{(ext)} = sum of 1PI one-loop graphs; Gamma_1 = -i * graph sum",
    "graviton_components": "independent h_(mn) (m<=n); multiplicity w = 1 (diag), 2 (off-diag) enters ONLY the
  +>  tensor form",
    "ghost": "vertices c-bar LEFT (cbar.c.h); <c cbar> = i M0^{-1}; closed ghost loop factor -1",
    "source": "K' LEFT of c in K'.c.h and in the Gamma_1 coefficient G_{a' nu} (K'_(a') G c^nu)",
    "loop_measure": "int d^d l/(2pi)^d (prescription: see P14_prescription_D11_matching)"
  },
  "rules": {
    "graphs": {
      "D01": {
        "S_f": "1/2",
        "loop_sign": 1,
        "ext_sign": 1,
        "ext_order": ["h", "h"]
      },
      "D02": {
        "S_f": "1/2",
        "loop_sign": 1,
```

```
      "ext_sign": 1,
      "ext_order": ["h", "h"]
    },
    "D03": {
      "S_f": "1",
      "loop_sign": -1,
      "ext_sign": 1,
      "ext_order": ["h", "h"]
    },
    "D11": {
      "S_f": "1/2",
      "loop_sign": 1,
      "ext_sign": 1,
      "ext_order": ["h", "h"]
    },
    "D12": {
      "S_f": "1/2",
      "loop_sign": 1,
      "ext_sign": 1,
      "ext_order": ["h", "h"]
    },
    "D13": {
      "S_f": "1",
      "loop_sign": 1,
      "ext_sign": 1,
      "ext_order": ["K", "c"]
    },
    "D14a": {
      "S_f": "1/2",
      "loop_sign": 1,
      "ext_sign": 1,
      "ext_order": ["h"]
    },
    "D14b": {
      "S_f": "1",
      "loop_sign": -1,
      "ext_sign": 1,
      "ext_order": ["h"]
    },
    "D14c": {
      "S_f": "1/2",
      "loop_sign": 1,
      "ext_sign": 1,
      "ext_order": ["h"]
    }
  },
  "leg_order": {
    "K_c_h": ["K", "c", "h"],
    "cb_c_h": ["cb", "c", "h"],
    "ghost_kinetic": ["cb", "c"]
  },
  "graviton_contraction": "independent_components",
  "ghost_propagator": "<c^mu(q) cbar_rho(-q)> = i (M0^{-1})_{mu rho},  M0_{rho mu} = V[cbar_rho(-q), c^mu(q)
+> ]"
},
"vertex_inventory": [
  {
    "name": "V2_grav",
    "legs": ["h", "h"],
    "kappa_power": 0,
    "phase_i_power": 0,
    "components": 90,
    "nonzeros": 156,
    "sha256": "44e354ba23d84f09afa496fc205cc7d862cab2c175f2abb229f35857398313e5",
    "origin": "exports/ell2_generating_polynomial.srepr (sha 8af76ef7b97d1901)"
  },
  {
    "name": "V3_grav",
    "legs": ["h", "h", "h"],
    "kappa_power": 1,
    "phase_i_power": 0,
    "components": 894,
    "nonzeros": 3924,
    "sha256": "e3eafa0b0c4cb3047a0c6f56da288b3c3984935953d7a2f2554599c04ddef422",
    "origin": "exports/ell3_generating_polynomial.srepr (sha 60c73cd39c9e1531)"
  },
  {
    "name": "V4_grav",
    "legs": ["h", "h", "h", "h"],
    "kappa_power": 2,
    "phase_i_power": 0,
```

```
  "components": 8796,
  "nonzeros": 68688,
  "sha256": "6f1ead09b1e1ecd4453bbfa7cb1a9b7ab95bd22a39c8362322410fa850ab3f99",
  "origin": "exports/ell4_generating_polynomial.srepr (sha 33ac9e49c4d1644e)"
},
{
  "name": "M0_ghost",
  "legs": ["cb", "c"],
  "kappa_power": 0,
  "phase_i_power": 1,
  "components": 16,
  "nonzeros": 40,
  "sha256": "a705e13333b4a2a6c6c1d200545a07290959795f3c3fa4b6cd19d9b8f9e44a5c",
  "origin": "X^(0): cbar.c (c-bar left)"
},
{
  "name": "V_cb_c_h",
  "legs": ["cb", "c", "h"],
  "kappa_power": 1,
  "phase_i_power": 1,
  "components": 136,
  "nonzeros": 272,
  "sha256": "0b1a615ccce1aec8b96f35e733a87196e91ae604217d48ebc80632ed3b0887d0",
  "origin": "X^(1): cbar.c.h"
},
{
  "name": "V_K_c",
  "legs": ["K", "c"],
  "kappa_power": 0,
  "phase_i_power": 1,
  "components": 16,
  "nonzeros": 16,
  "sha256": "8f5e321559f13cbc1d385c9f7175724c858df09484da94ad569704c700580de9",
  "origin": "X^(0): K'.c (source-linear, tree)"
},
{
  "name": "V_K_c_h",
  "legs": ["K", "c", "h"],
  "kappa_power": 1,
  "phase_i_power": 1,
  "components": 88,
  "nonzeros": 104,
  "sha256": "b9feeb4a2218b176e1c41d969dde0fcc787b47437d94667fcb803c50b13075df",
  "origin": "X^(1): K'.c.h"
},
{
  "name": "V_cs_c_c",
  "legs": ["cs", "c", "c"],
  "kappa_power": 1,
  "phase_i_power": 1,
  "components": 28,
  "nonzeros": 32,
  "sha256": "e5cf0ac5b0ac8b15232a95d0142e021bee0d08e9e1797ee02ceadd62e37223f8",
  "origin": "X^(1): c*.c.c"
},
{
  "name": "V_phis_c_phi",
  "legs": ["phis", "c", "phi"],
  "kappa_power": 1,
  "phase_i_power": 1,
  "components": 4,
  "nonzeros": 4,
  "sha256": "13ad47dc9753e69558a315f2192412e643682c23c3e4ce743b9c494cb5e96e6a",
  "origin": "X^(1): phi*.c.phi"
},
{
  "name": "V_b_h",
  "legs": ["b", "h"],
  "kappa_power": 0,
  "phase_i_power": 1,
  "components": 28,
  "nonzeros": 28,
  "sha256": "007bdb36f1ff2bd8edf72701dbe17261665af2e5a797ce38dccf0fd9e4a16002",
  "origin": "X^(0): b.h (gauge fixing)"
},
{
  "name": "V_b_b",
  "legs": ["b", "b"],
  "kappa_power": 0,
  "phase_i_power": 0,
  "components": 4,
```

```
      "nonzeros": 4,
      "sha256": "8f31a01eafd2a4f290a460b325380533a3d0d2d3f3db60a43d10b6b9fcb132c4",
      "origin": "X^(0): b.b"
    },
    {
      "name": "V_cbs_b",
      "legs": ["cbs", "b"],
      "kappa_power": 0,
      "phase_i_power": 1,
      "components": 4,
      "nonzeros": 4,
      "sha256": "7628280e5d25da0c2cd56c2f873603698a0b81e20056b13d957ef0a37b5306f2",
      "origin": "X^(0): cbar*'.b (nonminimal, tree)"
    },
    {
      "name": "M_phi",
      "legs": ["phi", "phi"],
      "kappa_power": 0,
      "phase_i_power": 0,
      "components": 1,
      "nonzeros": 4,
      "sha256": "13dcf1e38f25de0fa9d362a4696e87e3871e049fdb0d175e78f87b23c36cc346",
      "origin": "S_phi^(0)"
    },
    {
      "name": "V_h_phi_phi",
      "legs": ["h", "phi", "phi"],
      "kappa_power": 1,
      "phase_i_power": 0,
      "components": 10,
      "nonzeros": 28,
      "sha256": "ac0b6145d174f59e3bcfb91d46cce14950ba28a0a72e6e3db7a6b54b6bece537",
      "origin": "S_phi^(1): a1"
    },
    {
      "name": "V_hh_phi_phi",
      "legs": ["h", "h", "phi", "phi"],
      "kappa_power": 2,
      "phase_i_power": 0,
      "components": 94,
      "nonzeros": 232,
      "sha256": "00498bb5467c33d467b6037ad94f933a73dad4d2b9ed0d54003be1cf2b90178a",
      "origin": "S_phi^(2): a2"
    }
  ],
  "propagators": {
    "graviton": {
      "numerator": "i (P^{-1})_{ab}",
      "denominator": "q^2 + i0",
      "Pinv": [
        ["1/2", "0", "0", "0", "1/2", "0", "0", "1/2", "0", "1/2"],
        ["0", "-1/2", "0", "0", "0", "0", "0", "0", "0", "0"],
        ["0", "0", "-1/2", "0", "0", "0", "0", "0", "0", "0"],
        ["0", "0", "0", "-1/2", "0", "0", "0", "0", "0", "0"],
        ["1/2", "0", "0", "0", "1/2", "0", "0", "-1/2", "0", "-1/2"],
        ["0", "0", "0", "0", "0", "1/2", "0", "0", "0", "0"],
        ["0", "0", "0", "0", "0", "0", "1/2", "0", "0", "0"],
        ["1/2", "0", "0", "0", "-1/2", "0", "0", "1/2", "0", "-1/2"],
        ["0", "0", "0", "0", "0", "0", "0", "0", "1/2", "0"],
        ["1/2", "0", "0", "0", "-1/2", "0", "0", "-1/2", "0", "1/2"]
      ],
      "Pinv_equals_deDonder_tensor_at_representatives": true,
      "schur_equals_q2P": true,
      "full_h_b_inverse_verified": true,
      "b_block": "Matrix([[-2, 0, 0, 0], [0, 2, 0, 0], [0, 0, 2, 0], [0, 0, 0, 2]])",
      "hb_entry_example": "-I*q0/(q0**2 - q1**2 - q2**2 - q3**2) (entry [00,0])"
    },
    "ghost": {
      "rule": "<c^mu(q) cbar_rho(-q)> = i (M0^{-1})_{mu rho} = numerator_{mu rho}/(q^2+i0)",
      "M0": "-i q^2 delta_{rho mu}",
      "numerator": "-delta_{mu rho}",
      "M0_is_q2_times_constant": true
    },
    "scalar": {
      "rule": "i/(q^2+i0)",
      "V2": "q0**2 - q1**2 - q2**2 - q3**2"
    },
    "i0": "Feynman: every denominator q^2 + i0 (eta = +,-,-,-); D_F(k) = i/(k^2+i0) (Gate 0)"
  },
  "graphs": {
    "D01": {
```

```
  "status": "integrand_available",
  "integrand_available": true,
  "integrated": false,
  "value": "not evaluated",
  "prefactor": "kappa**2/2",
  "denominators": ["l^2+i0", "(l-p)^2+i0"],
  "external": ["h_a(p)", "h_b(-p)"],
  "routing": {
    "A": ["h_a: p", "h_c: -l", "h_e: l-p"],
    "B": ["h_b: -p", "h_d: l", "h_f: p-l"],
    "propagators": ["<h_c h_d>(l)", "<h_e h_f>(l-p)"]
  },
  "numerator_components": 100,
  "numerator_monomials": 3888,
  "numerator_degree": 4,
  "file": "exports/P14_integrands/D01.json",
  "sha256": "869a5db72f275c25a68b9b0295e3381fdf57b9c71de3c632172d70dce2fabce8",
  "rule": {
    "S_f": "1/2",
    "loop_sign": 1,
    "ext_sign": 1,
    "ext_order": ["h", "h"]
  }
},
"D02": {
  "status": "integrand_available",
  "integrand_available": true,
  "integrated": false,
  "value": "not evaluated",
  "prefactor": "-kappa**2/2",
  "denominators": ["l^2+i0"],
  "external": ["h_a(p)", "h_b(-p)"],
  "routing": {
    "V4": ["h_a: p", "h_b: -p", "h_c: l", "h_d: -l"],
    "propagators": ["<h_c h_d>(l)"]
  },
  "numerator_components": 94,
  "numerator_monomials": 220,
  "numerator_degree": 2,
  "file": "exports/P14_integrands/D02.json",
  "sha256": "e37cb4152690018ec8ce2ac1a0539a29864528e65e06a9efe09441b14fb6a063",
  "rule": {
    "S_f": "1/2",
    "loop_sign": 1,
    "ext_sign": 1,
    "ext_order": ["h", "h"]
  }
},
"D03": {
  "status": "integrand_available",
  "integrand_available": true,
  "integrated": false,
  "value": "not evaluated",
  "prefactor": "-kappa**2",
  "denominators": ["(l-p)^2+i0", "l^2+i0"],
  "external": ["h_a(p)", "h_b(-p)"],
  "routing": {
    "A": ["cbar_rho: -l", "c^mu: l-p", "h_a: p"],
    "B": ["cbar_sigma: p-l", "c^nu: l", "h_b: -p"],
    "propagators": ["<c^mu cbar_sigma>(l-p)", "<c^nu cbar_rho>(l)"],
    "trace": "closed ghost loop"
  },
  "numerator_components": 100,
  "numerator_monomials": 2728,
  "numerator_degree": 4,
  "file": "exports/P14_integrands/D03.json",
  "sha256": "728cd2564ff9024d77e950c07642f4a287d59c0d9c6ff56f4de03d1ed7a978bc",
  "rule": {
    "S_f": "1",
    "loop_sign": -1,
    "ext_sign": 1,
    "ext_order": ["h", "h"]
  }
},
"D11": {
  "status": "reference: inherited XVI-O kernel (O5/o70); P-convention integrand for matching M1 only",
  "integrand_available": true,
  "integrated": false,
  "value": "not evaluated",
  "prefactor": "kappa**2/2",
  "denominators": ["l^2+i0", "(l-p)^2+i0"],
```

```
  "external": ["h_a(p)", "h_b(-p)"],
  "routing": {
    "A": ["h_a: p", "phi: -l", "phi: l-p"],
    "B": ["h_b: -p", "phi: l", "phi: p-l"],
    "propagators": ["<phi phi>(l)", "<phi phi>(l-p)"]
  },
  "numerator_components": 100,
  "numerator_monomials": 1722,
  "numerator_degree": 4,
  "file": "exports/P14_integrands/D11.json",
  "sha256": "6afa9cc07776ed4a77077986ffa3aa2ab50936cff13068f98fba9c3017958609",
  "rule": {
    "S_f": "1/2",
    "loop_sign": 1,
    "ext_sign": 1,
    "ext_order": ["h", "h"]
  }
},
"D12": {
  "status": "Missing (integrand available; no value)",
  "integrand_available": true,
  "integrated": false,
  "value": "not evaluated",
  "prefactor": "-kappa**2/2",
  "denominators": ["l^2+i0"],
  "external": ["h_a(p)", "h_b(-p)"],
  "routing": {
    "V": ["h_a: p", "h_b: -p", "phi: l", "phi: -l"],
    "propagators": ["<phi phi>(l)"]
  },
  "numerator_components": 94,
  "numerator_monomials": 160,
  "numerator_degree": 2,
  "file": "exports/P14_integrands/D12.json",
  "sha256": "044080e17dc3acdfcbd2640995d831b69ab2d637712f27148fa60a6a2a99b6fa",
  "rule": {
    "S_f": "1/2",
    "loop_sign": 1,
    "ext_sign": 1,
    "ext_order": ["h", "h"]
  },
  "D12_status": "Missing"
},
"D13": {
  "status": "integrand_available",
  "integrand_available": true,
  "integrated": false,
  "value": "not evaluated",
  "prefactor": "I*kappa**2",
  "denominators": ["l^2+i0", "(p-l)^2+i0"],
  "external": ["K'_(a')(-p)  [left]", "c^nu(p)  [right]"],
  "routing": {
    "K_vertex": ["K'_(a'): -p", "c^mu: l", "h_c: p-l"],
    "ghost_vertex": ["cbar_rho: -l", "c^nu (external): p", "h_d: l-p"],
    "propagators": ["<c^mu cbar_rho>(l)", "<h_c h_d>(p-l)"],
    "Gamma1_term": "Gamma_1 contains int K'_(a') G_{a' nu} c^nu, G = ordered extraction (K', c)"
  },
  "numerator_components": 40,
  "numerator_monomials": 396,
  "numerator_degree": 3,
  "file": "exports/P14_integrands/D13.json",
  "sha256": "42d999fbc4eb3aa01b57050fbe5ff97525976e56dca0e748bae6ca9f03c67026",
  "rule": {
    "S_f": "1",
    "loop_sign": 1,
    "ext_sign": 1,
    "ext_order": ["K", "c"]
  }
},
"D14a": {
  "status": "integrand_available",
  "integrand_available": true,
  "integrated": false,
  "value": "not evaluated",
  "prefactor": "-kappa/2",
  "denominators": ["l^2+i0"],
  "external": ["h_a(0)"],
  "routing": {
    "V3": ["h_a: 0", "h_c: l", "h_d: -l"],
    "propagators": ["<h_c h_d>(l)"]
  },
```

```
      "numerator_components": 10,
      "numerator_monomials": 22,
      "numerator_degree": 2,
      "file": "exports/P14_integrands/D14a.json",
      "sha256": "e141db64f2ed3ef1cb09709b03b8bbaf6c55d29604cb6873e09d05387473e606",
      "rule": {
        "S_f": "1/2",
        "loop_sign": 1,
        "ext_sign": 1,
        "ext_order": ["h"]
      }
    },
    "D14b": {
      "status": "integrand_available",
      "integrand_available": true,
      "integrated": false,
      "value": "not evaluated",
      "prefactor": "kappa",
      "denominators": ["l^2+i0"],
      "external": ["h_a(0)"],
      "routing": {
        "V": ["cbar_rho: -l", "c^mu: l", "h_a: 0"],
        "propagators": ["<c^mu cbar_rho>(l)"]
      },
      "numerator_components": 4,
      "numerator_monomials": 16,
      "numerator_degree": 2,
      "file": "exports/P14_integrands/D14b.json",
      "sha256": "630ff664eec930d2c3eab60f5782dc9a3d4151c85304549f315cd94ad68d94de",
      "rule": {
        "S_f": "1",
        "loop_sign": -1,
        "ext_sign": 1,
        "ext_order": ["h"]
      }
    },
    "D14c": {
      "status": "integrand_available",
      "integrand_available": true,
      "integrated": false,
      "value": "not evaluated",
      "prefactor": "-kappa/2",
      "denominators": ["l^2+i0"],
      "external": ["h_a(0)"],
      "routing": {
        "V": ["h_a: 0", "phi: l", "phi: -l"],
        "propagators": ["<phi phi>(l)"]
      },
      "numerator_components": 10,
      "numerator_monomials": 22,
      "numerator_degree": 2,
      "file": "exports/P14_integrands/D14c.json",
      "sha256": "26d8a239783f101b997843a91ba11acaea453cab04a9fa1266974ad0e2a7f120",
      "rule": {
        "S_f": "1/2",
        "loop_sign": 1,
        "ext_sign": 1,
        "ext_order": ["h"]
      }
    }
  },
  "controls": {
    "W1_wick_berezin_shadow": {
      "checks": {
        "D01": true,
        "D11": true,
        "D02": true,
        "D12": true,
        "D14a": true,
        "D14c": true,
        "ghost_propagator": true,
        "D03": true,
        "D14b": true,
        "D13": true
      },
      "pass": true
    },
    "W2_multiplicity": {
      "checks": {
        "D02": true,
        "D14a": true,
```

```
      "D13": true,
      "Pinv_equals_deDonder_numerator": true,
      "schur_equals_q2P": true,
      "full_h_b_inverse_verified": true
    },
    "pass": true,
    "rule": "independent components h_(mn): vertex = d/dh_(mn) (no 1/w), propagator i (P^{-1})_{ab}/(q^2+i0)
+> , contraction sum over 10x10; tensor form needs V^T = V/w (w=2 off-diagonal) with the 16x16 de Donder num
+> erator"
  },
  "W3_closed_form_vertices": {
    "checks": {
      "cbar_c_h_equals_2ia_D_Lch": true,
      "cbar_c_h_mismatched_components": 0,
      "K_c_h_equals_a_Lch": true,
      "K_c_equals_a_dc_plus_dc": true,
      "h_phi_phi_equals_a1_closed_form": true,
      "ghost_kinetic_equals_minus_i_q2_delta": true
    },
    "pass": true,
    "closed_forms": {
      "cbar.c.h": "kappa * cbar_rho 2 i a D^rho[L_c h], L_c h = c^r d_r h + h_rn d_m c^r + h_mr d_n c^r",
      "K'.c.h": "kappa K'_(mn) a (L_c h)_mn",
      "K'.c": "K'_(mn) a (d_m c_n + d_n c_m)",
      "h.phi.phi": "kappa (1/2) a1^{mn}(h) d_m phi d_n phi, a1 = tau eta/2 - h^{mn}",
      "ghost kinetic": "cbar_mu i Box c^mu"
    }
  },
  "W5_regeneration": {
    "checks": {
      "D14c": true,
      "D14b": true,
      "D14a": true,
      "D11": true,
      "D13": true,
      "D01_bose_N_ab_equals_N_ba": true
    },
    "pass": true,
    "covers": ["D14a", "D14b", "D14c", "D11", "D13"]
  },
  "W6_hessian_route": {
    "checks": {
      "D01_hessian_route_equals_integrand": true,
      "D02_hessian_route_equals_integrand": true,
      "D03_hessian_route_equals_integrand": true,
      "D12_hessian_route_equals_integrand": true,
      "graviton_Hbb_constant": true,
      "graviton_schur_is_q2_times_constant": true,
      "graviton_full_14x14_inverse_verified": true,
      "ghost_M0_is_q2_times_constant": true,
      "structural: X1_has_no_field_only_monomial (no b.h.h, h^3 in X)": true,
      "structural: X2_is_zero (no cbar.c.h.h, no ghost tadpole at kappa^2)": true
    },
    "pass": true,
    "route": "Tr log of the field-dependent Hessian of S_Psi (independent of the Feynman-rule assembler)",
    "X0_hh_part_zero": true,
    "seconds": 12.5
  },
  "W4_ward_bose": {
    "checks": {
      "kappa1_ward_offshell_symbolic": true,
      "kappa1_V3_part_nonzero": true,
      "kappa0_ward_symbolic": true,
      "kappa2_ward_offshell_symbolic": true,
      "kappa2_chunks": 1000,
      "kappa2_chunks_failing": 0,
      "kappa2_V4_part_nonzero_chunks": 996,
      "seconds": 15.7,
      "ward_contraction_two_free_onshell_graviton_legs_offshell_gauge_leg": true,
      "ward_contraction_instances": 144,
      "bose_V3": true,
      "bose_V4": true,
      "fast_extractor_equals_polarisation_V3_symbolic": true
    },
    "pass": true,
    "acceptance_keys": ["kappa1_ward_offshell_symbolic", "kappa1_V3_part_nonzero", "kappa0_ward_symbolic", "
+> kappa2_ward_offshell_symbolic", "ward_contraction_two_free_onshell_graviton_legs_offshell_gauge_leg", "bo
+> se_V3", "bose_V4", "fast_extractor_equals_polarisation_V3_symbolic"]
  }
},
"W1_detail": {
```

```
  "D01": {
    "value": "-I*g_a*g_b/(2*m**2)",
    "rule": "-I*g_a*g_b/(2*m**2)",
    "agree": true
  },
  "D11": {
    "value": "-I*g_a*g_b/(2*m**2)",
    "rule": "-I*g_a*g_b/(2*m**2)",
    "agree": true
  },
  "D02": {
    "value": "I*g/(2*m)",
    "rule": "I*g/(2*m)",
    "agree": true
  },
  "D12": {
    "value": "I*g/(2*m)",
    "rule": "I*g/(2*m)",
    "agree": true
  },
  "D14a": {
    "value": "I*g_a/(2*m)",
    "rule": "I*g_a/(2*m)",
    "agree": true
  },
  "D14c": {
    "value": "I*g_a/(2*m)",
    "rule": "I*g_a/(2*m)",
    "agree": true
  },
  "ghost_propagator": {
    "berezin": "Matrix([[-40*I/247, -42*I/247], [-84*I/247, 60*I/247]])",
    "rule": "Matrix([[-40*I/247, -42*I/247], [-84*I/247, 60*I/247]])",
    "agree": true
  },
  "D03": {
    "value": "335057*I/305045",
    "rule": "335057*I/305045",
    "agree": true
  },
  "D14b": {
    "value": "-714*I/247",
    "rule": "-714*I/247",
    "agree": true
  },
  "D13": {
    "values_vs_rule": {
      "00": ["-6111*I/(4940*m)", "-6111*I/(4940*m)"],
      "01": ["1809*I/(1235*m)", "1809*I/(1235*m)"],
      "10": ["4229*I/(1235*m)", "4229*I/(1235*m)"],
      "11": ["-3308*I/(1235*m)", "-3308*I/(1235*m)"]
    },
    "agree": true
  }
},
"W6_detail": {
  "D01": {
    "components": 100,
    "nonzero_components": 100,
    "mismatches": [],
    "n_mismatches": 0,
    "agree": true
  },
  "D02": {
    "components": 94,
    "nonzero_components": 94,
    "mismatches": [],
    "n_mismatches": 0,
    "agree": true
  },
  "D03": {
    "components": 100,
    "nonzero_components": 100,
    "mismatches": [],
    "n_mismatches": 0,
    "agree": true
  },
  "D12": {
    "components": 94,
    "nonzero_components": 94,
    "mismatches": [],
```

```
    "n_mismatches": 0,
    "agree": true
  }
},
"W6_exports": {
  "D01": {
    "components": 100,
    "nonzero_components": 100,
    "mismatches": [],
    "n_mismatches": 0,
    "agree": true,
    "export_sha256": "869a5db72f275c25a68b9b0295e3381fdf57b9c71de3c632172d70dce2fabce8"
  },
  "D02": {
    "components": 94,
    "nonzero_components": 94,
    "mismatches": [],
    "n_mismatches": 0,
    "agree": true,
    "export_sha256": "e37cb4152690018ec8ce2ac1a0539a29864528e65e06a9efe09441b14fb6a063"
  },
  "D03": {
    "components": 100,
    "nonzero_components": 100,
    "mismatches": [],
    "n_mismatches": 0,
    "agree": true,
    "export_sha256": "728cd2564ff9024d77e950c07642f4a287d59c0d9c6ff56f4de03d1ed7a978bc"
  },
  "D12": {
    "components": 94,
    "nonzero_components": 94,
    "mismatches": [],
    "n_mismatches": 0,
    "agree": true,
    "export_sha256": "044080e17dc3acdfcbd2640995d831b69ab2d637712f27148fa60a6a2a99b6fa"
  },
  "all_agree": true
},
"regeneration_coverage": {
  "W5_independent_sympy_route": ["D14a", "D14b", "D14c", "D11", "D13"],
  "W6_hessian_trlog_route": ["D01", "D02", "D03", "D12"],
  "D01_bose_symmetry": "W5 (symmetry check only)"
},
"mutations": {
  "wrong_tadpole_factor": {
    "amplitude_rules_verified": false,
    "rejected": true,
    "failed_checks": ["W1_wick_berezin_shadow.D02", "W6_hessian_route.D02_hessian_route_equals_integrand"],
    "seconds": 10.1
  },
  "wrong_ghost_loop_sign": {
    "amplitude_rules_verified": false,
    "rejected": true,
    "failed_checks": ["W1_wick_berezin_shadow.D03", "W1_wick_berezin_shadow.D14b", "W6_hessian_route.D03_hes
+> sian_route_equals_integrand"],
    "seconds": 10.1
  },
  "offdiag_multiplicity": {
    "amplitude_rules_verified": false,
    "rejected": true,
    "failed_checks": ["W2_multiplicity.D02", "W2_multiplicity.D13", "W2_multiplicity.D14a", "W5_regeneration
+> .D13", "W5_regeneration.D14a", "W6_hessian_route.D01_hessian_route_equals_integrand", "W6_hessian_route.D
+> 02_hessian_route_equals_integrand"],
    "seconds": 10.2
  },
  "reversed_source_ghost_order": {
    "amplitude_rules_verified": false,
    "rejected": true,
    "failed_checks": ["W1_wick_berezin_shadow.D13"],
    "seconds": 17.7
  },
  "omit_ghost_transport_term": {
    "amplitude_rules_verified": false,
    "rejected": true,
    "failed_checks": ["W3_closed_form_vertices.cbar_c_h_equals_2ia_D_Lch", "W6_hessian_route.D03_hessian_rou
+> te_equals_integrand"],
    "seconds": 17.2
  }
},
"mutations_all_rejected": true,
```

```
  "D12_status": "Missing",
  "D11_role": "inherited kernel reference; matching defined (see D11 matching note)",
  "exports": {
    "vertex_tensors": "exports/P14_vertex_tensors.json",
    "integrands": "exports/P14_integrands/*.json"
  },
  "execution_pass": true,
  "seconds": 145.1
}
```


71 P.5 entry: operational DRED rules certificate (p14r rules; p15 R5 correction; p16 F3 hardening)

[listing 38/133] certificates/P14_dred_rules.json


```
{
  "landmark": "XVI-P",
  "revision": "p14r",
  "deliverable": "P.5 entry: operational DRED rules (executable)",
  "dred_rules_verified": true,
  "rules": {
    "R1_spaces": "Q4S = QdS (+) Q(4-d)S; g4 = ghat + gtilde; tr g4 = 4, tr ghat = d, tr gtilde = 4-d; ghat.gha
  +> t = ghat, gtilde.gtilde = gtilde, ghat.gtilde = 0, g4.X = X",
    "R2_momenta": "loop momentum l in QdS; external momenta p in QdS (p.gtilde = 0, stated choice); field/vert
  +> ex/polarisation indices in Q4S",
    "R3_lifting": "exported lower components l_r = l.e_(r) = l.ehat_(r), p_r = p.e_(r), e_(r) the eta-orthonor
  +> mal basis of Q4S; vertex tensors lifted with eta -> g4",
    "R4_reduction": "Gram/PV projection on symmetric ghat/p tensors with symbolic d; ghat_rs = eta_rs - Gt_rs,
  +>  eta^{rs} Gt_rs = 4-d, Gt_rs p^s = 0",
    "R5_order": "all contractions (incl. external 4D projection) BEFORE the Laurent expansion; d kept symbolic
  +>  through MS-bar subtraction and finite-part extraction; (d-4) x pole retained",
    "R6_evanescent": "evanescent remainders arise in general and are retained (Gt structures; (4-d) x pole fin
  +> ite terms); the p14 'none arise' statement is withdrawn"
  },
  "literature_background_only": ["W. Siegel, Phys. Lett. B84 (1979) 193", "W. Siegel, Phys. Lett. B94 (1980) 3
  +> 7", "D. Stockinger, JHEP 0503 (2005) 076, hep-ph/0503129", "A. Signer, D. Stockinger, Nucl. Phys. B808 (2
  +> 009) 88, arXiv:0807.4424"],
  "trace_rules_check": {
    "tr_g4_is_4": true,
    "tr_ghat_is_symbolic_d": true,
    "tr_ghat_plus_tr_gtilde_is_4": true,
    "tr_gtilde_is_4_minus_d_nonzero": true,
    "pass": true
  },
  "known_reductions": {
    "rank2_tadpole_eq_ghat_over_d": true,
    "rank4_tadpole_eq_sym_ghatghat_over_d_dplus2": true,
    "rank2_two_point_B00": true,
    "rank2_two_point_B11": true,
    "pass": true
  },
  "toy_checks": {
    "placeholders": "J = a/eps + b + c eps (generic symbols; not a loop value); d = 4 - 2 eps",
    "T1_four_d_part": {
      "finite": "a/8 + b/4",
      "pole": "a/(4*epsilon)",
      "equals_reference": true
    },
    "T2_evanescent_trace": {
      "finite": "-a/2",
      "pole": "0",
      "equals_reference": true,
      "pre_laurent": "J_T[1,0] - 4*J_T[1,0]/d",
      "is_d_minus_4_times_pole": true
    },
    "reference": {
      "T1_finite": "a/8 + b/4",
      "T1_pole": "a/(4*epsilon)",
      "T2_finite": "-a/2",
      "T2_pole": "0"
    }
  },
  "mutations": {
    "premature_d4": {
      "rejected": true,
      "reason": "PrematureD4Substitution: d -> 4 outside substitute_d (R5: d is opaque before the Laurent expa
  +> nsion)",
```

```
    "finite_part_if_not_caught": "b/4",
    "finite_part_differs": true
  },
  "trace_ghat_eq_4": {
    "rules_check": {
      "tr_g4_is_4": true,
      "tr_ghat_is_symbolic_d": false,
      "tr_ghat_plus_tr_gtilde_is_4": true,
      "tr_gtilde_is_4_minus_d_nonzero": false,
      "pass": false
    },
    "structurally_refused": true,
    "T1_finite": "b/4",
    "T2_finite": "0",
    "rejected": true
  },
  "contract_after_laurent": {
    "rejected": true,
    "reason": "UncontractedObjects: ['Gt_00', 'Gt_11', 'Gt_22', 'Gt_33']"
  }
},
"mutations_all_rejected": true,
"lift_of_exported_integrands": {
  "D01": {
    "present": true,
    "components": 100,
    "max_l_rank": 4,
    "components_with_evanescent_part": 100,
    "scalar_integrals": "J_D01[a,b] symbols (NOT evaluated)"
  },
  "D02": {
    "present": true,
    "components": 94,
    "max_l_rank": 2,
    "components_with_evanescent_part": 70,
    "scalar_integrals": "J_D02[a,b] symbols (NOT evaluated)"
  },
  "D03": {
    "present": true,
    "components": 100,
    "max_l_rank": 4,
    "components_with_evanescent_part": 100,
    "scalar_integrals": "J_D03[a,b] symbols (NOT evaluated)"
  },
  "D11": {
    "present": true,
    "components": 100,
    "max_l_rank": 4,
    "components_with_evanescent_part": 100,
    "scalar_integrals": "J_D11[a,b] symbols (NOT evaluated)"
  },
  "D12": {
    "present": true,
    "components": 94,
    "max_l_rank": 2,
    "components_with_evanescent_part": 94,
    "scalar_integrals": "J_D12[a,b] symbols (NOT evaluated)"
  },
  "D13": {
    "present": true,
    "components": 40,
    "max_l_rank": 3,
    "components_with_evanescent_part": 40,
    "scalar_integrals": "J_D13[a,b] symbols (NOT evaluated)"
  },
  "D14a": {
    "present": true,
    "components": 10,
    "max_l_rank": 2,
    "components_with_evanescent_part": 10,
    "scalar_integrals": "J_D14a[a,b] symbols (NOT evaluated)"
  },
  "D14b": {
    "present": true,
    "components": 4,
    "max_l_rank": 2,
    "components_with_evanescent_part": 4,
    "scalar_integrals": "J_D14b[a,b] symbols (NOT evaluated)"
  },
  "D14c": {
    "present": true,
```

```
      "components": 10,
      "max_l_rank": 2,
      "components_with_evanescent_part": 10,
      "scalar_integrals": "J_D14c[a,b] symbols (NOT evaluated)"
    }
  },
  "evanescent_parts_present_in_exports": true,
  "p15_contract_correction": {
    "finding": "F1_premature_d4_guard_heuristic (reproducibility/independent_rebuild_report_p14r.json)",
    "old_safeguard": "heuristic: raise iff the reduction introduced d and d is absent before the Laurent expan
  +> sion (false positive and false negative on the reviewer's eta-trace toy)",
    "new_safeguard": "structural/provenance: OpaqueDimension d (subs/xreplace raise outside substitute_d); sub
  +> stitute_d records every authorised substitution; numeric traces of ghat/gtilde refused (NumericTraceForbi
  +> dden) unless display_only",
    "released_p14r_values_affected": false,
    "why_unaffected": "in p14r Pipeline.laurent was applied only to the toy placeholders; no loop integral was
  +>  evaluated",
    "regression_toys": {
      "eta_trace_correct": {
        "finite": "a1*m**4",
        "raised": false,
        "ok": true
      },
      "eta_trace_premature_d4": {
        "raised": true,
        "reason": "d -> 4 outside substitute_d (R5: d is opaque before the Laurent expansion)",
        "finite_if_forced": "m**4*(a1 - 1/2)",
        "ok": true
      },
      "A0_mixing": {
        "correct_finite": "a1*m**4",
        "mixed_refused": true,
        "finite_if_forced": "m**4*(a1 + 1/2)",
        "shift": "m**4/2",
        "ok": true
      },
      "B0_mixing": {
        "correct_finite": "0",
        "mixed_refused": true,
        "finite_if_forced": "-p2/6",
        "shift": "-p2/6",
        "ok": true
      },
      "pass": true
    }
  },
  "loop_evaluations_performed": false,
  "seconds": 85.0
}
```


72 Symbolic integrand D01 (p14r export; evaluated in p16)

[listing 39/133] exports/P14_integrands/D01.json


```
{"denominators":["l^2+i0","(l-p)^2+i0"],"external":["h_a(p)","h_b(-p)"],"external_components":"independent h_(
  +> mn) index 0..9 = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]; vector
  +>  index 0..3","graph":"D01","integrated":false,"numerator":{"0,0":[[["l0","l0","l0","l0"],"3/2"],[["l0","l
  +> 0","l0","p0"],"-3/1"],[["l0","l0","l1","l1"],"-1/1"],[["l0","l0","l1","p1"],"1/1"],[["l0","l0","l2","l2"]
  +> ,"-1/1"],[["l0","l0","l2","p2"],"1/1"],[["l0","l0","l3","l3"],"-1/1"],[["l0","l0","l3","p3"],"1/1"],[["l0
  +> ","l0","p0","p0"],"3/2"],[["l0","l0","p1","p1"],"1/2"],[["l0","l0","p2","p2"],"1/2"],[["l0","l0","p3","p3
  +> "],"1/2"],[["l0","l1","l1","p0"],"1/1"],[["l0","l2","l2","p0"],"1/1"],[["l0","l3","l3","p0"],"1/1"],[["l0
  +> ","p0","p1","p1"],"-1/1"],[["l0","p0","p2","p2"],"-1/1"],[["l0","p0","p3","p3"],"-1/1"],[["l1","l1","l1",
  +> "l1"],"3/2"],[["l1","l1","l1","p1"],"-3/1"],[["l1","l1","l2","l2"],"3/1"],[["l1","l1","l2","p2"],"-3/1"],
  +> [["l1","l1","l3","l3"],"3/1"],[["l1","l1","l3","p3"],"-3/1"],[["l1","l1","p0","p0"],"-1/2"],[["l1","l1","
  +> p1","p1"],"5/2"],[["l1","l1","p2","p2"],"3/2"],[["l1","l1","p3","p3"],"3/2"],[["l1","l2","l2","p1"],"-3/1
  +> "],[["l1","l2","p1","p2"],"2/1"],[["l1","l3","l3","p1"],"-3/1"],[["l1","l3","p1","p3"],"2/1"],[["l1","p1"
  +> ,"p1","p1"],"-1/1"],[["l1","p1","p2","p2"],"-1/1"],[["l1","p1","p3","p3"],"-1/1"],[["l2","l2","l2","l2"],
  +> "3/2"],[["l2","l2","l2","p2"],"-3/1"],[["l2","l2","l3","l3"],"3/1"],[["l2","l2","l3","p3"],"-3/1"],[["l2"
  +> ,"l2","p0","p0"],"-1/2"],[["l2","l2","p1","p1"],"3/2"],[["l2","l2","p2","p2"],"5/2"],[["l2","l2","p3","p3
  +> "],"3/2"],[["l2","l3","l3","p2"],"-3/1"],[["l2","l3","p2","p3"],"2/1"],[["l2","p1","p1","p2"],"-1/1"],[["
  +> l2","p2","p2","p2"],"-1/1"],[["l2","p2","p3","p3"],"-1/1"],[["l3","l3","l3","l3"],"3/2"],[["l3","l3","l3"
  +> ,"p3"],"-3/1"],[["l3","l3","p0","p0"],"-1/2"],[["l3","l3","p1","p1"],"3/2"],[["l3","l3","p2","p2"],"3/2"]
  +> ,[["l3","l3","p3","p3"],"5/2"],[["l3","p1","p1","p3"],"-1/1"],[["l3","p2","p2","p3"],"-1/1"],[["l3","p3",
  +> "p3","p3"],"-1/1"],[["p1","p1","p1","p1"],"2/1"],[["p1","p1","p2","p2"],"4/1"],[["p1","p1","p3","p3"],"4/
  +> 1"],[["p2","p2","p2","p2"],"2/1"],[["p2","p2","p3","p3"],"4/1"],[["p3","p3","p3","p3"],"2/1"]],"0,1":[[["
  +> l0","l0","l0","l1"],"-2/1"],[["l0","l0","l0","p1"],"1/1"],[["l0","l0","l1","p0"],"3/1"],[["l0","l0","p0",
  +> "p1"],"-2/1"],[["l0","l1","l1","l1"],"-2/1"],[["l0","l1","l1","p1"],"3/1"],[["l0","l1","l2","l2"],"-2/1"]
  +> ,[["l0","l1","l2","p2"],"2/1"],[["l0","l1","l3","l3"],"-2/1"],[["l0","l1","l3","p3"],"2/1"],[["l0","l1","
```

```
+> p0","p0"],"-1/1"],[["l0","l1","p1","p1"],"-3/1"],[["l0","l1","p2","p2"],"-2/1"],[["l0","l1","p3","p3"],"-
+> 2/1"],[["l0","l2","l2","p1"],"1/1"],[["l0","l2","p1","p2"],"-1/1"],[["l0","l3","l3","p1"],"1/1"],[["l0","
+> l3","p1","p3"],"-1/1"],[["l0","p0","p0","p1"],"1/1"],[["l0","p1","p1","p1"],"1/1"],[["l0","p1","p2","p2"]
+> ,"1/1"],[["l0","p1","p3","p3"],"1/1"],[["l1","l1","l1","p0"],"1/1"],[["l1","l1","p0","p1"],"-2/1"],[["l1"
+> ,"l2","l2","p0"],"1/1"],[["l1","l2","p0","p2"],"-1/1"],[["l1","l3","l3","p0"],"1/1"],[["l1","l3","p0","p3
+> "],"-1/1"],[["l1","p0","p1","p1"],"2/1"],[["l1","p0","p2","p2"],"1/1"],[["l1","p0","p3","p3"],"1/1"],[["l
+> 2","l2","p0","p1"],"-1/1"],[["l2","p0","p1","p2"],"1/1"],[["l3","l3","p0","p1"],"-1/1"],[["l3","p0","p1",
+> "p3"],"1/1"],[["p0","p1","p1","p1"],"-4/1"],[["p0","p1","p2","p2"],"-4/1"],[["p0","p1","p3","p3"],"-4/1"]
+> ],"0,2":[[["l0","l0","l0","l2"],"-2/1"],[["l0","l0","l0","p2"],"1/1"],[["l0","l0","l2","p0"],"3/1"],[["l0
+> ","l0","p0","p2"],"-2/1"],[["l0","l1","l1","l2"],"-2/1"],[["l0","l1","l1","p2"],"1/1"],[["l0","l1","l2","
+> p1"],"2/1"],[["l0","l1","p1","p2"],"-1/1"],[["l0","l2","l2","l2"],"-2/1"],[["l0","l2","l2","p2"],"3/1"],[
+> ["l0","l2","l3","l3"],"-2/1"],[["l0","l2","l3","p3"],"2/1"],[["l0","l2","p0","p0"],"-1/1"],[["l0","l2","p
+> 1","p1"],"-2/1"],[["l0","l2","p2","p2"],"-3/1"],[["l0","l2","p3","p3"],"-2/1"],[["l0","l3","l3","p2"],"1/
+> 1"],[["l0","l3","p2","p3"],"-1/1"],[["l0","p0","p0","p2"],"1/1"],[["l0","p1","p1","p2"],"1/1"],[["l0","p2
+> ","p2","p2"],"1/1"],[["l0","p2","p3","p3"],"1/1"],[["l1","l1","l2","p0"],"1/1"],[["l1","l1","p0","p2"],"-
+> 1/1"],[["l1","l2","p0","p1"],"-1/1"],[["l1","p0","p1","p2"],"1/1"],[["l2","l2","l2","p0"],"1/1"],[["l2","
+> l2","p0","p2"],"-2/1"],[["l2","l3","l3","p0"],"1/1"],[["l2","l3","p0","p3"],"-1/1"],[["l2","p0","p1","p1"
+> ],"1/1"],[["l2","p0","p2","p2"],"2/1"],[["l2","p0","p3","p3"],"1/1"],[["l3","l3","p0","p2"],"-1/1"],[["l3
+> ","p0","p2","p3"],"1/1"],[["p0","p1","p1","p2"],"-4/1"],[["p0","p2","p2","p2"],"-4/1"],[["p0","p2","p3","
+> p3"],"-4/1"]],"0,3":[[["l0","l0","l0","l3"],"-2/1"],[["l0","l0","l0","p3"],"1/1"],[["l0","l0","l3","p0"],
+> "3/1"],[["l0","l0","p0","p3"],"-2/1"],[["l0","l1","l1","l3"],"-2/1"],[["l0","l1","l1","p3"],"1/1"],[["l0"
+> ,"l1","l3","p1"],"2/1"],[["l0","l1","p1","p3"],"-1/1"],[["l0","l2","l2","l3"],"-2/1"],[["l0","l2","l2","p
+> 3"],"1/1"],[["l0","l2","l3","p2"],"2/1"],[["l0","l2","p2","p3"],"-1/1"],[["l0","l3","l3","l3"],"-2/1"],[
+> "l0","l3","l3","p3"],"3/1"],[["l0","l3","p0","p0"],"-1/1"],[["l0","l3","p1","p1"],"-2/1"],[["l0","l3","p2
+> ","p2"],"-2/1"],[["l0","l3","p3","p3"],"-3/1"],[["l0","p0","p0","p3"],"1/1"],[["l0","p1","p1","p3"],"1/1"
+> ],[["l0","p2","p2","p3"],"1/1"],[["l0","p3","p3","p3"],"1/1"],[["l1","l1","l3","p0"],"1/1"],[["l1","l1","
+> p0","p3"],"-1/1"],[["l1","l3","p0","p1"],"-1/1"],[["l1","p0","p1","p3"],"1/1"],[["l2","l2","l3","p0"],"1/
+> 1"],[["l2","l2","p0","p3"],"-1/1"],[["l2","l3","p0","p0"],"-1/1"],[["l2","l3","p1","p1"],"-2/1"],[["l2","
+> l3","l3","p0"],"1/1"],[["l3","l3","p0","p3"],"-2/1"],[["l3","l3","p1","p2"],"-1/1"],[["l3","p0","p0","p2
+> "],"-2/1"],[["l0","l3","p3","p3"],"-3/1"],[["l0","p0","p0","p3"],"1/1"],[["l0","p1","p1","p3"],"1/1"
+> ],[["l0","p2","p2","p3"],"1/1"],[["l0","p3","p3","p3"],"1/1"],[["l1","l1","l3","p0"],"1/1"],[["l1","l1","
+> p0","p3"],"-1/1"],[["l1","l3","p0","p1"],"-1/1"],[["l1","p0","p1","p3"],"1/1"],[["l2","l2","l3","p0"],"1/
+> 1"],[["l2","l2","p0","p3"],"-1/1"],[["l2","l3","p0","p2"],"1/1"],[["l2","l3","p1","p3"],"6/1"],[["l3","l
+> 3","l3","p0"],"1/1"],[["l3","l3","p0","p3"],"-2/1"],[["l3","p0","p1","p1"],"1/1"],[["l3","p0","p2","p2"],
+> "1/1"],[["l3","p0","p3","p3"],"2/1"],[["p0","p1","p1","p3"],"-4/1"],[["p0","p2","p2","p3"],"-4/1"],[["p0"
+> ,"p3","p3","p3"],"-4/1"]],"0,4":[[["l0","l0","l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0"
+> ,"l1","l1"],"3/1"],[["l0","l0","l1","p1"],"-3/1"],[["l0","l0","p1","p1"],"2/1"],[["l0","l0","p2","p2"],"-
+> 3/2"],[["l0","l0","p3","p3"],"-3/2"],[["l0","l1","l1","p0"],"-3/1"],[["l0","l2","p0","p2"],"-1/1"],[["l0"
+> ,"l3","p0","p3"],"-1/1"],[["l0","p0","p0","p0"],"-1/2"],[["l0","p0","p1","p1"],"-1/2"],[["l0","p0","p2","
+> p2"],"2/1"],[["l0","p0","p3","p3"],"2/1"],[["l1","l1","l1","l1"],"-1/2"],[["l1","l1","l1","p1"],"1/1"],[[
+> "l1","l1","p0","p0"],"2/1"],[["l1","l1","p2","p2"],"3/2"],[["l1","l1","p3","p3"],"3/2"],[["l1","l2","p1",
+> "p2"],"1/1"],[["l1","l3","p1","p3"],"1/1"],[["l1","p0","p0","p1"],"-1/2"],[["l1","p1","p1","p1"],"-1/2"],
+> [["l1","p1","p2","p2"],"-2/1"],[["l1","p1","p3","p3"],"-2/1"],[["l2","l2","l2","l2"],"1/2"],[["l2","l2","
+> l2","p2"],"-1/1"],[["l2","l2","l3","l3"],"1/1"],[["l2","l2","l3","p3"],"-1/1"],[["l2","l2","p2","p2"],"5/
+> 2"],[["l2","l2","p3","p3"],"-1/2"],[["l2","l3","l3","p2"],"-1/1"],[["l2","l3","p2","p3"],"6/1"],[["l2","p
+> 0","p0","p2"],"1/2"],[["l2","p1","p1","p2"],"-1/2"],[["l2","p2","p2","p2"],"-2/1"],[["l2","p2","p3","p3"]
+> ,"-2/1"],[["l3","l3","l3","l3"],"1/2"],[["l3","l3","l3","p3"],"-1/1"],[["l3","l3","p2","p2"],"-1/2"],[["l
+> 3","l3","p3","p3"],"5/2"],[["l3","p0","p0","p3"],"1/2"],[["l3","p1","p1","p3"],"-1/2"],[["l3","p2","p2","
+> p3"],"-2/1"],[["l3","p3","p3","p3"],"-2/1"],[["p0","p0","p1","p1"],"2/1"]],"0,5":[[["l0","l0","l1","l2"],
+> "6/1"],[["l0","l0","l1","p2"],"-3/1"],[["l0","l0","l2","p1"],"-3/1"],[["l0","l0","p1","p2"],"7/1"],[["l0"
+> ,"l1","l2","p0"],"-6/1"],[["l0","l1","p0","p2"],"1/1"],[["l0","l2","p0","p1"],"1/1"],[["l0","p0","p1","p2
+> "],"-5/1"],[["l1","l1","l1","l2"],"-2/1"],[["l1","l1","l1","p2"],"1/1"],[["l1","l1","l2","p1"],"3/1"],[["
+> l1","l1","p1","p2"],"-4/1"],[["l1","l2","l2","l2"],"-2/1"],[["l1","l2","l2","p2"],"3/1"],[["l1","l2","l3"
+> ,"l3"],"-2/1"],[["l1","l2","l3","p3"],"2/1"],[["l1","l2","p0","p0"],"4/1"],[["l1","l2","p1","p1"],"-1/1"]
+> ,[["l1","l2","p2","p2"],"-1/1"],[["l1","l2","p3","p3"],"4/1"],[["l1","l3","l3","p2"],"1/1"],[["l1","l3","
+> p2","p3"],"-5/1"],[["l1","p0","p0","p2"],"-1/1"],[["l1","p1","p1","p2"],"3/1"],[["l2","l2","l2","p1"],"1/
+> 1"],[["l2","l2","p1","p2"],"-4/1"],[["l2","l3","l3","p1"],"1/1"],[["l2","l3","p1","p3"],"-5/1"],[["l2","p
+> 0","p0","p1"],"-1/1"],[["l2","p1","p2","p2"],"3/1"],[["l3","l3","p1","p2"],"1/1"],[["l3","p1","p2","p3"],
+> "3/1"],[["p0","p0","p1","p2"],"4/1"]],"0,6":[[["l0","l0","l1","l3"],"6/1"],[["l0","l0","l1","p3"],"-3/1"]
+> ,[["l0","l0","l3","p1"],"-3/1"],[["l0","l0","p1","p3"],"7/1"],[["l0","l1","l3","p0"],"-6/1"],[["l0","l1",
+> "p0","p3"],"1/1"],[["l0","l3","p0","p1"],"1/1"],[["l0","p0","p1","p3"],"-5/1"],[["l1","l1","l1","l3"],"-2
+> /1"],[["l1","l1","l1","p3"],"1/1"],[["l1","l1","l3","p1"],"3/1"],[["l1","l1","p1","p3"],"-4/1"],[["l1","l
+> 2","l2","l3"],"-2/1"],[["l1","l2","l2","p3"],"1/1"],[["l1","l2","l3","p2"],"2/1"],[["l1","l2","p2","p3"],
+> "-5/1"],[["l1","l3","l3","l3"],"-2/1"],[["l1","l3","l3","p3"],"3/1"],[["l1","l3","p0","p0"],"4/1"],[["l1"
+> ,"l3","p1","p1"],"-1/1"],[["l1","l3","p2","p2"],"4/1"],[["l1","l3","p3","p3"],"-1/1"],[["l1","p0","p0","p
+> 3"],"-1/1"],[["l1","p1","p1","p3"],"3/1"],[["l2","l2","l3","p1"],"1/1"],[["l2","l2","l2","p1","p3"],"1/1"],[["
+> l2","l3","p1","p2"],"-5/1"],[["l2","p1","p2","p3"],"3/1"],[["l3","l3","l3","p1"],"1/1"],[["l3","l3","p1",
+> "p3"],"-4/1"],[["l3","p0","p0","p1"],"-1/1"],[["l3","p1","p3","p3"],"3/1"],[["p0","p0","p1","p3"],"4/1"]]
+> ,"0,7":[[["l0","l0","l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l2","l2"],"3/1"],[["l0"
+> ,"l0","l2","p2"],"-3/1"],[["l0","l0","p1","p1"],"-3/2"],[["l0","l0","p2","p2"],"2/1"],[["l0","l0","p3","p
+> 3"],"-3/2"],[["l0","l1","p0","p1"],"-1/1"],[["l0","l2","l2","p0"],"-3/1"],[["l0","l3","p0","p3"],"-1/1"],
+> [["l0","p0","p0","p0"],"-1/2"],[["l0","p0","p1","p1"],"2/1"],[["l0","p0","p2","p2"],"-1/2"],[["l0","p0","
+> p3","p3"],"2/1"],[["l1","l1","l1","l1"],"1/2"],[["l1","l1","l1","p1"],"-1/1"],[["l1","l1","l3","l3"],"1/1
+> "],[["l1","l1","l3","p3"],"-1/1"],[["l1","l1","p1","p1"],"5/2"],[["l1","l1","p3","p3"],"-1/2"],[["l1","l2
+> ","p1","p2"],"1/1"],[["l1","l3","l3","p1"],"-1/1"],[["l1","l3","p1","p3"],"6/1"],[["l1","p0","p0","p1"],"
+> 1/2"],[["l1","p1","p1","p1"],"-2/1"],[["l1","p1","p2","p2"],"-1/2"],[["l1","p1","p3","p3"],"-2/1"],[["l2"
+> ,"l2","l2","l2"],"-1/2"],[["l2","l2","l2","p2"],"1/1"],[["l2","l2","p0","p0"],"2/1"],[["l2","l2","p1","p1
+> "],"3/2"],[["l2","l2","p3","p3"],"3/2"],[["l2","l3","p2","p3"],"1/1"],[["l2","p0","p0","p2"],"-1/2"],[["l
+> 2","p1","p1","p2"],"-2/1"],[["l2","p2","p2","p2"],"-1/2"],[["l2","p2","p3","p3"],"-2/1"],[["l3","l3","l3"
+> ,"l3"],"1/2"],[["l3","l3","l3","p3"],"-1/1"],[["l3","l3","p1","p1"],"-1/2"],[["l3","l3","p3","p3"],"5/2"]
+> ,[["l3","p0","p0","p3"],"1/2"],[["l3","p1","p1","p3"],"-2/1"],[["l3","p2","p2","p3"],"-1/2"],[["l3","p3",
+> "p3","p3"],"-2/1"],[["p0","p0","p2","p2"],"2/1"]],"0,8":[[["l0","l0","l2","l3"],"6/1"],[["l0","l0","l2","
+> p3"],"-3/1"],[["l0","l0","l3","p2"],"-3/1"],[["l0","l0","p2","p3"],"7/1"],[["l0","l2","l3","p0"],"-6/1"],
+> [["l0","l2","p0","p3"],"1/1"],[["l0","l3","p0","p2"],"1/1"],[["l0","p0","p2","p3"],"-5/1"],[["l1","l1","l
+> 2","l3"],"-2/1"],[["l1","l1","l2","p3"],"1/1"],[["l1","l1","l3","p2"],"1/1"],[["l1","l1","p2","p3"],"1/1"
+> ],[["l1","l2","l3","p1"],"2/1"],[["l1","l2","p1","p3"],"-5/1"],[["l1","l3","p1","p2"],"-5/1"],[["l1","p1"
```

```
+> ,"p2","p3"],"3/1"],[["12","12","12","13"],"-2/1"],[["12","12","12","p3"],"1/1"],[["12","12","13","p2"],"3
+> /1"],[["12","12","p2","p3"],"-4/1"],[["12","13","13","13"],"-2/1"],[["12","13","13","p3"],"3/1"],[["12","
+> 13","p0","p0"],"4/1"],[["12","13","p1","p1"],"4/1"],[["12","13","p2","p2"],"-1/1"],[["12","13","p3","p3"]
+> ,"-1/1"],[["12","p0","p0","p3"],"-1/1"],[["12","p2","p2","p3"],"3/1"],[["13","13","13","p2"],"1/1"],[["13
+> ","13","p2","p3"],"-4/1"],[["13","p0","p0","p2"],"-1/1"],[["13","p2","p3","p3"],"3/1"],[["p0","p0","p2","
+> p3"],"4/1"]],"0,9":[[["10","10","10","10"],"-1/2"],[["10","10","10","p0"],"1/1"],[["10","10","13","13"],"
+> 3/1"],[["10","10","13","p3"],"-3/1"],[["10","10","p1","p1"],"-3/2"],[["10","10","p2","p2"],"-3/2"],[["10"
+> ,"10","p3","p3"],"2/1"],[["10","11","p0","p1"],"-1/1"],[["10","12","p0","p2"],"-1/1"],[["10","13","13","p
+> 0"],"-3/1"],[["10","p0","p0","p0"],"-1/2"],[["10","p0","p1","p1"],"2/1"],[["10","p0","p2","p2"],"2/1"],[[
+> "10","p0","p3","p3"],"-1/2"],[["11","11","11","11"],"1/2"],[["11","11","11","p1"],"-1/1"],[["11","11","12
+> ","12"],"1/1"],[["11","11","12","p2"],"-1/1"],[["11","11","p1","p1"],"5/2"],[["11","11","p2","p2"],"-1/2"
+> ],[["11","12","12","p1"],"-1/1"],[["11","12","p1","p2"],"6/1"],[["11","13","p1","p3"],"1/1"],[["11","p0",
+> "p0","p1"],"1/2"],[["11","p1","p1","p1"],"-2/1"],[["11","p1","p2","p2"],"-2/1"],[["11","p1","p3","p3"],"-
+> 1/2"],[["12","12","12","12"],"1/2"],[["12","12","12","p2"],"-1/1"],[["12","12","p1","p1"],"-1/2"],[["12",
+> "12","p2","p2"],"5/2"],[["12","13","p2","p3"],"1/1"],[["12","p0","p0","p2"],"1/2"],[["12","p1","p1","p2"]
+> ,"-2/1"],[["12","p2","p2","p2"],"-2/1"],[["12","p2","p3","p3"],"-1/2"],[["13","13","13","13"],"-1/2"],[["
+> 13","13","13","p3"],"1/1"],[["13","13","p0","p0"],"2/1"],[["13","13","p1","p1"],"3/2"],[["13","13","p2","
+> p2"],"3/2"],[["13","p0","p0","p3"],"-1/2"],[["13","p1","p1","p3"],"-2/1"],[["13","p2","p2","p3"],"-2/1"],
+> [["13","p3","p3","p3"],"-1/2"],[["p0","p0","p3","p3"],"2/1"]],"1,0":[[["10","10","10","11"],"-2/1"],[["10
+> ","10","10","p1"],"1/1"],[["10","10","11","p0"],"3/1"],[["10","10","p0","p1"],"-2/1"],[["10","11","11","1
+> 1"],"-2/1"],[["10","11","11","p1"],"3/1"],[["10","11","12","12"],"-2/1"],[["10","11","12","p2"],"2/1"],[[
+> "10","11","13","13"],"-2/1"],[["10","11","13","p3"],"2/1"],[["10","11","p0","p0"],"-1/1"],[["10","11","p1
+> ","p1"],"-3/1"],[["10","11","p2","p2"],"-2/1"],[["10","11","p3","p3"],"-2/1"],[["10","12","12","p1"],"1/1
+> "],[["10","12","p1","p2"],"-1/1"],[["10","13","13","p1"],"1/1"],[["10","13","p1","p3"],"-1/1"],[["10","p0
+> ","p0","p1"],"1/1"],[["10","p1","p1","p1"],"1/1"],[["10","p1","p2","p2"],"1/1"],[["10","p1","p3","p3"],"1
+> /1"],[["11","11","11","p0"],"1/1"],[["11","11","p0","p1"],"-2/1"],[["11","12","12","p0"],"1/1"],[["11","1
+> 2","p0","p2"],"-1/1"],[["11","13","13","p0"],"1/1"],[["11","13","p0","p3"],"-1/1"],[["11","p0","p1","p1"]
+> ,"2/1"],[["11","p0","p2","p2"],"1/1"],[["11","p0","p3","p3"],"1/1"],[["12","12","p0","p1"],"-1/1"],[["12"
+> ,"p0","p1","p2"],"1/1"],[["13","13","p0","p1"],"-1/1"],[["13","p0","p1","p3"],"1/1"],[["p0","p1","p1","p1
+> "],"-4/1"],[["p0","p1","p2","p2"],"-4/1"],[["p0","p1","p3","p3"],"-4/1"]],"1,1":[[["10","10","11","11"],"
+> 8/1"],[["10","10","11","p1"],"-8/1"],[["10","10","12","12"],"4/1"],[["10","10","12","p2"],"-4/1"],[["10",
+> "10","13","13"],"4/1"],[["10","10","13","p3"],"-4/1"],[["10","10","p2","p2"],"4/1"],[["10","10","p3","p3"
+> ],"4/1"],[["10","11","11","p0"],"-8/1"],[["10","11","p0","p1"],"16/1"],[["10","12","12","p0"],"-4/1"],[["
+> 10","12","p0","p2"],"4/1"],[["10","13","13","p0"],"-4/1"],[["10","13","p0","p3"],"4/1"],[["10","p0","p1",
+> "p1"],"-4/1"],[["10","p0","p2","p2"],"-4/1"],[["10","p0","p3","p3"],"-4/1"],[["11","11","12","12"],"-4/1"
+> ],[["11","11","12","p2"],"4/1"],[["11","11","13","13"],"-4/1"],[["11","11","13","p3"],"4/1"],[["11","11",
+> "p2","p2"],"-4/1"],[["11","11","p3","p3"],"-4/1"],[["11","12","12","p1"],"4/1"],[["11","12","p1","p2"],"-
+> 4/1"],[["11","13","13","p1"],"4/1"],[["11","13","p1","p3"],"-4/1"],[["11","p0","p0","p1"],"-4/1"],[["11",
+> "p1","p2","p2"],"4/1"],[["11","p1","p3","p3"],"4/1"],[["12","12","12","12"],"-4/1"],[["12","12","12","p2"
+> ],"8/1"],[["12","12","13","13"],"-8/1"],[["12","12","13","p3"],"8/1"],[["12","12","p0","p0"],"2/1"],[["12
+> ","12","p1","p1"],"-2/1"],[["12","12","p2","p2"],"-10/1"],[["12","12","p3","p3"],"-2/1"],[["12","13","13"
+> ,"p2"],"8/1"],[["12","13","p2","p3"],"-16/1"],[["12","p0","p0","p2"],"-2/1"],[["12","p1","p1","p2"],"2/1"
+> ],[["12","p2","p2","p2"],"6/1"],[["12","p2","p3","p3"],"6/1"],[["13","13","13","13"],"-4/1"],[["13","13",
+> "13","p3"],"8/1"],[["13","13","p0","p0"],"2/1"],[["13","13","p1","p1"],"-2/1"],[["13","13","p2","p2"],"-2
+> /1"],[["13","13","p3","p3"],"-10/1"],[["13","p0","p0","p3"],"-2/1"],[["13","p1","p1","p3"],"2/1"],[["13",
+> "p2","p2","p3"],"6/1"],[["13","p3","p3","p3"],"6/1"],[["p0","p0","p1","p1"],"8/1"],[["p0","p0","p2","p2"]
+> ,"4/1"],[["p0","p0","p3","p3"],"4/1"],[["p1","p1","p2","p2"],"-4/1"],[["p1","p1","p3","p3"],"-4/1"],[["p2
+> ","p2","p2","p2"],"-4/1"],[["p2","p2","p3","p3"],"-8/1"],[["p3","p3","p3","p3"],"-4/1"]],"1,2":[[["10","1
+> 0","11","12"],"4/1"],[["10","10","11","p2"],"-2/1"],[["10","10","12","p1"],"-2/1"],[["10","10","p1","p2"]
+> ,"-4/1"],[["10","11","12","p0"],"-4/1"],[["10","11","p0","p2"],"6/1"],[["10","12","p0","p1"],"6/1"],[["11
+> ","11","11","12"],"4/1"],[["11","11","11","p2"],"-2/1"],[["11","11","12","p1"],"-6/1"],[["11","11","p1","
+> p2"],"6/1"],[["11","12","12","12"],"4/1"],[["11","12","12","p2"],"-6/1"],[["11","12","13","13"],"4/1"],[[
+> "11","12","13","p3"],"-4/1"],[["11","12","p0","p0"],"-2/1"],[["11","12","p1","p1"],"4/1"],[["11","12","p2
+> ","p2"],"4/1"],[["11","12","p3","p3"],"-2/1"],[["11","13","13","p2"],"-2/1"],[["11","13","p2","p3"],"6/1"
+> ],[["11","p0","p0","p2"],"-1/1"],[["11","p1","p1","p2"],"-5/1"],[["11","p2","p2","p2"],"-1/1"],[["11","p2
+> ","p3","p3"],"-1/1"],[["12","12","12","p1"],"-2/1"],[["12","12","p1","p2"],"6/1"],[["12","13","13","p1"],
+> "-2/1"],[["12","13","p1","p3"],"6/1"],[["12","p0","p0","p1"],"-1/1"],[["12","p1","p1","p1"],"-1/1"],[["12
+> ","p1","p2","p2"],"-5/1"],[["12","p1","p3","p3"],"-1/1"],[["13","p1","p2","p3"],"-4/1"],[["p0","p0","p1",
+> "p2"],"4/1"],[["p1","p1","p1","p2"],"4/1"],[["p1","p2","p2","p2"],"4/1"],[["p1","p2","p3","p3"],"4/1"]],"
+> 1,3":[[["10","10","11","13"],"4/1"],[["10","10","11","p3"],"-2/1"],[["10","10","13","p1"],"-2/1"],[["10",
+> "10","p1","p3"],"-4/1"],[["10","11","13","p0"],"-4/1"],[["10","11","p0","p3"],"6/1"],[["10","13","p0","p1
+> "],"6/1"],[["11","11","11","13"],"4/1"],[["11","11","11","p3"],"-2/1"],[["11","11","13","p1"],"-6/1"],[["
+> 11","11","p1","p3"],"6/1"],[["11","12","12","13"],"4/1"],[["11","12","12","p3"],"-2/1"],[["11","12","13",
+> "p2"],"-4/1"],[["11","12","p2","p3"],"6/1"],[["11","13","13","13"],"4/1"],[["11","13","13","p3"],"-6/1"],
+> [["11","13","p0","p0"],"-2/1"],[["11","13","p1","p1"],"4/1"],[["11","13","p2","p2"],"-2/1"],[["11","13","
+> p3","p3"],"4/1"],[["11","p0","p0","p3"],"-1/1"],[["11","p1","p1","p3"],"-5/1"],[["11","p2","p2","p3"],"-1
+> /1"],[["11","p3","p3","p3"],"-1/1"],[["12","12","13","p1"],"-2/1"],[["12","13","p1","p2"],"6/1"],[["12","
+> p1","p2","p3"],"-4/1"],[["13","13","13","p1"],"-2/1"],[["13","13","p1","p3"],"6/1"],[["13","p0","p0","p1"
+> ],"-1/1"],[["13","p1","p1","p1"],"-1/1"],[["13","p1","p2","p2"],"-1/1"],[["13","p1","p3","p3"],"-5/1"],[[
+> "p0","p0","p1","p3"],"4/1"],[["p1","p1","p1","p3"],"4/1"],[["p1","p2","p2","p3"],"4/1"],[["p1","p3","p3",
+> "p3"],"4/1"]],"1,4":[[["10","10","10","11"],"-2/1"],[["10","10","10","p1"],"1/1"],[["10","10","11","p0"],
+> "3/1"],[["10","10","p0","p1"],"-2/1"],[["10","11","11","11"],"-2/1"],[["10","11","11","p1"],"3/1"],[["10"
+> ,"11","12","12"],"2/1"],[["10","11","12","p2"],"-2/1"],[["10","11","13","13"],"2/1"],[["10","11","13","p3
+> "],"-2/1"],[["10","11","p0","p0"],"-3/1"],[["10","11","p1","p1"],"-1/1"],[["10","11","p2","p2"],"2/1"],[[
+> "10","11","p3","p3"],"2/1"],[["10","12","12","p1"],"-1/1"],[["10","12","p1","p2"],"1/1"],[["10","13","13"
+> ,"p1"],"-1/1"],[["10","13","p1","p3"],"1/1"],[["10","p0","p0","p1"],"2/1"],[["10","p1","p2","p2"],"-1/1"]
+> ,[["10","p1","p3","p3"],"-1/1"],[["11","11","11","p0"],"1/1"],[["11","11","p0","p1"],"-2/1"],[["11","11",
+> "12","p0"],"-1/1"],[["11","12","p0","p2"],"1/1"],[["11","13","13","p0"],"-1/1"],[["11","13","p0","p3"],"1
+> /1"],[["11","p0","p0","p0"],"1/1"],[["11","p0","p1","p1"],"1/1"],[["11","p0","p2","p2"],"-1/1"],[["11","p
+> 0","p3","p3"],"-1/1"],[["12","12","p0","p1"],"1/1"],[["12","p0","p1","p2"],"-1/1"],[["13","13","p0","p1"]
+> ,"1/1"],[["13","p0","p1","p3"],"-1/1"],[["p0","p0","p0","p1"],"-4/1"],[["p0","p1","p2","p2"],"4/1"],[["p0
```

```
+> ","p1","p3","p3"],"4/1"]],"1,5":[[["l0","l0","l0","12"],"-4/1"],[["l0","l0","l0","p2"],"2/1"],[["l0","l0"
+> ,"12","p0"],"6/1"],[["l0","l0","p0","p2"],"-6/1"],[["l0","l1","l1","12"],"-4/1"],[["l0","l1","l1","p2"],"
+> 2/1"],[["l0","l1","12","p1"],"4/1"],[["l0","l1","p1","p2"],"-6/1"],[["l0","12","12","12"],"4/1"],[["l0","
+> l2","12","p2"],"-6/1"],[["l0","12","13","13"],"4/1"],[["l0","12","13","p3"],"-4/1"],[["l0","12","p0","p0"
+> ],"-4/1"],[["l0","12","p1","p1"],"2/1"],[["l0","12","p2","p2"],"4/1"],[["l0","12","p3","p3"],"-2/1"],[["l
+> 0","13","13","p2"],"-2/1"],[["l0","13","p2","p3"],"6/1"],[["l0","p0","p0","p2"],"5/1"],[["l0","p1","p1","
+> p2"],"1/1"],[["l0","p2","p2","p2"],"-1/1"],[["l0","p2","p3","p3"],"-1/1"],[["l1","l1","12","p0"],"2/1"],[
+> ["l1","l1","p0","p2"],"4/1"],[["l1","12","p0","p1"],"-6/1"],[["12","12","12","p0"],"-2/1"],[["12","12","p
+> 0","p2"],"6/1"],[["12","13","13","p0"],"-2/1"],[["12","13","p0","p3"],"6/1"],[["12","p0","p0","p0"],"1/1"
+> ],[["12","p0","p1","p1"],"1/1"],[["12","p0","p2","p2"],"-5/1"],[["12","p0","p3","p3"],"-1/1"],[["13","p0"
+> ,"p2","p3"],"-4/1"],[["p0","p0","p0","p2"],"-4/1"],[["p0","p1","p1","p2"],"-4/1"],[["p0","p2","p2","p2"],
+> "4/1"],[["p0","p2","p3","p3"],"4/1"]],1,6":[[["l0","l0","l0","13"],"-4/1"],[["l0","l0","l0","p3"],"2/1"]
+> ,[["l0","l0","13","p0"],"6/1"],[["l0","l0","p0","p3"],"-6/1"],[["l0","l1","l1","13"],"-4/1"],[["l0","l1",
+> "l1","p3"],"2/1"],[["l0","l1","13","p1"],"4/1"],[["l0","l1","p1","p3"],"-6/1"],[["l0","12","12","13"],"4/
+> 1"],[["l0","12","12","p3"],"-2/1"],[["l0","12","13","p2"],"-4/1"],[["l0","12","p2","p3"],"6/1"],[["l0","l
+> 3","13","13"],"4/1"],[["l0","13","13","p3"],"-6/1"],[["l0","13","p0","p0"],"-4/1"],[["l0","13","p1","p1"]
+> ,"2/1"],[["l0","13","p2","p2"],"-2/1"],[["l0","13","p3","p3"],"4/1"],[["l0","p0","p0","p3"],"5/1"],[["l0"
+> ,"p1","p1","p3"],"1/1"],[["l0","p2","p2","p3"],"-1/1"],[["l0","p3","p3","p3"],"-1/1"],[["l1","l1","13","p
+> 0"],"2/1"],[["l1","l1","p0","p3"],"4/1"],[["l1","13","p0","p1"],"-6/1"],[["12","12","13","p0"],"-2/1"],[[
+> "12","13","p0","p2"],"6/1"],[["12","p0","p2","p3"],"-4/1"],[["13","13","13","p0"],"-2/1"],[["13","13","p0
+> ","p3"],"6/1"],[["13","p0","p0","p0"],"1/1"],[["13","p0","p1","p1"],"1/1"],[["13","p0","p2","p2"],"-1/1"]
+> ,[["13","p0","p3","p3"],"-5/1"],[["p0","p0","p0","p3"],"-4/1"],[["p0","p1","p1","p3"],"-4/1"],[["p0","p2"
+> ,"p2","p3"],"4/1"],[["p0","p3","p3","p3"],"4/1"]],"1,7":[[["l0","l0","l0","11"],"2/1"],[["l0","l0","l0","
+> p1"],"-1/1"],[["l0","l0","l1","p0"],"-3/1"],[["l0","l0","p0","p1"],"4/1"],[["l0","l1","l1","l1"],"-2/1"],
+> [["l0","l1","l1","p1"],"3/1"],[["l0","l1","12","12"],"-6/1"],[["l0","l1","12","p2"],"6/1"],[["l0","l1","l
+> 3","13"],"-2/1"],[["l0","l1","13","p3"],"2/1"],[["l0","l1","p0","p0"],"1/1"],[["l0","l1","p1","p1"],"-1/1
+> "],[["l0","l1","p2","p2"],"-4/1"],[["l0","l1","p3","p3"],"4/1"],[["l0","12","12","p1"],"3/1"],[["l0","12"
+> ,"p1","p2"],"-1/1"],[["l0","l3","13","p1"],"1/1"],[["l0","l3","p1","p3"],"-5/1"],[["l0","p0","p0","p1","
+> -3/1"],[["l0","p1","p2","p2"],"1/1"],[["l1","l1","l1","p0"],"1/1"],[["l1","l1","p0","p1"],"-4/1"],[["l1",
+> "l2","12","p0"],"3/1"],[["l1","12","p0","p2"],"-1/1"],[["l1","l3","13","p0"],"1/1"],[["l1","13","p0","p3"
+> ],"-5/1"],[["l1","p0","p1","p1"],"3/1"],[["l1","p0","p2","p2"],"1/1"],[["12","12","p0","p1"],"-7/1"],[["1
+> 2","p0","p1","p2"],"5/1"],[["13","13","p0","p1"],"1/1"],[["13","p0","p1","p3"],"3/1"],[["p0","p1","p2","p
+> 2"],"-4/1"]],"1,8":[[["l0","l1","12","13"],"-8/1"],[["l0","l1","12","p3"],"4/1"],[["l0","l1","13","p2"],"
+> 4/1"],[["l0","l1","p2","p3"],"-16/1"],[["l0","12","13","p1"],"4/1"],[["l0","12","p1","p3"],"4/1"],[["l0",
+> "13","p1","p2"],"4/1"],[["l0","p1","p2","p3"],"2/1"],[["l1","12","13","p0"],"4/1"],[["l1","12","p0","p3"]
+> ,"4/1"],[["l1","13","p0","p2"],"4/1"],[["l1","p0","p2","p3"],"2/1"],[["12","13","p0","p1"],"-16/1"],[["12
+> ","p0","p1","p3"],"2/1"],[["13","p0","p1","p2"],"2/1"],[["p0","p1","p2","p3"],"-8/1"]],"1,9":[[["l0","l0"
+> ,"l0","l1"],"2/1"],[["l0","l0","l0","p1"],"-1/1"],[["l0","l0","l1","p0"],"-3/1"],[["l0","l0","p0","p1"],"
+> 4/1"],[["l0","l1","l1","l1"],"-2/1"],[["l0","l1","l1","p1"],"3/1"],[["l0","l1","12","12"],"-2/1"],[["l0",
+> "l1","12","p2"],"2/1"],[["l0","l1","13","13"],"-6/1"],[["l0","l1","13","p3"],"6/1"],[["l0","l1","p0","p0"
+> ],"1/1"],[["l0","l1","p1","p1"],"-1/1"],[["l0","l1","p2","p2"],"4/1"],[["l0","l1","p3","p3"],"-4/1"],[["l
+> 0","12","12","p1"],"1/1"],[["l0","12","p1","p2"],"-5/1"],[["l0","13","13","p1"],"3/1"],[["l0","13","p1","
+> p3"],"-1/1"],[["l0","p0","p0","p1"],"-3/1"],[["l0","p1","p3","p3"],"1/1"],[["l1","l1","l1","p0"],"1/1"],[
+> ["l1","l1","p0","p1"],"-4/1"],[["l1","12","12","p0"],"1/1"],[["l1","12","p0","p2"],"-5/1"],[["l1","13","l
+> 3","p0"],"3/1"],[["l1","13","p0","p3"],"-1/1"],[["l1","p0","p1","p1"],"3/1"],[["l1","p0","p3","p3"],"1/1"
+> ],[["12","12","p0","p1"],"1/1"],[["12","p0","p1","p2"],"3/1"],[["13","13","p0","p1"],"-7/1"],[["13","p0",
+> "p1","p3"],"5/1"],[["p0","p1","p3","p3"],"-4/1"]],"2,0":[[["l0","l0","l0","12"],"-2/1"],[["l0","l0","l0",
+> "p2"],"1/1"],[["l0","l0","12","p0"],"3/1"],[["l0","l0","p0","p2"],"-2/1"],[["l0","l1","l1","12"],"-2/1",
+> [["l0","l1","l1","p2"],"1/1"],[["l0","l1","12","p1"],"2/1"],[["l0","l1","p1","p2"],"-1/1"],[["l0","12","l
+> 2","12"],"-2/1"],[["l0","12","12","p2"],"3/1"],[["l0","12","13","13"],"-2/1"],[["l0","12","13","p3"],"2/1
+> "],[["l0","12","p0","p0"],"-1/1"],[["l0","12","p1","p1"],"-2/1"],[["l0","12","p2","p2"],"-3/1"],[["l0","l
+> 2","p3","p3"],"-2/1"],[["l0","13","13","p2"],"1/1"],[["l0","13","p2","p3"],"-1/1"],[["l0","p0","p0","p2"]
+> ,"1/1"],[["l0","p1","p1","p2"],"1/1"],[["l0","p2","p2","p2"],"1/1"],[["l0","p2","p3","p3"],"1/1"],[["l1",
+> "l1","12","p0"],"1/1"],[["l1","l1","p0","p2"],"-1/1"],[["l1","l2","p0","p1"],"-1/1"],[["l1","p0","p1","p2
+> "],"1/1"],[["l2","12","12","p0"],"1/1"],[["12","12","p0","p2"],"-2/1"],[["12","13","13","p0"],"1/1"],[["1
+> 2","13","p0","p3"],"-1/1"],[["12","p0","p1","p1"],"1/1"],[["12","p0","p2","p2"],"2/1"],[["12","p0","p3","
+> p3"],"1/1"],[["13","13","p0","p2"],"-1/1"],[["13","p0","p2","p3"],"1/1"],[["p0","p1","p1","p2"],"-4/1"],[
+> ["p0","p2","p2","p2"],"-4/1"],[["p0","p2","p3","p3"],"-4/1"]],"2,1":[[["l0","l0","l1","12"],"4/1"],[["l0"
+> ,"l0","l1","p2"],"-2/1"],[["l0","l0","12","p1"],"-2/1"],[["l0","l0","p1","p2"],"-4/1"],[["l0","l1","12","
+> p0"],"-4/1"],[["l0","l1","p0","p2"],"6/1"],[["l0","12","p0","p1"],"6/1"],[["l1","l1","l1","12"],"4/1"],[[
+> "l1","l1","l1","p2"],"-2/1"],[["l1","l1","12","p1"],"-6/1"],[["l1","l1","p1","p2"],"6/1"],[["l1","12","12
+> ","12"],"4/1"],[["l1","12","12","p2"],"-6/1"],[["l1","12","13","13"],"4/1"],[["l1","12","13","p3"],"-4/1"
+> ],[["l1","12","p0","p0"],"-2/1"],[["l1","12","p1","p1"],"4/1"],[["l1","12","p2","p2"],"4/1"],[["l1","12",
+> "p3","p3"],"-2/1"],[["l1","13","13","p2"],"-2/1"],[["l1","13","p2","p3"],"6/1"],[["l1","p0","p0","p2"],"-
+> 1/1"],[["l1","p1","p1","p2"],"-5/1"],[["l1","p2","p2","p2"],"-1/1"],[["l1","p2","p3","p3"],"-1/1"],[["12"
+> ,"12","12","p1"],"-2/1"],[["12","12","p1","p2"],"6/1"],[["12","13","13","p1"],"-2/1"],[["12","13","p1","p
+> 3"],"6/1"],[["12","p0","p0","p1"],"-1/1"],[["12","p1","p1","p1"],"-1/1"],[["12","p1","p2","p2"],"-5/1"],[
+> ["12","p1","p3","p3"],"-1/1"],[["13","p1","p2","p3"],"-4/1"],[["p0","p0","p1","p2"],"4/1"],[["p1","p1","p
+> 1","p2"],"4/1"],[["p1","p2","p2","p2"],"4/1"],[["p1","p2","p3","p3"],"4/1"]],"2,2":[[["l0","l0","l1","l1"
+> ],"4/1"],[["l0","l0","l1","p1"],"-4/1"],[["l0","l0","12","12"],"8/1"],[["l0","l0","12","p2"],"-8/1"],[["l
+> 0","l0","13","13"],"4/1"],[["l0","l0","13","p3"],"-4/1"],[["l0","l0","p1","p1"],"4/1"],[["l0","l0","p3","
+> p3"],"4/1"],[["l0","l1","l1","p0"],"-4/1"],[["l0","l1","p0","p1"],"4/1"],[["l0","12","12","p0"],"-8/1"],[
+> ["l0","12","p0","p2"],"16/1"],[["l0","13","13","p0"],"-4/1"],[["l0","13","p0","p3"],"4/1"],[["l0","p0","p
+> 1","p1"],"-4/1"],[["l0","p0","p2","p2"],"-4/1"],[["l0","p0","p3","p3"],"-4/1"],[["l1","l1","l1","l1"],"-4
+> /1"],[["l1","l1","l1","p1"],"8/1"],[["l1","l1","12","12"],"-4/1"],[["l1","l1","12","p2"],"4/1"],[["l1","l
+> 1","13","13"],"-8/1"],[["l1","l1","13","p3"],"8/1"],[["l1","l1","p0","p0"],"2/1"],[["l1","l1","p1","p1"],
+> "-10/1"],[["l1","l1","p2","p2"],"-2/1"],[["l1","l1","p3","p3"],"-2/1"],[["l1","12","12","p1"],"4/1"],[["l
+> 1","12","p1","p2"],"-4/1"],[["l1","13","13","p1"],"8/1"],[["l1","13","p1","p3"],"-16/1"],[["l1","p0","p0"
+> ,"p1"],"-2/1"],[["l1","p1","p1","p1"],"6/1"],[["l1","p1","p2","p2"],"2/1"],[["l1","p1","p3","p3"],"6/1"],
+> [["12","12","13","13"],"-4/1"],[["12","12","13","p3"],"4/1"],[["12","12","p1","p1"],"-4/1"],[["12","12","
+> p3","p3"],"-4/1"],[["12","13","13","p2"],"4/1"],[["12","13","p2","p3"],"-4/1"],[["12","p0","p0","p2"],"-4
```

```
+> /1"],[["12","p1","p1","p2"],"4/1"],[["12","p2","p3","p3"],"4/1"],[["13","13","13","13"],"-4/1"],[["13","1
+> 3","13","p3"],"8/1"],[["13","13","p0","p0"],"2/1"],[["13","13","p1","p1"],"-2/1"],[["13","13","p2","p2"],
+> "-2/1"],[["13","13","p3","p3"],"-10/1"],[["13","p0","p0","p3"],"-2/1"],[["13","p1","p1","p3"],"6/1"],[["1
+> 3","p2","p2","p3"],"2/1"],[["13","p3","p3","p3"],"6/1"],[["p0","p0","p1","p1"],"4/1"],[["p0","p0","p2","p
+> 2"],"8/1"],[["p0","p0","p3","p3"],"4/1"],[["p1","p1","p1","p1"],"-4/1"],[["p1","p1","p2","p2"],"-4/1"],[[
+> "p1","p1","p3","p3"],"-8/1"],[["p2","p2","p3","p3"],"-4/1"],[["p3","p3","p3","p3"],"-4/1"]],"2,3":[[["10"
+> ,"10","12","13"],"4/1"],[["10","10","12","p3"],"-2/1"],[["10","10","13","p2"],"-2/1"],[["10","10","p2","p
+> 3"],"-4/1"],[["10","12","13","p0"],"-4/1"],[["10","12","p0","p3"],"6/1"],[["10","13","p0","p2"],"6/1"],[[
+> "11","11","12","13"],"4/1"],[["11","11","12","p3"],"-2/1"],[["11","11","13","p2"],"-2/1"],[["11","12","13
+> ","p1"],"-4/1"],[["11","12","p1","p3"],"6/1"],[["11","13","p1","p2"],"6/1"],[["11","p1","p2","p3"],"-4/1"
+> ],[["12","12","12","13"],"4/1"],[["12","12","12","p3"],"-2/1"],[["12","12","13","p2"],"-6/1"],[["12","12"
+> ,"p2","p3"],"6/1"],[["12","13","13","13"],"4/1"],[["12","13","13","p3"],"-6/1"],[["12","13","p0","p0"],"-
+> 2/1"],[["12","13","p1","p1"],"-2/1"],[["12","13","p2","p2"],"4/1"],[["12","13","p3","p3"],"4/1"],[["12","
+> p0","p0","p3"],"-1/1"],[["12","p1","p1","p3"],"-1/1"],[["12","p2","p2","p3"],"-5/1"],[["12","p3","p3","p3
+> "],"-1/1"],[["13","13","13","p2"],"-2/1"],[["13","13","p2","p3"],"6/1"],[["13","p0","p0","p2"],"-1/1"],[[
+> "13","p1","p1","p2"],"-1/1"],[["13","p2","p2","p2"],"-1/1"],[["13","p2","p3","p3"],"-5/1"],[["p0","p0","p
+> 2","p3"],"4/1"],[["p1","p1","p2","p3"],"4/1"],[["p2","p2","p2","p3"],"4/1"],[["p2","p3","p3","p3"],"4/1"]
+> ],"2,4":[[["10","10","10","12"],"2/1"],[["10","10","10","p2"],"-1/1"],[["10","10","12","p0"],"-3/1"],[["1
+> 0","10","p0","p2"],"4/1"],[["10","11","11","12"],"-6/1"],[["10","11","11","p2"],"3/1"],[["10","11","12","
+> p1"],"6/1"],[["10","11","p1","p2"],"-1/1"],[["10","12","12","12"],"-2/1"],[["10","12","12","p2"],"3/1"],[
+> ["10","12","13","13"],"-2/1"],[["10","12","13","p3"],"2/1"],[["10","12","p0","p0"],"1/1"],[["10","12","p1
+> ","p1"],"-4/1"],[["10","12","p2","p2"],"-1/1"],[["10","12","p3","p3"],"4/1"],[["10","13","13","p2"],"1/1"
+> ],[["10","13","p2","p3"],"-5/1"],[["10","p0","p0","p2"],"-3/1"],[["10","p1","p1","p2"],"1/1"],[["11","11"
+> ,"12","p0"],"3/1"],[["11","11","p0","p2"],"-7/1"],[["11","12","p0","p1"],"-1/1"],[["11","p0","p1","p2"],"
+> 5/1"],[["12","12","12","p0"],"1/1"],[["12","12","p0","p2"],"-4/1"],[["12","13","13","p0"],"1/1"],[["12","
+> 13","p0","p3"],"-5/1"],[["12","p0","p1","p1"],"1/1"],[["12","p0","p2","p2"],"3/1"],[["13","13","p0","p2"]
+> ,"1/1"],[["13","p0","p2","p3"],"3/1"],[["p0","p1","p1","p2"],"-4/1"]],"2,5":[[["10","10","10","11"],"-4/1
+> "],[["10","10","10","p1"],"2/1"],[["10","10","11","p0"],"6/1"],[["10","10","p0","p1"],"-6/1"],[["10","11"
+> ,"11","11"],"4/1"],[["10","11","11","p1"],"-6/1"],[["10","11","12","12"],"-4/1"],[["10","11","12","p2"],"
+> 4/1"],[["10","11","13","13"],"4/1"],[["10","11","13","p3"],"-4/1"],[["10","11","p0","p0"],"-4/1"],[["10",
+> "11","p1","p1"],"4/1"],[["10","11","p2","p2"],"2/1"],[["10","11","p3","p3"],"-2/1"],[["10","12","12","p1"
+> ],"2/1"],[["10","12","p1","p2"],"-6/1"],[["10","13","13","p1"],"-2/1"],[["10","13","p1","p3"],"6/1"],[["1
+> 0","p0","p0","p1"],"5/1"],[["10","p1","p1","p1"],"-1/1"],[["10","p1","p2","p2"],"1/1"],[["10","p1","p3","
+> p3"],"-1/1"],[["11","11","11","p0"],"-2/1"],[["11","11","p0","p1"],"6/1"],[["11","12","12","p0"],"2/1"],[
+> ["11","12","p0","p2"],"-6/1"],[["11","13","13","p0"],"-2/1"],[["11","13","p0","p3"],"6/1"],[["11","p0","p
+> 0","p0"],"1/1"],[["11","p0","p1","p1"],"-5/1"],[["11","p0","p2","p2"],"1/1"],[["11","p0","p3","p3"],"-1/1
+> "],[["12","12","p0","p1"],"4/1"],[["13","p0","p1","p3"],"-4/1"],[["p0","p0","p0","p1"],"-4/1"],[["p0","p1
+> ","p1","p1"],"4/1"],[["p0","p1","p2","p2"],"-4/1"],[["p0","p1","p3","p3"],"4/1"]],"2,6":[[["10","11","12"
+> ,"13"],"-8/1"],[["10","11","12","p3"],"4/1"],[["10","11","13","p2"],"4/1"],[["10","11","p2","p3"],"4/1"],
+> [["10","12","13","p1"],"4/1"],[["10","12","p1","p3"],"-16/1"],[["10","13","p1","p2"],"4/1"],[["10","p1","
+> p2","p3"],"2/1"],[["11","12","13","p0"],"4/1"],[["11","12","p0","p3"],"4/1"],[["11","13","p0","p2"],"-16/
+> 1"],[["11","p0","p2","p3"],"2/1"],[["12","13","p0","p1"],"4/1"],[["12","p0","p1","p3"],"2/1"],[["13","p0"
+> ,"p1","p2"],"2/1"],[["p0","p1","p2","p3"],"-8/1"]],"2,7":[[["10","10","10","12"],"-2/1"],[["10","10","10"
+> ,"p2"],"1/1"],[["10","10","12","p0"],"3/1"],[["10","10","p0","p2"],"-2/1"],[["10","11","11","12"],"2/1"],
+> [["10","11","11","p2"],"-1/1"],[["10","11","12","p1"],"-2/1"],[["10","11","p1","p2"],"1/1"],[["10","12","
+> 12","12"],"-2/1"],[["10","12","12","p2"],"3/1"],[["10","12","13","13"],"2/1"],[["10","12","13","p3"],"-2/
+> 1"],[["10","12","p0","p0"],"-3/1"],[["10","12","p1","p1"],"2/1"],[["10","12","p2","p2"],"-1/1"],[["10","1
+> 2","p3","p3"],"2/1"],[["10","13","13","p2"],"-1/1"],[["10","13","p2","p3"],"1/1"],[["10","p0","p0","p2"],
+> "2/1"],[["10","p1","p1","p2"],"-1/1"],[["10","p2","p3","p3"],"-1/1"],[["11","11","12","p0"],"-1/1"],[["11
+> ","11","p0","p2"],"1/1"],[["11","12","p0","p1"],"1/1"],[["11","p0","p1","p2"],"-1/1"],[["12","12","12","p
+> 0"],"1/1"],[["12","12","p0","p2"],"-2/1"],[["12","13","13","p0"],"-1/1"],[["12","13","p0","p3"],"1/1"],[[
+> "12","p0","p0","p0"],"1/1"],[["12","p0","p1","p1"],"-1/1"],[["12","p0","p2","p2"],"1/1"],[["12","p0","p3"
+> ,"p3"],"-1/1"],[["13","13","p0","p2"],"1/1"],[["13","p0","p2","p3"],"-1/1"],[["p0","p0","p0","p2"],"-4/1"
+> ],[["p0","p1","p1","p2"],"4/1"],[["p0","p2","p3","p3"],"4/1"]],"2,8":[[["10","10","10","13"],"-4/1"],[["1
+> 0","10","10","p3"],"2/1"],[["10","10","13","p0"],"6/1"],[["10","10","p0","p3"],"-6/1"],[["10","11","11","
+> 13"],"4/1"],[["10","11","11","p3"],"-2/1"],[["10","11","13","p1"],"-4/1"],[["10","11","p1","p3"],"6/1"],[
+> ["10","12","12","13"],"-4/1"],[["10","12","12","p3"],"2/1"],[["10","12","13","p2"],"4/1"],[["10","12","p2
+> ","p3"],"-6/1"],[["10","13","13","13"],"4/1"],[["10","13","13","p3"],"-6/1"],[["10","13","p0","p0"],"-4/1
+> "],[["10","13","p1","p1"],"-2/1"],[["10","13","p2","p2"],"2/1"],[["10","13","p3","p3"],"4/1"],[["10","p0"
+> ,"p0","p3"],"5/1"],[["10","p1","p1","p3"],"-1/1"],[["10","p2","p2","p3"],"1/1"],[["10","p3","p3","p3"],"-
+> 1/1"],[["11","11","13","p0"],"-2/1"],[["11","13","p0","p1"],"6/1"],[["11","p0","p1","p3"],"-4/1"],[["12",
+> "12","13","p0"],"2/1"],[["12","12","p0","p3"],"4/1"],[["12","13","p0","p2"],"-6/1"],[["13","13","13","p0"
+> ],"-2/1"],[["13","13","p0","p3"],"6/1"],[["13","p0","p0","p0"],"1/1"],[["13","p0","p1","p1"],"-1/1"],[["1
+> 3","p0","p2","p2"],"1/1"],[["13","p0","p3","p3"],"-5/1"],[["p0","p0","p0","p3"],"-4/1"],[["p0","p1","p1",
+> "p3"],"4/1"],[["p0","p2","p2","p3"],"-4/1"],[["p0","p3","p3","p3"],"4/1"]],"2,9":[[["10","10","10","12"],
+> "2/1"],[["10","10","10","p2"],"-1/1"],[["10","10","12","p0"],"-3/1"],[["10","10","p0","p2"],"4/1"],[["10"
+> ,"11","11","12"],"-2/1"],[["10","11","11","p2"],"1/1"],[["10","11","12","p1"],"2/1"],[["10","11","p1","p2
+> "],"-5/1"],[["10","12","12","12"],"-2/1"],[["10","12","12","p2"],"3/1"],[["10","12","13","13"],"-6/1"],[[
+> "10","12","13","p3"],"6/1"],[["10","12","p0","p0"],"1/1"],[["10","12","p1","p1"],"4/1"],[["10","12","p2",
+> "p2"],"-1/1"],[["10","12","p3","p3"],"-4/1"],[["10","13","13","p2"],"3/1"],[["10","13","p2","p3"],"-1/1"
+> ,[["10","p0","p0","p2"],"-3/1"],[["10","p2","p3","p3"],"1/1"],[["11","11","12","p0"],"1/1"],[["11","11","
+> p0","p2"],"1/1"],[["11","12","p0","p1"],"-5/1"],[["11","p0","p1","p2"],"3/1"],[["12","12","12","p0"],"1/1
+> "],[["12","12","p0","p2"],"-4/1"],[["12","13","13","p0"],"3/1"],[["12","13","p0","p3"],"-1/1"],[["12","p0
+> ","p2","p2"],"3/1"],[["12","p0","p3","p3"],"1/1"],[["13","13","p0","p2"],"-7/1"],[["13","p0","p2","p3"],"
+> 5/1"],[["p0","p2","p3","p3"],"-4/1"]],"3,0":[[["10","10","10","13"],"-2/1"],[["10","10","10","p3"],"1/1"]
+> ,[["10","10","13","p0"],"3/1"],[["10","10","p0","p3"],"-2/1"],[["10","11","11","13"],"-2/1"],[["10","11",
+> "11","p3"],"1/1"],[["10","11","13","p1"],"2/1"],[["10","11","p1","p3"],"-1/1"],[["10","12","12","13"],"-2
+> /1"],[["10","12","12","p3"],"1/1"],[["10","12","13","p2"],"2/1"],[["10","12","p2","p3"],"-1/1"],[["10","1
+> 3","13","13"],"-2/1"],[["10","13","13","p3"],"3/1"],[["10","13","p0","p0"],"-1/1"],[["10","13","p1","p1"]
+> ,"-2/1"],[["10","13","p2","p2"],"-2/1"],[["10","13","p3","p3"],"-3/1"],[["10","p0","p0","p3"],"1/1"],[["1
+> 0","p1","p1","p3"],"1/1"],[["10","p2","p2","p3"],"1/1"],[["10","p3","p3","p3"],"1/1"],[["11","11","13","p
```

+> 0"],"1/1"],[["l1","l1","p0","p3"],"-1/1"],[["l1","l3","p0","p1"],"-1/1"],[["l1","p0","p1","p3"],"1/1"],[[
+> "l2","l2","l3","p0"],"1/1"],[["l2","l2","p0","p3"],"-1/1"],[["l2","l3","p0","p2"],"-1/1"],[["l2","p0","p2
+> ","p3"],"1/1"],[["l3","l3","l3","p0"],"1/1"],[["l3","l3","p0","p3"],"-2/1"],[["l3","p0","p1","p1"],"1/1"]
+> ,[["l3","p0","p2","p2"],"1/1"],[["l3","p0","p3","p3"],"2/1"],[["p0","p1","p1","p3"],"-4/1"],[["p0","p2","
+> p2","p3"],"-4/1"],[["p0","p3","p3","p3"],"-4/1"]],"3,1":[[["l0","l0","l1","l3"],"4/1"],[["l0","l0","l1","
+> p3"],"-2/1"],[["l0","l0","l3","p1"],"-2/1"],[["l0","l0","p1","p3"],"-4/1"],[["l0","l1","l3","p0"],"-4/1"]
+> ,[["l0","l1","p0","p3"],"6/1"],[["l0","l3","p0","p1"],"6/1"],[["l1","l1","l1","l3"],"4/1"],[["l1","l1","l
+> 1","p3"],"-2/1"],[["l1","l1","l3","p1"],"-6/1"],[["l1","l1","p1","p3"],"6/1"],[["l1","l2","l2","l3"],"4/1
+> "],[["l1","l2","l2","p3"],"-2/1"],[["l1","l2","l3","p2"],"-4/1"],[["l1","l2","p2","p3"],"6/1"],[["l1","l3
+> ","l3","l3"],"4/1"],[["l1","l3","l3","p3"],"-6/1"],[["l1","l3","p0","p0"],"-2/1"],[["l1","l3","p1","p1"],
+> "4/1"],[["l1","l3","p2","p2"],"-2/1"],[["l1","l3","p3","p3"],"4/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1"
+> ,"p1","p1","p3"],"-5/1"],[["l1","p2","p2","p3"],"-1/1"],[["l1","p3","p3","p3"],"-1/1"],[["l2","l2","l3","
+> p1"],"-2/1"],[["l2","l3","p1","p2"],"6/1"],[["l2","p1","p2","p3"],"-4/1"],[["l3","l3","l3","p1"],"-2/1"],
+> [["l3","l3","p1","p3"],"6/1"],[["l3","p0","p0","p1"],"-1/1"],[["l3","p1","p1","p1"],"-1/1"],[["l3","p1","
+> p2","p2"],"-1/1"],[["l3","p1","p3","p3"],"-5/1"],[["p0","p0","p1","p3"],"4/1"],[["p1","p1","p1","p3"],"4/
+> 1"],[["p1","p2","p2","p3"],"4/1"],[["p1","p3","p3","p3"],"4/1"]],"3,2":[[["l0","l0","l2","l3"],"4/1"],[["
+> l0","l0","l2","p3"],"-2/1"],[["l0","l0","l3","p2"],"-2/1"],[["l0","l0","p2","p3"],"-4/1"],[["l0","l2","l3
+> ","p0"],"-4/1"],[["l0","l2","p0","p3"],"6/1"],[["l0","l3","p0","p2"],"6/1"],[["l1","l1","l2","l3"],"4/1"]
+> ,[["l1","l1","l2","p3"],"-2/1"],[["l1","l1","l3","p2"],"-2/1"],[["l1","l2","l3","p1"],"-4/1"],[["l1","l2"
+> ,"p1","p3"],"6/1"],[["l1","l3","p1","p2"],"6/1"],[["l1","p1","p2","p3"],"-4/1"],[["l2","l2","l2","l3"],"4
+> /1"],[["l2","l2","l2","p3"],"-2/1"],[["l2","l2","l3","p2"],"-6/1"],[["l2","l2","p2","p3"],"6/1"],[["l2","
+> l3","l3","l3"],"4/1"],[["l2","l3","l3","p3"],"-6/1"],[["l2","l3","p0","p0"],"-2/1"],[["l2","l3","p1","p1"
+>],"-2/1"],[["l2","l3","p2","p2"],"4/1"],[["l2","l3","p3","p3"],"4/1"],[["l2","p0","p0","p3"],"-1/1"],[["l
+> 2","p1","p1","p3"],"-1/1"],[["l2","p2","p2","p3"],"-5/1"],[["l2","p3","p3","p3"],"-1/1"],[["l3","l3","l3"
+> ,"p2"],"-2/1"],[["l3","l3","p2","p3"],"6/1"],[["l3","p0","p0","p2"],"-1/1"],[["l3","p1","p1","p2"],"-1/1"
+>],[["l3","p2","p2","p2"],"-1/1"],[["l3","p2","p3","p3"],"-5/1"],[["p0","p0","p2","p3"],"4/1"],[["p1","p1"
+> ,"p2","p3"],"4/1"],[["p2","p2","p2","p3"],"4/1"],[["p2","p3","p3","p3"],"4/1"]],"3,3":[[["l0","l0","l1","
+> l1"],"4/1"],[["l0","l0","l1","p1"],"-4/1"],[["l0","l0","l2","l2"],"4/1"],[["l0","l0","l2","p2"],"-4/1"],[
+> ["l0","l0","l3","l3"],"8/1"],[["l0","l0","l3","p3"],"-8/1"],[["l0","l0","p1","p1"],"4/1"],[["l0","l0","p2
+> ","p2"],"4/1"],[["l0","l1","l1","p0"],"-4/1"],[["l0","l1","p0","p1"],"4/1"],[["l0","l2","l2","p0"],"-4/1"
+>],[["l0","l2","p0","p2"],"4/1"],[["l0","l3","l3","p0"],"-8/1"],[["l0","l3","p0","p3"],"16/1"],[["l0","p0"
+> ,"p1","p1"],"-4/1"],[["l0","p0","p2","p2"],"-4/1"],[["l0","p0","p3","p3"],"-4/1"],[["l1","l1","l1","l1"],
+> "-4/1"],[["l1","l1","l1","p1"],"8/1"],[["l1","l1","l2","l2"],"-8/1"],[["l1","l1","l2","p2"],"8/1"],[["l1"
+> ,"l1","l3","l3"],"-4/1"],[["l1","l1","l3","p3"],"4/1"],[["l1","l1","p0","p0"],"2/1"],[["l1","l1","p1","p1
+> "],"-10/1"],[["l1","l1","p2","p2"],"-2/1"],[["l1","l1","p3","p3"],"-2/1"],[["l1","l2","l2","p1"],"8/1"],[
+> ["l1","l2","p1","p2"],"-16/1"],[["l1","l3","l3","p1"],"4/1"],[["l1","l3","p1","p3"],"-4/1"],[["l1","p0","
+> p0","p1"],"-2/1"],[["l1","p1","p1","p1"],"6/1"],[["l1","p1","p2","p2"],"6/1"],[["l1","p1","p3","p3"],"2/1
+> "],[["l2","l2","l2","l2"],"-4/1"],[["l2","l2","l2","p2"],"8/1"],[["l2","l2","l3","l3"],"-4/1"],[["l2","l2
+> ","l3","p3"],"4/1"],[["l2","l2","p0","p0"],"2/1"],[["l2","l2","p1","p1"],"-2/1"],[["l2","l2","p2","p2"],"
+> -10/1"],[["l2","l2","p3","p3"],"-2/1"],[["l2","l3","l3","p2"],"4/1"],[["l2","l3","p2","p3"],"-4/1"],[["l2
+> ","p0","p0","p2"],"-2/1"],[["l2","p1","p1","p2"],"6/1"],[["l2","p2","p2","p2"],"6/1"],[["l2","p2","p3","p
+> 3"],"2/1"],[["l3","l3","p1","p1"],"-4/1"],[["l3","l3","p2","p2"],"-4/1"],[["l3","p0","p0","p3"],"-4/1"],[
+> ["l3","p1","p1","p3"],"4/1"],[["l3","p2","p2","p3"],"4/1"],[["p0","p0","p1","p1"],"4/1"],[["p0","p0","p2"
+> ,"p2"],"4/1"],[["p0","p0","p3","p3"],"8/1"],[["p1","p1","p1","p1"],"-4/1"],[["p1","p1","p2","p2"],"-8/1"]
+> ,[["p1","p1","p3","p3"],"-4/1"],[["p2","p2","p2","p2"],"-4/1"],[["p2","p2","p3","p3"],"-4/1"]],"3,4":[[["
+> l0","l0","l0","l3"],"2/1"],[["l0","l0","l0","p3"],"-1/1"],[["l0","l0","l3","p0"],"-3/1"],[["l0","l0","p0"
+> ,"p3"],"4/1"],[["l0","l1","l1","l3"],"-6/1"],[["l0","l1","l1","p3"],"3/1"],[["l0","l1","l3","p1"],"6/1"],
+> [["l0","l1","p1","p3"],"-1/1"],[["l0","l2","l2","l3"],"-2/1"],[["l0","l2","l2","p3"],"1/1"],[["l0","l2","
+> l3","p2"],"2/1"],[["l0","l2","p2","p3"],"-5/1"],[["l0","l3","l3","l3"],"-2/1"],[["l0","l3","l3","p3"],"3/
+> 1"],[["l0","l3","p0","p0"],"1/1"],[["l0","l3","p1","p1"],"-4/1"],[["l0","l3","p2","p2"],"4/1"],[["l0","l3
+> ","p3","p3"],"-1/1"],[["l0","p0","p0","p3"],"-3/1"],[["l0","p1","p1","p3"],"1/1"],[["l1","l1","l3","p0"],
+> "3/1"],[["l1","l1","p0","p3"],"-7/1"],[["l1","l3","p0","p1"],"-1/1"],[["l1","p0","p1","p3"],"5/1"],[["l2"
+> ,"l2","l3","p0"],"1/1"],[["l2","l2","p0","p3"],"1/1"],[["l2","l3","p0","p2"],"-5/1"],[["l2","p0","p2","p3
+> "],"3/1"],[["l3","l3","l3","p0"],"1/1"],[["l3","l3","p0","p3"],"-4/1"],[["l3","p0","p1","p1"],"1/1"],[["l
+> 3","p0","p3","p3"],"3/1"],[["p0","p1","p1","p3"],"-4/1"]],"3,5":[[["l0","l1","l2","l3"],"-8/1"],[["l0","l
+> 1","l2","p3"],"4/1"],[["l0","l1","l3","p2"],"4/1"],[["l0","l1","p2","p3"],"4/1"],[["l0","l2","l3","p1"],"
+> 4/1"],[["l0","l2","p1","p3"],"4/1"],[["l0","l3","p1","p2"],"-16/1"],[["l0","p1","p2","p3"],"2/1"],[["l1",
+> "l2","l3","p0"],"4/1"],[["l1","l2","p0","p3"],"-16/1"],[["l1","l3","p1","p2"],"-16/1"],[["l1","p0","p2","
+> p3"],"2/1"],[["l2","l3","p0","p1"],"4/1"],[["l2","p0","p1","p3"],"2/1"],[["l3","p0","p1","p2"],"2/1"],[["p0
+> ","p1","p2","p3"],"-8/1"]],"3,6":[[["l0","l0","l0","l1"],"-4/1"],[["l0","l0","l0","p1"],"2/1"],[["l0","l0
+> ","l1","p0"],"6/1"],[["l0","l0","p0","p1"],"-6/1"],[["l0","l1","l1","l1"],"4/1"],[["l0","l1","l1","p1","
+> -6/1"],[["l0","l1","l2","l2"],"4/1"],[["l0","l1","l2","p2"],"-4/1"],[["l0","l1","l3","l3"],"-4/1"],[["l0"
+> ,"l1","l3","p3"],"4/1"],[["l0","l1","p0","p0"],"-4/1"],[["l0","l1","p1","p1"],"4/1"],[["l0","l1","p2","p2
+> "],"-2/1"],[["l0","l1","p3","p3"],"2/1"],[["l0","l2","l2","p1"],"-2/1"],[["l0","l2","p1","p2"],"6/1"],[["
+> l0","l3","l3","p1"],"2/1"],[["l0","l3","p1","p3"],"-6/1"],[["l0","p0","p0","p1"],"5/1"],[["l0","p1","p1",
+> "p1"],"-1/1"],[["l0","p1","p2","p2"],"-1/1"],[["l0","p1","p3","p3"],"1/1"],[["l1","l1","l1","p0"],"-2/1"]
+> ,[["l1","l1","p0","p1"],"6/1"],[["l1","l2","l2","p0"],"-2/1"],[["l1","l2","p0","p2"],"6/1"],[["l1","l3","
+> l3","p0"],"2/1"],[["l1","l3","p0","p3"],"-6/1"],[["l1","p0","p0","p0"],"1/1"],[["l1","p0","p1","p1"],"-5/
+> 1"],[["l1","p0","p2","p2"],"-1/1"],[["l1","p0","p3","p3"],"1/1"],[["l2","p0","p1","p2"],"-4/1"],[["l3","l
+> 3","p0","p1"],"4/1"],[["p0","p0","p0","p1"],"-4/1"],[["p0","p1","p1","p1"],"4/1"],[["p0","p1","p2","p2",
+> "4/1"],[["p0","p1","p3","p3"],"-4/1"]],"3,7":[[["l0","l0","l0","l3"],"2/1"],[["l0","l0","l0","p3"],"-1/1"
+>],[["l0","l0","l3","p0"],"-3/1"],[["l0","l0","p0","p3"],"4/1"],[["l0","l1","l1","l3"],"-2/1"],[["l0","l1"
+> ,"l1","p3"],"1/1"],[["l0","l1","l3","p1"],"2/1"],[["l0","l1","p1","p3"],"-5/1"],[["l0","l2","l2","l3"],"-
+> 6/1"],[["l0","l2","l2","p3"],"3/1"],[["l0","l2","l3","p2"],"6/1"],[["l0","l2","p2","p3"],"-1/1"],[["l0","
+> l3","l3","l3"],"-2/1"],[["l0","l3","l3","p3"],"3/1"],[["l0","l3","p0","p0"],"1/1"],[["l0","l3","p1","p1"]
+> ,"4/1"],[["l0","l3","p2","p2"],"-4/1"],[["l0","l3","p3","p3"],"-1/1"],[["l0","p0","p0","p3"],"-3/1"],[["l
+> 0","p2","p2","p3"],"1/1"],[["l1","l1","l3","p0"],"1/1"],[["l1","l1","p0","p3"],"1/1"],[["l1","l3","p0","p
+> 1"],"-5/1"],[["l1","p0","p1","p3"],"3/1"],[["l2","l2","l3","p0"],"3/1"],[["l2","l2","p0","p3"],"-7/1"],[[
+> "l2","l3","p0","p2"],"-1/1"],[["l2","p0","p2","p3"],"5/1"],[["l3","l3","l3","p0"],"1/1"],[["l3","l3","p0"
+> ,"p3"],"-4/1"],[["l3","p0","p2","p2"],"1/1"],[["l3","p0","p3","p3"],"3/1"],[["p0","p2","p2","p3"],"-4/1"]
+>],"3,8":[[["l0","l0","l0","l2"],"-4/1"],[["l0","l0","l0","p2"],"2/1"],[["l0","l0","l2","p0"],"6/1"],[["l0

```
+> ","l0","p0","p2"],"-6/1"],[["l0","l1","l1","l2"],"4/1"],[["l0","l1","l1","p2"],"-2/1"],[["l0","l1","l2","
+> p1"],"-4/1"],[["l0","l1","p1","p2"],"6/1"],[["l0","l2","l2","l2"],"4/1"],[["l0","l2","l2","p2"],"-6/1"],[
+> ["l0","l2","l3","l3"],"-4/1"],[["l0","l2","l3","p3"],"4/1"],[["l0","l2","p0","p0"],"-4/1"],[["l0","l2","p
+> 1","p1"],"-2/1"],[["l0","l2","p2","p2"],"4/1"],[["l0","l2","p3","p3"],"2/1"],[["l0","l3","l3","p2"],"2/1"
+> ],[["l0","l3","p2","p3"],"-6/1"],[["l0","p0","p0","p2"],"5/1"],[["l0","p1","p1","p2"],"-1/1"],[["l0","p2"
+> ,"p2","p2"],"-1/1"],[["l0","p2","p3","p3"],"1/1"],[["l1","l1","l2","p0"],"-2/1"],[["l1","l2","p0","p1"],"
+> 6/1"],[["l1","p0","p1","p2"],"-4/1"],[["l2","l2","l2","p0"],"-2/1"],[["l2","l2","p0","p2"],"6/1"],[["l2",
+> "l3","l3","p0"],"2/1"],[["l2","l3","p0","p3"],"-6/1"],[["l2","p0","p0","p0"],"1/1"],[["l2","p0","p1","p1"
+> ],"-1/1"],[["l2","p0","p2","p2"],"-5/1"],[["l2","p0","p3","p3"],"1/1"],[["l3","l3","p0","p2"],"4/1"],[["p
+> 0","p0","p0","p2"],"-4/1"],[["p0","p1","p1","p2"],"4/1"],[["p0","p2","p2","p2"],"4/1"],[["p0","p2","p3","
+> p3"],"-4/1"]],3,9":[[["l0","l0","l0","l3"],"-2/1"],[["l0","l0","l0","p3"],"1/1"],[["l0","l0","l0","p3"],
+> "3/1"],[["l0","l0","p0","p3"],"-2/1"],[["l0","l1","l1","l3"],"2/1"],[["l0","l1","l1","p3"],"-1/1"],[["l0
+> ","l1","l3","p1"],"-2/1"],[["l0","l1","p1","p3"],"1/1"],[["l0","l2","l2","l3"],"2/1"],[["l0","l2","l2","p3
+> "],"-1/1"],[["l0","l2","l3","p2"],"-2/1"],[["l0","l2","p2","p3"],"1/1"],[["l0","l3","l3","l3"],"-2/1"],[[
+> "l0","l3","l3","p3"],"3/1"],[["l0","l3","p0","p0"],"-3/1"],[["l0","l3","p1","p1"],"2/1"],[["l0","l3","p2"
+> ,"p2"],"2/1"],[["l0","l3","p3","p3"],"-1/1"],[["l0","p0","p0","p3"],"2/1"],[["l0","p1","p1","p3"],"-1/1"
+> ,[["l0","p2","p2","p3"],"-1/1"],[["l1","l1","l3","p0"],"-1/1"],[["l1","l1","p0","p3"],"1/1"],[["l1","l3",
+> "p0","p1"],"1/1"],[["l1","p0","p1","p3"],"-1/1"],[["l2","l2","l3","p0"],"-1/1"],[["l2","l2","p0","p3"],"1
+> /1"],[["l2","l3","p0","p2"],"1/1"],[["l2","p0","p2","p3"],"-1/1"],[["l3","l3","l3","p0"],"1/1"],[["l3","l
+> 3","p0","p3"],"-2/1"],[["l3","p0","p0","p0"],"1/1"],[["l3","p0","p1","p1"],"-1/1"],[["l3","p0","p2","p2"]
+> ,"-1/1"],[["l3","p0","p3","p3"],"1/1"],[["p0","p0","p0","p3"],"-4/1"],[["p0","p1","p1","p3"],"4/1"],[["p0
+> ","p2","p2","p3"],"4/1"]],"4,0":[[["l0","l0","l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0"
+> ,"l1","l1"],"3/1"],[["l0","l0","l1","p1"],"-3/1"],[["l0","l0","p1","p1"],"2/1"],[["l0","l0","p2","p2"],"-
+> 3/2"],[["l0","l0","p3","p3"],"-3/2"],[["l0","l1","l1","p0"],"-3/1"],[["l0","l2","p0","p2"],"-1/1"],[["l0"
+> ,"l3","p0","p3"],"-1/1"],[["l0","p0","p0","p0"],"-1/2"],[["l0","p0","p1","p1"],"-1/2"],[["l0","p0","p2","
+> p2"],"2/1"],[["l0","p0","p3","p3"],"2/1"],[["l1","l1","l1","l1"],"-1/2"],[["l1","l1","l1","p1"],"1/1"],[[
+> "l1","l1","p0","p0"],"2/1"],[["l1","l1","p2","p2"],"3/2"],[["l1","l1","p3","p3"],"3/2"],[["l1","l2","p1",
+> "p2"],"1/1"],[["l1","l3","p1","p3"],"1/1"],[["l1","p0","p0","p1"],"-1/2"],[["l1","p1","p1","p1"],"-1/2",
+> [["l1","p1","p2","p2"],"-2/1"],[["l1","p1","p3","p3"],"-2/1"],[["l2","l2","l2","l2"],"1/2"],[["l2","l2","
+> l2","p2"],"-1/1"],[["l2","l2","l3","l3"],"1/1"],[["l2","l2","l3","p3"],"-1/1"],[["l2","l2","p2","p2"],"5/
+> 2"],[["l2","l2","p3","p3"],"-1/2"],[["l2","l3","l3","p2"],"-1/1"],[["l2","l3","p2","p3"],"6/1"],[["l2","p
+> 0","p0","p2"],"1/2"],[["l2","p1","p1","p2"],"-1/2"],[["l2","p2","p2","p2"],"-2/1"],[["l2","p2","p3","p3"]
+> ,"-2/1"],[["l3","l3","l3","l3"],"1/2"],[["l3","l3","l3","p3"],"-1/1"],[["l3","l3","p2","p2"],"-1/2"],[["l
+> 3","l3","p3","p3"],"5/2"],[["l3","p0","p0","p3"],"1/2"],[["l3","p1","p1","p3"],"-1/2"],[["l3","p2","p2","
+> p3"],"-2/1"],[["l3","p3","p3","p3"],"-2/1"],[["p0","p0","p1","p1"],"2/1"]],"4,1":[[["l0","l0","l0","l1"],
+> "-2/1"],[["l0","l0","l0","p1"],"1/1"],[["l0","l0","l1","p0"],"3/1"],[["l0","l0","p0","p1"],"-2/1"],[["l0"
+> ,"l1","l1","l1"],"-2/1"],[["l0","l1","l1","p1"],"3/1"],[["l0","l1","l2","l2"],"2/1"],[["l0","l1","l2","p2
+> "],"-2/1"],[["l0","l1","l3","l3"],"2/1"],[["l0","l1","l3","p3"],"-2/1"],[["l0","l1","p0","p0"],"-3/1"],[[
+> "l0","l1","p1","p1"],"-1/1"],[["l0","l1","p2","p2"],"2/1"],[["l0","l1","p3","p3"],"2/1"],[["l0","l2","l2"
+> ,"p1"],"-1/1"],[["l0","l2","l2","p2"],"1/1"],[["l0","l3","l3","p1"],"-1/1"],[["l0","l3","p1","p3"],"1/1"
+> ,[["l0","p0","p0","p1"],"2/1"],[["l0","p1","p2","p2"],"-1/1"],[["l0","p1","p3","p3"],"-1/1"],[["l1","l1",
+> "l1","p0"],"1/1"],[["l1","l1","p0","p1"],"-2/1"],[["l1","l2","l2","p0"],"-1/1"],[["l1","l2","p0","p2"],"1
+> /1"],[["l1","l3","l3","p0"],"-1/1"],[["l1","l3","p0","p3"],"1/1"],[["l1","p0","p0","p0"],"1/1"],[["l1","p
+> 0","p1","p1"],"1/1"],[["l1","p0","p2","p2"],"-1/1"],[["l1","p0","p3","p3"],"-1/1"],[["l2","l2","p0","p1"]
+> ,"1/1"],[["l2","p0","p1","p2"],"-1/1"],[["l3","l3","p0","p1"],"1/1"],[["l3","p0","p1","p3"],"-1/1"],[["p0
+> ","p0","p0","p1"],"-4/1"],[["p0","p1","p2","p2"],"4/1"],[["p0","p1","p3","p3"],"4/1"]],"4,2":[[["l0","l0"
+> ,"l0","l2"],"2/1"],[["l0","l0","l0","p2"],"-1/1"],[["l0","l0","l2","p0"],"-3/1"],[["l0","l0","p0","p2"],"
+> 4/1"],[["l0","l1","l1","l2"],"-6/1"],[["l0","l1","l1","p2"],"3/1"],[["l0","l1","l2","p1"],"6/1"],[["l0","
+> l1","p1","p2"],"-1/1"],[["l0","l2","l2","l2"],"-2/1"],[["l0","l2","l2","p2"],"3/1"],[["l0","l2","l3","l3"
+> ],"-2/1"],[["l0","l2","l3","p3"],"2/1"],[["l0","l2","p0","p0"],"1/1"],[["l0","l2","p1","p1"],"-4/1"],[["l
+> 0","l2","p2","p2"],"-1/1"],[["l0","l2","p3","p3"],"4/1"],[["l0","l3","l3","p2"],"1/1"],[["l0","l3","p2","
+> p3"],"-5/1"],[["l0","p0","p0","p2"],"-3/1"],[["l0","p1","p1","p2"],"1/1"],[["l1","l1","l2","p0"],"3/1"],[
+> ["l1","l1","p0","p2"],"-7/1"],[["l1","l2","p0","p1"],"-1/1"],[["l1","p0","p1","p2"],"5/1"],[["l2","l2","l
+> 2","p0"],"1/1"],[["l2","l2","p0","p2"],"-4/1"],[["l2","l3","l3","p0"],"1/1"],[["l2","l3","p0","p3"],"-5/1
+> "],[["l2","p0","p1","p1"],"1/1"],[["l2","p0","p2","p2"],"3/1"],[["l3","l3","p0","p2"],"1/1"],[["l3","p0",
+> "p2","p3"],"3/1"],[["p0","p1","p1","p2"],"-4/1"]],"4,3":[[["l0","l0","l0","l3"],"2/1"],[["l0","l0","l0","
+> p3"],"-1/1"],[["l0","l0","l3","p0"],"-3/1"],[["l0","l0","p0","p3"],"4/1"],[["l0","l1","l1","l3"],"-6/1"],
+> [["l0","l1","l1","p3"],"3/1"],[["l0","l1","l3","p1"],"6/1"],[["l0","l1","p1","p3"],"-1/1"],[["l0","l2","l
+> 2","l3"],"-2/1"],[["l0","l2","l2","p3"],"1/1"],[["l0","l2","l3","p2"],"2/1"],[["l0","l2","p2","p3"],"-5/1
+> "],[["l0","l3","l3","l3"],"-2/1"],[["l0","l3","l3","p3"],"3/1"],[["l0","l3","p0","p0"],"1/1"],[["l0","l3"
+> ,"p1","p1"],"-4/1"],[["l0","l3","p2","p2"],"4/1"],[["l0","l3","p3","p3"],"-1/1"],[["l0","p0","p0","p3"],"
+> -3/1"],[["l0","p1","p1","p3"],"1/1"],[["l1","l1","l3","p0"],"3/1"],[["l1","l1","p0","p3"],"-7/1"],[["l1",
+> "l3","p0","p1"],"-1/1"],[["l1","p0","p1","p3"],"5/1"],[["l2","l2","l3","p0"],"1/1"],[["l2","l2","p0","p3"
+> ],"1/1"],[["l2","l3","p0","p2"],"-5/1"],[["l2","p0","p2","p3"],"3/1"],[["l3","l3","l3","p0"],"1/1"],[["l3
+> ","l3","p0","p3"],"-4/1"],[["l3","p0","p1","p1"],"1/1"],[["l3","p0","p3","p3"],"3/1"],[["p0","p1","p1","p
+> 3"],"-4/1"]],"4,4":[[["l0","l0","l0","l0"],"3/2"],[["l0","l0","l0","p0"],"-3/1"],[["l0","l0","l1","l1","
+> -1/1"],[["l0","l0","l1","p1"],"1/1"],[["l0","l0","l2","l2"],"-3/1"],[["l0","l0","l2","p2"],"3/1"],[["l0",
+> "l0","l3","l3"],"-3/1"],[["l0","l0","l3","p3"],"3/1"],[["l0","l0","p0","p0"],"5/2"],[["l0","l0","p1","p1"
+> ],"-1/2"],[["l0","l0","p2","p2"],"-3/2"],[["l0","l0","p3","p3"],"-3/2"],[["l0","l1","l1","p0"],"1/1"],[["
+> l0","l2","l2","p0"],"3/1"],[["l0","l2","p0","p2"],"-2/1"],[["l0","l3","l3","p0"],"3/1"],[["l0","l3","p0",
+> "p3"],"-2/1"],[["l0","p0","p0","p0"],"-1/1"],[["l0","p0","p2","p2"],"1/1"],[["l0","p0","p3","p3"],"1/1"],
+> [["l1","l1","l1","l1"],"3/2"],[["l1","l1","l1","p1"],"-3/1"],[["l1","l1","l2","l2"],"1/1"],[["l1","l1","l
+> 2","p2"],"-1/1"],[["l1","l1","l3","l3"],"1/1"],[["l1","l1","l3","p3"],"-1/1"],[["l1","l1","p0","p0"],"1/2
+> "],[["l1","l1","p1","p1"],"3/2"],[["l1","l1","p2","p2"],"-1/2"],[["l1","l1","p3","p3"],"-1/2"],[["l1","l2
+> ","l2","p1"],"-1/1"],[["l1","l3","l3","p1"],"-1/1"],[["l1","p0","p0","p1"],"-1/1"],[["l1","p1","p2","p2"]
+> ,"1/1"],[["l1","p1","p3","p3"],"1/1"],[["l2","l2","l2","l2"],"3/2"],[["l2","l2","l2","p2"],"-3/1"],[["l2"
+> ,"l2","l3","l3"],"3/1"],[["l2","l2","l3","p3"],"-3/1"],[["l2","l2","p0","p0"],"-3/2"],[["l2","l2","p1","p
+> 1"],"1/2"],[["l2","l2","p2","p2"],"5/2"],[["l2","l2","p3","p3"],"3/2"],[["l2","l3","l3","p2"],"-3/1"],[["
+> l2","l3","p2","p3"],"2/1"],[["l2","p0","p0","p2"],"1/1"],[["l2","p2","p2","p2"],"-1/1"],[["l2","p2","p3",
+> "p3"],"-1/1"],[["l3","l3","l3","l3"],"3/2"],[["l3","l3","l3","p3"],"-3/1"],[["l3","l3","p0","p0"],"-3/2"]
+> ,[["l3","l3","p1","p1"],"1/2"],[["l3","l3","p2","p2"],"3/2"],[["l3","l3","p3","p3"],"5/2"],[["l3","p0","p
```

```
+> 0","p3"],"1/1"],[["l3","p2","p2","p3"],"-1/1"],[["l3","p3","p3","p3"],"-1/1"],[["p0","p0","p0","p0"],"2/1
+> "],[["p0","p0","p2","p2"],"-4/1"],[["p0","p0","p3","p3"],"-4/1"],[["p2","p2","p2","p2"],"2/1"],[["p2","p2
+> ","p3","p3"],"4/1"],[["p3","p3","p3","p3"],"2/1"]],"4,5":[[["l0","l0","l1","l2"],"2/1"],[["l0","l0","l1",
+> "p2"],"-1/1"],[["l0","l0","l2","p1"],"-1/1"],[["l0","l0","p1","p2"],"1/1"],[["l0","l1","l2","p0"],"-2/1"]
+> ,[["l0","l1","p0","p2"],"1/1"],[["l0","l2","p0","p1"],"1/1"],[["l0","p0","p1","p2"],"-1/1"],[["l1","l1","
+> l1","l2"],"2/1"],[["l1","l1","l1","p2"],"-1/1"],[["l1","l1","l2","p1"],"-3/1"],[["l1","l1","p1","p2"],"2/
+> 1"],[["l1","l2","l2","l2"],"-2/1"],[["l1","l2","l2","p2"],"3/1"],[["l1","l2","l3","l3"],"-2/1"],[["l1","l
+> 2","l3","p3"],"2/1"],[["l1","l2","p0","p0"],"2/1"],[["l1","l2","p1","p1"],"1/1"],[["l1","l2","p2","p2"],"
+> -3/1"],[["l1","l2","p3","p3"],"-2/1"],[["l1","l3","l3","p2"],"1/1"],[["l1","l3","p2","p3"],"-1/1"],[["l1"
+> ,"p0","p0","p2"],"-1/1"],[["l1","p1","p1","p2"],"-1/1"],[["l1","p2","p2","p2"],"1/1"],[["l1","p2","p3","p
+> 3"],"1/1"],[["l2","l2","l2","p1"],"1/1"],[["l2","l2","p1","p2"],"-2/1"],[["l2","l3","l3","p1"],"1/1"],[["
+> l2","l3","p1","p3"],"-1/1"],[["l2","p0","p0","p1"],"-1/1"],[["l2","p1","p2","p2"],"2/1"],[["l2","p1","p3"
+> ,"p3"],"1/1"],[["l3","l3","p1","p2"],"-1/1"],[["l3","p1","p2","p3"],"1/1"],[["p0","p0","p1","p2"],"4/1"],
+> [["p1","p2","p2","p2"],"-4/1"],[["p1","p2","p3","p3"],"-4/1"]],"4,6":[[["l0","l0","l1","l3"],"2/1"],[["l0
+> ","l0","l1","p3"],"-1/1"],[["l0","l0","l3","p1"],"-1/1"],[["l0","l0","p1","p3"],"1/1"],[["l0","l1","l3","
+> p0"],"-2/1"],[["l0","l1","p0","p3"],"1/1"],[["l0","l3","p0","p1"],"1/1"],[["l0","p0","p1","p3"],"-1/1"],[
+> ["l1","l1","l1","l3"],"2/1"],[["l1","l1","l1","p3"],"-1/1"],[["l1","l1","l3","p1"],"-3/1"],[["l1","l1","p
+> 1","p3"],"2/1"],[["l1","l2","l2","l3"],"-2/1"],[["l1","l2","l2","p3"],"1/1"],[["l1","l2","l3","p2"],"2/1"
+> ],[["l1","l2","p2","p3"],"-1/1"],[["l1","l3","l3","l3"],"-2/1"],[["l1","l3","l3","p3"],"3/1"],[["l1","l3"
+> ,"p0","p0"],"2/1"],[["l1","l3","p1","p1"],"1/1"],[["l1","l3","p2","p2"],"-2/1"],[["l1","l3","p3","p3"],"-
+> 3/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1","p1","p1","p3"],"-1/1"],[["l1","p2","p2","p3"],"1/1"],[["l1",
+> "p3","p3","p3"],"1/1"],[["l2","l2","l3","p1"],"1/1"],[["l2","l2","p1","p3"],"-1/1"],[["l2","l3","p1","p2"
+> ],"-1/1"],[["l2","p1","p2","p3"],"1/1"],[["l3","l3","l3","p1"],"1/1"],[["l3","l3","p1","p3"],"-2/1"],[["l
+> 3","p0","p0","p1"],"-1/1"],[["l3","p1","p2","p2"],"1/1"],[["l3","p1","p3","p3"],"2/1"],[["p0","p0","p1","
+> p3"],"4/1"],[["p1","p2","p2","p3"],"-4/1"],[["p1","p3","p3","p3"],"-4/1"]],"4,7":[[["l0","l0","l0","l0"],
+> "-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l3","l3"],"1/1"],[["l0","l0","l3","p3"],"-1/1"],[["l0"
+> ,"l0","p0","p0"],"-5/2"],[["l0","l0","p3","p3"],"-1/2"],[["l0","l1","p0","p1"],"1/1"],[["l0","l2","p0","p
+> 2"],"1/1"],[["l0","l3","l3","p0"],"-1/1"],[["l0","l3","p0","p3"],6/1"],[["l0","p0","p0","p0"],"2/1"],[["
+> l0","p0","p1","p1"],"-1/2"],[["l0","p0","p2","p2"],"-1/2"],[["l0","p0","p3","p3"],"-2/1"],[["l1","l1","l1
+> ","l1"],"1/2"],[["l1","l1","l1","p1"],"-1/1"],[["l1","l1","l2","l2"],"3/1"],[["l1","l1","l2","p2"],"-3/1"
+> ],[["l1","l1","p0","p0"],"3/2"],[["l1","l1","p2","p2"],"2/1"],[["l1","l1","p3","p3"],"-3/2"],[["l1","l2",
+> "l2","p1"],"-3/1"],[["l1","l3","p1","p3"],"-1/1"],[["l1","p0","p0","p1"],"-2/1"],[["l1","p1","p1","p1"],"
+> 1/2"],[["l1","p1","p2","p2"],"-1/2"],[["l1","p1","p3","p3"],"-1/1"],[["l1","p2","p2","p2"],"1/2"],[["l2","
+> l2","l2","p2"],"-1/1"],[["l2","l2","p0","p0"],"3/2"],[["l2","l2","p1","p1"],"2/1"],[["l2","l2","p3","p3"
+> ,"-3/2"],[["l2","l3","p2","p3"],"-1/1"],[["l2","p0","p0","p2"],"-2/1"],[["l2","p1","p1","p2"],"-1/2"],[["
+> l2","p2","p2","p2"],"1/2"],[["l2","p2","p3","p3"],"2/1"],[["l3","l3","l3","l3"],"-1/2"],[["l3","l3","l3",
+> "p3"],"1/1"],[["l3","l3","p0","p0"],"-1/2"],[["l3","l3","p3","p3"],"-5/2"],[["l3","p0","p0","p3"],"-2/1"
+> ,[["l3","p1","p1","p3"],"1/2"],[["l3","p2","p2","p3"],"1/2"],[["l3","p3","p3","p3"],"2/1"],[["p1","p1","p
+> 2","p2"],"2/1"]],"4,8":[[["l0","l0","l2","l3"],"-2/1"],[["l0","l0","l2","p3"],"1/1"],[["l0","l0","l3","p2
+> "],"1/1"],[["l0","l0","p2","p3"],"1/1"],[["l0","l2","l3","p0"],"2/1"],[["l0","l2","p0","p3"],"-5/1"],[["l
+> 0","l3","p0","p2"],"-5/1"],[["l0","p0","p2","p3"],"3/1"],[["l1","l1","l2","l3"],"6/1"],[["l1","l1","l2","
+> p3"],"-3/1"],[["l1","l1","l3","p2"],"-3/1"],[["l1","l1","p2","p3"],"7/1"],[["l1","l2","l3","p1"],"-6/1"],
+> [["l1","l2","p1","p3"],"1/1"],[["l1","l3","p1","p2"],"1/1"],[["l1","p1","p2","p3"],"-5/1"],[["l2","l2","l
+> 2","l3"],"2/1"],[["l2","l2","l2","p3"],"-1/1"],[["l2","l2","l3","p2"],"-3/1"],[["l2","l2","p2","p3"],"4/1
+> "],[["l2","l3","l3","l3"],"2/1"],[["l2","l3","l3","p3"],"-3/1"],[["l2","l3","p0","p0"],"4/1"],[["l2","l3"
+> ,"p1","p1"],"4/1"],[["l2","l3","p2","p2"],"1/1"],[["l2","l3","p3","p3"],"1/1"],[["l2","p1","p1","p3"],"-1
+> /1"],[["l2","p2","p2","p3"],"-3/1"],[["l3","l3","l3","p2"],"-1/1"],[["l3","l3","p2","p3"],"4/1"],[["l3","
+> p1","p1","p2"],"-1/1"],[["l3","p2","p3","p3"],"-3/1"],[["p1","p1","p2","p3"],"4/1"]],"4,9":[[["l0","l0","
+> l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l2","l2"],"1/1"],[["l0","l0","l2","p2"],"-1/
+> 1"],[["l0","l0","p0","p0"],"-5/2"],[["l0","l0","p2","p2"],"-1/2"],[["l0","l1","p0","p1"],"1/1"],[["l0","l
+> 2","l2","p0"],"-1/1"],[["l0","l2","p0","p2"],"6/1"],[["l0","l3","p0","p3"],"1/1"],[["l0","p0","p0","p0"],
+> "2/1"],[["l0","p0","p1","p1"],"-1/2"],[["l0","p0","p2","p2"],"-2/1"],[["l0","p0","p3","p3"],"-1/2"],[["l1
+> ","l1","l1","l1"],"1/2"],[["l1","l1","l1","p1"],"-1/1"],[["l1","l1","l3","l3"],"3/1"],[["l1","l1","l3","p
+> 3"],"-3/1"],[["l1","l1","p0","p0"],"3/2"],[["l1","l1","p2","p2"],"-3/2"],[["l1","l1","p3","p3"],"2/1"],[[
+> "l1","l2","p1","p2"],"-1/1"],[["l1","l3","l3","p1"],"-3/1"],[["l1","p0","p0","p1"],"-2/1"],[["l1","p1","p
+> 1","p1"],"1/2"],[["l1","p1","p2","p2"],"2/1"],[["l1","p1","p3","p3"],"-1/2"],[["l2","l2","l2","l2"],"-1/2
+> "],[["l2","l2","l2","p2"],"1/1"],[["l2","l2","p0","p0"],"-1/2"],[["l2","l2","p2","p2"],"-5/2"],[["l2","l3
+> ","p2","p3"],"-1/1"],[["l2","p0","p0","p2"],"-2/1"],[["l2","p1","p1","p2"],"1/2"],[["l2","p2","p2","p2"],
+> "2/1"],[["l2","p2","p3","p3"],"1/2"],[["l3","l3","l3","l3"],"1/2"],[["l3","l3","l3","p3"],"-1/1"],[["l3",
+> "l3","p0","p0"],"3/2"],[["l3","l3","p1","p1"],"2/1"],[["l3","l3","p2","p2"],"-3/2"],[["l3","p0","p0","p3"
+> ],"-2/1"],[["l3","p1","p1","p3"],"-1/2"],[["l3","p2","p2","p3"],"2/1"],[["l3","p3","p3","p3"],"1/2"],[["p
+> 1","p1","p3","p3"],"2/1"]],"5,0":[[["l0","l0","l1","l2"],"6/1"],[["l0","l0","l1","p2"],"-3/1"],[["l0","l0
+> ","l2","p1"],"-3/1"],[["l0","l0","p1","p2"],"7/1"],[["l0","l1","l2","p0"],"-6/1"],[["l0","l1","p0","p2"],
+> "1/1"],[["l0","l2","p0","p1"],"1/1"],[["l0","p0","p1","p2"],"-5/1"],[["l1","l1","l1","l2"],"-2/1"],[["l1"
+> ,"l1","l1","p2"],"1/1"],[["l1","l1","l2","p1"],"3/1"],[["l1","l1","p1","p2"],"-4/1"],[["l1","l2","l2","l2
+> "],"-2/1"],[["l1","l2","l2","p2"],"3/1"],[["l1","l2","l3","l3"],"-2/1"],[["l1","l2","l3","p3"],"2/1"],[["
+> l1","l2","p0","p0"],"4/1"],[["l1","l2","p1","p1"],"-1/1"],[["l1","l2","p2","p2"],"-1/1"],[["l1","l2","p3"
+> ,"p3"],"4/1"],[["l1","l3","l3","p2"],"1/1"],[["l1","l3","p2","p3"],"-5/1"],[["l1","p0","p0","p2"],"-1/1"]
+> ,[["l1","p1","p1","p2"],"3/1"],[["l2","l2","l2","p1"],"1/1"],[["l2","l2","p1","p2"],"-4/1"],[["l2","l3","
+> l3","p1"],"1/1"],[["l2","l3","p1","p3"],"-5/1"],[["l2","p0","p0","p1"],"-1/1"],[["l2","p1","p2","p2"],"3/
+> 1"],[["l3","l3","p1","p2"],"1/1"],[["l3","p1","p2","p3"],"3/1"],[["p0","p0","p1","p2"],"4/1"]],"5,1":[[["
+> l0","l0","l0","l2"],"-4/1"],[["l0","l0","l0","p2"],"2/1"],[["l0","l0","l2","p0"],"6/1"],[["l0","l0","p0",
+> "p2"],"-6/1"],[["l0","l1","l1","l2"],"-4/1"],[["l0","l1","l1","p2"],"2/1"],[["l0","l1","l2","p1"],"4/1"],
+> [["l0","l1","p1","p2"],"-6/1"],[["l0","l2","l2","l2"],"4/1"],[["l0","l2","l2","p2"],"-6/1"],[["l0","l2","
+> l3","l3"],"4/1"],[["l0","l2","l3","p3"],"-4/1"],[["l0","l2","p0","p0"],"-4/1"],[["l0","l2","p1","p1"],"2/
+> 1"],[["l0","l2","p2","p2"],"4/1"],[["l0","l2","p3","p3"],"-2/1"],[["l0","l3","l3","p2"],"-2/1"],[["l0","l
+> 3","p2","p3"],"6/1"],[["l0","p0","p0","p2"],"5/1"],[["l0","p1","p1","p2"],"1/1"],[["l0","p2","p2","p2"],"
+> -1/1"],[["l0","p2","p3","p3"],"-1/1"],[["l1","l1","l2","p0"],"2/1"],[["l1","l1","p0","p2"],"4/1"],[["l1",
+> "l2","p0","p1"],"-6/1"],[["l2","l2","l2","p0"],"-2/1"],[["l2","l2","p0","p2"],"6/1"],[["l2","l3","l3","p0
+> "],"-2/1"],[["l2","l3","p0","p3"],"6/1"],[["l2","p0","p0","p0"],"1/1"],[["l2","p0","p1","p1"],"1/1"],[["l
+> 2","p0","p2","p2"],"-5/1"],[["l2","p0","p3","p3"],"-1/1"],[["l3","p0","p2","p3"],"-4/1"],[["p0","p0","p0"
```

```
+> ,"p2"],"-4/1"],[["p0","p1","p1","p2"],"-4/1"],[["p0","p2","p2","p2"],"4/1"],[["p0","p2","p3","p3"],"4/1"]
+> ],"5,2":[[["l0","l0","l0","l1"],"-4/1"],[["l0","l0","l0","p1"],"2/1"],[["l0","l0","l1","p0"],"6/1"],[["l0
+> ","l0","p0","p1"],"-6/1"],[["l0","l1","l1","l1"],"4/1"],[["l0","l1","l1","p1"],"-6/1"],[["l0","l1","l2","
+> l2"],"-4/1"],[["l0","l1","l2","p2"],"4/1"],[["l0","l1","l3","l3"],"4/1"],[["l0","l1","l3","p3"],"-4/1"],[
+> ["l0","l1","p0","p0"],"-4/1"],[["l0","l1","p1","p1"],"4/1"],[["l0","l1","p2","p2"],"2/1"],[["l0","l1","p3
+> ","p3"],"-2/1"],[["l0","l2","l2","p1"],"2/1"],[["l0","l2","p1","p2"],"-6/1"],[["l0","l3","l3","p1"],"-2/1
+> "],[["l0","l3","p1","p3"],"6/1"],[["l0","p0","p0","p1"],"5/1"],[["l0","p1","p1","p1"],"-1/1"],[["l0","p1"
+> ,"p2","p2"],"1/1"],[["l0","p1","p3","p3"],"-1/1"],[["l1","l1","l1","p0"],"-2/1"],[["l1","l1","p0","p1"],"
+> 6/1"],[["l1","l2","l2","p0"],"2/1"],[["l1","l2","p0","p2"],"-6/1"],[["l1","l3","l3","p0"],"-2/1"],[["l1",
+> "l3","p0","p3"],"6/1"],[["l1","p0","p0","p0"],"1/1"],[["l1","p0","p1","p1"],"-5/1"],[["l1","p0","p2","p2"
+> ],"1/1"],[["l1","p0","p3","p3"],"-1/1"],[["l2","l2","p0","p1"],"4/1"],[["l3","p0","p1","p3"],"-4/1"],[["p
+> 0","p0","p0","p1"],"-4/1"],[["p0","p1","p1","p1"],"4/1"],[["p0","p1","p2","p2"],"-4/1"],[["p0","p1","p3",
+> "p3"],"4/1"]],"5,3":[[["l0","l1","l2","l3"],"-8/1"],[["l0","l1","l2","p3"],"4/1"],[["l0","l1","l3","p2"],
+> "4/1"],[["l0","l1","p2","p3"],"4/1"],[["l0","l2","l3","p1"],"4/1"],[["l0","l2","p1","p3"],"4/1"],[["l0","
+> l3","p1","p2"],"-16/1"],[["l0","p1","p2","p3"],"2/1"],[["l1","l2","l3","p0"],"4/1"],[["l1","l2","p0","p3"
+> ],"-16/1"],[["l1","l3","p0","p2"],"4/1"],[["l1","p0","p2","p3"],"2/1"],[["l2","l3","p0","p1"],"4/1"],[["l
+> 2","p0","p1","p3"],"2/1"],[["l3","p0","p1","p2"],"2/1"],[["p0","p1","p2","p3"],"-8/1"]],"5,4":[[["l0","l0
+> ","l1","l2"],"2/1"],[["l0","l0","l1","p2"],"-1/1"],[["l0","l0","l2","p1"],"-1/1"],[["l0","l0","p1","p2"],
+> "1/1"],[["l0","l1","l2","p0"],"-2/1"],[["l0","l1","p0","p2"],"1/1"],[["l0","l2","p0","p1"],"1/1"],[["l0",
+> "p0","p1","p2"],"-1/1"],[["l1","l1","l1","l2"],"2/1"],[["l1","l1","l1","p2"],"-1/1"],[["l1","l1","l2","p1
+> "],"-3/1"],[["l1","l1","p1","p2"],"2/1"],[["l1","l2","l2","l2"],"-2/1"],[["l1","l2","l2","p2"],"3/1"],[["
+> l1","l2","l3","l3"],"-2/1"],[["l1","l2","l3","p3"],"2/1"],[["l1","l2","p0","p0"],"2/1"],[["l1","l2","p1",
+> "p1"],"1/1"],[["l1","l2","p2","p2"],"-3/1"],[["l1","l2","p3","p3"],"-2/1"],[["l1","l3","l3","p2"],"1/1"],
+> [["l1","l3","p2","p3"],"-1/1"],[["l1","p0","p0","p2"],"-1/1"],[["l1","p1","p1","p2"],"-1/1"],[["l1","p2",
+> "p2","p2"],"1/1"],[["l1","p2","p3","p3"],"1/1"],[["l2","l2","l2","p1"],"1/1"],[["l2","l2","p1","p2"],"-2/
+> 1"],[["l2","l3","l3","p1"],"1/1"],[["l2","l3","p1","p3"],"-1/1"],[["l2","p0","p0","p1"],"-1/1"],[["l2","p
+> 1","p2","p2"],"2/1"],[["l2","p1","p3","p3"],"1/1"],[["l3","l3","p1","p2"],"-1/1"],[["l3","p1","p2"],"-1/1
+> "],[["l3","p1","p3","p3"],"1/1"],[["p0","p0","p1","p2"],"4/1"],[["p1","p2","p2","p2"],"-4/1"],[["p1","p2
+> ","p3","p3"],"-4/1"]],"5,5":[[["l0","l0","l0","l0"],"4/1"],[["l0","l0","l0","p0"],"-8/1"],[["l0","l0","l1
+> ","l1"],"-4/1"],[["l0","l0","l1","p1"],"4/1"],[["l0","l0","l2","l2"],"-4/1"],[["l0","l0","l2","p2"],"4/1
+> "],[["l0","l0","l3","l3"],"-8/1"],[["l0","l0","l3","p3"],"8/1"],[["l0","l0","p0","p0"],"10/1"],[["l0","l0
+> ","p1","p1"],"-2/1"],[["l0","l0","p2","p2"],"-2/1"],[["l0","l0","p3","p3"],"-2/1"],[["l0","l1","l1","p0"
+> ],"4/1"],[["l0","l1","l1","p1"],"-8/1"],[["l0","l1","l2","l2"],"16/1"],[["l0","l1","l3","l3"],"-4/1"],[["
+> l0","l3","p0","p3"],"-16/1"],[["l0","p0","p0","p0"],"-6/1"],[["l0","p0","p1","p1"],"2/1"],[["l0","p0","p2
+> ","p2"],"2/1"],[["l0","p0","p3","p3"],"6/1"],[["l1","l1","l2","l2"],"8/1"],[["l1","l1","l2","p2"],"-8/1"]
+> ,[["l1","l1","l3","l3"],"4/1"],[["l1","l1","l3","p3"],"-4/1"],[["l1","l1","p0","p0"],"-4/1"],[["l1","l1",
+> "p3","p3"],"4/1"],[["l1","l2","l2","p1"],"-8/1"],[["l1","l2","p1","p2"],"16/1"],[["l1","l3","l3","p1"],"-
+> 4/1"],[["l1","l3","p1","p3"],"4/1"],[["l1","p0","p0","p1"],"4/1"],[["l1","p1","p2","p2"],"-4/1"],[["l1","
+> p1","p3","p3"],"-4/1"],[["l2","l2","l3","l3"],"4/1"],[["l2","l2","l3","p3"],"-4/1"],[["l2","l2","p0","p0"
+> ],"-4/1"],[["l2","l2","p3","p3"],"4/1"],[["l2","l3","l3","p2"],"-4/1"],[["l2","l3","p2","p3"],"4/1"],[["l
+> 2","p0","p0","p2"],"4/1"],[["l2","p1","p1","p2"],"-4/1"],[["l2","p2","p3","p3"],"-4/1"],[["l3","l3","l3",
+> "l3"],"4/1"],[["l3","l3","l3","p3"],"-8/1"],[["l3","l3","p0","p0"],"-2/1"],[["l3","l3","p1","p1"],"2/1"],
+> [["l3","l3","p2","p2"],"2/1"],[["l3","l3","p3","p3"],"10/1"],[["l3","p0","p0","p3"],"6/1"],[["l3","p1","p
+> 1","p3"],"-2/1"],[["l3","p2","p2","p3"],"-2/1"],[["l3","p3","p3","p3"],"-6/1"],[["p0","p0","p0","p0"],"4/
+> 1"],[["p0","p0","p1","p1"],"-4/1"],[["p0","p0","p2","p2"],"-4/1"],[["p0","p0","p3","p3"],"-8/1"],[["p1","
+> p1","p2","p2"],"8/1"],[["p1","p1","p3","p3"],"4/1"],[["p2","p2","p3","p3"],"4/1"],[["p3","p3","p3","p3"],
+> "4/1"]],"5,6":[[["l0","l0","l2","l3"],"4/1"],[["l0","l0","l2","p3"],"-2/1"],[["l0","l0","l3","p2"],"-2/1"
+> ],[["l0","l2","l3","p0"],"-4/1"],[["l0","l2","p0","p3"],"6/1"],[["l0","l3","p0","p2"],"6/1"],[["l0","p0",
+> "p2","p3"],"-4/1"],[["l1","l1","l2","l3"],"4/1"],[["l1","l1","l2","p3"],"-2/1"],[["l1","l1","l3","p2"],"-
+> 2/1"],[["l1","l1","p2","p3"],"-4/1"],[["l1","l2","l3","p1"],"-4/1"],[["l1","l2","p1","p3"],"6/1"],[["l1",
+> "l3","p1","p2"],"6/1"],[["l2","l2","l2","l3"],"-4/1"],[["l2","l2","l2","p3"],"2/1"],[["l2","l2","l3","p2"
+> ],"6/1"],[["l2","l2","p2","p3"],"-6/1"],[["l2","l3","l3","l3"],"-4/1"],[["l2","l3","l3","p3"],"6/1"],[["l
+> 2","l3","p0","p0"],"-2/1"],[["l2","l3","p1","p1"],"-2/1"],[["l2","l3","p2","p2"],"-4/1"],[["l2","l3","p3"
+> ,"p3"],"-4/1"],[["l2","p0","p0","p3"],"-1/1"],[["l2","p1","p1","p3"],"-1/1"],[["l2","p2","p2","p3"],"5/1"
+> ],[["l2","p3","p3","p3"],"1/1"],[["l3","l3","l3","p2"],"2/1"],[["l3","l3","p2","p3"],"-6/1"],[["l3","p0",
+> "p0","p2"],"-1/1"],[["l3","p1","p1","p2"],"-1/1"],[["l3","p2","p2","p2"],"1/1"],[["l3","p2","p3","p3"],"5
+> /1"],[["p0","p0","p2","p3"],"4/1"],[["p1","p1","p2","p3"],"4/1"],[["p2","p2","p2","p3"],"-4/1"],[["p2","p
+> 3","p3","p3"],"-4/1"]],"5,7":[[["l0","l0","l1","l2"],"2/1"],[["l0","l0","l1","p2"],"-1/1"],[["l0","l0","l
+> 2","p1"],"-1/1"],[["l0","l0","p1","p2"],"1/1"],[["l0","l1","l2","p0"],"-2/1"],[["l0","l1","p0","p2"],"1/1
+> "],[["l0","l2","p0","p1"],"1/1"],[["l0","p0","p1","p2"],"-1/1"],[["l1","l1","l1","l2"],"-2/1"],[["l1","l1
+> ","l1","p2"],"1/1"],[["l1","l1","l2","p1"],"3/1"],[["l1","l1","p1","p2"],"-2/1"],[["l1","l2","l2","l2"],"
+> 2/1"],[["l1","l2","l2","p2"],"-3/1"],[["l1","l2","l3","l3"],"-2/1"],[["l1","l2","l3","p3"],"2/1"],[["l1",
+> "l2","p0","p0"],"2/1"],[["l1","l2","p1","p1"],"-3/1"],[["l1","l2","p2","p2"],"1/1"],[["l1","l2","p3","p3"
+> ],"-2/1"],[["l1","l3","l3","p2"],"1/1"],[["l1","l3","p2","p3"],"-1/1"],[["l1","p0","p0","p2"],"-1/1"],[["
+> l1","p1","p1","p2"],"2/1"],[["l1","p2","p3","p3"],"1/1"],[["l2","l2","l2","p1"],"-1/1"],[["l2","l2","p1",
+> "p2"],"2/1"],[["l2","l3","l3","p1"],"1/1"],[["l2","l3","p1","p3"],"-1/1"],[["l2","p0","p0","p1"],"-1/1"],
+> [["l2","p1","p1","p1"],"1/1"],[["l2","p1","p2","p2"],"-1/1"],[["l2","p1","p3","p3"],"1/1"],[["l3","l3","p
+> 1","p2"],"-1/1"],[["l3","p1","p2","p3"],"1/1"],[["p0","p0","p1","p2"],"4/1"],[["p1","p1","p1","p2"],"-4/1
+> "],[["p1","p2","p3","p3"],"-4/1"]],"5,8":[[["l0","l0","l1","l3"],"4/1"],[["l0","l0","l1","p3"],"-2/1"],[[
+> "l0","l0","l3","p1"],"-2/1"],[["l0","l1","l3","p0"],"-4/1"],[["l0","l1","p0","p3"],"6/1"],[["l0","l3","p0
+> ","p1"],"6/1"],[["l0","p0","p1","p3"],"-4/1"],[["l1","l1","l1","l3"],"-4/1"],[["l1","l1","l1","p3"],"2/1"
+> ],[["l1","l1","l3","p1"],"6/1"],[["l1","l1","p1","p3"],"-6/1"],[["l1","l2","l2","l3"],"4/1"],[["l1","l2",
+> "l2","p3"],"-2/1"],[["l1","l2","l3","p2"],"-4/1"],[["l1","l2","p2","p3"],"6/1"],[["l1","l3","l3","l3"],"-
+> 4/1"],[["l1","l3","l3","p3"],"6/1"],[["l1","l3","p0","p0"],"-2/1"],[["l1","l3","p1","p1"],"-4/1"],[["l1",
+> "l3","p2","p2"],"-2/1"],[["l1","l3","p3","p3"],"-4/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1","p1","p1","p
+> 3"],"5/1"],[["l1","p2","p2","p3"],"-1/1"],[["l1","p3","p3","p3"],"1/1"],[["l2","l2","l3","p1"],"-2/1"],[[
+> "l2","l2","p1","p3"],"-4/1"],[["l2","l3","p1","p2"],"6/1"],[["l3","l3","l3","p1"],"2/1"],[["l3","l3","p1"
+> ,"p3"],"-6/1"],[["l3","p0","p0","p1"],"-1/1"],[["l3","p1","p1","p1"],"1/1"],[["l3","p1","p2","p2"],"-1/1"
+> ],[["l3","p1","p3","p3"],"5/1"],[["p0","p0","p1","p3"],"4/1"],[["p1","p1","p1","p3"],"-4/1"],[["p1","p2",
+> "p2","p3"],"4/1"],[["p1","p3","p3","p3"],"-4/1"]],"5,9":[[["l0","l0","l1","l2"],"-2/1"],[["l0","l0","l1",
+> "p2"],"1/1"],[["l0","l0","l2","p1"],"1/1"],[["l0","l0","p1","p2"],"1/1"],[["l0","l1","l2","p0"],"2/1"],[[
```

```
+> "l0","l1","p0","p2"],"-5/1"],[["l0","l2","p0","p1"],"-5/1"],[["l0","p0","p1","p2"],"3/1"],[["l1","l1","l1
+> ","l2"],"2/1"],[["l1","l1","l1","p2"],"-1/1"],[["l1","l1","l2","p1"],"-3/1"],[["l1","l1","p1","p2"],"4/1"
+> ],[["l1","l2","l2","l2"],"2/1"],[["l1","l2","l2","p2"],"-3/1"],[["l1","l2","l3","l3"],"6/1"],[["l1","l2",
+> "l3","p3"],"-6/1"],[["l1","l2","p0","p0"],"4/1"],[["l1","l2","p1","p1"],"1/1"],[["l1","l2","p2","p2"],"1/
+> 1"],[["l1","l2","p3","p3"],"4/1"],[["l1","l3","l3","p2"],"-3/1"],[["l1","l3","p2","p3"],"1/1"],[["l1","p1
+> ","p1","p2"],"-3/1"],[["l1","p2","p3","p3"],"-1/1"],[["l2","l2","l2","p1"],"-1/1"],[["l2","l2","p1","p2"]
+> ,"4/1"],[["l2","l3","l3","p1"],"-3/1"],[["l2","l3","p1","p3"],"1/1"],[["l2","p1","p2","p2"],"-3/1"],[["l2
+> ","p1","p3","p3"],"-1/1"],[["l3","l3","p1","p2"],"7/1"],[["l3","p1","p2","p3"],"-5/1"],[["p1","p2","p3","
+> p3"],"4/1"]],"6,0":[[["l0","l0","l1","l3"],"6/1"],[["l0","l0","l1","p3"],"-3/1"],[["l0","l0","l3","p1"],"
+> -3/1"],[["l0","l0","p1","p3"],"7/1"],[["l0","l1","l3","p0"],"-6/1"],[["l0","l1","p0","p3"],"1/1"],[["l0",
+> "l3","p0","p1"],"1/1"],[["l0","p0","p1","p3"],"-5/1"],[["l1","l1","l1","l3"],"-2/1"],[["l1","l1","l1","p3
+> "],"1/1"],[["l1","l1","l3","p1"],"3/1"],[["l1","l1","p1","p3"],"-4/1"],[["l1","l2","l2","l3"],"-2/1"],[["
+> l1","l2","l2","p3"],"1/1"],[["l1","l2","l3","p2"],"2/1"],[["l1","l2","p2","p3"],"-5/1"],[["l1","l3","l3",
+> "l3"],"-2/1"],[["l1","l3","l3","p3"],"3/1"],[["l1","l3","p0","p0"],"4/1"],[["l1","l3","p1","p1"],"-1/1"],
+> [["l1","l3","p2","p2"],"4/1"],[["l1","l3","p3","p3"],"-1/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1","p1","
+> p1","p3"],"3/1"],[["l2","l2","l3","p1"],"1/1"],[["l2","l2","p1","p3"],"1/1"],[["l2","l3","p1","p2"],"-5/1
+> "],[["l2","p1","p2","p3"],"3/1"],[["l3","l3","l3","p1"],"1/1"],[["l3","l3","p1","p3"],"-4/1"],[["l3","p0"
+> ,"p0","p1"],"-1/1"],[["l3","p1","p3","p3"],"3/1"],[["p0","p0","p1","p3"],"4/1"]],"6,1":[[["l0","l0","l0",
+> "l3"],"-4/1"],[["l0","l0","l0","p3"],"2/1"],[["l0","l0","l3","p0"],"6/1"],[["l0","l0","p0","p3"],"-6/1"],
+> [["l0","l1","l1","l3"],"-4/1"],[["l0","l1","l1","p3"],"2/1"],[["l0","l1","l3","p1"],"4/1"],[["l0","l1","p
+> 1","p3"],"-6/1"],[["l0","l2","l2","l3"],"4/1"],[["l0","l2","l2","p3"],"-2/1"],[["l0","l2","l3","p2"],"-4/
+> 1"],[["l0","l2","p2","p3"],"6/1"],[["l0","l3","l3","l3"],"4/1"],[["l0","l3","l3","p3"],"-6/1"],[["l0","l3
+> ","p0","p0"],"-4/1"],[["l0","l3","p1","p1"],"2/1"],[["l0","l3","p2","p2"],"-2/1"],[["l0","l3","p3","p3"],
+> "4/1"],[["l0","p0","p0","p3"],"5/1"],[["l0","p1","p1","p3"],"1/1"],[["l0","p2","p2","p3"],"-1/1"],[["l0",
+> "p3","p3","p3"],"-1/1"],[["l1","l1","l3","p0"],"2/1"],[["l1","l1","p0","p3"],"4/1"],[["l1","l3","p0","p1"
+> ],"-6/1"],[["l2","l2","l3","p0"],"-2/1"],[["l2","l3","p0","p2"],"6/1"],[["l2","p0","p2","p3"],"-4/1"],[["
+> l3","l3","l3","p0"],"-2/1"],[["l3","l3","p0","p3"],"6/1"],[["l3","p0","p0","p0"],"1/1"],[["l3","p0","p1",
+> "p1"],"1/1"],[["l3","p0","p2","p2"],"-1/1"],[["l3","p0","p3","p3"],"-5/1"],[["p0","p0","p0","p3"],"-4/1"]
+> ,[["p0","p1","p1","p3"],"-4/1"],[["p0","p2","p2","p3"],"4/1"],[["p0","p3","p3","p3"],"4/1"]],"6,2":[[["l0
+> ","l1","l2","l3"],"-8/1"],[["l0","l1","l2","p3"],"4/1"],[["l0","l1","l3","p2"],"4/1"],[["l0","l1","p2","p
+> 3"],"4/1"],[["l0","l2","l3","p1"],"4/1"],[["l0","l2","p1","p3"],"-16/1"],[["l0","l3","p1","p2"],"4/1"],[[
+> "l0","p1","p2","p3"],"2/1"],[["l1","l2","l3","p0"],"4/1"],[["l1","l2","p0","p3"],"4/1"],[["l1","l3","p0",
+> "p2"],"-16/1"],[["l1","p0","p2","p3"],"2/1"],[["l2","l3","p0","p1"],"4/1"],[["l2","p0","p1","p3"],"2/1"],
+> [["l3","p0","p1","p2"],"2/1"],[["p0","p1","p2","p3"],"-8/1"]],"6,3":[[["l0","l0","l0","l1"],"-4/1"],[["l0
+> ","l0","l0","p1"],"2/1"],[["l0","l0","l1","p0"],"6/1"],[["l0","l0","p0","p1"],"-6/1"],[["l0","l1","l1","l
+> 1"],"4/1"],[["l0","l1","l1","p1"],"-6/1"],[["l0","l1","l2","l2"],"4/1"],[["l0","l1","l2","p2"],"-4/1"],[[
+> "l0","l1","l3","l3"],"-4/1"],[["l0","l1","l3","p3"],"4/1"],[["l0","l1","p0","p0"],"-4/1"],[["l0","l1","p1
+> ","p1"],"4/1"],[["l0","l1","p2","p2"],"-2/1"],[["l0","l1","p3","p3"],"2/1"],[["l0","l2","l2","p1"],"-2/1"
+> ],[["l0","l2","p1","p2"],"6/1"],[["l0","l3","l3","p1"],"2/1"],[["l0","l3","p1","p3"],"-6/1"],[["l0","p0",
+> "p0","p1"],"5/1"],[["l0","p1","p1","p1"],"-1/1"],[["l0","p1","p2","p2"],"-1/1"],[["l0","p1","p3","p3"],"1
+> /1"],[["l1","l1","l1","p0"],"-2/1"],[["l1","l1","p0","p1"],"6/1"],[["l1","l2","l2","p0"],"-2/1"],[["l1","
+> l2","p0","p2"],"6/1"],[["l1","l3","l3","p0"],"2/1"],[["l1","l3","p0","p3"],"-6/1"],[["l1","p0","p0","p0"]
+> ,"1/1"],[["l1","p0","p1","p1"],"-5/1"],[["l1","p0","p2","p2"],"-1/1"],[["l1","p0","p3","p3"],"1/1"],[["l2
+> ","p0","p1","p2"],"-4/1"],[["l3","l3","p0","p1"],"4/1"],[["p0","p0","p0","p1"],"-4/1"],[["p0","p1","p1","
+> p1"],"4/1"],[["p0","p1","p2","p2"],"4/1"],[["p0","p1","p3","p3"],"-4/1"]],"6,4":[[["l0","l0","l1","l3"],"
+> 2/1"],[["l0","l0","l1","p3"],"-1/1"],[["l0","l0","l3","p1"],"-1/1"],[["l0","l0","p1","p3"],"1/1"],[["l0",
+> "l1","l3","p0"],"-2/1"],[["l0","l1","p0","p3"],"1/1"],[["l0","l3","p0","p1"],"1/1"],[["l0","p0","p1","p3"
+> ],"-1/1"],[["l1","l1","l1","l3"],"2/1"],[["l1","l1","l1","p3"],"-1/1"],[["l1","l1","l3","p1"],"-3/1"],[["
+> l1","l1","p1","p3"],"2/1"],[["l1","l2","l2","l3"],"-2/1"],[["l1","l2","l2","p3"],"1/1"],[["l1","l2","l3",
+> "p2"],"2/1"],[["l1","l2","p2","p3"],"-1/1"],[["l1","l3","l3","l3"],"-2/1"],[["l1","l3","l3","p3"],"3/1"],
+> [["l1","l3","p0","p0"],"2/1"],[["l1","l3","p1","p1"],"1/1"],[["l1","l3","p2","p2"],"-2/1"],[["l1","l3","p
+> 3","p3"],"-3/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1","p1","p1","p3"],"-1/1"],[["l1","p2","p2","p3"],"1/
+> 1"],[["l1","p3","p3","p3"],"1/1"],[["l2","l2","l3","p1"],"1/1"],[["l2","l2","p1","p3"],"-1/1"],[["l2","l3
+> ","p1","p2"],"-1/1"],[["l2","p1","p2","p3"],"1/1"],[["l3","l3","l3","p1"],"1/1"],[["l3","l3","p1","p3"],"
+> -2/1"],[["l3","p0","p0","p1"],"-1/1"],[["l3","p1","p2","p2"],"1/1"],[["l3","p1","p3","p3"],"2/1"],[["p0",
+> "p0","p1","p3"],"4/1"],[["p1","p2","p2","p3"],"-4/1"],[["p1","p3","p3","p3"],"-4/1"]],"6,5":[[["l0","l0","
+> "l2","l3"],"4/1"],[["l0","l0","l2","p3"],"-2/1"],[["l0","l0","l3","p2"],"-2/1"],[["l0","l2","l3","p0"],"-
+> 4/1"],[["l0","l2","p0","p3"],"6/1"],[["l0","l3","p0","p2"],"6/1"],[["l0","p0","p2","p3"],"-4/1"],[["l1","
+> l1","l2","l3"],"4/1"],[["l1","l1","l2","p3"],"-2/1"],[["l1","l1","l3","p2"],"-2/1"],[["l1","l1","p2","p3"
+> ],"-4/1"],[["l1","l2","l3","p1"],"-4/1"],[["l1","l2","p1","p3"],"6/1"],[["l1","l3","p1","p2"],"6/1"],[["l
+> 2","l2","l2","l3"],"-4/1"],[["l2","l2","l2","p3"],"2/1"],[["l2","l2","l3","p2"],"6/1"],[["l2","l2","p2","
+> p3"],"-6/1"],[["l2","l3","l3","l3"],"-4/1"],[["l2","l3","l3","p3"],"6/1"],[["l2","l3","p0","p0"],"-2/1"],
+> [["l2","l3","p1","p1"],"-2/1"],[["l2","l3","p2","p2"],"-4/1"],[["l2","l3","p3","p3"],"-4/1"],[["l2","p0",
+> "p0","p3"],"-1/1"],[["l2","p1","p1","p3"],"-1/1"],[["l2","p2","p2","p3"],"5/1"],[["l2","p3","p3","p3"],"1
+> /1"],[["l3","l3","l3","p2"],"2/1"],[["l3","l3","p2","p3"],"-6/1"],[["l3","p0","p0","p2"],"-1/1"],[["l3","
+> p1","p1","p2"],"-1/1"],[["l3","p2","p2","p2"],"1/1"],[["l3","p2","p3","p3"],"5/1"],[["p0","p0","p2","p3"]
+> ,"4/1"],[["p1","p1","p2","p3"],"4/1"],[["p2","p2","p2","p3"],"-4/1"],[["p2","p3","p3","p3"],"-4/1"]],"6,6
+> ":[[["l0","l0","l0","l0"],"4/1"],[["l0","l0","l0","p0"],"-8/1"],[["l0","l0","l1","l1"],"-4/1"],[["l0","l0
+> ","l1","p1"],"4/1"],[["l0","l0","l2","l2"],"-8/1"],[["l0","l0","l2","p2"],"8/1"],[["l0","l0","l3","l3","
+> -4/1"],[["l0","l0","l3","p3"],"4/1"],[["l0","l0","p0","p0"],"10/1"],[["l0","l0","p1","p1"],"-2/1"],[["l0"
+> ,"l0","p2","p2"],"-2/1"],[["l0","l0","p3","p3"],"-2/1"],[["l0","l1","l1","p0"],"4/1"],[["l0","l1","p0","p
+> 1"],"-4/1"],[["l0","l2","l2","p0"],"8/1"],[["l0","l2","p0","p2"],"-16/1"],[["l0","l3","l3","p0"],"4/1"],[
+> ["l0","l3","p0","p3"],"-4/1"],[["l0","p0","p0","p0"],"-6/1"],[["l0","p0","p1","p1"],"2/1"],[["l0","p0","p
+> 2","p2"],"6/1"],[["l0","p0","p3","p3"],"2/1"],[["l1","l1","l2","l2"],"4/1"],[["l1","l1","l2","p2"],"-4/1"
+> ],[["l1","l1","l3","l3"],"8/1"],[["l1","l1","l3","p3"],"-8/1"],[["l1","l1","p0","p0"],"-4/1"],[["l1","l1"
+> ,"p2","p2"],"4/1"],[["l1","l2","l2","p1"],"-4/1"],[["l1","l2","p1","p2"],"4/1"],[["l1","l3","l3","p1"],"-
+> 8/1"],[["l1","l3","p1","p3"],"16/1"],[["l1","p0","p0","p1"],"4/1"],[["l1","p1","p2","p2"],"-4/1"],[["l1",
+> "p1","p3","p3"],"-4/1"],[["l2","l2","l2","l2"],"4/1"],[["l2","l2","l2","p2"],"-8/1"],[["l2","l2","l3","l3
+> "],"4/1"],[["l2","l2","l3","p3"],"-4/1"],[["l2","l2","p0","p0"],"-2/1"],[["l2","l2","p1","p1"],"2/1"],[["
+> l2","l2","p2","p2"],"10/1"],[["l2","l2","p3","p3"],"2/1"],[["l2","l3","l3","p2"],"-4/1"],[["l2","l3","p2"
+> ,"p3"],"4/1"],[["l2","p0","p0","p2"],"6/1"],[["l2","p1","p1","p2"],"-2/1"],[["l2","p2","p2","p2"],"-6/1"]
```

+> ,[["l2","p2","p3","p3"],"-2/1"],[["l3","l3","p0","p0"],"-4/1"],[["l3","l3","p2","p2"],"4/1"],[["l3","p0",
+> "p0","p3"],"4/1"],[["l3","p1","p1","p3"],"-4/1"],[["l3","p2","p2","p3"],"-4/1"],[["p0","p0","p0","p0"],"4
+> /1"],[["p0","p0","p1","p1"],"-4/1"],[["p0","p0","p2","p2"],"-8/1"],[["p0","p0","p3","p3"],"-4/1"],[["p1",
+> "p1","p2","p2"],"4/1"],[["p1","p1","p3","p3"],"8/1"],[["p2","p2","p2","p2"],"4/1"],[["p2","p2","p3","p3"]
+> ,"4/1"]],"6,7":[[["l0","l0","l1","l3"],"-2/1"],[["l0","l0","l1","p3"],"1/1"],[["l0","l0","l3","p1"],"1/1"
+>],[["l0","l0","p1","p3"],"1/1"],[["l0","l1","l3","p0"],"2/1"],[["l0","l1","p0","p3"],"-5/1"],[["l0","l3",
+> "p0","p1"],"-5/1"],[["l0","p0","p1","p3"],"3/1"],[["l1","l1","l1","l3"],"2/1"],[["l1","l1","l1","p3"],"-1
+> /1"],[["l1","l1","l3","p1"],"-3/1"],[["l1","l1","p1","p3"],"4/1"],[["l1","l2","l2","l3"],"6/1"],[["l1","l
+> 2","l2","p3"],"-3/1"],[["l1","l2","l3","p2"],"-6/1"],[["l1","l2","p2","p3"],"1/1"],[["l1","l3","l3","l3"]
+> ,"2/1"],[["l1","l3","l3","p3"],"-3/1"],[["l1","l3","p0","p0"],"4/1"],[["l1","l3","p1","p1"],"1/1"],[["l1"
+> ,"l3","p2","p2"],"4/1"],[["l1","l3","p3","p3"],"1/1"],[["l1","p1","p1","p3"],"-3/1"],[["l1","p2","p2","p3
+> "],"-1/1"],[["l2","l2","l3","p1"],"-3/1"],[["l2","l2","p1","p3"],"7/1"],[["l2","l3","p1","p2"],"1/1"],[["
+> l2","p1","p2","p3"],"-5/1"],[["l3","l3","l3","p1"],"-1/1"],[["l3","l3","p1","p3"],"4/1"],[["l3","p1","p2"
+> ,"p2"],"-1/1"],[["l3","p1","p3","p3"],"-3/1"],[["p1","p2","p2","p3"],"4/1"]],"6,8":[[["l0","l0","l1","l2"
+>],"4/1"],[["l0","l0","l1","p2"],"-2/1"],[["l0","l0","l2","p1"],"-2/1"],[["l0","l1","l2","p0"],"-4/1"],[["
+> l0","l1","p0","p2"],"6/1"],[["l0","l2","p0","p1"],"6/1"],[["l0","p0","p1","p2"],"-4/1"],[["l1","l1","l1",
+> "l2"],"-4/1"],[["l1","l1","l1","p2"],"2/1"],[["l1","l1","l2","p1"],"6/1"],[["l1","l1","p1","p2"],"-6/1"],
+> [["l1","l2","l2","l2"],"-4/1"],[["l1","l2","l2","p2"],"6/1"],[["l1","l2","l3","l3"],"4/1"],[["l1","l2","l
+> 3","p3"],"-4/1"],[["l1","l2","p0","p0"],"-2/1"],[["l1","l2","p1","p1"],"-4/1"],[["l1","l2","p2","p2"],"-4
+> /1"],[["l1","l2","p3","p3"],"-2/1"],[["l1","l3","l3","p2"],"-2/1"],[["l1","l3","p2","p3"],"6/1"],[["l1","
+> p0","p0","p2"],"-1/1"],[["l1","p1","p1","p2"],"5/1"],[["l1","p2","p2","p2"],"1/1"],[["l1","p2","p3","p3"]
+> ,"-1/1"],[["l2","l2","l2","p1"],"2/1"],[["l2","l2","p1","p2"],"-6/1"],[["l2","l3","l3","p1"],"-2/1"],[["l
+> 2","l3","p1","p3"],"6/1"],[["l2","p0","p0","p1"],"-1/1"],[["l2","p1","p1","p1"],"1/1"],[["l2","p1","p2","
+> p2"],"5/1"],[["l2","p1","p3","p3"],"-1/1"],[["l3","l3","p1","p2"],"-4/1"],[["p0","p0","p1","p2"],"4/1"],[
+> ["p1","p1","p1","p2"],"-4/1"],[["p1","p2","p2","p2"],"-4/1"],[["p1","p2","p3","p3"],"4/1"]],"6,9":[[["l0"
+> ,"l0","l1","l3"],"2/1"],[["l0","l0","l1","p3"],"-1/1"],[["l0","l0","l3","p1"],"-1/1"],[["l0","l0","p1","p
+> 3"],"1/1"],[["l0","l1","l3","p0"],"-2/1"],[["l0","l1","p0","p3"],"1/1"],[["l0","l3","p0","p1"],"1/1"],[["
+> l0","p0","p1","p3"],"-1/1"],[["l1","l1","l1","l3"],"-2/1"],[["l1","l1","l1","p3"],"1/1"],[["l1","l1","l3"
+> ,"p1"],"3/1"],[["l1","l1","p1","p3"],"-2/1"],[["l1","l2","l2","l3"],"-2/1"],[["l1","l2","l2","p3"],"1/1"]
+> ,[["l1","l2","l3","p2"],"2/1"],[["l1","l2","p2","p3"],"-1/1"],[["l1","l3","l3","l3"],"2/1"],[["l1","l3","
+> l3","p3"],"-3/1"],[["l1","l3","p0","p0"],"2/1"],[["l1","l3","p1","p1"],"-3/1"],[["l1","l3","p2","p2"],"-2
+> /1"],[["l1","l3","p3","p3"],"1/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1","p1","p1","p3"],"2/1"],[["l1","p
+> 2","p2","p3"],"1/1"],[["l2","l2","l3","p1"],"1/1"],[["l2","l2","p1","p3"],"-1/1"],[["l2","l3","p1","p2"],
+> "-1/1"],[["l2","p1","p2","p3"],"1/1"],[["l3","l3","l3","p1"],"-1/1"],[["l3","l3","p1","p3"],"4/1"],[["l3"
+> ,"p0","p0","p1"],"-1/1"],[["l3","p1","p1","p1"],"1/1"],[["l3","p1","p2","p2"],"1/1"],[["l3","p1","p3","p3
+> "],"-1/1"],[["p0","p0","p1","p3"],"4/1"],[["p1","p1","p1","p3"],"-4/1"],[["p1","p2","p2","p3"],"-4/1"]],"
+> 7,0":[[["l0","l0","l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l2","l2"],"3/1"],[["l0","
+> l0","l2","p2"],"-3/1"],[["l0","l0","p1","p1"],"-3/2"],[["l0","l0","p2","p2"],"2/1"],[["l0","l0","p3","p3"
+>],"-3/2"],[["l0","l1","p0","p1"],"-1/1"],[["l0","l2","l2","p0"],"-3/1"],[["l0","l3","p0","p3"],"-1/1"],[[
+> "l0","p0","p0","p0"],"-1/2"],[["l0","p0","p1","p1"],"2/1"],[["l0","p0","p2","p2"],"-1/2"],[["l0","p0","p3
+> ","p3"],"2/1"],[["l1","l1","l1","l1"],"1/2"],[["l1","l1","l1","p1"],"-1/1"],[["l1","l1","l3","l3"],"1/1"]
+> ,[["l1","l1","l3","p3"],"-1/1"],[["l1","l1","p1","p1"],"5/2"],[["l1","l1","p3","p3"],"-1/2"],[["l1","l2",
+> "p1","p2"],"1/1"],[["l1","l3","l3","p1"],"-1/1"],[["l1","l3","p1","p3"],"6/1"],[["l1","p0","p0","p1"],"1/
+> 2"],[["l1","p1","p1","p1"],"-2/1"],[["l1","p1","p2","p2"],"-1/2"],[["l1","p1","p3","p3"],"-2/1"],[["l2","
+> l2","l2","l2"],"-1/2"],[["l2","l2","l2","p2"],"1/1"],[["l2","l2","p0","p0"],"2/1"],[["l2","l2","p1","p1"]
+> ,"3/2"],[["l2","l2","p3","p3"],"3/2"],[["l2","l3","p2","p3"],"1/1"],[["l2","p0","p0","p2"],"-1/2"],[["l2"
+> ,"p1","p1","p2"],"-2/1"],[["l2","p2","p2","p2"],"-1/2"],[["l2","p2","p3","p3"],"-2/1"],[["l3","l3","l3","
+> l3"],"1/2"],[["l3","l3","l3","p3"],"-1/1"],[["l3","l3","p1","p1"],"-1/2"],[["l3","l3","p3","p3"],"5/2"],[
+> ["l3","p0","p0","p3"],"1/2"],[["l3","p1","p1","p3"],"-2/1"],[["l3","p2","p2","p3"],"-1/2"],[["l3","p3","p
+> 3","p3"],"-2/1"],[["p0","p0","p2","p2"],"2/1"]],"7,1":[[["l0","l0","l0","l1"],"2/1"],[["l0","l0","l0","p1
+> "],"-1/1"],[["l0","l0","l1","p0"],"-3/1"],[["l0","l0","p0","p1"],"4/1"],[["l0","l1","l1","l1"],"-2/1"],[[
+> "l0","l1","l1","p1"],"3/1"],[["l0","l1","l2","l2"],"-6/1"],[["l0","l1","l2","p2"],"6/1"],[["l0","l1","l3"
+> ,"l3"],"-2/1"],[["l0","l1","l3","p3"],"2/1"],[["l0","l1","p0","p0"],"1/1"],[["l0","l1","p1","p1"],"-1/1"]
+> ,[["l0","l1","p2","p2"],"-4/1"],[["l0","l1","p3","p3"],"4/1"],[["l0","l2","l2","p1"],"3/1"],[["l0","l2","
+> p1","p2"],"-1/1"],[["l0","l3","l3","p1"],"1/1"],[["l0","l3","p1","p3"],"-5/1"],[["l0","p0","p0","p1"],"-3
+> /1"],[["l0","p1","p2","p2"],"1/1"],[["l1","l1","l1","p0"],"1/1"],[["l1","l1","p0","p1"],"-4/1"],[["l1","l
+> 2","l2","p0"],"3/1"],[["l1","l2","p0","p2"],"-1/1"],[["l1","l3","l3","p0"],"1/1"],[["l1","l3","p0","p3"],
+> "-5/1"],[["l1","p0","p1","p1"],"3/1"],[["l1","p0","p2","p2"],"1/1"],[["l2","l2","p0","p1"],"-7/1"],[["l2"
+> ,"p0","p1","p2"],"5/1"],[["l3","l3","p0","p1"],"1/1"],[["l3","p0","p1","p3"],"3/1"],[["p0","p1","p2","p2"
+>],"-4/1"]],"7,2":[[["l0","l0","l0","l2"],"-2/1"],[["l0","l0","l0","p2"],"1/1"],[["l0","l0","l2","p0"],"3/
+> 1"],[["l0","l0","p0","p2"],"-2/1"],[["l0","l1","l1","l2"],"2/1"],[["l0","l1","l1","p2"],"-1/1"],[["l0","l
+> 1","l2","p1"],"-2/1"],[["l0","l1","p1","p2"],"1/1"],[["l0","l2","l2","l2"],"-2/1"],[["l0","l2","l2","p2"]
+> ,"3/1"],[["l0","l2","l3","l3"],"2/1"],[["l0","l2","l3","p3"],"-2/1"],[["l0","l2","p0","p0"],"-3/1"],[["l0
+> ","l2","p1","p1"],"2/1"],[["l0","l2","p2","p2"],"-1/1"],[["l0","l2","p3","p3"],"2/1"],[["l0","l3","l3","p
+> 2"],"-1/1"],[["l0","l3","p2","p3"],"1/1"],[["l0","p0","p0","p2"],"2/1"],[["l0","p1","p1","p2"],"-1/1"],[[
+> "l0","p2","p3","p3"],"-1/1"],[["l1","l1","l2","p0"],"-1/1"],[["l1","l1","p0","p2"],"1/1"],[["l1","l2","p0
+> ","p1"],"1/1"],[["l1","p0","p1","p2"],"-1/1"],[["l2","l2","l2","p0"],"1/1"],[["l2","l2","p0","p2"],"-2/1"
+>],[["l2","l3","l3","p0"],"-1/1"],[["l2","l3","p0","p3"],"1/1"],[["l2","p0","p0","p0"],"1/1"],[["l2","p0",
+> "p1","p1"],"-1/1"],[["l2","p0","p2","p2"],"1/1"],[["l2","p0","p3","p3"],"-1/1"],[["l3","l3","p0","p2"],"1
+> /1"],[["l3","p0","p2","p3"],"-1/1"],[["p0","p0","p0","p2"],"-4/1"],[["p0","p1","p1","p2"],"4/1"],[["p0","
+> p2","p3","p3"],"4/1"]],"7,3":[[["l0","l0","l0","l3"],"2/1"],[["l0","l0","l0","p3"],"-1/1"],[["l0","l0","l
+> 3","p0"],"-3/1"],[["l0","l0","p0","p3"],"4/1"],[["l0","l1","l1","l3"],"-2/1"],[["l0","l1","l1","p3"],"1/1
+> "],[["l0","l1","l3","p1"],"2/1"],[["l0","l1","p1","p3"],"-5/1"],[["l0","l2","l2","l3"],"-6/1"],[["l0","l2
+> ","l2","p3"],"3/1"],[["l0","l2","l3","p2"],"6/1"],[["l0","l2","p2","p3"],"-1/1"],[["l0","l3","l3","l3"],"
+> -2/1"],[["l0","l3","l3","p3"],"3/1"],[["l0","l3","p0","p0"],"1/1"],[["l0","l3","p1","p1"],"4/1"],[["l0","
+> l3","p2","p2"],"-4/1"],[["l0","l3","p3","p3"],"-1/1"],[["l0","p0","p0","p3"],"-3/1"],[["l0","p2","p2","p3
+> "],"1/1"],[["l1","l1","l3","p0"],"1/1"],[["l1","l1","p0","p3"],"1/1"],[["l1","l3","p0","p1"],"-5/1"],[["l
+> 1","p0","p1","p3"],"3/1"],[["l2","l2","l3","p0"],"3/1"],[["l2","l2","p0","p3"],"-7/1"],[["l2","l3","p0","
+> p2"],"-1/1"],[["l2","p0","p2","p3"],"5/1"],[["l3","l3","l3","p0"],"1/1"],[["l3","l3","p0","p3"],"-4/1"],[
+> ["l3","p0","p2","p2"],"1/1"],[["l3","p0","p3","p3"],"3/1"],[["p0","p2","p2","p3"],"-4/1"]],"7,4":[[["l0",
+> "l0","l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l3","l3"],"1/1"],[["l0","l0","l3","p3"

```
+> ],"-1/1"],[["l0","l0","p0","p0"],"-5/2"],[["l0","l0","p3","p3"],"-1/2"],[["l0","l1","p0","p1"],"1/1"],[["
+> l0","l2","p0","p2"],"1/1"],[["l0","l3","l3","p0"],"-1/1"],[["l0","l3","p0","p3"],"6/1"],[["l0","p0","p0",
+> "p0"],"2/1"],[["l0","p0","p1","p1"],"-1/2"],[["l0","p0","p2","p2"],"-1/2"],[["l0","p0","p3","p3"],"-2/1"]
+> ,[["l1","l1","l1","l1"],"1/2"],[["l1","l1","l1","p1"],"-1/1"],[["l1","l1","l2","l2"],"3/1"],[["l1","l1","
+> l2","p2"],"-3/1"],[["l1","l1","p0","p0"],"3/2"],[["l1","l1","p2","p2"],"2/1"],[["l1","l1","p3","p3"],"-3/
+> 2"],[["l1","l2","l2","p1"],"-3/1"],[["l1","l3","p1","p3"],"-1/1"],[["l1","p0","p0","p1"],"-2/1"],[["l1","
+> p1","p1","p1"],"1/2"],[["l1","p1","p2","p2"],"-1/2"],[["l1","p1","p3","p3"],"2/1"],[["l2","l2","l2","l2"]
+> ,"1/2"],[["l2","l2","l2","p2"],"-1/1"],[["l2","l2","p0","p0"],"3/2"],[["l2","l2","p1","p1"],"2/1"],[["l2
+> ","l2","p3","p3"],"-3/2"],[["l2","l3","p2","p3"],"-1/1"],[["l2","p0","p0","p2"],"-2/1"],[["l2","p1","p1","
+> p2"],"-1/2"],[["l2","p2","p2","p2"],"1/2"],[["l2","p2","p3","p3"],"2/1"],[["l3","l3","l3","l3"],"-1/2"],[
+> ["l3","l3","l3","p3"],"1/1"],[["l3","l3","p0","p0"],"-1/2"],[["l3","l3","p3","p3"],"-5/2"],[["l3","p0","p
+> 0","p3"],"-2/1"],[["l3","p1","p1","p3"],"1/2"],[["l3","p2","p2","p3"],"1/2"],[["l3","p3","p3","p3"],"2/1"
+> ],[["p1","p1","p2","p2"],"2/1"]],"7,5":[[["l0","l0","l1","l2"],"2/1"],[["l0","l0","l1","p2"],"-1/1"],[["l
+> 0","l0","l2","p1"],"-1/1"],[["l0","l0","p1","p2"],"1/1"],[["l0","l1","l2","p0"],"-2/1"],[["l0","l1","p0",
+> "p2"],"1/1"],[["l0","l2","p0","p1"],"1/1"],[["l0","p0","p1","p2"],"-1/1"],[["l1","l1","l1","l2"],"-2/1"],
+> [["l1","l1","l1","p2"],"1/1"],[["l1","l1","l2","p1"],"3/1"],[["l1","l1","p1","p2"],"-2/1"],[["l1","l2","l
+> 2","l2"],"2/1"],[["l1","l2","l2","p2"],"-3/1"],[["l1","l2","l3","l3"],"-2/1"],[["l1","l2","l3","p3"],"2/1
+> "],[["l1","l2","p0","p0"],"2/1"],[["l1","l2","p1","p1"],"-3/1"],[["l1","l2","p2","p2"],"1/1"],[["l1","l2
+> ","p3","p3"],"-2/1"],[["l1","l3","l3","p2"],"1/1"],[["l1","l3","p2","p3"],"-1/1"],[["l1","p0","p0","p2","
+> -1/1"],[["l1","p1","p1","p2"],"2/1"],[["l1","p2","p3","p3"],"1/1"],[["l2","l2","l2","p1"],"-1/1"],[["l2",
+> "l2","p1","p2"],"2/1"],[["l2","l3","l3","p1"],"1/1"],[["l2","l3","p1","p3"],"-1/1"],[["l2","p0","p0","p1"
+> ],"-1/1"],[["l2","p1","p1","p1"],"1/1"],[["l2","p1","p2","p2"],"-1/1"],[["l2","p1","p3","p3"],"1/1"],[["l
+> 3","l3","p1","p2"],"-1/1"],[["l3","p1","p2","p3"],"1/1"],[["p0","p0","p1","p2"],"4/1"],[["p1","p1","p1","
+> p2"],"-4/1"],[["p1","p2","p3","p3"],"-4/1"]],"7,6":[[["l0","l0","l1","l3"],"-2/1"],[["l0","l0","l1","p3"]
+> ,"1/1"],[["l0","l0","l3","p1"],"1/1"],[["l0","l0","p1","p3"],"1/1"],[["l0","l1","l3","p0"],"2/1"],[["l0",
+> "l1","p0","p3"],"-5/1"],[["l0","l3","p0","p1"],"-5/1"],[["l0","p0","p1","p3"],"3/1"],[["l1","l1","l1","l3
+> "],"2/1"],[["l1","l1","l1","p3"],"-1/1"],[["l1","l1","l3","p1"],"-3/1"],[["l1","l1","p1","p3"],"4/1"],[["
+> l1","l2","l2","l3"],"6/1"],[["l1","l2","l2","p3"],"-3/1"],[["l1","l2","l3","p2"],"-6/1"],[["l1","l2","p2"
+> ,"p3"],"1/1"],[["l1","l3","l3","l3"],"2/1"],[["l1","l3","l3","p3"],"-3/1"],[["l1","l3","p0","p0"],"4/1"],
+> [["l1","l3","p1","p1"],"1/1"],[["l1","l3","p2","p2"],"4/1"],[["l1","l3","p3","p3"],"1/1"],[["l1","p1","p1
+> ","p3"],"-3/1"],[["l1","p2","p2","p3"],"-1/1"],[["l2","l2","l3","p1"],"-3/1"],[["l2","l2","p1","p3"],"7/1
+> "],[["l2","l3","p1","p2"],"1/1"],[["l2","p1","p2","p3"],"-5/1"],[["l3","l3","l3","p1"],"-1/1"],[["l3","l3
+> ","p1","p3"],"4/1"],[["l3","p1","p2","p2"],"-1/1"],[["l3","p1","p3","p3"],"-3/1"],[["p1","p2","p2","p3"],
+> "4/1"]],"7,7":[[["l0","l0","l0","l0"],"3/2"],[["l0","l0","l0","p0"],"-3/1"],[["l0","l0","l1","l1"],"-3/1"
+> ],[["l0","l0","l1","p1"],"3/1"],[["l0","l0","l2","l2"],"-1/1"],[["l0","l0","l2","p2"],"1/1"],[["l0","l0",
+> "l3","l3"],"-3/1"],[["l0","l0","l3","p3"],"3/1"],[["l0","l0","p0","p0"],"5/2"],[["l0","l0","p1","p1"],"-3
+> /2"],[["l0","l0","p2","p2"],"-1/2"],[["l0","l0","p3","p3"],"-3/2"],[["l0","l1","l1","p0"],"3/1"],[["l0","
+> l1","p0","p1"],"-2/1"],[["l0","l2","l2","p0"],"1/1"],[["l0","l3","l3","p0"],"3/1"],[["l0","l3","p0","p3"]
+> ,"-2/1"],[["l0","p0","p0","p0"],"-1/1"],[["l0","p0","p1","p1"],"1/1"],[["l0","p0","p3","p3"],"1/1"],[["l1
+> ","l1","l1","l1"],"3/2"],[["l1","l1","l1","p1"],"-3/1"],[["l1","l1","l2","l2"],"1/1"],[["l1","l1","l2","p
+> 2"],"-1/1"],[["l1","l1","l3","l3"],"3/1"],[["l1","l1","l3","p3"],"-3/1"],[["l1","l1","p0","p0"],"-3/2"],[
+> ["l1","l1","p1","p1"],"5/2"],[["l1","l1","p2","p2"],"1/2"],[["l1","l1","p3","p3"],"3/2"],[["l1","l2","l2"
+> ,"p1"],"-1/1"],[["l1","l3","l3","p1"],"-3/1"],[["l1","l3","p1","p3"],"2/1"],[["l1","p0","p0","p1"],"1/1"]
+> ,[["l1","p1","p1","p1"],"-1/1"],[["l1","p1","p3","p3"],"-1/1"],[["l2","l2","l2","l2"],"3/2"],[["l2","l2",
+> "l2","p2"],"-3/1"],[["l2","l2","l3","l3"],"1/1"],[["l2","l2","l3","p3"],"-1/1"],[["l2","l2","p0","p0"],"1
+> /2"],[["l2","l2","p1","p1"],"-1/2"],[["l2","l2","p2","p2"],"3/2"],[["l2","l2","p3","p3"],"-1/2"],[["l2","
+> l3","l3","p2"],"-1/1"],[["l2","p0","p0","p2"],"-1/1"],[["l2","p1","p1","p2"],"1/1"],[["l2","p2","p3","p3"
+> ],"1/1"],[["l3","l3","l3","l3"],"3/2"],[["l3","l3","l3","p3"],"-3/1"],[["l3","l3","p0","p0"],"-3/2"],[["l
+> 3","l3","p1","p1"],"3/2"],[["l3","l3","p2","p2"],"1/2"],[["l3","l3","p3","p3"],"5/2"],[["l3","p0","p0","p
+> 3"],"1/1"],[["l3","p1","p1","p3"],"-1/1"],[["l3","p3","p3","p3"],"-1/1"],[["p0","p0","p0","p0"],"2/1"],[[
+> "p0","p0","p1","p1"],"-4/1"],[["p0","p0","p3","p3"],"-4/1"],[["p1","p1","p1","p1"],"2/1"],[["p1","p1","p3
+> ","p3"],"4/1"],[["p3","p3","p3","p3"],"2/1"]],"7,8":[[["l0","l0","l2","l3"],"2/1"],[["l0","l0","l2","p3"]
+> ,"-1/1"],[["l0","l0","l3","p2"],"-1/1"],[["l0","l0","p2","p3"],"1/1"],[["l0","l2","l3","p0"],"-2/1"],[["l
+> 0","l2","p0","p3"],"1/1"],[["l0","l3","p0","p2"],"1/1"],[["l0","p0","p2","p3"],"-1/1"],[["l1","l1","l2","
+> l3"],"-2/1"],[["l1","l1","l2","p3"],"1/1"],[["l1","l1","l3","p2"],"1/1"],[["l1","l1","p2","p3"],"-1/1"],[
+> ["l1","l2","l3","p1"],"2/1"],[["l1","l2","p1","p3"],"-1/1"],[["l1","l3","p1","p2"],"-1/1"],[["l1","p1","p
+> 2","p3"],"1/1"],[["l2","l2","l2","l3"],"2/1"],[["l2","l2","l2","p3"],"-1/1"],[["l2","l2","l3","p2"],"-3/1
+> "],[["l2","l2","p2","p3"],"2/1"],[["l2","l3","l3","l3"],"-2/1"],[["l2","l3","l3","p3"],"3/1"],[["l2","l3"
+> ,"p0","p0"],"2/1"],[["l2","l3","p1","p1"],"-2/1"],[["l2","l3","p2","p2"],"1/1"],[["l2","l3","p3","p3"],"-
+> 3/1"],[["l2","p0","p0","p3"],"-1/1"],[["l2","p1","p1","p3"],"1/1"],[["l2","p2","p2","p3"],"-1/1"],[["l2",
+> "p3","p3","p3"],"1/1"],[["l3","l3","l3","p2"],"1/1"],[["l3","l3","p2","p3"],"-2/1"],[["l3","p0","p0","p2"
+> ],"-1/1"],[["l3","p1","p1","p2"],"1/1"],[["l3","p2","p3","p3"],"2/1"],[["p0","p0","p2","p3"],"4/1"],[["p1
+> ","p1","p2","p3"],"-4/1"],[["p2","p3","p3","p3"],"-4/1"]],"7,9":[[["l0","l0","l0","l0"],"-1/2"],[["l0","l
+> 0","l0","p0"],"1/1"],[["l0","l0","l1","l1"],"1/1"],[["l0","l0","l1","p1"],"-1/1"],[["l0","l0","p0","p0"],
+> "-5/2"],[["l0","l0","p1","p1"],"-1/2"],[["l0","l1","l1","p0"],"-1/1"],[["l0","l1","p0","p1"],"6/1"],[["l0
+> ","l2","p0","p2"],"1/1"],[["l0","l3","p0","p3"],"1/1"],[["l0","p0","p0","p0"],"2/1"],[["l0","p0","p1","p1
+> "],"-2/1"],[["l0","p0","p2","p2"],"-1/2"],[["l0","p0","p3","p3"],"-1/2"],[["l1","l1","l1","l1"],"-1/2"],[
+> ["l1","l1","l1","p1"],"1/1"],[["l1","l1","p0","p0"],"-1/2"],[["l1","l1","p1","p1"],"-5/2"],[["l1","l2","p
+> 1","p2"],"-1/1"],[["l1","l3","p1","p3"],"-1/1"],[["l1","p0","p0","p1"],"-2/1"],[["l1","p1","p1","p1"],"2/
+> 1"],[["l1","p1","p2","p2"],"1/2"],[["l1","p1","p3","p3"],"1/2"],[["l2","l2","l2","l2"],"1/2"],[["l2","l2"
+> ,"l2","p2"],"-1/1"],[["l2","l2","l3","l3"],"3/1"],[["l2","l2","l3","p3"],"-3/1"],[["l2","l2","p0","p0"],"
+> 3/2"],[["l2","l2","p1","p1"],"-3/2"],[["l2","l2","p3","p3"],"2/1"],[["l2","l3","l3","p2"],"-3/1"],[["l2",
+> "p0","p0","p2"],"-2/1"],[["l2","p1","p1","p2"],"2/1"],[["l2","p2","p2","p2"],"1/2"],[["l2","p2","p3","p3"
+> ],"-1/2"],[["l3","l3","l3","l3"],"1/2"],[["l3","l3","l3","p3"],"-1/1"],[["l3","l3","p0","p0"],"3/2"],[["l
+> 3","l3","p1","p1"],"-3/2"],[["l3","l3","p2","p2"],"2/1"],[["l3","p0","p0","p3"],"-2/1"],[["l3","p1","p1",
+> "p3"],"2/1"],[["l3","p2","p2","p3"],"-1/2"],[["l3","p3","p3","p3"],"1/2"],[["p2","p2","p3","p3"],"2/1"]],
+> "8,0":[[["l0","l0","l2","l3"],"6/1"],[["l0","l0","l2","p3"],"-3/1"],[["l0","l0","l3","p2"],"-3/1"],[["l0"
+> ,"l0","p2","p3"],"7/1"],[["l0","l2","l3","p0"],"-6/1"],[["l0","l2","p0","p3"],"1/1"],[["l0","l3","p0","p2
+> "],"1/1"],[["l0","p0","p2","p3"],"-5/1"],[["l1","l1","l2","l3"],"-2/1"],[["l1","l1","l2","p3"],"1/1"],[["
+> l1","l1","l3","p2"],"1/1"],[["l1","l1","p2","p3"],"1/1"],[["l1","l2","l3","p1"],"2/1"],[["l1","l2","p1","
+> p3"],"-5/1"],[["l1","l3","p1","p2"],"-5/1"],[["l1","p1","p2","p3"],"3/1"],[["l2","l2","l2","l3"],"-2/1"],
```

```
+> [["l2","l2","l2","p3"],"1/1"],[["l2","l2","l3","p2"],"3/1"],[["l2","l2","p2","p3"],"-4/1"],[["l2","l3","l
+> 3","l3"],"-2/1"],[["l2","l3","l3","p3"],"3/1"],[["l2","l3","p0","p0"],"4/1"],[["l2","l3","p1","p1"],"4/1"
+> ],[["l2","l3","p2","p2"],"-1/1"],[["l2","l3","p3","p3"],"-1/1"],[["l2","p0","p0","p3"],"-1/1"],[["l2","p2
+> ","p2","p3"],"3/1"],[["l3","l3","l3","p2"],"1/1"],[["l3","l3","p2","p3"],"-4/1"],[["l3","p0","p0","p2"],"
+> -1/1"],[["l3","p2","p3","p3"],"3/1"],[["p0","p0","p2","p3"],"4/1"]],8,1":[[["l0","11","l2","l3"],"-8/1"]
+> ,[["l0","11","l2","p3"],"4/1"],[["l0","11","l3","p2"],"4/1"],[["l0","11","p2","p3"],"-16/1"],[["l0","l2",
+> "l3","p1"],"4/1"],[["l0","l2","p1","p3"],"4/1"],[["l0","l3","p1","p2","p3"],"4/1"],[["l0","p1","p2","p3"],"2/1
+> "],[["l1","l2","l3","p0"],"4/1"],[["l1","l2","p0","p3"],"4/1"],[["l1","l3","p0","p2"],"4/1"],[["l1","p0",
+> "p2","p3"],"2/1"],[["l2","l3","p0","p1"],"-16/1"],[["l2","p0","p1","p3"],"2/1"],[["l3","p0","p1","p2"],"2
+> /1"],[["p0","p1","p2","p3"],"-8/1"]],8,2":[[["l0","l0","l0","l3"],"-4/1"],[["l0","l0","l0","p3"],"2/1"],
+> [["l0","l0","l3","p0"],"6/1"],[["l0","l0","p0","p3"],"-6/1"],[["l0","l1","l1","l3"],"4/1"],[["l0","l1","l
+> 1","p3"],"-2/1"],[["l0","l1","l3","p1"],"-4/1"],[["l0","l1","p1","p3"],"6/1"],[["l0","l2","l2","l3"],"-4/
+> 1"],[["l0","l2","l2","p3"],"2/1"],[["l0","l2","l3","p2"],"4/1"],[["l0","l2","p2","p3"],"-6/1"],[["l0","l3
+> ","l3","l3"],"4/1"],[["l0","l3","l3","p3"],"-6/1"],[["l0","l3","p0","p0"],"-4/1"],[["l0","l3","p1","p1"],
+> "-2/1"],[["l0","l3","p2","p2"],"2/1"],[["l0","l3","p3","p3"],"4/1"],[["l0","p0","p0","p3"],"5/1"],[["l0",
+> "p1","p1","p3"],"-1/1"],[["l0","p2","p2","p3"],"1/1"],[["l0","p3","p3","p3"],"-1/1"],[["l1","l1","l3","p0
+> "],"-2/1"],[["l1","l3","p0","p1"],"6/1"],[["l1","p0","p1","p3"],"-4/1"],[["l2","l2","l3","p0"],"2/1"],[["
+> l2","l2","p0","p3"],"4/1"],[["l2","l3","p0","p2"],"-6/1"],[["l3","l3","l3","p0"],"-2/1"],[["l3","l3","p0"
+> ,"p3"],"6/1"],[["l3","p0","p0","p0"],"1/1"],[["l3","p0","p1","p1"],"-1/1"],[["l3","p0","p2","p2"],"1/1"],
+> [["l3","p0","p3","p3"],"-5/1"],[["p0","p0","p0","p3"],"-4/1"],[["p0","p1","p1","p3"],"4/1"],[["p0","p2","
+> p2","p3"],"-4/1"],[["p0","p3","p3","p3"],"4/1"]],8,3":[[["l0","l0","l0","l2"],"-4/1"],[["l0","l0","l0","
+> p2"],"2/1"],[["l0","l0","l2","p0"],"6/1"],[["l0","l0","p0","p2"],"-6/1"],[["l0","l1","l1","l2"],"4/1"],[[
+> "l0","l1","l1","p2"],"-2/1"],[["l0","l1","l2","p1"],"-4/1"],[["l0","l1","p1","p2"],"6/1"],[["l0","l2","l2
+> ","l2"],"4/1"],[["l0","l2","l2","p2"],"-6/1"],[["l0","l2","l3","l3"],"-4/1"],[["l0","l2","l3","p3"],"4/1"
+> ],[["l0","l2","p0","p0"],"-4/1"],[["l0","l2","p1","p1"],"-2/1"],[["l0","l2","p2","p2"],"4/1"],[["l0","l2"
+> ,"p3","p3"],"2/1"],[["l0","l3","l3","p2"],"2/1"],[["l0","l3","p2","p3"],"-6/1"],[["l0","p0","p0","p2"],"5
+> /1"],[["l0","p1","p1","p2"],"-1/1"],[["l0","p2","p2","p2"],"-1/1"],[["l0","p2","p3","p3"],"1/1"],[["l1","
+> l1","l2","p0"],"-2/1"],[["l1","l2","p0","p1"],"6/1"],[["l1","p0","p1","p2"],"-4/1"],[["l2","l2","l2","p0"
+> "],"-2/1"],[["l2","l2","p0","p2"],"6/1"],[["l2","l3","l3","p0"],"2/1"],[["l2","l3","p0","p3"],"-6/1"],[["l
+> 2","p0","p0","p0"],"1/1"],[["l2","p0","p1","p1"],"-1/1"],[["l2","p0","p2","p2"],"-5/1"],[["l2","p0","p3",
+> "p3"],"1/1"],[["l3","l3","p0","p2"],"4/1"],[["p0","p0","p0","p2"],"-4/1"],[["p0","p1","p1","p2"],"4/1"],[
+> ["p0","p2","p2","p2"],"4/1"],[["p0","p2","p3","p3"],"-4/1"]],8,4":[[["l0","l0","l2","l3"],"-2/1"],[["l0"
+> ,"l0","l2","p3"],"1/1"],[["l0","l0","l3","p2"],"1/1"],[["l0","l0","p2","p3"],"1/1"],[["l0","l2","l3","p0"
+> "],"2/1"],[["l0","l2","p0","p3"],"-5/1"],[["l0","l3","p0","p2"],"-5/1"],[["l0","p0","p2","p3"],"3/1"],[["l
+> 1","l1","l2","l3"],"6/1"],[["l1","l1","l2","p3"],"-3/1"],[["l1","l1","l3","p2"],"-3/1"],[["l1","l1","p2",
+> "p3"],"7/1"],[["l1","l2","l3","p1"],"-6/1"],[["l1","l2","p1","p3"],"1/1"],[["l1","l3","p1","p2"],"1/1"],[
+> ["l1","p1","p2","p3"],"-5/1"],[["l2","l2","l2","l3"],"2/1"],[["l2","l2","l2","p3"],"-1/1"],[["l2","l2","l
+> 3","p2"],"-3/1"],[["l2","l2","p2","p3"],"4/1"],[["l2","l3","l3","l3"],"2/1"],[["l2","l3","l3","p3"],"-3/1
+> "],[["l2","l3","p0","p0"],"4/1"],[["l2","l3","p1","p1"],"4/1"],[["l2","l3","p2","p2"],"1/1"],[["l2","l3",
+> "p3","p3"],"1/1"],[["l2","p1","p1","p3"],"-1/1"],[["l2","p2","p2","p3"],"-3/1"],[["l3","l3","l3","p2"],"-
+> 1/1"],[["l3","l3","p2","p3"],"4/1"],[["l3","p1","p1","p2"],"-1/1"],[["l3","p2","p3","p3"],"-3/1"],[["p1",
+> "p1","p2","p3"],"4/1"]],8,5":[[["l0","l0","l1","l3"],"4/1"],[["l0","l0","l1","p3"],"-2/1"],[["l0","l0","
+> l3","p1"],"-2/1"],[["l0","l1","l3","p0"],"-4/1"],[["l0","l1","p0","p3"],"6/1"],[["l0","l3","p0","p1"],"6/
+> 1"],[["l0","p0","p1","p3"],"-4/1"],[["l1","l1","l1","l3"],"-4/1"],[["l1","l1","l1","p3"],"2/1"],[["l1","l
+> 1","l3","p1"],"6/1"],[["l1","l1","p1","p3"],"-6/1"],[["l1","l2","l2","l3"],"4/1"],[["l1","l2","l2","p3"],
+> "-2/1"],[["l1","l2","l3","p2"],"-4/1"],[["l1","l2","p2","p3"],"6/1"],[["l1","l3","l3","l3"],"-4/1"],[["l1
+> ","l3","l3","p3"],"6/1"],[["l1","l3","p0","p0"],"-2/1"],[["l1","l3","p1","p1"],"-4/1"],[["l1","l3","p2","
+> p2"],"-2/1"],[["l1","l3","p3","p3"],"-4/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1","p1","p1","p3"],"5/1"],
+> [["l1","p2","p2","p3"],"-1/1"],[["l1","p3","p3","p3"],"1/1"],[["l2","l2","l3","p1"],"-2/1"],[["l2","l2","
+> p1","p3"],"-4/1"],[["l2","l3","p1","p2"],"6/1"],[["l3","l3","l3","p1"],"2/1"],[["l3","l3","p1","p3"],"-6/
+> 1"],[["l3","p0","p0","p1"],"-1/1"],[["l3","p1","p1","p1"],"1/1"],[["l3","p1","p2","p2"],"-1/1"],[["l3","p
+> 1","p3","p3"],"5/1"],[["p0","p0","p1","p3"],"4/1"],[["p1","p1","p1","p3"],"-4/1"],[["p1","p2","p2","p3"],
+> "4/1"],[["p1","p3","p3","p3"],"-4/1"]],8,6":[[["l0","l0","l1","l2"],"4/1"],[["l0","l0","l1","p2"],"-2/1"
+> ],[["l0","l0","l2","p1"],"-2/1"],[["l0","l1","l2","p0"],"-4/1"],[["l0","l1","p0","p2"],"6/1"],[["l0","l2"
+> ,"p0","p1"],"6/1"],[["l0","p0","p1","p2"],"-4/1"],[["l1","l1","l1","l2"],"-4/1"],[["l1","l1","l1","p2"],"
+> 2/1"],[["l1","l1","l2","p1"],"6/1"],[["l1","l1","l1","p1","p2"],"-6/1"],[["l1","l2","l2","l2"],"-4/1"],[["l1",
+> "l2","l2","p2"],"6/1"],[["l1","l2","l3","l3"],"4/1"],[["l1","l2","l3","p3"],"-4/1"],[["l1","l2","p0","p0"
+> ],"-2/1"],[["l1","l2","p1","p1"],"-4/1"],[["l1","l2","p2","p2"],"-4/1"],[["l1","l2","p3","p3"],"-2/1"],[[
+> "l1","l3","l3","p2"],"-2/1"],[["l1","l3","p2","p3"],"6/1"],[["l1","p0","p0","p2"],"-1/1"],[["l1","p1","p1
+> ","p2"],"5/1"],[["l1","p2","p2","p2"],"1/1"],[["l1","p2","p3","p3"],"-1/1"],[["l2","l2","l2","p1"],"2/1"]
+> ,[["l2","l2","p1","p2"],"-6/1"],[["l2","l3","l3","p1"],"-2/1"],[["l2","l3","p1","p3"],"6/1"],[["l2","p0",
+> "p0","p1"],"-1/1"],[["l2","p1","p1","p1"],"1/1"],[["l2","p1","p2","p2"],"5/1"],[["l2","p1","p3","p3"],"-1
+> /1"],[["l3","l3","p1","p2"],"-4/1"],[["p0","p0","p1","p2"],"4/1"],[["p1","p1","p1","p2"],"-4/1"],[["p1","
+> p2","p2","p2"],"-4/1"],[["p1","p2","p3","p3"],"4/1"]],8,7":[[["l0","l0","l2","l3"],"2/1"],[["l0","l0","l
+> 2","p3"],"-1/1"],[["l0","l0","l3","p2"],"-1/1"],[["l0","l0","p2","p3"],"1/1"],[["l0","l2","l3","p0"],"-2/
+> 1"],[["l0","l2","p0","p3"],"1/1"],[["l0","l3","p0","p2"],"1/1"],[["l0","p0","p2","p3"],"-1/1"],[["l1","l1
+> ","l2","l3"],"-2/1"],[["l1","l1","l2","p3"],"1/1"],[["l1","l3","p2","p3"],"6/1"],[["l1","p0","p0","p2"],"
+> -1/1"],[["l1","p1","p1","p2"],"5/1"],[["l1","p2","p2","p2"],"1/1"],[["l1","p2","p3","p3"],"-1/1"],[["l2","
+> l2","l2","p1"],"2/1"],[["l2","l2","p1","p2"],"-6/1"],[["l2","l3","l3","p1"],"-2/1"],[["l2","l3","p1","p3"
+> ],"6/1"],[["l2","p0","p0","p1"],"-1/1"],[["l2","p1","p1","p1"],"1/1"],[["l2","p1","p2","p2"],"5/1"],[["l2
+> ","p1","p3","p3"],"-1/1"],[["l3","l3","p1","p2"],"-4/1"],[["p0","p0","p1","p2"],"4/1"],[["p1","p1","p1","
+> p2"],"-4/1"],[["p1","p2","p2","p2"],"-4/1"],[["p1","p2","p3","p3"],"4/1"]],8,8":[[["l0","l0","l0","l0"],"4/1"],[[
+> "l0","l0","l0","p0"],"-8/1"],[["l0","l0","l1","l1"],"-8/1"],[["l0","l0","l1","p1"],"8/1"],[["l0","l0","12
+> ","l2"],"-4/1"],[["l0","l0","l2","p2"],"4/1"],[["l0","l0","l3","l3"],"-4/1"],[["l0","l0","l3","p3"],"4/1"
+> ],[["l0","l0","p0","p0"],"10/1"],[["l0","l0","p1","p1"],"-2/1"],[["l0","l0","p2","p2"],"-2/1"],[["l0","l0
+> ","p3","p3"],"-2/1"],[["l0","l1","l1","p0"],"8/1"],[["l0","l1","p0","p1"],"-16/1"],[["l0","l2","l2","p0"]
+> ,"4/1"],[["l0","l2","p0","p2"],"-4/1"],[["l0","l3","l3","p0"],"4/1"],[["l0","l3","p0","p3"],"-4/1"],[["l0
+> ","p0","p0","p0"],"-6/1"],[["l0","p0","p1","p1"],"6/1"],[["l0","p0","p2","p2"],"2/1"],[["l0","p0","p3","p
```

```
+> 3"],"2/1"],[["l1","l1","l1","l1"],"4/1"],[["l1","l1","l1","p1"],"-8/1"],[["l1","l1","l2","l2"],"4/1"],[["
+> l1","l1","l2","p2"],"-4/1"],[["l1","l1","l3","l3"],"4/1"],[["l1","l1","l3","p3"],"-4/1"],[["l1","l1","p0"
+> ,"p0"],"-2/1"],[["l1","l1","p1","p1"],"10/1"],[["l1","l1","p2","p2"],"2/1"],[["l1","l1","p3","p3"],"2/1"]
+> ,[["l1","l2","l2","p1"],"-4/1"],[["l1","l2","p1","p2"],"4/1"],[["l1","l3","l3","p1"],"-4/1"],[["l1","l3",
+> "p1","p3"],"4/1"],[["l1","p0","p0","p1"],"6/1"],[["l1","p1","p1","p1"],"-6/1"],[["l1","p1","p2","p2"],"-2
+> /1"],[["l1","p1","p3","p3"],"-2/1"],[["l2","l2","l3","l3"],"8/1"],[["l2","l2","l3","p3"],"-8/1"],[["l2","
+> l2","p0","p0"],"-4/1"],[["l2","l2","p1","p1"],"4/1"],[["l2","l3","l3","p2"],"-8/1"],[["l2","l3","p2","p3"
+> ],"16/1"],[["l2","p0","p0","p2"],"4/1"],[["l2","p1","p1","p2"],"-4/1"],[["l2","p2","p3","p3"],"-4/1"],[["
+> l3","l3","p0","p0"],"-4/1"],[["l3","l3","p1","p1"],"4/1"],[["l3","p0","p0","p3"],"4/1"],[["l3","p1","p1",
+> "p3"],"-4/1"],[["l3","p2","p2","p3"],"-4/1"],[["p0","p0","p0","p0"],"4/1"],[["p0","p0","p1","p1"],"-8/1"]
+> ,[["p0","p0","p2","p2"],"-4/1"],[["p0","p0","p3","p3"],"-4/1"],[["p1","p1","p1","p1"],"4/1"],[["p1","p1",
+> "p2","p2"],"4/1"],[["p1","p1","p3","p3"],"4/1"],[["p2","p2","p3","p3"],"8/1"]],"8,9":[[["l0","l0","l2","l
+> 3"],"2/1"],[["l0","l0","l2","p3"],"-1/1"],[["l0","l0","l3","p2"],"-1/1"],[["l0","l0","p2","p3"],"1/1"],[[
+> "l0","l2","l3","p0"],"-2/1"],[["l0","l2","p0","p3"],"1/1"],[["l0","l3","p0","p2"],"1/1"],[["l0","p0","p2"
+> ,"p3"],"-1/1"],[["l1","l1","l2","l3"],"-2/1"],[["l1","l1","l2","p3"],"1/1"],[["l1","l1","l3","p2"],"1/1"]
+> ,[["l1","l1","p2","p3"],"-1/1"],[["l1","l2","l3","p1"],"2/1"],[["l1","l2","p1","p3"],"-1/1"],[["l1","l3",
+> "p1","p2"],"-1/1"],[["l1","p1","p2","p3"],"1/1"],[["l2","l2","l2","l3"],"-2/1"],[["l2","l2","l2","p3"],"1
+> /1"],[["l2","l2","l3","p2"],"3/1"],[["l2","l2","p2","p3"],"-2/1"],[["l2","l3","l3","l3"],"2/1"],[["l2","l
+> 3","l3","p3"],"-3/1"],[["l2","l3","p0","p0"],"2/1"],[["l2","l3","p1","p1"],"-2/1"],[["l2","l3","p2","p2"]
+> ,"-3/1"],[["l2","l3","p3","p3"],"1/1"],[["l2","p0","p0","p3"],"-1/1"],[["l2","p1","p1","p3"],"1/1"],[["l2
+> ","p2","p2","p3"],"2/1"],[["l3","l3","l3","p2"],"-1/1"],[["l3","l3","p2","p3"],"2/1"],[["l3","p0","p0","p
+> 2"],"-1/1"],[["l3","p1","p1","p2"],"1/1"],[["l3","p2","p2","p2"],"1/1"],[["l3","p2","p3","p3"],"-1/1"],[[
+> "p0","p0","p2","p3"],"4/1"],[["p1","p1","p2","p3"],"-4/1"],[["p2","p2","p2","p3"],"-4/1"]],"9,0":[[["l0",
+> "l0","l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l3","l3"],"3/1"],[["l0","l0","l3","p3"
+> ],"-3/1"],[["l0","l0","p1","p1"],"-3/2"],[["l0","l0","p2","p2"],"-3/2"],[["l0","l0","p3","p3"],"2/1"],[["
+> l0","l1","p0","p1"],"-1/1"],[["l0","l2","p0","p2"],"-1/1"],[["l0","l3","l3","p0"],"-3/1"],[["l0","p0","p0
+> ","p0"],"-1/2"],[["l0","p0","p1","p1"],"2/1"],[["l0","p0","p2","p2"],"2/1"],[["l0","p0","p3","p3"],"-1/2"
+> ],[["l1","l1","l1","l1"],"1/2"],[["l1","l1","l1","p1"],"-1/1"],[["l1","l1","l2","l2"],"1/1"],[["l1","l1",
+> "l2","p2"],"-1/1"],[["l1","l1","p1","p1"],"5/2"],[["l1","l1","p2","p2"],"-1/2"],[["l1","l2","l2","p1"],"-
+> 1/1"],[["l1","l2","p1","p2"],"6/1"],[["l1","l3","p1","p3"],"1/1"],[["l1","p0","p0","p1"],"1/2"],[["l1","p
+> 1","p1","p1"],"-2/1"],[["l1","p1","p2","p2"],"-2/1"],[["l1","p1","p3","p3"],"-1/2"],[["l2","l2","l2","l2"
+> ],"1/2"],[["l2","l2","l2","p2"],"-1/1"],[["l2","l2","p1","p1"],"-1/2"],[["l2","l2","p2","p2"],"5/2"],[["l
+> 2","l3","p2","p3"],"1/1"],[["l2","p0","p0","p2"],"1/2"],[["l2","p1","p1","p2"],"-2/1"],[["l2","p2","p2","
+> p2"],"-2/1"],[["l2","p2","p3","p3"],"-1/2"],[["l3","l3","l3","l3"],"-1/2"],[["l3","l3","l3","p3"],"1/1"],
+> [["l3","l3","p0","p0"],"2/1"],[["l3","l3","p1","p1"],"3/2"],[["l3","l3","p2","p2"],"3/2"],[["l3","p0","p0
+> ","p3"],"-1/2"],[["l3","p1","p1","p3"],"-2/1"],[["l3","p2","p2","p3"],"-2/1"],[["l3","p3","p3","p3"],"-1/
+> 2"],[["p0","p0","p3","p3"],"2/1"]],"9,1":[[["l0","l0","l0","l1"],"2/1"],[["l0","l0","l0","p1"],"-1/1"],[[
+> "l0","l0","l1","p0"],"-3/1"],[["l0","l0","p0","p1"],"4/1"],[["l0","l1","l1","l1"],"-2/1"],[["l0","l1","l1
+> ","p1"],"3/1"],[["l0","l1","l2","l2"],"-2/1"],[["l0","l1","l2","p2"],"2/1"],[["l0","l1","l3","l3"],"-6/1"
+> ],[["l0","l1","l3","p3"],"6/1"],[["l0","l1","p0","p0"],"1/1"],[["l0","l1","p1","p1"],"-1/1"],[["l0","l1",
+> "p2","p2"],"4/1"],[["l0","l1","p3","p3"],"-4/1"],[["l0","l2","l2","p1"],"1/1"],[["l0","l2","p1","p2"],"-5
+> /1"],[["l0","l3","l3","p1"],"3/1"],[["l0","l3","p1","p3"],"-1/1"],[["l0","p0","p0","p1"],"-3/1"],[["l0","
+> p1","p3","p3"],"1/1"],[["l1","l1","l1","p0"],"1/1"],[["l1","l1","p0","p1"],"-4/1"],[["l1","l2","l2","p0"]
+> ,"1/1"],[["l1","l2","p0","p2"],"-5/1"],[["l1","l3","l3","p0"],"3/1"],[["l1","l3","p0","p3"],"-1/1"],[["l1
+> ","p0","p1","p1"],"3/1"],[["l1","p0","p3","p3"],"1/1"],[["l2","l2","p0","p1"],"1/1"],[["l2","p0","p1","p2
+> "],"3/1"],[["l3","l3","p0","p1"],"-7/1"],[["l3","p0","p1","p3"],"5/1"],[["p0","p1","p3","p3"],"-4/1"]],"9
+> ,2":[[["l0","l0","l0","l2"],"2/1"],[["l0","l0","l0","p2"],"-1/1"],[["l0","l0","l2","p0"],"-3/1"],[["l0","
+> l0","p0","p2"],"4/1"],[["l0","l1","l1","l2"],"-2/1"],[["l0","l1","l1","p2"],"1/1"],[["l0","l1","l2","p1"]
+> ,"2/1"],[["l0","l1","p1","p2"],"-5/1"],[["l0","l2","l2","l2"],"-2/1"],[["l0","l2","l2","p2"],"3/1"],[["l0
+> ","l2","l3","l3"],"-6/1"],[["l0","l2","l3","p3"],"6/1"],[["l0","l2","p0","p0"],"1/1"],[["l0","l2","p1","p
+> 1"],"4/1"],[["l0","l2","p2","p2"],"-1/1"],[["l0","l2","p3","p3"],"-4/1"],[["l0","l3","l3","p2"],"3/1"],[[
+> "l0","l3","p2","p3"],"-1/1"],[["l0","p0","p0","p2"],"-3/1"],[["l0","p2","p3","p3"],"1/1"],[["l1","l1","l2
+> ","p0"],"1/1"],[["l1","l1","p0","p2"],"1/1"],[["l1","l2","p0","p1"],"-5/1"],[["l1","p0","p1","p2"],"3/1"]
+> ,[["l2","l2","l2","p0"],"1/1"],[["l2","l2","p0","p2"],"-4/1"],[["l2","l3","l3","p0"],"3/1"],[["l2","l3","
+> p0","p3"],"-1/1"],[["l2","p0","p2","p2"],"3/1"],[["l2","p0","p3","p3"],"1/1"],[["l3","l3","p0","p2"],"-7/
+> 1"],[["l3","p0","p2","p3"],"5/1"],[["p0","p2","p3","p3"],"-4/1"]],"9,3":[[["l0","l0","l0","l3"],"-2/1"],[
+> ["l0","l0","l0","p3"],"1/1"],[["l0","l0","l3","p0"],"3/1"],[["l0","l0","p0","p3"],"-2/1"],[["l0","l1","l1
+> ","l3"],"2/1"],[["l0","l1","l1","p3"],"-1/1"],[["l0","l1","l3","p1"],"-2/1"],[["l0","l1","p1","p3"],"1/1"
+> ],[["l0","l2","l2","l3"],"2/1"],[["l0","l2","l2","p3"],"-1/1"],[["l0","l2","l3","p2"],"-2/1"],[["l0","l2"
+> ,"p2","p3"],"1/1"],[["l0","l3","l3","l3"],"-2/1"],[["l0","l3","l3","p3"],"3/1"],[["l0","l3","p0","p0"],"-
+> 3/1"],[["l0","l3","p1","p1"],"2/1"],[["l0","l3","p2","p2"],"2/1"],[["l0","l3","p3","p3"],"-1/1"],[["l0","
+> p0","p0","p3"],"2/1"],[["l0","p1","p1","p3"],"-1/1"],[["l0","p2","p2","p3"],"-1/1"],[["l1","l1","l3","p0"
+> ],"-1/1"],[["l1","l1","p0","p3"],"1/1"],[["l1","l3","p0","p1"],"1/1"],[["l1","p0","p1","p3"],"-1/1"],[["l
+> 2","l2","l3","p0"],"-1/1"],[["l2","l2","p0","p3"],"1/1"],[["l2","l3","p0","p2"],"1/1"],[["l2","p0","p2","
+> p3"],"-1/1"],[["l3","l3","l3","p0"],"1/1"],[["l3","l3","p0","p3"],"-2/1"],[["l3","p0","p0","p0"],"1/1"],[
+> ["l3","p0","p1","p1"],"-1/1"],[["l3","p0","p2","p2"],"-1/1"],[["l3","p0","p3","p3"],"1/1"],[["p0","p0","p
+> 0","p3"],"-4/1"],[["p0","p1","p1","p3"],"4/1"],[["p0","p2","p2","p3"],"4/1"]],"9,4":[[["l0","l0","l0","l0
+> "],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l2","l2"],"1/1"],[["l0","l0","l2","p2"],"-1/1"],[["
+> l0","l0","p0","p0"],"-5/2"],[["l0","l0","p2","p2"],"-1/2"],[["l0","l1","p0","p1"],"1/1"],[["l0","l2","l2"
+> ,"p0"],"-1/1"],[["l0","l2","p0","p2"],"6/1"],[["l0","l3","p0","p3"],"1/1"],[["l0","p0","p0","p0"],"2/1"],
+> [["l0","p0","p1","p1"],"-1/2"],[["l0","p0","p2","p2"],"-2/1"],[["l0","p0","p3","p3"],"-1/2"],[["l1","l1",
+> "l1","l1"],"1/2"],[["l1","l1","l1","p1"],"-1/1"],[["l1","l1","l3","l3"],"3/1"],[["l1","l1","l3","p3"],"-3
+> /1"],[["l1","l1","p0","p0"],"3/2"],[["l1","l1","p2","p2"],"-3/2"],[["l1","l1","p3","p3"],"2/1"],[["l1","l
+> 2","p1","p2"],"-1/1"],[["l1","l3","l3","p1"],"-3/1"],[["l1","p0","p0","p1"],"-2/1"],[["l1","p1","p1","p1"
+> ],"1/2"],[["l1","p1","p2","p2"],"2/1"],[["l1","p1","p3","p3"],"-1/2"],[["l2","l2","l2","l2"],"-1/2"],[["l
+> 2","l2","l2","p2"],"1/1"],[["l2","l2","p0","p0"],"-1/2"],[["l2","l2","p2","p2"],"-5/2"],[["l2","l3","p2",
+> "p3"],"-1/1"],[["l2","p0","p0","p2"],"-2/1"],[["l2","p1","p1","p2"],"1/2"],[["l2","p2","p2","p2"],"2/1"],
+> [["l2","p2","p3","p3"],"1/2"],[["l3","l3","l3","l3"],"1/2"],[["l3","l3","l3","p3"],"-1/1"],[["l3","l3","p
+> 0","p0"],"3/2"],[["l3","l3","p1","p1"],"2/1"],[["l3","l3","p2","p2"],"-3/2"],[["l3","p0","p0","p3"],"-2/1
+> "],[["l3","p1","p1","p3"],"-1/2"],[["l3","p2","p2","p3"],"2/1"],[["l3","p3","p3","p3"],"1/2"],[["p1","p1"
+> ,"p3","p3"],"2/1"]],"9,5":[[["l0","l0","l1","l2"],"-2/1"],[["l0","l0","l1","p2"],"1/1"],[["l0","l0","l2",
```

+> "p1"],"1/1"],[["l0","l0","p1","p2"],"1/1"],[["l0","l1","l2","p0"],"2/1"],[["l0","l1","p0","p2"],"-5/1"],[
+> ["l0","l2","p0","p1"],"-5/1"],[["l0","p0","p1","p2"],"3/1"],[["l1","l1","l1","l2"],"2/1"],[["l1","l1","l1
+> ","p2"],"-1/1"],[["l1","l1","l2","p1"],"-3/1"],[["l1","l1","p1","p2"],"4/1"],[["l1","l2","l2","l2"],"2/1"
+>],[["l1","l2","l2","p2"],"-3/1"],[["l1","l2","l3","l3"],"6/1"],[["l1","l2","l3","p3"],"-6/1"],[["l1","l2"
+> ,"p0","p0"],"4/1"],[["l1","l2","p1","p1"],"1/1"],[["l1","l2","p2","p2"],"1/1"],[["l1","l2","p3","p3"],"4/
+> 1"],[["l1","l3","l3","p2"],"-3/1"],[["l1","l3","p2","p3"],"1/1"],[["l1","p1","p1","p2"],"-3/1"],[["l1","p
+> 2","p3","p3"],"-1/1"],[["l2","l2","l2","p1"],"-1/1"],[["l2","l2","p1","p2"],"4/1"],[["l2","l3","l3","p1"]
+> ,"-3/1"],[["l2","l3","p1","p3"],"1/1"],[["l2","p1","p2","p2"],"-3/1"],[["l2","p1","p3","p3"],"-1/1"],[["l
+> 3","l3","p1","p2"],"7/1"],[["l3","p1","p2","p3"],"-5/1"],[["p1","p2","p3","p3"],"4/1"]],"9,6":[[["l0","l0
+> ","l1","l3"],"2/1"],[["l0","l0","l1","p3"],"-1/1"],[["l0","l0","l3","p1"],"-1/1"],[["l0","l0","p1","p3"],
+> "1/1"],[["l0","l1","l3","p0"],"-2/1"],[["l0","l1","p0","p3"],"1/1"],[["l0","l3","p0","p1"],"1/1"],[["l0",
+> "p0","p1","p3"],"-1/1"],[["l1","l1","l1","l3"],"-2/1"],[["l1","l1","l1","p3"],"1/1"],[["l1","l1","l3","p1
+> "],"3/1"],[["l1","l1","p1","p3"],"-2/1"],[["l1","l2","l2","l3"],"-2/1"],[["l1","l2","l2","p3"],"1/1"],[["
+> l1","l2","l3","p2"],"2/1"],[["l1","l2","p2","p3"],"-1/1"],[["l1","l3","l3","l3"],"2/1"],[["l1","l3","l3",
+> "p3"],"-3/1"],[["l1","l3","p0","p0"],"2/1"],[["l1","l3","p1","p1"],"-3/1"],[["l1","l3","p2","p2"],"-2/1"]
+> ,[["l1","l3","p3","p3"],"1/1"],[["l1","p0","p0","p3"],"-1/1"],[["l1","p1","p1","p3"],"2/1"],[["l1","p2","
+> p2","p3"],"1/1"],[["l2","l2","l3","p1"],"1/1"],[["l2","l2","p1","p3"],"-1/1"],[["l2","l3","p1","p2"],"-1/
+> 1"],[["l2","p1","p2","p3"],"1/1"],[["l3","l3","l3","p1"],"-1/1"],[["l3","l3","p1","p3"],"2/1"],[["l3","p0
+> ","p0","p1"],"-1/1"],[["l3","p1","p1","p1"],"1/1"],[["l3","p1","p2","p2"],"1/1"],[["l3","p1","p3","p3"],"
+> -1/1"],[["p0","p0","p1","p3"],"4/1"],[["p1","p1","p1","p3"],"-4/1"],[["p1","p2","p2","p3"],"-4/1"]],"9,7"
+> :[[["l0","l0","l0","l0"],"-1/2"],[["l0","l0","l0","p0"],"1/1"],[["l0","l0","l1","l1"],"1/1"],[["l0","l0",
+> "l1","p1"],"-1/1"],[["l0","l0","p0","p0"],"-5/2"],[["l0","l0","p1","p1"],"-1/2"],[["l0","l1","l1","p0"],"
+> -1/1"],[["l0","l1","p0","p1"],"6/1"],[["l0","l2","p0","p2"],"1/1"],[["l0","l3","p0","p3"],"1/1"],[["l0","
+> p0","p0","p0"],"2/1"],[["l0","p0","p1","p1"],"-2/1"],[["l0","p0","p2","p2"],"-1/2"],[["l0","p0","p3","p3"
+>],"-1/2"],[["l1","l1","l1","l1"],"-1/2"],[["l1","l1","l1","p1"],"1/1"],[["l1","l1","p0","p0"],"-1/2"],[["
+> l1","l1","p1","p1"],"-5/2"],[["l1","l2","p1","p2"],"-1/1"],[["l1","l3","p1","p3"],"-1/1"],[["l1","p0","p0
+> ","p1"],"-2/1"],[["l1","p1","p1","p1"],"2/1"],[["l1","p1","p2","p2"],"1/2"],[["l1","p1","p3","p3"],"1/2"]
+> ,[["l2","l2","l2","l2"],"1/2"],[["l2","l2","l2","p2"],"-1/1"],[["l2","l2","l3","l3"],"3/1"],[["l2","l2","
+> l3","p3"],"-3/1"],[["l2","l2","p0","p0"],"3/2"],[["l2","l2","p1","p1"],"-3/2"],[["l2","l2","p3","p3"],"2/
+> 1"],[["l2","l3","l3","p2"],"-3/1"],[["l2","p0","p0","p2"],"-2/1"],[["l2","p1","p1","p2"],"2/1"],[["l2","p
+> 2","p2","p2"],"1/2"],[["l2","p2","p3","p3"],"-1/2"],[["l3","l3","l3","l3"],"1/2"],[["l3","l3","l3","p3"],
+> "-1/1"],[["l3","l3","p0","p0"],"3/2"],[["l3","l3","p1","p1"],"-3/2"],[["l3","l3","p2","p2"],"2/1"],[["l3"
+> ,"p0","p0","p3"],"-2/1"],[["l3","p1","p1","p3"],"2/1"],[["l3","p2","p2","p3"],"-1/2"],[["l3","p3","p3","p
+> 3"],"1/2"],[["p2","p2","p3","p3"],"2/1"]],"9,8":[[["l0","l0","l2","l3"],"2/1"],[["l0","l0","l2","p3"],"-1
+> /1"],[["l0","l0","l3","p2"],"-1/1"],[["l0","l0","p2","p3"],"1/1"],[["l0","l2","l3","p0"],"-2/1"],[["l0","
+> l2","p0","p3"],"1/1"],[["l0","l3","p0","p2"],"1/1"],[["l0","p0","p2","p3"],"-1/1"],[["l1","l1","l2","l3"]
+> ,"-2/1"],[["l1","l1","l2","p3"],"1/1"],[["l1","l1","l3","p2"],"1/1"],[["l1","l1","p2","p3"],"-1/1"],[["l1
+> ","l2","l3","p1"],"2/1"],[["l1","l2","p1","p3"],"-1/1"],[["l1","l3","p1","p2"],"-1/1"],[["l1","p1","p2","
+> p3"],"1/1"],[["l2","l2","l2","l3"],"-2/1"],[["l2","l2","l2","p3"],"1/1"],[["l2","l2","l3","p2"],"3/1"],[[
+> "l2","l2","p2","p3"],"-2/1"],[["l2","l3","l3","l3"],"2/1"],[["l2","l3","l3","p3"],"-3/1"],[["l2","l3","p0
+> ","p0"],"2/1"],[["l2","l3","p1","p1"],"-2/1"],[["l2","l3","p2","p2"],"-3/1"],[["l2","l3","p3","p3"],"1/1"
+>],[["l2","p0","p0","p3"],"-1/1"],[["l2","p1","p1","p3"],"1/1"],[["l2","p2","p2","p3"],"2/1"],[["l3","l3",
+> "l3","p2"],"-1/1"],[["l3","l3","p2","p3"],"2/1"],[["l3","p0","p0","p2"],"-1/1"],[["l3","p1","p1","p2"],"1
+> /1"],[["l3","p2","p2","p2"],"1/1"],[["l3","p2","p3","p3"],"-1/1"],[["p0","p0","p2","p3"],"4/1"],[["p1","p
+> 1","p2","p3"],"-4/1"],[["p2","p2","p2","p3"],"-4/1"]],"9,9":[[["l0","l0","l0","l0"],"3/2"],[["l0","l0","l
+> 0","p0"],"-3/1"],[["l0","l0","l1","l1"],"-3/1"],[["l0","l0","l1","p1"],"3/1"],[["l0","l0","l2","l2"],"-3/
+> 1"],[["l0","l0","l2","p2"],"3/1"],[["l0","l0","l3","l3"],"-1/1"],[["l0","l0","l3","p3"],"1/1"],[["l0","l0
+> ","p0","p0"],"5/2"],[["l0","l0","p1","p1"],"-3/2"],[["l0","l0","p2","p2"],"-3/2"],[["l0","l0","p3","p3"],
+> "-1/2"],[["l0","l1","l1","p0"],"3/1"],[["l0","l1","p0","p1"],"-2/1"],[["l0","l2","l2","p0"],"3/1"],[["l0"
+> ,"l2","p0","p2"],"-2/1"],[["l0","l3","l3","p0"],"1/1"],[["l0","p0","p0","p0"],"-1/1"],[["l0","p0","p1","p
+> 1"],"1/1"],[["l0","p0","p2","p2"],"1/1"],[["l1","l1","l1","l1"],"3/2"],[["l1","l1","l1","p1"],"-3/1"],[["
+> l1","l1","l2","l2"],"3/1"],[["l1","l1","l2","p2"],"-3/1"],[["l1","l1","l3","l3"],"1/1"],[["l1","l1","l3",
+> "p3"],"-1/1"],[["l1","l1","p0","p0"],"-3/2"],[["l1","l1","p1","p1"],"5/2"],[["l1","l1","p2","p2"],"3/2"],
+> [["l1","l1","p3","p3"],"1/2"],[["l1","l2","l2","p1"],"-3/1"],[["l1","l2","p1","p2"],"2/1"],[["l1","l3","l
+> 3","p1"],"-1/1"],[["l1","p0","p0","p1"],"1/1"],[["l1","p1","p1","p1"],"-1/1"],[["l1","p1","p2","p2"],"-1/
+> 1"],[["l2","l2","l2","l2"],"3/2"],[["l2","l2","l2","p2"],"-3/1"],[["l2","l2","l3","l3"],"1/1"],[["l2","l2
+> ","l3","p3"],"-1/1"],[["l2","l2","p0","p0"],"-3/2"],[["l2","l2","p1","p1"],"3/2"],[["l2","l2","p2","p2",
+> "5/2"],[["l2","l2","p3","p3"],"1/2"],[["l2","l3","l3","p2"],"-1/1"],[["l2","p0","p0","p2"],"1/1"],[["l2",
+> "p1","p1","p2"],"-1/1"],[["l2","p2","p2","p2"],"-1/1"],[["l3","l3","l3","l3"],"3/2"],[["l3","l3","l3","p3
+> "],"-3/1"],[["l3","l3","p0","p0"],"1/2"],[["l3","l3","p1","p1"],"-1/2"],[["l3","l3","p2","p2"],"-1/2"],[[
+> "l3","l3","p3","p3"],"3/2"],[["l3","p0","p0","p3"],"-1/1"],[["l3","p1","p1","p3"],"1/1"],[["l3","p2","p2"
+> ,"p3"],"1/1"],[["p0","p0","p0","p0"],"2/1"],[["p0","p0","p1","p1"],"-4/1"],[["p0","p0","p2","p2"],"-4/1"]
+> ,[["p1","p1","p1","p1"],"2/1"],[["p1","p1","p2","p2"],"4/1"],[["p2","p2","p2","p2"],"2/1"]]},"numerator_v
+> ariables":"l0..l3, p0..p3 (lower)","prefactor":"kappa**2/2","routing":{"A":["h_a: p","h_c: -l","h_e: l-p"
+>],"B":["h_b: -p","h_d: l","h_f: p-l"],"propagators":["<h_c h_d>(l)","<h_e h_f>(l-p)"]},"sha256":"869a5db7
+> 2f275c25a68b9b0295e3381fdf57b9c71de3c632172d70dce2fabce8"}

73 Symbolic integrand D03 (p14r export; evaluated in p16)

[listing 40/133] exports/P14_integrands/D03.json


```
{"denominators":["(l-p)^2+i0","l^2+i0"],"external":["h_a(p)","h_b(-p)"],"external_components":"independent h_(
  +> mn) index 0..9 = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]; vector
  +>  index 0..3","graph":"D03","integrated":false,"numerator":{"0,0":[[["l0","l0","l0","l0"],"1/1"],[["l0","l
  +> 0","p0"],"-2/1"],[["l0","l0","l1","l1"],"-2/1"],[["l0","l0","l1","p1"],"2/1"],[["l0","l0","l2","l2"]
  +> ,"-2/1"],[["l0","l0","l2","p2"],"2/1"],[["l0","l0","l3","l3"],"-2/1"],[["l0","l0","l3","p3"],"2/1"],[["l0
  +> ","l0","p0","p0"],"5/4"],[["l0","l0","p1","p1"],"-1/1"],[["l0","l0","p2","p2"],"-1/1"],[["l0","l0","p3","
  +> p3"],"-1/1"],[["l0","l1","l1","p0"],"2/1"],[["l0","l1","p0","p1"],"-3/2"],[["l0","l2","l2","p0"],"2/1"],[
```

```
+> ["l0","l2","p0","p2"],"-3/2"],[["l0","l3","l3","p0"],"2/1"],[["l0","l3","p0","p3"],"-3/2"],[["l0","p0","p
+> 0","p0"],"-1/4"],[["l0","p0","p1","p1"],"3/4"],[["l0","p0","p2","p2"],"3/4"],[["l0","p0","p3","p3"],"3/4"
+> ],[["l1","l1","l1","l1"],"1/1"],[["l1","l1","l1","p1"],"-2/1"],[["l1","l1","l2","l2"],"2/1"],[["l1","l1",
+> "l2","p2"],"-2/1"],[["l1","l1","l3","l3"],"2/1"],[["l1","l1","l3","p3"],"-2/1"],[["l1","l1","p0","p0"],"-
+> 1/2"],[["l1","l1","p1","p1"],"3/4"],[["l1","l2","l2","p1"],"-2/1"],[["l1","l2","p1","p2"],"3/2"],[["l1","
+> l3","l3","p1"],"-2/1"],[["l1","l3","p1","p3"],"3/2"],[["l1","p0","p0","p1"],"1/4"],[["l1","p1","p1","p1"]
+> ,"1/4"],[["l1","p1","p2","p2"],"1/4"],[["l1","p1","p3","p3"],"1/4"],[["l2","l2","l2","l2"],"1/1"],[["l2",
+> "l2","l2","p2"],"-2/1"],[["l2","l2","l3","l3"],"2/1"],[["l2","l2","l3","p3"],"-2/1"],[["l2","l2","p0","p0
+> "],"-1/2"],[["l2","l2","p2","p2"],"3/4"],[["l2","l3","l3","p2"],"-2/1"],[["l2","l3","p2","p3"],"3/2"],[["
+> l2","p0","p0","p2"],"1/4"],[["l2","p1","p1","p2"],"1/4"],[["l2","p2","p2","p2"],"1/4"],[["l2","p2","p3","
+> p3"],"1/4"],[["l3","l3","l3","l3"],"1/1"],[["l3","l3","l3","p3"],"-2/1"],[["l3","l3","p0","p0"],"-1/2"],[
+> ["l3","l3","p3","p3"],"3/4"],[["l3","p0","p0","p3"],"1/4"],[["l3","p1","p1","p3"],"1/4"],[["l3","p2","p2"
+> ,"p3"],"1/4"],[["l3","p3","p3","p3"],"1/4"]],"0,1":[[["l0","l0","l0","p1"],"3/2"],[["l0","l0","l1","p0"],
+> "-1/2"],[["l0","l0","p0","p1"],"-3/2"],[["l0","l1","l1","p1"],"-3/2"],[["l0","l1","p0","p0"],"1/2"],[["l0
+> ","l1","p1","p1"],"5/2"],[["l0","l1","p2","p2"],"1/1"],[["l0","l1","p3","p3"],"1/1"],[["l0","l2","l2","p1
+> "],"-3/2"],[["l0","l2","p1","p2"],"3/2"],[["l0","l3","l3","p1"],"-3/2"],[["l0","l3","p1","p3"],"3/2"],[["
+> l0","p1","p1","p1"],"-1/1"],[["l0","p1","p2","p2"],"-1/1"],[["l0","p1","p3","p3"],"-1/1"],[["l1","l1","l1
+> ","p0"],"1/2"],[["l1","l1","p0","p1"],"1/2"],[["l1","l2","l2","p0"],"1/2"],[["l1","l2","p0","p2"],"-1/2"]
+> ,[["l1","l3","l3","p0"],"1/2"],[["l1","l3","p0","p3"],"-1/2"],[["l1","p0","p1","p1"],"-1/1"],[["l2","l2",
+> "p0","p1"],"1/1"],[["l2","p0","p1","p2"],"-1/1"],[["l3","l3","p0","p1"],"1/1"],[["l3","p0","p1","p3"],"-1
+> /1"]],"0,2":[[["l0","l0","l0","p2"],"3/2"],[["l0","l0","l2","p0"],"-1/2"],[["l0","l0","p0","p2"],"-3/2"],
+> [["l0","l1","l1","p2"],"-3/2"],[["l0","l1","p1","p2"],"3/2"],[["l0","l2","l2","p2"],"-3/2"],[["l0","l2","
+> p0","p0"],"1/2"],[["l0","l2","p1","p1"],"1/1"],[["l0","l2","p2","p2"],"5/2"],[["l0","l2","p3","p3"],"1/1"
+> ],[["l0","l3","l3","p2"],"-3/2"],[["l0","l3","p2","p3"],"3/2"],[["l0","p1","p1","p2"],"-1/1"],[["l0","p2"
+> ,"p2","p2"],"-1/1"],[["l0","p2","p3","p3"],"-1/1"],[["l1","l1","l2","p0"],"1/2"],[["l1","l1","p0","p2"],"
+> 1/1"],[["l1","l2","p0","p1"],"-1/2"],[["l1","p0","p1","p2"],"-1/1"],[["l2","l2","l2","p0"],"1/2"],[["l2",
+> "l2","p0","p2"],"1/2"],[["l2","l3","l3","p0"],"1/2"],[["l2","l3","p0","p3"],"-1/2"],[["l2","p0","p2","p2"
+> ],"-1/1"],[["l3","l3","p0","p2"],"1/1"],[["l3","p0","p2","p3"],"-1/1"]],"0,3":[[["l0","l0","l0","p3"],"3/
+> 2"],[["l0","l0","l3","p0"],"-1/2"],[["l0","l0","p0","p3"],"-3/2"],[["l0","l1","l1","p3"],"-3/2"],[["l0","
+> l1","p1","p3"],"3/2"],[["l0","l2","l2","p3"],"-3/2"],[["l0","l2","p2","p3"],"3/2"],[["l0","l3","l3","p3"]
+> ,"-3/2"],[["l0","l3","p0","p0"],"1/2"],[["l0","l3","p1","p1"],"1/1"],[["l0","l3","p2","p2"],"1/1"],[["l0"
+> ,"l3","p3","p3"],"5/2"],[["l0","p1","p1","p3"],"-1/1"],[["l0","p2","p2","p3"],"-1/1"],[["l0","p3","p3","p
+> 3"],"-1/1"],[["l1","l1","l3","p0"],"1/2"],[["l1","l1","p0","p3"],"1/1"],[["l1","l3","p0","p1"],"-1/2"],[[
+> "l1","p0","p1","p3"],"-1/1"],[["l2","l2","l3","p0"],"1/2"],[["l2","l2","p0","p3"],"1/1"],[["l2","l3","p0"
+> ,"p2"],"-1/2"],[["l2","p0","p2","p3"],"-1/1"],[["l3","l3","l3","p0"],"1/2"],[["l3","l3","p0","p3"],"1/2"
+> ,[["l3","p0","p3","p3"],"-1/1"]],"0,4":[[["l0","l0","l0","p0"],"-1/2"],[["l0","l0","l1","p1"],"-1/2"],[["
+> l0","l0","p0","p0"],"3/4"],[["l0","l0","p1","p1"],"1/2"],[["l0","l0","p2","p2"],"-1/2"],[["l0","l0","p3",
+> "p3"],"-1/2"],[["l0","l1","l1","p0"],"1/2"],[["l0","l1","p0","p1"],"-3/2"],[["l0","l2","l2","p0"],"1/2"],
+> [["l0","l3","l3","p0"],"1/2"],[["l0","p0","p0","p0"],"-1/4"],[["l0","p0","p1","p1"],"3/4"],[["l0","p0","p
+> 2","p2"],"1/4"],[["l0","p0","p3","p3"],"1/4"],[["l1","l1","l1","p1"],"1/2"],[["l1","l1","p1","p1"],"-3/4"
+> ],[["l1","l1","p2","p2"],"1/2"],[["l1","l1","p3","p3"],"1/2"],[["l1","l2","l2","p1"],"1/2"],[["l1","l2","
+> p1","p2"],"-1/1"],[["l1","l3","l3","p1"],"1/2"],[["l1","l3","p1","p3"],"-1/1"],[["l1","p0","p0","p1"],"1/
+> 4"],[["l1","p1","p1","p1"],"1/4"],[["l1","p1","p2","p2"],"-1/4"],[["l1","p1","p3","p3"],"-1/4"],[["l2","l
+> 2","p0","p0"],"-1/2"],[["l2","l2","p2","p2"],"1/4"],[["l2","l3","p2","p3"],"1/2"],[["l2","p0","p0","p2",
+> "1/4"],[["l2","p1","p1","p2"],"1/4"],[["l2","p2","p2","p2"],"-1/4"],[["l2","p2","p3","p3"],"-1/4"],[["l3
+> ","l3","p0","p0"],"-1/2"],[["l3","l3","p3","p3"],"1/4"],[["l3","p0","p0","p3"],"1/4"],[["l3","p1","p1","p3
+> "],"1/4"],[["l3","p2","p2","p3"],"-1/4"],[["l3","p3","p3","p3"],"-1/4"]],"0,5":[[["l0","l0","l1","p2"],"-
+> 1/2"],[["l0","l0","l2","p1"],"-1/2"],[["l0","l0","p1","p2"],"2/1"],[["l0","l1","p0","p2"],"-3/2"],[["l0",
+> "l2","p0","p1"],"-3/2"],[["l0","p0","p1","p2"],"1/1"],[["l1","l1","l1","p2"],"1/2"],[["l1","l1","l2","p1"
+> ],"1/2"],[["l1","l1","p1","p2"],"-3/2"],[["l1","l2","l2","p2"],"1/2"],[["l1","l2","p0","p0"],"1/1"],[["l1
+> ","l2","p1","p1"],"-1/2"],[["l1","l2","p2","p2"],"-1/2"],[["l1","l2","p3","p3"],"1/1"],[["l1","l3","l3","
+> p2"],"1/2"],[["l1","l3","p2","p3"],"-3/2"],[["l1","p1","p1","p2"],"1/1"],[["l2","l2","l2","p1"],"1/2"],[[
+> "l2","l2","p1","p2"],"-3/2"],[["l2","l3","l3","p1"],"1/2"],[["l2","l3","p1","p3"],"-3/2"],[["l2","p1","p2
+> ","p2"],"1/1"],[["l3","p1","p2","p3"],"1/1"]],"0,6":[[["l0","l0","l1","p3"],"-1/2"],[["l0","l0","l3","p1"
+> ],"-1/2"],[["l0","l0","p1","p3"],"2/1"],[["l0","l1","p0","p3"],"-3/2"],[["l0","l3","p0","p1"],"-3/2"],[["
+> l0","p0","p1","p3"],"1/1"],[["l1","l1","l1","p3"],"1/2"],[["l1","l1","l3","p1"],"1/2"],[["l1","l1","p1","
+> p3"],"-3/2"],[["l1","l2","l2","p3"],"1/2"],[["l1","l2","p2","p3"],"-3/2"],[["l1","l3","l3","p3"],"1/2"],[
+> ["l1","l3","p0","p0"],"1/1"],[["l1","l3","p1","p1"],"-1/2"],[["l1","l3","p2","p2"],"1/1"],[["l1","l3","p3
+> ","p3"],"-1/2"],[["l1","p1","p1","p3"],"1/1"],[["l2","l2","l3","p1"],"1/2"],[["l2","l3","p1","p2"],"-3/2"
+> ],[["l2","p1","p2","p3"],"1/1"],[["l3","l3","l3","p1"],"1/2"],[["l3","l3","p1","p3"],"-3/2"],[["l3","p1",
+> "p3","p3"],"1/1"]],"0,7":[[["l0","l0","l0","p0"],"-1/2"],[["l0","l0","l2","p2"],"-1/2"],[["l0","l0","p0",
+> "p0"],"3/4"],[["l0","l0","p1","p1"],"-1/2"],[["l0","l0","p2","p2"],"1/2"],[["l0","l0","p3","p3"],"-1/2"],
+> [["l0","l1","l1","p0"],"1/2"],[["l0","l2","l2","p0"],"1/2"],[["l0","l2","p0","p2"],"-3/2"],[["l0","l3","l
+> 3","p0"],"1/2"],[["l0","p0","p0","p0"],"-1/4"],[["l0","p0","p1","p1"],"1/4"],[["l0","p0","p2","p2"],"3/4"
+> ],[["l0","p0","p3","p3"],"1/4"],[["l1","l1","l2","p2"],"1/2"],[["l1","l1","p0","p0"],"-1/2"],[["l1","l1",
+> "p1","p1"],"1/4"],[["l1","l2","p1","p2"],"-1/1"],[["l1","l3","p1","p3"],"1/2"],[["l1","p0","p0","p1"],"1/
+> 4"],[["l1","p1","p1","p1"],"-1/4"],[["l1","p1","p2","p2"],"1/4"],[["l1","p1","p3","p3"],"-1/4"],[["l2","l
+> 2","l2","p2"],"1/2"],[["l2","l2","p1","p1"],"1/2"],[["l2","l2","p2","p2"],"-3/4"],[["l2","l2","p3","p3"],
+> "1/2"],[["l2","l3","l3","p2"],"1/2"],[["l2","l3","p2","p3"],"-1/1"],[["l2","p0","p0","p2"],"1/4"],[["l2",
+> "p1","p1","p2"],"-1/4"],[["l2","p2","p2","p2"],"1/4"],[["l2","p2","p3","p3"],"-1/4"],[["l3","l3","p0","p0
+> "],"-1/2"],[["l3","l3","p3","p3"],"1/4"],[["l3","p0","p0","p3"],"1/4"],[["l3","p1","p1","p3"],"-1/4"],[["
+> l3","p2","p2","p3"],"1/4"],[["l3","p3","p3","p3"],"-1/4"]],"0,8":[[["l0","l0","l2","p3"],"-1/2"],[["l0","
+> l0","l3","p2"],"-1/2"],[["l0","l0","p2","p3"],"2/1"],[["l0","l2","p0","p3"],"-3/2"],[["l0","l3","p0","p2"
+> ],"-3/2"],[["l0","p0","p2","p3"],"1/1"],[["l1","l1","l2","p3"],"1/2"],[["l1","l1","l3","p2"],"1/2"],[["l1
+> ","l2","p1","p3"],"-3/2"],[["l1","l3","p1","p2"],"-3/2"],[["l1","p1","p2","p3"],"1/1"],[["l2","l2","l2","
+> p3"],"1/2"],[["l2","l2","l3","p2"],"1/2"],[["l2","l2","p2","p3"],"-3/2"],[["l2","l3","l3","p3"],"1/2"],[[
+> "l2","l3","p0","p0"],"1/1"],[["l2","l3","p1","p1"],"1/1"],[["l2","l3","p2","p2"],"-1/2"],[["l2","l3","p3"
+> ,"p3"],"-1/2"],[["l2","p2","p2","p3"],"1/1"],[["l3","l3","l3","p2"],"1/2"],[["l3","l3","p2","p3"],"-3/2"
+> ,[["l3","p2","p3","p3"],"1/1"]],"0,9":[[["l0","l0","l0","p0"],"-1/2"],[["l0","l0","l3","p3"],"-1/2"],[["l
+> 0","l0","p0","p0"],"3/4"],[["l0","l0","p1","p1"],"-1/2"],[["l0","l0","p2","p2"],"-1/2"],[["l0","l0","p3",
+> "p3"],"1/2"],[["l0","l1","l1","p0"],"1/2"],[["l0","l2","l2","p0"],"1/2"],[["l0","l3","l3","p0"],"1/2"],[[
```

```
+> "l0","l3","p0","p3"],"-3/2"],[["l0","p0","p0","p0"],"-1/4"],[["l0","p0","p1","p1"],"1/4"],[["l0","p0","p2
+> ","p2"],"1/4"],[["l0","p0","p3","p3"],"3/4"],[["l1","l1","l3","p3"],"1/2"],[["l1","l1","p0","p0"],"-1/2"]
+> ,[["l1","l1","p1","p1"],"1/4"],[["l1","l2","p1","p2"],"1/2"],[["l1","l3","p1","p3"],"-1/1"],[["l1","p0","
+> p0","p1"],"1/4"],[["l1","p1","p1","p1"],"-1/4"],[["l1","p1","p2","p2"],"-1/4"],[["l1","p1","p3","p3"],"1/
+> 4"],[["l2","l2","l3","p3"],"1/2"],[["l2","l2","p0","p0"],"-1/2"],[["l2","l2","p2","p2"],"1/4"],[["l2","l3
+> ","p2","p3"],"-1/1"],[["l2","p0","p0","p2"],"1/4"],[["l2","p1","p1","p2"],"-1/4"],[["l2","p2","p2","p2"],
+> "-1/4"],[["l2","p2","p3","p3"],"1/4"],[["l3","l3","l3","p3"],"1/2"],[["l3","l3","p1","p1"],"1/2"],[["l3",
+> "l3","p2","p2"],"1/2"],[["l3","l3","p3","p3"],"-3/4"],[["l3","p0","p0","p3"],"1/4"],[["l3","p1","p1","p3"
+> ],"-1/4"],[["l3","p2","p2","p3"],"-1/4"],[["l3","p3","p3","p3"],"1/4"]],"1,0":[[["l0","l0","l0","p1"],"-3
+> /2"],[["l0","l0","l1","p0"],"1/2"],[["l0","l0","p0","p1"],"5/2"],[["l0","l1","l1","p1"],"3/2"],[["l0","l1
+> ","p0","p0"],"-1/2"],[["l0","l1","p1","p1"],"-1/2"],[["l0","l1","p2","p2"],"1/1"],[["l0","l1","p3","p3"],
+> "1/1"],[["l0","l2","l2","p1"],"3/2"],[["l0","l2","p1","p2"],"-3/2"],[["l0","l3","l3","p1"],"3/2"],[["l0",
+> "l3","p1","p3"],"-3/2"],[["l0","p0","p0","p1"],"-1/1"],[["l1","l1","l1","p0"],"-1/2"],[["l1","l1","p0","p
+> 1"],"1/2"],[["l1","l2","l2","p0"],"-1/2"],[["l1","l2","p0","p2"],"1/2"],[["l1","l3","l3","p0"],"-1/2"],[[
+> "l1","l3","p0","p3"],"1/2"],[["l1","p0","p1","p1"],"-1/1"],[["l1","p0","p2","p2"],"-1/1"],[["l1","p0","p3
+> ","p3"],"-1/1"]],"1,1":[[["l0","l0","l0","l0"],"-2/1"],[["l0","l0","l0","p0"],"4/1"],[["l0","l0","l1","l1
+> "],"4/1"],[["l0","l0","l1","p1"],"-4/1"],[["l0","l0","l2","l2"],"4/1"],[["l0","l0","l2","p2"],"-4/1"],[["
+> l0","l0","l3","l3"],"4/1"],[["l0","l0","l3","p3"],"-4/1"],[["l0","l0","p0","p0"],"-3/1"],[["l0","l0","p2"
+> ,"p2"],"2/1"],[["l0","l0","p3","p3"],"2/1"],[["l0","l1","l1","p0"],"-4/1"],[["l0","l1","p0","p1"],"2/1"],
+> [["l0","l2","l2","p0"],"-4/1"],[["l0","l2","p0","p2"],"2/1"],[["l0","l3","l3","p0"],"-4/1"],[["l0","l3","
+> p0","p3"],"2/1"],[["l0","p0","p0","p0"],"1/1"],[["l0","p0","p1","p1"],"1/1"],[["l0","p0","p2","p2"],"-1/1
+> "],[["l0","p0","p3","p3"],"-1/1"],[["l1","l1","l1","l1"],"-2/1"],[["l1","l1","l1","p1"],"4/1"],[["l1","l1
+> ","l2","l2"],"-4/1"],[["l1","l1","l2","p2"],"4/1"],[["l1","l1","l3","l3"],"-4/1"],[["l1","l1","l3","p3"],
+> "4/1"],[["l1","l1","p1","p1"],"-3/1"],[["l1","l1","p2","p2"],"-2/1"],[["l1","l1","p3","p3"],"-2/1"],[["l1
+> ","l2","l2","p1"],"4/1"],[["l1","l2","p1","p2"],"-2/1"],[["l1","l3","l3","p1"],"4/1"],[["l1","l3","p1","p
+> 3"],"-2/1"],[["l1","p0","p0","p1"],"1/1"],[["l1","p1","p1","p1"],"1/1"],[["l1","p1","p2","p2"],"1/1"],[["
+> l1","p1","p3","p3"],"1/1"],[["l2","l2","l2","l2"],"-2/1"],[["l2","l2","l2","p2"],"4/1"],[["l2","l2","l3",
+> "l3"],"-4/1"],[["l2","l2","l3","p3"],"4/1"],[["l2","l2","p0","p0"],"1/1"],[["l2","l2","p1","p1"],"-1/1"],
+> [["l2","l2","p2","p2"],"-2/1"],[["l2","l3","l3","p2"],"4/1"],[["l2","l3","p2","p3"],"-4/1"],[["l3","l3","
+> l3","l3"],"-2/1"],[["l3","l3","l3","p3"],"4/1"],[["l3","l3","p0","p0"],"1/1"],[["l3","l3","p1","p1"],"-1/
+> 1"],[["l3","l3","p3","p3"],"-2/1"]],"1,2":[[["l0","l0","l1","p2"],"-1/1"],[["l0","l0","l2","p1"],"1/1"],[
+> ["l0","l0","p1","p2"],"-2/1"],[["l0","l1","p0","p2"],"1/1"],[["l0","l2","p0","p1"],"-1/1"],[["l0","p0","p
+> 1","p2"],"2/1"],[["l1","l1","l1","p2"],"1/1"],[["l1","l1","l2","p1"],"-1/1"],[["l1","l1","p1","p2"],"-1/1
+> "],[["l1","l2","l2","p2"],"1/1"],[["l1","l2","p0","p0"],"-1/1"],[["l1","l2","p2","p2"],"-2/1"],[["l1","l2
+> ","p3","p3"],"-2/1"],[["l1","l3","l3","p2"],"1/1"],[["l1","p0","p0","p2"],"1/1"],[["l1","p1","p1","p2"],"
+> 1/1"],[["l1","p2","p2","p2"],"1/1"],[["l1","p2","p3","p3"],"1/1"],[["l2","l2","l2","p1"],"-1/1"],[["l2","
+> l2","p1","p2"],"1/1"],[["l2","l3","l3","p1"],"-1/1"],[["l2","l3","l3","p1","p3"],"1/1"]],"1,4":[[["l0","l
+> 0","l0","p1"],"-1/2"],[["l0","l0","l1","p0"],"3/2"],[["l0","l0","p0","p1"],"1/2"],[["l0","l1","l1","p1"],
+> "1/2"],[["l0","l1","p0","p0"],"-1/2"],[["l0","l1","p1","p1"],"-1/2"],[["l0","l1","p2","p2"],"-1/1"],[["l0
+> ","l1","p3","p3"],"-1/1"],[["l0","l2","l2","p1"],"1/2"],[["l0","l2","p1","p2"],"-1/2"],[["l0","l3","l3","
+> p1"],"1/2"],[["l0","l3","p1","p3"],"-1/2"],[["l0","p0","p0","p1"],"-1/1"],[["l0","p1","p2","p2"],"1/1"],[
+> ["l0","p1","p3","p3"],"1/1"],[["l1","l1","l1","p0"],"-3/2"],[["l1","l1","p0","p1"],"5/2"],[["l1","l2","l2
+> ","p0"],"-3/2"],[["l1","l2","p0","p2"],"3/2"],[["l1","l3","l3","p0"],"-3/2"],[["l1","l3","p0","p3"],"3/2"
+> ],[["l1","p0","p1","p1"],"-1/1"]],"1,5":[[["l0","l0","l0","p2"],"-1/1"],[["l0","l0","l2","p0"],"1/1"],[["
+> l0","l0","p0","p2"],"1/1"],[["l0","l1","l1","p2"],"1/1"],[["l0","l1","p1","p2"],"-1/1"],[["l0","l2","l2",
+> "p2"],"1/1"],[["l0","l2","p1","p1"],"1/1"],[["l0","l2","p2","p2"],"-2/1"],[["l0","l2","p3","p3"],"-2/1"],
+> [["l0","l3","l3","p2"],"1/1"],[["l0","p0","p0","p2"],"-1/1"],[["l0","p1","p1","p2"],"-1/1"],[["l0","p2","
+> p2","p2"],"1/1"],[["l0","p2","p3","p3"],"1/1"],[["l1","l1","l2","p0"],"-1/1"],[["l1","l1","p0","p2"],"2/1
+> "],[["l1","l2","p0","p1"],"1/1"],[["l1","p0","p1","p2"],"-2/1"],[["l2","l2","l2","p0"],"-1/1"],[["l2","l2
+> ","p0","p2"],"1/1"],[["l2","l3","l3","p0"],"-1/1"],[["l2","l3","p0","p3"],"2/1"],[["l3","l3","p0","p2","
+> -1/1"]],"1,6":[[["l0","l0","l0","p3"],"-1/1"],[["l0","l0","l3","p0"],"1/1"],[["l0","l0","p0","p3"],"1/1"]
+> ,[["l0","l1","l1","p3"],"1/1"],[["l0","l1","p1","p3"],"-1/1"],[["l0","l2","l2","p3"],"1/1"],[["l0","l3","
+> l3","p3"],"1/1"],[["l0","l3","p1","p1"],"1/1"],[["l0","l3","p2","p2"],"-2/1"],[["l0","l3","p3","p3"],"-2/
+> 1"],[["l0","p0","p0","p3"],"-1/1"],[["l0","p1","p1","p3"],"-1/1"],[["l0","p2","p2","p3"],"1/1"],[["l0","p
+> 3","p3","p3"],"1/1"],[["l1","l1","l3","p0"],"-1/1"],[["l1","l1","p0","p3"],"2/1"],[["l1","l3","p0","p1"],
+> "1/1"],[["l1","p0","p1","p3"],"-2/1"],[["l2","l2","l3","p0"],"-1/1"],[["l2","l2","p0","p3"],"-1/1"],[["l2
+> ","l3","p0","p2"],"2/1"],[["l3","l3","l3","p0"],"-1/1"],[["l3","l3","p0","p3"],"1/1"]],"1,7":[[["l0","l0"
+> ,"l0","p1"],"1/2"],[["l0","l0","l1","p0"],"1/2"],[["l0","l0","p0","p1"],"-1/2"],[["l0","l1","l1","p1"],"-
+> 1/2"],[["l0","l1","p0","p0"],"-1/2"],[["l0","l1","p1","p1"],"1/2"],[["l0","l1","p2","p2"],"-1/1"],[["l0",
+> "l1","p3","p3"],"1/1"],[["l0","l2","l2","p1"],"-1/2"],[["l0","l2","p1","p2"],"5/2"],[["l0","l3","l3","p1"
+> ],"-1/2"],[["l0","l3","p1","p3"],"-1/2"],[["l0","p1","p2","p2"],"-1/1"],[["l1","l1","l1","p0"],"-1/2"],[[
+> "l1","l1","p0","p1"],"1/2"],[["l1","l2","l2","p0"],"-1/2"],[["l1","l2","p0","p2"],"5/2"],[["l1","l3","l3
+> ",p0"],"-1/2"],[["l1","l3","p0","p3"],"-1/2"],[["l1","p0","p2","p2"],"-1/1"],[["l2","l2","p0","p1"],"-1/1
+> "],[["l3","l3","p0","p1"],"1/1"]],"1,8":[[["l0","l1","p2","p3"],"-4/1"],[["l0","l2","p1","p3"],"3/1"],[["
+> l0","l3","p1","p2"],"3/1"],[["l0","p1","p2","p3"],"-2/1"],[["l1","l2","p0","p3"],"3/1"],[["l1","l3","p0",
+> "p2"],"3/1"],[["l1","p0","p2","p3"],"-2/1"],[["l2","l3","p0","p1"],"-4/1"]],"1,9":[[["l0","l0","l0","p1"]
+> ,"1/2"],[["l0","l0","l1","p0"],"1/2"],[["l0","l0","p0","p1"],"-1/2"],[["l0","l1","l1","p1"],"-1/2"],[["l0
+> ","l1","p0","p0"],"-1/2"],[["l0","l1","p1","p1"],"1/2"],[["l0","l1","p2","p2"],"1/1"],[["l0","l1","p3","p
+> 3"],"-1/1"],[["l0","l2","l2","p1"],"-1/2"],[["l0","l2","p1","p2"],"-1/2"],[["l0","l3","l3","p1"],"-1/2"],
+> [["l0","l3","p1","p3"],"5/2"],[["l0","p1","p3","p3"],"-1/1"],[["l1","l1","l1","p0"],"-1/2"],[["l1","l1","
+> p0","p1"],"1/2"],[["l1","l2","l2","p0"],"-1/2"],[["l1","l2","p0","p2"],"-1/2"],[["l1","l3","l3","p0"],"-1
+> /2"],[["l1","l3","p0","p3"],"5/2"],[["l1","p0","p3","p3"],"-1/1"],[["l2","l2","p0","p1"],"1/1"],[["l3","l
+> 3","p0","p1"],"-1/1"]],"2,0":[[["l0","l0","l0","p2"],"-3/2"],[["l0","l0","l2","p0"],"1/2"],[["l0","l0","p
+> 0","p2"],"5/2"],[["l0","l1","l1","p2"],"3/2"],[["l0","l1","p1","p2"],"-3/2"],[["l0","l2","l2","p2"],"3/2"
```

```
+> ],[["l0","l2","p0","p0"],"-1/2"],[["l0","l2","p1","p1"],"1/1"],[["l0","l2","p2","p2"],"-1/2"],[["l0","l2"
+> ,"p3","p3"],"1/1"],[["l0","l3","l3","p2"],"3/2"],[["l0","l3","p2","p3"],"-3/2"],[["l0","p0","p0","p2"],"-
+> 1/1"],[["l1","l1","l2","p0"],"-1/2"],[["l1","l2","p0","p1"],"1/2"],[["l2","l2","l2","p0"],"-1/2"],[["l2",
+> "l2","p0","p2"],"1/2"],[["l2","l3","l3","p0"],"-1/2"],[["l2","l3","p0","p3"],"1/2"],[["l2","p0","p1","p1"
+> ],"-1/1"],[["l2","p0","p2","p2"],"-1/1"],[["l2","p0","p3","p3"],"-1/1"]],"2,1":[["[["l0","l0","l1","p2"],"1
+> /1"],[["l0","l0","l2","p1"],"-1/1"],[["l0","l0","p1","p2"],"-2/1"],[["l0","l1","p0","p2"],"-1/1"],[["l0",
+> "l2","p0","p1"],"1/1"],[["l0","p0","p1","p2"],"2/1"],[["l1","l1","l1","p2"],"-1/1"],[["l1","l1","l2","p1"
+> ],"1/1"],[["l1","l1","p1","p2"],"1/1"],[["l1","l2","l2","p2"],"-1/1"],[["l1","l2","p0","p0"],"-1/1"],[["l
+> 1","l2","p1","p1"],"-2/1"],[["l1","l2","p3","p3"],"-2/1"],[["l1","l3","l3","p2"],"-1/1"],[["l1","l3","p2"
+> ,"p3"],"2/1"],[["l2","l2","l2","p1"],"1/1"],[["l2","l2","p1","p2"],"-1/1"],[["l2","l3","l3","p1"],"1/1"],
+> [["l2","p0","p0","p1"],"1/1"],[["l2","p1","p1","p1"],"1/1"],[["l2","p1","p2","p2"],"1/1"],[["l2","p1","p3
+> ","p3"],"1/1"],[["l3","l3","p1","p2"],"-1/1"]],"2,2":[["[["l0","l0","l0","l0"],"-2/1"],[["l0","l0","l0","p0
+> "],"4/1"],[["l0","l0","l1","l1"],"4/1"],[["l0","l0","l1","p1"],"-4/1"],[["l0","l0","l2","l2"],"4/1"],[["l
+> 0","l0","l2","p2"],"-4/1"],[["l0","l0","l3","l3"],"4/1"],[["l0","l0","l3","p3"],"-4/1"],[["l0","l0","p0",
+> "p0"],"-3/1"],[["l0","l0","p1","p1"],"2/1"],[["l0","l0","p3","p3"],"2/1"],[["l0","l1","l1","p0"],"-4/1"],
+> [["l0","l1","p0","p1"],"2/1"],[["l0","l2","l2","p0"],"-4/1"],[["l0","l2","p0","p2"],"2/1"],[["l0","l3","l
+> 3","p0"],"-4/1"],[["l0","l3","p0","p3"],"2/1"],[["l0","p0","p0","p0"],"1/1"],[["l0","p0","p1","p1"],"-1/1
+> "],[["l0","p0","p2","p2"],"1/1"],[["l0","p0","p3","p3"],"-1/1"],[["l1","l1","l1","l1"],"-2/1"],[["l1","l1
+> ","l1","p1"],"4/1"],[["l1","l1","l2","l2"],"-4/1"],[["l1","l1","l2","p2"],"4/1"],[["l1","l1","l3","l3","
+> -4/1"],[["l1","l1","l3","p3"],"4/1"],[["l1","l1","p0","p0"],"1/1"],[["l1","l1","p1","p1"],"-2/1"],[["l1",
+> "l1","p2","p2"],"-1/1"],[["l1","l2","l2","p1"],"4/1"],[["l1","l2","p1","p2"],"-2/1"],[["l1","l3","l3","p1
+> "],"4/1"],[["l1","l3","p1","p3"],"-4/1"],[["l2","l2","l2","l2"],"-2/1"],[["l2","l2","l2","p2"],"4/1"],[["
+> l2","l2","l3","l3"],"-4/1"],[["l2","l2","l3","p3"],"4/1"],[["l2","l2","p1","p1"],"-2/1"],[["l2","l2","p2"
+> ,"p2"],"-3/1"],[["l2","l2","p3","p3"],"-2/1"],[["l2","l3","l3","p2"],"4/1"],[["l2","l3","p2","p3"],"-2/1"
+> ],[["l2","p0","p0","p2"],"1/1"],[["l2","p1","p1","p2"],"1/1"],[["l2","p2","p2","p2"],"1/1"],[["l2","p2","
+> p3","p3"],"1/1"],[["l3","l3","l3","l3"],"-2/1"],[["l3","l3","l3","p3"],"4/1"],[["l3","l3","p0","p0"],"1/1
+> "],[["l3","l3","p2","p2"],"-1/1"],[["l3","l3","p3","p3"],"-2/1"]],"2,3":[["[["l0","l0","l2","p3"],"-1/1",[
+> ["l0","l0","l3","p2"],"1/1"],[["l0","l0","p2","p3"],"-2/1"],[["l0","l2","p0","p3"],"1/1"],[["l0","l3","p0
+> ","p2"],"-1/1"],[["l0","p0","p2","p3"],"2/1"],[["l1","l1","l2","p3"],"1/1"],[["l1","l1","l3","p2"],"-1/1"
+> ],[["l1","l1","p2","p3"],"-1/1"],[["l1","l3","p1","p2"],"2/1"],[["l2","l2","l2","p3"],"1/1"],[["l2","l2",
+> "l3","p2"],"-1/1"],[["l2","l2","l2","p2"],"-1/1"],[["l2","l2","p1","p1"],"-1/1"],[["l0","l2","p2","p2"],
+> "1/2"],[["l0","l2","p3","p3"],"1/1"],[["l3","l3","l3","p2"],"3/2"],[["l0","l3","p2","p3"],"-3/2"],[["l0",
+> "p0","p0","p2"],"1/2"],[["l0","p0","p1","p1"],"-1/1"],[["l0","p1","p2","p2"],"-1/1"],[["l0","p1","p3","p3
+> "],"1/1"],[["l1","l1","l1","p0"],"-1/1"],[["l1","l1","p0","p1"],"1/1"],[["l1","l2","l2","p0"],"-1/1"],[["l1","l2","
+> p0","p2"],"1/1"],[["l1","l3","l3","p0"],"-1/1"],[["l1","l3","p0","p3"],"2/1"],[["l2","l2","p0","p1"],"2/1
+> "],[["l2","p0","p1","p2"],"-2/1"],[["l3","l3","p0","p1"],"-1/1"]],"2,6":[["[["l0","l1","p2","p3"],"3/1"],[[
+> "l0","l2","p1","p3"],"-4/1"],[["l0","l3","p1","p2"],"3/1"],[["l0","p1","p2","p3"],"-2/1"],[["l1","l2","p0
+> ","p3"],"3/1"],[["l1","l3","p0","p2"],"-4/1"],[["l2","l3","p0","p1"],"3/1"],[["l2","p0","p1","p3"],"-2/1"
+> ]],"2,7":[["[["l0","l0","l0","p2"],"-1/2"],[["l0","l0","l2","p0"],"3/2"],[["l0","l0","p0","p2"],"1/2"],[["l
+> 0","l1","l1","p2"],"1/2"],[["l0","l1","p1","p2"],"-1/2"],[["l0","l2","l2","p2"],"1/2"],[["l0","l2","p0","
+> p0"],"-1/2"],[["l0","l2","p1","p1"],"-1/1"],[["l0","l2","p2","p2"],"-1/2"],[["l0","l2","p3","p3"],"-1/1"]
+> ,[["l0","l3","l3","p2"],"1/2"],[["l0","l3","p2","p3"],"-1/2"],[["l0","p0","p0","p2"],"-1/1"],[["l0","p1",
+> "p1","p2"],"1/1"],[["l0","p2","p3","p3"],"1/1"],[["l1","l1","l2","p0"],"-3/2"],[["l1","l2","p0","p1"],"3/
+> 2"],[["l2","l2","l2","p0"],"-3/2"],[["l2","l2","p0","p2"],"5/2"],[["l2","l3","l3","p0"],"-3/2"],[["l2","l
+> 3","p0","p3"],"3/2"],[["l2","p0","p2","p2"],"-1/1"]],"2,8":[["[["l0","l0","l0","p3"],"-1/1"],[["l0","l0","l
+> 3","p0"],"1/1"],[["l0","l0","p0","p3"],"1/1"],[["l0","l1","l1","p3"],"1/1"],[["l0","l2","l2","p3"],"1/1"]
+> ,[["l0","l2","p2","p3"],"-1/1"],[["l0","l3","l3","p3"],"1/1"],[["l0","l3","p1","p1"],"-2/1"],[["l0","l3",
+> "p2","p2"],"1/1"],[["l0","l3","p3","p3"],"-2/1"],[["l0","p0","p0","p3"],"-1/1"],[["l0","p1","p1","p3"],"1
+> /1"],[["l0","p2","p2","p3"],"-1/1"],[["l0","p3","p3","p3"],"1/1"],[["l1","l1","l3","p0"],"-1/1"],[["l1","
+> l1","p0","p3"],"-1/1"],[["l1","l3","p0","p1"],"2/1"],[["l2","l2","l3","p0"],"-1/1"],[["l2","l2","l2","p0","p3"
+> ],"2/1"],[["l2","l3","p0","p2"],"1/1"],[["l2","p0","p2","p3"],"-2/1"],[["l3","l3","l3","p0"],"-1/1"],[["l
+> 3","l3","p0","p3"],"1/1"]],"2,9":[["[["l0","l0","l0","p2"],"1/2"],[["l0","l0","l2","p0"],"1/2"],[["l0","l0"
+> ,"p0","p2"],"-1/2"],[["l0","l1","l1","p2"],"-1/2"],[["l0","l1","p1","p2"],"-1/2"],[["l0","l2","l2","p2"],
+> "-1/2"],[["l0","l2","p0","p0"],"-1/2"],[["l0","l2","p1","p1"],"1/1"],[["l0","l2","p2","p2"],"1/2"],[["l0"
+> ,"l2","p3","p3"],"-1/1"],[["l0","l3","l3","p2"],"-1/2"],[["l0","l3","p2","p3"],"5/2"],[["l0","p2","p3","p
+> 3"],"-1/1"],[["l1","l1","l2","p0"],"-1/2"],[["l1","l1","p0","p2"],"1/1"],[["l1","l2","p0","p1"],"-1/2"],[
+> ["l2","l2","l2","p0"],"-1/2"],[["l2","l2","p0","p2"],"1/2"],[["l2","l3","l3","p0"],"-1/2"],[["l2","l3","p
+> 0","p3"],"5/2"],[["l2","p0","p3","p3"],"-1/1"],[["l3","l3","p0","p2"],"-1/1"]],"3,0":[["[["l0","l0","l0","p
+> 3"],"-3/2"],[["l0","l0","l3","p0"],"1/2"],[["l0","l0","p0","p3"],"5/2"],[["l0","l1","l1","p3"],"3/2"],[["
+> l0","l1","p1","p3"],"-3/2"],[["l0","l2","l2","p3"],"3/2"],[["l0","l2","p2","p3"],"-3/2"],[["l0","l3","l3"
+> ,"p3"],"3/2"],[["l0","l3","p0","p0"],"-1/2"],[["l0","l3","p1","p1"],"1/1"],[["l0","l3","p2","p2"],"1/1"],
+> [["l0","l3","p3","p3"],"-1/2"],[["l0","p0","p0","p3"],"-1/1"],[["l1","l1","l3","p0"],"-1/2"],[["l1","l3",
+> "p0","p1"],"1/2"],[["l2","l2","l3","p0"],"-1/2"],[["l2","l3","p0","p2"],"1/2"],[["l3","l3","l3","p0"],"-1
+> /2"],[["l3","l3","p0","p3"],"1/2"],[["l3","p0","p1","p1"],"-1/1"],[["l3","p0","p2","p2"],"-1/1"],[["l3","
+> p0","p3","p3"],"-1/1"]],"3,1":[["[["l0","l0","l1","p3"],"1/1"],[["l0","l0","l3","p1"],"-1/1"],[["l0","l0","
+> p1","p3"],"-2/1"],[["l0","l1","p0","p3"],"-1/1"],[["l0","l3","p0","p1"],"1/1"],[["l0","p0","p1","p3"],"2/
+> 1"],[["l1","l1","l1","p3"],"-1/1"],[["l1","l1","l3","p1"],"1/1"],[["l1","l1","p1","p3"],"1/1"],[["l1","l2
+> ","l2","p3"],"-1/1"],[["l1","l2","p2","p3"],"2/1"],[["l1","l3","l3","p3"],"-1/1"],[["l1","l3","p0","p0"],
+> "-1/1"],[["l1","l3","p1","p1"],"-2/1"],[["l1","l3","p2","p2"],"-2/1"],[["l2","l2","l3","p1"],"1/1"],[["l2
+> ","l2","p1","p3"],"-1/1"],[["l3","l3","l3","p1"],"1/1"],[["l3","l3","p1","p3"],"-1/1"],[["l3","p0","p0","
```

```
+> p1"],"1/1"],[["l3","p1","p1","p1"],"1/1"],[["l3","p1","p2","p2"],"1/1"],[["l3","p1","p3","p3"],"1/1"]],"3
+> ,2":[[["l0","10","l2","p3"],"1/1"],[["l0","10","l3","p2"],"-1/1"],[["l0","10","p2","p3"],"-2/1"],[["l0","
+> l2","p0","p3"],"-1/1"],[["10","l3","p0","p2"],"1/1"],[["l0","p0","p2","p3"],"2/1"],[["11","11","12","p3"]
+> ,"-1/1"],[["11","11","l3","p2"],"1/1"],[["11","11","p2","p3"],"-1/1"],[["11","12","p1","p3"],"2/1"],[["12
+> ","12","12","p3"],"-1/1"],[["12","12","l3","p2"],"1/1"],[["12","12","p2","p3"],"1/1"],[["l2","l3","l3","p
+> 3"],"-1/1"],[["12","l3","p0","p0"],"-1/1"],[["12","l3","p1","p1"],"-2/1"],[["12","l3","p2","p2"],"-2/1"],
+> [["l3","l3","l3","p2"],"1/1"],[["l3","l3","p2","p3"],"-1/1"],[["l3","p0","p0","p2"],"1/1"],[["l3","p1","p
+> 1","p2"],"1/1"],[["l3","p2","p2","p2"],"1/1"],[["l3","p2","p3","p3"],"1/1"]],"3,3":[[["l0","l0","l0","l0"
+> ],"-2/1"],[["l0","l0","l0","p0"],"4/1"],[["l0","l0","11","11"],"4/1"],[["l0","l0","11","p1"],"-4/1"],[["1
+> 0","l0","12","12"],"4/1"],[["l0","l0","12","p2"],"-4/1"],[["l0","l0","l3","l3"],"4/1"],[["l0","l0","l3","
+> p3"],"-4/1"],[["l0","l0","p0","p0"],"-3/1"],[["l0","l0","p1","p1"],"2/1"],[["l0","l0","p2","p2"],"2/1"],[
+> ["l0","11","11","p0"],"-4/1"],[["l0","11","p0","p1"],"2/1"],[["l0","12","12","p0"],"-4/1"],[["l0","12","p
+> 0","p2"],"2/1"],[["l0","l3","l3","p0"],"-4/1"],[["l0","l3","p0","p3"],"2/1"],[["l0","p0","p0","p0"],"1/1"
+> ],[["l0","p0","p1","p1"],"-1/1"],[["l0","p0","p2","p2"],"-1/1"],[["l0","p0","p3","p3"],"1/1"],[["11","11"
+> ,"11","11"],"-2/1"],[["11","11","11","p1"],"4/1"],[["11","11","12","12"],"-4/1"],[["11","11","12","p2"],"
+> 4/1"],[["11","11","l3","l3"],"-4/1"],[["11","11","l3","p3"],"4/1"],[["11","11","p0","p0"],"1/1"],[["11","
+> 11","p1","p1"],"-2/1"],[["11","11","p3","p3"],"-1/1"],[["11","12","12","p1"],"4/1"],[["11","12","p1","p2"
+> ],"-4/1"],[["11","l3","l3","p1"],"4/1"],[["11","l3","p1","p3"],"-2/1"],[["12","12","12","12"],"-2/1"],[["
+> 12","12","12","p2"],"4/1"],[["12","12","l3","l3"],"-4/1"],[["12","12","l3","p3"],"4/1"],[["12","12","p0",
+> "p0"],"1/1"],[["12","12","p2","p2"],"-2/1"],[["12","12","p3","p3"],"-1/1"],[["12","l3","l3","p2"],"4/1"],
+> [["12","l3","p2","p3"],"-2/1"],[["l3","l3","l3","l3"],"-2/1"],[["l3","l3","l3","p3"],"4/1"],[["l3","l3","
+> p1","p1"],"-2/1"],[["l3","l3","p2","p2"],"-2/1"],[["l3","l3","p3","p3"],"-3/1"],[["l3","p0","p0","p3"],"1
+> /1"],[["l3","p1","p1","p3"],"1/1"],[["l3","p2","p2","p3"],"1/1"],[["l3","p3","p3","p3"],"1/1"]],"3,4":[[[
+> "l0","l0","l0","p3"],"1/2"],[["l0","l0","l3","p0"],"1/2"],[["l0","l0","p0","p3"],"-1/2"],[["l0","11","11"
+> ,"p3"],"-1/2"],[["l0","11","p1","p3"],"5/2"],[["l0","12","12","p3"],"-1/2"],[["l0","12","p2","p3"],"-1/2"
+> ],[["l0","l3","l3","p3"],"-1/2"],[["l0","l3","p0","p0"],"-1/2"],[["l0","l3","p1","p1"],"-1/1"],[["l0","l3
+> ","p2","p2"],"1/1"],[["l0","l3","p3","p3"],"1/2"],[["l0","p1","p1","p3"],"-1/1"],[["11","11","l3","p0","
+> -1/2"],[["11","11","p0","p3"],"-1/1"],[["11","l3","p0","p1"],"5/2"],[["12","12","l3","p0"],"-1/2"],[["12"
+> ,"12","p0","p3"],"1/1"],[["12","l3","p0","p2"],"-1/2"],[["l3","l3","l3","p0"],"-1/2"],[["l3","l3","p0","p
+> 3"],"1/2"],[["l3","p0","p1","p1"],"-1/1"]],"3,5":[[["l0","11","p2","p3"],"3/1"],[["l0","12","p1","p3"],"3
+> /1"],[["l0","l3","p1","p2"],"-4/1"],[["l0","p1","p2","p3"],"-2/1"],[["11","12","p0","p3"],"-4/1"],[["11",
+> "l3","p0","p2"],"3/1"],[["12","l3","p0","p1"],"3/1"],[["l3","p0","p1","p2"],"-2/1"]],"3,6":[[["l0","l0","
+> l0","p1"],"-1/1"],[["l0","l0","11","p0"],"1/1"],[["l0","l0","p0","p1"],"1/1"],[["l0","11","11","p1"],"1/1
+> "],[["l0","11","p1","p1"],"-2/1"],[["l0","11","p2","p2"],"-2/1"],[["l0","11","p3","p3"],"1/1"],[["l0","12
+> ","12","p1"],"1/1"],[["l0","l3","l3","p1"],"1/1"],[["l0","l3","p1","p3"],"-1/1"],[["l0","p0","p0","p1"],"
+> -1/1"],[["l0","p1","p1","p1"],"1/1"],[["l0","p1","p2","p2"],"1/1"],[["l0","p1","p3","p3"],"-1/1"],[["11",
+> "11","11","p0"],"-1/1"],[["11","11","p0","p1"],"1/1"],[["11","12","12","p0"],"-1/1"],[["11","12","p0","p2
+> "],"2/1"],[["11","l3","l3","p0"],"-1/1"],[["11","l3","p0","p3"],"1/1"],[["12","12","p0","p1"],"-1/1"],[["
+> l3","l3","p0","p1"],"2/1"],[["l3","p0","p1","p3"],"-2/1"]],"3,7":[[["l0","l0","l0","p3"],"1/2"],[["l0","1
+> 0","l3","p0"],"1/2"],[["l0","l0","p0","p3"],"-1/2"],[["l0","11","11","p3"],"-1/2"],[["l0","11","p1","p3"]
+> ,"-1/2"],[["l0","12","12","p3"],"-1/2"],[["l0","12","p2","p3"],"5/2"],[["l0","l3","l3","p3"],"-1/2"],[["1
+> 0","l3","p0","p0"],"-1/2"],[["l0","l3","p1","p1"],"1/1"],[["l0","l3","p2","p2"],"-1/1"],[["l0","l3","p3",
+> "p3"],"1/2"],[["l0","p2","p2","p3"],"-1/1"],[["11","11","l3","p0"],"-1/2"],[["11","11","p0","p3"],"1/1"],
+> [["11","l3","p0","p1"],"-1/2"],[["12","12","l3","p0"],"-1/2"],[["12","12","p0","p3"],"-1/1"],[["12","l3",
+> "p0","p2"],"5/2"],[["l3","l3","l3","p0"],"-1/2"],[["l3","l3","p0","p3"],"1/2"],[["l3","p0","p2","p2"],"-1
+> /1"]],"3,8":[[["l0","l0","l0","p2"],"-1/1"],[["l0","l0","12","p0"],"1/1"],[["l0","l0","p0","p2"],"1/1"],[
+> ["l0","11","11","p2"],"1/1"],[["l0","12","12","p2"],"1/1"],[["l0","12","p1","p1"],"-2/1"],[["l0","12","p2
+> ","p2"],"-2/1"],[["l0","12","p3","p3"],"1/1"],[["l0","13","13","p2"],"1/1"],[["l0","13","l2","p3"],"-1/1"
+> ],[["l0","p0","p0","p2"],"-1/1"],[["l0","p1","p1","p2"],"1/1"],[["l0","p2","p2","p2"],"1/1"],[["l0","p2",
+> "p3","p3"],"-1/1"],[["11","11","l2","p0"],"-1/1"],[["11","11","p0","p2"],"-1/1"],[["11","l2","p0","p1"],"
+> 2/1"],[["12","12","12","p0"],"-1/1"],[["12","12","p0","p2"],"1/1"],[["12","13","13","p0"],"-1/1"],[["12",
+> "13","p0","p3"],"1/1"],[["13","13","p0","p2"],"2/1"],[["13","p0","p2","p3"],"-2/1"]],"3,9":[[["l0","l0","
+> l0","p3"],"-1/2"],[["l0","l0","13","p0"],"3/2"],[["l0","l0","p0","p3"],"1/2"],[["l0","11","11","p3"],"1/2
+> "],[["l0","11","p1","p3"],"-1/2"],[["l0","12","12","p3"],"1/2"],[["l0","12","p2","p3"],"-1/2"],[["l0","13
+> ","13","p3"],"1/2"],[["l0","13","p0","p0"],"-1/2"],[["l0","13","p1","p1"],"-1/1"],[["l0","l0","p2","p2"],
+> "-1/1"],[["l0","13","p3","p3"],"-1/2"],[["l0","p0","p0","p3"],"-1/1"],[["l0","p1","p1","p3"],"1/1"],[["l0
+> ","p2","p2","p3"],"1/1"],[["11","11","13","p0"],"-3/2"],[["11","13","p0","p1"],"3/2"],[["12","12","13","p
+> 0"],"-3/2"],[["12","13","p0","p2"],"3/2"],[["13","13","13","p0"],"-3/2"],[["13","13","p0","p3"],"5/2"],[[
+> "13","p0","p3","p3"],"-1/1"]],"4,0":[[["10","10","10","p0"],"1/2"],[["10","10","11","p1"],"1/2"],[["10","
+> 10","p0","p0"],"-3/4"],[["10","10","p2","p2"],"-1/2"],[["10","10","p3","p3"],"-1/2"],[["10","11","11","p0
+> "],"-1/2"],[["10","11","p0","p1"],"-3/2"],[["10","12","12","p0"],"-1/2"],[["10","12","p0","p2"],"1/1"],[[
+> "10","13","13","p0"],"-1/2"],[["10","13","p0","p3"],"1/1"],[["10","p0","p0","p0"],"1/4"],[["10","p0","p1"
+> ,"p1"],"1/4"],[["10","p0","p2","p2"],"1/4"],[["10","p0","p3","p3"],"1/4"],[["11","11","11","p1"],"-1/2"],
+> [["11","11","p0","p0"],"1/2"],[["11","11","p1","p1"],"3/4"],[["11","11","p2","p2"],"1/2"],[["11","11","p3
+> ","p3"],"1/2"],[["11","12","12","p1"],"-1/2"],[["11","13","13","p1"],"-1/2"],[["11","p0","p0","p1"],"3/4"
+> ],[["11","p1","p1","p1"],"-1/4"],[["11","p1","p2","p2"],"-1/4"],[["11","p1","p3","p3"],"-1/4"],[["12","12
+> ","p1","p1"],"1/2"],[["12","12","p2","p2"],"1/4"],[["12","13","p2","p3"],"1/2"],[["12","p0","p0","p2"],"-
+> 1/4"],[["12","p1","p1","p2"],"-1/4"],[["12","p2","p2","p2"],"-1/4"],[["12","p2","p3","p3"],"-1/4"],[["13"
+> ,"13","p1","p1"],"1/2"],[["13","13","p3","p3"],"1/4"],[["13","p0","p0","p3"],"-1/4"],[["13","p1","p1","p3
+> "],"-1/4"],[["13","p2","p2","p3"],"-1/4"],[["13","p3","p3","p3"],"-1/4"]],"4,1":[[["10","10","10","p1"],"
+> 1/2"],[["10","10","11","p0"],"-3/2"],[["10","10","p0","p1"],"1/2"],[["10","11","11","p1"],"-1/2"],[["10",
+> "11","p0","p0"],"5/2"],[["10","11","p1","p1"],"1/2"],[["10","11","p2","p2"],"-1/1"],[["10","11","p3","p3"
+> ],"-1/1"],[["10","12","12","p1"],"-1/2"],[["10","12","p1","p2"],"1/2"],[["10","13","13","p1"],"-1/2"],[["
+> 10","13","p1","p3"],"1/2"],[["10","p0","p0","p1"],"-1/1"],[["11","11","11","p0"],"3/2"],[["11","11","p0",
+> "p1"],"-3/2"],[["11","12","12","p0"],"3/2"],[["11","12","p0","p2"],"-3/2"],[["11","13","13","p0"],"3/2"],
+> [["11","13","p0","p3"],"-3/2"],[["11","p0","p0","p0"],"-1/1"],[["11","p0","p2","p2"],"1/1"],[["11","p0","
+> p3","p3"],"1/1"],[["12","12","p0","p1"],"-1/1"],[["12","p0","p1","p2"],"1/1"],[["13","13","p0","p1"],"-1/
+> 1"],[["13","p0","p1","p3"],"1/1"]],"4,2":[[["10","10","10","p2"],"-1/2"],[["10","10","12","p0"],"-1/2"],[
+> ["10","10","p0","p2"],"3/2"],[["10","11","11","p2"],"1/2"],[["10","11","p1","p2"],"3/2"],[["10","12","12"
+> ,"p2"],"1/2"],[["10","12","p0","p0"],"1/2"],[["10","12","p1","p1"],"-1/1"],[["10","12","p2","p2"],"-1/2"]
+> ,[["10","12","p3","p3"],"1/1"],[["10","13","13","p2"],"1/2"],[["10","13","p2","p3"],"-3/2"],[["10","p0","
```

```
+> p0","p2"],"-1/1"],[["l1","l1","l2","p0"],"1/2"],[["l1","l1","p0","p2"],"-2/1"],[["l1","l2","p0","p1"],"3/
+> 2"],[["l1","p0","p1","p2"],"-1/1"],[["l2","l2","l2","p0"],"1/2"],[["l2","l2","p0","p2"],"-3/2"],[["l2","l
+> 3","l3","p0"],"1/2"],[["l2","l3","p0","p3"],"-3/2"],[["l2","p0","p2","p2"],"1/1"],[["l3","p0","p2","p3"],
+> "1/1"]],"4,3":[[["l0","l0","l0","p3"],"-1/2"],[["l0","l0","l3","p0"],"-1/2"],[["l0","l0","p0","p3"],"3/2"
+> ],[["l0","l1","l1","p3"],"1/2"],[["l0","l1","p1","p3"],"3/2"],[["l0","l2","l2","p3"],"1/2"],[["l0","l2","
+> p2","p3"],"-3/2"],[["l0","l3","l3","p3"],"1/2"],[["l0","l3","p0","p0"],"1/2"],[["l0","l3","p1","p1"],"-1/
+> 1"],[["l0","l3","p2","p2"],"1/1"],[["l0","l3","p3","p3"],"-1/2"],[["l0","p0","p0","p3"],"-1/1"],[["l1","l
+> 1","l3","p0"],"1/2"],[["l1","l1","p0","p3"],"-2/1"],[["l1","l3","p0","p1"],"3/2"],[["l1","p0","p1","p3"],
+> "-1/1"],[["l2","l2","l3","p0"],"1/2"],[["l2","l3","p0","p2"],"-3/2"],[["l2","p0","p2","p3"],"1/1"],[["l3"
+> ,"l3","l3","p0"],"1/2"],[["l3","l3","p0","p3"],"-3/2"],[["l3","p0","p3","p3"],"1/1"]],"4,4":[[["l0","l0",
+> "l0","l0"],"1/1"],[["l0","l0","l0","p0"],"-2/1"],[["l0","l0","l1","l1"],"-2/1"],[["l0","l0","l1","p1"],"2
+> /1"],[["l0","l0","l2","l2"],"-2/1"],[["l0","l0","l2","p2"],"2/1"],[["l0","l0","l3","l3"],"-2/1"],[["l0","
+> l0","l3","p3"],"2/1"],[["l0","l0","p0","p0"],"3/4"],[["l0","l0","p1","p1"],"-1/2"],[["l0","l1","l1","p0"]
+> ,"2/1"],[["l0","l1","p0","p1"],"-3/2"],[["l0","l2","l2","p0"],"2/1"],[["l0","l2","p0","p2"],"-3/2"],[["l0
+> ","l3","l3","p0"],"2/1"],[["l0","l3","p0","p3"],"-3/2"],[["l0","p0","p0","p0"],"1/4"],[["l0","p0","p1","p
+> 1"],"1/4"],[["l0","p0","p2","p2"],"-1/4"],[["l0","p0","p3","p3"],"-1/4"],[["l1","l1","l1","l1"],"1/1"],[[
+> "l1","l1","l1","p1"],"-2/1"],[["l1","l1","l2","l2"],"2/1"],[["l1","l1","l2","p2"],"-2/1"],[["l1","l1","l3
+> ","l3"],"2/1"],[["l1","l1","l3","p3"],"-2/1"],[["l1","l1","p0","p0"],"-1/1"],[["l1","l1","p1","p1"],"5/4"
+> ],[["l1","l1","p2","p2"],"1/1"],[["l1","l1","p3","p3"],"1/1"],[["l1","l2","l2","p1"],"-2/1"],[["l1","l2",
+> "p1","p2"],"3/2"],[["l1","l3","l3","p1"],"-2/1"],[["l1","l3","p1","p3"],"3/2"],[["l1","p0","p0","p1"],"3/
+> 4"],[["l1","p1","p1","p1"],"-1/4"],[["l1","p1","p2","p2"],"-3/4"],[["l1","p1","p3","p3"],"-3/4"],[["l2","
+> l2","l2","l2"],"1/1"],[["l2","l2","l2","p2"],"-2/1"],[["l2","l2","l3","l3"],"2/1"],[["l2","l2","l3","p3"]
+> ,"-2/1"],[["l2","l2","p1","p1"],"1/2"],[["l2","l2","p2","p2"],"3/4"],[["l2","l3","l3","p2"],"-2/1"],[["l2
+> ","l3","p2","p3"],"3/2"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"-1/4"],[["l2","p2","p2","
+> p2"],"1/4"],[["l2","p2","p3","p3"],"1/4"],[["l3","l3","l3","l3"],"1/1"],[["l3","l3","l3","p3"],"-2/1"],[[
+> "l3","l3","p1","p1"],"1/2"],[["l3","l3","p3","p3"],"3/4"],[["l3","p0","p0","p3"],"-1/4"],[["l3","p1","p1"
+> ,"p3"],"-1/4"],[["l3","p2","p2","p3"],"1/4"],[["l3","p3","p3","p3"],"1/4"]],"4,5":[[["l0","l0","l1","p2"]
+> ,"3/2"],[["l0","l0","l2","p1"],"-1/2"],[["l0","l0","p1","p2"],"-1/1"],[["l0","l1","p0","p2"],"-3/2"],[["l
+> 0","l2","p0","p1"],"1/2"],[["l0","p0","p1","p2"],"1/1"],[["l1","l1","l1","p2"],"-3/2"],[["l1","l1","l2","
+> p1"],"1/2"],[["l1","l1","p1","p2"],"3/2"],[["l1","l2","l2","p2"],"-3/2"],[["l1","l2","p0","p0"],"-1/1"],[
+> ["l1","l2","p1","p1"],"-1/2"],[["l1","l2","p2","p2"],"5/2"],[["l1","l2","p3","p3"],"1/1"],[["l1","l3","l3
+> ","p2"],"-3/2"],[["l1","l3","p2","p3"],"3/2"],[["l1","p0","p0","p2"],"1/1"],[["l1","p2","p2","p2"],"-1/1"
+> ],[["l1","p2","p3","p3"],"-1/1"],[["l2","l2","l2","p1"],"1/2"],[["l2","l2","p1","p2"],"1/2"],[["l2","l3",
+> "l3","p1"],"1/2"],[["l2","l3","p1","p3"],"-1/2"],[["l2","p1","p2","p2"],"-1/1"],[["l3","l3","p1","p2"],"1
+> /1"],[["l3","p1","p2","p3"],"-1/1"]],"4,6":[[["l0","l0","l1","p3"],"3/2"],[["l0","l0","l3","p1"],"-1/2"],
+> [["l0","l0","p1","p3"],"-1/1"],[["l0","l1","p0","p3"],"-3/2"],[["l0","l3","p0","p1"],"1/2"],[["l0","p0","
+> p1","p3"],"1/1"],[["l1","l1","l1","p3"],"-3/2"],[["l1","l1","l3","p1"],"1/2"],[["l1","l1","p1","p3"],"3/2
+> "],[["l1","l2","l2","p3"],"-3/2"],[["l1","l2","p2","p3"],"3/2"],[["l1","l3","l3","p3"],"-3/2"],[["l1","l3
+> ","p0","p0"],"-1/1"],[["l1","l3","p1","p1"],"-1/2"],[["l1","l3","p2","p2"],"1/1"],[["l1","l3","p3","p3",
+> "5/2"],[["l1","p0","p0","p3"],"1/1"],[["l1","p2","p2","p3"],"-1/1"],[["l1","p3","p3","p3"],"-1/1"],[["l2"
+> ,"l2","l3","p1"],"1/2"],[["l2","l2","p1","p3"],"1/1"],[["l2","l3","p1","p2"],"-1/2"],[["l2","p1","p2","p3
+> "],"-1/1"],[["l3","l3","l3","p1"],"1/2"],[["l3","l3","p1","p3"],"1/2"],[["l3","p1","p3","p3"],"-1/1"]],"4
+> ,7":[[["l0","l0","l1","p1"],"-1/2"],[["l0","l0","l2","p2"],"1/2"],[["l0","l0","p0","p0"],"-1/4"],[["l0","
+> l0","p1","p1"],"1/2"],[["l0","l2","p0","p2"],"-1/1"],[["l0","l3","p0","p3"],"1/2"],[["l0","p0","p0","p0"]
+> ,"1/4"],[["l0","p0","p1","p1"],"-1/4"],[["l0","p0","p2","p2"],"1/4"],[["l0","p0","p3","p3"],"-1/4"],[["l1
+> ","l1","l1","p1"],"1/2"],[["l1","l1","l2","p2"],"-1/2"],[["l1","l1","p0","p0"],"1/2"],[["l1","l1","p1","p
+> 1"],"-3/4"],[["l1","l1","p2","p2"],"1/2"],[["l1","l1","p3","p3"],"-1/2"],[["l1","l2","l2","p1"],"1/2"],[[
+> "l1","l2","p1","p2"],"-3/2"],[["l1","l3","l3","p1"],"1/2"],[["l1","p0","p0","p1"],"-1/4"],[["l1","p1","p1
+> ","p1"],"1/4"],[["l1","p1","p2","p2"],"3/4"],[["l1","p1","p3","p3"],"1/4"],[["l2","l2","l2","p2"],"-1/2"
+> ],[["l2","l2","p0","p0"],"1/2"],[["l2","l2","p2","p2"],"3/4"],[["l2","l2","p3","p3"],"-1/2"],[["l2","l3","
+> l3","p2"],"-1/2"],[["l2","l3","p2","p3"],"1/1"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"1/
+> 4"],[["l2","p2","p2","p2"],"-1/4"],[["l2","p2","p3","p3"],"1/4"],[["l3","l3","p1","p1"],"-1/2"],[["l3","l
+> 3","p0","p3"],"-1/4"],[["l3","p0","p0","p3"],"-1/4"],[["l3","p1","p1","p3"],"1/4"],[["l3","p2","p2","p3"]
+> ,"-1/4"],[["l3","p3","p3","p3"],"1/4"]],"4,8":[[["l0","l0","l2","p3"],"1/2"],[["l0","l0","l3","p2"],"1/2"
+> ],[["l0","l2","p0","p3"],"-3/2"],[["l0","l3","p0","p2"],"-3/2"],[["l0","p0","p2","p3"],"1/1"],[["l1","l1"
+> ,"l2","p3"],"-1/2"],[["l1","l1","l3","p2"],"-1/2"],[["l1","l1","p2","p3"],"2/1"],[["l1","l2","p1","p3"],"
+> -3/2"],[["l1","l3","p1","p2"],"-3/2"],[["l1","p1","p2","p3"],"1/1"],[["l2","l2","l2","p3"],"-1/2"],[["l2"
+> ,"l2","l3","p2"],"-1/2"],[["l2","l2","p2","p3"],"3/2"],[["l2","l3","l3","p3"],"-1/2"],[["l2","l3","p0","p
+> 0"],"1/1"],[["l2","l3","p1","p1"],"1/1"],[["l2","l3","p2","p2"],"1/2"],[["l2","l3","p3","p3"],"1/2"],[["l
+> 2","p2","p2","p3"],"-1/1"],[["l3","l3","l3","p2"],"-1/2"],[["l3","l3","p2","p3"],"3/2"],[["l3","p2","p3",
+> "p3"],"-1/1"]],"4,9":[[["l0","l0","l1","p1"],"-1/2"],[["l0","l0","l3","p3"],"1/2"],[["l0","l0","p0","p0"]
+> ,"-1/4"],[["l0","l0","p1","p1"],"1/2"],[["l0","l2","p0","p2"],"1/2"],[["l0","l3","p0","p3"],"-1/1"],[["l0
+> ","p0","p0","p0"],"1/4"],[["l0","p0","p1","p1"],"-1/4"],[["l0","p0","p2","p2"],"-1/4"],[["l0","p0","p3","
+> p3"],"1/4"],[["l1","l1","l1","p1"],"1/2"],[["l1","l1","l3","p3"],"-1/2"],[["l1","l1","p0","p0"],"1/2"],[[
+> "l1","l1","p1","p1"],"-3/4"],[["l1","l1","p2","p2"],"-1/2"],[["l1","l1","p3","p3"],"1/2"],[["l1","l2","l2
+> ","p1"],"1/2"],[["l1","l3","l3","p1"],"1/2"],[["l1","l3","p1","p3"],"-3/2"],[["l1","p0","p0","p1"],"-1/4"
+> ],[["l1","p1","p1","p1"],"1/4"],[["l1","p1","p2","p2"],"1/4"],[["l1","p1","p3","p3"],"3/4"],[["l2","l2","
+> l3","p3"],"-1/2"],[["l2","l2","p1","p1"],"-1/2"],[["l2","l2","p2","p2"],"-1/4"],[["l2","l3","p2","p3"],"1
+> /1"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"1/4"],[["l2","p2","p2","p2"],"1/4"],[["l2","p
+> 2","p3","p3"],"-1/4"],[["l3","l3","l3","p3"],"-1/2"],[["l3","l3","p0","p0"],"1/2"],[["l3","l3","p2","p2"]
+> ,"-1/2"],[["l3","l3","p3","p3"],"3/4"],[["l3","p0","p0","p3"],"-1/4"],[["l3","p1","p1","p3"],"1/4"],[["l3
+> ","p2","p2","p3"],"1/4"],[["l3","p3","p3","p3"],"-1/4"]],"5,0":[[["l0","l0","l1","p2"],"1/2"],[["l0","l0"
+> ,"l2","p1"],"1/2"],[["l0","l0","p1","p2"],"1/1"],[["l0","l1","p0","p2"],"-5/2"],[["l0","l2","p0","p1"],"-
+> 5/2"],[["l1","l1","l1","p2"],"-1/2"],[["l1","l1","l2","p1"],"-1/2"],[["l1","l1","p1","p2"],"1/2"],[["l1",
+> "l2","l2","p2"],"-1/2"],[["l1","l2","p0","p0"],"1/1"],[["l1","l2","p1","p1"],"1/2"],[["l1","l2","p2","p2"
+> ],"1/2"],[["l1","l2","p3","p3"],"1/1"],[["l1","l3","l3","p2"],"-1/2"],[["l1","l3","p2","p3"],"-1/2"],[["l
+> 1","p0","p0","p2"],"1/1"],[["l2","l2","l2","p1"],"-1/2"],[["l2","l2","p1","p2"],"1/2"],[["l2","l3","l3","
+> p1"],"-1/2"],[["l2","l3","p1","p3"],"-1/2"],[["l2","p0","p0","p1"],"1/1"],[["l3","l3","p1","p2"],"1/1"]],
+> "5,1":[[["l0","l0","l0","p2"],"1/1"],[["l0","l0","l2","p0"],"-1/1"],[["l0","l0","p0","p2"],"-1/1"],[["l0"
+> ,"l1","l1","p2"],"-1/1"],[["l0","l1","p1","p2"],"1/1"],[["l0","l2","l2","p2"],"-1/1"],[["l0","l2","p0","p
+> 0"],"2/1"],[["l0","l2","p1","p1"],"1/1"],[["l0","l2","p3","p3"],"-2/1"],[["l0","l3","l3","p2"],"-1/1"],[[
```

```
+> "l0","l3","p2","p3"],"2/1"],[["l1","l1","l2","p0"],"1/1"],[["l1","l1","p0","p2"],"2/1"],[["l1","l2","p0",
+> "p1"],"-1/1"],[["l1","p0","p1","p2"],"-2/1"],[["l2","l2","l2","p0"],"1/1"],[["l2","l2","l2","p0","p2"],"-1/1"]
+> ,[["l2","l3","l3","p0"],"1/1"],[["l2","p0","p0","p0"],"-1/1"],[["l2","p0","p1","p1"],"-1/1"],[["l2","p0",
+> "p2","p2"],"1/1"],[["l2","p0","p3","p3"],"1/1"],[["l3","l3","p0","p2"],"-1/1"]],"5,2":[[["l0","l0","l0","
+> p1"],"1/1"],[["l0","l0","l1","p0"],"-1/1"],[["l0","l0","p0","p1"],"-1/1"],[["l0","l1","l1","p1"],"-1/1"],
+> [["l0","l1","p0","p0"],"2/1"],[["l0","l1","p2","p2"],"1/1"],[["l0","l1","p3","p3"],"-2/1"],[["l0","l2","l
+> 2","p1"],"-1/1"],[["l0","l2","l1","p2"],"1/1"],[["l0","l3","l3","p1"],"-1/1"],[["l0","l3","l1","p3"],"2/1
+> "],[["l1","l1","l1","p0"],"1/1"],[["l1","l1","p0","p1"],"-1/1"],[["l1","l2","l2","p0"],"1/1"],[["l1","l2"
+> ,"p0","p2"],"-1/1"],[["l1","l3","l3","p0"],"1/1"],[["l1","p0","p0","p0"],"-1/1"],[["l1","p0","p1","p1"],"
+> 1/1"],[["l1","p0","p2","p2"],"-1/1"],[["l1","p0","p3","p3"],"1/1"],[["l2","l2","p0","p1"],"2/1"],[["l2","
+> p0","p1","p2"],"-2/1"],[["l3","l3","p0","p1"],"-1/1"]],"5,3":[[["l0","l1","p2","p3"],"3/1"],[["l0","l2","
+> p1","p3"],"3/1"],[["l0","l3","p1","p2"],"-4/1"],[["l1","l2","p0","p3"],"-4/1"],[["l1","l3","p0","p2"],"3/
+> 1"],[["l1","p0","p2","p3"],"-2/1"],[["l2","l3","p0","p1"],"3/1"],[["l2","p0","p1","p3"],"-2/1"]],"5,4":[[
+> ["l0","l0","l1","p2"],"-3/2"],[["l0","l0","l2","p1"],"1/2"],[["l0","l1","p0","p2"],"3/2"],[["l0","l2","p0
+> ","p1"],"-1/2"],[["l1","l1","l1","p2"],"3/2"],[["l1","l1","l2","p1"],"-1/2"],[["l1","l1","p1","p2"],"-5/2
+> "],[["l1","l2","l2","p2"],"3/2"],[["l1","l2","p0","p0"],"-1/1"],[["l1","l2","p1","p1"],"1/2"],[["l1","l2"
+> ,"p2","p2"],"-1/2"],[["l1","l2","p3","p3"],"1/1"],[["l1","l3","l3","p2"],"3/2"],[["l1","l3","p2","p3"],"-
+> 3/2"],[["l1","p1","p1","p2"],"1/1"],[["l2","l2","l2","p1"],"-1/2"],[["l2","l2","p1","p2"],"1/2"],[["l2","
+> l3","l3","p1"],"-1/2"],[["l2","l3","p1","p3"],"1/2"],[["l2","p0","p0","p1"],"1/1"],[["l2","p1","p2","p2"]
+> ,"-1/1"],[["l2","p1","p3","p3"],"-1/1"]],"5,5":[[["l0","l0","l0","l0"],"2/1"],[["l0","l0","l0","p0"],"-4/
+> 1"],[["l0","l0","l1","l1"],"-4/1"],[["l0","l0","l1","p1"],"4/1"],[["l0","l0","l2","l2"],"-4/1"],[["l0","l
+> 0","l2","p2"],"4/1"],[["l0","l0","l3","l3"],"-4/1"],[["l0","l0","l3","p3"],"4/1"],[["l0","l0","p0","p0"],
+> "2/1"],[["l0","l0","p1","p1"],"-1/1"],[["l0","l0","p2","p2"],"-1/1"],[["l0","l1","l1","p0"],"4/1"],[["l0"
+> ,"l1","p0","p1"],"-2/1"],[["l0","l2","l2","p0"],"4/1"],[["l0","l2","p0","p2"],"-2/1"],[["l0","l3","l3","p
+> 0"],"4/1"],[["l0","l3","p0","p3"],"-4/1"],[["l1","l1","l1","l1"],"2/1"],[["l1","l1","l1","p1"],"-4/1"],[[
+> "l1","l1","l2","l2"],"4/1"],[["l1","l1","l2","p2"],"-4/1"],[["l1","l1","l3","l3"],"4/1"],[["l1","l1","l3"
+> ,"p3"],"-4/1"],[["l1","l1","p0","p0"],"-2/1"],[["l1","l1","p1","p1"],"3/1"],[["l1","l1","p3","p3"],"2/1"]
+> ,[["l1","l2","l2","p1"],"-4/1"],[["l1","l2","p1","p2"],"2/1"],[["l1","l3","l3","p1"],"-4/1"],[["l1","l3",
+> "p1","p3"],"2/1"],[["l1","p0","p0","p1"],"1/1"],[["l1","p1","p1","p1"],"-1/1"],[["l1","p1","p2","p2"],"1/
+> 1"],[["l1","p1","p3","p3"],"-1/1"],[["l2","l2","l2","l2"],"2/1"],[["l2","l2","l2","p2"],"-4/1"],[["l2","l
+> 2","l3","l3"],"4/1"],[["l2","l2","l3","p3"],"-4/1"],[["l2","l2","p0","p0"],"-2/1"],[["l2","l2","p2","p2"]
+> ,"3/1"],[["l2","l2","p3","p3"],"2/1"],[["l2","l3","l3","p2"],"-4/1"],[["l2","l3","p2","p3"],"2/1"],[["l2"
+> ,"p0","p0","p2"],"1/1"],[["l2","p1","p1","p2"],"1/1"],[["l2","p2","p2","p2"],"-1/1"],[["l2","p2","p3","p3
+> "],"-1/1"],[["l3","l3","l3","l3"],"2/1"],[["l3","l3","l3","p3"],"-4/1"],[["l3","l3","p1","p1"],"1/1"],[["
+> l3","l3","p2","p2"],"1/1"],[["l3","l3","p3","p3"],"2/1"]],"5,6":[[["l0","l0","l2","p3"],"1/1"],[["l0","l0
+> ","l3","p2"],"-1/1"],[["l0","l0","p2","p3"],"-1/1"],[["l0","l3","p0","p2"],"2/1"],[["l1","l1","l2","p3"],
+> "-1/1"],[["l1","l1","l3","p2"],"1/1"],[["l1","l1","p2","p3"],"-2/1"],[["l1","l2","p1","p3"],"1/1"],[["l1
+> ","l3","p1","p2"],"-1/1"],[["l1","p1","p2","p3"],"2/1"],[["l2","l2","l2","p3"],"-1/1"],[["l2","l2","l3","p
+> 2"],"1/1"],[["l2","l2","p2","p3"],"1/1"],[["l2","l3","l3","p3"],"-1/1"],[["l2","l3","p0","p0"],"-2/1"],[[
+> "l2","l3","p1","p1"],"-1/1"],[["l2","l3","p3","p3"],"2/1"],[["l2","p0","p0","p3"],"1/1"],[["l2","p1","p1"
+> ,"p3"],"1/1"],[["l2","p2","p2","p3"],"-1/1"],[["l2","p3","p3","p3"],"-1/1"],[["l3","l3","l3","p2"],"1/1"]
+> ,[["l3","l3","p2","p3"],"-1/1"]],"5,7":[[["l0","l0","l1","p2"],"1/2"],[["l0","l0","l2","p1"],"-3/2"],[["l
+> 0","l1","p0","p2"],"-1/2"],[["l0","l2","p0","p1"],"3/2"],[["l1","l1","l1","p2"],"-1/2"],[["l1","l1","l2",
+> "p1"],"3/2"],[["l1","l1","p1","p2"],"1/2"],[["l1","l2","l2","p2"],"-1/2"],[["l1","l2","p0","p0"],"-1/1"],
+> [["l1","l2","p1","p1"],"-1/2"],[["l1","l2","p2","p2"],"1/2"],[["l1","l2","p3","p3"],"1/1"],[["l1","l3","l
+> 3","p2"],"-1/2"],[["l1","l3","p2","p3"],"1/2"],[["l1","p0","p0","p2"],"1/1"],[["l1","p1","p1","p2"],"-1/1
+> "],[["l1","p2","p3","p3"],"-1/1"],[["l2","l2","l2","p1"],"3/2"],[["l2","l2","p1","p2"],"-5/2"],[["l2","l3
+> ","l3","p1"],"3/2"],[["l2","l3","p1","p3"],"-3/2"],[["l2","p1","p2","p2"],"1/1"]],"5,8":[[["l0","l0","l1"
+> ,"p3"],"1/1"],[["l0","l0","l3","p1"],"-1/1"],[["l0","l0","p1","p3"],"-1/1"],[["l0","l3","p0","p1"],"2/1"]
+> ,[["l1","l1","l1","p3"],"-1/1"],[["l1","l1","l3","p1"],"1/1"],[["l1","l1","p1","p3"],"1/1"],[["l1","l2","
+> l2","p3"],"-1/1"],[["l1","l2","p2","p3"],"1/1"],[["l1","l3","l3","p3"],"-1/1"],[["l1","l3","p0","p0"],"-2
+> /1"],[["l1","l3","p2","p2"],"-1/1"],[["l1","l3","p3","p3"],"2/1"],[["l1","p0","p0","p3"],"1/1"],[["l1","p
+> 1","p1","p3"],"-1/1"],[["l1","p2","p2","p3"],"1/1"],[["l1","p3","p3","p3"],"-1/1"],[["l2","l2","l3","p1"]
+> ,"1/1"],[["l2","l2","p1","p3"],"-2/1"],[["l2","l3","p1","p2"],"-1/1"],[["l2","p1","p2","p3"],"2/1"],[["l3
+> ","l3","l3","p1"],"1/1"],[["l3","l3","p1","p3"],"-1/1"]],"5,9":[[["l0","l0","l1","p2"],"-1/2"],[["l0","l0
+> ","l2","p1"],"-1/2"],[["l0","l0","p1","p2"],"1/1"],[["l0","l1","p0","p2"],"-1/2"],[["l0","l2","p0","p1"],
+> "-1/2"],[["l1","l1","l1","p2"],"1/2"],[["l1","l1","l2","p1"],"1/2"],[["l1","l1","p1","p2"],"-1/2"],[["l1"
+> ,"l2","l2","p2"],"1/2"],[["l1","l2","p0","p0"],"1/1"],[["l1","l2","p1","p1"],"-1/2"],[["l1","l2","p2","p2
+> "],"-1/2"],[["l1","l2","p3","p3"],"1/1"],[["l1","l3","l3","p2"],"1/2"],[["l1","l3","p2","p3"],"-5/2"],[["
+> l1","p2","p3","p3"],"1/1"],[["l2","l2","l2","p1"],"1/2"],[["l2","l2","p1","p2"],"-1/2"],[["l2","l3","l3",
+> "p1"],"1/2"],[["l2","l3","p1","p3"],"-5/2"],[["l2","p1","p3","p3"],"1/1"],[["l3","l3","p1","p2"],"1/1"]],
+> "6,0":[[["l0","l0","l1","p3"],"1/2"],[["l0","l0","l3","p1"],"1/2"],[["l0","l0","p1","p3"],"1/1"],[["l0","
+> l1","p0","p3"],"-5/2"],[["l0","l3","p0","p1"],"-5/2"],[["l1","l1","l1","p3"],"-1/2"],[["l1","l1","l3","p1
+> "],"-1/2"],[["l1","l1","p1","p3"],"1/2"],[["l1","l2","l2","p3"],"-1/2"],[["l1","l2","p2","p3"],"-1/2"],[[
+> "l1","l3","l3","p3"],"-1/2"],[["l1","l3","p0","p0"],"1/1"],[["l1","l3","p1","p1"],"1/2"],[["l1","l3","p2"
+> ,"p2"],"1/1"],[["l1","l3","p3","p3"],"1/2"],[["l1","p0","p0","p3"],"1/1"],[["l2","l2","l3","p1"],"-1/2"],
+> [["l2","l2","p1","p3"],"1/1"],[["l2","l3","p1","p2"],"-1/2"],[["l3","l3","l3","p1"],"-1/2"],[["l3","l3","
+> p1","p3"],"1/2"],[["l3","p0","p0","p1"],"1/1"]],"6,1":[[["l0","l0","l0","p3"],"1/1"],[["l0","l0","l3","p0
+> "],"-1/1"],[["l0","l0","p0","p3"],"-1/1"],[["l0","l1","l1","p3"],"-1/1"],[["l0","l1","p1","p3"],"1/1"],[[
+> "l0","l2","l2","p3"],"-1/1"],[["l0","l2","p2","p3"],"2/1"],[["l0","l3","l3","p3"],"-1/1"],[["l0","l3","p0
+> ","p0"],"2/1"],[["l0","l3","p1","p1"],"1/1"],[["l0","l3","p2","p2"],"-2/1"],[["l1","l1","l3","p0"],"1/1"]
+> ,[["l1","l1","p0","p3"],"2/1"],[["l1","l3","p0","p1"],"-1/1"],[["l1","l3","p1","p3"],"1/1"],[["l1","l3","
+> l3","p0"],"1/1"],[["l2","l2","p0","p3"],"-1/1"],[["l3","l3","l3","p0"],"-1/1"],[["l3","l3","p0","p3"],"-1
+> /1"],[["l3","p0","p0","p0"],"-1/1"],[["l3","p0","p1","p1"],"-1/1"],[["l3","p0","p2","p2"],"1/1"],[["l3","
+> p0","p3","p3"],"1/1"]],"6,2":[[["l0","l1","p2","p3"],"3/1"],[["l0","l2","p1","p3"],"-4/1"],[["l0","l3","p
+> 1","p2"],"3/1"],[["l1","l2","p0","p3"],"3/1"],[["l1","l3","p0","p2"],"-4/1"],[["l1","p0","p2","p3"],"-2/1
+> "],[["l2","l3","p0","p1"],"3/1"],[["l3","p0","p1","p2"],"-2/1"]],"6,3":[[["l0","l0","l0","p1"],"1/1"],[["
+> l0","l0","l1","p0"],"-1/1"],[["l0","l0","p0","p1"],"-1/1"],[["l0","l1","l1","p1"],"-1/1"],[["l0","l1","p0
+> ","p0"],"2/1"],[["l0","l1","p2","p2"],"-2/1"],[["l0","l1","p3","p3"],"1/1"],[["l0","l2","l2","p1"],"-1/1"
+> ],[["l0","l2","p1","p2"],"2/1"],[["l0","l3","l3","p1"],"-1/1"],[["l0","l3","p1","p3"],"1/1"],[["l1","l1",
+> "l1","p0"],"1/1"],[["l1","l1","p0","p1"],"-1/1"],[["l1","l2","l2","p0"],"1/1"],[["l1","l3","l3","p0"],"1/
```

```
+> 1"],[["l1","l3","p0","p3"],"-1/1"],[["l1","p0","p0","p0"],"-1/1"],[["l1","p0","p1","p1"],"1/1"],[["l1","p
+> 0","p2","p2"],"1/1"],[["l1","p0","p3","p3"],"-1/1"],[["12","12","p0","p1"],"-1/1"],[["l3","l3","p0","p1"]
+> ,"2/1"],[["l3","p0","p1","p3"],"-2/1"]],"6,4":[[["l0","l0","l1","p3"],"-3/2"],[["l0","l0","l3","p1"],"1/2
+> "],[["l0","l1","p0","p3"],"3/2"],[["l0","l3","p0","p1"],"-1/2"],[["l1","l1","l1","p3"],"3/2"],[["l1","l1"
+> ,"l3","p1"],"-1/2"],[["l1","l1","p1","p3"],"-5/2"],[["l1","12","12","p3"],"3/2"],[["l1","l2","p2","p3","
+> -3/2"],[["l1","l3","l3","p3"],"3/2"],[["l1","l3","p0","p0"],"-1/1"],[["l1","l3","p1","p1"],"1/2"],[["l1",
+> "l3","p2","p2"],"1/1"],[["l1","l3","p3","p3"],"-1/2"],[["l1","p1","p1","p3"],"1/1"],[["12","12","l3","p1"
+> ],"-1/2"],[["12","l3","p1","p2"],"1/2"],[["l3","l3","l3","p1"],"-1/2"],[["l3","l3","p1","p3"],"1/2"],[["l
+> 3","p0","p0","p1"],"1/1"],[["l3","p1","p2","p2"],"-1/1"],[["l3","p1","p3","p3"],"-1/1"]],"6,5":[[["l0","l
+> 0","12","p3"],"-1/1"],[["l0","l0","l3","p2"],"1/1"],[["l0","l0","p2","p3"],"-1/1"],[["l0","12","p0","p3"]
+> ,"2/1"],[["l1","l1","12","p3"],"1/1"],[["l1","l1","l3","p2"],"-1/1"],[["l1","l1","p2","p3"],"-2/1"],[["l1
+> ","12","p1","p3"],"-1/1"],[["l1","l3","p1","p2"],"1/1"],[["l1","p1","p2","p3"],"2/1"],[["12","12","12","p
+> 3"],"1/1"],[["12","12","l3","p2"],"-1/1"],[["12","12","p2","p3"],"-1/1"],[["12","13","l3","p3"],"1/1"],[[
+> "12","l3","p0","p0"],"-2/1"],[["12","l3","p1","p1"],"-1/1"],[["12","l3","p2","p2"],"2/1"],[["l3","l3","l3
+> ","p2"],"-1/1"],[["l3","l3","p2","p3"],"1/1"],[["l3","p0","p0","p2"],"1/1"],[["l3","p1","p1","p2"],"1/1"]
+> ,[["l3","p2","p2","p2"],"-1/1"],[["l3","p2","p3","p3"],"-1/1"]],"6,6":[[["l0","l0","l0","l0"],"2/1"],[["l
+> 0","l0","l0","p0"],"-4/1"],[["l0","l0","l1","l1"],"-4/1"],[["l0","l0","l1","p1"],"4/1"],[["l0","l0","12",
+> "12"],"-4/1"],[["l0","l0","12","p2"],"4/1"],[["l0","l0","l3","l3"],"-4/1"],[["l0","l0","l3","p3"],"4/1"],
+> [["l0","l0","p0","p0"],"2/1"],[["l0","l0","p1","p1"],"-1/1"],[["l0","l0","p3","p3"],"-1/1"],[["l0","l1","
+> l1","p0"],"4/1"],[["l0","l1","p0","p1"],"-2/1"],[["l0","12","12","p0"],"4/1"],[["l0","12","p0","p2"],"-4/
+> 1"],[["l0","l3","l3","p0"],"4/1"],[["l0","l3","p0","p3"],"-2/1"],[["l1","l1","l1","l1"],"2/1"],[["l1","l1
+> ","l1","p1"],"-4/1"],[["l1","l1","12","12"],"4/1"],[["l1","l1","12","p2"],"-4/1"],[["l1","l1","l3","l3"],
+> "4/1"],[["l1","l1","l3","p3"],"-4/1"],[["l1","l1","p0","p0"],"-2/1"],[["l1","l1","p1","p1"],"3/1"],[["l1"
+> ,"l1","p2","p2"],"2/1"],[["l1","12","12","p1"],"-4/1"],[["l1","12","p1","p2"],"2/1"],[["l1","l3","l3","p1
+> "],"-4/1"],[["l1","l3","p1","p3"],"2/1"],[["l1","p0","p0","p1"],"1/1"],[["l1","p1","p1","p1"],"-1/1"],[["
+> l1","p1","p2","p2"],"-1/1"],[["l1","p1","p3","p3"],"1/1"],[["12","12","12","12"],"2/1"],[["12","12","12",
+> "p2"],"-4/1"],[["12","12","l3","l3"],"4/1"],[["12","12","l3","p3"],"-4/1"],[["12","12","p1","p1"],"1/1"],
+> [["12","12","p2","p2"],"2/1"],[["12","12","p3","p3"],"1/1"],[["12","l3","l3","p2"],"-4/1"],[["12","l3","p
+> 2","p3"],"2/1"],[["l3","l3","l3","l3"],"2/1"],[["l3","l3","l3","p3"],"-4/1"],[["l3","l3","p0","p0"],"-2/1
+> "],[["l3","l3","p2","p2"],"2/1"],[["l3","l3","p3","p3"],"3/1"],[["l3","p0","p0","p3"],"1/1"],[["l3","p1",
+> "p1","p3"],"1/1"],[["l3","p2","p2","p3"],"-1/1"],[["l3","p3","p3","p3"],"-1/1"]],"6,7":[[["l0","l0","l1",
+> "p3"],"-1/2"],[["l0","l0","l3","p1"],"-1/2"],[["l0","l0","p1","p3"],"1/1"],[["l0","l1","p0","p3"],"-1/2"]
+> ,[["l0","l3","p0","p1"],"-1/2"],[["l1","l1","l1","p3"],"1/2"],[["l1","l1","l3","p1"],"1/2"],[["l1","l1","
+> p1","p3"],"-1/2"],[["l1","l2","l2","p3"],"1/2"],[["l1","l2","p2","p3"],"-5/2"],[["l1","l3","l3","p3"],"1/
+> 2"],[["l1","l3","p0","p0"],"1/1"],[["l1","l3","p1","p1"],"-1/2"],[["l1","l3","p2","p2"],"1/1"],[["l1","l3
+> ","p3","p3"],"-1/2"],[["l1","p2","p2","p3"],"1/1"],[["12","12","l3","p1"],"1/2"],[["12","12","p1","p3"],"
+> 1/1"],[["l2","l3","p1","p2"],"-5/2"],[["l3","l3","l3","p1"],"1/2"],[["l3","l3","p1","p3"],"-1/2"],[["l3",
+> "p1","p2","p2"],"1/1"]],"6,8":[[["l0","l0","l1","p2"],"1/1"],[["l0","l0","l2","p1"],"-1/1"],[["l0","l0","
+> p1","p2"],"-1/1"],[["l0","l2","p0","p1"],"2/1"],[["l1","l1","l1","p2"],"-1/1"],[["l1","l1","l2","p1"],"1/
+> 1"],[["l1","l1","p1","p2"],"1/1"],[["l1","l2","l2","p2"],"-1/1"],[["l1","l2","p0","p0"],"-2/1"],[["l1","l
+> 2","p2","p2"],"2/1"],[["l1","l2","p3","p3"],"-1/1"],[["l1","l3","l3","p2"],"-1/1"],[["l1","l3","p2","p3"]
+> ,"1/1"],[["l1","p0","p0","p2"],"1/1"],[["l1","p1","p1","p2"],"-1/1"],[["l1","p2","p2","p2"],"-1/1"],[["l1
+> ","p2","p3","p3"],"1/1"],[["l2","l2","l2","p1"],"1/1"],[["l2","l2","p1","p2"],"-1/1"],[["l2","l3","l3","p
+> 1"],"1/1"],[["l2","l3","p1","p3"],"-1/1"],[["l3","l3","l3","p1","p2"],"-2/1"],[["l3","p1","p2","p3"],"2/1"]],"
+> 6,9":[[["l0","l0","l1","p3"],"1/2"],[["l0","l0","l3","p1"],"-3/2"],[["l0","l1","p0","p3"],"-1/2"],[["l0",
+> "l3","p0","p1"],"3/2"],[["l1","l1","l1","p3"],"-1/2"],[["l1","l1","l3","p1"],"3/2"],[["l1","l1","p1","p3"
+> ],"1/2"],[["l1","l2","l2","p3"],"-1/2"],[["l1","l2","p2","p3"],"1/2"],[["l1","l3","l3","p3"],"-1/2"],[["l
+> 1","l3","p0","p0"],"-1/1"],[["l1","l3","p1","p1"],"-1/2"],[["l1","l3","p2","p2"],"1/1"],[["l1","l3","p3",
+> "p3"],"1/2"],[["l1","p0","p0","p3"],"1/1"],[["l1","p1","p1","p3"],"-1/1"],[["l1","p2","p2","p3"],"-1/1"],
+> [["12","l2","l3","p1"],"3/2"],[["12","l3","p1","p2"],"-3/2"],[["l3","l3","l3","p1"],"3/2"],[["l3","l3","p
+> 1","p3"],"-5/2"],[["l3","p1","p3","p3"],"1/1"]],"7,0":[[["l0","l0","l0","p0"],"1/2"],[["l0","l0","l2","p2
+> "],"1/2"],[["l0","l0","p0","p0"],"-3/4"],[["l0","l0","p1","p1"],"-1/2"],[["l0","l0","p3","p3"],"-1/2"],[[
+> "l0","l1","l1","p0"],"-1/2"],[["l0","l1","p0","p1"],"1/1"],[["l0","12","12","p0"],"-1/2"],[["l0","12","p0
+> ","p2"],"-3/2"],[["l0","l3","l3","p0"],"-1/2"],[["l0","l3","p0","p3"],"1/1"],[["l0","p0","p0","p0"],"1/4"
+> ],[["l0","p0","p1","p1"],"1/4"],[["l0","p0","p2","p2"],"1/4"],[["l0","p0","p3","p3"],"1/4"],[["l1","l1","
+> l2","p2"],"-1/2"],[["l1","l1","p1","p1"],"1/4"],[["l1","l1","p2","p2"],"1/2"],[["l1","l3","p1","p3"],"1/2
+> "],[["l1","p0","p0","p1"],"-1/4"],[["l1","p1","p1","p1"],"-1/4"],[["l1","p1","p2","p2"],"-1/4"],[["l1","p
+> 1","p3","p3"],"-1/4"],[["l2","l2","l2","p2"],"-1/2"],[["l2","l2","p0","p0"],"1/2"],[["l2","l2","p1","p1"]
+> ,"1/2"],[["l2","l2","p2","p2"],"3/4"],[["l2","l2","p3","p3"],"1/2"],[["l2","l3","l3","p2"],"-1/2"],[["l2"
+> ,"p0","p0","p2"],"3/4"],[["l2","p1","p1","p2"],"-1/4"],[["l2","p2","p2","p2"],"-1/4"],[["l2","p2","p3","p
+> 3"],"-1/4"],[["l3","l3","p2","p2"],"1/2"],[["l3","l3","p3","p3"],"1/4"],[["l3","p0","p0","p3"],"-1/4"],[[
+> "l3","p1","p1","p3"],"-1/4"],[["l3","p2","p2","p3"],"-1/4"],[["l3","p3","p3","p3"],"-1/4"]],"7,1":[[["l0"
+> ,"l0","l0","p1"],"-1/2"],[["l0","l0","l1","p0"],"-1/2"],[["l0","l0","p0","p1"],"3/2"],[["l0","l1","l1","p
+> 1"],"1/2"],[["l0","l1","p0","p0"],"1/2"],[["l0","l1","p1","p1"],"-1/2"],[["l0","l1","p2","p2"],"-1/1"],[[
+> "l0","l1","p3","p3"],"1/1"],[["l0","12","12","p1"],"1/2"],[["l0","12","p1","p2"],"3/2"],[["l0","l3","l3",
+> "p1"],"1/2"],[["l0","l3","p1","p3"],"-3/2"],[["l0","p0","p0","p1"],"-1/1"],[["l1","l1","l1","p0"],"1/2"],
+> [["l1","l1","p0","p1"],"-3/2"],[["l1","12","12","p0"],"1/2"],[["l1","12","p0","p2"],"3/2"],[["l1","l3","l
+> 3","p0"],"1/2"],[["l1","l3","p0","p3"],"-3/2"],[["l1","p0","p1","p1"],"1/1"],[["12","12","p0","p1"],"-2/1
+> "],[["12","p0","p1","p2"],"-1/1"],[["l3","p0","p1","p3"],"1/1"]],"7,2":[[["l0","l0","l0","p2"],"1/2"],[["
+> l0","l0","12","p0"],"-3/2"],[["l0","l0","p0","p2"],"1/2"],[["l0","l1","l1","p2"],"-1/2"],[["l0","l1","p1"
+> ,"p2"],"1/2"],[["l0","12","12","p2"],"-1/2"],[["l0","12","p0","p0"],"5/2"],[["l0","12","p1","p1"],"-1/1"]
+> ,[["l0","12","p2","p2"],"1/2"],[["l0","12","p3","p3"],"-1/1"],[["l0","l3","l3","p2"],"-1/2"],[["l0","l3",
+> "p2","p3"],"1/2"],[["l0","p0","p0","p2"],"-1/1"],[["l1","l1","12","p0"],"3/2"],[["l1","l1","p0","p2"],"-1
+> /1"],[["l1","12","p0","p1"],"-3/2"],[["l1","p0","p1","p2"],"1/1"],[["12","12","12","p0"],"3/2"],[["12","l
+> 2","p0","p2"],"-3/2"],[["12","l3","l3","p0"],"3/2"],[["12","l3","p0","p3"],"-3/2"],[["12","p0","p0","p0"]
+> ,"-1/1"],[["12","p0","p1","p1"],"1/1"],[["12","p0","p3","p3"],"1/1"],[["l3","l3","p0","p2"],"-1/1"],[["l3
+> ","p0","p2","p3"],"1/1"]],"7,3":[[["l0","l0","l0","p3"],"-1/2"],[["l0","l0","l3","p0"],"-1/2"],[["l0","l0
+> ","p0","p3"],"3/2"],[["l0","l1","l1","p3"],"1/2"],[["l0","l1","p1","p3"],"-3/2"],[["l0","l2","l2","p3"],"
+> 1/2"],[["l0","l2","p2","p3"],"3/2"],[["l0","l3","l3","p3"],"1/2"],[["l0","l3","p0","p0"],"1/2"],[["l0","l
+> 3","p1","p1"],"1/1"],[["l0","l3","p2","p2"],"-1/1"],[["l0","l3","p3","p3"],"-1/2"],[["l0","p0","p0","p3"]
+> ,"-1/1"],[["l1","l1","l3","p0"],"1/2"],[["l1","l3","p0","p1"],"-3/2"],[["l1","p0","p1","p3"],"1/1"],[["12
```

+> ","l2","l3","p0"],"1/2"],[["l2","l2","p0","p3"],"-2/1"],[["l2","l3","p0","p2"],"3/2"],[["l2","p0","p2","p
+> 3"],"-1/1"],[["l3","l3","l3","p0"],"1/2"],[["l3","l3","p0","p3"],"-3/2"],[["l3","p0","p3","p3"],"1/1"]],"
+> 7,4":[[["l0","l0","l1","p1"],"1/2"],[["l0","l0","l2","p2"],"-1/2"],[["l0","l0","p0","p0"],"-1/4"],[["l0",
+> "l0","p2","p2"],"1/2"],[["l0","l1","p0","p1"],"-1/1"],[["l0","l3","p0","p3"],"1/2"],[["l0","p0","p0","p0"
+>],"1/4"],[["l0","p0","p1","p1"],"1/4"],[["l0","p0","p2","p2"],"-1/4"],[["l0","p0","p3","p3"],"-1/4"],[["l
+> 1","l1","l1","p1"],"-1/2"],[["l1","l1","l2","p2"],"1/2"],[["l1","l1","p0","p0"],"1/2"],[["l1","l1","p1","
+> p1"],"3/4"],[["l1","l1","p3","p3"],"-1/2"],[["l1","l2","l2","p1"],"-1/2"],[["l1","l2","p1","p2"],"-3/2"],
+> [["l1","l3","l3","p1"],"-1/2"],[["l1","l3","p1","p3"],"1/1"],[["l1","p0","p0","p1"],"-1/4"],[["l1","p1","
+> p1","p1"],"-1/4"],[["l1","p1","p2","p2"],"1/4"],[["l1","p1","p3","p3"],"1/4"],[["l2","l2","l2","p2"],"1/2
+> "],[["l2","l2","p0","p0"],"1/2"],[["l2","l2","p1","p1"],"1/2"],[["l2","l2","p2","p2"],"-3/4"],[["l2","l2"
+> ,"p3","p3"],"-1/2"],[["l2","l3","l3","p2"],"1/2"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"
+> 3/4"],[["l2","p2","p2","p2"],"1/4"],[["l2","p2","p3","p3"],"1/4"],[["l3","l3","p2","p2"],"-1/2"],[["l3","
+> l3","p3","p3"],"-1/4"],[["l3","p0","p0","p3"],"-1/4"],[["l3","p1","p1","p3"],"-1/4"],[["l3","p2","p2","p3
+> "],"1/4"],[["l3","p3","p3","p3"],"1/4"]],"7,5":[[["l0","l0","l1","p2"],"-1/2"],[["l0","l0","l2","p1"],"3/
+> 2"],[["l0","l0","p1","p2"],"-1/1"],[["l0","l1","p0","p2"],"1/2"],[["l0","l2","p0","p1"],"-3/2"],[["l0","p
+> 0","p1","p2"],"1/1"],[["l1","l1","l1","p2"],"1/2"],[["l1","l1","l2","p1"],"-3/2"],[["l1","l1","p1","p2"],
+> "1/2"],[["l1","l2","l2","p2"],"1/2"],[["l1","l2","p0","p0"],"-1/1"],[["l1","l2","p1","p1"],"5/2"],[["l1",
+> "l2","p2","p2"],"-1/2"],[["l1","l2","p3","p3"],"1/1"],[["l1","l3","l3","p2"],"1/2"],[["l1","l3","p2","p3"
+>],"-1/2"],[["l1","p1","p1","p2"],"-1/1"],[["l2","l2","l2","p1"],"-3/2"],[["l2","l2","p1","p2"],"3/2"],[["
+> l2","l3","l3","p1"],"-3/2"],[["l2","l3","p1","p3"],"3/2"],[["l2","p0","p0","p1"],"1/1"],[["l2","p1","p1",
+> "p1"],"-1/1"],[["l2","p1","p3","p3"],"-1/1"],[["l3","l3","p1","p2"],"1/1"],[["l3","p1","p2","p3"],"-1/1"]
+>],"7,6":[[["l0","l0","l1","p3"],"1/2"],[["l0","l0","l3","p1"],"1/2"],[["l0","l1","p0","p3"],"-3/2"],[["l0
+> ","l3","p0","p1"],"-3/2"],[["l0","p0","p1","p3"],"1/1"],[["l1","l1","l1","p3"],"-1/2"],[["l1","l1","l3","
+> p1"],"-1/2"],[["l1","l1","p1","p3"],"3/2"],[["l1","l2","l2","p3"],"-1/2"],[["l1","l2","p2","p3"],"-3/2"],
+> [["l1","l3","l3","p3"],"-1/2"],[["l1","l3","p0","p0"],"1/1"],[["l1","l3","p1","p1"],"1/2"],[["l1","l3","p
+> 2","p2"],"1/1"],[["l1","l3","p3","p3"],"1/2"],[["l1","p1","p1","p3"],"-1/1"],[["l2","l2","l3","p1"],"-1/2
+> "],[["l2","l2","p1","p3"],"2/1"],[["l2","l3","p1","p2"],"-3/2"],[["l2","p1","p2","p3"],"1/1"],[["l3","l3"
+> ,"l3","p1"],"-1/2"],[["l3","l3","p1","p3"],"3/2"],[["l3","p1","p3","p3"],"-1/1"]],"7,7":[[["l0","l0","l0"
+> ,"l0"],"1/1"],[["l0","l0","l0","p0"],"-2/1"],[["l0","l0","l1","l1"],"-2/1"],[["l0","l0","l1","p1"],"2/1"]
+> ,[["l0","l0","l2","l2"],"-2/1"],[["l0","l0","l2","p2"],"2/1"],[["l0","l0","l3","l3"],"-2/1"],[["l0","l0",
+> "l3","p3"],"2/1"],[["l0","l0","p0","p0"],"3/4"],[["l0","l0","p2","p2"],"-1/2"],[["l0","l1","l1","p0"],"2/
+> 1"],[["l0","l1","p0","p1"],"-3/2"],[["l0","l2","l2","p0"],"2/1"],[["l0","l2","p0","p2"],"-3/2"],[["l0","l
+> 3","l3","p0"],"2/1"],[["l0","l3","p0","p3"],"-3/2"],[["l0","p0","p0","p0"],"1/4"],[["l0","p0","p1","p1"],
+> "-1/4"],[["l0","p0","p2","p2"],"1/4"],[["l0","p0","p3","p3"],"-1/4"],[["l1","l1","l1","l1"],"1/1"],[["l1"
+> ,"l1","l1","p1"],"-2/1"],[["l1","l1","l2","l2"],"2/1"],[["l1","l1","l2","p2"],"-2/1"],[["l1","l1","l3","l
+> 3"],"2/1"],[["l1","l1","l3","p3"],"-2/1"],[["l1","l1","p1","p1"],"3/4"],[["l1","l1","p2","p2"],"1/2"],[["
+> l1","l2","l2","p1"],"-2/1"],[["l1","l2","p1","p2"],"3/2"],[["l1","l3","l3","p1"],"-2/1"],[["l1","l3","p1"
+> ,"p3"],"3/2"],[["l1","p0","p0","p1"],"-1/4"],[["l1","p1","p1","p1"],"1/4"],[["l1","p1","p2","p2"],"-1/4"]
+> ,[["l1","p1","p3","p3"],"1/4"],[["l2","l2","l2","l2"],"1/1"],[["l2","l2","l2","p2"],"-2/1"],[["l2","l2","
+> l3","l3"],"2/1"],[["l2","l2","l3","p3"],"-2/1"],[["l2","l2","p0","p0"],"-1/1"],[["l2","l2","p1","p1"],"1/
+> 1"],[["l2","l2","p2","p2"],"5/4"],[["l2","l2","p3","p3"],"1/1"],[["l2","l3","l3","p2"],"-2/1"],[["l2","l3
+> ","p2","p3"],"3/2"],[["l2","p0","p0","p2"],"3/4"],[["l2","p1","p1","p2"],"-3/4"],[["l2","p2","p2","p2"],"
+> -1/4"],[["l2","p2","p3","p3"],"-3/4"],[["l3","l3","l3","l3"],"1/1"],[["l3","l3","l3","p3"],"-2/1"],[["l3"
+> ,"l3","p2","p2"],"1/2"],[["l3","l3","p3","p3"],"3/4"],[["l3","p0","p0","p3"],"-1/4"],[["l3","p1","p1","p3
+> "],"1/4"],[["l3","p2","p2","p3"],"-1/4"],[["l3","p3","p3","p3"],"1/4"]],"7,8":[[["l0","l0","l2","p3"],"3/
+> 2"],[["l0","l0","l3","p2"],"-1/2"],[["l0","l0","p2","p3"],"-1/1"],[["l0","l2","p0","p3"],"-3/2"],[["l0","
+> l3","p0","p2"],"1/2"],[["l0","p0","p2","p3"],"1/1"],[["l1","l1","l2","p3"],"-3/2"],[["l1","l1","l3","p2"]
+> ,"1/2"],[["l1","l1","p2","p3"],"1/1"],[["l1","l2","p1","p3"],"3/2"],[["l1","l3","p1","p2"],"-1/2"],[["l1"
+> ,"p1","p2","p3"],"-1/1"],[["l2","l2","l2","p3"],"-3/2"],[["l2","l2","l3","p2"],"1/2"],[["l2","l2","p2","p
+> 3"],"3/2"],[["l2","l3","l3","p3"],"-3/2"],[["l2","l3","p0","p0"],"-1/1"],[["l2","l3","p1","p1"],"1/1"],[[
+> "l2","l3","p2","p2"],"-1/2"],[["l2","l3","p3","p3"],"5/2"],[["l2","p0","p0","p3"],"1/1"],[["l2","p1","p1"
+> ,"p3"],"-1/1"],[["l2","p3","p3","p3"],"-1/1"],[["l3","l3","l3","p2"],"1/2"],[["l3","l3","p2","p3"],"1/2"]
+> ,[["l3","p2","p3","p3"],"-1/1"]],"7,9":[[["l0","l0","l2","p2"],"-1/2"],[["l0","l0","l3","p3"],"1/2"],[["l
+> 0","l0","p0","p0"],"-1/4"],[["l0","l0","p2","p2"],"1/2"],[["l0","l1","p0","p1"],"1/2"],[["l0","l3","p0","
+> p3"],"-1/1"],[["l0","p0","p0","p0"],"1/4"],[["l0","p0","p1","p1"],"-1/4"],[["l0","p0","p2","p2"],"-1/4"],
+> [["l0","p0","p3","p3"],"1/4"],[["l1","l1","l2","p2"],"1/2"],[["l1","l1","l3","p3"],"-1/2"],[["l1","l1","p
+> 1","p1"],"-1/4"],[["l1","l1","p2","p2"],"-1/2"],[["l1","l3","p1","p3"],"1/1"],[["l1","p0","p0","p1"],"-1/
+> 4"],[["l1","p1","p1","p1"],"1/4"],[["l1","p1","p2","p2"],"1/4"],[["l1","p1","p3","p3"],"-1/4"],[["l2","l2
+> ","l2","p2"],"1/2"],[["l2","l2","l3","p3"],"-1/2"],[["l2","l2","p0","p0"],"1/2"],[["l2","l2","p1","p1"],"
+> -1/2"],[["l2","l2","p2","p2"],"-3/4"],[["l2","l2","p3","p3"],"1/2"],[["l2","l3","l3","p2"],"1/2"],[["l2",
+> "l3","p2","p3"],"-3/2"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"1/4"],[["l2","p2","p2","p2
+> "],"1/4"],[["l2","p2","p3","p3"],"3/4"],[["l3","l3","l3","p3"],"-1/2"],[["l3","l3","p0","p0"],"1/2"],[["l
+> 3","l3","p1","p1"],"-1/2"],[["l3","l3","p3","p3"],"3/4"],[["l3","p0","p0","p3"],"-1/4"],[["l3","p1","p1",
+> "p3"],"1/4"],[["l3","p2","p2","p3"],"1/4"],[["l3","p3","p3","p3"],"-1/4"]],"8,0":[[["l0","l0","l2","p3"],
+> "1/2"],[["l0","l0","l3","p2"],"1/2"],[["l0","l0","p2","p3"],"1/1"],[["l0","l2","p0","p3"],"-5/2"],[["l0",
+> "l3","p0","p2"],"-5/2"],[["l1","l1","l2","p3"],"-1/2"],[["l1","l1","l3","p2"],"-1/2"],[["l1","l1","p2","p
+> 3"],"1/1"],[["l1","l2","p1","p3"],"-1/2"],[["l1","l3","p1","p2"],"-1/2"],[["l2","l2","l2","p3"],"-1/2"],[
+> ["l2","l2","l3","p2"],"-1/2"],[["l2","l2","p2","p3"],"1/2"],[["l2","l3","l3","p3"],"-1/2"],[["l2","l3","p
+> 0","p0"],"1/1"],[["l2","l3","p1","p1"],"1/1"],[["l2","l3","p2","p2"],"1/2"],[["l2","l3","p3","p3"],"1/2"]
+> ,[["l2","p0","p0","p3"],"1/1"],[["l3","l3","l3","p2"],"-1/2"],[["l3","l3","p2","p3"],"1/2"],[["l3","p0","
+> p0","p2"],"1/1"]],"8,1":[[["l0","l1","p2","p3"],"-4/1"],[["l0","l2","p1","p3"],"3/1"],[["l0","l3","p1","p
+> 2"],"3/1"],[["l1","l2","p0","p3"],"3/1"],[["l1","l3","p0","p2"],"3/1"],[["l2","l3","p0","p1"],"-4/1"],[["
+> l2","p0","p1","p3"],"-2/1"],[["l3","p0","p1","p2"],"-2/1"]],"8,2":[[["l0","l0","l0","p3"],"1/1"],[["l0","
+> l0","l3","p0"],"-1/1"],[["l0","l0","p0","p3"],"-1/1"],[["l0","l1","l1","p3"],"-1/1"],[["l0","l1","p1","p3
+> "],"2/1"],[["l0","l2","l2","p3"],"-1/1"],[["l0","l2","p2","p3"],"1/1"],[["l0","l3","l3","p3"],"-1/1"],[["
+> l0","l3","p0","p0"],"2/1"],[["l0","l3","p1","p1"],"-2/1"],[["l0","l3","p2","p2"],"1/1"],[["l1","l1","l3",
+> "p0"],"1/1"],[["l1","l1","p0","p3"],"-1/1"],[["l2","l2","l3","p0"],"1/1"],[["l2","l2","p0","p3"],"2/1"],[
+> ["l2","l3","p0","p2"],"-1/1"],[["l2","p0","p2","p3"],"-2/1"],[["l3","l3","l3","p0"],"1/1"],[["l3","l3","p
+> 0","p3"],"-1/1"],[["l3","p0","p0","p0"],"-1/1"],[["l3","p0","p1","p1"],"1/1"],[["l3","p0","p2","p2"],"-1/
+> 1"],[["l3","p0","p3","p3"],"1/1"]],"8,3":[[["l0","l0","l0","p2"],"1/1"],[["l0","l0","l2","p0"],"-1/1"],[[
+> "l0","l0","p0","p2"],"-1/1"],[["l0","l1","l1","p2"],"-1/1"],[["l0","l1","p1","p2"],"2/1"],[["l0","l2","l2

+> ","p2"],"-1/1"],[["l0","l2","p0","p0"],"2/1"],[["l0","l2","p1","p1"],"-2/1"],[["l0","l2","p3","p3"],"1/1"
+>],[["l0","l3","l3","p2"],"-1/1"],[["l0","l3","p2","p3"],"1/1"],[["l1","l1","l2","p0"],"1/1"],[["l1","l1",
+> "p0","p2"],"-1/1"],[["l2","l2","l2","p0"],"1/1"],[["l2","l2","p0","p2"],"-1/1"],[["l2","l3","l3","p0"],"1
+> /1"],[["l2","l3","p0","p3"],"-1/1"],[["l2","p0","p0","p0"],"-1/1"],[["l2","p0","p1","p1"],"1/1"],[["l2","
+> p0","p2","p2"],"1/1"],[["l2","p0","p3","p3"],"-1/1"],[["l3","l3","p0","p2"],"2/1"],[["l3","p0","p2","p3"]
+> ,"-2/1"]],"8,4":[[["l0","l0","l2","p3"],"-1/2"],[["l0","l0","l3","p2"],"-1/2"],[["l0","l0","p2","p3"],"1/
+> 1"],[["l0","l2","p0","p3"],"-1/2"],[["l0","l3","p0","p2"],"-1/2"],[["l1","l1","l2","p3"],"1/2"],[["l1","l
+> 1","l3","p2"],"1/2"],[["l1","l1","p2","p3"],"1/1"],[["l1","l2","p1","p3"],"-5/2"],[["l1","l3","p1","p2"],
+> "-5/2"],[["l2","l2","l2","p3"],"1/2"],[["l2","l2","l3","p2"],"1/2"],[["l2","l2","p2","p3"],"-1/2"],[["l2"
+> ,"l3","l3","p3"],"1/2"],[["l2","l3","p0","p0"],"1/1"],[["l2","l3","p1","p1"],"1/1"],[["l2","l3","p2","p2"
+>],"-1/2"],[["l2","l3","p3","p3"],"-1/2"],[["l2","p1","p1","p3"],"1/1"],[["l3","l3","l3","p2"],"1/2"],[["l
+> 3","l3","p2","p3"],"-1/2"],[["l3","p1","p1","p2"],"1/1"]],"8,5":[[["l0","l0","l1","p3"],"-1/1"],[["l0","l
+> 0","l3","p1"],"1/1"],[["l0","l0","p1","p3"],"-1/1"],[["l0","l1","p0","p3"],"2/1"],[["l1","l1","l1","p3"],
+> "1/1"],[["l1","l1","l3","p1"],"-1/1"],[["l1","l1","p1","p3"],"-1/1"],[["l1","l2","l2","p3"],"1/1"],[["l1"
+> ,"l2","p2","p3"],"-1/1"],[["l1","l3","l3","p3"],"1/1"],[["l1","l3","p0","p0"],"-2/1"],[["l1","l3","p1","p
+> 1"],"2/1"],[["l1","l3","p2","p2"],"-1/1"],[["l2","l2","l3","p1"],"-1/1"],[["l2","l2","p1","p3"],"-2/1"],[
+> ["l2","l3","p1","p2"],"1/1"],[["l2","p1","p2","p3"],"2/1"],[["l3","l3","l3","p1"],"-1/1"],[["l3","l3","p1
+> ","p3"],"1/1"],[["l3","p0","p0","p1"],"1/1"],[["l3","p1","p1","p1"],"-1/1"],[["l3","p1","p2","p2"],"1/1"]
+> ,[["l3","p1","p3","p3"],"-1/1"]],"8,6":[[["l0","l0","l1","p2"],"-1/1"],[["l0","l0","l2","p1"],"1/1"],[["l
+> 0","l0","p1","p2"],"-1/1"],[["l0","l1","p0","p2"],"2/1"],[["l1","l1","l1","p2"],"1/1"],[["l1","l1","l2","
+> p1"],"-1/1"],[["l1","l1","p1","p2"],"-1/1"],[["l1","l2","l2","p2"],"1/1"],[["l1","l2","p0","p0"],"-2/1"],
+> [["l1","l2","p1","p1"],"2/1"],[["l1","l2","p3","p3"],"-1/1"],[["l1","l3","l3","p2"],"1/1"],[["l1","l3","p
+> 2","p3"],"-1/1"],[["l2","l2","l2","p1"],"-1/1"],[["l2","l2","p1","p2"],"1/1"],[["l2","l3","l3","p1"],"-1/
+> 1"],[["l2","l3","p1","p3"],"1/1"],[["l2","p0","p0","p1"],"1/1"],[["l2","p1","p1","p1"],"-1/1"],[["l2","p1
+> ","p2","p2"],"-1/1"],[["l2","p1","p3","p3"],"1/1"],[["l3","l3","p1","p2"],"-2/1"],[["l3","p1","p2","p3"],
+> "2/1"]],"8,7":[[["l0","l0","l2","p3"],"-3/2"],[["l0","l0","l3","p2"],"1/2"],[["l0","l2","p0","p3"],"3/2"]
+> ,[["l0","l3","p0","p2"],"-1/2"],[["l1","l1","l2","p3"],"3/2"],[["l1","l1","l3","p2"],"-1/2"],[["l1","l2",
+> "p1","p3"],"-3/2"],[["l1","l3","p1","p2"],"1/2"],[["l2","l2","l2","p3"],"3/2"],[["l2","l2","l3","p2"],"-1
+> /2"],[["l2","l2","p2","p3"],"-5/2"],[["l2","l3","l3","p3"],"3/2"],[["l2","l3","p0","p0"],"-1/1"],[["l2","
+> l3","p1","p1"],"1/1"],[["l2","l3","p2","p2"],"1/2"],[["l2","l3","p3","p3"],"-1/2"],[["l2","p2","p2","p3"]
+> ,"1/1"],[["l3","l3","l3","p2"],"-1/2"],[["l3","l3","p2","p3"],"1/2"],[["l3","p0","p0","p2"],"1/1"],[["l3"
+> ,"p1","p1","p2"],"-1/1"],[["l3","p2","p3","p3"],"-1/1"]],"8,8":[[["l0","l0","l0","l0"],"2/1"],[["l0","l0"
+> ,"l0","p0"],"-4/1"],[["l0","l0","l1","l1"],"-4/1"],[["l0","l0","l1","p1"],"4/1"],[["l0","l0","l2","l2"],"
+> -4/1"],[["l0","l0","l2","p2"],"4/1"],[["l0","l0","l3","l3"],"-4/1"],[["l0","l0","l3","p3"],"4/1"],[["l0",
+> "l0","p0","p0"],"2/1"],[["l0","l0","p2","p2"],"-1/1"],[["l0","l0","p3","p3"],"-1/1"],[["l0","l1","l1","p0
+> "],"4/1"],[["l0","l1","p0","p1"],"-4/1"],[["l0","l2","l2","p0"],"4/1"],[["l0","l2","p0","p2"],"-2/1"],[["
+> l0","l3","l3","p0"],"4/1"],[["l0","l3","p0","p3"],"-2/1"],[["l1","l1","l1","l1"],"2/1"],[["l1","l1","l1",
+> "p1"],"-4/1"],[["l1","l1","l2","l2"],"4/1"],[["l1","l1","l2","p2"],"-4/1"],[["l1","l1","l3","l3"],"4/1"],
+> [["l1","l1","l3","p3"],"-4/1"],[["l1","l1","p1","p1"],"2/1"],[["l1","l1","p2","p2"],"1/1"],[["l1","l1","p
+> 3","p3"],"1/1"],[["l1","l2","l2","p1"],"-4/1"],[["l1","l2","p1","p2"],"2/1"],[["l1","l3","l3","p1"],"-4/1
+> "],[["l1","l3","p1","p3"],"2/1"],[["l2","l2","l2","l2"],"2/1"],[["l2","l2","l2","p2"],"-4/1"],[["l2","l2"
+> ,"l3","l3"],"4/1"],[["l2","l2","l3","p3"],"-4/1"],[["l2","l2","p0","p0"],"-2/1"],[["l2","l2","p1","p1"],"
+> 2/1"],[["l2","l2","p2","p2"],"3/1"],[["l2","l3","l3","p2"],"-4/1"],[["l2","l3","p2","p3"],"2/1"],[["l2","
+> p0","p0","p2"],"1/1"],[["l2","p1","p1","p2"],"-1/1"],[["l2","p2","p2","p2"],"-1/1"],[["l2","p2","p3","p3"
+>],"1/1"],[["l3","l3","l3","l3"],"2/1"],[["l3","l3","l3","p3"],"-4/1"],[["l3","l3","p0","p0"],"-2/1"],[["l
+> 3","l3","p1","p1"],"2/1"],[["l3","l3","p3","p3"],"3/1"],[["l3","p0","p0","p3"],"1/1"],[["l3","p1","p1","p
+> 3"],"-1/1"],[["l3","p2","p2","p3"],"1/1"],[["l3","p3","p3","p3"],"-1/1"]],"8,9":[[["l0","l0","l2","p3"],"
+> 1/2"],[["l0","l0","l3","p2"],"-3/2"],[["l0","l2","p0","p3"],"-1/2"],[["l0","l3","p0","p2"],"3/2"],[["l1",
+> "l1","l2","p3"],"-1/2"],[["l1","l1","l3","p2"],"3/2"],[["l1","l2","p1","p3"],"1/2"],[["l1","l3","p1","p2"
+>],"-3/2"],[["l2","l2","l2","p3"],"-1/2"],[["l2","l2","l3","p2"],"3/2"],[["l2","l2","p2","p3"],"1/2"],[["l
+> 2","l3","l3","p3"],"-1/2"],[["l2","l3","p0","p0"],"-1/1"],[["l2","l3","p1","p1"],"1/1"],[["l2","l3","p2",
+> "p2"],"-1/2"],[["l2","l3","p3","p3"],"1/2"],[["l2","p0","p0","p3"],"1/1"],[["l2","p1","p1","p3"],"-1/1"],
+> [["l2","p2","p2","p3"],"-1/1"],[["l3","l3","l3","p2"],"3/2"],[["l3","l3","p2","p3"],"-5/2"],[["l3","p2","
+> p3","p3"],"1/1"]],"9,0":[[["l0","l0","l0","p0"],"1/2"],[["l0","l0","l3","p3"],"1/2"],[["l0","l0","p0","p0
+> "],"-3/4"],[["l0","l0","p1","p1"],"-1/2"],[["l0","l0","p2","p2"],"-1/2"],[["l0","l1","l1","p0"],"-1/2"],[
+> ["l0","l1","p0","p1"],"1/1"],[["l0","l2","l2","p0"],"-1/2"],[["l0","l2","p0","p2"],"1/1"],[["l0","l3","l3
+> ","p0"],"-1/2"],[["l0","l3","p0","p3"],"-3/2"],[["l0","p0","p0","p0"],"1/4"],[["l0","p0","p1","p1"],"1/4"
+>],[["l0","p0","p2","p2"],"1/4"],[["l0","p0","p3","p3"],"1/4"],[["l1","l1","l3","p3"],"-1/2"],[["l1","l1",
+> "p1","p1"],"1/4"],[["l1","l1","p3","p3"],"1/2"],[["l1","l2","p1","p2"],"1/2"],[["l1","p0","p0","p1"],"-1/
+> 4"],[["l1","p1","p1","p1"],"-1/4"],[["l1","p1","p2","p2"],"-1/4"],[["l1","p1","p3","p3"],"-1/4"],[["l2","
+> l2","l3","p3"],"-1/2"],[["l2","l2","p2","p2"],"1/4"],[["l2","l2","p3","p3"],"1/2"],[["l2","p0","p0","p2"]
+> ,"-1/4"],[["l2","p1","p1","p2"],"-1/4"],[["l2","p2","p2","p2"],"-1/4"],[["l2","p2","p3","p3"],"-1/4"],[["
+> l3","l3","l3","p3"],"-1/2"],[["l3","l3","p0","p0"],"1/2"],[["l3","l3","p1","p1"],"1/2"],[["l3","l3","p2",
+> "p2"],"1/2"],[["l3","l3","p3","p3"],"3/4"],[["l3","p0","p0","p3"],"3/4"],[["l3","p1","p1","p3"],"-1/4"],[
+> ["l3","p2","p2","p3"],"-1/4"],[["l3","p3","p3","p3"],"-1/4"]],"9,1":[[["l0","l0","l0","p1"],"-1/2"],[["l0
+> ","l0","l1","p0"],"-1/2"],[["l0","l0","p0","p1"],"3/2"],[["l0","l1","l1","p1"],"1/2"],[["l0","l1","p0","p
+> 0"],"1/2"],[["l0","l1","p1","p1"],"-1/2"],[["l0","l1","p2","p2"],"1/1"],[["l0","l1","p3","p3"],"-1/1"],[[
+> "l0","l2","l2","p1"],"1/2"],[["l0","l2","p1","p2"],"-3/2"],[["l0","l3","l3","p1"],"1/2"],[["l0","l3","p1"
+> ,"p3"],"3/2"],[["l0","p0","p0","p1"],"-1/1"],[["l1","l1","l1","p0"],"1/2"],[["l1","l1","p0","p1"],"-3/2"]
+> ,[["l1","l2","l2","p0"],"1/2"],[["l1","l2","p0","p2"],"-3/2"],[["l1","l3","l3","p0"],"1/2"],[["l1","l3","
+> p0","p3"],"3/2"],[["l1","p0","p1","p1"],"1/1"],[["l2","p0","p1","p2"],"1/1"],[["l3","l3","p0","p1"],"-2/1
+> "],[["l3","p0","p1","p3"],"-1/1"]],"9,2":[[["l0","l0","l0","p2"],"-1/2"],[["l0","l0","l2","p0"],"-1/2"],[
+> ["l0","l0","p0","p2"],"3/2"],[["l0","l1","l1","p2"],"1/2"],[["l0","l1","p1","p2"],"-3/2"],[["l0","l2","l2
+> ","p2"],"1/2"],[["l0","l2","p0","p0"],"1/2"],[["l0","l2","p1","p1"],"1/1"],[["l0","l2","p2","p2"],"-1/2"]
+> ,[["l0","l2","p3","p3"],"-1/1"],[["l0","l3","l3","p2"],"1/2"],[["l0","l3","p2","p3"],"3/2"],[["l0","p0","
+> p0","p2"],"-1/1"],[["l1","l1","l2","p0"],"1/2"],[["l1","l2","p0","p1"],"-3/2"],[["l1","p0","p1","p2"],"1/
+> 1"],[["l2","l2","l2","p0"],"1/2"],[["l2","l2","p0","p2"],"-3/2"],[["l2","l3","l3","p0"],"1/2"],[["l2","l3
+> ","p0","p3"],"3/2"],[["l2","p0","p2","p2"],"1/1"],[["l3","l3","p0","p2"],"-2/1"],[["l3","p0","p2","p3"],"
+> -1/1"]],"9,3":[[["l0","l0","l0","p3"],"1/2"],[["l0","l0","l3","p0"],"-3/2"],[["l0","l0","p0","p3"],"1/2"]
+> ,[["l0","l1","l1","p3"],"-1/2"],[["l0","l1","p1","p3"],"1/2"],[["l0","l2","l2","p3"],"-1/2"],[["l0","l2",
+> "p2","p3"],"1/2"],[["l0","l3","l3","p3"],"-1/2"],[["l0","l3","p0","p0"],"5/2"],[["l0","l3","p1","p1"],"-1

```
  +> /1"],[["l0","l3","p2","p2"],"-1/1"],[["l0","l3","p3","p3"],"1/2"],[["l0","p0","p0","p3"],"-1/1"],[["l1","
  +> l1","l3","p0"],"3/2"],[["l1","l1","p0","p3"],"-1/1"],[["l1","l3","p0","p1"],"-3/2"],[["l1","p0","p1","p3"
  +> ],"1/1"],[["l2","l2","l3","p0"],"3/2"],[["l2","l2","p0","p3"],"-1/1"],[["l2","l3","p0","p2"],"-3/2"],[["l
  +> 2","p0","p2","p3"],"1/1"],[["l3","l3","l3","p0"],"3/2"],[["l3","l3","p0","p3"],"-3/2"],[["l3","p0","p0","
  +> p0"],"-1/1"],[["l3","p0","p1","p1"],"1/1"],[["l3","p0","p2","p2"],"1/1"]],"9,4":[[["l0","l0","l1","p1"],"
  +> 1/2"],[["l0","l0","l3","p3"],"-1/2"],[["l0","l0","p0","p0"],"-1/4"],[["l0","l0","p3","p3"],"1/2"],[["l0",
  +> "l1","p0","p1"],"-1/1"],[["l0","l2","p0","p2"],"1/2"],[["l0","p0","p0","p0"],"1/4"],[["l0","p0","p1","p1"
  +> ],"1/4"],[["l0","p0","p2","p2"],"-1/4"],[["l0","p0","p3","p3"],"-1/4"],[["l1","l1","l1","p1"],"-1/2"],[["
  +> l1","l1","l3","p3"],"1/2"],[["l1","l1","p0","p0"],"1/2"],[["l1","l1","p1","p1"],"3/4"],[["l1","l1","p2","
  +> p2"],"-1/2"],[["l1","l2","l2","p1"],"-1/2"],[["l1","l2","p1","p2"],"1/1"],[["l1","l3","l3","p1"],"-1/2"],
  +> [["l1","l3","p1","p3"],"-3/2"],[["l1","p0","p0","p1"],"-1/4"],[["l1","p1","p1","p1"],"-1/4"],[["l1","p1",
  +> "p2","p2"],"1/4"],[["l1","p1","p3","p3"],"1/4"],[["l2","l2","l3","p3"],"1/2"],[["l2","l2","p2","p2"],"-1/
  +> 4"],[["l2","l2","p3","p3"],"-1/2"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"-1/4"],[["l2","
  +> p2","p2","p2"],"1/4"],[["l2","p2","p3","p3"],"1/4"],[["l3","l3","l3","p3"],"1/2"],[["l3","l3","p0","p0"],
  +> "1/2"],[["l3","l3","p1","p1"],"1/2"],[["l3","l3","p2","p2"],"-1/2"],[["l3","l3","p3","p3"],"-3/4"],[["l3"
  +> ,"p0","p0","p3"],"-1/4"],[["l3","p1","p1","p3"],"3/4"],[["l3","p2","p2","p3"],"1/4"],[["l3","p3","p3","p3
  +> "],"1/4"]],"9,5":[[["l0","l0","l1","p2"],"1/2"],[["l0","l0","l2","p1"],"1/2"],[["l0","l1","p0","p2"],"-3/
  +> 2"],[["l0","l2","p0","p1"],"-3/2"],[["l0","p0","p1","p2"],"1/1"],[["l1","l1","l1","p2"],"-1/2"],[["l1","l
  +> 1","l2","p1"],"-1/2"],[["l1","l1","p1","p2"],"3/2"],[["l1","l2","l2","p2"],"-1/2"],[["l1","l2","p0","p0"]
  +> ,"1/1"],[["l1","l2","p1","p1"],"1/2"],[["l1","l2","p2","p2"],"1/2"],[["l1","l2","p3","p3"],"1/1"],[["l1",
  +> "l3","l3","p2"],"-1/2"],[["l1","l3","p2","p3"],"-3/2"],[["l1","p1","p1","p2"],"-1/1"],[["l2","l2","l2","p
  +> 1"],"-1/2"],[["l2","l2","p1","p2"],"3/2"],[["l2","l3","l3","p1"],"-1/2"],[["l2","l3","p1","p3"],"-3/2"],[
  +> ["l2","p1","p2","p2"],"-1/1"],[["l3","l3","p1","p2"],"2/1"],[["l3","p1","p2","p3"],"1/1"]],"9,6":[[["l0",
  +> "l0","l1","p3"],"-1/2"],[["l0","l0","l3","p1"],"3/2"],[["l0","l0","p1","p3"],"-1/1"],[["l0","l1","p0","p3
  +> "],"1/2"],[["l0","l3","p0","p1"],"-3/2"],[["l0","p0","p1","p3"],"1/1"],[["l1","l1","l1","p3"],"1/2"],[["l
  +> 1","l1","l3","p1"],"-3/2"],[["l1","l1","p1","p3"],"1/2"],[["l1","l2","l2","p3"],"1/2"],[["l1","l2","p2","
  +> p3"],"-1/2"],[["l1","l3","l3","p3"],"1/2"],[["l1","l3","p0","p0"],"-1/1"],[["l1","l3","p1","p1"],"5/2"],[
  +> ["l1","l3","p2","p2"],"1/1"],[["l1","l3","p3","p3"],"-1/2"],[["l1","p1","p1","p3"],"-1/1"],[["l2","l2","l
  +> 3","p1"],"-3/2"],[["l2","l2","p1","p3"],"1/1"],[["l2","l3","p1","p2"],"3/2"],[["l2","p1","p2","p3"],"-1/1
  +> "],[["l3","l3","l3","p1"],"-3/2"],[["l3","l3","p1","p3"],"3/2"],[["l3","p0","p0","p1"],"1/1"],[["l3","p1"
  +> ,"p1","p1"],"-1/1"],[["l3","p1","p2","p2"],"-1/1"]],"9,7":[[["l0","l0","l2","p2"],"1/2"],[["l0","l0","l3"
  +> ,"p3"],"-1/2"],[["l0","l0","p0","p0"],"-1/4"],[["l0","l0","p3","p3"],"1/2"],[["l0","l1","p0","p1"],"1/2"]
  +> ,[["l0","l2","p0","p2"],"-1/1"],[["l0","p0","p0","p0"],"1/4"],[["l0","p0","p1","p1"],"-1/4"],[["l0","p0",
  +> "p2","p2"],"1/4"],[["l0","p0","p3","p3"],"-1/4"],[["l1","l1","l2","p2"],"-1/2"],[["l1","l1","l3","p3"],"1
  +> /2"],[["l1","l1","p1","p1"],"-1/4"],[["l1","l1","p3","p3"],"-1/2"],[["l1","l2","p1","p2"],"1/1"],[["l1","
  +> p0","p0","p1"],"-1/4"],[["l1","p1","p1","p1"],"1/4"],[["l1","p1","p2","p2"],"-1/4"],[["l1","p1","p3","p3"
  +> ],"1/4"],[["l2","l2","l2","p2"],"-1/2"],[["l2","l2","l3","p3"],"1/2"],[["l2","l2","p0","p0"],"1/2"],[["l2
  +> ","l2","p1","p1"],"-1/2"],[["l2","l2","p2","p2"],"3/4"],[["l2","l3","l3","p2"],"-1/2"],[["l2","l3","p2","
  +> p3"],"-3/2"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"1/4"],[["l2","p2","p2","p2"],"-1/4"],
  +> [["l2","p2","p3","p3"],"1/4"],[["l3","l3","l3","p3"],"1/2"],[["l3","l3","p0","p0"],"1/2"],[["l3","l3","p1
  +> ","p1"],"-1/2"],[["l3","l3","p2","p2"],"1/2"],[["l3","l3","p3","p3"],"-3/4"],[["l3","p0","p0","p3"],"-1/4
  +> "],[["l3","p1","p1","p3"],"1/4"],[["l3","p2","p2","p3"],"3/4"],[["l3","p3","p3","p3"],"1/4"]],"9,8":[[["l
  +> 0","l0","l2","p3"],"-1/2"],[["l0","l0","l3","p2"],"3/2"],[["l0","l0","p2","p3"],"-1/1"],[["l0","l2","p0",
  +> "p3"],"1/2"],[["l0","l3","p0","p2"],"-3/2"],[["l0","p0","p2","p3"],"1/1"],[["l1","l1","l2","p3"],"1/2"],[
  +> ["l1","l1","l3","p2"],"-3/2"],[["l1","l1","p2","p3"],"1/1"],[["l1","l2","p1","p3"],"-1/2"],[["l1","l3","p
  +> 1","p2"],"3/2"],[["l1","p1","p2","p3"],"-1/1"],[["l2","l2","l2","p3"],"1/2"],[["l2","l2","l3","p2"],"-3/2
  +> "],[["l2","l2","p2","p3"],"1/2"],[["l2","l3","l3","p3"],"1/2"],[["l2","l3","p0","p0"],"-1/1"],[["l2","l3"
  +> ,"p1","p1"],"1/1"],[["l2","l3","p2","p2"],"5/2"],[["l2","l3","p3","p3"],"-1/2"],[["l2","p2","p2","p3"],"-
  +> 1/1"],[["l3","l3","l3","p2"],"-3/2"],[["l3","l3","p2","p3"],"3/2"],[["l3","p0","p0","p2"],"1/1"],[["l3","
  +> p1","p1","p2"],"-1/1"],[["l3","p2","p2","p2"],"-1/1"]],"9,9":[[["l0","l0","l0","l0"],"1/1"],[["l0","l0","
  +> l0","p0"],"-2/1"],[["l0","l0","l1","l1"],"-2/1"],[["l0","l0","l1","p1"],"2/1"],[["l0","l0","l2","l2"],"-2
  +> /1"],[["l0","l0","l2","p2"],"2/1"],[["l0","l0","l3","l3"],"-2/1"],[["l0","l0","l3","p3"],"2/1"],[["l0","l
  +> 0","p0","p0"],"3/4"],[["l0","l0","p3","p3"],"-1/2"],[["l0","l1","l1","p0"],"2/1"],[["l0","l1","p0","p1"],
  +> "-3/2"],[["l0","l2","l2","p0"],"2/1"],[["l0","l2","p0","p2"],"-3/2"],[["l0","l3","l3","p0"],"2/1"],[["l0"
  +> ,"l3","p0","p3"],"-3/2"],[["l0","p0","p0","p0"],"1/4"],[["l0","p0","p1","p1"],"-1/4"],[["l0","p0","p2","p
  +> 2"],"-1/4"],[["l0","p0","p3","p3"],"1/4"],[["l1","l1","l1","l1"],"1/1"],[["l1","l1","l1","p1"],"-2/1"],[[
  +> "l1","l1","l2","l2"],"2/1"],[["l1","l1","l2","p2"],"-2/1"],[["l1","l1","l3","l3"],"2/1"],[["l1","l1","l3"
  +> ,"p3"],"-2/1"],[["l1","l1","p1","p1"],"3/4"],[["l1","l1","p3","p3"],"1/2"],[["l1","l2","l2","p1"],"-2/1"
  +> ,[["l1","l2","p1","p2"],"3/2"],[["l1","l3","l3","p1"],"-2/1"],[["l1","l3","p1","p3"],"3/2"],[["l1","p0","
  +> p0","p1"],"-1/4"],[["l1","p1","p1","p1"],"1/4"],[["l1","p1","p2","p2"],"1/4"],[["l1","p1","p3","p3"],"-1/
  +> 4"],[["l2","l2","l2","l2"],"1/1"],[["l2","l2","l2","p2"],"-2/1"],[["l2","l2","l3","l3"],"2/1"],[["l2","l2
  +> ","l3","p3"],"-2/1"],[["l2","l2","p2","p2"],"3/4"],[["l2","l2","p3","p3"],"1/2"],[["l2","l3","l3","p2"],"
  +> -2/1"],[["l2","l3","p2","p3"],"3/2"],[["l2","p0","p0","p2"],"-1/4"],[["l2","p1","p1","p2"],"1/4"],[["l2",
  +> "p2","p2","p2"],"1/4"],[["l2","p2","p3","p3"],"-1/4"],[["l3","l3","l3","l3"],"1/1"],[["l3","l3","l3","p3"
  +> ],"-2/1"],[["l3","l3","p0","p0"],"-1/1"],[["l3","l3","p1","p1"],"1/1"],[["l3","l3","p2","p2"],"1/1"],[["l
  +> 3","l3","p3","p3"],"5/4"],[["l3","p0","p0","p3"],"3/4"],[["l3","p1","p1","p3"],"-3/4"],[["l3","p2","p2","
  +> p3"],"-3/4"],[["l3","p3","p3","p3"],"-1/4"]]},"numerator_variables":"l0..l3, p0..p3 (lower)","prefactor":
  +> "-kappa**2","routing":{"A":["cbar_rho: -l","c^mu: l-p","h_a: p"],"B":["cbar_sigma: p-l","c^nu: l","h_b: -
  +> p"],"propagators":["<c^mu cbar_sigma>(l-p)","<c^nu cbar_rho>(l)"],"trace":"closed ghost loop"},"sha256":"
  +> 728cd2564ff9024d77e950c07642f4a287d59c0d9c6ff56f4de03d1ed7a978bc"}
```


74 Symbolic integrand D13 (p14r export; evaluated in p15)

[listing 41/133] exports/P14_integrands/D13.json


```
{"denominators":["l^2+i0","(p-l)^2+i0"],"external":["K'_(a')(-p)  [left]","c^nu(p)  [right]"],"external_compon
  +> ents":"independent h_(mn) index 0..9 = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (
  +> 2, 3), (3, 3)]; vector index 0..3","graph":"D13","integrated":false,"numerator":{"0,0":[["l0","l0","l0"]
```

```
+> ,"2/1"],[["l0","l0","p0"],"1/4"],[["l0","l1","p1"],"-3/2"],[["l0","l2","p2"],"-3/2"],[["l0","l3","p3"],"-
+> 3/2"],[["l0","p0","p0"],"-1/4"],[["l0","p1","p1"],"-1/4"],[["l0","p2","p2"],"-1/4"],[["l0","p3","p3"],"-1
+> /4"],[["l1","l1","p0"],"-1/4"],[["l2","l2","p0"],"-1/4"],[["l3","l3","p0"],"-1/4"]],"0,1":[[["l0","l0","l
+> 1"],"2/1"],[["l0","l0","p1"],"-5/4"],[["l0","p0","p1"],"-1/2"],[["l1","l1","p1"],"-1/4"],[["l1","l2","p2"
+> ],"-1/2"],[["l1","l3","p3"],"-1/2"],[["l1","p0","p0"],"-1/4"],[["l1","p1","p1"],"1/4"],[["l1","p2","p2"],
+> "1/4"],[["l1","p3","p3"],"1/4"],[["l2","l2","p1"],"1/4"],[["l3","l3","p1"],"1/4"]],"0,2":[[["l0","l0","l2
+> "],"2/1"],[["l0","l0","p2"],"-5/4"],[["l0","p0","p2"],"-1/2"],[["l1","l1","p2"],"1/4"],[["l1","l2","p1"],
+> "-1/2"],[["l2","l2","p2"],"-1/4"],[["l2","l3","p2"],"-1/2"],[["l2","p0","p0"],"-1/4"],[["l2","p1","p1"],"
+> 1/4"],[["l2","p2","p2"],"1/4"],[["l2","p3","p3"],"1/4"],[["l3","l3","p2"],"1/4"]],"0,3":[[["l0","l0","l3"
+> ],"2/1"],[["l0","l0","p3"],"-5/4"],[["l0","p0","p3"],"-1/2"],[["l1","l1","p3"],"1/4"],[["l1","l3","p1"],"
+> -1/2"],[["l2","l2","p3"],"1/4"],[["l2","l3","p2"],"-1/2"],[["l3","l3","p3"],"-1/4"],[["l3","p0","p0"],"-1
+> /4"],[["l3","p1","p1"],"1/4"],[["l3","p2","p2"],"1/4"],[["l3","p3","p3"],"1/4"]],"1,0":[[["l0","l0","l1"]
+> ,"2/1"],[["l0","l1","p0"],"-3/4"],[["l0","p0","p1"],"-1/4"],[["l1","l1","p1"],"-3/4"],[["l1","l2","p2"],"
+> -1/2"],[["l1","l3","p3"],"-1/2"],[["l1","p1","p1"],"-1/4"],[["l1","p2","p2"],"-1/4"],[["l1","p3","p3"],"-
+> 1/4"],[["l2","l2","p1"],"-1/4"],[["l3","l3","p1"],"-1/4"]],"1,1":[[["l0","l0","p0"],"-3/4"],[["l0","l1","
+> l1"],"2/1"],[["l0","l1","p1"],"-3/4"],[["l0","l2","p2"],"1/2"],[["l0","l3","p3"],"1/2"],[["l0","p0","p0"]
+> ,"-1/4"],[["l0","p2","p2"],"1/4"],[["l0","p3","p3"],"1/4"],[["l1","p0","p1"],"-1/4"],[["l2","l2","p0"],"1
+> /4"],[["l3","l3","p0"],"1/4"]],"1,2":[[["l0","l1","l2"],"2/1"],[["l0","l1","p2"],"-1/1"],[["l0","l2","p1"
+> ],"-1/4"],[["l0","p1","p2"],"-1/4"],[["l1","l2","p0"],"-1/4"],[["l1","p0","p2"],"-1/4"]],"1,3":[[["l0","l
+> 1","l3"],"2/1"],[["l0","l1","p3"],"-1/1"],[["l0","l3","p1"],"-1/4"],[["l0","p1","p3"],"-1/4"],[["l1","l3"
+> ,"p0"],"-1/4"],[["l1","p0","p3"],"-1/4"]],"2,0":[[["l0","l0","l2"],"2/1"],[["l0","l2","p0"],"-3/4"],[["l0
+> ","p0","p2"],"-1/4"],[["l1","l1","p2"],"-1/4"],[["l1","l2","p1"],"-1/2"],[["l2","l2","p2"],"-3/4"],[["l2"
+> ,"l3","p3"],"-1/2"],[["l2","p1","p1"],"-1/4"],[["l2","p2","p2"],"-1/4"],[["l2","p3","p3"],"-1/4"],[["l3",
+> "l3","p2"],"-1/4"]],"2,1":[[["l0","l1","l2"],"2/1"],[["l0","l1","p2"],"-1/4"],[["l0","l2","p1"],"-1/1"],[
+> ["l0","p1","p2"],"-1/4"],[["l1","l2","p0"],"-1/4"],[["l2","p0","p1"],"-1/4"]],"2,2":[[["l0","l0","p0"],"-
+> 3/4"],[["l0","l1","p1"],"1/2"],[["l0","l2","l2"],"2/1"],[["l0","l2","p2"],"-3/4"],[["l0","l3","p3"],"1/2"
+> ],[["l0","p0","p0"],"-1/4"],[["l0","p1","p1"],"1/4"],[["l0","p3","p3"],"1/4"],[["l1","l1","p0"],"1/4"],[[
+> "l2","p0","p2"],"-1/4"],[["l3","l3","p0"],"1/4"]],"2,3":[[["l0","l2","l3"],"2/1"],[["l0","l2","p3"],"-1/1
+> "],[["l0","l3","p2"],"-1/4"],[["l0","p2","p3"],"-1/4"],[["l2","l3","p0"],"-1/4"],[["l2","p0","p3"],"-1/4"
+> ]],"3,0":[[["l0","l0","l3"],"2/1"],[["l0","l3","p0"],"-3/4"],[["l0","p0","p3"],"-1/4"],[["l1","l1","p3"],
+> "-1/4"],[["l1","l3","p1"],"-1/2"],[["l2","l2","p3"],"-1/4"],[["l2","l3","p2"],"-1/2"],[["l3","l3","p3"],"
+> -3/4"],[["l3","p1","p1"],"-1/4"],[["l3","p2","p2"],"-1/4"],[["l3","p3","p3"],"-1/4"]],"3,1":[[["l0","l1",
+> "l3"],"2/1"],[["l0","l1","p3"],"-1/4"],[["l0","l3","p1"],"-1/1"],[["l0","p1","p3"],"-1/4"],[["l1","l3","p
+> 0"],"-1/4"],[["l3","p0","p1"],"-1/4"]],"3,2":[[["l0","l2","l3"],"2/1"],[["l0","l2","p3"],"-1/4"],[["l0","
+> l3","p2"],"-1/1"],[["l0","p2","p3"],"-1/4"],[["l2","l3","p0"],"-1/4"],[["l3","p0","p2"],"-1/4"]],"3,3":[[
+> ["l0","l0","p0"],"-3/4"],[["l0","l1","p1"],"1/2"],[["l0","l2","p2"],"1/2"],[["l0","l3","l3"],"2/1"],[["l0
+> ","l3","p3"],"-3/4"],[["l0","p0","p0"],"-1/4"],[["l0","p1","p1"],"1/4"],[["l0","p2","p2"],"1/4"],[["l1","
+> l1","p0"],"1/4"],[["l2","l2","p0"],"1/4"],[["l3","p0","p3"],"-1/4"]],"4,0":[[["l0","l0","p0"],"-1/4"],[["
+> l0","l1","l1"],"2/1"],[["l0","l2","p2"],"1/2"],[["l0","l3","p3"],"1/2"],[["l0","p0","p0"],"1/4"],[["l0","
+> p1","p1"],"-1/4"],[["l0","p2","p2"],"-1/4"],[["l0","p3","p3"],"-1/4"],[["l1","l1","p0"],"-5/4"],[["l1","p
+> 0","p1"],"-1/2"],[["l2","l2","p0"],"-1/4"],[["l3","l3","p0"],"-1/4"]],"4,1":[[["l0","l0","p1"],"-1/4"],[[
+> "l0","l1","p0"],"-3/2"],[["l1","l1","l1"],"2/1"],[["l1","l1","p1"],"1/4"],[["l1","l2","p2"],"3/2"],[["l1"
+> ,"l3","p3"],"3/2"],[["l1","p0","p0"],"-1/4"],[["l1","p1","p1"],"-1/4"],[["l1","p2","p2"],"1/4"],[["l1","p
+> 3","p3"],"1/4"],[["l2","l2","p1"],"1/4"],[["l3","l3","p1"],"1/4"]],"4,2":[[["l0","l0","p2"],"1/4"],[["l0"
+> ,"l2","p0"],"-1/2"],[["l1","l1","l2"],"2/1"],[["l1","l1","p2"],"-5/4"],[["l1","p1","p2"],"-1/2"],[["l2","
+> l2","p2"],"1/4"],[["l2","l3","p3"],"1/2"],[["l2","p0","p0"],"1/4"],[["l2","p1","p1"],"-1/4"],[["l2","p2",
+> "p2"],"-1/4"],[["l2","p3","p3"],"-1/4"],[["l3","l3","p2"],"-1/4"]],"4,3":[[["l0","l0","p3"],"1/4"],[["l0"
+> ,"l3","p0"],"-1/2"],[["l1","l1","l3"],"2/1"],[["l1","l1","p3"],"-5/4"],[["l1","p1","p3"],"-1/2"],[["l2","
+> l2","p3"],"-1/4"],[["l2","l3","p2"],"1/2"],[["l3","l3","p3"],"1/4"],[["l3","p0","p0"],"1/4"],[["l3","p1",
+> "p1"],"-1/4"],[["l3","p2","p2"],"-1/4"],[["l3","p3","p3"],"-1/4"]],"5,0":[[["l0","l1","l2"],"2/1"],[["l0"
+> ,"l1","p2"],"-1/4"],[["l0","l2","p1"],"-1/4"],[["l1","l2","p0"],"-1/1"],[["l1","p0","p2"],"-1/4"],[["l2",
+> "p0","p1"],"-1/4"]],"5,1":[[["l0","l0","p2"],"-1/4"],[["l0","l2","p0"],"-1/2"],[["l1","l1","l2"],"2/1"],[
+> ["l1","l2","p1"],"-3/4"],[["l1","p1","p2"],"-1/4"],[["l2","l2","p2"],"3/4"],[["l2","l3","p3"],"1/2"],[["l
+> 2","p0","p0"],"-1/4"],[["l2","p2","p2"],"1/4"],[["l2","p3","p3"],"1/4"],[["l3","l3","p2"],"1/4"]],"5,2":[
+> [["l0","l0","p1"],"-1/4"],[["l0","l1","p0"],"-1/2"],[["l1","l1","p1"],"3/4"],[["l1","l2","l2"],"2/1"],[["
+> l1","l2","p2"],"-3/4"],[["l1","l3","p3"],"1/2"],[["l1","p0","p0"],"-1/4"],[["l1","p1","p1"],"1/4"],[["l1"
+> ,"p3","p3"],"1/4"],[["l2","p1","p2"],"-1/4"],[["l3","l3","p1"],"1/4"]],"5,3":[[["l1","l2","l3"],"2/1"],[[
+> "l1","l2","p3"],"-1/1"],[["l1","l3","p2"],"-1/4"],[["l1","p2","p3"],"-1/4"],[["l2","l3","p1"],"-1/4"],[["
+> l2","p1","p3"],"-1/4"]],"6,0":[[["l0","l1","l3"],"2/1"],[["l0","l1","p3"],"-1/4"],[["l0","l3","p1"],"-1/4
+> "],[["l1","l3","p0"],"-1/1"],[["l1","p0","p3"],"-1/4"],[["l3","p0","p1"],"-1/4"]],"6,1":[[["l0","l0","p3"
+> ],"-1/4"],[["l0","l3","p0"],"-1/2"],[["l1","l1","l3"],"2/1"],[["l1","l3","p1"],"-3/4"],[["l1","p1","p3"],
+> "-1/4"],[["l2","l2","p3"],"1/4"],[["l2","l3","p2"],"1/2"],[["l3","l3","p3"],"3/4"],[["l3","p0","p0"],"-1/
+> 4"],[["l3","p2","p2"],"1/4"],[["l3","p3","p3"],"1/4"]],"6,2":[[["l1","l2","l3"],"2/1"],[["l1","l2","p3"],
+> "-1/4"],[["l1","l3","p2"],"-1/1"],[["l1","p2","p3"],"-1/4"],[["l2","l3","p1"],"-1/4"],[["l3","p1","p2"],"
+> -1/4"]],"6,3":[[["l0","l0","p1"],"-1/4"],[["l0","l1","p0"],"-1/2"],[["l1","l1","p1"],"3/4"],[["l1","l2","
+> p2"],"1/2"],[["l1","l3","l3"],"2/1"],[["l1","l3","p3"],"-3/4"],[["l1","p0","p0"],"-1/4"],[["l1","p1","p1"
+> ],"1/4"],[["l1","p2","p2"],"1/4"],[["l2","l2","p1"],"1/4"],[["l3","p1","p3"],"-1/4"]],"7,0":[[["l0","l0",
+> "p0"],"-1/4"],[["l0","l1","p1"],"1/2"],[["l0","l2","l2"],"2/1"],[["l0","l3","p3"],"1/2"],[["l0","p0","p0"
+> ],"1/4"],[["l0","p1","p1"],"-1/4"],[["l0","p2","p2"],"-1/4"],[["l0","p3","p3"],"-1/4"],[["l1","l1","p0"],
+> "-1/4"],[["l2","l2","p0"],"-5/4"],[["l2","p0","p2"],"-1/2"],[["l3","l3","p0"],"-1/4"]],"7,1":[[["l0","l0"
+> ,"p1"],"1/4"],[["l0","l1","p0"],"-1/2"],[["l1","l1","p1"],"1/4"],[["l1","l2","l2"],"2/1"],[["l1","l3","p3
+> "],"1/2"],[["l1","p0","p0"],"1/4"],[["l1","p1","p1"],"-1/4"],[["l1","p2","p2"],"-1/4"],[["l1","p3","p3"],
+> "-1/4"],[["l2","l2","p1"],"-5/4"],[["l2","p1","p2"],"-1/2"],[["l3","l3","p1"],"-1/4"]],"7,2":[[["l0","l0"
+> ,"p2"],"-1/4"],[["l0","l2","p0"],"-3/2"],[["l1","l1","p2"],"1/4"],[["l1","l2","p1"],"3/2"],[["l2","l2","l
+> 2"],"2/1"],[["l2","l2","p2"],"1/4"],[["l2","l3","p3"],"3/2"],[["l2","p0","p0"],"-1/4"],[["l2","p1","p1"],
+> "1/4"],[["l2","p2","p2"],"-1/4"],[["l2","p3","p3"],"1/4"],[["l3","l3","p2"],"1/4"]],"7,3":[[["l0","l0","p
+> 3"],"1/4"],[["l0","l3","p0"],"-1/2"],[["l1","l1","p3"],"-1/4"],[["l1","l3","p1"],"1/2"],[["l2","l2","l3"]
+> ,"2/1"],[["l2","l2","p3"],"-5/4"],[["l2","p2","p3"],"-1/2"],[["l3","l3","p3"],"1/4"],[["l3","p0","p0"],"1
+> /4"],[["l3","p1","p1"],"-1/4"],[["l3","p2","p2"],"-1/4"],[["l3","p3","p3"],"-1/4"]],"8,0":[[["l0","l2","l
+> 3"],"2/1"],[["l0","l2","p3"],"-1/4"],[["l0","l3","p2"],"-1/4"],[["l2","l3","p0"],"-1/1"],[["l2","p0","p3"
+> ],"-1/4"],[["l3","p0","p2"],"-1/4"]],"8,1":[[["l1","l2","l3"],"2/1"],[["l1","l2","p3"],"-1/4"],[["l1","l3
```

```
  +> ","p2"],"-1/4"],[["l2","l3","p1"],"-1/1"],[["l2","p1","p3"],"-1/4"],[["l3","p1","p2"],"-1/4"]],"8,2":[[["
  +> l0","l0","p3"],"-1/4"],[["l0","l3","p0"],"-1/2"],[["l1","l1","p3"],"1/4"],[["l1","l3","p1"],"1/2"],[["l2"
  +> ,"l2","l3"],"2/1"],[["l2","l3","p2"],"-3/4"],[["l2","p2","p3"],"-1/4"],[["l3","l3","p3"],"3/4"],[["l3","p
  +> 0","p0"],"-1/4"],[["l3","p1","p1"],"1/4"],[["l3","p3","p3"],"1/4"]],"8,3":[[["l0","l0","p2"],"-1/4"],[["l
  +> 0","l2","p0"],"-1/2"],[["l1","l1","p2"],"1/4"],[["l1","l2","p1"],"1/2"],[["l2","l2","p2"],"3/4"],[["l2","
  +> l3","l3"],"2/1"],[["l2","l3","p3"],"-3/4"],[["l2","p0","p0"],"-1/4"],[["l2","p1","p1"],"1/4"],[["l2","p2"
  +> ,"p2"],"1/4"],[["l3","p2","p3"],"-1/4"]],"9,0":[[["l0","l0","p0"],"-1/4"],[["l0","l1","p1"],"1/2"],[["l0"
  +> ,"l2","p2"],"1/2"],[["l0","l3","l3"],"2/1"],[["l0","p0","p0"],"1/4"],[["l0","p1","p1"],"-1/4"],[["l0","p2
  +> ","p2"],"-1/4"],[["l0","p3","p3"],"-1/4"],[["l1","l1","p0"],"-1/4"],[["l2","l2","p0"],"-1/4"],[["l3","l3"
  +> ,"p0"],"-5/4"],[["l3","p0","p3"],"-1/2"]],"9,1":[[["l0","l0","p1"],"1/4"],[["l0","l1","p0"],"-1/2"],[["l1
  +> ","l1","p1"],"1/4"],[["l1","l2","p2"],"1/2"],[["l1","l3","l3"],"2/1"],[["l1","p0","p0"],"1/4"],[["l1","p1
  +> ","p1"],"-1/4"],[["l1","p2","p2"],"-1/4"],[["l1","p3","p3"],"-1/4"],[["l2","l2","p1"],"-1/4"],[["l3","l3"
  +> ,"p1"],"-5/4"],[["l3","p1","p3"],"-1/2"]],"9,2":[[["l0","l0","p2"],"1/4"],[["l0","l2","p0"],"-1/2"],[["l1
  +> ","l1","p2"],"-1/4"],[["l1","l2","p1"],"1/2"],[["l2","l2","p2"],"1/4"],[["l2","l3","l3"],"2/1"],[["l2","p
  +> 0","p0"],"1/4"],[["l2","p1","p1"],"-1/4"],[["l2","p2","p2"],"-1/4"],[["l2","p3","p3"],"-1/4"],[["l3","l3"
  +> ,"p2"],"-5/4"],[["l3","p2","p3"],"-1/2"]],"9,3":[[["l0","l0","p3"],"-1/4"],[["l0","l3","p0"],"-3/2"],[["l
  +> 1","l1","p3"],"1/4"],[["l1","l3","p1"],"3/2"],[["l2","l2","p3"],"1/4"],[["l2","l3","p2"],"3/2"],[["l3","l
  +> 3","l3"],"2/1"],[["l3","l3","p3"],"1/4"],[["l3","p0","p0"],"-1/4"],[["l3","p1","p1"],"1/4"],[["l3","p2","
  +> p2"],"1/4"],[["l3","p3","p3"],"-1/4"]]},"numerator_variables":"l0..l3, p0..p3 (lower)","prefactor":"I*kap
  +> pa**2","routing":{"Gamma1_term":"Gamma_1 contains int K'_(a') G_{a' nu} c^nu, G = ordered extraction (K',
  +>  c)","K_vertex":["K'_(a'): -p","c^mu: l","h_c: p-l"],"ghost_vertex":["cbar_rho: -l","c^nu (external): p",
  +> "h_d: l-p"],"propagators":["<c^mu cbar_rho>(l)","<h_c h_d>(p-l)"]},"sha256":"42d999fbc4eb3aa01b57050fbe5f
  +> f97525976e56dca0e748bae6ca9f03c67026"}
```


75 Symbolic integrand D14a (p14r export; evaluated in p15)

[listing 42/133] exports/P14_integrands/D14a.json


```
{"denominators":["l^2+i0"],"external":["h_a(0)"],"external_components":"independent h_(mn) index 0..9 = [(0, 0
  +> ), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]; vector index 0..3","graph":"D
  +> 14a","integrated":false,"numerator":{"0":[[["l0","l0"],"-3/1"],[["l1","l1"],"1/1"],[["l2","l2"],"1/1"],[[
  +> "l3","l3"],"1/1"]],"1":[[["l0","l1"],"4/1"]],"2":[[["l0","l2"],"4/1"]],"3":[[["l0","l3"],"4/1"]],"4":[[["
  +> l0","l0"],"1/1"],[["l1","l1"],"-3/1"],[["l2","l2"],"-1/1"],[["l3","l3"],"-1/1"]],"5":[[["l1","l2"],"-4/1"
  +> ]],"6":[[["l1","l3"],"-4/1"]],"7":[[["l0","l0"],"1/1"],[["l1","l1"],"-1/1"],[["l2","l2"],"-3/1"],[["l3","
  +> l3"],"-1/1"]],"8":[[["l2","l3"],"-4/1"]],"9":[[["l0","l0"],"1/1"],[["l1","l1"],"-1/1"],[["l2","l2"],"-1/1
  +> "],[["l3","l3"],"-3/1"]]},"numerator_variables":"l0..l3, p0..p3 (lower)","prefactor":"-kappa/2","routing"
  +> :{"V3":["h_a: 0","h_c: l","h_d: -l"],"propagators":["<h_c h_d>(l)"]},"sha256":"e141db64f2ed3ef1cb09709b03
  +> b8bbaf6c55d29604cb6873e09d05387473e606"}
```


76 Symbolic integrand D14b (p14r export; evaluated in p15)

[listing 43/133] exports/P14_integrands/D14b.json


```
{"denominators":["l^2+i0"],"external":["h_a(0)"],"external_components":"independent h_(mn) index 0..9 = [(0, 0
  +> ), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]; vector index 0..3","graph":"D
  +> 14b","integrated":false,"numerator":{"0":[[["l0","l0"],"1/1"],[["l1","l1"],"-1/1"],[["l2","l2"],"-1/1"],[
  +> ["l3","l3"],"-1/1"]],"4":[[["l0","l0"],"-1/1"],[["l1","l1"],"1/1"],[["l2","l2"],"1/1"],[["l3","l3"],"1/1"
  +> ]],"7":[[["l0","l0"],"-1/1"],[["l1","l1"],"1/1"],[["l2","l2"],"1/1"],[["l3","l3"],"1/1"]],"9":[[["l0","l0
  +> "],"-1/1"],[["l1","l1"],"1/1"],[["l2","l2"],"1/1"],[["l3","l3"],"1/1"]]},"numerator_variables":"l0..l3, p
  +> 0..p3 (lower)","prefactor":"kappa","routing":{"V":["cbar_rho: -l","c^mu: l","h_a: 0"],"propagators":["<c^
  +> mu cbar_rho>(l)"]},"sha256":"630ff664eec930d2c3eab60f5782dc9a3d4151c85304549f315cd94ad68d94de"}
```


77 Symbolic integrand D14c (p14r export; evaluated in p15)

[listing 44/133] exports/P14_integrands/D14c.json


```
{"denominators":["l^2+i0"],"external":["h_a(0)"],"external_components":"independent h_(mn) index 0..9 = [(0, 0
  +> ), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]; vector index 0..3","graph":"D
  +> 14c","integrated":false,"numerator":{"0":[[["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"-1/2"],
  +> [["l3","l3"],"-1/2"]],"1":[[["l0","l1"],"2/1"]],"2":[[["l0","l2"],"2/1"]],"3":[[["l0","l3"],"2/1"]],"4":[
  +> [["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"1/2"],[["l3","l3"],"1/2"]],"5":[[["l1","l2"],"-2/
  +> 1"]],"6":[[["l1","l3"],"-2/1"]],"7":[[["l0","l0"],"-1/2"],[["l1","l1"],"1/2"],[["l2","l2"],"-1/2"],[["l3"
  +> ,"l3"],"1/2"]],"8":[[["l2","l3"],"-2/1"]],"9":[[["l0","l0"],"-1/2"],[["l1","l1"],"1/2"],[["l2","l2"],"1/2
  +> "],[["l3","l3"],"-1/2"]]},"numerator_variables":"l0..l3, p0..p3 (lower)","prefactor":"-kappa/2","routing"
  +> :{"V":["h_a: 0","phi: l","phi: -l"],"propagators":["<phi phi>(l)"]},"sha256":"26d8a239783f101b997843a91ba
  +> 11acaea453cab04a9fa1266974ad0e2a7f120"}
```


78 Symbolic integrand D02 (p14r export; evaluated in p15)

[listing 45/133] exports/P14_integrands/D02.json

```
{"denominators":["l^2+i0"],"external":["h_a(p)","h_b(-p)"],"external_components":"independent h_(mn) index 0..
  +> 9 = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]; vector index 0..3",
  +> "graph":"D02","integrated":false,"numerator":{"0,0":[["l0","l0"],"9/2"],[["l1","l1"],"-5/2"],[["l2","l2"
  +> ],"-5/2"],[["l3","l3"],"-5/2"]],"0,1":[["l0","l1"],"-2/1"]],"0,2":[["l0","l2"],"-2/1"]],"0,3":[["l0","
  +> l3"],"-2/1"]],"0,4":[["l0","l0"],"-1/2"],[["l1","l1"],"1/2"],[["l2","l2"],"-1/2"],[["l3","l3"],"-1/2"],[
  +> ["p2","p2"],"-4/1"],[["p3","p3"],"-4/1"]],"0,5":[["l1","l2"],"2/1"],[["p1","p2"],"8/1"]],"0,6":[["l1","
  +> l3"],"2/1"],[["p1","p3"],"8/1"]],"0,7":[["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"1/2"],[["
  +> l3","l3"],"-1/2"],[["p1","p1"],"-4/1"],[["p3","p3"],"-4/1"]],"0,8":[["l2","l3"],"2/1"],[["p2","p3"],"8/1
  +> "]],"0,9":[["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"-1/2"],[["l3","l3"],"1/2"],[["p1","p1"
  +> ],"-4/1"],[["p2","p2"],"-4/1"]],"1,0":[["l0","l1"],"-2/1"]],"1,1":[["l0","l0"],"-6/1"],[["l1","l1"],"6/
  +> 1"],[["l2","l2"],"6/1"],[["l3","l3"],"6/1"],[["p2","p2"],"8/1"],[["p3","p3"],"8/1"]],"1,2":[["p1","p2"],
  +> "-8/1"]],"1,3":[["p1","p3"],"-8/1"]],"1,4":[["l0","l1"],"2/1"]],"1,5":[["p0","p2"],"-8/1"]],"1,6":[["
  +> p0","p3"],"-8/1"]],"1,7":[["l0","l1"],"2/1"],[["p0","p1"],"8/1"]],"1,9":[["l0","l1"],"2/1"],[["p0","p1"
  +> ],"8/1"]],"2,0":[["l0","l2"],"-2/1"]],"2,1":[["p1","p2"],"-8/1"]],"2,2":[["l0","l0"],"-6/1"],[["l1","l
  +> 1"],"6/1"],[["l2","l2"],"6/1"],[["l3","l3"],"6/1"],[["p1","p1"],"8/1"],[["p3","p3"],"8/1"]],"2,3":[["p2"
  +> ,"p3"],"-8/1"]],"2,4":[["l0","l2"],"2/1"],[["p0","p2"],"8/1"]],"2,5":[["p0","p1"],"-8/1"]],"2,7":[["l0
  +> ","l2"],"2/1"]],"2,8":[["p0","p3"],"-8/1"]],"2,9":[["l0","l2"],"2/1"],[["p0","p2"],"8/1"]],"3,0":[["l0
  +> ","l3"],"-2/1"]],"3,1":[["p1","p3"],"-8/1"]],"3,2":[["p2","p3"],"-8/1"]],"3,3":[["l0","l0"],"-6/1"],[[
  +> "l1","l1"],"6/1"],[["l2","l2"],"6/1"],[["l3","l3"],"6/1"],[["p1","p1"],"8/1"],[["p2","p2"],"8/1"]],"3,4":
  +> [[["l0","l3"],"2/1"],[["p0","p3"],"8/1"]],"3,6":[["p0","p1"],"-8/1"]],"3,7":[["l0","l3"],"2/1"],[["p0",
  +> "p3"],"8/1"]],"3,8":[["p0","p2"],"-8/1"]],"3,9":[["l0","l3"],"2/1"]],"4,0":[["l0","l0"],"-1/2"],[["l1"
  +> ,"l1"],"1/2"],[["l2","l2"],"-1/2"],[["l3","l3"],"-1/2"],[["p2","p2"],"-4/1"],[["p3","p3"],"-4/1"]],"4,1":
  +> [[["l0","l1"],"2/1"]],"4,2":[["l0","l2"],"2/1"],[["p0","p2"],"8/1"]],"4,3":[["l0","l3"],"2/1"],[["p0","
  +> p3"],"8/1"]],"4,4":[["l0","l0"],"5/2"],[["l1","l1"],"-9/2"],[["l2","l2"],"-5/2"],[["l3","l3"],"-5/2"]],"
  +> 4,5":[["l1","l2"],"-2/1"]],"4,6":[["l1","l3"],"-2/1"]],"4,7":[["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"]
  +> ,[["l2","l2"],"-1/2"],[["l3","l3"],"1/2"],[["p0","p0"],"-4/1"],[["p3","p3"],"4/1"]],"4,8":[["l2","l3"],"
  +> -2/1"],[["p2","p3"],"-8/1"]],"4,9":[["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"1/2"],[["l3",
  +> "l3"],"-1/2"],[["p0","p0"],"-4/1"],[["p2","p2"],"4/1"]],"5,0":[["l1","l2"],"2/1"],[["p1","p2"],"8/1"]],"
  +> 5,1":[["p0","p2"],"-8/1"]],"5,2":[["p0","p1"],"-8/1"]],"5,4":[["l1","l2"],"-2/1"]],"5,5":[["l0","l0"]
  +> ,"6/1"],[["l1","l1"],"-6/1"],[["l2","l2"],"-6/1"],[["l3","l3"],"-6/1"],[["p0","p0"],"8/1"],[["p3","p3"],"
  +> -8/1"]],"5,6":[["p2","p3"],"8/1"]],"5,7":[["l1","l2"],"-2/1"]],"5,8":[["p1","p3"],"8/1"]],"5,9":[["l1
  +> ","l2"],"-2/1"],[["p1","p2"],"-8/1"]],"6,0":[["l1","l3"],"2/1"],[["p1","p3"],"8/1"]],"6,1":[["p0","p3"]
  +> ,"-8/1"]],"6,3":[["p0","p1"],"-8/1"]],"6,4":[["l1","l3"],"-2/1"]],"6,5":[["p2","p3"],"8/1"]],"6,6":[[[
  +> "l0","l0"],"6/1"],[["l1","l1"],"-6/1"],[["l2","l2"],"-6/1"],[["l3","l3"],"-6/1"],[["p0","p0"],"8/1"],[["p
  +> 2","p2"],"-8/1"]],"6,7":[["l1","l3"],"-2/1"],[["p1","p3"],"-8/1"]],"6,8":[["p1","p2"],"8/1"]],"6,9":[[[
  +> "l1","l3"],"-2/1"]],"7,0":[["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"1/2"],[["l3","l3"],"-1
  +> /2"],[["p1","p1"],"-4/1"],[["p3","p3"],"-4/1"]],"7,1":[["l0","l1"],"2/1"],[["p0","p1"],"8/1"]],"7,2":[[[
  +> "l0","l2"],"2/1"]],"7,3":[["l0","l3"],"2/1"],[["p0","p3"],"8/1"]],"7,4":[["l0","l0"],"-1/2"],[["l1","l1
  +> "],"-1/2"],[["l2","l2"],"-1/2"],[["l3","l3"],"1/2"],[["p0","p0"],"-4/1"],[["p3","p3"],"4/1"]],"7,5":[[["l
  +> 1","l2"],"-2/1"]],"7,6":[["l1","l3"],"-2/1"],[["p1","p3"],"-8/1"]],"7,7":[["l0","l0"],"5/2"],[["l1","l1
  +> "],"-5/2"],[["l2","l2"],"-9/2"],[["l3","l3"],"-5/2"]],"7,8":[["l2","l3"],"-2/1"]],"7,9":[["l0","l0"],"-
  +> 1/2"],[["l1","l1"],"1/2"],[["l2","l2"],"-1/2"],[["l3","l3"],"-1/2"],[["p0","p0"],"-4/1"],[["p1","p1"],"4/
  +> 1"]],"8,0":[["l2","l3"],"2/1"],[["p2","p3"],"8/1"]],"8,2":[["p0","p3"],"-8/1"]],"8,3":[["p0","p2"],"-8
  +> /1"]],"8,4":[["l2","l3"],"-2/1"],[["p2","p3"],"-8/1"]],"8,5":[["p1","p3"],"8/1"]],"8,6":[["p1","p2"],"
  +> 8/1"]],"8,7":[["l2","l3"],"-2/1"]],"8,8":[["l0","l0"],"6/1"],[["l1","l1"],"-6/1"],[["l2","l2"],"-6/1"],
  +> [["l3","l3"],"-6/1"],[["p0","p0"],"8/1"],[["p1","p1"],"-8/1"]],"8,9":[["l2","l3"],"-2/1"]],"9,0":[["l0"
  +> ,"l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"-1/2"],[["l3","l3"],"1/2"],[["p1","p1"],"-4/1"],[["p2",
  +> "p2"],"-4/1"]],"9,1":[["l0","l1"],"2/1"],[["p0","p1"],"8/1"]],"9,2":[["l0","l2"],"2/1"],[["p0","p2"],"8
  +> /1"]],"9,3":[["l0","l3"],"2/1"]],"9,4":[["l0","l0"],"-1/2"],[["l1","l1"],"-1/2"],[["l2","l2"],"1/2"],[[
  +> "l3","l3"],"-1/2"],[["p0","p0"],"-4/1"],[["p2","p2"],"4/1"]],"9,5":[["l1","l2"],"-2/1"],[["p1","p2"],"-8
  +> /1"]],"9,6":[["l1","l3"],"-2/1"]],"9,7":[["l0","l0"],"-1/2"],[["l1","l1"],"1/2"],[["l2","l2"],"-1/2"],[
  +> ["l3","l3"],"-1/2"],[["p0","p0"],"-4/1"],[["p1","p1"],"4/1"]],"9,8":[["l2","l3"],"-2/1"]],"9,9":[["l0",
  +> "l0"],"5/2"],[["l1","l1"],"-5/2"],[["l2","l2"],"-5/2"],[["l3","l3"],"-9/2"]]},"numerator_variables":"l0..
  +> l3, p0..p3 (lower)","prefactor":"-kappa**2/2","routing":{"V4":["h_a: p","h_b: -p","h_c: l","h_d: -l"],"pr
  +> opagators":["<h_c h_d>(l)"]},"sha256":"e37cb4152690018ec8ce2ac1a0539a29864528e65e06a9efe09441b14fb6a063"}
```


79 Symbolic integrand D12 (p14r export; evaluated in p15)

[listing 46/133] exports/P14_integrands/D12.json


```
{"denominators":["l^2+i0"],"external":["h_a(p)","h_b(-p)"],"external_components":"independent h_(mn) index 0..
  +> 9 = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]; vector index 0..3",
  +> "graph":"D12","integrated":false,"numerator":{"0,0":[["l0","l0"],"3/4"],[["l1","l1"],"1/4"],[["l2","l2"]
  +> ,"1/4"],[["l3","l3"],"1/4"]],"0,1":[["l0","l1"],"-1/1"]],"0,2":[["l0","l2"],"-1/1"]],"0,3":[["l0","l3"
  +> ],"-1/1"]],"0,4":[["l0","l0"],"1/4"],[["l1","l1"],"-1/4"],[["l2","l2"],"1/4"],[["l3","l3"],"1/4"]],"0,5
  +> :[["l1","l2"],"-1/1"]],"0,6":[["l1","l3"],"-1/1"]],"0,7":[["l0","l0"],"1/4"],[["l1","l1"],"1/4"],[["l2
  +> ","l2"],"-1/4"],[["l3","l3"],"1/4"]],"0,8":[["l2","l3"],"-1/1"]],"0,9":[["l0","l0"],"1/4"],[["l1","l1"]
  +> ,"1/4"],[["l2","l2"],"1/4"],[["l3","l3"],"-1/4"]],"1,0":[["l0","l1"],"-1/1"]],"1,1":[["l0","l0"],"-1/1"
  +> ],[["l1","l1"],"1/1"],[["l2","l2"],"-1/1"],[["l3","l3"],"-1/1"]],"1,2":[["l1","l2"],"2/1"]],"1,3":[["l1
  +> ","l3"],"2/1"]],"1,4":[["l0","l1"],"1/1"]],"1,5":[["l0","l2"],"2/1"]],"1,6":[["l0","l3"],"2/1"]],"1,7"
  +> :[["l0","l1"],"-1/1"]],"1,9":[["l0","l1"],"-1/1"]],"2,0":[["l0","l2"],"-1/1"]],"2,1":[["l1","l2"],"2/
  +> 1"]],"2,2":[["l0","l0"],"-1/1"],[["l1","l1"],"-1/1"],[["l2","l2"],"1/1"],[["l3","l3"],"-1/1"]],"2,3":[[[
  +> "l2","l3"],"2/1"]],"2,4":[["l0","l2"],"-1/1"]],"2,5":[["l0","l1"],"2/1"]],"2,7":[["l0","l2"],"1/1"]],"
  +> 2,8":[["l0","l3"],"2/1"]],"2,9":[["l0","l2"],"-1/1"]],"3,0":[["l0","l3"],"-1/1"]],"3,1":[["l1","l3"],"
  +> "2/1"]],"3,2":[["l2","l3"],"2/1"]],"3,3":[["l0","l0"],"-1/1"],[["l1","l1"],"-1/1"],[["l2","l2"],"-1/1"]
  +> ,[["l3","l3"],"1/1"]],"3,4":[["l0","l3"],"-1/1"]],"3,6":[["l0","l1"],"2/1"]],"3,7":[["l0","l3"],"-1/1"
  +> ]],"3,8":[["l0","l2"],"2/1"]],"3,9":[["l0","l3"],"1/1"]],"4,0":[["l0","l0"],"1/4"],[["l1","l1"],"-1/4"
  +> ],[["l2","l2"],"1/4"],[["l3","l3"],"1/4"]],"4,1":[["l0","l1"],"1/1"]],"4,2":[["l0","l2"],"-1/1"]],"4,3"
```

```
+> :[[["l0","l3"],"-1/1"]],"4,4":[[["l0","l0"],"-1/4"],[["l1","l1"],"-3/4"],[["l2","l2"],"1/4"],[["l3","l3"]
+> ,"1/4"]],"4,5":[[["l1","l2"],"-1/1"]],"4,6":[[["l1","l3"],"-1/1"]],"4,7":[[["l0","l0"],"1/4"],[["l1","l1"
+> ],"1/4"],[["l2","l2"],"1/4"],[["l3","l3"],"-1/4"]],"4,8":[[["l2","l3"],"1/1"]],"4,9":[[["l0","l0"],"1/4"]
+> ,[["l1","l1"],"1/4"],[["l2","l2"],"-1/4"],[["l3","l3"],"1/4"]],"5,0":[[["l1","l2"],"-1/1"]],"5,1":[[["l0"
+> ,"l2"],"2/1"]],"5,2":[[["l0","l1"],"2/1"]],"5,4":[[["l1","l2"],"-1/1"]],"5,5":[[["l0","l0"],"-1/1"],[["l1
+> ","l1"],"-1/1"],[["l2","l2"],"-1/1"],[["l3","l3"],"1/1"]],"5,6":[[["l2","l3"],"-2/1"]],"5,7":[[["l1","l2"
+> ],"-1/1"]],"5,8":[[["l1","l3"],"-2/1"]],"5,9":[[["l1","l2"],"1/1"]],"6,0":[[["l1","l3"],"-1/1"]],"6,1":[[
+> ["l0","l3"],"2/1"]],"6,3":[[["l0","l1"],"2/1"]],"6,4":[[["l1","l3"],"-1/1"]],"6,5":[[["l2","l3"],"-2/1"]]
+> ,"6,6":[[["l0","l0"],"-1/1"],[["l1","l1"],"-1/1"],[["l2","l2"],"1/1"],[["l3","l3"],"-1/1"]],"6,7":[[["l1"
+> ,"l3"],"1/1"]],"6,8":[[["l1","l2"],"-2/1"]],"6,9":[[["l1","l3"],"-1/1"]],"7,0":[[["l0","l0"],"1/4"],[["l1
+> ","l1"],"1/4"],[["l2","l2"],"-1/4"],[["l3","l3"],"1/4"]],"7,1":[[["l0","l1"],"-1/1"]],"7,2":[[["l0","l2"]
+> ,"1/1"]],"7,3":[[["l0","l3"],"-1/1"]],"7,4":[[["l0","l0"],"1/4"],[["l1","l1"],"1/4"],[["l2","l2"],"1/4"],
+> [["l3","l3"],"-1/4"]],"7,5":[[["l1","l2"],"-1/1"]],"7,6":[[["l1","l3"],"1/1"]],"7,7":[[["l0","l0"],"-1/4"
+> ],[["l1","l1"],"1/4"],[["l2","l2"],"-3/4"],[["l3","l3"],"1/4"]],"7,8":[[["l2","l3"],"-1/1"]],"7,9":[[["l0
+> ","l0"],"1/4"],[["l1","l1"],"-1/4"],[["l2","l2"],"1/4"],[["l3","l3"],"1/4"]],"8,0":[[["l2","l3"],"-1/1"]]
+> ,"8,2":[[["l0","l3"],"2/1"]],"8,3":[[["l0","l2"],"2/1"]],"8,4":[[["l2","l3"],"1/1"]],"8,5":[[["l1","l3"],
+> "-2/1"]],"8,6":[[["l1","l2"],"-2/1"]],"8,7":[[["l2","l3"],"-1/1"]],"8,8":[[["l0","l0"],"-1/1"],[["l1","l1
+> "],"1/1"],[["l2","l2"],"-1/1"],[["l3","l3"],"-1/1"]],"8,9":[[["l2","l3"],"-1/1"]],"9,0":[[["l0","l0"],"1/
+> 4"],[["l1","l1"],"1/4"],[["l2","l2"],"1/4"],[["l3","l3"],"-1/4"]],"9,1":[[["l0","l1"],"-1/1"]],"9,2":[[["
+> l0","l2"],"-1/1"]],"9,3":[[["l0","l3"],"1/1"]],"9,4":[[["l0","l0"],"1/4"],[["l1","l1"],"1/4"],[["l2","l2"
+> ],"-1/4"],[["l3","l3"],"1/4"]],"9,5":[[["l1","l2"],"1/1"]],"9,6":[[["l1","l3"],"-1/1"]],"9,7":[[["l0","l0
+> "],"1/4"],[["l1","l1"],"-1/4"],[["l2","l2"],"1/4"],[["l3","l3"],"1/4"]],"9,8":[[["l2","l3"],"-1/1"]],"9,9
+> ":[[["l0","l0"],"-1/4"],[["l1","l1"],"1/4"],[["l2","l2"],"1/4"],[["l3","l3"],"-3/4"]]},"numerator_variabl
+> es":"l0..l3, p0..p3 (lower)","prefactor":"-kappa**2/2","routing":{"V":["h_a: p","h_b: -p","phi: l","phi:
+> -l"],"propagators":["<phi phi>(l)"]},"sha256":"044080e17dc3acdfcbd2640995d831b69ab2d637712f27148fa60a6a2a
+> 99b6fa"}
```


80 Boundary identity certificate (C3)

[listing 47/133] certificates/P13_boundary_identity.json


```
{
  "revision": "p13",
  "gate": "C3_boundary_identity",
  "statement": "sqrt(-g) R - L_GG = - d_mu B^mu,  B^mu = sqrt(-g)(g^{mu nu} G^l_{nu l} - g^{l nu} G^mu_{l nu})
  +> ",
  "L_GG": "L_GG = sqrt(-g) g^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab})",
  "orientation_correction": "p09/p10 stated sqrt(-q)R - L_GG = + d_mu B^mu; correct is - d_mu B^mu",
  "old_C3_vacuity": {
    "support": "g = eta + A(x0) e_00 (p09/p10 C3)",
    "nonzero_Christoffels": ["G^0_00"],
    "sqrt_g_R": "0",
    "L_GG": "0",
    "B": ["0", "0", "0", "0"],
    "div_B": "0",
    "vacuous": true,
    "why": "locally flat: x0 -> int sqrt(1+A) dx0 removes A; Gamma^0_00 alone gives R = 0, both GammaGamma con
  +> tractions equal (Gamma^0_00)^2 g^00 and cancel, and B^0 = sqrt(-g)(g^00 G^0_00 - g^00 G^0_00) = 0. The p1
  +> 0 source comment 'keeps dB nontrivial' was false."
  },
  "conformal_control_correct_orientation": {
    "metric": "g = f(x0,x1,x2,x3) eta, generic f (all four coordinates), exact",
    "orientation": "sqrt(-g)R - L_GG + d_mu B^mu",
    "residual": "0",
    "residual_zero": true,
    "closed_forms_verified": {
      "sqrt(-g)R = -3 Box f + (3/(2f))(df)^2": true,
      "L_GG = (3/(2f))(df)^2": true,
      "B^mu = 3 d^mu f": true
    },
    "nontrivial": true
  },
  "conformal_control_old_orientation_fails": {
    "residual": "-6*Derivative(f(x0, x1, x2, x3), (x0, 2)) + 6*Derivative(f(x0, x1, x2, x3), (x1, 2)) + 6*Deri
  +> vative(f(x0, x1, x2, x3), (x2, 2)) + 6*Derivative(f(x0, x1, x2, x3), (x3, 2))",
    "fails": true,
    "equals": "-6 Box f"
  },
  "generic_jet_proof": {
    "variables": {
      "inverse_metric_free_symmetric": 10,
      "first_jets": 40,
      "second_jets": 100
    },
    "orientation": "+d_mu B^mu (correct)",
    "terms": {
      "R": 2406,
      "l": 1764,
```

```
      "div_b": 2442
    },
    "residual_terms": 0,
    "identically_zero": true,
    "holds_for_free_G": true,
    "seconds": 1.3
  },
  "generic_jet_proof_old_orientation": {
    "variables": {
      "inverse_metric_free_symmetric": 10,
      "first_jets": 40,
      "second_jets": 100
    },
    "orientation": "-d_mu B^mu (p09/p10)",
    "terms": {
      "R": 2406,
      "l": 1764,
      "div_b": 2442
    },
    "residual_terms": 2736,
    "identically_zero": false,
    "holds_for_free_G": false,
    "seconds": 0.94
  },
  "jacobi_lemma": {
    "derivative_of_det_equals_tr_adj": true,
    "adj_g_equals_det_one": true,
    "consequence": "d_mu sqrt(-g) = (1/2) sqrt(-g) g^{ab} d_mu g_ab ; d_c g^{ab} = - g^{ar} d_c g_rs g^{sb}"
  },
  "integrated_action_unaffected": "int d_mu B^mu = 0 under compact support for either sign, so S_EH = -(2/kapp
  +> a^2) int L_GG and ell_k = -2[eps^k]L_GG are unchanged",
  "evidence_kind": "deterministic exact symbolic (sympy; polynomial ring over QQ for (iii))",
  "C3_proved": true,
  "execution_pass": true,
  "claim_status": "proved",
  "scope": "exact non-perturbative identity on the jet space of a generic metric (any signature-compatible g)"
  +> ,
  "seconds": 3.4
}
```


81 Quartic verification certificate

[listing 48/133] certificates/P13_quartic_verified.json


```
{
  "revision": "p13",
  "gate": "quartic_generator_verified",
  "execution_pass": true,
  "claim_status": "verified",
  "scope": "classical EH quartic density ell_4 (Gamma-Gamma form, affine q = eta + kappa h) on declared conven
  +> tions",
  "quartic_generator_verified": true,
  "acceptance_controls_deterministic": {
    "C1": true,
    "C2": true,
    "C3": true,
    "C4": true,
    "C5": true,
    "C6": true,
    "C7": true
  },
  "all_acceptance_deterministic": true,
  "DO2_vertex_available": true,
  "DO2_quantum_evaluated": false,
  "ell2_sha256": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
  "ell3_sha256": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
  "ell4_sha256": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
  "terms_reproducibility": {
    "2": 84,
    "3": 678,
    "4": 3360
  },
  "NOT_acceptance_by_term_count": true,
  "build": {
    "2": {
      "sha256": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "terms_reproducibility": 84
    },
```

```
  "3": {
    "sha256": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
    "terms_reproducibility": 678
  },
  "4": {
    "sha256": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
    "terms_reproducibility": 3360
  },
  "ell0_zero": true,
  "ell1_zero": true,
  "build": "fresh build ell4_full_polynomial.expand_L_gamma_jet(order=4)",
  "seconds": 92.0
},
"conventions": {
  "revision": "p13",
  "eta": "diag(+,-,-,-)",
  "q": "eta + eps H, eps = kappa, H = h (lowered, symmetric); affine parametrisation",
  "S_EH": "-(2/kappa^2) int sqrt(-q) R",
  "density": "L_GG = sqrt(-g) g^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab})",
  "boundary_identity": "sqrt(-g) R - L_GG = - d_mu B^mu,  B^mu = sqrt(-g)(g^{mu nu} G^l_{nu l} - g^{l nu} G^
+> mu_{l nu})",
  "ell_k": "-2 [eps^k] L_GG ; S_EH = sum_k kappa^{k-2} int ell_k (integrated, compact support)",
  "IBP": "total derivatives dropped only inside the integrated action (orientation of the dropped divergence
+>  irrelevant)",
  "truncation": "O(eps^4) inclusive",
  "jets": "H{m}{n} = h_mn (m<=n), dH{m}{n}_{r} = d_r h_mn",
  "a_ghost": "1/2",
  "polarization": "V4[h1..h4] = [t1 t2 t3 t4] ell_4(sum t_i h_i) = D^4 ell_4(0)[h1..h4] (unnormalised); norm
+> alised symmetric form = V4/4!; Feynman vertex i kappa^2 V4; D02 graph symmetry factor 1/2"
},
"controls": {
  "C1_inverse": {
    "test": "q q^{-1} = 1 + O(eps^5), 10 symbolic components",
    "pass": true,
    "failures": []
  },
  "C2_det_tracelog_generic": {
    "test": "log(-det q) = tr log(eta^{-1} q) and sqrt-series^2 = -det q, through eps^4, 10 symbolic compone
+> nts",
    "logdet_equals_trlog": true,
    "sqrt_series_squared_equals_minus_det": true,
    "pass": true
  },
  "C3_boundary_identity": "P13_boundary_identity.json (C3_proved=True)",
  "C4_export_identity": {
    "test": "export == independent series evaluator over QQ[50 jet symbols] (polynomial identity)",
    "results": {
      "ell2": true,
      "ell3": true,
      "ell4": true
    },
    "pass": true
  },
  "C5_C6_regression": {
    "cubic": {
      "test": "E_h(ell3 - ell3_XIIC) = 0 for all 10 components (exact)",
      "difference_terms": 1530,
      "euler_terms": {
        "00": 0,
        "01": 0,
        "02": 0,
        "03": 0,
        "11": 0,
        "12": 0,
        "13": 0,
        "22": 0,
        "23": 0,
        "33": 0
      },
      "divergence": true,
      "NOT_XII_C_quartic": true,
      "NOT_Ward_identity": true
    },
    "quadratic": {
      "test": "E_h(ell2 - FP_XIA) = 0 for all 10 components (exact)",
      "difference_terms": 48,
      "euler_terms": {
        "00": 0,
        "01": 0,
        "02": 0,
        "03": 0,
```

```
        "11": 0,
        "12": 0,
        "13": 0,
        "22": 0,
        "23": 0,
        "33": 0
      },
      "divergence": true
    },
    "sign_discrimination": {
      "ell3 + ell3_XIIC is a divergence": false
    },
    "source": "XVI-N-r10/code/prep/grav_density.py",
    "source_sha256": "855ab9e2124acf66f4440b2f1745470acf137caf77dc7bd538d13b7c878ea8e0",
    "source_pin_ok": true
  },
  "C7_noether_exact": {
    "kappa0": {
      "density": "s0 ell2",
      "W_terms": 132,
      "Q_terms": [0, 0, 0, 0],
      "Q_identically_zero": true,
      "s0_piece_alone_Q_terms": [0, 0, 0, 0],
      "residual_head": []
    },
    "kappa1": {
      "density": "s0 ell3 + s1 ell2",
      "W_terms": 2148,
      "Q_terms": [0, 0, 0, 0],
      "Q_identically_zero": true,
      "s0_piece_alone_Q_terms": [159, 159, 159, 159],
      "residual_head": []
    },
    "kappa2": {
      "density": "s0 ell4 + s1 ell3 (+ s2 ell2, s2 = 0)",
      "W_terms": 15428,
      "Q_terms": [0, 0, 0, 0],
      "Q_identically_zero": true,
      "s0_piece_alone_Q_terms": [2088, 2088, 2088, 2088],
      "residual_head": []
    }
  },
  "C7_parameters": {
    "a_ghost": "1/2",
    "transport_term": 1,
    "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
    "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
    "ell_sha256": {
      "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
    }
  },
  "C7_accepted_orders": {
    "kappa0": true,
    "kappa1": true,
    "kappa2": true
  }
},
"regression_controls_sampled_NOT_acceptance": {
  "evaluator_numeric": {
    "role": "regression control (sampled; NOT acceptance)",
    "samples_per_k": 3,
    "agree": true
  },
  "planewave_cubic": {
    "role": "regression control (sampled; NOT acceptance)",
    "samples": 1,
    "agree": true
  }
},
"identification_of_exported_coefficients": "exports/ell_k (sha above) == -2[eps^k] L_GG(eta + eps h) as poly
+> nomials (C4, independent evaluator); L_GG = sqrt(-g)R + d_mu B^mu exactly (C3); hence int ell_k = [kappa^
+> {k-2}] S_EH for compact support.",
"mutations": [
  {
    "mutation": "M1_corrupt_one_ell4_coefficient_x7/5",
    "failing_test": "C7 kappa^2 (Q != 0)",
    "rejected": true
  },
  {
```

```
      "mutation": "M2_wrong_overall_norm_ell4_sign",
      "failing_test": "C7 kappa^2 (Q != 0)",
      "rejected": true
    },
    {
      "mutation": "M3_swapped_GammaGamma_contraction",
      "failing_test": "C4, C5, C6, C7",
      "detail": {
        "C4": {
          "ell2": false,
          "ell3": false,
          "ell4": false
        },
        "C5": false,
        "C6": false,
        "C7": {
          "kappa0": false,
          "kappa1": false,
          "kappa2": false
        }
      },
      "rejected": true
    },
    {
      "mutation": "M4_missing_sqrt_eps2_coefficient",
      "failing_test": "C4 ell4, C7 kappa^2",
      "detail": {
        "C4": {
          "ell2": true,
          "ell3": true,
          "ell4": false
        },
        "C7": {
          "kappa0": true,
          "kappa1": true,
          "kappa2": false
        }
      },
      "rejected": true
    },
    {
      "mutation": "M5_wrong_inverse_sign",
      "failing_test": "C1",
      "rejected": true
    },
    {
      "mutation": "M6_stale_ell4_hash",
      "failing_test": "sha256 vs certificate",
      "rejected": true
    },
    {
      "mutation": "M7_wrong_boundary_orientation (+d_mu B^mu)",
      "failing_test": "C3 generic + conformal",
      "rejected": true
    }
  ],
  "mutations_all_rejected": true,
  "polarization": {
    "convention": "V4[h1..h4] = [t1 t2 t3 t4] ell_4(sum t_i h_i) = D^4 ell_4(0)[h1..h4] (unnormalised); normal
  +> ised symmetric form = V4/4!; Feynman vertex i kappa^2 V4; D02 graph symmetry factor 1/2",
    "definition": "V4[h1,h2,h3,h4] := [t1 t2 t3 t4] ell_4(sum_i t_i h_i) = D^4 ell_4(0)[h1,h2,h3,h4]",
    "normalisation": "UNnormalised fourth variation; the normalised symmetric 4-linear form is V4/4! (not used
  +> )",
    "implemented_by": "planewave_exact.polarize / series_eval.polarized_ell (inclusion-exclusion) compute V4",
    "where_factors_enter": {
      "1/4!": "only in S2 = (1/4!) V4[h,h,h,h]; cancelled by the 4! leg assignments",
      "i": "vertex factor i from e^{iS}",
      "kappa": "kappa^2 (S = sum kappa^{k-2} int ell_k)",
      "D02_symmetry_factor": "1/2 at graph level (two legs contracted)",
      "D02_vertex_rule": "i kappa^2 V4[h1,h2,h3,h4]"
    },
    "D02_quantum_evaluated": false
  },
  "seconds": 124.5,
  "pass": true
}
```

82 BRST derivation certificate

[listing 49/133] certificates/P13_brst_derivation.json


```
{
  "revision": "p13",
  "gate": "P13_brst_from_geometry",
  "declared_inputs": {
    "parametrisation": "g_{mu nu} = eta_{mu nu} + kappa h_{mu nu}  (h lowered, affine in h)",
    "ghost_scaling": "xi^mu = kappa a c^mu  (geometric diffeo parameter; XVI-N: s h = (1/2) L_c(eta + kappa h)
  +> )",
    "nonminimal_pair": "s cbar_mu = i b_mu, s b_mu = 0 (contractible pair; XVI-N normalisation i)",
    "scalar": "phi -> phi o flow  =>  s phi = xi.d phi"
  },
  "ghost_law_derivation": {
    "ansatz": "s xi^mu = lam xi^r d_r xi^mu",
    "condition": "s^2 h = 0 and s^2 phi = 0 for all jets",
    "lam_solutions": ["1"],
    "n_independent_conditions": 115,
    "s2_c_terms_any_lam": 0,
    "s2_c_identically_zero_any_lam": true,
    "derived_lam": 1
  },
  "s_images": {
    "h": "s h_mn = (1/kappa) L_xi(eta+kappa h)_mn = a(d_m c_n + d_n c_m) + kappa a (L_c h)_mn   [exact]",
    "c": "s c^mu = (1/(kappa a)) xi.d xi^mu = kappa a c^r d_r c^mu   [exact]",
    "phi": "s phi = kappa a c^r d_r phi   [exact]",
    "cbar": "s cbar_mu = i b_mu",
    "b": "s b_mu = 0"
  },
  "per_sector_kappa_expansion": {
    "h": {
      "terms_by_kappa_order": {
        "0": 16,
        "1": 104,
        "2": 0,
        "3": 0
      },
      "max_kappa_degree": "1",
      "s0_nonzero": true,
      "s1_nonzero": true,
      "s2_nonzero": false,
      "s3_nonzero": false,
      "s2": "derived_zero"
    },
    "c": {
      "terms_by_kappa_order": {
        "0": 0,
        "1": 16,
        "2": 0,
        "3": 0
      },
      "max_kappa_degree": "1",
      "s0_nonzero": false,
      "s1_nonzero": true,
      "s2_nonzero": false,
      "s3_nonzero": false,
      "s2": "derived_zero"
    },
    "phi": {
      "terms_by_kappa_order": {
        "0": 0,
        "1": 4,
        "2": 0,
        "3": 0
      },
      "max_kappa_degree": "1",
      "s0_nonzero": false,
      "s1_nonzero": true,
      "s2_nonzero": false,
      "s3_nonzero": false,
      "s2": "derived_zero"
    },
    "cb": {
      "terms_by_kappa_order": {
        "0": 4,
        "1": 0,
        "2": 0,
        "3": 0
      },
      "max_kappa_degree": "0",
```

```
    "s0_nonzero": true,
    "s1_nonzero": false,
    "s2_nonzero": false,
    "s3_nonzero": false,
    "s2": "derived_zero"
  },
  "b": {
    "terms_by_kappa_order": {
      "0": 0,
      "1": 0,
      "2": 0,
      "3": 0
    },
    "max_kappa_degree": -1,
    "s0_nonzero": false,
    "s1_nonzero": false,
    "s2_nonzero": false,
    "s3_nonzero": false,
    "s2": "derived_zero"
  }
},
"s2_per_sector": {
  "h": "derived_zero",
  "c": "derived_zero",
  "phi": "derived_zero",
  "cb": "derived_zero",
  "b": "derived_zero"
},
"s2_reason": "derived: g is affine in h and L_xi is linear in g, so (1/kappa)L_{kappa a c}g has kappa-degree
+>  exactly 1; s xi quadratic in xi gives s c of degree exactly 1; s phi linear in xi; (cbar,b) pair kappa-i
+> ndependent. 'No invented s2' is NOT used.",
"nilpotency_exact_all_orders": true,
"nilpotency_terms_by_order": {
  "h(0, 0)": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "h(0, 1)": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "h(0, 2)": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "h(0, 3)": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "h(1, 1)": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "h(1, 2)": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "h(1, 3)": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
```

```
},
"h(2, 2)": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"h(2, 3)": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"h(3, 3)": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"c0": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"c1": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"c2": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"c3": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"phi()": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"cb0": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"b0": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"cb1": {
  "0": 0,
  "1": 0,
  "2": 0,
  "3": 0,
  "4": 0
},
"b1": {
  "0": 0,
  "1": 0,
```

```
    "2": 0,
    "3": 0,
    "4": 0
  },
  "cb2": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "b2": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "cb3": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  },
  "b3": {
    "0": 0,
    "1": 0,
    "2": 0,
    "3": 0,
    "4": 0
  }
},
"matches_XVI_N_s0_s1_sc": true,
"ghost_number_violations": [],
"control_nonlinear_parametrisation": {
  "parametrisation": "g = eta + kappa h + kappa^2/2 h.eta.h (control only, NOT the declared one)",
  "s2_h_terms": 100,
  "s2_h_nonzero": true,
  "with_s2": {
    "s2h_terms_by_order_through_kappa2": {
      "h00": {
        "0": 0,
        "1": 0,
        "2": 0
      },
      "h01": {
        "0": 0,
        "1": 0,
        "2": 0
      },
      "h12": {
        "0": 0,
        "1": 0,
        "2": 0
      }
    },
    "nilpotent_through_kappa2": true
  },
  "s2_omitted": {
    "s2h_terms_by_order_through_kappa2": {
      "h00": {
        "0": 0,
        "1": 0,
        "2": 36
      },
      "h01": {
        "0": 0,
        "1": 0,
        "2": 60
      },
      "h12": {
        "0": 0,
        "1": 0,
        "2": 60
      }
    },
    "nilpotent_through_kappa2": false
  }
},
"execution_pass": true,
```

```
  "claim_status": "s2_derived_zero_all_sectors",
  "scope": "declared affine parametrisation g=eta+kappa h, ghost scaling xi=kappa a c, a=1/2; formal power ser
  +> ies in kappa",
  "pass": true,
  "seconds": 7.65
}
```


83 Source action certificate

[listing 50/133] certificates/P13_source_action.json


```
{
  "revision": "p13",
  "gate": "P13_source_action",
  "bracket": "(F,G) = sum_A int [ (d^R F/d Phi^A)(d^L G/d Phi*_A) - (d^R F/d Phi*_A)(d^L G/d Phi^A) ]",
  "generator_rule": "s Phi^A = (S, Phi^A);  s Phi*_A = (S, Phi*_A)  (XVI-N: NOT (Phi*,S))",
  "sigma_solutions": {
    "h": [1],
    "c": [-1],
    "cb": [-1],
    "phi": [1]
  },
  "sigma_unique": true,
  "sigma": {
    "h": 1,
    "c": -1,
    "cb": -1,
    "phi": 1
  },
  "sigma_closed_form": "sigma_A = (-1)^{|Phi^A|}  (derived: right derivative of Phi* X past X gives (-1)^{|Phi
  +> |+1}, generator rule adds -1)",
  "sigma_closed_form_check": {
    "h": true,
    "c": true,
    "cb": true,
    "phi": true
  },
  "matches_XVI_N_S_af": true,
  "S_af": "int K^{mn}(s h)_mn + int phi*(s phi) - int c*_mu (s c^mu) - int cbar*^mu (s cbar_mu) = int K^{mn}[a
  +> (d_m c_n+d_n c_m) + kappa a (L_c h)_mn] + kappa a int phi* c.d phi - kappa a int c*_mu c^r d_r c^mu - i i
  +> nt cbar*^mu b_mu",
  "K_vs_symmetric_hstar": "K^{mn} = w_mn h*^{mn}, w=1 (m=n), 2 (m<n); pairing over all 16 components",
  "nonminimal_included": true,
  "b_star_term": "absent (s b = 0)",
  "X_terms_by_kappa_order": {
    "0": 17,
    "1": 124,
    "2": 0,
    "3": 0
  },
  "X_kappa2_terms": 0,
  "X_ghost_number": [0],
  "antifield_linear": true,
  "antifield_transformations_X_part": {
    "K(0, 0)": {
      "terms": 11,
      "kappa_orders": [
        "1"
      ]
    },
    "cs0": {
      "terms": 38,
      "kappa_orders": [
        "1"
      ]
    },
    "cbs0": {
      "terms": 0,
      "kappa_orders": []
    },
    "phis()": {
      "terms": 8,
      "kappa_orders": [
        "1"
      ]
    }
  },
  "S_field": {
```

```
    "grav": "S_EH = -(2/kappa^2) int sqrt(-g) R ;  ell_k = -2 [eps^k] L_GammaGamma ; S_EH = sum_k kappa^(k-2)
  +> int ell_k (mod boundary)",
    "scalar": "S_phi = (1/2) int sqrt(-g) g^{mu nu} d_mu phi d_nu phi (m=0);  sqrt(-g) g^{mu nu} = eta + kappa
  +>  a1 + kappa^2 a2 (XI-A a1,a2; O2_scalar_action_authoritative)",
    "scalar_B_identification": {
      "B0_eta": true,
      "a1_matches_XI_A": true,
      "a2_matches_XI_A": true
    },
    "grav_identification": "exports/ell{2,3,4}_generating_polynomial.srepr (sha in P13_quartic_verified)"
  },
  "baseline": "S_inv (unprepared, sigma_eps == 1, no insertion I_t); prepared S_D keeps (S_D,S_D) != 0 (XVI-N
  +> prop:broken-square)",
  "execution_pass": true,
  "claim_status": "source_action_constructed",
  "scope": "S_inv extended action through O(kappa^2), antifield-linear (closed algebra), nonminimal (cbar,b) p
  +> air",
  "pass": true,
  "seconds": 4.64
}
```


84 Gauge-fixing map certificate

[listing 51/133] certificates/P13_gauge_fixing_map.json


```
{
  "revision": "p13",
  "gate": "gauge_fixing_map_constructed",
  "Psi": "Psi = -i int cbar_mu (2 D^mu[h] - b^mu),  D^mu[h] = d_nu h^{mu nu} - (1/2) d^mu tr h",
  "Psi_parity_odd": true,
  "Psi_ghost_number": [-1],
  "left_equals_right_for_odd_Psi": true,
  "dPsi": {
    "dPsi/dh_mn": "2i (d^(m cbar^n) - 1/2 eta^{mn} d.cbar) * w_mn-pairing (independent comps m<=n)",
    "dPsi/dcbar_mu": "-i (2 D^mu[h] - b^mu)",
    "dPsi/db_mu": "+i cbar^mu",
    "dPsi/dc": "0",
    "dPsi/dphi": "0"
  },
  "dPsi_nonzero_components": ["b0", "b1", "b2", "b3", "cb0", "cb1", "cb2", "cb3", "h(0, 0)", "h(0, 1)", "h(0,
  +> 2)", "h(0, 3)", "h(1, 1)", "h(1, 2)", "h(1, 3)", "h(2, 2)", "h(2, 3)", "h(3, 3)"],
  "S_af_at_dPsi_vs_sPsi": {
    "S_af(dPsi) ~ -sPsi": true,
    "S_af(dPsi) ~ +sPsi": false
  },
  "orientation_scan": {
    "1": {
      "hessian": {
        "momentum": "generic symbolic k = (k0,k1,k2,k3); identities in Q(k) (no sample)",
        "det_Mbb": "-16",
        "Mbb_det_nonzero": true,
        "schur_equals_k2P": false,
        "det_k2P": "64*(k0**2 - k1**2 - k2**2 - k3**2)**10",
        "propagator_hh_equals_inv_k2P": false,
        "propagator_argument": "block-inverse formula: (M^{-1})_hh = (A - B D^{-1} C)^{-1} = (k^2 P)^{-1}"
      },
      "ghost_kinetic_i_box": false
    },
    "-1": {
      "hessian": {
        "momentum": "generic symbolic k = (k0,k1,k2,k3); identities in Q(k) (no sample)",
        "det_Mbb": "-16",
        "Mbb_det_nonzero": true,
        "schur_equals_k2P": true,
        "det_k2P": "64*(k0**2 - k1**2 - k2**2 - k3**2)**10",
        "propagator_hh_equals_inv_k2P": true,
        "propagator_argument": "block-inverse formula: (M^{-1})_hh = (A - B D^{-1} C)^{-1} = (k^2 P)^{-1}"
      },
      "ghost_kinetic_i_box": true
    }
  },
  "derived_eps": -1,
  "map": {
    "fields": "Phi'^A = Phi^A (all fields unchanged)",
    "antifields": "Phi*_A = Phi*'_A + (-1) dPsi/dPhi^A   i.e.  Phi*'_A = Phi*_A - (-1) dPsi/dPhi^A",
    "inverse": "Phi^A = Phi'^A ; Phi*'_A = Phi*_A - (-1) dPsi/dPhi^A",
    "explicit": {
      "K'^{mn}": "K^{mn} - (-1) dPsi/dh_mn",
```

```
      "cbar*'^mu": "cbar*^mu - (-1)(-i)(2D^mu[h]-b^mu)",
      "b*'^mu": "b*^mu - (-1)(i cbar^mu)",
      "c*'": "c*",
      "phi*'": "phi*"
    }
  },
  "S_Psi_equals_S_plus_sPsi_mod_d": true,
  "gauge_fixed_pieces": {
    "kappa0_ghost_kinetic_i_box": true,
    "kappa1_ghost_graviton_2ia_D[L_c h]": true,
    "kappa2_field_only_terms": 0,
    "kappa2_ghost_vertex_absent_derived": true,
    "b_sector": "b(2D-b): eliminating b gives +D_mu D^mu"
  },
  "propagators": {
    "graviton": {
      "stated": "K = -Box P (includes S_gf)",
      "momentum": "generic symbolic k = (k0,k1,k2,k3); identities in Q(k) (no sample)",
      "det_Mbb": "-16",
      "Mbb_det_nonzero": true,
      "schur_equals_k2P": true,
      "det_k2P": "64*(k0**2 - k1**2 - k2**2 - k3**2)**10",
      "propagator_hh_equals_inv_k2P": true,
      "propagator_argument": "block-inverse formula: (M^{-1})_hh = (A - B D^{-1} C)^{-1} = (k^2 P)^{-1}"
    },
    "ghost": {
      "stated": "i cbar Box c",
      "match": true
    },
    "scalar": {
      "stated": "-Box (m=0)",
      "match": true
    },
    "momentum": "generic symbolic (k0,k1,k2,k3)"
  },
  "propagators_match_transformed_Hessians": true,
  "canonicity": {
    "antifield_antifield_fails": [],
    "antifield_field_fails": [],
    "pass": true,
    "pairs_tested": 299
  },
  "external_antifields_retained": ["K", "cbs", "cs0", "cs1", "cs2", "cs3", "phis"],
  "invariant_vs_transformed_kept_separate": {
    "invariant_extended": "S_ext = S_field + S_af(Phi*)  [nonminimal pair included; Psi not applied]",
    "transformed": "S_Psi(Phi,Phi*') = S_ext(Phi, Phi*' + (-1) dPsi/dPhi)  [external antifields retained]",
    "NOT_S_D": true
  },
  "source_identity_relation": {
    "relation": "Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' + (-1) dPsi/dPhi)",
    "classical_instance_verified": true,
    "status": "DEFINED classical pullback/interface along the canonical antifield shift; NOT an independently
  +> established quantum gauge-change theorem (no quantum Gamma is constructed here)",
    "ch_relevant_feed": "cbar*-dependence of the invariant functional maps to h-dependence: cbar* = cbar*' + (
  +> -1)(-i)(2D[h]-b); h*, b* shifts carry cbar (vanish on ch pattern)",
    "related_not_identical": true
  },
  "source_identity_relation_defined": true,
  "gauge_fixing_map_constructed": true,
  "pass": true,
  "execution_pass": true,
  "mutations_from_env": [],
  "claim_status": "Constructed",
  "scope": "classical BV gauge fixing of S_inv extended action by declared Psi; linear de Donder; through O(ka
  +> ppa^2)",
  "NOT_import_shift_sign": true,
  "obstruction_respected": "No insertion completion; XVI-N thm:massless-obstruction untouched; XV-B not opened
  +> ; Route B not discharged",
  "seconds": 10.1
}
```


85 Master equation certificate

[listing 52/133] certificates/P13_master_kappa2.json


```
{
  "revision": "p13",
  "gate": "required_classical_BV_components_verified",
```

```
"equation": "(S,S) at kappa^2: 2(S0,S2)+(S1,S1) = 0  [plus regressions (S0,S0)=0, 2(S0,S1)=0]",
"bracket": "(F,G) = sum_A int [ (d^R F/d Phi^A)(d^L G/d Phi*_A) - (d^R F/d Phi*_A)(d^L G/d Phi^A) ]",
"baseline": "S_inv (unprepared, sigma_eps == 1, no insertion I_t); prepared S_D keeps (S_D,S_D) != 0 (XVI-N
+> prop:broken-square)",
"identification_with_exports": {
  "ell2_ell3_ell4": {
    "2": {
      "sha256": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
      "terms": 84
    },
    "3": {
      "sha256": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
      "terms": 678
    },
    "4": {
      "sha256": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
      "terms": 3360
    }
  },
  "ell2_ell3_ell4_sha_match_P13_quartic_verified": true,
  "quartic_generator_verified": true,
  "scalar_a1_a2": "re-derived and identified in P13_source_action (B = sqrt(-g) g^{-1})",
  "s_images": "P13_brst_derivation (s2 derived zero)"
},
"table_by_kappa_and_antifield_number": {
  "kappa0": {
    "af0_grav_Noether": true,
    "af0_scalar_Noether": true,
    "af0_ghost_gf_transformed": true,
    "af1_invariant": true,
    "af1_transformed": true,
    "af2_invariant": true,
    "af2_transformed": true
  },
  "kappa1": {
    "af0_grav_Noether": true,
    "af0_scalar_Noether": true,
    "af0_ghost_gf_transformed": true,
    "af1_invariant": true,
    "af1_transformed": true,
    "af2_invariant": true,
    "af2_transformed": true
  },
  "kappa2": {
    "af0_grav_Noether": true,
    "af0_scalar_Noether": true,
    "af0_ghost_gf_transformed": true,
    "af1_invariant": true,
    "af1_transformed": true,
    "af2_invariant": true,
    "af2_transformed": true
  }
},
"details": {
  "invariant_variables": {
    "0": {
      "total": {
        "is_divergence": true,
        "n_terms": 0,
        "nonzero_euler_components": []
      },
      "by_antifield_number": {
        "0": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        },
        "1": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        },
        "2": {
```

```
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        }
      }
    },
    "1": {
      "total": {
        "is_divergence": true,
        "n_terms": 180,
        "nonzero_euler_components": []
      },
      "by_antifield_number": {
        "0": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        },
        "1": {
          "total": {
            "is_divergence": true,
            "n_terms": 180,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {
            "h": {
              "is_divergence": false,
              "n_terms": 148,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            },
            "c": {
              "is_divergence": false,
              "n_terms": 64,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            }
          },
          "mixed_cancellation": true
        },
        "2": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        }
      }
    },
    "2": {
      "total": {
        "is_divergence": true,
        "n_terms": 1356,
        "nonzero_euler_components": []
      },
      "by_antifield_number": {
        "0": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        },
        "1": {
          "total": {
            "is_divergence": true,
            "n_terms": 1356,
            "nonzero_euler_components": []
          },
```

```
          "per_bracket_pair": {
            "h": {
              "is_divergence": false,
              "n_terms": 992,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            },
            "c": {
              "is_divergence": false,
              "n_terms": 564,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            },
            "phi": {
              "is_divergence": false,
              "n_terms": 28,
              "nonzero_euler_components": ["c0", "c1", "c2", "c3", "phi()", "phis()"]
            }
          },
          "mixed_cancellation": true
        },
        "2": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        }
      }
    }
  },
  "transformed_variables": {
    "0": {
      "total": {
        "is_divergence": true,
        "n_terms": 32,
        "nonzero_euler_components": []
      },
      "by_antifield_number": {
        "0": {
          "total": {
            "is_divergence": true,
            "n_terms": 32,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {
            "h": {
              "is_divergence": false,
              "n_terms": 16,
              "nonzero_euler_components": ["b0", "b1", "b2", "b3", "c0", "c1", "c2", "c3"]
            },
            "cb": {
              "is_divergence": false,
              "n_terms": 16,
              "nonzero_euler_components": ["b0", "b1", "b2", "b3", "c0", "c1", "c2", "c3"]
            }
          },
          "mixed_cancellation": true
        },
        "1": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        },
        "2": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        }
      }
    },
```

```
    "1": {
      "total": {
        "is_divergence": true,
        "n_terms": 552,
        "nonzero_euler_components": []
      },
      "by_antifield_number": {
        "0": {
          "total": {
            "is_divergence": true,
            "n_terms": 372,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {
            "h": {
              "is_divergence": false,
              "n_terms": 324,
              "nonzero_euler_components": ["b0", "b1", "b2", "b3", "c0", "c1", "c2", "c3", "cb0", "cb1", "cb
+> 2", "cb3"]
            },
            "c": {
              "is_divergence": false,
              "n_terms": 64,
              "nonzero_euler_components": ["c0", "c1", "c2", "c3", "cb0", "cb1", "cb2", "cb3"]
            },
            "cb": {
              "is_divergence": false,
              "n_terms": 36,
              "nonzero_euler_components": ["b0", "b1", "b2", "b3", "c0", "c1", "c2", "c3", "h(0, 0)", "h(0,
+> 1)", "h(0, 2)", "h(0, 3)"]
            }
          },
          "mixed_cancellation": true
        },
        "1": {
          "total": {
            "is_divergence": true,
            "n_terms": 180,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {
            "h": {
              "is_divergence": false,
              "n_terms": 148,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            },
            "c": {
              "is_divergence": false,
              "n_terms": 64,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            }
          },
          "mixed_cancellation": true
        },
        "2": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        }
      }
    },
    "2": {
      "total": {
        "is_divergence": true,
        "n_terms": 3748,
        "nonzero_euler_components": []
      },
      "by_antifield_number": {
        "0": {
          "total": {
            "is_divergence": true,
            "n_terms": 2392,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {
```

```
            "h": {
              "is_divergence": false,
              "n_terms": 2192,
              "nonzero_euler_components": ["c0", "c1", "c2", "c3", "cb0", "cb1", "cb2", "cb3", "h(0, 0)", "h
+> (0, 1)", "h(0, 2)", "h(0, 3)"]
            },
            "c": {
              "is_divergence": false,
              "n_terms": 512,
              "nonzero_euler_components": ["c0", "c1", "c2", "c3", "cb0", "cb1", "cb2", "cb3", "h(0, 0)", "h
+> (0, 1)", "h(0, 2)", "h(0, 3)"]
            }
          },
          "mixed_cancellation": true
        },
        "1": {
          "total": {
            "is_divergence": true,
            "n_terms": 1356,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {
            "h": {
              "is_divergence": false,
              "n_terms": 992,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            },
            "c": {
              "is_divergence": false,
              "n_terms": 564,
              "nonzero_euler_components": ["K(0, 0)", "K(0, 1)", "K(0, 2)", "K(0, 3)", "K(1, 1)", "K(1, 2)",
+>  "K(1, 3)", "K(2, 2)", "K(2, 3)", "K(3, 3)", "c0", "c1"]
            },
            "phi": {
              "is_divergence": false,
              "n_terms": 28,
              "nonzero_euler_components": ["c0", "c1", "c2", "c3", "phi()", "phis()"]
            }
          },
          "mixed_cancellation": true
        },
        "2": {
          "total": {
            "is_divergence": true,
            "n_terms": 0,
            "nonzero_euler_components": []
          },
          "per_bracket_pair": {},
          "mixed_cancellation": false
        }
      }
    }
  },
  "scalar_af0": {
    "0": {
      "total": {
        "is_divergence": true,
        "n_terms": 0,
        "nonzero_euler_components": []
      },
      "pieces_individually_divergence": {
        "2(L0,X0)": true
      }
    },
    "1": {
      "total": {
        "is_divergence": true,
        "n_terms": 20,
        "nonzero_euler_components": []
      },
      "pieces_individually_divergence": {
        "2(L0,X1)": false,
        "2(L1,X0)": false
      }
    },
    "2": {
      "total": {
        "is_divergence": true,
        "n_terms": 20,
        "nonzero_euler_components": []
```

```
    },
    "pieces_individually_divergence": {
      "2(L1,X1)": false,
      "2(L2,X0)": false
    }
  }
},
"grav_af0_deterministic": {
  "kappa0": {
    "density": "s0 ell2",
    "W_terms": 132,
    "Q_terms": [0, 0, 0, 0],
    "Q_identically_zero": true,
    "s0_piece_alone_Q_terms": [0, 0, 0, 0],
    "residual_head": []
  },
  "kappa1": {
    "density": "s0 ell3 + s1 ell2",
    "W_terms": 2148,
    "Q_terms": [0, 0, 0, 0],
    "Q_identically_zero": true,
    "s0_piece_alone_Q_terms": [159, 159, 159, 159],
    "residual_head": []
  },
  "kappa2": {
    "density": "s0 ell4 + s1 ell3 (+ s2 ell2, s2 = 0)",
    "W_terms": 15428,
    "Q_terms": [0, 0, 0, 0],
    "Q_identically_zero": true,
    "s0_piece_alone_Q_terms": [2088, 2088, 2088, 2088],
    "residual_head": []
  }
},
"grav_af0_parameters": {
  "a_ghost": "1/2",
  "transport_term": 1,
  "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)",
  "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
  "ell_sha256": {
    "ell2": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
    "ell3": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
    "ell4": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123"
  }
},
"grav_af0_accepted_orders": {
  "kappa0": true,
  "kappa1": true,
  "kappa2": true
},
"grav_af0_planewave_regression_control_NOT_acceptance": {
  "0": {
    "samples": 3,
    "all_sums_exactly_zero": true,
    "s0_piece_nonzero_in_samples": 0,
    "s1_piece_nonzero_in_samples": 0,
    "max_abs_sum": "0",
    "example_pieces": ["0", "0"],
    "density": "s0 ell2"
  },
  "1": {
    "samples": 3,
    "all_sums_exactly_zero": true,
    "s0_piece_nonzero_in_samples": 3,
    "s1_piece_nonzero_in_samples": 3,
    "max_abs_sum": "0",
    "example_pieces": ["82091533477/1800000", "-82091533477/1800000"],
    "density": "s0 ell3 + s1 ell2"
  },
  "2": {
    "samples": 3,
    "all_sums_exactly_zero": true,
    "s0_piece_nonzero_in_samples": 3,
    "s1_piece_nonzero_in_samples": 3,
    "max_abs_sum": "0",
    "example_pieces": ["13042093527791/1620000000", "-13042093527791/1620000000"],
    "density": "s0 ell4 + s1 ell3 (+ s2 ell2, s2=0 derived)"
  }
},
"grav_inputs": {
  "2": {
    "sha256": "8af76ef7b97d190159001df33e715dabb759060e49fb1eb6d896f74ec7745ce0",
```

```
        "terms": 84
      },
      "3": {
        "sha256": "60c73cd39c9e15316d366288ad133237bf4afbd151dfef7b0a89a6dec55e39c1",
        "terms": 678
      },
      "4": {
        "sha256": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
        "terms": 3360
      }
    },
    "residuals_all_orders": [],
    "residuals_kappa2": [],
    "seconds": 56.65,
    "mutations": []
  },
  "required_master_residuals": [],
  "required_classical_BV_components_verified": true,
  "evidence_kind": {
    "engine_parts": "exact symbolic (graded jets; Euler-operator divergence test)",
    "grav_af0": "DETERMINISTIC exact polynomial identity: E_{c^mu}(s0 ell_{n+2} + s1 ell_{n+1}) = 0 for n = 0,
  +> 1,2 on the exported polynomials (grav_noether.py); structural reason: sqrt(-g)R is a scalar density, s(sq
  +> rt(-g)R) = d(xi sqrt(-g)R), and L_GG = sqrt(-g)R + d_mu B^mu (P13_boundary_identity: sqrt(-g)R - L_GG = -
  +> d_mu B^mu), with s d_mu B^mu = d_mu s B^mu",
    "grav_af0_planewave": "REGRESSION CONTROL only: exact rational plane-wave evaluation at 3 points per order
  +> ; not cited as proof"
  },
  "transformed_variables_note": "(S_Psi,S_Psi) = 2(S_field,X_Psi) + (X_Psi,X_Psi); X is antifield-linear, so d
  +> X_Psi/dPhi*' = dX/dPhi* and 2(S_field,X_Psi) = 2(S_field,X) (af 0, shared Noether table); the s Psi field
  +> -only part has no antifields and brackets only through (X_Psi,X_Psi).",
  "mixed_cancellations": {
    "kappa1_af1_invariant": ["c", "h"],
    "kappa2_af1_invariant": ["c", "h", "phi"],
    "kappa0_af0_transformed": ["cb", "h"],
    "kappa1_af0_transformed": ["c", "cb", "h"],
    "kappa1_af1_transformed": ["c", "h"],
    "kappa2_af0_transformed": ["c", "h"],
    "kappa2_af1_transformed": ["c", "h", "phi"]
  },
  "execution_pass": true,
  "claim_status": "verified",
  "mutations_from_env": [],
  "scope": "classical BV master equation of the S_inv extended action (grav ell_2..ell_4 + massless scalar + m
  +> inimal ghosts + nonminimal (cbar,b)) through O(kappa^2), in invariant and Psi-transformed variables with
  +> external antifields; NOT the prepared S_D (broken square kept), NOT the insertion, NOT quantum.",
  "note_on_pass": "p11: no unconditional 'pass' field; execution_pass (ran), claim_status and scope are separa
  +> te."
}
```


86 ch source audit certificate

[listing 53/133] certificates/P13_ch_source_audit.json


```
{
  "revision": "p13",
  "gate": "ch_projection_defined / ch_dependency_inventory_complete",
  "variables": "transformed (Phi, Phi*'), S_Psi with external antifields; Psi = Psi = -i int cbar_mu (2 D^mu[h
  +> ] - b^mu),  D^mu[h] = d_nu h^{mu nu} - (1/2) d^mu tr h",
  "ch_pattern": {
    "c": 1,
    "h": 1,
    "phi": 0,
    "cbar": 0,
    "b": 0,
    "antifields": 0,
    "ghost_number": 1,
    "coupling": "kappa^2",
    "loop": "one",
    "domain": "off-shell, compact smooth smearings"
  },
  "method": "p11 (e): mechanical ch dependency audit in TRANSFORMED variables + generated one-loop graph inven
  +> tory.",
  "vertex_inventory": {
    "kappa0": ["K.c", "b.cbs", "b.h", "b^2", "c.cb", "h^2", "phi^2"],
    "kappa1": ["K.c.h", "c.cb.h", "c.phi.phis", "c^2.cs", "h.phi^2", "h^3"],
    "kappa2": ["h^2.phi^2", "h^4"]
  },
  "vertex_sources": {
```

```
  "0|K.c": ["X_Psi (antifield + s Psi)"],
  "0|b.cbs": ["X_Psi (antifield + s Psi)"],
  "0|b.h": ["X_Psi (antifield + s Psi)"],
  "0|b^2": ["X_Psi (antifield + s Psi)"],
  "0|c.cb": ["X_Psi (antifield + s Psi)"],
  "1|K.c.h": ["X_Psi (antifield + s Psi)"],
  "1|c.cb.h": ["X_Psi (antifield + s Psi)"],
  "1|c.phi.phis": ["X_Psi (antifield + s Psi)"],
  "1|c^2.cs": ["X_Psi (antifield + s Psi)"],
  "0|phi^2": ["scalar"],
  "1|h.phi^2": ["scalar"],
  "2|h^2.phi^2": ["scalar"],
  "0|h^2": ["ell_2"],
  "1|h^3": ["ell_3"],
  "2|h^4": ["ell_4"]
},
"rows_total": 6,
"rows_surviving": 3,
"rows_derived_zero": 3,
"surviving_rows": [
  {
    "id": "(S0,Gamma1^[2]) s2 K:K.c",
    "bracket": "(S0, Gamma1^[2])",
    "summand": 2,
    "S_monomial": "K.c",
    "differentiated": "K",
    "Gamma1_pattern": "h^2",
    "Gamma1_order": "kappa^2",
    "graphs": ["D01_hh_bubble_cubic", "D02_hh_tadpole_quartic", "D03_hh_ghost_loop", "D11_hh_scalar_loop", "
+> D12_hh_scalar_tadpole"],
    "status": "surviving_unevaluated",
    "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
  },
  {
    "id": "(S0,Gamma1^[2]) s1 h:h^2",
    "bracket": "(S0, Gamma1^[2])",
    "summand": 1,
    "S_monomial": "h^2",
    "differentiated": "h",
    "Gamma1_pattern": "K.c",
    "Gamma1_order": "kappa^2",
    "graphs": ["D13_Kc_ghost_graviton_loop"],
    "status": "surviving_unevaluated",
    "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
  },
  {
    "id": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "bracket": "(S1, Gamma1^[1])",
    "summand": 2,
    "S_monomial": "K.c.h",
    "differentiated": "K",
    "Gamma1_pattern": "h",
    "Gamma1_order": "kappa^1",
    "graphs": ["D14a_h_onepoint_grav_tadpole", "D14b_h_onepoint_ghost_tadpole", "D14c_h_onepoint_scalar_tadp
+> ole"],
    "status": "surviving_unevaluated",
    "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
  }
],
"hstar_c_row": [
  {
    "id": "(S0,Gamma1^[2]) s1 h:h^2",
    "bracket": "(S0, Gamma1^[2])",
    "summand": 1,
    "S_monomial": "h^2",
    "differentiated": "h",
    "Gamma1_pattern": "K.c",
    "Gamma1_order": "kappa^2",
    "graphs": ["D13_Kc_ghost_graviton_loop"],
    "status": "surviving_unevaluated",
    "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
  }
],
"onepoint_kappa_slot": [
  {
    "id": "(S1,Gamma1^[1]) s2 K:K.c.h",
    "bracket": "(S1, Gamma1^[1])",
    "summand": 2,
    "S_monomial": "K.c.h",
    "differentiated": "K",
    "Gamma1_pattern": "h",
```

```
      "Gamma1_order": "kappa^1",
      "graphs": ["D14a_h_onepoint_grav_tadpole", "D14b_h_onepoint_ghost_tadpole", "D14c_h_onepoint_scalar_tadp
  +> ole"],
      "status": "surviving_unevaluated",
      "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
    }
  ],
  "phi_rows": {
    "count": 0,
    "all_derived_zero_or_impossible": true,
    "parity_argument": "phi -> -phi is a symmetry of S_Psi (phi appears in even powers in every non-antifield
  +> monomial and phis.phi pairs); ch is phi-even"
  },
  "checks": {
    "names_TBD_empty": true,
    "regenerated_rows_match_published": true,
    "hstar_c_row_present": true,
    "onepoint_slot_distinct_from_D12": true,
    "b_has_no_interaction_vertex": true,
    "no_kappa2_ghost_vertex": true,
    "cbar_star_rows_derived_zero": true,
    "b_star_rows_derived_zero": true,
    "S2_rows_derived_zero": true,
    "enumeration_lemma_holds": true
  },
  "enumeration_lemma": {
    "statement": "(p11; justifies max_len = 2).  (i) A connected one-loop graph is 1PI iff every internal line
  +>  lies\non its unique cycle (a line off the cycle is a bridge); hence every vertex of a one-loop 1PI graph
  +>  lies on the cycle\nwith exactly two internal legs, and all quadratic (kappa^0, two-field) terms are abso
  +> rbed into propagators, so the\ngraph is a cycle of interaction vertices.  (ii) Every retained interaction
  +>  vertex has coupling order kappa^n, n >= 1\n(checked mechanically: the kappa^0 inventory contains only tw
  +> o-field monomials).  (iii) Hence a kappa^order\ncontribution with order <= 2 has at most `order` <= 2 ver
  +> tices: max_len = 2 is exhaustive.  (iv) The auxiliary b\noccurs only in kappa^0 two-field monomials (chec
  +> ked), so its Gaussian elimination produces only bilinear terms\n(the de Donder kinetic term and source-bi
  +> linear terms) and no omitted interaction.  (v) Source-linear monomials\n(exactly one quantum leg) cannot
  +> sit on a cycle (two internal legs needed); attached off the cycle they create a\nbridge, i.e. a one-parti
  +> cle-reducible graph, so they cannot contribute extra 1PI coefficients.",
    "mechanical_checks": {
      "kappa0_monomials_all_two_field": true,
      "kappa0_multi_field_monomials": [],
      "min_interaction_vertex_order": 1,
      "all_interaction_vertices_kappa_ge_1": true,
      "max_vertices_at_kappa2": 2,
      "max_len_used": 2,
      "max_len_exhaustive": true,
      "b_monomials": [
        [0, "b.cbs"],
        [0, "b.h"],
        [0, "b^2"]
      ],
      "b_only_in_kappa0_bilinears": true,
      "source_linear_excluded_reason": "vertex_options requires >= 2 quantum legs; one-leg insertions are brid
  +> ges (1PR)"
    },
    "holds": true,
    "interface": "Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi) is a defined classical pullback/interf
  +> ace, not an independently established quantum gauge-change theorem"
  },
  "dropped_rows_mutation": [],
  "regeneration": {
    "missing_from_published": [],
    "extra_in_published": [],
    "complete": true
  },
  "names_TBD": [],
  "ch_projection_defined": true,
  "ch_dependency_inventory_complete": true,
  "values": "no surviving coefficient is evaluated in this release (no loop evaluated here); D11 carries an in
  +> herited O5 kernel constructed under prescription, not re-evaluated in this release",
  "execution_pass": true,
  "claim_status": "earned",
  "scope": "pattern-level dependency inventory of the projected kappa^2 one-loop source identity; not evaluati
  +> on",
  "seconds": 5.6
}
```

87 ch dependency rows ledger

[listing 54/133] ledgers/P13_ch_dependency_rows.json


```
{
  "revision": "p13",
  "rows": [
    {
      "id": "(S0,Gamma1^[2]) s2 K:K.c",
      "bracket": "(S0, Gamma1^[2])",
      "summand": 2,
      "S_monomial": "K.c",
      "differentiated": "K",
      "Gamma1_pattern": "h^2",
      "Gamma1_order": "kappa^2",
      "graphs": ["D01_hh_bubble_cubic", "D02_hh_tadpole_quartic", "D03_hh_ghost_loop", "D11_hh_scalar_loop", "
  +> D12_hh_scalar_tadpole"],
      "status": "surviving_unevaluated",
      "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
    },
    {
      "id": "(S0,Gamma1^[2]) s1 b:b.h",
      "bracket": "(S0, Gamma1^[2])",
      "summand": 1,
      "S_monomial": "b.h",
      "differentiated": "b",
      "Gamma1_pattern": "bs.c",
      "Gamma1_order": "kappa^2",
      "graphs": [],
      "status": "derived_zero",
      "reason": "derived zero: Gamma1 cannot depend on bs (no S_Psi vertex carries it with two quantum legs)"
    },
    {
      "id": "(S0,Gamma1^[2]) s1 cb:c.cb",
      "bracket": "(S0, Gamma1^[2])",
      "summand": 1,
      "S_monomial": "c.cb",
      "differentiated": "cb",
      "Gamma1_pattern": "cbs.h",
      "Gamma1_order": "kappa^2",
      "graphs": [],
      "status": "derived_zero",
      "reason": "derived zero: Gamma1 cannot depend on cbs (no S_Psi vertex carries it with two quantum legs)"
    },
    {
      "id": "(S0,Gamma1^[2]) s1 h:h^2",
      "bracket": "(S0, Gamma1^[2])",
      "summand": 1,
      "S_monomial": "h^2",
      "differentiated": "h",
      "Gamma1_pattern": "K.c",
      "Gamma1_order": "kappa^2",
      "graphs": ["D13_Kc_ghost_graviton_loop"],
      "status": "surviving_unevaluated",
      "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
    },
    {
      "id": "(S1,Gamma1^[1]) s2 K:K.c.h",
      "bracket": "(S1, Gamma1^[1])",
      "summand": 2,
      "S_monomial": "K.c.h",
      "differentiated": "K",
      "Gamma1_pattern": "h",
      "Gamma1_order": "kappa^1",
      "graphs": ["D14a_h_onepoint_grav_tadpole", "D14b_h_onepoint_ghost_tadpole", "D14c_h_onepoint_scalar_tadp
  +> ole"],
      "status": "surviving_unevaluated",
      "reason": "surviving: one-loop graphs exist; coefficient not evaluated in this release"
    },
    {
      "id": "(S1,Gamma1^[1]) s1 cb:c.cb.h",
      "bracket": "(S1, Gamma1^[1])",
      "summand": 1,
      "S_monomial": "c.cb.h",
      "differentiated": "cb",
      "Gamma1_pattern": "cbs",
      "Gamma1_order": "kappa^1",
      "graphs": [],
      "status": "derived_zero",
      "reason": "derived zero: Gamma1 cannot depend on cbs (no S_Psi vertex carries it with two quantum legs)"
```

```
    }
  ]
}
```

88 Generated graph inventory

[listing 55/133] ledgers/P13_graph_inventory.json


```
{
  "revision": "p13",
  "generated_from": "S_Psi vertex inventory (ch_source_audit.enumerate_graphs)",
  "graphs": {
    "D03_hh_ghost_loop": {
      "external": "h^2",
      "kappa": 2,
      "signature": [
        ["c.cb.h", "ccb", 1],
        ["c.cb.h", "ccb", 1]
      ],
      "status": "listed_unevaluated"
    },
    "D11_hh_scalar_loop": {
      "external": "h^2",
      "kappa": 2,
      "signature": [
        ["h.phi^2", "phiphi", 1],
        ["h.phi^2", "phiphi", 1]
      ],
      "status": "Constructed_kernel_under_prescription (inherited O5; not re-evaluated)"
    },
    "D12_hh_scalar_tadpole": {
      "external": "h^2",
      "kappa": 2,
      "signature": [
        ["h^2.phi^2", "phiphi", 2]
      ],
      "status": "Missing (D12_quantum_evaluated=false)"
    },
    "D01_hh_bubble_cubic": {
      "external": "h^2",
      "kappa": 2,
      "signature": [
        ["h^3", "hh", 1],
        ["h^3", "hh", 1]
      ],
      "status": "listed_unevaluated"
    },
    "D02_hh_tadpole_quartic": {
      "external": "h^2",
      "kappa": 2,
      "signature": [
        ["h^4", "hh", 2]
      ],
      "status": "vertex_available_quantum_unevaluated"
    },
    "D13_Kc_ghost_graviton_loop": {
      "external": "K.c",
      "kappa": 2,
      "signature": [
        ["K.c.h", "ch", 1],
        ["c.cb.h", "cbh", 1]
      ],
      "status": "listed_unevaluated (generated from S_Psi; h* c row; coefficient not evaluated)"
    },
    "D14b_h_onepoint_ghost_tadpole": {
      "external": "h",
      "kappa": 1,
      "signature": [
        ["c.cb.h", "ccb", 1]
      ],
      "status": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12"
    },
    "D14c_h_onepoint_scalar_tadpole": {
      "external": "h",
      "kappa": 1,
      "signature": [
        ["h.phi^2", "phiphi", 1]
      ],
```

```
      "status": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12"
    },
    "D14a_h_onepoint_grav_tadpole": {
      "external": "h",
      "kappa": 1,
      "signature": [
        ["h^3", "hh", 1]
      ],
      "status": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12"
    }
  },
  "names_TBD": [],
  "D02_quantum_evaluated": false,
  "D12_quantum_evaluated": false,
  "D12_status": "Missing",
  "no_loop_evaluated": true
}
```


89 V4_grav export

[listing 56/133] exports/V4_grav.json


```
{
  "id": "V4_grav",
  "revision": "p13",
  "status": "Constructed",
  "quartic_generator_verified": true,
  "D02_vertex_available": true,
  "D02_quantum_evaluated": false,
  "ell4_sha256": "33ac9e49c4d1644ee47f09b80983f4ea29827d33af93248d7037bdafa5c6e123",
  "ell4_srepr_path": "exports/ell4_generating_polynomial.srepr",
  "terms_reproducibility": 3360,
  "NOT_acceptance_by_term_count": true,
  "conventions": {
    "revision": "p13",
    "eta": "diag(+,-,-,-)",
    "q": "eta + eps H, eps = kappa, H = h (lowered, symmetric); affine parametrisation",
    "S_EH": "-(2/kappa^2) int sqrt(-q) R",
    "density": "L_GG = sqrt(-g) g^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab})",
    "boundary_identity": "sqrt(-g) R - L_GG = - d_mu B^mu,  B^mu = sqrt(-g)(g^{mu nu} G^l_{nu l} - g^{l nu} G^
  +> mu_{l nu})",
    "ell_k": "-2 [eps^k] L_GG ; S_EH = sum_k kappa^{k-2} int ell_k (integrated, compact support)",
    "IBP": "total derivatives dropped only inside the integrated action (orientation of the dropped divergence
  +>  irrelevant)",
    "truncation": "O(eps^4) inclusive",
    "jets": "H{m}{n} = h_mn (m<=n), dH{m}{n}_{r} = d_r h_mn",
    "a_ghost": "1/2",
    "polarization": "V4[h1..h4] = [t1 t2 t3 t4] ell_4(sum t_i h_i) = D^4 ell_4(0)[h1..h4] (unnormalised); norm
  +> alised symmetric form = V4/4!; Feynman vertex i kappa^2 V4; D02 graph symmetry factor 1/2"
  },
  "polarization": {
    "convention": "V4[h1..h4] = [t1 t2 t3 t4] ell_4(sum t_i h_i) = D^4 ell_4(0)[h1..h4] (unnormalised); normal
  +> ised symmetric form = V4/4!; Feynman vertex i kappa^2 V4; D02 graph symmetry factor 1/2",
    "definition": "V4[h1,h2,h3,h4] := [t1 t2 t3 t4] ell_4(sum_i t_i h_i) = D^4 ell_4(0)[h1,h2,h3,h4]",
    "normalisation": "UNnormalised fourth variation; the normalised symmetric 4-linear form is V4/4! (not used
  +> )",
    "implemented_by": "planewave_exact.polarize / series_eval.polarized_ell (inclusion-exclusion) compute V4",
    "where_factors_enter": {
      "1/4!": "only in S2 = (1/4!) V4[h,h,h,h]; cancelled by the 4! leg assignments",
      "i": "vertex factor i from e^{iS}",
      "kappa": "kappa^2 (S = sum kappa^{k-2} int ell_k)",
      "D02_symmetry_factor": "1/2 at graph level (two legs contracted)",
      "D02_vertex_rule": "i kappa^2 V4[h1,h2,h3,h4]"
    },
    "D02_quantum_evaluated": false
  }
}
```


90 ell4 conventions export

[listing 57/133] exports/ell4_conventions.json


```
{
  "revision": "p13",
  "eta": "diag(+,-,-,-)",
```

```
  "q": "eta + eps H, eps = kappa, H = h (lowered, symmetric); affine parametrisation",
  "S_EH": "-(2/kappa^2) int sqrt(-q) R",
  "density": "L_GG = sqrt(-g) g^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab})",
  "boundary_identity": "sqrt(-g) R - L_GG = - d_mu B^mu,  B^mu = sqrt(-g)(g^{mu nu} G^l_{nu l} - g^{l nu} G^mu
  +> _{l nu})",
  "ell_k": "-2 [eps^k] L_GG ; S_EH = sum_k kappa^{k-2} int ell_k (integrated, compact support)",
  "IBP": "total derivatives dropped only inside the integrated action (orientation of the dropped divergence i
  +> rrelevant)",
  "truncation": "O(eps^4) inclusive",
  "jets": "H{m}{n} = h_mn (m<=n), dH{m}{n}_{r} = d_r h_mn",
  "a_ghost": "1/2",
  "polarization": "V4[h1..h4] = [t1 t2 t3 t4] ell_4(sum t_i h_i) = D^4 ell_4(0)[h1..h4] (unnormalised); normal
  +> ised symmetric form = V4/4!; Feynman vertex i kappa^2 V4; D02 graph symmetry factor 1/2"
}
```

91 Source: conventions.py

[listing 58/133] code/classical/conventions.py

```
#!/usr/bin/env python3
"""XVI-P single source of conventions (declared once, read by every builder; p13 adds the milestone definition
  +> s).

Nothing here is a result. Every sign that is a *result* (antifield-shift orientation,
odd left/right derivative signs, sigma_A in S_af, s2 per sector) is DERIVED in the
builders and compared against these declarations by the tests.
"""
from __future__ import annotations
from fractions import Fraction

REVISION = "p13"
DATE = "2026-10-08"
XVI_M_PIN = "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea"

ETA = (1, -1, -1, -1)                # eta = diag(+---)
A_GHOST = Fraction(1, 2)             # a = 1/2 (XVI-N)
INDEX = [(m, n) for m in range(4) for n in range(m, 4)]   # 10 independent h_{mn}, m<=n

# Metric parametrisation (declared; s2 is DERIVED from it, not assumed)
METRIC_PARAMETRISATION = "g_{mu nu} = eta_{mu nu} + kappa h_{mu nu}  (h lowered, affine in h)"
GHOST_SCALING = "xi^mu = kappa a c^mu  (geometric diffeo parameter; XVI-N: s h = (1/2) L_c(eta + kappa h))"

# XVI-N def:antibracket (frozen orientation; used verbatim by graded_jets.bracket)
ANTIBRACKET = ("(F,G) = sum_A int [ (d^R F/d Phi^A)(d^L G/d Phi*_A) "
               "- (d^R F/d Phi*_A)(d^L G/d Phi^A) ]")
GENERATOR_RULE = "s Phi^A = (S, Phi^A);  s Phi*_A = (S, Phi*_A)  (XVI-N: NOT (Phi*,S))"

# Field table: name -> (Grassmann parity, ghost number, conjugate antifield name)
FIELDS = {
    "h":   (0, 0, "K"),     # h_{mn}, m<=n ; K^{mn} its canonical conjugate
    "c":   (1, 1, "cs"),    # c^mu (upper)
    "cb":  (1, -1, "cbs"),  # cbar_mu (lower)
    "b":   (0, 0, "bs"),    # b_mu (lower)  Nakanishi-Lautrup
    "phi": (0, 0, "phis"),  # massless scalar
}
ANTIFIELDS = {v[2]: (1 - v[0], -1 - v[1], k) for k, v in FIELDS.items()}

# Off-diagonal multiplicity of the symmetric pairing int h*^{mu nu} X_{mu nu} (all 16 comps):
#   K^{mn} = w_{mn} h*^{mn},  w = 1 (m=n), 2 (m<n).  Canonical pair: (h_{mn}, K^{mn}), m<=n.
def w_mult(m, n):
    return 1 if m == n else 2

# Gauge fermion (declared, XVI-N): Psi = -i int cbar_mu (2 D^mu[h] - b^mu),
#   D^mu[h] = d_nu h^{mu nu} - (1/2) d^mu h^rho_rho   (linear de Donder)
PSI = "Psi = -i int cbar_mu (2 D^mu[h] - b^mu),  D^mu[h] = d_nu h^{mu nu} - (1/2) d^mu tr h"

# Actions (declared normalisations)
S_EH = "S_EH = -(2/kappa^2) int sqrt(-g) R ;  ell_k = -2 [eps^k] L_GammaGamma ; S_EH = sum_k kappa^(k-2) int e
  +> ll_k (mod boundary)"

# Boundary identity (p11, CORRECTED orientation; proved generically in boundary_identity.py):
#   L_GG := sqrt(-g) g^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab}),
#   B^mu := sqrt(-g)(g^{mu nu} G^l_{nu l} - g^{l nu} G^mu_{l nu}),
#   Ricci convention of ell4_verify: R^r_{s m n} = d_m G^r_{n s} - d_n G^r_{m s} + G^r_{m l}G^l_{n s} - G^r_{n
  +>  l}G^l_{m s},
#   R_{s n} = R^r_{s r n}, R = g^{s n} R_{s n}
#   =>  sqrt(-g) R - L_GG = - d_mu B^mu     (p09/p10 stated "+ d_mu B^mu": wrong; integrated action unaffected
  +> )
```

```
BOUNDARY_IDENTITY = "sqrt(-g) R - L_GG = - d_mu B^mu,  B^mu = sqrt(-g)(g^{mu nu} G^l_{nu l} - g^{l nu} G^mu_{l
  +>  nu})"
L_GG = "L_GG = sqrt(-g) g^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab})"

# Polarisation convention (p11, ONE convention everywhere):
#   V4[h1,h2,h3,h4] := [t1 t2 t3 t4] ell_4(sum_i t_i h_i) = D^4 ell_4(0)[h1,h2,h3,h4]   (UNnormalised 4th vari
  +> ation)
#   normalised symmetric form = V4 / 4!  (not used for vertices).
#   Feynman vertex (from e^{iS}, S2 = int ell_4 at order kappa^2): i kappa^2 V4 (no 1/4!: the 1/4! of
#   S2 = (1/4!) D^4 S2[h,h,h,h] is cancelled by the 4! Wick assignments of external legs).
#   D02 (quartic tadpole, two legs contracted with the graviton propagator): graph symmetry factor 1/2 enters
  +> at
#   GRAPH level only.  D02 is not evaluated.
POLARIZATION = ("V4[h1..h4] = [t1 t2 t3 t4] ell_4(sum t_i h_i) = D^4 ell_4(0)[h1..h4] (unnormalised); "
                "normalised symmetric form = V4/4!; Feynman vertex i kappa^2 V4; D02 graph symmetry factor 1/2
  +> ")
CERT_PREFIX = "P13"
S_SCALAR = ("S_phi = (1/2) int sqrt(-g) g^{mu nu} d_mu phi d_nu phi (m=0);  "
            "sqrt(-g) g^{mu nu} = eta + kappa a1 + kappa^2 a2 (XI-A a1,a2; O2_scalar_action_authoritative)")

BASELINE = "S_inv (unprepared, sigma_eps == 1, no insertion I_t); prepared S_D keeps (S_D,S_D) != 0 (XVI-N pro
  +> p:broken-square)"

BOUNDARY_POLICY = ("Integrated identities modulo total derivatives only for the integrated action "
                   "(the sign/orientation of a discarded divergence never affects the integrated equivalence)
  +> "
                   "under compact-support (or periodic plane-wave momentum-conserving) boundary data. "
                   "No divergence is discarded after multiplication by a non-constant switch or test.")

HARD_STOPS = {
    "quantum_inputs_evaluated": "not_earned",
    "projected_identity_verified": "not_earned",
    "D02_quantum_evaluated": False,
    "D12_quantum_evaluated": False,
    "D12_status": "Missing",
    "symmetry_restored": False,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": False,
    "Route_B_discharged": False,
    "P5_opened": False,
    "loop_evaluations_performed": False,
    "u_R_c_delta_added": False,
}

NOT_PREMISES = ["En01-A", "En01-B", "En01-C", "Em01 family", "Em02-B", "Em02-D",
                "En02-A", "En02-B", "VII-E", "VIII-B"]

# ------------------------------------------------------------------------------------------------------------
  +> ---
# p13: AUTHORITATIVE milestone definitions (used verbatim by the manuscript, RELEASE_STATUS, the milestone led
  +> ger and
# the completion dossier).  Older wording ("P-next.N", "P4 = coupled one-loop restoration", "P.3--P.5 out of s
  +> cope")
# is history only: archival/p02_p09/notes/P_scope.tex and archival/p12/.  P.4 is classical readiness, NOT coup
  +> led
# one-loop restoration.
MILESTONES = {
    "P.3": "fixed output projection + exhaustive coefficient dependencies + graph topology inventory "
           "(complete for the declared ch target)",
    "P.4": "required classical action, transformations, source terms, gauge map and master verification "
           "(classical readiness earned); P.4 means classical readiness, NOT coupled one-loop restoration",
    "P.5_entry": "quantum prescription, amplitude rules and counterterm domain fixed",
    "P.5_calculation": "coefficients evaluated and renormalized",
    "P.5_identity": "projected defect computed and, if possible, cancelled",
}
MILESTONE_ORDER = ("P.3", "P.4", "P.5_entry", "P.5_calculation", "P.5_identity")
MILESTONE_HISTORY = ("archival/p02_p09/notes/P_scope.tex (p06-p08 'P-next.N' labels; its table row 'P4 coupled
  +>  one-loop "
                     "restoration' is SUPERSEDED: P.4 = classical readiness); archival/p12/ (p12 used 'classic
  +> al "
                     "readiness' without milestone labels)")

# p13: the three quantum input slots of the projected identity: NAMED DEPENDENCIES, not evaluated coefficients
  +> .
# Row ids are the surviving rows of certificates/P13_ch_source_audit.json (checked by dossier.py).
QUANTUM_INPUT_SLOTS = [
    {"slot": "hh at kappa^2", "Gamma1_pattern": "h^2", "row": "(S0,Gamma1^[2]) s2 K:K.c",
     "graphs": ["D01", "D02", "D03", "D11", "D12"]},
    {"slot": "K'c at kappa^2", "Gamma1_pattern": "K.c", "row": "(S0,Gamma1^[2]) s1 h:h^2", "graphs": ["D13"]},
    {"slot": "h one-point at kappa", "Gamma1_pattern": "h", "row": "(S1,Gamma1^[1]) s2 K:K.c.h",
```

```
    "graphs": ["D14a", "D14b", "D14c"]},
]
```

92 Source: boundary_identity.py (C3)

[listing 59/133] code/classical/boundary_identity.py

```
#!/usr/bin/env python3
"""p11 C3: the boundary identity  sqrt(-g) R - L_GG = - d_mu B^mu  (CORRECTED orientation), exact in sympy.

Definitions (Ricci convention of the former, now archived, ell4_verify.py, kept verbatim):
  G^r_{mn} = 1/2 g^{rs}(d_m g_{sn} + d_n g_{sm} - d_s g_{mn})
  R^r_{s m n} = d_m G^r_{n s} - d_n G^r_{m s} + G^r_{m l} G^l_{n s} - G^r_{n l} G^l_{m s},  R_{sn} = R^r_{s r
  +> n},
  R = g^{sn} R_{sn};   L_GG = sqrt(-g) g^{mn}(G^a_{mb}G^b_{na} - G^a_{mn}G^b_{ab});
  B^mu = sqrt(-g)(g^{mu nu} G^l_{nu l} - g^{l nu} G^mu_{l nu}).

(0) VACUITY of the p09/p10 C3: on g = eta + A(x0) e_00 e_00 only G^0_00 != 0 and R = L_GG = B = 0, so the old
    check compared 0 with 0 (it could not detect the orientation error).
(i) Orientation: the correct statement is sqrt(-g)R - L_GG + d_mu B^mu = 0; "+ d_mu B^mu" (p09/p10) is wrong.
(ii) Nontrivial conformal control: g = f(x0,x1,x2,x3) eta, generic f, exact:
     sqrt(-g)R = -3 Box f + (3/(2f))(df)^2, L_GG = (3/(2f))(df)^2, B^mu = 3 d^mu f; the old orientation leaves
  +>  -6 Box f.
(iii) Generic jet proof (all 10 metric components, symbolic first and second jets, exact polynomial ring over
  +> QQ):
     with G^{ab} the inverse metric as 10 free symmetric symbols, d_c G^{ab} := -G^{ar} d_c g_{rs} G^{sb} (fro
  +> m
     G g = 1) and the Jacobi lemma d_mu sqrt(-g) = (1/2) sqrt(-g) G^{ab} d_mu g_{ab}, the identity divided by
     sqrt(-g) reads   R - l + d_mu b^mu + (1/2) G^{ab} d_mu g_{ab} b^mu = 0   (L_GG = sqrt(-g) l, B = sqrt(-g)
  +>  b),
     which is verified as an IDENTICALLY vanishing polynomial in 10 + 40 + 100 variables (it holds even for fr
  +> ee G).
     Jacobi lemma: d/dt det(g + t delta)|_0 = tr(adj(g) delta) and adj(g) g = det(g) 1, polynomial identities
  +> in
     10 + 10 generic symmetric symbols; hence d ln(-det g) = G^{ab} dg_{ab} on G = g^{-1}.
Consequence: S_EH = -(2/kappa^2) int sqrt(-g)R = -(2/kappa^2) int L_GG for compactly supported h (a divergence
integrates to zero whatever its sign), so the integrated-action identification of ell_k is UNAFFECTED by the
orientation correction; only the stated boundary vector orientation changes.
"""
from __future__ import annotations
import json, sys, time
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
import conventions as C  # noqa: E402
import certio  # noqa: E402

ETA = C.ETA
X = sp.symbols("x0:4")


def _geom(g, ginv, sq):
    Gam = [[[sp.simplify(sum(ginv[r, s] * (sp.diff(g[s, n], X[m]) + sp.diff(g[s, m], X[n]) - sp.diff(g[m, n],
  +> X[s]))
                             for s in range(4)) / 2) for n in range(4)] for m in range(4)] for r in range(4)]

    def riem(r, sig, m, n):
        e = sp.diff(Gam[r][n][sig], X[m]) - sp.diff(Gam[r][m][sig], X[n])
        e += sum(Gam[r][m][l] * Gam[l][n][sig] - Gam[r][n][l] * Gam[l][m][sig] for l in range(4))
        return e
    Ric = sp.Matrix(4, 4, lambda s, n: sum(riem(r, s, r, n) for r in range(4)))
    SR = sp.simplify(sq * sum(ginv[a, b] * Ric[a, b] for a in range(4) for b in range(4)))
    L = 0
    for mu in range(4):
        for nu in range(4):
            L += ginv[mu, nu] * (sum(Gam[a][mu][b] * Gam[b][nu][a] for a in range(4) for b in range(4))
                                 - sum(Gam[a][mu][nu] * Gam[b][a][b] for a in range(4) for b in range(4)))
    L = sp.simplify(sq * L)
    B = [sp.simplify(sq * sum(ginv[mu, nu] * Gam[l][nu][l] - ginv[l, nu] * Gam[mu][l][nu]
                              for nu in range(4) for l in range(4))) for mu in range(4)]
    divB = sp.simplify(sum(sp.diff(B[m], X[m]) for m in range(4)))
    return Gam, SR, L, B, divB


def vacuity_of_old_support():
```

```
    A = sp.Function("A")(X[0])
    eta = sp.diag(*ETA)
    H = sp.zeros(4)
    H[0, 0] = A
    g = eta + H
    Gam, SR, L, B, divB = _geom(g, g.inv(), sp.sqrt(-g.det()))
    nz = [f"G^{r}_{m}{n}" for r in range(4) for m in range(4) for n in range(m, 4) if sp.simplify(Gam[r][m][n]
  +> ) != 0]
    return {"support": "g = eta + A(x0) e_00 (p09/p10 C3)", "nonzero_Christoffels": nz,
            "sqrt_g_R": str(SR), "L_GG": str(L), "B": [str(b) for b in B], "div_B": str(divB),
            "vacuous": SR == 0 and L == 0 and all(b == 0 for b in B) and divB == 0,
            "why": "locally flat: x0 -> int sqrt(1+A) dx0 removes A; Gamma^0_00 alone gives R = 0, both GammaG
  +> amma "
                   "contractions equal (Gamma^0_00)^2 g^00 and cancel, and B^0 = sqrt(-g)(g^00 G^0_00 - g^00 G
  +> ^0_00) = 0. "
                   "The p10 source comment 'keeps dB nontrivial' was false."}


def conformal_control(orientation=+1):
    """orientation=+1: sqrt(-g)R - L + dB (correct); orientation=-1: sqrt(-g)R - L - dB (p09/p10 statement)."""
  +> "
    f = sp.Function("f")(*X)
    eta = sp.diag(*ETA)
    Gam, SR, L, B, divB = _geom(f * eta, eta / f, f ** 2)
    box = sum(ETA[m] * sp.diff(f, X[m], 2) for m in range(4))
    df2 = sum(ETA[m] * sp.diff(f, X[m]) ** 2 for m in range(4))
    resid = sp.simplify(SR - L + orientation * divB)
    return {
        "metric": "g = f(x0,x1,x2,x3) eta, generic f (all four coordinates), exact",
        "orientation": "sqrt(-g)R - L_GG + d_mu B^mu" if orientation > 0 else "sqrt(-g)R - L_GG - d_mu B^mu (p
  +> 09/p10)",
        "residual": str(resid), "residual_zero": resid == 0,
        "closed_forms_verified": {
            "sqrt(-g)R = -3 Box f + (3/(2f))(df)^2": sp.simplify(SR - (-3 * box + sp.Rational(3, 2) * df2 / f)
  +> ) == 0,
            "L_GG = (3/(2f))(df)^2": sp.simplify(L - sp.Rational(3, 2) * df2 / f) == 0,
            "B^mu = 3 d^mu f": all(sp.simplify(B[m] - 3 * ETA[m] * sp.diff(f, X[m])) == 0 for m in range(4)),
        },
        "nontrivial": SR != 0 and L != 0 and divB != 0 and any(b != 0 for b in B),
    }


def _ring_setup():
    from sympy.polys.rings import ring
    from sympy import QQ
    IDX = [(a, b) for a in range(4) for b in range(a, 4)]
    names = ([f"G{a}{b}" for a, b in IDX] + [f"g{c}_{a}{b}" for c in range(4) for a, b in IDX]
             + [f"gg{c}{d}_{a}{b}" for c in range(4) for d in range(c, 4) for a, b in IDX])
    R_, *gens = ring(",".join(names), QQ)
    return R_, dict(zip(names, gens)), QQ


def generic_jet_proof(orientation=+1):
    t0 = time.time()
    R_, V, QQ = _ring_setup()
    S = lambda a, b: (min(a, b), max(a, b))  # noqa: E731
    G = lambda a, b: V["G%d%d" % S(a, b)]  # noqa: E731
    dg = lambda c, a, b: V["g%d_%d%d" % ((c,) + S(a, b))]  # noqa: E731

    def ddg(c, d, a, b):
        c, d = min(c, d), max(c, d)
        return V["gg%d%d_%d%d" % ((c, d) + S(a, b))]
    Z = R_(0)
    r4 = range(4)
    dG = {(c, a, b): sum((-G(a, r) * dg(c, r, s) * G(s, b) for r in r4 for s in r4), Z) for c in r4 for a in r
  +> 4 for b in r4}
    T = lambda m, s, n: dg(m, s, n) + dg(n, s, m) - dg(s, m, n)  # noqa: E731
    dT = lambda c, m, s, n: ddg(c, m, s, n) + ddg(c, n, s, m) - ddg(c, s, m, n)  # noqa: E731
    half = QQ(1, 2)
    Gam = {(r, m, n): sum((G(r, s) * T(m, s, n) for s in r4), Z) * half for r in r4 for m in r4 for n in r4}
    dGam = {(c, r, m, n): sum((dG[(c, r, s)] * T(m, s, n) + G(r, s) * dT(c, m, s, n) for s in r4), Z) * half
            for c in r4 for r in r4 for m in r4 for n in r4}

    def riem(r, sig, m, n):
        e = dGam[(m, r, n, sig)] - dGam[(n, r, m, sig)]
        for l in r4:
            e += Gam[(r, m, l)] * Gam[(l, n, sig)] - Gam[(r, n, l)] * Gam[(l, m, sig)]
        return e
    Ric = {(s, n): sum((riem(r, s, r, n) for r in r4), Z) for s in r4 for n in r4}
    Rs = sum((G(a, b) * Ric[(a, b)] for a in r4 for b in r4), Z)
    trG = [sum((Gam[(b, a, b)] for b in r4), Z) for a in r4]
```

```
    ell = Z
    for mu in r4:
        for nu in r4:
            ell += G(mu, nu) * (sum((Gam[(a, mu, b)] * Gam[(b, nu, a)] for a in r4 for b in r4), Z)
                                - sum((Gam[(a, mu, nu)] * trG[a] for a in r4), Z))
    b = [sum((G(mu, nu) * trG[nu] - sum((G(l, nu) * Gam[(mu, l, nu)] for l in r4), Z) for nu in r4), Z) for mu
  +>  in r4]
    dtrG = {(c, a): sum((dGam[(c, bb, a, bb)] for bb in r4), Z) for c in r4 for a in r4}
    divb = Z
    for mu in r4:
        for nu in r4:
            divb += dG[(mu, mu, nu)] * trG[nu] + G(mu, nu) * dtrG[(mu, nu)]
            for l in r4:
                divb -= dG[(mu, l, nu)] * Gam[(mu, l, nu)] + G(l, nu) * dGam[(mu, mu, l, nu)]
    jac = [sum((G(a, bb) * dg(m, a, bb) for a in r4 for bb in r4), Z) * half for m in r4]
    divB_over_sqrt = divb + sum((jac[m] * b[m] for m in r4), Z)        # (1/sqrt(-g)) d_mu (sqrt(-g) b^mu)
    resid = Rs - ell + orientation * divB_over_sqrt
    return {"variables": {"inverse_metric_free_symmetric": 10, "first_jets": 40, "second_jets": 100},
            "orientation": "+d_mu B^mu (correct)" if orientation > 0 else "-d_mu B^mu (p09/p10)",
            "terms": {"R": len(Rs.terms()), "l": len(ell.terms()), "div_b": len(divb.terms())},
            "residual_terms": len(resid.terms()), "identically_zero": resid == 0,
            "holds_for_free_G": resid == 0, "seconds": round(time.time() - t0, 2)}


def jacobi_lemma():
    """d/dt det(g + t d)|_0 = tr(adj(g) d)  and  adj(g) g = det(g) 1 for generic symmetric g, d (10 + 10 symbo
  +> ls)."""
    gs = sp.Matrix(4, 4, lambda a, b: sp.Symbol(f"q{min(a, b)}{max(a, b)}"))
    ds = sp.Matrix(4, 4, lambda a, b: sp.Symbol(f"d{min(a, b)}{max(a, b)}"))
    t = sp.Symbol("t")
    det = (gs + t * ds).det(method="berkowitz")
    lhs = sp.expand(sp.diff(det, t).subs(t, 0))
    adj = gs.adjugate()
    rhs = sp.expand(sum(adj[a, b] * ds[b, a] for a in range(4) for b in range(4)))
    adj_ok = sp.expand(adj * gs - gs.det() * sp.eye(4)) == sp.zeros(4, 4)
    return {"derivative_of_det_equals_tr_adj": sp.expand(lhs - rhs) == 0, "adj_g_equals_det_one": adj_ok,
            "consequence": "d_mu sqrt(-g) = (1/2) sqrt(-g) g^{ab} d_mu g_ab ; d_c g^{ab} = - g^{ar} d_c g_rs g
  +> ^{sb}"}


def run():
    t0 = time.time()
    vac = vacuity_of_old_support()
    conf = conformal_control(+1)
    conf_old = conformal_control(-1)
    gen = generic_jet_proof(+1)
    gen_old = generic_jet_proof(-1)
    jac = jacobi_lemma()
    ok = (vac["vacuous"] and conf["residual_zero"] and conf["nontrivial"] and all(conf["closed_forms_verified"
  +> ].values())
          and not conf_old["residual_zero"] and gen["identically_zero"] and not gen_old["identically_zero"]
          and jac["derivative_of_det_equals_tr_adj"] and jac["adj_g_equals_det_one"])
    return {
        "revision": C.REVISION, "gate": "C3_boundary_identity",
        "statement": C.BOUNDARY_IDENTITY, "L_GG": C.L_GG,
        "orientation_correction": "p09/p10 stated sqrt(-q)R - L_GG = + d_mu B^mu; correct is - d_mu B^mu",
        "old_C3_vacuity": vac,
        "conformal_control_correct_orientation": conf,
        "conformal_control_old_orientation_fails": {"residual": conf_old["residual"], "fails": not conf_old["r
  +> esidual_zero"],
                                                    "equals": "-6 Box f"},
        "generic_jet_proof": gen, "generic_jet_proof_old_orientation": gen_old,
        "jacobi_lemma": jac,
        "integrated_action_unaffected": ("int d_mu B^mu = 0 under compact support for either sign, so "
                                         "S_EH = -(2/kappa^2) int L_GG and ell_k = -2[eps^k]L_GG are unchanged
  +> "),
        "evidence_kind": "deterministic exact symbolic (sympy; polynomial ring over QQ for (iii))",
        "C3_proved": ok, "execution_pass": True, "claim_status": "proved" if ok else "Partial",
        "scope": "exact non-perturbative identity on the jet space of a generic metric (any signature-compatib
  +> le g)",
        "seconds": round(time.time() - t0, 1),
    }


def main():
    cert = run()
    (ROOT / "certificates" / f"{C.CERT_PREFIX}_boundary_identity.json").write_text(certio.dumps(cert))
    print(json.dumps({k: cert[k] for k in ("C3_proved", "seconds")}, indent=1),
          cert["generic_jet_proof"]["residual_terms"], cert["generic_jet_proof_old_orientation"]["residual_ter
  +> ms"])
    return 0 if cert["C3_proved"] else 1
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

93 Source: jetpoly.py

[listing 60/133] code/classical/jetpoly.py

```
#!/usr/bin/env python3
"""p11: exact sparse polynomial algebra on the (even) jet space of h_{mn} (m<=n) and c^mu, with total
derivatives D_r, partial derivatives and Euler operators.  Coefficients: fractions.Fraction.  Deterministic.

Variable key = (field, comp, J): field in {"h","c"}, comp=(m,n) m<=n for h, mu for c (upper index),
J = sorted tuple of derivative directions.  Polynomial = dict{monomial(sorted tuple of var ids): Fraction}.
Here c is treated as a commuting placeholder: every density used is LINEAR in c, so grading is irrelevant.

Divergence lemma used for c-linear densities W:  W = c^mu Q_mu + d_r(...) with Q_mu = E_{c^mu}(W)
(integration by parts), so W is a total divergence iff E_{c^mu}(W) = 0 for all mu.
"""
from __future__ import annotations
from collections import defaultdict
from fractions import Fraction as Fr

KEYS: list = []
IDS: dict = {}


def vid(key):
    i = IDS.get(key)
    if i is None:
        i = len(KEYS)
        KEYS.append(key)
        IDS[key] = i
    return i


def var(field, comp, J=()):
    return {(vid((field, comp, tuple(sorted(J)))),): Fr(1)}


def const(x):
    return {(): Fr(x)} if x else {}


def add(*ps):
    out = defaultdict(Fr)
    for p in ps:
        for m, c in p.items():
            out[m] += c
    return {m: c for m, c in out.items() if c}


def scale(p, x):
    x = Fr(x)
    return {m: c * x for m, c in p.items()} if x else {}


def mul(p, q):
    out = defaultdict(Fr)
    for m1, c1 in p.items():
        for m2, c2 in q.items():
            out[tuple(sorted(m1 + m2))] += c1 * c2
    return {m: c for m, c in out.items() if c}


def shifted(i, r):
    f, comp, J = KEYS[i]
    return vid((f, comp, tuple(sorted(J + (r,)))))


def D(p, r):
    out = defaultdict(Fr)
    for m, c in p.items():
        for pos in range(len(m)):
            if pos and m[pos] == m[pos - 1]:
                continue
            k = m.count(m[pos])
```

```
            new = list(m)
            new[pos] = shifted(m[pos], r)
            out[tuple(sorted(new))] += c * k
    return {m: c for m, c in out.items() if c}


def pdiff(p, i):
    out = defaultdict(Fr)
    for m, c in p.items():
        k = m.count(i)
        if k:
            new = list(m)
            new.remove(i)
            out[tuple(new)] += c * k
    return {m: c for m, c in out.items() if c}


def euler(p, field, comp):
    """E_u(p) = sum_J (-D)_J dp/du_J."""
    js = sorted({KEYS[i][2] for m in p for i in m if KEYS[i][0] == field and KEYS[i][1] == comp})
    out = {}
    for J in js:
        t = pdiff(p, vid((field, comp, J)))
        for r in J:
            t = scale(D(t, r), -1)
        out = add(out, t)
    return out


def from_sympy(expr, symmap):
    """sympy polynomial -> jet poly, symmap: sympy Symbol name -> key."""
    import sympy as sp
    syms = [sp.Symbol(n) for n in symmap]
    P = sp.Poly(expr, *syms)
    ids = [vid(symmap[s.name]) for s in syms]
    out = {}
    for mon, c in P.terms():
        m = []
        for i, e in zip(ids, mon):
            m += [i] * e
        c = sp.Rational(c)
        out[tuple(sorted(m))] = Fr(int(c.p), int(c.q))
    return out


def ell_symmap():
    INDEX = [(m, n) for m in range(4) for n in range(m, 4)]
    sm = {f"H{m}{n}": ("h", (m, n), ()) for m, n in INDEX}
    sm.update({f"dH{m}{n}_{r}": ("h", (m, n), (r,)) for m, n in INDEX for r in range(4)})
    return sm


def describe(p, k=3):
    return [(" ".join(f"{KEYS[i][0]}{KEYS[i][1]}{'_' + ''.join(map(str, KEYS[i][2])) if KEYS[i][2] else ''}"
                      for i in m), str(c)) for m, c in list(p.items())[:k]]
```

94 Source: certio.py (certificate serialization + invariants)

[listing 61/133] code/classical/certio.py

```
#!/usr/bin/env python3
"""p12: certificate serialization + residual-record invariants (shared by every certificate writer and the rel
  +> ease gate).

Why (p12 root cause): the p11 appendix typeset multi-line JSON arrays; a page break inside a Q_terms array
put the footer page number ("12", "29") between two array elements, and a plain text extraction of the PDF rea
  +> d
the kappa0 quartic counts as 0,0,120,0 and the kappa1 master counts as 290,0,0,0.  The certificate files thems
  +> elves were correct.  p12 therefore
  * writes every list of scalars on ONE line (cannot be split by a page break),
  * enforces, before serialization, the invariant  Q_identically_zero == all(n == 0 for n in Q_terms)
    on every residual record (any dict carrying both keys), and
  * provides the acceptance and cross-certificate checks used by quartic.py, master_kappa2.py and release.py.
"""
from __future__ import annotations
import json
from pathlib import Path
```

```
ORDERS = ("kappa0", "kappa1", "kappa2")
REC_KEYS = ("W_terms", "Q_terms", "Q_identically_zero", "s0_piece_alone_Q_terms")


class CertificateInvariantError(AssertionError):
    pass


def _scalar(x):
    return x is None or isinstance(x, (bool, int, float, str))


def _enc(o, ind, step=2):
    pad, pad2 = " " * ind, " " * (ind + step)
    if isinstance(o, dict):
        if not o:
            return "{}"
        items = [f"{pad2}{json.dumps(str(k))}: {_enc(v, ind + step)}" for k, v in o.items()]
        return "{\n" + ",\n".join(items) + "\n" + pad + "}"
    if isinstance(o, (list, tuple)):
        if not o:
            return "[]"
        if all(_scalar(x) for x in o):
            return "[" + ", ".join(json.dumps(x) for x in o) + "]"
        return "[\n" + ",\n".join(pad2 + _enc(x, ind + step) for x in o) + "\n" + pad + "]"
    if _scalar(o):
        return json.dumps(o)
    return json.dumps(str(o))


def residual_records(obj, path=""):
    """Yield (path, record) for every dict carrying both Q_terms and Q_identically_zero."""
    if isinstance(obj, dict):
        if "Q_terms" in obj and "Q_identically_zero" in obj:
            yield path, obj
        for k, v in obj.items():
            yield from residual_records(v, f"{path}/{k}")
    elif isinstance(obj, (list, tuple)):
        for i, v in enumerate(obj):
            yield from residual_records(v, f"{path}[{i}]")


def invariant_violations(obj):
    bad = []
    for p, r in residual_records(obj):
        counts = r["Q_terms"]
        if not (isinstance(counts, list) and all(isinstance(n, int) and n >= 0 for n in counts)):
            bad.append(f"{p}: Q_terms not a list of nonnegative ints: {counts!r}")
        elif bool(r["Q_identically_zero"]) != all(n == 0 for n in counts):
            bad.append(f"{p}: Q_identically_zero={r['Q_identically_zero']} but Q_terms={counts}")
    return bad


def dumps(obj):
    bad = invariant_violations(obj)
    if bad:
        raise CertificateInvariantError("; ".join(bad))
    return _enc(obj, 0) + "\n"


def write(path, obj):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(dumps(obj))


def accepted_orders(block):
    """Acceptance: Q_terms == [0,0,0,0] AND Q_identically_zero at every order kappa0..kappa2."""
    out = {}
    for o in ORDERS:
        r = block.get(o, {})
        out[o] = r.get("Q_terms") == [0, 0, 0, 0] and r.get("Q_identically_zero") is True
    return out


def cross_check(q_block, q_params, m_block, m_params):
    """Same export hashes + transformation parameters must give identical residual records."""
    same_inputs = q_params == m_params
    diffs = []
    if same_inputs:
        for o in ORDERS:
            for k in REC_KEYS:
```

```
                a, b = q_block.get(o, {}).get(k), m_block.get(o, {}).get(k)
                if a != b:
                    diffs.append(f"{o}.{k}: quartic={a} master={b}")
    return {"same_inputs": same_inputs, "differences": diffs, "agree": same_inputs and not diffs,
            "quartic_parameters": q_params, "master_parameters": m_params}


# ---------------- p13: content hashes that ignore wall-clock timings (reproducibility comparison) -----------
  +> -----
VOLATILE_KEYS = frozenset({"seconds"})          # wall-clock timings: the only run-dependent values in a certi
  +> ficate


def strip_volatile(obj, extra=()):
    drop = VOLATILE_KEYS | frozenset(extra)
    if isinstance(obj, dict):
        return {k: strip_volatile(v, extra) for k, v in obj.items() if k not in drop}
    if isinstance(obj, (list, tuple)):
        return [strip_volatile(v, extra) for v in obj]
    return obj


def content_sha(obj, extra=()):
    """sha256 of the canonical JSON (sorted keys, compact) after removing timing keys (and `extra` keys)."""
    import hashlib
    txt = json.dumps(strip_volatile(obj, extra), sort_keys=True, separators=(",", ":"), ensure_ascii=True)
    return hashlib.sha256(txt.encode()).hexdigest()


def file_content_sha(path, extra=()):
    return content_sha(json.loads(Path(path).read_text()), extra)
```

95 Source: grav_noether.py

[listing 62/133] code/classical/grav_noether.py

```
#!/usr/bin/env python3
"""p11: DETERMINISTIC gravitational Noether identities and divergence-equality tests (exact, jetpoly).

Noether at order kappa^n (n = 0, 1, 2), antifield number 0 of 2(S_field, X):
    W_n = sum_{m<=n'} E_{(mn')}(ell_{n+2}) (s0 h)_{mn'} + E_{(mn')}(ell_{n+1}) (s1 h)_{mn'}   [+ E(ell_n) s2 h
  +> , s2 = 0 derived]
with E the Euler operator for the independent component h_{mn'} (m<=n'), s0 h = a(d_m c_n + d_n c_m),
s1 h = a L_c h = a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r), a = 1/2, c_n = eta_nn c^n.
W_n is linear in c, so W_n = c^mu Q_mu + divergence with Q_mu = E_{c^mu}(W_n); hence
    W_n is a total divergence  <=>  Q_mu = 0 identically (polynomial identity on the jet space).
This is an exact symbolic statement about the EXPORTED polynomials (no sampling).
"""
from __future__ import annotations
from fractions import Fraction as Fr
import jetpoly as J

ETA = (1, -1, -1, -1)
INDEX = [(m, n) for m in range(4) for n in range(m, 4)]
A = Fr(1, 2)


def _h(m, n):
    return J.var("h", (min(m, n), max(m, n)))


def _dh(m, n, r):
    return J.var("h", (min(m, n), max(m, n)), (r,))


def _c(mu, Jd=()):
    return J.var("c", mu, Jd)


def s0(m, n, a=A):
    return J.scale(J.add(J.scale(_c(n, (m,)), ETA[n]), J.scale(_c(m, (n,)), ETA[m])), a)


def s1(m, n, a=A, transport=1):
    t = []
    for r in range(4):
        if transport:
            t.append(J.mul(_c(r), _dh(m, n, r)))
```

```
        t.append(J.mul(_h(r, n), _c(r, (m,))))
        t.append(J.mul(_h(m, r), _c(r, (n,))))
    return J.scale(J.add(*t), a)


def load_export_polys():
    import planewave_exact as PW
    sm = J.ell_symmap()
    out, sha = {}, {}
    for k in (2, 3, 4):
        e, h = PW.load_ell(k)
        out[k] = J.from_sympy(e, sm)
        sha[k] = h
    return out, sha


def ring_to_jet(poly, names):
    sm = J.ell_symmap()
    ids = [J.vid(sm[nm]) for nm in names]
    out = {}
    for mon, c in poly.terms():
        m = []
        for i, e in zip(ids, mon):
            m += [i] * e
        out[tuple(sorted(m))] = Fr(int(c.numerator), int(c.denominator))
    return out


def euler_all(p):
    return {mn: J.euler(p, "h", mn) for mn in INDEX}


def _nz(p):
    """Drop zero-valued entries (jetpoly already never stores them; this makes the count/flag pair robust)."""
    return {m: c for m, c in p.items() if c != 0}


def parameters(shas=None, transport=1, a=A):
    return {"a_ghost": str(a), "transport_term": int(transport),
            "s0": "a(eta_nn d_m c^n + eta_mm d_n c^m)", "s1": "a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r)",
            "ell_sha256": {f"ell{k}": shas[k] for k in sorted(shas)} if shas else None}


def noether(ell, transport=1):
    """ell: {2,3,4: jetpoly}.  Returns per order: Q term counts, zero flag, and the s0-piece-alone Q (mixing).
    Count and flag are derived from the SAME normalised list Q: zero == all(n == 0 for n in counts) (asserted)
  +> ."""
    E = {k: euler_all(ell[k]) for k in (2, 3, 4)}
    out = {}
    for n in (0, 1, 2):
        W0 = J.add(*[J.mul(E[n + 2][mn], s0(*mn)) for mn in INDEX])
        W = W0
        if n >= 1:
            W = J.add(W0, *[J.mul(E[n + 1][mn], s1(*mn, transport=transport)) for mn in INDEX])
        Q = [_nz(J.euler(W, "c", mu)) for mu in range(4)]
        Q0 = [_nz(J.euler(W0, "c", mu)) for mu in range(4)] if n >= 1 else Q
        counts = [len(q) for q in Q]
        zero = all(c == 0 for c in counts)
        assert zero == all(not q for q in Q), "count/flag invariant violated"
        out[f"kappa{n}"] = {"density": {0: "s0 ell2", 1: "s0 ell3 + s1 ell2", 2: "s0 ell4 + s1 ell3 (+ s2 ell2
  +> , s2 = 0)"}[n],
                            "W_terms": len(_nz(W)), "Q_terms": counts, "Q_identically_zero": zero,
                            "s0_piece_alone_Q_terms": [len(q) for q in Q0],
                            "residual_head": J.describe(Q[0]) if Q[0] else []}
    return out


def divergence_equal(p, q):
    """p - q is a total divergence  <=>  E_{h_mn}(p - q) = 0 for all 10 components (no constant term)."""
    d = J.add(p, J.scale(q, -1))
    res = {f"{m}{n}": len(J.euler(d, "h", (m, n))) for m, n in INDEX}
    return {"difference_terms": len(d), "euler_terms": res, "divergence": all(v == 0 for v in res.values())}
```

96 Source: graded_jets.py

[listing 63/133] code/classical/graded_jets.py

```
#!/usr/bin/env python3
```

```
"""Graded (Grassmann) jet algebra for local BV functionals (p10; unchanged in p11).

Element = density in the superjet algebra:  {odd_monomial(tuple of odd jet keys, canonical
order): commuting sympy coefficient in even jet symbols}.  Jet key = (field, comp, J) with
J a sorted tuple of derivative directions.  Provides graded product, total derivatives D_a,
left/right partial derivatives, left/right Euler operators, the XVI-N antibracket
    (F,G) = sum_A [ E^R_{Phi^A}(F) E^L_{Phi*_A}(G) - E^R_{Phi*_A}(F) E^L_{Phi^A}(G) ],
and the divergence test  f ~ 0  <=>  E_u(f) = 0 for every generator u (no constant term).
Nothing in this module chooses a physics sign; it only implements graded calculus.
"""
from __future__ import annotations
import itertools
import sympy as sp

FIELD_ORDER = ["h", "c", "cb", "b", "phi", "K", "cs", "cbs", "bs", "phis", "lam", "mu", "z"]
_FORD = {f: i for i, f in enumerate(FIELD_ORDER)}
PARITY = {"h": 0, "c": 1, "cb": 1, "b": 0, "phi": 0,
          "K": 1, "cs": 0, "cbs": 0, "bs": 1, "phis": 1,
          "lam": 0, "mu": 0, "z": 1}
PAIRS = [("h", "K"), ("c", "cs"), ("cb", "cbs"), ("b", "bs"), ("phi", "phis")]
H_COMPS = [(m, n) for m in range(4) for n in range(m, 4)]
VEC_COMPS = [0, 1, 2, 3]


def comps(field):
    if field in ("h", "K"):
        return H_COMPS
    if field in ("phi", "phis"):
        return [()]
    return VEC_COMPS


def set_parity(field, p):
    PARITY[field] = p


def _ckey(comp):
    return comp if isinstance(comp, tuple) else (comp,)


def sortkey(key):
    f, c, J = key
    return (_FORD[f], _ckey(c), len(J), J)


_SYM = {}
_KEY = {}


def esym(key):
    s = _SYM.get(key)
    if s is None:
        f, c, J = key
        cs = "".join(map(str, _ckey(c)))
        js = ("_d" + "".join(map(str, J))) if J else ""
        s = sp.Symbol(f"{f}{cs}{js}")
        _SYM[key] = s
        _KEY[s] = key
    return s


def shift(key, a):
    f, c, J = key
    return (f, c, tuple(sorted(J + (a,))))


def _sort_sign(seq):
    """Return (sign, sorted tuple) or (0, None) if a repeated odd generator occurs."""
    lst = list(seq)
    n = len(lst)
    sign = 1
    keys = [sortkey(x) for x in lst]
    for i in range(n):
        for j in range(n - 1 - i):
            if keys[j] > keys[j + 1]:
                keys[j], keys[j + 1] = keys[j + 1], keys[j]
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
                sign = -sign
            elif keys[j] == keys[j + 1]:
                return 0, None
    return sign, tuple(lst)
```

```
class GE:
    __slots__ = ("t",)

    def __init__(self, t=None):
        self.t = {}
        if t:
            for k, v in t.items():
                v = sp.expand(v)
                if v != 0:
                    self.t[k] = v

    # constructors
    @staticmethod
    def const(x):
        return GE({(): sp.sympify(x)})

    @staticmethod
    def gen(field, comp, J=()):
        key = (field, comp, tuple(sorted(J)))
        if PARITY[field] == 0:
            return GE({(): esym(key)})
        return GE({(key,): sp.Integer(1)})

    def copy(self):
        g = GE()
        g.t = dict(self.t)
        return g

    def is_zero(self):
        return not self.t

    def __add__(self, o):
        if not isinstance(o, GE):
            o = GE.const(o)
        r = dict(self.t)
        for k, v in o.t.items():
            r[k] = sp.expand(r.get(k, 0) + v)
            if r[k] == 0:
                del r[k]
        g = GE(); g.t = r
        return g

    __radd__ = __add__

    def __neg__(self):
        g = GE(); g.t = {k: -v for k, v in self.t.items()}
        return g

    def __sub__(self, o):
        return self + (-o if isinstance(o, GE) else GE.const(-o))

    def __rsub__(self, o):
        return GE.const(o) + (-self)

    def scale(self, x):
        x = sp.sympify(x)
        return GE({k: v * x for k, v in self.t.items()})

    def __mul__(self, o):
        if not isinstance(o, GE):
            return self.scale(o)
        r = {}
        for k1, v1 in self.t.items():
            for k2, v2 in o.t.items():
                sgn, k = _sort_sign(k1 + k2)
                if sgn == 0:
                    continue
                r[k] = r.get(k, 0) + sgn * v1 * v2
        return GE(r)

    def __rmul__(self, x):
        return self.scale(x)

    def parity_terms(self):
        return {len(k) % 2 for k in self.t}

    def keys_of(self, field=None, comp=None):
        """All jet keys (even & odd) occurring."""
        out = set()
```

```
        for k, v in self.t.items():
            for x in k:
                out.add(x)
            for s in v.free_symbols:
                kk = _KEY.get(s)
                if kk is not None:
                    out.add(kk)
        if field is not None:
            out = {x for x in out if x[0] == field and (comp is None or x[1] == comp)}
        return out

    def __repr__(self):
        return " + ".join(f"({v})*{list(k)}" for k, v in self.t.items()) or "0"


def D(F: GE, a: int) -> GE:
    """Total derivative d_a (even derivation)."""
    r = {}
    for k, v in F.t.items():
        # coefficient
        dv = 0
        for s in v.free_symbols:
            kk = _KEY.get(s)
            if kk is not None:
                dv += sp.diff(v, s) * esym(shift(kk, a))
        if dv != 0:
            r[k] = r.get(k, 0) + dv
        # odd generators
        for i in range(len(k)):
            nk = k[:i] + (shift(k[i], a),) + k[i + 1:]
            sgn, sk = _sort_sign(nk)
            if sgn:
                r[sk] = r.get(sk, 0) + sgn * v
    return GE(r)


def DJ(F, J):
    for a in J:
        F = D(F, a)
    return F


def pdiff(F: GE, key, side="L") -> GE:
    """Left/right partial derivative wrt a jet generator."""
    f = key[0]
    r = {}
    if PARITY[f] == 0:
        s = esym(key)
        for k, v in F.t.items():
            dv = sp.diff(v, s)
            if dv != 0:
                r[k] = r.get(k, 0) + dv
    else:
        for k, v in F.t.items():
            if key in k:
                i = k.index(key)
                sgn = (-1) ** i if side == "L" else (-1) ** (len(k) - 1 - i)
                nk = k[:i] + k[i + 1:]
                r[nk] = r.get(nk, 0) + sgn * v
    return GE(r)


def euler(F: GE, field, comp, side="L") -> GE:
    out = GE()
    for key in F.keys_of(field, comp):
        J = key[2]
        term = DJ(pdiff(F, key, side), J)
        if len(J) % 2:
            term = -term
        out = out + term
    return out


def bracket(F: GE, G: GE, pairs=PAIRS) -> GE:
    """XVI-N antibracket density (orientation frozen in conventions.ANTIBRACKET)."""
    out = GE()
    for f, fs in pairs:
        for c in comps(f):
            a = euler(F, f, c, "R")
            if not a.is_zero():
                b = euler(G, fs, c, "L")
```

```
                if not b.is_zero():
                    out = out + a * b
            a2 = euler(F, fs, c, "R")
            if not a2.is_zero():
                b2 = euler(G, f, c, "L")
                if not b2.is_zero():
                    out = out - a2 * b2
    return out


def divergence_residual(F: GE):
    """Return {(field,comp): E^L_u(F)} for all u with nonzero Euler image (empty <=> F ~ 0)."""
    res = {}
    fields = {k[0] for k in F.keys_of()}
    for f in fields:
        for c in comps(f) if f not in ("lam", "mu", "z") else sorted({k[1] for k in F.keys_of(f)}, key=str):
            e = euler(F, f, c, "L")
            if not e.is_zero():
                res[(f, c)] = e
    return res


def is_divergence(F: GE) -> bool:
    return not divergence_residual(F)


def kappa_coeff(F: GE, kappa, n) -> GE:
    return GE({k: sp.expand(v).coeff(kappa, n) for k, v in F.t.items()})


def subst(F: GE, key_map: dict) -> GE:
    """Substitute undifferentiated generators: key_map[(field,comp)] = GE (same parity).
    Generators with derivative jets in F for substituted fields raise."""
    out = GE()
    for k, v in F.t.items():
        # even part: expand v as polynomial in substituted even symbols
        evsubs = {}
        for s in v.free_symbols:
            kk = _KEY.get(s)
            if kk is not None and (kk[0], kk[1]) in key_map and PARITY[kk[0]] == 0:
                if kk[2]:
                    raise ValueError(f"derivative jet of substituted even field: {kk}")
                evsubs[s] = key_map[(kk[0], kk[1])]
        # build coefficient as GE
        if evsubs:
            syms = list(evsubs)
            P = sp.Poly(v, *syms)
            coefGE = GE()
            for mon, cc in P.terms():
                t = GE.const(cc)
                for s, e in zip(syms, mon):
                    for _ in range(e):
                        t = t * evsubs[s]
                coefGE = coefGE + t
        else:
            coefGE = GE.const(v)
        # odd part, left to right
        prod = coefGE
        for x in k:
            if (x[0], x[1]) in key_map:
                if x[2]:
                    raise ValueError(f"derivative jet of substituted odd field: {x}")
                prod = prod * key_map[(x[0], x[1])]
            else:
                prod = prod * GE({(x,): sp.Integer(1)})
        out = out + prod
    return out


def s_apply(F: GE, s_of: dict) -> GE:
    """Left odd derivation s:  s F = sum_u (s u) d^L F / d u,  s(u_J) = D_J(s u).
    s_of[(field, comp)] = GE image of the undifferentiated generator."""
    out = GE()
    for key in F.keys_of():
        fc = (key[0], key[1])
        if fc not in s_of:
            continue
        su = DJ(s_of[fc], key[2])
        dF = pdiff(F, key, "L")
        if not dF.is_zero() and not su.is_zero():
            out = out + su * dF
```

```
    return out


def ghost_number(F: GE, gh: dict) -> set:
    vals = set()
    for k, v in F.t.items():
        g0 = sum(gh[x[0]] for x in k)
        for term in sp.Add.make_args(sp.expand(v)):
            g = g0
            for s, e in term.as_powers_dict().items():
                kk = _KEY.get(s)
                if kk is not None:
                    g += gh[kk[0]] * int(e)
            vals.add(g)
    return vals
```

97 Source: bv__model.py

[listing 64/133] code/classical/bv_model.py

```
#!/usr/bin/env python3
"""p10/p11 model builders on the graded jet engine: geometry -> s, source action, Psi.

Every object is built from declared geometry (conventions.py); signs that are results are
derived in brst_from_geometry.py / source_action.py / gauge_fixing_map.py, not here.
"""
from __future__ import annotations
import sympy as sp
from graded_jets import GE, D, H_COMPS, s_apply, kappa_coeff, euler, comps

ETA = (1, -1, -1, -1)
KAPPA = sp.Symbol("kappa")
A = sp.Rational(1, 2)
I = sp.I


def hc(m, n, J=()):
    return GE.gen("h", (min(m, n), max(m, n)), J)


def cv(mu, J=()):
    return GE.gen("c", mu, J)


def Dn(F, J):
    for a in J:
        F = D(F, a)
    return F


# ---------------- geometry ----------------
def metric(param="affine", kappa=KAPPA):
    """g_{mn}(h). 'affine': eta + kappa h ; 'exp2': eta + kappa h + kappa^2/2 h.eta.h (control)."""
    g = [[None] * 4 for _ in range(4)]
    for m in range(4):
        for n in range(4):
            e = GE.const(ETA[m] if m == n else 0) + hc(m, n).scale(kappa)
            if param == "exp2":
                acc = GE()
                for r in range(4):
                    acc = acc + (hc(m, r) * hc(r, n)).scale(ETA[r])
                e = e + acc.scale(kappa ** 2 / 2)
            g[m][n] = e
    return g


def lie_sym(xi, T):
    """(L_xi T)_{mn} = xi^r d_r T_mn + T_rn d_m xi^r + T_mr d_n xi^r  (xi left of T)."""
    out = [[None] * 4 for _ in range(4)]
    for m in range(4):
        for n in range(m, 4):
            acc = GE()
            for r in range(4):
                acc = acc + xi[r] * D(T[m][n], r)
                acc = acc + D(xi[r], m) * T[r][n]
                acc = acc + D(xi[r], n) * T[m][r]
            out[m][n] = out[n][m] = acc
    return out
```

```
def xi_field(kappa=KAPPA, a=A):
    return [cv(mu).scale(kappa * a) for mu in range(4)]


def ghost_ansatz(lam, kappa=KAPPA, a=A):
    """s xi^mu = lam xi^r d_r xi^mu  =>  s c^mu = (1/(kappa a)) s xi^mu."""
    xi = xi_field(kappa, a)
    out = {}
    for mu in range(4):
        acc = GE()
        for r in range(4):
            acc = acc + xi[r] * D(xi[mu], r)
        out[mu] = acc.scale(lam / (kappa * a))
    return out


def s_h_affine(kappa=KAPPA, a=A):
    """s h_{mn} = (1/kappa) L_xi g_{mn},  g = eta + kappa h (exact, all orders)."""
    g = metric("affine", kappa)
    Lg = lie_sym(xi_field(kappa, a), g)
    return {(m, n): Lg[m][n].scale(1 / kappa) for (m, n) in H_COMPS}


def s_phi(kappa=KAPPA, a=A):
    xi = xi_field(kappa, a)
    acc = GE()
    for r in range(4):
        acc = acc + xi[r] * D(GE.gen("phi", ()), r)
    return acc


def s_map(lam=1, kappa=KAPPA, a=A, nonminimal=True):
    """Full s on generators (affine parametrisation)."""
    s = {}
    for k, v in s_h_affine(kappa, a).items():
        s[("h", k)] = v
    for mu, v in ghost_ansatz(lam, kappa, a).items():
        s[("c", mu)] = v
    s[("phi", ())] = s_phi(kappa, a)
    if nonminimal:
        for mu in range(4):
            s[("cb", mu)] = GE.gen("b", mu).scale(I)
            s[("b", mu)] = GE()
    return s


def truncate_kappa(F, nmax, kappa=KAPPA):
    out = GE()
    for n in range(nmax + 1):
        out = out + kappa_coeff(F, kappa, n).scale(kappa ** n)
    return out


# --------------- source / antifield action ---------------
ANTI = {"h": "K", "c": "cs", "cb": "cbs", "b": "bs", "phi": "phis"}


def S_af(s, sigma, fields=("h", "c", "phi", "cb")):
    """S_af = sum_A sigma_A Phi*_A (s Phi^A)  (antifield written to the LEFT)."""
    out = GE()
    for f in fields:
        for c in comps(f):
            img = s.get((f, c))
            if img is None or img.is_zero():
                continue
            out = out + (GE.gen(ANTI[f], c) * img).scale(sigma[f])
    return out


# --------------- gauge fermion ---------------
def h_up(m, n):
    return hc(m, n).scale(ETA[m] * ETA[n])


def trace_h():
    acc = GE()
    for r in range(4):
        acc = acc + hc(r, r).scale(ETA[r])
    return acc
```

```
def deDonder_up(mu):
    """D^mu[h] = d_nu h^{mu nu} - 1/2 d^mu tr h."""
    acc = GE()
    for nu in range(4):
        acc = acc + D(h_up(mu, nu), nu)
    acc = acc - D(trace_h(), mu).scale(sp.Rational(1, 2) * ETA[mu])
    return acc


def deDonder_of(Tlow):
    """D^mu applied to an arbitrary symmetric lowered tensor (list of lists of GE)."""
    out = []
    for mu in range(4):
        acc = GE()
        for nu in range(4):
            acc = acc + D(Tlow[mu][nu].scale(ETA[mu] * ETA[nu]), nu)
        tr = GE()
        for r in range(4):
            tr = tr + Tlow[r][r].scale(ETA[r])
        acc = acc - D(tr, mu).scale(sp.Rational(1, 2) * ETA[mu])
        out.append(acc)
    return out


def Psi_density(coef_D=2, coef_b=1):
    """Psi = -i cbar_mu (coef_D D^mu[h] - coef_b b^mu)   (declared: coef_D=2, coef_b=1)."""
    acc = GE()
    for mu in range(4):
        bup = GE.gen("b", mu).scale(ETA[mu])
        acc = acc + GE.gen("cb", mu) * (deDonder_up(mu).scale(coef_D) - bup.scale(coef_b))
    return acc.scale(-I)


def delta_Psi(Psi, side="L"):
    """Variational derivatives of the gauge fermion w.r.t. every field component."""
    return {(f, c): euler(Psi, f, c, side) for f in ("h", "c", "cb", "b", "phi") for c in comps(f)}
```

98 Source: brst__from__geometry.py

[listing 65/133] code/classical/brst_from_geometry.py

```
#!/usr/bin/env python3
"""p11 (b) [unchanged since p10]: derive s = s0 + kappa s1 + kappa^2 s2 from the finite geometric transformati
  +> ons.

Finite data: g -> phi_t^* g (flow of xi), phi -> phi o flow;  infinitesimal L_xi.
Declared parametrisation g = eta + kappa h and ghost scaling xi = kappa a c (conventions.py).
Ghost law: ansatz s xi = lam xi.d xi; lam is SOLVED from s^2 g = 0 (not imported).
s2 per sector is READ OFF the exact kappa-expansion; it is a result, not a policy.
Control: the exponential parametrisation g = eta + kappa h + kappa^2/2 h eta h gives s2 != 0,
and dropping that s2 breaks s^2 = 0 at O(kappa^2) (mutation 'omitted s2').
"""
from __future__ import annotations
import json, sys, time
from pathlib import Path
import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parent))
from graded_jets import GE, D, s_apply, kappa_coeff, H_COMPS, ghost_number  # noqa: E402
from bv_model import (KAPPA, A, I, metric, lie_sym, xi_field, s_map, hc, cv,  # noqa: E402
                      ghost_ansatz)
import conventions as C  # noqa: E402
import certio  # noqa: E402

ROOT = Path(__file__).resolve().parents[2]
GH = {"h": 0, "c": 1, "cb": -1, "b": 0, "phi": 0, "K": -1, "cs": -2, "cbs": 0, "bs": -1, "phis": -1}


def kdeg(F):
    return max((sp.degree(sp.expand(v), KAPPA) for v in F.t.values()), default=-1)


def solve_ghost_law():
    lam = sp.Symbol("lam")
    s = s_map(lam=lam)
    eqs = set()
```

```
    for k in H_COMPS:
        for v in s_apply(s[("h", k)], s).t.values():
            eqs.add(sp.factor(v))
    for v in s_apply(s[("phi", ())], s).t.values():
        eqs.add(sp.factor(v))
    sols = None
    for e in eqs:
        r = set(sp.solve(sp.Eq(e, 0), lam)) if e.has(lam) else None
        if r is None:
            continue
        sols = r if sols is None else sols & r
    s2c = sum(len(s_apply(s[("c", mu)], s).t) for mu in range(4))
    return {"ansatz": "s xi^mu = lam xi^r d_r xi^mu",
            "condition": "s^2 h = 0 and s^2 phi = 0 for all jets",
            "lam_solutions": sorted(str(x) for x in (sols or [])),
            "n_independent_conditions": len(eqs),
            "s2_c_terms_any_lam": s2c,
            "s2_c_identically_zero_any_lam": s2c == 0,
            "derived_lam": 1 if sols == {1} else None}


def per_sector_expansion(s):
    out = {}
    for (f, c), img in s.items():
        if img.is_zero():
            orders = {k: 0 for k in range(4)}
        else:
            orders = {k: len(kappa_coeff(img, KAPPA, k).t) for k in range(4)}
        out.setdefault(f, {"terms_by_kappa_order": {}, "max_kappa_degree": -1})
        for k, n in orders.items():
            out[f]["terms_by_kappa_order"][k] = out[f]["terms_by_kappa_order"].get(k, 0) + n
        out[f]["max_kappa_degree"] = max(out[f]["max_kappa_degree"], kdeg(img))
    for f, d in out.items():
        d["s0_nonzero"] = d["terms_by_kappa_order"][0] > 0
        d["s1_nonzero"] = d["terms_by_kappa_order"][1] > 0
        d["s2_nonzero"] = d["terms_by_kappa_order"][2] > 0
        d["s3_nonzero"] = d["terms_by_kappa_order"][3] > 0
        d["s2"] = "nonzero" if d["s2_nonzero"] else "derived_zero"
    return out


def compare_to_xvin(s):
    """Regression: s0 h = a(d_m c_n + d_n c_m), s1 h = a L_c h, s c = kappa a c.dc (XVI-N)."""
    ok = True
    xi = [cv(r) for r in range(4)]
    hlow = [[hc(m, n) for n in range(4)] for m in range(4)]
    Lch = lie_sym(xi, hlow)
    for (m, n) in H_COMPS:
        img = s[("h", (m, n))]
        s0 = kappa_coeff(img, KAPPA, 0)
        R0 = (D(cv(n), m).scale(C.ETA[n]) + D(cv(m), n).scale(C.ETA[m])).scale(A)
        s1 = kappa_coeff(img, KAPPA, 1)
        ok &= (s0 - R0).is_zero() and (s1 - Lch[m][n].scale(A)).is_zero()
    for mu in range(4):
        ref = GE()
        for r in range(4):
            ref = ref + cv(r) * D(cv(mu), r)
        ok &= (s[("c", mu)] - ref.scale(KAPPA * A)).is_zero()
    return ok


def nilpotency(s, nmax=2):
    res = {}
    for (f, c), img in s.items():
        s2 = s_apply(img, s)
        res[f"{f}{c}"] = {k: len(kappa_coeff(s2, KAPPA, k).t) for k in range(nmax + 3)}
    allzero = all(all(v == 0 for v in d.values()) for d in res.values())
    return allzero, res


def exp2_control():
    """Nonlinear parametrisation control: derive s_k h (k<=2) from s g = L_xi g."""
    g = metric("exp2")
    Lg = lie_sym(xi_field(), g)
    sk = {}
    for k in range(3):
        cur = {}
        for (m, n) in H_COMPS:
            rhs = kappa_coeff(Lg[m][n], KAPPA, k + 1)
            if k >= 1:
                corr = GE()
```

```
                for r in range(4):
                    corr = corr + (sk[k - 1][(min(m, r), max(m, r))] * hc(r, n)
                                   + hc(m, r) * sk[k - 1][(min(r, n), max(r, n))]).scale(C.ETA[r])
                rhs = rhs - corr.scale(sp.Rational(1, 2))
            cur[(m, n)] = rhs
        sk[k] = cur
    s2_terms = sum(len(v.t) for v in sk[2].values())

    def smap(include_s2):
        s = {}
        for (m, n) in H_COMPS:
            img = sk[0][(m, n)] + sk[1][(m, n)].scale(KAPPA)
            if include_s2:
                img = img + sk[2][(m, n)].scale(KAPPA ** 2)
            s[("h", (m, n))] = img
        for mu, v in ghost_ansatz(1).items():
            s[("c", mu)] = v
        return s

    out = {}
    for inc in (True, False):
        s = smap(inc)
        bad = {}
        for (m, n) in [(0, 0), (0, 1), (1, 2)]:
            s2h = s_apply(s[("h", (m, n))], s)
            bad[f"h{m}{n}"] = {k: len(kappa_coeff(s2h, KAPPA, k).t) for k in range(3)}
        out["with_s2" if inc else "s2_omitted"] = {
            "s2h_terms_by_order_through_kappa2": bad,
            "nilpotent_through_kappa2": all(all(v == 0 for v in d.values()) for d in bad.values()),
        }
    return {"parametrisation": "g = eta + kappa h + kappa^2/2 h.eta.h (control only, NOT the declared one)",
            "s2_h_terms": s2_terms, "s2_h_nonzero": s2_terms > 0, **out}


def ghost_numbers(s):
    bad = []
    for (f, c), img in s.items():
        if img.is_zero():
            continue
        g = ghost_number(img, GH)
        if g != {GH[f] + 1}:
            bad.append((f, c, sorted(g)))
    return bad


def main():
    t0 = time.time()
    law = solve_ghost_law()
    s = s_map(lam=1)
    sectors = per_sector_expansion(s)
    nil_ok, nil = nilpotency(s)
    xvin = compare_to_xvin(s)
    ctrl = exp2_control()
    ghbad = ghost_numbers(s)
    s2_per_sector = {f: d["s2"] for f, d in sectors.items()}
    passed = bool(law["derived_lam"] == 1 and nil_ok and xvin and not ghbad
                  and all(v == "derived_zero" for v in s2_per_sector.values())
                  and ctrl["s2_h_nonzero"] and ctrl["with_s2"]["nilpotent_through_kappa2"]
                  and not ctrl["s2_omitted"]["nilpotent_through_kappa2"])
    cert = {
        "revision": C.REVISION, "gate": "P13_brst_from_geometry",
        "declared_inputs": {"parametrisation": C.METRIC_PARAMETRISATION, "ghost_scaling": C.GHOST_SCALING,
                            "nonminimal_pair": "s cbar_mu = i b_mu, s b_mu = 0 (contractible pair; XVI-N norma
  +> lisation i)",
                            "scalar": "phi -> phi o flow  =>  s phi = xi.d phi"},
        "ghost_law_derivation": law,
        "s_images": {
            "h": "s h_mn = (1/kappa) L_xi(eta+kappa h)_mn = a(d_m c_n + d_n c_m) + kappa a (L_c h)_mn   [exact
  +> ]",
            "c": "s c^mu = (1/(kappa a)) xi.d xi^mu = kappa a c^r d_r c^mu   [exact]",
            "phi": "s phi = kappa a c^r d_r phi   [exact]",
            "cbar": "s cbar_mu = i b_mu", "b": "s b_mu = 0"},
        "per_sector_kappa_expansion": sectors,
        "s2_per_sector": s2_per_sector,
        "s2_reason": ("derived: g is affine in h and L_xi is linear in g, so (1/kappa)L_{kappa a c}g has "
                      "kappa-degree exactly 1; s xi quadratic in xi gives s c of degree exactly 1; "
                      "s phi linear in xi; (cbar,b) pair kappa-independent. 'No invented s2' is NOT used."),
        "nilpotency_exact_all_orders": nil_ok,
        "nilpotency_terms_by_order": nil,
        "matches_XVI_N_s0_s1_sc": xvin,
        "ghost_number_violations": ghbad,
```

```
        "control_nonlinear_parametrisation": ctrl,
        "execution_pass": passed,
        "claim_status": "s2_derived_zero_all_sectors" if passed else "not_derived",
        "scope": "declared affine parametrisation g=eta+kappa h, ghost scaling xi=kappa a c, a=1/2; formal pow
  +> er series in kappa",
        "pass": passed,
        "seconds": round(time.time() - t0, 2),
    }
    (ROOT / "certificates" / "P13_brst_derivation.json").write_text(certio.dumps(cert))
    print(json.dumps({"pass": passed, "lam": law["lam_solutions"], "s2": s2_per_sector,
                      "nil": nil_ok, "xvin": xvin, "exp2_s2_nonzero": ctrl["s2_h_nonzero"],
                      "exp2_omit_fails": not ctrl["s2_omitted"]["nilpotent_through_kappa2"],
                      "sec": cert["seconds"]}, indent=1))
    return 0 if passed else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

99 Source: source_action.py

[listing 66/133] code/classical/source_action.py

```
#!/usr/bin/env python3
"""p11 (c) [unchanged since p10]: source/antifield action built from the derived transformations.

S_ext = S_field + S_af,   S_af = sum_A sigma_A int Phi*_A (s Phi^A)   (antifield on the left)
sigma_A is SOLVED from the generator rule  (S, Phi^A) = s Phi^A  under the XVI-N bracket,
using the graded engine (no sign table imported).  Includes the nonminimal (cbar,b) pair
required by Psi.  S_field = sum_k kappa^(k-2) int ell_k  +  (1/2) int B^{mn}(h) d_m phi d_n phi,
with B = sqrt(-g) g^{-1} = eta + kappa a1 + kappa^2 a2 re-derived here and identified with the
exported XI-A a1, a2 (O2_scalar_action_authoritative).
"""
from __future__ import annotations
import json, sys, time
from pathlib import Path
import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parent))
from graded_jets import GE, D, bracket, euler, comps, kappa_coeff, ghost_number  # noqa: E402
from bv_model import KAPPA, A, I, s_map, S_af, hc, ANTI  # noqa: E402
import conventions as C  # noqa: E402
import certio  # noqa: E402

ROOT = Path(__file__).resolve().parents[2]
GH = {"h": 0, "c": 1, "cb": -1, "b": 0, "phi": 0, "K": -1, "cs": -2, "cbs": 0, "bs": -1, "phis": -1}


def derive_sigma(s):
    """For each field: find sigma in {+1,-1} with (sigma Phi* sPhi, int mu Phi) = mu sPhi."""
    out = {}
    for f in ("h", "c", "cb", "phi"):
        found = []
        for sig in (1, -1):
            ok = True
            for c in comps(f):
                img = s[(f, c)]
                Str = (GE.gen(ANTI[f], c) * img).scale(sig)
                test = GE.gen("mu", c if not isinstance(c, tuple) or c else 0) * GE.gen(f, c)
                br = bracket(Str, test)
                ref = GE.gen("mu", c if not isinstance(c, tuple) or c else 0) * img
                if not (br - ref).is_zero():
                    ok = False
                    break
            if ok:
                found.append(sig)
        out[f] = found
    return out


def sigma_formula_check(sig):
    """Closed form: sigma_A = (-1)^{|Phi^A|}  (derived above; compared, not assumed)."""
    par = {"h": 0, "c": 1, "cb": 1, "phi": 0}
    return {f: (sig[f] == [(-1) ** par[f]]) for f in par}


def scalar_B_series():
    """B^{mn} = sqrt(-g) g^{mn} through kappa^2 at a point (algebraic in h); compare to XI-A a1,a2."""
```

```
    H = sp.Matrix(4, 4, lambda i, j: sp.Symbol(f"h{min(i,j)}{max(i,j)}"))
    eta = sp.diag(*C.ETA)
    k = KAPPA
    g = eta + k * H
    ginv = eta - k * eta * H * eta + k**2 * eta * H * eta * H * eta
    det = sp.expand(g.det())
    sq = sp.series(sp.sqrt(-det), k, 0, 3).removeO()
    B = (sq * ginv).applyfunc(lambda e: sp.expand(sp.series(sp.expand(e), k, 0, 3).removeO()))
    hup = eta * H * eta
    tau = sum(eta[i, i] * H[i, i] for i in range(4))
    gg = sum(H[i, j] * hup[i, j] for i in range(4) for j in range(4))
    a1 = sp.Rational(1, 2) * tau * eta - hup
    b2 = sp.Rational(1, 8) * tau**2 - sp.Rational(1, 4) * gg
    hmix = eta * H  # h^mu_rho
    a2 = hmix * hup - sp.Rational(1, 2) * tau * hup + b2 * eta
    ok1 = all(sp.expand(B[i, j].coeff(k, 1) - a1[i, j]) == 0 for i in range(4) for j in range(4))
    ok2 = all(sp.expand(B[i, j].coeff(k, 2) - a2[i, j]) == 0 for i in range(4) for j in range(4))
    ok0 = all(sp.expand(B[i, j].coeff(k, 0) - eta[i, j]) == 0 for i in range(4) for j in range(4))
    return B, {"B0_eta": ok0, "a1_matches_XI_A": ok1, "a2_matches_XI_A": ok2}


def scalar_density(nmax=2, mutate_a2=None):
    """L_phi = 1/2 B^{mn}(h) d_m phi d_n phi as engine element through kappa^nmax."""
    B, _ = scalar_B_series()
    dphi = [D(GE.gen("phi", ()), m) for m in range(4)]
    out = GE()
    for m in range(4):
        for n in range(4):
            e = sp.expand(sum(B[m, n].coeff(KAPPA, j) * KAPPA**j for j in range(nmax + 1)))
            if mutate_a2 is not None:
                e = e + (mutate_a2 * KAPPA**2 * sp.Symbol("h00") ** 2 if (m, n) == (0, 0) else 0)
            if e == 0:
                continue
            # build GE coefficient: polynomial in undifferentiated h symbols
            coef = GE()
            P = sp.Poly(e, *[sp.Symbol(f"h{i}{j}") for i in range(4) for j in range(i, 4)])
            for mon, cc in P.terms():
                t = GE.const(cc)
                idx = 0
                for i in range(4):
                    for j in range(i, 4):
                        for _ in range(mon[idx]):
                            t = t * hc(i, j)
                        idx += 1
                coef = coef + t
            out = out + coef * dphi[m] * dphi[n]
    return out.scale(sp.Rational(1, 2))


def build_X_inv(sigma=None, drop=(), s=None):
    """Antifield + nonminimal part of the invariant extended action (antifield-linear)."""
    s = s or s_map(lam=1)
    sigma = sigma or {"h": 1, "c": -1, "cb": -1, "phi": 1}
    fields = tuple(f for f in ("h", "c", "phi", "cb") if f not in drop)
    return S_af(s, sigma, fields)


def antifield_transformations(X):
    """s Phi*_A = (S, Phi*_A) restricted to the X part (S_field part adds E_Phi(S_field))."""
    out = {}
    for f, fs in [("h", "K"), ("c", "cs"), ("cb", "cbs"), ("phi", "phis")]:
        c = comps(f)[0]
        test = GE.gen("mu", 0) * GE.gen(fs, c)
        br = bracket(X, test)
        out[f"{fs}{c}"] = {"terms": len(br.t), "kappa_orders": sorted({sp.degree(v, KAPPA) for v in br.t.value
  +> s()})}
    return out


def main():
    t0 = time.time()
    s = s_map(lam=1)
    sig = derive_sigma(s)
    sig_closed = sigma_formula_check(sig)
    unique = all(len(v) == 1 for v in sig.values())
    sigma = {f: v[0] for f, v in sig.items()} if unique else None
    xvin = {"h": 1, "phi": 1, "c": -1, "cb": -1}
    matches_xvin = sigma == xvin
    X = build_X_inv(sigma)
    gh = ghost_number(X, GH)
    by_kappa = {n: len(kappa_coeff(X, KAPPA, n).t) for n in range(4)}
```

```
    _, Bchk = scalar_B_series()
    aft = antifield_transformations(X)
    passed = bool(unique and matches_xvin and all(sig_closed.values()) and gh == {0}
                  and all(Bchk.values()) and by_kappa[2] == 0 and by_kappa[3] == 0)
    cert = {
        "revision": C.REVISION, "gate": "P13_source_action",
        "bracket": C.ANTIBRACKET, "generator_rule": C.GENERATOR_RULE,
        "sigma_solutions": sig, "sigma_unique": unique, "sigma": sigma,
        "sigma_closed_form": "sigma_A = (-1)^{|Phi^A|}  (derived: right derivative of Phi* X past X gives (-1)
  +> ^{|Phi|+1}, generator rule adds -1)",
        "sigma_closed_form_check": sig_closed,
        "matches_XVI_N_S_af": matches_xvin,
        "S_af": ("int K^{mn}(s h)_mn + int phi*(s phi) - int c*_mu (s c^mu) - int cbar*^mu (s cbar_mu)"
                 " = int K^{mn}[a(d_m c_n+d_n c_m) + kappa a (L_c h)_mn] + kappa a int phi* c.d phi"
                 " - kappa a int c*_mu c^r d_r c^mu - i int cbar*^mu b_mu"),
        "K_vs_symmetric_hstar": "K^{mn} = w_mn h*^{mn}, w=1 (m=n), 2 (m<n); pairing over all 16 components",
        "nonminimal_included": True, "b_star_term": "absent (s b = 0)",
        "X_terms_by_kappa_order": by_kappa,
        "X_kappa2_terms": by_kappa[2],
        "X_ghost_number": sorted(gh),
        "antifield_linear": True,
        "antifield_transformations_X_part": aft,
        "S_field": {"grav": C.S_EH, "scalar": C.S_SCALAR, "scalar_B_identification": Bchk,
                    "grav_identification": "exports/ell{2,3,4}_generating_polynomial.srepr (sha in P13_quartic
  +> _verified)"},
        "baseline": C.BASELINE,
        "execution_pass": passed, "claim_status": "source_action_constructed" if passed else "incomplete",
        "scope": "S_inv extended action through O(kappa^2), antifield-linear (closed algebra), nonminimal (cba
  +> r,b) pair",
        "pass": passed, "seconds": round(time.time() - t0, 2),
    }
    (ROOT / "certificates" / "P13_source_action.json").write_text(certio.dumps(cert))
    print(json.dumps({"pass": passed, "sigma": sig, "gh": sorted(gh), "by_kappa": by_kappa, "B": Bchk,
                      "sec": cert["seconds"]}, indent=1, default=str))
    return 0 if passed else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

100 Source: gauge_fixing_map.py

[listing 67/133] code/classical/gauge_fixing_map.py

```
#!/usr/bin/env python3
"""p11 (a): gauge-fixing map from Psi = -i int cbar_mu (2 D^mu[h] - b^mu), in transformed variables.

Derived here (graded engine + exact plane waves), nothing imported:
  * odd left/right derivatives of Psi agree (Psi odd);
  * S_af(Phi* = dPsi/dPhi) = - s Psi  (mod d) under the XVI-N bracket and derived sigma_A;
  * shift orientation eps in  Phi* = Phi*' + eps dPsi/dPhi  is fixed by requiring the transformed
    Hessians to equal the stated propagator kernels (K = -Box P, ghost i cbar Box c, scalar -Box);
  * canonicity of the map (bracket preservation) on ALL 23 antifield components (276 pairs + 23 mixed);
  * Hessians for a GENERIC symbolic momentum k (p11): Schur complement of the b-block equals k^2 P identically
    and, by the block-inverse formula, (M^{-1})_hh = (k^2 P)^{-1} (deterministic; no momentum sample);
  * transformed action S_Psi(Phi,Phi*') = S(Phi, Phi*' + eps dPsi/dPhi) keeps external antifields;
  * inverse map; Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' + eps dPsi/dPhi) is a DEFINED classical
    pullback/interface, NOT an independently established quantum gauge-change theorem.
"""
from __future__ import annotations
import json, os, random, sys, time
from fractions import Fraction as Fr
from pathlib import Path
import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parent))
from graded_jets import (GE, D, bracket, euler, comps, kappa_coeff, subst, s_apply,  # noqa: E402
                         is_divergence, divergence_residual, ghost_number, _KEY, H_COMPS)
from bv_model import KAPPA, A, I, s_map, Psi_density, delta_Psi, hc, cv, deDonder_of, lie_sym  # noqa: E402
from source_action import build_X_inv, scalar_density  # noqa: E402
import planewave_exact as PW  # noqa: E402
import conventions as C  # noqa: E402
import certio  # noqa: E402

ROOT = Path(__file__).resolve().parents[2]
GH = {"h": 0, "c": 1, "cb": -1, "b": 0, "phi": 0, "K": -1, "cs": -2, "cbs": 0, "bs": -1, "phis": -1,
      "lam": 0, "mu": 0}
ANTI_OF = {"h": "K", "c": "cs", "cb": "cbs", "b": "bs", "phi": "phis"}
```

```
def shift_map(Psi, eps, mutate=None):
    """Old antifields in terms of new: Phi*_A = Phi*'_A + eps dPsi/dPhi^A."""
    dP = delta_Psi(Psi, "L")
    km = {}
    for (f, c), v in dP.items():
        if mutate == "halve_offdiag_h" and f == "h" and c[0] != c[1]:
            v = v.scale(sp.Rational(1, 2))
        km[(ANTI_OF[f], c)] = GE.gen(ANTI_OF[f], c) + v.scale(eps)
    return km


def transformed_X(X_inv, Psi, eps, mutate=None):
    return subst(X_inv, shift_map(Psi, eps, mutate))


def field_only(F):
    """Set every antifield to zero."""
    zero = {(af, c): GE() for af in ("K", "cs", "cbs", "bs", "phis") for c in comps(af)}
    return subst(F, zero)


# ---------- plane-wave bilinear forms of even quadratic GE densities ----------
def pw_bilinear(F, w1, w2):
    """F even, quadratic in even fields. w = {field: (pol dict comp->Fr, k list)}.
    Returns coefficient of t1 t2 with d -> i k (exact, sympy)."""
    t1, t2 = sp.symbols("t1 t2")
    subs = {}
    for coef in F.t.values():
        for s_ in coef.free_symbols:
            kk = _KEY.get(s_)
            if kk is None:
                continue
            f, c, J = kk
            val = 0
            for t, w in ((t1, w1), (t2, w2)):
                if f in w:
                    pol, k = w[f]
                    v = sp.Rational(pol.get(c, 0))
                    for j in J:
                        v *= I * sp.sympify(k[j])
                    val += t * v
            subs[s_] = val
    tot = sum(sp.expand(v.xreplace(subs)) for key, v in F.t.items() if key == ())
    return sp.expand(tot).coeff(t1, 1).coeff(t2, 1)


def basis_waves(k):
    out = []
    for (m, n) in H_COMPS:
        out.append(("h", (m, n), {"h": ({(m, n): Fr(1)}, k)}))
    for mu in range(4):
        out.append(("b", mu, {"b": ({mu: Fr(1)}, k)}))
    return out


_ELL2 = {}


def ell2_bilinear_symbolic(e1, e2, k):
    """ell2 bilinear on symmetric basis tensors, legs (e1, k) and (e2, -k), symbolic k; d -> i k."""
    if "e" not in _ELL2:
        _ELL2["e"] = PW.load_ell(2)[0]
    t1, t2 = sp.symbols("t1 t2")
    sub = {}
    for m, n in H_COMPS:
        a = (1 if (m, n) == e1 else 0)
        b = (1 if (m, n) == e2 else 0)
        sub[sp.Symbol(f"H{m}{n}")] = t1 * a + t2 * b
        for r in range(4):
            sub[sp.Symbol(f"dH{m}{n}_{r}")] = t1 * a * I * k[r] - t2 * b * I * k[r]
    return sp.expand(sp.expand(_ELL2["e"].xreplace(sub)).coeff(t1, 1).coeff(t2, 1))


def ell2_bilinear(cp2, e1, e2, k):
    """Exact ell2 bilinear on symmetric basis tensors (uniform 2 derivatives: i*(-i)=+1 for k,-k)."""
    def E(mn):
        e = [[Fr(0)] * 4 for _ in range(4)]
        e[mn[0]][mn[1]] = e[mn[1]][mn[0]] = Fr(1)
        return e
```

```
    km = [-x for x in k]
    val = PW.polarize(lambda j: PW.eval_poly(cp2, j), [PW.tensor_jet(E(e1), k), PW.tensor_jet(E(e2), km)])
    return sp.Rational(-val.numerator, val.denominator)   # i^2 = -1 stripped factor


def P_bilinear(e1, e2):
    def E(mn):
        e = [[0] * 4 for _ in range(4)]
        e[mn[0]][mn[1]] = e[mn[1]][mn[0]] = 1
        return e
    a, b = E(e1), E(e2)
    dot = sum(C.ETA[m] * C.ETA[n] * a[m][n] * b[m][n] for m in range(4) for n in range(4))
    t1 = sum(C.ETA[m] * a[m][m] for m in range(4))
    t2 = sum(C.ETA[m] * b[m][m] for m in range(4))
    return sp.Rational(dot) - sp.Rational(1, 2) * t1 * t2


def hessian_check(gf_density, cp2=None, k=None):
    """(h,b) block Hessian of ell2 + gf for GENERIC symbolic k = (k0,k1,k2,k3): exact identities in Q(k).
    M = [[A, B], [C, D]] (h|b).  If det D != 0 and S = A - B D^{-1} C is invertible, the block-inverse formula
  +>  gives
    (M^{-1})_hh = S^{-1}.  We prove S = k^2 P identically, so the hh propagator is (k^2 P)^{-1} = P^{-1}/k^2."
  +> ""
    ks = list(sp.symbols("k0:4")) if k is None else k
    km = [-x for x in ks]
    basis = basis_waves(ks)
    n = len(basis)
    M = sp.zeros(n, n)
    for i, (fi, ci, wi) in enumerate(basis):
        for j, (fj, cj, wj) in enumerate(basis):
            w2 = {f: (pol, km) for f, (pol, _) in wj.items()}
            v = pw_bilinear(gf_density, wi, w2)
            if fi == "h" and fj == "h":
                v += ell2_bilinear_symbolic(ci, cj, ks)
            M[i, j] = sp.expand(v)
    k2 = sp.expand(sum(C.ETA[r] * ks[r] ** 2 for r in range(4)))
    Mhh, Mhb, Mbh, Mbb = M[:10, :10], M[:10, 10:], M[10:, :10], M[10:, 10:]
    dD = sp.factor(Mbb.det())
    res = {"momentum": "generic symbolic k = (k0,k1,k2,k3); identities in Q(k) (no sample)",
           "det_Mbb": str(dD), "Mbb_det_nonzero": dD != 0}
    if dD != 0:
        K = (Mhh - Mhb * Mbb.inv() * Mbh).applyfunc(sp.cancel)
        Pm = sp.Matrix(10, 10, lambda i, j: k2 * P_bilinear(H_COMPS[i], H_COMPS[j]))
        res["schur_equals_k2P"] = (K - Pm).applyfunc(sp.cancel) == sp.zeros(10, 10)
        detP = sp.factor(Pm.det())
        res["det_k2P"] = str(detP)
        res["propagator_hh_equals_inv_k2P"] = bool(res["schur_equals_k2P"] and detP != 0)
        res["propagator_argument"] = "block-inverse formula: (M^{-1})_hh = (A - B D^{-1} C)^{-1} = (k^2 P)^{-1
  +> }"
    else:
        res["schur_equals_k2P"] = False
        res["propagator_hh_equals_inv_k2P"] = False
    return res


def box(F):
    acc = GE()
    for r in range(4):
        acc = acc + D(D(F, r), r).scale(C.ETA[r])
    return acc


def ghost_kinetic_check(Xf0, sign_expected=1):
    """E^L_{cbar_mu}(field-only kappa^0) == i Box c^mu (stated XVI-N V2_ghost)."""
    ok = True
    for mu in range(4):
        lhs = euler(Xf0, "cb", mu, "L")
        rhs = box(cv(mu)).scale(I * sign_expected)
        ok &= (lhs - rhs).is_zero()
    return ok


def ghost_graviton_vertex_check(Xf1):
    """E^L_{cbar_mu}(field-only kappa^1) == 2 i a D^mu[L_c h] (XVI-N V3_cbar_c_h factor kappa*2ia)."""
    xi = [cv(r) for r in range(4)]
    Lch = lie_sym(xi, [[hc(m, n) for n in range(4)] for m in range(4)])
    DL = deDonder_of(Lch)
    ok = True
    for mu in range(4):
        lhs = euler(Xf1, "cb", mu, "L")
        rhs = DL[mu].scale(2 * I * A)
```

```
        ok &= (lhs - rhs).is_zero()
    return ok


def canonicity(Psi, eps, mutate=None):
    """(Phi*'_A, Phi*'_B) ~ 0 and (Phi*'_A, Phi^B) = (Phi*_A, Phi^B) with smeared even tests."""
    new = {}
    dP = delta_Psi(Psi, "L")
    for (f, c), v in dP.items():
        if mutate == "halve_offdiag_h" and f == "h" and c[0] != c[1]:
            v = v.scale(sp.Rational(1, 2))
        new[(ANTI_OF[f], c)] = GE.gen(ANTI_OF[f], c) - v.scale(eps)   # Phi*' = Phi* - eps dPsi
    keys = ([("K", mn) for mn in H_COMPS] + [("cbs", m) for m in range(4)] + [("bs", m) for m in range(4)]
            + [("cs", m) for m in range(4)] + [("phis", ())])
    fails = []
    for i, ka in enumerate(keys):
        for kb in keys[i:]:
            F = GE.gen("lam", 0) * new[ka]
            G = GE.gen("mu", 0) * new[kb]
            if not is_divergence(bracket(F, G)):
                fails.append(f"({ka},{kb})")
    mixed = []
    for ka in keys:
        f = {v: k for k, v in ANTI_OF.items()}[ka[0]]
        F = GE.gen("lam", 0) * new[ka]
        G = GE.gen("mu", 0) * GE.gen(f, ka[1])
        Fo = GE.gen("lam", 0) * GE.gen(ka[0], ka[1])
        if not (bracket(F, G) - bracket(Fo, G)).is_zero():
            mixed.append(str(ka))
    return {"antifield_antifield_fails": fails, "antifield_field_fails": mixed,
            "pass": not fails and not mixed, "pairs_tested": len(keys) * (len(keys) + 1) // 2 + len(keys)}


def build(eps_override=None, mutate=None):
    s = s_map(lam=1)
    X_inv = build_X_inv()
    Psi = Psi_density()
    out = {}
    out["Psi_parity_odd"] = Psi.parity_terms() == {1}
    out["Psi_ghost_number"] = sorted(ghost_number(Psi, GH))
    dL, dR = delta_Psi(Psi, "L"), delta_Psi(Psi, "R")
    out["left_equals_right_for_odd_Psi"] = all((dL[k] - dR[k]).is_zero() for k in dL)
    out["dPsi"] = {
        "dPsi/dh_mn": "2i (d^(m cbar^n) - 1/2 eta^{mn} d.cbar) * w_mn-pairing (independent comps m<=n)",
        "dPsi/dcbar_mu": "-i (2 D^mu[h] - b^mu)", "dPsi/db_mu": "+i cbar^mu", "dPsi/dc": "0", "dPsi/dphi": "0"
  +> }
    out["dPsi_nonzero_components"] = sorted(f"{f}{c}" for (f, c), v in dL.items() if not v.is_zero())
    sPsi = s_apply(Psi, s)
    SafdPsi = subst(X_inv, {(ANTI_OF[f], c): v for (f, c), v in dL.items()})
    SafdPsi = field_only(SafdPsi)
    rel_minus = is_divergence(SafdPsi + sPsi)
    rel_plus = is_divergence(SafdPsi - sPsi)
    out["S_af_at_dPsi_vs_sPsi"] = {"S_af(dPsi) ~ -sPsi": rel_minus, "S_af(dPsi) ~ +sPsi": rel_plus}
    # Hessian-based orientation
    cp2 = None
    k = list(sp.symbols("k0:4"))
    orient = {}
    for eps in (1, -1):
        Xt = transformed_X(X_inv, Psi, eps)
        Xf = field_only(Xt)
        Xf0 = kappa_coeff(Xf, KAPPA, 0)
        gf_hb = GE({key: v for key, v in Xf0.t.items() if key == ()})
        hess = hessian_check(gf_hb, cp2, k)
        orient[eps] = {"hessian": hess, "ghost_kinetic_i_box": ghost_kinetic_check(Xf0)}
    good = [e for e, d in orient.items()
            if d["hessian"]["schur_equals_k2P"] and d["hessian"]["propagator_hh_equals_inv_k2P"]
            and d["ghost_kinetic_i_box"]]
    eps = eps_override if eps_override is not None else (good[0] if len(good) == 1 else None)
    out["orientation_scan"] = {str(e): d for e, d in orient.items()}
    out["derived_eps"] = good[0] if len(good) == 1 else None
    out["map"] = {
        "fields": "Phi'^A = Phi^A (all fields unchanged)",
        "antifields": f"Phi*_A = Phi*'_A + ({eps}) dPsi/dPhi^A   i.e.  Phi*'_A = Phi*_A - ({eps}) dPsi/dPhi^A"
  +> ,
        "inverse": f"Phi^A = Phi'^A ; Phi*'_A = Phi*_A - ({eps}) dPsi/dPhi^A",
        "explicit": {"K'^{mn}": f"K^{{mn}} - ({eps}) dPsi/dh_mn", "cbar*'^mu": f"cbar*^mu - ({eps})(-i)(2D^mu[
  +> h]-b^mu)",
                     "b*'^mu": f"b*^mu - ({eps})(i cbar^mu)", "c*'": "c*", "phi*'": "phi*"},
    }
    if eps is None:
        out["pass"] = False
```

```
        return out, None
    Xt = transformed_X(X_inv, Psi, eps, mutate)
    out["S_Psi_equals_S_plus_sPsi_mod_d"] = is_divergence(field_only(Xt) - sPsi) if True else None
    Xf = field_only(Xt)
    out["gauge_fixed_pieces"] = {
        "kappa0_ghost_kinetic_i_box": ghost_kinetic_check(kappa_coeff(Xf, KAPPA, 0)),
        "kappa1_ghost_graviton_2ia_D[L_c h]": ghost_graviton_vertex_check(kappa_coeff(Xf, KAPPA, 1)),
        "kappa2_field_only_terms": len(kappa_coeff(Xf, KAPPA, 2).t),
        "kappa2_ghost_vertex_absent_derived": len(kappa_coeff(Xf, KAPPA, 2).t) == 0,
        "b_sector": "b(2D-b): eliminating b gives +D_mu D^mu",
    }
    hess = orient[eps]["hessian"]
    Ssc = scalar_density(0)
    ph = pw_bilinear(Ssc, {"phi": ({(): Fr(1)}, k)}, {"phi": ({(): Fr(1)}, [-x for x in k])})
    k2 = sum(C.ETA[r] * k[r] ** 2 for r in range(4))
    out["propagators"] = {
        "graviton": {"stated": "K = -Box P (includes S_gf)", **hess},
        "ghost": {"stated": "i cbar Box c", "match": orient[eps]["ghost_kinetic_i_box"]},
        "scalar": {"stated": "-Box (m=0)", "match": sp.simplify(ph - k2) == 0},
        "momentum": "generic symbolic (k0,k1,k2,k3)",
    }
    out["propagators_match_transformed_Hessians"] = bool(hess["schur_equals_k2P"] and hess["propagator_hh_equa
+> ls_inv_k2P"]
                                                          and orient[eps]["ghost_kinetic_i_box"] and sp.simplif
+> y(ph - k2) == 0)
    out["canonicity"] = canonicity(Psi, eps, mutate)
    out["external_antifields_retained"] = sorted({x[0] for kk in Xt.t for x in kk if x[0] in ("K", "phis", "bs
+> ")} |
                                                 {str(s_)[:3] for v in Xt.t.values() for s_ in v.free_symbols
                                                  if str(s_).startswith(("cs", "cbs"))})
    out["invariant_vs_transformed_kept_separate"] = {
        "invariant_extended": "S_ext = S_field + S_af(Phi*)  [nonminimal pair included; Psi not applied]",
        "transformed": f"S_Psi(Phi,Phi*') = S_ext(Phi, Phi*' + ({eps}) dPsi/dPhi)  [external antifields retain
+> ed]",
        "NOT_S_D": True}
    out["source_identity_relation"] = {
        "relation": f"Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' + ({eps}) dPsi/dPhi)",
        "classical_instance_verified": out["S_Psi_equals_S_plus_sPsi_mod_d"],
        "status": ("DEFINED classical pullback/interface along the canonical antifield shift; NOT an independe
+> ntly "
                   "established quantum gauge-change theorem (no quantum Gamma is constructed here)"),
        "ch_relevant_feed": ("cbar*-dependence of the invariant functional maps to h-dependence: "
                             f"cbar* = cbar*' + ({eps})(-i)(2D[h]-b); h*, b* shifts carry cbar (vanish on ch p
+> attern)"),
        "related_not_identical": True}
    out["source_identity_relation_defined"] = True
    ok = bool(out["Psi_parity_odd"] and out["Psi_ghost_number"] == [-1] and out["left_equals_right_for_odd_Psi
+> "]
              and rel_minus and not rel_plus and out["derived_eps"] == -1 and out["S_Psi_equals_S_plus_sPsi_mo
+> d_d"]
              and all(v for kk, v in out["gauge_fixed_pieces"].items() if isinstance(v, bool))
              and out["propagators_match_transformed_Hessians"] and out["canonicity"]["pass"])
    out["gauge_fixing_map_constructed"] = ok
    out["pass"] = ok
    return out, Xt


def main():
    t0 = time.time()
    muts = [m for m in os.environ.get("P13_MUTATIONS", "").split(",") if m]
    out, _ = build(mutate="halve_offdiag_h" if "alter_gf_shift" in muts else None)
    cert = {"revision": C.REVISION, "gate": "gauge_fixing_map_constructed", "Psi": C.PSI,
            **out,
            "execution_pass": True, "mutations_from_env": muts,
            "claim_status": "Constructed" if out["pass"] else "Partial",
            "scope": "classical BV gauge fixing of S_inv extended action by declared Psi; linear de Donder; th
+> rough O(kappa^2)",
            "NOT_import_shift_sign": True,
            "obstruction_respected": "No insertion completion; XVI-N thm:massless-obstruction untouched; XV-B
+> not opened; Route B not discharged",
            "seconds": round(time.time() - t0, 2)}
    (ROOT / "certificates" / "P13_gauge_fixing_map.json").write_text(certio.dumps(cert))
    print(json.dumps({k: cert[k] for k in ("pass", "derived_eps", "S_af_at_dPsi_vs_sPsi", "S_Psi_equals_S_plus
+> _sPsi_mod_d",
                                           "gauge_fixed_pieces", "propagators_match_transformed_Hessians", "ca
+> nonicity",
                                           "seconds")}, indent=1, default=str))
    return 0


if __name__ == "__main__":
```

```
    raise SystemExit(main())
```

101 Source: planewave_exact.py

[listing 68/133] code/classical/planewave_exact.py

```
#!/usr/bin/env python3
"""Exact-rational plane-wave evaluator for local action densities (p10; unchanged in p11).

Method (declared): a local density f that is multilinear after polarisation is a total
divergence iff its polarised plane-wave value vanishes for all momentum-conserving
momenta (sum k = 0): int d_mu V^mu -> i (sum k)_mu V^mu = 0.  We evaluate polarised
values EXACTLY over Q (fractions.Fraction) at random rational momenta/polarisations.
Every density used here has a uniform total derivative count n_d, so the common
factor i^{n_d} is stripped (d_mu -> k_mu).  p11: plane-wave evaluations are REGRESSION CONTROLS only.
No acceptance statement relies on them; the deterministic proofs are the Euler-operator identities in
grav_noether.py / quartic.py and the exact symbolic identities listed in P13_quartic_verified.json.

Jets: H{m}{n} = h_{mn}, dH{m}{n}_{r} = d_r h_{mn} (ell4_full_polynomial naming).
"""
from __future__ import annotations
import hashlib, itertools, json, random
from fractions import Fraction as Fr
from pathlib import Path
import sympy as sp

ROOT = Path(__file__).resolve().parents[2]
EXPORTS = ROOT / "exports"
ETA = (1, -1, -1, -1)
INDEX = [(m, n) for m in range(4) for n in range(m, 4)]
VARS = [f"H{m}{n}" for m, n in INDEX] + [f"dH{m}{n}_{r}" for r in range(4) for m, n in INDEX]
VPOS = {v: i for i, v in enumerate(VARS)}


def load_ell(k: int, path: Path | None = None):
    """Load exported ell_k (sympy) and its sha256(str)."""
    p = path or (EXPORTS / f"ell{k}_generating_polynomial.srepr")
    expr = sp.sympify(p.read_text().strip())
    return expr, hashlib.sha256(str(expr).encode()).hexdigest()


def compile_poly(expr):
    """sympy polynomial in jet symbols -> list[(Fraction, ((pos,pow),...))]."""
    syms = [sp.Symbol(v) for v in VARS]
    P = sp.Poly(expr, *syms)
    out = []
    for mon, c in P.terms():
        c = sp.Rational(c)
        fac = tuple((i, e) for i, e in enumerate(mon) if e)
        out.append((Fr(int(c.p), int(c.q)), fac))
    return out


def eval_poly(cp, jet):
    tot = Fr(0)
    for c, fac in cp:
        v = c
        for i, e in fac:
            x = jet[i]
            if x == 0:
                v = 0
                break
            v *= x ** e if e > 1 else x
        if v:
            tot += v
    return tot


# ---------- plane-wave jets ----------
def tensor_jet(eps, k):
    """Symmetric lowered polarisation eps[m][n], momentum k (lower) -> jet vector."""
    j = [Fr(0)] * len(VARS)
    for m, n in INDEX:
        j[VPOS[f"H{m}{n}"]] = Fr(eps[m][n])
        for r in range(4):
            j[VPOS[f"dH{m}{n}_{r}"]] = Fr(k[r]) * Fr(eps[m][n])
    return j
```

```
def jet_add(*js):
    return [sum(x) for x in zip(*js)]


def polarize(evalf, Xs):
    """Coefficient of t1..tn in f(sum t_i X_i) for f homogeneous of degree n=len(Xs)."""
    n = len(Xs)
    tot = Fr(0)
    for r in range(1, n + 1):
        for S in itertools.combinations(range(n), r):
            tot += (-1) ** (n - r) * evalf(jet_add(*[Xs[i] for i in S]))
    return tot


# ---------- random rational data ----------
def rnd_frac(rng, lo=-9, hi=9):
    while True:
        p = rng.randint(lo, hi)
        q = rng.randint(1, 5)
        if p:
            return Fr(p, q)


def rnd_sym(rng):
    e = [[Fr(0)] * 4 for _ in range(4)]
    for m, n in INDEX:
        e[m][n] = e[n][m] = rnd_frac(rng)
    return e


def rnd_vec(rng):
    return [rnd_frac(rng) for _ in range(4)]


# ---------- BRST images of plane waves (d -> k, uniform i stripped) ----------
def lower(v):
    return [ETA[i] * v[i] for i in range(4)]


def s0_h_pol(zeta_up, kc, a):
    """s0 h_{mn} = a (d_m c_n + d_n c_m), c = zeta e^{i kc x} -> polarisation, momentum kc."""
    zl = lower(zeta_up)
    return [[a * (kc[m] * zl[n] + kc[n] * zl[m]) for n in range(4)] for m in range(4)]


def lie_c_h_pol(zeta_up, kc, eps, kh, a, transport=1, rotation=1):
    """a (L_c h)_{mn} = a( c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r ); momentum kc+kh."""
    out = [[Fr(0)] * 4 for _ in range(4)]
    for m in range(4):
        for n in range(4):
            v = transport * sum(zeta_up[r] * kh[r] for r in range(4)) * eps[m][n]
            v += rotation * sum(eps[r][n] * kc[m] * zeta_up[r] for r in range(4))
            v += rotation * sum(eps[m][r] * kc[n] * zeta_up[r] for r in range(4))
            out[m][n] = a * v
    return out


def vadd(*vs):
    return [sum(x) for x in zip(*vs)]


def neg(v):
    return [-x for x in v]
```

102 Source: series_eval.py

[listing 69/133] code/classical/series_eval.py

```
#!/usr/bin/env python3
"""Independent evaluator of ell_n = -2 [eps^n] L_GammaGamma(eta + eps H) (truncated power series in eps).

Coded independently of ell4_full_polynomial (Neumann-series inverse, permutation determinant, binomial sqrt).
Generic over the coefficient domain `num`:
  * num = fractions.Fraction  -> exact numeric evaluation at a jet (regression controls);
  * num = symbolic ring        -> exact POLYNOMIAL in the 50 jet symbols (deterministic identity with the expo
  +> rt).
```

```
contraction: "correct"   L = sqrt(-q) q^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab})
             "swapped"   L = sqrt(-q) q^{mn}(G^a_{ma} G^b_{nb} - G^a_{mn} G^b_{ab})   (mutation)
"""
from __future__ import annotations
import itertools
from fractions import Fraction as Fr

ETA = (1, -1, -1, -1)
N = 5


def ell_from_jet(H, dH, nmax=4, contraction="correct", drop_sqrt_eps2=False, num=Fr):
    """H[m][n], dH[r][m][n] in the coefficient domain.  Returns {k: ell_k} for k <= nmax."""
    n = nmax + 1
    Z = num(0)

    def sc(c):
        return [num(c)] + [Z] * (n - 1)

    def smul(a, b):
        out = [Z] * n
        for i in range(n):
            if a[i] == 0:
                continue
            for j in range(n - i):
                if b[j] != 0:
                    out[i + j] = out[i + j] + a[i] * b[j]
        return out

    def sadd(*xs):
        out = [Z] * n
        for x in xs:
            out = [u + v for u, v in zip(out, x)]
        return out

    def sscale(a, c):
        c = num(c)
        return [c * x for x in a]

    g = [[[num(ETA[i]) if i == j else Z, H[i][j]] + [Z] * (n - 2) for j in range(4)] for i in range(4)]
    M = [[num(ETA[i]) * H[i][j] for j in range(4)] for i in range(4)]
    ginv = [[[Z] * n for _ in range(4)] for _ in range(4)]
    P = [[num(1) if i == j else Z for j in range(4)] for i in range(4)]
    for k in range(n):
        for i in range(4):
            for j in range(4):
                ginv[i][j][k] = ginv[i][j][k] + num((-1) ** k * ETA[j]) * P[i][j]
        P = [[sum((P[i][l] * M[l][j] for l in range(4)), Z) for j in range(4)] for i in range(4)]
    det = [Z] * n
    for perm in itertools.permutations(range(4)):
        sgn = 1
        for i in range(4):
            for j in range(i + 1, 4):
                if perm[i] > perm[j]:
                    sgn = -sgn
        t = sc(sgn)
        for i in range(4):
            t = smul(t, g[i][perm[i]])
        det = sadd(det, t)
    x = [Z] + [-v for v in det[1:]]           # -det = 1 + x
    sq = sc(1)
    term = sc(1)
    coef = Fr(1)
    for k in range(1, n):
        coef = coef * (Fr(1, 2) - (k - 1)) / k
        term = smul(term, x)
        sq = sadd(sq, sscale(term, coef))
    if drop_sqrt_eps2:
        sq[2] = Z

    def dg(m, s, nn):
        return [Z, dH[m][s][nn]] + [Z] * (n - 2)
    Gam = [[[None] * 4 for _ in range(4)] for _ in range(4)]
    for r in range(4):
        for m in range(4):
            for nn in range(m, 4):
                acc = [Z] * n
                for s in range(4):
                    br = sadd(dg(m, s, nn), dg(nn, s, m), sscale(dg(s, m, nn), -1))
                    acc = sadd(acc, smul(ginv[r][s], br))
                Gam[r][m][nn] = Gam[r][nn][m] = sscale(acc, Fr(1, 2))
```

```
    trG = [sadd(*[Gam[b][a][b] for b in range(4)]) for a in range(4)]
    L0 = [Z] * n
    for mu in range(4):
        for nu in range(4):
            if contraction == "correct":
                quad = sadd(*[smul(Gam[a][mu][b], Gam[b][nu][a]) for a in range(4) for b in range(4)])
            elif contraction == "swapped":
                quad = smul(trG[mu], trG[nu])
            else:
                raise ValueError(contraction)
            tr = sadd(*[smul(Gam[a][mu][nu], trG[a]) for a in range(4)])
            L0 = sadd(L0, smul(ginv[mu][nu], sadd(quad, sscale(tr, -1))))
    L = smul(sq, L0)
    return {k: num(-2) * L[k] for k in range(n)}


def jet_from_legs(legs):
    """legs: list of (eps[m][n], k[r]) plane waves, d -> k (uniform i stripped). Sum of legs."""
    H = [[sum(e[m][nn] for e, _ in legs) for nn in range(4)] for m in range(4)]
    dH = [[[sum(k[r] * e[m][nn] for e, k in legs) for nn in range(4)] for m in range(4)] for r in range(4)]
    return H, dH


def polarized_ell(nn, legs, contraction="correct", **kw):
    """[t_1..t_n] ell_nn(sum t_i leg_i)  (= unnormalised n-th variation D^n ell_nn(0)[legs])."""
    tot = Fr(0)
    m = len(legs)
    for r in range(1, m + 1):
        for S in itertools.combinations(range(m), r):
            H, dH = jet_from_legs([legs[i] for i in S])
            tot += (-1) ** (m - r) * ell_from_jet(H, dH, nmax=nn, contraction=contraction, **kw)[nn]
    return tot


def symbolic_ell(nmax=4, **kw):
    """Exact polynomials ell_k in the 50 jet symbols H{m}{n}, dH{m}{n}_{r} (sympy PolyRing over QQ)."""
    from sympy.polys.rings import ring
    from sympy import QQ
    INDEX = [(m, n) for m in range(4) for n in range(m, 4)]
    names = [f"H{m}{n}" for m, n in INDEX] + [f"dH{m}{n}_{r}" for r in range(4) for m, n in INDEX]
    R, *gens = ring(",".join(names), QQ)
    V = dict(zip(names, gens))

    def num(x):
        x = Fr(x)
        return R(QQ(x.numerator, x.denominator))
    H = [[V["H%d%d" % (min(m, n), max(m, n))] for n in range(4)] for m in range(4)]
    dH = [[[V["dH%d%d_%d" % (min(m, n), max(m, n), r)] for n in range(4)] for m in range(4)] for r in range(4)
  +> ]
    return R, ell_from_jet(H, dH, nmax=nmax, num=num, **kw)
```

103 Source: quartic.py

[listing 70/133] code/classical/quartic.py

```
#!/usr/bin/env python3
"""p11 quartic stage: build + export ell_2, ell_3, ell_4 (Gamma-Gamma density) and verify DETERMINISTICALLY.

Exports: exports/ell{2,3,4}_generating_polynomial.srepr, exports/V4_grav.json, exports/ell4_conventions.json
Certificates: certificates/P13_quartic_verified.json (+ P13_boundary_identity.json via boundary_identity.run)

ACCEPTANCE (all exact symbolic, no sampling):
  C1 q q^{-1} = 1 + O(eps^5), all 10 components symbolic
  C2 log(-det q) = tr log(eta^{-1} q) through eps^4 and [sqrt-series]^2 = -det q through eps^4, all 10 compone
  +> nts
  C3 boundary identity sqrt(-g)R - L_GG = - d_mu B^mu (boundary_identity.py: generic jets + conformal control)
  C4 exported ell_k == independent series evaluator run over the polynomial ring of the 50 jet symbols (identi
  +> ty)
  C5 ell_3 - XII-C ell_3 (XVI-N grav_density.ell3_and_curvature, symbolic jets) has identically vanishing Eule
  +> r
     derivatives (=> equal modulo a total divergence; same sign and normalisation)
  C6 ell_2 - XI-A Fierz-Pauli (XVI-N grav_density.FP_density): same Euler test
  C7 Noether s0 ell_{n+2} + s1 ell_{n+1} ~ 0, n = 0,1,2, on the EXPORTED polynomials (grav_noether.py, Euler t
  +> est)
REGRESSION CONTROLS (sampled, NOT acceptance): numeric evaluator vs export at random jets; plane-wave polarise
  +> d
  cubic/quadratic regression; plane-wave Noether.
MUTATIONS (each must fail a deterministic acceptance test):
```

```
  M1 one ell_4 coefficient x 7/5 -> C7 kappa^2; M2 ell_4 sign -> C7 kappa^2; M3 swapped contraction -> C4, C5,
  +>  C6, C7;
  M4 missing sqrt eps^2 coefficient -> C4, C7 kappa^2; M5 wrong inverse sign -> C1; M6 stale ell_4 hash -> sha
  +>  check;
  M7 wrong boundary orientation (+d B) -> C3 (generic and conformal).
"""
from __future__ import annotations
import contextlib, hashlib, io, itertools, json, random, sys, time
from fractions import Fraction as Fr
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
import conventions as C  # noqa: E402
import certio  # noqa: E402
import planewave_exact as PW  # noqa: E402
import series_eval as SE  # noqa: E402
import grav_noether as GN  # noqa: E402
import jetpoly as J  # noqa: E402
import boundary_identity as BI  # noqa: E402
from inverse_metric_series import product_qid_minus_one, eta_matrix  # noqa: E402

import inherited  # noqa: E402  (p13: pinned inherited inputs; workspace or vendored copy, sha256-verified)
GRAV_DENSITY = "XVI-N-r10/code/prep/grav_density.py"
EXPORTS = ROOT / "exports"
CERT = ROOT / "certificates"
eps = sp.symbols("eps")

CONVENTIONS = {
    "revision": C.REVISION, "eta": "diag(+,-,-,-)",
    "q": "eta + eps H, eps = kappa, H = h (lowered, symmetric); affine parametrisation",
    "S_EH": "-(2/kappa^2) int sqrt(-q) R",
    "density": C.L_GG, "boundary_identity": C.BOUNDARY_IDENTITY,
    "ell_k": "-2 [eps^k] L_GG ; S_EH = sum_k kappa^{k-2} int ell_k (integrated, compact support)",
    "IBP": "total derivatives dropped only inside the integrated action (orientation of the dropped divergence
  +>  irrelevant)",
    "truncation": "O(eps^4) inclusive", "jets": "H{m}{n} = h_mn (m<=n), dH{m}{n}_{r} = d_r h_mn", "a_ghost": "
  +> 1/2",
    "polarization": C.POLARIZATION,
}


def build_and_export(from_cache=None):
    t0 = time.time()
    if from_cache is not None and Path(from_cache).exists():
        import pickle
        ell = {k: sp.sympify(v) for k, v in pickle.load(open(from_cache, "rb"))["ell"].items()}
        how = f"cache {Path(from_cache).name} (written by the same builder)"
    else:
        from ell4_full_polynomial import expand_L_gamma_jet
        with contextlib.redirect_stdout(io.StringIO()):
            coeffs, _, _ = expand_L_gamma_jet(order=4)
        ell = {k: sp.expand(-2 * coeffs[k]) for k in range(5)}
        how = "fresh build ell4_full_polynomial.expand_L_gamma_jet(order=4)"
    out = {}
    for k in (2, 3, 4):
        (EXPORTS / f"ell{k}_generating_polynomial.srepr").write_text(sp.srepr(ell[k]) + "\n")
        e, h = PW.load_ell(k)
        out[k] = {"sha256": h, "terms_reproducibility": len(sp.Add.make_args(e))}
    out["ell0_zero"] = ell[0] == 0
    out["ell1_zero"] = ell[1] == 0
    out["build"] = how
    out["seconds"] = round(time.time() - t0, 1)
    (EXPORTS / "ell4_conventions.json").write_text(certio.dumps(CONVENTIONS))
    return out


# ---------------- C1 / M5 ----------------
def inverse_control(order=4):
    Hs = sp.Matrix(4, 4, lambda m, n: sp.Symbol(f"Hc{min(m, n)}{max(m, n)}"))
    rem = product_qid_minus_one(Hs, eps, order=order)
    fails = [(i, j) for i in range(4) for j in range(4) if sp.expand(rem[i, j]) != 0]
    return {"test": "q q^{-1} = 1 + O(eps^5), 10 symbolic components", "pass": not fails, "failures": fails}


def wrong_inverse_rejected():
    Hs = sp.Matrix(4, 4, lambda m, n: sp.Symbol(f"Hc{min(m, n)}{max(m, n)}"))
    eta = eta_matrix()
    bad = eta + eps * (eta * Hs * eta)
```

```
    prod = sp.expand((eta + eps * Hs) * bad) - sp.eye(4)
    return any(sp.expand(prod[i, j]) != 0 for i in range(4) for j in range(4))


# ---------------- C2 ----------------
def det_tracelog_generic():
    from sympy.polys.rings import ring
    from sympy import QQ
    IDX = [(m, n) for m in range(4) for n in range(m, 4)]
    R, e, *hs = ring("e," + ",".join(f"H{m}{n}" for m, n in IDX), QQ)
    Hm = {(m, n): hs[IDX.index((min(m, n), max(m, n)))] for m in range(4) for n in range(4)}
    Mm = [[C.ETA[i] * Hm[(i, j)] for j in range(4)] for i in range(4)]       # eta^{-1} H

    def trunc(p, n=4):
        return R({mon: c for mon, c in p.terms() if mon[0] <= n})
    # -det(eta + e H) = det(1 + e M)
    one_eM = [[(R(1) if i == j else R(0)) + e * Mm[i][j] for j in range(4)] for i in range(4)]
    det = R(0)
    for perm in itertools.permutations(range(4)):
        sgn = 1
        for i in range(4):
            for j in range(i + 1, 4):
                if perm[i] > perm[j]:
                    sgn = -sgn
        t = R(sgn)
        for i in range(4):
            t = t * one_eM[i][perm[i]]
        det += t
    x = det - 1
    logdet, xp = R(0), R(1)
    for j in range(1, 5):
        xp = trunc(xp * x)
        logdet += xp * QQ((-1) ** (j + 1), j)
    trlog, Mp = R(0), [[R(1) if i == j else R(0) for j in range(4)] for i in range(4)]
    for k in range(1, 5):
        Mp = [[sum((Mp[i][l] * Mm[l][j] for l in range(4)), R(0)) for j in range(4)] for i in range(4)]
        trlog += e ** k * sum((Mp[i][i] for i in range(4)), R(0)) * QQ((-1) ** (k + 1), k)
    # binomial sqrt series used by the independent evaluator: [sqrt(1+x)]^2 = 1+x mod e^5
    sq, term, coef = R(1), R(1), Fr(1)
    for k in range(1, 5):
        coef = coef * (Fr(1, 2) - (k - 1)) / k
        term = trunc(term * x)
        sq += term * QQ(coef.numerator, coef.denominator)
    return {"test": "log(-det q) = tr log(eta^{-1} q) and sqrt-series^2 = -det q, through eps^4, 10 symbolic c
  +> omponents",
            "logdet_equals_trlog": trunc(logdet) == trunc(trlog),
            "sqrt_series_squared_equals_minus_det": trunc(sq * sq) == trunc(det),
            "pass": trunc(logdet) == trunc(trlog) and trunc(sq * sq) == trunc(det)}


# ---------------- C4 ----------------
def export_identity(contraction="correct", drop_sqrt_eps2=False):
    R, ell = SE.symbolic_ell(contraction=contraction, drop_sqrt_eps2=drop_sqrt_eps2)
    res = {}
    for k in (2, 3, 4):
        e, _ = PW.load_ell(k)
        res[f"ell{k}"] = ell[k] == R.from_expr(e)
    return {"test": "export == independent series evaluator over QQ[50 jet symbols] (polynomial identity)",
            "results": res, "pass": all(res.values())}, R, ell


def evaluator_regression(samples=3, seed=11):
    rng = random.Random(seed)
    ok = True
    for k in (2, 3, 4):
        cp = PW.compile_poly(PW.load_ell(k)[0])
        for _ in range(samples):
            H = PW.rnd_sym(rng)
            dH = [PW.rnd_sym(rng) for _ in range(4)]
            j = [Fr(0)] * len(PW.VARS)
            for m, n in PW.INDEX:
                j[PW.VPOS[f"H{m}{n}"]] = H[m][n]
                for r in range(4):
                    j[PW.VPOS[f"dH{m}{n}_{r}"]] = dH[r][m][n]
            ok &= PW.eval_poly(cp, j) == SE.ell_from_jet(H, dH)[k]
    return {"role": "regression control (sampled; NOT acceptance)", "samples_per_k": samples, "agree": ok}


# ---------------- C5 / C6 ----------------
def _gd():
    sys.path.insert(0, str(inherited.resolve(GRAV_DENSITY).parent))
```

```
    with contextlib.redirect_stdout(io.StringIO()):
        import grav_density as GD
    return GD


def xvin_symbolic():
    GD = _gd()
    sm = {}
    S = lambda m, n: (min(m, n), max(m, n))  # noqa: E731
    G = [[None] * 4 for _ in range(4)]
    dG = [[[None] * 4 for _ in range(4)] for _ in range(4)]
    ddG = [[[[None] * 4 for _ in range(4)] for _ in range(4)] for _ in range(4)]
    for m in range(4):
        for n in range(4):
            a, b = S(m, n)
            nm = f"G{a}{b}"
            G[m][n] = sp.Symbol(nm)
            sm[nm] = ("h", (a, b), ())
            for r in range(4):
                nm = f"dG{a}{b}_{r}"
                dG[r][m][n] = sp.Symbol(nm)
                sm[nm] = ("h", (a, b), (r,))
                for q in range(4):
                    r1, q1 = min(r, q), max(r, q)
                    nm = f"ddG{a}{b}_{r1}{q1}"
                    ddG[r][q][m][n] = sp.Symbol(nm)
                    sm[nm] = ("h", (a, b), (r1, q1))
    x3 = GD.ell3_and_curvature(G, dG, ddG)["ell3"]
    fp = GD.FP_density(G, dG)
    return J.from_sympy(x3, sm), J.from_sympy(fp, sm)


def regression_deterministic(ell_jet, X3, FP):
    c3 = GN.divergence_equal(ell_jet[3], X3)
    c2 = GN.divergence_equal(ell_jet[2], FP)
    wrong = GN.divergence_equal(ell_jet[3], J.scale(X3, -1))
    return {"cubic": {"test": "E_h(ell3 - ell3_XIIC) = 0 for all 10 components (exact)", **c3,
                      "NOT_XII_C_quartic": True, "NOT_Ward_identity": True},
            "quadratic": {"test": "E_h(ell2 - FP_XIA) = 0 for all 10 components (exact)", **c2},
            "sign_discrimination": {"ell3 + ell3_XIIC is a divergence": wrong["divergence"]},
            "source": GRAV_DENSITY,
            "source_sha256": inherited.sha(inherited.resolve(GRAV_DENSITY)),
            "source_pin_ok": inherited.sha(inherited.resolve(GRAV_DENSITY)) == inherited.PINS[GRAV_DENSITY]["s
  +> ha256"]}


def planewave_regression(samples=1, seed=17):
    """Sampled regression control (NOT acceptance)."""
    GD = _gd()
    rng = random.Random(seed)

    def jet(legs):
        G = [[sp.Rational(sum(e[m][n] for e, _ in legs)) for n in range(4)] for m in range(4)]
        dG = [[[sp.Rational(sum(k[a] * e[m][n] for e, k in legs)) for n in range(4)] for m in range(4)] for a
  +> in range(4)]
        ddG = [[[[sp.Rational(sum(k[a] * k[b] * e[m][n] for e, k in legs)) for n in range(4)] for m in range(4
  +> )]
                 for b in range(4)] for a in range(4)]
        return G, dG, ddG

    def pol(f, legs):
        tot = 0
        for r in range(1, len(legs) + 1):
            for S in itertools.combinations(range(len(legs)), r):
                tot += (-1) ** (len(legs) - r) * f([legs[i] for i in S])
        return sp.Rational(tot)
    ok = True
    for _ in range(samples):
        ks = [PW.rnd_vec(rng) for _ in range(2)]
        ks.append(PW.neg(PW.vadd(*ks)))
        legs = [(PW.rnd_sym(rng), k) for k in ks]
        ok &= sp.Rational(SE.polarized_ell(3, legs)) == pol(lambda L: GD.ell3_and_curvature(*jet(L))["ell3"],
  +> legs)
    return {"role": "regression control (sampled; NOT acceptance)", "samples": samples, "agree": ok}


def V4_polarization():
    return {"convention": C.POLARIZATION,
            "definition": "V4[h1,h2,h3,h4] := [t1 t2 t3 t4] ell_4(sum_i t_i h_i) = D^4 ell_4(0)[h1,h2,h3,h4]",
            "normalisation": "UNnormalised fourth variation; the normalised symmetric 4-linear form is V4/4! (
  +> not used)",
```

```
          "implemented_by": "planewave_exact.polarize / series_eval.polarized_ell (inclusion-exclusion) comp
  +> ute V4",
          "where_factors_enter": {"1/4!": "only in S2 = (1/4!) V4[h,h,h,h]; cancelled by the 4! leg assignme
  +> nts",
                                  "i": "vertex factor i from e^{iS}", "kappa": "kappa^2 (S = sum kappa^{k-2}
  +>  int ell_k)",
                                  "DO2_symmetry_factor": "1/2 at graph level (two legs contracted)",
                                  "DO2_vertex_rule": "i kappa^2 V4[h1,h2,h3,h4]"},
          "DO2_quantum_evaluated": False}


def main(from_cache=None):
    t0 = time.time()
    built = build_and_export(from_cache)
    print("built", {k: built[k] for k in (2, 3, 4)}, flush=True)
    c1 = inverse_control()
    c2 = det_tracelog_generic()
    bi = BI.run()
    (CERT / f"{C.CERT_PREFIX}_boundary_identity.json").write_text(certio.dumps(bi))
    c4, R, ell_ring = export_identity()
    names = [str(g) for g in R.gens]
    ell_jet, shas = GN.load_export_polys()
    X3, FP = xvin_symbolic()
    reg = regression_deterministic(ell_jet, X3, FP)
    c7 = GN.noether(ell_jet)
    c7_params = GN.parameters(shas, transport=1)
    c7_accept = certio.accepted_orders(c7)
    c7_ok = all(c7_accept.values()) and not certio.invariant_violations(c7)
    print("C1..C7", c1["pass"], c2["pass"], bi["C3_proved"], c4["pass"], reg["cubic"]["divergence"],
          reg["quadratic"]["divergence"], c7_ok, flush=True)
    # mutations (deterministic)
    e4 = dict(ell_jet[4])
    k0 = sorted(e4)[len(e4) // 2]
    e4[k0] *= Fr(7, 5)
    m1 = GN.noether({2: ell_jet[2], 3: ell_jet[3], 4: e4})["kappa2"]["Q_identically_zero"]
    m2 = GN.noether({2: ell_jet[2], 3: ell_jet[3], 4: J.scale(ell_jet[4], -1)})["kappa2"]["Q_identically_zero"
  +> ]
    c4_sw, Rs, sw = export_identity(contraction="swapped")
    swj = {k: GN.ring_to_jet(sw[k], names) for k in (2, 3, 4)}
    reg_sw = regression_deterministic(swj, X3, FP)
    no_sw = GN.noether(swj)
    c4_ds, Rd, ds = export_identity(drop_sqrt_eps2=True)
    dsj = {k: GN.ring_to_jet(ds[k], names) for k in (2, 3, 4)}
    no_ds = GN.noether(dsj)
    stale_txt = (EXPORTS / "ell4_generating_polynomial.srepr").read_text().strip().replace("Integer(2)", "Inte
  +> ger(3)", 1)
    stale = hashlib.sha256(str(sp.sympify(stale_txt)).encode()).hexdigest() != built[4]["sha256"]
    muts = [
        {"mutation": "M1_corrupt_one_ell4_coefficient_x7/5", "failing_test": "C7 kappa^2 (Q != 0)", "rejected"
  +> : not m1},
        {"mutation": "M2_wrong_overall_norm_ell4_sign", "failing_test": "C7 kappa^2 (Q != 0)", "rejected": not
  +>  m2},
        {"mutation": "M3_swapped_GammaGamma_contraction", "failing_test": "C4, C5, C6, C7",
         "detail": {"C4": c4_sw["results"], "C5": reg_sw["cubic"]["divergence"], "C6": reg_sw["quadratic"]["di
  +> vergence"],
                    "C7": {k: v["Q_identically_zero"] for k, v in no_sw.items()}},
         "rejected": (not c4_sw["pass"]) and not reg_sw["cubic"]["divergence"]
         and not all(v["Q_identically_zero"] for v in no_sw.values())},
        {"mutation": "M4_missing_sqrt_eps2_coefficient", "failing_test": "C4 ell4, C7 kappa^2",
         "detail": {"C4": c4_ds["results"], "C7": {k: v["Q_identically_zero"] for k, v in no_ds.items()}},
         "rejected": (not c4_ds["pass"]) and not no_ds["kappa2"]["Q_identically_zero"]},
        {"mutation": "M5_wrong_inverse_sign", "failing_test": "C1", "rejected": wrong_inverse_rejected()},
        {"mutation": "M6_stale_ell4_hash", "failing_test": "sha256 vs certificate", "rejected": stale},
        {"mutation": "M7_wrong_boundary_orientation (+d_mu B^mu)", "failing_test": "C3 generic + conformal",
         "rejected": (not bi["generic_jet_proof_old_orientation"]["identically_zero"])
         and bi["conformal_control_old_orientation_fails"]["fails"]},
    ]
    controls = {"C1": c1["pass"], "C2": c2["pass"], "C3": bi["C3_proved"], "C4": c4["pass"],
                "C5": reg["cubic"]["divergence"], "C6": reg["quadratic"]["divergence"], "C7": c7_ok}
    muts_ok = all(m["rejected"] for m in muts)
    verified = all(controls.values()) and muts_ok
    cert = {
        "revision": C.REVISION, "gate": "quartic_generator_verified",
        "execution_pass": True,
        "claim_status": "verified" if verified else "Partial",
        "scope": "classical EH quartic density ell_4 (Gamma-Gamma form, affine q = eta + kappa h) on declared
  +> conventions",
        "quartic_generator_verified": verified,
        "acceptance_controls_deterministic": controls,
        "all_acceptance_deterministic": True,
        "DO2_vertex_available": verified, "DO2_quantum_evaluated": False,
```

```
      "ell2_sha256": built[2]["sha256"], "ell3_sha256": built[3]["sha256"], "ell4_sha256": built[4]["sha256"
  +> ],
      "terms_reproducibility": {k: built[k]["terms_reproducibility"] for k in (2, 3, 4)},
      "NOT_acceptance_by_term_count": True, "build": built, "conventions": CONVENTIONS,
      "controls": {"C1_inverse": c1, "C2_det_tracelog_generic": c2,
                   "C3_boundary_identity": f"{C.CERT_PREFIX}_boundary_identity.json (C3_proved={bi['C3_prove
  +> d']})",
                   "C4_export_identity": c4, "C5_C6_regression": reg, "C7_noether_exact": c7,
                   "C7_parameters": c7_params, "C7_accepted_orders": c7_accept},
      "regression_controls_sampled_NOT_acceptance": {"evaluator_numeric": evaluator_regression(),
                                                     "planewave_cubic": planewave_regression()},
      "identification_of_exported_coefficients": (
          "exports/ell_k (sha above) == -2[eps^k] L_GG(eta + eps h) as polynomials (C4, independent evaluato
  +> r); "
          "L_GG = sqrt(-g)R + d_mu B^mu exactly (C3); hence int ell_k = [kappa^{k-2}] S_EH for compact suppo
  +> rt."),
      "mutations": muts, "mutations_all_rejected": muts_ok,
      "polarization": V4_polarization(),
      "seconds": round(time.time() - t0, 1),
      "pass": verified,
    }
    (CERT / f"{C.CERT_PREFIX}_quartic_verified.json").write_text(certio.dumps(cert))
    v4 = {"id": "V4_grav", "revision": C.REVISION, "status": "Constructed" if verified else "not_verified",
          "quartic_generator_verified": verified, "D02_vertex_available": verified, "D02_quantum_evaluated": F
  +> alse,
          "ell4_sha256": built[4]["sha256"], "ell4_srepr_path": "exports/ell4_generating_polynomial.srepr",
          "terms_reproducibility": built[4]["terms_reproducibility"], "NOT_acceptance_by_term_count": True,
          "conventions": CONVENTIONS, "polarization": V4_polarization()}
    (EXPORTS / "V4_grav.json").write_text(certio.dumps(v4))
    print(json.dumps({"verified": verified, "controls": controls,
                      "mutations": {m["mutation"]: m["rejected"] for m in muts}, "sec": cert["seconds"]}, inde
  +> nt=1))
    return 0 if verified else 1


if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1] if len(sys.argv) > 1 else None))
```

104 Source: ell4_full_polynomial.py (builder library)

[listing 71/133] code/classical/ell4_full_polynomial.py

```
#!/usr/bin/env python3
"""p08: Full indexed ell_4 / V4_grav generating polynomial (order-by-order).

ell_4 = -2 [eps^4] L_Gamma,  L_Gamma = sqrt(-q) q^{mn}(Gamma^a_{mb} Gamma^b_{na} - Gamma^a_{mn} Gamma^b_{ab})
      == sqrt(-q) R  (modulo total derivatives; IBP policy).

Jet coordinates: 10 independent H_{munu} and d_rho H_{munu}.
"""
from __future__ import annotations
import hashlib, json, time, sys
from pathlib import Path
import sympy as sp

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Path(__file__).resolve().parent))
from inverse_metric_series import (  # noqa: E402
    inverse_series,
    product_qid_minus_one,
    negative_control_q11_series,
    expected_negative_control,
    ETA_SIG,
)

ETA = ETA_SIG
eps = sp.symbols("eps")
OUT = ROOT / "exports"
CERT = ROOT / "certificates"
OUT.mkdir(parents=True, exist_ok=True)
CERT.mkdir(parents=True, exist_ok=True)
INDEX = [(m, n) for m in range(4) for n in range(m, 4)]
ORDER = 4


def jet_symbols():
    H = {(m, n): sp.symbols(f"H{m}{n}") for m, n in INDEX}
    dH = {(r, m, n): sp.symbols(f"dH{m}{n}_{r}") for r in range(4) for m, n in INDEX}
```

```
    return H, dH


def sym_mat(comp):
    M = sp.zeros(4)
    for m in range(4):
        for n in range(4):
            key = (min(m, n), max(m, n))
            M[m, n] = comp[key]
    return M


def acceptance_controls():
    results = {"qid_identity": [], "negative_control": None, "pass": True}
    for name, H in [
        ("h11_const", sp.Matrix(4, 4, lambda i, j: sp.symbols("B") if i == j == 1 else 0)),
        ("h00_const", sp.Matrix(4, 4, lambda i, j: sp.symbols("A") if i == j == 0 else 0)),
        ("h01_const", sp.Matrix(4, 4, lambda i, j: sp.symbols("C") if {i, j} == {0, 1} else 0)),
    ]:
        rem = product_qid_minus_one(H, eps, order=4)
        ok = all(sp.simplify(rem[i, j]) == 0 for i in range(4) for j in range(4))
        results["qid_identity"].append({"support": name, "pass": bool(ok)})
        if not ok:
            results["pass"] = False
    B = sp.symbols("B")
    got = negative_control_q11_series(B, eps, order=4)
    exp = expected_negative_control(B, eps, order=4)
    nc_ok = sp.simplify(got - exp) == 0
    buggy = -1 - eps * B + eps**2 * B**2 - eps**3 * B**3 + eps**4 * B**4
    not_buggy = sp.simplify(got - buggy) != 0
    results["negative_control"] = {"pass": bool(nc_ok and not_buggy), "rejected_alternating_bug": bool(not_bug
  +> gy)}
    if not (nc_ok and not_buggy):
        results["pass"] = False
    return results


# --- Power series helpers: dict order -> sympy expr ---
def ps_const(expr, order=ORDER):
    return {0: sp.expand(expr), **{k: 0 for k in range(1, order + 1)}}


def ps_trunc(d, order=ORDER):
    return {k: sp.expand(d.get(k, 0)) for k in range(order + 1)}


def ps_add(a, b, order=ORDER):
    return {k: sp.expand(a.get(k, 0) + b.get(k, 0)) for k in range(order + 1)}


def ps_mul(a, b, order=ORDER):
    out = {k: 0 for k in range(order + 1)}
    for i in range(order + 1):
        ai = a.get(i, 0)
        if ai == 0:
            continue
        for j in range(order - i + 1):
            bj = b.get(j, 0)
            if bj == 0:
                continue
            out[i + j] = sp.expand(out[i + j] + ai * bj)
    return out


def ps_scale(a, c, order=ORDER):
    return {k: sp.expand(c * a.get(k, 0)) for k in range(order + 1)}


def mat_ps_from_series_matrix(M, order=ORDER):
    """Convert Matrix of polynomials in eps to Matrix of ps dicts."""
    out = [[None] * 4 for _ in range(4)]
    for i in range(4):
        for j in range(4):
            e = sp.expand(M[i, j])
            out[i][j] = {k: sp.expand(e.coeff(eps, k)) for k in range(order + 1)}
    return out


def mat_ps_mul(A, B, order=ORDER):
    C = [[ps_const(0) for _ in range(4)] for _ in range(4)]
    for i in range(4):
```

```
        for j in range(4):
            acc = ps_const(0)
            for k in range(4):
                acc = ps_add(acc, ps_mul(A[i][k], B[k][j], order), order)
            C[i][j] = acc
    return C


def expand_L_gamma_jet(order=ORDER):
    tmark = time.time()
    Hsyms, dHsyms = jet_symbols()
    H = sym_mat(Hsyms)

    def dH_mat(r):
        return sym_mat({(m, n): dHsyms[(r, m, n)] for m, n in INDEX})

    print("  inverse series...", flush=True)
    ginv_M = inverse_series(H, eps, order=order)
    ginv = mat_ps_from_series_matrix(ginv_M, order)

    print("  sqrt(-det)...", flush=True)
    eta = sp.diag(*ETA)
    g = sp.Matrix(4, 4, lambda i, j: eta[i, j] + eps * H[i, j])
    det = sp.expand(g.det())
    # series of sqrt(-det)
    sqrt_series = sp.series(sp.sqrt(sp.expand(-det)), eps, 0, order + 1).removeO()
    sqrt_ps = {k: sp.expand(sqrt_series.coeff(eps, k)) for k in range(order + 1)}
    print("    sqrt done", round(time.time() - tmark, 2), flush=True)

    print("  Christoffel...", flush=True)
    # d_m g_sn = eps * dH_mat(m)[s,n]  => ps with only order 1
    def dg(m, s, n):
        return {0: 0, 1: dH_mat(m)[s, n], **{k: 0 for k in range(2, order + 1)}}

    Gam = [[[None] * 4 for _ in range(4)] for _ in range(4)]
    for r in range(4):
        for m in range(4):
            for n in range(4):
                acc = ps_const(0)
                for s in range(4):
                    # ginv[r,s] * (d_m g_sn + d_n g_sm - d_s g_mn)
                    bracket = ps_add(ps_add(dg(m, s, n), dg(n, s, m), order), ps_scale(dg(s, m, n), -1, order)
  +> , order)
                    acc = ps_add(acc, ps_mul(ginv[r][s], bracket, order), order)
                Gam[r][m][n] = ps_scale(acc, sp.Rational(1, 2), order)
        print(f"    r={r}", round(time.time() - tmark, 2), flush=True)

    print("  L_Gamma contraction...", flush=True)
    # L0 = q^{mu nu} (Gamma^a_{mu b} Gamma^b_{nu a} - Gamma^a_{mu nu} Gamma^b_{a b})
    L0 = ps_const(0)
    for mu in range(4):
        for nu in range(4):
            quad = ps_const(0)
            for a in range(4):
                for b in range(4):
                    quad = ps_add(quad, ps_mul(Gam[a][mu][b], Gam[b][nu][a], order), order)
            tr = ps_const(0)
            for a in range(4):
                for b in range(4):
                    tr = ps_add(tr, ps_mul(Gam[a][mu][nu], Gam[b][a][b], order), order)
            struct = ps_add(quad, ps_scale(tr, -1, order), order)
            L0 = ps_add(L0, ps_mul(ginv[mu][nu], struct, order), order)
        print(f"    mu={mu}", round(time.time() - tmark, 2), flush=True)

    L = ps_mul(sqrt_ps, L0, order)
    print("  expand coeffs...", flush=True)
    coeffs = {k: sp.expand(L[k]) for k in range(order + 1)}
    print("  done expand", round(time.time() - tmark, 2), flush=True)
    return coeffs, Hsyms, dHsyms


def substitute_sparse(ell4, Hsyms, dHsyms, components):
    X = sp.symbols("x0:4")
    funs = {}
    for (m, n), stem in components.items():
        funs[(m, n)] = sp.Function(stem)(*X)
    reps = {}
    for m, n in INDEX:
        key = (m, n)
        f = funs.get(key, funs.get((n, m), 0))
        reps[Hsyms[key]] = 0 if f == 0 else f
```

```
        for r in range(4):
            reps[dHsyms[(r, m, n)]] = 0 if f == 0 else sp.diff(f, X[r])
    return sp.expand(ell4.xreplace(reps))


def main():
    t0 = time.time()
    controls = acceptance_controls()
    print("controls", controls["pass"], flush=True)
    if not controls["pass"]:
        (CERT / "P4_ell4_full_polynomial.json").write_text(json.dumps({
            "revision": "p08", "pass": False, "acceptance_controls": controls
        }, indent=2) + "\n")
        return 1

    coeffs, Hsyms, dHsyms = expand_L_gamma_jet(order=ORDER)
    ell4 = sp.expand(-2 * coeffs[4])
    ell3 = sp.expand(-2 * coeffs[3])
    print("ell4 terms", len(sp.Add.make_args(ell4)) if ell4 != 0 else 0, "len", len(str(ell4)), flush=True)

    s_ell = str(ell4)
    H_present = []
    for m, n in INDEX:
        if str(Hsyms[(m, n)]) in s_ell or any(str(dHsyms[(r, m, n)]) in s_ell for r in range(4)):
            H_present.append(f"H{m}{n}")
    max_der = 1 if any(str(v) in s_ell for v in dHsyms.values()) else 0

    sparse_checks = []
    for name, spec in [
        ("h00_only", {(0, 0): "A"}),
        ("h11_only", {(1, 1): "B"}),
        ("h01_only", {(0, 1): "C"}),
        ("h12_only", {(1, 2): "D"}),
        ("h00_h11", {(0, 0): "A", (1, 1): "B"}),
    ]:
        sub = substitute_sparse(ell4, Hsyms, dHsyms, spec)
        nz = sp.simplify(sub) != 0
        sparse_checks.append({
            "support": name,
            "ell4_jet_nonzero_on_support": bool(nz),
            "sha_head": hashlib.sha256(str(sp.expand(sub)).encode()).hexdigest()[:16],
        })
        print(name, "nz", nz, flush=True)

    term_count = len(sp.Add.make_args(ell4)) if ell4 != 0 else 0
    ell4_sha = hashlib.sha256(str(ell4).encode()).hexdigest()
    ell4_path = OUT / "ell4_generating_polynomial.srepr"
    ell4_path.write_text(sp.srepr(ell4) + "\n")

    export = {
        "id": "V4_grav",
        "revision": "p08",
        "status": "Constructed",
        "definition": "ell_4=-2[eps^4]L_Gamma; L_Gamma=sqrt(-q)q^{mn}(GammaGamma-GammaGamma) == sqrt(-q)R modu
 +> lo d(.)",
        "IBP_policy": "Compact support; discard total derivatives; Gamma-Gamma action density",
        "metric_normalization": "q=eta+eps H, eta=diag(+---), S_EH=-(2/kappa^2)int sqrt(-q)R",
        "n_independent_components": 10,
        "components": [f"H{m}{n}" for m, n in INDEX],
        "H_present_in_ell4": H_present,
        "max_derivative_order": max_der,
        "field_order": 4,
        "ell4_sha256": ell4_sha,
        "ell4_length": len(str(ell4)),
        "ell4_term_count": term_count,
        "ell4_srepr_path": str(ell4_path),
        "ell4_srepr_sha256": hashlib.sha256(ell4_path.read_bytes()).hexdigest(),
        "ell4_str_head": str(ell4)[:500],
        "ell3_sha256_from_same_prescription": hashlib.sha256(str(ell3).encode()).hexdigest(),
        "ell3_note": "Cubic from same L_Gamma expansion for regression vs XII-C ell_3; NOT claiming XII-C is q
 +> uartic",
        "NOT_XII_C_quartic": True,
        "pairing": {
            "vertex_id": "V4_grav",
            "legs": 4,
            "species": ["h", "h", "h", "h"],
            "D02_rule": "contract two h-legs with gauge-fixed <greek_capital_letter_delta>(h); two external",
            "evaluation_entry_point": "code/classical/v4_grav_pairings.py:evaluate_D02_template",
            "P33_connecting_map": "Unresolved",
        },
        "D02_availability": "unblocked_vertex_Constructed",
```

```
        "usable_by_graph_generator": True,
    }
    (OUT / "V4_grav.json").write_text(json.dumps(export, indent=2) + "\n")
    (OUT / "ell4_jet_symbols.json").write_text(json.dumps({
        "H": {f"{m}{n}": str(Hsyms[(m, n)]) for m, n in INDEX},
        "dH": {f"{m}{n}_{r}": str(dHsyms[(r, m, n)]) for r in range(4) for m, n in INDEX},
    }, indent=2) + "\n")

    nz_sparse = sum(1 for r in sparse_checks if r["ell4_jet_nonzero_on_support"])
    constructed = bool(
        controls["pass"] and ell4 != 0 and max_der <= 1
        and len(H_present) >= 8 and nz_sparse >= 3 and term_count > 0
    )
    cert = {
        "revision": "p08",
        "landmark": "XVI-P",
        "gate": "P-next.4_ell4_full_polynomial",
        "pass": constructed,
        "pass_means": (
            "Full indexed ell_4 Constructed as -2[eps^4]L_Gamma in 10 H-components + dH jets; "
            "IBP/Gamma-Gamma declared; controls+sparse checks passed; exports/V4_grav.json + srepr. "
            "NOT XII-C quartic."
        ) if constructed else "Construction checks failed",
        "acceptance_controls": controls,
        "status": "Constructed" if constructed else "Unresolved",
        "method": "order_by_order_jet_GammaGamma_correct_inverse",
        "IBP_policy": export["IBP_policy"],
        "polynomial": {
            "n_components": 10,
            "H_present": H_present,
            "max_derivative_order": max_der,
            "field_order": 4,
            "term_count": term_count,
            "sha256": ell4_sha,
            "srepr_path": str(ell4_path),
            "export_path": str(OUT / "V4_grav.json"),
        },
        "sparse_support_checks": sparse_checks,
        "cubic_regression_record": {
            "ell3_sha256": export["ell3_sha256_from_same_prescription"],
            "note": export["ell3_note"],
            "NOT_XII_C_quartic": True,
        },
        "D02_availability": "unblocked_vertex_Constructed" if constructed else "still_blocked",
        "seconds": round(time.time() - t0, 3),
        "no_invented_s2": True,
    }
    (CERT / "P4_ell4_full_polynomial.json").write_text(json.dumps(cert, indent=2) + "\n")
    print(json.dumps({
        "pass": cert["pass"], "terms": term_count, "H_present": len(H_present),
        "max_der": max_der, "seconds": cert["seconds"], "D02": cert["D02_availability"],
    }, indent=2), flush=True)
    return 0 if constructed else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

105 Source: inverse_metric_series.py

[listing 72/133] code/classical/inverse_metric_series.py

```
#!/usr/bin/env python3
"""Shared weak-field inverse-metric series (X-A / XI-A controls).

For q = eta + eps H (lowered indices), with Minkowski eta = diag(+1,-1,-1,-1)
so eta^{-1} = eta and eta^2 = I:

  q^{-1} = eta - eps (eta H eta) + eps^2 (eta H eta H eta)
         - eps^3 (eta H eta H eta H eta) + eps^4 (...) + O(eps^5)

Equivalently with h1 = eta*H*eta:
  h2 = h1 * eta * h1
  h3 = h2 * eta * h1
  h4 = h3 * eta * h1

WRONG (o57-v4 bug): h2 = h1*h1 (matrix product of two raised tensors without eta).
"""
```

```
from __future__ import annotations
import sympy as sp

ETA_SIG = (1, -1, -1, -1)


def eta_matrix():
    return sp.diag(*ETA_SIG)


def inverse_series(H: sp.Matrix, eps, order: int = 4) -> sp.Matrix:
    """Return q^{-1} series truncated through eps^order (inclusive)."""
    eta = eta_matrix()
    h1 = sp.expand(eta * H * eta)
    powers = [h1]
    for _ in range(2, order + 1):
        powers.append(sp.expand(powers[-1] * eta * h1))
    ginv = eta
    sign = -1
    for k in range(1, order + 1):
        ginv = ginv + sign * (eps**k) * powers[k - 1]
        sign = -sign
    return sp.Matrix(4, 4, lambda i, j: sp.expand(ginv[i, j]))


def q_matrix(H: sp.Matrix, eps) -> sp.Matrix:
    eta = eta_matrix()
    return sp.Matrix(4, 4, lambda i, j: sp.expand(eta[i, j] + eps * H[i, j]))


def product_qid_minus_one(H: sp.Matrix, eps, order: int = 4) -> sp.Matrix:
    """Return q * q^{-1}_series - I, coefficients through eps^order should vanish."""
    q = q_matrix(H, eps)
    qinv = inverse_series(H, eps, order=order)
    prod = sp.expand(q * qinv)
    I = sp.eye(4)
    return sp.Matrix(4, 4, lambda i, j: sp.series(prod[i, j] - I[i, j], eps, 0, order + 1).removeO())


def negative_control_q11_series(B, eps, order: int = 4):
    """For H with only H_11=B: q^{11} = -1 - eps B - eps^2 B^2 - ... (all minus)."""
    H = sp.zeros(4)
    H[1, 1] = B
    qinv = inverse_series(H, eps, order=order)
    return sp.series(qinv[1, 1], eps, 0, order + 1).removeO()


def expected_negative_control(B, eps, order: int = 4):
    s = -1
    for k in range(1, order + 1):
        s = s - (eps**k) * (B**k)
    return sp.expand(s)
```

106 Source: master_kappa2.py

[listing 73/133] code/classical/master_kappa2.py

```
#!/usr/bin/env python3
"""p11 (d): evaluate 2(S0,S2)+(S1,S1) (and the kappa^0, kappa^1 regressions) by antifield number,
in invariant AND transformed (gauge-fixed, external antifields retained) variables.

S = S_field + X,  X antifield-linear.  (S_field,S_field)=0 identically, so
  (S,S) = 2 (S_field, X) + (X, X),   2(S_field,X) = 2 int E_Phi(S_field) . sigma s Phi  (af 0)
* (X,X): exact graded-engine bracket (XVI-N orientation), split by kappa order, antifield number
  and bracket pair; divergence test by Euler operators (exact).
* 2(S_field,X), scalar sector: exact engine bracket + divergence test.
* 2(S_field,X), gravitational sector (p11, DETERMINISTIC): W_n = s0 ell_{n+2} + s1 ell_{n+1} built as exact
  jet polynomials from the EXPORTED ell_2, ell_3, ell_4 (sha-checked); W_n is c-linear, so W_n ~ c^mu E_{c^mu}
  +> (W_n)
  and W_n is a divergence iff E_{c^mu}(W_n) = 0 identically (grav_noether.py).  The s0-alone piece is reported
  +>  to
  exhibit the mixed cancellation.  Plane-wave evaluation is retained as a sampled REGRESSION CONTROL only.
Multidegree lemma (corrected p11): with P_d the projection onto field multidegree d and E_u the Euler operator
  +>  of
field u, E_u P_d = P_{d - e_u} E_u (E_u lowers the u-degree by one and preserves the others; total derivatives
preserve multidegree).  Hence if F = sum_d P_d F has E_u F = 0 for all u, then P_{d-e_u} E_u F = E_u P_d F = 0
  +>  for every
d and u, so every homogeneous component P_d F with |d| >= 1 has vanishing Euler derivatives and is a divergenc
```

```
  +> e
(no constant term occurs); conversely a sum of divergences is a divergence.  So a sum is a divergence iff each
multidegree component is.  Within one multidegree all contributions are summed before testing.
"""
from __future__ import annotations
import hashlib, json, random, sys, time
from fractions import Fraction as Fr
from pathlib import Path
import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parent))
from graded_jets import (GE, D, bracket, euler, comps, kappa_coeff, divergence_residual, PAIRS,  # noqa: E402
                         _KEY, set_parity)
from bv_model import KAPPA, A, s_map, Psi_density, hc  # noqa: E402
from source_action import build_X_inv, scalar_density  # noqa: E402
from gauge_fixing_map import transformed_X  # noqa: E402
import planewave_exact as PW  # noqa: E402
import conventions as C  # noqa: E402
import certio  # noqa: E402
import grav_noether as GN  # noqa: E402
import jetpoly as J  # noqa: E402
import os

ROOT = Path(__file__).resolve().parents[2]
AF_ODD = {"K", "bs", "phis"}
AF_EVEN = ("cs", "cbs")


def af_number_split(F):
    out = {}
    for k, v in F.t.items():
        n_odd = sum(1 for x in k if x[0] in AF_ODD)
        for term in sp.Add.make_args(sp.expand(v)):
            n = n_odd
            for s_, e in term.as_powers_dict().items():
                kk = _KEY.get(s_)
                if kk is not None and kk[0] in AF_EVEN:
                    n += int(e)
            out.setdefault(n, {})
            out[n][k] = out[n].get(k, 0) + term
    return {n: GE(t) for n, t in out.items()}


def kappa_parts(X, nmax=2):
    return {n: kappa_coeff(X, KAPPA, n) for n in range(nmax + 1)}


def bracket_pairs(F, G):
    return {f: bracket(F, G, pairs=[(f, fs)]) for f, fs in PAIRS}


def residual_summary(F):
    res = divergence_residual(F)
    return {"is_divergence": not res, "n_terms": len(F.t),
            "nonzero_euler_components": sorted(f"{f}{c}" for f, c in res)[:12]}


def XX_by_order(X, label, nmax=2, per_pair=True):
    Xk = kappa_parts(X, nmax)
    out = {}
    for n in range(nmax + 1):
        pairs_total = {f: GE() for f, _ in PAIRS}
        for i in range(n + 1):
            j = n - i
            if Xk[i].is_zero() or Xk[j].is_zero():
                continue
            for f, v in bracket_pairs(Xk[i], Xk[j]).items():
                pairs_total[f] = pairs_total[f] + v
        tot = GE()
        for v in pairs_total.values():
            tot = tot + v
        split = af_number_split(tot)
        rec = {"total": residual_summary(tot), "by_antifield_number": {}}
        for af in sorted(set(split) | {0, 1, 2}):
            part = split.get(af, GE())
            prs = {}
            if per_pair:
                for f, v in pairs_total.items():
                    pv = af_number_split(v).get(af, GE())
                    if not pv.is_zero():
                        prs[f] = residual_summary(pv)
```

```
            rec["by_antifield_number"][af] = {"total": residual_summary(part), "per_bracket_pair": prs,
                                              "mixed_cancellation": (len(prs) > 1 and residual_summary(part)[
  +> "is_divergence"]
                                                                    and any(not p["is_divergence"] for p in
  +>  prs.values()))}
            out[n] = rec
        return out


def scalar_sector(X, nmax=2, mutate_a2=None):
    L = scalar_density(nmax, mutate_a2=mutate_a2)
    Lk = kappa_parts(L, nmax)
    Xk = kappa_parts(X, nmax)
    out = {}
    for n in range(nmax + 1):
        tot = GE()
        pieces = {}
        for i in range(n + 1):
            j = n - i
            if Lk[i].is_zero() or Xk[j].is_zero():
                continue
            b = bracket(Lk[i], Xk[j]).scale(2)
            pieces[f"2(L{i},X{j})"] = residual_summary(b)["is_divergence"]
            tot = tot + b
        out[n] = {"total": residual_summary(tot), "pieces_individually_divergence": pieces}
    return out


# ---------------- gravitational sector (exact plane waves) ----------------
def load_grav(mutate_ell4=None):
    data = {}
    for k in (2, 3, 4):
        expr, sha = PW.load_ell(k)
        cp = PW.compile_poly(expr)
        if k == 4 and mutate_ell4 is not None:
            cp = [(c * mutate_ell4 if i == 0 else c, f) for i, (c, f) in enumerate(cp)]
        data[k] = {"cp": cp, "sha": sha, "terms": len(cp)}
    return data


def grav_noether(data, n, rng, a=Fr(1, 2), s1_scale=Fr(1), transport=1, s1_sign=1):
    ev = {k: (lambda j, cp=data[k]["cp"]: PW.eval_poly(cp, j)) for k in data}
    nh = n + 1
    eps = [PW.rnd_sym(rng) for _ in range(nh)]
    ks = [PW.rnd_vec(rng) for _ in range(nh)]
    kc = PW.neg(PW.vadd(*ks))
    zeta = PW.rnd_vec(rng)
    W = [PW.tensor_jet(eps[i], ks[i]) for i in range(nh)]
    S0 = PW.tensor_jet(PW.s0_h_pol(zeta, kc, a), kc)
    Y = [PW.tensor_jet(PW.lie_c_h_pol(zeta, kc, eps[i], ks[i], a * s1_scale * s1_sign, transport=transport),
                       PW.vadd(kc, ks[i])) for i in range(nh)]
    piece_s0 = PW.polarize(ev[n + 2], W + [S0])
    piece_s1 = Fr(0)
    if n >= 1:
        for i in range(nh):
            others = [W[j] for j in range(nh) if j != i]
            piece_s1 += PW.polarize(ev[n + 1], others + [Y[i]])
    return piece_s0, piece_s1


def grav_sector(data, samples=10, seed=2026, **kw):
    out = {}
    for n in (0, 1, 2):
        rng = random.Random(seed + n)
        vals = []
        for _ in range(samples):
            p0, p1 = grav_noether(data, n, rng, **kw)
            vals.append((p0, p1))
        out[n] = {
            "samples": samples,
            "all_sums_exactly_zero": all(p0 + p1 == 0 for p0, p1 in vals),
            "s0_piece_nonzero_in_samples": sum(1 for p0, _ in vals if p0 != 0),
            "s1_piece_nonzero_in_samples": sum(1 for _, p1 in vals if p1 != 0),
            "max_abs_sum": str(max(abs(p0 + p1) for p0, p1 in vals)),
            "example_pieces": [str(vals[0][0]), str(vals[0][1])],
            "density": {0: "s0 ell2", 1: "s0 ell3 + s1 ell2", 2: "s0 ell4 + s1 ell3 (+ s2 ell2, s2=0 derived)"
  +> }[n],
        }
    return out
```

```
def grav_deterministic(mutations=()):
    ell, sha = GN.load_export_polys()
    if "corrupt_ell4" in mutations:
        e4 = dict(ell[4])
        k0 = sorted(e4)[len(e4) // 2]
        e4[k0] *= Fr(7, 5)
        ell[4] = e4
    tr = 0 if "drop_s1_transport" in mutations else 1
    return GN.noether(ell, transport=tr), sha, GN.parameters(sha, transport=tr)


def run(mutations=(), samples=3):
    t0 = time.time()
    muts = set(mutations)
    sigma = {"h": 1, "c": -1, "cb": -1, "phi": 1}
    if "flip_sigma_c" in muts:
        sigma["c"] = 1
    drop = ("c",) if "drop_cstar" in muts else ()
    X_inv = build_X_inv(sigma=sigma, drop=drop)
    Psi = Psi_density()
    X_psi = transformed_X(X_inv, Psi, -1, mutate="halve_offdiag_h" if "alter_gf_shift" in muts else None)
    if "flip_odd_derivative_sign" in muts:
        set_parity("c", 0)   # treat the ghost as even: odd left/right derivative signs lost
    try:
        inv = XX_by_order(X_inv, "invariant")
        trf = XX_by_order(X_psi, "transformed")
        sc = scalar_sector(X_inv, mutate_a2=Fr(1, 3) if "corrupt_a2" in muts else None)
    finally:
        set_parity("c", 1)
    gdet, gsha, gparams = grav_deterministic(muts)
    data = load_grav(mutate_ell4=Fr(7, 5) if "corrupt_ell4" in muts else None)
    grav = grav_sector(data, samples=samples,
                       transport=0 if "drop_s1_transport" in muts else 1)

    def ok_order(d, n):
        return all(v["total"]["is_divergence"] for v in d[n]["by_antifield_number"].values())

    residuals = []
    for n in (0, 1, 2):
        for name, d in (("invariant", inv), ("transformed", trf)):
            for af, v in d[n]["by_antifield_number"].items():
                if not v["total"]["is_divergence"]:
                    residuals.append(f"{name}_kappa{n}_af{af}:{','.join(v['total']['nonzero_euler_components']
  +> [:4])}")
        if not sc[n]["total"]["is_divergence"]:
            residuals.append(f"scalar_noether_kappa{n}_af0")
        if not (gdet[f"kappa{n}"]["Q_identically_zero"] and gdet[f"kappa{n}"]["Q_terms"] == [0, 0, 0, 0]):
            residuals.append(f"grav_noether_kappa{n}_af0:Q_terms={gdet[f'kappa{n}']['Q_terms']}")
    k2_res = [r for r in residuals if "kappa2" in r]
    return {
        "invariant_variables": inv, "transformed_variables": trf,
        "scalar_af0": sc, "grav_af0_deterministic": gdet, "grav_af0_parameters": gparams,
        "grav_af0_accepted_orders": certio.accepted_orders(gdet),
        "grav_af0_planewave_regression_control_NOT_acceptance": grav,
        "grav_inputs": {k: {"sha256": data[k]["sha"], "terms": data[k]["terms"]} for k in data},
        "residuals_all_orders": residuals, "residuals_kappa2": k2_res,
        "seconds": round(time.time() - t0, 2), "mutations": sorted(muts),
    }


def master_residuals(r, sha_ok):
    """Explicit list: every non-divergent component at any order, plus an ell4 provenance failure."""
    res = list(r["residuals_all_orders"])
    if not sha_ok:
        res.append("ell4_sha256_mismatch_vs_P13_quartic_verified")
    return res


def env_mutations():
    return tuple(m for m in os.environ.get("P13_MUTATIONS", "").split(",") if m)


def main(samples=3):
    muts = env_mutations()
    r = run(mutations=muts, samples=samples)
    q = json.loads((ROOT / "certificates" / "P13_quartic_verified.json").read_text()) \
        if (ROOT / "certificates" / "P13_quartic_verified.json").exists() else {}
    sha_ok = (q.get("ell4_sha256") == r["grav_inputs"][4]["sha256"]
              and q.get("ell3_sha256") == r["grav_inputs"][3]["sha256"]
              and q.get("ell2_sha256") == r["grav_inputs"][2]["sha256"])
    quartic_ok = bool(q.get("quartic_generator_verified"))
```

```
    prereq = [] if quartic_ok else ["prerequisite_failed:P13_quartic_verified.quartic_generator_verified"]
    verified = (not r["residuals_all_orders"]) and sha_ok and quartic_ok
    af_table = {}
    for n in (0, 1, 2):
        af_table[f"kappa{n}"] = {
            "af0_grav_Noether": r["grav_af0_deterministic"][f"kappa{n}"]["Q_identically_zero"],
            "af0_scalar_Noether": r["scalar_af0"][n]["total"]["is_divergence"],
            "af0_ghost_gf_transformed": r["transformed_variables"][n]["by_antifield_number"][0]["total"]["is_d
  +> ivergence"],
            "af1_invariant": r["invariant_variables"][n]["by_antifield_number"][1]["total"]["is_divergence"],
            "af1_transformed": r["transformed_variables"][n]["by_antifield_number"][1]["total"]["is_divergence
  +> "],
            "af2_invariant": r["invariant_variables"][n]["by_antifield_number"][2]["total"]["is_divergence"],
            "af2_transformed": r["transformed_variables"][n]["by_antifield_number"][2]["total"]["is_divergence
  +> "],
        }
    cert = {
        "revision": C.REVISION, "gate": "required_classical_BV_components_verified",
        "equation": "(S,S) at kappa^2: 2(S0,S2)+(S1,S1) = 0  [plus regressions (S0,S0)=0, 2(S0,S1)=0]",
        "bracket": C.ANTIBRACKET, "baseline": C.BASELINE,
        "identification_with_exports": {
            "ell2_ell3_ell4": r["grav_inputs"],
            "ell2_ell3_ell4_sha_match_P13_quartic_verified": sha_ok,
            "quartic_generator_verified": quartic_ok,
            "scalar_a1_a2": "re-derived and identified in P13_source_action (B = sqrt(-g) g^{-1})",
            "s_images": "P13_brst_derivation (s2 derived zero)"},
        "table_by_kappa_and_antifield_number": af_table,
        "details": r,
        "required_master_residuals": master_residuals(r, sha_ok) + prereq,
        "required_classical_BV_components_verified": verified,
        "evidence_kind": {
            "engine_parts": "exact symbolic (graded jets; Euler-operator divergence test)",
            "grav_af0": ("DETERMINISTIC exact polynomial identity: E_{c^mu}(s0 ell_{n+2} + s1 ell_{n+1}) = 0 f
  +> or n = 0,1,2 "
                         "on the exported polynomials (grav_noether.py); structural reason: sqrt(-g)R is a sca
  +> lar density, "
                         "s(sqrt(-g)R) = d(xi sqrt(-g)R), and L_GG = sqrt(-g)R + d_mu B^mu (P13_boundary_ident
  +> ity: "
                         "sqrt(-g)R - L_GG = -d_mu B^mu), with s d_mu B^mu = d_mu s B^mu"),
            "grav_af0_planewave": (f"REGRESSION CONTROL only: exact rational plane-wave evaluation at "
                                   f"{r['grav_af0_planewave_regression_control_NOT_acceptance'][2]['samples']}
  +>  points per order; not cited as proof"),
        },
        "transformed_variables_note": ("(S_Psi,S_Psi) = 2(S_field,X_Psi) + (X_Psi,X_Psi); X is antifield-linea
  +> r, so "
            "dX_Psi/dPhi*' = dX/dPhi* and 2(S_field,X_Psi) = 2(S_field,X) (af 0, shared Noether table); "
            "the s Psi field-only part has no antifields and brackets only through (X_Psi,X_Psi)."),
        "mixed_cancellations": {f"kappa{n}_af{af}_{name}": sorted(v["per_bracket_pair"])
                                for name, d in (("invariant", r["invariant_variables"]), ("transformed", r["tr
  +> ansformed_variables"]))
                                for n in (0, 1, 2) for af, v in d[n]["by_antifield_number"].items() if v["mixe
  +> d_cancellation"]},
        "execution_pass": True,
        "claim_status": "verified" if verified else "Partial",
        "mutations_from_env": list(muts),
        "scope": ("classical BV master equation of the S_inv extended action (grav ell_2..ell_4 + massless sca
  +> lar + "
                  "minimal ghosts + nonminimal (cbar,b)) through O(kappa^2), in invariant and Psi-transformed
  +> variables "
                  "with external antifields; NOT the prepared S_D (broken square kept), NOT the insertion, NOT
  +>  quantum."),
        "note_on_pass": "p11: no unconditional 'pass' field; execution_pass (ran), claim_status and scope are
  +> separate.",
    }
    (ROOT / "certificates" / "P13_master_kappa2.json").write_text(certio.dumps(cert))
    print(json.dumps({"verified": verified, "residuals": cert["required_master_residuals"], "table": af_table,
                      "sec": r["seconds"]}, indent=1))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

107 Source: ch_source_audit.py

[listing 74/133] code/p4_classical/ch_source_audit.py

```
#!/usr/bin/env python3
```

```
"""p11 (e): mechanical ch dependency audit in TRANSFORMED variables + generated one-loop graph inventory.

Input: the transformed gauge-fixed action S_Psi(Phi, Phi*') = S_field + X(Phi, Phi*' - dPsi/dPhi)
(external antifields kept), built by source_action/gauge_fixing_map, plus the exported ell_2..ell_4.
Nothing below is typed by hand: every vertex pattern is read off S_Psi monomials.

ch pattern (exact): one c, one h; zero phi, cbar, b, antifields; ghost number 1; kappa^2; one loop; off-shell.
Projection of hbar^1 kappa^2 part of (S_Psi, Gamma_Psi) = sum_i (S_i, Gamma1^[2-i]) onto ch:
  summand 1 (A field):     dS_i/dA^R  *  dGamma1/dA*'^L      -> Gamma1 pattern P = ch - (M - A) + A*'
  summand 2 (A* antifield): dS_i/dA*'^R * dGamma1/dA^L        -> Gamma1 pattern P = ch - (M - A*') + A
for every monomial M of S_i.  P with a negative count => impossible.  Otherwise the one-loop graphs with
external pattern P at kappa^(2-i) are ENUMERATED from the S_Psi vertex inventory (cycles of vertices with
two internal legs each, lines h-h, c-cbar, phi-phi; b has no interaction vertex and is integrated out).
No graph => derived zero (with reason).  Surviving rows carry the generated graph names.

ENUMERATION LEMMA (p11; justifies max_len = 2).  (i) A connected one-loop graph is 1PI iff every internal line
  +>  lies
on its unique cycle (a line off the cycle is a bridge); hence every vertex of a one-loop 1PI graph lies on the
  +>  cycle
with exactly two internal legs, and all quadratic (kappa^0, two-field) terms are absorbed into propagators, so
  +>  the
graph is a cycle of interaction vertices.  (ii) Every retained interaction vertex has coupling order kappa^n,
  +> n >= 1
(checked mechanically: the kappa^0 inventory contains only two-field monomials).  (iii) Hence a kappa^order
contribution with order <= 2 has at most `order` <= 2 vertices: max_len = 2 is exhaustive.  (iv) The auxiliary
  +>  b
occurs only in kappa^0 two-field monomials (checked), so its Gaussian elimination produces only bilinear terms
(the de Donder kinetic term and source-bilinear terms) and no omitted interaction.  (v) Source-linear monomial
  +> s
(exactly one quantum leg) cannot sit on a cycle (two internal legs needed); attached off the cycle they create
  +>  a
bridge, i.e. a one-particle-reducible graph, so they cannot contribute extra 1PI coefficients.
Gamma_Psi <-> Gamma is the DEFINED classical pullback/interface along the antifield shift (gauge_fixing_map),
not an independently established quantum gauge-change theorem.
"""
from __future__ import annotations
import hashlib, itertools, json, os, sys, time
from collections import Counter
from pathlib import Path
import sympy as sp

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "code" / "classical"))
from graded_jets import kappa_coeff, _KEY  # noqa: E402
from bv_model import KAPPA, Psi_density  # noqa: E402
from source_action import build_X_inv, scalar_density  # noqa: E402
from gauge_fixing_map import transformed_X  # noqa: E402
import planewave_exact as PW  # noqa: E402
import conventions as C  # noqa: E402
import certio  # noqa: E402

SPECIES = ["h", "c", "cb", "b", "phi", "K", "cs", "cbs", "bs", "phis"]
QUANTUM = {"h", "c", "cb", "b", "phi"}
ANTI = {"h": "K", "c": "cs", "cb": "cbs", "b": "bs", "phi": "phis"}
LINES = {("h", "h"), ("phi", "phi"), ("c", "cb"), ("cb", "c")}
GH = {"h": 0, "c": 1, "cb": -1, "b": 0, "phi": 0, "K": -1, "cs": -2, "cbs": 0, "bs": -1, "phis": -1}
CH = Counter({"c": 1, "h": 1})


def pat(c):
    return tuple(sorted((k, v) for k, v in Counter(c).items() if v))


def show(p):
    return "".join(f"{k}{'^'+str(v) if v > 1 else ''}." for k, v in p).rstrip(".") or "1"


# ---------------- vertex inventory of S_Psi ----------------
def monomial_patterns(F, nmax=2):
    out = {}
    for n in range(nmax + 1):
        Fn = kappa_coeff(F, KAPPA, n)
        pats = set()
        for key, coef in Fn.t.items():
            odd = Counter(k[0] for k in key)
            for term in sp.Add.make_args(sp.expand(coef)):
                cnt = Counter(odd)
                for s_, e in term.as_powers_dict().items():
                    kk = _KEY.get(s_)
                    if kk is not None:
                        cnt[kk[0]] += int(e)
```

```
                pats.add(pat(cnt))
        out[n] = sorted(pats)
    return out


def S_psi_inventory(mutate=None):
    X_inv = build_X_inv()
    X_psi = transformed_X(X_inv, Psi_density(), -1)
    inv = {n: set() for n in range(3)}
    src = {}
    for name, F in (("X_Psi (antifield + s Psi)", X_psi), ("scalar", scalar_density(2))):
        for n, ps in monomial_patterns(F).items():
            for p in ps:
                inv[n].add(p)
                src.setdefault(f"{n}|{show(p)}", set()).add(name)
    for k in (2, 3, 4):
        e, h = PW.load_ell(k)
        if e != 0:
            p = pat({"h": k})
            inv[k - 2].add(p)
            src.setdefault(f"{k-2}|{show(p)}", set()).add(f"ell_{k}")
    return {n: sorted(v) for n, v in inv.items()}, {k: sorted(v) for k, v in src.items()}


# ---------------- one-loop graph enumeration ----------------
def vertex_options(inv):
    """(kappa, pattern, internal pair (x,y), external pattern) for every vertex with >=2 quantum legs, kappa>=
  +> 1."""
    opts = []
    for n, ps in inv.items():
        for p in ps:
            c = Counter(dict(p))
            q = [s for s in c if s in QUANTUM]
            if sum(c[s] for s in q) < 2:
                continue
            if n == 0:
                if sum(c.values()) <= 2:
                    continue  # kinetic / source-linear terms are propagators or external insertions
            for x in q:
                for y in q:
                    if x > y:
                        continue
                    cc = Counter(c)
                    cc[x] -= 1
                    if cc[y] <= 0:
                        continue
                    cc[y] -= 1
                    opts.append((n, p, (x, y), pat(cc)))
    return opts


def b_has_no_interaction(inv):
    return all(dict(p).get("b", 0) == 0 or n == 0 and sum(dict(p).values()) <= 2
               for n, ps in inv.items() for p in ps)


def enumerate_graphs(inv, P, order, max_len=2):
    """One-loop graphs with external pattern P at kappa^order (canonical signatures)."""
    opts = [o for o in vertex_options(inv) if 1 <= o[0] <= order]
    found = set()
    target = Counter(dict(P))
    for L in range(1, max_len + 1):
        for combo in itertools.combinations_with_replacement(range(len(opts)), L):
            vs = [opts[i] for i in combo]
            if sum(v[0] for v in vs) != order:
                continue
            ext = Counter()
            for v in vs:
                ext.update(dict(v[3]))
            if +ext != +target or any(ext[k] != target[k] for k in set(ext) | set(target)):
                continue
            # line matching: L=1 internal pair must be a line; L=2 pairs must match crosswise
            if L == 1:
                x, y = vs[0][2]
                ok = (x, y) in LINES
            else:
                (a1, b1), (a2, b2) = vs[0][2], vs[1][2]
                ok = ((a1, a2) in LINES and (b1, b2) in LINES) or ((a1, b2) in LINES and (b1, a2) in LINES)
            if ok:
                found.add(tuple(sorted((show(v[1]), f"{v[2][0]}{v[2][1]}", v[0]) for v in vs)))
    return sorted(found)
```

```
GRAPH_NAMES = {
    # (external pattern, kappa, signature) -> name ; unknown signature => TBD (acceptance fails)
    (("h^2", 2), (("h^3", "hh", 1), ("h^3", "hh", 1))): "D01_hh_bubble_cubic",
    (("h^2", 2), (("h^4", "hh", 2),)): "D02_hh_tadpole_quartic",
    (("h^2", 2), (("c.cb.h", "ccb", 1), ("c.cb.h", "ccb", 1))): "D03_hh_ghost_loop",
    (("h^2", 2), (("h.phi^2", "phiphi", 1), ("h.phi^2", "phiphi", 1))): "D11_hh_scalar_loop",
    (("h^2", 2), (("h^2.phi^2", "phiphi", 2),)): "D12_hh_scalar_tadpole",
    (("K.c", 2), (("K.c.h", "ch", 1), ("c.cb.h", "cbh", 1))): "D13_Kc_ghost_graviton_loop",
    (("h", 1), (("h^3", "hh", 1),)): "D14a_h_onepoint_grav_tadpole",
    (("h", 1), (("c.cb.h", "ccb", 1),)): "D14b_h_onepoint_ghost_tadpole",
    (("h", 1), (("h.phi^2", "phiphi", 1),)): "D14c_h_onepoint_scalar_tadpole",
}
GRAPH_STATUS = {
    "D01_hh_bubble_cubic": "listed_unevaluated",
    "D02_hh_tadpole_quartic": "vertex_available_quantum_unevaluated",
    "D03_hh_ghost_loop": "listed_unevaluated",
    "D11_hh_scalar_loop": "Constructed_kernel_under_prescription (inherited O5; not re-evaluated)",
    "D12_hh_scalar_tadpole": "Missing (D12_quantum_evaluated=false)",
    "D13_Kc_ghost_graviton_loop": "listed_unevaluated (generated from S_Psi; h* c row; coefficient not evaluat
  +> ed)",
    "D14a_h_onepoint_grav_tadpole": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12",
    "D14b_h_onepoint_ghost_tadpole": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12",
    "D14c_h_onepoint_scalar_tadpole": "listed_unevaluated; order-kappa one-point slot, DISTINCT from D12",
}


def show_sig(sig):
    return tuple((a, b, c) for a, b, c in sig)


def name_graph(P, order, sig):
    return GRAPH_NAMES.get(((show(P), order), tuple(sig)))


def audit(inv, drop_rows=()):
    rows, graphs, tbd = [], {}, []
    for i in range(3):
        for M in inv[i]:
            m = Counter(dict(M))
            for A in QUANTUM:
                for summand, X, Y in ((1, A, ANTI[A]), (2, ANTI[A], A)):
                    if m[X] == 0:
                        continue
                    R = Counter(m)
                    R[X] -= 1
                    Pc = Counter(CH)
                    Pc.subtract(R)          # what Gamma1 must supply besides the differentiated leg Y
                    if any(v < 0 for v in Pc.values()):
                        continue            # pattern impossible (S-monomial remainder exceeds ch)
                    Pc[Y] += 1
                    P = pat(Pc)
                    gs = enumerate_graphs(inv, P, 2 - i)
                    names = []
                    for g in gs:
                        nm = name_graph(P, 2 - i, g)
                        if nm is None:
                            tbd.append({"P": show(P), "order": 2 - i, "sig": g})
                            nm = "TBD"
                        names.append(nm)
                        graphs[nm] = {"external": show(P), "kappa": 2 - i, "signature": g,
                                      "status": GRAPH_STATUS.get(nm, "TBD")}
                    rid = f"(S{i},Gamma1^[{2-i}]) s{summand} {X}:{show(M)}"
                    if rid in drop_rows:
                        continue
                    if gs:
                        why = "surviving: one-loop graphs exist; coefficient not evaluated in this release"
                    elif dict(P).keys() & {"cbs", "bs"}:
                        why = ("derived zero: Gamma1 cannot depend on " + ",".join(sorted(dict(P).keys() & {'c
  +> bs', 'bs'}))
                               + " (no S_Psi vertex carries it with two quantum legs)")
                    else:
                        why = "derived zero: no one-loop graph with this external pattern at this order"
                    rows.append({"id": rid, "bracket": f"(S{i}, Gamma1^[{2-i}])", "summand": summand,
                                 "S_monomial": show(M), "differentiated": X, "Gamma1_pattern": show(P),
                                 "Gamma1_order": f"kappa^{2-i}", "graphs": sorted(set(names)),
                                 "status": "surviving_unevaluated" if gs else "derived_zero", "reason": why})
    return rows, graphs, tbd
```

```
def regenerate_and_compare(published_rows, inv):
    rows, _, _ = audit(inv)
    want = {r["id"] for r in rows}
    have = {r["id"] for r in published_rows}
    return {"missing_from_published": sorted(want - have), "extra_in_published": sorted(have - want),
            "complete": want == have}


def enumeration_lemma(inv):
    opts = vertex_options(inv)
    k0_multi = [show(p) for p in inv[0] if sum(dict(p).values()) > 2]
    min_k = min(o[0] for o in opts if o[0] >= 1)
    b_pats = [(n, show(p)) for n, ps in inv.items() for p in ps if dict(p).get("b", 0)]
    return {
        "statement": __doc__.split("ENUMERATION LEMMA")[1].split("Gamma_Psi <->")[0].strip(),
        "mechanical_checks": {
            "kappa0_monomials_all_two_field": not k0_multi, "kappa0_multi_field_monomials": k0_multi,
            "min_interaction_vertex_order": min_k, "all_interaction_vertices_kappa_ge_1": min_k >= 1,
            "max_vertices_at_kappa2": 2 // min_k, "max_len_used": 2, "max_len_exhaustive": 2 // min_k <= 2,
            "b_monomials": b_pats, "b_only_in_kappa0_bilinears": b_has_no_interaction(inv),
            "source_linear_excluded_reason": "vertex_options requires >= 2 quantum legs; one-leg insertions ar
  +> e bridges (1PR)",
        },
        "holds": (not k0_multi) and min_k >= 1 and b_has_no_interaction(inv),
        "interface": ("Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi) is a defined classical pullback/i
  +> nterface, "
                      "not an independently established quantum gauge-change theorem"),
    }


def env_drop_rows(inv):
    muts = [m for m in os.environ.get("P13_MUTATIONS", "").split(",") if m]
    if "drop_hstar_c_row" not in muts:
        return ()
    rows, _, _ = audit(inv)
    return tuple(r["id"] for r in rows if r["status"] == "surviving_unevaluated" and r["differentiated"] == "h
  +> "
                 and r["Gamma1_pattern"] == "K.c")


def main(drop_rows=(), write=True):
    t0 = time.time()
    inv, src = S_psi_inventory()
    drop_rows = tuple(drop_rows) + env_drop_rows(inv)
    rows, graphs, tbd = audit(inv, drop_rows=drop_rows)
    lemma = enumeration_lemma(inv)
    surviving = [r for r in rows if r["status"] == "surviving_unevaluated"]
    hstar_c = [r for r in surviving if r["differentiated"] == "h" and r["Gamma1_pattern"] == "K.c"]
    onept = [r for r in surviving if r["Gamma1_pattern"] == "h"]
    cmp = regenerate_and_compare(rows, inv)
    phi_rows = [r for r in rows if "phi" in r["differentiated"]]
    k2_ghost_vertex = any(dict(p).get("c", 0) and dict(p).get("cb", 0) for p in inv[2])
    checks = {
        "names_TBD_empty": not tbd,
        "regenerated_rows_match_published": cmp["complete"],
        "hstar_c_row_present": bool(hstar_c),
        "onepoint_slot_distinct_from_D12": bool(onept) and all("D12_hh_scalar_tadpole" not in r["graphs"] for
  +> r in onept),
        "b_has_no_interaction_vertex": b_has_no_interaction(inv),
        "no_kappa2_ghost_vertex": not k2_ghost_vertex,
        "cbar_star_rows_derived_zero": all(r["status"] == "derived_zero" for r in rows if "cbs" in r["Gamma1_p
  +> attern"]),
        "b_star_rows_derived_zero": all(r["status"] == "derived_zero" for r in rows if "bs" in r["Gamma1_patte
  +> rn"]),
        "S2_rows_derived_zero": all(r["status"] == "derived_zero" for r in rows if r["bracket"].startswith("(S
  +> 2")),
        "enumeration_lemma_holds": lemma["holds"],
    }
    complete = all(checks.values())
    cert = {
        "revision": C.REVISION, "gate": "ch_projection_defined / ch_dependency_inventory_complete",
        "variables": "transformed (Phi, Phi*'), S_Psi with external antifields; Psi = " + C.PSI,
        "ch_pattern": {"c": 1, "h": 1, "phi": 0, "cbar": 0, "b": 0, "antifields": 0, "ghost_number": 1,
                       "coupling": "kappa^2", "loop": "one", "domain": "off-shell, compact smooth smearings"},
        "method": __doc__.strip().splitlines()[0],
        "vertex_inventory": {f"kappa{n}": [show(p) for p in ps] for n, ps in inv.items()},
        "vertex_sources": src,
        "rows_total": len(rows), "rows_surviving": len(surviving),
        "rows_derived_zero": len(rows) - len(surviving),
        "surviving_rows": surviving,
        "hstar_c_row": hstar_c,
```

```
        "onepoint_kappa_slot": onept,
        "phi_rows": {"count": len(phi_rows), "all_derived_zero_or_impossible": all(r["status"] == "derived_zer
  +> o" for r in phi_rows),
                     "parity_argument": "phi -> -phi is a symmetry of S_Psi (phi appears in even powers in eve
  +> ry non-antifield "
                                        "monomial and phis.phi pairs); ch is phi-even"},
        "checks": checks, "enumeration_lemma": lemma, "dropped_rows_mutation": list(drop_rows),
        "regeneration": cmp,
        "names_TBD": tbd,
        "ch_projection_defined": True,
        "ch_dependency_inventory_complete": complete,
        "values": ("no surviving coefficient is evaluated in this release (no loop evaluated here); D11 carrie
  +> s an "
                   "inherited O5 kernel constructed under prescription, not re-evaluated in this release"),
        "execution_pass": True, "claim_status": "earned" if complete else "Partial",
        "scope": "pattern-level dependency inventory of the projected kappa^2 one-loop source identity; not ev
  +> aluation",
        "seconds": round(time.time() - t0, 1),
    }
    gl = {"revision": C.REVISION, "generated_from": "S_Psi vertex inventory (ch_source_audit.enumerate_graphs)
  +> ",
          "graphs": graphs, "names_TBD": [t for t in tbd],
          "D02_quantum_evaluated": False, "D12_quantum_evaluated": False, "D12_status": "Missing",
          "no_loop_evaluated": True}
    rl = {"revision": C.REVISION, "rows": rows}
    if write:
        (ROOT / "certificates" / "P13_ch_source_audit.json").write_text(certio.dumps(cert))
        (ROOT / "ledgers" / "P13_ch_dependency_rows.json").write_text(certio.dumps(rl))
        (ROOT / "ledgers" / "P13_graph_inventory.json").write_text(certio.dumps(gl))
    return cert, rows, graphs


if __name__ == "__main__":
    cert, rows, graphs = main()
    print(json.dumps({"complete": cert["ch_dependency_inventory_complete"], "checks": cert["checks"],
                      "rows": cert["rows_total"], "surviving": [(r["id"], r["Gamma1_pattern"], r["graphs"]) fo
  +> r r in cert["surviving_rows"]],
                      "graphs": list(graphs), "inv": cert["vertex_inventory"], "TBD": cert["names_TBD"], "sec"
  +> : cert["seconds"]}, indent=1, default=str))
```

108 Source: release.py (frozen p13 classical gate; forked verbatim by release_p14)

[listing 75/133] code/classical/release.py

```
#!/usr/bin/env python3
"""XVI-P p13 acceptance / release gate (assembled from the P13 certificates only).

p12 additions: residual-record invariant (Q_identically_zero == all(Q_terms == 0)) re-checked on EVERY certifi
  +> cate;
per-order acceptance Q_terms == [0,0,0,0] and Q_identically_zero at kappa0..kappa2 recomputed from the quartic
  +>  AND
master records (never trusted from a stored Boolean); cross-certificate agreement (same export hashes + transf
  +> ormation
parameters => identical residual records).  Any failure => release_claim_accepted = false => publication rejec
  +> ted.

execution_pass         : the pipeline ran and every hard stop holds (package consistency).
claim_status           : the classical-readiness claim (five-flag conjunction), separate from execution.
scope                  : what the claim is about.
release_claim_accepted : execution_pass AND classical_readiness AND every acceptance prerequisite is
                         deterministic and true AND hash consistency (export file == quartic cert == master in
  +> puts).
p13 additions: milestone definitions/status (conventions.MILESTONES; P.4 = classical readiness, NOT coupled on
  +> e-loop
restoration), the milestone ledger ledgers/P13_milestones.json, and the reproducibility flags
  author_rebuild_passed       : reproducibility/author_rebuild_report.json passed AND was produced from exactl
  +> y the
                                current source tree (source_tree_sha256 equal) -- otherwise false;
  independent_rebuild_passed  : "pending" until reproducibility/independent_rebuild_report.json exists (writte
  +> n by a
                                separate clean environment); then the same binding rule.
These are package evidence (XVI-0 conv:package), not mathematical certificates, and do not enter
release_claim_accepted.  Certificate input hashes are content hashes with wall-clock timings removed.

Publication rule (p11): the final closure artifacts (certificates/P13_release.json, certificates/P13_closure_r
  +> elease.json)
```

```
are written ONLY if release_claim_accepted and classical_readiness are both true.  Otherwise any stale closure
  +>  artifact is
deleted, certificates/P13_release_rejected.json is written, RELEASE_STATUS.json records the rejection, and the
  +>  exit is 1.
"""
from __future__ import annotations
import hashlib, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "tools"))
import conventions as C  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402

import inherited  # noqa: E402


def j(name):
    return json.loads((ROOT / "certificates" / name).read_text())


def rebuild_flag(report_rel, pending_if_absent):
    f = ROOT / report_rel
    cur = SM.tree_sha(ROOT)
    if not f.exists():
        return ("pending" if pending_if_absent else False), {"report": report_rel, "present": False,
                                                             "current_source_tree_sha256": cur}
    try:
        rep = json.loads(f.read_text())
    except Exception as exc:  # noqa: BLE001
        return False, {"report": report_rel, "present": True, "error": str(exc)}
    same = rep.get("source_tree_sha256") == cur
    ok = rep.get("pass") is True and same
    return ok, {"report": report_rel, "present": True, "report_pass": rep.get("pass"),
                "report_source_tree_sha256": rep.get("source_tree_sha256"), "current_source_tree_sha256": cur,
                "bound_to_current_source": same,
                # the report's source_zip_sha256 is NOT copied here: the zip is a manifest-listed file and its
  +>  hash must
                # not appear inside the PDFs (this record is typeset); build.py checks it against dist/ at the
  +>  manifest gate
                "report_records_source_zip_sha256": "source_zip_sha256" in rep,
                "inherited_mode": rep.get("inherited_mode"), "environment": rep.get("environment")}


def milestones(ch, gi_names_tbd, conj, accepted, independent):
    p3 = bool(ch["ch_projection_defined"] and ch["ch_dependency_inventory_complete"] and ch["enumeration_lemma
  +> "]["holds"]
              and ch["checks"]["regenerated_rows_match_published"] and gi_names_tbd == [])
    p4 = bool(conj and accepted)
    st = {"P.3": "complete (declared ch target)" if p3 else "incomplete",
          "P.4": "earned (classical readiness)" if p4 else "not_earned",
          "P.5_entry": "next (not started; p13 does no P.5 work)",
          "P.5_calculation": "open", "P.5_identity": "open"}
    return {"definitions": C.MILESTONES, "order": list(C.MILESTONE_ORDER), "status": st,
            "P3_complete": p3, "P4_classical_readiness_earned": p4,
            "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified
  +>  not earned)",
            "P3_P4_frozen_as_P5_inputs": independent is True,
            "frozen_rule": "P.3/P.4 become frozen P.5 inputs only after independent_rebuild_passed is true",
            "history": C.MILESTONE_HISTORY,
            "XVI_O_unaffected": "XVI-P is unregistered package evidence; XVI-O prop:kappa2-master stays Partia
  +> l"}


def _main():
    try:
        pin = inherited.sha(inherited.resolve("XVI-M.tex"))
    except inherited.PinError:
        pin = "unavailable"
    pin_ok = pin == C.XVI_M_PIN
    q, br, sa = j("P13_quartic_verified.json"), j("P13_brst_derivation.json"), j("P13_source_action.json")
    gf, ms, ch = j("P13_gauge_fixing_map.json"), j("P13_master_kappa2.json"), j("P13_ch_source_audit.json")
    bi = j("P13_boundary_identity.json")
    flags = {
        "quartic_generator_verified": bool(q["quartic_generator_verified"]),
        "ch_projection_defined": bool(ch["ch_projection_defined"]),
        "ch_dependency_inventory_complete": bool(ch["ch_dependency_inventory_complete"]),
        "gauge_fixing_map_constructed": bool(gf["gauge_fixing_map_constructed"]),
```

```
      "required_classical_BV_components_verified": bool(ms["required_classical_BV_components_verified"]),
  }
  residuals = list(ms["required_master_residuals"])
  # ---- hash consistency: export files == quartic certificate == master inputs ----
  import sympy as sp
  file_sha = {}
  for k in (2, 3, 4):
      txt = (ROOT / "exports" / f"ell{k}_generating_polynomial.srepr").read_text().strip()
      file_sha[k] = hashlib.sha256(str(sp.sympify(txt)).encode()).hexdigest()
  hashes = {f"ell{k}": {"export_file": file_sha[k], "quartic_cert": q.get(f"ell{k}_sha256"),
                        "master_input": ms["identification_with_exports"]["ell2_ell3_ell4"][str(k)]["sha256"
+> ]
                        if str(k) in ms["identification_with_exports"]["ell2_ell3_ell4"]
                        else ms["identification_with_exports"]["ell2_ell3_ell4"][k]["sha256"]} for k in (2,
+> 3, 4)}
  hash_consistent = all(len({v["export_file"], v["quartic_cert"], v["master_input"]}) == 1 for v in hashes.v
+> alues())
  # ---- deterministic acceptance prerequisites ----
  gdet = ms["details"]["grav_af0_deterministic"]
  hess = gf["propagators"]["graviton"]
  det_prereq = {
      "C1_inverse_10comp": q["acceptance_controls_deterministic"]["C1"],
      "C2_det_tracelog_10comp": q["acceptance_controls_deterministic"]["C2"],
      "C3_boundary_identity_generic_and_conformal": bool(bi["C3_proved"]) and q["acceptance_controls_determi
+> nistic"]["C3"],
      "C4_export_equals_independent_evaluator": q["acceptance_controls_deterministic"]["C4"],
      "C5_cubic_regression_Euler": q["acceptance_controls_deterministic"]["C5"],
      "C6_quadratic_regression_Euler": q["acceptance_controls_deterministic"]["C6"],
      "C7_grav_noether_k0_k1_k2_on_exports": q["acceptance_controls_deterministic"]["C7"],
      "quartic_mutations_all_rejected": bool(q["mutations_all_rejected"]),
      "master_grav_noether_deterministic_k0_k1_k2": all(gdet[f"kappa{n}"]["Q_identically_zero"] for n in (0,
+>  1, 2)),
      "master_engine_parts_exact": not [r for r in residuals],
      "graviton_hessian_generic_symbolic_k": bool(hess.get("schur_equals_k2P")) and bool(hess.get("propagato
+> r_hh_equals_inv_k2P"))
                                             and "generic" in str(hess.get("momentum", "")),
      "canonicity_all_23_antifield_components": bool(gf["canonicity"]["pass"]) and gf["canonicity"]["pairs_t
+> ested"] == 299,
      "ch_enumeration_lemma_mechanical": bool(ch["enumeration_lemma"]["holds"]),
  }
  # ---- p12: residual-record invariants, per-order acceptance, cross-certificate agreement ----
  allc = {"quartic": q, "brst": br, "source": sa, "gauge": gf, "master": ms, "ch": ch, "boundary": bi}
  inv_bad = {k: certio.invariant_violations(v) for k, v in allc.items()}
  inv_bad = {k: v for k, v in inv_bad.items() if v}
  q_c7 = q["controls"]["C7_noether_exact"]
  acc_q = certio.accepted_orders(q_c7)
  acc_m = certio.accepted_orders(gdet)
  cross = certio.cross_check(q_c7, q["controls"].get("C7_parameters"), gdet, ms["details"].get("grav_af0_par
+> ameters"))
  stored_vs_recomputed = (q["acceptance_controls_deterministic"]["C7"] == all(acc_q.values()))
  det_prereq.update({
      "residual_record_invariants_all_certificates": not inv_bad,
      "quartic_C7_Q_terms_zero_and_flag_true_all_orders": all(acc_q.values()),
      "master_grav_Q_terms_zero_and_flag_true_all_orders": all(acc_m.values()),
      "cross_certificate_quartic_vs_master_agree": cross["agree"],
      "stored_C7_flag_equals_recomputation": stored_vs_recomputed,
  })
  det_prereq["C7_grav_noether_k0_k1_k2_on_exports"] = det_prereq["C7_grav_noether_k0_k1_k2_on_exports"] and
+> all(acc_q.values())
  det_prereq["master_grav_noether_deterministic_k0_k1_k2"] = det_prereq["master_grav_noether_deterministic_k
+> 0_k1_k2"] and all(acc_m.values())
  deterministic_ok = all(det_prereq.values())
  failed_prereq = [k for k, v in det_prereq.items() if not v] + ([] if hash_consistent else ["hash_consisten
+> cy"])
  flags["quartic_generator_verified"] = (flags["quartic_generator_verified"] and hash_consistent and bool(bi
+> ["C3_proved"])
                                         and all(acc_q.values()) and "quartic" not in inv_bad)
  flags["required_classical_BV_components_verified"] = (flags["required_classical_BV_components_verified"]
                                                        and hash_consistent and not residuals and all(acc_m.
+> values())
                                                        and cross["agree"] and "master" not in inv_bad)
  conj = all(flags.values())
  hard = dict(C.HARD_STOPS)
  hard_ok = (hard["quantum_inputs_evaluated"] == "not_earned" and hard["projected_identity_verified"] == "no
+> t_earned"
             and hard["D02_quantum_evaluated"] is False and hard["D12_quantum_evaluated"] is False
             and hard["D12_status"] == "Missing" and hard["symmetry_restored"] is False
             and hard["O1_closure"] == "NOT_CLOSED" and hard["XV_B_opened"] is False
             and hard["Route_B_discharged"] is False and hard["loop_evaluations_performed"] is False
             and hard["u_R_c_delta_added"] is False
             and q["D02_quantum_evaluated"] is False)
```

```
  execution_pass = bool(pin_ok and hard_ok and all(x["execution_pass"] for x in (q, br, sa, gf, ms, ch)))
  statuses = {k: ("earned" if v else "Partial") for k, v in flags.items()}
  release_claim_accepted = bool(execution_pass and conj and deterministic_ok and hash_consistent)
  statuses.update({
      "classical_readiness_for_ch_identity": "earned" if conj else "not_earned",
      "quantum_inputs_evaluated": "not_earned",
      "projected_identity_verified": "not_earned",
      "propagators_match_transformed_Hessians": bool(gf["propagators_match_transformed_Hessians"]),
      "source_identity_relation_defined": bool(gf["source_identity_relation_defined"]),
      "source_identity_relation": ("defined classical pullback/interface (related, not identical): "
                                   "Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi); not a quantum the
+> orem"),
      "D02_vertex_available": bool(q["D02_vertex_available"]),
      "D02_quantum_evaluated": False, "D12_quantum_evaluated": False, "D12_status": "Missing",
  })
  author_ok, author_rec = rebuild_flag("reproducibility/author_rebuild_report.json", pending_if_absent=False
+> )
  indep_ok, indep_rec = rebuild_flag("reproducibility/independent_rebuild_report.json", pending_if_absent=Tr
+> ue)
  gi = json.loads((ROOT / "ledgers" / "P13_graph_inventory.json").read_text())
  ms_block = milestones(ch, gi.get("names_TBD"), conj, release_claim_accepted, indep_ok)
  surviving = [{"row": r["id"], "Gamma1_pattern": r["Gamma1_pattern"], "graphs": r["graphs"],
                "value": "not evaluated in this release (D11 inherited from O5, not re-evaluated)"}
               for r in ch["surviving_rows"]]
  cert = {
      "landmark": "XVI-P", "revision": C.REVISION, "date": C.DATE, "deliverable": "evidence_and_closure",
      "execution_pass": execution_pass,
      "claim_status": "classical_readiness_for_ch_identity: " + ("earned" if conj else "not_earned"),
      "scope": ("classical readiness for the fixed one-loop kappa^2 ch identity on the S_inv baseline (decla
+> red "
                "conventions, affine metric, a=1/2, linear de Donder Psi, massless scalar); classical only;
+> "
                "NOT quantum inputs, NOT the projected identity, NOT the prepared S_D, NOT the insertion"),
      "execution_pass_means": "pipeline executed in dependency order; every hard stop holds; pin verified. "
                              "Does NOT mean any classical or quantum claim.",
      "release_claim_accepted": release_claim_accepted,
      "release_claim_accepted_definition": ("execution_pass AND classical_readiness_for_ch_identity AND all
+> "
                                            "deterministic acceptance prerequisites AND hash consistency"),
      "deterministic_prerequisites": det_prereq, "failed_prerequisites": failed_prereq,
      "hash_consistency": {"consistent": hash_consistent, "ell": hashes},
      "residual_records": {"invariant_violations": inv_bad,
                           "quartic_C7_per_order": {o: {"residual_counts": q_c7[o]["Q_terms"], "zero_flag":
+> q_c7[o]["Q_identically_zero"],
                                                        "accepted": acc_q[o]} for o in certio.ORDERS},
                           "master_grav_per_order": {o: {"residual_counts": gdet[o]["Q_terms"], "zero_flag":
+>  gdet[o]["Q_identically_zero"],
                                                         "accepted": acc_m[o]} for o in certio.ORDERS},
                           "cross_certificate": cross},
      "sampled_checks_role": "regression controls only (never acceptance): numeric evaluator samples, plane-
+> wave "
                             "cubic/Noether samples",
      "flags": flags, "classical_readiness_for_ch_identity": conj,
      "classical_readiness_conjunction_definition": " AND ".join(flags),
      "statuses": statuses,
      "required_master_residuals": residuals,
      "closed_blocker": {"id": "complete_af_ghost_O_kappa2_on_D_h",
                         "closed": not residuals and flags["required_classical_BV_components_verified"],
                         "by": "P13_master_kappa2.json (transformed and invariant variables, by antifield nu
+> mber; "
                               "grav af0 deterministic)"},
      "boundary_identity": {"statement": C.BOUNDARY_IDENTITY, "C3_proved": bi["C3_proved"],
                            "orientation_correction": bi.get("orientation_correction"),
                            "integrated_action_unaffected": bi.get("integrated_action_unaffected")},
      "s2_per_sector": br["s2_per_sector"], "s2_reason": br["s2_reason"],
      "master_table": ms["table_by_kappa_and_antifield_number"],
      "mixed_cancellations": ms["mixed_cancellations"],
      "derived_signs": {"sigma_A": sa["sigma"], "antifield_shift_eps": gf["derived_eps"],
                        "S_Psi": "S + s Psi (mod d)", "odd_Psi_left_equals_right": gf["left_equals_right_for
+> _odd_Psi"]},
      "ch_surviving_rows_unevaluated": surviving,
      "graph_inventory": "ledgers/P13_graph_inventory.json",
      "source_identity_interface": ("Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi): defined classica
+> l "
                                    "pullback/interface, NOT an independently established quantum gauge-chan
+> ge theorem"),
      "remaining_blockers_outside_classical_scope": [
          {"id": "Gamma1_one_loop_coefficients (D01,D02,D03,D11,D12,D13,D14a-c)",
           "status": "not evaluated in this release (D11: inherited O5 kernel under prescription, not re-eva
+> luated)",
           "blocks": "quantum_inputs_evaluated"},
```

```
            {"id": "D12_quantum_evaluation", "status": "Missing", "blocks": "quantum_inputs_evaluated"},
            {"id": "D02_quantum_tadpole_evaluation", "status": "not_performed (vertex available)",
             "blocks": "quantum_inputs_evaluated"},
        ],
        "hard_stops": hard, "hard_stops_hold": hard_ok,
        "milestones": ms_block,
        "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
        "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
  +> ource.py",
                            "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                            "author": author_rec, "independent": indep_rec,
                            "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"},
        "XVI_P_registered": False,
        "symmetry_restored": False, "O1_closure": "NOT_CLOSED",
        "insertion_status": "Frozen_negative_obstruction_preserved", "XV_B_opened": False, "Route_B_discharged
  +> ": False,
        "P5_opened": False,
        "obstruction": "XVI-N thm:massless-obstruction respected: no completion of the prepared/inserted funct
  +> ional is "
                       "claimed; the closure is the S_inv classical master equation only. No c delta added to
  +> u^R_mu "
                       "(XVI-J prop:extension).",
        "not_premises": C.NOT_PREMISES,
        "baseline": C.BASELINE, "ghost_normalization_a": "1/2",
        "XVI_M_sha256": pin, "XVI_M_pin_ok": pin_ok,
        "XVI_A_to_O_edited": False, "XVI_O_o70_sync": "read-only",
        "inputs_content_sha256_ignoring_timings": {n: certio.file_content_sha(ROOT / "certificates" / n) for n
  +>  in
                    ("P13_quartic_verified.json", "P13_boundary_identity.json", "P13_brst_derivation.json",
                     "P13_source_action.json", "P13_gauge_fixing_map.json", "P13_master_kappa2.json",
                     "P13_ch_source_audit.json")},
    }
    cdir = ROOT / "certificates"
    closure = [cdir / "P13_release.json", cdir / "P13_closure_release.json"]
    publish = release_claim_accepted and conj
    cert["publication"] = "accepted" if publish else "REJECTED"
    txt = certio.dumps(cert)
    (ROOT / "RELEASE_STATUS.json").write_text(txt)
    (cdir / "RELEASE_STATUS.json").write_text(txt)
    certio.write(ROOT / "ledgers" / "P13_milestones.json", {
        "landmark": "XVI-P", "revision": C.REVISION, "ledger": "milestones (authoritative definitions + status
  +> )",
        **ms_block, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
        "hard_stops": hard, "dossier": "certificates/P13_dossier.json"})
    if publish:
        (cdir / "P13_release_rejected.json").unlink(missing_ok=True)
        closure[0].write_text(txt)
        closure[1].write_text(certio.dumps({
            "landmark": "XVI-P", "revision": C.REVISION, "closed_blocker": cert["closed_blocker"],
            "release_claim_accepted": True, "classical_readiness_for_ch_identity": True, "flags": flags,
            "scope": cert["scope"], "quantum_inputs_evaluated": "not_earned", "symmetry_restored": False,
            "O1_closure": "NOT_CLOSED", "release_certificate": "certificates/P13_release.json",
            "hashes_of_release_files": "external manifest only (RELEASE_MANIFEST_EXTERNAL.txt)"}))
    else:
        for f in closure:
            f.unlink(missing_ok=True)
        (cdir / "P13_release_rejected.json").write_text(certio.dumps({
            "landmark": "XVI-P", "revision": C.REVISION, "publication": "REJECTED",
            "release_claim_accepted": release_claim_accepted, "classical_readiness_for_ch_identity": conj,
            "execution_pass": execution_pass, "flags": flags, "failed_prerequisites": failed_prereq,
            "required_master_residuals": residuals, "hash_consistency": hash_consistent}))
    print(json.dumps({"execution_pass": execution_pass, "flags": flags, "conj": conj, "residuals": residuals,
                      "release_claim_accepted": release_claim_accepted, "failed_prerequisites": failed_prereq,
                      "publication": cert["publication"], "pin_ok": pin_ok}, indent=1))
    return 0 if publish else 1


def reject_on_error(exc):
    """Fail closed: an unreadable/inconsistent certificate set is a rejection, never a crash that leaves artif
  +> acts."""
    cdir = ROOT / "certificates"
    for f in (cdir / "P13_release.json", cdir / "P13_closure_release.json"):
        f.unlink(missing_ok=True)
    rec = {"landmark": "XVI-P", "revision": C.REVISION, "publication": "REJECTED", "release_claim_accepted": F
  +> alse,
           "classical_readiness_for_ch_identity": False, "error": f"{type(exc).__name__}: {exc}",
           "failed_prerequisites": ["release_evaluation_error"]}
    txt = certio.dumps(rec)
    (cdir / "P13_release_rejected.json").write_text(txt)
    (ROOT / "RELEASE_STATUS.json").write_text(txt)
    (cdir / "RELEASE_STATUS.json").write_text(txt)
```

```
    print(json.dumps({"publication": "REJECTED", "error": rec["error"]}))
    return 1


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001
        return reject_on_error(exc)


if __name__ == "__main__":
    raise SystemExit(main())
```

109 Source: dossier.py (frozen p13 dossier library)

[listing 76/133] code/classical/dossier.py

```
#!/usr/bin/env python3
"""p13: P.3 / P.4 completion dossier (authoritative; NOT a release-history retelling).

Each row links one claim to its executable evidence: file, function, certificate key(s) and test.  dossier.py
  +> RESOLVES
every link mechanically on the current tree:
  * the function exists in the file (AST),
  * the certificate key exists and holds the expected value,
  * the test exists in the test file (AST; pass/fail is recorded separately in P13_test_results.json),
and writes
  certificates/P13_dossier.json          (machine-readable; all_links_resolved must be true for publication)
  manuscript/body/P13-dossier-table.tex  (the typeset table, generated from the same rows)
Milestone definitions come from conventions.MILESTONES; milestone status from RELEASE_STATUS.json.
The three quantum input slots are NAMED DEPENDENCIES (rows of the ch audit), never evaluated coefficients.
"""
from __future__ import annotations
import ast, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
import conventions as C  # noqa: E402
import certio  # noqa: E402

T = "tests/test_p13.py"
QA, BI, BR, SA = ("certificates/P13_quartic_verified.json", "certificates/P13_boundary_identity.json",
                  "certificates/P13_brst_derivation.json", "certificates/P13_source_action.json")
GF, MS, CH, RS = ("certificates/P13_gauge_fixing_map.json", "certificates/P13_master_kappa2.json",
                  "certificates/P13_ch_source_audit.json", "RELEASE_STATUS.json")
GI = "ledgers/P13_graph_inventory.json"
F_CH, F_Q, F_GN = "code/p4_classical/ch_source_audit.py", "code/classical/quartic.py", "code/classical/grav_no
  +> ether.py"
F_BI, F_BR, F_SA = "code/classical/boundary_identity.py", "code/classical/brst_from_geometry.py", "code/classi
  +> cal/source_action.py"
F_GF, F_MS, F_RL = "code/classical/gauge_fixing_map.py", "code/classical/master_kappa2.py", "code/classical/re
  +> lease.py"
F_CI, F_BD = "code/classical/certio.py", "tools/build.py"
ALL_TRUE, NONEMPTY = "ALL_TRUE", "NONEMPTY"


def R(rid, claim, file, funcs, checks, tests):
    return {"id": rid, "claim": claim, "file": file, "functions": list(funcs),
            "checks": [{"source": s, "key": k, "expect": e} for s, k, e in checks], "tests": list(tests)}


CHAINS = [
    ("P.3", "projection -> dependency rows -> graphs", [
        R("P3.1", "ch output projection fixed (c^1 h^1, ghost number 1, kappa^2, one loop, transformed variabl
  +> es)",
          F_CH, ["main"], [(CH, "/ch_projection_defined", True), (CH, "/ch_pattern/ghost_number", 1)],
          ["test_ch_rows_and_quantum_input_slots"]),
        R("P3.2", "vertex inventory generated from S_Psi (not copied)", F_CH, ["S_psi_inventory", "vertex_opti
  +> ons"],
          [(CH, "/vertex_inventory", NONEMPTY), (CH, "/checks/b_has_no_interaction_vertex", True)],
          ["test_ch_rows_and_quantum_input_slots"]),
        R("P3.3", "dependency rows exhaustive: regenerated rows equal the published rows", F_CH,
          ["audit", "regenerate_and_compare"],
          [(CH, "/checks/regenerated_rows_match_published", True), (CH, "/regeneration/complete", True)],
          ["test_e2e_dropped_hstar_c_row"]),
```

```
      R("P3.4", "6 rows: 3 surviving, 3 derived zero (S2, cbar*, b* rows)", F_CH, ["audit"],
        [(CH, "/rows_total", 6), (CH, "/rows_surviving", 3), (CH, "/rows_derived_zero", 3)],
        ["test_ch_rows_and_quantum_input_slots"]),
      R("P3.5", "enumeration lemma (justifies max_len = 2) checked mechanically", F_CH, ["enumeration_lemma"
+> ],
        [(CH, "/enumeration_lemma/holds", True)], ["test_enumeration_lemma"]),
      R("P3.6", "graph topologies generated and named; no unnamed graph", F_CH, ["enumerate_graphs", "name_g
+> raph"],
        [(GI, "/names_TBD", []), (GI, "/graphs", NONEMPTY), (CH, "/checks/names_TBD_empty", True)],
        ["test_ch_rows_and_quantum_input_slots"]),
      R("P3.7", "P.3 flags", F_RL, ["_main"],
        [(RS, "/flags/ch_projection_defined", True), (RS, "/flags/ch_dependency_inventory_complete", True)],
        ["test_milestones_consistent"]),
  ]),
  ("P.4", "classical coefficient -> export -> verification", [
      R("P4.A1", "C3 boundary identity sqrt(-g)R - L_GG = -d_mu B^mu on generic jets", F_BI, ["generic_jet_p
+> roof"],
        [(BI, "/C3_proved", True)], ["test_C3_boundary_identity_generic_and_conformal"]),
      R("P4.A2", "ell_2, ell_3, ell_4 built and exported; export hash == quartic cert == master input", F_Q,
        ["build_and_export"], [(RS, "/hash_consistency/consistent", True), (QA, "/ell4_sha256", NONEMPTY)],
        ["test_stale_ell4_hash_blocks_master"]),
      R("P4.A3", "C1 inverse, C2 det/trace-log (10 symbolic components)", F_Q, ["inverse_control", "det_trac
+> elog_generic"],
        [(QA, "/acceptance_controls_deterministic/C1", True), (QA, "/acceptance_controls_deterministic/C2",
+> True)],
        ["test_quartic_acceptance_all_deterministic"]),
      R("P4.A4", "C4 export == independent evaluator (exact polynomial identity)", F_Q, ["export_identity"],
        [(QA, "/acceptance_controls_deterministic/C4", True)], ["test_quartic_acceptance_all_deterministic"]
+> ),
      R("P4.A5", "C5/C6 ell_3 = XII-C, ell_2 = XI-A Fierz-Pauli modulo divergences (pinned XVI-N grav_densit
+> y)", F_Q,
        ["regression_deterministic", "xvin_symbolic"],
        [(QA, "/acceptance_controls_deterministic/C5", True), (QA, "/acceptance_controls_deterministic/C6",
+> True)],
        ["test_quartic_acceptance_all_deterministic"]),
      R("P4.A6", "C7 gravitational Noether identities on the exports, kappa^0..kappa^2", F_GN, ["noether"],
        [(QA, "/controls/C7_accepted_orders", ALL_TRUE)], ["test_grav_noether_exact_and_mutation"]),
      R("P4.A7", "quartic mutations rejected; quartic flag", F_Q, ["main"],
        [(QA, "/mutations_all_rejected", True), (RS, "/flags/quartic_generator_verified", True)],
        ["test_e2e_corrupted_ell4_coefficient"]),
  ]),
  ("P.4", "transformations -> source terms", [
      R("P4.B1", "ghost law derived from s^2 = 0 (unique lambda = 1)", F_BR, ["solve_ghost_law"],
        [(BR, "/ghost_law_derivation/lam_solutions", ["1"])], ["test_brst_and_source_terms_derived"]),
      R("P4.B2", "s_2 derived zero in every sector; s nilpotent at all orders", F_BR, ["per_sector_expansion
+> ", "nilpotency"],
        [(BR, "/s2_per_sector", "ALL_EQ:derived_zero"), (BR, "/nilpotency_exact_all_orders", True)],
        ["test_brst_and_source_terms_derived"]),
      R("P4.B3", "sigma_A derived from the generator rule (unique); antifield-linear, no kappa^2 X terms", F
+> _SA,
        ["derive_sigma", "build_X_inv"],
        [(SA, "/sigma_unique", True), (SA, "/X_kappa2_terms", 0), (SA, "/antifield_linear", True)],
        ["test_brst_and_source_terms_derived", "test_e2e_flipped_odd_sign", "test_e2e_omitted_cstar_term"]),
  ]),
  ("P.4", "gauge map -> Hessians -> source convention", [
      R("P4.C1", "antifield shift orientation derived (eps = -1); S_Psi = S + s Psi mod d", F_GF, ["build",
+> "shift_map"],
        [(GF, "/derived_eps", -1), (GF, "/S_Psi_equals_S_plus_sPsi_mod_d", True)], ["test_e2e_altered_gauge_
+> shift"]),
      R("P4.C2", "canonicity of the map on all 299 (anti)field pairs", F_GF, ["canonicity"],
        [(GF, "/canonicity/pass", True), (GF, "/canonicity/pairs_tested", 299)],
        ["test_gauge_hessian_generic_and_full_canonicity"]),
      R("P4.C3", "graviton Hessian at generic symbolic k: Schur complement k^2 P, propagator (k^2 P)^-1", F_
+> GF,
        ["hessian_check"],
        [(GF, "/propagators/graviton/schur_equals_k2P", True), (GF, "/propagators/graviton/propagator_hh_equ
+> als_inv_k2P", True)],
        ["test_gauge_hessian_generic_and_full_canonicity"]),
      R("P4.C4", "ghost (i cbar Box c) and scalar (-Box) kinetic terms; propagators match transformed Hessia
+> ns", F_GF,
        ["ghost_kinetic_check"],
        [(GF, "/propagators/ghost/match", True), (GF, "/propagators/scalar/match", True),
         (GF, "/propagators_match_transformed_Hessians", True)], ["test_gauge_hessian_generic_and_full_canon
+> icity"]),
      R("P4.C5", "source convention Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi): classical pullbac
+> k, NOT a quantum theorem",
        F_GF, ["build"],
        [(GF, "/source_identity_relation_defined", True),
         (GF, "/source_identity_relation/status", "CONTAINS:NOT an independently established quantum")],
        ["test_gauge_hessian_generic_and_full_canonicity"]),
      R("P4.C6", "gauge-map flag", F_RL, ["_main"], [(RS, "/flags/gauge_fixing_map_constructed", True)],
```

```
          ["test_milestones_consistent"]),
    ]),
    ("P.4", "master component -> residual record -> acceptance gate", [
        *[R(f"P4.D{n + 1}", f"master equation at kappa^{n}: every antifield-number component; grav af0 record;
  +>  gate",
           F_MS, ["run", "XX_by_order", "grav_deterministic"],
           [(MS, f"/table_by_kappa_and_antifield_number/kappa{n}", ALL_TRUE),
            (MS, f"/details/grav_af0_deterministic/kappa{n}/Q_terms", [0, 0, 0, 0]),
            (MS, f"/details/grav_af0_deterministic/kappa{n}/Q_identically_zero", True),
            (RS, f"/residual_records/master_grav_per_order/kappa{n}/accepted", True)],
           ["test_residual_records_regenerated_and_agree"]) for n in (0, 1, 2)],
        R("P4.D4", "no required master residual; no unconditional pass field", F_MS, ["master_residuals"],
          [(MS, "/required_master_residuals", [])], ["test_master_no_unconditional_pass_and_deterministic"]),
        R("P4.D5", "count/flag invariant on every record; quartic and master records agree", F_CI,
          ["invariant_violations", "cross_check"],
          [(RS, "/deterministic_prerequisites/residual_record_invariants_all_certificates", True),
           (RS, "/residual_records/cross_certificate/agree", True)],
          ["test_certificate_mutation_rejects_publication", "test_certio_rejects_inconsistent_record_before_se
  +> rialization"]),
        R("P4.D6", "acceptance gate: release_claim_accepted and classical readiness (P.4)", F_RL, ["_main"],
          [(RS, "/release_claim_accepted", True), (RS, "/classical_readiness_for_ch_identity", True),
           (RS, "/flags/required_classical_BV_components_verified", True)],
          ["test_release_accepted_and_hard_stops"]),
        R("P4.D7", "publication gate re-invoked before every pdf/zip/manifest stage, also on resumed builds",
  +> F_BD,
          ["publication_gate", "pdf_readback"], [], ["test_resumed_stage_build_blocked"]),
    ]),
]


def ptr(obj, key):
    cur = obj
    for part in [p for p in key.split("/") if p != ""]:
        if isinstance(cur, list):
            cur = cur[int(part)]
        else:
            cur = cur[part]
    return cur


def matches(val, exp):
    if exp == ALL_TRUE:
        return isinstance(val, dict) and bool(val) and all(v is True for v in val.values())
    if exp == NONEMPTY:
        return bool(val)
    if isinstance(exp, str) and exp.startswith("ALL_EQ:"):
        return isinstance(val, dict) and bool(val) and all(v == exp[7:] for v in val.values())
    if isinstance(exp, str) and exp.startswith("CONTAINS:"):
        return isinstance(val, str) and exp[9:] in val
    return val == exp and type(val) is type(exp)


def defs(rel):
    return {n.name for n in ast.walk(ast.parse((ROOT / rel).read_text())) if isinstance(n, ast.FunctionDef)}


def short(v, n=48):
    s = json.dumps(v)
    return s if len(s) <= n else s[:n - 3] + "..."


def resolve_row(row, cache):
    out = dict(row)
    fdefs = defs(row["file"]) if (ROOT / row["file"]).exists() else set()
    out["functions_found"] = {f: f in fdefs for f in row["functions"]}
    tdefs = cache.setdefault(T, defs(T))
    out["tests_found"] = {t: t in tdefs for t in row["tests"]}
    res = []
    for c in row["checks"]:
        try:
            doc = cache.setdefault(c["source"], json.loads((ROOT / c["source"]).read_text()))
            val = ptr(doc, c["key"])
            ok = matches(val, c["expect"])
            res.append({**c, "value": short(val), "ok": ok})
        except Exception as exc:  # noqa: BLE001
            res.append({**c, "value": f"UNRESOLVED ({type(exc).__name__})", "ok": False})
    out["checks"] = res
    out["resolved"] = all(out["functions_found"].values()) and all(out["tests_found"].values()) and \
        all(c["ok"] for c in res)
    return out
```

```
def slots(cache):
    ch = cache.setdefault(CH, json.loads((ROOT / CH).read_text()))
    gi = cache.setdefault(GI, json.loads((ROOT / GI).read_text()))
    rows = {r["id"]: r for r in ch["surviving_rows"]}
    out = []
    for s in C.QUANTUM_INPUT_SLOTS:
        r = rows.get(s["row"], {})
        graphs = r.get("graphs", [])
        ok = (r.get("Gamma1_pattern") == s["Gamma1_pattern"] and r.get("status") == "surviving_unevaluated"
              and [g.split("_")[0] for g in graphs] == s["graphs"] and all(g in gi["graphs"] for g in graphs))
        out.append({**s, "graph_names": graphs, "status": "named dependency; NOT an evaluated coefficient",
                    "graph_status": {g: gi["graphs"].get(g, {}).get("status") for g in graphs},
                    "evidence": {"file": F_CH, "function": "audit", "certificate": CH,
                                 "key": f"/surviving_rows/{list(rows).index(s['row']) if s['row'] in rows else
  +>  '?'}",
                                 "test": "test_ch_rows_and_quantum_input_slots"},
                    "resolved": ok})
    return out


def build():
    cache = {}
    rs = json.loads((ROOT / RS).read_text())
    chains = [{"milestone": m, "chain": name, "rows": [resolve_row(r, cache) for r in rows]} for m, name, rows
  +>  in CHAINS]
    sl = slots(cache)
    resolved = all(r["resolved"] for ch in chains for r in ch["rows"]) and all(s["resolved"] for s in sl)
    d = {
        "landmark": "XVI-P", "revision": C.REVISION, "document": "P.3/P.4 completion dossier",
        "status_of_this_document": ("package evidence for XVI-P (unregistered; XVI-O conv:package): an archive
  +> , a listing "
                                    "and a manifest are not a series certificate. Does not update XVI-O "
                                    "(prop:kappa2-master stays Partial there; symmetry_restored false; O.1 NOT
  +> _CLOSED)."),
        "milestone_definitions": C.MILESTONES, "milestone_status": rs.get("milestones", {}),
        "chains": chains, "quantum_input_slots": sl,
        "hard_stops": rs.get("hard_stops", {}),
        "reproducibility": {k: rs.get(k) for k in ("author_rebuild_passed", "independent_rebuild_passed")},
        "P3_P4_frozen_as_P5_inputs": rs.get("milestones", {}).get("P3_P4_frozen_as_P5_inputs"),
        "all_links_resolved": resolved,
        "n_rows": sum(len(ch["rows"]) for ch in chains), "n_slots": len(sl),
    }
    return d


# ---------------- LaTeX ----------------
def tex(s):
    rep = {"\\": r"\textbackslash{}", "_": r"\_", "^": r"\^{}", "{": r"\{", "}": r"\}", "&": r"\&", "%": r"\%"
  +> ,
           "#": r"\#", "$": r"\$", "~": r"\~{}", "<": r"\textless{}", ">": r"\textgreater{}"}
    return "".join(rep.get(ch, ch) for ch in str(s))


def brk(s):
    """escaped text with permitted line breaks after '/', '_' and '.' (long certificate keys in narrow columns
  +> )"""
    return tex(s).replace("/", r"/\allowbreak{}").replace(r"\_", r"\_\allowbreak{}").replace(".", r".\allowbre
  +> ak{}")


def disp(v):
    try:
        x = json.loads(v)
        return x[:16] + "..." if isinstance(x, str) and len(x) > 24 else short(x, 34)
    except Exception:  # noqa: BLE001            (already truncated by short())
        return v if len(v) <= 34 else v[:18] + "..."


def table(d):
    L = [r"{\scriptsize\setlength{\tabcolsep}{2.5pt}",
         r"\begin{longtable}{@{}p{0.9cm}p{4.1cm}p{3.5cm}p{5.0cm}p{3.0cm}@{}}",
         r"\toprule id & claim & file :: function & certificate key = value & test \\ \midrule \endhead"]
    for ch in d["chains"]:
        L.append(r"\multicolumn{5}{@{}l}{\textbf{" + tex(ch["milestone"]) + ": " + tex(ch["chain"]) + r"}} \\"
  +> )
        for r in ch["rows"]:
            fn = r"\texttt{" + brk(r["file"].split("/")[-1]) + "} :: " + ", ".join(
                r"\texttt{" + brk(f) + "}" for f in r["functions"])
            ks = r"\newline ".join(r"\texttt{" + brk(c["source"].split("/")[-1].replace("P13_", "").replace(".
  +> json", "")
```

```
                                                    + c["key"]) + "} = " + (r"\emph{all entries true}" if c["
  +> expect"] == ALL_TRUE and c["ok"] else brk(disp(c["value"])))
                                    for c in r["checks"]) or "(build-time gate)"
            ts = r"\newline ".join(r"\texttt{" + brk(t.replace("test_", "")) + "}" for t in r["tests"])
            L.append(f"{tex(r['id'])} & {tex(r['claim'])} & {fn} & {ks} & {ts} \\\\")
        L.append(r"\midrule")
    L.append(r"\multicolumn{5}{@{}l}{\textbf{Quantum input slots (named dependencies, NOT evaluated coefficien
  +> ts)}} \\")
    for s in d["quantum_input_slots"]:
        L.append(f"slot & {tex(s['slot'])} ($\\Gamma_1$ pattern \\texttt{{{tex(s['Gamma1_pattern'])}}}) & "
                 r"\texttt{ch\_source\_audit.py} :: \texttt{audit} & \texttt{ch\_source\_audit" + brk(s["evide
  +> nce"]["key"])
                 + "} = " + tex(", ".join(s["graphs"])) + r" & \texttt{" + brk("ch_rows_and_quantum_input_slot
  +> s") + r"} \\")
    rp = d["reproducibility"]
    L.append(r"\midrule \multicolumn{5}{@{}p{16.5cm}@{}}{\textbf{Reproducibility flags (package evidence, read
  +>  from "
             r"RELEASE\_STATUS at build time):} \texttt{author\_rebuild\_passed} = " + tex(json.dumps(rp.get("
  +> author_rebuild_passed")))
             + r"; \texttt{independent\_rebuild\_passed} = " + tex(json.dumps(rp.get("independent_rebuild_pass
  +> ed")))
             + r"; \texttt{P3\_P4\_frozen\_as\_P5\_inputs} = " + tex(json.dumps(d.get("P3_P4_frozen_as_P5_inpu
  +> ts"))) + r".} \\")
    L += [r"\bottomrule", r"\end{longtable}}"]
    return "\n".join(L) + "\n"


def main():
    d = build()
    certio.write(ROOT / "certificates" / "P13_dossier.json", d)
    (ROOT / "manuscript" / "body").mkdir(parents=True, exist_ok=True)
    (ROOT / "manuscript" / "body" / "P13-dossier-table.tex").write_text(table(d))
    bad = [r["id"] for ch in d["chains"] for r in ch["rows"] if not r["resolved"]] + \
          [s["slot"] for s in d["quantum_input_slots"] if not s["resolved"]]
    print(json.dumps({"all_links_resolved": d["all_links_resolved"], "rows": d["n_rows"], "slots": d["n_slots"
  +> ],
                      "unresolved": bad}))
    return 0 if d["all_links_resolved"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

110 Source: inherited.py (pinned inherited inputs)

[listing 77/133] code/classical/inherited.py

```
#!/usr/bin/env python3
"""p13: inherited (read-only) external inputs, each pinned by sha256.

Every file this package reads from OUTSIDE its own tree is listed here.  resolve(name) returns a path whose sh
  +> a256
equals the pin, or raises.  Search order (XVIP_INHERITED = auto | workspace | vendored, default auto):
  workspace : the sibling landmark tree on the author's machine (e.g. /workspace/XVI-M.tex);
  vendored  : the byte-identical copy shipped inside this package under provenance/ (so a clean environment th
  +> at has
              only XVI-P-source-only.zip can rebuild).
A pin mismatch is never tolerated in either location.  Certificates record only the logical name and the sha25
  +> 6,
never the absolute path, so certificate contents do not depend on where the rebuild runs.
"""
from __future__ import annotations
import hashlib
import os
from pathlib import Path

ROOT = Path(__file__).resolve().parents[2]

PINS = {
    "XVI-M.tex": {
        "sha256": "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea",
        "workspace": "/workspace/XVI-M.tex",
        "vendored": "provenance/XVI-M.tex",
        "used_for": "landmark pin (read-only; XVI-M supplies M.4 selected-graph data, not premises of P.3/P.4)
  +> ",
    },
    "XVI-N-r10/code/prep/grav_density.py": {
        "sha256": "855ab9e2124acf66f4440b2f1745470acf137caf77dc7bd538d13b7c878ea8e0",
```

```
        "workspace": "/workspace/XVI-N-r10/code/prep/grav_density.py",
        "vendored": "provenance/inherited/XVI-N-r10/code/prep/grav_density.py",
        "used_for": "C5/C6 regression references: XII-C ell_3 (ell3_and_curvature) and XI-A Fierz-Pauli (FP_de
  +> nsity)",
    },
}


class PinError(RuntimeError):
    pass


def sha(p: Path) -> str:
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def candidates(name, mode=None):
    mode = mode or os.environ.get("XVIP_INHERITED", "auto")
    pin = PINS[name]
    ws, vd = Path(pin["workspace"]), ROOT / pin["vendored"]
    return {"workspace": [ws], "vendored": [vd], "auto": [ws, vd]}[mode]


def resolve(name, mode=None):
    """Path of a pinned inherited input (sha256 verified)."""
    pin = PINS[name]
    tried = []
    for p in candidates(name, mode):
        if p.exists():
            h = sha(p)
            if h == pin["sha256"]:
                return p
            tried.append(f"{p}: sha256 {h} != pin")
        else:
            tried.append(f"{p}: absent")
    raise PinError(f"inherited input {name} not available with pinned sha256 {pin['sha256']}: " + "; ".join(tr
  +> ied))


def verify_all(mode=None):
    """{name: {pin, ok, location, sha256}} without absolute paths in the location field."""
    out = {}
    for name, pin in PINS.items():
        try:
            p = resolve(name, mode)
            loc = "vendored" if Path(p).resolve() == (ROOT / pin["vendored"]).resolve() else "workspace"
            out[name] = {"sha256": pin["sha256"], "ok": True, "location": loc}
        except PinError as exc:
            out[name] = {"sha256": pin["sha256"], "ok": False, "error": str(exc)}
    # the vendored copies themselves must also match (the source-only zip ships them)
    for name, pin in PINS.items():
        vd = ROOT / pin["vendored"]
        out[name]["vendored_copy_matches_pin"] = vd.exists() and sha(vd) == pin["sha256"]
    return out


if __name__ == "__main__":
    import json
    r = verify_all()
    print(json.dumps(r, indent=1))
    raise SystemExit(0 if all(v["ok"] and v["vendored_copy_matches_pin"] for v in r.values()) else 1)
```

111 Source: tools/source_manifest.py (source set, source-only zip)

[listing 78/133] tools/source_manifest.py

```
#!/usr/bin/env python3
"""p14/p15/p16: the SOURCE set of XVI-P (single definition, used by the source-only zip, the rebuild entrypoin
  +> t, the release
gate and the publication gate).

source = hand-written code/tools/tests, hand-written manuscript sources, provenance (pins + vendored inherited
  +>  inputs),
       archival READMEs and archival notes typeset by the appendix, the reproducibility contract files, and
  +> the
       INHERITED DATA inputs listed (with sha256 pins) in reproducibility/inherited_data_pins.json.
```

```
Everything else in a built tree is generated (certificates P13_*/P14_*, RELEASE_STATUS, exports ell*/V4_grav/
ell4_conventions/P14_*, ledgers P13_*/P14_*, regenerated O5 matter-loop records, listings, appendix body, doss
  +> ier and
summary tables, PDFs, zips, manifest, build/, caches; p15: certificates/ledgers P15_*, the P15 results table)
  +> and is
NOT in XVI-P-p15-source-only.zip.
p19: the p18 archive records are source as well (archival/p18/README.md, reference_values_p18.json, the p18 au
  +> thor
and independent rebuild reports and p18_source_files.json, the per-file base of the p19 change log).
p20: likewise the p19 archive records (archival/p19/README.md, reference_values_p19.json, the p19 author and
independent rebuild reports and p19_source_files.json, the per-file base of the p20 change log).
p21: likewise the p20 archive records (archival/p20/README.md, reference_values_p20.json, the p20 author and
independent rebuild reports and p20_source_files.json, the per-file base of the p21 change log).
p22: likewise the p21 archive records (archival/p21/README.md, reference_values_p21.json, the p21 author and
independent rebuild reports and p21_source_files.json, the per-file base of the p22 change log).
p18: the p17 archive records are source as well (archival/p17/README.md, reference_values_p17.json, the p17 au
  +> thor
and independent rebuild reports and p17_source_files.json, the per-file base of the p18 change log).
p17: the p16 archive records are source as well (archival/p16/README.md, reference_values_p16.json, the p16 au
  +> thor
and independent rebuild reports and p16_source_files.json, the per-file base of the p17 change log).
p16: the p15 archive records are source as well (archival/p15/README.md, reference_values_p15.json and the p15
  +>  author
and independent rebuild reports, byte-identical copies, pinned in p16_conventions.py); generated p16 outputs
(certificates/ledgers P16_*, the P16 results table) are NOT source.  The live reproducibility/*_p15.json files
  +>  are NOT
p16 source and are never written by p16.
p15: the p14r archive records are source (archival/p14r/README.md, reference_values_p14r.json and the p14r aut
  +> hor and
independent rebuild reports, byte-identical copies; the independent report is pinned in p15_conventions.py); t
  +> he live
reproducibility/*_p14r.json files are NOT p15 source and are never written by p15.
p14: the p13 freeze inputs are source (archival/p13/reference_values_p13.json, archival/p13/frozen_p13_inputs.
  +> json,
archival/p13/reproducibility/*.json incl. the archived independent report); the reviewer's live
reproducibility/independent_rebuild_report.json and the p13 reproducibility/reference_values.json are NOT p14
  +> source
(they are left untouched in the author tree; their archived copies are byte-identical).

Usage:  python3 tools/source_manifest.py [--list] [--tree-sha] [--zip OUT.zip] [--check-source-only]
"""
from __future__ import annotations
import hashlib, json, sys, zipfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
PINS_FILE = "reproducibility/inherited_data_pins.json"
REFERENCE_FILE = "reproducibility/reference_values_p23.json"
ZIP_NAME = "XVI-P-p23-source-only.zip"
ZIP_DATE = (2026, 10, 9, 0, 0, 0)

TOP_FILES = ("README.md", "REBUILD_INSTRUCTIONS.md")
GLOBS = ("code/**/*.py", "tools/**/*.py", "tests/**/*.py",
         "manuscript/preamble.tex", "manuscript/body/XVI-P-body.tex", "manuscript/notes/*.tex",
         "notes/*.tex", "provenance/**/*", "archival/*/README.md", "archival/p02_p09/notes/*.tex",
         "archival/p13/reference_values_p13.json", "archival/p13/frozen_p13_inputs.json",
         "archival/p13/reproducibility/independent_rebuild_report_p13.json",
         "archival/p13/reproducibility/author_rebuild_report_p13.json",
         "archival/p14r/reference_values_p14r.json",
         "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
         "archival/p14r/reproducibility/author_rebuild_report_p14r.json",
         "archival/p15/reference_values_p15.json",
         "archival/p15/reproducibility/independent_rebuild_report_p15.json",
         "archival/p15/reproducibility/author_rebuild_report_p15.json",
         "archival/p15/p15_source_files.json",
         "archival/p16/reference_values_p16.json",
         "archival/p16/reproducibility/independent_rebuild_report_p16.json",
         "archival/p16/reproducibility/author_rebuild_report_p16.json",
         "archival/p16/p16_source_files.json",
         "archival/p17/reference_values_p17.json",
         "archival/p17/reproducibility/independent_rebuild_report_p17.json",
         "archival/p17/reproducibility/author_rebuild_report_p17.json",
         "archival/p17/p17_source_files.json",
         "archival/p18/reference_values_p18.json",
         "archival/p18/reproducibility/independent_rebuild_report_p18.json",
         "archival/p18/reproducibility/author_rebuild_report_p18.json",
         "archival/p18/p18_source_files.json",
         "archival/p19/reference_values_p19.json",
         "archival/p19/reproducibility/independent_rebuild_report_p19.json",
         "archival/p19/reproducibility/author_rebuild_report_p19.json",
         "archival/p19/p19_source_files.json",
```

```
         "archival/p20/reference_values_p20.json",
         "archival/p20/reproducibility/independent_rebuild_report_p20.json",
         "archival/p20/reproducibility/author_rebuild_report_p20.json",
         "archival/p20/p20_source_files.json",
         "archival/p21/reference_values_p21.json",
         "archival/p21/reproducibility/independent_rebuild_report_p21.json",
         "archival/p21/reproducibility/author_rebuild_report_p21.json",
         "archival/p21/p21_source_files.json",
         "archival/p22/reference_values_p22.json",
         "archival/p22/reproducibility/independent_rebuild_report_p22.json",
         "archival/p22/reproducibility/author_rebuild_report_p22.json",
         "archival/p22/p22_source_files.json",
         "archival/p22/certificates/P22_evanescent.json",
         PINS_FILE, REFERENCE_FILE)
NEVER = ("__pycache__", ".pytest_cache")
NEVER_SUFFIX = (".pyc", ".pkl", ".aux", ".log", ".toc", ".out", ".pdf", ".zip")


def inherited_data(root=ROOT):
    f = Path(root) / PINS_FILE
    return json.loads(f.read_text())["files"] if f.exists() else {}


def source_files(root=ROOT):
    root = Path(root)
    out = set()
    for t in TOP_FILES:
        if (root / t).is_file():
            out.add(t)
    for g in GLOBS:
        for p in root.glob(g):
            if p.is_file():
                out.add(p.relative_to(root).as_posix())
    for rel in inherited_data(root):
        if (root / rel).is_file():
            out.add(rel)
    return sorted(r for r in out if not any(n in r.split("/") for n in NEVER) and not r.endswith(NEVER_SUFFIX)
  +> )


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def tree_sha(root=ROOT, exclude=()):
    """sha256 over the sorted 'relpath<TAB>sha256' lines of the source set (optionally excluding some paths)."
  +> ""
    root = Path(root)
    lines = [f"{r}\t{sha(root / r)}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def tree_sha_without_reference(root=ROOT):
    """Hash of everything except the reference-values file (which records this hash, so it cannot include itse
  +> lf)."""
    return tree_sha(root, exclude=(REFERENCE_FILE,))


def non_source_files(root=ROOT):
    """Files present in a tree that are not source (i.e. generated artifacts or caches)."""
    root = Path(root)
    src = set(source_files(root))
    extra = []
    for p in sorted(root.rglob("*")):
        if p.is_file():
            rel = p.relative_to(root).as_posix()
            if rel not in src:
                extra.append(rel)
    return extra


def make_zip(out, root=ROOT):
    """Deterministic zip (sorted entries, fixed date and permissions) of the source set under XVI-P/."""
    root, out = Path(root), Path(out)
    out.parent.mkdir(parents=True, exist_ok=True)
    out.unlink(missing_ok=True)
    files = source_files(root)
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for rel in files:
            info = zipfile.ZipInfo("XVI-P/" + rel, date_time=ZIP_DATE)
            info.compress_type = zipfile.ZIP_DEFLATED
```

```
            info.external_attr = 0o644 << 16
            z.writestr(info, (root / rel).read_bytes())
    return {"zip": out.name, "files": len(files), "bytes": out.stat().st_size, "sha256": sha(out)}


def zip_matches_tree(zp, root=ROOT):
    """[] if the zip holds exactly the current source set byte-for-byte; else a list of differences."""
    root = Path(root)
    bad = []
    try:
        zf = zipfile.ZipFile(zp)
    except (zipfile.BadZipFile, FileNotFoundError) as exc:
        return [f"{Path(zp).name}: {type(exc).__name__}"]
    with zf:
        names = {n.split("/", 1)[1]: n for n in zf.namelist()}
        src = set(source_files(root))
        bad += [f"missing_in_zip:{r}" for r in sorted(src - set(names))]
        bad += [f"not_source:{r}" for r in sorted(set(names) - src)]
        bad += [f"differs:{r}" for r in sorted(src & set(names)) if zf.read(names[r]) != (root / r).read_bytes
  +> ()]
    return bad


XVIO_TREE = "provenance/XVI-O_o70_source_tree.json"


def check_xvio_provenance(root=ROOT):
    """The XVI-O (o70-sync) code and notes carried verbatim in this package (code/matter_loop/*.py, notes/*.te
  +> x)
    must match the sha256 list recorded in provenance/XVI-O_o70_source_tree.json."""
    root = Path(root)
    tree = json.loads((root / XVIO_TREE).read_text())
    out = {}
    for f in tree["files"]:
        p = root / f["path"]
        out[f["path"]] = {"pin": f["sha256"], "ok": p.is_file() and sha(p) == f["sha256"]}
    return out


def check_inherited_data(root=ROOT):
    """Every inherited data input is present and matches its sha256 pin."""
    root = Path(root)
    out = {}
    for rel, pin in inherited_data(root).items():
        p = root / rel
        out[rel] = {"pin": pin["sha256"], "present": p.is_file(),
                    "ok": p.is_file() and sha(p) == pin["sha256"]}
    return out


if __name__ == "__main__":
    a = sys.argv[1:]
    if "--list" in a:
        print("\n".join(source_files()))
    if "--tree-sha" in a:
        print(json.dumps({"source_tree_sha256": tree_sha(), "without_reference": tree_sha_without_reference(),
                          "files": len(source_files())}))
    if "--check-source-only" in a:
        extra = non_source_files()
        print(json.dumps({"non_source_files": extra[:50], "count": len(extra)}))
        raise SystemExit(0 if not extra else 1)
    if "--zip" in a:
        print(json.dumps(make_zip(a[a.index("--zip") + 1])))
```

112 Source: tools/rebuild_from_source.py (rebuild entrypoint)

[listing 79/133] tools/rebuild_from_source.py

```
#!/usr/bin/env python3
"""XVI-P p23 REBUILD ENTRYPOINT (the reproducibility contract; see REBUILD_INSTRUCTIONS.md).

Run from the root of an unpacked XVI-P-p23-source-only.zip (a tree containing SOURCE ONLY):

    python3 tools/rebuild_from_source.py [--inherited auto|workspace|vendored] [--report PATH]

It
  1. records the environment (Python, sympy, mpmath, pytest, pdfTeX, pdftotext) and compares it with the refer
  +> ence;
```

```
  2. refuses to run unless the tree is source-only: no generated certificates, exports, ledgers, listings, PDF
  +> s,
     zips, build/ or __pycache__ caches (tools/source_manifest.py defines the source set);
  3. verifies the inherited dependencies and pins: XVI-M (sha 41aa1099...739ea), the XVI-N grav_density.py rea
  +> d
     by C5/C6, the verbatim XVI-O (o70-sync) code and notes (provenance/XVI-O_o70_source_tree.json) and every
     inherited data input listed in reproducibility/inherited_data_pins.json;
  4. runs the full pipeline tools/build.py (fresh: no cache) with sync redirected INSIDE the copy
     (build/rebuild_sync) and every Python file access traced (tools/read_trace/sitecustomize.py);
     this regenerates the polynomial exports (ell2/ell3/ell4) and all residual records, the inherited O5
     matter-loop records, the P.5 entry contract (symbolic amplitude rules and integrands, counterterm domain,
     quantum object, prescription + D11 matching), the p15 calculation (integral library, Batch A, D13), the p
  +> 15
     release gate, the p16 calculation (E2 decoupling proof, Batch C D01/D03), the p16 release gate, the p17 c
  +> alculation (D01/D03
     evanescent poles vs the ch slot), the p17 release gate, the p18 calculation (M2 discharge, D11, full hh s
  +> lot)
     and the p18 release gate, the p19 calculation (projected ch-identity defect, counterterm solve, p20: full
     closure) and the p19 layer gate, the p20 D11 pin (p21: absolute pin derivation) and the p20 layer gate, t
  +> he p21 layer gate, the p22 evanescent
     calculation (EV-M2, EV-CT) and the p22 layer gate (incl. the live runtime d-guard probe), the p23 power c
  +> ounting / 0.1 scope
     (power_counting.py) and the p23 release gate (live poly-refusal probe, EV value check), runs the baseline
  +> , certificate/gate-mutation
     and resumed-build tests, typesets the PDFs, zips and writes the manifest;
  5. checks: release accepted; test groups baseline / certificate_mutation / resumed_build all passed; the PDF
     readback equals the machine-readable residual records AND (p16, F4) the D13 / Batch A / Batch C / E2 / (p
  +> 17) hh-slot / (p18) M2, D11, full-slot value
     blocks, and every page footer carries the single version label; the external manifest verifies; the rebui
  +> lt
     source-only zip equals this tree; no file outside the tree was read except a pinned inherited input;
  6. compares with the author's reference values (reproducibility/reference_values_p23.json): export hashes, c
  +> ertificate
     content hashes ignoring wall-clock timings (and the run-specific reproducibility block), Q/W counts, flag
  +> s,
     milestone status and test counts, and writes a comparison report (default build/rebuild_report.json).
Exit code 0 iff every step passed and every compared value is equal.

Author mode (once, on the author's fully built tree): --write-reference writes reproducibility/reference_value
  +> s_p23.json.
(The p13 reproducibility/reference_values.json is NOT written or read by p14; its archived copy
archival/p13/reference_values_p13.json is pinned and read by the p13 freeze check F3.)
This is package evidence (XVI-O conv:package), not a mathematical certificate.
"""
from __future__ import annotations
import sys
sys.dont_write_bytecode = True          # the source-only check must not be defeated by our own caches
import json, os, platform, re, subprocess, tempfile, time  # noqa: E402
from pathlib import Path  # noqa: E402

ROOT = Path(__file__).resolve().parents[1]
PY = sys.executable
sys.path.insert(0, str(ROOT / "tools"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import source_manifest as SM  # noqa: E402
import certio  # noqa: E402
import inherited  # noqa: E402
import conventions as C  # noqa: E402   (frozen p13 classical conventions)
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
import p14_conventions as PC  # noqa: E402
sys.path.insert(0, str(ROOT / "code" / "p5_calc"))
import p15_conventions as Q15  # noqa: E402
import p23_conventions as QC  # noqa: E402
import p22_conventions as Q22  # noqa: E402
import p21_conventions as Q21  # noqa: E402
import p20_conventions as Q20  # noqa: E402
import p19_conventions as Q19  # noqa: E402
import p18_conventions as Q18  # noqa: E402
import p17_conventions as Q17  # noqa: E402
import p16_conventions as Q16  # noqa: E402

XVI_M_PIN = "41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea"
TRACE = ROOT / "build" / "rebuild_trace.log"
SYNC = ROOT / "build" / "rebuild_sync"
CERTS = ("P13_boundary_identity", "P13_quartic_verified", "P13_brst_derivation", "P13_source_action",
         "P13_gauge_fixing_map", "P13_ch_source_audit", "P13_master_kappa2", "P14_release", "P14_closure_relea
  +> se",
         "P14_dossier", "P14_quantum_object", "P14_prescription_D11_matching", "P14_counterterm_domain",
         "P14_amplitude_rules", "P14_dred_rules", "P15_integral_library", "P15_batchA", "P15_D13", "P15_releas
  +> e",
         "P16_E2_evanescent", "P16_batchC", "P16_release", "P17_hh_slot", "P17_release", "P18_D11_M2", "P18_D1
```

```
  +> 1", "P18_hh_slot", "P18_release",
        "P19_defect", "P19_release", "P20_D11_pin", "P20_release", "P21_release", "P22_evanescent", "P22_rele
  +> ase", "P23_O1_scope", "P23_release")
LEDGERS = ("P13_ch_dependency_rows", "P13_graph_inventory", "P14_milestones", "P15_milestones", "P16_milestone
  +> s", "P17_milestones", "P18_milestones", "P19_milestones", "P20_milestones", "P21_milestones", "P22_milest
  +> ones", "P23_milestones")
# p13 freeze inputs read by the p14 pipeline: pinned by sha256 in code/p5_entry/p14_conventions.py
PINNED_ARCHIVAL = (PC.P13_REFERENCE_FILE, PC.P13_INDEPENDENT_REPORT, PC.P13_FROZEN_LIST) + tuple(PC.P13_ARCHIV
  +> ED_RECORDS)
ARCHIVAL_PINS = {PC.P13_REFERENCE_FILE: PC.P13_REFERENCE_VALUES_SHA256, PC.P13_INDEPENDENT_REPORT: PC.P13_INDE
  +> PENDENT_REPORT_SHA256,
                 PC.P13_FROZEN_LIST: PC.P13_FROZEN_LIST_SHA256, **PC.P13_ARCHIVED_RECORDS,
                 "archival/p14r/reproducibility/independent_rebuild_report_p14r.json": Q15.P14R_INDEPENDENT_RE
  +> PORT_SHA256,
                 **Q15.P14R_ARCHIVED_RECORDS,
                 "archival/p15/reproducibility/independent_rebuild_report_p15.json": Q16.P15_INDEPENDENT_REPOR
  +> T_SHA256,
                 **Q16.P15_ARCHIVED_RECORDS, **Q17.P16_ARCHIVED_RECORDS, **Q18.P17_ARCHIVED_RECORDS,
                 **Q19.P18_ARCHIVED_RECORDS, **Q20.P19_ARCHIVED_RECORDS, **Q21.P20_ARCHIVED_RECORDS,
                 **Q22.P21_ARCHIVED_RECORDS, **QC.P22_ARCHIVED_RECORDS}
PINNED_ARCHIVAL = PINNED_ARCHIVAL + ("archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
                                     *Q15.P14R_ARCHIVED_RECORDS,
                                     "archival/p15/reproducibility/independent_rebuild_report_p15.json",
                                     *Q16.P15_ARCHIVED_RECORDS, *Q17.P16_ARCHIVED_RECORDS, *Q18.P17_ARCHIVED_R
  +> ECORDS,
                                     *Q19.P18_ARCHIVED_RECORDS, *Q20.P19_ARCHIVED_RECORDS, *Q21.P20_ARCHIVED_R
  +> ECORDS,
                                     *Q22.P21_ARCHIVED_RECORDS, *QC.P22_ARCHIVED_RECORDS)
# run-specific keys (not mathematical content): the rebuild flags/records differ between the author tree (whic
  +> h holds
# the author's rebuild report) and any fresh copy.  Wall-clock 'seconds' are removed by certio.strip_volatile.
RUN_SPECIFIC = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed",
                "live_report_identical_to_archived",
                # p14 freeze records that depend on files which are (correctly) absent from a source-only copy
  +> :
                # the archived p13 zip and the reviewer's live p13 report in the author tree
                "live_report_present", "live_report_identical_to_archived", "archived_zip_sha256_ok",
                "list_equals_archived_zip")


def versions():
    import sympy, mpmath
    try:
        import pytest
        pv = pytest.__version__
    except Exception:  # noqa: BLE001
        pv = None

    def first(cmd):
        try:
            r = subprocess.run(cmd, capture_output=True, text=True)
            return ((r.stdout or r.stderr).strip().splitlines() or [""])[0]
        except FileNotFoundError:
            return None
    return {"python": platform.python_version(), "sympy": sympy.__version__, "mpmath": mpmath.__version__,
            "pytest": pv, "pdftex": first(["pdflatex", "--version"]), "pdftotext": first(["pdftotext", "-v"]),
            "platform": platform.platform()}


# ---------------- reference values (identical code path for author and rebuild) ----------------
def jload(rel, root=ROOT):
    return json.loads((Path(root) / rel).read_text())


def reference_values(root=ROOT):
    root = Path(root)
    q = jload("certificates/P13_quartic_verified.json", root)
    m = jload("certificates/P13_master_kappa2.json", root)
    rs = jload("RELEASE_STATUS.json", root)
    tr = jload(f"certificates/{QC.CERT_PREFIX}_test_results.json", root)
    cl = rs["classical"]
    rec = lambda r: {k: r[k] for k in ("W_terms", "Q_terms", "Q_identically_zero", "s0_piece_alone_Q_terms")}
  +>  # noqa: E731
    return {
        "source": {"tree_sha256_without_reference": SM.tree_sha_without_reference(root),
                   "n_source_files": len(SM.source_files(root))},
        "inherited_pins": {**{k: v["sha256"] for k, v in inherited.PINS.items()},
                           **{k: v["sha256"] for k, v in SM.inherited_data(root).items()},
                           "XVI-O_o70_source_tree": json.loads((root / SM.XVIO_TREE).read_text())["sha256"]},
        "exports": {**{f"ell{k}": {"file_sha256": SM.sha(root / "exports" / f"ell{k}_generating_polynomial.sre
  +> pr"),
```

```
                                "polynomial_sha256": q[f"ell{k}_sha256"],
                                "terms": q["terms_reproducibility"][str(k)]} for k in (2, 3, 4)},
                    "V4_grav.json": SM.sha(root / "exports" / "V4_grav.json"),
                    "ell4_conventions.json": SM.sha(root / "exports" / "ell4_conventions.json"),
                    "P14_vertex_tensors.json": SM.sha(root / "exports" / "P14_vertex_tensors.json"),
                    "P14_integrands": {f.name: SM.sha(f) for f in sorted((root / "exports" / "P14_integrands")
+> .glob("*.json"))}},
      "certificate_content_sha256_ignoring_timings": {
          **{c: certio.file_content_sha(root / "certificates" / f"{c}.json", RUN_SPECIFIC) for c in CERTS},
          "RELEASE_STATUS": certio.file_content_sha(root / "RELEASE_STATUS.json", RUN_SPECIFIC),
          **{f"ledgers/{l_}": certio.file_content_sha(root / "ledgers" / f"{l_}.json", RUN_SPECIFIC) for l_
+> in LEDGERS}},
      "residual_records": {
          "quartic_C7": {o: rec(q["controls"]["C7_noether_exact"][o]) for o in certio.ORDERS},
          "master_grav_af0": {o: rec(m["details"]["grav_af0_deterministic"][o]) for o in certio.ORDERS},
          "master_table": m["table_by_kappa_and_antifield_number"],
          "required_master_residuals": m["required_master_residuals"]},
      "p23": {k: rs[k] for k in ("O1", "power_counting", "P6_entry", "EV_values", "title_qualifier", "contra
+> ct",
                                 "independent_rebuild_passed_p22", "O1_requirement_recorded",
                                 "one_loop_power_counting_verified", "P6_entry_inventory",
                                 "d_poly_domain_refusal_verified", "EV_values_verified")}
      | {"d_poly_domain_refusal": {k: rs["d_poly_domain_refusal"][k] for k in ("ok", "n_plants", "not_raised
+> ",
                                                                              "algebra_ok")},
         "change_log_vs_p22": {k: rs["change_log_vs_p22"][k] for k in ("added", "modified", "removed",
                                                                       "undisclosed_changes")}},
      "p22": {k: rs[k] for k in ("evanescent_M2_verified", "evanescent_counterterms_verified",
                                 "d_runtime_guard_verified", "evanescent", "P5_calculation_status",
                                 "D11_matching_verified_scope", "evanescent_hh_counterterm_added", "contract
+> ",
                                 "p21_layer", "manuscript_disclosure", "open_pins", "independent_rebuild_pas
+> sed_p21")}
      | {"lower_layers": {k: v for k, v in rs["lower_layers"].items() if k != "contract_detail"},
         "d_runtime_guard": {k: rs["d_runtime_guard"][k] for k in ("ok", "n_plants", "not_raised", "algebra_
+> ok",
                                                                   "entry_points")},
         "change_log_vs_p21": {k: rs["change_log_vs_p21"][k] for k in ("added", "modified", "removed",
                                                                       "undisclosed_changes")}},
      "p21": {"D11_absolute_pin_verified": rs["D11_absolute_pin_verified"],
              "absolute_pin": rs["p21_layer_absolute_pin"],
              "lower_layers": {k: v for k, v in rs["p21_layer_lower_layers"].items() if k != "contract_detai
+> l"},
              "p20_layer": rs["p21_layer_p20_layer"], "manuscript_disclosure": rs["p21_layer_manuscript_disc
+> losure"],
              "open_pins": rs["p21_layer_open_pins"],
              # recomputed by the p21 gate on the CURRENT tree (not the p21 release log)
              "p21_gate_change_log_vs_p20_recomputed_on_current_tree": {
                  k: rs["p21_layer_change_log_vs_p20"][k] for k in ("added", "modified", "removed",
                                                                    "undisclosed_package_changes")},
              "independent_rebuild_passed_p20": rs["p21_layer_independent_rebuild_passed_p20"]},
      "p20": {"D11_ratio_pin_verified": rs["D11_ratio_pin_verified"], "D11_pin": rs["p20_layer_D11_pin"],
              "K1_correction": rs["p20_layer_K1_correction"], "retraction_check": rs["p20_layer_retraction_c
+> heck"],
              "open_pins_p20_layer": rs["p20_layer_open_pins"],
              "full_closure_mutation_detector": rs["full_closure_mutation_detector"],
              # p20 reviewer: this log is RECOMPUTED by the p20 gate on the CURRENT tree (not the p20 releas
+> e log)
              "p20_gate_change_log_vs_p19_recomputed_on_current_tree": {
                  k: rs["p20_layer_change_log_vs_p19"][k] for k in ("added", "modified", "removed")},
              "independent_rebuild_passed_p19": rs["p20_layer_independent_rebuild_passed_p19"]},
      "p19": {"projected_defect_computed": rs["projected_defect_computed"],
              "projected_identity_verified": rs["projected_identity_verified"],
              "projected_restoration_verified": rs["projected_restoration_verified"],
              "bulk_symmetry_restored": rs["bulk_symmetry_restored"],
              "counterterm_solve_outcome": rs["counterterm_solve"]["outcome"],
              "projected_defect_summary": rs["projected_defect_summary"],
              # p20 reviewer: renamed (was /p19/change_log_vs_p18, recomputed on the current tree); the p19
              # release log as released is frozen in archival/p19/reference_values_p19.json (pinned)
              "p19_gate_change_log_vs_p18_recomputed_on_current_tree": {
                  k: rs["p19_gate_change_log_vs_p18_recomputed_on_current_tree"][k] for k in ("added", "modi
+> fied", "removed")},
              "change_log_vs_p18_as_released_p19": "archival/p19/reference_values_p19.json (sha256 pinned)",
              "independent_rebuild_passed_p18": rs["independent_rebuild_passed_p18"]},
      "p18": {"graphs": rs["graphs"], "D12_status": rs["D12_status"], "batchA_evaluated": rs["batchA_evaluat
+> ed"],
              "D13_evaluated": rs["D13_evaluated"], "batchC_evaluated": rs["batchC_evaluated"],
              "E2_status": rs["E2_status"], "E2_decoupling_proven": rs["E2_decoupling_proven"],
              "D11_evaluated": rs["D11_evaluated"], "library_certified": rs["library_certified"],
              "contract_changed_files": rs["contract_identity"]["changed_vs_p14r"],
              "contract_files_added": rs["contract_identity"]["added"],
```

```
                "hh_evanescent_status": rs["hh_evanescent_status"],
                "hh_evanescent_decoupling_proven": rs["hh_evanescent_decoupling_proven"],
                "evanescent_hh_counterterm_added": rs["evanescent_hh_counterterm_added"],
                "D11_matching_verified": rs["D11_matching_verified"],
                "quantum_inputs_evaluated": rs["quantum_inputs_evaluated"],
                "hh_full_row_decoupling_proven": rs["hh_full_row_decoupling_proven"],
                "independent_rebuild_passed_p14r": rs["independent_rebuild_passed_p14r"],
                "independent_rebuild_passed_p15": rs["independent_rebuild_passed_p15"],
                "independent_rebuild_passed_p16": rs["independent_rebuild_passed_p16"],
                "independent_rebuild_passed_p17": rs["independent_rebuild_passed_p17"]},
        "release": {"release_claim_accepted": rs["release_claim_accepted"], "publication": rs["publication"],
                    "classical_readiness_for_ch_identity": cl["classical_readiness_for_ch_identity"],
                    "flags": cl["flags"], "milestones": rs["milestones"]["status"], "hard_stops": rs["hard_sto
  +> ps"],
                    "P5_entry_flags": rs["P5_entry_flags"], "P5_entry_earned": rs["P5_entry_earned"],
                    "P3_P4_frozen_as_P5_inputs": rs["P3_P4_frozen_as_P5_inputs"],
                    "D11_matching_verified": rs["D11_matching_verified"], "D12_status": rs["D12_status"]},
        "tests": tr.get("groups", {}),
    }


def flatten(d, p=""):
    if isinstance(d, dict):
        out = {}
        for k, v in d.items():
            out.update(flatten(v, f"{p}/{k}"))
        return out
    return {p: d}


def compare(ref, cur):
    a, b = flatten(ref), flatten(cur)
    diffs = [{"key": k, "reference": a.get(k, "<absent>"), "rebuild": b.get(k, "<absent>")}
             for k in sorted(set(a) | set(b)) if a.get(k, "<absent>") != b.get(k, "<absent>")]
    return {"compared": len(set(a) | set(b)), "equal": len(set(a) | set(b)) - len(diffs), "differences": diffs
  +> }


# ---------------- trace analysis ----------------
SYSTEM_PREFIXES = ("/usr/", "/lib", "/etc/", "/proc/", "/dev/", "/sys/", "/opt/", "/var/lib/", "/run/")


def analyse_trace(source_before):
    if not TRACE.exists():
        return {"present": False, "unpinned_external_reads": ["<no trace>"]}
    reads, writes = set(), set()
    for ln in TRACE.read_text(errors="replace").splitlines():
        if len(ln) < 3:
            continue
        (writes if ln[0] == "W" else reads).add(ln[2:])
    root = str(ROOT) + "/"
    tmp = tempfile.gettempdir().rstrip("/") + "/"
    site = tuple(sorted({str(Path(p).resolve()) + "/" for p in sys.path if p and ("site-packages" in p or "dis
  +> t-packages" in p)}))
    pinned_ws = {v["workspace"] for v in inherited.PINS.values()}
    ext_pinned, ext_other, in_tree_inputs, unknown = set(), set(), set(), set()
    home = str(Path.home()) + "/"
    for r in reads:
        if r.startswith(root):
            rel = r[len(root):]
            if rel in source_before:
                in_tree_inputs.add(rel)
            elif r not in writes and not (ROOT / rel).exists() and "__pycache__" not in rel:
                unknown.add(rel)
            continue
        if r in pinned_ws:
            ext_pinned.add(r)
        elif r.startswith(SYSTEM_PREFIXES) or r.startswith(site) or "__pycache__" in r or r.endswith((".pyc",)
  +> ):
            continue
        elif r == tmp.rstrip("/") or r.startswith(tmp):
            continue                            # TemporaryDirectory / tempfile / pytest / this entrypoint's own
  +>  scratch
        elif r.startswith(home + ".") or r in ("/", ".") or r.startswith(str(TRACE)):
            continue
        elif "/" not in r and not Path(r).is_absolute():
            continue                            # relative basename from Path("subdir").exists() / mkdir (not a
  +> file read)
        else:
            ext_other.add(r)
    data_inputs = sorted(x for x in in_tree_inputs if not x.endswith((".py", ".tex", ".md")))
```

```
    pins = SM.inherited_data(ROOT)
    unpinned_data = [x for x in data_inputs if x not in pins and not x.startswith("provenance/")
                     and x not in (SM.REFERENCE_FILE, SM.PINS_FILE) and x not in PINNED_ARCHIVAL]
    return {"present": True, "reads": len(reads), "writes": len(writes),
            "external_reads_pinned": sorted(ext_pinned), "unpinned_external_reads": sorted(ext_other),
            "in_tree_source_files_read": len(in_tree_inputs), "in_tree_data_inputs_read": data_inputs,
            "unpinned_in_tree_data_inputs": unpinned_data, "reads_of_unknown_files": sorted(unknown)[:50]}


# ---------------- main ----------------
def write_reference():
    rs = jload("RELEASE_STATUS.json")
    assert rs["release_claim_accepted"] and rs["publication"] == "accepted", "build the release first"
    ref = {"landmark": "XVI-P", "revision": QC.REVISION, "kind": "author reference values for the p21 rebuild
  +> contract",
           "generated_from": "the author's full build of this source tree (tools/build.py)",
           "environment": versions(), "values": reference_values(ROOT)}
    (ROOT / SM.REFERENCE_FILE).write_text(json.dumps(ref, indent=1) + "\n")
    print(json.dumps({"written": SM.REFERENCE_FILE, "tree_sha256_without_reference":
                      ref["values"]["source"]["tree_sha256_without_reference"]}))
    return 0


def main():
    a = sys.argv[1:]
    if "--write-reference" in a:
        return write_reference()
    mode = a[a.index("--inherited") + 1] if "--inherited" in a else "auto"
    report_path = Path(a[a.index("--report") + 1]) if "--report" in a else ROOT / "build" / "rebuild_report.js
  +> on"
    t0 = time.time()
    steps, rep = {}, {"landmark": "XVI-P", "revision": QC.REVISION, "entrypoint": "tools/rebuild_from_source.p
  +> y",
                      "inherited_mode": mode, "kind": "package evidence (XVI-O conv:package); not a certificat
  +> e"}
    env = versions()
    ref = jload(SM.REFERENCE_FILE)
    rep["environment"] = env
    rep["environment_reference"] = ref.get("environment")
    rep["environment_matches_reference"] = {k: env.get(k) == ref.get("environment", {}).get(k)
                                            for k in ("python", "sympy", "mpmath", "pytest", "pdftex", "pdftot
  +> ext")}
    # 2. source only
    extra = SM.non_source_files(ROOT)
    steps["source_only_tree"] = {"ok": not extra, "non_source_files": extra[:40], "count": len(extra)}
    source_before = set(SM.source_files(ROOT))
    rep["source_tree_sha256"] = SM.tree_sha(ROOT)
    rep["source_tree_sha256_without_reference"] = SM.tree_sha_without_reference(ROOT)
    rep["n_source_files"] = len(source_before)
    with tempfile.TemporaryDirectory(prefix="p23src_") as td:
        z = SM.make_zip(Path(td) / SM.ZIP_NAME, ROOT)
    rep["source_zip_sha256"] = z["sha256"]
    if extra:
        rep.update({"pass": False, "steps": steps, "stopped": "tree is not source-only (generated artifacts or
  +>  caches present)"})
        report_path.parent.mkdir(parents=True, exist_ok=True)
        report_path.write_text(json.dumps(rep, indent=1) + "\n")
        print(json.dumps({"pass": False, "reason": rep["stopped"], "examples": extra[:10]}))
        return 1
    # 3. pins
    os.environ["XVIP_INHERITED"] = mode
    pins = inherited.verify_all(mode)
    data = SM.check_inherited_data(ROOT)
    xvio = SM.check_xvio_provenance(ROOT)
    arch = {p: (ROOT / p).is_file() and SM.sha(ROOT / p) == h for p, h in ARCHIVAL_PINS.items()}
    steps["inherited_pins"] = {"ok": all(v["ok"] and v["vendored_copy_matches_pin"] for v in pins.values())
                               and all(v["ok"] for v in data.values()) and all(v["ok"] for v in xvio.values())
                               and all(arch.values())
                               and C.XVI_M_PIN == XVI_M_PIN
                               and inherited.PINS["XVI-M.tex"]["sha256"] == XVI_M_PIN,
                               "external": pins, "inherited_data_files": len(data),
                               "inherited_data_failures": [k for k, v in data.items() if not v["ok"]],
                               "xvi_o_o70_files_verified": len(xvio),
                               "xvi_o_o70_failures": [k for k, v in xvio.items() if not v["ok"]],
                               "XVI_M_pin": XVI_M_PIN}
    # 4. full build (fresh, traced, sync inside the copy)
    (ROOT / "build").mkdir(exist_ok=True)
    TRACE.unlink(missing_ok=True)
    benv = dict(os.environ, P14_SYNC_DIR=str(SYNC), XVIP_INHERITED=mode, P13_MUTATIONS="", P14_MUTATIONS="",
                XVIP_TRACE_LOG=str(TRACE),
                PYTHONPATH=os.pathsep.join([str(ROOT / "tools" / "read_trace")] +
```

```
                                       ([os.environ["PYTHONPATH"]] if os.environ.get("PYTHONPATH") else []
+> )))
  tb = time.time()
  with open(ROOT / "build" / "rebuild_build.log", "w") as lf:
      r = subprocess.run([PY, "tools/build.py"], cwd=str(ROOT), env=benv, stdout=lf, stderr=subprocess.STDOU
+> T,
                         timeout=4 * 3600)
  steps["full_build"] = {"ok": r.returncode == 0, "rc": r.returncode, "seconds": round(time.time() - tb, 1),
                         "log": "build/rebuild_build.log"}
  if r.returncode == 0:
      import build as B                      # noqa: E402  (tools/build.py; import has no side effects)
      from manifest import verify            # noqa: E402
      rs = jload("RELEASE_STATUS.json")
      steps["release_accepted"] = {"ok": rs["release_claim_accepted"] is True and rs["publication"] == "acce
+> pted"
                                   and rs["classical"]["classical_readiness_for_ch_identity"] is True
                                   and rs["P5_entry_earned"] is True and rs["P3_P4_frozen_as_P5_inputs"] is
+> True
                                   and rs.get("revision") == QC.REVISION and rs.get("batchA_evaluated") is T
+> rue
                                   and rs.get("D13_evaluated") is True and rs.get("batchC_evaluated") is Tru
+> e
                                   and rs.get("E2_decoupling_proven") is True and rs.get("D11_evaluated") is
+>  True
                                   # p18 (disclosed): D11 is earned in p18; the p17 criterion "D11_evaluated
+>  is False" was stale
                                   and rs.get("D11_matching_verified") is True
                                   # p22 (disclosed): the scope is widened by EV-M2 (evanescent components)
                                   and str(rs.get("D11_matching_verified_scope", "")).startswith(
                                       "4D components: M2 discharged")
                                   and rs.get("quantum_inputs_evaluated") == "earned"
                                   and rs.get("hh_full_row_decoupling_proven") is True
                                   and rs.get("hard_stops_hold") is True
                                   # p19: the projected-defect flags (P19 gate) and the bulk hard stop
                                   and rs.get("projected_defect_computed") == "earned"
                                   and rs.get("projected_identity_verified") == "earned"
                                   and rs.get("projected_restoration_verified") == "earned"
                                   and rs.get("bulk_symmetry_restored") is False
                                   # p20: the corrected restoration (full closure), D11 ratio pin, no retrac
+> ted text
                                   and rs.get("D11_ratio_pin_verified") == "earned"
                                   and rs.get("p20_layer_K1_correction", {}).get("full_closure_complete_S_ct
+> ") is True
                                   and rs.get("p20_layer_retraction_check", {}).get("hits") == {}
                                   # p21: absolute pin, lower layers re-checked live, every package change d
+> isclosed
                                   and rs.get("D11_absolute_pin_verified") == "earned"
                                   and rs.get("lower_layers", {}).get("rechecked_live") is True
                                   # p22: evanescent M2 + counterterms, runtime guard, no undisclosed change
                                   and all(rs.get(k) == "earned" for k in Q22.EARNABLE_P22)
                                   and rs.get("P5_calculation_status") == Q22.P5_STATUS_P22
                                   # p23: O.1 scope / power counting / P.6 entry / poly refusal / EV values
                                   and all(rs.get(k) == "earned" for k in QC.EARNABLE_P23)
                                   and rs.get("O1_closure") == "NOT_CLOSED"
                                   and rs.get("change_log_vs_p22", {}).get("undisclosed_changes") == [],
                                   "milestones": rs["milestones"]["status"], "P5_entry_flags": rs["P5_entry_
+> flags"]}
      groups = B.junit_groups()
      steps["tests"] = {"ok": set(groups) == {"baseline", "certificate_mutation", "resumed_build"} and
                        all(g["passed"] > 0 and g["failed"] == 0 and g["skipped"] == 0 for g in groups.value
+> s()),
                        "groups": groups, "summary": (ROOT / "build" / "p14" / "pytest_summary.txt").read_te
+> xt().strip()}
      rb = B.readback_compare(ROOT)
      steps["pdf_readback_vs_records"] = {"ok": rb["pass"], "records": rb["expected_records"],
                                          "appendix": rb["XVI-P-appendix"]["pass"], "combined": rb["XVI-P"][
+> "pass"],
                                          "value_blocks": {st: {r: x["pass"] for r, x in rb[st]["value_block
+> s"].items()}
                                                           for st in ("XVI-P-appendix", "XVI-P")},
                                          "footer": {st: rb[st]["footer"] for st in ("XVI-P-appendix", "XVI-
+> P")}}
      v = verify(ROOT / "RELEASE_MANIFEST_EXTERNAL.txt", ROOT)
      synced = {f.name: SM.sha(f) for f in sorted(SYNC.glob("*"))} if SYNC.exists() else {}
      steps["external_manifest"] = {"ok": v["pass"] and len(synced) == 6, "entries": v["entries"],
                                    "failures": v["failures"], "synced_inside_copy": sorted(synced)}
      zp = ROOT / "dist" / SM.ZIP_NAME
      zbad = SM.zip_matches_tree(zp, ROOT)
      steps["source_only_zip_rebuilt"] = {"ok": not zbad and SM.sha(zp) == rep["source_zip_sha256"],
                                          "differences": zbad[:20]}
      tr = analyse_trace(source_before)
```

```
        steps["file_access_trace"] = {"ok": tr["present"] and not tr["unpinned_external_reads"]
                                      and not tr["unpinned_in_tree_data_inputs"], **tr}
        cur = reference_values(ROOT)
        cmp_ = compare(ref["values"], cur)
        steps["comparison_with_reference"] = {"ok": not cmp_["differences"], **cmp_}
        rep["rebuilt_values"] = cur
    rep["steps"] = steps
    rep["pass"] = all(s.get("ok") for s in steps.values()) and "comparison_with_reference" in steps
    rep["seconds"] = round(time.time() - t0, 1)
    report_path.parent.mkdir(parents=True, exist_ok=True)
    report_path.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps({"pass": rep["pass"], "steps": {k: v.get("ok") for k, v in steps.items()},
                      "report": str(report_path), "source_tree_sha256": rep["source_tree_sha256"],
                      "source_zip_sha256": rep["source_zip_sha256"]}, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

113 REBUILD_INSTRUCTIONS.md

[listing 80/133] REBUILD_INSTRUCTIONS.md

```
# XVI-P p23: rebuild instructions (reproducibility contract)

p23 (stage release from p22; disclosed contract revision: polynomial-domain refusal block appended to dred.py)
  +> : the
0.1 requirement record, the one-loop power counting and the P.6 entry inventory are in certificates/P23_01_sco
  +> pe.json
(code/p5_calc/power_counting.py), and the gate is code/p5_calc/release_p23.py (live poly-refusal probe, EV val
  +> ue check).
The p23 change log is computed against archival/p22/p22_source_files.json. The source zip is
XVI-P-p23-source-only.zip, the reference is reproducibility/reference_values_p23.json, and the reports are
reproducibility/{author,independent}_rebuild_report_p23.json. The top-level RELEASE_STATUS.json is the p23 sta
  +> tus
(certificates/P22_release.json is the p22 layer record). independent_rebuild_passed_p22 = true (archived repor
  +> t
023e5b83..., same box); the p23 independent rebuild is pending. The text below applies with p22 -> p23 where i
  +> t names
the current release.


p22 (from p21; disclosed contract revision): EV-M2 proven, evanescent counterterms (EV-CT) earned with full cl
  +> osure and
the projected identity unchanged, P.5 calculation status renormalized (one loop, kappa^2, two-point functions,
QdS externals), and a runtime d guard in dred.py (delimited additive block, live probe of 30 forms x 4 express
  +> ions in
release_p22.py). The p22 change log is computed against archival/p21/p21_source_files.json (pinned in
code/p5_calc/p22_conventions.py) and every change must be disclosed (no tools/ or notes prefix exemption). The
  +>  source
zip is XVI-P-p22-source-only.zip, the reference is reproducibility/reference_values_p22.json, the reports are
reproducibility/{author,independent}_rebuild_report_p22.json, and the top-level RELEASE_STATUS.json is the p22
  +>  status
(certificates/P21_release.json is the p21 layer record). independent_rebuild_passed_p21 = true (archived repor
  +> t
8a582b33..., same box); the p22 independent rebuild is pending. The p21 text below applies with p21 -> p22 whe
  +> re it
names the current release.


p21 (small release from p20): the absolute D11 pin is derived (sign and magnitude, from 1/2 Tr ln under the de
  +> clared
conventions, equal to the heat-kernel a_2); the p20 gate checks the top-level pin status and re-checks the low
  +> er layers;
an allow-list scan and a runtime opaque-d check complement the pattern scans; the p20 edit of the P19 note is
  +> disclosed.
The p20 independent rebuild PASSED (report e8f358f9..., same box) and is recorded as independent_rebuild_passe
  +> d_p20;
the p21 independent rebuild is pending.

This contract lets anyone check, from **source only**, that the XVI-P evidence (the frozen classical P.3/P.4
pipeline, the p14r P.5 entry contract, the p15 first P.5 calculation -- Batch A D02, D12, D14a-c and Batch B D
  +> 13 --
the p16 second step -- the E2 decoupling proof and Batch C D01, D03 -- and the p17 third step -- the proof tha
  +> t the
D01/D03 evanescent poles decouple from the ch-projected slot as the sum D01 + D03 -- and the p18 fourth step -
  +> - the
```

```
discharge of the D11 matching premise M2, D11 evaluated, and the full hh slot D01 + D02 + D03 + D11 + D12 -- a
  +> nd the
p19 identity step -- the projected one-loop kappa^2 ch-identity defect assembled from every evaluated input, a
  +> nd the
counterterm solve -- and the p20 correction -- K1 orientation erratum, full sigma_0 closure of the one-loop ka
  +> ppa^2
MS-bar counterterm, the D11 ratio pin)
regenerates and matches the author's reference values. A passing rebuild is package evidence (XVI-O `conv:pack
  +> age`), **not** a
mathematical certificate. XVI-P stays unregistered.

## Flags

| flag | meaning | set by |
|---|---|---|
| `author_rebuild_passed` | `reproducibility/author_rebuild_report_p21.json` passed **and** was produced from
  +> exactly the current source tree (`source_tree_sha256` equal) | `code/p5_calc/release_p21.py` |
| `independent_rebuild_passed` | `pending` until `reproducibility/independent_rebuild_report_p21.json` exists;
  +>  then the same rule | a separate clean environment (not the author) |
| `independent_rebuild_passed_p20` | the archived p20 independent report (`archival/p20/reproducibility/indepe
  +> ndent_rebuild_report_p20.json`, sha256 `e8f358f9...` pinned) passed and is bound to the p20 tree `2c0aa15
  +> 3...` / zip `ca32ce01...` (same box) | `code/p5_calc/release_p21.py` |
| `independent_rebuild_passed_p19` | the archived p19 independent report (`archival/p19/reproducibility/indepe
  +> ndent_rebuild_report_p19.json`, sha256 `1b01f1d8...` pinned) passed and is bound to the p19 tree `fcf9035
  +> 4...` / zip `da924275...` | `code/p5_calc/release_p20.py` |
| `independent_rebuild_passed_p18` | the archived p18 independent report (`archival/p18/reproducibility/indepe
  +> ndent_rebuild_report_p18.json`, sha256 `893f167e...` pinned) passed and is bound to the p18 tree `bfab3b4
  +> b...` / zip `7c7d5451...` | `code/p5_calc/release_p19.py` |
| `independent_rebuild_passed_p17` | the archived p17 independent report (`archival/p17/reproducibility/indepe
  +> ndent_rebuild_report_p17.json`, sha256 `92542a22...` pinned) passed and is bound to the p17 tree `2a5a631
  +> 7...` / zip `7f4f61df...` | `code/p5_calc/release_p18.py` |
| `independent_rebuild_passed_p16` | the archived p16 independent report (`archival/p16/reproducibility/indepe
  +> ndent_rebuild_report_p16.json`, sha256 `1a886196...` pinned) passed and is bound to the p16 tree `90b4a66
  +> 5...` / zip `56d5887d...` | `code/p5_calc/release_p17.py` |
| `independent_rebuild_passed_p15` | the archived p15 independent report (`archival/p15/reproducibility/indepe
  +> ndent_rebuild_report_p15.json`, sha256 `6bc760c2...` pinned) passed and is bound to the p15 tree `79f5375
  +> 2...` / zip `54325c1f...` | `code/p5_calc/release_p16.py` |
| `independent_rebuild_passed_p14r` | the archived p14r independent report (`archival/p14r/reproducibility/ind
  +> ependent_rebuild_report_p14r.json`, sha256 pinned) passed and is bound to the p14r tree `a64c0ae3...` / z
  +> ip `19e8bf59...` | `code/p5_calc/release_p15.py` |

None of these flags enters `release_claim_accepted`. (The integrity of the pinned archived records does: a tam
  +> pered
archive rejects publication.)

**p19 independent rebuild (bound to the p19 tree).** PASSED with findings (SAME-BOX caveat: same machine, fres
  +> h
directory and venv, from the source-only zip; not a second machine). It is not inherited by p20. Its findings
  +> (K1:
the p19 full-closure statement was an orientation bookkeeping error, now retracted; quotient orientation; rest
  +> oration
flag not independent; blind spots; J1 wording; scan gaps; stale RELEASE_STATUS on a standalone reject) are add
  +> ressed
in the p20 correction release, with no contract revision.

**p18 independent rebuild (bound to the p18 tree).** PASSED (SAME-BOX caveat: same machine, fresh directory an
  +> d
venv, from the source-only zip; not a second machine). It is not inherited by p19. Its findings I1 (publicatio
  +> n not
gated on quantum_inputs_evaluated), I2 (remaining static-scan pattern gaps) and I3 (milestone wording) are add
  +> ressed
in p19 with no contract revision.

**p17 independent rebuild (bound to the p17 tree).** PASSED (SAME-BOX caveat: same machine, fresh directory an
  +> d
venv, from the source-only zip; not a second machine). It is not inherited by p18. Its findings H1 (static-sca
  +> n
pattern gaps), H2 (c hat / h tilde assignment), H3 (stale comments), H4 (curly quotes in listings) and H5 (par
  +> se_d of
srepr raises, undocumented) are addressed in p18; the dred.py change of p18 is documentation only (its AST wit
  +> hout
docstrings is pinned equal to p17).

**p16 independent rebuild (bound to the p16 tree).** PASSED (272/272 values equal, 0 mismatches; SAME-BOX cave
  +> at:
same machine, fresh directory and venv, from the source-only zip; not a second machine). It is not inherited b
  +> y p17.
Its findings G1 (static-scan scope), G2 (plain `Symbol('d')` from re-parsed strings), G3 (guard wording), G4 (
  +> change
log omitted the new reference file) and G6 (phase convention) are addressed in p17. The fix for G2 (the guarde
  +> d
```

loader `dred.parse_d`) is the one disclosed contract revision of p17 (`dred.py`).

The top-level `RELEASE_STATUS.json` is the p21 status. The layer records are `certificates/P20_release.json`,
  +> `certificates/P19_release.json`,
`certificates/P18_release.json`,
`certificates/P17_release.json`,
`certificates/P16_release.json` and `certificates/P15_release.json`.

p15 independent rebuild (bound to the p15 tree). PASSED (0 mismatches; SAME-BOX caveat: same machine, fres
  +> h
directory and venv, from the source-only zip; not a second machine). It is not inherited by p16. Its findings
  +> F3
(opaque-d bypass via `replace`/`lambdify`), F4 (stale footer label; readback scope) and F5 (non-contract chang
  +> e log)
are addressed in p16; F3 is the one disclosed contract revision of p16 (`dred.py`).

p14r independent rebuild (bound to the p14r tree). PASSED: 143/143 compared values equal, 60 tests, review
  +> er toy
checks OK. SAME-BOX caveat: it ran on the same machine as the author build (fresh directory and venv, from the
source-only zip), not on a second machine. It is not inherited by p15: the p15 tree has its own pending flag.
  +> Its
finding F1 (the heuristic PrematureD4 guard of `dred.py`) is repaired in p15 as a disclosed contract revision.

The p14r contract record is `certificates/P14_release.json`.

p13 freeze (separate, bound to the p13 tree). `P3_P4_frozen_as_P5_inputs` is true because the independent
rebuild of **p13** passed (source tree `2b1baf33...`, zip `2f85a566...`; archived report
`archival/p13/reproducibility/independent_rebuild_report_p13.json`, pinned by sha256), the 54 frozen classical
  +> files
are byte-identical and the classical values equal the archived p13 reference values. Caveat: that rebuild ran
  +> on the
same box (shared kernel, TeX Live, poppler) in a fresh venv from the source-only zip; it was not on a second m
  +> achine.
The p13 flag is not inherited by later changes: each tree has its own independent flag.

Environment used by the author

component	version
Python	3.13.5
sympy	1.14.0
mpmath	1.3.0
pytest	9.1.1
pdfTeX	3.141592653-2.6-1.40.26 (TeX Live 2025/dev/Debian)
poppler (`pdftotext`, `pdfinfo`)	25.03.0

LaTeX packages used: lmodern, amsmath/amssymb/amsthm, booktabs, hyperref, bookmark, geometry, longtable,
listings, xcolor, graphicx, verbatim, fancyhdr. No other Python packages are needed. Other versions may work.
  +> The
report records the versions found and compares them with the reference. A different sympy version may change t
  +> he
`srepr` text of the exports, and therefore the export **file** hashes, without changing the polynomials. The
polynomial hashes are compared separately.

Steps

```
unzip XVI-P-p23-source-only.zip         # creates XVI-P/ (source only: no certificates, exports, PDFs, caches)
cd XVI-P
python3 tools/rebuild_from_source.py --inherited vendored --report build/rebuild_report.json
```

The command exits 0 if and only if every step passed and every compared value is equal. It takes about 30-45 m
  +> inutes
on one machine. The full pipeline uses about 2.5 minutes for the quartic, about 1 minute for the master, about
1.5 minutes for the matter-loop gates, about 7 minutes for the P.5 entry stages (amplitude rules incl. the W6
Hessian route <almost_equal_to> 3, operational DRED <almost_equal_to> 0.5, counterterm domain <almost_equal_to
  +> > 3), about 4 minutes for the p15 calculation
(integral library <almost_equal_to> 0.75, Batch A <almost_equal_to> 2, D13 <almost_equal_to> 1.2), about 4 min
  +> utes for the p16 calculation (E2 <almost_equal_to> 0.1, Batch C <almost_equal_to> 3.5),
about 1 minute for the p17 calculation (hh slot), about 2 minutes for the p18 calculation (M2, D11, full slot)
  +> , about 5 minutes for the p19 calculation (projected defect, full closure), a few seconds for the p20 D11
  +> pin (p21: with the absolute pin derivation) and several minutes for the tests.

`--inherited` selects where the pinned external inputs come from:
* `vendored`: only the copies shipped in `provenance/` (recommended for a clean environment);
* `workspace`: the sibling landmark trees, e.g. `/workspace/XVI-M.tex`;
* `auto`: workspace if present, otherwise vendored.

Every location is sha256-verified, and a mismatch fails the rebuild.

```
## What the entrypoint does

1. Records the environment (table above) and compares it with `reproducibility/reference_values_p23.json`.
2. Refuses to run unless the tree is source-only (`tools/source_manifest.py` defines the source set). No gener
  +> ated
   certificate, export, ledger, listing, PDF, zip, `build/` or `__pycache__` may be present.
3. Verifies inherited dependencies and pins:
   * XVI-M `41aa109935f40ddee3109de62c46329c189b5408007970cc90ed5611dc4739ea`;
   * XVI-N-r10 `code/prep/grav_density.py`, read by C5/C6 (pin in `code/classical/inherited.py`);
   * the verbatim XVI-O o70-sync code and notes (`code/matter_loop/*.py`, `notes/*.tex`) against
     `provenance/XVI-O_o70_source_tree.json`;
   * the inherited data input `ledgers/quantum_factor_ledger.json` (`reproducibility/inherited_data_pins.json`
  +> );
   * the p13 freeze inputs under `archival/p13/` (pinned in `code/p5_entry/p14_conventions.py`);
   * the archived p14r independent report (pinned in `code/p5_calc/p15_conventions.py`);
   * the archived p15 records: independent and author reports, reference values and the per-file source list
     `archival/p15/p15_source_files.json` (pinned in `code/p5_calc/p16_conventions.py`);
   * the archived p16 records: independent and author reports, reference values and the per-file source list
     `archival/p16/p16_source_files.json` (pinned in `code/p5_calc/p17_conventions.py`);
   * the archived p17 records: independent and author reports, reference values and the per-file source list
     `archival/p17/p17_source_files.json` (pinned in `code/p5_calc/p18_conventions.py`);
   * the archived p18 records: independent and author reports, reference values and the per-file source list
     `archival/p18/p18_source_files.json` (pinned in `code/p5_calc/p19_conventions.py`), and the p20 change lo
  +> g is
     computed against `archival/p19/p19_source_files.json` (pinned in `code/p5_calc/p20_conventions.py`); the
  +> p21 change log is computed against `archival/p20/p20_source_files.json` (pinned in `code/p5_calc/p21_conv
  +> entions.py`).
4. Runs `tools/build.py` from a fresh state, with no cache and sync redirected to `build/rebuild_sync`, so not
  +> hing is
   written outside the copy except the tests' temporary directories. Every Python file access is traced
   (`tools/read_trace/sitecustomize.py`). The build regenerates the ell2/ell3/ell4 exports and all residual re
  +> cords, the O5
   matter-loop records and the P.5 entry certificates (including the operational DRED rules and the W6 Hessian
  +> -route
   regeneration) and symbolic integrands, then the p15 calculation (`code/p5_calc`: integral library, Batch A,
  +>  D13)
   and the p15 release gate, then the p16 calculation (`e2_evanescent.py`, `batchC.py`) and the p16 release ga
  +> te
   `release_p16.py`, then the p17 calculation (`hh_slot.py`) and the p17 release gate `release_p17.py`, then t
  +> he p18
   calculation (`d11.py`) and the p18 release gate `release_p18.py`, then the p19 calculation (`defect.py`), t
  +> he
   p19 layer gate `release_p19.py`, the p20 D11 pin (`d11_pin.py`), the p20 layer gate `release_p20.py` and th
  +> e p21 release gate `release_p21.py`. It runs
   the baseline, certificate/gate-mutation and resumed-build tests, typesets the PDFs, reads them back, builds
  +>  the zips
   (including the source-only zip), and writes the external manifest.
5. Checks:
   * the p16 layer is accepted (batchA_evaluated, D13_evaluated, batchC_evaluated, E2 decoupling proven,
     D11 NOT evaluated, p16 hard stops), the p17 release is accepted (hh-slot decoupling proven, no evanescent
  +>  h.h
     counterterm, D11 NOT evaluated in the p17 layer, p17 hard stops), the p18 release is accepted (M2 dischar
  +> ged, D11
     evaluated, full-row slot decoupling, projected identity/restoration NOT earned in the p18 layer, p18 hard
  +>  stops),
     the p19 release is accepted (quantum_inputs_evaluated earned -- I1, projected_defect_computed,
     projected_identity_verified and projected_restoration_verified earned, bulk_symmetry_restored false, p19
  +> hard
     stops), the p20 release is accepted (restoration on the p20 definition = full sigma_0 closure of the comp
  +> lete
     MS-bar counterterm, K1 correction content, no retracted p19 text, D11_ratio_pin_verified earned, absolute
  +>  D11
     pin derived and PASS since p21, p20 hard stops), the p21 release is accepted (absolute D11 pin earned, lo
  +> wer layers re-checked live, every package change vs the p20 zip disclosed, p21 hard stops), P.5 entry is
  +> earned and the p13 freeze holds;
   * the test groups `baseline`, `certificate_mutation` and `resumed_build` all pass (junit);
   * the PDF readback equals the machine-readable residual records and (p16, F4) the D13, Batch A, Batch C and
  +>  E2
     value blocks (p17) the hh-slot value block and (p18) the M2, D11 and full-slot value blocks and (p19) the
  +>  defect value block and (p20) the D11 pin value block, and every page footer carries the
     version label `XVI-P p21`;
   * the external manifest verifies, and all six synced files exist;
   * the rebuilt source-only zip equals the tree;
   * no file outside the tree was read, except a pinned input.
6. Compares with the reference values:
   * ell2/ell3/ell4 export file hashes, polynomial hashes and term counts;
   * `V4_grav.json`, `ell4_conventions.json`, `P14_vertex_tensors.json` and every `exports/P14_integrands/*.js
  +> on`;
   * content hashes of every P13 classical, P14, P15 and P16 certificate (including `P14_dred_rules`,
     `P15_integral_library`, `P15_batchA`, `P15_D13`, `P15_release`, `P16_E2_evanescent`, `P16_batchC`,
     `P16_release`, `P17_hh_slot`, `P17_release`, `P18_D11_M2`, `P18_D11`, `P18_hh_slot`, `P18_release`, `P19_
```

```
  +> defect`, `P19_release`, `P20_D11_pin`, `P20_release`) and ledger, and of `RELEASE_STATUS`;
   * the graph statuses, D12 status, the flags, the list of changed contract files, the p19 change log vs p18
  +> and the p19 defect summary and flags, the p20 change log vs p19 (recomputed on the current tree; renamed
  +> in p21), the p21 change log vs p20, the absolute D11 pin, the K1 correction, the
     retraction check, the D11 pin and the open pins;
   * W/Q counts of the quartic C7 and master gravitational records at kappa^0, kappa^1, kappa^2;
   * the master table, classical flags, P.5 entry flags, freeze, milestone status, hard stops and test counts.

   The report goes to `--report` (default `build/rebuild_report.json`).

**Ignored in the comparison:** wall-clock timings (`seconds`), the run-specific reproducibility block (the two
  +>  flags
and the report records), and four freeze sub-records that depend on files correctly absent from a source-only
  +> copy
(`archived_zip_sha256_ok`, `list_equals_archived_zip`, `live_report_present`, `live_report_identical_to_archiv
  +> ed`;
the last also for the p14r, p15, p16, p17 and p18 records). The author tree holds the author's report, the arc
  +> hived p13 zip and the
reviewers' live p13/p14r reports, and a fresh copy
does not, so these legitimately differ. PDF and zip bytes are not compared because they embed the timings thro
  +> ugh the listed certificates. Their
content is checked through the readback and the manifest instead.

## Recording a rebuild

* **Author:** copy the passing report to `reproducibility/author_rebuild_report_p19.json` in the author tree,
  +> then run
  `python3 tools/build.py`.
* **Independent (separate clean environment):** return the passing report. It is stored as
  `reproducibility/independent_rebuild_report_p19.json`. `release_p19.py` sets `independent_rebuild_passed = t
  +> rue`
  only if the report passed and its `source_tree_sha256` equals the source tree being released.
```

114 Source: tools/build.py

[listing 81/133] tools/build.py

```
#!/usr/bin/env python3
"""XVI-P p23 single active release (p23 = stage release on the 0.1 direction: 0.1 requirement, one-loop power
counting, P.6 entry inventory, poly-domain refusal; previously p22 (p22 = bigger fronts after p21, WITH a disc
  +> losed contract revision: EV-M2 (evanescent
matching statement), EV-CT (evanescent counterterms, sigma_0-closed, identity unchanged; P.5 status renormaliz
  +> ed),
runtime d guard in code/p5_entry/dred.py (additive block, contract_registry), gate without prefix exemptions;
  +> stage
p22calc runs code/p5_calc/evanescent.py; after the p21 gate release_p21.py (now a LAYER gate writing
certificates/P21_release.json) the p22 gate code/p5_calc/release_p22.py runs).
Previous header (p21 = small release from p20: absolute D11 pin derived from 1/2 Tr ln under the
declared conventions; gate checks top-level pin status and re-checks lower layers; allow-list d-evaluation sca
  +> n +
runtime opaque-d check; P19 note edit disclosed; no scope expansion; no contract revision; after the p20 gate
release_p20.py (now a LAYER gate writing certificates/P20_release.json; d11_pin.py and release_p20.py are disc
  +> losed
p21 revisions) the p21 gate code/p5_calc/release_p21.py runs).
Previous header (p20 = CORRECTION of p19 -- K1 orientation erratum, restoration redefined as full
sigma_0 closure of the one-loop kappa^2 MS-bar counterterm, full-closure mutation detector, D11 ratio pin, J1
  +> wording,
scan patterns; no scope expansion; no contract revision), regenerated in dependency order (p20: after the p19
  +> gate
(release_p19.py, now a LAYER gate writing certificates/P19_release.json; defect.py and release_p19.py are disc
  +> losed
p20 revisions), the D11 pin code/p5_calc/d11_pin.py and the p20 release gate code/p5_calc/release_p20.py, whic
  +> h
requires the p19 layer accepted, the K1 correction content, the retraction check, the D11 ratio pin, the pinne
  +> d p19
independent record and the p20 hard stops; p19: defect.py + release_p19.py; p18: d11.py + release_p18.py; p17,
  +>  p16,
p15 as before):

conventions -> boundary identity (C3) -> quartic (deterministic C1-C7) -> classical transformations / source a
  +> ction
-> gauge map -> ch audit -> graphs -> master verification  [the FROZEN p13 classical P.3/P.4 code, unchanged]
-> matter-loop gates (regenerate the inherited O5 records) -> P.5 entry contract (amplitude rules, counterterm
domain, quantum object, prescription + D11 matching; NO loop evaluation) -> contract gate code/p5_entry/releas
  +> e_p14.py
(classical acceptance AND p13 freeze AND all five P.5-entry flags AND contract hard stops) -> dossier (every l
  +> ink
resolved) -> p15 calculation (code/p5_calc: integral library, Batch A, D13) -> p15 release gate
```

```
code/p5_calc/release_p15.py (contract accepted AND contract byte-identity except the disclosed dred.py correct
  +> ion AND
library AND batchA_evaluated AND D13_evaluated AND p15 hard stops) -> p16 calculation (code/p5_calc: E2 evanes
  +> cent
decoupling proof, Batch C D01/D03 via the covariant route) -> p16 release gate code/p5_calc/release_p16.py (p1
  +> 4r and
p15 gates accepted AND contract identity: only dred.py changed vs p14r (disclosed p15 F1 + p16 F3), no new con
  +> tract
file AND p15 calc files byte-identical AND E2 proof AND batchC_evaluated AND p16 hard stops; a rejected releas
  +> e exits
nonzero and STOPS the build before any PDF/zip/sync) -> tests (baseline, certificate and
gate mutations, resumed builds; junit) -> PDFs (science, appendix, combined) + readback -> zips (review-focus,
  +>  full,
SOURCE-ONLY XVI-P-p23-source-only.zip) -> external manifest LAST (PDFs never contain PDF hashes) -> sync to
$P14_SYNC_DIR (default /workspace; earlier source-only zips in /workspace are never written).

PUBLICATION GATE (since p12): before ANY pdf, zip or manifest/sync stage (also when resuming with --from-stage
  +> ) the
contract gate release_p14.py, the dossier dossier_p14.py, the p15 gate release_p15.py, the p16 gate
release_p16.py, the p17 gate release_p17.py, the p18 gate release_p18.py, the p19 gate release_p19.py, the p20
  +>  gate release_p20.py, the p21 gate release_p21.py AND the p22 gate release_p22.py are re-invoked and the
  +> current
artifacts are checked (export hashes, residual-record invariants, cross-certificate agreement, dossier links,
listings == executed/written files, zips == current files, source-only zip == current source set).  If anythin
  +> g
fails, existing PDFs/zips/manifest are moved to stale/ with a REJECTED marker, PUBLICATION_REJECTED.txt is wri
  +> tten,
nothing is synced, and the build exits nonzero.

p13/p14: the source set and the source-only zip are defined in tools/source_manifest.py; the clean-copy rebuil
  +> d
contract is tools/rebuild_from_source.py (REBUILD_INSTRUCTIONS.md).  Generated-file cleaning NEVER touches the
p13 records reproducibility/reference_values.json, reproducibility/independent_rebuild_report.json,
reproducibility/author_rebuild_report.json or anything under archival/.

Usage: python3 tools/build.py [--from-cache] [--skip-tests] [--from-stage NAME]
"""
from __future__ import annotations
import hashlib, json, os, re, shutil, subprocess, sys, time, unicodedata, zipfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
PY = sys.executable
MS = ROOT / "manuscript"
LST = MS / "listing_src"
DIST = ROOT / "dist"
TOP = Path(os.environ.get("P14_SYNC_DIR", "/workspace"))
STALE = ROOT / "stale"
PDF_STEMS = ("XVI-P-science", "XVI-P-appendix", "XVI-P")
SRC_ZIP = "XVI-P-p23-source-only.zip"
ZIPS = ("XVI-P-review-focus.zip", "XVI-P-full.zip", SRC_ZIP)
sys.path.insert(0, str(ROOT / "code" / "classical"))
sys.path.insert(0, str(ROOT / "tools"))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
import conventions as C  # noqa: E402   (frozen p13 classical conventions: pins only)
import p14_conventions as PC  # noqa: E402   (p14r CONTRACT constants: hard stops, flags, p13 pins)
sys.path.insert(0, str(ROOT / "code" / "p5_calc"))
import p15_conventions as Q15  # noqa: E402   (p15 layer constants: hard stops of the p15 gate)
import p23_conventions as QC  # noqa: E402   (revision/date/version label of THIS release)
import p22_conventions as Q22  # noqa: E402   (p22 layer)
import p21_conventions as Q21  # noqa: E402   (p21 layer: hard stops)
import p20_conventions as Q20  # noqa: E402   (p20 layer: hard stops)
import p19_conventions as Q19  # noqa: E402   (p19 layer: hard stops)
import p18_conventions as Q18  # noqa: E402   (p18 layer: hard stops)
import p17_conventions as Q17  # noqa: E402   (p17 layer: hard stops)
import p16_conventions as Q16  # noqa: E402   (p16 layer: hard stops)
import certio  # noqa: E402
import inherited  # noqa: E402
import source_manifest as SM  # noqa: E402
from manifest import file_line, verify, pdf_pages, sha  # noqa: E402

LOG = []


def step(name, cmd, timeout=3600):
    t0 = time.time()
    r = subprocess.run(cmd, cwd=str(ROOT), capture_output=True, text=True, timeout=timeout)
    LOG.append({"stage": name, "cmd": " ".join(map(str, cmd)), "rc": r.returncode, "sec": round(time.time() -
  +> t0, 1)})
    print(f"[{name}] rc={r.returncode} {LOG[-1]['sec']}s", flush=True)
    if r.returncode != 0:
```

```
        print(r.stdout[-4000:], r.stderr[-4000:])
        raise SystemExit(f"stage {name} failed")
    return r


# ---------------- listings (ASCII-sanitised copies of CURRENT sources) ----------------
GREEK = {"alpha": "alpha", "beta": "beta", "gamma": "gamma", "delta": "delta", "eps": "eps", "zeta": "zeta", "
  +> kappa": "kappa", "lambda": "lambda",
        "mu": "mu", "nu": "nu", "pi": "pi", "rho": "rho", "sigma": "sigma", "tau": "tau", "phi": "phi", "xi":
  +>  "xi", "psi": "psi",
        "Psi": "Psi", "Phi": "Phi", "Gamma": "Gamma", "Pi": "Pi", "d": "d", "<=": "<=", ">=": ">=", "!=": "!=
  +> ", "->": "->",
        "=>": "=>", "AND": "AND", "int": "int", "sqrt": "sqrt", "^2": "^2", "^4": "^4", "_1": "_1", "~=": "~=
  +> ", "~": "~",
        "x": "x", ".": ".", "--": "--", "-": "-", "'": "'", """: '"', """: '"', "...": "...", "<": "<", ">":
  +> ">",
        "ell": "ell", "hbar": "hbar", "Box": "Box", "==": "==", "in": "in", "subset": "subset", "sum": "sum"}


def ascii_clean(s: str) -> str:
    out = []
    for ch in s:
        if ord(ch) < 128:
            out.append(ch)
        elif ch in GREEK:
            out.append(GREEK[ch])
        else:
            out.append("<" + unicodedata.name(ch, "U+%04X" % ord(ch)).lower().replace(" ", "_") + ">")
    return "".join(out)


LISTINGS = [
    ("Release status / acceptance certificate (p23)", "certificates/P23_release.json"),
    ("Milestone ledger (p23)", "ledgers/P23_milestones.json"),
    ("Test results (p23)", "certificates/P23_test_results.json"),
    ("O.1 requirement, one-loop power counting, P.6 entry inventory (p23)", "certificates/P23_O1_scope.json"),
    ("p22 layer record (release gate p22, revised in p23)", "certificates/P22_release.json"),
    ("Evanescent components: EV-M2 statement and evanescent counterterms (p22)", "certificates/P22_evanescent.
  +> json"),
    ("p21 layer record (release gate p21, revised in p22)", "certificates/P21_release.json"),
    ("D11 pin: ratio (p20) and absolute sign and magnitude (derived in p21)", "certificates/P20_D11_pin.json")
  +> ,
    ("p20 layer record (release gate p20, revised in p21)", "certificates/P20_release.json"),
    ("P.5 identity: projected ch-identity defect, counterterm solve, full closure (p19 layer, corrected in p20
  +> )", "certificates/P19_defect.json"),
    ("p19 layer record (release gate p19, revised in p20)", "certificates/P19_release.json"),
    ("Milestone ledger (p19 layer)", "ledgers/P19_milestones.json"),
    ("p18 layer record (release gate p18)", "certificates/P18_release.json"),
    ("Milestone ledger (p18 layer)", "ledgers/P18_milestones.json"),
    ("P.5 calculation: D11 matching premise M2 (discharge; p18)", "certificates/P18_D11_M2.json"),
    ("P.5 calculation: D11 evaluated under the declared prescription (p18)", "certificates/P18_D11.json"),
    ("P.5 calculation: full hh slot D01+D02+D03+D11+D12 vs the ch projection (p18)", "certificates/P18_hh_slot
  +> .json"),
    ("p17 layer record (release gate p17)", "certificates/P17_release.json"),
    ("Milestone ledger (p17 layer)", "ledgers/P17_milestones.json"),
    ("P.5 calculation: D01/D03 evanescent poles vs the ch-projected slot (decoupling proof; p17)", "certificat
  +> es/P17_hh_slot.json"),
    ("p16 layer record (release gate p16)", "certificates/P16_release.json"),
    ("Milestone ledger (p16 layer)", "ledgers/P16_milestones.json"),
    ("P.5 calculation: E2 evanescent decoupling proof (D13; p16)", "certificates/P16_E2_evanescent.json"),
    ("P.5 calculation: Batch C certificate (D01, D03; p16)", "certificates/P16_batchC.json"),
    ("p15 layer release record (Batch A, D13; p15 gate)", "certificates/P15_release.json"),
    ("Milestone ledger (p15)", "ledgers/P15_milestones.json"),
    ("P.5 calculation: integral library certificate (p15)", "certificates/P15_integral_library.json"),
    ("P.5 calculation: Batch A certificate (D02, D12, D14a-c; p15)", "certificates/P15_batchA.json"),
    ("P.5 calculation: Batch B certificate (D13; p15)", "certificates/P15_D13.json"),
    ("p14r contract record (P.5 entry; contract gate)", "certificates/P14_release.json"),
    ("Closure release artifact of the p14r contract", "certificates/P14_closure_release.json"),
    ("P.5 entry dossier (machine-readable)", "certificates/P14_dossier.json"),
    ("Milestone ledger (p14)", "ledgers/P14_milestones.json"),
    ("P.5 entry: quantum object certificate", "certificates/P14_quantum_object.json"),
    ("P.5 entry: prescription and D11 matching certificate", "certificates/P14_prescription_D11_matching.json"
  +> ),
    ("P.5 entry: counterterm domain certificate", "certificates/P14_counterterm_domain.json"),
    ("P.5 entry: amplitude rules certificate", "certificates/P14_amplitude_rules.json"),
    ("P.5 entry: operational DRED rules certificate (p14r rules; p15 R5 correction; p16 F3 hardening)", "certi
  +> ficates/P14_dred_rules.json"),
    ("Symbolic integrand D01 (p14r export; evaluated in p16)", "exports/P14_integrands/D01.json"),
    ("Symbolic integrand D03 (p14r export; evaluated in p16)", "exports/P14_integrands/D03.json"),
    ("Symbolic integrand D13 (p14r export; evaluated in p15)", "exports/P14_integrands/D13.json"),
    ("Symbolic integrand D14a (p14r export; evaluated in p15)", "exports/P14_integrands/D14a.json"),
```

```
  ("Symbolic integrand D14b (p14r export; evaluated in p15)", "exports/P14_integrands/D14b.json"),
  ("Symbolic integrand D14c (p14r export; evaluated in p15)", "exports/P14_integrands/D14c.json"),
  ("Symbolic integrand D02 (p14r export; evaluated in p15)", "exports/P14_integrands/D02.json"),
  ("Symbolic integrand D12 (p14r export; evaluated in p15)", "exports/P14_integrands/D12.json"),
  ("Boundary identity certificate (C3)", "certificates/P13_boundary_identity.json"),
  ("Quartic verification certificate", "certificates/P13_quartic_verified.json"),
  ("BRST derivation certificate", "certificates/P13_brst_derivation.json"),
  ("Source action certificate", "certificates/P13_source_action.json"),
  ("Gauge-fixing map certificate", "certificates/P13_gauge_fixing_map.json"),
  ("Master equation certificate", "certificates/P13_master_kappa2.json"),
  ("ch source audit certificate", "certificates/P13_ch_source_audit.json"),
  ("ch dependency rows ledger", "ledgers/P13_ch_dependency_rows.json"),
  ("Generated graph inventory", "ledgers/P13_graph_inventory.json"),
  ("V4\\_grav export", "exports/V4_grav.json"),
  ("ell4 conventions export", "exports/ell4_conventions.json"),
  ("Source: conventions.py", "code/classical/conventions.py"),
  ("Source: boundary\\_identity.py (C3)", "code/classical/boundary_identity.py"),
  ("Source: jetpoly.py", "code/classical/jetpoly.py"),
  ("Source: certio.py (certificate serialization + invariants)", "code/classical/certio.py"),
  ("Source: grav\\_noether.py", "code/classical/grav_noether.py"),
  ("Source: graded\\_jets.py", "code/classical/graded_jets.py"),
  ("Source: bv\\_model.py", "code/classical/bv_model.py"),
  ("Source: brst\\_from\\_geometry.py", "code/classical/brst_from_geometry.py"),
  ("Source: source\\_action.py", "code/classical/source_action.py"),
  ("Source: gauge\\_fixing\\_map.py", "code/classical/gauge_fixing_map.py"),
  ("Source: planewave\\_exact.py", "code/classical/planewave_exact.py"),
  ("Source: series\\_eval.py", "code/classical/series_eval.py"),
  ("Source: quartic.py", "code/classical/quartic.py"),
  ("Source: ell4\\_full\\_polynomial.py (builder library)", "code/classical/ell4_full_polynomial.py"),
  ("Source: inverse\\_metric\\_series.py", "code/classical/inverse_metric_series.py"),
  ("Source: master\\_kappa2.py", "code/classical/master_kappa2.py"),
  ("Source: ch\\_source\\_audit.py", "code/p4_classical/ch_source_audit.py"),
  ("Source: release.py (frozen p13 classical gate; forked verbatim by release\\_p14)", "code/classical/relea
+> se.py"),
  ("Source: dossier.py (frozen p13 dossier library)", "code/classical/dossier.py"),
  ("Source: inherited.py (pinned inherited inputs)", "code/classical/inherited.py"),
  ("Source: tools/source\\_manifest.py (source set, source-only zip)", "tools/source_manifest.py"),
  ("Source: tools/rebuild\\_from\\_source.py (rebuild entrypoint)", "tools/rebuild_from_source.py"),
  ("REBUILD\\_INSTRUCTIONS.md", "REBUILD_INSTRUCTIONS.md"),
  ("Source: tools/build.py", "tools/build.py"),
  ("Source: tools/manifest.py", "tools/manifest.py"),
  ("Source: p14\\_conventions.py (P.5 entry contract constants)", "code/p5_entry/p14_conventions.py"),
  ("Source: freeze\\_p13.py (p13 freeze F1-F3)", "code/p5_entry/freeze_p13.py"),
  ("Source: amplitude\\_rules.py", "code/p5_entry/amplitude_rules.py"),
  ("Source: counterterm\\_domain.py", "code/p5_entry/counterterm_domain.py"),
  ("Source: quantum\\_object.py", "code/p5_entry/quantum_object.py"),
  ("Source: prescription\\_d11.py", "code/p5_entry/prescription_d11.py"),
  ("Source: dred.py (operational DRED rules, p14r; disclosed revisions: p15 structural R5 safeguard, p16 F3
+> hardening, p17 G2 guarded loader parse\\_d, p18 H3/H5 documentation only, p22 runtime d guard -- additive
+>  block)", "code/p5_entry/dred.py"),
  ("Source: hessian\\_route.py (W6 independent regeneration, p14r)", "code/p5_entry/hessian_route.py"),
  ("Source: release\\_p14.py (p14 release gate)", "code/p5_entry/release_p14.py"),
  ("Source: dossier\\_p14.py (P.5 entry dossier)", "code/p5_entry/dossier_p14.py"),
  ("Source: tests/test\\_p14.py", "tests/test_p14.py"),
  ("Source: p15\\_conventions.py (p15 constants, contract pins)", "code/p5_calc/p15_conventions.py"),
  ("Source: integrals.py (p15 integral library, S0)", "code/p5_calc/integrals.py"),
  ("Source: batchA.py (p15 Batch A)", "code/p5_calc/batchA.py"),
  ("Source: d13.py (p15 Batch B: D13)", "code/p5_calc/d13.py"),
  ("Source: release\\_p15.py (p15 release gate)", "code/p5_calc/release_p15.py"),
  ("Source: tests/test\\_p15.py", "tests/test_p15.py"),
  ("Source: p16\\_conventions.py (p16 constants, version label, pins)", "code/p5_calc/p16_conventions.py"),
  ("Source: covariant.py (p16 covariant route: exact covariant integrand, FP and PV loop routes)", "code/p5_
+> calc/covariant.py"),
  ("Source: e2\\_evanescent.py (p16 E2 decoupling proof)", "code/p5_calc/e2_evanescent.py"),
  ("Source: batchC.py (p16 Batch C: D01, D03)", "code/p5_calc/batchC.py"),
  ("Source: release\\_p16.py (p16 release gate)", "code/p5_calc/release_p16.py"),
  ("Source: tests/test\\_p16.py", "tests/test_p16.py"),
  ("Source: p17\\_conventions.py (p17 constants, version label, pins, disclosures)", "code/p5_calc/p17_conve
+> ntions.py"),
  ("Source: hh\\_slot.py (p17 D01/D03 evanescent poles vs the ch slot: proof, controls, mutations)", "code/p
+> 5_calc/hh_slot.py"),
  ("Source: release\\_p17.py (p17 release gate)", "code/p5_calc/release_p17.py"),
  ("Source: tests/test\\_p17.py", "tests/test_p17.py"),
  ("Source: p18\\_conventions.py (p18 constants, version label, pins, disclosures)", "code/p5_calc/p18_conve
+> ntions.py"),
  ("Source: d11.py (p18 M2 discharge, D11 evaluation, full hh slot)", "code/p5_calc/d11.py"),
  ("Source: release\\_p18.py (p18 release gate)", "code/p5_calc/release_p18.py"),
  ("Source: tests/test\\_p18.py", "tests/test_p18.py"),
  ("Source: p19\\_conventions.py (p19 constants, version label, pins, disclosures)", "code/p5_calc/p19_conve
+> ntions.py"),
  ("Source: defect.py (p19 layer, disclosed p20 revision: K1 orientation fix, full sigma-0 closure, restorat
```

```
  +> ion redefined)", "code/p5_calc/defect.py"),
    ("Source: release\\_p19.py (p19 layer gate, disclosed p20 revision)", "code/p5_calc/release_p19.py"),
    ("Source: tests/test\\_p19.py", "tests/test_p19.py"),
    ("Source: p20\\_conventions.py (p20 constants, version label, pins, disclosures)", "code/p5_calc/p20_conve
  +> ntions.py"),
    ("Source: d11\\_pin.py (p20 D11 pin; disclosed p21 revision: absolute pin derivation)", "code/p5_calc/d11_
  +> pin.py"),
    ("Source: release\\_p20.py (p20 layer gate, disclosed p21 revision)", "code/p5_calc/release_p20.py"),
    ("Source: tests/test\\_p20.py", "tests/test_p20.py"),
    ("Source: p21\\_conventions.py (p21 constants, version label, pins, disclosures, scan allow-list)", "code/
  +> p5_calc/p21_conventions.py"),
    ("Source: release\\_p21.py (p21 layer gate, disclosed p22 revision)", "code/p5_calc/release_p21.py"),
    ("Source: tests/test\\_p21.py", "tests/test_p21.py"),
    ("Source: p22\\_conventions.py (p22 constants, version label, disclosures, guard plants)", "code/p5_calc/p
  +> 22_conventions.py"),
    ("Source: evanescent.py (p22 EV-M2 and evanescent counterterms)", "code/p5_calc/evanescent.py"),
    ("Source: contract\\_registry.py (p22 registry of the post-p18 contract revision)", "code/p5_calc/contract
  +> _registry.py"),
    ("Source: release\\_p22.py (p22 layer gate, disclosed p23 revision)", "code/p5_calc/release_p22.py"),
    ("Source: tests/test\\_p22.py", "tests/test_p22.py"),
    ("Source: p23\\_conventions.py (p23 constants, disclosures, pinned EV values, poly plants)", "code/p5_calc
  +> /p23_conventions.py"),
    ("Source: power\\_counting.py (p23 0.1 requirement, power counting, P.6 entry inventory)", "code/p5_calc/p
  +> ower_counting.py"),
    ("Source: release\\_p23.py (p23 release gate)", "code/p5_calc/release_p23.py"),
    ("Source: tests/test\\_p23.py", "tests/test_p23.py"),
]


WRAP = 110


def wrap_lines(txt: str) -> str:
    """Hard-wrap for verbatim listings; continuation lines are prefixed with '  +> ' (content unchanged)."""
    out = []
    for ln in txt.splitlines():
        while len(ln) > WRAP:
            out.append(ln[:WRAP])
            ln = "  +> " + ln[WRAP:]
        out.append(ln)
    return "\n".join(out) + "\n"


def make_listings():
    if LST.exists():
        shutil.rmtree(LST)
    LST.mkdir(parents=True)
    names = []
    for title, rel in LISTINGS:
        src = ROOT / rel
        dst = LST / (rel.replace("/", "__"))
        dst.write_text(wrap_lines(ascii_clean(src.read_text())))
        names.append((title, rel, dst.name))
    return names


def listing_marker(i, n):
    return f"[listing {i:02d}/{n:02d}]"


def appendix_body(names):
    arch_notes = sorted((ROOT / "archival" / "p02_p09" / "notes").glob("*.tex"))
    lines = [r"\section{Appendix overview}",
             "All listings below are generated by \\texttt{tools/build.py} from exactly the source files execu
  +> ted and the "
             "certificate files written in this build (ASCII-sanitised copies in \\texttt{manuscript/listing\\
  +> _src}; "
             "the build verifies byte-identity after typesetting and again before zipping). Lists of scalars a
  +> re printed "
             "on one line, so a page break cannot split them; page numbers are printed as ``" + QC.VERSION_LAB
  +> EL + " | page N'' (one version variable, "
             "\\texttt{p22\\_conventions.VERSION\\_LABEL}, written to \\texttt{manuscript/version.tex}). "
             "The external manifest (with PDF/zip hashes) is written after the PDFs and is deliberately not re
  +> produced "
             "inside any PDF.", ""]
    for i, (title, rel, fn) in enumerate(names, 1):
        lines += [f"\\section{{{title}}}",
                  f"\\noindent\\texttt{{{listing_marker(i, len(names))} {rel.replace('_', chr(92) + '_')}}}",
                  f"{{\\scriptsize\\verbatiminput{{listing_src/{fn}}}}}", ""]
    lines += [r"\section{Archival material (p22): evanescent M2/CT and runtime d guard, superseded by p23}",
              r"\scopebox{Archival.}{The p22 release (\texttt{archival/p22/}) is superseded by p23. Its indepe
```

```
+> ndent "
            r"rebuild PASSED (same box; report 023e5b83\ldots), bound to tree 1bc6edff\ldots{} and zip "
            r"ade2fa5c\ldots; p23 records this as independent\_rebuild\_passed\_p22 and does not inherit it.
+>  "
            r"Findings (polynomial-algebra bypass of the d guard; EV values unchecked by the gate; title-lin
+> e "
            r"qualifier) are addressed in p23.}"]
  lines += [r"\section{Archival material (p21): small release, superseded by p22}",
            r"\scopebox{Archival.}{The p21 release (\texttt{archival/p21/}) is superseded by p22. Its indepe
+> ndent "
            r"rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p21 tr
+> ee "
            r"b835353f\ldots{} and zip a71d6b52\ldots; p22 records this as independent\_rebuild\_passed\_p21
+>  and "
            r"does not inherit it. Its findings (gate prefix exemptions; eight scan misses and the power-bas
+> e "
            r"subs hole of the d guard; stale OPEN text in the p20 section) are addressed in p22.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p21_README.md}}",
            r"\section{Archival material (p20): correction of p19, superseded by p21}",
            r"\scopebox{Archival.}{The p20 release (\texttt{archival/p20/}) is superseded by p21. Its indepe
+> ndent "
            r"rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p20 tr
+> ee "
            r"2c0aa153\ldots{} and zip ca32ce01\ldots; p21 records this as independent\_rebuild\_passed\_p20
+>  and "
            r"does not inherit it. Its findings (absolute D11 pin derivable; top-level pin status unchecked;
+>  eight "
            r"scan misses; P19 note edit not listed; misleading change-log key) are addressed in p21.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p20_README.md}}",
            r"\section{Archival material (p19): P.5 identity step, superseded by the p20 correction}",
            r"\scopebox{Archival.}{The p19 release (\texttt{archival/p19/}) is superseded by the p20 correct
+> ion "
            r"release. Its independent rebuild PASSED with findings (same box, fresh directory and venv; NOT
+>  a second "
            r"machine), bound to the p19 tree fcf90354\ldots{} and zip da924275\ldots; p20 records this as "
            r"independent\_rebuild\_passed\_p19 and does not inherit it. Finding K1 (the p19 claim that full
+>  closure "
            r"fails with the derived factor) was an orientation bookkeeping error and is retracted (erratum
+> in the "
            r"p20 note); all findings are addressed in p20.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p19_README.md}}",
            r"\section{Archival material (p18): M2 discharged, D11 evaluated, full hh slot, superseded by p1
+> 9}",
            r"\scopebox{Archival.}{The p18 release (\texttt{archival/p18/}) is superseded as a release by p1
+> 9; its "
            r"calculation code (d11, release\_p18) stays byte-identical and active as the p18 layer. Its "
            r"independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound t
+> o the p18 "
            r"tree bfab3b4b\ldots{} and zip 7c7d5451\ldots; p19 records this as independent\_rebuild\_passed
+> \_p18 "
            r"and does not inherit it. Reviewer findings I1--I3 are addressed in p19 (I1 fail-closed gate, I
+> 2 "
            r"extended scan -- a pattern check, not a proof, I3 milestone wording); no contract revision.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p18_README.md}}",
            r"\section{Archival material (p17): D01/D03 evanescent poles vs the ch slot, superseded by p18}"
+> ,
            r"\scopebox{Archival.}{The p17 release (\texttt{archival/p17/}) is superseded as a release by p1
+> 8; its "
            r"calculation code (hh\_slot, release\_p17) stays byte-identical and active as the p17 layer. It
+> s "
            r"independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound t
+> o the p17 "
            r"tree 2a5a6317\ldots{} and zip 7f4f61df\ldots; p18 records this as independent\_rebuild\_passed
+> \_p17 "
            r"and does not inherit it. Reviewer findings H1--H5 are addressed in p18 (one disclosed dred.py
+> revision, "
            r"documentation only).}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p17_README.md}}",
            r"\section{Archival material (p16): E2 decoupling proof and Batch C (D01, D03), superseded by p1
+> 7}",
            r"\scopebox{Archival.}{The p16 release (\texttt{archival/p16/}) is superseded as a release by p1
+> 7; its "
            r"calculation code (covariant route, Batch C, E2 proof, release\_p16) stays byte-identical and a
+> ctive as "
            r"the p16 layer. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a secon
+> d machine), "
            r"bound to the p16 tree 90b4a665\ldots{} and zip 56d5887d\ldots; p17 records this as "
            r"independent\_rebuild\_passed\_p16 and does not inherit it. Reviewer findings G1--G4 and G6 are
+>  addressed "
            r"in p17 (G2: one disclosed dred.py revision, the guarded loader parse\_d).}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p16_README.md}}",
```

```
            r"\section{Archival material (p15): first P.5 calculation (Batch A, D13), superseded by p16}",
            r"\scopebox{Archival.}{The p15 release (\texttt{archival/p15/}) is superseded as a release by p1
+> 6; its "
            r"calculation code (integral library, Batch A, D13, release\_p15) stays byte-identical and activ
+> e as the "
            r"p15 layer. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second ma
+> chine), "
            r"bound to the p15 tree 79f53752\ldots{} and zip 54325c1f\ldots; p16 records this as "
            r"independent\_rebuild\_passed\_p15 and does not inherit it. Reviewer findings F3--F5 are addres
+> sed in "
            r"p16 (F3: one disclosed dred.py revision).}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p15_README.md}}",
            r"\section{Archival material (p14r): the P.5-entry contract release (contract active in p15, p16
+> )}",
            r"\scopebox{Archival.}{The p14r release (\texttt{archival/p14r/}) is superseded as a release by
+> p15; its "
            r"contract code stays active as the frozen contract on which p15 evaluates. Its independent rebu
+> ild "
            r"PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p14r tree a64c
+> 0ae3\ldots "
            r"and zip 19e8bf59\ldots. ONE disclosed contract revision inside p15: the heuristic PrematureD4
+> guard of "
            r"dred.py (finding F1: a false positive and a false negative) is replaced by a structural safegu
+> ard; "
            r"released p14r values are unaffected (Pipeline.laurent was used only on toys).}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p14r_README.md}}",
            r"\section{Archival material (p14): superseded by the p14r repair}",
            r"\scopebox{Archival.}{The p14 release (\texttt{archival/p14/}) is superseded by p14r, a narrow
+> repair: "
            r"p14 stated a DRED-type prescription without executable index rules and claimed that no evanesc
+> ent "
            r"remainder arises (withdrawn); its M3 labels conflated the scalar-extension and kernel scaling
+> degrees; "
            r"it called the counterterm quotient ``essential'' directions; its W5 regeneration did not cover
+>  D01, D02, "
            r"D03, D12; and its gate-mutation test copies lacked frozen inputs. P.3/P.4 code is unchanged.}"
+> ,
            r"{\scriptsize\verbatiminput{listing_src/archival_p14_README.md}}",
            r"\section{Archival material (p13): frozen classical P.3/P.4 inputs}",
            r"\scopebox{Archival.}{The p13 release (\texttt{archival/p13/}) is superseded by p14 as a releas
+> e but its "
            r"classical P.3/P.4 code is kept byte-identical and is the frozen P.5 input. The archive holds t
+> he p13 "
            r"source-only zip (tree 2b1baf33\ldots, zip 2f85a566\ldots), its reference values, its author re
+> build "
            r"report and the independent rebuild report (passed; SAME box with a fresh venv and a source-onl
+> y build, "
            r"NOT a second machine). The p14 freeze check re-verifies these pins on every release.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p13_README.md}}",
            r"\section{Archival material (p12): NOT p14 evidence}",
            r"\scopebox{Archival.}{The p12 release (\texttt{archival/p12/}) is superseded by p13, which adds
+>  no "
            r"mathematics. p12 used ``classical readiness'' without the milestone labels fixed in p13 (P.3,
+> P.4 = "
            r"classical readiness, P.5 entry/calculation/identity), had no completion dossier, and had no so
+> urce-only "
            r"rebuild contract or rebuild flags.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p12_README.md}}",
            r"\section{Archival material (p11): NOT p14 evidence}",
            r"\scopebox{Archival.}{The p11 release (\texttt{archival/p11/}) is superseded. Its certificate F
+> ILES were "
            r"internally consistent (all Noether residual counts zero), but its appendix typeset multi-line
+> JSON arrays: "
            r"a page break inside \texttt{Q\_terms} placed the footer page number (12, 29) between array ele
+> ments, so a "
            r"text extraction read [0,0,120,0] and [290,0,0,0]. p12 prints scalar arrays on one line, prefix
+> es page "
            r"numbers, enforces the count/flag invariant before serialization and in the release gate, cross
+> -checks the "
            r"quartic and master records, and gates every pdf/zip/sync stage.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p11_README.md}}",
            r"\section{Archival material (p10): NOT p14 evidence}",
            r"\scopebox{Archival.}{The p10 release (\texttt{archival/p10/}) is superseded and is not p14 evi
+> dence: its C3 "
            r"was vacuous and stated the wrong orientation $+\partial_\mu B^\mu$; it cited samples as proof;
+>  its master "
            r"certificate carried an unconditional pass; its multidegree lemma and polarization statement we
+> re imprecise. "
            r"Its note is preserved verbatim in \texttt{archival/p10/notes/} and is not typeset here.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_p10_README.md}}",
            r"\section{Archival material (p02--p09): NOT p14 evidence; superseded milestone wording}",
```

```
            r"\scopebox{Archival.}{Everything in this section is superseded material from revisions p02--p09
  +> , kept "
            r"for provenance only (\texttt{archival/p02\_p09/}). It is not read by the p14 pipeline and is n
  +> ot "
            r"acceptance evidence. HISTORY ONLY: its milestone wording (``P-next.N'', and the table row ``P4
  +>  coupled "
            r"one-loop restoration'' in \texttt{P\_scope.tex}) is superseded by the p13 definitions kept in
  +> p14 (P.4 = classical "
            r"readiness, NOT coupled one-loop restoration; see the milestone section of the science volume).
  +>  The archived \texttt{ell4\_verify.py} and \texttt{ell4\_generator.py} state the "
            r"WRONG boundary orientation ($+\partial_\mu B^\mu$) and a vacuous C3; see "
            r"\texttt{P13\_boundary\_identity.json}.}",
            r"{\scriptsize\verbatiminput{listing_src/archival_README.md}}",
            r"\begingroup\let\section\subsection\let\subsection\subsubsection\let\subsubsection\paragraph"]
    for n in arch_notes:
        lines.append(f"\\input{{../archival/p02_p09/notes/{n.name}}}")
    lines.append(r"\endgroup")
    (LST / "archival_p21_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p21" / "README.md
  +> ").read_text())))
    (LST / "archival_p20_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p20" / "README.md
  +> ").read_text())))
    (LST / "archival_p19_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p19" / "README.md
  +> ").read_text())))
    (LST / "archival_p18_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p18" / "README.md
  +> ").read_text())))
    (LST / "archival_p17_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p17" / "README.md
  +> ").read_text())))
    (LST / "archival_p16_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p16" / "README.md
  +> ").read_text())))
    (LST / "archival_p15_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p15" / "README.md
  +> ").read_text())))
    (LST / "archival_p14r_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p14r" / "README.
  +> md").read_text())))
    (LST / "archival_p14_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p14" / "README.md
  +> ").read_text())))
    (LST / "archival_p13_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p13" / "README.md
  +> ").read_text())))
    (LST / "archival_p12_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p12" / "README.md
  +> ").read_text())))
    (LST / "archival_p11_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p11" / "README.md
  +> ").read_text())))
    (LST / "archival_p10_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p10" / "README.md
  +> ").read_text())))
    (LST / "archival_README.md").write_text(wrap_lines(ascii_clean((ROOT / "archival" / "p02_p09" / "README.md
  +> ").read_text())))
    (MS / "body" / "XVI-P-appendix-body.tex").write_text("\n".join(lines) + "\n")


FRONT = {
    "XVI-P-science.tex": ("Landmark XVI-P --- Scientific manuscript\\\\\n\\large Science volume (p23: O.1 requ
  +> irement recorded, one-loop power counting, P.6 entry inventory (kappa\\textsuperscript{3} three-point, no
  +> t evaluated); P.5 calculation renormalized at one loop kappa\\textsuperscript{2} for the two-point functi
  +> ons with QdS external legs; bulk symmetry not claimed; O.1 NOT\\_CLOSED)",
                          "\\noindent\\textbf{Science volume.} Certificates and listings: \\texttt{XVI-P-appen
  +> dix.pdf}; "
                          "combined: \\texttt{XVI-P.pdf}.\n\n\\tableofcontents\n\\newpage\n\\input{body/XVI-P-
  +> body.tex}"),
    "XVI-P-appendix.tex": ("Landmark XVI-P --- Appendix\\\\\n\\large Certificates, listings, archival section
  +> (p23)",
                           "\\tableofcontents\n\\newpage\n\\input{body/XVI-P-appendix-body.tex}"),
    "XVI-P.tex": ("Landmark XVI-P --- Combined release\\\\\n\\large (p23: O.1 requirement recorded, one-loop p
  +> ower counting, P.6 entry inventory (kappa\\textsuperscript{3} three-point, not evaluated); P.5 calculatio
  +> n renormalized at one loop kappa\\textsuperscript{2} for the two-point functions with QdS external legs;
  +> bulk symmetry not claimed; O.1 NOT\\_CLOSED)",
                "\\tableofcontents\n\\newpage\n\\part{Science}\n\\input{body/XVI-P-body.tex}\n\\newpage\n"
                "\\part{Appendix}\n\\input{body/XVI-P-appendix-body.tex}"),
}


def write_version():
    """F4 (p16): the footer label comes from ONE variable, the VERSION_LABEL of the current conventions (p23).
  +> """
    assert re.fullmatch(r"XVI-P p\d+[a-z]?", QC.VERSION_LABEL) and QC.VERSION_LABEL.endswith(QC.REVISION)
    (MS / "version.tex").write_text("% generated by tools/build.py from code/p5_calc/p23_conventions.py VERSIO
  +> N_LABEL\n"
                                    "\\newcommand{\\XVIPversion}{" + QC.VERSION_LABEL + "}\n")


def write_fronts():
    write_version()
    for fn, (title, body) in FRONT.items():
```

```
        (MS / fn).write_text("\\documentclass{article}\n\\input{preamble.tex}\n\n"
                             f"\\title{{{title}}}\n\\author{{XVI-P {QC.REVISION}}}\n\\date{{{QC.DATE}}}\n\n"
                             "\\begin{document}\n\\maketitle\n" + body + "\n\\end{document}\n")


def build_pdfs():
    for stem in ("XVI-P-science", "XVI-P-appendix", "XVI-P"):
        for ext in (".aux", ".toc", ".out", ".log", ".pdf"):
            (MS / (stem + ext)).unlink(missing_ok=True)
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", stem + ".tex"],
                               cwd=str(MS), capture_output=True, text=True, timeout=900)
            if r.returncode != 0:
                print(r.stdout[-3000:])
                raise SystemExit(f"pdflatex failed for {stem}")
        LOG.append({"stage": f"pdf {stem}", "pages": pdf_pages(MS / (stem + ".pdf"))})
        print(f"[pdf] {stem} pages={pdf_pages(MS / (stem + '.pdf'))}", flush=True)
    assert not any("MANIFEST" in p.name for p in LST.glob("*")), "manifest must not be listed in a PDF"


def assert_no_hash_in_pdfs():
    """After the manifest: no PDF/zip sha256 from the manifest occurs in the text of any PDF."""
    hashes = re.findall(r"sha256=([0-9a-f]{64})", (ROOT / "RELEASE_MANIFEST_EXTERNAL.txt").read_text())
    found = []
    for stem in ("XVI-P-science", "XVI-P-appendix", "XVI-P"):
        txt = subprocess.run(["pdftotext", str(MS / (stem + ".pdf")), "-"], capture_output=True, text=True).st
  +> dout
        txt = re.sub(r"\s+", "", txt)
        found += [f"{stem}:{h[:12]}" for h in hashes if h in txt]
    assert not found, found
    return True


def check_listings_current():
    for title, rel in LISTINGS:
        cur = wrap_lines(ascii_clean((ROOT / rel).read_text()))
        if (LST / rel.replace("/", "__")).read_text() != cur:
            raise SystemExit(f"listing stale: {rel}")


EXCL_DIRS = {"__pycache__", "dist", ".pytest_cache", "stale"}
EXCL_SUFFIX = {".aux", ".log", ".toc", ".out", ".pkl", ".pyc", ".zip"}  # no nested zips


def _walk(base: Path):
    for p in sorted(base.rglob("*")):
        if p.is_dir() or any(part in EXCL_DIRS for part in p.relative_to(ROOT).parts):
            continue
        if p.suffix in EXCL_SUFFIX or p.name in ("RELEASE_MANIFEST_EXTERNAL.txt", "PUBLICATION_REJECTED.txt"):
            continue
        if p.relative_to(ROOT).parts[:2] in (("build", "p10"), ("build", "p13"), ("build", "p14")) and p.suffi
  +> x == ".log":
            continue
        yield p


def make_zips():
    DIST.mkdir(exist_ok=True)
    focus = ["README.md", "REBUILD_INSTRUCTIONS.md", "RELEASE_STATUS.json", "manuscript/XVI-P-science.pdf",
             "manuscript/XVI-P-appendix.pdf", "manuscript/XVI-P.pdf", "manuscript/notes/P16_E2_and_batchC.tex"
  +> ,
             "manuscript/body/P16-results-table.tex", "ledgers/P16_milestones.json", "tests/test_p16.py",
             "manuscript/notes/P17_hh_slot.tex", "manuscript/body/P17-results-table.tex", "ledgers/P17_milesto
  +> nes.json",
             "tests/test_p17.py", "manuscript/notes/P18_D11.tex", "manuscript/body/P18-results-table.tex",
             "ledgers/P18_milestones.json", "tests/test_p18.py", "manuscript/notes/P19_defect.tex",
             "manuscript/body/P19-results-table.tex", "ledgers/P19_milestones.json", "tests/test_p19.py",
             "manuscript/notes/P20_correction.tex", "certificates/P20_D11_pin.json", "ledgers/P20_milestones.j
  +> son",
             "tests/test_p20.py", "manuscript/notes/P21_absolute_pin.tex", "ledgers/P21_milestones.json",
             "tests/test_p21.py", "manuscript/notes/P22_evanescent.tex", "ledgers/P22_milestones.json",
             "tests/test_p22.py", "certificates/P22_evanescent.json", "manuscript/notes/P23_01_scope.tex",
             "ledgers/P23_milestones.json", "tests/test_p23.py", "certificates/P23_01_scope.json",
             "archival/p22/README.md", "archival/p21/README.md",
             "archival/p21/reference_values_p21.json",
             "archival/p21/reproducibility/independent_rebuild_report_p21.json",
             "archival/p21/reproducibility/author_rebuild_report_p21.json", "archival/p20/README.md", "archiva
  +> l/p20/reference_values_p20.json",
             "archival/p20/reproducibility/independent_rebuild_report_p20.json",
             "archival/p20/reproducibility/author_rebuild_report_p20.json",
             "archival/p19/README.md", "archival/p19/reference_values_p19.json",
```

```
            "archival/p19/reproducibility/independent_rebuild_report_p19.json",
            "archival/p19/reproducibility/author_rebuild_report_p19.json",
            "archival/p18/README.md", "archival/p18/reference_values_p18.json",
            "archival/p18/reproducibility/independent_rebuild_report_p18.json",
            "archival/p18/reproducibility/author_rebuild_report_p18.json", "archival/p17/README.md",
            "archival/p17/reference_values_p17.json",
            "archival/p17/reproducibility/independent_rebuild_report_p17.json",
            "archival/p17/reproducibility/author_rebuild_report_p17.json", "archival/p16/README.md", "archiva
  +> l/p16/reference_values_p16.json",
            "archival/p16/reproducibility/independent_rebuild_report_p16.json",
            "archival/p16/reproducibility/author_rebuild_report_p16.json",
            "manuscript/preamble.tex", "manuscript/version.tex",
            "archival/p15/README.md", "archival/p15/reference_values_p15.json",
            "archival/p15/reproducibility/independent_rebuild_report_p15.json",
            "archival/p15/reproducibility/author_rebuild_report_p15.json",
            "manuscript/notes/P15_first_calculation.tex",
            "manuscript/body/P15-results-table.tex", "ledgers/P15_milestones.json", "tests/test_p15.py",
            "archival/p14r/README.md", "archival/p14r/reference_values_p14r.json",
            "archival/p14r/reproducibility/independent_rebuild_report_p14r.json",
            "archival/p14r/reproducibility/author_rebuild_report_p14r.json",
            "manuscript/notes/P14_P5_entry_contract.tex",
            "manuscript/notes/P14_frozen_classical_record.tex", "manuscript/body/P14-dossier-table.tex",
            "manuscript/body/P14-entry-summary.tex", "ledgers/P13_ch_dependency_rows.json",
            "ledgers/P13_graph_inventory.json", "ledgers/P14_milestones.json", "exports/V4_grav.json",
            "exports/ell4_generating_polynomial.srepr", "exports/ell3_generating_polynomial.srepr",
            "exports/ell2_generating_polynomial.srepr", "exports/ell4_conventions.json", "tools/build.py",
            "tools/manifest.py", "tools/source_manifest.py", "tools/rebuild_from_source.py", "tests/test_p14.
  +> py",
            "build/p14/test_results/e2e_mutations.json", "build/p14/pdf_readback.json", "build/p14/pdf_inputs
  +> .json",
            "build/p14/junit.xml", SM.REFERENCE_FILE, SM.PINS_FILE, "exports/P14_vertex_tensors.json",
            "archival/p13/README.md", "archival/p13/reference_values_p13.json", "archival/p13/frozen_p13_inpu
  +> ts.json",
            "archival/p13/reproducibility/independent_rebuild_report_p13.json",
            "archival/p13/reproducibility/author_rebuild_report_p13.json", "archival/p12/README.md", "archiva
  +> l/p11/README.md", "archival/p10/README.md",
            "code/p4_classical/ch_source_audit.py", "code/p5_entry/dred.py", "code/p5_entry/hessian_route.py"
  +> ,
            "certificates/P14_dred_rules.json", "archival/p14/README.md"]
    for opt in (QC.AUTHOR_REPORT, QC.INDEPENDENT_REPORT_CURRENT):
        if (ROOT / opt).exists():
            focus.append(opt)
    focus += [str(p.relative_to(ROOT)) for p in sorted((ROOT / "certificates").glob("P1[3456]_*.json"))]
    focus += [str(p.relative_to(ROOT)) for p in sorted((ROOT / "exports" / "P14_integrands").glob("*.json"))]
    focus += [str(p.relative_to(ROOT)) for p in sorted((ROOT / "code" / "classical").glob("*.py"))]
    focus += [str(p.relative_to(ROOT)) for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))]
    focus += [str(p.relative_to(ROOT)) for p in sorted((ROOT / "code" / "p5_calc").glob("*.py"))]
    focus = [f for f in focus if (ROOT / f).exists()]
    out = {}
    for name, files in (("XVI-P-review-focus.zip", focus),
                        ("XVI-P-full.zip", [str(p.relative_to(ROOT)) for p in _walk(ROOT)])):
        zp = DIST / name
        zp.unlink(missing_ok=True)
        with zipfile.ZipFile(zp, "w", zipfile.ZIP_DEFLATED) as z:
            for rel in sorted(set(files)):
                info = zipfile.ZipInfo("XVI-P/" + rel, date_time=(2026, 10, 8, 0, 0, 0))
                info.compress_type = zipfile.ZIP_DEFLATED
                info.external_attr = 0o644 << 16
                z.writestr(info, (ROOT / rel).read_bytes())
        out[name] = len(set(files))
        print(f"[zip] {name} files={out[name]} bytes={zp.stat().st_size}", flush=True)
    src = SM.make_zip(DIST / SRC_ZIP, ROOT)
    out[SRC_ZIP] = src["files"]
    print(f"[zip] {SRC_ZIP} files={src['files']} bytes={src['bytes']} (source only; tools/source_manifest.py)"
  +> , flush=True)
    return out


MANIFEST_FILES = ("manuscript/XVI-P-science.pdf", "manuscript/XVI-P-appendix.pdf", "manuscript/XVI-P.pdf",
                  "dist/XVI-P-review-focus.zip", "dist/XVI-P-full.zip", f"dist/{SRC_ZIP}",
                  "RELEASE_STATUS.json", "certificates/P23_release.json", "ledgers/P23_milestones.json",
                  "certificates/P23_test_results.json", "certificates/P23_01_scope.json",
                  "certificates/P22_release.json", "ledgers/P22_milestones.json", "certificates/P22_evanescent
  +> .json",
                  "certificates/P21_release.json", "ledgers/P21_milestones.json",
                  "certificates/P20_release.json", "ledgers/P20_milestones.json", "certificates/P20_D11_pin.js
  +> on",
                  "certificates/P19_release.json", "ledgers/P19_milestones.json", "certificates/P19_defect.jso
  +> n",
                  "certificates/P18_release.json", "ledgers/P18_milestones.json",
                  "certificates/P18_D11_M2.json", "certificates/P18_D11.json", "certificates/P18_hh_slot.json"
```

```
  +> ,
                  "certificates/P17_release.json", "ledgers/P17_milestones.json", "certificates/P17_hh_slot.js
  +> on",
                  "certificates/P16_release.json", "ledgers/P16_milestones.json",
                  "certificates/P16_E2_evanescent.json", "certificates/P16_batchC.json",
                  "certificates/P15_release.json", "ledgers/P15_milestones.json",
                  "certificates/P15_integral_library.json", "certificates/P15_batchA.json", "certificates/P15_
  +> D13.json",
                  "certificates/P14_release.json", "certificates/P14_closure_release.json",
                  "certificates/P14_dossier.json", "ledgers/P14_milestones.json",
                  "certificates/P14_quantum_object.json", "certificates/P14_prescription_D11_matching.json",
                  "certificates/P14_counterterm_domain.json", "certificates/P14_amplitude_rules.json",
                  "certificates/P14_dred_rules.json",
                  "exports/P14_vertex_tensors.json",
                  "exports/ell2_generating_polynomial.srepr", "exports/ell3_generating_polynomial.srepr",
                  "exports/ell4_generating_polynomial.srepr", "exports/V4_grav.json",
                  "REBUILD_INSTRUCTIONS.md", SM.REFERENCE_FILE,
                  "reproducibility/inherited_data_pins.json", QC.AUTHOR_REPORT)
# pinned p13 inputs: their sha256 are printed in the PDFs on purpose (p14_conventions listing), so they are NO
  +> T hash
# entries of this manifest (whose hashes must never occur inside a PDF); they are listed by path, verified by
  +> F1-F3.
PINNED_P13 = ("archival/p13/reference_values_p13.json", "archival/p13/frozen_p13_inputs.json",
              "archival/p13/XVI-P-source-only_p13.zip", "archival/p13/reproducibility/independent_rebuild_repo
  +> rt_p13.json")


def write_manifest():
    rs = json.loads((ROOT / "RELEASE_STATUS.json").read_text())
    tr = json.loads((ROOT / "certificates" / f"{QC.CERT_PREFIX}_test_results.json").read_text())
    cl = rs["classical"]
    lines = [f"XVI-P RELEASE_MANIFEST_EXTERNAL -- revision {QC.REVISION} (contract {QC.CONTRACT_REVISION})",
             f"date: {QC.DATE}",
             f"XVI-M_sha256: {rs['XVI_M_sha256']}  pin_ok: {rs['XVI_M_pin_ok']}",
             f"execution_pass: {rs['execution_pass']}", f"claim_status: {rs['claim_status']}",
             f"release_claim_accepted: {rs['release_claim_accepted']}", f"publication: {rs['publication']}",
             f"scope: {rs['scope']}", ""]
    lines += [f"milestone {k}: {v}" for k, v in rs["milestones"]["status"].items()]
    lines += [f"library_certified: {rs['library_certified']}", f"batchA_evaluated: {rs['batchA_evaluated']}",
              f"D13_evaluated: {rs['D13_evaluated']}", f"batchC_evaluated: {rs['batchC_evaluated']}",
              f"E2_status: {rs['E2_status']}  (E2_decoupling_proven: {rs['E2_decoupling_proven']})",
              f"hh_evanescent_status: {rs['hh_evanescent_status']}  (hh_evanescent_decoupling_proven: "
              f"{rs['hh_evanescent_decoupling_proven']}; evanescent_hh_counterterm_added: "
              f"{rs['evanescent_hh_counterterm_added']})",
              f"hh_full_row_decoupling_proven: {rs['hh_full_row_decoupling_proven']}",
              f"D11_evaluated: {rs['D11_evaluated']}", f"D12_status: {rs['D12_status']}",
              f"D11_matching_verified_scope: {rs['D11_matching_verified_scope']}  (evanescent components: "
              f"{rs['D11_evanescent_components_matching']})",
              f"M3_status: {rs['M3_status']}",
              f"quantum_inputs_evaluated: {rs['quantum_inputs_evaluated']}  (inventory: "
              + "; ".join(f"{x['graph']} [{x['slot_row']}] {x['status']}" for x in rs['quantum_inputs_inventor
  +> y']) + ")",
              f"projected_identity_verified: {rs['projected_identity_verified']}",
              f"projected_restoration_verified: {rs['projected_restoration_verified']}",
              f"projected_defect_computed: {rs['projected_defect_computed']}",
              f"bulk_symmetry_restored: {rs['bulk_symmetry_restored']}",
              f"counterterm_solve: {rs['counterterm_solve']['outcome']}  (rank P_ch sigma_0 "
              f"{rs['counterterm_solve']['rank_P_ch_sigma0_routeG']}; {rs['counterterm_solve']['solution']})",
              f"why_zero: {rs['why_zero']}",
              f"rests_on: {'; '.join(rs['rests_on'])}"]
    lines += [f"flag_definition {k}: {v}" for k, v in rs["flag_definitions"].items()]
    lines += [f"finding {k}: {v}" for k, v in rs["findings"].items()]
    lines += [f"graph {k}: {v.get('status')}" for k, v in rs["graphs"].items()]
    lines += [f"contract_revision_disclosed: {', '.join(rs['contract_identity']['changed_vs_p14r'])} "
              f"(p15: finding F1 of the p14r independent rebuild; p16: finding F3 of the p15 independent rebui
  +> ld; "
              f"p17: findings G1-G3 of the p16 independent rebuild; p18: H3/H5 of the p17 independent rebuild
  +> "
              f"(documentation only); p19: none; p20: none; p21: none; p22: runtime d guard, one additive bloc
  +> k (contract_registry); p23: polynomial-domain refusal, one appended block; "
              f"released values unaffected); new contract files: {rs['contract_identity']['added'] or 'none'}"
  +> ]
    cl22 = rs["change_log_vs_p21"]
    lines += [f"change_log_vs_p21 {k}: {', '.join(cl22[k]) or 'none'}" for k in ("added", "modified", "removed
  +> ")]
    ev = rs["evanescent"]
    lines += [f"EV_M2: {ev['EV_M2_verdict']}: {ev['EV_M2']}",
              f"D11_matching_verified_scope (p22): {rs['D11_matching_verified_scope']}",
              f"evanescent counterterm coefficients (hh sum): {ev['coefficients_hh_sum']}; K'.c E2: "
              f"{ev['coefficient_Kc_E2']}; cb.c SE: {ev['coefficient_ccb_SE']}",
              f"evanescent checks: {ev['check']}",
```

```
            f"Q4S trace remnant: {ev['remnants']['O_eps_times_pole_Q4S_trace_external']}",
            f"P5_calculation_status: {rs['P5_calculation_status']}",
            f"d_runtime_guard: ok {rs['d_runtime_guard']['ok']}; plants {rs['d_runtime_guard']['n_plants']};
+>  "
            f"not raised {rs['d_runtime_guard']['not_raised']}; entry points {rs['d_runtime_guard']['entry_p
+> oints']}",
            f"d_runtime_guard residue: {rs['d_runtime_guard']['residue_not_interceptable']}",
            f"evanescent_M2_verified: {rs['evanescent_M2_verified']}",
            f"evanescent_counterterms_verified: {rs['evanescent_counterterms_verified']}",
            f"d_runtime_guard_verified: {rs['d_runtime_guard_verified']}"]
  o1 = rs["O1"]
  lines += [f"O1_requirement (p23): {o1['requirement']}", f"O1_closable_in_XVI_P: {o1['closable_in_XVI_P']}
+> ({o1['reason']})",
            f"power_counting (p23): vertex rule {rs['power_counting']['vertex_rule']}; one-loop kappa^E "
            f"{rs['power_counting']['kappa_equals_E']}; frozen E2 match {rs['power_counting']['frozen_E2_mat
+> ch']['equal']}; "
            f"enlarge rule triggered {rs['power_counting']['identity_at_kappa2']['enlarge_rule_triggered']}"
+> ,
            f"P6_entry (p23): {rs['P6_entry']['n_graphs']} graphs {rs['P6_entry']['by_external']}; {rs['P6_e
+> ntry']['status']}",
            f"d_poly_domain_refusal (p23): {rs['d_poly_domain_refusal']['ok']}; not raised {rs['d_poly_domai
+> n_refusal']['not_raised']}",
            f"EV_values (p23): {all(v['ok'] for v in rs['EV_values']['values'].values())}",
            f"title_qualifier (p23): {QC.TITLE_QUALIFIER}"]
  cl23 = rs["change_log_vs_p22"]
  lines += [f"change_log_vs_p22 {k}: {', '.join(cl23[k]) or 'none'}" for k in ("added", "modified", "removed
+> ")]
  lines += [f"erratum_p19 (p20 layer) {k}: {v}" for k, v in rs["p20_layer_erratum_p19"].items()]
  ap = rs["p21_layer_absolute_pin"]["absolute_pin"]
  lines += [f"D11_absolute_pin: status {ap['status']}; package Gamma2 pole per kappa^2/eps "
            f"{ap['derivation_p21']['package_Gamma2_pole_per_kappa2_over_eps']}; heat-kernel prediction "
            f"{ap['derivation_p21']['heat_kernel_prediction_per_kappa2_over_eps']}; external convention choi
+> ces: "
            f"{ap['derivation_p21']['convention_choices_external']}",
            f"D11_ratio_pin_verified: {rs['D11_ratio_pin_verified']}",
            f"D11_absolute_pin_verified: {rs['D11_absolute_pin_verified']}", f"open_pins: {rs['open_pins']}"
+> ,
            f"runtime_guard_limitation_p21 (CLOSED in p22): {Q21.RUNTIME_GUARD_LIMITATION_P21}"]
  p13i, p14ri, p15i = rs["p13_independent_rebuild"], rs["p14r_independent_rebuild"], rs["p15_independent_reb
+> uild"]
  p16i, p17i, p18i = rs["p16_independent_rebuild"], rs["p17_independent_rebuild"], rs["p18_independent_rebui
+> ld"]
  lines += [f"P5_entry_earned: {rs['P5_entry_earned']}"]
  lines += [f"P5_entry_flag {k}: {v}" for k, v in rs["P5_entry_flags"].items()]
  lines += [f"P3_P4_frozen_as_P5_inputs: {rs['P3_P4_frozen_as_P5_inputs']}  (bound to p13 source tree "
            f"{p13i.get('bound_to_source_tree_sha256')})",
            f"independent_rebuild_passed_p13: {rs['independent_rebuild_passed_p13']}  caveat: {p13i.get('cav
+> eat')}",
            f"independent_rebuild_passed_p14r: {rs['independent_rebuild_passed_p14r']}  (bound to p14r tree
+> "
            f"{p14ri.get('bound_to_source_tree_sha256')} / zip {p14ri.get('bound_to_source_zip_sha256')}; "
            f"caveat: {p14ri.get('caveat')})",
            f"independent_rebuild_passed_p15: {rs['independent_rebuild_passed_p15']}  (bound to p15 tree "
            f"{p15i.get('bound_to_source_tree_sha256')} / zip {p15i.get('bound_to_source_zip_sha256')}; "
            f"caveat: {p15i.get('caveat')})",
            f"independent_rebuild_passed_p16: {rs['independent_rebuild_passed_p16']}  (bound to p16 tree "
            f"{p16i.get('bound_to_source_tree_sha256')} / zip {p16i.get('bound_to_source_zip_sha256')}; "
            f"caveat: {p16i.get('caveat')})",
            f"independent_rebuild_passed_p17: {rs['independent_rebuild_passed_p17']}  (bound to p17 tree "
            f"{p17i.get('bound_to_source_tree_sha256')} / zip {p17i.get('bound_to_source_zip_sha256')}; "
            f"caveat: {p17i.get('caveat')})",
            f"independent_rebuild_passed_p18: {rs['independent_rebuild_passed_p18']}  (bound to p18 tree "
            f"{p18i.get('bound_to_source_tree_sha256')} / zip {p18i.get('bound_to_source_zip_sha256')}; "
            f"caveat: {p18i.get('caveat')})",
            f"independent_rebuild_passed_p19: {rs['p20_layer_independent_rebuild_passed_p19']}  (bound to p1
+> 9 tree "
            f"{rs['p20_layer_p19_independent_rebuild'].get('bound_to_source_tree_sha256')} / zip "
            f"{rs['p20_layer_p19_independent_rebuild'].get('bound_to_source_zip_sha256')}; "
            f"caveat: {rs['p20_layer_p19_independent_rebuild'].get('caveat')})",
            f"independent_rebuild_passed_p20: {rs['p21_layer_independent_rebuild_passed_p20']}  (bound to p2
+> 0 tree "
            f"{rs['p21_layer_p20_independent_rebuild'].get('bound_to_source_tree_sha256')} / zip "
            f"{rs['p21_layer_p20_independent_rebuild'].get('bound_to_source_zip_sha256')}; "
            f"caveat: {rs['p21_layer_p20_independent_rebuild'].get('caveat')})",
            f"independent_rebuild_passed_p21: {rs['independent_rebuild_passed_p21']}  (bound to p21 tree "
            f"{rs['p21_independent_rebuild'].get('bound_to_source_tree_sha256')} / zip "
            f"{rs['p21_independent_rebuild'].get('bound_to_source_zip_sha256')}; "
            f"caveat: {rs['p21_independent_rebuild'].get('caveat')})",
            f"{QC.REVISION} author_rebuild_passed: {rs['author_rebuild_passed']}",
            f"{QC.REVISION} independent_rebuild_passed: {rs['independent_rebuild_passed']}",
            f"D11_matching_verified: {rs['D11_matching_verified']}",
```

```
            f"source_tree_sha256: {SM.tree_sha(ROOT)}", ""]
    lines += [f"classical {k}: {v}" for k, v in cl["flags"].items()]
    lines += [f"classical_readiness_for_ch_identity: {cl['classical_readiness_for_ch_identity']}",
            f"required_master_residuals: {cl['required_master_residuals']}"]
    lines += [f"{k}: {v}" for k, v in rs["hard_stops"].items()]
    lines += [f"tests: {tr['summary']}", ""]
    lines += [f"pinned p13 input (pin in code/p5_entry/p14_conventions.py; verified by the freeze F1-F3): {rel
  +> }"
            f"  present={(ROOT / rel).exists()}" for rel in PINNED_P13]
    lines += ["", "Files (written after the PDFs and zips; no PDF contains any of these hashes):"]
    for rel in MANIFEST_FILES:
        if (ROOT / rel).exists():
            lines.append(file_line(ROOT, rel))
    man = ROOT / "RELEASE_MANIFEST_EXTERNAL.txt"
    man.write_text("\n".join(lines) + "\n")
    v = verify(man, ROOT)
    assert v["pass"], v
    v["no_pdf_hash_inside_pdfs"] = assert_no_hash_in_pdfs()
    return v


def sync_top():
    pairs = [(MS / "XVI-P-science.pdf", TOP / "XVI-P-science.pdf"), (MS / "XVI-P-appendix.pdf", TOP / "XVI-P-a
  +> ppendix.pdf"),
            (MS / "XVI-P.pdf", TOP / "XVI-P.pdf"), (DIST / "XVI-P-review-focus.zip", TOP / "XVI-P-review-focu
  +> s.zip"),
            (DIST / "XVI-P-full.zip", TOP / "XVI-P-full.zip"), (DIST / SRC_ZIP, TOP / SRC_ZIP)]
    TOP.mkdir(parents=True, exist_ok=True)
    out = {}
    for s, d in pairs:
        shutil.copyfile(s, d)
        assert sha(s) == sha(d)
        out[d.name] = {"bytes": d.stat().st_size, "pages": pdf_pages(d) if d.suffix == ".pdf" else None, "sha2
  +> 56": sha(d)}
    return out


def collect_tests():
    d = ROOT / "build" / "p14" / "test_results"
    res = {p.stem: json.loads(p.read_text()) for p in sorted(d.glob("*.json"))} if d.exists() else {}
    junit = ROOT / "build" / "p14" / "pytest_summary.txt"
    summ = junit.read_text().strip() if junit.exists() else "not run"
    (ROOT / "certificates" / f"{QC.CERT_PREFIX}_test_results.json").write_text(certio.dumps(
        {"revision": QC.REVISION, "summary": summ, "groups": junit_groups(), "results": res}))
    return summ


def junit_groups(path=None):
    """Per-group outcomes from the pytest junit XML: baseline / certificate_mutation / resumed_build."""
    import xml.etree.ElementTree as ET
    path = Path(path) if path else ROOT / "build" / "p14" / "junit.xml"
    if not path.exists():
        return {}
    groups = {"baseline": {"passed": 0, "failed": 0, "skipped": 0},
              "certificate_mutation": {"passed": 0, "failed": 0, "skipped": 0},
              "resumed_build": {"passed": 0, "failed": 0, "skipped": 0}}
    for tc in ET.parse(path).getroot().iter("testcase"):
        n = tc.get("name", "")
        g = ("resumed_build" if n.startswith("test_resumed_stage_build_blocked") else
             "certificate_mutation" if n.startswith(("test_certificate_mutation", "test_e2e_", "test_stale_ell
  +> 4_hash",
                                                     "test_p5_entry_gate_mutation", "test_amplitude_rules
  +> _and_mutations"))
             else "baseline")
        st = ("failed" if tc.find("failure") is not None or tc.find("error") is not None else
              "skipped" if tc.find("skipped") is not None else "passed")
        groups[g][st] += 1
    return groups


# ---------------- p12/p13/p14: fresh clean, publication gate, rejection, readback ----------------
GENERATED = (["RELEASE_STATUS.json", "RELEASE_MANIFEST_EXTERNAL.txt", "PUBLICATION_REJECTED.txt",
              "certificates/RELEASE_STATUS.json", "exports/V4_grav.json", "exports/ell4_conventions.json",
              "manuscript/body/P13-dossier-table.tex", "manuscript/body/P14-dossier-table.tex",
              "manuscript/body/P14-entry-summary.tex", "manuscript/body/XVI-P-appendix-body.tex",
              "exports/P14_vertex_tensors.json", "reproducibility/reference_values_p15.candidate.json",
              "reproducibility/reference_values_p16.candidate.json", "manuscript/body/P15-results-table.tex",
              "manuscript/body/P16-results-table.tex", "manuscript/version.tex",
              "reproducibility/reference_values_p17.candidate.json", "manuscript/body/P17-results-table.tex",
              "reproducibility/reference_values_p18.candidate.json", "manuscript/body/P18-results-table.tex",
              "reproducibility/reference_values_p19.candidate.json", "manuscript/body/P19-results-table.tex",
```

```
             "reproducibility/reference_values_p20.candidate.json",
             "reproducibility/reference_values_p21.candidate.json",
             "reproducibility/reference_values_p22.candidate.json",
             "reproducibility/reference_values_p23.candidate.json"]
            + [f"exports/ell{k}_generating_polynomial.srepr" for k in (2, 3, 4)]
            + [f"manuscript/{s}.pdf" for s in PDF_STEMS] + [f"dist/{z}" for z in ZIPS])


def clean_fresh():
    """Full build: remove every generated P13/P14 artifact first, so nothing shown or shipped can be a stale c
  +> opy.
    (reproducibility/*rebuild_report*.json are records of rebuilds of the SOURCE and are kept; the p13
    reproducibility/reference_values.json and everything under archival/ are never touched.)"""
    for pat in ("certificates/P13_*.json", "ledgers/P13_*.json", "certificates/P14_*.json", "ledgers/P14_*.jso
  +> n",
                "certificates/P15_*.json", "ledgers/P15_*.json", "certificates/P16_*.json", "ledgers/P16_*.jso
  +> n",
                "certificates/P17_*.json", "ledgers/P17_*.json", "certificates/P18_*.json", "ledgers/P18_*.jso
  +> n",
                "certificates/P19_*.json", "ledgers/P19_*.json",
                "certificates/P20_*.json", "ledgers/P20_*.json",
                "certificates/P21_*.json", "ledgers/P21_*.json",
                "certificates/P22_*.json", "ledgers/P22_*.json",
                "certificates/P23_*.json", "ledgers/P23_*.json"):
        for f in ROOT.glob(pat):
            f.unlink()
    for rel in GENERATED:
        assert not rel.startswith(("archival/", "reproducibility/reference_values.json",
                                   "reproducibility/independent_rebuild_report"))
        (ROOT / rel).unlink(missing_ok=True)
    shutil.rmtree(ROOT / "exports" / "P14_integrands", ignore_errors=True)
    shutil.rmtree(ROOT / "build" / "p14", ignore_errors=True)
    shutil.rmtree(ROOT / "build" / "p13", ignore_errors=True)   # p13 build outputs: archived in archival/p13/
  +> build
    shutil.rmtree(LST, ignore_errors=True)
    shutil.rmtree(STALE, ignore_errors=True)
    (ROOT / "build" / "p14").mkdir(parents=True, exist_ok=True)
    LOG.append({"stage": "clean_fresh"})


def reject_artifacts(stage, reasons):
    """Rejected build: existing PDFs/zips/manifest cannot pass as accepted artifacts; nothing is synced."""
    STALE.mkdir(exist_ok=True)
    moved = []
    for rel in [f"manuscript/{s}.pdf" for s in PDF_STEMS] + [f"dist/{z}" for z in ZIPS] + ["RELEASE_MANIFEST_E
  +> XTERNAL.txt"]:
        f = ROOT / rel
        if f.exists():
            dst = STALE / ("REJECTED__" + rel.replace("/", "__"))
            shutil.move(str(f), str(dst))
            moved.append(str(dst.relative_to(ROOT)))
    msg = {"publication": "REJECTED", "stage": stage, "reasons": reasons, "moved_to_stale": moved,
           "synced_to_top": False, "revision": QC.REVISION}
    (STALE / "REJECTED.json").write_text(json.dumps(msg, indent=1) + "\n")
    (ROOT / "PUBLICATION_REJECTED.txt").write_text(json.dumps(msg, indent=1) + "\n")
    (ROOT / "build" / "p14").mkdir(parents=True, exist_ok=True)
    (ROOT / "build" / "p14" / "build_log.json").write_text(json.dumps({"log": LOG, "rejected": msg}, indent=1)
  +> )
    print("[gate] PUBLICATION REJECTED", json.dumps(msg), flush=True)
    raise SystemExit(3)


def listed_files():
    return [rel for _, rel in LISTINGS]


def file_sha(rel):
    return hashlib.sha256((ROOT / rel).read_bytes()).hexdigest()


def zips_current():
    bad = []
    for z in ZIPS:
        zp = DIST / z
        if not zp.exists():
            bad.append(f"missing {z}")
            continue
        if z == SRC_ZIP:
            bad += [f"{z}:{b}" for b in SM.zip_matches_tree(zp, ROOT)]
            continue
        try:
```

```
            zf = zipfile.ZipFile(zp)
        except zipfile.BadZipFile:
            bad.append(f"{z}: not a valid zip")
            continue
        with zf:
            for n in zf.namelist():
                rel = n.split("/", 1)[1]
                if rel.startswith(("certificates/", "exports/", "ledgers/", "code/", "tools/", "tests/", "manu
  +> script/",
                                   "reproducibility/")) or rel == "RELEASE_STATUS.json":
                    f = ROOT / rel
                    if not f.exists() or zf.read(n) != f.read_bytes():
                        bad.append(f"{z}:{rel}")
    return bad


def publication_gate(stage):
    """Re-invoke the release gate and the dossier and check current artifact consistency; reject (and stop) on
  +>  any
    failure.  Fail closed: an exception inside the checks is itself a rejection."""
    try:
        reasons = _gate_reasons(stage)
    except Exception as exc:  # noqa: BLE001
        reasons = [f"gate evaluation error: {type(exc).__name__}: {exc}"]
    if reasons:
        reject_artifacts(stage, reasons)
    (ROOT / "PUBLICATION_REJECTED.txt").unlink(missing_ok=True)
    print(f"[gate] before {stage}: accepted", flush=True)
    return True


def _gate_reasons(stage):
    reasons = []
    r = subprocess.run([PY, "code/p5_entry/release_p14.py"], cwd=str(ROOT), capture_output=True, text=True, ti
  +> meout=900)
    LOG.append({"stage": f"gate_before_{stage}", "release_rc": r.returncode})
    if r.returncode != 0:
        reasons.append(f"release_p14.py exit {r.returncode}")
    try:
        rs = json.loads((ROOT / "certificates" / "P14_release.json").read_text())      # the p14r CONTRACT rec
  +> ord
    except Exception as exc:  # noqa: BLE001
        rs = {}
        reasons.append(f"P14_release (contract record) unreadable: {exc}")
    cl = rs.get("classical", {})
    if not (rs.get("release_claim_accepted") is True and cl.get("classical_readiness_for_ch_identity") is True
            and cl.get("release_claim_accepted") is True and rs.get("publication") == "accepted"):
        reasons.append("release_claim_accepted/readiness not true")
    if not (rs.get("P5_entry_earned") is True and all(v is True for v in rs.get("P5_entry_flags", {}).values()
  +> )
            and len(rs.get("P5_entry_flags", {})) == len(PC.P5_ENTRY_FLAGS)):
        reasons.append("P5 entry not earned (all five contract flags required)")
    if rs.get("P3_P4_frozen_as_P5_inputs") is not True:
        reasons.append("p13 freeze (P3_P4_frozen_as_P5_inputs) not true")
    for k, v in PC.HARD_STOPS.items():
        if rs.get("hard_stops", {}).get(k) != v:
            reasons.append(f"hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
    reasons += [f"failed_prerequisite:{x}" for x in cl.get("failed_prerequisites", [])]
    if not (ROOT / "certificates" / "P14_closure_release.json").exists():
        reasons.append("closure artifact absent")
    d = subprocess.run([PY, "code/p5_entry/dossier_p14.py"], cwd=str(ROOT), capture_output=True, text=True, ti
  +> meout=600)
    LOG.append({"stage": f"dossier_before_{stage}", "rc": d.returncode})
    if d.returncode != 0:
        reasons.append(f"dossier links unresolved: {d.stdout.strip()[-400:]}")
    r15 = subprocess.run([PY, "code/p5_calc/release_p15.py"], cwd=str(ROOT), capture_output=True, text=True, t
  +> imeout=900)
    LOG.append({"stage": f"p15_gate_before_{stage}", "release_rc": r15.returncode})
    if r15.returncode != 0:
        reasons.append(f"release_p15.py exit {r15.returncode}")
    try:
        rs = json.loads((ROOT / "certificates" / "P15_release.json").read_text())       # the p15 LAYER record
    except Exception as exc:  # noqa: BLE001
        rs = {}
        reasons.append(f"P15_release (p15 layer record) unreadable: {exc}")
    if not (rs.get("revision") == Q15.REVISION and rs.get("publication") == "accepted"
            and rs.get("release_claim_accepted") is True and rs.get("batchA_evaluated") is True
            and rs.get("D13_evaluated") is True and rs.get("library_certified") is True
            and rs.get("hard_stops_hold") is True):
        reasons.append("p15 layer not accepted")
    for k, v in Q15.HARD_STOPS_P15.items():
```

```
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p15 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r16 = subprocess.run([PY, "code/p5_calc/release_p16.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=900)
  LOG.append({"stage": f"p16_gate_before_{stage}", "release_rc": r16.returncode})
  if r16.returncode != 0:
      reasons.append(f"release_p16.py exit {r16.returncode}: {r16.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "certificates" / "P16_release.json").read_text())       # the p16 layer record
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"P16_release (p16 layer record) unreadable: {exc}")
  if not (rs.get("revision") == Q16.REVISION and rs.get("publication") == "accepted"
          and rs.get("release_claim_accepted") is True and rs.get("batchA_evaluated") is True
          and rs.get("D13_evaluated") is True and rs.get("batchC_evaluated") is True
          and rs.get("E2_decoupling_proven") is True and rs.get("library_certified") is True
          and rs.get("D11_evaluated") is False and rs.get("hard_stops_hold") is True):
      reasons.append("p16 release not accepted")
  for k, v in Q16.HARD_STOPS_P16.items():
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p16 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r17 = subprocess.run([PY, "code/p5_calc/release_p17.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=900)
  LOG.append({"stage": f"p17_gate_before_{stage}", "release_rc": r17.returncode})
  if r17.returncode != 0:
      reasons.append(f"release_p17.py exit {r17.returncode}: {r17.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "certificates" / "P17_release.json").read_text())       # the p17 layer record
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"P17_release (p17 layer record) unreadable: {exc}")
  if not (rs.get("revision") == Q17.REVISION and rs.get("publication") == "accepted"
          and rs.get("release_claim_accepted") is True and rs.get("batchC_evaluated") is True
          and rs.get("E2_decoupling_proven") is True and rs.get("hh_evanescent_decoupling_proven") is True
          and rs.get("evanescent_hh_counterterm_added") is False
          and rs.get("D11_evaluated") is False and rs.get("hard_stops_hold") is True):
      reasons.append("p17 release not accepted")
  for k, v in Q17.HARD_STOPS_P17.items():
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p17 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r18 = subprocess.run([PY, "code/p5_calc/release_p18.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=900)
  LOG.append({"stage": f"p18_gate_before_{stage}", "release_rc": r18.returncode})
  if r18.returncode != 0:
      reasons.append(f"release_p18.py exit {r18.returncode}: {r18.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "certificates" / "P18_release.json").read_text())       # the p18 layer record
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"P18_release (p18 layer record) unreadable: {exc}")
  if not (rs.get("revision") == Q18.REVISION and rs.get("publication") == "accepted"
          and rs.get("quantum_inputs_evaluated") == "earned"
          and rs.get("release_claim_accepted") is True and rs.get("D11_matching_verified") is True
          and rs.get("D11_evaluated") is True and rs.get("hh_full_row_decoupling_proven") is True
          and rs.get("evanescent_hh_counterterm_added") is False
          and rs.get("projected_identity_verified") == "not_earned"
          and rs.get("projected_restoration_verified") == "not_earned"
          and rs.get("bulk_symmetry_restored") is False and rs.get("hard_stops_hold") is True):
      reasons.append("p18 release not accepted")
  for k, v in Q18.HARD_STOPS_P18.items():
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p18 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r19 = subprocess.run([PY, "code/p5_calc/release_p19.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=900)
  LOG.append({"stage": f"p19_gate_before_{stage}", "release_rc": r19.returncode})
  if r19.returncode != 0:
      reasons.append(f"release_p19.py exit {r19.returncode}: {r19.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "certificates" / "P19_release.json").read_text())       # the p19 layer record
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"P19_release (p19 layer record) unreadable: {exc}")
  if not (rs.get("revision") == Q19.REVISION and rs.get("publication") == "accepted"       # I1: fail closed
          and rs.get("release_claim_accepted") is True and rs.get("quantum_inputs_evaluated") == "earned"
          and rs.get("projected_defect_computed") == "earned"
          and rs.get("bulk_symmetry_restored") is False and rs.get("symmetry_restored") is False
          and rs.get("O1_closure") == "NOT_CLOSED" and rs.get("hard_stops_hold") is True
          and (rs.get("projected_restoration_verified") != "earned" or rs.get("projected_identity_verified")
+>  == "earned")):
      reasons.append("p19 release not accepted")
  for k, v in Q19.HARD_STOPS_P19.items():
```

```
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p19 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r20 = subprocess.run([PY, "code/p5_calc/release_p20.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=900)
  LOG.append({"stage": f"p20_gate_before_{stage}", "release_rc": r20.returncode})
  if r20.returncode != 0:
      reasons.append(f"release_p20.py exit {r20.returncode}: {r20.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "certificates" / "P20_release.json").read_text())       # the p20 layer record
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"P20_release (p20 layer record) unreadable: {exc}")
  if not (rs.get("revision") == Q20.REVISION and rs.get("publication") == "accepted"
          and rs.get("release_claim_accepted") is True and rs.get("quantum_inputs_evaluated") == "earned"
          and rs.get("projected_defect_computed") == "earned"
          and rs.get("bulk_symmetry_restored") is False and rs.get("symmetry_restored") is False
          and rs.get("O1_closure") == "NOT_CLOSED" and rs.get("hard_stops_hold") is True
          and rs.get("retraction_check", {}).get("hits") == {}
          and (rs.get("projected_restoration_verified") != "earned" or rs.get("projected_identity_verified")
+>  == "earned")):
      reasons.append("p20 release not accepted")
  for k, v in Q20.HARD_STOPS_P20.items():
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p20 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r21 = subprocess.run([PY, "code/p5_calc/release_p21.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=900)
  LOG.append({"stage": f"p21_gate_before_{stage}", "release_rc": r21.returncode})
  if r21.returncode != 0:
      reasons.append(f"release_p21.py exit {r21.returncode}: {r21.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "certificates" / "P21_release.json").read_text())       # the p21 layer record
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"P21_release (p21 layer record) unreadable: {exc}")
  if not (rs.get("revision") == Q21.REVISION and rs.get("publication") == "accepted"
          and rs.get("release_claim_accepted") is True and rs.get("quantum_inputs_evaluated") == "earned"
          and rs.get("D11_absolute_pin_verified") == "earned" and rs.get("bulk_symmetry_restored") is False
          and rs.get("symmetry_restored") is False and rs.get("O1_closure") == "NOT_CLOSED"
          and rs.get("hard_stops_hold") is True and rs.get("lower_layers", {}).get("rechecked_live") is True
          and rs.get("change_log_vs_p20", {}).get("undisclosed_package_changes") == []):
      reasons.append("p21 release not accepted")
  for k, v in Q21.HARD_STOPS_P21.items():
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p21 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r22 = subprocess.run([PY, "code/p5_calc/release_p22.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=1800)
  LOG.append({"stage": f"p22_gate_before_{stage}", "release_rc": r22.returncode})
  if r22.returncode != 0:
      reasons.append(f"release_p22.py exit {r22.returncode}: {r22.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "certificates" / "P22_release.json").read_text())       # the p22 layer record
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"P22_release (p22 layer record) unreadable: {exc}")
  if not (rs.get("revision") == Q22.REVISION and rs.get("publication") == "accepted"
          and rs.get("release_claim_accepted") is True and rs.get("quantum_inputs_evaluated") == "earned"
          and all(rs.get(k) == "earned" for k in Q22.EARNABLE_P22) and rs.get("bulk_symmetry_restored") is F
+> alse
          and rs.get("symmetry_restored") is False and rs.get("O1_closure") == "NOT_CLOSED"
          and rs.get("hard_stops_hold") is True and rs.get("change_log_vs_p21", {}).get("undisclosed_changes
+> ") == []):
      reasons.append("p22 release not accepted")
  for k, v in Q22.HARD_STOPS_P22.items():
      if rs.get("hard_stops", {}).get(k) != v:
          reasons.append(f"p22 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
  r23 = subprocess.run([PY, "code/p5_calc/release_p23.py"], cwd=str(ROOT), capture_output=True, text=True, t
+> imeout=1800)
  LOG.append({"stage": f"p23_gate_before_{stage}", "release_rc": r23.returncode})
  if r23.returncode != 0:
      reasons.append(f"release_p23.py exit {r23.returncode}: {r23.stdout.strip()[-400:]}")
  try:
      rs = json.loads((ROOT / "RELEASE_STATUS.json").read_text())                     # the p23 status
  except Exception as exc:  # noqa: BLE001
      rs = {}
      reasons.append(f"RELEASE_STATUS unreadable: {exc}")
  if not (rs.get("revision") == QC.REVISION and rs.get("publication") == "accepted"
          and rs.get("release_claim_accepted") is True and all(rs.get(k) == "earned" for k in QC.EARNABLE_P2
+> 3)
          and rs.get("bulk_symmetry_restored") is False and rs.get("symmetry_restored") is False
          and rs.get("O1_closure") == "NOT_CLOSED" and rs.get("hard_stops_hold") is True
          and rs.get("change_log_vs_p22", {}).get("undisclosed_changes") == []):
```

```
        reasons.append("p23 release not accepted")
    for k, v in QC.HARD_STOPS_P23.items():
        if rs.get("hard_stops", {}).get(k) != v:
            reasons.append(f"p23 hard stop violated: {k}={rs.get('hard_stops', {}).get(k)!r}")
    for c in ("P15_release.json", "P16_release.json", "P17_release.json", "P18_release.json", "P19_release.jso
  +> n",
              "P20_release.json", "P21_release.json", "P22_release.json", f"{QC.CERT_PREFIX}_release.json"):
        if not (ROOT / "certificates" / c).exists():
            reasons.append(f"release certificate absent: {c}")
    for f in sorted(list((ROOT / "certificates").glob("P[12][0-9]_*.json")) + list((ROOT / "ledgers").glob("P[
  +> 12][0-9]_*.json"))):
        try:
            bad = certio.invariant_violations(json.loads(f.read_text()))
        except Exception as exc:  # noqa: BLE001
            bad = [f"unreadable: {exc}"]
        reasons += [f"{f.name}:{b}" for b in bad]
    if stage in ("zip", "manifest"):
        pi = ROOT / "build" / "p14" / "pdf_inputs.json"
        if not pi.exists():
            reasons.append("pdf_inputs.json absent (PDFs not built by this pipeline)")
        else:
            rec = json.loads(pi.read_text())
            for rel, h in rec["listed"].items():
                if not (ROOT / rel).exists() or file_sha(rel) != h:
                    reasons.append(f"appendix listing no longer equals file: {rel}")
            for st in PDF_STEMS:
                f = MS / (st + ".pdf")
                if not f.exists() or sha(f) != rec["pdfs"].get(st):
                    reasons.append(f"pdf missing or not the gated build output: {st}")
        rb = ROOT / "build" / "p14" / "pdf_readback.json"
        if not rb.exists() or not json.loads(rb.read_text()).get("pass"):
            reasons.append("pdf readback absent or failed")
    if stage == "manifest":
        reasons += [f"zip_not_current:{b}" for b in zips_current()[:20]]
        if rs.get("author_rebuild_passed") is True:     # the flag must refer to exactly the zip being publish
  +> ed
            rep = json.loads((ROOT / QC.AUTHOR_REPORT).read_text())
            if (DIST / SRC_ZIP).exists() and rep.get("source_zip_sha256") != sha(DIST / SRC_ZIP):
                reasons.append("author_rebuild_passed refers to a different source-only zip")
    return reasons


def _text(pdf):
    return subprocess.run(["pdftotext", str(pdf), "-"], capture_output=True, text=True).stdout


# F4 (p16): value blocks read back from the PDFs in addition to the residual records.  For each certificate be
  +> low, the
# ordered list of `"key": value` snippets (exactly as serialized in the file) must occur IN ORDER inside that
# certificate's typeset listing slice, after footers ("<VERSION_LABEL> | page N") and wrap markers are strippe
  +> d.
VALUE_BLOCKS = {
    "certificates/P15_D13.json": ("result", ("structure", "A_exact_d_times_MB", "UV_pole", "MSbar_finite",
                                             "nonlocal_coefficient_of_L", "local_polynomial")),
    "certificates/P15_batchA.json": ("graphs", ("graph", "kappa_order", "prefactor", "status", "finite_part"))
  +> ,
    "certificates/P16_batchC.json": ("graphs.*.result", ("evanescent", "A_exact_d_times_MB", "UV_pole", "MSbar
  +> _finite",
                                                         "nonlocal_coefficient_of_L", "local_polynomial")),
    "certificates/P16_E2_evanescent.json": ("", ("outcome", "E2_status", "D13_slot_contribution", "E2_decoupli
  +> ng_proven")),
    "certificates/P17_hh_slot.json": ("", ("verdict", "status", "surviving_evanescent_structures_in_slot",
                                           "decoupling_proven")),
    "certificates/P18_D11_M2.json": ("", ("verdict", "D11_matching_verified")),
    "certificates/P18_D11.json": ("result", ("evanescent", "A_exact_d_times_MB", "UV_pole", "MSbar_finite",
                                             "nonlocal_coefficient_of_L", "local_polynomial")),
    "certificates/P18_hh_slot.json": ("", ("verdict", "decoupling_proven_full_row", "surviving_evanescent_stru
  +> ctures_in_slot",
                                           "D11_reopens_evanescent_question", "quantum_inputs_evaluated")),
    "certificates/P19_defect.json": ("", ("defect_zero_exact_in_d", "quotient_coordinates", "D_cap_ker_sigma0"
  +> ,
                                          "projected_defect_computed", "projected_identity_verified",
                                          "projected_restoration_verified", "bulk_symmetry_restored")),
    "certificates/P20_D11_pin.json": ("", ("c2_spin2", "c0_spin0s", "ratio_c0_over_c2", "ratio_pin_status",
                                           "absolute_pin_status", "D11_ratio_pin_verified",
                                           "D11_absolute_pin_verified")),
    "certificates/P22_evanescent.json": ("", ("evanescent_counterterm_earned", "EV_M2_status", "EV_CT_status")
  +> ),
    "certificates/P23_O1_scope.json": ("", ("O1_closure_status", "P6_entry_n_graphs", "one_point_n_graphs",
                                            "O1_closable_in_XVI_P")),
}
```

```
FOOTER_RE = re.compile(r"XVI-P p\d+[a-z]? \| page \d+")
FOOTER_LINE_RE = re.compile(r"^XVI-P p\d+[a-z]? \| page \d+$", re.M)     # a footer is a line of its own


def _snip(k, v):
    return json.dumps(k) + ": " + json.dumps(v)


def value_snippets(root, rel):
    """Ordered `"key": value` snippets of the value block of one certificate (file order)."""
    path, keys = VALUE_BLOCKS[rel]
    obj = json.loads((Path(root) / rel).read_text())
    nodes = [obj]
    for part in [x for x in path.split(".") if x]:
        nodes = [v for n in nodes for v in (n.values() if part == "*" else [n[part]])]
    out = []

    def walk(o):
        if isinstance(o, dict):
            for k, v in o.items():
                if k in keys and not isinstance(v, (dict, list)):
                    if not (isinstance(v, str) and (not v.isascii() or "'" in v or "`" in v)):
                        out.append(_snip(k, v))     # typographic-quote / non-ASCII values are not read back
                else:
                    walk(v)
        elif isinstance(o, list):
            for v in o:
                walk(v)
    for n in nodes:
        walk(n)
    return out


def _squash(t):
    return re.sub(r"\s+", "", t.replace("+>", ""))


def readback_values(text, root, rel, slice_text):
    """True iff every snippet of rel occurs in order in slice_text (footers/wrap markers/whitespace removed)."
  +> ""
    src = _squash((Path(root) / rel).read_text())
    snips = value_snippets(root, rel)
    missing_in_file = [x for x in snips if _squash(x) not in src]
    t = _squash(FOOTER_RE.sub(" ", slice_text))
    pos, miss = 0, []
    for x in snips:
        i = t.find(_squash(x), pos)
        if i < 0:
            miss.append(x)
        else:
            pos = i + len(_squash(x))
    return {"snippets": len(snips), "not_in_order_in_pdf": miss[:5], "n_missing": len(miss),
            "builder_error_not_in_file": missing_in_file[:5],
            "pass": bool(snips) and not miss and not missing_in_file}


def readback_compare(root=None):
    """Read the typeset residual records back from the appendix and combined PDFs and compare with the files
    (pure: no side effects; used by pdf_readback and by tools/rebuild_from_source.py)."""
    root = Path(root) if root else ROOT
    exp_q, exp_z = [], []
    for rel in listed_files():
        if rel.endswith(".json"):
            for _, rec in certio.residual_records(json.loads((root / rel).read_text())):
                exp_q.append(rec["Q_terms"])
                exp_z.append(bool(rec["Q_identically_zero"]))
    out = {"expected_records": len(exp_q), "expected_values": exp_q}
    ok = True
    n = len(LISTINGS)
    for st in ("XVI-P-appendix", "XVI-P"):
        raw = _text(root / "manuscript" / (st + ".pdf"))
        footers = FOOTER_LINE_RE.findall(raw)
        pages = pdf_pages(root / "manuscript" / (st + ".pdf"))
        foot = {"pages": pages, "footers": len(footers), "labels": sorted({f.rsplit(" | ", 1)[0] for f in foot
  +> ers}),
                "expected_label": QC.VERSION_LABEL}
        foot["pass"] = foot["footers"] == pages and foot["labels"] == [QC.VERSION_LABEL]
        t = re.sub(r"\s+", " ", raw)
        # scope: only the typeset certificate/ledger/export JSON listings (sliced by the listing markers)
        slices, pos, missing, vals = [], 0, [], {}
        for i, (_, rel) in enumerate(LISTINGS, 1):
```

```
            a = t.find(listing_marker(i, n), pos)
            if a < 0:
                missing.append(rel)
                continue
            b = t.find(listing_marker(i + 1, n), a) if i < n else t.find("Archival material", a)
            b = len(t) if b < 0 else b
            if rel.endswith(".json"):
                slices.append(t[a:b])
            if rel in VALUE_BLOCKS:
                vals[rel] = readback_values(t, root, rel, t[a:b])
            pos = a
        for rel in VALUE_BLOCKS:
            vals.setdefault(rel, {"pass": False, "reason": "listing not found"})
        t = " ".join(slices)
        got_q, bad = [], []
        for m in re.finditer(r'"Q_terms": ?\[([^\]]*)\]', t):
            body = m.group(1)
            if re.search(r"[A-Za-z_(]", body):
                continue          # source code, not a certificate value
            if not re.fullmatch(r" ?\d+(, ?\d+)* ?", body):
                bad.append(body)
                continue
            got_q.append([int(x) for x in body.split(",")])
        got_z = [x == "true" for x in re.findall(r'"Q_identically_zero": ?(true|false)', t)]
        res = {"Q_terms_equal": got_q == exp_q, "zero_flags_equal": got_z == exp_z, "malformed": bad,
               "read": len(got_q), "read_values": got_q, "missing_listing_markers": missing}
        res["value_blocks"] = vals
        res["footer"] = foot
        res["pass"] = (res["Q_terms_equal"] and res["zero_flags_equal"] and not bad and not missing
                       and all(v["pass"] for v in vals.values()) and foot["pass"])
        out[st] = res
        ok &= res["pass"]
    out["pass"] = ok
    return out


def pdf_readback():
    out = readback_compare()
    (ROOT / "build" / "p14" / "pdf_readback.json").write_text(json.dumps(out, indent=1) + "\n")
    print("[pdf readback]", json.dumps({k: ({"pass": v["pass"], "values": {r: x["pass"] for r, x in v.get("val
  +> ue_blocks", {}).items()},
                                             "footer": v.get("footer")} if isinstance(v, dict) else v) for k,
  +> v in out.items()
                                        if k != "expected_values"}), flush=True)
    if not out["pass"]:
        reject_artifacts("pdf_readback", [f"pdf readback mismatch: {out}"])
    return out


def record_pdf_inputs():
    rec = {"listed": {rel: file_sha(rel) for rel in listed_files()},
           "pdfs": {st: sha(MS / (st + ".pdf")) for st in PDF_STEMS}}
    (ROOT / "build" / "p14" / "pdf_inputs.json").write_text(json.dumps(rec, indent=1) + "\n")


def main():
    args = sys.argv[1:]
    t0 = time.time()
    assert inherited.sha(inherited.resolve("XVI-M.tex")) == C.XVI_M_PIN
    for sub in ("certificates", "exports", "ledgers", "dist", "manuscript/body", "build/p14"):   # absent in a
  +>  source-only tree
        (ROOT / sub).mkdir(parents=True, exist_ok=True)
    stages = ["quartic", "brst", "source", "gauge", "ch", "master", "matterloop", "p5entry", "release", "dossi
  +> er",
              "p15calc", "release15", "p16calc", "release16", "p17calc", "release17", "p18calc", "release18",
  +> "p19calc", "release19", "p20calc", "release20", "release21", "p22calc", "release22", "p23calc", "release2
  +> 3", "tests", "pdf", "zip",
              "manifest"]
    (ROOT / "build" / "p14").mkdir(parents=True, exist_ok=True)
    start = args[args.index("--from-stage") + 1] if "--from-stage" in args else "quartic"
    run = stages[stages.index(start):]
    if "quartic" in run:
        clean_fresh()
        step("conventions", [PY, "-c", "import sys; sys.path.insert(0,'code/classical'); import conventions as
  +>  C; print(C.REVISION)"])
        step("inherited_pins", [PY, "code/classical/inherited.py"])
        step("boundary_identity_C3", [PY, "code/classical/boundary_identity.py"])
        step("quartic", [PY, "code/classical/quartic.py"] + (["build/p14/ell_cache.pkl"] if "--from-cache" in
  +> args else []))
    if "brst" in run:
        step("brst_transformations", [PY, "code/classical/brst_from_geometry.py"])
```

```
  if "source" in run:
      step("source_action", [PY, "code/classical/source_action.py"])
  if "gauge" in run:
      step("gauge_map", [PY, "code/classical/gauge_fixing_map.py"])
  if "ch" in run:
      step("ch_audit_and_graphs", [PY, "code/p4_classical/ch_source_audit.py"])
  if "master" in run:
      step("master", [PY, "code/classical/master_kappa2.py"])
  if "matterloop" in run:     # regenerates the inherited O5 matter-loop records read by the D11 matching no
+> te
      step("matter_loop_gates", [PY, "tests/test_matter_loop_gates.py"])
  if "p5entry" in run:        # P.5 entry contract: symbolic rules and definitions only, NO loop evaluation
      step("p5_amplitude_rules", [PY, "code/p5_entry/amplitude_rules.py"])
      step("p5_dred_rules", [PY, "code/p5_entry/dred.py"])          # p14r: operational DRED (reads the expo
+> rts)
      step("p5_counterterm_domain", [PY, "code/p5_entry/counterterm_domain.py"])
      step("p5_quantum_object", [PY, "code/p5_entry/quantum_object.py"])
      step("p5_prescription_d11", [PY, "code/p5_entry/prescription_d11.py"])
  if "release" in run:
      r = subprocess.run([PY, "code/p5_entry/release_p14.py"], cwd=str(ROOT), capture_output=True, text=True
+> )
      LOG.append({"stage": "release_gate", "rc": r.returncode})
      print(f"[release_gate] rc={r.returncode}", flush=True)
      if r.returncode != 0:
          reject_artifacts("release", ["release_p14.py rejected publication", r.stdout[-2000:]])
  if "dossier" in run:
      r = subprocess.run([PY, "code/p5_entry/dossier_p14.py"], cwd=str(ROOT), capture_output=True, text=True
+> )
      LOG.append({"stage": "dossier", "rc": r.returncode})
      print(f"[dossier] rc={r.returncode} {r.stdout.strip()}", flush=True)
      if r.returncode != 0:
          reject_artifacts("dossier", ["dossier links unresolved", r.stdout[-2000:]])
  if "p15calc" in run:        # p15: FIRST P.5 CALCULATION on the p14r contract (Batch A, Batch B = D13)
      step("p15_integral_library", [PY, "code/p5_calc/integrals.py"])
      step("p15_batchA", [PY, "code/p5_calc/batchA.py"])
      step("p15_D13", [PY, "code/p5_calc/d13.py"])
  if "release15" in run:
      r = subprocess.run([PY, "code/p5_calc/release_p15.py"], cwd=str(ROOT), capture_output=True, text=True)
      LOG.append({"stage": "release_gate_p15", "rc": r.returncode})
      print(f"[release_gate_p15] rc={r.returncode}", flush=True)
      if r.returncode != 0:
          reject_artifacts("release15", ["release_p15.py rejected publication", r.stdout[-2000:]])
  if "p16calc" in run:        # p16: E2 evanescent decoupling proof (D13 ch slot), Batch C (D01, D03; covari
+> ant route)
      step("p16_E2_evanescent", [PY, "code/p5_calc/e2_evanescent.py"])
      step("p16_batchC", [PY, "code/p5_calc/batchC.py"], timeout=5400)
  if "release16" in run:
      r = subprocess.run([PY, "code/p5_calc/release_p16.py"], cwd=str(ROOT), capture_output=True, text=True)
      LOG.append({"stage": "release_gate_p16", "rc": r.returncode})
      print(f"[release_gate_p16] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
      if r.returncode != 0:
          reject_artifacts("release16", ["release_p16.py rejected publication", r.stdout[-2000:]])
  if "p17calc" in run:        # p17: D01/D03 evanescent poles vs the ch-projected slot (decoupling proof)
      step("p17_hh_slot", [PY, "code/p5_calc/hh_slot.py"], timeout=3600)
  if "release17" in run:
      r = subprocess.run([PY, "code/p5_calc/release_p17.py"], cwd=str(ROOT), capture_output=True, text=True)
      LOG.append({"stage": "release_gate_p17", "rc": r.returncode})
      print(f"[release_gate_p17] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
      if r.returncode != 0:
          reject_artifacts("release17", ["release_p17.py rejected publication", r.stdout[-2000:]])
  if "p18calc" in run:        # p18: M2 discharge, D11 evaluation, full hh slot
      step("p18_d11", [PY, "code/p5_calc/d11.py"], timeout=3600)
  if "release18" in run:
      r = subprocess.run([PY, "code/p5_calc/release_p18.py"], cwd=str(ROOT), capture_output=True, text=True)
      LOG.append({"stage": "release_gate_p18", "rc": r.returncode})
      print(f"[release_gate_p18] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
      if r.returncode != 0:
          reject_artifacts("release18", ["release_p18.py rejected publication", r.stdout[-2000:]])
  if "p19calc" in run:        # p19: projected ch-identity defect, Route C / Route G, counterterm solve
      step("p19_defect", [PY, "code/p5_calc/defect.py"], timeout=3600)
  if "release19" in run:
      r = subprocess.run([PY, "code/p5_calc/release_p19.py"], cwd=str(ROOT), capture_output=True, text=True)
      LOG.append({"stage": "release_gate_p19", "rc": r.returncode})
      print(f"[release_gate_p19] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
      if r.returncode != 0:
          reject_artifacts("release19", ["release_p19.py rejected publication", r.stdout[-2000:]])
  if "p20calc" in run:        # p20: D11 normalisation pin (trace-anomaly spin-2 / spin-0 ratio)
      step("p20_d11_pin", [PY, "code/p5_calc/d11_pin.py"], timeout=1800)
  if "release20" in run:
      r = subprocess.run([PY, "code/p5_calc/release_p20.py"], cwd=str(ROOT), capture_output=True, text=True)
      LOG.append({"stage": "release_gate_p20", "rc": r.returncode})
```

```
        print(f"[release_gate_p20] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
        if r.returncode != 0:
            reject_artifacts("release20", ["release_p20.py rejected publication", r.stdout[-2000:]])
    if "release21" in run:
        r = subprocess.run([PY, "code/p5_calc/release_p21.py"], cwd=str(ROOT), capture_output=True, text=True)
        LOG.append({"stage": "release_gate_p21", "rc": r.returncode})
        print(f"[release_gate_p21] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
        if r.returncode != 0:
            reject_artifacts("release21", ["release_p21.py rejected publication", r.stdout[-2000:]])
    if "p22calc" in run:        # p22: EV-M2 and the evanescent counterterms
        step("p22_evanescent", [PY, "code/p5_calc/evanescent.py"], timeout=3600)
    if "release22" in run:
        r = subprocess.run([PY, "code/p5_calc/release_p22.py"], cwd=str(ROOT), capture_output=True, text=True)
        LOG.append({"stage": "release_gate_p22", "rc": r.returncode})
        print(f"[release_gate_p22] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
        if r.returncode != 0:
            reject_artifacts("release22", ["release_p22.py rejected publication", r.stdout[-2000:]])
    if "p23calc" in run:        # p23: 0.1 requirement, one-loop power counting, P.6 entry inventory
        step("p23_power_counting", [PY, "code/p5_calc/power_counting.py"], timeout=3600)
    if "release23" in run:
        r = subprocess.run([PY, "code/p5_calc/release_p23.py"], cwd=str(ROOT), capture_output=True, text=True)
        LOG.append({"stage": "release_gate_p23", "rc": r.returncode})
        print(f"[release_gate_p23] rc={r.returncode} {r.stdout.strip()[-600:]}", flush=True)
        if r.returncode != 0:
            reject_artifacts("release23", ["release_p23.py rejected publication", r.stdout[-2000:]])
    if "tests" in run and "--skip-tests" not in args:
        shutil.rmtree(ROOT / "build" / "p14" / "test_results", ignore_errors=True)
        r = subprocess.run([PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "--junitxml=build/p14/junit.xm
  +> l",
                            "tests/test_p14.py", "tests/test_p15.py", "tests/test_p16.py", "tests/test_p17.py"
  +> , "tests/test_p18.py", "tests/test_p19.py", "tests/test_p20.py",
                            "tests/test_p21.py", "tests/test_p22.py", "tests/test_p23.py", "tests/test_inverse
  +> _metric_series.py"],
                           cwd=str(ROOT), capture_output=True, text=True)
        (ROOT / "build" / "p14" / "pytest_summary.txt").write_text(r.stdout.strip().splitlines()[-1] + "\n")
        print("[tests]", r.stdout.strip().splitlines()[-1], json.dumps(junit_groups()), flush=True)
        if r.returncode != 0:
            print(r.stdout[-5000:])
            raise SystemExit("tests failed")
    collect_tests()
    if "pdf" in run:
        publication_gate("pdf")
        names = make_listings()
        appendix_body(names)
        write_fronts()
        build_pdfs()
        check_listings_current()
        record_pdf_inputs()
        pdf_readback()
    if "zip" in run:
        publication_gate("zip")
        (ROOT / "RELEASE_MANIFEST_EXTERNAL.txt").unlink(missing_ok=True)
        make_zips()
    if "manifest" in run:
        publication_gate("manifest")
        v = write_manifest()
        top = sync_top()
        shutil.rmtree(STALE, ignore_errors=True)
        print(json.dumps({"manifest": v, "top": top}, indent=1))
    (ROOT / "build" / "p14" / "build_log.json").write_text(json.dumps({"log": LOG, "sec": round(time.time() -
  +> t0, 1)}, indent=1))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

115 Source: tools/manifest.py

[listing 82/133] tools/manifest.py

```
#!/usr/bin/env python3
"""External manifest writer/verifier (written LAST; PDFs never embed their own hashes).

Machine-checkable lines:  FILE <relpath> pages=<n|-> bytes=<n> sha256=<hex>
verify(): recompute bytes + sha256 of every FILE line; any mismatch (stale hash) => fail.
"""
from __future__ import annotations
```

```
import hashlib, re, subprocess, sys
from pathlib import Path

LINE = re.compile(r"^FILE (\S+) pages=(\S+) bytes=(\d+) sha256=([0-9a-f]{64})$")


def sha(p: Path) -> str:
    return hashlib.sha256(p.read_bytes()).hexdigest()


def pdf_pages(p: Path):
    try:
        out = subprocess.run(["pdfinfo", str(p)], capture_output=True, text=True).stdout
        m = re.search(r"^Pages:\s+(\d+)", out, re.M)
        return int(m.group(1)) if m else None
    except FileNotFoundError:
        return None


def file_line(root: Path, rel: str) -> str:
    p = root / rel
    pages = pdf_pages(p) if p.suffix == ".pdf" else None
    return f"FILE {rel} pages={pages if pages is not None else '-'} bytes={p.stat().st_size} sha256={sha(p)}"


def verify(manifest: Path, root: Path) -> dict:
    bad, n = [], 0
    for ln in manifest.read_text().splitlines():
        m = LINE.match(ln.strip())
        if not m:
            continue
        n += 1
        rel, _, b, h = m.groups()
        p = root / rel
        if not p.exists():
            bad.append(f"missing {rel}")
        elif p.stat().st_size != int(b) or sha(p) != h:
            bad.append(f"stale {rel}")
    return {"entries": n, "failures": bad, "pass": n > 0 and not bad}


if __name__ == "__main__":
    r = verify(Path(sys.argv[1]), Path(sys.argv[2]))
    print(r)
    raise SystemExit(0 if r["pass"] else 1)
```

116 Source: p14_conventions.py (P.5 entry contract constants)

[listing 83/133] code/p5_entry/p14_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p14 / p14r conventions (P.5 ENTRY; revision label p14r, certificate prefix kept P14).  The frozen p13
  +>  conventions (code/classical/conventions.py) are NOT edited:
the classical P.3/P.4 code is byte-identical to p13 (archival/p13/frozen_p13_inputs.json) so that the p13 rebu
  +> ild
evidence stays valid for it.  The classical stages therefore keep writing P13_* certificates (frozen P.3/P.4 i
  +> nputs,
regenerated by unchanged code); p14 writes P14_* certificates."""
from __future__ import annotations

REVISION = "p14r"          # narrow p14 repair (operational DRED, M3, wording, W6, gate tests)
CERT_PREFIX = "P14"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p14r.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p14r.json"
DATE = "2026-10-08"

# ---- p13 pins (the frozen P.3/P.4 inputs) ----
P13_SOURCE_TREE_SHA256 = "2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70"
P13_SOURCE_TREE_SHA256_WITHOUT_REFERENCE = "289a5d03e3adda6a63703a36bb6379ae90209ea09a4d6c533d6dd4809d1ce345"
P13_SOURCE_ZIP_SHA256 = "2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266"
P13_REFERENCE_VALUES_SHA256 = "d8bf3d97457913dbd1fa03a2e801630005aeca601b93ebb53307fb30e1bc4095"
P13_INDEPENDENT_REPORT_SHA256 = "7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4"
P13_FROZEN_LIST_SHA256 = "39f5dae88768921972769ac8f9bbcbcdc6efc317619b1f593125e34245f58e06"
# archived p13 records shipped as p14 source (records only, not freeze inputs): pinned so that reading them
# (e.g. when hashing the source tree) is a read of a pinned file
P13_ARCHIVED_RECORDS = {
    "archival/p13/reproducibility/author_rebuild_report_p13.json":
        "fa8996b41522dfbd8dceef65eac9c8bf0bb90abbd04578bf6e7136a9d9687291"}
```

```
# (archival/p13/reproducibility/inherited_data_pins_p13.json is byte-identical to reproducibility/inherited_da
  +> ta_pins.json
#  and is kept in the archive only, not in the p14 source set)
P13_ARCHIVE = "archival/p13"
P13_REFERENCE_FILE = "archival/p13/reference_values_p13.json"
P13_INDEPENDENT_REPORT = "archival/p13/reproducibility/independent_rebuild_report_p13.json"
P13_INDEPENDENT_REPORT_LIVE = "reproducibility/independent_rebuild_report.json"   # written by the reviewer; n
  +> ever written here
P13_FROZEN_LIST = "archival/p13/frozen_p13_inputs.json"
P13_SOURCE_ZIP_ARCHIVED = "archival/p13/XVI-P-source-only_p13.zip"
INDEPENDENT_CAVEAT = ("the p13 independent rebuild ran on the SAME box (shared kernel, TeX Live, poppler) in a
  +>  fresh venv "
                      "from the source-only zip; it was NOT run on a second machine")

# ---- P.5 entry contract ----
P5_ENTRY_FLAGS = ("quantum_action_defined", "quantum_prescription_defined", "D11_matching_defined",
                  "counterterm_domain_defined", "amplitude_rules_verified")
P5_ENTRY_RULE = "P.5 entry is earned iff all five contract flags hold (and is stated against the P.3/P.4 freez
  +> e)"
P5_ENTRY_QUALIFICATION = (
    "P.5 entry = the contract is fixed, nothing is evaluated. Qualifications (p14r): the DRED prescription is
  +> operational "
    "at the level of index algebra, lifting, ghat-projected tensor reduction and the order contraction -> Laur
  +> ent "
    "(tested on generic placeholders and applied to the lifted exported integrands), but no scalar integral is
  +>  "
    "evaluated and the scaleless-integral rules are not applied; D11 M3 is conditional on M2 and its degree-4
  +> form on "
    "the imposed normalization condition N4; the counterterm count 8 is a count of candidate directions modulo
  +>  the "
    "computed exact equivalences, before the identity constraints (D cap ker sigma_0 not computed); "
    "D11_matching_verified = false; D12 Missing.")

HARD_STOPS = {
    "quantum_inputs_evaluated": "not_earned",
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": False,
    "symmetry_restored": False,
    "D02_quantum_evaluated": False,
    "D12_quantum_evaluated": False,
    "D12_status": "Missing",
    "D11_matching_verified": False,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": False,
    "XIV_J_opened": False,
    "Route_B_discharged": False,
    "P5_entry_opened": True,
    "P5_calculation_opened": False,
    "P5_identity_opened": False,
    "loop_evaluations_performed": False,
    "u_R_c_delta_added": False,
}

PROJECTED_VS_BULK = ("projected (XVI-N lem:field-only; the ch projection of the fixed one-loop kappa^2 identit
  +> y) is "
                     "distinct from bulk BV restoration; the XVI-O kernel two-point Ward identity d(Pi_w + L)
  +> = 0 is "
                     "NOT functional ST4b")
```

117 Source: freeze_p13.py (p13 freeze F1-F3)

[listing 84/133] code/p5_entry/freeze_p13.py

```
#!/usr/bin/env python3
"""XVI-P p14: the P.3/P.4 freeze (P3_P4_frozen_as_P5_inputs), bound to the p13 SOURCE TREE, not to the p14 tre
  +> e.

P3_P4_frozen_as_P5_inputs := F1 AND F2 AND F3
  F1 independent_rebuild_passed (p13): the archived reviewer report archival/p13/reproducibility/
     independent_rebuild_report_p13.json has the pinned sha256, pass = true, no mismatches, and is bound to th
  +> e p13
     source tree 2b1baf33... and source-only zip 2f85a566... (the live reproducibility/independent_rebuild_rep
  +> ort.json,
     when present, must be byte-identical to the archived copy; it is never written by this package).
  F2 frozen code: every file of archival/p13/frozen_p13_inputs.json (classical P.3/P.4 code, inherited XVI-O c
```

```
  +> ode and
     notes, pins, inherited data; 54 files) is byte-identical in the current tree; the list itself has its pin
  +> ned sha
     and, where the archived p13 zip is present, equals the zip (zip sha256 pinned).
  F3 frozen values: the classical reference values recomputed now (export hashes, the 7 classical P13_* certif
  +> icate
     content hashes ignoring timings, the 2 classical ledgers, residual records, inherited pins) equal
     archival/p13/reference_values_p13.json (pinned sha).
The p13 flag cannot be inherited by p14 changes: it is bound to the p13 tree sha, and F2/F3 fail as soon as a
  +> frozen
file or a frozen value changes.  p14's own author/independent rebuild flags are separate (release_p14.py).
"""
from __future__ import annotations
import hashlib
import json
import sys
import zipfile
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
sys.path.insert(0, str(ROOT / "tools"))
import p14_conventions as PC  # noqa: E402
import certio  # noqa: E402
import inherited  # noqa: E402
import source_manifest as SM  # noqa: E402

CLASSICAL_CERTS = ("P13_boundary_identity", "P13_quartic_verified", "P13_brst_derivation", "P13_source_action"
  +> ,
                   "P13_gauge_fixing_map", "P13_ch_source_audit", "P13_master_kappa2")
CLASSICAL_LEDGERS = ("P13_ch_dependency_rows", "P13_graph_inventory")
P13_RUN_SPECIFIC = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def F1_independent(root=ROOT):
    root = Path(root)
    arch = root / PC.P13_INDEPENDENT_REPORT
    rec = {"report": PC.P13_INDEPENDENT_REPORT, "present": arch.is_file()}
    if not arch.is_file():
        return False, rec
    rec["sha256_ok"] = sha(arch) == PC.P13_INDEPENDENT_REPORT_SHA256
    rep = json.loads(arch.read_text())
    rec.update({"report_pass": rep.get("pass"), "report_independent_rebuild_passed": rep.get("independent_rebu
  +> ild_passed"),
                "mismatches": len(rep.get("mismatches", [])),
                "independent_comparisons": len(rep.get("independent_comparisons", [])),
                "compared_with_reference": (rep.get("checks", {}).get("2_entrypoint_rebuild_from_source", {})
                                            .get("comparison_with_in_zip_reference")),
                "bound_source_tree_sha256": rep.get("source_tree_sha256"),
                "bound_zip_sha256": rep.get("zip_sha256") or rep.get("source_zip_sha256"),
                "bound_to_p13_tree": rep.get("source_tree_sha256") == PC.P13_SOURCE_TREE_SHA256,
                "bound_to_p13_zip": (rep.get("zip_sha256") or rep.get("source_zip_sha256")) == PC.P13_SOURCE_Z
  +> IP_SHA256,
                "environment": rep.get("environment"), "caveat": PC.INDEPENDENT_CAVEAT})
    live = root / PC.P13_INDEPENDENT_REPORT_LIVE
    rec["live_report_present"] = live.is_file()
    rec["live_report_identical_to_archived"] = (sha(live) == sha(arch)) if live.is_file() else None
    ok = bool(rec["sha256_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and not rep.get("mismatches") and rec["bound_to_p13_tree"] and rec["bound_to_p13_zip"]
              and rec["live_report_identical_to_archived"] is not False)
    return ok, rec


def F2_frozen_code(root=ROOT):
    root = Path(root)
    lst = root / PC.P13_FROZEN_LIST
    rec = {"list": PC.P13_FROZEN_LIST, "present": lst.is_file()}
    if not lst.is_file():
        return False, rec
    rec["list_sha256_ok"] = sha(lst) == PC.P13_FROZEN_LIST_SHA256
    L = json.loads(lst.read_text())
    zp = root / PC.P13_SOURCE_ZIP_ARCHIVED
    if zp.is_file():
        rec["archived_zip_sha256_ok"] = sha(zp) == PC.P13_SOURCE_ZIP_SHA256
        with zipfile.ZipFile(zp) as z:
```

```
            names = {n.split("/", 1)[1]: n for n in z.namelist()}
            rec["list_equals_archived_zip"] = all(
                f["path"] in names and hashlib.sha256(z.read(names[f["path"]])).hexdigest() == f["sha256"]
                for f in L["files"])
    else:
        rec["archived_zip_sha256_ok"] = None          # not in a source-only copy (zips are never source)
        rec["list_equals_archived_zip"] = None
    changed = [f["path"] for f in L["files"] if not (root / f["path"]).is_file() or sha(root / f["path"]) != f
  +> ["sha256"]]
    rec.update({"files": len(L["files"]), "changed_or_missing": changed})
    ok = bool(rec["list_sha256_ok"] and not changed and rec["archived_zip_sha256_ok"] is not False
              and rec["list_equals_archived_zip"] is not False and L.get("p13_source_zip_sha256") == PC.P13_SO
  +> URCE_ZIP_SHA256)
    return ok, rec


def classical_values(root=ROOT):
    """the classical subset of tools/rebuild_from_source.reference_values (p13 definitions, verbatim key layou
  +> t)"""
    root = Path(root)
    q = json.loads((root / "certificates" / "P13_quartic_verified.json").read_text())
    m = json.loads((root / "certificates" / "P13_master_kappa2.json").read_text())
    rec = lambda r: {k: r[k] for k in ("W_terms", "Q_terms", "Q_identically_zero", "s0_piece_alone_Q_terms")}
  +>  # noqa: E731
    return {
        "inherited_pins": {**{k: v["sha256"] for k, v in inherited.PINS.items()},
                           **{k: v["sha256"] for k, v in SM.inherited_data(root).items()},
                           "XVI-O_o70_source_tree": json.loads((root / SM.XVIO_TREE).read_text())["sha256"]},
        "exports": {**{f"ell{k}": {"file_sha256": SM.sha(root / "exports" / f"ell{k}_generating_polynomial.sre
  +> pr"),
                                   "polynomial_sha256": q[f"ell{k}_sha256"],
                                   "terms": q["terms_reproducibility"][str(k)]} for k in (2, 3, 4)},
                    "V4_grav.json": SM.sha(root / "exports" / "V4_grav.json"),
                    "ell4_conventions.json": SM.sha(root / "exports" / "ell4_conventions.json")},
        "certificate_content_sha256_ignoring_timings": {
            **{c: certio.file_content_sha(root / "certificates" / f"{c}.json", P13_RUN_SPECIFIC) for c in CLAS
  +> SICAL_CERTS},
            **{f"ledgers/{l_}": certio.file_content_sha(root / "ledgers" / f"{l_}.json", P13_RUN_SPECIFIC)
               for l_ in CLASSICAL_LEDGERS}},
        "residual_records": {
            "quartic_C7": {o: rec(q["controls"]["C7_noether_exact"][o]) for o in certio.ORDERS},
            "master_grav_af0": {o: rec(m["details"]["grav_af0_deterministic"][o]) for o in certio.ORDERS},
            "master_table": m["table_by_kappa_and_antifield_number"],
            "required_master_residuals": m["required_master_residuals"]},
    }


def _flat(d, p=""):
    if isinstance(d, dict):
        out = {}
        for k, v in d.items():
            out.update(_flat(v, f"{p}/{k}"))
        return out
    return {p: d}


def F3_frozen_values(root=ROOT):
    root = Path(root)
    ref_f = root / PC.P13_REFERENCE_FILE
    rec = {"reference": PC.P13_REFERENCE_FILE, "present": ref_f.is_file()}
    if not ref_f.is_file():
        return False, rec
    rec["reference_sha256_ok"] = sha(ref_f) == PC.P13_REFERENCE_VALUES_SHA256
    refv = json.loads(ref_f.read_text())["values"]
    cur = classical_values(root)
    ref_sub = {k: (refv[k] if k != "certificate_content_sha256_ignoring_timings" else
                   {c: refv[k][c] for c in cur[k]}) for k in cur}
    a, b = _flat(ref_sub), _flat(cur)
    diffs = [k for k in sorted(set(a) | set(b)) if a.get(k, "<absent>") != b.get(k, "<absent>")]
    rec.update({"compared": len(set(a) | set(b)), "equal": len(set(a) | set(b)) - len(diffs), "differences": d
  +> iffs[:30]})
    return bool(rec["reference_sha256_ok"] and not diffs), rec


def freeze(root=ROOT):
    f1, r1 = F1_independent(root)
    f2, r2 = F2_frozen_code(root)
    f3, r3 = F3_frozen_values(root)
    return {"P3_P4_frozen_as_P5_inputs": bool(f1 and f2 and f3),
            "definition": "F1 independent_rebuild_passed(p13 tree) AND F2 frozen code byte-identical AND F3 fr
  +> ozen values equal",
```

```
            "independent_rebuild_passed_p13": f1, "frozen_code_unchanged": f2, "frozen_values_unchanged": f3,
            "bound_to": {"p13_source_tree_sha256": PC.P13_SOURCE_TREE_SHA256, "p13_source_zip_sha256": PC.P13_
  +> SOURCE_ZIP_SHA256},
            "caveat": PC.INDEPENDENT_CAVEAT,
            "F1": r1, "F2": r2, "F3": r3}


if __name__ == "__main__":
    out = freeze()
    print(json.dumps({k: out[k] for k in ("P3_P4_frozen_as_P5_inputs", "independent_rebuild_passed_p13",
                                          "frozen_code_unchanged", "frozen_values_unchanged")}, indent=1))
    print(json.dumps(out["F3"], indent=0)[:600])
```

118 Source: amplitude_rules.py

[listing 85/133] code/p5_entry/amplitude_rules.py

```
#!/usr/bin/env python3
"""XVI-P p14r Deliverable 4: executable amplitude rules for the ch-relevant one-loop graphs (INTEGRANDS ONLY).

Graphs: D01 (h^3-h^3 bubble), D02 (h^4 tadpole), D03 (ghost loop), D11 (scalar loop; reference to the inherite
  +> d XVI-O
kernel only -- P-convention integrand exported for the D11 matching identity M1), D12 (h^2 phi^2 tadpole; inte
  +> grand
exported, status stays Missing), D13 (K'.c.h + cbar.c.h; external K', c), D14a-c (one-point tadpoles).
No integral is evaluated: every record carries integrated = false.

i Gamma_1^{(ext)} = int d^d l/(2pi)^d  prefactor * N(l,p) / prod (q_j^2 + i0)     (Gamma_1 = -i * graph sum, G
  +> ate 0)
N is an exact rational polynomial in l_r, p_r (lower components); prefactor collects S_f, loop sign, external-
  +> order
sign, (i kappa^n) per vertex, vertex phases and the propagator i's.

Controls (all must pass for amplitude_rules_verified):
  W1 0-dim Wick/Berezin shadow (shadow.py): symmetry factors, closed-ghost-loop sign, <c cbar>, external order
  +>  (D13)
  W2 multiplicity: (P^{-1}) == de Donder tensor numerator at representatives; generator contraction (independe
  +> nt
     components) == independent 16-component tensor recomputation with V^T = V/w (D02, D14a, D13)
  W3 closed forms: cbar.c.h == 2 i a D^rho[L_c h] (all derivative terms), K'.c.h == a (L_c h), K'.c == a(dc+dc
  +> ),
     h.phi.phi == -a1(k1 k2 + k2 k1)/2, ghost kinetic == -i q^2 delta  (generic symbolic momenta, conservation
  +>  imposed)
  W4 graviton Ward identities -- PRIMARY evidence: the generic symbolic OFF-SHELL identities
     kappa^1  V3(d0 xi,e2,e3) + V2(d1(xi,e2),e3) + V2(d1(xi,e3),e2) = 0,
     kappa^2  V4(d0 xi,e2,e3,e4) + sum_i V3(d1(xi,e_i),e_j,e_k) = 0 (chunked over basis polarisations);
     supplementary: "Ward contraction with two free on-shell graviton legs and an off-shell gauge leg" (V3 wit
  +> h a pure-
     gauge leg d0 xi at the off-shell momentum -(ka+kb), the two other legs on shell with polarisations in the
  +>  kernel of
     the linearised field equation); Bose symmetry of V3, V4; fast extractor == sympy polarisation (V3 symboli
  +> c)
  W5 regeneration (covers ONLY D14a, D14b, D14c, D11 and D13): integrands regenerated by an independent sympy
  +> route ==
     exported; plus D01 Bose symmetry N_ab == N_ba (a symmetry check, not a regeneration)
  W6 independent Hessian / Tr-log route (hessian_route.py) for D01, D02, D03, D12: -1/2 Tr(G U2) + 1/2 Tr(G U1
  +>  G U1)
     (bosons), -Tr(G U1 G U1) (ghosts) from the field-dependent Hessian of S_Psi, own (h,b) inverse == integra
  +> nds
Mutations (env P14_MUTATIONS, and in-process table): wrong_tadpole_factor, wrong_ghost_loop_sign,
offdiag_multiplicity, reversed_source_ghost_order, omit_ghost_transport_term -- each must be REJECTED.
"""
from __future__ import annotations
import copy
import hashlib
import itertools
import json
import sys
import time
from fractions import Fraction as Fr
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
```

```
import feynman as F  # noqa: E402
import shadow  # noqa: E402
import conventions as C  # noqa: E402
import p14_conventions as PC  # noqa: E402
import certio  # noqa: E402

I = sp.I
HALF = sp.Rational(1, 2)
GRAPH_ORDER = ["D01", "D02", "D03", "D11", "D12", "D13", "D14a", "D14b", "D14c"]
MUTATION_NAMES = ["wrong_tadpole_factor", "wrong_ghost_loop_sign", "offdiag_multiplicity",
                  "reversed_source_ghost_order", "omit_ghost_transport_term"]
LP_NAMES = {r: f"l{r}" for r in range(4)} | {4 + r: f"p{r}" for r in range(4)}


def base_rules():
    g = {}
    for k in GRAPH_ORDER:
        g[k] = {"S_f": HALF if k in ("D01", "D02", "D11", "D12", "D14a", "D14c") else sp.Integer(1),
                "loop_sign": -1 if k in ("D03", "D14b") else 1, "ext_sign": 1,
                "ext_order": ("K", "c") if k == "D13" else ("h",) * (1 if k.startswith("D14") else 2)}
    return {"graphs": g,
            "leg_order": {"K_c_h": ("K", "c", "h"), "cb_c_h": ("cb", "c", "h"), "ghost_kinetic": ("cb", "c")},
            "graviton_contraction": "independent_components",
            "ghost_propagator": "<c^mu(q) cbar_rho(-q)> = i (M0^{-1})_{mu rho},  M0_{rho mu} = V[cbar_rho(-q),
  +>  c^mu(q)]"}


def apply_mutations(rules, muts):
    r = copy.deepcopy(rules)
    if "wrong_tadpole_factor" in muts:
        r["graphs"]["D02"]["S_f"] = sp.Integer(1)
    if "wrong_ghost_loop_sign" in muts:
        for k in ("D03", "D14b"):
            r["graphs"][k]["loop_sign"] = 1
    if "offdiag_multiplicity" in muts:
        r["graviton_contraction"] = "tensor16_on_independent_vertices"
    if "reversed_source_ghost_order" in muts:
        r["leg_order"]["K_c_h"] = ("c", "K", "h")
    return r


def rules_json(r):
    return {"graphs": {k: {"S_f": str(v["S_f"]), "loop_sign": v["loop_sign"], "ext_sign": v["ext_sign"],
                           "ext_order": list(v["ext_order"])} for k, v in r["graphs"].items()},
            "leg_order": {k: list(v) for k, v in r["leg_order"].items()},
            "graviton_contraction": r["graviton_contraction"], "ghost_propagator": r["ghost_propagator"]}


# ---------------------------------------------------------------------------------------------------
def K_vertex(rules):
    """V_K'ch with the leg order of the rule table, re-indexed to canonical (K', c, h) slots (sign retained)."
  +> ""
    order = rules["leg_order"]["K_c_h"]
    if tuple(order) == ("K", "c", "h"):
        return F.vertices()["V_K_c_h"]
    X = F.S_psi_parts()["X"][1]
    vt = F.ge_vertex("V_K_c_h[" + ",".join(order) + "]", X, [(f, 0) for f in order], 1, "X^(1) reordered")
    pos = [order.index(x) for x in ("K", "c", "h")]
    T = {}
    for comps, poly in vt.T.items():
        mp = {}
        for new, old in enumerate(pos):
            for r in range(4):
                mp[4 * old + r] = F.SP.var(4 * new + r)
        T[tuple(comps[p] for p in pos)] = poly.subs(mp)
    return F.VT(vt.name, ["K", "c", "h"], vt.phase, T, 1, vt.origin)


def grav_weights(rules):
    if rules["graviton_contraction"] == "independent_components":
        return lambda c, d: 1
    return lambda c, d: F.W_OFF[c] * F.W_OFF[d]          # MUTATION: 16-component sum on independent-comp vert
  +> ices


def pinv_sparse():
    Pi = F.propagators()["graviton"]["Pinv"]
    return {(i, j): Fr(int(Pi[i, j].p), int(Pi[i, j].q)) for i in range(10) for j in range(10) if Pi[i, j] !=
  +> 0}
```

```
def ghost_num():
    G = F.propagators()["ghost"]["numerator"]
    out = {}
    for i in range(4):
        for j in range(4):
            if G[i, j] != 0:
                v = sp.nsimplify(G[i, j])
                assert v.is_rational, "ghost numerator expected real rational"
                out[(i, j)] = Fr(int(v.p), int(v.q))
    return out


def phase_factor(*vts):
    return I ** sum(v.phase for v in vts)


def build_integrands(rules):
    V = F.vertices()
    Pi = pinv_sparse()
    w = grav_weights(rules)
    Gh = ghost_num()
    Pcols = {}
    for (c, d), v in Pi.items():
        Pcols.setdefault(d, []).append((c, v))
    kap = F.KAPPA
    R = rules["graphs"]
    out = {}

    def rec(name, prefactor, N, ext, routing, dens, extra=None):
        out[name] = {"prefactor": sp.nsimplify(prefactor), "N": N, "external": ext, "routing": routing,
                     "denominators": dens, **(extra or {})}

    # ---- D01 ----
    V3 = V["V3_grav"]
    VA = F.vt_route(V3, [(0, 1), (-1, 0), (1, -1)])      # (a,p) (c,-l) (e,l-p)
    VB = F.vt_route(V3, [(0, -1), (1, 0), (-1, 1)])      # (b,-p) (d,l) (f,p-l)
    WB = {}
    for (b, d, f), poly in VB.items():
        for c, pcd in Pcols.get(d, []):
            for e, pef in Pcols.get(f, []):
                WB.setdefault((b, c, e), F.SP()).iadd(poly, pcd * pef * w(c, d) * w(e, f))
    N = {}
    for (a, c, e), pa in VA.items():
        for b in range(10):
            wb = WB.get((b, c, e))
            if wb is not None:
                N.setdefault((a, b), F.SP()).iadd_prod(pa, wb)
    rec("D01", R["D01"]["S_f"] * R["D01"]["loop_sign"] * (I * kap) ** 2 * phase_factor(V3, V3) * I ** 2,
        N, ["h_a(p)", "h_b(-p)"],
        {"A": ["h_a: p", "h_c: -l", "h_e: l-p"], "B": ["h_b: -p", "h_d: l", "h_f: p-l"],
         "propagators": ["<h_c h_d>(l)", "<h_e h_f>(l-p)"]}, ["l^2+i0", "(l-p)^2+i0"])
    # ---- D02 ----
    V4 = V["V4_grav"]
    VA = F.vt_route(V4, [(0, 1), (0, -1), (1, 0), (-1, 0)])
    N = {}
    for (a, b, c, d), poly in VA.items():
        pcd = Pi.get((c, d))
        if pcd:
            N.setdefault((a, b), F.SP()).iadd(poly, pcd * w(c, d))
    rec("D02", R["D02"]["S_f"] * R["D02"]["loop_sign"] * (I * kap ** 2) * phase_factor(V4) * I, N,
        ["h_a(p)", "h_b(-p)"], {"V4": ["h_a: p", "h_b: -p", "h_c: l", "h_d: -l"], "propagators": ["<h_c h_d>(l
  +> )"]},
        ["l^2+i0"])
    # ---- D03 ----
    Vg = V["V_cb_c_h"]
    VA = F.vt_route(Vg, [(-1, 0), (1, -1), (0, 1)])      # cbar_rho(-l) c^mu(l-p) h_a(p)
    VB = F.vt_route(Vg, [(-1, 1), (1, 0), (0, -1)])      # cbar_sig(p-l) c^nu(l) h_b(-p)
    WBg = {}
    for (s, nu, b), poly in VB.items():                  # sum_nu V_B[s,nu,b] G_{nu rho}
        for (n2, rho), gv in Gh.items():
            if n2 == nu:
                WBg.setdefault((s, rho, b), F.SP()).iadd(poly, gv)
    N = {}
    for (rho, mu, a), pa in VA.items():                  # sum V_A[rho,mu,a] G_{mu s} WBg[s,rho,b]
        for (m2, s), gv in Gh.items():
            if m2 != mu:
                continue
            for b in range(10):
                wb = WBg.get((s, rho, b))
                if wb is not None:
                    N.setdefault((a, b), F.SP()).iadd_prod(pa, wb, gv)
```

```
    rec("D03", R["D03"]["S_f"] * R["D03"]["loop_sign"] * (I * kap) ** 2 * phase_factor(Vg, Vg), N,
        ["h_a(p)", "h_b(-p)"],
        {"A": ["cbar_rho: -l", "c^mu: l-p", "h_a: p"], "B": ["cbar_sigma: p-l", "c^nu: l", "h_b: -p"],
         "propagators": ["<c^mu cbar_sigma>(l-p)", "<c^nu cbar_rho>(l)"], "trace": "closed ghost loop"},
        ["(l-p)^2+i0", "l^2+i0"])
    # ---- D11 (P-convention integrand of the inherited kernel; matching only) ----
    Vs = V["V_h_phi_phi"]
    VA = F.vt_route(Vs, [(0, 1), (-1, 0), (1, -1)])
    VB = F.vt_route(Vs, [(0, -1), (1, 0), (-1, 1)])
    N = {}
    for (a, _, _), pa in VA.items():
        for (b, _, _), pb in VB.items():
            N.setdefault((a, b), F.SP()).iadd_prod(pa, pb)
    rec("D11", R["D11"]["S_f"] * R["D11"]["loop_sign"] * (I * kap) ** 2 * phase_factor(Vs, Vs) * I ** 2, N,
        ["h_a(p)", "h_b(-p)"], {"A": ["h_a: p", "phi: -l", "phi: l-p"], "B": ["h_b: -p", "phi: l", "phi: p-l"]
 +> ,
                                "propagators": ["<phi phi>(l)", "<phi phi>(l-p)"]}, ["l^2+i0", "(l-p)^2+i0"],
        {"role": "reference to the inherited XVI-O kernel only (D11 matching M1); not re-evaluated"})
    # ---- D12 ----
    V2s = V["V_hh_phi_phi"]
    VA = F.vt_route(V2s, [(0, 1), (0, -1), (1, 0), (-1, 0)])
    N = {}
    for (a, b, _, _), pa in VA.items():
        N.setdefault((a, b), F.SP()).iadd(pa)
    rec("D12", R["D12"]["S_f"] * R["D12"]["loop_sign"] * (I * kap ** 2) * phase_factor(V2s) * I, N,
        ["h_a(p)", "h_b(-p)"], {"V": ["h_a: p", "h_b: -p", "phi: l", "phi: -l"], "propagators": ["<phi phi>(l)
 +> "]},
        ["l^2+i0"], {"role": "integrand only; D12 status stays Missing (no value)"})
    # ---- D13 ----
    VK = K_vertex(rules)
    VA = F.vt_route(VK, [(0, -1), (1, 0), (-1, 1)])      # K'_(a')(-p) c^mu(l) h_c(p-l)
    VB = F.vt_route(Vg, [(-1, 0), (0, 1), (1, -1)])      # cbar_rho(-l) c^nu_ext(p) h_d(l-p)
    WB = {}
    for (rho, nu, d), poly in VB.items():
        for c, pcd in Pcols.get(d, []):
            WB.setdefault((rho, nu, c), F.SP()).iadd(poly, pcd * w(c, d))
    N = {}
    for (ap, mu, c), pa in VA.items():
        for (m2, rho), gv in Gh.items():
            if m2 != mu:
                continue
            for nu in range(4):
                wb = WB.get((rho, nu, c))
                if wb is not None:
                    N.setdefault((ap, nu), F.SP()).iadd_prod(pa, wb, gv)
    rec("D13", R["D13"]["ext_sign"] * R["D13"]["S_f"] * R["D13"]["loop_sign"] * (I * kap) ** 2
        * phase_factor(VK, Vg) * I, N, ["K'_(a')(-p)  [left]", "c^nu(p)  [right]"],
        {"K_vertex": ["K'_(a'): -p", "c^mu: l", "h_c: p-l"], "ghost_vertex": ["cbar_rho: -l", "c^nu (external)
 +> : p", "h_d: l-p"],
         "propagators": ["<c^mu cbar_rho>(l)", "<h_c h_d>(p-l)"],
         "Gamma1_term": "Gamma_1 contains int K'_(a') G_{a' nu} c^nu, G = ordered extraction (K', c)"},
        ["l^2+i0", "(p-l)^2+i0"])
    # ---- D14a/b/c ----
    VA = F.vt_route(V3, [(0, 0), (1, 0), (-1, 0)])
    N = {}
    for (a, c, d), poly in VA.items():
        pcd = Pi.get((c, d))
        if pcd:
            N.setdefault((a,), F.SP()).iadd(poly, pcd * w(c, d))
    rec("D14a", R["D14a"]["S_f"] * R["D14a"]["loop_sign"] * (I * kap) * phase_factor(V3) * I, N, ["h_a(0)"],
        {"V3": ["h_a: 0", "h_c: l", "h_d: -l"], "propagators": ["<h_c h_d>(l)"]}, ["l^2+i0"])
    VA = F.vt_route(Vg, [(-1, 0), (1, 0), (0, 0)])
    N = {}
    for (rho, mu, a), poly in VA.items():
        gv = Gh.get((mu, rho))
        if gv:
            N.setdefault((a,), F.SP()).iadd(poly, gv)
    rec("D14b", R["D14b"]["S_f"] * R["D14b"]["loop_sign"] * (I * kap) * phase_factor(Vg), N, ["h_a(0)"],
        {"V": ["cbar_rho: -l", "c^mu: l", "h_a: 0"], "propagators": ["<c^mu cbar_rho>(l)"]}, ["l^2+i0"])
    VA = F.vt_route(Vs, [(0, 0), (1, 0), (-1, 0)])
    N = {}
    for (a, _, _), poly in VA.items():
        N.setdefault((a,), F.SP()).iadd(poly)
    rec("D14c", R["D14c"]["S_f"] * R["D14c"]["loop_sign"] * (I * kap) * phase_factor(Vs) * I, N, ["h_a(0)"],
        {"V": ["h_a: 0", "phi: l", "phi: -l"], "propagators": ["<phi phi>(l)"]}, ["l^2+i0"])
    for r in out.values():
        r["N"] = {k: v for k, v in r["N"].items() if not v.is_zero()}
    return out


def integrand_record(name, r):
```

```
    num = {",".join(map(str, k)): v.canonical(LP_NAMES) for k, v in sorted(r["N"].items())}
    body = {"graph": name, "prefactor": str(r["prefactor"]), "denominators": r["denominators"],
            "external": r["external"], "routing": r["routing"], "numerator_variables": "l0..l3, p0..p3 (lower)
  +> ",
            "external_components": "independent h_(mn) index 0..9 = " + str(F.HC) + "; vector index 0..3",
            "numerator": num}
    sha = hashlib.sha256(json.dumps(body, sort_keys=True, separators=(",", ":")).encode()).hexdigest()
    return body, sha


# ---------------------------------------------------------------------------------------------------------
# W2 multiplicity: independent 16-component tensor recomputation
# ---------------------------------------------------------------------------------------------------------
def _tensor_pairs():
    return [(m, n) for m in range(4) for n in range(4)]


def dd_tensor(mn, rs):
    e = lambda a, b: F.ETA[a] if a == b else 0  # noqa: E731
    (m, n), (r, s) = mn, rs
    return Fr(e(m, r) * e(n, s) + e(m, s) * e(n, r), 2) - Fr(e(m, n) * e(r, s), 2)


def W2(ints):
    """tensor route: V^T_{..(mn)..} = V_{..(min,max)..}/w ; 16x16 de Donder numerator; compare with generator.
  +> """
    V = F.vertices()
    TP = _tensor_pairs()
    idx = lambda mn: F.HIDX[(min(mn), max(mn))]  # noqa: E731
    res = {}
    # D02
    VA = F.vt_route(V["V4_grav"], [(0, 1), (0, -1), (1, 0), (-1, 0)])
    N = {}
    for mn in TP:
        for rs in TP:
            dv = dd_tensor(mn, rs)
            if not dv:
                continue
            c, d = idx(mn), idx(rs)
            for a in range(10):
                for b in range(10):
                    p = VA.get((a, b, c, d))
                    if p is not None:
                        N.setdefault((a, b), F.SP()).iadd(p, dv / (F.W_OFF[c] * F.W_OFF[d]))
    N = {k: v for k, v in N.items() if not v.is_zero()}
    res["D02"] = N == ints["D02"]["N"]
    # D14a
    VA = F.vt_route(V["V3_grav"], [(0, 0), (1, 0), (-1, 0)])
    N = {}
    for mn in TP:
        for rs in TP:
            dv = dd_tensor(mn, rs)
            if not dv:
                continue
            c, d = idx(mn), idx(rs)
            for a in range(10):
                p = VA.get((a, c, d))
                if p is not None:
                    N.setdefault((a,), F.SP()).iadd(p, dv / (F.W_OFF[c] * F.W_OFF[d]))
    N = {k: v for k, v in N.items() if not v.is_zero()}
    res["D14a"] = N == ints["D14a"]["N"]
    # D13 (one graviton propagator)
    base = base_rules()
    VA = F.vt_route(K_vertex(base), [(0, -1), (1, 0), (-1, 1)])
    VB = F.vt_route(V["V_cb_c_h"], [(-1, 0), (0, 1), (1, -1)])
    Gh = ghost_num()
    N = {}
    for (ap, mu, c), pa in VA.items():
        for (rho, nu, d), pb in VB.items():
            gv = Gh.get((mu, rho))
            if not gv:
                continue
            tot = Fr(0)
            for mn in TP:
                if idx(mn) != c:
                    continue
                for rs in TP:
                    if idx(rs) == d:
                        tot += dd_tensor(mn, rs) / (F.W_OFF[c] * F.W_OFF[d])
            if tot:
                N.setdefault((ap, nu), F.SP()).iadd_prod(pa, pb, gv * tot)
```

```
    N = {k: v for k, v in N.items() if not v.is_zero()}
    # D13 generator under base rules may differ in sign only if the rule table is mutated (handled by W1)
    res["D13"] = N == ints["D13_base_order"]["N"] if "D13_base_order" in ints else N == ints["D13"]["N"]
    P = F.propagators()["graviton"]
    res["Pinv_equals_deDonder_numerator"] = bool(P["Pinv_equals_deDonder_tensor_at_representatives"])
    res["schur_equals_q2P"] = bool(P["schur_equals_q2P"])
    res["full_h_b_inverse_verified"] = bool(P["full_h_b_inverse_verified"])
    return {"checks": res, "pass": all(res.values()),
            "rule": "independent components h_(mn): vertex = d/dh_(mn) (no 1/w), propagator i (P^{-1})_{ab}/(q
  +> ^2+i0), "
                    "contraction sum over 10x10; tensor form needs V^T = V/w (w=2 off-diagonal) with the 16x16
  +>  "
                    "de Donder numerator"}


# ---------------------------------------------------------------------------------------------------
# W3 closed forms
# ---------------------------------------------------------------------------------------------------
def W3():
    V = F.vertices()
    k = [F.ksym(leg) for leg in range(4)]
    names = {4 * leg + r: k[leg][r] for leg in range(4) for r in range(4)}

    def val(vt, comps):
        p = vt.T.get(comps)
        return (I ** vt.phase) * p.to_sympy(names) if p is not None else sp.Integer(0)
    res = {}
    # ghost-graviton vertex, all derivative terms, momentum conservation k0 = -(k1+k2)
    cons = {k[0][r]: -(k[1][r] + k[2][r]) for r in range(4)}
    bad = 0
    for rho in range(4):
        for mu in range(4):
            for a in range(10):
                lhs = sp.expand(val(V["V_cb_c_h"], (rho, mu, a)).xreplace(cons))
                rhs = F.ghost_vertex_closed(rho, mu, a, k[0], k[1], k[2])
                bad += sp.expand(lhs - rhs) != 0
    res["cbar_c_h_equals_2ia_D_Lch"] = bad == 0
    res["cbar_c_h_mismatched_components"] = bad
    bad = 0
    for ap in range(10):
        for mu in range(4):
            for a in range(10):
                lhs = sp.expand(val(V["V_K_c_h"], (ap, mu, a)).xreplace(cons))
                rhs = F.K_vertex_closed(ap, mu, a, k[0], k[1], k[2])
                bad += sp.expand(lhs - rhs) != 0
    res["K_c_h_equals_a_Lch"] = bad == 0
    cons2 = {k[0][r]: -k[1][r] for r in range(4)}
    bad = 0
    for ap in range(10):
        m, n = F.HC[ap]
        for mu in range(4):
            lhs = sp.expand(val(V["V_K_c"], (ap, mu)).xreplace(cons2))
            dc = lambda x, y: (F.ETA[y] if y == mu else 0) * I * k[1][x]  # noqa: E731  d_x c_y, c_y = eta_yy
  +> c^y
            rhs = sp.expand(F.A_GH * (dc(m, n) + dc(n, m)))
            bad += sp.expand(lhs - rhs) != 0
    res["K_c_equals_a_dc_plus_dc"] = bad == 0
    bad = 0
    for a in range(10):
        lhs = sp.expand(val(V["V_h_phi_phi"], (a, 0, 0)).xreplace(cons))
        bad += sp.expand(lhs - F.h_phi_phi_closed(a, k[1], k[2])) != 0
    res["h_phi_phi_equals_a1_closed_form"] = bad == 0
    q2 = sum(F.ETA[r] * k[1][r] ** 2 for r in range(4))
    bad = 0
    for rho in range(4):
        for mu in range(4):
            lhs = sp.expand(val(V["M0_ghost"], (rho, mu)).xreplace(cons2))
            bad += sp.expand(lhs - (-I * q2 if rho == mu else 0)) != 0
    res["ghost_kinetic_equals_minus_i_q2_delta"] = bad == 0
    return {"checks": res, "pass": all(v for kk, v in res.items() if isinstance(v, bool)),
            "closed_forms": {"cbar.c.h": "kappa * cbar_rho 2 i a D^rho[L_c h], L_c h = c^r d_r h + h_rn d_m c^
  +> r + h_mr d_n c^r",
                             "K'.c.h": "kappa K'_(mn) a (L_c h)_mn", "K'.c": "K'_(mn) a (d_m c_n + d_n c_m)",
                             "h.phi.phi": "kappa (1/2) a1^{mn}(h) d_m phi d_n phi, a1 = tau eta/2 - h^{mn}",
                             "ghost kinetic": "cbar_mu i Box c^mu"}}


# ---------------------------------------------------------------------------------------------------
# W4 graviton Ward identities, Bose symmetry, extractor consistency
# ---------------------------------------------------------------------------------------------------
def _d0_wave(xi_ids, k1):
```

```
    """i-stripped delta_0 h_(mn) = a (k1_m xi_n + k1_n xi_m), xi_n = eta_nn xi^n; k1: list of SP."""
    out = {}
    for a, (m, n) in enumerate(F.HC):
        v = (k1[m] * F.SP.var(xi_ids[n], F.ETA[n])) + (k1[n] * F.SP.var(xi_ids[m], F.ETA[m]))
        out[a] = v.scale(F.A_GH)
    return out


def _d1_wave(xi_ids, k1, e, ke):
    """i-stripped delta_1 h_(mn) = a (xi^r ke_r e_mn + e_rn k1_m xi^r + e_mr k1_n xi^r); e: {comp idx: SP}."""
    def E(r, s):
        return e.get(F.HIDX[(min(r, s), max(r, s))], F.SP())
    out = {}
    for a, (m, n) in enumerate(F.HC):
        v = F.SP()
        for r in range(4):
            x = F.SP.var(xi_ids[r])
            v.iadd(x * ke[r] * E(m, n))
            v.iadd(x * k1[m] * E(r, n))
            v.iadd(x * k1[n] * E(m, r))
        out[a] = v.scale(F.A_GH)
    return out


def W4(full_kappa2=True):
    V2, V3, V4 = F.grav_vertex(2), F.grav_vertex(3), F.grav_vertex(4)
    res = {}
    # ---- kappa^1 symbolic: vars xi 0..3, k2 4..7, k3 8..11, e2 12..21, e3 22..31
    xi = [0, 1, 2, 3]
    k2 = [F.SP.var(4 + r) for r in range(4)]
    k3 = [F.SP.var(8 + r) for r in range(4)]
    k1 = [-(k2[r] + k3[r]) for r in range(4)]
    e2 = {a: F.SP.var(12 + a) for a in range(10)}
    e3 = {a: F.SP.var(22 + a) for a in range(10)}
    d0 = _d0_wave(xi, k1)
    W = F.vt_eval(V3, [d0, e2, e3], [k1, k2, k3])
    W.iadd(F.vt_eval(V2, [_d1_wave(xi, k1, e2, k2), e3], [[k1[r] + k2[r] for r in range(4)], k3]))
    W.iadd(F.vt_eval(V2, [_d1_wave(xi, k1, e3, k3), e2], [[k1[r] + k3[r] for r in range(4)], k2]))
    res["kappa1_ward_offshell_symbolic"] = W.is_zero()
    # trivial-ness guard: V3 part alone is non-zero
    res["kappa1_V3_part_nonzero"] = not F.vt_eval(V3, [d0, e2, e3], [k1, k2, k3]).is_zero()
    # ---- kappa^0
    res["kappa0_ward_symbolic"] = F.vt_eval(V2, [_d0_wave(xi, [-x for x in k2]), e2],
                                            [[-x for x in k2], k2]).is_zero()
    # ---- kappa^2 symbolic, chunked over basis polarisations of legs 2,3,4: vars xi 0..3, k2,k3,k4 4..15
    t0 = time.time()
    k2 = [F.SP.var(4 + r) for r in range(4)]
    k3 = [F.SP.var(8 + r) for r in range(4)]
    k4 = [F.SP.var(12 + r) for r in range(4)]
    k1 = [-(k2[r] + k3[r] + k4[r]) for r in range(4)]
    d0 = _d0_wave(xi, k1)
    ks = {2: k2, 3: k3, 4: k4}
    by_tail = {}
    for comps, poly in V4.T.items():
        by_tail.setdefault(comps[1:], []).append((comps[0], poly))
    V4idx = F.VT("V4", V4.legs, 0, None, 2, "")
    combos = list(itertools.product(range(10), repeat=3)) if full_kappa2 else \
        [c for c in itertools.product(range(10), repeat=3) if c[0] <= c[1] <= c[2]]
    bad = 0
    nonzero_V4_part = 0
    for (b2, b3, b4) in combos:
        T4 = {(c0, b2, b3, b4): p for c0, p in by_tail.get((b2, b3, b4), [])}
        V4idx.T = T4
        unit = lambda b: {b: F.SP.const(1)}  # noqa: E731
        Wk = F.vt_eval(V4idx, [d0, unit(b2), unit(b3), unit(b4)], [k1, k2, k3, k4])
        if not Wk.is_zero():
            nonzero_V4_part += 1
        legs = {2: b2, 3: b3, 4: b4}
        for i in (2, 3, 4):
            j, kk = [x for x in (2, 3, 4) if x != i]
            wave = _d1_wave(xi, k1, unit(legs[i]), ks[i])
            Wk.iadd(F.vt_eval(V3, [wave, unit(legs[j]), unit(legs[kk])],
                              [[k1[r] + ks[i][r] for r in range(4)], ks[j], ks[kk]]))
        bad += not Wk.is_zero()
    res["kappa2_ward_offshell_symbolic"] = bad == 0
    res["kappa2_chunks"] = len(combos)
    res["kappa2_chunks_failing"] = bad
    res["kappa2_V4_part_nonzero_chunks"] = nonzero_V4_part
    res["seconds"] = round(time.time() - t0, 1)      # wall-clock (volatile key "seconds": excluded from conte
  +> nt hashes)
    # ---- Ward contraction with two free on-shell graviton legs and an off-shell gauge leg (supplementary; th
```

```
  +> e
    #      generic symbolic off-shell identities above are the primary evidence; exact null momenta, ell2 kern
  +> el pols)
    def ker(kv):
        M = sp.Matrix(10, 10, lambda i, j: F.vt_eval(F.VT("V2", [], 0, {(i, j): V2.T[(i, j)]} if (i, j) in V2.
  +> T else {}, 0, ""),
                                                     [{i: F.SP.const(1)}, {j: F.SP.const(1)}],
                                                     [[F.SP.const(-x) for x in kv], [F.SP.const(x) for x in k
  +> v]]).d.get((), 0))
        return [v.T for v in M.nullspace()]
    ka, kb = (5, 3, 4, 0), (13, -5, 0, 12)
    kc = tuple(-(x + y) for x, y in zip(ka, kb))
    ok, ninst = True, 0
    for ea in ker(ka):
        for eb in ker(kb):
            for mu in range(4):
                xiv = [1 if r == mu else 0 for r in range(4)]
                d0w = {a: F.SP.const(F.A_GH * (kc[m] * F.ETA[n] * xiv[n] + kc[n] * F.ETA[m] * xiv[m]))
                       for a, (m, n) in enumerate(F.HC)}
                pa = {i: F.SP.const(sp.Rational(ea[i])) for i in range(10) if ea[i] != 0}
                pb = {i: F.SP.const(sp.Rational(eb[i])) for i in range(10) if eb[i] != 0}
                v = F.vt_eval(V3, [d0w, pa, pb], [[F.SP.const(x) for x in kc], [F.SP.const(x) for x in ka],
                                                  [F.SP.const(x) for x in kb]])
                ok &= v.is_zero()
                ninst += 1
    res["ward_contraction_two_free_onshell_graviton_legs_offshell_gauge_leg"] = ok
    res["ward_contraction_instances"] = ninst
    # ---- Bose symmetry of V3, V4 (all leg permutations; dict relabelling)
    def bose(vt, n):
        for perm in itertools.permutations(range(n)):
            T2 = {}
            for comps, poly in vt.T.items():
                mp = {4 * leg + r: F.SP.var(4 * perm[leg] + r) for leg in range(n) for r in range(4)}
                nc = [None] * n
                for leg in range(n):
                    nc[perm[leg]] = comps[leg]
                T2[tuple(nc)] = poly.subs(mp)
            if T2 != vt.T:
                return False
        return True
    res["bose_V3"] = bose(V3, 3)
    res["bose_V4"] = bose(V4, 4)
    # ---- fast extractor == sympy polarisation (V3 fully symbolic)
    e, _ = F.PW.load_ell(3)
    t = sp.symbols("t1:4")
    pol = [{mn: sp.Symbol(f"E{j}_{mn[0]}{mn[1]}") for mn in F.HC} for j in range(3)]
    kk = [F.ksym(j) for j in range(3)]
    sub = {}
    for mn in F.HC:
        sub[sp.Symbol(f"H{mn[0]}{mn[1]}")] = sum(t[j] * pol[j][mn] for j in range(3))
        for r in range(4):
            sub[sp.Symbol(f"dH{mn[0]}{mn[1]}_{r}")] = sum(t[j] * pol[j][mn] * I * kk[j][r] for j in range(3))
    ref = sp.expand(e.xreplace(sub))
    ref = sp.expand(ref.coeff(t[0], 1).coeff(t[1], 1).coeff(t[2], 1))
    names = {4 * leg + r: kk[leg][r] for leg in range(3) for r in range(4)}
    fast = sp.expand(sum(poly.to_sympy(names) * sp.Mul(*[pol[j][F.HC[c]] for j, c in enumerate(comps)])
                         for comps, poly in V3.T.items()))
    res["fast_extractor_equals_polarisation_V3_symbolic"] = sp.expand(fast - ref) == 0
    acc = [k for k, v in res.items() if isinstance(v, bool)]
    return {"checks": res, "pass": all(res[k] for k in acc), "acceptance_keys": acc}


# ---------------------------------------------------------------------------------------------------------
# W5 independent sympy regeneration
# ---------------------------------------------------------------------------------------------------------
def W5(ints, rules):
    l = sp.symbols("l0:4")
    p = sp.symbols("p0:4")
    names = {r: l[r] for r in range(4)} | {4 + r: p[r] for r in range(4)}
    res = {}
    P = F.propagators()
    Pinv = P["graviton"]["Pinv"]
    Gh = P["ghost"]["numerator"]
    S = F.S_psi_parts()
    X, Ph = S["X"], S["phi"]

    def mom(cl, cp):
        return [cl * l[r] + cp * p[r] for r in range(4)]

    def pol(a):
        return {F.HC[a]: 1}
```

```
  def gen_sym(name):
      return {k: v.to_sympy(names) for k, v in ints[name]["N"].items()}

  # D14c
  ref = {}
  for a in range(10):
      v = F.pw_vertex(Ph[1], [{"f": "h", "pol": pol(a), "k": mom(0, 0)}, {"f": "phi", "k": mom(1, 0)},
                              {"f": "phi", "k": mom(-1, 0)}])
      if v != 0:
          ref[(a,)] = sp.expand(v)
  res["D14c"] = ref == {k: sp.expand(v) for k, v in gen_sym("D14c").items()}
  # D14b  (phase: vertex i stripped consistently -> compare i^phase * N)
  Vg = F.vertices()["V_cb_c_h"]
  ref = {}
  for a in range(10):
      tot = 0
      for rho in range(4):
          for mu in range(4):
              if Gh[mu, rho] == 0:
                  continue
              v = F.pw_vertex(X[1], [{"f": "cb", "c": rho, "k": mom(-1, 0)}, {"f": "c", "c": mu, "k": mom(1,
+>  0)},
                                     {"f": "h", "pol": pol(a), "k": mom(0, 0)}])
              tot += v * Gh[mu, rho]
      tot = sp.expand(tot)
      if tot != 0:
          ref[(a,)] = tot
  res["D14b"] = ref == {k: sp.expand((I ** Vg.phase) * v) for k, v in gen_sym("D14b").items()}
  # D14a (sympy polarisation of ell3, independent of the dict extractor)
  e3, _ = F.PW.load_ell(3)
  ref = {}
  t = sp.symbols("t1:4")
  for a in range(10):
      tot = 0
      for (c, d) in [(i, j) for i in range(10) for j in range(10) if Pinv[i, j] != 0]:
          sub = {}
          for mn in F.HC:
              ws = [(t[0], a, mom(0, 0)), (t[1], c, mom(1, 0)), (t[2], d, mom(-1, 0))]
              sub[sp.Symbol(f"H{mn[0]}{mn[1]}")] = sum(tt for tt, idx, _ in ws if F.HC[idx] == mn)
              for r in range(4):
                  sub[sp.Symbol(f"dH{mn[0]}{mn[1]}_{r}")] = sum(tt * I * kk[r] for tt, idx, kk in ws if F.HC
+> [idx] == mn)
          v = sp.expand(e3.xreplace(sub)).coeff(t[0], 1).coeff(t[1], 1).coeff(t[2], 1)
          tot += Pinv[c, d] * v
      tot = sp.expand(tot)
      if tot != 0:
          ref[(a,)] = tot
  res["D14a"] = ref == {k: sp.expand(v) for k, v in gen_sym("D14a").items()}
  # D11
  ref = {}
  VA = {}
  VB = {}
  for a in range(10):
      VA[a] = F.pw_vertex(Ph[1], [{"f": "h", "pol": pol(a), "k": mom(0, 1)}, {"f": "phi", "k": mom(-1, 0)},
                                  {"f": "phi", "k": mom(1, -1)}])
      VB[a] = F.pw_vertex(Ph[1], [{"f": "h", "pol": pol(a), "k": mom(0, -1)}, {"f": "phi", "k": mom(1, 0)},
                                  {"f": "phi", "k": mom(-1, 1)}])
  for a in range(10):
      for b in range(10):
          v = sp.expand(VA[a] * VB[b])
          if v != 0:
              ref[(a, b)] = v
  res["D11"] = ref == {k: sp.expand(v) for k, v in gen_sym("D11").items()}
  # D13 (rule-table leg order; K' vertex extracted by pw_vertex in that order)
  order = rules["leg_order"]["K_c_h"]
  ref = {}
  momK = {"K": mom(0, -1), "c": mom(1, 0), "h": mom(-1, 1)}
  for ap in range(10):
      for nu in range(4):
          tot = 0
          for mu in range(4):
              for rho in range(4):
                  if Gh[mu, rho] == 0:
                      continue
                  for c in range(10):
                      for d in range(10):
                          if Pinv[c, d] == 0:
                              continue
                          lg = {"K": {"f": "K", "c": F.HC[ap], "k": momK["K"]}, "c": {"f": "c", "c": mu, "k"
+> : momK["c"]},
```

```
                                "h": {"f": "h", "pol": pol(c), "k": momK["h"]}}
                            vk = F.pw_vertex(X[1], [lg[x] for x in order])
                            if vk == 0:
                                continue
                            vg = F.pw_vertex(X[1], [{"f": "cb", "c": rho, "k": mom(-1, 0)},
                                                    {"f": "c", "c": nu, "k": mom(0, 1)},
                                                    {"f": "h", "pol": pol(d), "k": mom(1, -1)}])
                            tot += vk * Gh[mu, rho] * vg * Pinv[c, d]
                tot = sp.expand(tot)
                if tot != 0:
                    ref[(ap, nu)] = tot
    VK = K_vertex(rules)
    res["D13"] = ref == {k: sp.expand((I ** (VK.phase + Vg.phase)) * v) for k, v in gen_sym("D13").items()}
    # D01 Bose symmetry of the integrand
    N = ints["D01"]["N"]
    res["D01_bose_N_ab_equals_N_ba"] = all(N.get((a, b), F.SP()) == N.get((b, a), F.SP()) for a in range(10) f
  +> or b in range(10))
    return {"checks": res, "pass": all(res.values())}


# ---------------------------------------------------------------------------------------------------
def verify(rules, w4=None, ints=None):
    t0 = time.time()
    ints = ints or build_integrands(rules)
    if tuple(rules["leg_order"]["K_c_h"]) != ("K", "c", "h"):
        ints["D13_base_order"] = build_integrands(base_rules())["D13"]
    w1 = shadow.run_shadows(rules)
    W1 = {"checks": {k: v["agree"] for k, v in w1.items()}, "detail": w1}
    W1["pass"] = all(W1["checks"].values())
    W2r = W2(ints)
    W3r = W3()
    W5r = W5(ints, rules)
    W5r["covers"] = ["D14a", "D14b", "D14c", "D11", "D13"]
    import hessian_route as HR
    W6r = HR.W6(ints)
    controls = {"W1_wick_berezin_shadow": W1, "W2_multiplicity": W2r, "W3_closed_form_vertices": W3r,
                "W5_regeneration": W5r, "W6_hessian_route": W6r}
    if w4 is not None:
        controls["W4_ward_bose"] = w4
    ok = all(c["pass"] for c in controls.values())
    return ok, controls, ints, round(time.time() - t0, 1)


def main():
    t0 = time.time()
    muts_env = F.mutations() & set(MUTATION_NAMES)
    rules = apply_mutations(base_rules(), muts_env)
    w4 = W4(full_kappa2=True)
    verified, controls, ints, _ = verify(rules, w4)
    # in-process mutation table (graviton Ward/Bose W4 is unaffected by these mutations and is reused)
    mut_table = {}
    if not muts_env:
        for mname in MUTATION_NAMES:
            if mname == "omit_ghost_transport_term":
                import os
                os.environ["P14_MUTATIONS"] = mname
                F._CACHE.clear()
                ok_m, ctl_m, _, sec = verify(base_rules(), w4)
                os.environ["P14_MUTATIONS"] = ""
                F._CACHE.clear()
            else:
                ok_m, ctl_m, _, sec = verify(apply_mutations(base_rules(), {mname}), w4)
            failed = sorted(f"{cn}.{k}" for cn, c in ctl_m.items() for k, v in c["checks"].items()
                            if isinstance(v, bool) and not v)
            mut_table[mname] = {"amplitude_rules_verified": ok_m, "rejected": not ok_m, "failed_checks": faile
  +> d,
                                "seconds": sec}
        F.vertices()
        F.propagators()
    mutations_all_rejected = bool(mut_table) and all(v["rejected"] for v in mut_table.values())
    amplitude_rules_verified = bool(verified and (mutations_all_rejected or bool(muts_env)) and not muts_env)
    # exports
    exd = ROOT / "exports" / "P14_integrands"
    exd.mkdir(parents=True, exist_ok=True)
    graphs = {}
    status = {"D01": "integrand_available", "D02": "integrand_available", "D03": "integrand_available",
              "D11": "reference: inherited XVI-O kernel (O5/o70); P-convention integrand for matching M1 only"
  +> ,
              "D12": "Missing (integrand available; no value)", "D13": "integrand_available",
              "D14a": "integrand_available", "D14b": "integrand_available", "D14c": "integrand_available"}
    for g in GRAPH_ORDER:
```

```
      body, sha = integrand_record(g, ints[g])
      (exd / f"{g}.json").write_text(json.dumps({**body, "sha256": sha, "integrated": False}, indent=None,
                                                separators=(",", ":"), sort_keys=True))
      graphs[g] = {"status": status[g], "integrand_available": True, "integrated": False, "value": "not eval
+> uated",
                   "prefactor": body["prefactor"], "denominators": body["denominators"], "external": body["e
+> xternal"],
                   "routing": body["routing"], "numerator_components": len(body["numerator"]),
                   "numerator_monomials": sum(len(v) for v in body["numerator"].values()),
                   "numerator_degree": max((v.degree() for v in ints[g]["N"].values()), default=0),
                   "file": f"exports/P14_integrands/{g}.json", "sha256": sha,
                   "rule": {k: (str(v) if not isinstance(v, (int, tuple, list)) else v)
                            for k, v in rules["graphs"][g].items()}}
  graphs["D12"]["D12_status"] = "Missing"
  import hessian_route as HR
  w6_exports = HR.compare_exports(ROOT)
  amplitude_rules_verified = bool(amplitude_rules_verified and w6_exports["all_agree"])
  V = F.vertices()
  names = {4 * leg + r: f"k{leg}_{r}" for leg in range(4) for r in range(4)}
  vt_export = {}
  inv = []
  for nm, vt in V.items():
      rows = [[list(c), p.canonical(names)] for c, p in sorted(vt.T.items())]
      sha = hashlib.sha256(json.dumps(rows, separators=(",", ":")).encode()).hexdigest()
      vt_export[nm] = {"legs": vt.legs, "phase_i_power": vt.phase, "kappa_power": vt.kappa_power,
                       "feynman_vertex": f"i kappa^{vt.kappa_power} * i^{vt.phase} * T", "origin": vt.origin
+> ,
                       "tensor": rows, "sha256": sha}
      inv.append({"name": nm, "legs": vt.legs, "kappa_power": vt.kappa_power, "phase_i_power": vt.phase,
                  "components": len(vt.T), "nonzeros": vt.nnz(), "sha256": sha, "origin": vt.origin})
  (ROOT / "exports" / "P14_vertex_tensors.json").write_text(json.dumps(
      {"revision": PC.REVISION, "convention": "ordered extraction (leg 1 leftmost); momenta k{leg}_{r} lower
+> , incoming;"
                                              " graviton comps = independent h_(mn) index; V = i^phase * sum
+>  T",
       "vertices": vt_export}, separators=(",", ":"), sort_keys=True))
  P = F.propagators()
  props = {"graviton": {"numerator": "i (P^{-1})_{ab}", "denominator": "q^2 + i0",
                        "Pinv": [[str(x) for x in P["graviton"]["Pinv"].row(i)] for i in range(10)],
                        "Pinv_equals_deDonder_tensor_at_representatives": P["graviton"]["Pinv_equals_deDonde
+> r_tensor_at_representatives"],
                        "schur_equals_q2P": P["graviton"]["schur_equals_q2P"],
                        "full_h_b_inverse_verified": P["graviton"]["full_h_b_inverse_verified"],
                        "b_block": P["graviton"]["Mbb"], "hb_entry_example": P["graviton"]["inverse_blocks"]
+> ["hb"]},
           "ghost": {"rule": P["ghost"]["rule"], "M0": "-i q^2 delta_{rho mu}", "numerator": "-delta_{mu rho
+> }",
                     "M0_is_q2_times_constant": P["ghost"]["M0_is_q2_times_constant"]},
           "scalar": {"rule": "i/(q^2+i0)", "V2": str(P["scalar"]["V2"])},
           "i0": "Feynman: every denominator q^2 + i0 (eta = +,-,-,-); D_F(k) = i/(k^2+i0) (Gate 0)"}
  cert = {
      "landmark": "XVI-P", "revision": PC.REVISION, "deliverable": "P.5 entry D4: executable amplitude rules
+> ",
      "amplitude_rules_verified": amplitude_rules_verified,
      "verified_definition": "W1 AND W2 AND W3 AND W4 AND W5 AND W6 pass AND the W6 Hessian route equals the
+>  EXPORTED "
                             "D01/D02/D03/D12 integrands AND every listed mutation rejected (no env mutation
+> )",
      "mutations_from_env": sorted(muts_env),
      "integrands_only": True, "loop_evaluations_performed": False,
      "conventions": {"weight": "e^{iS}", "fourier": "f(x) = int d^4k/(2pi)^4 e^{ikx} f~(k); d -> i k; incom
+> ing momenta",
                      "extraction": "ordered: V[L1..Ln] = d^L_{Ln}...d^L_{L1} S (leg 1 leftmost)",
                      "vertex_factor": "i kappa^n V (V unnormalised n-th variation; frozen V4 convention)",
                      "Gamma1": "i Gamma_1^{(ext)} = sum of 1PI one-loop graphs; Gamma_1 = -i * graph sum",
                      "graviton_components": "independent h_(mn) (m<=n); multiplicity w = 1 (diag), 2 (off-d
+> iag) "
                                             "enters ONLY the tensor form",
                      "ghost": "vertices c-bar LEFT (cbar.c.h); <c cbar> = i M0^{-1}; closed ghost loop fact
+> or -1",
                      "source": "K' LEFT of c in K'.c.h and in the Gamma_1 coefficient G_{a' nu} (K'_(a') G
+> c^nu)",
                      "loop_measure": "int d^d l/(2pi)^d (prescription: see P14_prescription_D11_matching)"}
+> ,
      "rules": rules_json(rules),
      "vertex_inventory": inv, "propagators": props, "graphs": graphs,
      "controls": {k: {kk: vv for kk, vv in v.items() if kk != "detail"} for k, v in controls.items()},
      "W1_detail": controls["W1_wick_berezin_shadow"]["detail"],
      "W6_detail": controls["W6_hessian_route"]["detail"], "W6_exports": w6_exports,
      "regeneration_coverage": {"W5_independent_sympy_route": ["D14a", "D14b", "D14c", "D11", "D13"],
                                "W6_hessian_trlog_route": ["D01", "D02", "D03", "D12"],
```

```
                                "D01_bose_symmetry": "W5 (symmetry check only)"},
        "mutations": mut_table, "mutations_all_rejected": mutations_all_rejected,
        "D12_status": "Missing", "D11_role": "inherited kernel reference; matching defined (see D11 matching n
  +> ote)",
        "exports": {"vertex_tensors": "exports/P14_vertex_tensors.json", "integrands": "exports/P14_integrands
  +> /*.json"},
        "execution_pass": True,
        "seconds": round(time.time() - t0, 1),
    }
    certio.write(ROOT / "certificates" / f"{PC.CERT_PREFIX}_amplitude_rules.json", cert)
    print(json.dumps({"amplitude_rules_verified": amplitude_rules_verified,
                      "controls": {k: v["pass"] for k, v in controls.items()},
                      "mutations": {k: v["rejected"] for k, v in mut_table.items()},
                      "seconds": cert["seconds"]}, indent=1))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

119 Source: counterterm_domain.py

[listing 86/133] code/p5_entry/counterterm_domain.py

```
#!/usr/bin/env python3
"""XVI-P p14 Deliverable 3: the one-loop counterterm DOMAIN at kappa^1 and kappa^2 (an order-bounded EFT const
  +> raint
list; NOT an admission -- XIV-J and XV-B are not opened).

Mechanical enumeration (no hand-written basis):
  * species table (dimension, ghost number, phi -> -phi parity, statistics) derived from S_Psi: [kappa] = -1,
  +> every
    interaction vertex is kappa^{legs-2}  =>  the one-loop kappa^n part of Gamma_1 has exactly n external legs
  +>  and the
    local counterterm operator has mass dimension 4 + n (massless theory, no other scale in the prescription);
  * permitted external species = legs of loop-capable vertices (>= 2 quantum legs) of the verified vertex inve
  +> ntory;
  * for every admissible species pair the Lorentz-covariant two-leg structures are GENERATED from eta and the
  +> leg
    momentum p (all index pairings), turned into GE densities X_I d_J Y_K, and represented by their ordered
    two-point vertex V[X(-p), Y(p)] (total derivatives vanish identically in this representation);
  * dimensions are exact-rational ranks.
Equivalences: sigma_0 = (S_Psi^(0), . ) (XVI-N antibracket, frozen orientation) applied to every generated
ghost-number -1, dimension-5 bilinear Xi (antifield-linear Xi = field redefinitions / canonical transformation
  +> s;
field-only Xi = s-exact gauge-fixing directions); the quotient dimension is dim D - dim(D cap Im sigma_0).
"""
from __future__ import annotations
import itertools
import json
import sys
import time
from fractions import Fraction as Fr
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import feynman as F  # noqa: E402
import p14_conventions as PC  # noqa: E402
import certio  # noqa: E402
from graded_jets import GE, DJ, bracket, PARITY  # noqa: E402

I = sp.I
ETA = F.ETA
# field: (mass dimension, ghost number, phi-parity (Z2), index type)   index types: 'L2' sym lower pair, 'U2'
  +> sym upper
# pair (dual), 'U1' upper vector, 'L1' lower vector, '' scalar
SPECIES = {
    "h": (1, 0, 0, "L2"), "c": (1, 1, 0, "U1"), "cb": (1, -1, 0, "L1"), "b": (2, 0, 0, "U1"), "phi": (1, 0, 1,
  +>  ""),
    "K": (2, -1, 0, "U2"), "cs": (2, -2, 0, "L1"), "cbs": (2, 0, 0, "U1"), "bs": (1, -1, 0, "L1"), "phis": (2,
  +>  -1, 1, ""),
}
ORDER = ["h", "c", "cb", "b", "phi", "K", "cs", "cbs", "bs", "phis"]
```

```
NIDX = {"L2": 2, "U2": 2, "U1": 1, "L1": 1, "": 0}



def _num_word(n):
    w = {0: "Zero", 1: "One", 2: "Two", 3: "Three", 4: "Four", 5: "Five", 6: "Six", 7: "Seven", 8: "Eight", 9:
  +>  "Nine",
         10: "Ten", 11: "Eleven"}
    return w.get(n, str(n))

def loop_capable_species():
    """species occurring as legs of vertices with >= 2 quantum (field) legs, from the verified inventory."""
    V = F.vertices()
    fields = {"h", "c", "cb", "b", "phi"}
    out, used = set(), {}
    for nm, vt in V.items():
        if nm in ("M0_ghost", "M_phi", "V2_grav", "V_b_b", "V_b_h"):
            continue                                # kinetic/gauge-fixing bilinears (propagators)
        nq = sum(1 for f in vt.legs if f in fields)
        if nq >= 2 and vt.kappa_power >= 1:
            out |= set(vt.legs)
            used[nm] = vt.legs
    return out, used


# ---------------------------------------------------------------------------------------------------
def structures(slots, k):
    """all covariant structures: slots = list of (slot id, position 'L'|'U'); returns list of dicts
    {component tuple: sympy poly in p0..p3}.  p is the lower momentum p_a (d_a -> i p_a on leg 2)."""
    p = sp.symbols("p0:4")
    p2 = sum(ETA[a] * p[a] ** 2 for a in range(4))
    n = len(slots)
    out = []
    for npv in range(0, min(n, k) + 1):
        if (k - npv) % 2:
            continue
        for psl in itertools.combinations(range(n), npv):
            rest = [i for i in range(n) if i not in psl]
            for match in _matchings(rest):
                T = {}
                for comps in itertools.product(range(4), repeat=n):
                    v = p2 ** ((k - npv) // 2)
                    for i in psl:
                        a = comps[i]
                        v = v * (ETA[a] * p[a] if slots[i][1] == "L" else p[a])     # p^a for a lower slot
                    for (i, j) in match:
                        a, b = comps[i], comps[j]
                        if a != b:
                            v = 0
                            break
                        if slots[i][1] == slots[j][1]:
                            v = v * ETA[a]
                    if v != 0:
                        T[comps] = sp.expand(v)
                if T:
                    out.append(T)
    return out


def _matchings(lst):
    if not lst:
        yield []
        return
    a = lst[0]
    for i in range(1, len(lst)):
        rest = lst[1:i] + lst[i + 1:]
        for m in _matchings(rest):
            yield [(a, lst[i])] + m


def slots_of(f, base):
    t = SPECIES[f][3]
    pos = "L" if t.startswith("L") else "U"
    return [(base + j, pos) for j in range(NIDX[t])]


def gen_comp(f, comps):
    """GE generator for a tensor component; symmetric pairs map to the independent component (dual U2 source
    K^{mn} = K'_(mn)/w so that sum_{m,n} K^{mn} X_mn = sum_{m<=n} K'_(mn) X_mn)."""
    t = SPECIES[f][3]
    if t in ("L2", "U2"):
```

```
        m, n = comps
        g = GE.gen(f, (min(m, n), max(m, n)))
        return g.scale(sp.Rational(1, 2)) if (t == "U2" and m != n) else g
    if t in ("U1", "L1"):
        return GE.gen(f, comps[0])
    return GE.gen(f, ())


def density(X, Y, T, nx):
    """sum_{comps} T(comps)(p) X_{comps[:nx]} Y_{comps[nx:]} with p_a -> -i d_a acting on Y."""
    p = sp.symbols("p0:4")
    out = GE()
    for comps, poly in T.items():
        P = sp.Poly(poly, *p)
        gx = gen_comp(X, comps[:nx])
        for mon, cf in P.terms():
            J = sum(([a] * e for a, e in enumerate(mon)), [])
            gy = DJ(gen_comp(Y, comps[nx:]), J)
            out = out + (gx * gy).scale(cf * (-I) ** len(J))
    return out


def comp_range(f):
    t = SPECIES[f][3]
    if t in ("L2", "U2"):
        return [(m, n) for m in range(4) for n in range(m, 4)]
    if t in ("U1", "L1"):
        return list(range(4))
    return [()]


def two_point_vector(G, sectors):
    """coordinates of a gh-0 bilinear density: ordered two-point vertex V[X_I(-p), Y_J(p)] for every target se
  +> ctor
    (X <= Y in ORDER) -> {(sector, I, J, monomial, re|im): Fraction}."""
    p = sp.symbols("p0:4")
    pm = [-x for x in p]
    vec = {}
    for (X, Y) in sectors:
        for ci in comp_range(X):
            for cj in comp_range(Y):
                legs = []
                for f, cc, k in ((X, ci, pm), (Y, cj, list(p))):
                    if f == "h":
                        legs.append({"f": "h", "pol": {cc: 1}, "k": k})
                    elif f == "phi":
                        legs.append({"f": "phi", "pol": {(): 1}, "k": k})
                    elif PARITY[f] == 0:
                        legs.append({"f": f, "pol": {cc: 1}, "k": k})
                    else:
                        legs.append({"f": f, "c": cc, "k": k})
                v = F.pw_vertex(G, legs)
                if v == 0:
                    continue
                for t in sp.Add.make_args(sp.expand(v)):
                    num, rest = t.as_independent(*p, as_Add=False)
                    mono = tuple(sp.Poly(rest, *p).monoms()[0]) if rest != 1 else (0, 0, 0, 0)
                    for part, val in (("re", sp.re(num)), ("im", sp.im(num))):
                        if val != 0:
                            key = (X, Y, str(ci), str(cj), mono, part)
                            vec[key] = vec.get(key, Fr(0)) + Fr(int(sp.Rational(val).p), int(sp.Rational(val).
  +> q))
    return {k: v for k, v in vec.items() if v}


def rank(vectors):
    """exact rational rank (Gaussian elimination on sparse dict vectors)."""
    rows = [dict(v) for v in vectors if v]
    basis = []                        # list of (pivot key, row)
    for r in rows:
        r = dict(r)
        for piv, b in basis:
            if piv in r:
                f = r[piv] / b[piv]
                for k, v in b.items():
                    nv = r.get(k, 0) - f * v
                    if nv:
                        r[k] = nv
                    else:
                        r.pop(k, None)
        if r:
```

```
            piv = min(r)
            basis.append((piv, r))
    return len(basis)


def admissible_pairs(n, species):
    """unordered species pairs (n = 2) or singles (n = 1): ghost number 0, phi-parity even, derivative count >
  +> = 0."""
    out = []
    if n == 1:
        for f in sorted(species, key=ORDER.index):
            d, g, z, _ = SPECIES[f]
            if g == 0 and z == 0:
                out.append(((f,), 4 + n - d))
        return out
    sp_ = sorted(species, key=ORDER.index)
    for i, X in enumerate(sp_):
        for Y in sp_[i:]:
            dX, gX, zX, _ = SPECIES[X]
            dY, gY, zY, _ = SPECIES[Y]
            if X == Y and PARITY[X] == 1:
                pass
            k = 4 + n - dX - dY
            if gX + gY == 0 and (zX + zY) % 2 == 0 and k >= 0:
                out.append(((X, Y), k))
    return out


def sector_basis(X, Y, k):
    sl = slots_of(X, 0) + slots_of(Y, 10)
    nx = NIDX[SPECIES[X][3]]
    dens = [density(X, Y, T, nx) for T in structures(sl, k)]
    return [d for d in dens if not d.is_zero()]


def independent(dens, sectors):
    vecs, keep = [], []
    r0 = 0
    for d in dens:
        v = two_point_vector(d, sectors)
        r1 = rank(vecs + [v])
        if r1 > r0:
            vecs.append(v)
            keep.append(d)
            r0 = r1
    return keep, vecs


def S0_density():
    """S_Psi at kappa^0 as one GE density: ell_2 (exports, rebuilt as a GE density) + X^(0) + S_phi^(0)."""
    P = F.S_psi_parts()
    e2, _ = F.PW.load_ell(2)
    sub = {}
    for (m, n) in F.HC:
        sub[sp.Symbol(f"H{m}{n}")] = sp.Symbol(f"h{m}{n}")
        for r in range(4):
            sub[sp.Symbol(f"dH{m}{n}_{r}")] = sp.Symbol(f"h{m}{n}_d{r}")
    from graded_jets import esym, _KEY  # noqa: F401  (register jet symbols)
    for (m, n) in F.HC:
        esym(("h", (m, n), ()))
        for r in range(4):
            esym(("h", (m, n), (r,)))
    ell2 = GE.const(e2.xreplace(sub))
    return ell2 + P["X"][0] + P["phi"][0]


def S0_check(S0):
    """S_Psi^(0) density reproduces the verified kappa^0 Hessians (V2_grav all 100 components; M0 = -i q^2 del
  +> ta)."""
    q = sp.symbols("q0:4")
    qm = [-x for x in q]
    vt = F.vertices()["V2_grav"]
    mq = [[F.SP.var(r, -1) for r in range(4)], [F.SP.var(r) for r in range(4)]]
    bad = 0
    for a in F.HC:
        for b in F.HC:
            v = F.pw_vertex(S0, [{"f": "h", "pol": {a: 1}, "k": qm}, {"f": "h", "pol": {b: 1}, "k": list(q)}])
            w = F.vt_eval(vt, [{F.HIDX[a]: F.SP.const(1)}, {F.HIDX[b]: F.SP.const(1)}], mq).to_sympy(q) * I **
  +>  vt.phase
            bad += sp.expand(v - w) != 0
    q2 = sum(ETA[a] * q[a] ** 2 for a in range(4))
```

```
    gh = all(sp.expand(F.pw_vertex(S0, [{"f": "cb", "c": m, "k": qm}, {"f": "c", "c": n, "k": list(q)}])
                       - (-I * q2 if m == n else 0)) == 0 for m in range(4) for n in range(4))
    return {"hh_vs_V2_grav_mismatches": int(bad), "ghost_M0_ok": bool(gh), "ok": bool(bad == 0 and gh)}


def main():
    t0 = time.time()
    lc, used = loop_capable_species()
    permitted = sorted(lc, key=ORDER.index)
    excluded = [f for f in ORDER if f not in lc]
    # ---- kappa^1: one leg, dimension 5, momentum zero
    k1 = []
    for (f,), k in admissible_pairs(1, lc):
        k1.append({"species": f, "derivatives": k, "dimension_ok": k >= 0,
                   "surviving_mod_total_derivatives": k == 0, "reason": "a one-leg local functional with k>0 d
  +> erivatives "
                   "is a total derivative; k = 0 needs a dimension-5 field (none)"})
    k1_basis = [x for x in k1 if x["surviving_mod_total_derivatives"]]
    # ---- kappa^2: two legs, dimension 6
    pairs = admissible_pairs(2, lc)
    sectors = [pr for pr, _ in pairs]
    basis, vecs, per_sector = [], [], {}
    for (X, Y), k in pairs:
        keep, v = independent(sector_basis(X, Y, k), [(X, Y)])
        per_sector[f"{X}.{Y}"] = {"derivatives": k, "dimension": len(keep),
                                  "structures": [repr(d)[:160] for d in keep]}
        basis += keep
        vecs += [two_point_vector(d, sectors) for d in keep]
    dimD = rank(vecs)
    # ---- equivalences: sigma_0 on gh -1, dim 5 bilinears (all species incl. b, cbar*', b*')
    S0 = S0_density()
    s0chk = S0_check(S0)
    xi_pairs = []
    allsp = list(SPECIES)
    for i, X in enumerate(sorted(allsp, key=ORDER.index)):
        for Y in sorted(allsp, key=ORDER.index)[i:]:
            dX, gX, zX, _ = SPECIES[X]
            dY, gY, zY, _ = SPECIES[Y]
            k = 5 - dX - dY
            if gX + gY == -1 and (zX + zY) % 2 == 0 and k >= 0:
                xi_pairs.append(((X, Y), k))
    all_sectors = []
    for i, X in enumerate(sorted(allsp, key=ORDER.index)):
        for Y in sorted(allsp, key=ORDER.index)[i:]:
            if SPECIES[X][1] + SPECIES[Y][1] == 0 and (SPECIES[X][2] + SPECIES[Y][2]) % 2 == 0 \
                    and 6 - SPECIES[X][0] - SPECIES[Y][0] >= 0:
                all_sectors.append((X, Y))
    D_full = [two_point_vector(d, all_sectors) for d in basis]
    images, xi_rec = [], {}
    for (X, Y), k in xi_pairs:
        dens = sector_basis(X, Y, k)
        ims = []
        for d in dens:
            im = bracket(S0, d)
            ims.append(two_point_vector(im, all_sectors))
        kind = "antifield-linear (field redefinition / canonical transformation)" if \
            any(SPECIES[s][1] < 0 and s in ("K", "cs", "phis", "bs") or s == "cbs" for s in (X, Y)) else \
            "field-only (s-exact gauge-fixing direction)"
        xi_rec[f"{X}.{Y}"] = {"derivatives": k, "candidates": len(dens), "image_rank": rank(ims), "kind": kind
  +> }
        images += ims
    r_im = rank(images)
    r_sum = rank(D_full + images)
    inter = dimD + r_im - r_sum
    by_sector_cap = {}
    off = 0
    for (X, Y), k in pairs:
        n_ = per_sector[f"{X}.{Y}"]["dimension"]
        Ds = D_full[off:off + n_]
        off += n_
        by_sector_cap[f"{X}.{Y}"] = n_ + r_im - rank(Ds + images)
    rec = {
        "landmark": "XVI-P", "revision": PC.REVISION, "deliverable": "P.5 entry D3: counterterm domain",
        "kind": "order-bounded EFT constraint list (one loop, kappa^1 and kappa^2); NOT an admission",
        "XIV_J_opened": False, "XV_B_opened": False,
        "dimensions": {f: SPECIES[f][0] for f in ORDER} | {"kappa": -1},
        "ghost_numbers": {f: SPECIES[f][1] for f in ORDER},
        "phi_parity": {f: SPECIES[f][2] for f in ORDER},
        "power_counting": "every interaction vertex is kappa^{legs-2} (amplitude_rules vertex inventory) => on
  +> e-loop "
                          "kappa^n terms have n external legs and local operator dimension 4+n; massless, no o
```

```
  +> ther scale",
        "permitted_species": permitted, "excluded_species": excluded,
        "exclusion_reason": "not a leg of any loop-capable vertex (b only in kappa^0 bilinears; cbar*' only in
  +>  cbar*'.b; "
                            "b*' absent since s b = 0)",
        "loop_capable_vertices": used,
        "permitted_source_dependence": "K' only in the K'.c sector at kappa^2; c*' and phi*' enter first in th
  +> ree-leg "
                                       "terms (c*'.c.c, phi*'.c.phi) at kappa^3 (outside the order bound); cba
  +> r*', b*' "
                                       "never",
        "symmetries": ["Lorentz covariance (structures generated from eta and p; parity-odd eps structures van
  +> ish for "
                       "two-leg terms with one momentum)", "phi -> -phi (with phi*' -> -phi*')", "ghost number
  +>  0",
                       "Bose/Fermi symmetry of identical legs (ordered two-point representation)"],
        "kappa1": {"candidates": k1, "basis": k1_basis, "basis_size": len(k1_basis)},
        "kappa2": {"sectors": per_sector, "basis_size": dimD,
                   "basis_size_by_sector": {s: v["dimension"] for s, v in per_sector.items()}},
        "equivalences": {
            "total_derivatives": "built in: two-point momentum representation with momentum conservation",
            "sigma0": "sigma_0 = (S_Psi^(0), .) with the frozen XVI-N antibracket (graded_jets.bracket)",
            "S0_density_check": s0chk,
            "Xi_classes": xi_rec, "image_rank": r_im, "domain_cap_image": inter,
            "sector_cap_image": by_sector_cap,
            "candidate_directions_by_sector": {s_: per_sector[s_]["dimension"] - v for s_, v in by_sector_cap.
  +> items()},
            "quotient_dimension": dimD - inter,
            "reading": "domain elements in Im sigma_0 are removable by field redefinitions/canonical transform
  +> ations "
                       "(antifield-linear Xi) or are s-exact (field-only Xi). "
                       f"{_num_word(dimD - inter)} candidate directions modulo the computed exact equivalences
  +> , before "
                       "imposing the relevant identity constraints.",
            "cohomology_caveat": "NOT a cohomological classification: that needs D cap ker sigma_0 (the identi
  +> ty "
                                 "constraints, i.e. sigma_0-closedness of the counterterm) and the quotient "
                                 "(D cap ker sigma_0)/(D cap Im sigma_0); D cap ker sigma_0 has NOT been compu
  +> ted. The "
                                 "count above is D/(D cap Im sigma_0) only, an upper bound for the cohomologic
  +> al count",
            "D_cap_ker_sigma0_computed": False},
        "normalization_conditions": [
            "MS-bar in the declared prescription (P14_prescription_D11_matching): counterterms are pure poles
  +> at scale mu",
            "no dimension-4 (two-derivative) counterterm at kappa^2: kappa and the h normalisation are not ren
  +> ormalised at "
            "this order -- an IMPOSED normalization condition (N4 of P14_prescription_D11_matching M3: no mu^2
  +>  p^2 or "
            "mu^4 contacts), not a consequence of masslessness, since mu is a scale; no kappa^1 counterterm (e
  +> mpty domain)",
            "finite adjustments only inside this domain; the D11 matching difference L_match is booked in the
  +> hh sector",
            "u^R_mu (XVI-O) is never modified: no c delta"],
        "counterterm_domain_defined": False,
        "seconds": None,
    }
    expected_ok = (dimD == sum(v["dimension"] for v in per_sector.values()) and not k1_basis)
    rec["counterterm_domain_defined"] = bool(expected_ok and inter <= dimD and r_im > 0 and s0chk["ok"])
    rec["seconds"] = round(time.time() - t0, 1)
    certio.write(ROOT / "certificates" / f"{PC.CERT_PREFIX}_counterterm_domain.json", rec)
    print(json.dumps({"kappa1": len(k1_basis), "kappa2": dimD, "by_sector": rec["kappa2"]["basis_size_by_secto
  +> r"],
                      "sigma0_image_rank": r_im, "domain_cap_image": inter, "sector_cap": by_sector_cap, "quot
  +> ient": dimD - inter,
                      "Xi": {k: (v["candidates"], v["image_rank"]) for k, v in xi_rec.items()},
                      "defined": rec["counterterm_domain_defined"], "seconds": rec["seconds"]}, indent=1))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

120 Source: quantum_object.py

[listing 87/133] code/p5_entry/quantum_object.py

```
#!/usr/bin/env python3
```

```
"""XVI-P p14 Deliverable 1: the quantum object, computed directly in TRANSFORMED variables.

  S_Psi(Phi, Phi*') = sum_k kappa^{k-2} ell_k + S_phi + X(Phi, Phi*' + eps dPsi/dPhi),  eps = -1 (derived, P13
  +>  gate)
  Gamma_Psi = S_Psi + hbar Gamma_1 + O(hbar^2);  Gamma_1 = -i log Z_1PI;  i Gamma_1 = sum of 1PI one-loop grap
  +> hs.

Mechanical content (all recomputed here): term patterns of X by kappa order; integrated fields vs external
antifields; the b decision (retained & integrated) incl. the explicit elimination computation; source spectato
  +> rs of
Gamma_1; kappa bookkeeping (every vertex kappa^{legs-2}; per graph sum(n_v - 2) = E = kappa power of the integ
  +> rand);
Fourier/propagator/i0 from the verified Hessian inverses; the classical pullback kept separate from any quantu
  +> m
gauge-change theorem.  No loop is evaluated.
"""
from __future__ import annotations
import json
import re
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import feynman as F  # noqa: E402
import p14_conventions as PC  # noqa: E402
import counterterm_domain as CT  # noqa: E402
import certio  # noqa: E402
from graded_jets import _KEY  # noqa: E402

FIELDS = ("h", "c", "cb", "b", "phi")
ANTIFIELDS = ("K", "cs", "cbs", "bs", "phis")
GRAPH_VERTICES = {"D01": ["V3_grav", "V3_grav"], "D02": ["V4_grav"], "D03": ["V_cb_c_h", "V_cb_c_h"],
                  "D11": ["V_h_phi_phi", "V_h_phi_phi"], "D12": ["V_hh_phi_phi"], "D13": ["V_K_c_h", "V_cb_c_h
  +> "],
                  "D14a": ["V3_grav"], "D14b": ["V_cb_c_h"], "D14c": ["V_h_phi_phi"]}
GRAPH_EXTERNAL = {"D01": 2, "D02": 2, "D03": 2, "D11": 2, "D12": 2, "D13": 2, "D14a": 1, "D14b": 1, "D14c": 1}


def patterns(G):
    """species multiset of every monomial of a GE density (odd keys + even jet symbols)."""
    out = set()
    for key, val in G.t.items():
        odd = [k[0] for k in key]
        for t in sp.Add.make_args(sp.expand(val)):
            ev = []
            for s, e in t.as_powers_dict().items():
                if s in _KEY:
                    ev += [_KEY[s][0]] * int(e)
            out.add(".".join(sorted(odd + ev, key=lambda f: (FIELDS + ANTIFIELDS).index(f))))
    return sorted(out)


def b_elimination(X0):
    """solve E_b(X^(0)) = 0 algebraically (b enters undifferentiated) and substitute."""
    v = sp.expand(X0.t.get((), 0))
    bsyms = [s for s in v.free_symbols if s in _KEY and _KEY[s][0] == "b"]
    assert all(_KEY[s][2] == () for s in bsyms), "b differentiated: elimination not algebraic"
    bsyms = sorted(bsyms, key=lambda s: _KEY[s][1])
    sol = sp.solve([sp.diff(v, s) for s in bsyms], bsyms, dict=True)
    assert len(sol) == 1
    red = sp.expand(v.xreplace(sol[0]))
    cbs = [s for s in red.free_symbols if s in _KEY and _KEY[s][0] == "cbs"]
    quad = [t for t in sp.Add.make_args(red) if sum(sp.degree(t, s) for s in cbs) == 2]
    lin_h = [t for t in sp.Add.make_args(red) if sum(sp.degree(t, s) for s in cbs) == 1]
    hess = sp.hessian(v, bsyms)
    return {"b_components": [str(s) for s in bsyms], "b_hessian_constant": all(x.free_symbols == set() for x i
  +> n hess),
            "b_hessian_det": str(hess.det()), "solution_b0": str(sol[0][bsyms[0]])[:160],
            "after_elimination_terms": len(sp.Add.make_args(red)),
            "cbar_star_quadratic_terms": len(quad), "cbar_star_linear_terms": len(lin_h),
            "quadratic_example": str(quad[0]) if quad else None,
            "reading": "eliminating b makes S_Psi QUADRATIC in cbar*' (and shifts the cbar*'-linear terms into
  +>  "
                       "h-dependent ones); retaining b keeps S_Psi linear in all external antifields"}
```

```
def main():
    t0 = time.time()
    P = F.S_psi_parts()
    X, Ph = P["X"], P["phi"]
    pats = {n: patterns(X[n]) for n in (0, 1, 2)}
    phipats = {n: patterns(Ph[n]) for n in (0, 1, 2)}
    gf = json.loads((ROOT / "certificates" / "P13_gauge_fixing_map.json").read_text())
    sa = json.loads((ROOT / "certificates" / "P13_source_action.json").read_text())
    amp_f = ROOT / "certificates" / f"{PC.CERT_PREFIX}_amplitude_rules.json"
    amp = json.loads(amp_f.read_text()) if amp_f.is_file() else None
    conv = (ROOT / "notes" / "quantum_convention_sheet.tex").read_text()
    # -------- Q1 transformed variables
    Q1 = {"eps_derived": P["eps"], "gauge_fixing_cert_eps": gf["derived_eps"],
          "S_Psi_equals_S_plus_sPsi_mod_d": gf["S_Psi_equals_S_plus_sPsi_mod_d"],
          "NOT_S_D": gf["invariant_vs_transformed_kept_separate"]["NOT_S_D"],
          "Psi": gf["Psi"],
          "X_patterns_by_kappa": pats, "S_phi_patterns_by_kappa": phipats,
          "X_terms_by_kappa_P13": sa["X_terms_by_kappa_order"],
          "no_switch_or_insertion_fields": not any(f in p for n in pats for p in pats[n] for f in ("lam", "mu"
  +> , "z"))}
    Q1["ok"] = bool(Q1["eps_derived"] == -1 == Q1["gauge_fixing_cert_eps"] and Q1["S_Psi_equals_S_plus_sPsi_mo
  +> d_d"]
                    and Q1["NOT_S_D"] and Q1["no_switch_or_insertion_fields"] and not pats[2])
    # -------- Q2 integrated fields vs external antifields
    lc, used = CT.loop_capable_species()
    b_orders = [n for n in (0, 1, 2) if any("b" in p.split(".") for p in pats[n])]
    b_pats = [p for p in pats[0] if "b" in p.split(".")]
    elim = b_elimination(X[0])
    Pr = F.propagators()
    Q2 = {"integrated": ["h_(mn) (10 independent components)", "c^mu", "cbar_mu", "phi", "b^mu (retained)"],
          "external_antifields": {"K'_(mn)": "h*' (10 comps, sigma_h = +1)", "c*'_mu": "c*'", "cbar*'^mu": "cb
  +> ar*'",
                                  "phi*'": "phi*'", "b*'": "absent (s b = 0)"},
          "external_antifields_retained_P13": gf["external_antifields_retained"],
          "b_decision": "RETAINED and integrated",
          "b_justification": {
              "b_occurs_only_at_kappa_orders": b_orders, "b_patterns": b_pats,
              "no_b_interaction_vertex": b_orders == [0] and all(len(p.split(".")) == 2 for p in b_pats),
              "Mbb_constant_invertible": elim["b_hessian_constant"] and elim["b_hessian_det"] not in ("0",),
              "full_h_b_inverse_verified": Pr["graviton"]["full_h_b_inverse_verified"],
              "schur_complement_equals_q2_P": Pr["graviton"]["schur_equals_q2P"],
              "elimination": elim},
          "loop_capable_species": sorted(lc),
          "Gamma1_independent_of": sorted(set(ANTIFIELDS) - lc),
          "source_spectators": "cbar*' couples only via cbar*'.b (one quantum leg; tree, one-particle reducibl
  +> e "
                               "through the b-h mixing) and b*' is absent => Gamma_1 (1PI) is independent of c
  +> bar*', "
                               "b*'; K', c*', phi*' enter Gamma_1 through K'.c.h, c*'.c.c, phi*'.c.phi"}
    Q2["ok"] = bool(Q2["b_justification"]["no_b_interaction_vertex"] and Q2["b_justification"]["Mbb_constant_i
  +> nvertible"]
                    and Q2["b_justification"]["full_h_b_inverse_verified"] and elim["cbar_star_quadratic_terms
  +> "] > 0
                    and set(Q2["Gamma1_independent_of"]) == {"bs", "cbs"} and {"K", "cs", "phis"} <= lc)
    # -------- Q3 1PI / Gamma_1, vacuum and one-point normalization
    W1 = (amp or {}).get("controls", {}).get("W1_wick_berezin_shadow", {})
    Q3 = {"weight": "e^{i S_Psi/hbar}", "Gamma": "Gamma_Psi[Phi, Phi*'] = S_Psi + hbar Gamma_1 + O(hbar^2); Le
  +> gendre "
          "transform in the fields Phi = (h, c, cbar, b, phi) only; antifields Phi*' are spectators (not trans
  +> formed)",
          "Gamma1": "Gamma_1 = (i/2) STr log S_Psi^{(2)} (fields only) = -i log Z|_1PI; i Gamma_1 = sum of 1PI
  +>  one-loop "
                    "graphs (graph i's from vertices i kappa^n V and propagators i M^{-1})",
          "gate0_consistency": {"Gamma_1 = -i log Z in convention sheet": "Gamma_{1,\\mathrm{matter}}=-i\\log
  +> Z" in conv,
                                "e^{iS/hbar} in convention sheet": "$e^{iS/\\hbar}$" in conv,
                                "W1 Berezin/Wick shadow pass": W1.get("pass")},
          "vacuum_normalization": "Z normalised so that Gamma_1[0] = 0 (field-independent vacuum graphs droppe
  +> d)",
          "one_point_normalization": "the one-point function Gamma_1^{(1)} (graphs D14a, D14b, D14c) is RETAIN
  +> ED as "
                                     "part of the object: no tadpole-cancellation condition and no background
  +> shift "
                                     "imposed; its value is not assigned here (no zero is substituted)",
          "background": "fields of Gamma are the classical (mean) fields; <h> is not shifted"}
    Q3["ok"] = bool(all(v is True for v in Q3["gate0_consistency"].values()))
    # -------- Q4 Fourier / propagators / i0
    Q4 = {"fourier": "f(x) = int d^4k/(2pi)^4 e^{ikx} f~(k), d_mu -> i k_mu, incoming momenta (Gate 0: d -> ik
  +> )",
          "eta": "diag(+,-,-,-)",
```

```
        "graviton": "<h_a h_b>(q) = i (P^{-1})_{ab} / (q^2 + i0)  (independent components; de Donder)",
        "ghost": "<c^mu(q) cbar_rho(-q)> = i (M0^{-1})_{mu rho} = -delta_{mu rho}/(q^2 + i0), M0 = -i q^2 de
  +> lta",
        "scalar": "<phi phi>(q) = i/(q^2 + i0)",
        "b": "b retained: mixed (h,b) inverse verified; b-lines never enter a loop (no b vertex)",
        "i0": "Feynman: every propagator denominator q^2 + i0 (D_F = i/(k^2+i0), Gate 0)",
        "checks": {"Pinv_equals_deDonder": Pr["graviton"]["Pinv_equals_deDonder_tensor_at_representatives"],
                   "schur_q2P": Pr["graviton"]["schur_equals_q2P"],
                   "full_h_b_inverse": Pr["graviton"]["full_h_b_inverse_verified"],
                   "ghost_M0_q2_const": Pr["ghost"]["M0_is_q2_times_constant"],
                   "D_F convention sheet": "D_F(k)=i/(k^2+i0)" in conv,
                   "d -> ik convention sheet": "\\partial_\\mu\\to ik_\\mu" in conv,
                   "propagators_match_transformed_Hessians_P13": gf["propagators_match_transformed_Hessians"
  +> ]}}
    Q4["ok"] = bool(all(v is True for v in Q4["checks"].values()))
    # -------- Q5 coupling / loop bookkeeping
    V = F.vertices()
    vk = {nm: {"legs": len(vt.legs), "kappa_power": vt.kappa_power,
               "kappa_eq_legs_minus_2": vt.kappa_power == len(vt.legs) - 2} for nm, vt in V.items()}
    gk = {}
    for g, vs in GRAPH_VERTICES.items():
        s = sum(V[v].kappa_power for v in vs)
        pref = (amp or {}).get("graphs", {}).get(g, {}).get("prefactor")
        m = re.search(r"kappa\*\*(\d+)", pref or "")
        kp = int(m.group(1)) if m else (1 if pref and "kappa" in pref else None)
        gk[g] = {"vertices": vs, "sum_n_minus_2": s, "external_legs": GRAPH_EXTERNAL[g], "integrand_prefactor"
  +> : pref,
                 "integrand_kappa_power": kp, "ok": s == GRAPH_EXTERNAL[g] and (kp is None or kp == s)}
    Q5 = {"rule": "S_Psi = sum_n kappa^n S^(n); each vertex with n legs is O(kappa^{n-2}); one loop: Gamma_1 a
  +> t kappa^k "
                  "has exactly k external legs (sum over vertices of (n_v - 2) = E); hbar counts loops (one lo
  +> op = hbar^1)",
          "orders": "P.5 scope: one loop, kappa^1 (one-point) and kappa^2 (two-point incl. K'c)",
          "vertices": vk, "graphs": gk, "amplitude_cert_present": amp is not None}
    Q5["ok"] = bool(all(v["kappa_eq_legs_minus_2"] for v in vk.values()) and all(v["ok"] for v in gk.values())
                    and amp is not None and all(v["integrand_kappa_power"] is not None for v in gk.values()))
    # -------- Q6 classical pullback vs quantum theorem
    rel = gf["source_identity_relation"]
    Q6 = {"classical_pullback": rel["relation"], "classical_instance_verified": rel["classical_instance_verifi
  +> ed"],
          "status_P13": rel["status"],
          "statement": "Gamma_Psi is DEFINED here directly in transformed variables. The relation Gamma_Psi(Ph
  +> i,Phi*') = "
                       "Gamma(Phi, Phi*' - dPsi/dPhi) is used ONLY as the classical pullback (verified at tree
  +>  level, "
                       "P13). No quantum gauge-(in)dependence theorem is asserted or used; any such statement
  +> is a "
                       "separate (p15+) object.",
          "quantum_gauge_change_theorem_asserted": False}
    Q6["ok"] = bool(rel["classical_instance_verified"] and "NOT an independently established quantum" in rel["
  +> status"])
    ok = all(q["ok"] for q in (Q1, Q2, Q3, Q4, Q5, Q6))
    rec = {"landmark": "XVI-P", "revision": PC.REVISION, "deliverable": "P.5 entry D1: quantum object",
           "quantum_action_defined": ok, "definition": "Q1..Q6 all ok",
           "S_Psi": "sum_k kappa^{k-2} ell_k + S_phi + X(Phi, Phi*' + eps dPsi/dPhi), eps = -1",
           "Q1_transformed_variables": Q1, "Q2_fields_antifields_b": Q2, "Q3_1PI_vacuum_onepoint": Q3,
           "Q4_fourier_propagators_i0": Q4, "Q5_kappa_loop_bookkeeping": Q5, "Q6_pullback_vs_theorem": Q6,
           "loop_evaluations_performed": False, "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{PC.CERT_PREFIX}_quantum_object.json", rec)
    print(json.dumps({"quantum_action_defined": ok, **{k: rec[k]["ok"] for k in rec if k.startswith("Q")},
                      "b_elim": {k: elim[k] for k in ("b_hessian_det", "cbar_star_quadratic_terms")},
                      "Gamma1_indep": Q2["Gamma1_independent_of"], "seconds": rec["seconds"]}, indent=1))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

121 Source: prescription_d11.py

[listing 88/133] code/p5_entry/prescription_d11.py

```
#!/usr/bin/env python3
"""XVI-P p14r Deliverable 2: ONE quantum prescription and the D11 matching note against XVI-O's frozen D11.

Prescription (P side): dimensional reduction (DRED) made OPERATIONAL in code/p5_entry/dred.py (rules R1-R6 of
certificates/P14_dred_rules.json: g4 = ghat + gtilde, tr ghat = d, tr gtilde = 4-d; l and p in QdS; exported l
  +> _r lifted as
```

```
l.e_(r); Gram/PV reduction with ghat projections; all contractions before the Laurent expansion with d symboli
  +> c;
evanescent remainders retained), MS-bar at scale mu, Feynman i0 on every propagator denominator.
quantum_prescription_defined requires dred_rules_verified.
The rules for scaleless integrals are PART of the definition, but they are NOT applied in p14: no tadpole (D02
  +> , D12,
D14a-c) is assigned a value, and D12 stays Missing.

D11 matching (defined, NOT verified):
  M0 baseline S_inv (not S_D): no switch/insertion fields; a1 = (1/2) tau eta - h equals XVI-O's vertex.
  M1 EXACT integrand identity (computed here): -i * [P's exported D11 integrand] == XVI-O's
       (i kappa^2/4) V V (C~C~ + C~C~) with C~_ab(k) = k_a k_b D~_F(k), D~_F = i/(k^2+i0), V = XVI-O's own V_t
  +> ensor
       (imported from code/matter_loop/d11_tensor.py), mapped to P's independent lower components by the eta-r
  +> aising map
       with orbit multiplicity; all 100 component pairs, as rational functions of (l, p) with the i0-denominat
  +> ors.
  M2 nonlocal agreement on U = R^4 \\ {0}: DEFINED only (needs the evaluated DR amplitude; p15).
  M3 (CONDITIONAL on M2) local difference L_match := FT[P^DR] - Pi_tr^R.  Scalar extension u^R_mu: scaling deg
  +> ree 4;
       kernel K u^R_mu: scaling degree at most eight; degree of divergence at most four; hence the local ambig
  +> uity has
       derivative order at most four (sd(d^alpha delta) = 4 + |alpha|).  IF M2 holds, L_match is supported at
  +> {0} and lies
       in the bounded contact space (orders 0, 2, 4; enumerated).  Exactly degree four is an EXPLICIT normaliz
  +> ation
       condition N4 (mu^4 and mu^2 p^2 contacts excluded by N4, not by 'no other scale'); under N4 the space i
  +> s the
       degree-4 hh span (computed: 5; transverse subspace computed: 2).
  M4 finite adjustments: P-side counterterms in the hh sector of P14_counterterm_domain; u^R_mu untouched (no
  +> c delta).
  M5 residue r = -i pi^2 is an XVI-O-side statement valid only after constructive holomorphy
       (lem:analytic_FP_u_mu_R_o70); P's 1/eps pole is P-side; no pole <-> residue identification is asserted.
  M6 provenance: XVI-O o70 files byte-pinned (tools/source_manifest.check_xvio_provenance).
  M7 the XVI-O kernel Ward identity is NOT functional ST4b; projected != bulk.
"""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
sys.path.insert(0, str(ROOT / "code" / "matter_loop"))
sys.path.insert(0, str(ROOT / "tools"))
import feynman as F  # noqa: E402
import amplitude_rules as A  # noqa: E402
import counterterm_domain as CT  # noqa: E402
import p14_conventions as PC  # noqa: E402
import certio  # noqa: E402
import d11_tensor as O  # noqa: E402   (XVI-O frozen code; imported, not edited)

I = sp.I
ETA = sp.diag(1, -1, -1, -1)

PRESCRIPTION = {
    "id": "P14_DRED_MSbar_FeynmanI0",
    "regularization": "dimensional reduction (DRED), operational rules R1-R6 (certificates/P14_dred_rules.json
  +> , "
                      "code/p5_entry/dred.py): quasi-4D metric g4 = ghat + gtilde, tr g4 = 4, tr ghat = d, "
                      "tr gtilde = 4 - d; loop momentum l and external momenta p in the d-dimensional space Qd
  +> S "
                      "(p.gtilde = 0); field/vertex/polarisation indices quasi-4D; int d^d l/(2 pi)^d with mu^
  +> {2 eps}",
    "lifting": "exported numerators are polynomials in lower components l_r, p_r at fixed external indices; l_
  +> r := "
              "l.e_(r) (e_(r) the eta-orthonormal basis of Q4S), p_r = p.e_(r); vertex tensors lifted with et
  +> a -> g4",
    "subtraction": "MS-bar: subtract the pole in the combination 1/eps - gamma_E + log 4 pi; scale mu",
    "i0": "Feynman: every propagator denominator q^2 + i0 (eta = +,-,-,-); Wick rotation l0 = i l4 implied",
    "tensor_reduction": "Gram/Passarino-Veltman projection on symmetric ghat/p tensors with symbolic d; free q
  +> uasi-4D "
                        "indices are projected with ghat_rs = eta_rs - Gt_rs (Gt_rs evanescent, eta^{rs} Gt_rs
  +>  = 4-d, "
                        "Gt_rs p^s = 0)",
    "order_of_operations": "ALL index contractions (incl. the external 4D projection) BEFORE the Laurent expan
```

```
  +> sion; d "
                           "symbolic through MS-bar subtraction and finite-part extraction; (d-4) x pole terms
  +>  retained; "
                           "premature d = 4 is detected (PrematureD4Substitution)",
    "evanescent": "evanescent remainders DO arise (Gt structures in every exported graph; (4-d) x pole finite
  +> terms after "
                  "contraction); they are retained, never dropped (the p14 'none arise' statement is withdrawn
  +> )",
    "literature_background_only": "Siegel PLB 84 (1979) 193, PLB 94 (1980) 37; Stockinger hep-ph/0503129; "
                                  "Signer-Stockinger arXiv:0807.4424",
    "scaleless_integrals": "PART of the definition (DR value of a scaleless massless integral) but NOT APPLIED
  +>  in p14: "
                           "no tadpole value is assigned; D12 stays Missing; never replaced by zero",
    "massless": True, "IR": "no IR regulator needed at the integrand level; IR/UV separation of scaleless inte
  +> grals is "
                            "deferred together with their evaluation (p15+)",
    "applies_to": ["D01", "D02", "D03", "D11 (P side, matching only)", "D12", "D13", "D14a", "D14b", "D14c"],
    "consistency_with_XVI_O": "XVI-O fixes D12/one-point by dimensional_regularization_m0 (coincident prescrip
  +> tion); "
                              "P's DR is the same family; P does NOT import XVI-O's prescription VALUE 0 (D12
  +> Missing). "
                              "XVI-O's D11 extension is analytic FP in position space (NOT momentum MS); the t
  +> wo are "
                              "related only through matching (M2-M4), not identified.",
}


def raise_map(A_):
    """d h^{mu nu}/d h_(A): orbit multiplicity (both orderings for off-diagonal independent components)."""
    m, n = A_
    R = sp.zeros(4, 4)
    R[m, n] += ETA[m, m] * ETA[n, n]
    if m != n:
        R[n, m] += ETA[m, m] * ETA[n, n]
    return R


def M1_identity():
    t0 = time.time()
    ints = A.build_integrands(A.base_rules())
    r = ints["D11"]
    body, sha = A.integrand_record("D11", r)
    exp_f = ROOT / "exports" / "P14_integrands" / "D11.json"
    exported_sha = json.loads(exp_f.read_text())["sha256"] if exp_f.is_file() else None
    l = sp.symbols("l0:4")
    p = sp.symbols("p0:4")
    D1, D2 = sp.symbols("D1 D2")           # l^2 + i0, (l-p)^2 + i0  (P's recorded denominators)
    kap = F.KAPPA
    names = list(l) + list(p)
    q = [l[a] - p[a] for a in range(4)]
    Ct = lambda k, a, b, D: k[a] * k[b] * I / D  # noqa: E731   C~_ab(k) = k_a k_b D~_F(k)
    # XVI-O kernel in momentum space: Pi~_{mu nu rho sigma} = (i k^2/4) V^{ab}_{mu nu} V^{cd}_{rho sigma}
    #   [C~_ac(l) C~_bd(q) + C~_ad(l) C~_bc(q)]   (FT of C(z)C(z) = convolution; C~ even)
    V = {(mu, nu, a, b): O.V_tensor(mu, nu, a, b, ETA) for mu in range(4) for nu in range(4)
         for a in range(4) for b in range(4)}
    G = {}
    for a in range(4):
        for b in range(4):
            for c in range(4):
                for d in range(4):
                    G[(a, b, c, d)] = sp.expand(Ct(l, a, c, D1) * Ct(q, b, d, D2) + Ct(l, a, d, D1) * Ct(q, b,
  +>  c, D2))
    Vh = {}
    for Aidx, Acomp in enumerate(F.HC):
        R = raise_map(Acomp)
        Vh[Aidx] = {(a, b): sp.expand(sum(R[mu, nu] * V[(mu, nu, a, b)] for mu in range(4) for nu in range(4))
  +> )
                    for a in range(4) for b in range(4)}
    bad, checked, nonzero = [], 0, 0
    pref = r["prefactor"]
    for Aidx in range(10):
        for Bidx in range(10):
            O_ = sum(Vh[Aidx][(a, b)] * Vh[Bidx][(c, d)] * G[(a, b, c, d)]
                     for a in range(4) for b in range(4) for c in range(4) for d in range(4)
                     if Vh[Aidx][(a, b)] != 0 and Vh[Bidx][(c, d)] != 0)
            O_ = sp.expand(I * kap ** 2 / 4 * O_)
            Np = r["N"].get((Aidx, Bidx))
            Ps = Np.to_sympy(names) if Np is not None else sp.Integer(0)
            P_ = sp.expand(-I * pref * Ps / (D1 * D2))
            checked += 1
            nonzero += O_ != 0
```

```
            if sp.expand(P_ - O_) != 0:
                bad.append([str(F.HC[Aidx]), str(F.HC[Bidx])])
    return {"identity": "-i * prefactor_P * N^P_{AB}(l,p)/(D1 D2)  ==  (i kappa^2/4) R_A R_B V V [C~(l)C~(l-p)
  +>  + "
                        "C~(l)C~(l-p)]_{AB}",
            "D1": "l^2 + i0", "D2": "(l-p)^2 + i0", "P_prefactor": str(pref), "P_routing": r["routing"],
            "XVI_O_V_tensor": "code/matter_loop/d11_tensor.py::V_tensor (frozen, imported)",
            "XVI_O_prefactor": "i kappa^2/4 (O5_Gate2_normalized_D11)",
            "C_tilde": "C~_ab(k) = k_a k_b * i/(k^2+i0)  (FT of C = -d_a d_b D_F, d -> ik)",
            "raising_map": "h^{mu nu} = eta^{mu a} eta^{nu b} h_ab; d/dh_(ab) carries orbit multiplicity 2 for
  +>  a != b",
            "normalisation": "Gamma_1^(2) = (1/2) int h^{mu nu} Pi_{mu nu rho sigma} h^{rho sigma} (XVI-O) <->
  +>  "
                             "Gamma_1^{AB} = d^2 Gamma_1/dh_A dh_B (P ordered extraction); i Gamma_1 = graph s
  +> um",
            "component_pairs_checked": checked, "nonzero_pairs": int(nonzero), "mismatches": bad,
            "recomputed_integrand_sha256": sha, "exported_integrand_sha256": exported_sha,
            "bound_to_export": exported_sha == sha if exported_sha else None,
            "pass": bool(not bad and checked == 100 and nonzero > 0 and (exported_sha in (None, sha))),
            "seconds": round(time.time() - t0, 1)}


def M3_local_space(order=4):
    """degree-<order> local hh structures in XVI-O's convention (lower kernel indices; symmetric in mu<->nu, r
  +> ho<->sigma,
    (mu nu)<->(rho sigma)) and their transverse subspace (p^mu T_{mu nu rho sigma} = 0)."""
    p = sp.symbols("p0:4")
    slots = [(0, "U"), (1, "U"), (2, "U"), (3, "U")]   # field h^{mu nu} upper => kernel indices lower (p_a, e
  +> ta_ab)
    S = CT.structures(slots, order)
    perms = [(0, 1, 2, 3), (1, 0, 2, 3), (0, 1, 3, 2), (1, 0, 3, 2), (2, 3, 0, 1), (3, 2, 0, 1), (2, 3, 1, 0),
  +>  (3, 2, 1, 0)]
    comps = [c for c in __import__("itertools").product(range(4), repeat=4)]
    mons = None
    rows = []
    for T in S:
        sym = {}
        for c in comps:
            sym[c] = sp.expand(sum(T.get(tuple(c[i] for i in pm), 0) for pm in perms) * sp.Rational(1, 8))
        rows.append(sym)
    # rank of the symmetrised structures
    def vec(sym):
        out = {}
        for c, v in sym.items():
            if v != 0:
                for mon, cf in sp.Poly(v, *p).terms():
                    out[(c, mon)] = cf
        return out
    from fractions import Fraction as Fr
    vecs = [{k: Fr(int(v.p), int(v.q)) for k, v in vec(s).items()} for s in rows]
    keep, kv = [], []
    for s, v in zip(rows, vecs):
        if CT.rank(kv + [v]) > len(kv):
            kv.append(v)
            keep.append(s)
    dim = len(keep)
    # transversality: sum_mu p^mu T_{mu nu rho sigma} = 0 for the general combination
    x = sp.symbols(f"x0:{dim}")
    eqs = []
    for nu in range(4):
        for rho in range(4):
            for sg in range(4):
                e = sp.expand(sum(x[i] * sum(ETA[mu, mu] * p[mu] * keep[i][(mu, nu, rho, sg)] for mu in range(
  +> 4))
                                  for i in range(dim)))
                if e != 0:
                    eqs += [cf for _, cf in sp.Poly(e, *p).terms()]
    M = sp.Matrix([[sp.diff(e, xi) for xi in x] for e in eqs]) if eqs else sp.zeros(0, dim)
    tdim = dim - M.rank()
    return {"order": order, "dimension": dim, "transverse_subspace_dimension": int(tdim),
            "matches_counterterm_hh_sector": None}


M3_WORDING = ("Kernel scaling degree at most eight; degree of divergence at most four; hence the local ambigui
  +> ty has "
              "derivative order at most four.")
N4 = ("normalization condition N4 (EXPLICIT, imposed): L_match is homogeneous of degree exactly four in p, i.e
  +> . it "
      "contains no mu^4 (order-0) and no mu^2 p^2 (order-2) contact terms.  This is NOT implied by 'no other s
  +> cale': "
      "mu itself is a scale, so mu^{4-|alpha|} d^alpha delta with |alpha| = 0, 2 are dimensionally allowed")
```

```
def M3_contact_analysis(glob, ext):
    """corrected M3: scaling-degree labels from the XVI-O certificates, the conditional statement, and the exp
  +> licit
    enumeration of the bounded contact space (orders 0, 2, 4; odd orders impossible with 4 indices from p and
  +> eta)."""
    sd_u = ext["u_mu_R"]["sd_max"]
    kk = glob["Pi_tr"]["scaling_degree_K_uR"]
    sd_k = kk["sd_bound"]
    labels = {"scalar_extension_scaling_degree": sd_u, "kernel_differential_order": kk["differential_order_K"]
  +> ,
              "kernel_scaling_degree_bound": sd_k, "degree_of_divergence_bound": sd_k - 4,
              "local_contact_derivative_order_bound": sd_k - 4,
              "contact_scaling_degree_rule": "sd(d^alpha delta) = 4 + |alpha|  (in 4 dimensions)",
              "labels_consistent": bool(sd_u == 4 and sd_k == 8 and sd_u + kk["differential_order_K"] == sd_k)
  +> }
    orders = {o: M3_local_space(o) for o in (0, 2, 4)}
    return {
        "labels": labels, "wording": M3_WORDING,
        "conditional_on": "M2_nonlocal_agreement_on_U (defined, NOT verified)",
        "conditional_statement": "IF M2 holds, then L_match vanishes on U = R^4 \\ {0}, hence is supported at
  +> {0}; "
                                 "with kernel scaling degree <= 8 it is a finite sum of d^alpha delta with "
                                 "4 + |alpha| <= 8, i.e. it lies in the bounded contact space of derivative or
  +> der <= 4",
        "bounded_contact_space": {f"order_{o}": {"mu_power": 4 - o, "dimension": r["dimension"],
                                                 "transverse_subspace_dimension": r["transverse_subspace_dimen
  +> sion"]}
                                  for o, r in orders.items()},
        "odd_orders": "absent: four tensor indices built from p and eta need an even number of p's",
        "normalization_condition": N4,
        "lower_degree_contacts_excluded_by_N4": {f"order_{o}_mu{4 - o}": orders[o]["dimension"] for o in (0, 2
  +> )},
        "under_N4": "L_match lies in the order-4 (degree-4) hh span", "_order4": orders[4]}


def main():
    t0 = time.time()
    cert = lambda n: json.loads((ROOT / "certificates" / f"{n}.json").read_text())  # noqa: E731
    g2 = cert("O5_Gate2_normalized_D11")
    ext = cert("O5_D11_scalar_extension")
    glob = cert("O5_D11_global_representation")
    g4 = cert("O5_Gate4_Ward_D12_onepoint")
    ten = cert("O5_D11_tensor")
    ww = cert("O5_D11_Ww_reference")
    ctd_f = ROOT / "certificates" / f"{PC.CERT_PREFIX}_counterterm_domain.json"
    ctd = json.loads(ctd_f.read_text()) if ctd_f.is_file() else None
    qo_f = ROOT / "certificates" / f"{PC.CERT_PREFIX}_quantum_object.json"
    qo = json.loads(qo_f.read_text()) if qo_f.is_file() else None
    import source_manifest as SM
    prov = SM.check_xvio_provenance(ROOT)
    # ---- prescription checks
    P = F.propagators()
    pc = {"propagators_are_verified_Hessian_inverses": bool(P["graviton"]["full_h_b_inverse_verified"]
                                                            and P["ghost"]["M0_is_q2_times_constant"]),
          "scalar_i0_matches_Gate0": g2["D_F"] == "i/(k^2+i0)",
          "XVI_O_coincident_prescription_is_DR_m0": (g4["D12"]["coincident_prescription"] == "dimensional_regu
  +> larization_m0"
                                                     and g4["one_point"]["prescription"] == "dimensional_regul
  +> arization_m0"),
          "XVI_O_D11_is_NOT_momentum_MS": ext["NOT_momentum_MS"] is True,
          "no_value_imported_D12": True,
          "quantum_object_defined": bool(qo and qo.get("quantum_action_defined"))}
    dr_f = ROOT / "certificates" / f"{PC.CERT_PREFIX}_dred_rules.json"
    dr = json.loads(dr_f.read_text()) if dr_f.is_file() else None
    pc["operational_DRED_rules_verified"] = bool(dr and dr.get("dred_rules_verified") is True
                                                 and dr.get("mutations_all_rejected") is True)
    pc["evanescent_remainders_retained"] = bool(dr and dr.get("evanescent_parts_present_in_exports") is True)
    power = {}
    amp_f = ROOT / "certificates" / f"{PC.CERT_PREFIX}_amplitude_rules.json"
    amp = json.loads(amp_f.read_text()) if amp_f.is_file() else None
    if amp:
        for g, rec in amp["graphs"].items():
            nden = len(rec["denominators"])
            deg = rec["numerator_degree"]
            power[g] = {"denominators": rec["denominators"], "numerator_degree": deg,
                        "superficial_degree": 4 + deg - 2 * nden,
                        "all_feynman_i0": all(d.endswith("+i0") for d in rec["denominators"])}
    pc["all_denominators_feynman_i0"] = bool(power) and all(v["all_feynman_i0"] for v in power.values())
    prescription_ok = all(v is True for v in pc.values())
```

```
  # ---- D11 matching
  m0 = {"baseline": "S_inv (XVI-N invariant extended action), NOT S_D",
        "no_switch_fields": bool(qo and qo["Q1_transformed_variables"]["no_switch_or_insertion_fields"]),
        "a1_vertex_XVI_O": g2["vertex_tensor"]["from"], "a1_P": "a1 = (1/2) tau eta - h (scalar_B_series, XI
+> -A)",
        "NOT_S_D_P13": json.loads((ROOT / "certificates" / "P13_gauge_fixing_map.json").read_text())[
            "invariant_vs_transformed_kept_separate"]["NOT_S_D"]}
  m0["pass"] = bool(m0["no_switch_fields"] and m0["NOT_S_D_P13"] and g2["vertex_tensor"]["from"] == "a1 = (1
+> /2) tau eta - h")
  m1 = M1_identity()
  m3c = M3_contact_analysis(glob, ext)
  o4 = m3c.pop("_order4")
  m3 = {**m3c, "dimension": o4["dimension"], "transverse_subspace_dimension": o4["transverse_subspace_dimens
+> ion"],
        "matches_counterterm_hh_sector": None}
  if ctd:
      m3["matches_counterterm_hh_sector"] = ctd["kappa2"]["basis_size_by_sector"].get("h.h") == m3["dimensio
+> n"]
  ans = g4["contact_ansatz"]["basis"]
  m3["XVI_O_contact_ansatz_ids"] = [b["id"] for b in ans]
  m3["XVI_O_order4_elements"] = sum(1 for b in ans if b["order"] == 4)
  m3["XVI_O_ansatz_NOT_proved_spanning"] = g4["contact_ansatz"].get("NOT_proved_spanning")
  m3["comparison_with_XVI_O_ansatz"] = (
      "XVI-O's contact ansatz is a candidate list (NOT_proved_spanning) with 3 named order-4 entries, one of
+>  them "
      "the collective 'L4_mixed'; P enumerates the order-4 space explicitly: 5 structures (transverse subspa
+> ce 2). "
      "No contradiction; P's list is the explicit one used for M3/M4")
  m3["XVI_O_order_lt_4_elements_excluded_by_N4"] = [b["id"] for b in ans if b["order"] < 4]
  m3["pass"] = bool(m3["dimension"] == 5 and m3["matches_counterterm_hh_sector"] is True
                    and m3["labels"]["labels_consistent"]
                    and all(v["dimension"] > 0 for v in m3["bounded_contact_space"].values()))
  matching = {
      "baseline_M0": m0,
      "lemmas_referenced": {
          "lem:d11-prefactor": {"XVI_O": "Pi = (i kappa^2/4) T", "status": g2["status_prefactor"], "pass": g
+> 2["pass"],
                                "used_in": "M1 (prefactor i kappa^2/4 imported, compared exactly)"},
          "lem:analytic_FP_u_mu_R_o70": {"prescription_id": ext["prescription_id"], "pass": ext["pass"],
                                         "u_mu_R": ext["u_mu_R"], "used_in": "M2/M5 (XVI-O side only)"},
          "lem:BV_rules_on_U": {"status": glob["lem_BV_rules_on_U"]["status"], "pass": glob["lem_BV_rules_on
+> _U"]["pass"],
                                "all_256_symbolic": glob["all_256_symbolic"]["pass"],
                                "alpha": glob["coefficients"]["alpha"], "beta": glob["coefficients"]["beta"]
+> ,
                                "used_in": "M2 (XVI-O's Pi_tr^R on U is the matching target)"}},
      "M1_integrand_identity": m1,
      "M2_nonlocal_agreement_on_U": {
          "definition": "for all test functions f in D(U), U = R^4 \\ {0}: <FT^{-1}[P^DR_{AB}](z), f> = "
                        "<R_A R_B Pi_tr^R(z), f>, Pi_tr^R = (i kappa^2/4) K u^R_mu (alpha = A^2/60, beta = A
+> ^2/18, "
                        "A = 1/(4 pi^2)); mu-independent on U",
          "status": "defined_not_verified", "requires": "evaluated DR D11 amplitude (Batch, p15+)",
          "integrand_level_evidence": "M1 (exact integrand identity) -- not a proof of M2"},
      "M3_local_difference": {"definition": "L_match := FT^{-1}[P^DR_{AB}] - R_A R_B Pi_tr^R, a distribution
+>  "
                                            "supported at z = 0 IF M2 holds", **m3,
                              "status": "conditional on M2; space enumerated; degree 4 imposed by N4; L_matc
+> h not computed"},
      "M4_finite_adjustments": {
          "rule": "under M2 and N4, L_match (and any finite renormalization) is booked ONLY as P-side counte
+> rterms "
                  "in the hh sector of P14_counterterm_domain (5 elements; transverse subspace 2 if the P am
+> plitude "
                  "is transverse); u^R_mu is NOT modified; without N4 the order-0/2 contacts (mu^4, mu^2 p^2
+> ) would "
                  "have to be added to that sector",
          "u_R_extra_c_delta_XVI_O": ext["u_mu_R"]["extra_c_delta"], "u_R_c_delta_added_by_P": False,
          "scalar_extension_prescription_preserved": ext["prescription_id"] == "analytic_FP_u_mu_R_o70",
          "XVI_O_D11_kernel_counterterm": g4["D11_kernel_counterterm"]},
      "M5_residue": {"r": ext["minkowski_residue"]["r_with_sign_and_i"],
                     "derived_from": ext["minkowski_residue"]["r_derived_from"],
                     "holomorphy_proof_pass": ext["phi_m1_holomorphy_proof"]["pass"],
                     "statement": "r = -i pi^2 is XVI-O-side, valid only AFTER constructive holomorphy of "
                                  "F(zeta) = (Q-i0)^zeta; P's MS-bar 1/eps pole is P-side; no identification
+>  of the "
                                  "P pole with r is asserted in p14"},
      "M6_provenance": {"files": len(prov), "all_pinned_ok": all(v["ok"] for v in prov.values()),
                        "pins": {k: v["pin"] for k, v in prov.items() if "d11" in k or k.endswith(".tex")}},
      "M7_projected_vs_bulk": {"XVI_O_kernel_Ward_NOT_functional_ST4b":
                               g4["Ward_and_CT"]["kernel_two_point_Ward"]["NOT_functional_ST4b"],
```

```
                                "symmetry_restored_bulk_XVI_O": g4["symmetry_restored_bulk"],
                                "statement": PC.PROJECTED_VS_BULK},
        "conventions": {"XVI_O_index_convention": ten["index_convention"], "XVI_O_Pi": ten["Pi"],
                        "P_components": "independent lower h_(mn), m<=n", "map": m1["raising_map"],
                        "W_w_reference_id": ww["prescription_id"]},
    }
    prov_ok = bool(prov) and all(v["ok"] for v in prov.values())
    D11_defined = bool(m0["pass"] and m1["pass"] and m3["pass"] and prov_ok and g2["pass"] and ext["pass"]
                       and glob["pass"] and g4["Ward_and_CT"]["kernel_two_point_Ward"]["NOT_functional_ST4b"]
                       and ext["u_mu_R"]["extra_c_delta"] is False)
    rec = {"landmark": "XVI-P", "revision": PC.REVISION,
           "deliverable": "P.5 entry D2: quantum prescription and D11 matching note",
           "quantum_prescription_defined": prescription_ok, "D11_matching_defined": D11_defined,
           "D11_matching_verified": False,
           "D11_matching_verified_reason": "M2 (nonlocal agreement on U) and L_match require the evaluated DR
  +> D11 "
                                           "amplitude (p15+); only M0, M1, M3-space, M4-M7 are established",
           "prescription": PRESCRIPTION, "prescription_checks": pc, "power_counting": power,
           "D12_status": "Missing", "D12_value_imported": False,
           "matching": matching, "loop_evaluations_performed": False, "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{PC.CERT_PREFIX}_prescription_D11_matching.json", rec)
    print(json.dumps({"quantum_prescription_defined": prescription_ok, "D11_matching_defined": D11_defined,
                      "M1": {k: m1[k] for k in ("component_pairs_checked", "nonzero_pairs", "mismatches", "bou
  +> nd_to_export",
                                                "pass", "seconds")},
                      "M3": {k: m3[k] for k in ("labels", "bounded_contact_space", "dimension",
                                                "transverse_subspace_dimension", "XVI_O_order4_elements",
                                                "matches_counterterm_hh_sector", "pass")},
                      "prov_ok": prov_ok, "checks": pc, "seconds": rec["seconds"]}, indent=1))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

122 Source: dred.py (operational DRED rules, p14r; disclosed revisions: p15 structural R5 safeguard, p16 F3 hardening, p17 G2 guarded loader parse_d, p18 H3/H5 documentation only, p22 runtime d guard – additive block)

[listing 89/133] code/p5_entry/dred.py

```
#!/usr/bin/env python3
"""XVI-P p14r: OPERATIONAL dimensional reduction (DRED) for the P.5 prescription.

Index algebra, lifting rule for the exported integrands, tensor reduction with ghat projections, and the
Laurent / MS-bar finite-part extraction with d kept symbolic.  NO loop integral is evaluated: scalar integrals
  +>  stay
symbols J_<graph>[a,b]; the toy checks use GENERIC Laurent placeholders  a/eps + b + c*eps  (not values of any
integral of this theory).

Background literature (cited as background only; nothing is imported from it): W. Siegel, Phys. Lett. B84 (197
  +> 9)
193 and B94 (1980) 37; D. Stockinger, JHEP 0503 (2005) 076, hep-ph/0503129; A. Signer, D. Stockinger,
Nucl. Phys. B808 (2009) 88, arXiv:0807.4424.

R1 spaces.  Q4S (quasi-4-dimensional; metric g4, tr g4 = 4) = QdS (metric ghat, tr ghat = d) (+) Q(4-d)S (metr
  +> ic
   gtilde, tr gtilde = 4 - d = 2 eps):  g4 = ghat + gtilde,  ghat.ghat = ghat,  gtilde.gtilde = gtilde,
   ghat.gtilde = 0,  g4.X = X.
R2 momenta.  The loop momentum l lies in QdS.  The external momenta p lie in QdS as well (stated choice:
   p.gtilde = 0).  Field, vertex and external-polarisation indices are Q4S indices.
R3 lifting.  The exported numerators are polynomials in LOWER components l_r, p_r (r = 0..3) at fixed external
   component indices.  l_r is read as l.e_(r), with e_(r) the fixed eta-orthonormal basis of Q4S
   (e_(r).e_(s) = eta_rs), so l_r = l.ehat_(r); p_r = p.e_(r) = p.ehat_(r).  The vertex tensors (built from et
  +> a and
   momenta) are lifted with eta -> g4, so the exported component polynomials ARE the lifted objects at fixed
   Q4S components; nothing is truncated.
R4 reduction.  int l^{mu1..mun} F(l^2, l.p) = sum_k C_k B_k, where B_k are the symmetric tensors built from gh
  +> at and
   p.  C_k comes from the Gram (Passarino-Veltman) projection, exact in symbolic d.  Contracting with the lift
  +> ed
   e_(r) gives  ghat_rs := ehat_(r).ehat_(s) = eta_rs - Gt_rs,  with  Gt_rs := etilde_(r).etilde_(s)  evanesce
  +> nt:
```

```
   eta^{rs} Gt_rs = 4 - d,  Gt_rs p^s = 0.
R5 order.  Every index contraction, including the external 4D projection (Gt traces), is done BEFORE the Laure
  +> nt
   expansion.  d stays symbolic through MS-bar subtraction and finite-part extraction, so (d-4) x pole finite
  +> terms
   are retained.  Substituting d = 4 before the expansion is a rule violation.
   p15 CONTRACT CORRECTION (disclosed; finding F1 of the p14r independent rebuild, reproducibility/
   independent_rebuild_report_p14r.json): the p14r safeguard was a HEURISTIC ("the reduction introduced d but
  +> d is
   absent before the Laurent expansion").  It gave a FALSE POSITIVE on eta^{rs} int l_r l_s/(l^2-m^2) = J (a c
  +> orrect
   d-free result) and a FALSE NEGATIVE under the package's own premature_d4 mutation on the same integral
   (eta_trace_Gt reintroduces d via tr gtilde = 4-d: d J/4, finite error -m^4/2).  The p14r docstring claim th
  +> at
   premature d = 4 "is detected" did not hold in general.  It is replaced by a STRUCTURAL, provenance-based
   safeguard: d is an OPAQUE dimension symbol (OpaqueDimension) that cannot be substituted by subs/xreplace at
  +>  all;
   the only substitution path is substitute_d(expr, value, reason), used by Pipeline.laurent (d -> 4 - 2 eps)
  +> and by
   explicitly declared post-expansion/report uses, each recorded in D_PROVENANCE.  Any other substitution of d
  +>  raises
   PrematureD4Substitution AT THE SUBSTITUTION SITE, whatever the later shape of the expression.  Numeric trac
  +> es of
   ghat/gtilde (Rules(trace_hat=...)) are refused by the contraction engine (NumericTraceForbidden) unless con
  +> structed
   display_only=True for a mutation display.  Released p14r values are unaffected: in p14r Pipeline.laurent wa
  +> s used
   only on the toy placeholders (no loop integral was evaluated).
   p16 CONTRACT REVISION (disclosed; finding F3 of the p15 independent rebuild, reproducibility/
   independent_rebuild_report_p15.json): expr.replace(d, 4) and sympy.lambdify bypassed the opaque d.  OpaqueD
  +> imension
   now also refuses pattern matching on d (matches: closes expr.replace / expr.match) and the lambdify / code-
  +> printer
   hooks outside substitute_d.  Value paths that never call back into the symbol (Poly(expr, d).eval, string
   round-trips) are excluded by a static scan of the package (tests/test_p16.py).  Rules R1-R6 unchanged; no r
  +> eleased
   value affected (no package code used these paths).
   p17 CONTRACT REVISION (disclosed; findings G1-G3 of the p16 independent rebuild, reproducibility/
   independent_rebuild_report_p16.json).  G2: sympify of a stored string yields a PLAIN Symbol('d') that is no
  +> t the
   opaque d (subs/has against dred.d then silently do nothing).  New guarded loader parse_d(text) parses with
   locals {'d': opaque d} and refuses any result still containing a plain symbol named d; assert_opaque_d(expr
  +> )
   checks an expression.  G3 (wording): the runtime guard refuses subs, xreplace, replace(d, value), evalf(sub
  +> s), N,
   limit, series in d, Lambda(d, .)(value), Subs.doit, lambdify (python/math/mpmath/sympy printers) and
   expr.match(d) with d itself as the pattern; it does NOT refuse a Wild pattern containing d ((d+1).match(d+w
  +> )
   substitutes no value) nor C/Fortran-type code printers (string output).  G1: value paths with no hook --
   replace with ANY function query (lambda or named function), string round-trips (sympify/parse_expr/S of str
  +> (e),
   builtin eval/exec of srepr), Poly(.., d).eval and hand-written expression-tree rebuilds (e.func(*args)) --
  +> are
   excluded by a static AST scan of code/p5_calc and code/p5_entry (tests/test_p17.py); the scan is a pattern
  +> check
   (not a proof): a path written outside these patterns (e.g. dynamic getattr) would not be caught.
   p18 (findings H1/H3/H5 of the p17 independent rebuild; documentation only, no executable statement changed)
  +> : the
   scan (tests/test_p18.py) now also flags Poly(e, d)(4) / e.as_poly(d)(4) calls, all_coeffs (Horner evaluatio
  +> n),
   type(x)(*args) and x.__class__(*args) rebuilds, getattr with a non-literal or hazardous attribute name, ali
  +> ased
   imports or assignments of sympify/S/parse_expr, sympify locals whose 'd' value is not the opaque dred.d, an
  +> d plain
   Symbol('d') / symbols('d') creation -- still a pattern check, not a proof.  parse_d accepts sympy's str() f
  +> orm
   only: parse_d(srepr(e)) RAISES PlainDimensionSymbol (srepr spells Symbol('d') explicitly), i.e. it fails cl
  +> osed.
R6 evanescent.  Evanescent remainders DO arise in general: Gt structures, and (4-d) x pole terms after contrac
  +> tion.
   They are kept and never dropped.  The p14 statement that none arise is withdrawn.
"""
from __future__ import annotations
import itertools
import json
import sys
import time
from pathlib import Path

import sympy as sp
```

```
HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))

class PrematureD4Substitution(Exception):
    """d was substituted outside the authorised path (substitute_d), i.e. before the Laurent expansion."""


class NumericTraceForbidden(Exception):
    """a numeric trace of ghat/gtilde (e.g. tr ghat = 4) was requested in an enforced contraction."""


import threading  # noqa: E402

_AUTH = threading.local()
D_PROVENANCE = {}          # every authorised substitution of d: reason -> number of calls


class OpaqueDimension(sp.Symbol):
    """the DRED dimension d: opaque -- subs/xreplace of d raise unless issued by substitute_d (structural R5).
  +> """

    def _xreplace(self, rule):
        if self in rule and rule[self] != self:
            if not getattr(_AUTH, "active", False):
                raise PrematureD4Substitution(f"d -> {rule[self]} outside substitute_d (R5: d is opaque before
  +>  the "
                                              "Laurent expansion)")
            return rule[self], True
        return self, False

    def _subs(self, old, new, **hints):
        if old == self and new != self and not getattr(_AUTH, "active", False):
            raise PrematureD4Substitution(f"d -> {new} via subs outside substitute_d (R5)")
        return super()._subs(old, new, **hints)

    # p16 contract revision (finding F3 of the p15 independent rebuild): expr.replace(d, value) and pattern qu
  +> eries on
    # d go through d.matches; lambdify and the code printers go through the printmethod hooks below.  Both rai
  +> se
    # outside substitute_d.  (Value paths that bypass sympy substitution altogether -- Poly(expr, d).eval(4),
  +> string
    # round-trips -- cannot be intercepted by the symbol; the static scan, tests/test_p18.py since p18 (p16/p1
  +> 7:
    # tests/test_p16.py, tests/test_p17.py), checks the package code for them: a pattern check, not a proof.)
    def matches(self, expr, repl_dict=None, old=False):
        if not getattr(_AUTH, "active", False):
            raise PrematureD4Substitution("pattern matching on d (expr.replace(d, ...) / expr.match(d)) outsid
  +> e "
                                          "substitute_d (R5)")
        return super().matches(expr, repl_dict, old)

    def _refuse_code(self, printer, *args, **kwargs):
        if not getattr(_AUTH, "active", False):
            raise PrematureD4Substitution("lambdify / code printing of d outside substitute_d (R5: a numerical
  +>  function "
                                          "of d would evaluate it before the Laurent expansion)")
        return printer._print_Symbol(self)

    _lambdacode = _numpycode = _scipycode = _mpmathcode = _pythoncode = _refuse_code
    _cupycode = _jaxcode = _tensorflowcode = _numexprcode = _refuse_code


class PlainDimensionSymbol(ValueError):
    """a plain Symbol('d') (e.g. from sympify of a stored string) where the opaque d is required (p17, G2)."""


def assert_opaque_d(expr, where=""):
    """raise PlainDimensionSymbol if expr contains a symbol named 'd' that is not the opaque d (p17, G2)."""
    bad = [x for x in sp.sympify(expr).free_symbols if x.name == "d" and not isinstance(x, OpaqueDimension)]
    if bad:
        raise PlainDimensionSymbol(f"plain Symbol('d') in {where or 'expression'}: re-parse with dred.parse_d"
  +> )
    return expr


def parse_d(text, extra_locals=None):
    """the guarded loader for stored strings (p17, G2): 'd' parses to the OPAQUE d; a result that still contai
  +> ns a
    plain symbol named d (impossible unless extra_locals shadows it) is refused.  Input format: sympy str() fo
```

```
  +> rm only;
    an srepr string (Symbol('d') spelled out) yields a plain d and RAISES PlainDimensionSymbol (H5, p18: fails
    closed; srepr round-trips are not supported)."""
    loc = dict(extra_locals or {})
    if "d" in loc and loc["d"] is not d:
        raise PlainDimensionSymbol("extra_locals must not rebind 'd'")
    loc["d"] = d
    return assert_opaque_d(sp.sympify(text, locals=loc), f"parse_d({str(text)[:40]!r})")


def substitute_d(expr, value, reason):
    """the ONLY authorised substitution of d (recorded with its reason in D_PROVENANCE)."""
    _AUTH.active = True
    try:
        out = sp.sympify(expr).xreplace({d: sp.sympify(value)})
    finally:
        _AUTH.active = False
    D_PROVENANCE[reason] = D_PROVENANCE.get(reason, 0) + 1
    return out


d = OpaqueDimension("d")


# ============================================================================== p22 CONTRACT REVISIO
  +> N: runtime d guard
# (finding d_guard_power_base_hole + scan_misses_p21 of the p21 independent rebuild).  p21 relied on hooks of
  +> the symbol
# itself (_xreplace/_subs/matches/printers); sympy reaches values of d without calling them (Pow._eval_subs on
  +>  a power
# base, Basic.xreplace's top-level rule lookup on a bare d, Poly.eval, Expr.coeff reconstruction, evalf(subs))
  +> .  p22 guards
# the ENTRY POINTS instead, installed once at import of dred (every package module imports dred):
#   * Basic.subs / Basic.xreplace / Basic.replace / evalf(subs=..) whose rule has a key containing d;
#   * Poly.eval / __call__ / as_expr(values) and every Poly coefficient accessor (all_coeffs, coeffs, terms, a
  +> ll_terms,
#     as_dict, as_list, coeff_monomial, nth, LC, TC, EC) when d is a generator;
#   * Expr.coeff / as_independent / as_coeff_exponent / series / collect when the target contains d;
#   * d == d (an equality test against d) -- the hook of hand-written e.args rebuilds.
# Rule: from code OUTSIDE sympy/mpmath (package, tests, user code) every such call raises PrematureD4Substitut
  +> ion unless
# issued through substitute_d / authorised_d (recorded in D_PROVENANCE).  From INSIDE sympy (cancel, factor, s
  +> olve,
# Lambda.__call__, Subs.doit, limit ...) polynomial algebra over d is allowed, but a substitution of d by anyt
  +> hing that is
# not a bare symbol (a rename to a Dummy) raises -- so Lambda(d, e)(4), Subs(e, d, 4).doit(), limit(e, d, 4) a
  +> ll raise.
# Irreducible residue (disclosed; the static allow-list scan remains as the SECONDARY barrier): identity / nam
  +> e / type
# tests (`x is d`, x.name == 'd', isinstance) in hand-written tree rebuilds, string round-trips (str(e).replac
  +> e),
# low-level polynomial representations (Poly.rep, DMP, polyutils dict builders), and code that disguises its m
  +> odule name
# as sympy.  These are code-review matters, not runtime-interceptable without breaking sympy itself.
import sys as _sys  # noqa: E402
import contextlib as _contextlib  # noqa: E402
from sympy.core.basic import Basic as _Basic  # noqa: E402
from sympy.core.expr import Expr as _Expr  # noqa: E402
from sympy.core.evalf import EvalfMixin as _EvalfMixin  # noqa: E402
from sympy.polys.polytools import Poly as _Poly  # noqa: E402

GUARD_REVISION = "p22"
GUARDED_ENTRY_POINTS = []


def _internal(depth):
    """True iff the frame `depth` levels above the caller of the wrapper belongs to sympy/mpmath (or to dred i
  +> tself)."""
    try:
        f = _sys._getframe(depth + 1)
    except ValueError:
        return False
    mod = f.f_globals.get("__name__", "") or ""
    return mod.startswith(("sympy.", "mpmath.")) or mod in ("sympy", "mpmath") or mod == __name__


def _has_d(x):
    if x is d or isinstance(x, OpaqueDimension):     # any OpaqueDimension instance (e.g. Symbol.__new__ cache
  +>  miss)
        return True
    if isinstance(x, _Basic) and not x.is_Atom:
```

```
        return x.has(d) or bool(x.atoms(OpaqueDimension))
    return False


def _deny(what):
    raise PrematureD4Substitution(f"{what} outside substitute_d (R5, p22 runtime guard: d is opaque before the
  +>  "
                                  "Laurent expansion)")


def _check_rule(pairs, what, depth=2):
    if getattr(_AUTH, "active", False):
        return
    for old, new in pairs:
        if _has_d(old):
            if not _internal(depth):
                _deny(f"{what} of {old} (caller outside sympy)")
            nb = new if isinstance(new, _Basic) else sp.sympify(new, strict=False) if isinstance(new, (int, fl
  +> oat)) else new
            if not (isinstance(nb, _Basic) and (nb.is_Symbol or _has_d(nb))):
                _deny(f"{what}: {old} -> {new} inside sympy (d-free value)")


def _pairs_of_subs(args, kwargs):
    if len(args) == 2:
        return [(args[0], args[1])]
    if len(args) == 1:
        seq = args[0]
        if isinstance(seq, dict):
            return list(seq.items())
        try:
            return [tuple(x) for x in seq]
        except TypeError:
            return []
    return []


_DEFER = threading.local()          # nested-check suspension while a deferred original call runs


def _deferred(check, call):
    """Guard ordering (p22): if the rule is violated, run the ORIGINAL call first with nested guard checks sus
  +> pended,
    so a pre-existing (p14..p18) hook that raises keeps its own message; if the original call does not raise,
  +> the
    guard raises afterwards.  Either way a violating call raises PrematureD4Substitution (the calls are pure).
  +> """
    if getattr(_DEFER, "on", False):
        return call()
    try:
        check()
    except PrematureD4Substitution as err:
        _DEFER.on = True
        try:
            call()
        except PrematureD4Substitution:
            raise
        except Exception:
            pass
        finally:
            _DEFER.on = False
        raise err
    return call()


def _install():
    o_subs, o_xrep, o_repl, o_evalf = _Basic.subs, _Basic.xreplace, _Basic.replace, _EvalfMixin.evalf

    def subs(self, *args, **kwargs):
        if getattr(_AUTH, "active", False):
            return o_subs(self, *args, **kwargs)
        return _deferred(lambda: _check_rule(_pairs_of_subs(args, kwargs), "subs", 4),
                         lambda: o_subs(self, *args, **kwargs))

    def xreplace(self, rule, *a, **k):
        if not getattr(_AUTH, "active", False) and isinstance(rule, dict) and rule:
            if d in rule or any(_has_d(key) for key in rule if not getattr(key, "is_Atom", True)):
                return _deferred(lambda: _check_rule(rule.items(), "xreplace", 4),
                                 lambda: o_xrep(self, rule, *a, **k))
        return o_xrep(self, rule, *a, **k)
```

```
  def replace(self, query, value, *a, **k):
      if not getattr(_AUTH, "active", False) and isinstance(query, _Basic) and _has_d(query):
          return _deferred(lambda: _deny(f"replace({query}, ...)"), lambda: o_repl(self, query, value, *a, *
+> *k))
      return o_repl(self, query, value, *a, **k)

  def evalf(self, n=15, subs=None, *a, **k):
      if subs and not getattr(_AUTH, "active", False):
          return _deferred(lambda: _check_rule(subs.items(), "evalf(subs=...)", 4),
                           lambda: o_evalf(self, n, subs, *a, **k))
      return o_evalf(self, n, subs, *a, **k)

  _Basic.subs, _Basic.xreplace, _Basic.replace = subs, xreplace, replace
  from sympy.core.basic import Atom as _Atom
  o_axrep = _Atom.xreplace

  def atom_xreplace(self, rule, *a, **k):
      if self is d and not getattr(_AUTH, "active", False) and isinstance(rule, dict) and d in rule:
          return _deferred(lambda: _check_rule([(d, rule[d])], "xreplace on d", 4),
                           lambda: o_axrep(self, rule, *a, **k))
      return o_axrep(self, rule, *a, **k)
  _Atom.xreplace = atom_xreplace
  GUARDED_ENTRY_POINTS.append("Atom.xreplace (bare d)")
  _EvalfMixin.evalf = evalf
  _EvalfMixin.n = evalf
  GUARDED_ENTRY_POINTS.extend(["Basic.subs", "Basic.xreplace", "Basic.replace", "evalf/n(subs=...)"])

  def poly_guard(name, value_args=False):
      orig = getattr(_Poly, name)

      def w(self, *args, **kwargs):
          if not getattr(_AUTH, "active", False) and d in self.gens and (not value_args or args or kwargs):
              if not _internal(1):
                  _deny(f"Poly.{name} with d as a generator (caller outside sympy)")
              if name in ("eval", "__call__"):
                  _deny(f"Poly.{name} with d as a generator inside sympy")
          return orig(self, *args, **kwargs)
      w.__name__ = name
      setattr(_Poly, name, w)
      GUARDED_ENTRY_POINTS.append(f"Poly.{name}")
  for nm in ("eval", "__call__", "all_coeffs", "coeffs", "terms", "all_terms", "as_dict", "as_list",
             "coeff_monomial", "nth", "LC", "TC", "EC"):
      poly_guard(nm)
  poly_guard("as_expr", value_args=True)

  def expr_guard(name, pick):
      orig = getattr(_Expr, name)

      def w(self, *args, **kwargs):
          if not getattr(_AUTH, "active", False) and any(_has_d(x) for x in pick(args, kwargs)) and not _int
+> ernal(1):
              _deny(f"Expr.{name} targeting d (caller outside sympy)")
          return orig(self, *args, **kwargs)
      w.__name__ = name
      setattr(_Expr, name, w)
      GUARDED_ENTRY_POINTS.append(f"Expr.{name}")
  expr_guard("coeff", lambda a, k: a[:1] + tuple(k.get(x) for x in ("x",) if x in k))
  expr_guard("as_independent", lambda a, k: a)
  expr_guard("as_coeff_exponent", lambda a, k: a[:1])
  expr_guard("series", lambda a, k: a[:1] + tuple(k.get(x) for x in ("x",) if x in k))

  import importlib
  import sympy as _sp
  _rad = importlib.import_module("sympy.simplify.radsimp")
  _simp = importlib.import_module("sympy.simplify")
  o_collect = _rad.collect

  def collect(expr, syms, *a, **k):
      ss = syms if isinstance(syms, (list, tuple, set)) else [syms]
      if not getattr(_AUTH, "active", False) and any(_has_d(_sp.sympify(x)) for x in ss) and not _internal(1
+> ):
          _deny("collect(.., d) (caller outside sympy)")
      return o_collect(expr, syms, *a, **k)
  _rad.collect = _simp.collect = _sp.collect = collect
  GUARDED_ENTRY_POINTS.append("collect")
  _lim = importlib.import_module("sympy.series.limits")
  o_limit = _lim.limit

  def limit(e, z, z0, *a, **k):
      if not getattr(_AUTH, "active", False) and _has_d(_sp.sympify(z)) and not _internal(1):
          _deny("limit in d (caller outside sympy)")
```

```
        return o_limit(e, z, z0, *a, **k)
    _lim.limit = _sp.limit = limit
    from sympy.core.function import Lambda as _Lambda
    o_lnew = _Lambda.__new__

    def lambda_new(cls, signature, expr, *a, **k):
        sig = signature if isinstance(signature, (tuple, list)) else (signature,)
        if not getattr(_AUTH, "active", False) and any(_has_d(_sp.sympify(v)) for v in sig if isinstance(v, _B
  +> asic)) \
                and not _internal(1):
            _deny("Lambda with d as a bound variable (caller outside sympy)")
        return o_lnew(cls, signature, expr, *a, **k)
    _Lambda.__new__ = lambda_new
    GUARDED_ENTRY_POINTS.extend(["limit", "Lambda(d, ...)"])


def _d_eq(self, other):
    if other is self and not getattr(_AUTH, "active", False) and not _internal(1):
        _deny("equality test against d (hand-written expression-tree rebuild)")
    return _sp_Symbol_eq(self, other)


_sp_Symbol_eq = sp.Symbol.__eq__
OpaqueDimension.__eq__ = _d_eq
OpaqueDimension.__hash__ = sp.Symbol.__hash__
GUARDED_ENTRY_POINTS.append("OpaqueDimension.__eq__ (d == d)")
if not getattr(_Basic.subs, "_xvi_p22", False):
    _install()
    _Basic.subs._xvi_p22 = True


@_contextlib.contextmanager
def authorised_d(reason):
    """p22: the only other authorised access to d's value paths (recorded in D_PROVENANCE like substitute_d)."
  +> ""
    prev = getattr(_AUTH, "active", False)
    _AUTH.active = True
    try:
        yield
    finally:
        _AUTH.active = prev
    D_PROVENANCE[reason] = D_PROVENANCE.get(reason, 0) + 1
# ================================================================================== end p22 runtime d gu
  +> ard
eps = sp.Symbol("epsilon")
ETA = (1, -1, -1, -1)
VSPACE = {"l": "h", "p": "h"}          # R2: loop and external momenta in QdS


class UncontractedObjects(Exception):
    """a Laurent expansion was requested on an expression that still carries open index objects (Gt_rs, e-vect
  +> ors)."""


class Rules:
    """trace rules; trace_hat is overridable ONLY for the mutation test (tr ghat = 4 must be rejected)."""

    def __init__(self, trace_hat=None, display_only=False):
        th = d if trace_hat is None else sp.sympify(trace_hat)
        self.trace = {"4": sp.Integer(4), "h": th, "t": sp.Integer(4) - th}
        self.mutated = trace_hat is not None
        self.display_only = bool(display_only)        # p15: a numeric trace may only be used to DISPLAY a mut
  +> ation

    def enforce_symbolic(self):
        if self.trace["h"] != d and not self.display_only:
            raise NumericTraceForbidden(f"tr ghat = {self.trace['h']} (must be the opaque d; R1/R5)")

    def check(self):
        tr = self.trace
        res = {"tr_g4_is_4": tr["4"] == 4,
               "tr_ghat_is_symbolic_d": tr["h"] == d,
               "tr_ghat_plus_tr_gtilde_is_4": sp.simplify(tr["h"] + tr["t"] - 4) == 0,
               "tr_gtilde_is_4_minus_d_nonzero": sp.simplify(tr["t"] - (4 - d)) == 0 and tr["t"] != 0}
        res["pass"] = all(res.values())
        return res


# ------------------------------------------------------------------ vectors, metrics, contraction engine
def vspace(name):
    if name.endswith("^"):
```

```
        return "h"
    if name.endswith("~"):
        return "t"
    return VSPACE.get(name, "4")


def project(kind, name):
    s = vspace(name)
    if kind == "4" or kind == s:
        return name
    if s == "4":
        return name + ("^" if kind == "h" else "~")
    return None                                          # orthogonal subspaces


def mcomb(k1, k2):
    if k1 == "4":
        return k2
    if k2 == "4":
        return k1
    return k1 if k1 == k2 else None


def dot(a, b):
    sa, sb = vspace(a), vspace(b)
    if {sa, sb} == {"h", "t"}:
        return sp.Integer(0)
    if sa == "4" and sb != "4":
        a = project(sb, a)
    if sb == "4" and sa != "4":
        b = project(sa, b)
    x, y = sorted([a, b])
    return sp.Symbol(f"{x}.{y}")


def _canon(facs):
    out = []
    for f in facs:
        if f[0] == "g":
            i, j = sorted((f[2], f[3]))
            out.append(("g", f[1], i, j))
        else:
            out.append(f)
    return tuple(sorted(out))


def contract_term(coeff, facs, rules):
    """facs: ('g', kind, i, j) | ('v', name, i).  Repeated indices are contracted with R1/R2."""
    facs = list(facs)
    while True:
        occ = {}
        for fi, f in enumerate(facs):
            for ix in (f[2:] if f[0] == "g" else (f[2],)):
                occ.setdefault(ix, []).append(fi)
        rep = next(((ix, fl) for ix, fl in occ.items() if len(fl) >= 2), None)
        if rep is None:
            return coeff, _canon(facs)
        ix, fl = rep
        assert len(fl) == 2, f"index {ix} occurs more than twice"
        a, b = fl
        if a == b:                                       # trace of one metric
            coeff = coeff * rules.trace[facs[a][1]]
            facs.pop(a)
            continue
        fa, fb = facs[a], facs[b]
        if fa[0] == "v" and fb[0] == "g":
            fa, fb = fb, fa
        rest = [f for k, f in enumerate(facs) if k not in (a, b)]
        if fa[0] == "g" and fb[0] == "g":
            kc = mcomb(fa[1], fb[1])
            if kc is None:
                return sp.Integer(0), ()
            o1 = fa[3] if fa[2] == ix else fa[2]
            o2 = fb[3] if fb[2] == ix else fb[2]
            facs = rest + [("g", kc, o1, o2)]
        elif fa[0] == "g":
            nv = project(fa[1], fb[1])
            if nv is None:
                return sp.Integer(0), ()
            o = fa[3] if fa[2] == ix else fa[2]
            facs = rest + [("v", nv, o)]
```

```
        else:
            coeff = coeff * dot(fa[1], fb[1])
            facs = rest
        if coeff == 0:
            return sp.Integer(0), ()


def contract(terms, rules):
    rules.enforce_symbolic()
    out = {}
    for c, facs in terms:
        cc, ff = contract_term(c, facs, rules)
        if cc != 0:
            out[ff] = sp.expand(out.get(ff, 0) + cc)
    return {k: v for k, v in out.items() if v != 0}


# ---------------------------------------------------------------- R4 tensor reduction (Gram / PV projection)
def _matchings(items):
    if not items:
        yield []
        return
    a = items[0]
    for k in range(1, len(items)):
        rest = items[1:k] + items[k + 1:]
        for m in _matchings(rest):
            yield [(a, items[k])] + m


def sym_basis(idx, with_p):
    """B_k: symmetric tensors with k ghat's and (n-2k) p's over the free indices idx."""
    n, out = len(idx), []
    for k in range(n // 2 + 1):
        if not with_p and 2 * k != n:
            continue
        terms = []
        for sub in itertools.combinations(idx, 2 * k):
            for m in _matchings(list(sub)):
                rest = [i for i in idx if i not in sub]
                terms.append(tuple(("g", "h", a, b) for a, b in m) + tuple(("v", "p", i) for i in rest))
        out.append({"k": k, "terms": terms})
    return out


def _integral_functional(expr, tag):
    """polynomial in l.l, l.p (coefficients may carry p.p, d) -> sum coeff * J_tag[a,b]   (NOT evaluated)."""
    ll, lp = sp.Symbol("l.l"), sp.Symbol("l.p")
    P = sp.Poly(sp.expand(expr), ll, lp)
    return sp.expand(sum(cf * sp.Symbol(f"J_{tag}[{m[0]},{m[1]}]") for m, cf in P.terms()))


_RED = {}


def reduce_tensor(n, with_p, rules, tag="G"):
    """int l^{mu_1..mu_n} F = sum_k C_k B_k ; returns (basis, C) with C exact rational functions of d, p.p and
  +>  J."""
    key = (n, with_p, str(rules.trace["h"]), tag)
    if key in _RED:
        return _RED[key]
    idx = [f"m{i}" for i in range(n)]
    B = sym_basis(idx, with_p)
    if not B:
        _RED[key] = (B, [])
        return _RED[key]
    G = sp.zeros(len(B), len(B))
    for j, bj in enumerate(B):
        for k, bk in enumerate(B):
            tot = contract([(sp.Integer(1), tj + tk) for tj in bj["terms"] for tk in bk["terms"]], rules)
            G[j, k] = sp.expand(tot.get((), 0))
    R = []
    for bj in B:
        tot = contract([(sp.Integer(1), tj + tuple(("v", "l", i) for i in idx)) for tj in bj["terms"]], rules)
        R.append(_integral_functional(tot.get((), 0), tag))
    Cs = G.LUsolve(sp.Matrix(R))
    Cs = [sp.factor(sp.cancel(c)) for c in Cs]
    _RED[key] = (B, Cs)
    return _RED[key]


# ---------------------------------------------------------------- R3 lifting of component monomials
```

```
def Gt(r, s):
    a, b = sorted((r, s))
    return sp.Symbol(f"Gt_{a}{b}")


def _lift_symbols(expr):
    """e_(r)^.e_(s)^ -> eta_rs - Gt_rs ; e_(r)~.e_(s)~ -> Gt_rs ; e_(r)^.p -> p_r (p in QdS) ; e_(r).e_(s) ->
  +> eta_rs."""
    rep = {}
    for s in expr.free_symbols:
        nm = s.name
        if "." not in nm or not nm.startswith("e"):
            continue
        a, b = nm.split(".")
        if a.startswith("e") and b.startswith("e"):
            r, q = int(a[1]), int(b[1])
            eta = ETA[r] if r == q else 0
            if a.endswith("^") and b.endswith("^"):
                rep[s] = eta - Gt(r, q)
            elif a.endswith("~") and b.endswith("~"):
                rep[s] = Gt(r, q)
            else:
                rep[s] = eta
        elif b == "p" and a.startswith("e"):
            rep[s] = sp.Symbol(f"p{int(a[1])}")
    return sp.expand(expr.xreplace(rep))


_LIFT = {}


def lift_reduce_monomial(comps, with_p, rules, tag):
    """int l_{r1}..l_{rn} F  ->  scalar in eta, Gt_rs, p_r, d, J_tag   (R3 + R4)."""
    key = (tuple(sorted(comps)), with_p, str(rules.trace["h"]), tag)
    if key in _LIFT:
        return _LIFT[key]
    n = len(comps)
    if n == 0:
        _LIFT[key] = sp.Symbol(f"J_{tag}[0,0]")
        return _LIFT[key]
    B, Cs = reduce_tensor(n, with_p, rules, tag)
    idx = [f"m{i}" for i in range(n)]
    ev = tuple(("v", f"e{r}", i) for r, i in zip(comps, idx))
    tot = 0
    for b, c in zip(B, Cs):
        con = contract([(sp.Integer(1), t + ev) for t in b["terms"]], rules)
        tot += c * con.get((), 0)
    _LIFT[key] = _lift_symbols(sp.expand(tot))
    return _LIFT[key]


def lift_reduce_polynomial(poly_rows, with_p, rules, tag):
    """poly_rows: exported canonical rows [[var names...], 'p/q'] in l0..l3, p0..p3."""
    tot = 0
    for names, cf in poly_rows:
        lc = sorted(int(v[1]) for v in names if v.startswith("l"))
        pc = sp.Mul(*[sp.Symbol(v) for v in names if v.startswith("p")])
        tot += sp.Rational(cf) * pc * lift_reduce_monomial(lc, with_p, rules, tag)
    return sp.expand(tot)


def split_evanescent(expr):
    """expr = four_d + evanescent, evanescent = part linear in Gt symbols."""
    gts = [s for s in expr.free_symbols if s.name.startswith("Gt_")]
    ev = sp.expand(expr - expr.xreplace({g: 0 for g in gts}))
    return sp.expand(expr - ev), ev


def eta_trace_Gt(expr, rules):
    """external 4D projection of the evanescent part: an expression c * sum_r eta^{rr} Gt_rr (+ Gt-free terms)
  +>  is
    contracted to c * tr(gtilde) = c * (4 - d) (R4).  Any Gt pattern that is not of this trace form is an open
    index object and is refused."""
    gts = {g for g in expr.free_symbols if g.name.startswith("Gt_")}
    if not gts:
        return expr
    c = None
    for g in gts:
        a, b = int(g.name[3]), int(g.name[4])
        if a != b:
            raise UncontractedObjects([g.name])
```

```
        val = sp.expand(sp.expand(expr).coeff(g) / ETA[a])
        if c is None:
            c = val
        elif sp.expand(val - c) != 0:
            raise UncontractedObjects(sorted(x.name for x in gts))
    if len(gts) != 4:
        raise UncontractedObjects(sorted(x.name for x in gts))
    return sp.expand(sp.expand(expr).xreplace({g: 0 for g in gts}) + c * rules.trace["t"])


# ------------------------------------------------------------------ R5 Laurent / MS-bar with guards
class Pipeline:
    """lift -> reduce -> contract (incl. external 4D projection) -> Laurent.  The ledger records whether the
    reduction introduced d; the guards enforce R5."""

    def __init__(self, rules=None, mutation=None, enforce=True):
        self.rules = rules or Rules()
        self.mutation = mutation
        self.enforce = enforce                          # enforce=False only to DISPLAY what a mutation would
  +>  yield
        self.ledger = []
        if enforce:
            self.rules.enforce_symbolic()

    def reduced_component(self, comps, with_p=False, tag="T"):
        v = lift_reduce_monomial(comps, with_p, self.rules, tag)
        if v.has(d):
            self.ledger.append({"step": "reduce", "comps": list(comps), "introduces_d": True})
        if self.mutation == "premature_d4":
            if self.enforce:
                v = v.xreplace({d: 4})                  # MUTATION: d -> 4 before the expansion -> raises (opa
  +> que d)
            else:
                v = substitute_d(v, 4, "mutation display only (enforce=False): premature d = 4")
        return v

    def laurent(self, expr, Jvals, order=0):
        if self.mutation == "contract_after_laurent":
            pass                                        # (the caller hands over an uncontracted expression)
        if any(s.name.startswith(("Gt_", "e")) and s.name != "epsilon" for s in expr.free_symbols):
            raise UncontractedObjects(sorted(s.name for s in expr.free_symbols if s.name.startswith(("Gt_", "e
  +> ")) 
                                             and s.name != "epsilon"))
        # p15 contract correction: no heuristic test on the shape of expr (it gave a false positive and a fals
  +> e
        # negative, finding F1); R5 is enforced structurally by the opaque d (any earlier substitution has rai
  +> sed)
        if self.enforce:
            self.rules.enforce_symbolic()
        e = sp.expand(substitute_d(expr.xreplace(Jvals), 4 - 2 * eps, "Pipeline.laurent: d = 4 - 2 eps"))
        ser = sp.series(e, eps, 0, order + 1).removeO()
        pole = sp.expand(sum(ser.coeff(eps, -k) * eps ** (-k) for k in (1, 2)))
        fin = sp.expand(ser.coeff(eps, 0))
        return {"pole": pole, "finite": fin, "ms_bar_subtracted": fin, "pre_laurent": expr}


# ------------------------------------------------------------------ toy checks (generic placeholders; no integr
  +> al evaluated)
A_, B_, C_ = sp.symbols("a b c")
JTOY = {sp.Symbol("J_T[1,0]"): A_ / eps + B_ + C_ * eps}


def toy_four_d_part(rules=None, mutation=None, enforce=True):
    """T1: 4D (eta_00) coefficient of int l_0 l_0 F(l^2) = (1/d) J[1,0]."""
    P = Pipeline(rules, mutation, enforce)
    v = P.reduced_component([0, 0], with_p=False, tag="T")
    four, _ = split_evanescent(v)
    return P.laurent(four, JTOY)


def toy_evanescent_trace(rules=None, mutation=None, enforce=True):
    """T2: eta^{rs} x [Gt-part of int l_r l_s F]  =  -(4-d)/d J[1,0]: a (d-4) x pole term with finite part -a/
  +> 2."""
    P = Pipeline(rules, mutation, enforce)
    tot = sp.expand(sum(ETA[r] * split_evanescent(P.reduced_component([r, r], with_p=False, tag="T"))[1]
                        for r in range(4)))
    if mutation != "contract_after_laurent":          # MUTATION: skip the contraction and expand first
        tot = eta_trace_Gt(tot, P.rules)
    return P.laurent(tot, JTOY)
```

```
def toy_reference():
    """hand-derived values (d = 4 - 2 eps, J = a/eps + b + c eps)."""
    return {"T1_finite": sp.expand(B_ / 4 + A_ / 8), "T1_pole": sp.expand(A_ / (4 * eps)),
            "T2_finite": sp.expand(-A_ / 2), "T2_pole": sp.Integer(0)}


def known_reductions(rules):
    """closed forms: rank-2 / rank-4 tadpoles and the rank-2 two-point PV coefficients."""
    out = {}
    B, C = reduce_tensor(2, False, rules, "X")
    out["rank2_tadpole_eq_ghat_over_d"] = sp.simplify(C[0] - sp.Symbol("J_X[1,0]") / d) == 0
    B, C = reduce_tensor(4, False, rules, "X")
    out["rank4_tadpole_eq_sym_ghatghat_over_d_dplus2"] = sp.simplify(C[0] - sp.Symbol("J_X[2,0]") / (d * (d +
  +> 2))) == 0
    B, C = reduce_tensor(2, True, rules, "Y")
    pp = sp.Symbol("p.p")
    J10, J02 = sp.Symbol("J_Y[1,0]"), sp.Symbol("J_Y[0,2]")
    c_g = [c for b, c in zip(B, C) if b["k"] == 1][0]
    c_pp = [c for b, c in zip(B, C) if b["k"] == 0][0]
    out["rank2_two_point_B00"] = sp.simplify(c_g - (J10 * pp - J02) / ((d - 1) * pp)) == 0
    out["rank2_two_point_B11"] = sp.simplify(c_pp - (d * J02 - pp * J10) / ((d - 1) * pp ** 2)) == 0
    out["pass"] = all(out.values())
    return out


def reviewer_toys():
    """p15 regression toys from the p14r independent rebuild (finding F1/F2).
    eta-trace toy: eta^{rs} int l_r l_s/(l^2 - m^2) = eta^{rs} ghat_rs J/d = J, J = m^2 A0, A0 = m^2(1/eps + a
  +> 1 + a2 eps).
      correct -> no exception, finite a1 m^4 (p14r heuristic: FALSE POSITIVE);
      premature_d4 (enforced) -> PrematureD4Substitution (p14r heuristic: FALSE NEGATIVE, finite m^4 (a1 - 1/2
  +> )).
    A0 mixing toy: reduce with tr ghat = d, contract with tr ghat = 4: refused (NumericTraceForbidden); forced
  +>  display
      gives the +m^4/2 finite shift.
    B0 mixing toy: ghat_{mu nu} int l^mu l^nu/(l^2 (l-p)^2) = d B00 + p^2 B11 = J[1,0] = 0 (scaleless); mixing
      (contract with 4): (4-d) B00 -> forced display gives the -p^2/6 finite shift."""
    m, a1, a2, p2, b1, b2 = sp.symbols("m a1 a2 p2 b1 b2")
    A0 = m ** 2 * (1 / eps + a1 + a2 * eps)
    B0 = 1 / eps + b1 + b2 * eps
    out = {}

    def eta_toy(mutation, enforce=True):
        _RED.clear()
        _LIFT.clear()
        P = Pipeline(Rules(), mutation, enforce)
        tot = sp.expand(sum(ETA[r] * P.reduced_component([r, r], with_p=False, tag="RT") for r in range(4)))
        tot = eta_trace_Gt(tot, P.rules)
        return P.laurent(tot, {sp.Symbol("J_RT[1,0]"): m ** 2 * A0})
    good = eta_toy(None)
    out["eta_trace_correct"] = {"finite": str(sp.simplify(good["finite"])), "raised": False,
                                "ok": sp.simplify(good["finite"] - a1 * m ** 4) == 0}
    try:
        eta_toy("premature_d4")
        out["eta_trace_premature_d4"] = {"raised": False, "ok": False}
    except PrematureD4Substitution as exc:
        disp = eta_toy("premature_d4", enforce=False)
        out["eta_trace_premature_d4"] = {"raised": True, "reason": str(exc),
                                         "finite_if_forced": str(sp.simplify(disp["finite"])),
                                         "ok": sp.simplify(disp["finite"] - m ** 4 * (a1 - sp.Rational(1, 2)))
  +>  == 0}

    def ghat_contract(red_rules, con_rules, with_p, tag):
        B, C = reduce_tensor(2, with_p, red_rules, tag)
        tot = 0
        for b, c in zip(B, C):
            con = contract([(sp.Integer(1), t + (("g", "h", "m0", "m1"),)) for t in b["terms"]], con_rules)
            tot += c * con.get((), 0)
        return sp.expand(tot)
    for lab, with_p, J, target_shift in (
            ("A0", False, {sp.Symbol("J_RA[1,0]"): m ** 2 * A0}, m ** 4 / 2),
            ("B0", True, {sp.Symbol("J_RB[1,0]"): sp.Integer(0), sp.Symbol("J_RB[0,2]"): p2 ** 2 * B0 / 4},
             -p2 / 6)):
        tag = "RA" if lab == "A0" else "RB"
        _RED.clear()
        _LIFT.clear()
        e = ghat_contract(Rules(), Rules(), with_p, tag).xreplace({sp.Symbol("p.p"): p2})
        fin = sp.simplify(Pipeline(Rules()).laurent(e, J)["finite"])
        try:
            ghat_contract(Rules(), Rules(trace_hat=4), with_p, tag)
            refused = False
```

```
        except NumericTraceForbidden:
            refused = True
        _RED.clear()
        _LIFT.clear()
        em = ghat_contract(Rules(), Rules(trace_hat=4, display_only=True), with_p, tag).xreplace({sp.Symbol("p
  +> .p"): p2})
        finm = sp.simplify(Pipeline(Rules(), enforce=False).laurent(em, J)["finite"])
        _RED.clear()
        _LIFT.clear()
        correct = a1 * m ** 4 if lab == "A0" else sp.Integer(0)
        out[f"{lab}_mixing"] = {"correct_finite": str(fin), "mixed_refused": refused,
                                "finite_if_forced": str(finm), "shift": str(sp.simplify(finm - fin)),
                                "ok": refused and sp.simplify(fin - correct) == 0
                                and sp.simplify(finm - fin - target_shift) == 0}
    out["pass"] = all(v["ok"] for v in out.values())
    return out


def lift_exported(graphs=("D01", "D02", "D03", "D11", "D12", "D13", "D14a", "D14b", "D14c")):
    """R3+R4 on the exported integrands (tadpole topologies: with_p only for the p-dependent numerators).
    Returns per graph: components lifted, max l-rank, components with a nonzero evanescent (Gt) part."""
    rules = Rules()
    out = {}
    for g in graphs:
        f = ROOT / "exports" / "P14_integrands" / f"{g}.json"
        if not f.is_file():
            out[g] = {"present": False}
            continue
        body = json.loads(f.read_text())
        with_p = len(body["denominators"]) > 1           # two-point: tensor basis includes p
        n_ev, n_c, rank = 0, 0, 0
        for comp, rows in body["numerator"].items():
            rank = max(rank, max((sum(1 for v in r[0] if v.startswith("l")) for r in rows), default=0))
            v = lift_reduce_polynomial(rows, with_p, rules, g)
            _, ev = split_evanescent(v)
            n_c += 1
            n_ev += ev != 0
        out[g] = {"present": True, "components": n_c, "max_l_rank": rank, "components_with_evanescent_part": n
  +> _ev,
                  "scalar_integrals": "J_" + g + "[a,b] symbols (NOT evaluated)"}
    return out


def main():
    t0 = time.time()
    rules = Rules()
    rc = rules.check()
    kr = known_reductions(rules)
    ref = toy_reference()
    t1, t2 = toy_four_d_part(), toy_evanescent_trace()
    toys = {"T1_four_d_part": {"finite": str(t1["finite"]), "pole": str(t1["pole"]),
                               "equals_reference": sp.expand(t1["finite"] - ref["T1_finite"]) == 0
                               and sp.expand(t1["pole"] - ref["T1_pole"]) == 0},
            "T2_evanescent_trace": {"finite": str(t2["finite"]), "pole": str(t2["pole"]),
                                    "equals_reference": sp.expand(t2["finite"] - ref["T2_finite"]) == 0
                                    and sp.expand(t2["pole"] - ref["T2_pole"]) == 0,
                                    "pre_laurent": str(t2["pre_laurent"]),
                                    "is_d_minus_4_times_pole": sp.simplify(
                                        t2["pre_laurent"] + (4 - d) / d * sp.Symbol("J_T[1,0]")) == 0}}
    # premature d=4 changes the finite part of T1 (a/8 lost); the guard must raise
    naive = toy_four_d_part(mutation="premature_d4", enforce=False)["finite"]   # what d=4-first would give
    muts = {}
    try:
        toy_four_d_part(mutation="premature_d4")
        muts["premature_d4"] = {"rejected": False}
    except PrematureD4Substitution as exc:
        muts["premature_d4"] = {"rejected": True, "reason": f"PrematureD4Substitution: {exc}",
                                "finite_part_if_not_caught": str(sp.expand(naive)),
                                "finite_part_differs": sp.expand(naive - ref["T1_finite"]) != 0}
    mr = Rules(trace_hat=4)
    try:                                                # p15: structural refusal of a numeric trace
        toy_four_d_part(rules=mr)
        structural = False
    except NumericTraceForbidden:
        structural = True
    md = Rules(trace_hat=4, display_only=True)
    _RED.clear()
    _LIFT.clear()
    m1, m2 = toy_four_d_part(rules=md, enforce=False), toy_evanescent_trace(rules=md, enforce=False)
    _RED.clear()
    _LIFT.clear()
```

```
    muts["trace_ghat_eq_4"] = {"rules_check": mr.check(), "structurally_refused": structural,
                               "T1_finite": str(m1["finite"]), "T2_finite": str(m2["finite"]),
                               "rejected": structural and (not mr.check()["pass"]) and (
                                   sp.expand(m1["finite"] - ref["T1_finite"]) != 0
                                   or sp.expand(m2["finite"] - ref["T2_finite"]) != 0)}
    try:
        toy_evanescent_trace(mutation="contract_after_laurent")
        muts["contract_after_laurent"] = {"rejected": False}
    except UncontractedObjects as exc:
        muts["contract_after_laurent"] = {"rejected": True, "reason": f"UncontractedObjects: {exc}"}
    reviewer = reviewer_toys()
    lifts = lift_exported()
    ev_any = any(v.get("components_with_evanescent_part", 0) for v in lifts.values())
    verified = bool(rc["pass"] and kr["pass"] and all(t["equals_reference"] for t in toys.values())
                    and toys["T2_evanescent_trace"]["is_d_minus_4_times_pole"]
                    and all(m["rejected"] for m in muts.values()) and all(v.get("present") for v in lifts.valu
+> es())
                    and reviewer["pass"])
    import p14_conventions as PC
    import certio
    rec = {"landmark": "XVI-P", "revision": PC.REVISION, "deliverable": "P.5 entry: operational DRED rules (ex
+> ecutable)",
           "dred_rules_verified": verified,
           "rules": {"R1_spaces": "Q4S = QdS (+) Q(4-d)S; g4 = ghat + gtilde; tr g4 = 4, tr ghat = d, tr gtild
+> e = 4-d; "
                                  "ghat.ghat = ghat, gtilde.gtilde = gtilde, ghat.gtilde = 0, g4.X = X",
                     "R2_momenta": "loop momentum l in QdS; external momenta p in QdS (p.gtilde = 0, stated ch
+> oice); "
                                   "field/vertex/polarisation indices in Q4S",
                     "R3_lifting": "exported lower components l_r = l.e_(r) = l.ehat_(r), p_r = p.e_(r), e_(r)
+>  the "
                                   "eta-orthonormal basis of Q4S; vertex tensors lifted with eta -> g4",
                     "R4_reduction": "Gram/PV projection on symmetric ghat/p tensors with symbolic d; "
                                     "ghat_rs = eta_rs - Gt_rs, eta^{rs} Gt_rs = 4-d, Gt_rs p^s = 0",
                     "R5_order": "all contractions (incl. external 4D projection) BEFORE the Laurent expansion
+> ; d kept "
                                 "symbolic through MS-bar subtraction and finite-part extraction; (d-4) x pole
+>  retained",
                     "R6_evanescent": "evanescent remainders arise in general and are retained (Gt structures;
+>  "
                                      "(4-d) x pole finite terms); the p14 'none arise' statement is withdrawn
+> "},
           "literature_background_only": ["W. Siegel, Phys. Lett. B84 (1979) 193", "W. Siegel, Phys. Lett. B94
+>  (1980) 37",
                                          "D. Stockinger, JHEP 0503 (2005) 076, hep-ph/0503129",
                                          "A. Signer, D. Stockinger, Nucl. Phys. B808 (2009) 88, arXiv:0807.44
+> 24"],
           "trace_rules_check": rc, "known_reductions": kr,
           "toy_checks": {"placeholders": "J = a/eps + b + c eps (generic symbols; not a loop value); d = 4 -
+> 2 eps",
                          **toys, "reference": {k: str(v) for k, v in ref.items()}},
           "mutations": muts, "mutations_all_rejected": all(m["rejected"] for m in muts.values()),
           "lift_of_exported_integrands": lifts, "evanescent_parts_present_in_exports": ev_any,
           "p15_contract_correction": {
               "finding": "F1_premature_d4_guard_heuristic (reproducibility/independent_rebuild_report_p14r.js
+> on)",
               "old_safeguard": "heuristic: raise iff the reduction introduced d and d is absent before the La
+> urent "
                                "expansion (false positive and false negative on the reviewer's eta-trace toy)
+> ",
               "new_safeguard": "structural/provenance: OpaqueDimension d (subs/xreplace raise outside substit
+> ute_d); "
                                "substitute_d records every authorised substitution; numeric traces of ghat/gt
+> ilde "
                                "refused (NumericTraceForbidden) unless display_only",
               "released_p14r_values_affected": False,
               "why_unaffected": "in p14r Pipeline.laurent was applied only to the toy placeholders; no loop i
+> ntegral "
                                 "was evaluated",
               "regression_toys": reviewer},
           "loop_evaluations_performed": False, "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{PC.CERT_PREFIX}_dred_rules.json", rec)
    print(json.dumps({"dred_rules_verified": verified, "trace": rc["pass"], "known": kr, "toys": toys,
                      "mutations": {k: v["rejected"] for k, v in muts.items()}, "lifts": lifts,
                      "seconds": rec["seconds"]}, indent=1, default=str))
    return 0 if verified else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

```
# ============================================================================== p23 CONTRACT REVISIO
  +> N: polynomial-domain refusal
# p23 (disclosed; reviewer p22 runtime_guard_bypass_poly_algebra): (1) internal sympy renames of d are admitte
  +> d only to a
# Dummy (or to an expression still containing d) -- a rename to a PLAIN symbol (Matrix.subs({d: y}) then y ->
  +> 4) raises;
# (2) public polynomial algebra that can evaluate d (rem/prem/div/quo/resultant/discriminant/compose/decompose
  +> /shift/
# gcdex/invert/subresultants/sturm/root isolation) refuses d as a generator or domain symbol when called from
  +> outside
# sympy.  Rational-function normalisation (cancel/factor/together/apart/simplify/expand) is unaffected.
def _check_rule(pairs, what, depth=2):            # p23: overrides the p22 rule (module global, looked up at c
  +> all time)
    if getattr(_AUTH, "active", False):
        return
    for old, new in pairs:
        if _has_d(old):
            if not _internal(depth):
                _deny(f"{what} of {old} (caller outside sympy)")
            nb = new if isinstance(new, _Basic) else sp.sympify(new, strict=False) if isinstance(new, (int, fl
  +> oat)) else new
            if not (isinstance(nb, _Basic) and (isinstance(nb, sp.Dummy) or _has_d(nb))):
                _deny(f"{what}: {old} -> {new} inside sympy (only a Dummy rename or a d-containing value is ad
  +> mitted)")


_POLY_DENY = ("rem", "prem", "pdiv", "pquo", "pexquo", "div", "quo", "exquo", "gcdex", "half_gcdex", "invert",
              "subresultants", "resultant", "discriminant", "compose", "decompose", "sturm", "nroots", "real_r
  +> oots",
              "count_roots", "ground_roots", "intervals", "refine_root", "nth_power_roots_poly")
_POLY_METHOD_DENY = _POLY_DENY + ("shift", "transform", "shift_list")


def _install_poly_refusal():
    import sympy as _sympy
    import sympy.polys.polytools as _pt

    def _args_have_d(args, kwargs):
        return any(_has_d(a) or (isinstance(a, _pt.Poly) and (d in a.gens or a.free_symbols_in_domain & {d}))
                   for a in list(args) + list(kwargs.values()))

    for name in _POLY_DENY:
        orig = getattr(_pt, name, None)
        if orig is None:
            continue

        def w(*args, _orig=orig, _name=name, **kwargs):
            if not getattr(_AUTH, "active", False) and not _internal(1) and _args_have_d(args, kwargs):
                _deny(f"{_name}(...) with d in the polynomial domain")
            return _orig(*args, **kwargs)
        w.__name__ = name
        setattr(_pt, name, w)
        if getattr(_sympy, name, None) is orig:
            setattr(_sympy, name, w)
        GUARDED_ENTRY_POINTS.append(f"polytools.{name} (d in domain)")
    for name in _POLY_METHOD_DENY:
        orig = getattr(_pt.Poly, name, None)
        if orig is None:
            continue

        def m(self, *args, _orig=orig, _name=name, **kwargs):
            if (not getattr(_AUTH, "active", False) and not _internal(1)
                    and (d in self.gens or d in self.free_symbols_in_domain or _args_have_d(args, kwargs))):
                _deny(f"Poly.{_name}(...) with d in the polynomial domain")
            return _orig(self, *args, **kwargs)
        m.__name__ = name
        setattr(_pt.Poly, name, m)
        GUARDED_ENTRY_POINTS.append(f"Poly.{name} (d in domain)")
    GUARDED_ENTRY_POINTS.append("internal rename of d to a plain Symbol (Matrix.subs)")


_install_poly_refusal()
# ============================================================================== end p23 polynomial-d
  +> omain refusal
```

123 Source: hessian_route.py (W6 independent regeneration, p14r)

[listing 90/133] code/p5_entry/hessian_route.py

```
#!/usr/bin/env python3
"""XVI-P p14r W6: INDEPENDENT regeneration of D01, D02, D03, D12 from the one-loop expansion of the functional
(super)trace log of the field-dependent Hessian of S_Psi -- NOT from the Feynman-rule assembler.

Not used here: amplitude_rules.build_integrands, the rule table (S_f, loop signs, i^phase bookkeeping),
feynman.vertices(), feynman.grav_vertex/ell_terms/vt_route, feynman.propagators().
Used: the action itself -- the graviton generating polynomials ell_k (S_grav = sum_k kappa^{k-2} int ell_k, fr
  +> ozen
exports) differentiated by an own monomial routine, and the GE densities X^(n), S_phi^(n) of S_Psi
(feynman.S_psi_parts) differentiated by the generic plane-wave functional derivative (feynman.pw_vertex).

Weight e^{iS}.  Quadratic form S_2 = 1/2 int phi_X(-q) H_XY(q) phi_Y(q), H_XY(q) = V[X(-q), Y(q)]; G := H^{-1}
  +> .
  bosons (Gaussian, det^{-1/2}):  i Gamma_1 = -1/2 Tr log H(Phi)
     => i Gamma^{AB} = -1/2 Tr(G U2^{AB}) + 1/2 Tr(G U1^A G U1^B)
  ghosts (Berezin, det^{+1}, S_gh = cbar M c with M_{rho mu}(q) = V[cbar_rho(-q), c^mu(q)]):  i Gamma_1 = + Tr
  +>  log M
     => i Gamma^{AB} = - Tr(G U1^A G U1^B)        (no cbar.c.h.h term: X^(2) = 0, checked)
with U1^A = kappa d/dh_A of the field-dependent Hessian, U2^{AB} = kappa^2 d^2/dh_A dh_B, and the trace routin
  +> g
  Tr(G U_A G U_B) = sum G_XY(1) U_A[Y(-1), Z(1-p), h_A(p)] G_ZW(1-p) U_B[W(p-1), X(1), h_B(-p)].
The graviton G is the hh block of the inverse of the full 14x14 (h, b) Hessian (b has no interaction: X^(1) ha
  +> s no
field-only monomial, checked), computed here.  The result is compared with prefactor * N of the exported integ
  +> rands
(the Feynman-rule route) component by component, exactly, as polynomials after multiplying by the denominators
  +> .
"""
from __future__ import annotations
import json
import sys
import time
from fractions import Fraction as Fr
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import feynman as F  # noqa: E402
import planewave_exact as PW  # noqa: E402

I = sp.I
L = sp.symbols("l0:4")
P = sp.symbols("p0:4")
VARS = list(L) + list(P)
HC = list(F.HC)
HIDX = {mn: i for i, mn in enumerate(HC)}
GRAPHS = ("D01", "D02", "D03", "D12")
_REF = {}


# ------------------------------------------------------------------ own monomial differentiation of ell_k
def _ell_monomials(k):
    e, sha = PW.load_ell(k)
    gens = sorted(e.free_symbols, key=lambda s: s.name)
    out = []
    for mon, cf in sp.Poly(e, *gens).terms():
        facs = []
        for g, ex in zip(gens, mon):
            if not ex:
                continue
            nm = g.name
            if nm.startswith("dH"):
                head, r = nm[2:].split("_")
                facs += [(HIDX[(int(head[0]), int(head[1]))], int(r))] * ex
            else:
                facs += [(HIDX[(int(nm[1]), int(nm[2]))], None)] * ex
        out.append((Fr(int(cf.p), int(cf.q)), facs))
    return out, sha
```

```
def _lin(route, r):
    """component r of the leg momentum cl*l + cp*p as {var index: coeff}."""
    cl, cp = route
    d = {}
    if cl:
        d[r] = Fr(cl)
    if cp:
        d[4 + r] = Fr(cp)
    return d


def ell_table(k, routes):
    """V[h_c1(k1), ..., h_ck(kk)] of int ell_k for all component tuples; returns {comps: {(i,j): Fr}} (the two
    derivative factors give (i k)(i k) = -k k)."""
    import itertools
    mons, _ = _ell_monomials(k)
    T = {}
    perms = list(itertools.permutations(range(k)))
    for c, facs in mons:
        for perm in perms:
            comps = [None] * k
            der = []
            for (comp, r), leg in zip(facs, perm):
                comps[leg] = comp
                if r is not None:
                    der.append(_lin(routes[leg], r))
            assert len(der) == 2
            d = T.setdefault(tuple(comps), {})
            for i, a in der[0].items():
                for j, b in der[1].items():
                    key = (i, j) if i <= j else (j, i)
                    d[key] = d.get(key, 0) - c * a * b
    return {kk: {m: v for m, v in d.items() if v} for kk, d in T.items() if any(d.values())}


def _sym(d):
    return sp.Add(*[sp.Rational(v.numerator, v.denominator) * VARS[i] * VARS[j] for (i, j), v in d.items()])


# ---------------------------------------------------------------- Hessians and inverses (own)
def _pw(dens, legs):
    return F.pw_vertex(dens, legs)


def h_leg(a, k):
    return {"f": "h", "pol": {HC[a]: 1}, "k": k}


def mom(cl, cp):
    return [cl * L[r] + cp * P[r] for r in range(4)]


def graviton_inverse():
    """full (h, b) Hessian H(q) at q = l; G_hh = (H^{-1})_hh via the Schur complement; checks S = q^2 C, C con
  +> st."""
    S = F.S_psi_parts()
    X0 = S["X"][0]
    q2 = sum(F.ETA[r] * L[r] ** 2 for r in range(4))
    T2 = ell_table(2, [(-1, 0), (1, 0)])
    Hhh = sp.zeros(10, 10)
    for (a, b), d in T2.items():
        Hhh[a, b] += _sym(d)
    Xhh = sp.zeros(10, 10)
    for a in range(10):
        for b in range(10):
            Xhh[a, b] = _pw(X0, [h_leg(a, mom(-1, 0)), h_leg(b, mom(1, 0))])
    Hhh = (Hhh + Xhh).applyfunc(sp.expand)
    bl = lambda r, k: {"f": "b", "pol": {r: 1}, "k": k}  # noqa: E731
    Hbb = sp.Matrix(10, 4, lambda a, r: _pw(X0, [h_leg(a, mom(-1, 0)), bl(r, mom(1, 0))]))
    Hbh = sp.Matrix(4, 10, lambda r, a: _pw(X0, [bl(r, mom(-1, 0)), h_leg(a, mom(1, 0))]))
    Hbb = sp.Matrix(4, 4, lambda r, s: _pw(X0, [bl(r, mom(-1, 0)), bl(s, mom(1, 0))]))
    bb_const = all(sp.diff(x, v) == 0 for x in Hbb for v in L)
    Schur = (Hhh - Hhb * Hbb.inv() * Hbh).applyfunc(sp.cancel)
    C = (Schur / q2).applyfunc(sp.cancel)
    C_const = all(sp.diff(x, v) == 0 for x in C for v in L)
    Cinv = C.inv()
    H = sp.BlockMatrix([[Hhh, Hhb], [Hbh, Hbb]]).as_explicit()
    # full inverse check (block formula) H * Hinv == 1
    Ghh = Cinv / q2
    Ghb = -Ghh * Hhb * Hbb.inv()
    Gbh = -Hbb.inv() * Hbh * Ghh
```

```
    Gbb = Hbb.inv() + Hbb.inv() * Hbh * Ghh * Hhb * Hbb.inv()
    G = sp.BlockMatrix([[Ghh, Ghb], [Gbh, Gbb]]).as_explicit()
    full_ok = (H * G).applyfunc(sp.cancel) == sp.eye(14)
    return {"Cinv": Cinv, "X0_hh_part_zero": Xhh == sp.zeros(10, 10), "Hbb_constant": bb_const,
            "schur_is_q2_times_constant": C_const, "full_14x14_inverse_verified": full_ok}


def ghost_inverse():
    X0 = F.S_psi_parts()["X"][0]
    q2 = sum(F.ETA[r] * L[r] ** 2 for r in range(4))
    M0 = sp.Matrix(4, 4, lambda rho, mu: _pw(X0, [{"f": "cb", "c": rho, "k": mom(-1, 0)},
                                                  {"f": "c", "c": mu, "k": mom(1, 0)}]))
    m = (M0 / q2).applyfunc(sp.cancel)
    const = all(sp.diff(x, v) == 0 for x in m for v in L)
    return {"minv": m.inv(), "M0_is_q2_times_constant": const}        # G_gh(q) = minv / q^2


def scalar_inverse():
    Ph0 = F.S_psi_parts()["phi"][0]
    q2 = sum(F.ETA[r] * L[r] ** 2 for r in range(4))
    Hphi = sp.expand(_pw(Ph0, [{"f": "phi", "pol": {(): 1}, "k": mom(-1, 0)}, {"f": "phi", "pol": {(): 1}, "k"
  +> : mom(1, 0)}]))
    c = sp.cancel(Hphi / q2)
    return {"cinv": 1 / c, "Hphi_is_q2_times_constant": not c.free_symbols - {F.KAPPA}}


def structural_checks():
    S = F.S_psi_parts()
    X = S["X"]
    return {"X1_has_no_field_only_monomial (no b.h.h, h^3 in X)": () not in X[1].t,
            "X2_is_zero (no cbar.c.h.h, no ghost tadpole at kappa^2)": X[2].is_zero()}


# ------------------------------------------------------------------ the Hessian-route integrands
def reference():
    """{graph: {(A,B): numerator}} with numerator := (i Gamma^{AB} integrand) x prod of the denominators."""
    if "ref" in _REF:
        return _REF["ref"]
    t0 = time.time()
    kap = F.KAPPA
    gi, gh, sc = graviton_inverse(), ghost_inverse(), scalar_inverse()
    Ci = {(i, j): gi["Cinv"][i, j] for i in range(10) for j in range(10) if gi["Cinv"][i, j] != 0}
    Ci_cols = {}
    for (i, j), v in Ci.items():
        Ci_cols.setdefault(i, []).append((j, v))
    out = {}
    # D01: + 1/2 Tr(G U_A G U_B), U = kappa V3
    UA = ell_table(3, [(0, 1), (-1, 0), (1, -1)])     # h_A(p), Y(-l), Z(l-p)
    UB = ell_table(3, [(0, -1), (-1, 1), (1, 0)])     # h_B(-p), W(p-l), X(l)
    UAs = {k: _sym(v) for k, v in UA.items()}
    UBs = {k: _sym(v) for k, v in UB.items()}
    # W_B[(B, Z, Y)] = sum_{W,X} Cinv[Z,W] U_B[B,W,X] Cinv[X,Y]
    WB = {}
    for (B, W, X), u in UBs.items():
        for (Z, Wc) in [(z, v) for (z, w), v in Ci.items() if w == W]:
            for (Y, v2) in Ci_cols.get(X, []):
                WB[(B, Z, Y)] = WB.get((B, Z, Y), 0) + Wc * u * v2
    N = {}
    for (A, Y, Z), u in UAs.items():
        for B in range(10):
            w = WB.get((B, Z, Y))
            if w is not None:
                N[(A, B)] = N.get((A, B), 0) + u * w
    out["D01"] = {k: sp.expand(sp.Rational(1, 2) * kap ** 2 * v) for k, v in N.items()}
    # D02: - 1/2 Tr(G U2), U2 = kappa^2 V4[h_A(p), h_B(-p), Y(-l), X(l)]
    U4 = ell_table(4, [(0, 1), (0, -1), (-1, 0), (1, 0)])
    N = {}
    for (A, B, Y, X), d in U4.items():
        v = Ci.get((X, Y))
        if v is not None:
            N[(A, B)] = N.get((A, B), 0) + v * _sym(d)
    out["D02"] = {k: sp.expand(-sp.Rational(1, 2) * kap ** 2 * v) for k, v in N.items()}
    # D03: - Tr(G U_A G U_B), ghosts; U = kappa X^(1) third variation
    X1 = F.S_psi_parts()["X"][1]
    mi = gh["minv"]

    def gv(rho, mu, h, k_cb, k_c, k_h):
        return _pw(X1, [{"f": "cb", "c": rho, "k": k_cb}, {"f": "c", "c": mu, "k": k_c}, h_leg(h, k_h)])
    VA = {(r, m, a): gv(r, m, a, mom(-1, 0), mom(1, -1), mom(0, 1)) for r in range(4) for m in range(4) for a
  +> in range(10)}
    VB = {(r, m, b): gv(r, m, b, mom(-1, 1), mom(1, 0), mom(0, -1)) for r in range(4) for m in range(4) for b
```

```
  +> in range(10)}
    N = {}
    for a in range(10):
        for b in range(10):
            tot = 0
            for m1 in range(4):
                for r1 in range(4):
                    if mi[m1, r1] == 0:
                        continue
                    for m2 in range(4):
                        ua = VA[(r1, m2, a)]
                        if ua == 0:
                            continue
                        for r2 in range(4):
                            if mi[m2, r2] == 0:
                                continue
                            tot += mi[m1, r1] * ua * mi[m2, r2] * VB[(r2, m1, b)]
            tot = sp.expand(-kap ** 2 * tot)
            if tot != 0:
                N[(a, b)] = tot
    out["DO3"] = N
    # D12: - 1/2 Tr(G_phi U2), U2 = kappa^2 S_phi^(2) fourth variation
    Ph2 = F.S_psi_parts()["phi"][2]
    N = {}
    for a in range(10):
        for b in range(10):
            v = _pw(Ph2, [h_leg(a, mom(0, 1)), h_leg(b, mom(0, -1)), {"f": "phi", "pol": {(): 1}, "k": mom(-1,
  +>  0)},
                           {"f": "phi", "pol": {(): 1}, "k": mom(1, 0)}])
            v = sp.expand(-sp.Rational(1, 2) * kap ** 2 * sc["cinv"] * v)
            if v != 0:
                N[(a, b)] = v
    out["D12"] = N
    meta = {"graviton_inverse": {k: v for k, v in gi.items() if k != "Cinv"},
            "ghost": {"M0_is_q2_times_constant": gh["M0_is_q2_times_constant"]},
            "scalar": {"Hphi_is_q2_times_constant": sc["Hphi_is_q2_times_constant"]},
            "structural": structural_checks(), "seconds": round(time.time() - t0, 1)}
    _REF["ref"] = (out, meta)
    return _REF["ref"]


def _assembler_side(rec):
    """prefactor * N from a Feynman-rule record (in-memory SP polys)."""
    names = {r: L[r] for r in range(4)} | {4 + r: P[r] for r in range(4)}
    return {k: sp.expand(rec["prefactor"] * v.to_sympy(names)) for k, v in rec["N"].items()}


def _export_side(g, root=ROOT):
    body = json.loads((Path(root) / "exports" / "P14_integrands" / f"{g}.json").read_text())
    pref = sp.sympify(body["prefactor"], locals={"kappa": F.KAPPA})
    vm = {str(v): v for v in VARS}
    out = {}
    for key, rows in body["numerator"].items():
        poly = sp.Add(*[sp.Rational(cf) * sp.Mul(*[vm[n] for n in names]) for names, cf in rows])
        out[tuple(int(x) for x in key.split(","))] = sp.expand(pref * poly)
    return out, body["sha256"]


def compare(mine, theirs):
    keys = set(mine) | set(theirs)
    bad = [list(k) for k in sorted(keys) if sp.expand(mine.get(k, 0) - theirs.get(k, 0)) != 0]
    return {"components": len(keys), "nonzero_components": sum(1 for k in keys if mine.get(k, 0) != 0),
            "mismatches": bad[:10], "n_mismatches": len(bad), "agree": not bad and bool(keys)}


def W6(ints):
    """control for amplitude_rules.verify: Hessian route == assembler integrands (current rules)."""
    ref, meta = reference()
    checks, detail = {}, {}
    for g in GRAPHS:
        c = compare(ref[g], _assembler_side(ints[g]))
        detail[g] = c
        checks[f"{g}_hessian_route_equals_integrand"] = c["agree"]
    for k, v in meta["graviton_inverse"].items():
        if isinstance(v, bool) and k != "X0_hh_part_zero":
            checks[f"graviton_{k}"] = v
    checks["ghost_M0_is_q2_times_constant"] = meta["ghost"]["M0_is_q2_times_constant"]
    checks.update({f"structural: {k}": v for k, v in meta["structural"].items()})
    return {"checks": checks, "pass": all(checks.values()), "detail": detail,
            "route": "Tr log of the field-dependent Hessian of S_Psi (independent of the Feynman-rule assemble
  +> r)",
```

```
            "X0_hh_part_zero": meta["graviton_inverse"]["X0_hh_part_zero"], "seconds": meta["seconds"]}


def compare_exports(root=ROOT):
    ref, _ = reference()
    res = {}
    for g in GRAPHS:
        th, sha = _export_side(g, root)
        res[g] = {**compare(ref[g], th), "export_sha256": sha}
    res["all_agree"] = all(v["agree"] for v in res.values() if isinstance(v, dict))
    return res


if __name__ == "__main__":
    t0 = time.time()
    r = compare_exports()
    _, meta = reference()
    print(json.dumps({"exports": {g: {k: v for k, v in r[g].items() if k != "mismatches"} if isinstance(r[g],
  +> dict)
                                  else r[g] for g in r}, "meta": meta, "seconds": round(time.time() - t0, 1)},
                     indent=1, default=str))
    raise SystemExit(0 if r["all_agree"] else 1)
```

124 Source: release__p14.py (p14 release gate)

[listing 91/133] code/p5_entry/release_p14.py

```
#!/usr/bin/env python3
"""XVI-P p14/p14r release gate: P.5 ENTRY (contract) on top of the FROZEN P.3/P.4 classical inputs.

Classical part: verbatim fork of the p13 acceptance computation (classical_block) over the P13_* classical cer
  +> tificates,
which are regenerated by byte-identical p13 code (frozen; archival/p13/frozen_p13_inputs.json).

P.5 entry (p14): earned  <=>  quantum_action_defined AND quantum_prescription_defined AND D11_matching_defined
  +>  AND
counterterm_domain_defined AND amplitude_rules_verified  (each recomputed from its P14 certificate, never assu
  +> med),
stated against the P.3/P.4 freeze:  P3_P4_frozen_as_P5_inputs (freeze_p13.freeze: p13 independent rebuild pass
  +> ed --
bound to the p13 tree 2b1baf33.../zip 2f85a566... -- AND frozen code byte-identical AND frozen classical value
  +> s equal).
Opening P.5 entry earns NONE of the loop/restoration flags (HARD_STOPS).

Reproducibility (package evidence, XVI-O conv:package; not mathematical certificates; not part of release_clai
  +> m_accepted):
  author_rebuild_passed (p14r)     : reproducibility/author_rebuild_report_p14r.json, bound to the CURRENT (p1
  +> 4r) tree;
  independent_rebuild_passed (p14r): "pending" until reproducibility/independent_rebuild_report_p14r.json exis
  +> ts;
  independent_rebuild_passed (p13) : true only via the archived p13 report, bound to the p13 tree -- NOT inher
  +> ited by p14.
Publication: P14_release.json / P14_closure_release.json are written only if release_claim_accepted = (p13 cla
  +> ssical
claim, recomputed) AND (p14 hard stops) AND P5_entry_earned; otherwise P14_release_rejected.json and exit 1.
"""
from __future__ import annotations
import hashlib, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
sys.path.insert(0, str(ROOT / "tools"))
import conventions as C  # noqa: E402   (frozen p13 conventions; read only)
import p14_conventions as PC  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
import inherited  # noqa: E402
import freeze_p13  # noqa: E402
from release import rebuild_flag, j  # noqa: E402,F401   (frozen helpers, reused unchanged)

P14_CERTS = {"quantum_action_defined": ("P14_quantum_object.json", "quantum_action_defined"),
             "quantum_prescription_defined": ("P14_prescription_D11_matching.json", "quantum_prescription_defi
  +> ned"),
             "D11_matching_defined": ("P14_prescription_D11_matching.json", "D11_matching_defined"),
             "counterterm_domain_defined": ("P14_counterterm_domain.json", "counterterm_domain_defined"),
```

```
            "amplitude_rules_verified": ("P14_amplitude_rules.json", "amplitude_rules_verified")}


def classical_block():
    """VERBATIM fork of the p13 classical acceptance computation (code/classical/release.py _main, up to the
    reproducibility flags); reads the frozen P13_* classical certificates regenerated by unchanged code."""
    try:
        pin = inherited.sha(inherited.resolve("XVI-M.tex"))
    except inherited.PinError:
        pin = "unavailable"
    pin_ok = pin == C.XVI_M_PIN
    q, br, sa = j("P13_quartic_verified.json"), j("P13_brst_derivation.json"), j("P13_source_action.json")
    gf, ms, ch = j("P13_gauge_fixing_map.json"), j("P13_master_kappa2.json"), j("P13_ch_source_audit.json")
    bi = j("P13_boundary_identity.json")
    flags = {
        "quartic_generator_verified": bool(q["quartic_generator_verified"]),
        "ch_projection_defined": bool(ch["ch_projection_defined"]),
        "ch_dependency_inventory_complete": bool(ch["ch_dependency_inventory_complete"]),
        "gauge_fixing_map_constructed": bool(gf["gauge_fixing_map_constructed"]),
        "required_classical_BV_components_verified": bool(ms["required_classical_BV_components_verified"]),
    }
    residuals = list(ms["required_master_residuals"])
    # ---- hash consistency: export files == quartic certificate == master inputs ----
    import sympy as sp
    file_sha = {}
    for k in (2, 3, 4):
        txt = (ROOT / "exports" / f"ell{k}_generating_polynomial.srepr").read_text().strip()
        file_sha[k] = hashlib.sha256(str(sp.sympify(txt)).encode()).hexdigest()
    hashes = {f"ell{k}": {"export_file": file_sha[k], "quartic_cert": q.get(f"ell{k}_sha256"),
                          "master_input": ms["identification_with_exports"]["ell2_ell3_ell4"][str(k)]["sha256"
  +> ]
                          if str(k) in ms["identification_with_exports"]["ell2_ell3_ell4"]
                          else ms["identification_with_exports"]["ell2_ell3_ell4"][k]["sha256"]} for k in (2,
  +> 3, 4)}
    hash_consistent = all(len({v["export_file"], v["quartic_cert"], v["master_input"]}) == 1 for v in hashes.v
  +> alues())
    # ---- deterministic acceptance prerequisites ----
    gdet = ms["details"]["grav_af0_deterministic"]
    hess = gf["propagators"]["graviton"]
    det_prereq = {
        "C1_inverse_10comp": q["acceptance_controls_deterministic"]["C1"],
        "C2_det_tracelog_10comp": q["acceptance_controls_deterministic"]["C2"],
        "C3_boundary_identity_generic_and_conformal": bool(bi["C3_proved"]) and q["acceptance_controls_determi
  +> nistic"]["C3"],
        "C4_export_equals_independent_evaluator": q["acceptance_controls_deterministic"]["C4"],
        "C5_cubic_regression_Euler": q["acceptance_controls_deterministic"]["C5"],
        "C6_quadratic_regression_Euler": q["acceptance_controls_deterministic"]["C6"],
        "C7_grav_noether_k0_k1_k2_on_exports": q["acceptance_controls_deterministic"]["C7"],
        "quartic_mutations_all_rejected": bool(q["mutations_all_rejected"]),
        "master_grav_noether_deterministic_k0_k1_k2": all(gdet[f"kappa{n}"]["Q_identically_zero"] for n in (0,
  +>  1, 2)),
        "master_engine_parts_exact": not [r for r in residuals],
        "graviton_hessian_generic_symbolic_k": bool(hess.get("schur_equals_k2P")) and bool(hess.get("propagato
  +> r_hh_equals_inv_k2P"))
                                               and "generic" in str(hess.get("momentum", "")),
        "canonicity_all_23_antifield_components": bool(gf["canonicity"]["pass"]) and gf["canonicity"]["pairs_t
  +> ested"] == 299,
        "ch_enumeration_lemma_mechanical": bool(ch["enumeration_lemma"]["holds"]),
    }
    # ---- p12: residual-record invariants, per-order acceptance, cross-certificate agreement ----
    allc = {"quartic": q, "brst": br, "source": sa, "gauge": gf, "master": ms, "ch": ch, "boundary": bi}
    inv_bad = {k: certio.invariant_violations(v) for k, v in allc.items()}
    inv_bad = {k: v for k, v in inv_bad.items() if v}
    q_c7 = q["controls"]["C7_noether_exact"]
    acc_q = certio.accepted_orders(q_c7)
    acc_m = certio.accepted_orders(gdet)
    cross = certio.cross_check(q_c7, q["controls"].get("C7_parameters"), gdet, ms["details"].get("grav_af0_par
  +> ameters"))
    stored_vs_recomputed = (q["acceptance_controls_deterministic"]["C7"] == all(acc_q.values()))
    det_prereq.update({
        "residual_record_invariants_all_certificates": not inv_bad,
        "quartic_C7_Q_terms_zero_and_flag_true_all_orders": all(acc_q.values()),
        "master_grav_Q_terms_zero_and_flag_true_all_orders": all(acc_m.values()),
        "cross_certificate_quartic_vs_master_agree": cross["agree"],
        "stored_C7_flag_equals_recomputation": stored_vs_recomputed,
    })
    det_prereq["C7_grav_noether_k0_k1_k2_on_exports"] = det_prereq["C7_grav_noether_k0_k1_k2_on_exports"] and
  +> all(acc_q.values())
    det_prereq["master_grav_noether_deterministic_k0_k1_k2"] = det_prereq["master_grav_noether_deterministic_k
  +> 0_k1_k2"] and all(acc_m.values())
    deterministic_ok = all(det_prereq.values())
```

```
    failed_prereq = [k for k, v in det_prereq.items() if not v] + ([] if hash_consistent else ["hash_consisten
  +> cy"])
    flags["quartic_generator_verified"] = (flags["quartic_generator_verified"] and hash_consistent and bool(bi
  +> ["C3_proved"])
                                           and all(acc_q.values()) and "quartic" not in inv_bad)
    flags["required_classical_BV_components_verified"] = (flags["required_classical_BV_components_verified"]
                                                          and hash_consistent and not residuals and all(acc_m.
  +> values())
                                                          and cross["agree"] and "master" not in inv_bad)
    conj = all(flags.values())
    hard = dict(C.HARD_STOPS)
    hard_ok = (hard["quantum_inputs_evaluated"] == "not_earned" and hard["projected_identity_verified"] == "no
  +> t_earned"
               and hard["D02_quantum_evaluated"] is False and hard["D12_quantum_evaluated"] is False
               and hard["D12_status"] == "Missing" and hard["symmetry_restored"] is False
               and hard["O1_closure"] == "NOT_CLOSED" and hard["XV_B_opened"] is False
               and hard["Route_B_discharged"] is False and hard["loop_evaluations_performed"] is False
               and hard["u_R_c_delta_added"] is False
               and q["D02_quantum_evaluated"] is False)
    execution_pass = bool(pin_ok and hard_ok and all(x["execution_pass"] for x in (q, br, sa, gf, ms, ch)))
    statuses = {k: ("earned" if v else "Partial") for k, v in flags.items()}
    release_claim_accepted = bool(execution_pass and conj and deterministic_ok and hash_consistent)
    statuses.update({
        "classical_readiness_for_ch_identity": "earned" if conj else "not_earned",
        "quantum_inputs_evaluated": "not_earned",
        "projected_identity_verified": "not_earned",
        "propagators_match_transformed_Hessians": bool(gf["propagators_match_transformed_Hessians"]),
        "source_identity_relation_defined": bool(gf["source_identity_relation_defined"]),
        "source_identity_relation": ("defined classical pullback/interface (related, not identical): "
                                     "Gamma_Psi(Phi,Phi*') := Gamma(Phi, Phi*' - dPsi/dPhi); not a quantum the
  +> orem"),
        "D02_vertex_available": bool(q["D02_vertex_available"]),
        "D02_quantum_evaluated": False, "D12_quantum_evaluated": False, "D12_status": "Missing",
    })
    return {k: v for k, v in locals().items() if k in ('pin', 'pin_ok', 'q', 'br', 'sa', 'gf', 'ms', 'ch', 'bi
  +> ',
            'flags', 'residuals', 'hashes', 'hash_consistent', 'det_prereq', 'inv_bad', 'q_c7', 'acc_q', 'acc_
  +> m',
            'gdet', 'cross', 'failed_prereq', 'conj', 'hard', 'hard_ok', 'execution_pass', 'statuses',
            'release_claim_accepted', 'deterministic_ok')}

def p5_entry_flags():
    flags, recs, ok_exec = {}, {}, True
    for flag, (fn, key) in P14_CERTS.items():
        f = ROOT / "certificates" / fn
        if not f.is_file():
            flags[flag], ok_exec = False, False
            recs[flag] = {"certificate": fn, "present": False}
            continue
        d = json.loads(f.read_text())
        flags[flag] = d.get(key) is True and d.get("revision") == PC.REVISION
        recs[flag] = {"certificate": fn, "present": True, "value": d.get(key), "revision": d.get("revision"),
                      "content_sha256_ignoring_timings": certio.file_content_sha(f)}
    # recomputed sub-conditions (never trust a stored Boolean alone)
    amp = json.loads((ROOT / "certificates" / "P14_amplitude_rules.json").read_text()) if recs["amplitude_rule
  +> s_verified"]["present"] else {}
    if amp:
        ctl_ok = all(v.get("pass") is True for v in amp.get("controls", {}).values())
        muts = amp.get("mutations", {})
        mut_ok = sorted(muts) == sorted(PC_MUTATIONS) and all(v.get("rejected") is True for v in muts.values()
  +> )
        flags["amplitude_rules_verified"] = bool(flags["amplitude_rules_verified"] and ctl_ok and mut_ok
                                                 and not amp.get("mutations_from_env")
                                                 and amp.get("loop_evaluations_performed") is False)
        recs["amplitude_rules_verified"].update({"controls_all_pass": ctl_ok, "mutations_all_rejected": mut_ok
  +> })
    if amp:
        w6_ok = bool(amp.get("controls", {}).get("W6_hessian_route", {}).get("pass") is True
                     and amp.get("W6_exports", {}).get("all_agree") is True)
        flags["amplitude_rules_verified"] = bool(flags["amplitude_rules_verified"] and w6_ok)
        recs["amplitude_rules_verified"]["W6_hessian_route_and_exports"] = w6_ok
    # p14r: the prescription flag is re-earned only with the operational DRED rules (recomputed from their cer
  +> tificate)
    dr_f = ROOT / "certificates" / "P14_dred_rules.json"
    dr = json.loads(dr_f.read_text()) if dr_f.is_file() else {}
    dred_ok = bool(dr.get("dred_rules_verified") is True and dr.get("revision") == PC.REVISION
                   and dr.get("mutations_all_rejected") is True
                   and sorted(dr.get("mutations", {})) == sorted(DRED_MUTATIONS)
                   and all(v.get("rejected") is True for v in dr.get("mutations", {}).values())
                   and dr.get("trace_rules_check", {}).get("pass") is True
                   and dr.get("evanescent_parts_present_in_exports") is True
```

```
                and dr.get("loop_evaluations_performed") is False)
    flags["quantum_prescription_defined"] = bool(flags["quantum_prescription_defined"] and dred_ok)
    recs["quantum_prescription_defined"].update({"dred_certificate_present": dr_f.is_file(), "dred_rules_ok":
  +> dred_ok})
    pd_f = ROOT / "certificates" / "P14_prescription_D11_matching.json"
    if pd_f.is_file():
        pd = json.loads(pd_f.read_text())
        flags["quantum_prescription_defined"] = bool(
            flags["quantum_prescription_defined"]
            and pd.get("prescription_checks", {}).get("operational_DRED_rules_verified") is True)
        m1 = pd["matching"]["M1_integrand_identity"]
        flags["D11_matching_defined"] = bool(flags["D11_matching_defined"] and m1["pass"] and not m1["mismatch
  +> es"]
                                             and m1["component_pairs_checked"] == 100)
        recs["D11_matching_defined"].update({"M1_pass": m1["pass"], "D11_matching_verified": pd["D11_matching_
  +> verified"]})
    ct_f = ROOT / "certificates" / "P14_counterterm_domain.json"
    if ct_f.is_file():
        ct = json.loads(ct_f.read_text())
        bs = ct["kappa2"]["basis_size_by_sector"]
        flags["counterterm_domain_defined"] = bool(flags["counterterm_domain_defined"] and ct["kappa1"]["basis
  +> _size"] == 0
                                                   and ct["kappa2"]["basis_size"] == sum(bs.values())
                                                   and ct["XIV_J_opened"] is False and ct["XV_B_opened"] is Fa
  +> lse)
        recs["counterterm_domain_defined"].update({"kappa1": ct["kappa1"]["basis_size"], "kappa2": ct["kappa2"
  +> ]["basis_size"],
                                                   "by_sector": bs})
    return flags, recs, ok_exec


DRED_MUTATIONS = ("premature_d4", "trace_ghat_eq_4", "contract_after_laurent")
PC_MUTATIONS = ("wrong_tadpole_factor", "wrong_ghost_loop_sign", "offdiag_multiplicity", "reversed_source_ghos
  +> t_order",
                "omit_ghost_transport_term")


def milestones_p14(ch, gi_names_tbd, conj, accepted, frozen, entry_flags):
    p3 = bool(ch["ch_projection_defined"] and ch["ch_dependency_inventory_complete"] and ch["enumeration_lemma
  +> "]["holds"]
              and ch["checks"]["regenerated_rows_match_published"] and gi_names_tbd == [])
    p4 = bool(conj and accepted)
    entry = bool(all(entry_flags.values()))
    st = {"P.3": "complete (declared ch target)" if p3 else "incomplete",
          "P.4": "earned (classical readiness)" if p4 else "not_earned",
          "P.5_entry": ("earned (contract fixed; stated against the P.3/P.4 freeze)" if entry and frozen and p
  +> 3 and p4
                        else ("contract flags hold but P.3/P.4 NOT frozen -> not earned" if entry else "not_ea
  +> rned")),
          "P.5_calculation": "open (not started; Batch A/B/C integrals are p15+)",
          "P.5_identity": "open"}
    return {"definitions": C.MILESTONES, "order": list(C.MILESTONE_ORDER), "status": st,
            "P3_complete": p3, "P4_classical_readiness_earned": p4,
            "P5_entry_earned": bool(entry and frozen and p3 and p4),
            "P5_entry_rule": PC.P5_ENTRY_RULE, "P5_entry_flags": entry_flags,
            "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified
  +>  not earned)",
            "P3_P4_frozen_as_P5_inputs": frozen,
            "frozen_rule": "P.3/P.4 are frozen P.5 inputs iff the p13 independent rebuild passed (bound to the
  +>  p13 tree) "
                           "AND the frozen code/values are unchanged",
            "P5_entry_depends_on": "P3_P4_frozen_as_P5_inputs (p13 independent rebuild; caveat: " + PC.INDEPEN
  +> DENT_CAVEAT + ")",
            "history": C.MILESTONE_HISTORY + "; archival/p13/ (p13 release: P.3 complete, P.4 earned, P.5 entr
  +> y next)",
            "XVI_O_unaffected": "XVI-P is unregistered package evidence; XVI-O prop:kappa2-master stays Partia
  +> l"}


def _main():
    cb = classical_block()
    q, ms, ch = cb["q"], cb["ms"], cb["ch"]
    fr = freeze_p13.freeze(ROOT)
    frozen = fr["P3_P4_frozen_as_P5_inputs"]
    entry_flags, entry_recs, entry_exec = p5_entry_flags()
    hard = dict(PC.HARD_STOPS)
    hard_ok = bool(cb["hard_ok"] and all(hard[k] == v for k, v in PC.HARD_STOPS.items())
                   and hard["D11_matching_verified"] is False and hard["D12_status"] == "Missing"
                   and hard["P5_calculation_opened"] is False and hard["P5_identity_opened"] is False)
    # the P14 certificates themselves must respect the hard stops
    pd = json.loads((ROOT / "certificates" / "P14_prescription_D11_matching.json").read_text())
```

```
  amp = json.loads((ROOT / "certificates" / "P14_amplitude_rules.json").read_text())
  cert_stops = {"D11_matching_verified_false": pd.get("D11_matching_verified") is False,
                "D12_Missing_in_prescription": pd.get("D12_status") == "Missing" and pd.get("D12_value_impor
+> ted") is False,
                "D12_Missing_in_amplitudes": amp["graphs"]["D12"].get("D12_status") == "Missing",
                "no_graph_value": all(g.get("value") == "not evaluated" and g.get("integrated") is False
                                      for g in amp["graphs"].values()),
                "no_loop_evaluation": all(json.loads((ROOT / "certificates" / f).read_text()).get(
                    "loop_evaluations_performed") is False for f in ("P14_quantum_object.json",
                                                                     "P14_prescription_D11_matching.json",
                                                                     "P14_amplitude_rules.json"))}
  hard_ok = hard_ok and all(cert_stops.values())
  execution_pass = bool(cb["execution_pass"] and hard_ok and entry_exec)
  m = milestones_p14(ch, json.loads((ROOT / "ledgers" / "P13_graph_inventory.json").read_text()).get("names_
+> TBD"),
                     cb["conj"], cb["release_claim_accepted"], frozen, entry_flags)
  author_ok, author_rec = rebuild_flag(PC.AUTHOR_REPORT, pending_if_absent=False)
  indep_ok, indep_rec = rebuild_flag(PC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
  p13_indep = {"independent_rebuild_passed": fr["independent_rebuild_passed_p13"],
               "bound_to_source_tree_sha256": PC.P13_SOURCE_TREE_SHA256,
               "bound_to_source_zip_sha256": PC.P13_SOURCE_ZIP_SHA256,
               "report": PC.P13_INDEPENDENT_REPORT, "report_sha256": PC.P13_INDEPENDENT_REPORT_SHA256,
               "comparisons": {"reference_values_matched": fr["F1"].get("compared_with_reference"),
                               "independent_comparisons": fr["F1"].get("independent_comparisons"),
                               "mismatches": fr["F1"].get("mismatches")},
               "caveat": PC.INDEPENDENT_CAVEAT,
               "NOT_inherited_by_p14": "bound to the p13 tree; the p14 tree has its own (pending) independen
+> t flag"}
  release_claim_accepted = bool(cb["release_claim_accepted"] and hard_ok and m["P5_entry_earned"])
  publish = bool(release_claim_accepted and cb["conj"] and execution_pass)
  surviving = [{"row": r["id"], "Gamma1_pattern": r["Gamma1_pattern"], "graphs": r["graphs"],
                "value": "not evaluated (P.5 calculation is p15+); integrands exported where available"}
               for r in ch["surviving_rows"]]
  cert = {
      "landmark": "XVI-P", "revision": PC.REVISION, "date": PC.DATE, "deliverable": "P.5 entry contract",
      "execution_pass": execution_pass,
      "claim_status": ("P.5_entry: " + ("earned (contract fixed)" if m["P5_entry_earned"] else "not_earned")
                       + "; classical_readiness (frozen P.4): " + ("earned" if cb["conj"] else "not_earned")
+> ),
      "scope": ("P.5 entry: the quantum object, ONE prescription with the D11 matching note, the one-loop co
+> unterterm "
                "domain and executable amplitude rules for the fixed one-loop kappa^2 ch identity on the S_i
+> nv baseline; "
                "integrands only; NO loop evaluation; NOT quantum inputs, NOT the projected identity, NOT bu
+> lk "
                "restoration, NOT the prepared S_D, NOT the insertion"),
      "release_claim_accepted": release_claim_accepted,
      "release_claim_accepted_definition": ("p13 classical release claim (recomputed from the frozen inputs)
+>  AND p14/p14r hard "
                                            "stops AND P5_entry_earned (all five contract flags AND "
                                            "P3_P4_frozen_as_P5_inputs)"),
      "P5_entry_earned": m["P5_entry_earned"], "P5_entry_flags": entry_flags, "P5_entry_records": entry_recs
+> ,
      "P5_entry_qualification": PC.P5_ENTRY_QUALIFICATION,
      "P5_entry_dependency": ("P.5 entry is stated against the P.3/P.4 freeze: P3_P4_frozen_as_P5_inputs = "
                              f"{frozen} (p13 independent rebuild passed, bound to the p13 tree; {PC.INDEPEN
+> DENT_CAVEAT})"),
      "P3_P4_frozen_as_P5_inputs": frozen, "freeze": {k: fr[k] for k in fr if k not in ("F1", "F2", "F3")} |
+>  {
          "F1": {k: fr["F1"][k] for k in fr["F1"] if k != "environment"},
          "F2": {k: fr["F2"][k] for k in ("files", "changed_or_missing", "list_sha256_ok", "archived_zip_sha
+> 256_ok",
                                          "list_equals_archived_zip")},
          "F3": {k: fr["F3"][k] for k in ("compared", "equal", "differences", "reference_sha256_ok")}},
      "D11_matching_verified": False, "D12_status": "Missing",
      "projected_vs_bulk": PC.PROJECTED_VS_BULK,
      "classical": {"flags": cb["flags"], "classical_readiness_for_ch_identity": cb["conj"],
                    "release_claim_accepted": cb["release_claim_accepted"],
                    "deterministic_prerequisites": cb["det_prereq"], "failed_prerequisites": cb["failed_prer
+> eq"],
                    "hash_consistency": {"consistent": cb["hash_consistent"], "ell": cb["hashes"]},
                    "required_master_residuals": cb["residuals"],
                    "residual_records": {
                        "invariant_violations": cb["inv_bad"],
                        "quartic_C7_per_order": {o: {"residual_counts": cb["q_c7"][o]["Q_terms"],
                                                     "zero_flag": cb["q_c7"][o]["Q_identically_zero"],
                                                     "accepted": cb["acc_q"][o]} for o in certio.ORDERS},
                        "master_grav_per_order": {o: {"residual_counts": cb["gdet"][o]["Q_terms"],
                                                      "zero_flag": cb["gdet"][o]["Q_identically_zero"],
                                                      "accepted": cb["acc_m"][o]} for o in certio.ORDERS},
                        "cross_certificate": cb["cross"]},
```

```
                  "hard_stops_p13_hold": cb["hard_ok"], "hard_stops_p13": cb["hard"],
                  "certificates": "P13_* (frozen P.3/P.4 inputs, regenerated by byte-identical p13 code)"}
+> ,
      "statuses": {**cb["statuses"], **{k: ("earned" if v else "not_earned") for k, v in entry_flags.items()
+> },
                 "D11_matching_verified": False, "P5_entry": m["status"]["P.5_entry"]},
      "ch_surviving_rows_unevaluated": surviving,
      "remaining_blockers": [
          {"id": "Batch A/B/C one-loop integrals (D01, D02, D03, D12, D13, D14a-c; D11 matching M2/L_match)"
+> ,
           "status": "integrands exported; not evaluated (p15+)", "blocks": "quantum_inputs_evaluated"},
          {"id": "D12_quantum_evaluation", "status": "Missing", "blocks": "quantum_inputs_evaluated"},
          {"id": "D11_matching_verification (M2 on U, L_match)", "status": "defined, not verified",
           "blocks": "D11_matching_verified"}],
      "hard_stops": hard, "hard_stops_hold": hard_ok, "certificate_hard_stops": cert_stops,
      "milestones": m,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p13": p13_indep["independent_rebuild_passed"],
      "rebuild_flag_scope": (f"author_rebuild_passed and independent_rebuild_passed refer to the CURRENT ({P
+> C.REVISION}) source "
                             "tree only; independent_rebuild_passed_p13 refers to the p13 source tree "
                             f"{PC.P13_SOURCE_TREE_SHA256} / zip {PC.P13_SOURCE_ZIP_SHA256} and is not inher
+> ited by "
                             f"{PC.REVISION} changes; it carries the P3_P4_frozen_as_P5_inputs freeze. Cavea
+> t: "
                             + PC.INDEPENDENT_CAVEAT),
      "p13_independent_rebuild": p13_indep,
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p13_archive": PC.P13_ARCHIVE,
                          "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"},
      "XVI_P_registered": False,
      "symmetry_restored": False, "bulk_symmetry_restored": False, "O1_closure": "NOT_CLOSED",
      "insertion_status": "Frozen_negative_obstruction_preserved", "XV_B_opened": False, "XIV_J_opened": Fal
+> se,
      "Route_B_discharged": False,
      "obstruction": "XVI-N thm:massless-obstruction respected; no completion of the prepared/inserted funct
+> ional; no "
                     "c delta added to u^R_mu (XVI-J prop:extension); counterterm domain is a constraint lis
+> t, not an "
                     "admission",
      "not_premises": C.NOT_PREMISES, "baseline": C.BASELINE, "ghost_normalization_a": "1/2",
      "XVI_M_sha256": cb["pin"], "XVI_M_pin_ok": cb["pin_ok"],
      "XVI_A_to_O_edited": False, "XVI_O_o70_sync": "read-only",
      "inputs_content_sha256_ignoring_timings": {n: certio.file_content_sha(ROOT / "certificates" / n) for n
+>  in
                  ("P13_quartic_verified.json", "P13_boundary_identity.json", "P13_brst_derivation.json",
                   "P13_source_action.json", "P13_gauge_fixing_map.json", "P13_master_kappa2.json",
                   "P13_ch_source_audit.json", "P14_quantum_object.json", "P14_prescription_D11_matching.json
+> ",
                   "P14_counterterm_domain.json", "P14_amplitude_rules.json")},
  }
  cdir = ROOT / "certificates"
  closure = [cdir / "P14_release.json", cdir / "P14_closure_release.json"]
  cert["publication"] = "accepted" if publish else "REJECTED"
  txt = certio.dumps(cert)
  (ROOT / "RELEASE_STATUS.json").write_text(txt)
  (cdir / "RELEASE_STATUS.json").write_text(txt)
  certio.write(ROOT / "ledgers" / "P14_milestones.json", {
      "landmark": "XVI-P", "revision": PC.REVISION, "ledger": "milestones (authoritative definitions + statu
+> s)",
      **m, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "p13_independent_rebuild": p13_indep, "hard_stops": hard, "dossier": "certificates/P14_dossier.json"})
  if publish:
      (cdir / "P14_release_rejected.json").unlink(missing_ok=True)
      closure[0].write_text(txt)
      closure[1].write_text(certio.dumps({
          "landmark": "XVI-P", "revision": PC.REVISION, "P5_entry_earned": m["P5_entry_earned"],
          "P5_entry_flags": entry_flags, "P3_P4_frozen_as_P5_inputs": frozen,
          "P5_entry_dependency": cert["P5_entry_dependency"], "release_claim_accepted": True,
          "P5_entry_qualification": PC.P5_ENTRY_QUALIFICATION,
          "classical_readiness_for_ch_identity": True, "scope": cert["scope"],
          "quantum_inputs_evaluated": "not_earned", "projected_defect_computed": "not_earned",
          "projected_identity_verified": "not_earned", "projected_restoration_verified": "not_earned",
          "bulk_symmetry_restored": False, "D11_matching_verified": False, "D12_status": "Missing",
          "O1_closure": "NOT_CLOSED", "release_certificate": "certificates/P14_release.json",
          "hashes_of_release_files": "external manifest only (RELEASE_MANIFEST_EXTERNAL.txt)"}))
  else:
      for f in closure:
          f.unlink(missing_ok=True)
```

```
        (cdir / "P14_release_rejected.json").write_text(certio.dumps({
            "landmark": "XVI-P", "revision": PC.REVISION, "publication": "REJECTED",
            "release_claim_accepted": release_claim_accepted, "execution_pass": execution_pass,
            "P5_entry_flags": entry_flags, "P5_entry_earned": m["P5_entry_earned"],
            "P3_P4_frozen_as_P5_inputs": frozen, "hard_stops_hold": hard_ok, "certificate_hard_stops": cert_st
  +> ops,
            "failed_prerequisites": cb["failed_prereq"]}))
    print(json.dumps({"execution_pass": execution_pass, "release_claim_accepted": release_claim_accepted,
                      "P5_entry_flags": entry_flags, "P5_entry_earned": m["P5_entry_earned"],
                      "P3_P4_frozen_as_P5_inputs": frozen, "author": author_ok, "independent_p14": indep_ok,
                      "independent_p13": p13_indep["independent_rebuild_passed"], "hard_ok": hard_ok,
                      "publication": cert["publication"]}, indent=1))
    return 0 if publish else 1


def reject_on_error(exc):
    cdir = ROOT / "certificates"
    for f in (cdir / "P14_release.json", cdir / "P14_closure_release.json"):
        f.unlink(missing_ok=True)
    rec = {"landmark": "XVI-P", "revision": PC.REVISION, "publication": "REJECTED", "release_claim_accepted":
  +> False,
          "P5_entry_earned": False, "error": f"{type(exc).__name__}: {exc}",
          "failed_prerequisites": ["release_evaluation_error"]}
    txt = certio.dumps(rec)
    (cdir / "P14_release_rejected.json").write_text(txt)
    (ROOT / "RELEASE_STATUS.json").write_text(txt)
    (cdir / "RELEASE_STATUS.json").write_text(txt)
    print(json.dumps({"publication": "REJECTED", "error": rec["error"]}))
    return 1


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001
        return reject_on_error(exc)


if __name__ == "__main__":
    raise SystemExit(main())
```

125 Source: dossier__p14.py (P.5 entry dossier)

[listing 92/133] code/p5_entry/dossier_p14.py

```
#!/usr/bin/env python3
"""p14 dossier: the frozen P.3/P.4 chains (re-pointed to the p14 release gate, which recomputes the classical
acceptance from the frozen P13_* certificates) + the P.5 ENTRY chain.  Every link is resolved mechanically
(function in file via AST, certificate key = expected value, test present in tests/test_p14.py) by the frozen
p13 resolver (code/classical/dossier.py, imported, not edited).  Writes certificates/P14_dossier.json and
manuscript/body/P14-dossier-table.tex, and the P.5 entry summary manuscript/body/P14-entry-summary.tex (values
  +>  read
from the certificates of the build).
"""
from __future__ import annotations
import json, re, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dossier as D  # noqa: E402   (frozen p13 resolver)
import p14_conventions as PC  # noqa: E402
import certio  # noqa: E402

D.T = "tests/test_p14.py"
R, ALL_TRUE, NONEMPTY = D.R, D.ALL_TRUE, D.NONEMPTY
RS = "RELEASE_STATUS.json"
F_RL14 = "code/p5_entry/release_p14.py"
QO, PD = "certificates/P14_quantum_object.json", "certificates/P14_prescription_D11_matching.json"
CT, AM = "certificates/P14_counterterm_domain.json", "certificates/P14_amplitude_rules.json"
F_QO, F_PD, F_CT = "code/p5_entry/quantum_object.py", "code/p5_entry/prescription_d11.py", "code/p5_entry/coun
  +> terterm_domain.py"
F_AM, F_FZ, F_FY = "code/p5_entry/amplitude_rules.py", "code/p5_entry/freeze_p13.py", "code/p5_entry/feynman.p
  +> y"
F_SH = "code/p5_entry/shadow.py"
DR, F_DR = "certificates/P14_dred_rules.json", "code/p5_entry/dred.py"
```

```
F_HR = "code/p5_entry/hessian_route.py"


def classical_chains():
    """p13 chains, unchanged claims/evidence; RELEASE_STATUS keys moved under /classical; the release function
  +>  is the
    p14 fork of the p13 acceptance (release_p14.classical_block)."""
    out = []
    for m, name, rows in D.CHAINS:
        nr = []
        for r in rows:
            r = json.loads(json.dumps(r))
            if r["file"] == D.F_RL:
                r["file"], r["functions"] = F_RL14, ["classical_block"]
            for c in r["checks"]:
                if c["source"] == D.RS and not c["key"].startswith("/classical/"):
                    c["key"] = "/classical" + c["key"]
            nr.append(r)
        out.append((m, name + " [frozen P.3/P.4 input]", nr))
    return out


P5_CHAIN = ("P.5_entry", "quantum object -> prescription/D11 matching -> counterterm domain -> amplitude rules
  +>  -> gate", [
    R("P5E.1", "quantum object in transformed variables: S_Psi parts, eps = -1, b retained (elimination makes
  +> S quadratic "
                "in cbar*'), Gamma_1 independent of cbar*', b*'; pullback != quantum theorem", F_QO,
      ["main", "patterns", "b_elimination"],
      [(QO, "/quantum_action_defined", True), (QO, "/Q1_transformed_variables/eps_derived", -1),
       (QO, "/Q2_fields_antifields_b/b_decision", "RETAINED and integrated"),
       (QO, "/Q2_fields_antifields_b/Gamma1_independent_of", ["bs", "cbs"]),
       (QO, "/Q5_kappa_loop_bookkeeping/ok", True),
       (QO, "/Q6_pullback_vs_theorem/quantum_gauge_change_theorem_asserted", False)],
      ["test_quantum_object"]),
    R("P5E.2", "ONE prescription: operational DRED (rules R1-R6), MS-bar, Feynman i0; scaleless rules defined
  +> but not "
                "applied; quantum_prescription_defined requires the verified DRED rules", F_PD,
      ["main"], [(PD, "/quantum_prescription_defined", True), (PD, "/prescription/id", "P14_DRED_MSbar_Feynman
  +> I0"),
                 (PD, "/prescription_checks", ALL_TRUE),
                 (PD, "/prescription_checks/operational_DRED_rules_verified", True), (PD, "/D12_status", "Miss
  +> ing")],
      ["test_prescription_and_D11_matching"]),
    R("P5E.2b", "operational DRED: g4 = ghat + gtilde (tr d, 4-d), l and p in QdS, l_r = l.e_(r) lifting, ghat
  +> -projected "
                "Gram/PV reduction, contractions before Laurent with d symbolic; evanescent parts retained; pr
  +> emature "
                "d=4, tr ghat=4 and expand-before-contract are rejected", F_DR,
      ["reduce_tensor", "eta_trace_Gt", "toy_four_d_part", "toy_evanescent_trace", "known_reductions", "lift_e
  +> xported",
       "main"],
      [(DR, "/dred_rules_verified", True), (DR, "/mutations_all_rejected", True),
       (DR, "/mutations/premature_d4/rejected", True), (DR, "/mutations/trace_ghat_eq_4/rejected", True),
       (DR, "/mutations/contract_after_laurent/rejected", True),
       (DR, "/toy_checks/T1_four_d_part/finite", "a/8 + b/4"), (DR, "/toy_checks/T2_evanescent_trace/finite",
  +> "-a/2"),
       (DR, "/known_reductions/pass", True), (DR, "/evanescent_parts_present_in_exports", True),
       (DR, "/loop_evaluations_performed", False)],
      ["test_dred_rules_certificate", "test_dred_premature_d4_changes_finite_part_and_is_caught",
       "test_dred_evanescent_trace_is_d_minus_4_times_pole", "test_dred_trace_ghat_eq_4_rejected",
       "test_dred_known_reductions_and_contraction_engine"]),
    R("P5E.3", "D11 matching DEFINED (M1 exact integrand identity vs XVI-O V_tensor, 100 pairs; M3 CONDITIONAL
  +>  on M2: "
                "sd(u^R) 4, kernel sd <= 8, contact derivative order <= 4, contact space orders 0/2/4 = 2/4/5,
  +> degree 4 "
                "imposed by normalization condition N4; no c delta; r = -i pi^2 XVI-O side only); NOT verified"
  +> , F_PD,
      ["M1_identity", "M3_local_space", "M3_contact_analysis"],
      [(PD, "/D11_matching_defined", True), (PD, "/D11_matching_verified", False),
       (PD, "/matching/M1_integrand_identity/mismatches", []),
       (PD, "/matching/M1_integrand_identity/component_pairs_checked", 100),
       (PD, "/matching/M1_integrand_identity/bound_to_export", True),
       (PD, "/matching/M3_local_difference/labels/scalar_extension_scaling_degree", 4),
       (PD, "/matching/M3_local_difference/labels/kernel_scaling_degree_bound", 8),
       (PD, "/matching/M3_local_difference/labels/local_contact_derivative_order_bound", 4),
       (PD, "/matching/M3_local_difference/lower_degree_contacts_excluded_by_N4/order_2_mu2", 4),
       (PD, "/matching/M3_local_difference/dimension", 5),
       (PD, "/matching/M3_local_difference/transverse_subspace_dimension", 2),
       (PD, "/matching/M4_finite_adjustments/u_R_c_delta_added_by_P", False),
       (PD, "/matching/M5_residue/r", "-i * pi**2")],
      ["test_prescription_and_D11_matching"]),
```

```
    R("P5E.4", "counterterm domain: kappa^1 empty, kappa^2 basis 11 (hh 5, cbar c 2, K'c 3, phi phi 1) generat
  +> ed "
                "mechanically; eight candidate directions modulo the computed exact equivalences, before imposi
  +> ng the "
                "identity constraints (no cohomological classification: D cap ker sigma_0 not computed); constr
  +> aint "
                "list, not an admission", F_CT,
        ["structures", "admissible_pairs", "rank", "main"],
        [(CT, "/counterterm_domain_defined", True), (CT, "/kappa1/basis_size", 0), (CT, "/kappa2/basis_size", 11
  +> ),
         (CT, "/equivalences/quotient_dimension", 8), (CT, "/equivalences/D_cap_ker_sigma0_computed", False),
         (CT, "/equivalences/S0_density_check/ok", True), (CT, "/XIV_J_opened", False), (CT, "/XV_B_opened", Fal
  +> se)],
        ["test_counterterm_domain"]),
    R("P5E.5", "amplitude rules: vertices (incl. ghost-graviton derivative terms, K' vertex), propagators, ord
  +> ering, "
                "multiplicities, symmetry factors, signs; symbolic integrands; W1-W6 controls (W5 regenerates o
  +> nly "
                "D14a-c, D11, D13; W6 regenerates D01, D02, D03, D12)", F_AM,
        ["build_integrands", "verify", "W2", "W3", "W4", "W5"],
        [(AM, "/amplitude_rules_verified", True), (AM, "/controls/W1_wick_berezin_shadow/pass", True),
         (AM, "/controls/W4_ward_bose/pass", True), (AM, "/integrands_only", True),
         (AM, "/loop_evaluations_performed", False), (AM, "/graphs/D12/D12_status", "Missing")],
        ["test_amplitude_rules_and_mutations"]),
    R("P5E.5b", "independent Hessian/Tr-log regeneration of D01, D02, D03, D12 from the field-dependent Hessia
  +> n of "
                 "S_Psi (own (h,b) inverse; not the Feynman-rule assembler) == exported integrands", F_HR,
        ["ell_table", "graviton_inverse", "ghost_inverse", "reference", "W6", "compare_exports"],
        [(AM, "/controls/W6_hessian_route/pass", True), (AM, "/W6_exports/all_agree", True),
         (AM, "/W6_exports/D01/n_mismatches", 0), (AM, "/W6_exports/D03/n_mismatches", 0)],
        ["test_hessian_route_regenerates_D01_D02_D03_D12"]),
    R("P5E.6", "mutations rejected: tadpole factor, ghost-loop sign, off-diagonal multiplicity, source/ghost o
  +> rder, "
                "ghost transport term", F_AM, ["apply_mutations", "main"],
        [(AM, "/mutations_all_rejected", True), (AM, "/mutations/omit_ghost_transport_term/rejected", True),
         (AM, "/mutations/reversed_source_ghost_order/rejected", True)],
        ["test_amplitude_rules_and_mutations"]),
    R("P5E.7", "Berezin/Wick shadow fixes symmetry factors and Grassmann signs independently", F_SH,
        ["expect", "gamma1", "run_shadows"], [(AM, "/controls/W1_wick_berezin_shadow/pass", True)],
        ["test_amplitude_rules_and_mutations"]),
    R("P5E.8", "P.3/P.4 freeze bound to the p13 tree (independent rebuild passed; same-box caveat); frozen cod
  +> e and "
                "values unchanged", F_FZ, ["freeze", "F1_independent", "F2_frozen_code", "F3_frozen_values"],
        [(RS, "/P3_P4_frozen_as_P5_inputs", True),
         (RS, "/p13_independent_rebuild/bound_to_source_tree_sha256", PC.P13_SOURCE_TREE_SHA256),
         (RS, "/p13_independent_rebuild/caveat", "CONTAINS:SAME box"),
         (RS, "/freeze/F2/changed_or_missing", []), (RS, "/freeze/F3/differences", [])],
        ["test_freeze_binding"]),
    R("P5E.9", "P.5 entry gate: earned iff all five contract flags hold; loop/restoration flags NOT earned", F
  +> _RL14,
        ["_main", "p5_entry_flags", "milestones_p14"],
        [(RS, "/P5_entry_flags", ALL_TRUE), (RS, "/P5_entry_earned", True), (RS, "/hard_stops_hold", True),
         (RS, "/hard_stops/quantum_inputs_evaluated", "not_earned"),
         (RS, "/hard_stops/projected_identity_verified", "not_earned"),
         (RS, "/hard_stops/bulk_symmetry_restored", False), (RS, "/D11_matching_verified", False),
         (RS, "/D12_status", "Missing"), (RS, "/hard_stops/O1_closure", "NOT_CLOSED")],
        ["test_p5_entry_gate_and_hard_stops"]),
    R("P5E.10", "publication gate (p14) re-invoked before pdf/zip/manifest stages", "tools/build.py",
        ["publication_gate", "pdf_readback"], [], ["test_resumed_stage_build_blocked"]),
])


def build():
    cache = {}
    rs = json.loads((ROOT / RS).read_text())
    chains = [{"milestone": m, "chain": name, "rows": [D.resolve_row(r, cache) for r in rows]}
              for m, name, rows in classical_chains() + [P5_CHAIN]]
    sl = D.slots(cache)
    resolved = all(r["resolved"] for ch in chains for r in ch["rows"]) and all(s["resolved"] for s in sl)
    return {
        "landmark": "XVI-P", "revision": PC.REVISION, "document": "P.5 entry dossier (with the frozen P.3/P.4
  +> chains)",
        "status_of_this_document": ("package evidence for XVI-P (unregistered; XVI-O conv:package): an archive
  +> , a listing "
                                    "and a manifest are not a series certificate. Does not update XVI-O "
                                    "(prop:kappa2-master stays Partial there; symmetry_restored false; O.1 NOT
  +> _CLOSED)."),
        "milestone_definitions": rs.get("milestones", {}).get("definitions"), "milestone_status": rs.get("mile
  +> stones", {}),
        "chains": chains, "quantum_input_slots": sl, "hard_stops": rs.get("hard_stops", {}),
        "reproducibility": {k: rs.get(k) for k in ("author_rebuild_passed", "independent_rebuild_passed")},
```

```
        "p13_independent_rebuild": rs.get("p13_independent_rebuild"),
        "P3_P4_frozen_as_P5_inputs": rs.get("P3_P4_frozen_as_P5_inputs"),
        "all_links_resolved": resolved,
        "n_rows": sum(len(ch["rows"]) for ch in chains), "n_slots": len(sl),
    }


def table(d):
    t = D.table(d).replace("P14\\_", "")
    p13 = d.get("p13_independent_rebuild") or {}
    extra = (r"; p13 \texttt{independent\_rebuild\_passed} = " + D.tex(json.dumps(p13.get("independent_rebuild
  +> _passed")))
             + r" (bound to the p13 source tree \texttt{" + D.tex(PC.P13_SOURCE_TREE_SHA256[:16]) + r"...}; ca
  +> veat: "
             + D.tex(PC.INDEPENDENT_CAVEAT) + ")")
    t2, n = re.subn(r"(\\texttt\{P3\\_P4\\_frozen\\_as\\_P5\\_inputs\} = [^.]*)\.\}", lambda m_: m_.group(1) +
  +>  extra + ".}", t)
    assert n == 1
    return t2.replace("(package evidence, read from RELEASE\\_STATUS at build time):}",
                      "(package evidence, read from RELEASE\\_STATUS at build time; p14 flags bound to the p14
  +>  tree):}")


def summary_tex():
    """P.5 entry summary tables, every value read from the certificates of this build."""
    cj = lambda n: json.loads((ROOT / "certificates" / n).read_text())  # noqa: E731
    qo, pd, ct, am = cj("P14_quantum_object.json"), cj("P14_prescription_D11_matching.json"), \
        cj("P14_counterterm_domain.json"), cj("P14_amplitude_rules.json")
    dr = cj("P14_dred_rules.json")
    rs = json.loads((ROOT / RS).read_text())
    t = D.tex
    L = [r"\begin{center}\small", r"\begin{tabular}{@{}p{5.2cm}p{10.3cm}@{}}", r"\toprule contract item & valu
  +> e (from the certificates of this build)\\ \midrule"]
    q2 = qo["Q2_fields_antifields_b"]
    m1 = pd["matching"]["M1_integrand_identity"]
    m3 = pd["matching"]["M3_local_difference"]
    eq = ct["equivalences"]
    rows = [
        ("integrated fields", ", ".join(q2["integrated"])),
        ("external antifields", ", ".join(f"{k}: {v}" for k, v in q2["external_antifields"].items())),
        ("b", f"{q2['b_decision']}; det M_bb = {q2['b_justification']['elimination']['b_hessian_det']}; elimin
  +> ating b gives "
             f"{q2['b_justification']['elimination']['cbar_star_quadratic_terms']} terms quadratic in cbar*';
  +>  Gamma_1 "
             f"independent of {', '.join(q2['Gamma1_independent_of'])}"),
        ("antifield shift", f"eps = {qo['Q1_transformed_variables']['eps_derived']} (derived)"),
        ("prescription", pd["prescription"]["id"] + ": " + pd["prescription"]["subtraction"]),
        ("i0", pd["prescription"]["i0"]),
        ("DRED rules", f"verified = {json.dumps(dr['dred_rules_verified'])}; T1 finite {dr['toy_checks']['T1_f
  +> our_d_part']['finite']} "
                       f"(premature d=4 would give {dr['mutations']['premature_d4'].get('finite_part_if_not_ca
  +> ught')}); "
                       f"T2 finite {dr['toy_checks']['T2_evanescent_trace']['finite']} ((d-4) x pole); mutatio
  +> ns: "
                       + ", ".join(f"{k} {'rejected' if v['rejected'] else 'NOT rejected'}" for k, v in dr["mu
  +> tations"].items())),
        ("D11 matching", f"defined = {json.dumps(pd['D11_matching_defined'])}, verified = "
                         f"{json.dumps(pd['D11_matching_verified'])}; M1: {m1['component_pairs_checked']} comp
  +> onent pairs, "
                         f"{len(m1['mismatches'])} mismatches; M3 (conditional on M2): kernel sd <= "
                         f"{m3['labels']['kernel_scaling_degree_bound']}, contact order <= "
                         f"{m3['labels']['local_contact_derivative_order_bound']}; contact space "
                         + ", ".join(f"{k} {v['dimension']}" for k, v in m3['bounded_contact_space'].items())
                         + f"; under N4 {m3['dimension']} (transverse {m3['transverse_subspace_dimension']});
  +> M2 "
                         f"{pd['matching']['M2_nonlocal_agreement_on_U']['status']}"),
        ("counterterm domain", f"kappa^1: {ct['kappa1']['basis_size']}; kappa^2: {ct['kappa2']['basis_size']}
  +> ("
                               + ", ".join(f"{k} {v}" for k, v in ct['kappa2']['basis_size_by_sector'].items()
  +> )
                               + f"); sigma_0-equivalent: {eq['domain_cap_image']} ("
                               + ", ".join(f"{k} {v}" for k, v in eq['sector_cap_image'].items())
                               + f"); {eq['quotient_dimension']} candidate directions modulo the computed exac
  +> t "
                               "equivalences, before the identity constraints (D cap ker sigma_0 not computed)
  +> "),
        ("amplitude controls", ", ".join(f"{k}: {'pass' if v['pass'] else 'FAIL'}" for k, v in am["controls"].
  +> items())
         + "; W5 regenerates D14a-c, D11, D13; W6 (Hessian route) regenerates D01, D02, D03, D12 == exports: "
         + json.dumps(am["W6_exports"]["all_agree"])),
        ("mutations (must be rejected)", ", ".join(f"{k}: {'rejected' if v['rejected'] else 'NOT rejected'}"
```

```
                                              for k, v in am["mutations"].items())),
        ("P.5 entry flags", ", ".join(f"{k} = {json.dumps(v)}" for k, v in rs["P5_entry_flags"].items())),
        ("P3_P4_frozen_as_P5_inputs", f"{json.dumps(rs['P3_P4_frozen_as_P5_inputs'])} ({rs['p13_independent_re
  +> build']['caveat']})"),
        ("P.5 entry", rs["milestones"]["status"]["P.5_entry"]),
    ]
    for k, v in rows:
        L.append(t(k) + " & " + t(v) + r" \\")
    L += [r"\bottomrule", r"\end{tabular}", r"\end{center}", "",
          r"\begin{center}\small", r"\begin{tabular}{@{}lllrrrl@{}}",
          r"\toprule graph & prefactor & denominators & comps & monomials & deg & status \\ \midrule"]
    for g, r in am["graphs"].items():
        L.append(" & ".join([t(g), r"\texttt{" + t(r["prefactor"]) + "}", t(", ".join(r["denominators"])),
                             str(r["numerator_components"]), str(r["numerator_monomials"]), str(r["numerator_d
  +> egree"]),
                             t(r["status"].split(" (")[0].split(":")[0])]) + r" \\")
    L += [r"\bottomrule", r"\end{tabular}", r"\end{center}"]
    return "\n".join(L) + "\n"


def main():
    d = build()
    certio.write(ROOT / "certificates" / "P14_dossier.json", d)
    (ROOT / "manuscript" / "body").mkdir(parents=True, exist_ok=True)
    (ROOT / "manuscript" / "body" / "P14-dossier-table.tex").write_text(table(d))
    (ROOT / "manuscript" / "body" / "P14-entry-summary.tex").write_text(summary_tex())
    bad = [r["id"] for ch in d["chains"] for r in ch["rows"] if not r["resolved"]] + \
          [s["slot"] for s in d["quantum_input_slots"] if not s["resolved"]]
    print(json.dumps({"all_links_resolved": d["all_links_resolved"], "rows": d["n_rows"], "slots": d["n_slots"
  +> ],
                      "unresolved": bad}))
    return 0 if d["all_links_resolved"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

126 Source: tests/test__p14.py

[listing 93/133] tests/test_p14.py

```
#!/usr/bin/env python3
"""p14 tests: the p13 suite (classical P.3/P.4 code frozen; its certificates P13_* are regenerated by byte-ide
  +> ntical code
and gated by the p14 release, code/p5_entry/release_p14.py) plus the P.5 ENTRY tests: quantum object, prescrip
  +> tion
and D11 matching, counterterm domain, amplitude rules and their mutations, D12 Missing (never zero), projected
  +>  vs bulk,
the five-flag P.5 entry gate, the hard stops, and the P.3/P.4 freeze bound to the p13 tree.

End-to-end chain (each in an isolated copy of code/certificates/exports/ledgers/archival):
  mutation -> stage residual or failed prerequisite -> flag false -> classical readiness false (and the frozen
           values change) -> release_claim_accepted false -> publication rejected (exit != 0, no P14_closure_r
  +> elease.json).
"""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile
from concurrent.futures import ThreadPoolExecutor
from fractions import Fraction as Fr
from pathlib import Path
import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "code" / "classical"))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
import conventions as C  # noqa: E402
import p14_conventions as PC  # noqa: E402
import jetpoly as J  # noqa: E402
import certio  # noqa: E402

CERT = ROOT / "certificates"


def cj(name, root=ROOT):
    return json.loads((root / "certificates" / name).read_text())
```

```
# ------------------------------------------------------------- unit / deterministic
def test_C3_boundary_identity_generic_and_conformal():
    import boundary_identity as BI
    g = BI.generic_jet_proof(+1)
    assert g["identically_zero"] and g["residual_terms"] == 0
    bad = BI.generic_jet_proof(-1)
    assert not bad["identically_zero"] and bad["residual_terms"] > 0
    cc = BI.conformal_control(+1)
    assert cc["residual_zero"] and cc["nontrivial"]
    assert not BI.conformal_control(-1)["residual_zero"]


def test_old_C3_support_is_vacuous():
    import boundary_identity as BI
    v = BI.vacuity_of_old_support()
    assert v["vacuous"] and v["sqrt_g_R"] == "0" and v["L_GG"] == "0" and v["div_B"] == "0"
    assert v["nonzero_Christoffels"] == ["G^0_00"]


def test_boundary_orientation_recorded_everywhere():
    assert "- d_mu B^mu" in C.BOUNDARY_IDENTITY
    for f in (ROOT / "exports" / "ell4_conventions.json", ROOT / "exports" / "V4_grav.json"):
        t = f.read_text()
        assert "= - d_mu B^mu" in t and "= + d_mu B^mu" not in t, f
    bi = cj("P13_boundary_identity.json")
    assert bi["C3_proved"] and bi["integrated_action_unaffected"]
    active = list((ROOT / "code").rglob("*.py"))
    assert not [p for p in active if p.name in ("ell4_verify.py", "ell4_generator.py")]


def test_quartic_acceptance_all_deterministic():
    q = cj("P13_quartic_verified.json")
    assert q["quartic_generator_verified"] and q["all_acceptance_deterministic"]
    assert all(q["acceptance_controls_deterministic"].values()) and len(q["acceptance_controls_deterministic"]
  +> ) == 7
    assert q["mutations_all_rejected"] and len(q["mutations"]) == 7
    assert "NOT acceptance" in q["regression_controls_sampled_NOT_acceptance"]["evaluator_numeric"]["role"]


def test_polarization_convention_single():
    q = cj("P13_quartic_verified.json")
    v4 = json.loads((ROOT / "exports" / "V4_grav.json").read_text())
    for pol in (q["polarization"], v4["polarization"]):
        assert pol["convention"] == C.POLARIZATION
        assert "UNnormalised" in pol["normalisation"]
        assert set(pol["where_factors_enter"]) >= {"1/4!", "i", "kappa", "D02_symmetry_factor"}


def test_grav_noether_exact_and_mutation():
    import grav_noether as GN
    ell, _ = GN.load_export_polys()
    r = GN.noether(ell)
    assert all(r[f"kappa{n}"]["Q_identically_zero"] for n in (0, 1, 2))
    assert sum(r["kappa2"]["s0_piece_alone_Q_terms"]) > 0       # genuine mixed cancellation
    flipped = GN.noether({2: ell[2], 3: J.scale(ell[3], -1), 4: ell[4]})
    assert not flipped["kappa1"]["Q_identically_zero"]


def test_multidegree_lemma_corrected():
    """E_u P_d = P_{d-e_u} E_u on a mixed polynomial (h,c multidegree)."""
    h = J.var("h", (0, 1)); hd = J.var("h", (0, 1), (2,)); c = J.var("c", 0, (1,)); c0 = J.var("c", 0)
    F = J.add(J.mul(h, J.mul(hd, c)), J.mul(c0, hd), J.mul(J.mul(h, h), J.mul(hd, c0)), J.scale(J.mul(hd, hd),
  +>  3))

    def deg(m):
        dh = sum(1 for v in m if J.KEYS[v][0] == "h")
        return (dh, len(m) - dh)

    def P(p, d):
        return {m: k for m, k in p.items() if deg(m) == d}
    for d in [(1, 1), (3, 1), (2, 0), (2, 1)]:
        lhs = J.euler(P(F, d), "h", (0, 1))
        rhs = P(J.euler(F, "h", (0, 1)), (d[0] - 1, d[1]))
        assert lhs == rhs
    src = (ROOT / "code" / "classical" / "master_kappa2.py").read_text()
    assert "E_u P_d = P_{d - e_u} E_u" in src and "Euler operators commute with projection" not in src


def test_master_no_unconditional_pass_and_deterministic():
    m = cj("P13_master_kappa2.json")
    assert "pass" not in m
```

```
    assert m["execution_pass"] is True and m["claim_status"] == "verified" and m["scope"]
    assert m["required_master_residuals"] == []
    assert "DETERMINISTIC" in m["evidence_kind"]["grav_af0"]
    assert "REGRESSION CONTROL" in m["evidence_kind"]["grav_af0_planewave"]


def test_gauge_hessian_generic_and_full_canonicity():
    g = cj("P13_gauge_fixing_map.json")
    gr = g["propagators"]["graviton"]
    assert "generic" in gr["momentum"] and gr["schur_equals_k2P"] and gr["propagator_hh_equals_inv_k2P"]
    assert g["canonicity"]["pass"] and g["canonicity"]["pairs_tested"] == 299
    assert "NOT an independently established quantum" in g["source_identity_relation"]["status"]


def test_enumeration_lemma():
    ch = cj("P13_ch_source_audit.json")
    mc = ch["enumeration_lemma"]["mechanical_checks"]
    assert ch["enumeration_lemma"]["holds"] and mc["min_interaction_vertex_order"] >= 1 and mc["max_len_exhaus
  +> tive"]
    assert mc["b_only_in_kappa0_bilinears"] and mc["kappa0_monomials_all_two_field"]


# p15: the top-level RELEASE_STATUS.json is the p15 status; the p14r CONTRACT record is certificates/P14_relea
  +> se.json
CONTRACT_RS = ROOT / "certificates" / "P14_release.json"


def test_release_accepted_and_hard_stops():
    r = json.loads(CONTRACT_RS.read_text())
    assert r["revision"] == PC.REVISION == "p14r"
    assert r["execution_pass"] and r["classical"]["classical_readiness_for_ch_identity"] and r["release_claim_
  +> accepted"]
    assert r["publication"] == "accepted" and (CERT / "P14_closure_release.json").exists()
    assert not (CERT / "P14_release_rejected.json").exists()
    assert not (CERT / "P13_release.json").exists() and not (CERT / "P13_closure_release.json").exists()
    assert "pass" not in r
    hs = r["hard_stops"]
    assert hs["quantum_inputs_evaluated"] == "not_earned" and hs["D02_quantum_evaluated"] is False
    assert hs["D12_quantum_evaluated"] is False and hs["D12_status"] == "Missing" and hs["symmetry_restored"]
  +> is False
    assert hs["O1_closure"] == "NOT_CLOSED" and hs["XV_B_opened"] is False and hs["Route_B_discharged"] is Fal
  +> se
    assert hs["loop_evaluations_performed"] is False and hs["u_R_c_delta_added"] is False
    assert r["insertion_status"].startswith("Frozen")


def test_xvi_m_pin():
    import inherited
    assert inherited.sha(inherited.resolve("XVI-M.tex")) == C.XVI_M_PIN == inherited.PINS["XVI-M.tex"]["sha256
  +> "]
    assert all(v["ok"] and v["vendored_copy_matches_pin"] for v in inherited.verify_all().values())


# ---------------------------------------------------------------- end-to-end mutations
STAGES = {"gauge": "code/classical/gauge_fixing_map.py", "master": "code/classical/master_kappa2.py",
          "ch": "code/p4_classical/ch_source_audit.py", "release": "code/p5_entry/release_p14.py"}


def _frozen_input_tops():
    """top-level directories holding the frozen P.3/P.4 inputs (the freeze list) -- the isolated copies must c
  +> ontain
    all of them, otherwise every copy is 'unfrozen' and P.5 mutations would be rejected for the wrong reason."
  +> ""
    L = json.loads((ROOT / PC.P13_FROZEN_LIST).read_text())
    return sorted({f["path"].split("/")[0] for f in L["files"]})


COPY_SUBS = tuple(sorted(set(("code", "certificates", "exports", "ledgers", "tools", "provenance", "reproducib
  +> ility",
                             "archival", "notes", "tests")) | set(_frozen_input_tops())))


def _copy_pkg():
    d = Path(tempfile.mkdtemp(prefix="p14e2e_"))
    for sub in COPY_SUBS:
        if (ROOT / sub).exists():
            shutil.copytree(ROOT / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__"))
    shutil.copy(ROOT / "RELEASE_STATUS.json", d / "RELEASE_STATUS.json")
    return d
```

```
def _run(d, stage, env_mut=""):
    env = dict(os.environ, P13_MUTATIONS=env_mut, P14_MUTATIONS="")
    p = subprocess.run([sys.executable, str(d / STAGES[stage])], cwd=d, env=env, capture_output=True, text=Tru
  +> e,
                       timeout=3600)
    return p.returncode


def _corrupt_export(d):
    f = d / "exports" / "ell4_generating_polynomial.srepr"
    e = sp.sympify(f.read_text().strip())
    t = sp.Add.make_args(e)[0]
    f.write_text(sp.srepr(sp.expand(e + sp.Rational(2, 5) * t)) + "\n")


def _stale_hash(d):
    f = d / "certificates" / "P13_quartic_verified.json"
    q = json.loads(f.read_text())
    q["ell4_sha256"] = hashlib.sha256(b"stale-p10-ell4").hexdigest()
    f.write_text(json.dumps(q, indent=2) + "\n")


CASES = {
    "flipped_odd_sign": {"env": "flip_odd_derivative_sign", "stages": ["master"], "flag": "required_classical_
  +> BV_components_verified"},
    "corrupted_ell4_coefficient": {"pre": _corrupt_export, "stages": ["master"], "flag": "required_classical_B
  +> V_components_verified"},
    "dropped_hstar_c_row": {"env": "drop_hstar_c_row", "stages": ["ch"], "flag": "ch_dependency_inventory_comp
  +> lete"},
    "altered_gauge_shift": {"env": "alter_gf_shift", "stages": ["gauge", "master"], "flag": "gauge_fixing_map_
  +> constructed"},
    "omitted_cstar_term": {"env": "drop_cstar", "stages": ["master"], "flag": "required_classical_BV_component
  +> s_verified"},
    "stale_hash": {"pre": _stale_hash, "stages": ["master"], "flag": "required_classical_BV_components_verifie
  +> d"},
}


def _chain(name):
    case = CASES[name]
    d = _copy_pkg()
    assert (d / "certificates" / "P14_closure_release.json").exists()   # stale closure artifact present befor
  +> e
    if "pre" in case:
        case["pre"](d)
    codes = {s: _run(d, s, case.get("env", "")) for s in case["stages"]}
    codes["release"] = _run(d, "release")
    out = {"dir": str(d), "codes": codes,
           "master": cj("P13_master_kappa2.json", d), "gauge": cj("P13_gauge_fixing_map.json", d),
           "ch": cj("P13_ch_source_audit.json", d),
           "status": json.loads((d / "RELEASE_STATUS.json").read_text()),
           "closure_exists": (d / "certificates" / "P14_closure_release.json").exists(),
           "release_exists": (d / "certificates" / "P14_release.json").exists(),
           "rejected": json.loads((d / "certificates" / "P14_release_rejected.json").read_text())
           if (d / "certificates" / "P14_release_rejected.json").exists() else None}
    return name, out


@pytest.fixture(scope="module")
def e2e():
    with ThreadPoolExecutor(max_workers=6) as ex:
        res = dict(ex.map(_chain, list(CASES)))
    summary = {k: {"stage_codes": v["codes"], "flag": CASES[k]["flag"],
                   "flag_value": v["status"]["classical"]["flags"][CASES[k]["flag"]],
                   "readiness": v["status"]["classical"]["classical_readiness_for_ch_identity"],
                   "P3_P4_frozen_as_P5_inputs": v["status"]["P3_P4_frozen_as_P5_inputs"],
                   "P5_entry_earned": v["status"]["P5_entry_earned"],
                   "release_claim_accepted": v["status"]["release_claim_accepted"],
                   "publication": v["status"]["publication"], "closure_artifact_exists": v["closure_exists"],
                   "master_residuals": v["master"]["required_master_residuals"][:6],
                   "failed_prerequisites": v["status"]["classical"]["failed_prerequisites"]} for k, v in res.i
  +> tems()}
    outd = ROOT / "build" / "p14" / "test_results"
    outd.mkdir(parents=True, exist_ok=True)
    (outd / "e2e_mutations.json").write_text(json.dumps(summary, indent=2, default=str) + "\n")
    yield res
    for v in res.values():
        shutil.rmtree(v["dir"], ignore_errors=True)


def _assert_rejected(r, flag):
```

```
    assert r["codes"]["release"] != 0
    st = r["status"]
    assert st["classical"]["flags"][flag] is False
    assert st["classical"]["classical_readiness_for_ch_identity"] is False
    assert st["P3_P4_frozen_as_P5_inputs"] is False and st["P5_entry_earned"] is False   # frozen values chang
  +> ed
    assert st["release_claim_accepted"] is False and st["publication"] == "REJECTED"
    assert not r["closure_exists"] and not r["release_exists"] and r["rejected"] is not None


def test_e2e_flipped_odd_sign(e2e):
    r = e2e["flipped_odd_sign"]
    assert r["master"]["required_master_residuals"]
    _assert_rejected(r, "required_classical_BV_components_verified")


def test_e2e_corrupted_ell4_coefficient(e2e):
    r = e2e["corrupted_ell4_coefficient"]
    res = r["master"]["required_master_residuals"]
    assert any(x.startswith("grav_noether_kappa2") for x in res) and any("sha256_mismatch" in x for x in res)
    assert not any(x.startswith("grav_noether_kappa0") or x.startswith("grav_noether_kappa1") for x in res)
    assert "hash_consistency" in r["status"]["classical"]["failed_prerequisites"]
    _assert_rejected(r, "required_classical_BV_components_verified")


def test_e2e_dropped_hstar_c_row(e2e):
    r = e2e["dropped_hstar_c_row"]
    assert r["ch"]["checks"]["hstar_c_row_present"] is False and r["ch"]["checks"]["regenerated_rows_match_pub
  +> lished"] is False
    _assert_rejected(r, "ch_dependency_inventory_complete")


def test_e2e_altered_gauge_shift(e2e):
    r = e2e["altered_gauge_shift"]
    assert r["gauge"]["gauge_fixing_map_constructed"] is False and r["gauge"]["canonicity"]["antifield_antifie
  +> ld_fails"]
    assert r["master"]["required_master_residuals"]
    _assert_rejected(r, "gauge_fixing_map_constructed")
    assert r["status"]["classical"]["flags"]["required_classical_BV_components_verified"] is False


def test_e2e_omitted_cstar_term(e2e):
    r = e2e["omitted_cstar_term"]
    assert r["master"]["required_master_residuals"]
    _assert_rejected(r, "required_classical_BV_components_verified")


def test_stale_ell4_hash_blocks_master(e2e):
    """Actually mutates the recorded ell4 hash and asserts master AND release block."""
    r = e2e["stale_hash"]
    res = r["master"]["required_master_residuals"]
    assert res == ["ell4_sha256_mismatch_vs_P13_quartic_verified"]
    assert r["master"]["claim_status"] == "Partial"
    assert r["master"]["identification_with_exports"]["ell2_ell3_ell4_sha_match_P13_quartic_verified"] is Fals
  +> e
    assert "hash_consistency" in r["status"]["classical"]["failed_prerequisites"]
    _assert_rejected(r, "required_classical_BV_components_verified")


def test_matter_loop_controls():
    """Inherited matter-loop controls (route-A/B source comparison + wrong-prescription mutations), run as a s
  +> cript."""
    r = subprocess.run([sys.executable, str(ROOT / "tests" / "test_matter_loop_gates.py")], cwd=str(ROOT),
                       capture_output=True, text=True, timeout=3600)
    outd = ROOT / "build" / "p14" / "test_results"
    outd.mkdir(parents=True, exist_ok=True)
    (outd / "matter_loop_controls.json").write_text(json.dumps(
        {"rc": r.returncode, "tail": r.stdout.strip().splitlines()[-2:]}, indent=1) + "\n")
    assert r.returncode == 0 and "ALL test_matter_loop_gates: PASS" in r.stdout


# ---------------------------------------------------------------- p12: residual-record invariants
def test_certio_rejects_inconsistent_record_before_serialization():
    good = {"kappa0": {"Q_terms": [0, 0, 0, 0], "Q_identically_zero": True}}
    certio.dumps(good)
    for counts, flag in (([0, 0, 120, 0], True), ([290, 0, 0, 0], True), ([0, 0, 0, 0], False)):
        bad = {"kappa1": {"Q_terms": counts, "Q_identically_zero": flag}}
        with pytest.raises(certio.CertificateInvariantError):
            certio.dumps(bad)
    assert "\n" not in certio.dumps({"Q_terms": [0, 0, 0, 0], "Q_identically_zero": True}).split("Q_terms")[1]
  +> .split("]")[0]
```

```
def test_residual_records_regenerated_and_agree():
    q = cj("P13_quartic_verified.json")
    m = cj("P13_master_kappa2.json")
    qc7, mg = q["controls"]["C7_noether_exact"], m["details"]["grav_af0_deterministic"]
    for o in certio.ORDERS:
        assert qc7[o]["Q_terms"] == [0, 0, 0, 0] and qc7[o]["Q_identically_zero"] is True
        assert mg[o]["Q_terms"] == [0, 0, 0, 0] and mg[o]["Q_identically_zero"] is True
    cross = certio.cross_check(qc7, q["controls"]["C7_parameters"], mg, m["details"]["grav_af0_parameters"])
    assert cross["same_inputs"] and cross["agree"], cross
    for f in sorted(list((ROOT / "certificates").glob("P13_*.json")) + list((ROOT / "certificates").glob("P14_
  +> *.json"))):
        assert not certio.invariant_violations(json.loads(f.read_text())), f.name
    rs = json.loads(CONTRACT_RS.read_text())
    assert rs["classical"]["deterministic_prerequisites"]["cross_certificate_quartic_vs_master_agree"] is True
    assert rs["classical"]["deterministic_prerequisites"]["residual_record_invariants_all_certificates"] is Tr
  +> ue


def _mut_master_count(d):
    f = d / "certificates" / "P13_master_kappa2.json"
    c = json.loads(f.read_text())
    c["details"]["grav_af0_deterministic"]["kappa1"]["Q_terms"] = [0, 0, 1, 0]      # Boolean left true
    f.write_text(json.dumps(c, indent=2))


def _mut_quartic_count(d):
    f = d / "certificates" / "P13_quartic_verified.json"
    c = json.loads(f.read_text())
    c["controls"]["C7_noether_exact"]["kappa0"]["Q_terms"] = [0, 0, 120, 0]     # the pattern reported for p11
    f.write_text(json.dumps(c, indent=2))


def _mut_cross(d):
    f = d / "certificates" / "P13_master_kappa2.json"
    c = json.loads(f.read_text())
    c["details"]["grav_af0_deterministic"]["kappa2"]["W_terms"] += 1             # internally consistent, disa
  +> grees
    f.write_text(json.dumps(c, indent=2))


CERT_MUTS = {"master_count_flag_true": (_mut_master_count, "residual_record_invariants_all_certificates"),
             "quartic_count_flag_true": (_mut_quartic_count, "residual_record_invariants_all_certificates"),
             "cross_certificate_mismatch": (_mut_cross, "cross_certificate_quartic_vs_master_agree")}


def _cert_chain(name):
    d = _copy_pkg()
    CERT_MUTS[name][0](d)
    rc = _run(d, "release")
    out = {"rc": rc, "status": json.loads((d / "RELEASE_STATUS.json").read_text()),
           "closure_exists": (d / "certificates" / "P14_closure_release.json").exists(),
           "release_exists": (d / "certificates" / "P14_release.json").exists(),
           "rejected_exists": (d / "certificates" / "P14_release_rejected.json").exists()}
    shutil.rmtree(d, ignore_errors=True)
    return name, out


@pytest.fixture(scope="module")
def certmut():
    with ThreadPoolExecutor(max_workers=3) as ex:
        res = dict(ex.map(_cert_chain, list(CERT_MUTS)))
    outd = ROOT / "build" / "p14" / "test_results"
    outd.mkdir(parents=True, exist_ok=True)
    (outd / "certificate_mutations.json").write_text(json.dumps(
        {k: {"rc": v["rc"], "publication": v["status"].get("publication"),
             "release_claim_accepted": v["status"].get("release_claim_accepted"),
             "failed_prerequisites": v["status"].get("classical", {}).get("failed_prerequisites"),
             "closure_artifact_exists": v["closure_exists"]} for k, v in res.items()}, indent=1) + "\n")
    return res


@pytest.mark.parametrize("name", list(CERT_MUTS))
def test_certificate_mutation_rejects_publication(certmut, name):
    r = certmut[name]
    st = r["status"]
    assert r["rc"] != 0 and st["publication"] == "REJECTED" and st["release_claim_accepted"] is False
    assert st["classical"]["classical_readiness_for_ch_identity"] is False
    assert CERT_MUTS[name][1] in st["classical"]["failed_prerequisites"]
    assert not r["closure_exists"] and not r["release_exists"] and r["rejected_exists"]
```

```
# ------------------------------------------------------------------ milestones, dossier, reproducibility contra
  +> ct (p14)
def test_milestones_consistent():
    """One set of milestone definitions: conventions == contract record == ledger == dossier; p14r contract st
  +> atus."""
    rs = json.loads(CONTRACT_RS.read_text())
    led = json.loads((ROOT / "ledgers" / "P14_milestones.json").read_text())
    dos = cj("P14_dossier.json")
    assert rs["milestones"]["definitions"] == led["definitions"] == dos["milestone_definitions"] == C.MILESTON
  +> ES
    assert list(C.MILESTONES) == list(C.MILESTONE_ORDER) == ["P.3", "P.4", "P.5_entry", "P.5_calculation", "P.
  +> 5_identity"]
    st = rs["milestones"]["status"]
    assert st["P.3"].startswith("complete") and st["P.4"].startswith("earned (classical readiness)")
    assert st["P.5_entry"].startswith("earned (contract fixed")
    assert st["P.5_calculation"].startswith("open") and st["P.5_identity"] == "open"
    assert "NOT coupled one-loop restoration" in C.MILESTONES["P.4"]
    hs = rs["hard_stops"]
    assert hs["P5_entry_opened"] is True and hs["P5_calculation_opened"] is False and hs["P5_identity_opened"]
  +>  is False
    assert hs["quantum_inputs_evaluated"] == "not_earned"
    assert rs["milestones"]["P3_P4_frozen_as_P5_inputs"] is rs["P3_P4_frozen_as_P5_inputs"] is True
    assert rs["milestones"]["P5_entry_earned"] is (all(rs["P5_entry_flags"].values()) and rs["P3_P4_frozen_as_
  +> P5_inputs"])
    note = (ROOT / "manuscript" / "notes" / "P14_P5_entry_contract.tex").read_text()
    for k in ("P.3", "P.4", "P.5 entry", "P.5 calculation", "P.5 identity", "same box", "not on a second machi
  +> ne"):
        assert k in note, k
    for f in [ROOT / "manuscript" / "notes" / "P14_P5_entry_contract.tex", ROOT / "README.md"]:
        t = f.read_text().lower()
        assert "p4 coupled one-loop restoration" not in t and "p-next" not in t, f
    assert not list((ROOT / "notes").glob("P*.tex"))
    assert (ROOT / "archival" / "p02_p09" / "notes" / "P_scope.tex").exists()


def test_dossier_links_resolve():
    d = cj("P14_dossier.json")
    assert d["all_links_resolved"] is True and d["n_rows"] >= 40 and d["n_slots"] == 3
    chains = [c["chain"] for c in d["chains"]]
    fz = " [frozen P.3/P.4 input]"
    assert chains == ["projection -> dependency rows -> graphs" + fz, "classical coefficient -> export -> veri
  +> fication" + fz,
                      "transformations -> source terms" + fz, "gauge map -> Hessians -> source convention" + f
  +> z,
                      "master component -> residual record -> acceptance gate" + fz,
                      "quantum object -> prescription/D11 matching -> counterterm domain -> amplitude rules ->
  +>  gate"]
    tests_here = {n for n in globals() if n.startswith("test_")}
    for c in d["chains"]:
        for r in c["rows"]:
            assert r["resolved"] and all(r["functions_found"].values()), r["id"]
            assert set(r["tests"]) <= tests_here, r["id"]
    slots = {s["slot"]: s for s in d["quantum_input_slots"]}
    assert list(slots) == ["hh at kappa^2", "K'c at kappa^2", "h one-point at kappa"]
    assert slots["hh at kappa^2"]["graphs"] == ["D01", "D02", "D03", "D11", "D12"]
    assert slots["K'c at kappa^2"]["graphs"] == ["D13"] and slots["h one-point at kappa"]["graphs"] == ["D14a"
  +> , "D14b", "D14c"]
    assert all("NOT an evaluated coefficient" in s["status"] and s["resolved"] for s in slots.values())
    assert "not a series certificate" in d["status_of_this_document"]
    assert d["p13_independent_rebuild"]["independent_rebuild_passed"] is True
    tex = (ROOT / "manuscript" / "body" / "P14-dossier-table.tex").read_text()
    assert all(r["id"] in tex for c in d["chains"] for r in c["rows"])
    assert "SAME box" in tex and "second machine" in tex


def test_dossier_detects_broken_link():
    """Mutation: a wrong expected value / missing function / missing certificate key makes the row unresolved.
  +> """
    import dossier as D
    D.T = "tests/test_p14.py"            # the frozen p13 dossier library, pointed at the active test file (as
  +>  dossier_p14)
    cache = {}
    row = D.R("X", "x", D.F_MS, ["run"], [(D.MS, "/required_master_residuals", ["something"])], ["test_enumera
  +> tion_lemma"])
    assert not D.resolve_row(row, cache)["resolved"]
    row = D.R("X", "x", D.F_MS, ["no_such_function"], [(D.MS, "/required_master_residuals", [])], ["test_enume
  +> ration_lemma"])
    assert not D.resolve_row(row, cache)["resolved"]
    row = D.R("X", "x", D.F_MS, ["run"], [(D.MS, "/no/such/key", True)], ["test_enumeration_lemma"])
```

```
    assert not D.resolve_row(row, cache)["resolved"]
    row = D.R("X", "x", D.F_MS, ["run"], [(D.MS, "/required_master_residuals", [])], ["test_no_such_test"])
    assert not D.resolve_row(row, cache)["resolved"]
    assert D.matches(1, 1) and not D.matches(True, 1) and not D.matches([0, 0, 1, 0], [0, 0, 0, 0])


def test_brst_and_source_terms_derived():
    br, sa = cj("P13_brst_derivation.json"), cj("P13_source_action.json")
    assert br["ghost_law_derivation"]["lam_solutions"] == ["1"] and br["nilpotency_exact_all_orders"] is True
    assert set(br["s2_per_sector"].values()) == {"derived_zero"}
    assert sa["sigma_unique"] is True and sa["X_kappa2_terms"] == 0 and sa["antifield_linear"] is True


def test_ch_rows_and_quantum_input_slots():
    ch = cj("P13_ch_source_audit.json")
    gi = json.loads((ROOT / "ledgers" / "P13_graph_inventory.json").read_text())
    assert ch["ch_projection_defined"] and ch["ch_pattern"]["ghost_number"] == 1
    assert (ch["rows_total"], ch["rows_surviving"], ch["rows_derived_zero"]) == (6, 3, 3)
    assert gi["names_TBD"] == [] and ch["checks"]["names_TBD_empty"] and ch["vertex_inventory"]
    rows = {r["id"]: r for r in ch["surviving_rows"]}
    for s in C.QUANTUM_INPUT_SLOTS:
        r = rows[s["row"]]
        assert [g.split("_")[0] for g in r["graphs"]] == s["graphs"] and r["status"] == "surviving_unevaluated
  +> "
    assert gi["graphs"]["D12_hh_scalar_tadpole"]["status"].startswith("Missing")
    assert "not re-evaluated" in gi["graphs"]["D11_hh_scalar_loop"]["status"]


def test_source_only_contract():
    sys.path.insert(0, str(ROOT / "tools"))
    import source_manifest as SM
    src = SM.source_files(ROOT)
    gen_prefixes = ("certificates/P13_", "certificates/P14_", "certificates/P15_", "certificates/O5_", "ledger
  +> s/P13_",
                    "ledgers/P14_", "ledgers/P15_", "manuscript/body/P15-",
                    "certificates/P16_", "ledgers/P16_", "manuscript/body/P16-", "manuscript/version.tex",   #
  +>  p16
                    "certificates/P17_", "ledgers/P17_", "manuscript/body/P17-",                            # p1
  +> 7
                    "certificates/P18_", "ledgers/P18_", "manuscript/body/P18-",                            # p1
  +> 8
                    "certificates/P19_", "ledgers/P19_", "manuscript/body/P19-",                            # p1
  +> 9
                    "certificates/P20_", "ledgers/P20_", "manuscript/body/P20-",                            # p2
  +> 0
                    "certificates/P21_", "ledgers/P21_", "manuscript/body/P21-",                            # p2
  +> 1
                    "certificates/P22_", "ledgers/P22_", "manuscript/body/P22-",                            # p2
  +> 2
                    "certificates/P23_", "ledgers/P23_", "manuscript/body/P23-",                            # p2
  +> 3
                    "exports/P14_", "build/", "dist/", "stale/", "manuscript/listing_src/")
    assert not [f for f in src if f.startswith(gen_prefixes) or f.endswith((".pdf", ".zip", ".pyc", ".pkl"))]
    assert not [f for f in src if f in ("RELEASE_STATUS.json", "RELEASE_MANIFEST_EXTERNAL.txt",
                                        "exports/ell4_generating_polynomial.srepr", "exports/V4_grav.json",
                                        "manuscript/body/XVI-P-appendix-body.tex", "manuscript/body/P13-dossie
  +> r-table.tex",
                                        "manuscript/body/P14-dossier-table.tex")]
    # the reviewer's live p13 report and the p13 reference values are NOT p14 source (archived copies are)
    assert "reproducibility/independent_rebuild_report.json" not in src
    assert "reproducibility/reference_values.json" not in src and SM.REFERENCE_FILE == "reproducibility/refere
  +> nce_values_p23.json"
    # p15: the live p14r records are NOT p15 source; their archived copies are
    assert not [f for f in src if f.startswith("reproducibility/") and "_p14r" in f]
    assert [f for f in src if f.startswith("archival/p14r/")] == sorted([
        "archival/p14r/README.md", "archival/p14r/reference_values_p14r.json",
        "archival/p14r/reproducibility/author_rebuild_report_p14r.json",
        "archival/p14r/reproducibility/independent_rebuild_report_p14r.json"])
    assert not [f for f in src if f.startswith("archival/p13/") and f not in (
        "archival/p13/README.md", PC.P13_REFERENCE_FILE, PC.P13_FROZEN_LIST, PC.P13_INDEPENDENT_REPORT,
        "archival/p13/reproducibility/author_rebuild_report_p13.json",
        "archival/p13/reproducibility/inherited_data_pins_p13.json")]
    for need in ("tools/rebuild_from_source.py", "REBUILD_INSTRUCTIONS.md", SM.REFERENCE_FILE, SM.PINS_FILE,
                 "provenance/XVI-M.tex", "provenance/inherited/XVI-N-r10/code/prep/grav_density.py",
                 PC.P13_REFERENCE_FILE, PC.P13_FROZEN_LIST, PC.P13_INDEPENDENT_REPORT, "code/p5_entry/release_
  +> p14.py"):
        assert need in src, need
    assert SM.ZIP_NAME == "XVI-P-p23-source-only.zip"
    # the archived p14 release is provenance only: just its README is source
    assert [f for f in src if f.startswith("archival/p14/")] == ["archival/p14/README.md"]
    assert not [f for f in src if f.startswith("reproducibility/") and "_p14." in f]
```

```
    assert all(v["ok"] for v in SM.check_inherited_data(ROOT).values())
    assert all(v["ok"] for v in SM.check_xvio_provenance(ROOT).values())
    d = Path(tempfile.mkdtemp(prefix="p14src_"))
    try:
        a, b = SM.make_zip(d / "a.zip", ROOT), SM.make_zip(d / "b.zip", ROOT)
        assert a["sha256"] == b["sha256"] and a["files"] == len(src)          # deterministic
        assert SM.zip_matches_tree(d / "a.zip", ROOT) == []
    finally:
        shutil.rmtree(d, ignore_errors=True)
    rs = json.loads((ROOT / "RELEASE_STATUS.json").read_text())      # p23 status: flags for the CURRENT tree
    assert rs["revision"] == "p23" and rs["independent_rebuild_passed"] == "pending"
    assert isinstance(rs["author_rebuild_passed"], bool)
    if rs["author_rebuild_passed"]:
        rep = json.loads((ROOT / "reproducibility" / "author_rebuild_report_p23.json").read_text())
        assert rep["pass"] is True and rep["source_tree_sha256"] == SM.tree_sha(ROOT)


# ---------------------------------------------------------------- p12: gate enforcement for resumed builds
def _copy_full_pkg():
    d = Path(tempfile.mkdtemp(prefix="p14resume_"))
    for sub in ("code", "certificates", "exports", "ledgers", "tools", "manuscript", "build", "provenance",
                "reproducibility", "archival", "notes", "tests"):
        if (ROOT / sub).exists():
            shutil.copytree(ROOT / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__", "test_results"))
    for f in ("README.md", "RELEASE_STATUS.json", "REBUILD_INSTRUCTIONS.md"):
        if (ROOT / f).exists():
            shutil.copy(ROOT / f, d / f)
    (d / "dist").mkdir(exist_ok=True)
    for st in ("XVI-P-science", "XVI-P-appendix", "XVI-P"):           # pre-existing artifacts that must not s
  +> urvive
        (d / "manuscript" / f"{st}.pdf").write_bytes(b"%PDF-1.5 placeholder (pre-existing artifact)\n")
    for z in ("XVI-P-review-focus.zip", "XVI-P-full.zip", "XVI-P-p23-source-only.zip"):
        (d / "dist" / z).write_bytes(b"PK placeholder (pre-existing artifact)")
    (d / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("pre-existing manifest\n")
    return d


RESUME = {"pdf": _mut_master_count, "zip": _stale_hash, "manifest": _mut_quartic_count, "manifest_unverified_p
  +> dfs": None}


def _resume_chain(name):
    d = _copy_full_pkg()
    sync = Path(tempfile.mkdtemp(prefix="p14sync_"))
    if RESUME[name] is not None:
        RESUME[name](d)
    stage = name.split("_")[0]
    env = dict(os.environ, P14_SYNC_DIR=str(sync), P13_MUTATIONS="", P14_MUTATIONS="")
    p = subprocess.run([sys.executable, str(d / "tools" / "build.py"), "--from-stage", stage, "--skip-tests"],
                       cwd=d, env=env, capture_output=True, text=True, timeout=1800)
    out = {"rc": p.returncode, "stage": stage,
           "pdfs_left": [x.name for x in (d / "manuscript").glob("*.pdf")],
           "zips_left": [x.name for x in (d / "dist").glob("*.zip")],
           "manifest_left": (d / "RELEASE_MANIFEST_EXTERNAL.txt").exists(),
           "stale": sorted(x.name for x in (d / "stale").glob("*")) if (d / "stale").exists() else [],
           "marker": (d / "PUBLICATION_REJECTED.txt").exists(),
           "synced": sorted(x.name for x in sync.iterdir()),
           "reasons": json.loads((d / "stale" / "REJECTED.json").read_text())["reasons"][:6]
           if (d / "stale" / "REJECTED.json").exists() else None}
    shutil.rmtree(d, ignore_errors=True)
    shutil.rmtree(sync, ignore_errors=True)
    return name, out


@pytest.fixture(scope="module")
def resumed():
    with ThreadPoolExecutor(max_workers=4) as ex:
        res = dict(ex.map(_resume_chain, list(RESUME)))
    outd = ROOT / "build" / "p14" / "test_results"
    outd.mkdir(parents=True, exist_ok=True)
    (outd / "resumed_stage_gate.json").write_text(json.dumps(res, indent=1, default=str) + "\n")
    return res


@pytest.mark.parametrize("name", list(RESUME))
def test_resumed_stage_build_blocked(resumed, name):
    r = resumed[name]
    assert r["rc"] != 0, r
    assert r["marker"] and "REJECTED.json" in r["stale"]
    assert not r["pdfs_left"] and not r["zips_left"] and not r["manifest_left"]
    assert any(x.startswith("REJECTED__manuscript__") for x in r["stale"])
```

```
    assert any(x.startswith("REJECTED__dist__") for x in r["stale"])
    assert r["synced"] == []


# ============================================================ p14: P.5 ENTRY
def _rs():
    return json.loads(CONTRACT_RS.read_text())


def test_quantum_object():
    qo = cj("P14_quantum_object.json")
    assert qo["quantum_action_defined"] is True and qo["revision"] == PC.REVISION
    assert all(qo[k]["ok"] for k in qo if k.startswith("Q"))
    q1 = qo["Q1_transformed_variables"]
    assert q1["eps_derived"] == -1 and q1["NOT_S_D"] and q1["no_switch_or_insertion_fields"]
    assert q1["X_patterns_by_kappa"]["2"] == [] and "h.b" in q1["X_patterns_by_kappa"]["0"]
    q2 = qo["Q2_fields_antifields_b"]
    assert q2["b_decision"] == "RETAINED and integrated" and q2["Gamma1_independent_of"] == ["bs", "cbs"]
    bj = q2["b_justification"]
    assert bj["b_occurs_only_at_kappa_orders"] == [0] and bj["no_b_interaction_vertex"] and bj["Mbb_constant_i
  +> nvertible"]
    assert bj["elimination"]["cbar_star_quadratic_terms"] > 0 and bj["elimination"]["b_hessian_det"] == "-16"
    assert "one_point" in qo["Q3_1PI_vacuum_onepoint"]["one_point_normalization"] or \
        "RETAINED" in qo["Q3_1PI_vacuum_onepoint"]["one_point_normalization"]
    assert "no zero is substituted" in qo["Q3_1PI_vacuum_onepoint"]["one_point_normalization"]
    g = qo["Q5_kappa_loop_bookkeeping"]["graphs"]
    assert all(v["ok"] and v["sum_n_minus_2"] == v["external_legs"] == v["integrand_kappa_power"] for v in g.v
  +> alues())
    assert qo["Q6_pullback_vs_theorem"]["quantum_gauge_change_theorem_asserted"] is False
    assert qo["loop_evaluations_performed"] is False
    # independent recomputation of the b elimination (direct, not from the certificate)
    import feynman as F, quantum_object as QO
    e = QO.b_elimination(F.S_psi_parts()["X"][0])
    assert e["b_hessian_det"] == "-16" and e["cbar_star_quadratic_terms"] == bj["elimination"]["cbar_star_quad
  +> ratic_terms"]


def test_prescription_and_D11_matching():
    pd = cj("P14_prescription_D11_matching.json")
    assert pd["quantum_prescription_defined"] is True and pd["prescription"]["id"] == "P14_DRED_MSbar_FeynmanI
  +> 0"
    assert all(pd["prescription_checks"].values())
    assert "NOT APPLIED" in pd["prescription"]["scaleless_integrals"] and "Missing" in pd["prescription"]["sca
  +> leless_integrals"]
    assert pd["D11_matching_defined"] is True and pd["D11_matching_verified"] is False
    m = pd["matching"]
    assert m["baseline_M0"]["pass"] and "NOT S_D" in m["baseline_M0"]["baseline"]
    m1 = m["M1_integrand_identity"]
    assert m1["pass"] and m1["mismatches"] == [] and m1["component_pairs_checked"] == 100 and m1["bound_to_exp
  +> ort"]
    assert m["M2_nonlocal_agreement_on_U"]["status"] == "defined_not_verified"
    m3 = m["M3_local_difference"]
    assert m3["dimension"] == 5 and m3["transverse_subspace_dimension"] == 2 and m3["matches_counterterm_hh_se
  +> ctor"]
    lab = m3["labels"]
    assert (lab["scalar_extension_scaling_degree"], lab["kernel_scaling_degree_bound"],
            lab["degree_of_divergence_bound"], lab["local_contact_derivative_order_bound"]) == (4, 8, 4, 4)
    assert lab["contact_scaling_degree_rule"].startswith("sd(d^alpha delta) = 4 + |alpha|")
    assert m3["wording"] == ("Kernel scaling degree at most eight; degree of divergence at most four; hence th
  +> e local "
                             "ambiguity has derivative order at most four.")
    assert m3["conditional_on"].startswith("M2") and m3["conditional_statement"].startswith("IF M2 holds")
    assert "NOT implied by 'no other scale'" in m3["normalization_condition"]
    assert m3["lower_degree_contacts_excluded_by_N4"] == {"order_0_mu4": 2, "order_2_mu2": 4}
    assert {k: (v["dimension"], v["transverse_subspace_dimension"]) for k, v in m3["bounded_contact_space"].it
  +> ems()} == \
        {"order_0": (2, 0), "order_2": (4, 1), "order_4": (5, 2)}
    assert "none arise" not in pd["prescription"]["tensor_reduction"] and "withdrawn" in pd["prescription"]["e
  +> vanescent"]
    assert pd["prescription_checks"]["operational_DRED_rules_verified"] is True
    assert m["M4_finite_adjustments"]["u_R_c_delta_added_by_P"] is False
    assert m["M4_finite_adjustments"]["u_R_extra_c_delta_XVI_0"] is False
    assert m["M4_finite_adjustments"]["scalar_extension_prescription_preserved"] is True
    assert m["M5_residue"]["r"] == "-i * pi**2" and m["M5_residue"]["holomorphy_proof_pass"]
    assert "AFTER constructive holomorphy" in m["M5_residue"]["statement"]
    for lem in ("lem:d11-prefactor", "lem:analytic_FP_u_mu_R_o70", "lem:BV_rules_on_U"):
        assert m["lemmas_referenced"][lem]["pass"] is True
    assert m["M6_provenance"]["all_pinned_ok"] and pd["D12_status"] == "Missing" and pd["D12_value_imported"]
  +> is False
    # direct recomputation: exact M1 identity and the M3 space
    import prescription_d11 as PDm
```

```
    r1 = PDm.M1_identity()
    assert r1["pass"] and r1["recomputed_integrand_sha256"] == m1["recomputed_integrand_sha256"]
    r3 = PDm.M3_local_space()
    assert (r3["dimension"], r3["transverse_subspace_dimension"]) == (5, 2)
    r2 = PDm.M3_local_space(2)
    assert (r2["dimension"], r2["transverse_subspace_dimension"]) == (4, 1)


def test_counterterm_domain():
    ct = cj("P14_counterterm_domain.json")
    assert ct["counterterm_domain_defined"] is True and "NOT an admission" in ct["kind"]
    assert ct["XIV_J_opened"] is False and ct["XV_B_opened"] is False
    assert ct["kappa1"]["basis_size"] == 0
    assert ct["kappa2"]["basis_size"] == 11
    assert ct["kappa2"]["basis_size_by_sector"] == {"h.h": 5, "c.cb": 2, "c.K": 3, "phi.phi": 1}
    assert set(ct["excluded_species"]) == {"b", "cbs", "bs"}
    eq = ct["equivalences"]
    assert eq["S0_density_check"]["ok"] and eq["quotient_dimension"] == 11 - eq["domain_cap_image"]
    assert 0 < eq["quotient_dimension"] <= 11
    assert eq["quotient_dimension"] == 8 and "Eight candidate directions modulo the computed exact equivalence
  +> s, before " \
        "imposing the relevant identity constraints." in eq["reading"]
    assert "essential" not in json.dumps(ct) and eq["D_cap_ker_sigma0_computed"] is False
    assert "D cap ker sigma_0" in eq["cohomology_caveat"] and "NOT a cohomological classification" in eq["coho
  +> mology_caveat"]
    # direct recomputation of the small sectors (mechanical enumeration)
    import counterterm_domain as CT
    lc, _ = CT.loop_capable_species()
    pairs = CT.admissible_pairs(2, lc)
    assert [p for p, _ in pairs] == [("h", "h"), ("c", "cb"), ("c", "K"), ("phi", "phi")]
    for (X, Y), k in pairs:
        if (X, Y) != ("h", "h"):
            keep, _ = CT.independent(CT.sector_basis(X, Y, k), [(X, Y)])
            assert len(keep) == ct["kappa2"]["basis_size_by_sector"][f"{X}.{Y}"]


MUT_GRAPH = {"wrong_tadpole_factor": "D02", "wrong_ghost_loop_sign": "D03", "reversed_source_ghost_order": "D1
  +> 3"}


def test_amplitude_rules_and_mutations():
    am = cj("P14_amplitude_rules.json")
    assert am["amplitude_rules_verified"] is True and am["integrands_only"] and am["loop_evaluations_performed
  +> "] is False
    assert not am["mutations_from_env"]
    assert all(c["pass"] for c in am["controls"].values()) and len(am["controls"]) == 6
    w4 = am["controls"]["W4_ward_bose"]["checks"]
    assert w4["ward_contraction_two_free_onshell_graviton_legs_offshell_gauge_leg"] is True
    assert not any("onshell_3graviton" in k for k in w4)
    assert am["controls"]["W5_regeneration"]["covers"] == ["D14a", "D14b", "D14c", "D11", "D13"]
    assert am["regeneration_coverage"]["W6_hessian_trlog_route"] == ["D01", "D02", "D03", "D12"]
    assert am["controls"]["W6_hessian_route"]["pass"] and am["W6_exports"]["all_agree"]
    assert set(am["mutations"]) == set(PC_MUTS) and all(v["rejected"] for v in am["mutations"].values())
    assert am["mutations_all_rejected"] is True
    inv = {v["name"]: v for v in am["vertex_inventory"]}
    for nm in ("V3_grav", "V4_grav", "V_cb_c_h", "V_K_c_h", "V_cs_c_c", "V_phis_c_phi", "V_h_phi_phi", "V_hh_p
  +> hi_phi"):
        assert inv[nm]["kappa_power"] == len(inv[nm]["legs"]) - 2
    assert am["rules"]["graphs"]["D13"]["ext_order"] == ["K", "c"]
    for g, rec in am["graphs"].items():
        f = ROOT / rec["file"]
        body = json.loads(f.read_text())
        assert rec["integrand_available"] and rec["integrated"] is False and body["integrated"] is False
        assert body["sha256"] == rec["sha256"] and rec["value"] == "not evaluated"
        assert all(d.endswith("+i0") for d in rec["denominators"])
    assert am["graphs"]["D13"]["numerator_components"] > 0 and am["graphs"]["D13"]["prefactor"] == "I*kappa**2
  +> "
    # direct: the Berezin/Wick shadow rejects the rule-level mutations and accepts the base rules
    import amplitude_rules as A, shadow as S
    base = S.run_shadows(A.base_rules())
    assert all(v["agree"] for v in base.values())
    for mname, g in MUT_GRAPH.items():
        r = S.run_shadows(A.apply_mutations(A.base_rules(), {mname}))
        assert r[g]["agree"] is False, mname


PC_MUTS = ("wrong_tadpole_factor", "wrong_ghost_loop_sign", "offdiag_multiplicity", "reversed_source_ghost_ord
  +> er",
           "omit_ghost_transport_term")
```

```
def _walk(o, path=""):
    if isinstance(o, dict):
        for k, v in o.items():
            yield from _walk(v, f"{path}/{k}")
    elif isinstance(o, list):
        for i, v in enumerate(o):
            yield from _walk(v, f"{path}/{i}")
    else:
        yield path, o


def test_D12_missing_never_zero():
    rs = _rs()
    assert rs["D12_status"] == "Missing" and rs["hard_stops"]["D12_status"] == "Missing"
    assert rs["hard_stops"]["D12_quantum_evaluated"] is False
    am, pd = cj("P14_amplitude_rules.json"), cj("P14_prescription_D11_matching.json")
    assert am["graphs"]["D12"]["D12_status"] == "Missing" and am["graphs"]["D12"]["value"] == "not evaluated"
    assert pd["D12_status"] == "Missing" and pd["D12_value_imported"] is False
    d12 = json.loads((ROOT / "exports" / "P14_integrands" / "D12.json").read_text())
    assert d12["integrated"] is False
    for name in ("P14_release.json", "P14_closure_release.json", "P14_amplitude_rules.json",
                 "P14_prescription_D11_matching.json", "P14_quantum_object.json", "P14_dossier.json"):
        for p, v in _walk(cj(name)):
            if "D12" in p and ("value" in p.split("/")[-1] or "status" in p.split("/")[-1].lower()):
                # a Missing value is never a numeric zero (booleans such as D12_value_imported=false are not v
  +> alues)
                assert isinstance(v, bool) or v not in (0, "0", 0.0, "0.0"), (name, p)


def test_projected_vs_bulk():
    rs = _rs()
    assert "NOT functional ST4b" in rs["projected_vs_bulk"] and "lem:field-only" in rs["projected_vs_bulk"]
    hs = rs["hard_stops"]
    for k in ("projected_defect_computed", "projected_identity_verified", "projected_restoration_verified",
              "quantum_inputs_evaluated"):
        assert hs[k] == "not_earned", k
    assert hs["bulk_symmetry_restored"] is False and rs["bulk_symmetry_restored"] is False
    m7 = cj("P14_prescription_D11_matching.json")["matching"]["M7_projected_vs_bulk"]
    assert m7["XVI_O_kernel_Ward_NOT_functional_ST4b"] is True and m7["symmetry_restored_bulk_XVI_O"] is False
    assert rs["O1_closure"] == "NOT_CLOSED" and rs["XV_B_opened"] is False and rs["Route_B_discharged"] is Fal
  +> se
    assert rs["insertion_status"].startswith("Frozen")


def test_freeze_binding():
    import freeze_p13 as FZ
    sys.path.insert(0, str(ROOT / "tools"))
    import source_manifest as SM
    fr = FZ.freeze(ROOT)
    assert fr["P3_P4_frozen_as_P5_inputs"] is True and fr["frozen_code_unchanged"] and fr["frozen_values_uncha
  +> nged"]
    assert fr["F1"]["bound_to_p13_tree"] and fr["F1"]["bound_to_p13_zip"] and fr["F1"]["mismatches"] == 0
    assert fr["F1"]["independent_comparisons"] == 77
    assert fr["F2"]["files"] == 54 and fr["F2"]["changed_or_missing"] == [] and fr["F3"]["differences"] == []
    rs = _rs()
    p13 = rs["p13_independent_rebuild"]
    assert p13["independent_rebuild_passed"] is True and p13["bound_to_source_tree_sha256"] == PC.P13_SOURCE_T
  +> REE_SHA256
    assert p13["bound_to_source_zip_sha256"] == PC.P13_SOURCE_ZIP_SHA256 and "SAME box" in p13["caveat"]
    # NOT inherited: the p14 tree differs from the p13 tree, and the p14 independent flag is separate
    assert SM.tree_sha(ROOT) != PC.P13_SOURCE_TREE_SHA256
    # p15: the contract record's flag reads the p14r report path, which (where present) is bound to the p14r t
  +> ree, so
    # it is never true for this tree; the CURRENT-tree flag is the p15 status flag (pending)
    assert rs["independent_rebuild_passed"] in ("pending", False)
    assert json.loads((ROOT / "RELEASE_STATUS.json").read_text())["independent_rebuild_passed"] == "pending"
    # the reviewer's files are untouched where present (never written by this package)
    live = ROOT / "reproducibility" / "independent_rebuild_report.json"
    if live.exists():
        assert hashlib.sha256(live.read_bytes()).hexdigest() == PC.P13_INDEPENDENT_REPORT_SHA256
    ref = ROOT / "reproducibility" / "reference_values.json"
    if ref.exists():
        assert hashlib.sha256(ref.read_bytes()).hexdigest() == PC.P13_REFERENCE_VALUES_SHA256


def _flip(rel, path, value):
    def f(d):
        p = d / rel
        o = json.loads(p.read_text())
        cur = o
        for k in path[:-1]:
```

```
            cur = cur[k]
        cur[path[-1]] = value
        p.write_text(json.dumps(o, indent=1))
    return f


def _append(rel, text):
    def f(d):
        with open(d / rel, "a") as fh:
            fh.write(text)
    return f


GATE_MUTS = {
    "baseline_unmutated": lambda d: None,
    "dred_mutation_not_rejected": _flip("certificates/P14_dred_rules.json", ["mutations", "premature_d4", "rej
  +> ected"], False),
    "counterterm_flag_false": _flip("certificates/P14_counterterm_domain.json", ["counterterm_domain_defined"]
  +> , False),
    "D11_matching_verified_true": _flip("certificates/P14_prescription_D11_matching.json", ["D11_matching_veri
  +> fied"], True),
    "D12_value_zero": _flip("certificates/P14_amplitude_rules.json", ["graphs", "D12", "value"], 0),
    "amplitude_mutation_not_rejected": _flip("certificates/P14_amplitude_rules.json",
                                             ["mutations", "wrong_ghost_loop_sign", "rejected"], False),
    "frozen_classical_file_changed": _append("code/classical/conventions.py", "\n# p14 edit (must break the fr
  +> eeze)\n"),
    "p13_independent_report_tampered": _flip("archival/p13/reproducibility/independent_rebuild_report_p13.json
  +> ",
                                             ["pass"], False),
}


def _gate_chain(name):
    d = _copy_pkg()
    live = d / "reproducibility" / "independent_rebuild_report.json"
    if name == "p13_independent_report_tampered" and live.exists():
        live.unlink()          # the archived copy is authoritative; keep the live copy out of this mutation
    GATE_MUTS[name](d)
    rc = _run(d, "release")
    st = json.loads((d / "RELEASE_STATUS.json").read_text())
    out = {"rc": rc, "status": st, "closure_exists": (d / "certificates" / "P14_closure_release.json").exists(
  +> ),
           "rejected_exists": (d / "certificates" / "P14_release_rejected.json").exists()}
    shutil.rmtree(d, ignore_errors=True)
    return name, out


@pytest.fixture(scope="module")
def gatemut():
    with ThreadPoolExecutor(max_workers=6) as ex:
        res = dict(ex.map(_gate_chain, list(GATE_MUTS)))
    outd = ROOT / "build" / "p14" / "test_results"
    outd.mkdir(parents=True, exist_ok=True)
    (outd / "p5_entry_gate_mutations.json").write_text(json.dumps(
        {k: {"rc": v["rc"], "publication": v["status"].get("publication"),
             "P5_entry_earned": v["status"].get("P5_entry_earned"),
             "P3_P4_frozen_as_P5_inputs": v["status"].get("P3_P4_frozen_as_P5_inputs"),
             "P5_entry_flags": v["status"].get("P5_entry_flags"),
             "hard_stops_hold": v["status"].get("hard_stops_hold"),
             "closure_artifact_exists": v["closure_exists"]} for k, v in res.items()}, indent=1) + "\n")
    return res


def test_p5_entry_gate_and_hard_stops():
    rs = _rs()
    assert list(rs["P5_entry_flags"]) == list(PC.P5_ENTRY_FLAGS) and all(rs["P5_entry_flags"].values())
    assert rs["P5_entry_earned"] is True and rs["P3_P4_frozen_as_P5_inputs"] is True
    q = rs["P5_entry_qualification"]
    assert q == PC.P5_ENTRY_QUALIFICATION and "no scalar integral is evaluated" in q and "N4" in q
    assert "D cap ker sigma_0 not computed" in q and "conditional on M2" in q
    assert "P3_P4_frozen_as_P5_inputs" in rs["P5_entry_dependency"] and "SAME box" in rs["P5_entry_dependency"
  +> ]
    assert rs["hard_stops"] == PC.HARD_STOPS and rs["hard_stops_hold"] is True and all(rs["certificate_hard_st
  +> ops"].values())
    assert rs["D11_matching_verified"] is False
    assert rs["hard_stops"]["XIV_J_opened"] is False and rs["hard_stops"]["u_R_c_delta_added"] is False
    cl = cj("P14_closure_release.json")
    assert cl["P5_entry_earned"] is True and cl["quantum_inputs_evaluated"] == "not_earned"
    assert cl["projected_restoration_verified"] == "not_earned" and cl["bulk_symmetry_restored"] is False
```

```
# P.5-only mutations: the freeze must stay TRUE in the isolated copy and the named contract flag must turn fal
  +> se
P5_ONLY = {"dred_mutation_not_rejected": "quantum_prescription_defined",
           "counterterm_flag_false": "counterterm_domain_defined",
           "amplitude_mutation_not_rejected": "amplitude_rules_verified"}
HARD_STOP_MUTS = ("D11_matching_verified_true", "D12_value_zero")
CLASSICAL_MUTS = ("frozen_classical_file_changed", "p13_independent_report_tampered")


def test_isolated_copy_contains_frozen_inputs():
    assert set(_frozen_input_tops()) <= set(COPY_SUBS) and "tests" in COPY_SUBS
    d = _copy_pkg()
    try:
        import freeze_p13 as FZ
        fr = FZ.freeze(d)
        assert fr["frozen_code_unchanged"] and fr["F2"]["changed_or_missing"] == [] and fr["P3_P4_frozen_as_P5
  +> _inputs"]
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_isolated_copy_baseline_accepted(gatemut):
    r = gatemut["baseline_unmutated"]
    st = r["status"]
    assert r["rc"] == 0 and st["publication"] == "accepted" and st["release_claim_accepted"] is True
    assert st["P3_P4_frozen_as_P5_inputs"] is True and st["P5_entry_earned"] is True and all(st["P5_entry_flag
  +> s"].values())
    assert r["closure_exists"] and not r["rejected_exists"]


@pytest.mark.parametrize("name", [n for n in GATE_MUTS if n != "baseline_unmutated"])
def test_p5_entry_gate_mutation_rejected(gatemut, name):
    r = gatemut[name]
    st = r["status"]
    assert r["rc"] != 0 and st["publication"] == "REJECTED" and st["release_claim_accepted"] is False
    assert not r["closure_exists"] and r["rejected_exists"]
    if name in P5_ONLY:                                  # rejected for the INTENDED reason
        assert st["P3_P4_frozen_as_P5_inputs"] is True
        assert st["P5_entry_flags"][P5_ONLY[name]] is False and st["P5_entry_earned"] is False
        assert all(v for k, v in st["P5_entry_flags"].items() if k != P5_ONLY[name])
    if name in HARD_STOP_MUTS:
        assert st["P3_P4_frozen_as_P5_inputs"] is True and st["hard_stops_hold"] is False
    if name in CLASSICAL_MUTS:
        assert st["P3_P4_frozen_as_P5_inputs"] is False and st["P5_entry_earned"] is False
        assert all(st["P5_entry_flags"].values())            # contract flags alone do not earn P.5 entry


# ============================================================== p14r: operational DRED
def test_dred_rules_certificate():
    dr = cj("P14_dred_rules.json")
    assert dr["dred_rules_verified"] is True and dr["revision"] == PC.REVISION
    assert dr["trace_rules_check"]["pass"] and dr["known_reductions"]["pass"]
    assert set(dr["mutations"]) == {"premature_d4", "trace_ghat_eq_4", "contract_after_laurent"}
    assert all(v["rejected"] for v in dr["mutations"].values())
    assert dr["evanescent_parts_present_in_exports"] is True
    assert "withdrawn" in dr["rules"]["R6_evanescent"]
    assert dr["loop_evaluations_performed"] is False


def test_dred_premature_d4_changes_finite_part_and_is_caught():
    import dred as DR
    a, b = DR.A_, DR.B_
    good = DR.toy_four_d_part()
    assert sp.simplify(good["finite"] - (b / 4 + a / 8)) == 0
    # the premature-d=4 result would silently drop a/8 ...
    bad = DR.toy_four_d_part(mutation="premature_d4", enforce=False)
    assert sp.simplify(bad["finite"] - b / 4) == 0 and sp.simplify(good["finite"] - bad["finite"]) != 0
    # ... and the guarded pipeline refuses it
    with pytest.raises(DR.PrematureD4Substitution):
        DR.toy_four_d_part(mutation="premature_d4")


def test_dred_evanescent_trace_is_d_minus_4_times_pole():
    import dred as DR
    t2 = DR.toy_evanescent_trace()
    assert sp.simplify(t2["finite"] + DR.A_ / 2) == 0 and sp.simplify(t2["pole"]) == 0
    with pytest.raises(DR.UncontractedObjects):
        DR.toy_evanescent_trace(mutation="contract_after_laurent")


def test_dred_trace_ghat_eq_4_rejected():
```

```
    import dred as DR
    bad = DR.Rules(trace_hat=4)
    assert bad.check()["pass"] is False
    assert DR.Rules().check()["pass"] is True
    # p15 contract correction (finding F1/F2 of the p14r independent rebuild): a numeric trace is refused stru
  +> cturally
    with pytest.raises(DR.NumericTraceForbidden):
        DR.toy_evanescent_trace(rules=bad)
    # display only: with tr ghat = 4 the evanescent trace vanishes identically: the (d-4) x pole finite part i
  +> s lost
    t2 = DR.toy_evanescent_trace(rules=DR.Rules(trace_hat=4, display_only=True), enforce=False)
    assert sp.simplify(t2["finite"]) == 0 != sp.simplify(DR.toy_evanescent_trace()["finite"])


def test_dred_known_reductions_and_contraction_engine():
    import dred as DR
    kr = DR.known_reductions(DR.Rules())
    assert kr["pass"] and all(kr.values())
    # eta^{rs} ghat_rs = d after the Gt trace rule (contraction before expansion)
    tot = sum(DR.ETA[r] * (1 - DR.ETA[r] * DR.Gt(r, r)) * DR.ETA[r] for r in range(4))
    assert sp.simplify(DR.eta_trace_Gt(sp.expand(tot), DR.Rules()) - DR.d) == 0
    with pytest.raises(DR.UncontractedObjects):
        DR.eta_trace_Gt(DR.Gt(0, 1) + DR.Gt(0, 0), DR.Rules())


# ============================================================ p14r: independent Hessian-route regeneratio
  +> n (W6)
def test_hessian_route_regenerates_D01_D02_D03_D12():
    import hessian_route as HR, amplitude_rules as A
    ex = HR.compare_exports(ROOT)
    assert ex["all_agree"] is True
    for g, n in (("D01", 100), ("D02", 94), ("D03", 100), ("D12", 94)):
        assert ex[g]["agree"] and ex[g]["n_mismatches"] == 0 and ex[g]["components"] == n, g
        assert ex[g]["export_sha256"] == json.loads((ROOT / "exports" / "P14_integrands" / f"{g}.json").read_t
  +> ext())["sha256"]
    _, meta = HR.reference()
    assert all(meta["graviton_inverse"].values()) and all(meta["structural"].values())
    # the independent route rejects rule-level mutations of the graphs it covers
    for mname, g in (("wrong_tadpole_factor", "D02"), ("wrong_ghost_loop_sign", "D03"), ("offdiag_multiplicity
  +> ", "D01")):
        w = HR.W6(A.build_integrands(A.apply_mutations(A.base_rules(), {mname})))
        assert w["checks"][f"{g}_hessian_route_equals_integrand"] is False, mname
```

127 Source: p15_conventions.py (p15 constants, contract pins)

[listing 94/133] code/p5_calc/p15_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p15 conventions: FIRST P.5 CALCULATION (Batch A: D02, D12, D14a-c; Batch B: D13).

Layering.  The p14r P.5-entry contract (code/p5_entry/*, P14_* certificates, revision label p14r) is the froze
  +> n
contract on which p15 evaluates.  p15 adds a calculation layer (code/p5_calc/*, P15_* certificates); the p15 r
  +> elease
gate (release_p15.py) runs AFTER the p14r contract gate and requires it to be accepted.

ONE disclosed contract revision (DISCLOSED_CONTRACT_CORRECTIONS): code/p5_entry/dred.py, after finding F1 of t
  +> he p14r
independent rebuild (the heuristic PrematureD4 guard had a false positive and a false negative).  Every other
  +> p14r
contract file is byte-identical to p14r (P14R_CONTRACT_SHA256, checked by release_p15).  Released p14r values
  +> are
unaffected: p14r applied Pipeline.laurent only to toy placeholders.
"""
from __future__ import annotations

REVISION = "p15"
CERT_PREFIX = "P15"
CONTRACT_REVISION = "p14r"
DATE = "2026-10-08"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p15.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p15.json"
P14R_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p14r.json"   # written by the reviewer;
  +> never here
P14R_SOURCE_TREE_SHA256 = "a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894"
P14R_SOURCE_ZIP_SHA256 = "19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3"

BATCH_A = ("D02", "D12", "D14a", "D14b", "D14c")
```

```
BATCH_B = ("D13",)
NOT_EVALUATED_P15 = ("D01", "D03", "D11")            # Batch C (p16) and the D11 matching (M2)

STATUS_VALUES = ("unevaluated", "regulated", "vanishing_under_declared_prescription", "renormalized")

# the p15 calculation-layer hard stops (the p14r contract hard stops are unchanged and checked by release_p14)
HARD_STOPS_P15 = {
    "quantum_inputs_evaluated": "not_earned",          # needs all three slots in one matched framework
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": False,
    "symmetry_restored": False,
    "D11_matching_verified": False,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": False,
    "XIV_J_opened": False,
    "Route_B_discharged": False,
    "u_R_c_delta_added": False,
    "D01_evaluated": False, "D03_evaluated": False, "D11_evaluated": False,
}

# sha256 of the p14r contract code (from XVI-P-p14r-source-only.zip, tree a64c0ae3...); release_p15 recomputes
  +>  them
P14R_CONTRACT_SHA256 = {
    "code/p5_entry/amplitude_rules.py": "0daf63a3c8803d54514cb2aac29bea127c379e025f32c5b0219d365de5bdde4a",
    "code/p5_entry/counterterm_domain.py": "bf2eb6bdb9c64f4bd9e7fce459c6f8b1bf2167323589a3832e8c9ad4dcb91241",
    "code/p5_entry/dossier_p14.py": "94996ab4505219f9a40facffeebb3d9c419bbb8f4ecfbf3635d41705e53fb7e5",
    "code/p5_entry/dred.py": "ba85b00cd10e77d4274808f7dcaef1cb5b1934761fe8a81551e3c7405939ee20",
    "code/p5_entry/feynman.py": "4fab8b0d4a1cc0b4cd0be19ec30ace80559b92fa9808f1fc25c858599a389344",
    "code/p5_entry/freeze_p13.py": "9151ea2e22919319364c52fd42d2accab9af11ea17d15df8a3938149b31df0f1",
    "code/p5_entry/hessian_route.py": "a7bee01d140d07b06ae7ddfba5316ce4d7d43ac8fd00d497fa468ac8d1b6cee8",
    "code/p5_entry/p14_conventions.py": "687e4fa9bdcf0c87a2f71aec4470986997185b82507f608fdd888cf6a90aa5b4",
    "code/p5_entry/prescription_d11.py": "f05fc3de6c07bf542960388144f21aa51ba31993a890ef3fa306a287b25f33fb",
    "code/p5_entry/quantum_object.py": "ea4ebc5aa916ea975ba56ee9aacedeb67e3d483196b2920b77895f5b375e8c05",
    "code/p5_entry/release_p14.py": "d22cf1001e54ae4a4551497dbc1fd97a82d184984964d49d00826dc84e242217",
    "code/p5_entry/shadow.py": "de6556660359bc2d8ee00621d39004944a1a0f300200a0402c0b072a28e4648b",
}

DISCLOSED_CONTRACT_CORRECTIONS = {
    "code/p5_entry/dred.py": {
        "kind": "p14r contract revision (disclosed), applied inside p15",
        "finding": "F1_premature_d4_guard_heuristic, reproducibility/independent_rebuild_report_p14r.json",
        "why": ("the p14r PrematureD4Substitution guard was a heuristic on the shape of the expression before
  +> the "
                "Laurent expansion: FALSE POSITIVE on eta^{rs} int l_r l_s/(l^2-m^2) = J (correct, d-free) and
  +>  FALSE "
                "NEGATIVE under premature_d4 on the same integral (eta_trace_Gt reintroduces d via tr gtilde =
  +>  4-d: "
                "d J/4, finite error -m^4/2); the R5 docstring claim 'is detected' did not hold in general"),
        "change": ("structural, provenance-based safeguard: d is an OpaqueDimension (subs/xreplace raise "
                   "PrematureD4Substitution outside dred.substitute_d, which records every authorised use); nu
  +> meric "
                   "traces of ghat/gtilde refused (NumericTraceForbidden) unless display_only; the heuristic i
  +> s "
                   "removed; R5 docstring corrected; reviewer toys added as regression checks (reviewer_toys)"
  +> ),
        "rules_R1_R6_changed": False,
        "released_p14r_values_affected": False,
        "why_unaffected": "p14r applied Pipeline.laurent only to toy placeholders; no loop integral was evalua
  +> ted",
    },
}
# test-suite revision that accompanies the correction (tests are not contract code; disclosed for completeness
  +> )
DISCLOSED_TEST_REVISIONS = {
    "tests/test_p14.py": ("test_dred_trace_ghat_eq_4_rejected: a numeric trace is now refused (NumericTraceFor
  +> bidden); "
                          "the tr ghat = 4 display uses Rules(trace_hat=4, display_only=True); infrastructure
  +> "
                          "assertions read the p14r contract record from certificates/P14_release.json (the to
  +> p-level "
                          "RELEASE_STATUS.json is the p15 status)"),
}
P14R_INDEPENDENT_REPORT_SHA256 = "09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b"
# the other archived p14r records shipped as source (read when the zips are written): pinned by sha256
P14R_ARCHIVED_RECORDS = {
    "archival/p14r/reference_values_p14r.json": "9e81749235d3cb115aacc748de0ae4f89c6eae81b3e99f45585d1ad36b328
  +> f48",
    "archival/p14r/reproducibility/author_rebuild_report_p14r.json":
```

```
        "2f7def0c65f954c4866a8d2efafcf117f31f7a357a023e85ce6f67182aff6db1",
}
# run-specific keys of the contract record (rebuild flags/records; freeze sub-records that depend on files abs
  +> ent from
# a source-only copy): excluded when release_p15 hashes its inputs (same list as tools/rebuild_from_source.py)
RUN_SPECIFIC_KEYS = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed",
                     "live_report_identical_to_archived", "live_report_present", "archived_zip_sha256_ok",
                     "list_equals_archived_zip")
```

128 Source: integrals.py (p15 integral library, S0)

[listing 95/133] code/p5_calc/integrals.py

```
#!/usr/bin/env python3
"""XVI-P p15: certified one-loop scalar-integral library (A0, B0, massless two-point family, ghat tensor reduc
  +> tion)
under the FROZEN p14r prescription (DRED R1-R6 of code/p5_entry/dred.py, Feynman +i0, MS-bar).

Normalisation.  Every integral  I = mu^{2 eps} int d^d l/(2 pi)^d (...)  (Minkowski, eta = +,-,-,-, all denomi
  +> nators
q^2 - m^2 + i0, d = 4 - 2 eps) is returned in units of  N_eps = i S_eps/(16 pi^2),  S_eps = (4 pi)^eps e^{-gam
  +> ma_E eps}:
I = N_eps * Ihat.  MS-bar (pole in 1/eps - gamma_E + log 4 pi) == drop the 1/eps poles of Ihat; finite part =
  +> eps^0
coefficient of Ihat.  L := log(-p^2/mu^2 - i0),  Lm := log(m^2/mu^2).

Exact formulas (d symbolic):
  T(k,n;m)   = mu^{2eps} int (l^2)^k/(l^2-m^2)^n
             -> Ihat = (-1)^{n+k} e^{gamma eps} Gamma(k+d/2) Gamma(n-k-d/2)/(Gamma(d/2) Gamma(n)) (m^2)^{2+k-n
  +> } e^{-eps Lm}
  I(n1,n2)   = mu^{2eps} int 1/[(l^2)^{n1} ((l-p)^2)^{n2}]  (massless)
             -> Ihat = (-1)^{n1+n2} e^{gamma eps} G(n1,n2) (-p^2)^{2-n1-n2} e^{-eps L},
                G = Gamma(d/2-n1) Gamma(d/2-n2) Gamma(n1+n2-d/2) / (Gamma(n1) Gamma(n2) Gamma(d-n1-n2))
  A0(m) = T(0,1;m),  B0(p^2;0,0) = I(1,1) =: MB  (the only massless two-point master).

S0 SCALELESS RULE (explicit, stated, part of the declared prescription; applied ONLY to integrals with no scal
  +> e):
  int d^d l (l^2)^k / (l^2)^nu  (no mass, no external momentum in the integrand after shift)  :=  0.
  UV/IR reading (Euclidean radial split at an arbitrary Lambda; omega := 4 + 2k - 2 nu, s := d + 2k - 2 nu):
    IR part int_0^Lambda r^{s-1} dr = +Lambda^s/s (converges for Re s > 0), UV part int_Lambda^oo = -Lambda^s/
  +> s (converges
    for Re s < 0); each is continued in d and the sum is 0.
    omega = 0 (logarithmic): s = -2 eps, the two parts are -1/(2 eps_UV) and +1/(2 eps_IR) times Omega_d/(2pi)
  +> ^d, i.e.
       the zero is (1/eps_UV - 1/eps_IR)-type: a UV pole cancelled by an IR pole;
    omega > 0 (UV power divergence): neither part has a pole at d = 4 (IR part finite, UV part a pure Lambda^o
  +> mega
       power); the zero discards a power divergence; NO 1/eps pole on either side;
    omega < 0 (IR power divergence): mirror image.
  A ledger records omega and this classification for EVERY application.
"""
from __future__ import annotations
import itertools
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dred  # noqa: E402   (p14r DRED rules R1-R6; p15 contract correction = structural R5 safeguard only)

d, eps = dred.d, dred.eps
ETA = dred.ETA
L = sp.Symbol("L")            # log(-p^2/mu^2 - i0)
Lm = sp.Symbol("Lm")          # log(m^2/mu^2)
PP = sp.Symbol("p.p")          # p^2 (same symbol as the frozen contraction engine)
MSQ = sp.Symbol("m2")          # IR-regulator mass squared (analysis only; the declared action is massless)
MB = sp.Symbol("MB")           # massless bubble master Ihat(1,1) (symbol; exact value master_exact())
G_ = sp.gamma


class ScalelessRuleMisuse(Exception):
```

```
    """S0 requested for an integral that carries a scale."""


LEDGER = []


def scaleless(k, nu, context, has_scale=False):
    """S0: int d^d l (l^2)^k/(l^2)^nu := 0, with the UV/IR classification recorded."""
    if has_scale:
        raise ScalelessRuleMisuse(f"S0 applied to an integral with a scale ({context})")
    omega = 4 + 2 * k - 2 * nu
    kind = "logarithmic: (1/eps_UV - 1/eps_IR) cancellation" if omega == 0 else (
        f"UV power divergence Lambda^{omega}: no pole on either side at d = 4" if omega > 0 else
        f"IR power divergence Lambda^{omega}: no pole on either side at d = 4")
    LEDGER.append({"context": context, "k": int(k), "nu": int(nu), "omega": int(omega), "class": kind})
    return sp.Integer(0)


def lambda_split(k, nu):
    """symbolic radial split (Euclidean, per unit solid angle): IR + UV = 0; poles at d = 4 iff omega = 0."""
    Lam = sp.Symbol("Lambda", positive=True)
    s = d + 2 * k - 2 * nu
    ir, uv = Lam ** s / s, -Lam ** s / s
    at4 = dred.substitute_d(s, 4, "integrals.lambda_split: power-counting exponent at d = 4 (no integrand)")
    def pole(e):
        ser = sp.series(dred.substitute_d(e, 4 - 2 * eps, "integrals.lambda_split: Laurent expansion"), eps, 0
  +> ,
                        1).removeO()
        return sp.simplify(ser.coeff(eps, -1))
    return {"omega": int(4 + 2 * k - 2 * nu), "s_at_d4": int(at4), "sum_zero": sp.simplify(ir + uv) == 0,
            "IR_part": str(ir), "UV_part": str(uv),
            "IR_pole_at_d4": str(pole(ir)) if at4 == 0 else "0 (no pole)",
            "UV_pole_at_d4": str(pole(uv)) if at4 == 0 else "0 (no pole)"}


# ------------------------------------------------------------------------------------------------ exact maste
  +> rs
def G2(n1, n2):
    return G_(d / 2 - n1) * G_(d / 2 - n2) * G_(n1 + n2 - d / 2) / (G_(n1) * G_(n2) * G_(d - n1 - n2))


def master_exact():
    """Ihat(1,1) exact in d (with eps = (4-d)/2)."""
    e = (4 - d) / 2
    return sp.exp(sp.EulerGamma * e) * G2(1, 1) * sp.exp(-e * L)


def bubble_hat(n1, n2, context="I(n1,n2)"):
    """Ihat(n1,n2) for integer n1, n2: scaleless (S0) if n1 <= 0 or n2 <= 0; otherwise R(d) * MB with R ration
  +> al."""
    if n1 <= 0 or n2 <= 0:
        # after the shift l -> l + p (if n1 <= 0) the integrand is a polynomial in l over (l^2)^{n}: scaleless
        n = n2 if n1 <= 0 else n1
        return scaleless(0, n, context + f" [I({n1},{n2}) -> polynomial/(l^2)^{n}]")
    if (n1, n2) == (1, 1):
        return MB
    raise NotImplementedError("only I(1,1) and scaleless members are needed for Batch A/B")


def tadpole_hat(k, n, msq=MSQ):
    """T(k,n;m) exact in d (massive; used for the IR-regulator analysis and as A0)."""
    e = (4 - d) / 2
    return ((-1) ** (n + k) * sp.exp(sp.EulerGamma * e) * G_(k + d / 2) * G_(n - k - d / 2) / (G_(d / 2) * G_(
  +> n))
            * msq ** (2 + k - n) * sp.exp(-e * Lm))


def laurent(expr, order=1):
    e = sp.expand(dred.substitute_d(expr, 4 - 2 * eps, "integrals.laurent: d = 4 - 2 eps after all contraction
  +> s"))
    ser = sp.series(e, eps, 0, order + 1).removeO()
    return {k: sp.simplify(ser.coeff(eps, k)) for k in range(-2, order + 1)}


_MBSER = {}


def master_series(order=1):
    if order not in _MBSER:
        _MBSER[order] = laurent(master_exact(), order)
    return _MBSER[order]
```

```
# ------------------------------------------------------------------------------------------- two-point J
  +>  values
def J_two_point(a, b, context="J"):
    """J[a,b] = int (l.l)^a (l.p)^b / (D1 D2), D1 = l^2, D2 = (l-p)^2: l.p = (D1 + p^2 - D2)/2, every member w
  +> ith a
    cancelled propagator is scaleless (S0); returns R(d, p^2) * MB."""
    D1, D2 = sp.symbols("D1 D2")
    num = sp.expand(D1 ** a * ((D1 + PP - D2) / 2) ** b)
    tot = 0
    for (i, j), cf in sp.Poly(num, D1, D2).terms():
        tot += cf * bubble_hat(1 - i, 1 - j, context + f"[{a},{b}]")
    return sp.expand(tot)


def J_values(tag, pairs):
    return {sp.Symbol(f"J_{tag}[{a},{b}]"): J_two_point(a, b, f"J_{tag}") for (a, b) in pairs}


# ------------------------------------------------------------------------------------------ Feynman-par
  +> ameter route
def _matchings(items):
    if not items:
        yield []
        return
    a = items[0]
    for k in range(1, len(items)):
        rest = items[1:k] + items[k + 1:]
        for m in _matchings(rest):
            yield [(a, items[k])] + m


def ghat(r, s):
    return (ETA[r] if r == s else 0) - dred.Gt(r, s)


def sym_integrate(comps):
    """k_{r1}..k_{rn} -> coefficient of (k^2)^{n/2} under symmetric integration in QdS (k in QdS, ghat project
  +> ions)."""
    n = len(comps)
    if n % 2:
        return sp.Integer(0)
    if n == 0:
        return sp.Integer(1)
    j = n // 2
    den = sp.Mul(*[d + 2 * i for i in range(j)])
    tot = 0
    for m in _matchings(list(range(n))):
        tot += sp.Mul(*[ghat(comps[a], comps[b]) for a, b in m])
    return sp.expand(tot / den)


def fp_ratio(j, alpha, beta):
    """[int_0^1 dx x^alpha (1-x)^beta Ihat_j(x)] / MB exact (rational in d): Ihat_j(x) is the hat of
    int (k^2)^j/(k^2 - Delta)^2, Delta = -x(1-x) p^2."""
    rf = sp.rf
    h = d / 2
    return sp.factor((-1) ** j * (-PP) ** j * rf(h, j) / rf(2 - j - h, j)
                     * rf(h - 1, alpha + j) * rf(h - 1, beta + j) / rf(d - 2, alpha + beta + 2 * j))


def fp_two_point(poly, lsym, psym):
    """Feynman-parameter evaluation of int poly(l, p)/(D1 D2) (D1 = l^2, D2 = (l-p)^2) as R(d) * MB, R rationa
  +> l;
    l = k + x p (x D2 + (1-x) D1 = k^2 + x(1-x) p^2); symmetric integration in QdS; NO scaleless rule used."""
    x = sp.Symbol("x")
    ks = sp.symbols("k0:4")
    sub = {lsym[r]: ks[r] + x * psym[r] for r in range(4)}
    e = sp.expand(sp.sympify(poly).xreplace(sub))
    if e == 0:
        return sp.Integer(0)
    P = sp.Poly(e, *ks)
    tot = 0
    for mon, cf in P.terms():
        comps = sum(([r] * m for r, m in enumerate(mon)), [])
        si = sym_integrate(comps)
        if si == 0:
            continue
        j = len(comps) // 2
        Px = sp.Poly(sp.expand(cf * si), x)
```

```
        for (ax,), cx in Px.terms():
            # x^ax with (1-x)^0
            tot += cx * fp_ratio(j, ax, 0)
    return sp.expand(tot) * MB


# ---------------------------------------------------------------------------------------------- faithful Gt
  +>  realization
def realization(p_num, seed):
    """Gt -> (4-d) M with M symmetric (lower indices), M_rs eta^{ss} p_s = 0, eta^{rs} M_rs = 1: a faithful mo
  +> del of
    the R1/R2/R4 relations for expressions linear in Gt (tr Gt = 4-d, Gt.p = 0)."""
    import random
    rnd = random.Random(seed)
    pu = [ETA[r] * p_num[r] for r in range(4)]                # p^r
    basis = []
    for _ in range(12):
        v = [sp.Rational(rnd.randint(-9, 9), rnd.randint(1, 5)) for _ in range(4)]
        # project out p:  v.p = sum eta^{rr} v_r p_r ; make v.p = 0 by adjusting with a vector u with u.p != 0
        vp = sum(ETA[r] * v[r] * p_num[r] for r in range(4))
        pp = sum(ETA[r] * p_num[r] ** 2 for r in range(4))
        v = [v[r] - vp / pp * p_num[r] for r in range(4)]
        basis.append(v)
        if len(basis) == 3:
            break
    lam = [sp.Rational(rnd.randint(1, 9), rnd.randint(1, 4)) for _ in basis]
    M = sp.zeros(4, 4)
    for l_, v in zip(lam, basis):
        M += l_ * sp.Matrix(4, 1, v) * sp.Matrix(1, 4, v)
    tr = sum(ETA[r] * M[r, r] for r in range(4))
    assert tr != 0
    M = M / tr
    assert all(sp.simplify(sum(M[r, s] * pu[s] for s in range(4))) == 0 for r in range(4))
    return {sp.Symbol(f"Gt_{min(r, s)}{max(r, s)}"): (4 - d) * M[r, s] for r in range(4) for s in range(4)}


def realize(expr, p_num, gt_map, psym=None):
    psym = psym or sp.symbols("p0:4")
    pp = sum(ETA[r] * p_num[r] ** 2 for r in range(4))
    rep = {psym[r]: p_num[r] for r in range(4)}
    rep[PP] = pp
    rep.update(gt_map)
    return sp.expand(sp.sympify(expr).xreplace(rep))


# ---------------------------------------------------------------------------------------------- certificati
  +> on
def _numeric_master(dval):
    """Euclidean numerical check: int d^d l_E/(2pi)^d 1/(l^2 (l-p)^2) at p_E^2 = 1, non-integer d (convergent
  +> for
    2 < d < 4), by radial x polar integration, vs the Gamma formula."""
    import mpmath as mp
    mp.mp.dps = 20
    D = mp.mpf(dval)
    omega_dm2 = 2 * mp.pi ** ((D - 1) / 2) / mp.gamma((D - 1) / 2)       # area of S^{d-2}
    def f(r, th):
        den = (r - mp.cos(th)) ** 2 + mp.sin(th) ** 2
        return r ** (D - 3) * mp.sin(th) ** (D - 2) / den if den else mp.mpf(0)

    def radial(th):
        c = mp.cos(th)
        pts = [0, c, 2, mp.inf] if c > 0 else [0, 1, mp.inf]
        return mp.quad(lambda r: f(r, th), pts)
    val = mp.quad(radial, [0, mp.mpf(1) / 100, mp.mpf(1) / 10, mp.pi])
    num = omega_dm2 * val / (2 * mp.pi) ** D
    exact = mp.gamma(2 - D / 2) * mp.gamma(D / 2 - 1) ** 2 / mp.gamma(D - 2) / (4 * mp.pi) ** (D / 2)
    return float(num), float(exact)


def _numeric_tadpole(dval, k=0):
    """Euclidean: int d^d l_E/(2pi)^d (l^2)^k/(l^2+1), convergent for 0 < d + 2k < 2, vs the Gamma formula."""
    import mpmath as mp
    mp.mp.dps = 20
    D = mp.mpf(dval)
    area = 2 * mp.pi ** (D / 2) / mp.gamma(D / 2)
    val = mp.quad(lambda r: r ** (D - 1 + 2 * k) / (r * r + 1), [0, 1, mp.inf])
    num = area * val / (2 * mp.pi) ** D
    exact = mp.gamma(k + D / 2) * mp.gamma(1 - k - D / 2) / mp.gamma(D / 2) / (4 * mp.pi) ** (D / 2)
    return float(num), float(exact)
```

```
def certify():
    t0 = time.time()
    out = {}
    s = master_series(1)
    out["B0_massless_laurent"] = {"pole": str(s[-1]), "finite": str(s[0]), "order_eps": str(s[1]),
                                  "ok": sp.simplify(s[-1] - 1) == 0 and sp.simplify(s[0] - (2 - L)) == 0
                                  and s[-2] == 0}
    a0 = laurent(tadpole_hat(0, 1), 0)
    out["A0_laurent"] = {"pole": str(a0[-1]), "finite": str(a0[0]),
                         "ok": sp.simplify(a0[-1] - MSQ) == 0 and sp.simplify(a0[0] - MSQ * (1 - Lm)) == 0}
    nb = [_numeric_master(x) for x in (2.6, 3.3, 3.7)]
    out["B0_exact_d_numeric"] = {"d_values": [2.6, 3.3, 3.7], "numeric_vs_formula": nb,
                                 "ok": all(abs(a - b) < 1e-8 * abs(b) for a, b in nb)}
    nt = [_numeric_tadpole(x, 0) for x in (0.7, 1.5)] + [_numeric_tadpole(x, 1) for x in (-0.6,)]
    out["T_exact_d_numeric"] = {"cases": ["d=0.7,k=0", "d=1.5,k=0", "d=-0.6,k=1"], "numeric_vs_formula": nt,
                                "ok": all(abs(a - b) < 1e-8 * abs(b) for a, b in nt)}
    # tensor reduction (frozen dred R3/R4) + J values (S0)  ==  Feynman parameters (no S0), exact in d, all co
  +> mponents
    rules = dred.Rules()
    lsym, psym = sp.symbols("l0:4"), sp.symbols("p0:4")
    p_nums = [[sp.Integer(3), sp.Integer(1), sp.Integer(-2), sp.Integer(5)],
              [sp.Rational(7, 2), sp.Integer(-1), sp.Integer(4), sp.Integer(2)]]
    Jv = J_values("LIB", [(a, b) for a in range(3) for b in range(5)])
    bad, checked = [], 0
    for n in range(0, 4):
        for comps in itertools.combinations_with_replacement(range(4), n):
            r1 = dred.lift_reduce_monomial(list(comps), True, rules, "LIB")
            r1 = sp.expand(r1.xreplace(Jv))
            r2 = fp_two_point(sp.Mul(*[lsym[c] for c in comps]), lsym, psym)
            for pn in p_nums:
                for seed in (1, 2):
                    gm = realization(pn, seed)
                    diff = sp.simplify((realize(r1, pn, gm) - realize(r2, pn, gm)) / MB)
                    checked += 1
                    if diff != 0:
                        bad.append({"comps": list(comps), "p": [str(x) for x in pn], "diff": str(diff)})
    out["tensor_reduction_vs_feynman_parameters"] = {"ranks": [0, 1, 2, 3], "checks": checked, "mismatches": b
  +> ad[:5],
                                                     "n_mismatches": len(bad), "ok": not bad,
                                                     "note": "route 1 uses S0 for every cancelled propagator;
  +> route 2 "
                                                             "uses none (Delta != 0): agreement certifies S0 o
  +> n this family"}
    out["J_table"] = {str(k): str(v) for k, v in Jv.items() if v != 0}
    out["scaleless_rule"] = {"statement": "S0: int d^d l (l^2)^k/(l^2)^nu := 0 when the integrand carries no s
  +> cale",
                             "lambda_split": {f"omega={4 + 2 * k - 2 * nu}": lambda_split(k, nu)
                                              for (k, nu) in ((1, 1), (0, 1), (0, 2), (0, 3))},
                             "misuse_guard": _misuse_guard()}
    # mass-regulator limit of the scaleless members: T(k,1;m) -> 0 as m -> 0 for Re d > 2 - 2k
    out["mass_regulator_limit"] = {f"k={k}": {"exact": str(tadpole_hat(k, 1)),
                                              "m2_power": str(sp.simplify(d / 2 + k - 1)),
                                              "vanishes_as_m_to_0_for": f"Re d > {2 - 2 * k}"} for k in (0, 1,
  +>  2)}
    out["i0"] = {"L": "log(-p^2/mu^2 - i0) = log|p^2/mu^2| - i pi theta(p^2)  (Feynman +i0)",
                 "Lm": "log(m^2/mu^2) (m^2 - i0)"}
    out["ok"] = all(v.get("ok", True) for v in out.values() if isinstance(v, dict))
    out["seconds"] = round(time.time() - t0, 1)
    return out


def _misuse_guard():
    try:
        scaleless(0, 1, "A0 with m != 0 (mutation: S0 on a massive tadpole)", has_scale=True)
        return {"rejected": False}
    except ScalelessRuleMisuse as exc:
        return {"rejected": True, "reason": str(exc)}


def main():
    import p15_conventions as QC
    import certio
    LEDGER.clear()
    res = certify()
    rec = {"landmark": "XVI-P", "revision": QC.REVISION, "deliverable": "P.5 calculation: one-loop integral li
  +> brary",
           "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 (DRED R1-R6, Feynman +i0, MS-bar); S0 stated
  +> explicitly",
           "normalisation": "I = N_eps * Ihat, N_eps = i S_eps/(16 pi^2), S_eps = (4 pi)^eps e^{-gamma_E eps},
  +>  mu^{2 eps}; "
                            "MS-bar = drop the poles of Ihat",
```

```
          "library_certified": bool(res["ok"]), **res, "S0_ledger_size": len(LEDGER)}
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_integral_library.json", rec)
    print(json.dumps({"library_certified": rec["library_certified"], "seconds": res["seconds"],
                      "checks": {k: v.get("ok") for k, v in res.items() if isinstance(v, dict) and "ok" in v}}
  +> , indent=1))
    return 0 if rec["library_certified"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

129 Source: batchA.py (p15 Batch A)

[listing 96/133] code/p5_calc/batchA.py

```
#!/usr/bin/env python3
"""XVI-P p15 Batch A: the massless tadpoles D02 (kappa^2 hh, quartic graviton vertex), D12 (kappa^2 hh, h h ph
  +> i phi),
and the order-kappa one-point graphs D14a (graviton loop), D14b (ghost loop), D14c (scalar loop).

Prescription: frozen p14r P14_DRED_MSbar_FeynmanI0 + the explicit scaleless rule S0 (integrals.py).  The decla
  +> red
action is massless (graviton, ghost and scalar propagators i N/(q^2 + i0); no mass, no external scale in any B
  +> atch A
integrand: D02/D12 carry p only in the numerator, D14 has p = 0).

Per graph:
  route 1  frozen export (sha-pinned, = the p14r W5/W6-verified integrand) -> frozen dred R3/R4 lift (tadpole
  +> branch)
           -> int (l^2)^k / l^2  members  -> S0 (each application ledgered with omega and the UV/IR class);
  route 2  independent symmetric integration of the exported component polynomial (own code, ghat = eta - Gt);
  IR-regulator analysis  1/l^2 -> 1/(l^2 - m^2) (exact in d, A0-type), poles/finite parts and the m -> 0 limit
  +> ;
  analytic regulator     (l^2)^{-1-nu}: still scaleless (no separation without a scale);
  cutoff split           Lambda^omega UV and IR pieces, no 1/eps pole on either side (omega > 0 for every memb
  +> er).
Statuses: vanishing_under_declared_prescription (derived; never 'zero by convention').  D12 (kappa^2 two-point
  +> ) and the
order-kappa one-point slot (D14a+D14b+D14c) are recorded SEPARATELY.
"""
from __future__ import annotations
import hashlib
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dred  # noqa: E402
import integrals as IL  # noqa: E402
import p15_conventions as QC  # noqa: E402

d, eps = dred.d, dred.eps
KAPPA = sp.Symbol("kappa")
SLOTS = {"D02": "kappa^2 hh two-point (part of the hh slot with D01, D03 -- D01/D03 NOT evaluated in p15)",
         "D12": "kappa^2 hh two-point, scalar tadpole (D12 slot; kept separate from the one-point slot)",
         "D14a": "order-kappa one-point slot", "D14b": "order-kappa one-point slot",
         "D14c": "order-kappa one-point slot"}
ORDER = {"D02": 2, "D12": 2, "D14a": 1, "D14b": 1, "D14c": 1}


def load_export(g):
    f = ROOT / "exports" / "P14_integrands" / f"{g}.json"
    body = json.loads(f.read_text())
    stored = body.pop("sha256")
    body.pop("integrated", None)                    # written next to the hashed record (amplitude_rules.py)
    sha = hashlib.sha256(json.dumps(body, sort_keys=True, separators=(",", ":")).encode()).hexdigest()
    amp = json.loads((ROOT / "certificates" / "P14_amplitude_rules.json").read_text())
    pinned = amp["graphs"][g]["sha256"]
    return body, {"file": f"exports/P14_integrands/{g}.json", "sha256_recomputed": sha, "sha256_stored": store
  +> d,
                  "sha256_p14r_certificate": pinned, "pinned": sha == stored == pinned}
```

```
def _poly(rows):
    return sp.Add(*[sp.Rational(cf) * sp.Mul(*[sp.Symbol(v) for v in names]) for names, cf in rows])


def route1(body, g):
    """frozen dred lift (with_p = False: tadpole) -> {component: expression in J_g[k,0], Gt, p_r, d}."""
    rules = dred.Rules()
    out = {}
    for comp, rows in body["numerator"].items():
        v = dred.lift_reduce_polynomial(rows, False, rules, g)
        if v != 0:
            out[comp] = v
    return out


def route2(body):
    """own symmetric integration: l_{r1}..l_{rn} -> ghat-symmetrised (l^2)^{n/2}/(d(d+2)..) ; returns {comp: {
  +> k: coef}}."""
    ls = sp.symbols("l0:4")
    out = {}
    for comp, rows in body["numerator"].items():
        P = sp.Poly(sp.expand(_poly(rows)), *ls)
        acc = {}
        for mon, cf in P.terms():
            comps = sum(([r] * m for r, m in enumerate(mon)), [])
            si = IL.sym_integrate(comps)
            if si != 0:
                k = len(comps) // 2
                acc[k] = sp.expand(acc.get(k, 0) + cf * si)
        acc = {k: v for k, v in acc.items() if v != 0}
        if acc:
            out[comp] = acc
    return out


def evaluate(g):
    body, pin = load_export(g)
    pref = sp.sympify(body["prefactor"], locals={"kappa": KAPPA})
    r1 = route1(body, g)
    r2 = route2(body)
    # route 1 J symbols are J_g[k,0] = int (l^2)^k / l^2 ; route 2 coefficient of (l^2)^k
    Js = {k: sp.Symbol(f"J_{g}[{k},0]") for k in range(4)}
    agree, n_cmp = True, 0
    for comp in set(r1) | set(r2):
        e1 = r1.get(comp, 0)
        e2 = sum(c * Js[k] for k, c in r2.get(comp, {}).items())
        n_cmp += 1
        if sp.expand(e1 - e2) != 0:
            agree = False
    members = sorted({int(str(s).split("[")[1].split(",")[0]) for e in r1.values() for s in e.free_symbols
                      if str(s).startswith(f"J_{g}[")})
    # S0 on every member (massless, no scale): ledgered with omega
    IL.LEDGER.clear()
    s0 = {k: IL.scaleless(k, 1, f"{g}: int (l^2)^{k}/l^2") for k in members}
    led = list(IL.LEDGER)
    value = {comp: sp.expand(e.xreplace({Js[k]: s0[k] for k in members})) for comp, e in r1.items()}
    all_zero = all(v == 0 for v in value.values())
    logarithmic = [x for x in led if x["omega"] == 0]
    # evanescent content of the regulated expression (before S0)
    n_ev = sum(1 for e in r1.values() if dred.split_evanescent(e)[1] != 0)
    # ---- IR-regulator mass on the loop line: int (l^2)^k/(l^2 - m^2) = T(k,1;m), exact in d
    Tm = {Js[k]: IL.tadpole_hat(k, 1) for k in members}
    mass_pole, mass_fin, mass_struct = {}, {}, {}
    for comp, e in r1.items():
        ev = sp.expand(e.xreplace(Tm))
        # contract the evanescent pieces with the faithful realization only for the classification of p-powers
  +> ;
        # the Laurent expansion is done on the exact-d expression with Gt kept symbolic (R6)
        ser = IL.laurent(ev, 0)
        mass_pole[comp] = sp.expand(ser[-1])
        mass_fin[comp] = sp.expand(ser[0])
        pw = set()
        for t in sp.Add.make_args(mass_pole[comp]):
            if t != 0:
                pw.add(int(sp.degree(t, IL.MSQ)))
        mass_struct[comp] = sorted(pw)
    m_powers = sorted({x for v in mass_struct.values() for x in v})
    lim0 = all(sp.limit(sp.expand(v.xreplace({IL.Lm: sp.log(IL.MSQ)})), IL.MSQ, 0, "+") == 0
               for v in list(mass_fin.values())[:12])
```

```
  ex = sorted(r1)[:3]
  rec = {
      "graph": g, "slot": SLOTS[g], "kappa_order": ORDER[g], "export": pin, "prefactor": body["prefactor"],
      "denominators": body["denominators"], "massless_declared": all(x == "l^2+i0" for x in body["denominato
+> rs"]),
      "status": "vanishing_under_declared_prescription" if (all_zero and pin["pinned"] and agree) else "regu
+> lated",
      "regulated_expression": {
          "form": f"iGamma_1 = N_eps * ({body['prefactor']}) * sum_comp [ sum_k c_k(d, p, Gt) J_{g}[k,0] ],
+>  "
                  f"J_{g}[k,0] = int (l^2)^k / (l^2 + i0)",
          "components": len(r1), "J_members_k": members,
          "components_with_evanescent_Gt_part": n_ev,
          "examples": {c: str(r1[c]) for c in ex}},
      "UV_subtraction": "none: no 1/eps pole (every member is a pure power divergence, omega > 0); MS-bar co
+> unterterm 0",
      "finite_part": "0 (all components)" if all_zero else "NONZERO",
      "local_ambiguity": ("none generated in the declared prescription; the kappa^2 hh slot's local freedom
+> is the p14r "
                          "h.h counterterm directions (degree 4 in p) and is NOT touched by DO2/D12 (no pole
+> , no "
                          "finite local term)" if ORDER[g] == 2 else
                          "none: the p14r kappa^1 counterterm basis is empty (one-leg local functionals are
+> total "
                          "derivatives or need a dimension-5 field)"),
      "renormalization_conditions": "MS-bar at mu (P14 normalization condition 0) + N4 (no mu^4, mu^2 p^2 co
+> ntacts); "
                                    "no counterterm is required",
      "scaleless_analysis": {
          "SO_applications": led, "logarithmic_members": logarithmic,
          "UV_IR_reading": ("no member is logarithmic (omega = 0): the DR zero is NOT a 1/eps_UV - 1/eps_IR
+> "
                            "cancellation here; it discards pure power divergences (omega = "
                            + ", ".join(str(x["omega"]) for x in led) + "); the IR region converges at d = 4
+>  and "
                            "neither the UV nor the IR part has a pole at d = 4") if not logarithmic else
                           "logarithmic members present: (1/eps_UV - 1/eps_IR) cancellation"},
      "prescription_dependence": {
          "IR_regulator_mass": {
              "rule": "1/(l^2 + i0) -> 1/(l^2 - m^2 + i0) on the loop line (analysis only; m is not in the a
+> ction)",
              "pole_powers_of_m2": m_powers,
              "pole_examples": {c: str(mass_pole[c]) for c in ex},
              "finite_examples": {c: str(mass_fin[c]) for c in ex},
              "limit_m_to_0": "0 (pole ~ m^{2j}, finite ~ m^{2j}(a + b log m^2), j >= 1)" if lim0 else "CHEC
+> K",
              "counterterm_needed_at_finite_m": (
                  "poles ~ kappa^2 m^4 hh and kappa^2 m^2 p^2 hh: order-0/order-2 contacts, OUTSIDE the p14r
+>  h.h "
                  "basis (excluded by N4); they vanish as m -> 0" if ORDER[g] == 2 else
                  "pole ~ kappa m^4 h: a one-point (cosmological-type) contact, OUTSIDE the p14r basis (kapp
+> a^1 basis "
                  "empty); it vanishes as m -> 0")},
          "analytic_regulator": "(l^2)^{-1-nu}: the integrand stays scaleless for every nu, the value stays
+> 0; an "
                                "analytic regulator alone does not separate UV from IR here",
          "cutoff_split": "each member splits into IR (+Lambda^omega/omega...) and UV (-Lambda^omega/omega..
+> .) "
                          "pieces with no pole at d = 4; with a hard cutoff the would-be counterterms are "
                          + ("kappa^2 Lambda^4 hh and kappa^2 Lambda^2 p^2 hh (order 0/2: excluded by N4, no
+> t p14r "
                             "directions)" if ORDER[g] == 2 else
                             "kappa Lambda^4 h (one-point; no p14r kappa^1 direction)"),
          "p14r_counterterm_required": False},
      "verification_controls": {
          "export_sha_pinned_to_p14r_certificate": pin["pinned"],
          "independent_integrand_route_p14r": ("W6 Hessian/Tr-log route (P14_amplitude_rules W6_exports)"
                                               if g in ("DO2", "D12") else "W5 independent sympy regeneratio
+> n"),
          "route1_dred_lift_equals_route2_symmetric_integration": agree, "components_compared": n_cmp,
          "all_components_vanish": all_zero},
      "value_imported_from_XVI_O": False,
      "XVI_O_comparison": ("XVI-O declares the D12/one-point value 0 by dimensional_regularization_m0 (coinc
+> ident "
                           "prescription); p15 DERIVES the same vanishing under P's own declared prescriptio
+> n; "
                           "not imported" if g in ("D12", "D14c") else "n/a"),
  }
  return rec
```

```
def p14r_regeneration_checks():
    amp = json.loads((ROOT / "certificates" / "P14_amplitude_rules.json").read_text())
    w5 = amp["controls"]["W5_regeneration"]["checks"]
    w6 = amp["W6_exports"]
    return {"D02": bool(w6["D02"]["agree"]), "D12": bool(w6["D12"]["agree"]), "D14a": bool(w5["D14a"]),
            "D14b": bool(w5["D14b"]), "D14c": bool(w5["D14c"])}


def gate_checks(recs):
    """the Batch A acceptance conditions (re-checked by release_p15)."""
    regen = p14r_regeneration_checks()
    c = {}
    for g, r in recs.items():
        vc = r["verification_controls"]
        c[g] = bool(r["status"] == "vanishing_under_declared_prescription" and vc["export_sha_pinned_to_p14r_c
  +> ertificate"]
                    and vc["route1_dred_lift_equals_route2_symmetric_integration"] and vc["all_components_vani
  +> sh"]
                    and regen[g] and r["massless_declared"] and r["scaleless_analysis"]["S0_applications"]
                    and r["value_imported_from_XVI_O"] is False)
    c["D12_separate_from_one_point_slot"] = (recs["D12"]["kappa_order"] == 2
                                             and all(recs[g]["kappa_order"] == 1 for g in ("D14a", "D14b", "D1
  +> 4c")))
    return c


def main():
    import certio
    t0 = time.time()
    recs = {g: evaluate(g) for g in QC.BATCH_A}
    gc = gate_checks(recs)
    ok = all(gc.values())
    one_point = {"graphs": ["D14a", "D14b", "D14c"], "kappa_order": 1,
                 "status": "vanishing_under_declared_prescription" if all(
                     recs[g]["status"] == "vanishing_under_declared_prescription" for g in ("D14a", "D14b", "D
  +> 14c"))
                 else "regulated",
                 "note": "the order-kappa one-point slot; kept separate from D12 (kappa^2 hh)"}
    d12 = {"graph": "D12", "kappa_order": 2, "status": recs["D12"]["status"],
           "derivation": "certificates/P15_batchA.json graphs.D12 (route 1 frozen-dred lift + S0, route 2 symm
  +> etric "
                         "integration, IR-mass and cutoff analysis)",
           "previous_status": "Missing (p14r)"}
    batchA_evaluated = {
        "execution": ok,
        "claim": ("D02, D12, D14a, D14b, D14c evaluated under the declared prescription (frozen p14r DRED + MS
  +> -bar + "
                  "Feynman i0 + the explicit scaleless rule S0): each is vanishing_under_declared_prescription
  +>  with "
                  "no UV pole, finite part 0, no p14r counterterm required" if ok else "not established"),
        "scope": ("massless declared action; DR-specific (S0): with an IR-regulator mass or a cutoff the tadpo
  +> les "
                  "carry m^{2j}/Lambda^{omega} terms whose counterterms are order-0/2 contacts outside the p14
  +> r basis "
                  "(N4) and vanish as m -> 0; D02 is only one of the three hh graphs (D01, D03 not evaluated);
  +>  no "
                  "slot is complete; NOT quantum_inputs_evaluated")}
    rec = {"landmark": "XVI-P", "revision": QC.REVISION, "deliverable": "P.5 calculation Batch A (tadpoles)",
           "contract_revision": QC.CONTRACT_REVISION,
           "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 + S0 (certificates/P15_integral_library.json)
  +> ",
           "graphs": recs, "D12_slot": d12, "one_point_slot_kappa1": one_point, "gate_checks": gc,
           "batchA_evaluated": batchA_evaluated, "loop_evaluations_performed": True,
           "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_batchA.json", rec)
    print(json.dumps({"batchA_evaluated": ok, "gate": gc,
                      "status": {g: r["status"] for g, r in recs.items()},
                      "omegas": {g: sorted({x["omega"] for x in r["scaleless_analysis"]["S0_applications"]})
                                 for g, r in recs.items()},
                      "mass_pole_powers": {g: r["prescription_dependence"]["IR_regulator_mass"]["pole_powers_o
  +> f_m2"]
                                           for g, r in recs.items()},
                      "seconds": rec["seconds"]}, indent=1))
    return 0 if ok else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

130 Source: d13.py (p15 Batch B: D13)

[listing 97/133] code/p5_calc/d13.py

```
#!/usr/bin/env python3
"""XVI-P p15 Batch B: D13, the kappa^2 K'.c source coefficient (open ghost line + graviton line), fully evalua
  +> ted.

Gamma_1 contains  int K'_(a') G_{a' nu} c^nu ;  G_{a' nu}(p) = V_{Gamma_1}[K'_(a')(-p), c^nu(p)] (ordered extr
  +> action),
iGamma_1 integrand = prefactor * N / ((l^2 + i0)((p-l)^2 + i0))  =>  G = -i * prefactor * int N/(D1 D2).
Result convention:  G_{(mn) nu}(p) = kappa^2 * N_eps * Ghat_{(mn) nu}(p),  N_eps = i S_eps/(16 pi^2)  (integra
  +> ls.py).

Route 1  frozen export (sha-pinned) -> frozen dred Pipeline (R3 lift, R4 Gram/PV reduction in ghat, ledger) ->
  +>  J[a,b]
        via the library (S0 for every cancelled propagator) -> R1(d, p, Gt) * MB.
Route 2  integrand REGENERATED here from S_Psi (feynman.pw_vertex, own leg loop; rule table: S_f, loop sign, e
  +> xternal
        sign, K'/c leg order) -> Feynman parameters, l = k + x p, symmetric integration in QdS -> R2 * MB (no
  +>  S0).
Covariant form  Ghat = sum_i A_i(d) T_i + sum_j B_j(d) E_j  times MB, with
   T1 = eta_mn p_nu p^2, T2 = (eta_m nu p_n + eta_n nu p_m) p^2, T3 = p_m p_n p_nu      (4D structures)
   E1 = Gt_mn p_nu p^2,  E2 = (Gt_m nu p_n + Gt_n nu p_m) p^2                       (evanescent structures)
fitted EXACTLY in d on the faithful realization Gt = (4-d) M (integrals.realization), several p and M.
Laurent / MS-bar with d kept symbolic until the expansion (R5); premature d = 4 is caught (frozen exception cl
  +> ass).
"""
from __future__ import annotations
import itertools
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dred  # noqa: E402
import integrals as IL  # noqa: E402
import p15_conventions as QC  # noqa: E402

d, eps = dred.d, dred.eps
I = sp.I
ETA = dred.ETA
LS = sp.symbols("l0:4")
PS = sp.symbols("p0:4")
PP = IL.PP
MB = IL.MB
HC = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]
P_NUMS = [[sp.Integer(3), sp.Integer(1), sp.Integer(-2), sp.Integer(5)],
          [sp.Rational(7, 2), sp.Integer(-1), sp.Integer(4), sp.Integer(2)],
          [sp.Integer(2), sp.Rational(-3, 2), sp.Integer(1), sp.Integer(-1)]]
SEEDS = (11, 12)


# ============================================================================================== helpers
def eta(a, b):
    return ETA[a] if a == b else 0


def gt(a, b):
    return dred.Gt(a, b)


def pp_expr():
    return sum(ETA[r] * PS[r] ** 2 for r in range(4))


def structures():
    """component tensors (keyed (ap, nu)) of the 4D and evanescent structures (lower indices; p lower)."""
    T = {"T1": {}, "T2": {}, "T3": {}, "E1": {}, "E2": {}}
    for ap, (m, n) in enumerate(HC):
        for nu in range(4):
            k = (ap, nu)
            T["T1"][k] = eta(m, n) * PS[nu] * PP
```

```
            T["T2"][k] = (eta(m, nu) * PS[n] + eta(n, nu) * PS[m]) * PP
            T["T3"][k] = PS[m] * PS[n] * PS[nu]
            T["E1"][k] = gt(m, n) * PS[nu] * PP
            T["E2"][k] = (gt(m, nu) * PS[n] + gt(n, nu) * PS[m]) * PP
    return T


STRUCT_4D = ("T1", "T2", "T3")
STRUCT_EV = ("E1", "E2")


def key_of(s):
    a, b = s.split(",")
    return int(a), int(b)


# ============================================================================================== route 1
def load_export():
    import batchA
    return batchA.load_export("D13")


def route1(mutation=None, enforce=True):
    """frozen export -> frozen dred Pipeline -> {(ap,nu): R1 (coefficient of MB)} ; ledger."""
    body, pin = load_export()
    P = dred.Pipeline(mutation="premature_d4" if mutation == "premature_d4" else None, enforce=enforce)
    Jv = IL.J_values("D13", [(a, b) for a in range(3) for b in range(5)])
    out = {}
    for comp, rows in body["numerator"].items():
        tot = 0
        for names, cf in rows:
            lc = sorted(int(v[1]) for v in names if v.startswith("l"))
            pc = sp.Mul(*[sp.Symbol(v) for v in names if v.startswith("p")])
            tot += sp.Rational(cf) * pc * P.reduced_component(lc, with_p=True, tag="D13")
        tot = sp.expand(tot)
        if mutation == "drop_evanescent":
            tot = sp.expand(tot.xreplace({s: 0 for s in tot.free_symbols if s.name.startswith("Gt_")}))
        R = sp.expand(tot.xreplace(Jv))
        out[key_of(comp)] = sp.expand(R / MB) if R != 0 else sp.Integer(0)
    pref = sp.sympify(body["prefactor"], locals={"kappa": sp.Symbol("kappa")})
    norm = sp.simplify(-I * pref / sp.Symbol("kappa") ** 2)          # Ghat = norm * R1
    return {k: sp.expand(norm * v) for k, v in out.items()}, P.ledger, pin, str(pref)


# ============================================================================================== route 2 (re
  +> generation)
_VCACHE = {}


def _vertices():
    import feynman as F
    if "X" not in _VCACHE:
        _VCACHE["X"] = F.S_psi_parts()["X"]
        P = F.propagators()
        _VCACHE["Pinv"] = P["graviton"]["Pinv"]
        _VCACHE["Gh"] = P["ghost"]["numerator"]
        _VCACHE["F"] = F
    return _VCACHE


def mom(cl, cp):
    return [cl * LS[r] + cp * PS[r] for r in range(4)]


def regen_integrand(first="K", order=("K", "c", "h")):
    """N2[(a, nu)] = sum vk * <c cbar> * vg * <h h>  with the first vertex V[first(-p), c^mu(l), h_c(p-l)] tak
  +> en in the
    rule-table leg order; first = 'K' (D13) or 'cb' (ghost self-energy, same topology).  No exported data used
  +> ."""
    V = _vertices()
    F, X, Pinv, Gh = V["F"], V["X"][1], V["Pinv"], V["Gh"]
    ckey = ("first", first, tuple(order))
    if ckey in _VCACHE:
        return _VCACHE[ckey]
    comps_first = range(10) if first == "K" else range(4)
    vk = {}
    for a in comps_first:
        for mu in range(4):
            for c in range(10):
                if first == "K":
                    lg = {"K": {"f": "K", "c": HC[a], "k": mom(0, -1)}}
```

```
                else:
                    lg = {"K": {"f": "cb", "c": a, "k": mom(0, -1)}}
                lg["c"] = {"f": "c", "c": mu, "k": mom(1, 0)}
                lg["h"] = {"f": "h", "pol": {HC[c]: 1}, "k": mom(-1, 1)}
                v = F.pw_vertex(X, [lg[x] for x in order])
                if v != 0:
                    vk[(a, mu, c)] = v
    if "vg" not in _VCACHE:
        vg = {}
        for rho in range(4):
            for nu in range(4):
                for dd in range(10):
                    v = F.pw_vertex(X, [{"f": "cb", "c": rho, "k": mom(-1, 0)}, {"f": "c", "c": nu, "k": mom(0
  +> , 1)},
                                        {"f": "h", "pol": {HC[dd]: 1}, "k": mom(1, -1)}])
                    if v != 0:
                        vg[(rho, nu, dd)] = v
        _VCACHE["vg"] = vg
    vg = _VCACHE["vg"]
    gh = [(mu, rho, Gh[mu, rho]) for mu in range(4) for rho in range(4) if Gh[mu, rho] != 0]
    pv = [(c, dd, Pinv[c, dd]) for c in range(10) for dd in range(10) if Pinv[c, dd] != 0]
    N = {}
    for a in comps_first:
        for nu in range(4):
            tot = 0
            for mu, rho, g in gh:
                for c, dd, pval in pv:
                    x1 = vk.get((a, mu, c))
                    if x1 is None:
                        continue
                    x2 = vg.get((rho, nu, dd))
                    if x2 is None:
                        continue
                    tot += x1 * g * x2 * pval
            tot = sp.expand(tot)
            if tot != 0:
                N[(a, nu)] = tot
    _VCACHE[ckey] = N
    return N


def route2(S_f=1, loop_sign=1, ext_sign=1, order=("K", "c", "h"), first="K"):
    """Ghat2 = -i * pref2/kappa^2 * R2,  pref2 = ext * S_f * loop * (i kappa)^2 * i  (propagator i's: ghost nu
  +> merator
    already carries i (M0^{-1}) -> one explicit i for the graviton propagator, as in build_integrands)."""
    N = regen_integrand(first, order)
    pref_over_k2 = sp.Integer(ext_sign) * sp.nsimplify(S_f) * loop_sign * (I ** 2) * I
    norm = sp.expand(-I * pref_over_k2)
    out = {}
    for k, poly in N.items():
        R = IL.fp_two_point(poly, LS, PS)
        out[k] = sp.expand(norm * R / MB)
    return out


# ============================================================================================== fit / Laure
  +> nt
def _realized_rows(G, p_num, gm):
    S = structures()
    names = STRUCT_4D + STRUCT_EV
    rows, rhs = [], []
    for k in sorted(set(G) | set(S["T1"])):
        rows.append([IL.realize(S[nm][k], p_num, gm) for nm in names])
        rhs.append(IL.realize(G.get(k, 0), p_num, gm))
    return rows, rhs


def exact_fit(A, b, n):
    """exact solution of the overdetermined linear system A x = b (entries rational in d): n independent rows
  +> are
    selected at a generic numerical d, the n x n system is solved exactly, and EVERY row is verified exactly."
  +> ""
    dg = sp.Rational(173, 37)
    chosen, M = [], []
    for i, row in enumerate(A):
        cand = M + [[dred.substitute_d(x, dg, "d13.exact_fit: numerical rank test for row selection only (the
  +> solve "
                                           "and the verification of every row are exact in d)") for x in row]
  +> ]
        if sp.Matrix(cand).rank() > len(M):
            M, chosen = cand, chosen + [i]
```

```
        if len(chosen) == n:
            break
    if len(chosen) < n:
        return None, False
    As = sp.Matrix([A[i] for i in chosen])
    bs = sp.Matrix([b[i] for i in chosen])
    sol = [sp.factor(sp.cancel(x)) for x in As.LUsolve(bs)]
    ok = all(sp.cancel(sum(sp.sympify(a) * x for a, x in zip(row, sol)) - rhs) == 0 for row, rhs in zip(A, b))
    return sol, ok


def fit(G):
    """exact covariant fit; returns ({name: coefficient(d)}, residual_ok, n_equations)."""
    names = STRUCT_4D + STRUCT_EV
    A, b = [], []
    for pn in P_NUMS:
        for s in SEEDS:
            gm = IL.realization(pn, s)
            r, h = _realized_rows(G, pn, gm)
            A += r
            b += h
    sol, ok = exact_fit(A, b, len(names))
    return dict(zip(names, sol)), ok, len(b)


def expand_coeff(c):
    """A(d) * MB  ->  {pole, finite, nonlocal (coefficient of L), local}; d = 4 - 2 eps, MB = 1/eps + 2 - L +
  +> O(eps)."""
    ms = IL.master_series(1)
    mbs = ms[-1] / eps + ms[0] + ms[1] * eps
    a = dred.substitute_d(c, 4 - 2 * eps, "d13.expand_coeff: Laurent expansion after all contractions and the
  +> fit")
    ser = sp.series(sp.expand(a * mbs), eps, 0, 1).removeO()
    pole = sp.simplify(ser.coeff(eps, -1))
    fin = sp.expand(sp.simplify(ser.coeff(eps, 0)))
    nonloc = sp.expand(fin.coeff(IL.L))
    loc = sp.expand(fin - nonloc * IL.L)
    a4 = sp.simplify(dred.substitute_d(c, 4, "d13.expand_coeff: diagnostic A(4) (d-dependence proof only)"))
    da4 = sp.simplify(dred.substitute_d(sp.diff(sp.sympify(c), d), 4, "d13.expand_coeff: diagnostic A'(4)"))
    return {"exact_d": sp.factor(c), "A_at_4": a4, "dA_dd_at_4": da4, "pole": pole, "finite": fin,
            "nonlocal_L_coefficient": nonloc, "local": loc,
            "finite_if_d_set_to_4_first": sp.expand(a4 * (2 - IL.L)),
            "d_dependence_shift_of_local_part": sp.expand(loc - 2 * a4)}


# ============================================================================================== counterterm
  +>  basis (p14r)
def ck_basis():
    """the p14r c.K' counterterm directions as ordered two-point vertices V[K'_(a')(-p), c^nu(p)] (same conven
  +> tion
    as G), and their coordinates on T1..T3."""
    import counterterm_domain as CT
    import feynman as F
    dens = CT.sector_basis("c", "K", 3)
    keep, _ = CT.independent(dens, [("c", "K")])
    S = structures()
    basis = []
    for dn in keep:
        Vk = {}
        for ap in range(10):
            for nu in range(4):
                v = F.pw_vertex(dn, [{"f": "K", "c": HC[ap], "k": [-x for x in PS]}, {"f": "c", "c": nu, "k":
  +> list(PS)}])
                if v != 0:
                    Vk[(ap, nu)] = sp.expand(v.xreplace({}))
        # coordinates on T1..T3 (exact, symbolic p)
        cs = sp.symbols("c1:4")
        eqs = []
        for k in set(Vk) | set(S["T1"]):
            e = sp.expand(Vk.get(k, 0) - sum(c * S[nm][k].xreplace({PP: pp_expr()}) for c, nm in zip(cs, STRUC
  +> T_4D)))
            if e != 0:
                eqs += sp.Poly(e, *PS).coeffs()
        sol = sp.solve(eqs, cs, dict=True)
        basis.append({"density": str(dn)[:300], "T_coordinates": [sp.simplify(sol[0][c]) for c in cs] if sol e
  +> lse None})
    return basis


def to_ct_coordinates(Tcoef, basis):
    """express sum_i Tcoef_i T_i in the p14r c.K' basis: solve sum_k z_k basis_k = Tcoef."""
```

```
    M = sp.Matrix([[b["T_coordinates"][i] for b in basis] for i in range(3)])
    z = M.LUsolve(sp.Matrix(Tcoef))
    return [sp.simplify(x) for x in z], M.rank() == 3


# ============================================================================================== ghost equat
  +> ion (BRST sanity)
def deDonder_apply(G):
    """2 i D^rho[G_{..nu}] at K = p:  D^rho T = i p_s eta^{rr} eta^{ss} T_{r s} - 1/2 i eta^{rr} p_r tr T,
    T_mn = G_{(min,max) nu}, trace over the quasi-4D external index (R4/R5 external projection)."""
    out = {}
    idx = {mn: a for a, mn in enumerate(HC)}
    for rho in range(4):
        for nu in range(4):
            T = lambda m, n: G.get((idx[(min(m, n), max(m, n))], nu), 0)  # noqa: E731
            tr = sum(ETA[m] * T(m, m) for m in range(4))
            D = sum(I * PS[s] * ETA[rho] * ETA[s] * T(rho, s) for s in range(4)) - sp.Rational(1, 2) * I * ETA
  +> [rho] * PS[rho] * tr
            v = sp.expand(2 * I * D)
            if v != 0:
                out[(rho, nu)] = v
    return out


def compare_realized(G1, G2, keys=None):
    bad, n = [], 0
    for pn in P_NUMS[:2]:
        for s in SEEDS:
            gm = IL.realization(pn, s)
            for k in (keys or sorted(set(G1) | set(G2))):
                n += 1
                df = sp.cancel(IL.realize(G1.get(k, 0), pn, gm) - IL.realize(G2.get(k, 0), pn, gm))
                if df != 0:
                    bad.append({"comp": list(k), "diff": str(df)[:120]})
    return {"compared": n, "mismatches": bad[:4], "n_mismatches": len(bad), "agree": not bad}


# ============================================================================================== main evalua
  +> tion
def log(msg):
    print(f"[d13 {time.strftime('%H:%M:%S')}] {msg}", flush=True)


def evaluate():
    t0 = time.time()
    log("route 1")
    G1, ledger, pin, pref = route1()
    t1 = time.time()
    log("route 2")
    G2 = route2()
    t2 = time.time()
    log("compare")
    agree = compare_realized(G1, G2)
    log("fit")
    fit1, ok1, neq = fit(G1)
    fit2, ok2, _ = fit(G2)
    same_fit = all(sp.cancel(fit1[k] - fit2[k]) == 0 for k in fit1)
    log("laurent")
    exp = {k: expand_coeff(v) for k, v in fit1.items()}
    log("ck basis")
    basis = ck_basis()
    pole4 = [exp[k]["pole"] for k in STRUCT_4D]
    loc4 = [exp[k]["local"] for k in STRUCT_4D]
    nonl4 = [exp[k]["nonlocal_L_coefficient"] for k in STRUCT_4D]
    zpole, full = to_ct_coordinates(pole4, basis)
    zloc, _ = to_ct_coordinates(loc4, basis)
    znl, _ = to_ct_coordinates(nonl4, basis)
    ev_pole = {k: exp[k]["pole"] for k in STRUCT_EV}
    t3 = time.time()
    # ghost equation: evaluated ghost self-energy (route 2, cbar external) == 2 i D[G] (route 1)
    log("ghost equation")
    Sig = route2(first="cb")
    DG = deDonder_apply(G1)
    ghost_eq = compare_realized(Sig, DG)
    t4 = time.time()
    return {"G1": G1, "G2": G2, "fit": fit1, "fit2": fit2, "fit_exact": ok1 and ok2, "fit_equations": neq,
            "routes_agree": agree, "fits_agree": same_fit, "exp": exp, "basis": basis, "basis_full_rank": full
  +> ,
            "z_pole": zpole, "z_local": zloc, "z_nonlocal": znl, "ev_pole": ev_pole, "ledger": ledger, "pin":
  +> pin,
            "prefactor": pref, "ghost_equation": ghost_eq, "Sigma": Sig,
```

```
            "timing": {"route1": round(t1 - t0, 1), "route2": round(t2 - t1, 1), "fit_laurent_ct": round(t3 -
  +> t2, 1),
                       "ghost_eq": round(t4 - t3, 1)}}


def mutations(base):
    """each mutation must CHANGE the result and be REJECTED by a control."""
    out = {}
    ref = base["fit"]

    def differs(f):
        return any(sp.cancel(f[k] - ref[k]) != 0 for k in ref)
    # 1 wrong ghost-loop sign: a closed-ghost-loop (-1) applied to the OPEN ghost line of D13 (route 2 regener
  +> ated)
    Gm = route2(loop_sign=-1)
    fm, _, _ = fit(Gm)
    cmpm = compare_realized(base["G1"], Gm)
    out["wrong_ghost_loop_sign"] = {"changes_result": differs(fm), "rejected": not cmpm["agree"],
                                    "reason": f"route 1 (frozen export) != regenerated route 2: {cmpm['n_misma
  +> tches']} "
                                              "mismatches; W1 shadow: D13 has no closed ghost loop",
                                    "pole_T_if_accepted": [str(sp.simplify(dred.substitute_d(
                                        fm[k], 4, "d13.mutation display: pole"))) for k in STRUCT_4D]}
    # 2 dropped evanescent term via premature d = 4 (frozen Pipeline mutation) -> structural R5 safeguard
    #   (p15 contract correction: the opaque d raises AT the substitution; the p14r heuristic guard is withdra
  +> wn)
    try:
        route1(mutation="premature_d4")
        out["premature_d4"] = {"rejected": False, "changes_result": None}
    except dred.PrematureD4Substitution as exc:
        Gp, _, _, _ = route1(mutation="premature_d4", enforce=False)        # display only (authorised path)
        fp_, okp, _ = fit(Gp)
        exp_if = {k: expand_coeff(fp_[k]) for k in fp_}                   # authorised Laurent path (display)
        loc_if = [str(exp_if[k]["local"]) for k in STRUCT_4D]
        out["premature_d4"] = {"rejected": True, "changes_result": differs(fp_),
                               "reason": f"PrematureD4Substitution (structural, at the substitution site): {ex
  +> c}",
                               "fit_d_free_if_not_caught": not any(sp.sympify(v).has(d) for v in fp_.values())
  +> ,
                               "local_part_if_not_caught_T": loc_if,
                               "local_part_certified_T": [str(base["exp"][k]["local"]) for k in STRUCT_4D],
                               "note": ("the Gt realization Gt = (4-d) M reintroduces d after the premature "
                                        "substitution (the false-negative mechanism of finding F1); only the "
                                        "structural safeguard catches it"),
                               "pole_T_if_not_caught": [str(exp_if[k]["pole"]) for k in STRUCT_4D + STRUCT_EV]
  +> }
    # 3 dropped evanescent (Gt -> 0, i.e. ghat -> eta before the contraction)
    Gd, ledd, _, _ = route1(mutation="drop_evanescent")
    fd, _, _ = fit(Gd)
    cmpd = compare_realized(Gd, base["G2"])
    out["drop_evanescent_Gt"] = {"changes_result": differs(fd), "rejected": not cmpd["agree"],
                                 "reason": f"route 1 with Gt dropped != route 2 (ghat kept): {cmpd['n_mismatch
  +> es']} "
                                           "mismatches; R6 retention check fails",
                                 "fit_if_accepted": {k: str(v) for k, v in fd.items()}}
    # 4 wrong symmetry factor (S_f = 1/2 for D13)
    Gs = route2(S_f=sp.Rational(1, 2))
    fs, _, _ = fit(Gs)
    cmps = compare_realized(base["G1"], Gs)
    out["wrong_symmetry_factor"] = {"changes_result": differs(fs), "rejected": not cmps["agree"],
                                    "reason": f"route 1 != route 2 with S_f = 1/2: {cmps['n_mismatches']} mism
  +> atches"}
    # 5 wrong K'/c ordering (rule-table leg order (c, K', h))
    Go = route2(order=("c", "K", "h"))
    fo, _, _ = fit(Go)
    cmpo = compare_realized(base["G1"], Go)
    out["wrong_K_c_ordering"] = {"changes_result": differs(fo), "rejected": not cmpo["agree"],
                                 "reason": f"route 1 != route 2 with leg order (c, K', h): {cmpo['n_mismatches
  +> ']} "
                                           "mismatches"}
    for v in out.values():
        v["accepted_as_rejection"] = bool(v["rejected"] and v["changes_result"])
    return out


SIG_DESC = {"S1": "delta^rho_nu p^4", "S2": "p^rho p_nu p^2", "SE": "Gt^rho_nu p^4 (evanescent)"}


def sigma_structures(rho, nu):
    """mixed-index structures of Sigma^rho_nu (upper rho = eta^{rho rho} x lower; the ghost equation 2 i D^rho
  +> [G]
```

```
    carries the upper index)."""
    f = ETA[rho]
    return [f * eta(rho, nu) * PP ** 2, f * PS[rho] * PS[nu] * PP, f * gt(rho, nu) * PP ** 2]


def ccb_basis():
    """the p14r c.cb counterterm directions (CT.sector_basis('cb','c',4)) as two-point vertices V[cb_rho(-p),
    c^nu(p)] and their exact coordinates on S1, S2."""
    import counterterm_domain as CT
    import feynman as F
    dens = CT.sector_basis("cb", "c", 4)
    keep, _ = CT.independent(dens, [("cb", "c")])
    out = []
    for dn in keep:
        cs = sp.symbols("s1:3")
        eqs = []
        for rho in range(4):
            for nu in range(4):
                v = F.pw_vertex(dn, [{"f": "cb", "c": rho, "k": [-x for x in PS]}, {"f": "c", "c": nu, "k": li
  +> st(PS)}])
                st = sigma_structures(rho, nu)
                e = sp.expand(v - sum(c * x.xreplace({PP: pp_expr()}) for c, x in zip(cs, st[:2])))
                if e != 0:
                    eqs += sp.Poly(e, *PS).coeffs()
        sol = sp.solve(eqs, cs, dict=True)
        out.append({"density": str(dn)[:300], "S_coordinates": [sp.simplify(sol[0][c]) for c in cs] if sol els
  +> e None})
    return out


def sigma_pole_in_ccb(Sig):
    """the ghost self-energy (route 2, cbar external) fitted exactly on S1, S2, SE; its 4D pole expressed in t
  +> he p14r
    c.cb directions; the evanescent pole recorded separately."""
    A, b = [], []
    for pn in P_NUMS:
        for sd in SEEDS:
            gm = IL.realization(pn, sd)
            for rho in range(4):
                for nu in range(4):
                    A.append([IL.realize(x, pn, gm) for x in sigma_structures(rho, nu)])
                    b.append(IL.realize(Sig.get((rho, nu), 0), pn, gm))
    sol, ok = exact_fit(A, b, 3)
    if sol is None:
        return {"fit_exact": False, "pole_local_and_in_p14r_c_cb_span": False}
    ex = {nm: expand_coeff(c) for nm, c in zip(("S1", "S2", "SE"), sol)}
    basis = ccb_basis()
    full = all(bb["S_coordinates"] is not None for bb in basis) and len(basis) == 2
    z = None
    if full:
        M = sp.Matrix([[bb["S_coordinates"][i] for bb in basis] for i in range(2)])
        full = M.rank() == 2
        if full:
            z = [sp.simplify(x) for x in M.LUsolve(sp.Matrix([ex["S1"]["pole"], ex["S2"]["pole"]]))]
    return {"fit_exact": ok, "structures": SIG_DESC,
            "coefficients_exact_d": {k: str(v["exact_d"]) for k, v in ex.items()},
            "pole": {k: str(v["pole"]) for k, v in ex.items()},
            "local": {k: str(v["local"]) for k, v in ex.items()},
            "p14r_c_cb_basis": [{"density": bb["density"], "S_coordinates": [str(c) for c in bb["S_coordinates
  +> "]]}
                                for bb in basis],
            "pole_in_p14r_c_cb_coordinates": [str(x) for x in z] if z else None,
            "evanescent_pole_SE": str(ex["SE"]["pole"]),
            "pole_local_and_in_p14r_c_cb_span": bool(ok and full)}


def s(x):
    return str(sp.simplify(x)) if not isinstance(x, str) else x


def _opaque_d_refuses(fit1):
    """the structural R5 safeguard (p15 contract correction): any substitution of d outside dred.substitute_d
  +> raises."""
    try:
        sp.sympify(fit1["T2"]).subs(d, 4)
        return False
    except dred.PrematureD4Substitution:
        return True


def main():
```

```
  import certio
  t0 = time.time()
  IL.LEDGER.clear()
  base = evaluate()
  s0 = list(IL.LEDGER)
  log("mutations")
  muts = mutations(base)
  log("sigma pole")
  sig = sigma_pole_in_ccb(base["Sigma"])
  exp = base["exp"]
  ev_pole_nonzero = any(exp[k]["pole"] != 0 for k in STRUCT_EV)
  d_matters = any(exp[k]["d_dependence_shift_of_local_part"] != 0 for k in STRUCT_4D)
  controls = {
      "export_sha_pinned": base["pin"]["pinned"],
      "route1_equals_route2_exact_in_d": base["routes_agree"]["agree"],
      "covariant_fit_exact": base["fit_exact"], "fits_agree": base["fits_agree"],
      "pole_local_in_p14r_cK_span": base["basis_full_rank"],
      "ghost_equation_2iD_G_equals_ghost_self_energy": base["ghost_equation"]["agree"],
      "ghost_self_energy_pole_in_p14r_c_cb_span": sig["pole_local_and_in_p14r_c_cb_span"],
      "nonlocal_equals_minus_pole": all(sp.simplify(exp[k]["nonlocal_L_coefficient"] + exp[k]["pole"]) == 0
                                        for k in exp),
      "R5_d_retained": any(sp.sympify(v).has(d) for v in base["fit"].values()),
      "R5_structural_safeguard_active": _opaque_d_refuses(base["fit"]),
      "mutations_all_rejected": all(v["accepted_as_rejection"] for v in muts.values()),
  }
  ok = all(controls.values())
  T_desc = {"T1": "eta_mn p_nu p^2", "T2": "(eta_m nu p_n + eta_n nu p_m) p^2", "T3": "p_m p_n p_nu",
            "E1": "Gt_mn p_nu p^2", "E2": "(Gt_m nu p_n + Gt_n nu p_m) p^2"}
  result = {k: {"structure": T_desc[k], "A_exact_d_times_MB": s(exp[k]["exact_d"]), "UV_pole": s(exp[k]["pol
+> e"]),
                "MSbar_finite": s(exp[k]["finite"]), "nonlocal_coefficient_of_L": s(exp[k]["nonlocal_L_coeff
+> icient"]),
                "local_polynomial": s(exp[k]["local"]),
                "finite_if_d_set_to_4_first": s(exp[k]["finite_if_d_set_to_4_first"]),
                "local_shift_from_d_dependence": s(exp[k]["d_dependence_shift_of_local_part"])}
            for k in STRUCT_4D + STRUCT_EV}
  # graph status (one of QC.STATUS_VALUES): 'renormalized' only if EVERY pole is absorbed by a declared coun
+> terterm.
  # The p14r contract declares no evanescent counterterm, so a nonzero evanescent pole keeps the graph at
  # 'regulated'; the 4D sector is reported separately.
  status = "renormalized" if ok and not ev_pole_nonzero else "regulated"
  assert status in QC.STATUS_VALUES
  sector_status = {"4D_structures_T1_T3": "renormalized" if ok else "regulated",
                   "evanescent_structures_E1_E2": ("regulated: pole nonzero (E2), recorded and retained (R6)
+> ; its "
                                                   "counterterm is an evanescent K'.c operator NOT in the p1
+> 4r 4D "
                                                   "basis (the p14r contract declares no evanescent countert
+> erm: "
                                                   "open contract item, not absorbed)") if ev_pole_nonzero e
+> lse
                                                  "pole zero"}
  D13_evaluated = {
      "execution": ok,
      "claim": ("D13 (kappa^2 K'.c) evaluated in closed form under the declared prescription: the UV pole of
+>  the 4D "
                "structures is local and in the span of the three p14r c.K' directions (the evanescent E2 po
+> le is "
                "recorded, not absorbed), MS-bar finite part = nonlocal log(-p^2/mu^2) part + "
                "local cubic polynomial, two independent routes agree exactly in d, ghost equation verified,
+>  "
                "mutations rejected") if ok else "not established",
      "scope": ("one graph (D13) of the K'.c slot at kappa^2; MS-bar at mu with N_eps = i S_eps/(16 pi^2); e
+> xternal "
                "momenta in QdS (R2); evanescent Gt structures retained "
                + ("(their pole is NONZERO: an evanescent K'.c counterterm outside the p14r 4D basis is need
+> ed for "
                   "full DRED renormalization)" if ev_pole_nonzero else "(their pole vanishes)")
                + "; NOT quantum_inputs_evaluated; the slot's identity role is not evaluated (Batch C, D11 M
+> 2)")}
  rec = {"landmark": "XVI-P", "revision": QC.REVISION, "deliverable": "P.5 calculation Batch B: D13 (K'.c, k
+> appa^2)",
         "contract_revision": QC.CONTRACT_REVISION,
         "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 + S0 (certificates/P15_integral_library.json)
+> ",
         "convention": ("G_{(mn) nu}(p) = V_Gamma1[K'_(mn)(-p), c^nu(p)] = kappa^2 * (i S_eps/(16 pi^2)) * G
+> hat, "
                        "Ghat = sum A_i(d) T_i * MB, MB = Ihat(1,1) = 1/eps + 2 - L + O(eps), "
                        "L = log(-p^2/mu^2 - i0); indices lower (K' slot dual to h_(mn), c^nu -> lower nu);
+> "
                        "p_r lower components; p^2 = eta^{rs} p_r p_s"),
```

```
          "export": base["pin"], "prefactor_export": base["prefactor"],
          "status": status, "sector_status": sector_status,
          "evanescent_counterterm_declared_in_p14r": False, "result": result,
          "UV_pole_in_p14r_cK_basis": {"coordinates": [s(x) for x in base["z_pole"]],
                                       "basis": [{"density": b["density"], "T_coordinates": [s(c) for c in b[
  +> "T_coordinates"]]}
                                                 for b in base["basis"]]},
          "local_polynomial_in_p14r_cK_basis": [s(x) for x in base["z_local"]],
          "nonlocal_in_p14r_cK_basis": [s(x) for x in base["z_nonlocal"]],
          "MSbar_counterterm": ("delta S = - kappa^2 (i S_eps/(16 pi^2)) (1/eps) * sum_k z_k O_k  (O_k the p1
  +> 4r c.K' "
                                "densities, z = UV_pole coordinates); plus, for the evanescent structures, "
                                + ("an evanescent counterterm (Gt_mn p_nu p^2, (Gt p + Gt p) p^2) NOT in the
  +> p14r basis"
                                   if ev_pole_nonzero else "nothing (pole zero)")),
          "evanescent_pole_nonzero": ev_pole_nonzero,
          "local_ambiguity": ("finite renormalization along the three p14r c.K' directions: G^R -> G^R + kapp
  +> a^2 "
                              "(i/(16 pi^2)) sum_k f_k O_k; a change of scale mu -> mu' shifts the local poly
  +> nomial by "
                              "+ A_i(4) log(mu'^2/mu^2) along the UV-pole direction"),
          "renormalization_conditions": ("MS-bar at mu: counterterm = pure pole (P14 normalization condition
  +> 0); "
                                         "no lower-degree K'.c contact (mu^2 K' dc) is generated (no scale bu
  +> t mu; pole "
                                         "is cubic in p); u^R_mu (XVI-O) not modified; an alternative momentu
  +> m "
                                         "subtraction at p^2 = -M^2 differs by the finite renormalization "
                                         "A_i(4) log(M^2/mu^2) - (local MS-bar polynomial) along the same dir
  +> ections"),
          "absorptive_part": "for timelike p^2 > 0: L = log(p^2/mu^2) - i pi, Im Ghat = + pi * sum A_i(4) T_i
  +> ",
          "d_dependence": {"evanescent_d_terms_matter": bool(d_matters or ev_pole_nonzero),
                           "local_shift_from_d_dependence_T": {k: result[k]["local_shift_from_d_dependence"]
                                                               for k in STRUCT_4D},
                           "evanescent_structure_poles": {k: result[k]["UV_pole"] for k in STRUCT_EV},
                           "statement": ("the (d-4) x pole terms change the local finite polynomial (prematur
  +> e d = 4 "
                                         "gives a different MS-bar finite part); the difference is local and
  +> lies in "
                                         "the p14r c.K' span (a scheme-type finite renormalization), so it ma
  +> tters "
                                         "for the finite part in the declared scheme" if d_matters else
                                         "the (d-4) x pole terms vanish for the 4D structures")},
          "authorised_d_substitutions": dict(dred.D_PROVENANCE),
          "independent_cross_check_route": ("route 2: integrand regenerated from the rule tables (feynman.pw_
  +> vertex; "
                                            "no exported data) and evaluated by Feynman parameters with d sym
  +> bolic "
                                            "(integrals.fp_two_point; no Gram/PV reduction, no S0); agrees wi
  +> th route "
                                            "1 (frozen export -> dred Pipeline) exactly in d"),
          "scaleless_rule_applications": s0, "dred_ledger_reductions_introducing_d": sum(1 for x in base["led
  +> ger"]
                                                                                         if x.get("introduces
  +> _d")),
          "controls": controls,
          "route_comparison": base["routes_agree"], "fit_equations": base["fit_equations"],
          "ghost_equation": {**base["ghost_equation"],
                             "statement": "Sigma_{rho nu}(p) [one-loop ghost self-energy, same topology, eval
  +> uated "
                                          "by route 2 from the cbar.c.h vertex] == 2 i D^rho_p[G_{.. nu}] wit
  +> h "
                                          "D^rho T = d_s T^{rho s} - 1/2 d^rho tr T (linear de Donder gauge:
  +> ghost "
                                          "equation); exact in d on the faithful Gt realization"},
          "ghost_self_energy_pole": sig,
          "mutations": muts, "mutations_all_rejected": controls["mutations_all_rejected"],
          "D13_evaluated": D13_evaluated, "loop_evaluations_performed": True,
          "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_D13.json", rec)
    print(json.dumps({"D13_evaluated": ok, "status": status, "controls": controls,
                      "pole_T": {k: result[k]["UV_pole"] for k in result},
                      "local_T": {k: result[k]["local_polynomial"] for k in result},
                      "z_pole": rec["UV_pole_in_p14r_cK_basis"]["coordinates"],
                      "mutations": {k: [v["changes_result"], v["rejected"]] for k, v in muts.items()},
                      "timing": base["timing"], "seconds": rec["seconds"]}, indent=1))
    return 0 if ok else 1


if __name__ == "__main__":
```

```
    raise SystemExit(main())
```

131 Source: release_p15.py (p15 release gate)

[listing 98/133] code/p5_calc/release_p15.py

```
#!/usr/bin/env python3
"""XVI-P p15 release gate: FIRST P.5 CALCULATION (Batch A: D02, D12, D14a-c; Batch B: D13) on the p14r contrac
  +> t.

Runs AFTER the p14r contract gate (code/p5_entry/release_p14.py, which writes certificates/P14_release.json) a
  +> nd
requires it to be accepted.  Recomputes (never trusts a stored Boolean alone):
  contract layer : P14_release.json accepted (classical claim, P3/P4 freeze, five P.5-entry flags, contract ha
  +> rd stops);
  contract identity : every p14r contract file byte-identical to its p14r sha256 EXCEPT the disclosed correcti
  +> on
                  (QC.DISCLOSED_CONTRACT_CORRECTIONS: code/p5_entry/dred.py, finding F1), whose regression to
  +> ys pass;
  library        : P15_integral_library.json (A0, B0, tensor reduction in ghat, S0) certified;
  batchA_evaluated : P15_batchA.json (status per graph, two routes, gate checks, D12 separate from the one-poi
  +> nt slot);
  D13_evaluated  : P15_D13.json (two routes exact in d, exact fit, ghost equation, c.K' / c.cb maps, 5 mutatio
  +> ns);
  hard stops     : QC.HARD_STOPS_P15 (quantum_inputs_evaluated etc. NOT earned; O.1 NOT_CLOSED; insertion froz
  +> en ...).
Rebuild flags (package evidence, not part of release_claim_accepted): author/independent for the CURRENT (p15)
  +>  tree;
independent_rebuild_passed_p14r from the archived p14r report, bound to the p14r tree/zip (same-box caveat).
Publication: P15_release.json (+ RELEASE_STATUS.json, ledgers/P15_milestones.json) only if accepted; otherwise
P15_release_rejected.json and exit 1.
"""
from __future__ import annotations
import hashlib, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p15_conventions as QC  # noqa: E402
import p14_conventions as PC  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)

ARCH_P14R = "archival/p14r"
ARCH_P14R_INDEP = f"{ARCH_P14R}/reproducibility/independent_rebuild_report_p14r.json"
D13_MUTATIONS = ("wrong_ghost_loop_sign", "premature_d4", "drop_evanescent_Gt", "wrong_symmetry_factor",
                 "wrong_K_c_ordering")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def contract_layer():
    r = jl("certificates/P14_release.json")
    ok = bool(r.get("revision") == PC.REVISION == QC.CONTRACT_REVISION and r.get("publication") == "accepted"
              and r.get("release_claim_accepted") is True and r.get("P5_entry_earned") is True
              and r.get("P3_P4_frozen_as_P5_inputs") is True and r.get("hard_stops_hold") is True
              and all(r.get("P5_entry_flags", {}).values()) and len(r.get("P5_entry_flags", {})) == 5
              and (ROOT / "certificates" / "P14_closure_release.json").is_file())
    return ok, r


def contract_identity():
    cur = {p.relative_to(ROOT).as_posix(): sha(p) for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    changed = sorted(k for k, h in QC.P14R_CONTRACT_SHA256.items() if cur.get(k) != h)
    added = sorted(set(cur) - set(QC.P14R_CONTRACT_SHA256))
    disclosed = sorted(QC.DISCLOSED_CONTRACT_CORRECTIONS)
    dr = jl("certificates/P14_dred_rules.json")
    corr = dr.get("p15_contract_correction", {})
```

```
    toys = corr.get("regression_toys", {})
    toys_ok = bool(toys.get("pass") is True and toys.get("eta_trace_correct", {}).get("raised") is False
                   and toys.get("eta_trace_premature_d4", {}).get("raised") is True
                   and toys.get("A0_mixing", {}).get("mixed_refused") is True
                   and toys.get("B0_mixing", {}).get("mixed_refused") is True)
    dred_ok = bool(dr.get("dred_rules_verified") is True and dr.get("mutations_all_rejected") is True
                   and dr.get("mutations", {}).get("trace_ghat_eq_4", {}).get("structurally_refused") is True
                   and corr.get("released_p14r_values_affected") is False)
    ok = bool(changed == disclosed and not added and toys_ok and dred_ok)
    return ok, {"p14r_contract_files": len(QC.P14R_CONTRACT_SHA256), "changed": changed, "added": added,
                "disclosed_corrections": QC.DISCLOSED_CONTRACT_CORRECTIONS,
                "byte_identical_except_disclosed": changed == disclosed and not added,
                "regression_toys_pass": toys_ok, "dred_certificate_ok": dred_ok,
                "regression_toys": toys, "test_revisions_disclosed": QC.DISCLOSED_TEST_REVISIONS,
                "P3_P4_code": "frozen p13 classical code byte-identical (freeze F2, checked by the p14r contra
  +> ct gate)"}


def p14r_independent():
    f = ROOT / ARCH_P14R_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P14R_INDEP, "present": False}
    rep = json.loads(f.read_text())
    live = ROOT / QC.P14R_INDEPENDENT_REPORT
    rec = {"report": ARCH_P14R_INDEP, "present": True, "report_sha256": sha(f),
           "report_sha256_pinned": sha(f) == QC.P14R_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "bound_to_source_tree_sha256": rep.get("source_tree_sha256"),
           "bound_to_source_zip_sha256": rep.get("zip_sha256"),
           "tree_matches_p14r": rep.get("source_tree_sha256") == QC.P14R_SOURCE_TREE_SHA256,
           "zip_matches_p14r": rep.get("zip_sha256") == QC.P14R_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])), "findings": [x.get("id") for x in rep.get("findings",
  +>  [])],
           "caveat": rep.get("same_box_caveat"),
           "live_report_identical_to_archived": (live.is_file() and live.read_bytes() == f.read_bytes())
           if live.is_file() else "live report not in this tree",
           "NOT_inherited_by_p15": "bound to the p14r tree/zip; the p15 tree has its own (pending) independent
  +>  flag"}
    rec["archived_p14r_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                           for p, h in QC.P14R_ARCHIVED_RECORDS.items()}
    ok = bool(rec["report_sha256_pinned"] and all(rec["archived_p14r_records_pinned"].values()) and rep.get("p
  +> ass") is True and rep.get("independent_rebuild_passed") is True
            and rec["tree_matches_p14r"] and rec["zip_matches_p14r"] and rec["mismatches"] == 0
            and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec


def library():
    c = jl(f"certificates/{QC.CERT_PREFIX}_integral_library.json")
    return bool(c.get("library_certified") is True and c.get("revision") == QC.REVISION), c


def batch_a():
    c = jl(f"certificates/{QC.CERT_PREFIX}_batchA.json")
    if not c:
        return False, {}, c
    g = c.get("graphs", {})
    per = {k: {"status": g.get(k, {}).get("status"), "UV_pole": g.get(k, {}).get("UV_subtraction"),
               "finite_part": g.get(k, {}).get("finite_part"),
               "routes_agree": g.get(k, {}).get("verification_controls", {}).get(
                   "route1_dred_lift_equals_route2_symmetric_integration")} for k in QC.BATCH_A}
    ok = bool(c.get("revision") == QC.REVISION and set(g) == set(QC.BATCH_A)
              and all(v["status"] in QC.STATUS_VALUES and v["status"] != "unevaluated" for v in per.values())
              and all(v["routes_agree"] is True for v in per.values())
              and all(c.get("gate_checks", {}).values()) and c.get("gate_checks", {}).get("D12_separate_from_o
  +> ne_point_slot")
              and c.get("batchA_evaluated", {}).get("execution") is True
              and all(g[k].get("value_imported_from_XVI_0") is False for k in g))
    return ok, per, c


def d13():
    c = jl(f"certificates/{QC.CERT_PREFIX}_D13.json")
    if not c:
        return False, c
    muts = c.get("mutations", {})
    ok = bool(c.get("revision") == QC.REVISION and c.get("status") in QC.STATUS_VALUES
              and all(v is True for v in c.get("controls", {}).values()) and len(c.get("controls", {})) >= 10
              and sorted(muts) == sorted(D13_MUTATIONS)
              and all(v.get("rejected") is True and v.get("changes_result") is True for v in muts.values())
              and c.get("D13_evaluated", {}).get("execution") is True)
```

```
    return ok, c


def milestones(contract, a_ok, b_ok):
    st = dict(contract.get("milestones", {}).get("status", {}))
    st["P.5_calculation"] = ("opened (p15: Batch A D02/D12/D14a-c and Batch B D13 evaluated under the p14r "
                             "prescription; Batch C D01/D03 and the D11 matching M2 NOT evaluated)"
                             if a_ok and b_ok else "open (p15 evaluation not accepted)")
    st["P.5_identity"] = "open"
    return {"definitions": contract.get("milestones", {}).get("definitions"),
            "order": contract.get("milestones", {}).get("order"), "status": st,
            "P5_entry_earned": contract.get("P5_entry_earned"),
            "P3_P4_frozen_as_P5_inputs": contract.get("P3_P4_frozen_as_P5_inputs"),
            "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified
  +>  not earned)"}


def results_table(per, dc):
    rows = []
    for k in QC.BATCH_A:
        rows.append(f"{k} & {per[k]['status'].replace('_', chr(92) + '_')} & 0 & 0 \\\\")
    r = dc.get("result", {})
    pole = ", ".join(f"{k}: {r[k]['UV_pole']}" for k in ("T1", "T2", "T3", "E1", "E2") if k in r)
    loc = ", ".join(f"{k}: {r[k]['local_polynomial']}" for k in ("T1", "T2", "T3", "E1", "E2") if k in r)
    rows.append(f"D13 & {dc.get('status', '?')} (4D: renormalized) & {pole} & local {loc} \\\\")
    return ("% generated by code/p5_calc/release_p15.py -- do not edit\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{1.1cm}p{4.2cm}p{4.3cm}p{4.3cm}}\\hline\n"
            "graph & status & UV pole (units $\\kappa^2 N_\\epsilon/\\epsilon$) & finite part \\\\\\hline\n"
            + "\n".join(rows) + "\n\\hline\\end{tabular}\\end{center}\n")


def _main():
    c_ok, contract = contract_layer()
    id_ok, ident = contract_identity()
    lib_ok, lib = library()
    a_ok, per, ac = batch_a()
    b_ok, dc = d13()
    p14r_ok, p14r_rec = p14r_independent()
    flags = {
        "library_certified": lib_ok,
        "batchA_evaluated": {"execution": a_ok, "claim": ac.get("batchA_evaluated", {}).get("claim") if a_ok e
  +> lse
                             "not established", "scope": ac.get("batchA_evaluated", {}).get("scope")},
        "D13_evaluated": {"execution": b_ok, "claim": dc.get("D13_evaluated", {}).get("claim") if b_ok else
                          "not established", "scope": dc.get("D13_evaluated", {}).get("scope")},
    }
    d12 = ac.get("D12_slot", {})
    D12_status = d12.get("status") if (a_ok and d12.get("derivation")) else "Missing"
    hard = dict(QC.HARD_STOPS_P15)
    certs_claim = [jl(f"certificates/{QC.CERT_PREFIX}_{n}.json") for n in ("integral_library", "batchA", "D13"
  +> )]
    hard_ok = bool(all(hard[k] == v for k, v in QC.HARD_STOPS_P15.items())
                   and not any(c.get("quantum_inputs_evaluated") not in (None, "not_earned") for c in certs_cl
  +> aim)
                   and set(per) == set(QC.BATCH_A) and not set(QC.NOT_EVALUATED_P15) & set(per))
    execution_pass = bool(c_ok and id_ok and lib_ok and a_ok and b_ok and hard_ok)
    release_claim_accepted = execution_pass
    publish = release_claim_accepted
    author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
    indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
    m = milestones(contract, a_ok, b_ok)
    graphs = {k: {**per[k], "kappa_order": ac["graphs"][k]["kappa_order"], "slot": ac["graphs"][k]["slot"]}
              for k in QC.BATCH_A} if a_ok else {}
    if b_ok:
        graphs["D13"] = {"status": dc["status"], "sector_status": dc["sector_status"],
                         "UV_pole": {k: v["UV_pole"] for k, v in dc["result"].items()},
                         "MSbar_finite": {k: v["MSbar_finite"] for k, v in dc["result"].items()},
                         "UV_pole_in_p14r_cK_basis": dc["UV_pole_in_p14r_cK_basis"]["coordinates"],
                         "evanescent_pole_nonzero": dc["evanescent_pole_nonzero"]}
    for k in QC.NOT_EVALUATED_P15:
        graphs[k] = {"status": "unevaluated"}
    cert = {
        "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
  +> TE,
        "deliverable": "P.5 calculation, first batch (Batch A: D02, D12, D14a-c; Batch B: D13)",
        "execution_pass": execution_pass,
        "claim_status": ("P.5_calculation: Batch A and D13 evaluated under the p14r prescription; P.5_entry: e
  +> arned "
                         "(p14r contract); classical_readiness (frozen P.4): earned") if execution_pass else
                        "P.5_calculation: not accepted",
        "scope": ("first P.5 calculation: the Batch A graphs (D02, D12, D14a-c) and the Batch B graph D13 eval
```

```
+> uated "
                "under the frozen p14r prescription (DRED R1-R6, Feynman +i0, MS-bar) with the explicit scal
+> eless "
                "rule S0; D01, D03, D11 NOT evaluated; no slot of the identity is complete; NOT quantum inpu
+> ts, NOT "
                "the projected defect/identity, NOT bulk restoration, NOT the insertion"),
      "release_claim_accepted": release_claim_accepted,
      "release_claim_accepted_definition": ("p14r contract accepted (P14_release.json) AND contract byte-ide
+> ntity "
                                            "except the disclosed dred.py correction (regression toys pass)
+> AND "
                                            "library certified AND batchA_evaluated AND D13_evaluated AND p1
+> 5 hard "
                                            "stops"),
      "publication": "accepted" if publish else "REJECTED",
      "flags": flags, "batchA_evaluated": a_ok, "D13_evaluated": b_ok, "library_certified": lib_ok,
      "graphs": graphs, "D12_status": D12_status,
      "D12_status_derivation": d12.get("derivation") if D12_status != "Missing" else None,
      "D12_previous_status": "Missing (p14r)",
      "one_point_slot_kappa1": ac.get("one_point_slot_kappa1"),
      "D13_summary": {k: dc.get(k) for k in ("status", "sector_status", "convention", "UV_pole_in_p14r_cK_ba
+> sis",
                                             "local_polynomial_in_p14r_cK_basis", "MSbar_counterterm",
                                             "evanescent_pole_nonzero", "evanescent_counterterm_declared_in
+> _p14r",
                                             "local_ambiguity", "renormalization_conditions", "controls")},
      "contract_identity": ident, "contract_correction_disclosed": True,
      "contract_layer": {"certificate": "certificates/P14_release.json", "accepted": c_ok,
                         "revision": contract.get("revision"),
                         "P5_entry_earned": contract.get("P5_entry_earned"),
                         "P5_entry_flags": contract.get("P5_entry_flags"),
                         "P3_P4_frozen_as_P5_inputs": contract.get("P3_P4_frozen_as_P5_inputs"),
                         "contract_hard_stops": contract.get("hard_stops"),
                         "note": ("the contract-layer statements (e.g. D12_status Missing, P5_calculation_op
+> ened "
                                  "false) describe the p14r CONTRACT and are superseded for p15 by this stat
+> us; the "
                                  "contract record's rebuild flags refer to the current tree through the p14
+> r "
                                  "report paths and are superseded by the p15 flags below")},
      "P5_entry_earned": contract.get("P5_entry_earned"), "P5_entry_flags": contract.get("P5_entry_flags"),
      "P3_P4_frozen_as_P5_inputs": contract.get("P3_P4_frozen_as_P5_inputs"),
      "classical": contract.get("classical"),
      "D11_matching_verified": False,
      "hard_stops": hard, "hard_stops_hold": hard_ok,
      "milestones": m,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p14r": p14r_ok,
      "independent_rebuild_passed_p13": contract.get("independent_rebuild_passed_p13"),
      "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p15
+> ) source "
                             "tree only; independent_rebuild_passed_p14r refers to the p14r tree "
                             f"{QC.P14R_SOURCE_TREE_SHA256} / zip {QC.P14R_SOURCE_ZIP_SHA256} (same-box cave
+> at) and "
                             "is not inherited by p15; independent_rebuild_passed_p13 carries the P.3/P.4 fr
+> eeze"),
      "p14r_independent_rebuild": p14r_rec,
      "p13_independent_rebuild": contract.get("p13_independent_rebuild"),
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p14r_archive": ARCH_P14R,
                          "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"},
      "XVI_P_registered": False, "XVI_M_sha256": contract.get("XVI_M_sha256"), "XVI_M_pin_ok": contract.get(
+> "XVI_M_pin_ok"),
      "symmetry_restored": False, "bulk_symmetry_restored": False, "O1_closure": "NOT_CLOSED",
      "insertion_status": "Frozen_negative_obstruction_preserved", "XV_B_opened": False, "XIV_J_opened": Fal
+> se,
      "Route_B_discharged": False,
      "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
          n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
                 ("P14_release.json", "P14_dred_rules.json", f"{QC.CERT_PREFIX}_integral_library.json",
                  f"{QC.CERT_PREFIX}_batchA.json", f"{QC.CERT_PREFIX}_D13.json") if (ROOT / "certificates" /
+>  n).is_file()},
  }
  cdir = ROOT / "certificates"
  txt = certio.dumps(cert)
  if publish:
      (ROOT / "RELEASE_STATUS.json").write_text(txt)
      (cdir / "RELEASE_STATUS.json").write_text(txt)
      (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
```

```
        (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
        certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
            "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
  +> tatus)",
            **m, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
            "independent_rebuild_passed_p14r": p14r_ok, "hard_stops": hard,
            "flags": {"batchA_evaluated": a_ok, "D13_evaluated": b_ok, "library_certified": lib_ok}})
        (ROOT / "manuscript" / "body").mkdir(parents=True, exist_ok=True)
        (ROOT / "manuscript" / "body" / "P15-results-table.tex").write_text(results_table(per, dc))
    else:
        (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(certio.dumps({
            "landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED",
            "contract_layer_accepted": c_ok, "contract_identity_ok": id_ok, "contract_identity": ident,
            "library_certified": lib_ok, "batchA_evaluated": a_ok, "D13_evaluated": b_ok, "hard_stops_hold": h
  +> ard_ok}))
    print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
  +> ion"],
                      "contract_layer": c_ok, "contract_identity": id_ok, "changed_contract_files": ident["cha
  +> nged"],
                      "library": lib_ok, "batchA_evaluated": a_ok, "D13_evaluated": b_ok, "D12_status": D12_st
  +> atus,
                      "hard_ok": hard_ok, "author": author_ok, "independent_p15": indep_ok,
                      "independent_p14r": p14r_ok}, indent=1))
    return 0 if publish else 1


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        cdir = ROOT / "certificates"
        (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
            {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED",
             "error": f"{type(exc).__name__}: {exc}"}, indent=1) + "\n")
        print(f"release_p15 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

132 Source: tests/test__p15.py

[listing 99/133] tests/test_p15.py

```
"""XVI-P p15 tests: first P.5 calculation (Batch A, D13), the disclosed p14r contract correction (structural R
  +> 5
safeguard in dred.py, finding F1 of the p14r independent rebuild) with the reviewer's toys as regression tests
  +> , and
the p15 release status."""
from __future__ import annotations
import hashlib
import json
import sys
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools"):
    sys.path.insert(0, str(ROOT / p))
import p15_conventions as QC  # noqa: E402
import dred as DR  # noqa: E402

CERT = ROOT / "certificates"


def cj(name):
    return json.loads((CERT / name).read_text())


# ============================================================== the disclosed contract correction (finding
  +> F1)
M, A1, A2 = sp.symbols("m a1 a2")
A0 = M ** 2 * (1 / DR.eps + A1 + A2 * DR.eps)          # J = m^2 A0 = int l^2/(l^2 - m^2)
```

```
def _eta_toy(mutation=None, enforce=True):
    """reviewer toy A (eta-projected): eta^{rs} int l_r l_s/(l^2 - m^2) = eta^{rs} ghat_rs J/d = J."""
    DR._RED.clear()
    DR._LIFT.clear()
    P = DR.Pipeline(DR.Rules(), mutation, enforce)
    tot = sp.expand(sum(DR.ETA[r] * P.reduced_component([r, r], with_p=False, tag="RTT") for r in range(4)))
    tot = DR.eta_trace_Gt(tot, P.rules)
    return P.laurent(tot, {sp.Symbol("J_RTT[1,0]"): M ** 2 * A0})


def test_reviewer_eta_trace_toy_false_positive_fixed():
    """p14r heuristic: raised on this CORRECT d-free result.  Structural guard: no exception, finite a1 m^4.""
  +> "
    out = _eta_toy()
    assert sp.simplify(out["finite"] - A1 * M ** 4) == 0 and sp.simplify(out["pole"] - M ** 4 / DR.eps) == 0


def test_reviewer_eta_trace_toy_false_negative_fixed():
    """p14r heuristic: did NOT raise under premature_d4 (finite m^4 (a1 - 1/2)).  Structural guard: raises."""
    with pytest.raises(DR.PrematureD4Substitution):
        _eta_toy("premature_d4")
    forced = _eta_toy("premature_d4", enforce=False)          # display only (authorised path, recorded)
    assert sp.simplify(forced["finite"] - M ** 4 * (A1 - sp.Rational(1, 2))) == 0


def test_opaque_dimension_refuses_every_unauthorised_substitution():
    e = DR.d ** 2 / (DR.d - 1) + sp.Symbol("x")
    for bad in (lambda: e.subs(DR.d, 4), lambda: e.xreplace({DR.d: 4}), lambda: e.subs({DR.d: 4 - 2 * DR.eps})
  +> ,
                lambda: sp.Integer(3) * DR.d + 1 and (3 * DR.d).subs(DR.d, sp.Rational(7, 2))):
        with pytest.raises(DR.PrematureD4Substitution):
            bad()
    n0 = DR.D_PROVENANCE.get("test: authorised", 0)
    assert DR.substitute_d(e, 4, "test: authorised") == sp.Rational(16, 3) + sp.Symbol("x")
    assert DR.D_PROVENANCE["test: authorised"] == n0 + 1
    assert e.subs(sp.Symbol("x"), 1).has(DR.d)                # other substitutions are unaffected


def _ghat_contract(red, con, with_p, tag):
    B, C = DR.reduce_tensor(2, with_p, red, tag)
    tot = 0
    for b, c in zip(B, C):
        tot += c * DR.contract([(sp.Integer(1), t + (("g", "h", "m0", "m1"),)) for t in b["terms"]], con).get(
  +> (), 0)
    return sp.expand(tot)


def test_reviewer_A0_mixing_toy():
    """reduce with tr ghat = d but contract with tr ghat = 4: refused; forced display shifts the finite part b
  +> y +m^4/2."""
    DR._RED.clear()
    J = {sp.Symbol("J_RAT[1,0]"): M ** 2 * A0}
    good = DR.Pipeline().laurent(_ghat_contract(DR.Rules(), DR.Rules(), False, "RAT"), J)
    assert sp.simplify(good["finite"] - A1 * M ** 4) == 0
    with pytest.raises(DR.NumericTraceForbidden):
        _ghat_contract(DR.Rules(), DR.Rules(trace_hat=4), False, "RAT")
    mixed = _ghat_contract(DR.Rules(), DR.Rules(trace_hat=4, display_only=True), False, "RAT")
    forced = DR.Pipeline(enforce=False).laurent(mixed, J)
    assert sp.simplify(forced["finite"] - good["finite"] - M ** 4 / 2) == 0


def test_reviewer_B0_mixing_toy():
    """ghat_{mu nu} int l^mu l^nu/(l^2 (l-p)^2) = J[1,0] = 0 (scaleless); the d/4 mixing gives -p^2/6."""
    DR._RED.clear()
    p2, b1, b2 = sp.symbols("p2 b1 b2")
    J = {sp.Symbol("J_RBT[1,0]"): sp.Integer(0), sp.Symbol("J_RBT[0,2]"): p2 ** 2 * (1 / DR.eps + b1 + b2 * DR
  +> .eps) / 4}
    pp = {sp.Symbol("p.p"): p2}
    good = DR.Pipeline().laurent(_ghat_contract(DR.Rules(), DR.Rules(), True, "RBT").xreplace(pp), J)
    assert sp.simplify(good["finite"]) == 0 and sp.simplify(good["pole"]) == 0
    with pytest.raises(DR.NumericTraceForbidden):
        _ghat_contract(DR.Rules(), DR.Rules(trace_hat=4), True, "RBT")
    mixed = _ghat_contract(DR.Rules(), DR.Rules(trace_hat=4, display_only=True), True, "RBT").xreplace(pp)
    forced = DR.Pipeline(enforce=False).laurent(mixed, J)
    assert sp.simplify(forced["finite"] + p2 / 6) == 0


def test_dred_certificate_records_the_correction():
    dr = cj("P14_dred_rules.json")
```

```
    c = dr["p15_contract_correction"]
    assert c["released_p14r_values_affected"] is False and "F1" in c["finding"]
    assert c["regression_toys"]["pass"] is True
    assert dr["mutations"]["premature_d4"]["rejected"] and dr["mutations"]["trace_ghat_eq_4"]["structurally_re
  +> fused"]
    src = (ROOT / "code" / "p5_entry" / "dred.py").read_text()
    assert "is a rule violation and is detected (PrematureD4)" not in src           # the p14r claim is correc
  +> ted
    assert "FALSE POSITIVE" in src and "FALSE NEGATIVE" in src and "OpaqueDimension" in src


def test_contract_byte_identical_except_disclosed_correction():
    cur = {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest()
           for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    changed = sorted(k for k, h in QC.P14R_CONTRACT_SHA256.items() if cur.get(k) != h)
    assert changed == sorted(QC.DISCLOSED_CONTRACT_CORRECTIONS) == ["code/p5_entry/dred.py"]
    assert set(cur) == set(QC.P14R_CONTRACT_SHA256)
    assert QC.DISCLOSED_CONTRACT_CORRECTIONS["code/p5_entry/dred.py"]["released_p14r_values_affected"] is Fals
  +> e


# ============================================================== library, scaleless rule
def test_integral_library_certified():
    lib = cj("P15_integral_library.json")
    assert lib["library_certified"] is True and lib["revision"] == "p15"
    assert lib["tensor_reduction_vs_feynman_parameters"] and lib["B0_exact_d_numeric"] and lib["T_exact_d_nume
  +> ric"]


def test_scaleless_rule_is_explicit_and_refuses_scaleful_integrals():
    import integrals as IL
    with pytest.raises(IL.ScalelessRuleMisuse):
        IL.scaleless(0, 1, "test: massive tadpole", has_scale=True)
    n = len(IL.LEDGER)
    assert IL.scaleless(0, 2, "test: log") == 0 and IL.LEDGER[n]["omega"] == 0
    assert "1/eps_UV - 1/eps_IR" in IL.LEDGER[n]["class"]
    assert IL.scaleless(1, 1, "test: power") == 0 and "power divergence" in IL.LEDGER[n + 1]["class"]


# ============================================================== Batch A
def test_batchA_graphs():
    a = cj("P15_batchA.json")
    assert set(a["graphs"]) == set(QC.BATCH_A) and a["batchA_evaluated"]["execution"] is True
    for k, g in a["graphs"].items():
        assert g["status"] == "vanishing_under_declared_prescription", k
        assert g["value_imported_from_XVI_0"] is False
        assert g["verification_controls"]["route1_dred_lift_equals_route2_symmetric_integration"] is True
        assert g["scaleless_analysis"]["logarithmic_members"] == []          # NOT a 1/eps_UV - 1/eps_IR cance
  +> llation
        assert all(x["omega"] > 0 for x in g["scaleless_analysis"]["S0_applications"])
        assert g["prescription_dependence"]["p14r_counterterm_required"] is False
    assert a["D12_slot"]["status"] == "vanishing_under_declared_prescription" and a["D12_slot"]["derivation"]
    assert a["one_point_slot_kappa1"]["graphs"] == ["D14a", "D14b", "D14c"]
    assert a["gate_checks"]["D12_separate_from_one_point_slot"] is True


# ============================================================== Batch B: D13
def test_D13_values_and_controls():
    c = cj("P15_D13.json")
    r = c["result"]
    assert [r[k]["UV_pole"] for k in ("T1", "T2", "T3", "E1", "E2")] == ["0", "3/16", "-1/4", "0", "1/16"]
    assert [r[k]["local_polynomial"] for k in ("T1", "T2", "T3", "E1", "E2")] == ["0", "1/3", "-5/12", "0", "1
  +> /6"]
    assert all(sp.sympify(r[k]["nonlocal_coefficient_of_L"]) == -sp.sympify(r[k]["UV_pole"]) for k in r)
    assert c["UV_pole_in_p14r_cK_basis"]["coordinates"] == ["0", "3/8", "-1/4"]
    assert c["status"] == "regulated" and c["sector_status"]["4D_structures_T1_T3"] == "renormalized"
    assert c["evanescent_pole_nonzero"] is True and c["evanescent_counterterm_declared_in_p14r"] is False
    assert all(v is True for v in c["controls"].values()) and c["D13_evaluated"]["execution"] is True
    g = c["ghost_self_energy_pole"]
    assert g["pole_in_p14r_c_cb_coordinates"] == ["-3/8", "1/4"] and g["evanescent_pole_SE"] == "-1/8"


def test_D13_mutations_change_result_and_are_rejected():
    m = cj("P15_D13.json")["mutations"]
    assert set(m) == {"wrong_ghost_loop_sign", "premature_d4", "drop_evanescent_Gt", "wrong_symmetry_factor",
                      "wrong_K_c_ordering"}
    assert all(v["changes_result"] is True and v["rejected"] is True for v in m.values())
    assert "structural" in m["premature_d4"]["reason"]


def test_D13_premature_d4_is_refused_at_the_substitution():
```

```
    import d13
    with pytest.raises(DR.PrematureD4Substitution):
        d13.route1(mutation="premature_d4")


# ============================================================ p15 release status
def test_p15_release_status_and_hard_stops():
    rs = json.loads((CERT / "P15_release.json").read_text())        # p16: the p15 LAYER record (top level is
  +> p16)
    assert rs["revision"] == "p15" and rs["contract_revision"] == "p14r" and rs["publication"] == "accepted"
    assert rs["batchA_evaluated"] is True and rs["D13_evaluated"] is True and rs["library_certified"] is True
    for k in ("batchA_evaluated", "D13_evaluated"):
        assert set(rs["flags"][k]) == {"execution", "claim", "scope"}
    assert rs["hard_stops"] == QC.HARD_STOPS_P15 and rs["hard_stops_hold"] is True
    assert rs["hard_stops"]["quantum_inputs_evaluated"] == "not_earned" and rs["O1_closure"] == "NOT_CLOSED"
    assert rs["bulk_symmetry_restored"] is False and rs["Route_B_discharged"] is False
    assert all(rs["graphs"][k]["status"] == "unevaluated" for k in QC.NOT_EVALUATED_P15)
    assert rs["D12_status"] == "vanishing_under_declared_prescription" and rs["D12_status_derivation"]
    assert rs["contract_identity"]["changed"] == ["code/p5_entry/dred.py"]
    assert rs["milestones"]["status"]["P.5_identity"] == "open"
    # p16: this layer record's current-tree flag reads the p15 report path, which (where present) is bound to
  +> the p15
    # tree, so it is never true for the p16 tree; the p16 flag is in the top-level status
    assert rs["independent_rebuild_passed"] in ("pending", False)


def test_p14r_independent_rebuild_recorded_with_caveat():
    rs = json.loads((ROOT / "RELEASE_STATUS.json").read_text())
    rec = rs["p14r_independent_rebuild"]
    assert rs["independent_rebuild_passed_p14r"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P14R_SOURCE_TREE_SHA256
    assert rec["bound_to_source_zip_sha256"] == QC.P14R_SOURCE_ZIP_SHA256
    assert rec["report_sha256_pinned"] is True and "SAME-BOX" in rec["caveat"]
```

133 Source: p16_conventions.py (p16 constants, version label, pins)

[listing 100/133] code/p5_calc/p16_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p16 conventions: P.5 calculation, second step (E2 evanescent analysis; Batch C: D01, D03).

Layering.  p14r contract (code/p5_entry/*, P14_*)  ->  p15 calculation layer (Batch A, D13; code/p5_calc/{inte
  +> grals,
batchA,d13,p15_conventions,release_p15}.py, P15_*; byte-identical to p15)  ->  p16 layer (covariant route, Bat
  +> ch C,
E2 analysis; code/p5_calc/{covariant,batchC,e2_evanescent,p16_conventions,release_p16}.py, P16_*).  The p16 ga
  +> te
(release_p16.py) runs after the p14r and p15 gates and requires both to be accepted.

Disclosed CONTRACT revision in p16 (DISCLOSED_CONTRACT_REVISIONS_P16): code/p5_entry/dred.py only -- F3 harden
  +> ing of
the opaque d (expr.replace / pattern matching / lambdify code printers) on top of the p15 F1 correction.  NO e
  +> vanescent
counterterm is added: the E2 decoupling from the ch-projected slot is PROVEN (P16_E2_evanescent.json).
All other p14r contract files are byte-identical to p14r; all p15 calculation files are byte-identical to p15.
"""
from __future__ import annotations

REVISION = "p16"
VERSION_LABEL = "XVI-P p16"           # single version variable (manuscript footer is checked against it)
CERT_PREFIX = "P16"
CONTRACT_REVISION = "p16 (p14r contract + disclosed p15/p16 revisions)"
DATE = "2026-10-08"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p16.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p16.json"
P15_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p15.json"      # written by the reviewer;
  +>  never here
P15_INDEPENDENT_REPORT_SHA256 = "6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713"
P15_SOURCE_TREE_SHA256 = "79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f"
P15_SOURCE_ZIP_SHA256 = "54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3"
P15_ARCHIVED_RECORDS = {
    "archival/p15/reference_values_p15.json": "0866adfe0f9dc281053b84dcabbe751a8a7425f522d37839373b4d373415cda
  +> 8",
    "archival/p15/reproducibility/author_rebuild_report_p15.json":
        "1e729eba3f3faa6a5cfbd1b55947245d643c7855756604c8a5b886cc28dbc5e6",
    # per-file sha256 list of the released p15 source-only zip (base of the F5 change log; works in a source-o
```

```
  +> nly tree)
    "archival/p15/p15_source_files.json": "09536539460ad68e09e15d1f92d1046afa8f4a3ee946b0396fa852ba28d1fbe8",
}
P15_SOURCE_LIST = "archival/p15/p15_source_files.json"

BATCH_C = ("D01", "D03")
NOT_EVALUATED_P16 = ("D11",)                      # M2 undischarged
STATUS_VALUES = ("unevaluated", "regulated", "vanishing_under_declared_prescription", "renormalized")

HARD_STOPS_P16 = {
    "quantum_inputs_evaluated": "not_earned",          # D11 open -> not every required input is evaluated
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": False,
    "symmetry_restored": False,
    "D11_matching_verified": False,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": False,
    "XIV_J_opened": False,
    "Route_B_discharged": False,
    "u_R_c_delta_added": False,
    "XVI_P_registered": False,
    "D11_evaluated": False,
}

# p15 calculation layer: byte-identical in p16 (sha256 from XVI-P-p15-source-only.zip; release_p16 recomputes
  +> them)
P15_CALC_SHA256 = {
    "code/p5_calc/batchA.py": "5481edfed7331165a3ba68cb2cc229975b8e6e6c581bd0014ae983a7cf4d8e76",
    "code/p5_calc/d13.py": "55c01198a6074fb89a24c261d28bd413bbc5113b9d716aed4f43a55d7bb5dccb",
    "code/p5_calc/integrals.py": "3175f53d5c8ef2abd82ee5e1d19f3097185c0e121d612750485adf7ac82843dc",
    "code/p5_calc/p15_conventions.py": "4ff06f631512bc83196651057f41468debc6cf2d8d661f84139f097e436715d3",
    "code/p5_calc/release_p15.py": "9048a6c8414938ba79b7eae93cba760b6cbc5e5fcdc8e377efda39342f9ba9d0",
}
P14R_CONTRACT_FILES_CHANGED_IN_P16 = ("code/p5_entry/dred.py",)
NEW_CONTRACT_FILES_P16 = ()
P15_DRED_SHA256 = "290e440419cba3b6ba554bcb73dfad5cc2fc150af1b1567c3f90388cd6cc173b"     # dred.py as released
  +>  in p15

DISCLOSED_CONTRACT_REVISIONS_P16 = {
    "code/p5_entry/dred.py": {
        "kind": "contract revision (disclosed), p16 (on top of the p15 F1 correction)",
        "finding": "F3_opaque_d_bypass, reproducibility/independent_rebuild_report_p15.json",
        "why": ("OpaqueDimension guarded subs/xreplace/evalf(subs)/limit, but expr.replace(d, 4) and sympy.lam
  +> bdify "
                "bypassed it (the reviewer probe returned 0); no package code used those paths"),
        "change": ("OpaqueDimension.matches raises outside substitute_d (closes expr.replace(d, value) and pat
  +> tern "
                   "queries for d); the code-printer hooks used by lambdify (_lambdacode, _numpycode, _scipyco
  +> de, "
                   "_mpmathcode, _pythoncode, _cupycode, _jaxcode, _tensorflowcode) raise outside substitute_d
  +> ; "
                   "remaining value paths that the guard CANNOT hook -- replace with a callable query "
                   "(expr.replace(lambda x: x == d, ...)), Poly(.., d).eval, string round-trips -- are covered
  +>  by a "
                   "static AST scan of code/p5_calc and code/p5_entry (tests/test_p16.py) -- none is used"),
        "rules_R1_R6_changed": False,
        "released_values_affected": False,
        "why_unaffected": ("no package code used replace/lambdify/Poly.eval on d (static scan); the P14/P15 "
                           "certificate content hashes recomputed in p16 equal those of archival/p15/"
                           "reference_values_p15.json (tests/test_p16.py)"),
    },
}
DISCLOSED_TEST_REVISIONS_P16 = {    # test files changed vs the p15 zip (release_p16 change log, tests/test_p1
  +> 6.py F5)
    "tests/test_p14.py": ("source-set / zip / reference names p15 -> p16 (SM.ZIP_NAME, SM.REFERENCE_FILE, auth
  +> or report "
                          "path, resumed-build placeholder zip); generated-prefix list extended by P16_* certi
  +> ficates/"
                          "ledgers, the P16 results table and manuscript/version.tex; top-level status revisio
  +> n p16. "
                          "No assertion weakened."),
    "tests/test_p15.py": ("test_p15_release_status_and_hard_stops reads the p15 LAYER record certificates/"
                          "P15_release.json (the top-level RELEASE_STATUS.json is now the p16 status); its cur
  +> rent-tree "
                          "independent flag may be False (the p15 report is bound to the p15 tree) or pending
  +> (report "
                          "absent in a source-only copy). All other p15 assertions unchanged."),
}
```

```
RUN_SPECIFIC_KEYS = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed",
                     "live_report_identical_to_archived", "live_report_present", "archived_zip_sha256_ok",
                     "list_equals_archived_zip")
```

134 Source: covariant.py (p16 covariant route: exact covariant integrand, FP and PV loop routes)

[listing 101/133] code/p5_calc/covariant.py

```
#!/usr/bin/env python3
"""XVI-P p16: COVARIANT evaluation route for one-loop two-point graphs under the frozen p14r DRED prescription
  +> .

Why (p16 finding C1).  The p14r R3 component lifting (dred.lift_reduce_monomial) reduces each exported COMPONE
  +> NT
monomial l_{r1}..l_{rn} separately, AFTER the internal index sums of the exported numerator were done in compo
  +> nents.
For rank >= 4 numerators (D01, D03) this produces Gt-QUADRATIC component sums such as sum_rs eta^rr eta^ss Gt_
  +> rs Gt_rs,
whose abstract value (tr gtilde.gtilde = tr gtilde = 4 - d, gtilde.gtilde = gtilde) cannot be recovered from t
  +> he
summed component polynomial: a fixed 4-component basis of Q4S cannot carry a projector of non-integer trace (S
  +> iegel's
DRED consistency issue), and the linear realization Gt = (4-d) M (integrals.realization) is faithful only for
expressions LINEAR in Gt.  dred.eta_trace_Gt refuses such patterns (UncontractedObjects).  D13 (rank 3) is lin
  +> ear in
Gt and unaffected.

The covariant route evaluates the SAME rules R1-R6 without any component basis inside the loop:
  C1  the exported 4D component numerator N_A(l, p) (frozen, sha-pinned) is written EXACTLY as a covariant pol
  +> ynomial
      sum_j c_j B_j(eta, l, p) with free external indices only (all internal contractions appear as the scalar
  +> s l.l,
      l.p, p.p); exactness is VERIFIED as a polynomial identity on every exported component.  Uniqueness: with
  +>  at most
      8 vector slots no 4D dimension-dependent (Schouten/Gram) identity exists among the B_j (needs >= 10 slot
  +> s).
  C2  R3 lift: eta -> g4 on the free external indices; l, p in QdS (R2).
  C3a Feynman-parameter route: l = k + x p, symmetric integration in QdS over free indices only
      (k_a k_b -> ghat_ab k^2/d, k.p -> p), x-integrals exact (integrals.fp_ratio)  -- no S0, no Gram.
  C3b Passarino-Veltman route: frozen dred.reduce_tensor (Gram projection in ghat, abstract indices) on the fr
  +> ee
      l-indices, scalar J[a,b] from the certified library (S0 for cancelled propagators).
  C4  external 4D projection (R4/R5): ghat_ab = g4_ab - gt_ab on FREE indices (no trace arises); products gt_a
  +> b gt_cd
      with distinct free indices are genuine evanescent tensors (kept, R6).
  C5  Laurent / MS-bar with d symbolic (opaque d; dred.substitute_d only).
Output: {canonical structure: A(d)} with Ghat = sum_s A_s(d) * s * MB.
"""
from __future__ import annotations
import itertools
import random
import sys
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dred  # noqa: E402
import integrals as IL  # noqa: E402

d = dred.d
ETA = dred.ETA
HC = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]
LL, LP, PP = sp.Symbol("l.l"), sp.Symbol("l.p"), IL.PP
MB = IL.MB


# ---------------------------------------------------------------------------------------- structures
# a factor is ('g4', a, b) | ('gt', a, b) | ('gh', a, b) | ('l', a) | ('p', a)   (a, b external index labels)
def canon(facs):
    out = []
    for f in facs:
```

```
        if f[0] in ("g4", "gt", "gh"):
            a, b = sorted(f[1:])
            out.append((f[0], a, b))
        else:
            out.append(f)
    return tuple(sorted(out))


def relabel(facs, perm):
    return canon([(f[0],) + tuple(perm.get(x, x) for x in f[1:]) for f in facs])


def orbit(facs, group):
    return frozenset(relabel(facs, g) for g in group)


def sym_group(pairs):
    """the symmetry group generated by swapping the members of each symmetric external pair."""
    gens = [dict(zip(p, p[::-1])) for p in pairs]
    group = []
    for bits in itertools.product((0, 1), repeat=len(gens)):
        g = {}
        for b, gen in zip(bits, gens):
            if b:
                g.update(gen)
        group.append(g)
    return group


def _matchings(items):
    if not items:
        yield []
        return
    a = items[0]
    for k in range(1, len(items)):
        rest = items[1:k] + items[k + 1:]
        for m in _matchings(rest):
            yield [(a, items[k])] + m


def numerator_ansatz(labels, degree, pairs):
    """all covariant monomials in g4 (external pairs), l_a, p_a and scalars l.l, l.p, p.p of total momentum de
  +> gree
    `degree`, symmetrised over the symmetric external pairs -> list of (orbit, scalar)."""
    group = sym_group(pairs)
    seen, out = set(), []
    n = len(labels)
    for k in range(n // 2 + 1):
        for sub in itertools.combinations(labels, 2 * k):
            for m in _matchings(list(sub)):
                rest = [x for x in labels if x not in sub]
                for lp in itertools.product(("l", "p"), repeat=len(rest)):
                    R = degree - len(rest)
                    if R < 0 or R % 2:
                        continue
                    facs = canon([("g4", a, b) for a, b in m] + [(t, x) for t, x in zip(lp, rest)])
                    orb = orbit(facs, group)
                    for a_ in range(R // 2 + 1):
                        for b_ in range(R // 2 + 1 - a_):
                            c_ = R // 2 - a_ - b_
                            sc = LL ** a_ * LP ** b_ * PP ** c_
                            key = (orb, sc)
                            if key not in seen:
                                seen.add(key)
                                out.append(key)
    return out


# ------------------------------------------------------------------------------------------------ component e
  +> valuation
def _comp_value(facs, assign, lsym, psym):
    v = sp.Integer(1)
    for f in facs:
        if f[0] == "g4":
            a, b = assign[f[1]], assign[f[2]]
            v *= ETA[a] if a == b else 0
        elif f[0] == "l":
            v *= lsym[assign[f[1]]]
        elif f[0] == "p":
            v *= psym[assign[f[1]]]
        else:
```

```
            raise ValueError(f)
        if v == 0:
            return v
    return v


def orbit_value(orb, assign, lsym, psym):
    return sp.Add(*[_comp_value(f, assign, lsym, psym) for f in orb])


def scalars(lsym, psym):
    return {LL: sum(ETA[r] * lsym[r] ** 2 for r in range(4)), LP: sum(ETA[r] * lsym[r] * psym[r] for r in rang
  +> e(4)),
            PP: sum(ETA[r] * psym[r] ** 2 for r in range(4))}


def assignments(spec):
    """spec: list of ('pair', (a, b)) | ('vec', a) external legs in export order -> list of (comp key, {label:
  +>  idx})."""
    per = []
    for kind, lab in spec:
        if kind == "pair":
            per.append([(i, {lab[0]: m, lab[1]: n}) for i, (m, n) in enumerate(HC)])
        else:
            per.append([(i, {lab: i}) for i in range(4)])
    out = []
    for combo in itertools.product(*per):
        key = ",".join(str(c[0]) for c in combo)
        asg = {}
        for c in combo:
            asg.update(c[1])
        out.append((key, asg))
    return out


def export_poly(rows, lsym, psym):
    tot = 0
    for names, cf in rows:
        m = sp.Rational(cf)
        for v in names:
            m *= lsym[int(v[1])] if v[0] == "l" else psym[int(v[1])]
        tot += m
    return sp.expand(tot)


def pair_weight(spec, asg, offdiag, upper=False):
    """off-diagonal component weight per symmetric pair (offdiag: number or tuple, one entry per pair)."""
    w = sp.Integer(1)
    pairs = [lab for kind, lab in spec if kind == "pair"]
    ws = offdiag if isinstance(offdiag, (tuple, list)) else [offdiag] * len(pairs)
    for lab, wo in zip(pairs, ws):
        if asg[lab[0]] != asg[lab[1]]:
            w *= wo
    if upper:                       # coefficient tensor with UPPER external indices (dual to lower field comp
  +> onents)
        for x, i in asg.items():
            w *= ETA[i]
    return w


def covariant_fit(numerator, spec, degree, seeds=tuple(range(1, 13)), offdiag=1, upper=False, polys=None):
    """C1: exact covariant form of the exported component numerator (or of given component polynomials `polys`
  +>  in
    l0..l3, p0..p3); verified as a polynomial identity on every component."""
    labels = [x for kind, lab in spec for x in (lab if kind == "pair" else (lab,))]
    pairs = [lab for kind, lab in spec if kind == "pair"]
    ansatz = numerator_ansatz(labels, degree, pairs)
    asg = assignments(spec)
    ls, ps = sp.symbols("l0:4"), sp.symbols("p0:4")
    rows, rhs = [], []
    if polys is None:
        polys = {k: export_poly(numerator.get(k, []), ls, ps) for k, _ in asg}
    else:
        polys = {k: sp.expand(polys.get(k, 0)) for k, _ in asg}
    for sd in seeds:
        rnd = random.Random(sd)
        lv = [sp.Rational(rnd.randint(-9, 9), rnd.randint(1, 4)) for _ in range(4)]
        pv = [sp.Rational(rnd.randint(-9, 9), rnd.randint(1, 4)) for _ in range(4)]
        sub = dict(zip(ls, lv)) | dict(zip(ps, pv))
        scn = scalars(lv, pv)
        for k, a in asg:
```

```
            w = pair_weight(spec, a, offdiag, upper)
            rows.append([w * orbit_value(orb, a, lv, pv) * sc.xreplace(scn) for orb, sc in ansatz])
            rhs.append(polys[k].xreplace(sub))
    from sympy.polys.matrices import DomainMatrix
    n = len(ansatz)
    Aug = DomainMatrix([[sp.QQ.convert(sp.Rational(x)) for x in r] + [sp.QQ.convert(sp.Rational(y))]
                        for r, y in zip(rows, rhs)], (len(rows), n + 1), sp.QQ)
    R, piv = Aug.rref()
    cols = [p for p in piv if p < n]
    if n in piv:
        return {"terms": [], "n_ansatz": n, "n_used": 0, "exact": False, "bad_components": ["inconsistent"],
                "rank": len(cols), "unique": False}
    Rl = R.to_Matrix()
    coef = {}
    for i, j in enumerate(cols):
        c = sp.Rational(Rl[i, n])
        if c != 0:
            coef[j] = c
    # exact verification as polynomial identity on every component
    scs = scalars(ls, ps)
    bad = []
    for k, a in asg:
        e = pair_weight(spec, a, offdiag, upper) * sum(c * orbit_value(ansatz[j][0], a, ls, ps) * ansatz[j][1]
  +> .xreplace(scs)
                                                       for j, c in coef.items())
        if sp.expand(e - polys[k]) != 0:
            bad.append(k)
    terms = [(ansatz[j][0], ansatz[j][1], c) for j, c in coef.items()]
    return {"terms": terms, "n_ansatz": len(ansatz), "n_used": len(terms), "exact": not bad,
            "bad_components": bad[:5], "rank": len(cols), "unique": len(cols) == len(ansatz), "offdiag_weight"
  +> : str(offdiag), "upper": upper}


# ---------------------------------------------------------------------------------------------- C3a Feynman
  +>  parameters
_X = sp.Symbol("x")
_K2, _KP = sp.Symbol("k.k"), sp.Symbol("k.p")


def _sym_pairings(items):
    """symmetric integration of k_{items} (labels or 'P' for k.p): sum over pairings -> list of factor tuples,
  +>  scalar."""
    out = []
    for m in _matchings(list(range(len(items)))):
        facs, sc = [], sp.Integer(1)
        for i, j in m:
            a, b = items[i], items[j]
            if a == "P" and b == "P":
                sc *= PP
            elif a == "P":
                facs.append(("p", b))
            elif b == "P":
                facs.append(("p", a))
            else:
                facs.append(("gh", a, b))
        out.append((facs, sc))
    return out


def fp_term(facs, scal):
    """int [facs * scal] / (D1 D2) by Feynman parameters, covariant: returns {canon factors: coefficient * MB}
  +> ."""
    lv = [f[1] for f in facs if f[0] == "l"]
    rest = [f for f in facs if f[0] != "l"]
    ksym = {a: sp.Symbol(f"k[{a}]") for a in lv}
    psy = {a: sp.Symbol(f"pp[{a}]") for a in lv}
    expr = sp.Mul(*[ksym[a] + _X * psy[a] for a in lv]) * scal.xreplace(
        {LL: _K2 + 2 * _X * _KP + _X ** 2 * PP, LP: _KP + _X * PP})
    expr = sp.expand(expr)
    gens = list(ksym.values()) + list(psy.values()) + [_K2, _KP, _X]
    out = {}
    for mon, cf in sp.Poly(expr, *gens).terms():
        nk = len(ksym)
        kl = [a for (a, e) in zip(ksym, mon[:nk]) for _ in range(e)]
        pl = [("p", a) for (a, e) in zip(psy, mon[nk:2 * nk]) for _ in range(e)]
        e2, ep, ex = mon[2 * nk:]
        items = kl + ["P"] * ep
        if len(items) % 2:
            continue
        nh = len(items) // 2
        den = sp.Mul(*[d + 2 * i for i in range(nh)])
```

```
        j = e2 + nh
        rat = IL.fp_ratio(j, ex, 0)
        for pf, sc in _sym_pairings(items):
            key = canon(rest + pl + pf)
            out[key] = sp.expand(out.get(key, 0) + cf * sc * rat / den * MB)
    return out


# ------------------------------------------------------------------------------------------- C3b PV via
  +> frozen dred
def pv_term(facs, scal, tag):
    """int [facs * scal]/(D1 D2) via dred.reduce_tensor (Gram in ghat) on the free l-indices; J from the libra
  +> ry."""
    lv = [f[1] for f in facs if f[0] == "l"]
    rest = [f for f in facs if f[0] != "l"]
    a0 = sp.degree(scal, LL)
    b0 = sp.degree(scal, LP)
    pc = sp.expand(scal / (LL ** a0 * LP ** b0))
    rules = dred.Rules()
    out = {}
    if not lv:
        J = IL.J_two_point(a0, b0, f"J_{tag}")
        out[canon(rest)] = sp.expand(pc * J)
        return out
    B, Cs = dred.reduce_tensor(len(lv), True, rules, tag)
    idx = {f"m{i}": a for i, a in enumerate(lv)}
    for bk, ck in zip(B, Cs):
        Jsub = {}
        for s in ck.free_symbols:
            if s.name.startswith(f"J_{tag}["):
                i, j = (int(x) for x in s.name[len(tag) + 3:-1].split(","))
                Jsub[s] = IL.J_two_point(i + a0, j + b0, f"J_{tag}")
        cval = sp.expand(ck.xreplace(Jsub))
        for t in bk["terms"]:
            ff = []
            for f in t:
                if f[0] == "g":
                    ff.append(("gh", idx[f[2]], idx[f[3]]))
                else:
                    ff.append(("p", idx[f[2]]))
            key = canon(rest + ff)
            out[key] = sp.expand(out.get(key, 0) + pc * cval)
    return out


# ------------------------------------------------------------------------------------------- C4 projecti
  +> on, collect
def project_hat(res):
    """ghat_ab = g4_ab - gt_ab (free indices): expand products."""
    out = {}
    for key, c in res.items():
        hs = [f for f in key if f[0] == "gh"]
        rest = [f for f in key if f[0] != "gh"]
        for choice in itertools.product((0, 1), repeat=len(hs)):
            ff = list(rest)
            sgn = 1
            for h, ch in zip(hs, choice):
                if ch:
                    ff.append(("gt", h[1], h[2]))
                    sgn = -sgn
                else:
                    ff.append(("g4", h[1], h[2]))
            k2 = canon(ff)
            out[k2] = sp.expand(out.get(k2, 0) + sgn * c)
    return {k: v for k, v in out.items() if v != 0}


def evaluate(terms, method, tag="C"):
    tot = {}
    for orb, sc, c in terms:
        for facs in orb:
            r = fp_term(list(facs), sc) if method == "fp" else pv_term(list(facs), sc, tag)
            for k, v in r.items():
                tot[k] = sp.expand(tot.get(k, 0) + c * v)
    tot = {k: v for k, v in tot.items() if v != 0}
    return project_hat(tot)


def by_orbit(res, pairs, norm=1):
    """group canonical monomials into symmetrised structures (orbits); coefficient of MB, p.p powers kept in t
  +> he
```

```
    structure label.  Returns {(orbit, p.p power): A(d)} and a consistency flag (all members equal)."""
    group = sym_group(pairs)
    out, ok = {}, True
    seen = {}
    for k, v in res.items():
        orb = orbit(k, group)
        vv = sp.cancel(sp.expand(norm * v / MB))
        nu, de = sp.fraction(vv)
        e = sp.degree(nu, PP) - sp.degree(de, PP)
        cf = sp.factor(sp.cancel(vv / PP ** e))
        if cf.has(PP) or cf.has(MB):
            ok = False
        key = (orb, e)
        if key in seen:
            if sp.cancel(seen[key] - cf) != 0:
                ok = False
        else:
            seen[key] = cf
    for (orb, e), cf in seen.items():
        if cf != 0:
            out[(orb, e)] = cf
    return out, ok


def label(orb, e):
    rep = sorted(orb)[0]
    s = " ".join(f"{f[0]}_{''.join(f[1:])}" for f in rep)
    return (s + (f" (p.p)^{e}" if e else "")).strip()


def is_evanescent(orb):
    return any(f[0] == "gt" for f in sorted(orb)[0])
```

135 Source: e2_evanescent.py (p16 E2 decoupling proof)

[listing 102/133] code/p5_calc/e2_evanescent.py

```
#!/usr/bin/env python3
"""XVI-P p16 priority 1: does the D13 evanescent E2 pole (pole 1/16, A_E2 = 3/(16(d-1))) decouple from the
ch-projected slot?  Deterministic symbolic proof + mutation controls.

Where D13 enters the ch projection (P13 ch dependency rows, ledgers/P13_ch_dependency_rows.json): ONLY through
  +>  the row
(S0, Gamma1^[2]) s1 h:h^2, i.e. the slot functional
      X_D13[c, h] = int G_{(mn) nu}(p) c^nu(p) E^{(mn)(rs)}(p) h_(rs)(-p),
with G = delta Gamma1/delta K' (the D13 coefficient, G = kappa^2 N_eps sum_i A_i(d) T_i MB) and E = the operat
  +> or of
the h^2 monomials of S_Psi^(0) (delta S0/delta h at b = 0: the linearised Einstein-Hilbert operator; the b.h a
  +> nd b^2
monomials are separate rows and carry no h-only leg).

Proof (all steps exact in d, with the DRED objects of R1/R2: g4 = ghat + gtilde, p in QdS, external c, h with
  +> Q4S
indices as declared, and also restricted to QdS or to Q(4-d)S):
  L1  E is TRANSVERSE: p_m E^{(mn)(rs)} = 0 as a covariant identity (verified on the exact covariant form of E
      recovered from the frozen S0 density; the contraction uses only g4.p = p and needs no trace, so it holds
  +>  for
      every d and for Q4S external indices).
  L2  E2_{(mn) nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is LONGITUDINAL in (mn) (each term carries p_n or p_m
  +> ).
  L3  hence E2 . E = 0 identically: X_slot[E2] = 0 for every external assignment (Q4S, QdS, Q(4-d)S), exactly
  +> in d, to
      all orders in eps.  The E2 pole therefore DECOUPLES from the ch-projected slot (not merely O(eps)).
  L4  the slot map is not trivial: X_slot[T1] and X_slot[E1] are nonzero (non-longitudinal structures), and th
  +> e D13
      coefficients A_T1 = A_E1 = 0 (P15_D13.json), so D13's whole contribution to the ch slot vanishes.
Mutation controls (each must make the E2 slot contribution NONZERO and be rejected by the L1/L2 checks):
  M1 the gauge-fixed (de Donder) operator q^2 P (Schur complement, NOT the h^2-monomial operator) in place of
  +> E;
  M2 E1 = gt_mn p_nu p^2 (non-longitudinal) in place of E2;
  M3 E2 with its (mn) indices contracted in the wrong slot (E2_{(m nu) n}).
Scope: the ch-projected one-loop kappa^2 identity (one loop: the identity is linear in Gamma1, so no product o
  +> f two
evanescent poles occurs at this order).  The E2 pole remains a UV pole of the epsilon-scalar (Gt) component of
  +>  the
K'.c Green function of Gamma1; it does not enter the ch projection.
"""
from __future__ import annotations
```

```
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dred  # noqa: E402
import covariant as C  # noqa: E402
import p16_conventions as QC  # noqa: E402

d = dred.d
PP = C.PP
PS = sp.symbols("p0:4")
HC = C.HC
SPEC_HH = [("pair", ("m", "n")), ("pair", ("r", "s"))]

STRUCT = {"T1": [(("g4", "m", "n"), ("p", "v"))], "T2": [(("g4", "m", "v"), ("p", "n")), (("g4", "n", "v"), ("
  +> p", "m"))],
          "T3": [(("p", "m"), ("p", "n"), ("p", "v"))], "E1": [(("gt", "m", "n"), ("p", "v"))],
          "E2": [(("gt", "m", "v"), ("p", "n")), (("gt", "n", "v"), ("p", "m"))]}
PPOW = {"T1": 1, "T2": 1, "T3": 0, "E1": 1, "E2": 1}
KIND = {"g4": "4", "gt": "t", "gh": "h"}


def to_dred(facs):
    out = []
    for f in facs:
        if f[0] in KIND:
            out.append(("g", KIND[f[0]], f[1], f[2]))
        else:
            out.append(("v", f[0] if f[0] == "p" else f[0], f[1]))
    return out


def operator_from_polys(polys):
    """exact covariant form of an h.h two-point operator given by its component polynomials in p (upper indice
  +> s,
    off-diagonal weight 2 -- the field-component convention of the frozen extraction)."""
    fit = C.covariant_fit({}, SPEC_HH, 2, offdiag=2, upper=True, polys=polys)
    terms = []
    for orb, sc, c in fit["terms"]:
        if any(f[0] == "l" for f in sorted(orb)[0]):
            raise ValueError("operator depends on l")
        for facs in orb:
            terms.append((c * sc, list(facs)))
    return terms, fit


def S0_hh_operator():
    import counterterm_domain as CT
    import feynman as F
    S0 = CT.S0_density()
    polys = {}
    for a in range(10):
        for b in range(10):
            v = F.pw_vertex(S0, [{"f": "h", "pol": {HC[a]: 1}, "k": list(PS)},
                                 {"f": "h", "pol": {HC[b]: 1}, "k": [-x for x in PS]}])
            if v != 0:
                polys[f"{a},{b}"] = v
    return polys


def gauge_fixed_operator():
    """M1: q^2 P (Schur complement of the b block; the de Donder gauge-fixed kinetic operator), same conventio
  +> n."""
    import feynman as F
    Pm = F.P_matrix()
    q2 = sum(dred.ETA[r] * PS[r] ** 2 for r in range(4))
    return {f"{a},{b}": sp.expand(q2 * Pm[a, b]) for a in range(10) for b in range(10) if Pm[a, b] != 0}


def contract_all(terms):
    rules = dred.Rules()
    out = {}
    for c, facs in terms:
```

```
        r = dred.contract([(sp.Integer(1), tuple(facs))], rules)
        for k, v in r.items():
            out[k] = sp.expand(out.get(k, 0) + c * v)
    return {k: v for k, v in out.items() if v != 0}


def transversality(Eterms):
    """L1: p_m E^{(mn)(rs)} with free n, r, s."""
    res = contract_all([(c, to_dred(f) + [("v", "p", "m")]) for c, f in Eterms])
    return {"residual_terms": len(res), "transverse": not res}


def longitudinal(name, struct=None):
    """L2: every term of the structure carries p_m or p_n."""
    st = struct or STRUCT[name]
    return all(any(f == ("p", "m") or f == ("p", "n") for f in t) for t in st)


EXT = {"Q4S (declared, R2)": ("c", "a", "b"), "QdS (hat)": ("c^", "a^", "b^"),
       "c tilde, h hat": ("c~", "a^", "b^"), "c tilde, h mixed": ("c~", "a~", "b^"),
       "c hat, h mixed": ("c^", "a~", "b^"), "all tilde": ("c~", "a~", "b~")}


def slot(struct_terms, Eterms, ext, ppow=1):
    """X = G_{mn v} c^v E^{mn rs} a_r b_s  (h_rs = a_(r b_s) symmetrised)."""
    cn, an, bn = ext
    terms = []
    for gt_ in struct_terms:
        for c, ef in Eterms:
            for (x, y) in ((an, bn), (bn, an)):
                facs = to_dred(list(gt_)) + to_dred(ef) + [("v", cn, "v"), ("v", x, "r"), ("v", y, "s")]
                terms.append((c * PP ** ppow / 2, facs))
    res = contract_all(terms)
    assert all(k == () for k in res), "open indices left"
    return sp.expand(res.get((), 0))


def has_trace_factor(expr):
    """does the (nonzero) slot value carry an explicit (4 - d) factor (an O(eps) trace)?"""
    return bool(expr != 0 and sp.cancel(expr / (4 - d)).is_polynomial() and sp.factor(expr).has(4 - d))


def main():
    import certio
    t0 = time.time()
    polys = S0_hh_operator()
    E, fitE = operator_from_polys(polys)
    L1 = transversality(E)
    table = {}
    for nm in STRUCT:
        table[nm] = {lab: str(sp.factor(slot(STRUCT[nm], E, ext, PPOW[nm]))) for lab, ext in EXT.items()}
    E2_zero_everywhere = all(v == "0" for v in table["E2"].values())
    nontrivial = any(v != "0" for v in table["T1"].values()) and any(v != "0" for v in table["E1"].values())
    d13 = json.loads((ROOT / "certificates" / "P15_D13.json").read_text())
    A = {k: sp.sympify(d13["result"][k]["A_exact_d_times_MB"], locals={"d": d}) for k in STRUCT}
    d13_slot = {lab: str(sp.factor(sum(A[k] * slot(STRUCT[k], E, ext, PPOW[k]) for k in STRUCT if A[k] != 0)))
                for lab, ext in EXT.items()}
    # ---- mutations
    muts = {}
    Egf, fitgf = operator_from_polys(gauge_fixed_operator())
    L1gf = transversality(Egf)
    v1 = {lab: str(sp.factor(slot(STRUCT["E2"], Egf, ext))) for lab, ext in EXT.items()}
    muts["M1_gauge_fixed_operator_qq2P"] = {"E2_slot_nonzero": any(v != "0" for v in v1.values()),
                                            "rejected": not L1gf["transverse"], "slot_values": v1,
                                            "reason": f"L1 fails: p_m (q^2 P)^(mn)(rs) has {L1gf['residual_ter
  +> ms']} "
                                                      "residual terms (q^2 P is not the h^2-monomial operator
  +> of the "
                                                      "slot and is not transverse)"}
    v2 = {lab: table["E1"][lab] for lab in EXT}
    muts["M2_E1_in_place_of_E2"] = {"E2_slot_nonzero": any(v != "0" for v in v2.values()),
                                    "rejected": not longitudinal("E1"), "slot_values": v2,
                                    "reason": "L2 fails: gt_mn p_nu p^2 is not longitudinal in (mn)"}
    wrong = [(("gt", "m", "n"), ("p", "v")), (("gt", "v", "n"), ("p", "m"))]
    v3 = {lab: str(sp.factor(slot(wrong, E, ext))) for lab, ext in EXT.items()}
    muts["M3_wrong_index_slot"] = {"E2_slot_nonzero": any(v != "0" for v in v3.values()),
                                   "rejected": not longitudinal("E2w", wrong), "slot_values": v3,
                                   "reason": "L2 fails for the mis-slotted structure (gt_mn p_nu term)"}
    for v in muts.values():
        v["accepted_as_rejection"] = bool(v["E2_slot_nonzero"] and v["rejected"])
    controls = {"E_covariant_form_exact": fitE["exact"], "E_covariant_form_unique": fitE["unique"],
```

```
                "L1_E_transverse": L1["transverse"], "L2_E2_longitudinal": longitudinal("E2"),
                "L3_E2_slot_zero_all_externals": E2_zero_everywhere, "L4_slot_map_nontrivial": nontrivial,
                "D13_A_T1_and_A_E1_zero": A["T1"] == 0 and A["E1"] == 0,
                "D13_slot_contribution_zero": all(v == "0" for v in d13_slot.values()),
                "mutations_all_rejected": all(v["accepted_as_rejection"] for v in muts.values())}
    proven = all(controls.values())
    rec = {"landmark": "XVI-P", "revision": QC.REVISION, "deliverable": "p16 priority 1: E2 evanescent pole vs
  +>  the ch slot",
           "question": "does the D13 E2 pole (1/16; A_E2 = 3/(16(d-1))) decouple from the ch-projected slot?",
           "outcome": "DECOUPLED (proof)" if proven else "OPEN (proof incomplete)",
           "E2_status": "decoupled_from_ch_slot_proven" if proven else "open",
           "slot": {"row": "(S0, Gamma1^[2]) s1 h:h^2 (ledgers/P13_ch_dependency_rows.json; the only row conta
  +> ining D13)",
                    "functional": "X_D13[c,h] = int G_{(mn) nu}(p) c^nu(p) E^{(mn)(rs)}(p) h_(rs)(-p)",
                    "E": "operator of the h^2 monomials of S_Psi^(0) (counterterm_domain.S0_density, pw_vertex
  +>  "
                         "extraction; exact covariant form recovered, unique)",
                    "E_fit": {k: v for k, v in fitE.items() if k != "terms"},
                    "E_covariant": sorted(([str(c), [list(f) for f in fa]] for c, fa in E), key=json.dumps)[:4
  +> 0]},   # deterministic
           "proof": {"L1": "p_m E^{(mn)(rs)} = 0 (covariant identity; only g4.p = p used, no trace) -> " + str
  +> (L1),
                     "L2": "E2_{(mn)nu} = (gt_{m nu} p_n + gt_{n nu} p_m) p^2 is longitudinal in (mn)",
                     "L3": "E2 . E = 0 identically: slot value 0 for every external assignment, exact in d",
                     "L4": "T1, E1 (non-longitudinal) give nonzero slot values; A_T1 = A_E1 = 0 for D13",
                     "order": ("one loop: the ch projection of the one-loop identity is LINEAR in Gamma1; the
  +> E2 "
                               "contribution is exactly zero (not O(eps)); no product of evanescent poles aris
  +> es "
                               "at this order")},
           "slot_table": table, "external_assignments": {k: list(v) for k, v in EXT.items()},
           "D13_slot_contribution": d13_slot,
           "consequence": ("the E2 pole does not enter the ch-projected identity; no evanescent counterterm is
  +>  needed "
                           "for the slot, so NO contract revision is made for E2.  The E2 pole remains a UV po
  +> le of the "
                           "epsilon-scalar (Gt) component of the K'.c Green function of Gamma1 (R6: retained,
  +> recorded); "
                           "its renormalization is relevant only outside the ch projection (other projections,
  +>  higher "
                           "loops) and is NOT claimed here" if proven else
                           "proof incomplete: an evanescent counterterm is required (contract revision)"),
           "D13_graph_status": "regulated (unchanged: the E2 pole is not absorbed by a declared counterterm);
  +> "
                               "ch-slot status: zero contribution (proven)",
           "mutations": muts, "controls": controls, "E2_decoupling_proven": proven,
           "authorised_d_substitutions": dict(dred.D_PROVENANCE),
           "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_E2_evanescent.json", rec)
    print(json.dumps({"outcome": rec["outcome"], "controls": controls, "E2": table["E2"], "T1": table["T1"],
                      "E1": table["E1"], "D13_slot": d13_slot,
                      "mutations": {k: [v["E2_slot_nonzero"], v["rejected"]] for k, v in muts.items()},
                      "seconds": rec["seconds"]}, indent=1))
    return 0 if proven else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

136 Source: batchC.py (p16 Batch C: D01, D03)

[listing 103/133] code/p5_calc/batchC.py

```
#!/usr/bin/env python3
"""XVI-P p16 Batch C: D01 (h^3-h^3 graviton bubble) and D03 (closed ghost loop), the kappa^2 hh two-point grap
  +> hs of
the (S0, Gamma1^[2]) s2 K:K.c row, evaluated under the frozen p14r prescription (DRED R1-R6, Feynman +i0, MS-b
  +> ar,
S0) with d opaque (dred.substitute_d only).  D11 (scalar loop) is NOT evaluated (M2 undischarged).

Convention.  G_{(mn)(rs)}(p) = coefficient tensor of the ordered two-point vertex V_Gamma1[h_(mn)(p), h_(rs)(-
  +> p)] with
UPPER external indices (dual to the lower field components h_(mn); exported component (a, b) = weight(a) weigh
  +> t(b)
x tensor component, weight 2 for an off-diagonal field component), G = kappa^2 N_eps Ghat, N_eps = i S_eps/(16
  +>  pi^2),
Ghat = -i * prefactor/kappa^2 * int N/(D1 D2) = sum_s A_s(d) s MB,  MB = 1/eps + 2 - L,  L = log(-p^2/mu^2 - i
  +> 0).
```

```
Routes (covariant.py; the p14r component route is NOT used for rank-4 numerators -- finding C1):
  C1 exact covariant form of the frozen export (polynomial identity on all 100 components; unique ansatz solut
  +> ion);
  C3a Feynman parameters (no S0, no Gram);  C3b frozen dred.reduce_tensor (Gram/PV) + certified J library (S0)
  +> ;
  R  integrand REGENERATED from S_Psi (feynman.pw_vertex, own leg loops, rule-table S_f / loop sign / propagat
  +> or i's)
     and compared with the export as an exact polynomial identity (G = prefactor * N).
"""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dred  # noqa: E402
import integrals as IL  # noqa: E402
import covariant as C  # noqa: E402
import p16_conventions as QC  # noqa: E402

d, eps = dred.d, dred.eps
I = sp.I
KAPPA = sp.Symbol("kappa")
LS = sp.symbols("l0:4")
PS = sp.symbols("p0:4")
HC = C.HC
SPEC = [("pair", ("m", "n")), ("pair", ("r", "s"))]
PAIRS = [("m", "n"), ("r", "s")]
GRAPHS = ("D01", "D03")
SLOT = "kappa^2 hh two-point (row (S0,Gamma1^[2]) s2 K:K.c with D02, D11, D12; D11 NOT evaluated)"


def log(msg):
    print(f"[batchC {time.strftime('%H:%M:%S')}] {msg}", flush=True)


def load(g):
    import batchA
    return batchA.load_export(g)


def norm_of(pref):
    return sp.simplify(-I * pref / KAPPA ** 2)


# ------------------------------------------------------------------------------------------ regeneratio
  +> n
_V = {}


def _vt():
    import feynman as F
    if not _V:
        _V["F"] = F
        _V["X"] = F.S_psi_parts()["X"][1]
        P = F.propagators()
        _V["Pinv"] = P["graviton"]["Pinv"]
        _V["Gh"] = P["ghost"]["numerator"]
    return _V


def ell_density(k):
    """the EH vertex density ell_k (frozen export) as a GE density (same construction as
    counterterm_domain.S0_density for ell_2); pw_vertex then performs its own leg extraction."""
    import feynman as F
    from graded_jets import GE, esym
    e, _ = F.PW.load_ell(k)
    sub = {}
    for (m, n) in F.HC:
        sub[sp.Symbol(f"H{m}{n}")] = sp.Symbol(f"h{m}{n}")
        for r in range(4):
            sub[sp.Symbol(f"dH{m}{n}_{r}")] = sp.Symbol(f"h{m}{n}_d{r}")
    for (m, n) in F.HC:
```

```
        esym(("h", (m, n), ()))
        for r in range(4):
            esym(("h", (m, n), (r,)))
    return GE.const(e.xreplace(sub))


def mom(cl, cp):
    return [cl * LS[r] + cp * PS[r] for r in range(4)]


def regen(g, S_f=None, loop_sign=None):
    """G_regen = pref2 * N2 (component polynomials), own extraction from S_Psi."""
    V = _vt()
    F, X, Pinv, Gh = V["F"], V["X"], V["Pinv"], V["Gh"]
    hleg = lambda a, k: {"f": "h", "pol": {HC[a]: 1}, "k": k}  # noqa: E731
    N = {}
    if g == "D01":
        X = ell_density(3)
        S_f = sp.Rational(1, 2) if S_f is None else S_f
        loop_sign = 1 if loop_sign is None else loop_sign
        VA, VB = {}, {}
        for a in range(10):
            for c in range(10):
                for e in range(10):
                    v = F.pw_vertex(X, [hleg(a, mom(0, 1)), hleg(c, mom(-1, 0)), hleg(e, mom(1, -1))])
                    if v != 0:
                        VA[(a, c, e)] = v
                    w = F.pw_vertex(X, [hleg(a, mom(0, -1)), hleg(c, mom(1, 0)), hleg(e, mom(-1, 1))])
                    if w != 0:
                        VB[(a, c, e)] = w
        pv = [(c, dd, Pinv[c, dd]) for c in range(10) for dd in range(10) if Pinv[c, dd] != 0]
        WB = {}
        for (b, dd, f), poly in VB.items():
            for c, d2, pcd in pv:
                if d2 != dd:
                    continue
                for e, f2, pef in pv:
                    if f2 != f:
                        continue
                    WB[(b, c, e)] = WB.get((b, c, e), 0) + poly * pcd * pef
        for (a, c, e), pa in VA.items():
            for b in range(10):
                wb = WB.get((b, c, e))
                if wb is not None:
                    N[(a, b)] = N.get((a, b), 0) + pa * wb
        pref2 = S_f * loop_sign * I ** 2 * I ** 2          # vertices (i kappa)^2 / kappa^2 ; two graviton pro
  +> ps i
    elif g == "D03":
        S_f = 1 if S_f is None else S_f
        loop_sign = -1 if loop_sign is None else loop_sign
        VA, VB = {}, {}
        for rho in range(4):
            for mu in range(4):
                for a in range(10):
                    v = F.pw_vertex(X, [{"f": "cb", "c": rho, "k": mom(-1, 0)}, {"f": "c", "c": mu, "k": mom(1
  +> , -1)},
                                        hleg(a, mom(0, 1))])
                    if v != 0:
                        VA[(rho, mu, a)] = v
                    w = F.pw_vertex(X, [{"f": "cb", "c": rho, "k": mom(-1, 1)}, {"f": "c", "c": mu, "k": mom(1
  +> , 0)},
                                        hleg(a, mom(0, -1))])
                    if w != 0:
                        VB[(rho, mu, a)] = w
        gh = [(m, r, Gh[m, r]) for m in range(4) for r in range(4) if Gh[m, r] != 0]
        for (rho, mu, a), pa in VA.items():
            for m2, s, g1 in gh:
                if m2 != mu:
                    continue
                for (s2, nu, b), pb in VB.items():
                    if s2 != s:
                        continue
                    for n2, r2, g2 in gh:
                        if n2 == nu and r2 == rho:
                            N[(a, b)] = N.get((a, b), 0) + pa * g1 * pb * g2
        pref2 = S_f * loop_sign * I ** 2                    # ghost numerators carry their i (M0^{-1})
    else:
        raise ValueError(g)
    return {k: sp.expand(pref2 * v) for k, v in N.items() if sp.expand(v) != 0}
```

```
def regen_equals_export(g, body, G2):
    pref = sp.sympify(body["prefactor"], locals={"kappa": KAPPA}) / KAPPA ** 2
    bad = 0
    for a in range(10):
        for b in range(10):
            e = C.export_poly(body["numerator"].get(f"{a},{b}", []), LS, PS) * pref
            if sp.expand(e - G2.get((a, b), 0)) != 0:
                bad += 1
    return {"components": 100, "mismatches": bad, "agree": bad == 0}


# ---------------------------------------------------------------------------------------------- evaluation
def evaluate_graph(g, fit=None, drop_evanescent_in=None):
    body, pin = load(g)
    pref = sp.sympify(body["prefactor"], locals={"kappa": KAPPA})
    if fit is None:
        fit = C.covariant_fit(body["numerator"], SPEC, 4, offdiag=2, upper=True)
    res = {}
    for meth in ("fp", "pv"):
        r = C.evaluate(fit["terms"], meth, g)
        if drop_evanescent_in == meth:
            r = {k: v for k, v in r.items() if not any(f[0] == "gt" for f in k)}
        o, ok = C.by_orbit(r, PAIRS, norm_of(pref))
        res[meth] = (o, ok)
    o1, o2 = res["fp"][0], res["pv"][0]
    agree = set(o1) == set(o2) and all(sp.cancel(o1[k] - o2[k]) == 0 for k in o1)
    return {"body": body, "pin": pin, "pref": pref, "fit": fit, "A": o1, "A_pv": o2, "routes_agree": agree,
            "orbit_consistent": res["fp"][1] and res["pv"][1]}


def exchange_symmetric(A):
    """(mn) <-> (rs) exchange symmetry of the result (Bose symmetry of the hh two-point function)."""
    sw = {"m": "r", "n": "s", "r": "m", "s": "n"}
    group = C.sym_group(PAIRS)
    for (orb, e), cf in A.items():
        img = C.orbit(C.relabel(sorted(orb)[0], sw), group)
        if sp.cancel(A.get((img, e), 0) - cf) != 0:
            return False
    return True


def hh_basis():
    """the p14r h.h counterterm directions (CT.sector_basis('h','h',4)) as ordered two-point vertices
    V[h_(mn)(p), h_(rs)(-p)] (same extraction as the export), written covariantly (same convention as G)."""
    import counterterm_domain as CT
    import feynman as F
    dens = CT.sector_basis("h", "h", 4)
    keep, _ = CT.independent(dens, [("h", "h")])
    out = []
    for dn in keep:
        polys = {}
        for a in range(10):
            for b in range(10):
                v = F.pw_vertex(dn, [{"f": "h", "pol": {HC[a]: 1}, "k": list(PS)},
                                     {"f": "h", "pol": {HC[b]: 1}, "k": [-x for x in PS]}])
                if v != 0:
                    polys[f"{a},{b}"] = v
        fit = C.covariant_fit({}, SPEC, 4, offdiag=2, upper=True, polys=polys)
        coords = {}
        for orb, sc, c in fit["terms"]:
            e = sp.degree(sc, C.PP)
            coords[(orb, e)] = coords.get((orb, e), 0) + c
        out.append({"density": str(dn)[:200], "fit_exact": fit["exact"], "coords": coords})
    return out


def four_d_keys(A):
    return sorted([k for k in A if not C.is_evanescent(k[0])], key=lambda k: C.label(*k))


def pole_in_hh_basis(vec, basis):
    """solve sum_k z_k basis_k = vec (4D structures; covariant coordinates; the basis vertex is i x density)."
  +> ""
    keys = sorted(set(vec) | {k for b in basis for k in b["coords"]}, key=lambda k: C.label(*k))
    zs = sp.symbols(f"z0:{len(basis)}")
    eqs = [sp.expand(sum(z * b["coords"].get(k, 0) for z, b in zip(zs, basis)) - vec.get(k, 0)) for k in keys]
    sol = sp.solve(eqs, zs, dict=True)
    if not sol:
        return None, False
    return [sp.simplify(sol[0].get(z, z)) for z in zs], all(not sp.sympify(sol[0].get(z, z)).free_symbols
                                                            for z in zs)
```

```
def component_route_unfaithful():
    """demonstration of finding C1: sum_rs eta^rr eta^ss Gt_rs Gt_rs (abstract value tr(gtilde.gtilde) = 4 - d
  +> )
    under the linear realization Gt = (4-d) M is realization-dependent and != 4 - d."""
    vals = []
    for pn in ([3, 1, -2, 5], [sp.Rational(7, 2), -1, 4, 2]):
        for sd in (11, 12):
            gm = IL.realization([sp.Integer(x) for x in pn], sd)
            e = sum(dred.ETA[r] * dred.ETA[s] * dred.Gt(r, s) ** 2 for r in range(4) for s in range(4))
            vals.append(sp.factor(e.xreplace(gm)))
    distinct = len({str(v) for v in vals})
    return {"expression": "sum_rs eta^rr eta^ss Gt_rs Gt_rs", "abstract_value": "4 - d",
            "realized_values": [str(v) for v in vals], "realization_dependent": distinct > 1,
            "equals_abstract": all(sp.cancel(v - (4 - d)) == 0 for v in vals)}


def count_quadratic_component_terms(g, body):
    P = dred.Pipeline()
    n = 0
    for comp, rows in body["numerator"].items():
        tot = 0
        for names, cf in rows:
            lc = sorted(int(v[1]) for v in names if v.startswith("l"))
            pc = sp.Mul(*[sp.Symbol(v) for v in names if v.startswith("p")])
            tot += sp.Rational(cf) * pc * P.reduced_component(lc, with_p=True, tag=g)
        tot = sp.expand(tot)
        gts = [s for s in tot.free_symbols if s.name.startswith("Gt_")]
        if gts and max(sum(m) for m in sp.Poly(tot, *gts).monoms()) >= 2:
            n += 1
    return n


def s(x):
    return str(sp.simplify(x)) if not isinstance(x, str) else x


# ---------------------------------------------------------------------------------------------- main
def laurent_table(A):
    import d13 as D13
    out = {}
    for key in sorted(A, key=lambda k: C.label(*k)):
        ex = D13.expand_coeff(A[key])
        out[key] = ex
    return out


def longitudinal_projection(A):
    """slot role (row (S0,Gamma1^[2]) s2 K:K.c): s h_(mn) = d_(m c_n) is longitudinal, so only p_m G^{(mn)(rs)
  +> }
    enters; returns {label of remaining structure (free n, r, s): coefficient(d) of MB}."""
    import e2_evanescent as E2M
    terms = []
    for (orb, e), cf in A.items():
        for facs in orb:
            terms.append((cf * C.PP ** e, E2M.to_dred(list(facs)) + [("v", "p", "m")]))
    res = E2M.contract_all(terms)
    out = {}
    for k, v in res.items():
        lab = " ".join(f"{f[1] if f[0] == 'g' else f[1]}_{''.join(f[2:])}" for f in k)
        out[lab] = sp.factor(v)
    return out


def mutations(evs, regens):
    out = {}
    # M1 wrong ghost-loop sign (D03): regenerated with loop sign +1 != export
    Gm = regen("D03", loop_sign=1)
    cm = regen_equals_export("D03", evs["D03"]["body"], Gm)
    out["wrong_ghost_loop_sign_D03"] = {"changes_result": True, "rejected": not cm["agree"],
                                        "reason": f"regenerated (loop sign +1) != frozen export: {cm['mismatch
  +> es']} "
                                                  "components (the result flips sign: A -> -A)"}
    # M2 wrong symmetry factor (D01: S_f = 1 instead of 1/2); S_f enters only the prefactor -> 2 x regenerated
    Gs = {k: 2 * v for k, v in regens["D01"].items()}
    cs = regen_equals_export("D01", evs["D01"]["body"], Gs)
    out["wrong_symmetry_factor_D01"] = {"changes_result": True, "rejected": not cs["agree"],
                                        "reason": f"S_f = 1 regeneration != export: {cs['mismatches']} compone
  +> nts"}
    # M3 premature d = 4 (structural R5 safeguard)
```

```
    k0 = next(iter(evs["D01"]["A"]))
    a0 = evs["D01"]["A"][k0]
    try:
        sp.sympify(a0).subs(d, 4)
        out["premature_d4"] = {"rejected": False, "changes_result": None}
    except dred.PrematureD4Substitution as exc:
        lt = laurent_table({k0: a0})[k0]
        out["premature_d4"] = {"rejected": True,
                               "changes_result": any(laurent_table({k: v})[k]["d_dependence_shift_of_local_par
  +> t"] != 0
                                                     for g in GRAPHS for k, v in evs[g]["A"].items()),
                               "reason": f"PrematureD4Substitution at the substitution site: {exc}",
                               "example": {"structure": C.label(*k0), "local_if_d4_first":
                                           str(lt["finite_if_d_set_to_4_first"]), "certified_finite": str(lt["
  +> finite"])}}
    # M4 dropped evanescent (gt -> 0 in the Feynman-parameter route only)
    ev4 = evaluate_graph("D01", fit=evs["D01"]["fit"], drop_evanescent_in="fp")
    out["drop_evanescent_gt"] = {"changes_result": set(ev4["A"]) != set(evs["D01"]["A"]),
                                 "rejected": not ev4["routes_agree"],
                                 "reason": "Feynman-parameter route with gt dropped != PV route (ghat kept): R
  +> 6 "
                                           "retention check fails"}
    # M5 wrong external index position (lower instead of upper) -> no exact covariant form
    f5 = C.covariant_fit(evs["D03"]["body"]["numerator"], SPEC, 4, offdiag=2, upper=False)
    out["wrong_external_index_position"] = {"changes_result": True, "rejected": not f5["exact"],
                                            "reason": f"covariant form with lower external indices: exact = "
                                                      f"{f5['exact']} ({f5['bad_components'][:2]})"}
    # M6 off-diagonal component weight 1 -> no exact covariant form
    f6 = C.covariant_fit(evs["D03"]["body"]["numerator"], SPEC, 4, offdiag=1, upper=True)
    out["wrong_offdiagonal_weight"] = {"changes_result": True, "rejected": not f6["exact"],
                                       "reason": f"off-diagonal weight 1: exact = {f6['exact']}"}
    # M7 the p14r component route with the linear Gt realization (finding C1)
    cr = component_route_unfaithful()
    out["component_route_linear_realization"] = {"changes_result": not cr["equals_abstract"],
                                                 "rejected": cr["realization_dependent"],
                                                 "reason": "Gt-quadratic component sums are realization-depend
  +> ent "
                                                           f"under Gt = (4-d) M: {cr['realized_values'][:2]} v
  +> s 4 - d"}
    for v in out.values():
        v["accepted_as_rejection"] = bool(v["rejected"] and v["changes_result"])
    return out


def main():
    import certio
    t0 = time.time()
    IL.LEDGER.clear()
    evs, regens, recs = {}, {}, {}
    for g in GRAPHS:
        log(f"{g}: covariant fit + FP/PV routes")
        evs[g] = evaluate_graph(g)
        log(f"{g}: regeneration from S_Psi")
        regens[g] = regen(g)
    s0 = list(IL.LEDGER)
    log("D13 control through the covariant route")
    import batchA
    b13, _ = batchA.load_export("D13")
    f13 = C.covariant_fit(b13["numerator"], [("pair", ("m", "n")), ("vec", "v")], 3)
    r13 = C.evaluate(f13["terms"], "fp", "D13")
    o13, _ = C.by_orbit(r13, [("m", "n")], norm_of(sp.sympify(b13["prefactor"], locals={"kappa": KAPPA})))
    cert13 = json.loads((ROOT / "certificates" / "P15_D13.json").read_text())["result"]
    want = {"T2": "g4_mv p_n (p.p)^1", "T3": "p_m p_n p_v", "E2": "gt_mv p_n (p.p)^1"}
    got = {C.label(*k): v for k, v in o13.items()}
    d13_ok = (len(got) == 3 and all(sp.cancel(got[lab] - sp.sympify(cert13[t]["A_exact_d_times_MB"],
                                                                    locals={"d": d})) == 0 for t, lab in want
  +> .items())
                and all(cert13[t]["A_exact_d_times_MB"] == "0" for t in ("T1", "E1")))
    log("p14r h.h counterterm basis")
    basis = hh_basis()
    log("component-route demonstration")
    comp_quad = {g: count_quadratic_component_terms(g, evs[g]["body"]) for g in GRAPHS}
    log("mutations")
    muts = mutations(evs, regens)
    graphs = {}
    ok_all = True
    total4 = {}
    for g in GRAPHS:
        ev = evs[g]
        A = ev["A"]
        lt = laurent_table(A)
```

```
      rg = regen_equals_export(g, ev["body"], regens[g])
      pole4 = {k: lt[k]["pole"] for k in four_d_keys(A)}
      for k, v in pole4.items():
          total4[k] = total4.get(k, 0) + v
      z, zok = pole_in_hh_basis(pole4, basis)
      ev_pole = {C.label(*k): str(lt[k]["pole"]) for k in A if C.is_evanescent(k[0])}
      ev_nonzero = any(lt[k]["pole"] != 0 for k in A if C.is_evanescent(k[0]))
      ctrl = {"export_sha_pinned": ev["pin"]["pinned"], "covariant_form_exact": ev["fit"]["exact"],
              "covariant_form_unique": ev["fit"]["unique"], "orbit_consistent": ev["orbit_consistent"],
              "route_FP_equals_route_PV_exact_in_d": ev["routes_agree"],
              "regenerated_integrand_equals_export": rg["agree"],
              "exchange_symmetric_mn_rs": exchange_symmetric(A),
              "nonlocal_equals_minus_pole": all(sp.simplify(lt[k]["nonlocal_L_coefficient"] + lt[k]["pole"])
+>  == 0
                                                for k in A),
              "pole_4D_in_p14r_hh_span": bool(zok),
              "R5_d_retained": any(sp.sympify(v).has(d) for v in A.values())}
      ok = all(ctrl.values())
      ok_all = ok_all and ok
      status = "renormalized" if ok and not ev_nonzero else ("regulated" if ok else "unevaluated")
      assert status in QC.STATUS_VALUES
      table = {C.label(*k): {"evanescent": C.is_evanescent(k[0]), "A_exact_d_times_MB": s(lt[k]["exact_d"]),
                             "UV_pole": s(lt[k]["pole"]), "MSbar_finite": s(lt[k]["finite"]),
                             "nonlocal_coefficient_of_L": s(lt[k]["nonlocal_L_coefficient"]),
                             "local_polynomial": s(lt[k]["local"]),
                             "finite_if_d_set_to_4_first": s(lt[k]["finite_if_d_set_to_4_first"]),
                             "local_shift_from_d_dependence": s(lt[k]["d_dependence_shift_of_local_part"])}
               for k in sorted(A, key=lambda k: C.label(*k))}
      graphs[g] = {"slot": SLOT, "export": ev["pin"], "prefactor_export": str(ev["pref"]),
                   "normalization": f"Ghat = {norm_of(ev['pref'])} * int N/(D1 D2) / N_eps",
                   "covariant_form": {k: v for k, v in ev["fit"].items() if k != "terms"},
                   "result": table, "status": status,
                   "sector_status": {"4D": "renormalized (pole in the p14r h.h span)" if ok else "regulated"
+> ,
                                     "evanescent": ("regulated: evanescent poles NONZERO, retained (R6); no
+> "
                                                    "evanescent h.h counterterm is declared" if ev_nonzero
                                                    else "pole zero")},
                   "UV_pole_in_p14r_hh_basis": [s(x) for x in z] if z else None,
                   "evanescent_poles": ev_pole, "evanescent_pole_nonzero": ev_nonzero,
                   "longitudinal_projection_p_m_G": {k: str(v) for k, v in sorted(longitudinal_projection(A)
+> .items())},
                   "regeneration": rg, "component_route_Gt_quadratic_components": comp_quad[g],
                   "controls": ctrl, "evaluated": ok}
  controls = {"D13_reproduced_by_covariant_route": d13_ok,
              "hh_basis_fits_exact": all(b["fit_exact"] for b in basis) and len(basis) == 5,
              "R5_structural_safeguard_active": muts["premature_d4"]["rejected"],
              "mutations_all_rejected": all(v["accepted_as_rejection"] for v in muts.values()),
              **{f"{g}_evaluated": graphs[g]["evaluated"] for g in GRAPHS}}
  batchC_ok = ok_all and all(controls.values())
  zt, ztok = pole_in_hh_basis(total4, basis)
  rec = {"landmark": "XVI-P", "revision": QC.REVISION, "deliverable": "P.5 calculation Batch C: D01, D03 (ka
+> ppa^2 hh)",
         "prescription": "frozen p14r P14_DRED_MSbar_FeynmanI0 + S0 (P15_integral_library.json); covariant r
+> oute "
                         "(covariant.py, finding C1)",
         "convention": __doc__.split("Convention.")[1].split("Routes")[0].strip(),
         "finding_C1_component_route": {
             "statement": ("the p14r per-component R3 lift produces Gt-quadratic component sums for rank-4 "
                           "numerators (D01/D03) whose abstract value cannot be recovered after the componen
+> t sums; "
                           "the linear realization Gt = (4-d)M is not faithful there; Batch C therefore uses
+>  the "
                           "covariant route (C1-C5), which evaluates the same rules R1-R6 with free external
+>  indices "
                           "only"),
             "components_with_Gt_quadratic_terms": comp_quad, "demonstration": component_route_unfaithful(),
             "D13_unaffected": "rank 3, linear in Gt; reproduced exactly by the covariant route (control)",
             "contract_change": False},
         "graphs": graphs, "D01_plus_D03_4D_pole_in_p14r_hh_basis": [s(x) for x in zt] if zt else None,
         "hh_basis": [{"density": b["density"], "coords": {C.label(*k): str(v) for k, v in b["coords"].items
+> ()}}
                      for b in basis],
         "D11": "NOT evaluated (M2 undischarged); the hh slot is incomplete (D01 + D02 + D03 + D11 + D12)",
         "controls": controls, "mutations": muts,
         "batchC_evaluated": {
             "execution": batchC_ok,
             "claim": ("D01 and D03 evaluated in closed form under the declared prescription (covariant rout
+> e): "
                       "exact covariant integrands (verified on all 100 components, regenerated from S_Psi),
+>  two "
```

```
                        "independent loop-integration routes agree exactly in d, 4D UV poles local and in the
  +>  p14r "
                        "h.h span, MS-bar finite parts = nonlocal log + local quartic polynomial, evanescent
  +> poles "
                        "recorded (R6), mutations rejected") if batchC_ok else "not established",
              "scope": ("two graphs (D01, D03) of the kappa^2 hh slot; massless; MS-bar at mu; external momen
  +> ta in "
                        "QdS; evanescent structures retained (their poles are NONZERO and not absorbed: no ev
  +> anescent "
                        "h.h counterterm is declared); D11 NOT evaluated, so the hh slot is incomplete; NOT "
                        "quantum_inputs_evaluated")},
          "scaleless_rule_applications": sorted(s0, key=lambda r: json.dumps(r, sort_keys=True)),   # determi
  +> nistic order "authorised_d_substitutions": dict(dred.D_PROVENANCE),
          "loop_evaluations_performed": True, "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_batchC.json", rec)
    print(json.dumps({"batchC_evaluated": batchC_ok, "controls": controls,
                      "graphs": {g: {"status": graphs[g]["status"], "controls": graphs[g]["controls"],
                                     "z": graphs[g]["UV_pole_in_p14r_hh_basis"]} for g in GRAPHS},
                      "mutations": {k: [v["changes_result"], v["rejected"]] for k, v in muts.items()},
                      "seconds": rec["seconds"]}, indent=1))
    return 0 if batchC_ok else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

137 Source: release_p16.py (p16 release gate)

[listing 104/133] code/p5_calc/release_p16.py

```
#!/usr/bin/env python3
"""XVI-P p16 release gate: P.5 calculation, second step -- E2 evanescent analysis (priority 1) and Batch C (D0
  +> 1, D03),
on the p14r contract with the disclosed p15 (F1) and p16 (F3) dred.py revisions.

Runs AFTER the p14r contract gate (release_p14.py -> certificates/P14_release.json) and the p15 layer gate
(release_p15.py -> certificates/P15_release.json) and requires both to be accepted.  Recomputes:
  contract identity : p14r contract files byte-identical to p14r EXCEPT code/p5_entry/dred.py (disclosed p15 F
  +> 1 + p16
                      F3); NO new contract file (the E2 decoupling is proven, no evanescent counterterm added)
  +> ;
                      dred.py F3 hardening active (replace / lambdify / match refused outside substitute_d);
  p15 layer        : every p15 calculation file byte-identical to the p15 zip (QC.P15_CALC_SHA256); P15_releas
  +> e
                      accepted (Batch A, D13);
  E2               : P16_E2_evanescent.json -- decoupling proof controls all true, mutations rejected;
  batchC_evaluated : P16_batchC.json -- D01, D03 (covariant route, two loop routes, regeneration, hh basis, mu
  +> tations);
  hard stops       : QC.HARD_STOPS_P16 (D11 NOT evaluated; quantum_inputs_evaluated not_earned; ...).
Rebuild flags (package evidence): author/independent for the CURRENT (p16) tree; independent_rebuild_passed_p1
  +> 5 from
the archived p15 independent report, bound to the p15 tree 79f53752... / zip 54325c1f... (same-box caveat).
Change log (F5): every source file added/modified/removed relative to the p15 source-only zip, contract and
non-contract, is listed in the status (change_log_vs_p15).
"""
from __future__ import annotations
import hashlib, json, sys, zipfile
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p16_conventions as QC  # noqa: E402
import p15_conventions as Q15  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)

ARCH_P15 = "archival/p15"
ARCH_P15_INDEP = f"{ARCH_P15}/reproducibility/independent_rebuild_report_p15.json"
E2_MUTATIONS = ("M1_gauge_fixed_operator_qq2P", "M2_E1_in_place_of_E2", "M3_wrong_index_slot")
C_MUTATIONS = ("wrong_ghost_loop_sign_D03", "wrong_symmetry_factor_D01", "premature_d4", "drop_evanescent_gt",
               "wrong_external_index_position", "wrong_offdiagonal_weight", "component_route_linear_realizatio
  +> n")


def sha(p):
```

```
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def contract_identity():
    cur = {p.relative_to(ROOT).as_posix(): sha(p) for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    changed = sorted(k for k, h in Q15.P14R_CONTRACT_SHA256.items() if cur.get(k) != h)
    added = sorted(set(cur) - set(Q15.P14R_CONTRACT_SHA256))
    dred_changed_vs_p15 = cur.get("code/p5_entry/dred.py") != QC.P15_DRED_SHA256
    f3 = f3_hardening()
    ok = bool(changed == sorted(QC.P14R_CONTRACT_FILES_CHANGED_IN_P16) and added == sorted(QC.NEW_CONTRACT_FIL
  +> ES_P16)
              and dred_changed_vs_p15 and f3["all_refused"] and f3["authorised_path_works"])
    return ok, {"p14r_contract_files": len(Q15.P14R_CONTRACT_SHA256), "changed_vs_p14r": changed, "added": add
  +> ed,
                "dred_changed_vs_p15": dred_changed_vs_p15, "disclosed_revisions_p16": QC.DISCLOSED_CONTRACT_R
  +> EVISIONS_P16,
                "disclosed_corrections_p15": Q15.DISCLOSED_CONTRACT_CORRECTIONS, "F3_probe": f3,
                "evanescent_counterterm_added": False,
                "why_no_evanescent_counterterm": "E2 decoupling from the ch-projected slot proven (P16_E2_evan
  +> escent.json)"}


def f3_hardening():
    import dred
    import sympy as sp
    d = dred.d
    e = (d - 4) / (d - 1) + d ** 2
    probes = {"expr.replace(d, 4)": lambda: e.replace(d, 4), "sympy.lambdify(d, expr)": lambda: sp.lambdify(d,
  +>  e),
              "expr.match(d)": lambda: e.match(d), "expr.subs(d, 4)": lambda: e.subs(d, 4),
              "expr.xreplace({d: 4})": lambda: e.xreplace({d: 4}), "expr.evalf(subs={d: 4})": lambda: e.evalf(
  +> subs={d: 4})}
    res = {}
    for k, f in probes.items():
        try:
            f()
            res[k] = "NOT refused"
        except dred.PrematureD4Substitution:
            res[k] = "refused"
    auth = dred.substitute_d(e, 4, "release_p16.f3_hardening: authorised-path probe") == 16
    return {"probes": res, "all_refused": all(v == "refused" for v in res.values()), "authorised_path_works":
  +> auth}


def p15_layer():
    cur = {k: sha(ROOT / k) if (ROOT / k).is_file() else None for k in QC.P15_CALC_SHA256}
    ident = all(cur[k] == h for k, h in QC.P15_CALC_SHA256.items())
    r = jl("certificates/P15_release.json")
    ok = bool(ident and r.get("publication") == "accepted" and r.get("release_claim_accepted") is True
              and r.get("batchA_evaluated") is True and r.get("D13_evaluated") is True
              and r.get("library_certified") is True and r.get("hard_stops_hold") is True)
    return ok, {"p15_calc_files_byte_identical": ident, "P15_release_accepted": r.get("publication") == "accep
  +> ted",
                "batchA_evaluated": r.get("batchA_evaluated"), "D13_evaluated": r.get("D13_evaluated"),
                "D12_status": r.get("D12_status")}, r


def p15_independent():
    f = ROOT / ARCH_P15_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P15_INDEP, "present": False}
    rep = json.loads(f.read_text())
    live = ROOT / QC.P15_INDEPENDENT_REPORT
    rec = {"report": ARCH_P15_INDEP, "present": True, "report_sha256": sha(f),
           "report_sha256_pinned": sha(f) == QC.P15_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "bound_to_source_tree_sha256": rep.get("source_tree_sha256"),
           "bound_to_source_zip_sha256": rep.get("zip_sha256"),
           "tree_matches_p15": rep.get("source_tree_sha256") == QC.P15_SOURCE_TREE_SHA256,
           "zip_matches_p15": rep.get("zip_sha256") == QC.P15_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])), "findings": [x.get("id") for x in rep.get("findings",
  +>  [])],
           "caveat": rep.get("same_box_caveat"),
           "live_report_identical_to_archived": (live.read_bytes() == f.read_bytes()) if live.is_file()
           else "live report not in this tree",
           "NOT_inherited_by_p16": "bound to the p15 tree/zip; the p16 tree has its own (pending) independent
```

```
  +> flag"}
    rec["archived_p15_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P15_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p15_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p15"] and rec["zip_matches_p15"] and rec["mismatches"] == 0
              and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec


def e2():
    c = jl(f"certificates/{QC.CERT_PREFIX}_E2_evanescent.json")
    muts = c.get("mutations", {})
    ok = bool(c.get("revision") == QC.REVISION and c.get("E2_decoupling_proven") is True
              and all(v is True for v in c.get("controls", {}).values()) and len(c.get("controls", {})) >= 9
              and sorted(muts) == sorted(E2_MUTATIONS)
              and all(v.get("accepted_as_rejection") is True for v in muts.values()))
    return ok, c


def batch_c():
    c = jl(f"certificates/{QC.CERT_PREFIX}_batchC.json")
    g = c.get("graphs", {})
    muts = c.get("mutations", {})
    ok = bool(c.get("revision") == QC.REVISION and set(g) == set(QC.BATCH_C)
              and all(v.get("evaluated") is True and v.get("status") in QC.STATUS_VALUES
                      and v.get("status") != "unevaluated" for v in g.values())
              and all(all(x is True for x in v.get("controls", {}).values()) for v in g.values())
              and all(v is True for v in c.get("controls", {}).values())
              and sorted(muts) == sorted(C_MUTATIONS)
              and all(v.get("accepted_as_rejection") is True for v in muts.values())
              and c.get("batchC_evaluated", {}).get("execution") is True)
    return ok, c


def change_log():
    """F5: every source file added / modified / removed relative to the released p15 source-only zip.  The bas
  +> e is the
    pinned per-file list archival/p15/p15_source_files.json (so the log is identical in a source-only tree)."""
  +> "
    f = ROOT / QC.P15_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P15_ARCHIVED_RECORDS[QC.P15_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT) if r != SM.REFERENCE_FILE}   # reference: written a
  +> fter the build
    contract = set(Q15.P14R_CONTRACT_SHA256) | {p.relative_to(ROOT).as_posix() for p in (ROOT / "code" / "p5_e
  +> ntry").glob("*.py")}
    added = sorted(set(new) - set(old))
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])
    changed = sorted(added + removed + modified)
    return {"base": f"{QC.P15_SOURCE_LIST} (pinned; files of the p15 zip sha256 {QC.P15_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok, "excluded": f"{SM.REFERENCE_FILE} (new in p16; written after the author
  +>  build)", "added": added, "removed": removed, "modified": modified,
            "contract_files_changed": [x for x in changed if x in contract],
            "non_contract_files_changed": [x for x in changed if x not in contract],
            "counts": {"added": len(added), "removed": len(removed), "modified": len(modified)}}


def milestones(p15r, c_ok, e_ok):
    st = dict(p15r.get("milestones", {}).get("status", {}))
    st["P.5_calculation"] = ("in progress (p15: Batch A D02/D12/D14a-c and Batch B D13; p16: E2 decoupled from
  +>  the ch "
                             "slot (proof), Batch C D01/D03 evaluated; D11 matching M2 NOT evaluated)"
                             if c_ok and e_ok else "open (p16 evaluation not accepted)")
    st["P.5_identity"] = "open"
    return {"definitions": p15r.get("milestones", {}).get("definitions"), "order": p15r.get("milestones", {}).
  +> get("order"),
            "status": st, "P5_entry_earned": p15r.get("P5_entry_earned"),
            "P3_P4_frozen_as_P5_inputs": p15r.get("P3_P4_frozen_as_P5_inputs"),
            "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified
  +>  not earned)"}


def tex(x):
    """plain text-mode rendering (no math mode): ** -> ^, * kept, TeX specials escaped, typewriter font."""
    t = str(x).replace("**", "^").replace("\\", "/").replace("_", "\\_").replace("^", "\\^{}").replace("&", "\
  +> \&")
    return "\\texttt{" + t + "}"
```

```
def results_table(bc, ec):
    rows = []
    for g in QC.BATCH_C:
        for lab, v in bc["graphs"][g]["result"].items():
            rows.append(f"{g} & {tex(lab)} & {tex(v['A_exact_d_times_MB'])} & {tex(v['UV_pole'])} & "
                        f"{tex(v['MSbar_finite'])} \\\\")
    st = " ".join(f"{g}: {bc['graphs'][g]['status']}." for g in QC.BATCH_C)
    return ("% generated by code/p5_calc/release_p16.py -- do not edit\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{0.7cm}p{2.9cm}p{3.7cm}p{1.4cm}p{3.3cm}}\\hline\n"
            "graph & structure & $A(d)$ (coefficient of MB) & UV pole & MS-bar finite \\\\\\hline\n"
            + "\n".join(rows) + "\n\\hline\\end{tabular}\\end{center}\n"
            f"\\noindent Status: {st} E2 (D13): {ec.get('outcome')}.\n")


def _main():
    id_ok, ident = contract_identity()
    l15_ok, l15, p15r = p15_layer()
    e_ok, ec = e2()
    c_ok, bc = batch_c()
    p15i_ok, p15i = p15_independent()
    p14r = jl("certificates/P14_release.json")
    hard = dict(QC.HARD_STOPS_P16)
    graphs = dict(p15r.get("graphs", {}))
    for g in QC.BATCH_C:
        if c_ok:
            gg = bc["graphs"][g]
            graphs[g] = {"status": gg["status"], "sector_status": gg["sector_status"],
                         "UV_pole": {k: v["UV_pole"] for k, v in gg["result"].items()},
                         "MSbar_finite": {k: v["MSbar_finite"] for k, v in gg["result"].items()},
                         "UV_pole_in_p14r_hh_basis": gg["UV_pole_in_p14r_hh_basis"],
                         "evanescent_pole_nonzero": gg["evanescent_pole_nonzero"]}
        else:
            graphs[g] = {"status": "unevaluated"}
    graphs["D11"] = {"status": "unevaluated", "reason": "M2 undischarged (D11 matching not verified)"}
    if "D13" in graphs and e_ok:
        graphs["D13"]["E2_ch_slot"] = ec.get("E2_status")
    hard_ok = bool(all(hard[k] == v for k, v in QC.HARD_STOPS_P16.items())
                   and graphs.get("D11", {}).get("status") == "unevaluated"
                   and not any(c.get("quantum_inputs_evaluated") not in (None, "not_earned") for c in (ec, bc)
  +> )
                   and p14r.get("hard_stops_hold") is True and p15r.get("hard_stops_hold") is True)
    cl = change_log()
    # integrity (fail closed): the archived p15 records are pinned inputs; a tampered archive rejects publicat
  +> ion
    # (the VALUE of independent_rebuild_passed_p15 is package evidence, reported as a flag)
    execution_pass = bool(id_ok and l15_ok and e_ok and c_ok and hard_ok and cl["base_pinned_ok"]
                          and p15i.get("pins_ok") is True)
    publish = execution_pass
    author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
    indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
    m = milestones(p15r, c_ok, e_ok)
    flags = {"library_certified": p15r.get("library_certified"),
             "batchA_evaluated": p15r.get("flags", {}).get("batchA_evaluated"),
             "D13_evaluated": p15r.get("flags", {}).get("D13_evaluated"),
             "batchC_evaluated": {"execution": c_ok, "claim": bc.get("batchC_evaluated", {}).get("claim") if c
  +> _ok
                                  else "not established", "scope": bc.get("batchC_evaluated", {}).get("scope")
  +> },
             "E2_decoupled_from_ch_slot": {"execution": e_ok, "claim": ec.get("outcome") if e_ok else "not est
  +> ablished",
                                           "scope": ("the ch-projected one-loop kappa^2 identity (row (S0,Gamm
  +> a1^[2]) "
                                                     "s1 h:h^2); the E2 pole remains a UV pole of the epsilon-
  +> scalar "
                                                     "K'.c Green function outside this projection")}}
    cert = {
        "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
  +> TE,
        "version_label": QC.VERSION_LABEL,
        "deliverable": "P.5 calculation, second step (E2 evanescent analysis; Batch C: D01, D03)",
        "execution_pass": execution_pass,
        "claim_status": ("P.5_calculation: Batch A, D13 (p15) and Batch C D01, D03 (p16) evaluated under the d
  +> eclared "
                         "prescription; E2 pole decoupled from the ch-projected slot (proof); P.5_entry: earne
  +> d "
                         "(p14r contract); classical_readiness (frozen P.4): earned") if execution_pass else
                        "P.5_calculation: not accepted",
        "scope": ("P.5 calculation: Batch A (D02, D12, D14a-c), Batch B (D13) and Batch C (D01, D03) evaluated
  +>  under "
                  "the frozen prescription (DRED R1-R6, Feynman +i0, MS-bar, S0; d opaque); D11 NOT evaluated
  +> (M2 "
```

```
                "undischarged), so the hh slot is incomplete and no slot of the identity is complete; NOT qu
+> antum "
                "inputs, NOT the projected defect/identity, NOT bulk restoration, NOT the insertion"),
      "release_claim_accepted": execution_pass,
      "release_claim_accepted_definition": ("p14r contract accepted AND p15 layer accepted and byte-identica
+> l AND "
                                            "contract identity (only dred.py changed vs p14r; F3 hardening a
+> ctive; "
                                            "no new contract file) AND E2 decoupling proof AND batchC_evalua
+> ted AND "
                                            "p16 hard stops"),
      "publication": "accepted" if publish else "REJECTED",
      "flags": flags, "batchA_evaluated": p15r.get("batchA_evaluated"), "D13_evaluated": p15r.get("D13_evalu
+> ated"),
      "batchC_evaluated": c_ok, "library_certified": p15r.get("library_certified"),
      "E2_status": ec.get("E2_status") if e_ok else "open", "E2_decoupling_proven": e_ok,
      "D01_evaluated": c_ok, "D03_evaluated": c_ok, "D11_evaluated": False,
      "graphs": graphs, "D12_status": p15r.get("D12_status"), "D12_status_derivation": p15r.get("D12_status_
+> derivation"),
      "one_point_slot_kappa1": p15r.get("one_point_slot_kappa1"),
      "D13_summary": p15r.get("D13_summary"),
      "E2_summary": {k: ec.get(k) for k in ("outcome", "E2_status", "proof", "consequence", "D13_graph_statu
+> s",
                                            "D13_slot_contribution", "controls")},
      "batchC_summary": {"finding_C1_component_route": bc.get("finding_C1_component_route"),
                         "D01_plus_D03_4D_pole_in_p14r_hh_basis": bc.get("D01_plus_D03_4D_pole_in_p14r_hh_ba
+> sis"),
                         "controls": bc.get("controls"), "D11": bc.get("D11")},
      "contract_identity": ident, "contract_revision_disclosed": True,
      "p15_layer": l15, "change_log_vs_p15": cl,
      "P5_entry_earned": p15r.get("P5_entry_earned"), "P5_entry_flags": p15r.get("P5_entry_flags"),
      "P3_P4_frozen_as_P5_inputs": p15r.get("P3_P4_frozen_as_P5_inputs"), "classical": p15r.get("classical")
+> ,
      "D11_matching_verified": False, "quantum_inputs_evaluated": "not_earned",
      "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": m,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p15": p15i_ok,
      "independent_rebuild_passed_p14r": p15r.get("independent_rebuild_passed_p14r"),
      "independent_rebuild_passed_p13": p15r.get("independent_rebuild_passed_p13"),
      "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p16
+> ) source "
                             "tree only (independent: pending until a reviewer report for this tree exists);
+>  "
                             f"independent_rebuild_passed_p15 refers to the p15 tree {QC.P15_SOURCE_TREE_SHA
+> 256} / zip "
                             f"{QC.P15_SOURCE_ZIP_SHA256} (same-box caveat) and is not inherited by p16; "
                             "independent_rebuild_passed_p14r / _p13 as recorded by the p15 layer"),
      "p15_independent_rebuild": p15i,
      "p14r_independent_rebuild": p15r.get("p14r_independent_rebuild"),
      "p13_independent_rebuild": p15r.get("p13_independent_rebuild"),
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p15_archive": ARCH_P15,
                          "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"},
      "XVI_P_registered": False, "XVI_M_sha256": p15r.get("XVI_M_sha256"), "XVI_M_pin_ok": p15r.get("XVI_M_p
+> in_ok"),
      "symmetry_restored": False, "bulk_symmetry_restored": False, "O1_closure": "NOT_CLOSED",
      "insertion_status": "Frozen_negative_obstruction_preserved", "XV_B_opened": False, "XIV_J_opened": Fal
+> se,
      "Route_B_discharged": False, "u_R_c_delta_added": False,
      "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
          n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
          ("P14_release.json", "P14_dred_rules.json", "P15_release.json", f"{QC.CERT_PREFIX}_E2_evanescent.j
+> son",
           f"{QC.CERT_PREFIX}_batchC.json") if (ROOT / "certificates" / n).is_file()},
  }
  cdir = ROOT / "certificates"
  txt = certio.dumps(cert)
  if publish:
      (ROOT / "RELEASE_STATUS.json").write_text(txt)
      (cdir / "RELEASE_STATUS.json").write_text(txt)
      (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
      (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
      certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
          "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
+> tatus)",
          **m, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
          "independent_rebuild_passed_p15": p15i_ok, "hard_stops": hard,
          "flags": {"batchA_evaluated": p15r.get("batchA_evaluated"), "D13_evaluated": p15r.get("D13_evaluat
+> ed"),
```

```
                    "batchC_evaluated": c_ok, "E2_decoupling_proven": e_ok}})
        (ROOT / "manuscript" / "body").mkdir(parents=True, exist_ok=True)
        (ROOT / "manuscript" / "body" / "P16-results-table.tex").write_text(results_table(bc, ec))
    else:
        (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(certio.dumps({
            "landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", "contract_identity_ok": i
  +> d_ok,
            "contract_identity": ident, "p15_layer_ok": l15_ok, "p15_layer": l15, "E2_ok": e_ok,
            "batchC_evaluated": c_ok, "hard_stops_hold": hard_ok}))
    print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
  +> ion"],
                      "contract_identity": id_ok, "changed_vs_p14r": ident["changed_vs_p14r"], "added": ident[
  +> "added"],
                      "p15_layer": l15_ok, "E2": e_ok, "batchC_evaluated": c_ok, "hard_ok": hard_ok,
                      "author": author_ok, "independent_p16": indep_ok, "independent_p15": p15i_ok,
                      "change_log_counts": cl["counts"]}, indent=1))
    return 0 if publish else 1


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        cdir = ROOT / "certificates"
        (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
            {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED",
             "error": f"{type(exc).__name__}: {exc}"}, indent=1) + "\n")
        print(f"release_p16 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

138 Source: tests/test__p16.py

[listing 105/133] tests/test_p16.py

```
"""XVI-P p16 tests: F3 hardening of the opaque d, F4 version label + value-block readback, E2 decoupling proof
  +> ,
Batch C (D01, D03), p16 status / hard stops / p15 independent record, F5 change log, p16 gate mutations."""
from __future__ import annotations
import ast
import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools"):
    sys.path.insert(0, str(ROOT / p))
import p16_conventions as QC  # noqa: E402
import p15_conventions as Q15  # noqa: E402
import dred as DR  # noqa: E402

CERT = ROOT / "certificates"
L = sp.Symbol("L")


def cj(name):
    return json.loads((CERT / name).read_text())


def rs():
    """p17 disclosed test revision: the p16 LAYER record (the top-level RELEASE_STATUS.json is the p17 status)
  +> ."""
    return json.loads((CERT / "P16_release.json").read_text())


# ============================================================ F3: the opaque d cannot be bypassed
```

```
@pytest.mark.parametrize("probe", ["replace", "lambdify", "match", "subs", "xreplace", "evalf", "limit"])
def test_f3_opaque_dimension_refuses_bypass(probe):
    d = DR.d
    e = (d - 4) / (d - 1) + d ** 2 + sp.Symbol("x")
    f = {"replace": lambda: e.replace(d, 4), "lambdify": lambda: sp.lambdify(d, e),
         "match": lambda: e.match(d), "subs": lambda: e.subs(d, 4),
         "xreplace": lambda: e.xreplace({d: 4}), "evalf": lambda: e.evalf(subs={d: 4}),
         "limit": lambda: sp.limit(e, d, 4)}[probe]
    with pytest.raises(DR.PrematureD4Substitution):
        f()


def test_f3_authorised_path_and_harmless_operations_still_work():
    d = DR.d
    e = (d - 4) / (d - 1) + d ** 2
    n0 = DR.D_PROVENANCE.get("test_p16: authorised", 0)
    assert DR.substitute_d(e, 4, "test_p16: authorised") == 16
    assert DR.D_PROVENANCE["test_p16: authorised"] == n0 + 1
    assert sp.factor(d ** 2 - 1).has(d) and str(e) and sp.latex(e) and sp.srepr(e)
    assert sp.series(1 / (d - 1), DR.eps).has(d)            # series in another symbol keeps d opaque
    x = sp.Symbol("x")
    assert (e + x).replace(x, 1).has(d) and (e + x).subs(x, 2).has(d)


def _forbidden_calls(path):
    """AST scan: lambdify/ufuncify/autowrap calls, any .eval( call, .replace(d, ...) with d as first argument.
  +> """
    tree = ast.parse(path.read_text())
    bad = []
    allowed = {"f3_hardening"}                  # release_p16's own probe of the guard

    def visit(node, fn=None):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            fn = node.name
        if isinstance(node, ast.Call) and fn not in allowed:
            f = node.func
            name = f.attr if isinstance(f, ast.Attribute) else f.id if isinstance(f, ast.Name) else None
            if name in ("lambdify", "ufuncify", "autowrap") or name == "eval" and isinstance(f, ast.Attribute)
  +> :
                bad.append(f"{path.name}:{node.lineno}:{name}")
            if name == "replace" and isinstance(f, ast.Attribute) and node.args:
                a = node.args[0]
                if (isinstance(a, ast.Name) and a.id == "d") or (isinstance(a, ast.Attribute) and a.attr == "d
  +> "):
                    bad.append(f"{path.name}:{node.lineno}:replace(d)")
                if isinstance(a, ast.Lambda):        # callable-query replace: cannot be hooked by the guard
                    bad.append(f"{path.name}:{node.lineno}:replace(callable)")
        for ch in ast.iter_child_nodes(node):
            visit(ch, fn)
    visit(tree)
    return bad


def test_f3_static_scan_no_unguarded_value_paths():
    """Paths the guard cannot hook (Poly(expr, d).eval, compiled code) are excluded by a static scan of the pa
  +> ckage."""
    files = sorted((ROOT / "code" / "p5_calc").glob("*.py")) + sorted((ROOT / "code" / "p5_entry").glob("*.py"
  +> ))
    bad = [b for f in files for b in _forbidden_calls(f)]
    assert not bad, bad


def test_f3_static_scan_detects_a_planted_violation(tmp_path):
    f = tmp_path / "planted.py"
    f.write_text("import sympy as sp\nfrom dred import d\ndef g(e):\n    return sp.Poly(e, d).eval(4), e.repla
  +> ce(d, 4), "
                 "sp.lambdify(d, e), e.replace(lambda x: x == d, lambda x: 4)\n")
    assert len(_forbidden_calls(f)) == 4


def test_f3_documented_limitation_callable_replace_is_not_hookable():
    """Honest record: a callable-query replace bypasses the runtime guard (no hook exists); it is excluded by
  +> the
    static scan above, and the limitation is disclosed in p16_conventions.DISCLOSED_CONTRACT_REVISIONS_P16."""
    d = DR.d
    with pytest.raises(DR.PrematureD4Substitution):    # p22 (disclosed test revision): the d == d hook of the
  +>  p22 runtime
        (d ** 2).replace(lambda x: x == d, lambda x: sp.Integer(4))          # guard now intercepts this form
    assert "callable query" in json.dumps(QC.DISCLOSED_CONTRACT_REVISIONS_P16)
```

```
def test_f3_contract_revision_disclosed():
    rev = QC.DISCLOSED_CONTRACT_REVISIONS_P16
    assert list(rev) == ["code/p5_entry/dred.py"] and "F3" in json.dumps(rev)
    cur = {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest()
           for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    assert sorted(k for k, h in Q15.P14R_CONTRACT_SHA256.items() if cur.get(k) != h) == ["code/p5_entry/dred.p
  +> y"]
    assert set(cur) == set(Q15.P14R_CONTRACT_SHA256)                                    # no new contract file
    assert cur["code/p5_entry/dred.py"] != QC.P15_DRED_SHA256                           # the p16 revision is r
  +> eal
    for k, h in QC.P15_CALC_SHA256.items():                                              # p15 layer byte-ident
  +> ical
        assert hashlib.sha256((ROOT / k).read_bytes()).hexdigest() == h, k


# ============================================================== F4: version label and value-block readback
def test_f4_footer_from_single_version_variable():
    pre = (ROOT / "manuscript" / "preamble.tex").read_text()
    assert "\\XVIPversion" in pre and "\\input{version.tex}" in pre
    code_lines = [ln for ln in pre.splitlines() if not ln.lstrip().startswith("%")]
    assert not any("XVI-P p1" in ln for ln in code_lines)                           # no literal label left
    import build as B
    src = (ROOT / "tools" / "build.py").read_text()
    assert "XVI-P p15 |" not in src and "XVI-P p14 |" not in src
    assert QC.VERSION_LABEL == f"XVI-P {QC.REVISION}"
    assert B.QC.VERSION_LABEL == f"XVI-P {B.QC.REVISION}"        # p17 disclosed revision: the CURRENT label (
  +> p17)


def test_f4_value_block_readback_detects_a_changed_value():
    import build as B
    for rel in B.VALUE_BLOCKS:
        if not (ROOT / rel).exists():
            pytest.skip(f"{rel} not built")
        txt = B.wrap_lines(B.ascii_clean((ROOT / rel).read_text()))
        page = txt[: len(txt) // 2] + "\n" + QC.VERSION_LABEL + " | page 7\n" + txt[len(txt) // 2:]   # a page
  +>  break mid-block
        ok = B.readback_values(page, ROOT, rel, page)
        assert ok["pass"] and ok["snippets"] > 0, (rel, ok)
        snips = B.value_snippets(ROOT, rel)
        victim = next(s for s in snips if (s in page) and                # p18 disclosed revision: a snippet t
  +> hat is
                      ('"' in s.split(": ", 1)[1] or s.split(": ", 1)[1] not in ("true", "false")))   # verbat
  +> im (unwrapped)
        k, v = victim.split(": ", 1)
        bad_page = page.replace(victim, k + ": " + json.dumps("MUTATED"), 1)
        assert bad_page != page and B.readback_values(bad_page, ROOT, rel, bad_page)["pass"] is False, rel


# ============================================================== E2: decoupling from the ch-projected slot
def test_E2_decoupling_proof_certificate():
    c = cj("P16_E2_evanescent.json")
    assert c["revision"] == "p16" and c["E2_decoupling_proven"] is True
    assert all(v is True for v in c["controls"].values()) and len(c["controls"]) >= 9
    for k in ("L1_E_transverse", "L2_E2_longitudinal", "L3_E2_slot_zero_all_externals", "L4_slot_map_nontrivia
  +> l",
              "E_covariant_form_exact", "E_covariant_form_unique", "D13_A_T1_and_A_E1_zero"):
        assert c["controls"][k] is True, k
    assert c["D13_slot_contribution"] and all(v == "0" for v in c["D13_slot_contribution"].values())
    assert c["E2_status"] == "decoupled_from_ch_slot_proven" and c["outcome"].startswith("DECOUPLED")
    assert sorted(c["mutations"]) == ["M1_gauge_fixed_operator_qq2P", "M2_E1_in_place_of_E2", "M3_wrong_index_
  +> slot"]
    for k, v in c["mutations"].items():
        assert v["E2_slot_nonzero"] is True and v["rejected"] is True and v["accepted_as_rejection"] is True,
  +> k


def test_E2_consistent_with_p15_D13_values():
    r = cj("P15_D13.json")["result"]
    d = DR.d
    a = sp.sympify(r["E2"]["A_exact_d_times_MB"], locals={"d": d})
    assert sp.cancel(a - sp.Rational(3, 16) / (d - 1)) == 0
    assert DR.substitute_d(a, 4, "test_p16: E2 pole") == sp.Rational(1, 16)
    assert r["T1"]["A_exact_d_times_MB"] == "0" and r["E1"]["A_exact_d_times_MB"] == "0"


def test_E2_status_in_release_no_counterterm():
    s = rs()
    assert s["E2_decoupling_proven"] is True and s["E2_status"] == cj("P16_E2_evanescent.json")["E2_status"]
    assert s["contract_identity"]["evanescent_counterterm_added"] is False and s["contract_identity"]["added"]
  +>  == []
```

```
def test_p14_p15_certificate_values_unchanged_vs_p15_reference():
    """F3 changed dred.py: every P14/P15 certificate content hash (timings and run-specific keys ignored) equa
  +> ls the
    archived p15 author reference values."""
    import certio
    ref = json.loads((ROOT / "archival" / "p15" / "reference_values_p15.json").read_text())["values"]
    ch = ref["certificate_content_sha256_ignoring_timings"]
    run_specific = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed",
                    "live_report_identical_to_archived", "live_report_present", "archived_zip_sha256_ok",
                    "list_equals_archived_zip")
    diff = []
    for name, h in ch.items():
        if name.startswith(("P14_", "P15_", "P13_")) and name not in ("P15_release",):
            f = CERT / f"{name}.json"
            if certio.file_content_sha(f, run_specific) != h:
                diff.append(name)
    assert not diff, diff


# ============================================================== Batch C
def _laurent(A):
    """independent Laurent expansion of A(d) * MB, MB = 1/eps + 2 - L, d = 4 - 2 eps (authorised substitution)
  +> ."""
    eps = sp.Symbol("eps_t")
    e = DR.substitute_d(A, 4 - 2 * eps, "test_p16: independent Laurent check")
    ser = sp.series(e * (1 / eps + 2 - L), eps, 0, 1).removeO()
    return sp.simplify(ser.coeff(eps, -1)), sp.expand(ser.coeff(eps, 0))


def test_batchC_values_independent_laurent_check():
    c = cj("P16_batchC.json")
    d = DR.d
    for g in QC.BATCH_C:
        res = c["graphs"][g]["result"]
        assert len(res) == {"D01": 14, "D03": 12}[g]
        for lab, v in res.items():
            A = sp.sympify(v["A_exact_d_times_MB"], locals={"d": d, "I": sp.I})
            pole, fin = _laurent(A)
            assert sp.simplify(pole - sp.sympify(v["UV_pole"])) == 0, (g, lab)
            assert sp.simplify(fin - sp.sympify(v["MSbar_finite"], locals={"L": L})) == 0, (g, lab)
            assert sp.simplify(sp.sympify(v["nonlocal_coefficient_of_L"]) + pole) == 0
            assert v["evanescent"] == ("gt_" in lab)


def test_batchC_controls_status_and_scope():
    c = cj("P16_batchC.json")
    assert c["batchC_evaluated"]["execution"] is True and all(c["controls"].values())
    for g in QC.BATCH_C:
        G = c["graphs"][g]
        assert G["evaluated"] is True and all(G["controls"].values()) and len(G["controls"]) >= 10
        assert G["status"] == "regulated"                    # evanescent poles nonzero, retained (R6)
        assert G["evanescent_pole_nonzero"] is True and G["UV_pole_in_p14r_hh_basis"]
        assert G["regeneration"]["agree"] is True
    assert "NOT evaluated" in c["D11"] and "NOT quantum_inputs_evaluated" in c["batchC_evaluated"]["scope"]
    assert c["finding_C1_component_route"]["contract_change"] is False


def test_batchC_reference_values():
    c = cj("P16_batchC.json")["graphs"]
    I = sp.I
    d = DR.d
    want = {("D03", "g4_mn g4_rs (p.p)^2"): I * (d + 3) / (16 * (d - 1)),
            ("D03", "p_m p_n p_r p_s"): I * (d - 2) / (4 * (d - 1)),
            ("D01", "g4_mn g4_rs (p.p)^2"): -I * (2 * d ** 2 + 10 * d + 9) / (16 * (d - 1) * (d + 1)),
            ("D01", "p_m p_n p_r p_s"): -I * (13 * d ** 2 + 2 * d - 8) / (16 * (d - 1) * (d + 1)),
            ("D01", "gt_mn gt_rs (p.p)^2"): -I / (16 * (d - 1) * (d + 1))}
    for (g, lab), v in want.items():
        got = sp.sympify(c[g]["result"][lab]["A_exact_d_times_MB"], locals={"d": d, "I": I})
        assert sp.cancel(got - v) == 0, (g, lab)


def test_certificate_mutation_p16_batchC_mutations_rejected():
    m = cj("P16_batchC.json")["mutations"]
    assert len(m) == 7 and all(v["accepted_as_rejection"] is True for v in m.values())


# ============================================================== status, hard stops, rebuild flags, change l
  +> og
def test_p16_release_status_and_hard_stops():
```

```
    s = rs()
    assert s["revision"] == "p16" and s["publication"] == "accepted" and s["release_claim_accepted"] is True
    assert s["batchC_evaluated"] is True and s["D01_evaluated"] is True and s["D03_evaluated"] is True
    assert s["D11_evaluated"] is False and s["graphs"]["D11"]["status"] == "unevaluated"
    assert s["D11_matching_verified"] is False and s["quantum_inputs_evaluated"] == "not_earned"
    assert s["hard_stops"] == QC.HARD_STOPS_P16 and s["hard_stops_hold"] is True
    assert s["O1_closure"] == "NOT_CLOSED" and s["symmetry_restored"] is False and s["bulk_symmetry_restored"]
  +>  is False
    assert s["XVI_P_registered"] is False and s["Route_B_discharged"] is False and s["u_R_c_delta_added"] is F
  +> alse
    assert s["XV_B_opened"] is False and s["XIV_J_opened"] is False
    assert s["insertion_status"] == "Frozen_negative_obstruction_preserved"
    assert s["XVI_M_sha256"].startswith("41aa1099") and s["XVI_M_pin_ok"] is True
    assert s["milestones"]["status"]["P.5_identity"] == "open"
    for g in QC.BATCH_C:
        assert s["graphs"][g]["status"] == "regulated"
    assert s["independent_rebuild_passed"] in ("pending", False)   # p17 revision: bound to the p16 tree (laye
  +> r record)


def test_p15_independent_rebuild_recorded_and_not_inherited():
    s = rs()
    rec = s["p15_independent_rebuild"]
    assert s["independent_rebuild_passed_p15"] is True
    assert rec["report_sha256"] == QC.P15_INDEPENDENT_REPORT_SHA256 and rec["report_sha256_pinned"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P15_SOURCE_TREE_SHA256
    assert rec["bound_to_source_zip_sha256"] == QC.P15_SOURCE_ZIP_SHA256
    assert "SAME-BOX" in rec["caveat"] and rec["mismatches"] == 0
    assert all(rec["archived_p15_records_pinned"].values())
    import source_manifest as SM
    assert SM.tree_sha(ROOT) != QC.P15_SOURCE_TREE_SHA256


def test_f5_change_log_lists_every_changed_file():
    s = rs()
    cl = s["change_log_vs_p15"]
    assert cl["base_pinned_ok"] is True
    base = json.loads((ROOT / QC.P15_SOURCE_LIST).read_text())["files"]
    import source_manifest as SM
    cur = {r: hashlib.sha256((ROOT / r).read_bytes()).hexdigest() for r in SM.source_files(ROOT) if r != SM.RE
  +> FERENCE_FILE}
    changed = sorted(set(cur) ^ set(base) | {k for k in set(cur) & set(base) if cur[k] != base[k]})
    assert sorted(cl["added"] + cl["removed"] + cl["modified"]) == changed
    assert cl["contract_files_changed"] == ["code/p5_entry/dred.py"]
    tests_changed = sorted(x for x in cl["modified"] if x.startswith("tests/") and not __import__("contract_re
  +> gistry").p22_revised(x))   # p22 filter
    assert tests_changed == sorted(x for x in QC.DISCLOSED_TEST_REVISIONS_P16 if not __import__("contract_regi
  +> stry").p22_revised(x))


# ============================================================== p16 gate mutations (copy of the built packa
  +> ge)
def _copy():
    d = Path(tempfile.mkdtemp(prefix="p16gate_"))
    for sub in ("code", "certificates", "ledgers", "exports", "tools", "archival", "reproducibility", "provena
  +> nce", "tests"):
        if (ROOT / sub).exists():
            shutil.copytree(ROOT / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__"))
    for f in ("README.md", "REBUILD_INSTRUCTIONS.md", "RELEASE_STATUS.json"):
        if (ROOT / f).exists():
            shutil.copy(ROOT / f, d / f)
    (d / "manuscript").mkdir()
    shutil.copy(ROOT / "manuscript" / "preamble.tex", d / "manuscript" / "preamble.tex")
    return d


def _tamper_batchC(d):
    f = d / "certificates" / "P16_batchC.json"
    c = json.loads(f.read_text())
    c["graphs"]["D03"]["controls"]["route_FP_equals_route_PV_exact_in_d"] = False
    f.write_text(json.dumps(c, indent=1))


def _tamper_E2(d):
    f = d / "certificates" / "P16_E2_evanescent.json"
    c = json.loads(f.read_text())
    c["controls"]["L3_E2_slot_zero_all_externals"] = False
    f.write_text(json.dumps(c, indent=1))


def _tamper_p15_layer(d):
```

```
    f = d / "code" / "p5_calc" / "d13.py"
    f.write_text(f.read_text() + "\n# tampered\n")


def _tamper_contract(d):
    f = d / "code" / "p5_entry" / "counterterm_domain.py"
    f.write_text(f.read_text() + "\n# tampered\n")


def _tamper_p15_report(d):
    f = d / "archival" / "p15" / "reproducibility" / "independent_rebuild_report_p15.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


def _hard_stop(d):
    f = d / "certificates" / "P16_batchC.json"
    c = json.loads(f.read_text())
    c["quantum_inputs_evaluated"] = "earned"
    f.write_text(json.dumps(c, indent=1))


@pytest.mark.parametrize("mut", [_tamper_batchC, _tamper_E2, _tamper_p15_layer, _tamper_contract, _tamper_p15_
  +> report,
                                 _hard_stop], ids=lambda f: f.__name__)
def test_certificate_mutation_p16_gate_rejects(mut):
    d = _copy()
    try:
        mut(d)
        r = subprocess.run([sys.executable, "code/p5_calc/release_p16.py"], cwd=d, capture_output=True, text=T
  +> rue,
                           timeout=600)
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d / "certificates" / "P16_release.json").exists()
        assert (d / "certificates" / "P16_release_rejected.json").exists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_p16_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = subprocess.run([sys.executable, "code/p5_calc/release_p16.py"], cwd=d, capture_output=True, text=T
  +> rue,
                           timeout=600)
        assert r.returncode == 0, r.stdout[-1500:]
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

139 Source: p17_conventions.py (p17 constants, version label, pins, disclosures)

[listing 106/133] code/p5_calc/p17_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p17 conventions: P.5 calculation, third step (D01/D03 evanescent poles vs the ch-projected slot;
p16 reviewer findings G1-G6).

Layering.  p14r contract (code/p5_entry/*)  ->  p15 layer (byte-identical; P15_CALC_SHA256 in p16_conventions)
  +>   ->
p16 layer (code/p5_calc/{covariant,batchC,e2_evanescent,p16_conventions,release_p16}.py; byte-identical, sha25
  +> 6
below)  ->  p17 layer (code/p5_calc/{hh_slot,p17_conventions,release_p17}.py, P17_*).  The p17 gate (release_p
  +> 17.py)
runs after the p14r, p15 and p16 gates and requires all three to be accepted.

Disclosed CONTRACT revision in p17 (DISCLOSED_CONTRACT_REVISIONS_P17): code/p5_entry/dred.py only -- the guard
  +> ed
string loader parse_d / assert_opaque_d (G2) and corrected guard-coverage wording (G1, G3), on top of the p15
  +> F1 and
p16 F3 revisions.  NO evanescent h.h counterterm is added: the D01/D03 evanescent poles decouple from the
ch-projected slot as the SUM D01 + D03 (proof in certificates/P17_hh_slot.json); no new contract file.
"""
from __future__ import annotations

REVISION = "p17"
VERSION_LABEL = "XVI-P p17"           # single version variable (manuscript footer is checked against it)
CERT_PREFIX = "P17"
```

```
CONTRACT_REVISION = "p17 (p14r contract + disclosed p15/p16/p17 revisions)"
DATE = "2026-10-08"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p17.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p17.json"
P16_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p16.json"      # written by the reviewer;
  +>  never here
P16_INDEPENDENT_REPORT_SHA256 = "1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843"
P16_SOURCE_TREE_SHA256 = "90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e"
P16_SOURCE_ZIP_SHA256 = "56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57"
P16_ARCHIVED_RECORDS = {
    "archival/p16/reference_values_p16.json": "a1fda1e67a4a0033014f40b8bdac57b4daacfa507588e0765bb7139587b053d
  +> 9",
    "archival/p16/reproducibility/author_rebuild_report_p16.json":
        "b4f9bcf38244eb4f51d1ebbba335caf20c1f6cc405b7fca90f174808fa02691d",
    "archival/p16/reproducibility/independent_rebuild_report_p16.json": P16_INDEPENDENT_REPORT_SHA256,
    # per-file sha256 list of the released p16 source-only zip (base of the p17 change log; works source-only)
    "archival/p16/p16_source_files.json": "3e0412c82d161de998eb7bcdcb16fd01b608a6bcc48cc72d18409e2606715de8",
}
P16_SOURCE_LIST = "archival/p16/p16_source_files.json"

HH_SLOT_GRAPHS = ("D01", "D03")
SLOT_ROW = "(S0,Gamma1^[2]) s2 K:K.c"
NOT_EVALUATED_P17 = ("D11",)                       # M2 undischarged

HARD_STOPS_P17 = {
    "quantum_inputs_evaluated": "not_earned",          # D11 open -> not every required input is evaluated
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": False,
    "symmetry_restored": False,
    "D11_matching_verified": False,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": False,
    "XIV_J_opened": False,
    "Route_B_discharged": False,
    "u_R_c_delta_added": False,
    "XVI_P_registered": False,
    "D11_evaluated": False,
}

# p16 layer: byte-identical in p17 (sha256 from XVI-P-p16-source-only.zip; release_p17 recomputes them)
P16_CALC_SHA256 = {
    "code/p5_calc/covariant.py": "71976d189280ac49a4513b8b0e6503d69930a28a4d5e991dee3f1bdb63cbf817",
    "code/p5_calc/batchC.py": "017154a961ee2539ffb038dc8c60107e6bf9a922e340e326abc3bb82cde6574c",
    "code/p5_calc/e2_evanescent.py": "2027b5ea9c29a89ca5c6fd74bd6507999415ba56939f2cf661f9b3505d405ea4",
    "code/p5_calc/p16_conventions.py": "915fd26ea2d59313b9bd20208a6b360bb29809a4a9487fb5f912c49f917e0f71",
    "code/p5_calc/release_p16.py": "b016c1c5833970e0c891c55aaf6f318d449a5ab7d4c79819fb12b3f8cfd60736",
}
P14R_CONTRACT_FILES_CHANGED_IN_P17 = ("code/p5_entry/dred.py",)
NEW_CONTRACT_FILES_P17 = ()                        # no evanescent h.h counterterm file (decoupling proven)
P16_DRED_SHA256 = "57f6159aa8946f5a69781cf098227489fa07b66fd150fc6c58f8489df064a725"     # dred.py as released
  +>  in p16

DISCLOSED_CONTRACT_REVISIONS_P17 = {
    "code/p5_entry/dred.py": {
        "kind": "contract revision (disclosed), p17 (on top of the p15 F1 and p16 F3 revisions)",
        "finding": "G1/G2/G3, reproducibility/independent_rebuild_report_p16.json",
        "why": ("G2: sympify of a stored string yields a plain Symbol('d') that is not the opaque d (subs/has
  +> against "
                "dred.d then silently do nothing); G1/G3: the p16 docstring/disclosure overstated the coverage
  +>  of the "
                "guard and of the static scan"),
        "change": ("new guarded loader dred.parse_d(text) (locals {'d': opaque d}; refuses a result with a pla
  +> in "
                   "symbol named d), dred.assert_opaque_d(expr) and exception PlainDimensionSymbol; docstring
  +> now "
                   "states exactly what the runtime guard refuses and what it does not (Wild-pattern match, C/
  +> Fortran "
                   "printers); unhookable value paths (replace with ANY function query, string round-trips via
  +>  "
                   "sympify/S/parse_expr/eval/exec, srepr+eval, Poly(..).eval, e.func(*args) rebuilds) are exc
  +> luded "
                   "by the extended static AST scan (tests/test_p17.py), a pattern check, not a proof"),
        "rules_R1_R6_changed": False,
        "released_values_affected": False,
        "why_unaffected": ("only functions were added (no existing code path changed); the P14/P15/P16 certifi
  +> cate "
                           "content hashes recomputed in p17 equal those of archival/p16/reference_values_p16.
  +> json "
```

```
                          "(tests/test_p17.py)"),
    },
}
DISCLOSED_TEST_REVISIONS_P17 = {    # test files changed vs the p16 zip (release_p17 change log)
    "tests/test_p14.py": ("source-set / zip / reference names p16 -> p17 (SM.ZIP_NAME, SM.REFERENCE_FILE, auth
  +> or report "
                          "path, resumed-build placeholder zip); generated-prefix list extended by P17_* certi
  +> ficates/"
                          "ledgers and the P17 results table; top-level status revision p17. No assertion weak
  +> ened."),
    "tests/test_p16.py": ("rs() reads the p16 LAYER record certificates/P16_release.json (the top-level "
                          "RELEASE_STATUS.json is now the p17 status); the footer test checks that the build's
  +>  CURRENT "
                          "label is 'XVI-P <current revision>' (no longer equal to the p16 label); the p16 lay
  +> er's own "
                          "independent flag may be False (the live p16 report is bound to the p16 tree) or pen
  +> ding. "
                          "Three lines; all other p16 assertions unchanged."),
}
# G1: package call sites of sympify / S with a non-literal argument, reviewed: each receives a sympy object (n
  +> ever a
# stored string) and would FAIL CLOSED on a string through the downstream opaque-d checks; any new site fails
  +> the scan.
PARSE_SITES_REVIEWED = {
    'code/p5_calc/batchA.py::evaluate::sp.sympify(body["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::regen_equals_export::sp.sympify(body["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::evaluate_graph::sp.sympify(body["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::main::sp.sympify(b13["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/d13.py::route1::sp.sympify(body["prefactor"], locals={"kappa": sp.Symbol("kappa")})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_entry/hessian_route.py::_export_side::sp.sympify(body["prefactor"], locals={"kappa": F.KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::pole_in_hh_basis::sp.sympify(sol[0].get(z, z))':
        'argument is a sympy object computed in-process (never a stored string) (sympy.solve output)',
    'code/p5_calc/batchC.py::mutations::sp.sympify(a0)':
        'argument is a sympy object computed in-process (never a stored string) (A(d) of the evaluation; the p
  +> remature_d4 mutation must raise: certificate shows rejected=True)',
    'code/p5_calc/batchC.py::main::sp.sympify(v)':
        'argument is a sympy object computed in-process (never a stored string) (A(d) values; R5_d_retained ch
  +> eck)',
    'code/p5_calc/d13.py::exact_fit::sp.sympify(a)':
        'argument is a sympy object computed in-process (never a stored string) (matrix entries)',
    'code/p5_calc/d13.py::expand_coeff::sp.sympify(c)':
        'argument is a sympy object computed in-process (never a stored string) (fitted coefficient)',
    'code/p5_calc/d13.py::mutations::sp.sympify(v)':
        'argument is a sympy object computed in-process (never a stored string) (fitted values)',
    'code/p5_calc/d13.py::_opaque_d_refuses::sp.sympify(fit1["T2"])':
        'argument is a sympy object computed in-process (never a stored string) (fit result; the probe must ra
  +> ise: certificate shows it does)',
    'code/p5_calc/d13.py::main::sp.sympify(v)':
        'argument is a sympy object computed in-process (never a stored string) (fit values; R5_d_retained che
  +> ck)',
    'code/p5_calc/integrals.py::fp_two_point::sp.sympify(poly)':
        'argument is a sympy object computed in-process (never a stored string) (numerator polynomial)',
    'code/p5_calc/integrals.py::realize::sp.sympify(expr)':
        'argument is a sympy object computed in-process (never a stored string)',
    'code/p5_calc/release_p17.py::f3_hardening::sp.sympify(str(e))':
        "deliberate G2 hazard probe (the plain Symbol('d') must be caught by assert_opaque_d)",
    'code/p5_entry/dred.py::assert_opaque_d::sp.sympify(expr)':
        'argument is a sympy object computed in-process (never a stored string) (the checker itself)',
    'code/p5_entry/dred.py::parse_d::sp.sympify(text, locals=loc)':
        "the guarded loader itself (loc['d'] is the opaque d; the result is checked by assert_opaque_d)",
    'code/p5_entry/dred.py::substitute_d::sp.sympify(expr)':
        'argument is a sympy object computed in-process (never a stored string) (the authorised substitution)'
  +> ,
    'code/p5_entry/dred.py::substitute_d::sp.sympify(value)':
        'numeric substitution value (4 or 4 - 2 eps)',
    'code/p5_entry/dred.py::__init__::sp.sympify(trace_hat)':
        'display-only numeric trace (Rules(trace_hat=...), refused unless display_only)',
    'code/p5_entry/release_p14.py::classical_block::sp.sympify(txt)':
        'frozen p13 classical export ell*_generating_polynomial.srepr (4D, contains no d); hashed only',
    'code/p5_entry/shadow.py::even::sp.sympify(x)':
        'argument is a sympy object computed in-process (never a stored string) (scalar coefficient)',
}
RUN_SPECIFIC_KEYS = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed",
                     "live_report_identical_to_archived", "live_report_present", "archived_zip_sha256_ok",
                     "list_equals_archived_zip")
```

```
PHASE_CONVENTION_G6 = (
    "Overall phase (G6).  Every graph is exported as G = kappa^2 N_eps Ghat with N_eps = i S_eps/(16 pi^2) and
  +>  "
    "Ghat = -i * prefactor/kappa^2 * int N/(D1 D2).  For D01 (export prefactor kappa^2/2) and D03 "
    "(prefactor -kappa^2, closed ghost loop) the prefactor is real, so A(d) carries an overall factor I; for D
  +> 13 the "
    "exported prefactor is I*kappa^2, so A(d) is real.  The relative phase between graphs of the same slot is
  +> fixed by "
    "the common -i and the real export prefactors (D01 vs D03: real ratio); it is checked by the exact cancell
  +> ation of "
    "the summed hh slot and by the rejected mutation M1 (relative sign).  Physical overall phase: not asserted
  +> .")
```

140 Source: hh_slot.py (p17 D01/D03 evanescent poles vs the ch slot: proof, controls, mutations)

[listing 107/133] code/p5_calc/hh_slot.py

```
#!/usr/bin/env python3
"""XVI-P p17: do the evanescent poles of D01/D03 (kappa^2 hh self-energy) enter the ch-projected slot?

Row.  The hh two-point function of Gamma1 enters the ch projection of the one-loop identity ONLY through the r
  +> ow
(S0, Gamma1^[2]) s2 K:K.c (ledgers/P13_ch_dependency_rows.json; graphs D01, D02, D03, D11, D12):
    X_hh[c, h] = int (s h)_{(mn)}(p; c) G^{(mn)(rs)}(p) h_(rs)(-p),
where (s h)_{(mn)} = delta S0 / delta K^{(mn)} at linear order in c is read off the frozen S_Psi^(0) (the K.c
  +> monomial;
exact covariant form recovered by an exact unique fit, H1) and G is the certified covariant hh vertex (P16_bat
  +> chC.json;
recomputed here by batchC.evaluate_graph and compared with the certificate).

Proof (exact in d; R1/R2: p in QdS, so p.gt = 0; no trace taken):
  H1  s h = (i/2)(p_m eta_nv + p_n eta_mv) c^v up to the fitted normalisation: LONGITUDINAL in (mn).
  H2  hence X_g = (contraction of p_m G_g^{(mn)(rs)}) only: every structure with gt on m or n drops (p.gt = 0)
  +> .
  H3  p_m (G_D01 + G_D03)^{(mn)(rs)} = 0 as a covariant tensor identity in free n, r, s, exactly in d -- both
  +> the 4D
      and the evanescent (gt) parts separately; D02 = D12 = 0 (p15).  So X_{D01+D02+D03+D12} = 0 for every ext
  +> ernal
      assignment (Q4S, QdS, tilde, mixed): the D01/D03 evanescent poles DECOUPLE from the slot -- as a sum (th
  +> e ghost
      loop D03 cancels the longitudinal part of the graviton bubble D01), not graph by graph.
  H4  controls: X_D01 and X_D03 are individually NONZERO, including for gt-carrying external components (the
      cancellation is not a trivial drop-out of the evanescent structures); the 4D poles individually are not
      transverse, their sum is.
  Mutations (each must make the summed slot nonzero and be rejected): wrong relative sign of D03 (phase), D03
  +> x 2
  (symmetry factor), evanescent part of D03 dropped (naive 4D), trace operator eta_mn p_v in place of s h.
Sharper 4D statement: the five p14r h.h directions span ALL five symmetric 4-derivative 4D structures, so 'pol
  +> e in
the span' is automatic; the non-automatic statement is that the summed 4D pole lies in the TRANSVERSE subspace
(the kernel of z -> p_m sum_i z_i B_i), of dimension 2, while the D01 and D03 poles individually do not.
D11 is NOT evaluated (M2 undischarged): its slot contribution is open; the D01/D03 statement does not depend o
  +> n it.
Phase (G6): every Ghat is defined with the same N_eps = i S_eps/(16 pi^2); D01/D03 carry an overall I (Ghat =
  +> -i pref/
kappa^2 int, real export prefactors), D13 (prefactor I kappa^2) is real; the D01/D03 RELATIVE phase is tested
  +> by H3
(mutation 'wrong relative sign' breaks it); s h carries an explicit i, so X_hh is real like the D13 slot.
"""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import dred  # noqa: E402
import covariant as C  # noqa: E402
import e2_evanescent as E2M  # noqa: E402
import batchC as BC  # noqa: E402
import p17_conventions as QC  # noqa: E402
```

```
d = dred.d
PP = C.PP
PS = sp.symbols("p0:4")
I = sp.I
SPEC_KC = [("pair", ("m", "n")), ("vec", "v")]
GRAPHS = ("D01", "D03")
EXT = E2M.EXT


def log(m):
    print(f"[hh_slot {time.strftime('%H:%M:%S')}] {m}", flush=True)


def s(x):
    return str(sp.factor(x)) if x != 0 else "0"


# ------------------------------------------------------------------ H1: the slot operator from the frozen S0
def kc_operator():
    import counterterm_domain as CT
    import feynman as F
    S0 = CT.S0_density()
    polys = {}
    for a, mn in enumerate(F.HC):
        for v in range(4):
            val = F.pw_vertex(S0, [{"f": "K", "c": mn, "k": [-x for x in PS]}, {"f": "c", "c": v, "k": list(PS
  +> )}])
            if val != 0:
                polys[f"{a},{v}"] = sp.expand(val / I)            # the vertex is i x (real polynomial)
    fits = {}
    for off in (1, 2):
        for up in (False, True):
            f = C.covariant_fit({}, SPEC_KC, 1, offdiag=off, upper=up, polys=polys)
            fits[(off, up)] = f
    good = [(k, f) for k, f in fits.items() if f["exact"] and f["unique"] and f["terms"]]
    if not good:
        return None, {"exact": False, "candidates": {str(k): f["exact"] for k, f in fits.items()}}, polys
    (off, up), f = good[0]
    terms = []
    for orb, sc, c in f["terms"]:
        for facs in sorted(orb, key=str):
            terms.append((I * c * sc, list(facs)))
    return terms, {"exact": True, "unique": True, "offdiag_weight": off, "upper": up, "n_components": len(poly
  +> s),
                   "rank": f["rank"], "n_ansatz": f["n_ansatz"],
                   "covariant": sorted(([str(I * c * sc), [list(x) for x in sorted(orb, key=str)[0]]] for orb,
  +>  sc, c in f["terms"]),
                                       key=json.dumps)}, polys


def longitudinal_op(terms):
    return all(any(fct in (("p", "m"), ("p", "n")) for fct in facs) for _, facs in terms)


def trace_operator():
    """mutation M4: eta_mn p_v (not longitudinal)."""
    return [(I, [("g4", "m", "n"), ("p", "v")])]


# ------------------------------------------------------------------ the slot
def g_terms(A, scale=1, drop_evanescent=False):
    out = []
    for (orb, e), cf in A.items():
        if drop_evanescent and C.is_evanescent(orb):
            continue
        for facs in sorted(orb, key=str):
            out.append((scale * cf * PP ** e, list(facs)))
    return out


def slot_value(op_terms, gterms, ext):
    """X = op_{(mn) v} c^v G^{(mn)(rs)} a_r b_s (h_rs = a_(r b_s)), exact in d."""
    cn, an, bn = ext
    terms = []
    for co, fo in op_terms:
        for cg, fg in gterms:
            for (x, y) in ((an, bn), (bn, an)):
                facs = E2M.to_dred(fo) + E2M.to_dred(fg) + [("v", cn, "v"), ("v", x, "r"), ("v", y, "s")]
                terms.append((co * cg / 2, facs))
    res = E2M.contract_all(terms)
```

```
    assert all(k == () for k in res), "open indices left"
    return sp.factor(sp.cancel(sp.together(sp.expand(res.get((), 0)))))   # exact zero test in the opaque d


def p_dot_G(gterms):
    """p_m G^{(mn)(rs)} with free n, r, s (covariant tensor identity)."""
    res = E2M.contract_all([(c, E2M.to_dred(f) + [("v", "p", "m")]) for c, f in gterms])
    out = {}
    for k, v in sorted(res.items(), key=lambda kv: str(kv[0])):
        v = sp.cancel(sp.together(sp.expand(v)))      # exact zero test (rational functions of the opaque d)
        if v != 0:
            out[str(k)] = sp.factor(v)
    return out


def per_structure(A, op_terms):
    """for each structure orbit of G: does it reach the slot (any external assignment)?"""
    out = {}
    for k in sorted(A, key=lambda k: C.label(*k)):
        vals = {name: slot_value(op_terms, g_terms({k: sp.Integer(1)}), ext) for name, ext in EXT.items()}
        out[C.label(*k)] = {"evanescent": C.is_evanescent(k[0]),
                            "reaches_slot": any(v != 0 for v in vals.values()),
                            "gt_on_differentiated_pair": any(f[0] == "gt" and ("m" in f[1:] or "n" in f[1:])
                                                             for f in sorted(k[0])[0])}
    return out


# ------------------------------------------------------------------ 4D poles: transverse subspace of the p14r s
  +> pan
def pole_vec(A, keys):
    return {k: dred.substitute_d(A[k], 4, "hh_slot: UV pole A(4) of a 4D structure (after exact-d algebra)")
            for k in keys}


def pair_swap_class(label):
    """canonical class of a 4D structure label under the pair exchange (mn) <-> (rs)."""
    def canon(lab):
        toks = []
        for t in lab.split(" "):
            if "_" in t and not t.startswith("(p.p)"):
                head, idx = t.split("_", 1)
                t = head + "_" + "".join(sorted(idx))
            toks.append(t)
        return " ".join(sorted(toks))
    swapped = label.translate(str.maketrans({"m": "r", "n": "s", "r": "m", "s": "n"}))
    return min(canon(label), canon(swapped))


def transverse_analysis(polevecs, basis):
    zs = sp.symbols(f"z0:{len(basis)}")
    comb = {}
    for z, b in zip(zs, basis):
        for k, v in b["coords"].items():
            comb[k] = comb.get(k, 0) + z * v
    lon = BC.longitudinal_projection(comb)
    eqs = [sp.expand(v) for v in lon.values()]          # sympy objects (no string parsing)
    M = sp.Matrix([[sp.diff(e, z) for z in zs] for e in eqs]) if eqs else sp.zeros(0, len(zs))
    rank = M.rank()
    kernel = [list(v) for v in M.nullspace()]
    res = {"basis_dimension": len(basis), "span_is_all_4D_structures": None,
           "longitudinal_constraint_rank": rank, "transverse_subspace_dimension": len(basis) - rank,
           "transverse_basis_in_p14r_coordinates": [[str(x) for x in v] for v in kernel], "graphs": {}}
    keys4 = sorted({k for b in basis for k in b["coords"]}, key=lambda k: C.label(*k))
    classes = {pair_swap_class(C.label(*k)) for k in keys4}
    rank_b = sp.Matrix([[b["coords"].get(k, 0) for k in keys4] for b in basis]).rank()
    res["n_4D_orbit_labels"] = len(keys4)
    res["dim_pair_symmetric_4D_structures"] = len(classes)        # (mn) <-> (rs) symmetric combinations
    res["basis_rank"] = rank_b
    res["span_is_all_4D_structures"] = rank_b == len(classes) == len(basis)
    for name, vec in polevecs.items():
        z, ok = BC.pole_in_hh_basis(vec, basis)
        zz = [sp.nsimplify(x) for x in z] if z else None
        trans = bool(zz is not None and all(sp.expand(sum(r * x for r, x in zip(M.row(i), zz))) == 0
                                            for i in range(M.rows)))
        res["graphs"][name] = {"coordinates_p14r_basis": [str(x) for x in zz] if zz else None, "in_span": ok,
                               "transverse": trans}
    return res


# ------------------------------------------------------------------ main
def main():
```

```
  import certio
  t0 = time.time()
  cert = json.loads((ROOT / "certificates" / "P16_batchC.json").read_text())
  log("H1: K.c operator from S0")
  op, opfit, polys = kc_operator()
  evs, A = {}, {}
  for g in GRAPHS:
      log(f"{g}: covariant vertex (batchC.evaluate_graph)")
      evs[g] = BC.evaluate_graph(g)
      A[g] = evs[g]["A"]
  cert_equal = all(
      all(sp.cancel(v - dred.parse_d(cert["graphs"][g]["result"][C.label(*k)]["A_exact_d_times_MB"])) == 0
          for k, v in A[g].items()) and len(A[g]) == len(cert["graphs"][g]["result"]) for g in GRAPHS)
  log("H2/H3: slot values")
  slots = {g: {n: slot_value(op, g_terms(A[g]), e) for n, e in EXT.items()} for g in GRAPHS}
  slots_sum = {n: sp.factor(slots["D01"][n] + slots["D03"][n]) for n in EXT}
  ev_only = {g: {k: v for k, v in A[g].items() if C.is_evanescent(k[0])} for g in GRAPHS}
  d4_only = {g: {k: v for k, v in A[g].items() if not C.is_evanescent(k[0])} for g in GRAPHS}
  pdG = {g: p_dot_G(g_terms(A[g])) for g in GRAPHS}
  pdG_sum = p_dot_G(g_terms(A["D01"]) + g_terms(A["D03"]))
  pdG_ev_sum = p_dot_G(g_terms(ev_only["D01"]) + g_terms(ev_only["D03"]))
  pdG_4d_sum = p_dot_G(g_terms(d4_only["D01"]) + g_terms(d4_only["D03"]))
  pdG_ev = {g: p_dot_G(g_terms(ev_only[g])) for g in GRAPHS}
  ev_slot = {g: {n: slot_value(op, g_terms(ev_only[g]), e) for n, e in EXT.items()} for g in GRAPHS}
  log("per-structure table")
  struct = {g: per_structure(A[g], op) for g in GRAPHS}
  log("mutations")
  muts = {}
  for name, gt3 in (("M1_wrong_relative_sign_D03", g_terms(A["D03"], -1)),
                    ("M2_D03_times_2_symmetry_factor", g_terms(A["D03"], 2)),
                    ("M3_D03_evanescent_dropped_naive_4D", g_terms(A["D03"], drop_evanescent=True))):
      vals = {n: slot_value(op, g_terms(A["D01"]) + gt3, e) for n, e in EXT.items()}
      nz = any(v != 0 for v in vals.values())
      muts[name] = {"summed_slot_nonzero": nz, "rejected": nz, "accepted_as_rejection": nz,
                    "slot_values": {n: s(v) for n, v in vals.items()}}
  vals = {n: slot_value(trace_operator(), g_terms(A["D01"]) + g_terms(A["D03"]), e) for n, e in EXT.items()}
  nz = any(v != 0 for v in vals.values())
  muts["M4_trace_operator_in_place_of_s_h"] = {"summed_slot_nonzero": nz, "rejected": nz, "accepted_as_rejec
+> tion": nz,
                                               "slot_values": {n: s(v) for n, v in vals.items()}}
  log("4D poles: transverse subspace of the p14r h.h span")
  basis = BC.hh_basis()
  keys = {g: BC.four_d_keys(A[g]) for g in GRAPHS}
  pv = {g: pole_vec(A[g], keys[g]) for g in GRAPHS}
  tot = {}
  for g in GRAPHS:
      for k, v in pv[g].items():
          tot[k] = tot.get(k, 0) + v
  tr = transverse_analysis({"D01": pv["D01"], "D03": pv["D03"], "D01+D03": tot}, basis)
  ev_poles_sum = {}
  for g in GRAPHS:
      for k, v in ev_only[g].items():
          ev_poles_sum[C.label(*k)] = ev_poles_sum.get(C.label(*k), 0) + dred.substitute_d(
              v, 4, "hh_slot: evanescent UV pole A(4) (record only)")
  led = json.loads((ROOT / "ledgers" / "P13_ch_dependency_rows.json").read_text())["rows"]
  hh_rows = sorted(r["id"] for r in led if any(g.split("_")[0] in GRAPHS for g in r.get("graphs", [])))
  row_rec = {r["id"]: {"Gamma1_pattern": r.get("Gamma1_pattern"), "S_monomial": r.get("S_monomial"),
                       "differentiated": r.get("differentiated"), "graphs": r.get("graphs"),
                       "status": r.get("status")} for r in led if r["id"] in hh_rows}
  controls = {
      "ledger_hh_graphs_enter_exactly_one_row": hh_rows == [QC.SLOT_ROW]
      and row_rec[QC.SLOT_ROW]["Gamma1_pattern"] == "h^2" and row_rec[QC.SLOT_ROW]["S_monomial"] == "K.c",
      "H1_operator_exact_unique": bool(opfit.get("exact")),
      "H1_operator_longitudinal": bool(op) and longitudinal_op(op),
      "vertex_equals_P16_certificate": cert_equal,
      "H3_p_dot_G_sum_zero_exact_in_d": not pdG_sum,
      "H3_evanescent_part_p_dot_G_sum_zero": not pdG_ev_sum,
      "H3_4D_part_p_dot_G_sum_zero": not pdG_4d_sum,
      "H3_slot_sum_zero_all_external_assignments": all(v == 0 for v in slots_sum.values()),
      "H4_D01_slot_nonzero": any(v != 0 for v in slots["D01"].values()),
      "H4_D03_slot_nonzero": any(v != 0 for v in slots["D03"].values()),
      "H4_evanescent_parts_individually_reach_slot": all(any(v != 0 for v in ev_slot[g].values()) for g in G
+> RAPHS),
      "H4_4D_poles_individually_not_transverse": not tr["graphs"]["D01"]["transverse"] and
      not tr["graphs"]["D03"]["transverse"],
      "4D_pole_sum_transverse": tr["graphs"]["D01+D03"]["transverse"],
      "transverse_subspace_dimension_2": tr["transverse_subspace_dimension"] == 2,
      "hh_basis_spans_all_pair_symmetric_4D_structures": tr["span_is_all_4D_structures"],
      "D01_plus_D03_coordinates_in_transverse_kernel": tr["graphs"]["D01+D03"]["in_span"],
      "mutations_all_rejected": all(v["accepted_as_rejection"] for v in muts.values()),
  }
```

```
    proven = all(controls.values())
    surviving = []          # evanescent structures of D01+D03 reaching the slot after summation
    if not controls["H3_evanescent_part_p_dot_G_sum_zero"]:
        surviving = sorted(pdG_ev_sum)
    rec = {
        "landmark": "XVI-P", "revision": QC.REVISION,
        "deliverable": "D01/D03 evanescent poles vs the ch-projected slot (row (S0,Gamma1^[2]) s2 K:K.c)",
        "question": ("do the evanescent (gt) UV poles of D01/D03 enter the ch-projected one-loop identity?"),
        "verdict": ("DECOUPLED (proof): as the sum D01 + D03 (+ D02 = D12 = 0); not graph by graph" if proven
  +> else
                    ("PARTIALLY DECOUPLED" if surviving else "NOT ESTABLISHED")),
        "status": "decoupled_from_ch_slot_proven_as_sum" if proven else "open",
        "surviving_evanescent_structures_in_slot": surviving,
        "row": "(S0, Gamma1^[2]) s2 K:K.c (ledgers/P13_ch_dependency_rows.json; graphs D01, D02, D03, D11, D12
  +> ); "
               "the hh two-point function enters no other ch row (computed from the ledger: rows_with_D01_D03)
  +> ",
        "rows_with_D01_D03": row_rec,
        "functional": "X_hh[c,h] = int (s h)_{(mn)}(p; c) G^{(mn)(rs)}(p) h_(rs)(-p)",
        "operator_s_h": opfit,
        "proof": {
            "H1": "s h read off S_Psi^(0) (K.c monomial): exact unique covariant form, longitudinal in (mn)",
            "H2": "X_g depends on G only through p_m G^{(mn)(rs)}; p in QdS so p.gt = 0 removes gt on (mn)",
            "H3": "p_m (G_D01 + G_D03)^{(mn)(rs)} = 0 identically in d (4D and evanescent parts separately); "
                  "X_{D01+D03} = 0 for every external assignment; D02 = D12 = 0 (P15_batchA.json)",
            "H4": "controls: X_D01, X_D03 individually nonzero (also their evanescent parts); 4D poles individ
  +> ually "
                  "not transverse",
            "order": "one loop: the ch projection is linear in Gamma1; the cancellation is exact in d (not O(e
  +> ps)); "
                     "no product of evanescent poles arises"},
        "p_dot_G": {"D01": pdG["D01"], "D03": pdG["D03"], "D01+D03": pdG_sum,
                    "evanescent_D01": pdG_ev["D01"], "evanescent_D03": pdG_ev["D03"],
                    "evanescent_D01+D03": pdG_ev_sum},
        "slot_values": {g: {n: s(v) for n, v in slots[g].items()} for g in GRAPHS} |
                       {"D01+D03": {n: s(v) for n, v in slots_sum.items()}},
        "per_structure": struct,
        "evanescent_poles_D01_plus_D03_in_hh_green_function": {k: s(v) for k, v in sorted(ev_poles_sum.items()
  +> )},
        "consequence": ("no evanescent h.h counterterm is required by the ch-projected identity at one loop, s
  +> o NO "
                        "contract revision is made for it.  The summed evanescent poles of the hh Green functi
  +> on are "
                        "nonzero (list above) but TRANSVERSE; renormalising the hh Green function itself would
  +>  need "
                        "evanescent h.h counterterms -- relevant only outside this projection and NOT claimed.
  +>   D11 "
                        "(same row) is NOT evaluated: its slot contribution is open."),
        "four_D_poles": {**tr, "note": ("the five p14r h.h directions span all five symmetric 4-derivative 4D
  +> "
                                        "structures, so span membership is automatic; the sharper, non-automat
  +> ic "
                                        "statement is transversality: the summed pole lies in the 2-dim transv
  +> erse "
                                        "subspace, the individual poles do not")},
        "phase_convention_G6": ("G = kappa^2 N_eps Ghat, N_eps = i S_eps/(16 pi^2) for every graph; D01/D03 Gh
  +> at = "
                                "-i pref/kappa^2 int (real export prefactors kappa^2/2, -kappa^2) carry an ove
  +> rall I; "
                                "D13 Ghat (prefactor I kappa^2) is real; s h = i(...) so X_hh is real like the
  +>  D13 slot; "
                                "the D01/D03 relative phase is tested by H3 (mutation M1 breaks it); the D13 v
  +> s hh relative "
                                "phase is not tested by a nonzero identity (both slot contributions vanish)"),
        "D11": "NOT evaluated (M2 undischarged); its contribution to this row is open",
        "controls": controls, "mutations": muts, "decoupling_proven": proven,
        "authorised_d_substitutions": dict(sorted(dred.D_PROVENANCE.items())),
        "quantum_inputs_evaluated": "not_earned", "seconds": round(time.time() - t0, 1)}
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_hh_slot.json", rec)
    print(json.dumps({"verdict": rec["verdict"], "controls": controls,
                      "transverse": {k: v["transverse"] for k, v in tr["graphs"].items()},
                      "coords": {k: v["coordinates_p14r_basis"] for k, v in tr["graphs"].items()},
                      "mutations": {k: v["accepted_as_rejection"] for k, v in muts.items()}}, indent=1))
    return 0 if proven else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

141 Source: release__p17.py (p17 release gate)

[listing 108/133] code/p5_calc/release_p17.py

```
#!/usr/bin/env python3
"""XVI-P p17 release gate: P.5 calculation, third step -- the D01/D03 evanescent poles vs the ch-projected slo
  +> t
(priority 1) and the p16 reviewer findings G1-G6, on the p14r contract with the disclosed p15 (F1), p16 (F3) a
  +> nd
p17 (G1-G3) dred.py revisions.

Runs AFTER the p14r contract gate (release_p14.py), the p15 layer gate (release_p15.py) and the p16 layer gate
(release_p16.py -> certificates/P16_release.json) and requires all three to be accepted.  Recomputes:
  contract identity : p14r contract files byte-identical to p14r EXCEPT code/p5_entry/dred.py; dred.py differs
  +>  from
                      the p16 release (disclosed p17 revision: parse_d / assert_opaque_d); NO new contract fil
  +> e (no
                      evanescent h.h counterterm: the decoupling is proven); F3 guard and G2 loader active (pr
  +> obes);
  p16 layer        : every p16 layer file byte-identical to the p16 zip (QC.P16_CALC_SHA256); P16_release acce
  +> pted;
  hh slot          : P17_hh_slot.json -- decoupling proof controls all true, mutations M1-M4 rejected, no surv
  +> iving
                      evanescent structure in the slot;
  hard stops       : QC.HARD_STOPS_P17 (D11 NOT evaluated; quantum_inputs_evaluated not_earned; ...).
Rebuild flags (package evidence): author/independent for the CURRENT (p17) tree; independent_rebuild_passed_p1
  +> 6 from
the archived p16 independent report, bound to the p16 tree 90b4a665... / zip 56d5887d... (same-box caveat).
Change log (G4): every source file added/modified/removed relative to the p16 source-only zip, INCLUDING the p
  +> 17
reference-values file (listed as added by declaration: it is written after the author build).
"""
from __future__ import annotations
import hashlib, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p17_conventions as QC  # noqa: E402
import p15_conventions as Q15  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)

ARCH_P16 = "archival/p16"
ARCH_P16_INDEP = f"{ARCH_P16}/reproducibility/independent_rebuild_report_p16.json"
HH_MUTATIONS = ("M1_wrong_relative_sign_D03", "M2_D03_times_2_symmetry_factor", "M3_D03_evanescent_dropped_nai
  +> ve_4D",
                "M4_trace_operator_in_place_of_s_h")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def f3_hardening():
    """F3 runtime guard (unchanged) and the p17 G2 loader."""
    import dred
    import sympy as sp
    d = dred.d
    e = (d - 4) / (d - 1) + d ** 2
    probes = {"expr.replace(d, 4)": lambda: e.replace(d, 4), "sympy.lambdify(d, expr)": lambda: sp.lambdify(d,
  +>  e),
              "expr.match(d)": lambda: e.match(d), "expr.subs(d, 4)": lambda: e.subs(d, 4),
              "expr.xreplace({d: 4})": lambda: e.xreplace({d: 4}),
              "dred.parse_d(str(expr)).subs(d, 4)": lambda: dred.parse_d(str(e)).subs(d, 4)}
    res = {}
    for k, f in probes.items():
        try:
            f()
            res[k] = "NOT refused"
        except dred.PrematureD4Substitution:
            res[k] = "refused"
```

```
    plain = sp.sympify(str(e))                                  # the G2 hazard: a plain Symbol('d')
    try:
        dred.assert_opaque_d(plain)
        plain_caught = False
    except dred.PlainDimensionSymbol:
        plain_caught = True
    rt = dred.parse_d(str(e))
    try:
        dred.parse_d("d + 1", {"d": sp.Symbol("d")})
        rebind_refused = False
    except dred.PlainDimensionSymbol:
        rebind_refused = True
    g2 = {"parse_d_round_trip_equals_original": sp.cancel(rt - e) == 0, "parse_d_restores_opaque_d": rt.has(d)
  +> ,
          "plain_sympify_detected_by_assert_opaque_d": plain_caught, "extra_locals_rebinding_d_refused": rebin
  +> d_refused}
    auth = dred.substitute_d(e, 4, "release_p17.f3_hardening: authorised-path probe") == 16
    return {"probes": res, "all_refused": all(v == "refused" for v in res.values()), "G2": g2,
            "G2_ok": all(g2.values()), "authorised_path_works": auth}


def contract_identity():
    cur = {p.relative_to(ROOT).as_posix(): sha(p) for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    changed = sorted(k for k, h in Q15.P14R_CONTRACT_SHA256.items() if cur.get(k) != h)
    added = sorted(set(cur) - set(Q15.P14R_CONTRACT_SHA256))
    dred_changed_vs_p16 = cur.get("code/p5_entry/dred.py") != QC.P16_DRED_SHA256
    g = f3_hardening()
    ok = bool(changed == sorted(QC.P14R_CONTRACT_FILES_CHANGED_IN_P17) and added == sorted(QC.NEW_CONTRACT_FIL
  +> ES_P17)
              and dred_changed_vs_p16 and g["all_refused"] and g["G2_ok"] and g["authorised_path_works"])
    return ok, {"p14r_contract_files": len(Q15.P14R_CONTRACT_SHA256), "changed_vs_p14r": changed, "added": add
  +> ed,
                "dred_changed_vs_p16": dred_changed_vs_p16,
                "disclosed_revisions_p17": QC.DISCLOSED_CONTRACT_REVISIONS_P17, "guard_probes": g,
                "evanescent_hh_counterterm_added": False,
                "why_no_evanescent_hh_counterterm": ("D01/D03 evanescent poles decouple from the ch-projected
  +> slot as "
                                                     "the sum D01 + D03 (proof, certificates/P17_hh_slot.json)
  +> ")}


def p16_layer():
    cur = {k: sha(ROOT / k) if (ROOT / k).is_file() else None for k in QC.P16_CALC_SHA256}
    ident = all(cur[k] == h for k, h in QC.P16_CALC_SHA256.items())
    r = jl("certificates/P16_release.json")
    ok = bool(ident and r.get("publication") == "accepted" and r.get("release_claim_accepted") is True
              and r.get("batchC_evaluated") is True and r.get("E2_decoupling_proven") is True
              and r.get("hard_stops_hold") is True)
    return ok, {"p16_layer_files_byte_identical": ident, "P16_release_accepted": r.get("publication") == "acce
  +> pted",
                "batchC_evaluated": r.get("batchC_evaluated"), "E2_status": r.get("E2_status")}, r


def p16_independent():
    f = ROOT / ARCH_P16_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P16_INDEP, "present": False, "pins_ok": False}
    rep = json.loads(f.read_text())
    live = ROOT / QC.P16_INDEPENDENT_REPORT
    rec = {"report": ARCH_P16_INDEP, "present": True, "report_sha256": sha(f),
           "report_sha256_pinned": sha(f) == QC.P16_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "bound_to_source_tree_sha256": rep.get("source_tree_sha256"),
           "bound_to_source_zip_sha256": rep.get("zip_sha256"),
           "tree_matches_p16": rep.get("source_tree_sha256") == QC.P16_SOURCE_TREE_SHA256,
           "zip_matches_p16": rep.get("zip_sha256") == QC.P16_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])), "findings": [x.get("id") for x in rep.get("findings",
  +>  [])],
           "caveat": rep.get("same_box_caveat"),
           "live_report_identical_to_archived": (live.read_bytes() == f.read_bytes()) if live.is_file()
           else "live report not in this tree",
           "NOT_inherited_by_p17": "bound to the p16 tree/zip; the p17 tree has its own (pending) independent
  +> flag"}
    rec["archived_p16_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P16_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p16_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p16"] and rec["zip_matches_p16"] and rec["mismatches"] == 0
              and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec
```

```
def hh_slot():
    c = jl(f"certificates/{QC.CERT_PREFIX}_hh_slot.json")
    muts = c.get("mutations", {})
    ok = bool(c.get("revision") == QC.REVISION and c.get("decoupling_proven") is True
              and c.get("status") == "decoupled_from_ch_slot_proven_as_sum"
              and c.get("surviving_evanescent_structures_in_slot") == []
              and all(v is True for v in c.get("controls", {}).values()) and len(c.get("controls", {})) >= 16
              and sorted(muts) == sorted(HH_MUTATIONS)
              and all(v.get("accepted_as_rejection") is True for v in muts.values()))
    return ok, c


def change_log():
    """G4: every source file added / modified / removed relative to the released p16 source-only zip, base the
  +>  pinned
    per-file list archival/p16/p16_source_files.json.  The p17 reference-values file is written AFTER the auth
  +> or
    build; it is listed as ADDED by declaration (deterministic: identical before and after it exists)."""
    f = ROOT / QC.P16_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P16_ARCHIVED_RECORDS[QC.P16_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT) if r != SM.REFERENCE_FILE}
    contract = set(Q15.P14R_CONTRACT_SHA256) | {p.relative_to(ROOT).as_posix() for p in (ROOT / "code" / "p5_e
  +> ntry").glob("*.py")}
    added = sorted((set(new) - set(old)) | {SM.REFERENCE_FILE})
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])
    changed = sorted(added + removed + modified)
    return {"base": f"{QC.P16_SOURCE_LIST} (pinned; files of the p16 zip sha256 {QC.P16_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok,
            "reference_file": f"{SM.REFERENCE_FILE} listed as added by declaration (written after the author b
  +> uild)",
            "added": added, "removed": removed, "modified": modified,
            "contract_files_changed": [x for x in changed if x in contract],
            "non_contract_files_changed": [x for x in changed if x not in contract],
            "counts": {"added": len(added), "removed": len(removed), "modified": len(modified)}}


def milestones(p16r, h_ok):
    st = dict(p16r.get("milestones", {}).get("status", {}))
    st["P.5_calculation"] = ("in progress (p15: Batch A D02/D12/D14a-c and Batch B D13; p16: E2 decoupled from
  +>  the ch "
                             "slot (proof), Batch C D01/D03 evaluated; p17: D01/D03 evanescent poles decoupled
  +>  from "
                             "the ch slot as the sum D01 + D03 (proof); D11 matching M2 NOT evaluated)"
                             if h_ok else "open (p17 hh-slot analysis not accepted)")
    st["P.5_identity"] = "open"
    return {"definitions": p16r.get("milestones", {}).get("definitions"), "order": p16r.get("milestones", {}).
  +> get("order"),
            "status": st, "P5_entry_earned": p16r.get("P5_entry_earned"),
            "P3_P4_frozen_as_P5_inputs": p16r.get("P3_P4_frozen_as_P5_inputs"),
            "P4_is_NOT": "coupled one-loop restoration (quantum_inputs_evaluated / projected_identity_verified
  +>  not earned)"}


def tex(x):
    """plain text-mode rendering (no math mode): ** -> ^, TeX specials escaped, typewriter font."""
    t = str(x).replace("**", "^").replace("\\", "/").replace("_", "\\_").replace("^", "\\^{}").replace("&", "\
  +> \&")
    return "\\texttt{" + t + "}"


def results_table(hc):
    ps = hc["per_structure"]
    labs = sorted(set(ps["D01"]) | set(ps["D03"]))
    yn = {True: "yes", False: "no", None: "--"}
    rows = [f"{tex(l)} & {yn[ps['D01'].get(l, ps['D03'].get(l, {})).get('evanescent')]} & "
            f"{yn[ps['D01'][l]['reaches_slot']] if l in ps['D01'] else '--'} & "
            f"{yn[ps['D03'][l]['reaches_slot']] if l in ps['D03'] else '--'} \\\\" for l in labs]
    fd = hc["four_D_poles"]
    crow = [f"{g} & " + " & ".join(tex(x) for x in fd["graphs"][g]["coordinates_p14r_basis"])
            + f" & {yn[fd['graphs'][g]['transverse']]} \\\\" for g in ("D01", "D03", "D01+D03")]
    kern = "; ".join("(" + ", ".join(v) + ")" for v in fd["transverse_basis_in_p14r_coordinates"])
    return ("% generated by code/p5_calc/release_p17.py -- do not edit\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{4.2cm}p{1.6cm}p{2.2cm}p{2.2cm}}\\hline\n"
            "structure of $G^{(mn)(rs)}$ & evanescent & D01 reaches slot & D03 reaches slot \\\\\\hline\n"
            + "\n".join(rows) + "\n\\hline\\end{tabular}\\end{center}\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{1.3cm}p{1.5cm}p{1.5cm}p{1.5cm}p{1.5cm}p{1.5cm}p{1.3
  +> cm}}\\hline\n"
            "4D pole & $z_1$ & $z_2$ & $z_3$ & $z_4$ & $z_5$ & transverse \\\\\\hline\n"
```

```
            + "\n".join(crow) + "\n\\hline\\end{tabular}\\end{center}\n"
            f"\\noindent Verdict: {tex(hc['verdict'])}. Surviving evanescent structures in the slot: none. "
            f"Transverse kernel (p14r coordinates): {tex(kern)}. D11: NOT evaluated.\n")


def _main():
    id_ok, ident = contract_identity()
    l16_ok, l16, p16r = p16_layer()
    h_ok, hc = hh_slot()
    p16i_ok, p16i = p16_independent()
    p14r = jl("certificates/P14_release.json")
    p15r = jl("certificates/P15_release.json")
    hard = dict(QC.HARD_STOPS_P17)
    graphs = dict(p16r.get("graphs", {}))
    for g in QC.HH_SLOT_GRAPHS:
        if g in graphs and h_ok:
            graphs[g] = dict(graphs[g], ch_slot=("decoupled as the sum D01 + D03 (evanescent and 4D parts; "
                                                 "certificates/P17_hh_slot.json); NOT graph by graph"))
    graphs["D11"] = {"status": "unevaluated", "reason": "M2 undischarged (D11 matching not verified)",
                     "ch_slot": "open (same row (S0,Gamma1^[2]) s2 K:K.c)"}
    hard_ok = bool(all(hard[k] == v for k, v in QC.HARD_STOPS_P17.items())
                   and graphs.get("D11", {}).get("status") == "unevaluated"
                   and hc.get("quantum_inputs_evaluated") in (None, "not_earned")
                   and p14r.get("hard_stops_hold") is True and p15r.get("hard_stops_hold") is True
                   and p16r.get("hard_stops_hold") is True)
    cl = change_log()
    execution_pass = bool(id_ok and l16_ok and h_ok and hard_ok and cl["base_pinned_ok"] and p16i.get("pins_ok
  +> ") is True)
    publish = execution_pass
    author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
    indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
    m = milestones(p16r, h_ok)
    hh_status = hc.get("status") if h_ok else "open"
    flags = dict(p16r.get("flags", {}))
    flags["hh_evanescent_decoupled_from_ch_slot"] = {
        "execution": h_ok, "claim": hc.get("verdict") if h_ok else "not established",
        "scope": ("the ch-projected one-loop kappa^2 identity, row (S0,Gamma1^[2]) s2 K:K.c, contributions of
  +> D01 + D03 "
                  "(D02 = D12 = 0); the summed evanescent poles remain (transverse) UV poles of the hh Green f
  +> unction "
                  "outside this projection; D11 (same row) NOT evaluated")}
    cert = {
        "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
  +> TE,
        "version_label": QC.VERSION_LABEL,
        "deliverable": "P.5 calculation, third step (D01/D03 evanescent poles vs the ch-projected slot; G1-G6)
  +> ",
        "execution_pass": execution_pass,
        "claim_status": ("P.5_calculation: Batch A, D13 (p15), Batch C D01, D03 (p16) evaluated under the decl
  +> ared "
                         "prescription; E2 (p16) and the D01/D03 evanescent poles (p17, as the sum) decoupled
  +> from the "
                         "ch-projected slot (proofs); P.5_entry: earned (p14r contract); classical_readiness (
  +> frozen "
                         "P.4): earned") if execution_pass else "P.5_calculation: not accepted",
        "scope": ("P.5 calculation: Batch A (D02, D12, D14a-c), Batch B (D13) and Batch C (D01, D03) evaluated
  +>  under "
                  "the frozen prescription (DRED R1-R6, Feynman +i0, MS-bar, S0; d opaque); D01 + D03 contribu
  +> te zero "
                  "to the ch-projected slot (exact in d); D11 NOT evaluated (M2 undischarged), so the hh slot
  +> is "
                  "incomplete and no slot of the identity is complete; NOT quantum inputs, NOT the projected "
                  "defect/identity, NOT bulk restoration, NOT the insertion"),
        "release_claim_accepted": execution_pass,
        "release_claim_accepted_definition": ("p14r contract accepted AND p15 and p16 layers accepted and byte
  +> -identical "
                                              "AND contract identity (only dred.py changed vs p14r; changed vs
  +>  p16 by "
                                              "the disclosed G2 loader; guard probes; no new contract file) AN
  +> D the "
                                              "hh-slot decoupling proof AND p17 hard stops"),
        "publication": "accepted" if publish else "REJECTED",
        "flags": flags, "batchA_evaluated": p16r.get("batchA_evaluated"), "D13_evaluated": p16r.get("D13_evalu
  +> ated"),
        "batchC_evaluated": p16r.get("batchC_evaluated"), "library_certified": p16r.get("library_certified"),
        "E2_status": p16r.get("E2_status"), "E2_decoupling_proven": p16r.get("E2_decoupling_proven"),
        "hh_evanescent_status": hh_status, "hh_evanescent_decoupling_proven": h_ok,
        "evanescent_hh_counterterm_added": False,
        "D01_evaluated": p16r.get("D01_evaluated"), "D03_evaluated": p16r.get("D03_evaluated"), "D11_evaluated
  +> ": False,
        "graphs": graphs, "D12_status": p16r.get("D12_status"), "D12_status_derivation": p16r.get("D12_status_
```

```
+> derivation"),
      "hh_slot_summary": {k: hc.get(k) for k in ("verdict", "status", "row", "operator_s_h", "proof",
                                                 "surviving_evanescent_structures_in_slot", "consequence",
                                                 "four_D_poles", "evanescent_poles_D01_plus_D03_in_hh_green_
+> function",
                                                 "phase_convention_G6", "controls")},
      "phase_convention_G6": QC.PHASE_CONVENTION_G6,
      "contract_identity": ident, "contract_revision_disclosed": True,
      "disclosed_test_revisions_p17": QC.DISCLOSED_TEST_REVISIONS_P17,
      "p16_layer": l16, "change_log_vs_p16": cl,
      "P5_entry_earned": p16r.get("P5_entry_earned"), "P5_entry_flags": p16r.get("P5_entry_flags"),
      "P3_P4_frozen_as_P5_inputs": p16r.get("P3_P4_frozen_as_P5_inputs"), "classical": p16r.get("classical")
+> ,
      "D11_matching_verified": False, "quantum_inputs_evaluated": "not_earned",
      "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": m,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p16": p16i_ok,
      "independent_rebuild_passed_p15": p16r.get("independent_rebuild_passed_p15"),
      "independent_rebuild_passed_p14r": p16r.get("independent_rebuild_passed_p14r"),
      "independent_rebuild_passed_p13": p16r.get("independent_rebuild_passed_p13"),
      "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p17
+> ) source "
                             "tree only (independent: pending until a reviewer report for this tree exists);
+>  "
                             f"independent_rebuild_passed_p16 refers to the p16 tree {QC.P16_SOURCE_TREE_SHA
+> 256} / zip "
                             f"{QC.P16_SOURCE_ZIP_SHA256} (same-box caveat) and is not inherited by p17; "
                             "_p15 / _p14r / _p13 as recorded by the p16 layer"),
      "p16_independent_rebuild": p16i,
      "p15_independent_rebuild": p16r.get("p15_independent_rebuild"),
      "p14r_independent_rebuild": p16r.get("p14r_independent_rebuild"),
      "p13_independent_rebuild": p16r.get("p13_independent_rebuild"),
      "one_point_slot_kappa1": p16r.get("one_point_slot_kappa1"), "D13_summary": p16r.get("D13_summary"),
      "E2_summary": p16r.get("E2_summary"), "batchC_summary": p16r.get("batchC_summary"),
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p16_archive": ARCH_P16,
                          "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"},
      "XVI_P_registered": False, "XVI_M_sha256": p16r.get("XVI_M_sha256"), "XVI_M_pin_ok": p16r.get("XVI_M_p
+> in_ok"),
      "symmetry_restored": False, "bulk_symmetry_restored": False, "O1_closure": "NOT_CLOSED",
      "insertion_status": "Frozen_negative_obstruction_preserved", "XV_B_opened": False, "XIV_J_opened": Fal
+> se,
      "Route_B_discharged": False, "u_R_c_delta_added": False,
      "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
          n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
          ("P14_release.json", "P15_release.json", "P16_release.json", f"{QC.CERT_PREFIX}_hh_slot.json")
          if (ROOT / "certificates" / n).is_file()},
  }
  cdir = ROOT / "certificates"
  txt = certio.dumps(cert)
  if publish:
      (ROOT / "RELEASE_STATUS.json").write_text(txt)
      (cdir / "RELEASE_STATUS.json").write_text(txt)
      (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
      (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
      certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
          "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
+> tatus)",
          **m, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
          "independent_rebuild_passed_p16": p16i_ok, "hard_stops": hard,
          "flags": {"batchA_evaluated": p16r.get("batchA_evaluated"), "D13_evaluated": p16r.get("D13_evaluat
+> ed"),
                    "batchC_evaluated": p16r.get("batchC_evaluated"),
                    "E2_decoupling_proven": p16r.get("E2_decoupling_proven"),
                    "hh_evanescent_decoupling_proven": h_ok}})
      (ROOT / "manuscript" / "body").mkdir(parents=True, exist_ok=True)
      (ROOT / "manuscript" / "body" / "P17-results-table.tex").write_text(results_table(hc))
  else:
      (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
      (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(certio.dumps({
          "landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", "contract_identity_ok": i
+> d_ok,
          "contract_identity": ident, "p16_layer_ok": l16_ok, "p16_layer": l16, "hh_slot_ok": h_ok,
          "p16_independent_pins_ok": p16i.get("pins_ok"), "hard_stops_hold": hard_ok,
          "change_log_base_ok": cl["base_pinned_ok"]}))
  print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
+> ion"],
                    "contract_identity": id_ok, "changed_vs_p14r": ident["changed_vs_p14r"], "added": ident[
+> "added"],
```

```
                    "p16_layer": l16_ok, "hh_slot": h_ok, "hard_ok": hard_ok,
                    "author": author_ok, "independent_p17": indep_ok, "independent_p16": p16i_ok,
                    "change_log_counts": cl["counts"]}, indent=1))
    return 0 if publish else 1


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        cdir = ROOT / "certificates"
        (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
            {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED",
             "error": f"{type(exc).__name__}: {exc}"}, indent=1) + "\n")
        print(f"release_p17 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

142 Source: tests/test__p17.py

[listing 109/133] tests/test_p17.py

```
"""XVI-P p17 tests: D01/D03 evanescent poles vs the ch-projected slot (proof, controls, mutations, transverse
coordinates), G1 extended static scan (with planted violations), G2 guarded loader parse_d, G3 wording, G4 cha
  +> nge log,
G6 phase convention, p17 status / hard stops / p16 independent record, p17 gate mutations."""
from __future__ import annotations
import ast
import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools"):
    sys.path.insert(0, str(ROOT / p))
import contract_registry as CR  # p22 (disclosed): registered contract revision
import p17_conventions as QC  # noqa: E402
import p16_conventions as Q16  # noqa: E402
import p15_conventions as Q15  # noqa: E402
import dred as DR  # noqa: E402

CERT = ROOT / "certificates"
PKG = ("code/p5_calc", "code/p5_entry")


def cj(name):
    return json.loads((CERT / name).read_text())


def rs():
    # p18: the p17 LAYER record (the top-level RELEASE_STATUS.json is the p18 status)
    return json.loads((CERT / "P17_release.json").read_text())


def P(text):
    """certificate string -> expression with the OPAQUE d (G2 loader); 'p.p' is the scalar p^2."""
    return DR.parse_d(text.replace("p.p", "pp"), {"I": sp.I})


# ============================================================ G1: extended static scan
FORBIDDEN_NAMES = {"eval", "exec", "parse_expr", "lambdify", "ufuncify", "autowrap"}
PROBE_FUNCTIONS = {"f3_hardening"}         # release_p16 / release_p17 guard probes (deliberate hazards, must
  +> raise)


def scan(path, root=ROOT):
    """AST pattern scan for value paths the runtime guard cannot hook.  Returns (violations, parse_sites):
```

```
    violations: builtin eval/exec, parse_expr, lambdify/ufuncify/autowrap, any `.eval(` call, `.replace(` whos
  +> e first
    argument is not a string literal (d itself, a lambda or a NAMED function query), `.func(` expression-tree
  +> rebuilds;
    parse_sites: every sympify / S call that does not pass locals with the key 'd' (must be on the reviewed li
  +> st)."""
    src = path.read_text()
    tree = ast.parse(src)
    try:
        rel = path.relative_to(root).as_posix()
    except ValueError:
        rel = path.name
    bad, sites = [], []

    def visit(node, fn=None):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            fn = node.name
        if isinstance(node, ast.Call) and fn not in PROBE_FUNCTIONS:
            f = node.func
            name = f.attr if isinstance(f, ast.Attribute) else f.id if isinstance(f, ast.Name) else None
            if name in FORBIDDEN_NAMES:
                bad.append(f"{rel}:{node.lineno}:{name}")
            if name == "replace" and isinstance(f, ast.Attribute) and node.args:
                a = node.args[0]
                if not (isinstance(a, ast.Constant) and isinstance(a.value, str)):
                    bad.append(f"{rel}:{node.lineno}:replace(non-literal)")
            if name == "func" and isinstance(f, ast.Attribute):
                bad.append(f"{rel}:{node.lineno}:.func(rebuild)")
        if isinstance(node, ast.Call):
            f = node.func
            name = f.attr if isinstance(f, ast.Attribute) else f.id if isinstance(f, ast.Name) else None
            if name in ("sympify", "S"):
                loc = {k.arg: k.value for k in node.keywords}.get("locals")
                has_d = isinstance(loc, ast.Dict) and any(isinstance(k, ast.Constant) and k.value == "d" for k
  +>  in loc.keys)
                if not has_d:
                    sites.append(f"{rel}::{fn}::{ast.get_source_segment(src, node)}")
        for ch in ast.iter_child_nodes(node):
            visit(ch, fn)
    visit(tree)
    return bad, sites


def _pkg_files():
    return [f for d in PKG for f in sorted((ROOT / d).glob("*.py"))]


def test_g1_static_scan_package_clean():
    bad, sites = [], []
    for f in _pkg_files():
        b, s = scan(*CR.scan_view(f))   # p22: p18 contract core of the registered dred revision
        bad += b
        sites += s
    assert not bad, bad
    unreviewed = sorted(set(sites) - set(QC.PARSE_SITES_REVIEWED))
    assert not unreviewed, unreviewed                                # every parse site reviewed
    stale = sorted(set(QC.PARSE_SITES_REVIEWED) - set(sites))
    assert not stale, stale                                          # the reviewed list is exact (no stale entr
  +> y)


def test_g1_static_scan_detects_planted_violations(tmp_path):
    f = tmp_path / "planted.py"
    f.write_text(
        "import sympy as sp\nfrom sympy.parsing.sympy_parser import parse_expr\nfrom dred import d\n"
        "def isd(x):\n    return x == d\n"
        "def g(e):\n"
        "    a = sp.Poly(e, d).eval(4)\n"                                 # 1 .eval
        "    b = e.replace(d, 4)\n"                                       # 2 replace(d)
        "    c = e.replace(lambda x: x == d, lambda x: 4)\n"              # 3 replace(lambda)
        "    k = e.replace(isd, lambda x: 4)\n"                           # 4 replace(named function)  [G1]
        "    m = sp.sympify(str(e)).subs(sp.Symbol('d'), 4)\n"            # site: string round-trip  [G1]
        "    n = eval(sp.srepr(e))\n"                                     # 5 eval(srepr)  [G1]
        "    o = parse_expr(str(e))\n"                                    # 6 parse_expr  [G1]
        "    q = e.func(*[x if x != d else 4 for x in e.args])\n"         # 7 tree rebuild  [G1]
        "    r = sp.lambdify(d, e)\n"                                     # 8 lambdify
        "    exec('z = 1')\n"                                             # 9 exec
        "    s = sp.S(str(e))\n"                                          # site: S of a string  [G1]
        "    t = sp.sympify(str(e), locals={'d': d})\n"                   # guarded: not a site
        "    return a, b, c, k, m, n, o, q, r, s, t\n")
    bad, sites = scan(f, tmp_path)
```

```
    assert len(bad) == 9, bad
    assert len(sites) == 2 and not set(sites) & set(QC.PARSE_SITES_REVIEWED), sites


def test_g1_prefactor_strings_parsed_without_d_are_d_free():
    """the reviewed prefactor parse sites parse export prefactor strings: none contains d."""
    for f in sorted((ROOT / "exports" / "P14_integrands").glob("*.json")):
        pre = json.loads(f.read_text())["prefactor"]
        e = sp.sympify(pre, locals={"kappa": sp.Symbol("kappa")})
        assert {s.name for s in e.free_symbols} <= {"kappa"}, (f.name, pre)


def test_g1_disclosure_wording_matches_scan_scope():
    txt = json.dumps(QC.DISCLOSED_CONTRACT_REVISIONS_P17)
    for w in ("ANY function query", "string round-trips", "srepr", "e.func(*args)", "Poly(..).eval",
              "pattern check, not a proof"):
        assert w in txt, w
    doc = DR.__doc__ or ""
    assert "Wild pattern" in doc and "NOT refuse" in doc                       # G3 wording corrected


# ============================================================ G2: guarded loader
def test_g2_parse_d_restores_opaque_d_and_guard():
    e = P("(d - 4)/(d - 1) + d**2")
    assert e.has(DR.d) and all(isinstance(s, DR.OpaqueDimension) for s in e.free_symbols if s.name == "d")
    with pytest.raises(DR.PrematureD4Substitution):
        e.subs(DR.d, 4)
    assert DR.substitute_d(e, 4, "test_p17: parse_d authorised path") == 16


def test_g2_plain_symbol_from_sympify_is_detected():
    plain = sp.sympify("(d - 4)/(d - 1)")                     # the G2 hazard
    assert plain.subs(sp.Symbol("d"), 4) == 0                  # a plain d is NOT guarded ...
    with pytest.raises(DR.PlainDimensionSymbol):
        DR.assert_opaque_d(plain)                              # ... and is detected
    with pytest.raises(DR.PlainDimensionSymbol):
        DR.parse_d("d + 1", {"d": sp.Symbol("d")})             # rebinding d in extra_locals refused
    assert DR.assert_opaque_d(DR.d + 1) == DR.d + 1


def test_g2_stored_certificate_strings_round_trip_through_parse_d():
    c = cj("P16_batchC.json")
    for g in QC.HH_SLOT_GRAPHS:
        for lab, v in c["graphs"][g]["result"].items():
            e = P(v["A_exact_d_times_MB"])
            DR.assert_opaque_d(e)
            assert e.has(DR.d) or e.is_number, (g, lab)


def test_g3_wild_pattern_not_refused_and_documented():
    w = sp.Wild("w")
    assert (DR.d + 1).match(DR.d + w) == {w: 1}                # substitutes no value for d; not refused (docum
  +> ented)
    with pytest.raises(DR.PrematureD4Substitution):
        (DR.d + 1).match(DR.d)


def test_g_contract_revision_disclosed_and_layers_identical():
    rev = QC.DISCLOSED_CONTRACT_REVISIONS_P17
    assert list(rev) == ["code/p5_entry/dred.py"] and "G2" in json.dumps(rev)
    cur = {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest()
           for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    assert sorted(k for k, h in Q15.P14R_CONTRACT_SHA256.items() if cur.get(k) != h) == ["code/p5_entry/dred.p
  +> y"]
    assert set(cur) == set(Q15.P14R_CONTRACT_SHA256)                                  # no new contract file
    assert cur["code/p5_entry/dred.py"] != QC.P16_DRED_SHA256                         # the p17 revision is r
  +> eal
    for k, h in {**Q16.P15_CALC_SHA256, **QC.P16_CALC_SHA256}.items():                # p15 + p16 layers iden
  +> tical
        assert hashlib.sha256((ROOT / k).read_bytes()).hexdigest() == h, k


def test_p14_p15_p16_certificate_values_unchanged_vs_p16_reference():
    """the p17 dred.py revision only ADDS functions: every P13/P14/P15/P16 certificate content hash (timings a
  +> nd
    run-specific keys ignored; the layer release records excluded) equals the archived p16 author reference.""
  +> "
    import certio
    ref = json.loads((ROOT / "archival" / "p16" / "reference_values_p16.json").read_text())["values"]
    ch = ref["certificate_content_sha256_ignoring_timings"]
    diff = []
```

```
    for name, h in ch.items():
        if name.startswith(("P13_", "P14_", "P15_", "P16_")) and name not in ("P15_release", "P16_release"):
            if certio.file_content_sha(CERT / f"{name}.json", QC.RUN_SPECIFIC_KEYS) != h:
                diff.append(name)
    assert not diff, diff


# ============================================================== priority 1: hh evanescent poles vs the ch s
  +> lot
def test_hh_slot_verdict_controls_and_mutations():
    c = cj("P17_hh_slot.json")
    assert c["revision"] == "p17" and c["decoupling_proven"] is True
    assert c["status"] == "decoupled_from_ch_slot_proven_as_sum" and c["verdict"].startswith("DECOUPLED (proof
  +> )")
    assert c["surviving_evanescent_structures_in_slot"] == []
    assert len(c["controls"]) >= 17 and all(v is True for v in c["controls"].values())
    assert sorted(c["mutations"]) == sorted(("M1_wrong_relative_sign_D03", "M2_D03_times_2_symmetry_factor",
                                             "M3_D03_evanescent_dropped_naive_4D", "M4_trace_operator_in_place
  +> _of_s_h"))
    assert all(v["accepted_as_rejection"] is True for v in c["mutations"].values())
    assert c["quantum_inputs_evaluated"] == "not_earned" and "NOT evaluated" in c["D11"]


def test_hh_slot_row_and_operator():
    c = cj("P17_hh_slot.json")
    assert list(c["rows_with_D01_D03"]) == [QC.SLOT_ROW]
    led = json.loads((ROOT / "ledgers" / "P13_ch_dependency_rows.json").read_text())["rows"]
    rows = [r["id"] for r in led if any(g.split("_")[0] in ("D01", "D03") for g in r["graphs"])]
    assert rows == [QC.SLOT_ROW]                                  # independent reading of the ledger
    op = c["operator_s_h"]
    assert op["exact"] and op["unique"] and op["covariant"] == [["I/2", [["g4", "m", "v"], ["p", "n"]]]]


def test_hh_slot_cancellation_independent_of_hh_slot_code():
    """p_m G for D01 and D03 as recorded by the (byte-identical) p16 Batch C code: their sum vanishes exactly
  +> in d,
    structure by structure (4D and evanescent); each is nonzero."""
    c = cj("P16_batchC.json")["graphs"]
    a, b = c["D01"]["longitudinal_projection_p_m_G"], c["D03"]["longitudinal_projection_p_m_G"]
    keys = set(a) | set(b)
    assert any(k.startswith("t_") for k in keys)                  # evanescent structures present
    for k in keys:
        s = sp.cancel(P(a.get(k, "0")) + P(b.get(k, "0")))
        assert s == 0, k
    assert any(sp.cancel(P(v)) != 0 for v in a.values()) and any(sp.cancel(P(v)) != 0 for v in b.values())


def test_hh_slot_transverse_coordinates_exact():
    fd = cj("P17_hh_slot.json")["four_D_poles"]
    assert fd["basis_dimension"] == 5 and fd["basis_rank"] == 5 and fd["dim_pair_symmetric_4D_structures"] ==
  +> 5
    assert fd["span_is_all_4D_structures"] is True                # automatic (disclosed as such)
    assert fd["longitudinal_constraint_rank"] == 3 and fd["transverse_subspace_dimension"] == 2
    k1, k2 = [[sp.Rational(x) for x in v] for v in fd["transverse_basis_in_p14r_coordinates"]]
    z = {g: [sp.sympify(x, locals={"I": sp.I}) for x in fd["graphs"][g]["coordinates_p14r_basis"]]
         for g in ("D01", "D03", "D01+D03")}
    I = sp.I
    assert z["D01"] == [-27 * I / 160, -71 * I / 240, 19 * I / 60, 11 * I / 20, -13 * I / 30]
    assert z["D03"] == [7 * I / 96, I / 24, -I / 8, -I / 24, I / 12]
    assert [sp.simplify(x + y) for x, y in zip(z["D01"], z["D03"])] == z["D01+D03"]
    al, be = sp.symbols("al be")
    sol = sp.solve([al * u + be * v - w for u, v, w in zip(k1, k2, z["D01+D03"])], [al, be], dict=True)
    assert sol == [{al: 61 * I / 120, be: -7 * I / 20}]           # the summed pole lies in the transverse ker
  +> nel
    for g in ("D01", "D03"):
        assert not sp.solve([al * u + be * v - w for u, v, w in zip(k1, k2, z[g])], [al, be], dict=True)
        assert fd["graphs"][g]["transverse"] is False
    assert fd["graphs"]["D01+D03"]["transverse"] is True


def test_hh_slot_evanescent_poles_recorded_not_dropped():
    ev = cj("P17_hh_slot.json")["evanescent_poles_D01_plus_D03_in_hh_green_function"]
    assert len(ev) == 8 and all(v != "0" for v in ev.values()) and all("gt_" in k for k in ev)


def test_g6_phase_convention_documented():
    c = cj("P17_hh_slot.json")
    assert "overall I" in c["phase_convention_G6"] and "real" in c["phase_convention_G6"]
    assert "D13" in QC.PHASE_CONVENTION_G6 and rs()["phase_convention_G6"] == QC.PHASE_CONVENTION_G6
```

```
# ============================================================ status, hard stops, change log
def test_p17_release_status_and_hard_stops():
    s = rs()
    assert s["revision"] == "p17" and s["publication"] == "accepted" and s["release_claim_accepted"] is True
    assert s["hh_evanescent_decoupling_proven"] is True and s["evanescent_hh_counterterm_added"] is False
    assert s["contract_identity"]["added"] == [] and s["contract_identity"]["changed_vs_p14r"] == ["code/p5_en
  +> try/dred.py"]
    assert s["batchC_evaluated"] is True and s["E2_decoupling_proven"] is True
    assert s["D11_evaluated"] is False and s["graphs"]["D11"]["status"] == "unevaluated"
    assert s["D11_matching_verified"] is False and s["quantum_inputs_evaluated"] == "not_earned"
    assert s["hard_stops"] == QC.HARD_STOPS_P17 and s["hard_stops_hold"] is True
    assert s["O1_closure"] == "NOT_CLOSED" and s["symmetry_restored"] is False and s["bulk_symmetry_restored"]
  +>  is False
    assert s["XVI_P_registered"] is False and s["Route_B_discharged"] is False and s["u_R_c_delta_added"] is F
  +> alse
    assert s["XV_B_opened"] is False and s["XIV_J_opened"] is False
    assert s["insertion_status"] == "Frozen_negative_obstruction_preserved"
    assert s["XVI_M_sha256"].startswith("41aa1099") and s["XVI_M_pin_ok"] is True
    assert s["milestones"]["status"]["P.5_identity"] == "open"
    assert s["independent_rebuild_passed"] in ("pending", False)   # p18: the live p17 report is bound to the
  +> p17 tree
    assert s["contract_identity"]["guard_probes"]["all_refused"] is True
    assert s["contract_identity"]["guard_probes"]["G2_ok"] is True


def test_p16_independent_rebuild_recorded_and_not_inherited():
    s = rs()
    rec = s["p16_independent_rebuild"]
    assert s["independent_rebuild_passed_p16"] is True
    assert rec["report_sha256"] == QC.P16_INDEPENDENT_REPORT_SHA256 and rec["report_sha256_pinned"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P16_SOURCE_TREE_SHA256
    assert rec["bound_to_source_zip_sha256"] == QC.P16_SOURCE_ZIP_SHA256
    assert "SAME-BOX" in rec["caveat"] and rec["mismatches"] == 0
    assert all(rec["archived_p16_records_pinned"].values())
    import source_manifest as SM
    assert SM.tree_sha(ROOT) != QC.P16_SOURCE_TREE_SHA256


def test_g4_change_log_lists_every_changed_file_incl_reference():
    s = rs()
    cl = s["change_log_vs_p16"]
    assert cl["base_pinned_ok"] is True
    base = json.loads((ROOT / QC.P16_SOURCE_LIST).read_text())["files"]
    import source_manifest as SM
    cur = {r: hashlib.sha256((ROOT / r).read_bytes()).hexdigest() for r in SM.source_files(ROOT) if r != SM.RE
  +> FERENCE_FILE}
    changed = sorted(set(cur) ^ set(base) | {k for k in set(cur) & set(base) if cur[k] != base[k]} | {SM.REFER
  +> ENCE_FILE})
    assert sorted(cl["added"] + cl["removed"] + cl["modified"]) == changed
    assert SM.REFERENCE_FILE in cl["added"]                                    # G4
    assert cl["contract_files_changed"] == ["code/p5_entry/dred.py"]
    assert sorted(x for x in cl["modified"] if x.startswith("tests/") and not __import__("contract_registry").
  +> p22_revised(x)) == sorted(   # p22 filter
        x for x in QC.DISCLOSED_TEST_REVISIONS_P17 if not __import__("contract_registry").p22_revised(x))


def test_version_label_p17():
    import build as B
    assert QC.VERSION_LABEL == "XVI-P p17" and B.QC.VERSION_LABEL == f"XVI-P {B.QC.REVISION}"   # p18: current
  +>  label
    assert "certificates/P17_hh_slot.json" in B.VALUE_BLOCKS


# ============================================================ p17 gate mutations (copy of the built packa
  +> ge)
def _copy():
    d = Path(tempfile.mkdtemp(prefix="p17gate_"))
    for sub in ("code", "certificates", "ledgers", "exports", "tools", "archival", "reproducibility", "provena
  +> nce", "tests"):
        if (ROOT / sub).exists():
            shutil.copytree(ROOT / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__"))
    for f in ("README.md", "REBUILD_INSTRUCTIONS.md", "RELEASE_STATUS.json"):
        if (ROOT / f).exists():
            shutil.copy(ROOT / f, d / f)
    (d / "manuscript").mkdir()
    shutil.copy(ROOT / "manuscript" / "preamble.tex", d / "manuscript" / "preamble.tex")
    return d


def _edit_cert(d, name, fn):
    f = d / "certificates" / name
```

```
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c, indent=1))


def _tamper_hh_control(d):
    _edit_cert(d, "P17_hh_slot.json", lambda c: c["controls"].__setitem__("H3_p_dot_G_sum_zero_exact_in_d", Fa
  +> lse))


def _surviving_structure(d):
    _edit_cert(d, "P17_hh_slot.json", lambda c: c.__setitem__("surviving_evanescent_structures_in_slot",
                                                              ["gt_rs p_m p_n (p.p)^1"]))


def _tamper_p16_layer(d):
    f = d / "code" / "p5_calc" / "batchC.py"
    f.write_text(f.read_text() + "\n# tampered\n")


def _tamper_contract(d):
    f = d / "code" / "p5_entry" / "counterterm_domain.py"
    f.write_text(f.read_text() + "\n# tampered\n")


def _new_contract_file(d):
    (d / "code" / "p5_entry" / "evanescent_hh_counterterm.py").write_text("# undisclosed contract file\n")


def _tamper_p16_report(d):
    f = d / "archival" / "p16" / "reproducibility" / "independent_rebuild_report_p16.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


def _hard_stop(d):
    _edit_cert(d, "P17_hh_slot.json", lambda c: c.__setitem__("quantum_inputs_evaluated", "earned"))


@pytest.mark.parametrize("mut", [_tamper_hh_control, _surviving_structure, _tamper_p16_layer, _tamper_contract
  +> ,
                                 _new_contract_file, _tamper_p16_report, _hard_stop], ids=lambda f: f.__name__
  +> )
def test_certificate_mutation_p17_gate_rejects(mut):
    d = _copy()
    try:
        mut(d)
        r = subprocess.run([sys.executable, "code/p5_calc/release_p17.py"], cwd=d, capture_output=True, text=T
  +> rue,
                           timeout=600)
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d / "certificates" / "P17_release.json").exists()
        assert (d / "certificates" / "P17_release_rejected.json").exists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_p17_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = subprocess.run([sys.executable, "code/p5_calc/release_p17.py"], cwd=d, capture_output=True, text=T
  +> rue,
                           timeout=600)
        assert r.returncode == 0, r.stdout[-1500:]
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

143 Source: p18_conventions.py (p18 constants, version label, pins, disclosures)

[listing 110/133] code/p5_calc/p18_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p18 conventions: P.5 calculation, fourth step (D11: discharge of the matching premise M2, evaluation
  +> of D11
under the declared prescription, the full hh slot D01+D02+D03+D11+D12; p17 reviewer findings H1-H5).

Layering.  p14r contract (code/p5_entry/*)  ->  p15 layer  ->  p16 layer  ->  p17 layer (code/p5_calc/{hh_slot
```

```
  +> ,
p17_conventions,release_p17}.py; byte-identical, sha256 below)  ->  p18 layer (code/p5_calc/{d11,p18_conventio
  +> ns,
release_p18}.py, P18_*).  The p18 gate (release_p18.py) runs after the p14r, p15, p16 and p17 gates and requir
  +> es all
four to be accepted.

Disclosed CONTRACT revision in p18 (DISCLOSED_CONTRACT_REVISIONS_P18): code/p5_entry/dred.py documentation onl
  +> y
(H3 stale comment, H5 parse_d(srepr(e)) raises).  No new contract file: no evanescent h.h counterterm is neede
  +> d by the
ch-projected slot (D11 has p_m G_D11 = 0 identically in d; certificates/P18_hh_slot.json).
"""
from __future__ import annotations

REVISION = "p18"
VERSION_LABEL = "XVI-P p18"           # single version variable (manuscript footer is checked against it)
CERT_PREFIX = "P18"
CONTRACT_REVISION = "p18 (p14r contract + disclosed p15/p16/p17/p18 revisions)"
DATE = "2026-10-09"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p18.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p18.json"
P17_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p17.json"      # written by the reviewer;
  +>  never here
P17_INDEPENDENT_REPORT_SHA256 = "92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb"
P17_SOURCE_TREE_SHA256 = "2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92"
P17_SOURCE_ZIP_SHA256 = "7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416"
P17_ARCHIVED_RECORDS = {
    "archival/p17/reference_values_p17.json": "d4345c282aee89676ee7d0b077fd8fc94ebd8735cd78499550b2666a0b9103b
  +> 1",
    "archival/p17/reproducibility/author_rebuild_report_p17.json":
        "bd7d8baeabb6337a07234f93dd098b11ab0f95d6ccb640f016d827d952a0bd6f",
    "archival/p17/reproducibility/independent_rebuild_report_p17.json": P17_INDEPENDENT_REPORT_SHA256,
    # per-file sha256 list of the released p17 source-only zip (base of the p18 change log; works source-only)
    "archival/p17/p17_source_files.json": "2b4ccc98c86d0977b71921275b9cc797002ff877e0bf8f466ae86e9ace4cdc42",
}
P17_SOURCE_LIST = "archival/p17/p17_source_files.json"

HH_SLOT_GRAPHS = ("D01", "D03", "D11")            # + D02 = D12 = 0 (P15_batchA.json)
SLOT_ROW = "(SO,Gamma1^[2]) s2 K:K.c"
INVENTORY_P18 = ("D01", "D02", "D03", "D11", "D12", "D13", "D14a", "D14b", "D14c")

# the three ch slots (P13_ch_source_audit surviving rows) and where each input is evaluated
INPUT_EVALUATION_P18 = {
    "D01": "certificates/P16_batchC.json", "D03": "certificates/P16_batchC.json",
    "D02": "certificates/P15_batchA.json", "D12": "certificates/P15_batchA.json",
    "D13": "certificates/P15_D13.json",
    "D14a": "certificates/P15_batchA.json", "D14b": "certificates/P15_batchA.json",
    "D14c": "certificates/P15_batchA.json",
    "D11": "certificates/P18_D11.json",
}

HARD_STOPS_P18 = {
    "projected_defect_computed": "not_earned",
    "projected_identity_verified": "not_earned",
    "projected_restoration_verified": "not_earned",
    "bulk_symmetry_restored": False,
    "symmetry_restored": False,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": False,
    "XIV_J_opened": False,
    "Route_B_discharged": False,
    "u_R_c_delta_added": False,
    "XVI_P_registered": False,
}
# flags that p18 may EARN, each only if its certificate says so (release_p18 reads them; never set by hand)
EARNABLE_P18 = ("D11_matching_verified", "D11_evaluated", "quantum_inputs_evaluated")

# p17 layer: byte-identical in p18 (sha256 from XVI-P-p17-source-only.zip; release_p18 recomputes them)
P17_CALC_SHA256 = {
    "code/p5_calc/hh_slot.py": "14330d4b9337773e759fd36262b7bd4653d3cc0b83bed4cdc6dec62b2c216465",
    "code/p5_calc/p17_conventions.py": "a239a2b695e8d58bf8b8405c0ed4a98433da7034b6b255d2477866c5b6db306c",
    "code/p5_calc/release_p17.py": "390b263089aa9e794805030d61d8ed773878083927750272ca4e983f10600d92",
}
P14R_CONTRACT_FILES_CHANGED_IN_P18 = ("code/p5_entry/dred.py",)
NEW_CONTRACT_FILES_P18 = ()                       # no evanescent h.h counterterm file (slot decoupling incl.
  +> D11)
P17_DRED_SHA256 = "7b0e46c3d0086fce0f7fec01a7d5e33021435e88cb15c99de176683fb65638c8"     # dred.py as released
  +>  in p17
# sha256 of ast.dump of the p17 dred.py with every docstring removed (release_p18.ast_sha; Python-version depe
```

```
  +> ndent:
# computed with the build interpreter; the p18 revision must keep it -- comments/docstrings only)
P17_DRED_AST_SHA256 = "8a32857a194bd0de67bc7f529c06f975124b3989c97e04ec344582f6b9a4e6ea"

DISCLOSED_CONTRACT_REVISIONS_P18 = {
    "code/p5_entry/dred.py": {
        "kind": "contract revision (disclosed), p18: COMMENTS/DOCSTRING ONLY (on top of p15 F1, p16 F3, p17 G1
  +> -G3)",
        "finding": "H3/H5, reproducibility/independent_rebuild_report_p17.json",
        "why": ("H3: OpaqueDimension.matches comment named tests/test_p16.py as the scan (now tests/test_p18.p
  +> y); "
                "H5: parse_d(srepr(e)) raises PlainDimensionSymbol (fails closed) but this was not documented"
  +> ),
        "change": "comment/docstring text only; no statement executed by Python changed (AST-equal, tests/test
  +> _p18.py)",
        "rules_R1_R6_changed": False,
        "released_values_affected": False,
        "why_unaffected": ("the module AST without docstrings equals that of the p17 dred.py (tests/test_p18.p
  +> y); "
                           "the P14..P17 certificate content hashes recomputed in p18 equal "
                           "archival/p17/reference_values_p17.json"),
    },
}
DISCLOSED_TEST_REVISIONS_P18 = {    # test files changed vs the p17 zip (release_p18 change log)
    "tests/test_p16.py": ("test_f4_value_block_readback_detects_a_changed_value: the planted mutation is appli
  +> ed to a "
                          "snippet that occurs VERBATIM in the typeset page (the p18 full-slot verdict is long
  +> er than "
                          "the 110-column wrap, so str.replace on the wrapped page was a no-op and the test co
  +> uld not "
                          "plant its mutation). The readback itself was correct (a mutation inside the wrapped
  +>  verdict "
                          "is detected; tests/test_p18.py::test_readback_detects_mutation_inside_wrapped_value
  +> ). One "
                          "line; no assertion weakened."),
    "tests/test_p14.py": ("source-set / zip / reference / author-report names p17 -> p18; generated-prefix lis
  +> t "
                          "extended by P18_* certificates/ledgers and the P18 results table; top-level status
  +> revision "
                          "p18. (Its quantum_inputs_evaluated assertions read the p14r CONTRACT record "
                          "certificates/P14_release.json and are unchanged.) No assertion weakened."),
    "tests/test_p17.py": ("rs() reads the p17 LAYER record certificates/P17_release.json (the top-level "
                          "RELEASE_STATUS.json is now the p18 status); the version-label test checks that the
  +> build's "
                          "CURRENT label is 'XVI-P <current revision>'; the p17 layer's own independent flag (
  +> recomputed "
                          "by release_p17.py against the CURRENT tree) may be False (the reviewer's live p17 r
  +> eport is "
                          "bound to the p17 tree) or pending (source-only rebuild); the p17 PASS itself is rec
  +> orded by "
                          "release_p18 as independent_rebuild_passed_p17. Three lines; all other p17 assertion
  +> s "
                          "unchanged."),
}
# H1/H3: package call sites of sympify / S (and aliases) with a non-literal argument, reviewed.  Two kinds (H3
  +> : the
# p17 header said "never a stored string", which was inaccurate for 7 entries): (a) in-process sympy objects,
  +> and (b)
# STORED STRINGS that provably contain no d (export prefactor strings -- kappa only, checked by tests -- and t
  +> he frozen
# p13 classical srepr export).  Every entry says which.  Any new site fails the scan (tests/test_p18.py).
PARSE_SITES_REVIEWED = {
    'code/p5_calc/batchA.py::evaluate::sp.sympify(body["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::regen_equals_export::sp.sympify(body["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::evaluate_graph::sp.sympify(body["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::main::sp.sympify(b13["prefactor"], locals={"kappa": KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/d13.py::route1::sp.sympify(body["prefactor"], locals={"kappa": sp.Symbol("kappa")})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_entry/hessian_route.py::_export_side::sp.sympify(body["prefactor"], locals={"kappa": F.KAPPA})':
        'export prefactor STRING (kappa only; tests/test_p17.py checks every exported prefactor is d-free)',
    'code/p5_calc/batchC.py::pole_in_hh_basis::sp.sympify(sol[0].get(z, z))':
        'argument is a sympy object computed in-process (never a stored string) (sympy.solve output)',
    'code/p5_calc/batchC.py::mutations::sp.sympify(a0)':
        'argument is a sympy object computed in-process (never a stored string) (A(d) of the evaluation; the p
  +> remature_d4 mutation must raise: certificate shows rejected=True)',
    'code/p5_calc/batchC.py::main::sp.sympify(v)':
        'argument is a sympy object computed in-process (never a stored string) (A(d) values; R5_d_retained ch
```

```
  +> eck)',
    'code/p5_calc/d13.py::exact_fit::sp.sympify(a)':
        'argument is a sympy object computed in-process (never a stored string) (matrix entries)',
    'code/p5_calc/d13.py::expand_coeff::sp.sympify(c)':
        'argument is a sympy object computed in-process (never a stored string) (fitted coefficient)',
    'code/p5_calc/d13.py::mutations::sp.sympify(v)':
        'argument is a sympy object computed in-process (never a stored string) (fitted values)',
    'code/p5_calc/d13.py::_opaque_d_refuses::sp.sympify(fit1["T2"])':
        'argument is a sympy object computed in-process (never a stored string) (fit result; the probe must ra
  +> ise: certificate shows it does)',
    'code/p5_calc/d13.py::main::sp.sympify(v)':
        'argument is a sympy object computed in-process (never a stored string) (fit values; R5_d_retained che
  +> ck)',
    'code/p5_calc/integrals.py::fp_two_point::sp.sympify(poly)':
        'argument is a sympy object computed in-process (never a stored string) (numerator polynomial)',
    'code/p5_calc/integrals.py::realize::sp.sympify(expr)':
        'argument is a sympy object computed in-process (never a stored string)',
    'code/p5_calc/release_p17.py::f3_hardening::sp.sympify(str(e))':
        "deliberate G2 hazard probe (the plain Symbol('d') must be caught by assert_opaque_d)",
    'code/p5_entry/dred.py::assert_opaque_d::sp.sympify(expr)':
        'argument is a sympy object computed in-process (never a stored string) (the checker itself)',
    'code/p5_entry/dred.py::parse_d::sp.sympify(text, locals=loc)':
        "the guarded loader itself (loc['d'] is the opaque d; the result is checked by assert_opaque_d)",
    'code/p5_entry/dred.py::substitute_d::sp.sympify(expr)':
        'argument is a sympy object computed in-process (never a stored string) (the authorised substitution)'
  +> ,
    'code/p5_entry/dred.py::substitute_d::sp.sympify(value)':
        'numeric substitution value (4 or 4 - 2 eps)',
    'code/p5_entry/dred.py::__init__::sp.sympify(trace_hat)':
        'display-only numeric trace (Rules(trace_hat=...), refused unless display_only)',
    'code/p5_entry/release_p14.py::classical_block::sp.sympify(txt)':
        'frozen p13 classical export ell*_generating_polynomial.srepr (4D, contains no d); hashed only',
    'code/p5_entry/shadow.py::even::sp.sympify(x)':
        'argument is a sympy object computed in-process (never a stored string) (scalar coefficient)',
}
RUN_SPECIFIC_KEYS = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed",
                     "live_report_identical_to_archived", "live_report_present", "archived_zip_sha256_ok",
                     "list_equals_archived_zip")
PHASE_CONVENTION_G6 = (
    "Overall phase (G6).  Every graph is exported as G = kappa^2 N_eps Ghat with N_eps = i S_eps/(16 pi^2) and
  +>  "
    "Ghat = -i * prefactor/kappa^2 * int N/(D1 D2).  For D01 (export prefactor kappa^2/2) and D03 "
    "(prefactor -kappa^2, closed ghost loop) the prefactor is real, so A(d) carries an overall factor I; for D
  +> 13 the "
    "exported prefactor is I*kappa^2, so A(d) is real.  The relative phase between graphs of the same slot is
  +> fixed by "
    "the common -i and the real export prefactors (D01 vs D03: real ratio); it is checked by the exact cancell
  +> ation of "
    "the summed hh slot and by the rejected mutation M1 (relative sign).  Physical overall phase: not asserted
  +> .")
```

144 Source: d11.py (p18 M2 discharge, D11 evaluation, full hh slot)

[listing 111/133] code/p5_calc/d11.py

```
#!/usr/bin/env python3
"""XVI-P p18: D11 (scalar matter loop in the hh two-point function) -- discharge of the matching premise M2,
evaluation of D11 under the declared prescription, and the full hh slot D01 + D02 + D03 + D11 + D12.

M2 (certificates/P14_prescription_D11_matching.json, matching.M2_nonlocal_agreement_on_U):
    for all f in D(U), U = R^4 \\ {0}:  <FT^-1[P^DR_AB](z), f> = <R_A R_B Pi_tr^R(z), f>,
    Pi_tr^R = (i kappa^2/4) K u^R,  K = alpha D2 + beta D0,  alpha = A^2/60, beta = A^2/18, A = 1/(4 pi^2).

Route B (explicit, exact; primary).  P^DR = kappa^2 N_eps Ghat with Ghat = sum_s A_s(d) S_s MB, MB = 1/eps + 2
  +>  - L,
L = log(-p^2/mu^2 - i0): the NONLOCAL part of P^DR at d -> 4 is  -(i kappa^2/16 pi^2) L sum_s A_s(4) S_s(p); e
  +> very other
term is a polynomial in p.  On the XVI-O side, FT[u^R](p) = c_FT log(-p^2 - i0) + const (Gel'fand-Shilov: u^R
  +> is the
finite part of (Q - i0)^zeta at zeta = -2, whose residue is r delta; hence c_FT = -r = i pi^2), and K -> Khat(
  +> p) with
d_m -> -i p_m (P_tr_mn -> p_m p_n - eta_mn p^2; fourth order, so the sign convention of the FT drops out).  M2
  +>  then
holds iff the two L-coefficients agree, i.e. for every 4D component pair (A, B):
    X_AB := sum_s A_s(4) S_s|_AB  ==  -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB,   Y_AB := R_A R_B (Dhat2/60 + Dhat0
  +> /18).
```

```
If they agree, FT^-1[P^DR] - R R Pi_tr^R has a polynomial Fourier transform, so it is supported at {0}: M2.  (
  +> mu enters
only through polynomials: mu-independence on U.)  c_FT is calibrated twice: (a) independently of XVI-O, from t
  +> he
library DR master bubble: FT[D_F^2] = (i)^2 int 1/(k^2 (p-k)^2) = -N_eps MB, and D_F^2 = A^2 u; (b) -r with th
  +> e
XVI-O residue r = -i pi^2 (M5).
Route A (structural; corroboration).  M1 (re-run here): -i pref N^P/(D1 D2) = (i kappa^2/4) R R V V [C~C~ + C~
  +> C~]
exactly (100/100), i.e. the P integrand IS the Fourier convolution of the XVI-O product T = V V (C C + C C); a
regularised convolution agrees with the pointwise product on U (any two extensions differ at z = 0 only); and
T = K u on U (XVI-O all_256_symbolic, re-run here, frozen code imported read-only).  Route A uses the cited
distribution-theory fact; Route B uses only the stated Fourier transform of (Q - i0)^zeta.
Scope.  4D (hatted) components only: XVI-O is strictly four-dimensional.  The evanescent (gt) structures of D1
  +> 1 have
NONZERO L-coefficients (recorded); they are epsilon-scalar components and are not part of M2's 4D statement.
"""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "code" / "matter_loop", ROOT /
  +>  "tools"):
    sys.path.insert(0, str(p))
import dred  # noqa: E402
import covariant as C  # noqa: E402
import batchC as BC  # noqa: E402
import hh_slot as H  # noqa: E402
import integrals as IL  # noqa: E402
import p18_conventions as QC  # noqa: E402

d = dred.d
I = sp.I
PI = sp.pi
ETA = (1, -1, -1, -1)
PS = sp.symbols("p0:4")
LS = sp.symbols("l0:4")
PPV = sum(ETA[i] * PS[i] ** 2 for i in range(4))
A_XVIO = 1 / (4 * PI ** 2)
ALPHA, BETA = sp.Rational(1, 60), sp.Rational(1, 18)          # alpha/A^2, beta/A^2 (XVI-O, derived; re-derive
  +> d below)
EXT = dict(H.EXT, **{"c hat, h tilde": ("c^", "a~", "b~")})      # H2: the seventh assignment


def log(m):
    print(f"[d11 {time.strftime('%H:%M:%S')}] {m}", flush=True)


def s(x):
    return str(sp.factor(x)) if x != 0 else "0"


def a_at_4(A, why):
    return {k: dred.substitute_d(v, 4, f"d11: {why}") for k, v in A.items()}


# ------------------------------------------------------------------ the two L-coefficients, component by compon
  +> ent
def x_component(A4, Ai, Bi, offdiag=2):
    """sum_s A_s(4) S_s on the 4D component pair (A, B) (same convention as the covariant fit: pair weight, up
  +> per
    external indices via ETA, lower p_r); gt structures vanish on 4D components."""
    import feynman as F
    (m, n), (r, s_) = F.HC[Ai], F.HC[Bi]
    asg = {"m": m, "n": n, "r": r, "s": s_}
    w = C.pair_weight(BC.SPEC, asg, offdiag, upper=True)
    tot = 0
    for (orb, e), c in A4.items():
        if C.is_evanescent(orb):
            continue
        tot += c * PPV ** e * C.orbit_value(orb, asg, LS, PS)
    return sp.expand(w * tot)


def _ph(a, b):
```

```
    return PS[a] * PS[b] - (ETA[a] if a == b else 0) * PPV


def khat(a, b, c, e, al, be):
    D0 = _ph(a, b) * _ph(c, e)
    D2 = sp.Rational(1, 2) * (_ph(a, c) * _ph(b, e) + _ph(a, e) * _ph(b, c)) - sp.Rational(1, 3) * D0
    return al * D2 + be * D0


def y_component(Ai, Bi, al=ALPHA, be=BETA):
    import feynman as F
    from prescription_d11 import raise_map
    RA, RB = raise_map(F.HC[Ai]), raise_map(F.HC[Bi])
    return sp.expand(sum(RA[a, b] * RB[c, e] * khat(a, b, c, e, al, be)
                         for a in range(4) for b in range(4) for c in range(4) for e in range(4)
                         if RA[a, b] != 0 and RB[c, e] != 0))


def compare(A4, c_ft, al=ALPHA, be=BETA, offdiag=2):
    """X_AB == -4 pi^2 c_FT A^2 Y_AB on all 100 component pairs (exact)."""
    k = sp.simplify(-4 * PI ** 2 * c_ft * A_XVIO ** 2)
    bad, nz = [], 0
    for i in range(10):
        for j in range(10):
            x = x_component(A4, i, j, offdiag)
            nz += x != 0
            if sp.expand(x - k * y_component(i, j, al, be)) != 0:
                bad.append([i, j])
    return {"pairs": 100, "nonzero_pairs": nz, "mismatches": len(bad), "first": bad[:3], "agree": not bad,
            "constant": str(k)}


def ft_constant():
    """c_FT (coefficient of log(-p^2 - i0) in FT[u^R]) two ways."""
    ms = IL.master_series(1)
    pole, fin = sp.expand(ms[-1]), sp.expand(ms[0])
    n_eps = I / (16 * PI ** 2)                                  # S_eps -> 1 (MS-bar)
    c_bubble = sp.expand(-n_eps * fin.coeff(IL.L))              # FT[D_F^2] = (i)^2 N_eps MB
    c_a = sp.simplify(c_bubble / A_XVIO ** 2)
    m5 = json.loads((ROOT / "certificates" / "P14_prescription_D11_matching.json").read_text())["matching"]["M
  +> 5_residue"]
    r_ok = m5["r"] == "-i * pi**2"                              # string compared, not parsed
    r = -I * PI ** 2
    return {"master_pole": str(pole), "master_finite": str(fin), "pole_is_1": pole == 1,
            "L_coefficient_of_MB": str(fin.coeff(IL.L)),
            "c_FT_from_DR_bubble": str(c_a), "c_FT_from_minus_r": str(-r), "r_certificate": m5["r"],
            "r_certificate_as_expected": r_ok, "agree": bool(r_ok and sp.simplify(c_a + r) == 0),
            "value": c_a}


def xvio_controls():
    import d11_global_representation as O
    ab = O.derive_alpha_beta()
    Asym = O.A_SYM
    a_ok = sp.simplify(ab["alpha_sympy"] - Asym ** 2 * ALPHA) == 0 and sp.simplify(ab["beta_sympy"] - Asym **
  +> 2 * BETA) == 0
    full = O.all_256_symbolic(ab["alpha_sympy"], ab["beta_sympy"])
    return {"alpha_beta_rederived": {"alpha": str(ab["alpha_sympy"]), "beta": str(ab["beta_sympy"]), "match":
  +> a_ok},
            "K_u_equals_T_all_256": {"status": full.get("status"), "pass": bool(full.get("pass")),
                                     "components": full.get("symbolic_components_checked"),
                                     "failures": len(full.get("symbolic_failures", []))},
            "ok": bool(a_ok and full.get("pass"))}


# ---------------------------------------------------------------- M2
def m2(evs):
    import prescription_d11 as PD
    A = evs["D11"]["A"]
    A4 = a_at_4(A, "L-coefficient A(4) of D11 (after exact-d algebra; M2 nonlocal comparison)")
    log("M2 calibration of c_FT")
    ft = ft_constant()
    c = ft["value"]
    log("M2 route B: 100 component pairs")
    main = compare(A4, c)
    log("M2 route A: M1 integrand identity and XVI-O K u = T")
    m1 = PD.M1_identity()
    xo = xvio_controls()
    trans = {"p_dot_G_4D_part": H.p_dot_G(H.g_terms({k: v for k, v in A.items() if not C.is_evanescent(k[0])})
  +> ),
             "p_dot_G_evanescent_part": H.p_dot_G(H.g_terms({k: v for k, v in A.items() if C.is_evanescent(k[0
```

```
+> ])}))}
  trans["transverse_exact_in_d"] = not trans["p_dot_G_4D_part"] and not trans["p_dot_G_evanescent_part"]
  log("M2 mutations")
  A01_4 = a_at_4(evs["D01"]["A"], "M2 mutation: D01 in place of D11 (A(4))")
  muts = {
      "alpha_beta_swapped": compare(A4, c, al=BETA, be=ALPHA),
      "offdiagonal_weight_1": compare(A4, c, offdiag=1),
      "FT_log_constant_sign_flipped": compare(A4, -c),
      "symmetry_factor_doubled": compare({k: 2 * v for k, v in A4.items()}, c),
      "D01_in_place_of_D11": compare(A01_4, c),
      "D0_dropped_beta_0": compare(A4, c, be=0),
  }
  for v in muts.values():
      v["rejected"] = not v["agree"]
      v["accepted_as_rejection"] = v["rejected"]
  ev_L = {C.label(*k): s(-v) for k, v in sorted(A4.items(), key=lambda kv: C.label(*kv[0])) if C.is_evanesce
+> nt(k[0])}
  controls = {"route_B_all_100_pairs_agree": main["agree"] and main["nonzero_pairs"] > 0,
              "FT_constant_two_calibrations_agree": ft["agree"],
              "route_A_M1_integrand_identity": m1.get("pass") is True and not m1.get("mismatches"),
              "route_A_XVI_O_K_u_equals_T": xo["ok"],
              "D11_transverse_exact_in_d": trans["transverse_exact_in_d"],
              "routes_FP_PV_agree": evs["D11"]["routes_agree"],
              "covariant_fit_exact_unique": bool(evs["D11"]["fit"]["exact"] and evs["D11"]["fit"]["unique"])
+> ,
              "mutations_all_rejected": all(v["accepted_as_rejection"] for v in muts.values())}
  ok = all(controls.values())
  return {
      "statement": ("M2 (P14_prescription_D11_matching.json): for all f in D(U), U = R^4 \\ {0}: <FT^-1[P^DR
+> _AB], f> "
                    "= <R_A R_B Pi_tr^R, f>, Pi_tr^R = (i kappa^2/4) K u^R, alpha = A^2/60, beta = A^2/18"),
      "verdict": "DISCHARGED (4D components)" if ok else "NOT DISCHARGED",
      "D11_matching_verified": ok,
      "D11_matching_verified_scope": "4D components",
      "evanescent_components_matching": {
          "status": "not_applicable_not_verified",
          "what_they_are": ("the 8 D11 structures carrying gt (the (4-d)-dimensional part of g4 = ghat + gt)
+> : "
                            "components of the hh two-point function with at least one index pair in the "
                            "epsilon-scalar (4-d) directions of DRED; they vanish on hatted 4D components"),
          "why_not_matched": ("XVI-O (u^R_mu, K, Pi_tr^R) is strictly four-dimensional and has NO evanescent
+>  "
                              "counterpart; M2 is a 4D statement, so there is nothing to match them to. Thei
+> r "
                              "L-coefficients are nonzero (recorded below); they are retained (R6) and drop
+> out of "
                              "the ch-projected slot (p_m G_D11 = 0 for the evanescent part separately)")},
      "route_B": {"equation": "X_AB = -4 pi^2 c_FT A^2 Y_AB = -(i/4) Y_AB on every 4D component pair", **mai
+> n},
      "FT_constant": {k: v for k, v in ft.items() if k != "value"},
      "route_A": {"M1": {k: m1.get(k) for k in ("component_pairs_checked", "nonzero_pairs", "mismatches", "p
+> ass",
                                                "recomputed_integrand_sha256", "exported_integrand_sha256")}
+> ,
                  "XVI_O": xo},
      "transversality": {k: (v if isinstance(v, bool) else {kk: str(vv) for kk, vv in v.items()})
                         for k, v in trans.items()},
      "mutations": muts, "controls": controls,
      "analytic_inputs_cited": [
          "Fourier transform of (Q - i0)^zeta (Gel'fand-Shilov): meromorphic in zeta, residue at zeta = -2 i
+> s "
          "r delta, so FT[u^R] = -r log(-p^2 - i0) + const (Route B; cross-checked by the DR bubble)",
          "a polynomial in p is the Fourier transform of a distribution supported at {0}",
          "Route A only: two extensions of the product V V (C C + C C) from U differ by a distribution suppo
+> rted "
          "at 0 (XVI-O lem:BV_rules_on_U, Constructed)"],
      "scope": ("4D (hatted) components; the evanescent gt structures are epsilon-scalar components outside
+> the 4D "
                "statement of M2 (their L-coefficients are nonzero and recorded); IR: the massless bubble at
+>  "
                "p^2 != 0 is IR finite in DR (no scaleless pieces enter the nonlocal part); the XVI-N massle
+> ss "
                "obstruction concerns the classical insertion and is not touched"),
      "evanescent_L_coefficients": ev_L,
      "M3_status": {
          "status": "unconditional on the 4D components (M2 discharged there; cited analytic inputs above)",
          "statement": ("L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at {0}; with kernel scaling degree
+>  <= 8 it is "
                        "a finite sum of d^alpha delta with |alpha| <= 4 (bounds 4/8/4 of P14 M3)"),
          "evanescent_components": "not applicable (no XVI-O counterpart; see evanescent_components_matching
+> )"},
```

```
        "M3_consequence": ("with M2 discharged, L_match = FT^-1[P^DR] - R R Pi_tr^R is supported at 0; both si
  +> des are "
                           "transverse and homogeneous of degree four in p (massless; no mu^4, mu^2 p^2 contac
  +> ts, "
                           "consistent with N4), so L_match lies in the 2-dim transverse subspace of the p14r
  +> h.h "
                           "span; its coordinates need the XVI-O finite constant of u^R and are NOT computed (
  +> M4: "
                           "booked as P-side hh counterterm; u^R_mu unmodified, no c delta)"),
    }


# ------------------------------------------------------------------ D11 evaluation (batchC certificate style)
def d11_mutations(ev):
    import feynman as F  # noqa: F401
    out = {}
    k0 = next(iter(ev["A"]))
    a0 = ev["A"][k0]
    try:
        a0.subs(d, 4)
        out["premature_d4"] = {"rejected": False, "changes_result": None}
    except dred.PrematureD4Substitution as exc:
        lt = BC.laurent_table(ev["A"])
        out["premature_d4"] = {"rejected": True,
                               "changes_result": any(v["d_dependence_shift_of_local_part"] != 0 for v in lt.va
  +> lues()),
                               "reason": f"PrematureD4Substitution: {exc}"}
    ev4 = BC.evaluate_graph("D11", fit=ev["fit"], drop_evanescent_in="fp")
    out["drop_evanescent_gt"] = {"changes_result": set(ev4["A"]) != set(ev["A"]), "rejected": not ev4["routes_
  +> agree"],
                                 "reason": "FP route with gt dropped != PV route (R6 retention check)"}
    f5 = C.covariant_fit(ev["body"]["numerator"], BC.SPEC, 4, offdiag=2, upper=False)
    out["wrong_external_index_position"] = {"changes_result": True, "rejected": not f5["exact"]}
    f6 = C.covariant_fit(ev["body"]["numerator"], BC.SPEC, 4, offdiag=1, upper=True)
    out["wrong_offdiagonal_weight"] = {"changes_result": True, "rejected": not f6["exact"]}
    for v in out.values():
        v["accepted_as_rejection"] = bool(v["rejected"] and v["changes_result"])
    return out


def d11_record(ev, m2rec):
    A = ev["A"]
    lt = BC.laurent_table(A)
    basis = BC.hh_basis()
    pole4 = {k: lt[k]["pole"] for k in BC.four_d_keys(A)}
    z, zok = BC.pole_in_hh_basis(pole4, basis)
    ev_nonzero = any(lt[k]["pole"] != 0 for k in A if C.is_evanescent(k[0]))
    muts = d11_mutations(ev)
    degs = {C.label(*k): sum(1 for t in C.label(*k).split() if t.startswith("p_")) + 2 * k[1] for k in A}
    ctrl = {"exactly_degree_four_every_structure": all(v == 4 for v in degs.values()),
            "N4_no_order_0_or_2_contact_arises": all(v == 4 for v in degs.values()),"export_sha_pinned": ev["p
  +> in"]["pinned"], "covariant_form_exact": ev["fit"]["exact"],
            "covariant_form_unique": ev["fit"]["unique"], "orbit_consistent": ev["orbit_consistent"],
            "route_FP_equals_route_PV_exact_in_d": ev["routes_agree"],
            "integrand_equals_XVI_O_convolution_M1": m2rec["controls"]["route_A_M1_integrand_identity"],
            "exchange_symmetric_mn_rs": BC.exchange_symmetric(A),
            "nonlocal_equals_minus_pole": all(sp.simplify(lt[k]["nonlocal_L_coefficient"] + lt[k]["pole"]) ==
  +> 0
                                              for k in A),
            "pole_4D_in_p14r_hh_span": bool(zok),
            "R5_d_retained": any(v.has(d) for v in A.values()),
            "M2_discharged": m2rec["D11_matching_verified"],
            "mutations_all_rejected": all(v["accepted_as_rejection"] for v in muts.values())}
    ok = all(ctrl.values())
    table = {C.label(*k): {"evanescent": C.is_evanescent(k[0]), "A_exact_d_times_MB": BC.s(lt[k]["exact_d"]),
                           "UV_pole": BC.s(lt[k]["pole"]), "MSbar_finite": BC.s(lt[k]["finite"]),
                           "nonlocal_coefficient_of_L": BC.s(lt[k]["nonlocal_L_coefficient"]),
                           "local_polynomial": BC.s(lt[k]["local"]),
                           "finite_if_d_set_to_4_first": BC.s(lt[k]["finite_if_d_set_to_4_first"]),
                           "local_shift_from_d_dependence": BC.s(lt[k]["d_dependence_shift_of_local_part"])}
             for k in sorted(A, key=lambda k: C.label(*k))}
    return {"graph": "D11", "export": ev["pin"], "prefactor_export": str(ev["pref"]),
            "normalization": f"Ghat = {BC.norm_of(ev['pref'])} * int N/(D1 D2) / N_eps",
            "prescription": ("operational DRED (R1-R6), d opaque, MS-bar, Feynman +i0, N4 contact exclusions,
  +> M3 bounds "
                             "4/8/4, exactly-degree-four normalization (massless: every local term is a homoge
  +> neous "
                             "quartic in p)"),
            "covariant_form": {k: v for k, v in ev["fit"].items() if k != "terms"},
            "prescription_usage": {
                "exactly_degree_four_normalization": ("every structure S_s (p.p)^e of Ghat has total momentum
```

```
  +> degree 4 "
                                                        "(computed per structure: " + ", ".join(f"{k}: {v}" for
  +> k, v in
                                                                                                sorted(degs.items
  +> ())) + ")"),
               "N4_contact_exclusions": ("used as stated: the order-0/2 contacts (mu^4, mu^2 p^2) excluded by
  +>  N4 do not "
                                         "arise (massless; every pole and local MS-bar term is a homogeneous
  +> quartic "
                                         "in p), so N4 removes nothing here and no order-0/2 counterterm is n
  +> eeded"),
               "M3_bounds_4_8_4": "degree of divergence 4; local terms of derivative order exactly 4 <= 4"},
            "result": table, "evaluated": ok,
            "status": "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)" if ok and ev_non
  +> zero
            else ("renormalized" if ok else "unevaluated"),
            "UV_pole_in_p14r_hh_basis": [BC.s(x) for x in z] if z else None,
            "evanescent_poles": {C.label(*k): str(lt[k]["pole"]) for k in A if C.is_evanescent(k[0])},
            "evanescent_pole_nonzero": ev_nonzero,
            "longitudinal_projection_p_m_G": {k: str(v) for k, v in sorted(BC.longitudinal_projection(A).items
  +> ())},
            "controls": ctrl, "mutations": muts}


# ------------------------------------------------------------------ the full hh slot
def slot_record(evs):
    op, opfit, _ = H.kc_operator()
    A = {g: evs[g]["A"] for g in QC.HH_SLOT_GRAPHS}
    gt = {g: H.g_terms(A[g]) for g in A}
    ev_only = {g: {k: v for k, v in A[g].items() if C.is_evanescent(k[0])} for g in A}
    allg = gt["D01"] + gt["D03"] + gt["D11"]
    slots = {g: {n: H.slot_value(op, gt[g], e) for n, e in EXT.items()} for g in A}
    slot_sum = {n: H.slot_value(op, allg, e) for n, e in EXT.items()}
    ev_slot_sum = {n: H.slot_value(op, sum((H.g_terms(ev_only[g]) for g in A), []), e) for n, e in EXT.items()
  +> }
    ev_slot_d11 = {n: H.slot_value(op, H.g_terms(ev_only["D11"]), e) for n, e in EXT.items()}
    pdG = {"D11": H.p_dot_G(gt["D11"]), "D01+D03+D11": H.p_dot_G(allg),
           "evanescent_D01+D03+D11": H.p_dot_G(sum((H.g_terms(ev_only[g]) for g in A), []))}
    tr_d11 = {n: H.slot_value(H.trace_operator(), gt["D11"], e) for n, e in EXT.items()}
    muts = {}
    for name, terms in (("M1_wrong_relative_sign_D03", gt["D01"] + H.g_terms(A["D03"], -1) + gt["D11"]),
                        ("M3_D03_evanescent_dropped_naive_4D", gt["D01"] + H.g_terms(A["D03"], drop_evanescent
  +> =True)
                         + gt["D11"])):
        vals = {n: H.slot_value(op, terms, e) for n, e in EXT.items()}
        nz = any(v != 0 for v in vals.values())
        muts[name] = {"rejected": nz, "accepted_as_rejection": nz}
    vals = {n: H.slot_value(H.trace_operator(), allg, e) for n, e in EXT.items()}
    muts["M4_trace_operator_in_place_of_s_h"] = {"rejected": any(v != 0 for v in vals.values()),
                                                 "accepted_as_rejection": any(v != 0 for v in vals.values())}
    basis = BC.hh_basis()
    pv = {g: H.pole_vec(A[g], BC.four_d_keys(A[g])) for g in A}
    tot = {}
    for g in A:
        for k, v in pv[g].items():
            tot[k] = tot.get(k, 0) + v
    tr = H.transverse_analysis({"D11": pv["D11"], "D01+D03": {k: pv["D01"].get(k, 0) + pv["D03"].get(k, 0)
                                                              for k in set(pv["D01"]) | set(pv["D03"])},
                                "D01+D03+D11": tot}, basis)
    ev_poles = {}
    for g in A:
        for k, v in ev_only[g].items():
            ev_poles[C.label(*k)] = ev_poles.get(C.label(*k), 0) + dred.substitute_d(
                v, 4, "d11: evanescent UV pole A(4) of the summed hh function (record only)")
    controls = {
        "operator_longitudinal": H.longitudinal_op(op),
        "p_dot_G_D11_zero_exact_in_d": not pdG["D11"],
        "p_dot_G_sum_zero_exact_in_d": not pdG["D01+D03+D11"],
        "p_dot_G_evanescent_sum_zero": not pdG["evanescent_D01+D03+D11"],
        "slot_sum_zero_all_7_assignments": all(v == 0 for v in slot_sum.values()),
        "evanescent_slot_sum_zero_all_7_assignments": all(v == 0 for v in ev_slot_sum.values()),
        "D01_D03_individually_nonzero_incl_c_hat_h_tilde": (any(v != 0 for v in slots["D01"].values())
                                                            and slots["D01"]["c hat, h tilde"] != 0
                                                            and slots["D03"]["c hat, h tilde"] != 0),
        "D11_reaches_slot_under_trace_operator": any(v != 0 for v in tr_d11.values()),
        "4D_pole_sum_D01_D03_D11_transverse": tr["graphs"]["D01+D03+D11"]["transverse"],
        "4D_pole_D11_transverse": tr["graphs"]["D11"]["transverse"],
        "mutations_all_rejected": all(v["accepted_as_rejection"] for v in muts.values()),
    }
    ok = all(controls.values())
    surviving = sorted(pdG["evanescent_D01+D03+D11"]) if pdG["evanescent_D01+D03+D11"] else []
```

```
    return {
        "row": QC.SLOT_ROW, "graphs": "D01 + D02 + D03 + D11 + D12 (D02 = D12 = 0, P15_batchA.json)",
        "verdict": ("DECOUPLED (proof, full row): p_m G = 0 exactly in d for D11 by itself and for D01 + D03 a
  +> s a "
                    "sum; no evanescent structure survives in the slot" if ok else
                    ("EVANESCENT STRUCTURE SURVIVES" if surviving else "NOT ESTABLISHED")),
        "decoupling_proven_full_row": ok, "surviving_evanescent_structures_in_slot": surviving,
        "D11_reopens_evanescent_question": bool(surviving),
        "how_D11_decouples": ("graph by graph: D11 (scalar loop, conserved matter stress tensor) is transverse
  +>  by itself, "
                              "4D and evanescent parts separately; its evanescent poles are nonzero but transv
  +> erse. "
                              "Hence D11 mutations of phase or symmetry factor are NOT detectable by the slot
  +> (they are "
                              "tested by M2 instead); the slot sees D11 only through a non-longitudinal operat
  +> or "
                              "(trace operator control: nonzero)"),
        "operator_s_h": opfit,
        "slot_values": {g: {n: s(v) for n, v in slots[g].items()} for g in A} |
                       {"D01+D03+D11": {n: s(v) for n, v in slot_sum.items()},
                        "evanescent_parts_D01+D03+D11": {n: s(v) for n, v in ev_slot_sum.items()},
                        "evanescent_part_D11": {n: s(v) for n, v in ev_slot_d11.items()},
                        "D11_under_trace_operator": {n: s(v) for n, v in tr_d11.items()}},
        "p_dot_G": {k: {kk: str(vv) for kk, vv in v.items()} for k, v in pdG.items()},
        "four_D_poles": tr,
        "evanescent_poles_D01_D03_D11_in_hh_green_function": {k: s(v) for k, v in sorted(ev_poles.items())},
        "consequence": ("no evanescent h.h counterterm is required by the ch-projected identity at one loop (D
  +> 11 "
                        "included), so NO contract revision is made for it.  The summed evanescent poles of th
  +> e hh "
                        "Green function are nonzero but transverse; renormalising the hh Green function itself
  +>  would "
                        "need evanescent h.h counterterms -- outside this projection and NOT claimed."),
        "controls": controls, "mutations": muts}


# ------------------------------------------------------------------ inventory
def inventory():
    """every graph of every SURVIVING ch row (ledgers/P13_ch_dependency_rows.json), with its evaluation certif
  +> icate."""
    led = json.loads((ROOT / "ledgers" / "P13_ch_dependency_rows.json").read_text())["rows"]
    out = {}
    for r in led:
        if not str(r.get("status", "")).startswith("surviving"):
            continue
        for gid in r.get("graphs", []):
            g = gid.split("_")[0]
            f = QC.INPUT_EVALUATION_P18.get(g, "MISSING")
            out[g] = {"row": r["id"], "graph_id": gid, "certificate": f, "present": (ROOT / f).is_file()}
    def flag(f, *keys):
        try:
            x = json.loads((ROOT / f).read_text())
            for k in keys:
                x = x[k]
            return x
        except (OSError, KeyError, TypeError, ValueError):
            return None
    stat = {g: v.get("status") for g, v in (flag("certificates/P15_batchA.json", "graphs") or {}).items()}
    for g in ("D02", "D12", "D14a", "D14b", "D14c"):
        out[g]["status"] = stat.get(g)
    for g in ("D01", "D03"):
        out[g]["status"] = flag("certificates/P16_batchC.json", "graphs", g, "status")
    out["D13"]["status"] = "evaluated (P15_D13.json)"
    out["D01"]["evaluated"] = flag("certificates/P16_batchC.json", "graphs", "D01", "evaluated") is True
    out["D03"]["evaluated"] = flag("certificates/P16_batchC.json", "graphs", "D03", "evaluated") is True
    out["D13"]["evaluated"] = flag("certificates/P15_D13.json", "D13_evaluated", "execution") is True
    claim = flag("certificates/P15_batchA.json", "batchA_evaluated", "claim") or ""
    for g in ("D02", "D12", "D14a", "D14b", "D14c"):
        out[g]["evaluated"] = (flag("certificates/P15_batchA.json", "batchA_evaluated", "execution") is True
                               and g in claim)
    return out


def main():
    import certio
    t0 = time.time()
    evs = {}
    for g in QC.HH_SLOT_GRAPHS:
        log(f"{g}: covariant vertex (batchC.evaluate_graph)")
        evs[g] = BC.evaluate_graph(g)
    m2rec = m2(evs)
```

```
    log(f"M2: {m2rec['verdict']}")
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_D11_M2.json",
                 {"landmark": "XVI-P", "revision": QC.REVISION, "deliverable": "D11 matching premise M2", **m2
  +> rec})
    if not m2rec["D11_matching_verified"]:
        log("M2 NOT discharged: D11 evaluation STOPPED (D11 stays NOT evaluated)")
        return 1
    log("D11 evaluation record")
    rec = d11_record(evs["D11"], m2rec)
    log("full hh slot")
    sl = slot_record(evs)
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_D11.json",
                 {"landmark": "XVI-P", "revision": QC.REVISION, "deliverable": "D11 evaluation under the decla
  +> red "
                  "prescription", **rec, "D11_evaluated": rec["evaluated"],
                  "D11_matching_verified": m2rec["D11_matching_verified"],
                  "authorised_d_substitutions": dict(sorted(dred.D_PROVENANCE.items()))})
    # inventory AFTER P18_D11.json is written (fresh-build ordering fix: 'present' must see this run's cert)
    inv = inventory()
    inv["D11"]["evaluated"] = rec["evaluated"]
    inv["D11"]["status"] = rec["status"]
    no_missing = all(v["present"] and v.get("status") and "missing" not in str(v["status"]).lower()
                     and "unevaluated" not in str(v["status"]).lower() for v in inv.values())
    q_ok = (all(v.get("evaluated") for v in inv.values()) and set(inv) == set(QC.INVENTORY_P18) and no_missing
            and len({v["row"] for v in inv.values()}) == 3)
    certio.write(ROOT / "certificates" / f"{QC.CERT_PREFIX}_hh_slot.json",
                 {"landmark": "XVI-P", "revision": QC.REVISION,
                  "deliverable": "full hh slot (D01+D02+D03+D11+D12) vs the ch projection", **sl,
                  "inventory": inv, "quantum_inputs_evaluated": "earned" if q_ok else "not_earned",
                  "quantum_inputs_scope": ("every graph of the P13 inventory (three ch slots: hh, K.c, one-poi
  +> nt) is "
                                           "evaluated under the one frozen prescription; this is NOT "
                                           "projected_identity_verified (the projected identity is not assembl
  +> ed)"),
                 "seconds": round(time.time() - t0, 1)})
    print(json.dumps({"M2": m2rec["verdict"], "M2_controls": m2rec["controls"], "D11_controls": rec["controls"
  +> ],
                      "slot": sl["verdict"], "slot_controls": sl["controls"], "q": q_ok,
                      "coords": {k: v["coordinates_p14r_basis"] for k, v in sl["four_D_poles"]["graphs"].items
  +> ()}},
                     indent=1))
    return 0 if (rec["evaluated"] and sl["decoupling_proven_full_row"]) else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

145 Source: release_p18.py (p18 release gate)

[listing 112/133] code/p5_calc/release_p18.py

```
#!/usr/bin/env python3
"""XVI-P p18 release gate: P.5 calculation, fourth step -- the D11 matching premise M2 (discharge), D11 evalua
  +> ted
under the declared prescription, the full hh slot D01 + D02 + D03 + D11 + D12, and the p17 reviewer findings H
  +> 1-H5,
on the p14r contract with the disclosed p15 (F1), p16 (F3), p17 (G1-G3) and p18 (H3/H5, documentation only) dr
  +> ed.py
revisions.

Runs AFTER the p14r, p15, p16 and p17 gates (release_p17.py -> certificates/P17_release.json) and requires all
  +>  four
to be accepted.  Recomputes:
  contract identity : p14r contract files byte-identical EXCEPT code/p5_entry/dred.py; dred.py differs from th
  +> e p17
                      release only in comments/docstrings (AST without docstrings pinned: QC.P17_DRED_AST_SHA2
  +> 56); NO
                      new contract file (no evanescent h.h counterterm: the full-row decoupling is proven);
  p17 layer        : every p17 layer file byte-identical to the p17 zip (QC.P17_CALC_SHA256); P17_release acce
  +> pted;
  M2               : P18_D11_M2.json -- all controls true, all mutations rejected, verdict DISCHARGED;
  D11              : P18_D11.json -- evaluated, all controls true, mutations rejected;
  hh slot          : P18_hh_slot.json -- full-row decoupling, no surviving evanescent structure;
  earned flags     : D11_matching_verified, D11_evaluated, quantum_inputs_evaluated -- each ONLY from the cert
  +> ificates
                      (the inventory of the three ch slots is re-read here from the P15/P16/P18 certificates);
  hard stops       : QC.HARD_STOPS_P18 (projected identity/restoration NOT earned; symmetry flags false; ...).
Rebuild flags: author/independent for the CURRENT (p18) tree; independent_rebuild_passed_p17 from the archived
```

```
  +>  p17
independent report, bound to the p17 tree 2a5a6317... / zip 7f4f61df... (same-box caveat).
Change log: every source file added/modified/removed relative to the p17 source-only zip, INCLUDING the p18
reference-values file (listed as added by declaration: it is written after the author build).
"""
from __future__ import annotations
import ast, hashlib, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p18_conventions as QC  # noqa: E402
import p15_conventions as Q15  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)

ARCH_P17 = "archival/p17"
ARCH_P17_INDEP = f"{ARCH_P17}/reproducibility/independent_rebuild_report_p17.json"
M2_MUTATIONS = ("alpha_beta_swapped", "offdiagonal_weight_1", "FT_log_constant_sign_flipped", "symmetry_factor
  +> _doubled",
                "D01_in_place_of_D11", "D0_dropped_beta_0")
SLOT_MUTATIONS = ("M1_wrong_relative_sign_D03", "M3_D03_evanescent_dropped_naive_4D", "M4_trace_operator_in_pl
  +> ace_of_s_h")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def ast_sha(path):
    """sha256 of the module AST with every docstring removed (comment/docstring-only revisions keep it)."""
    tree = ast.parse(Path(path).read_text())
    for n in ast.walk(tree):
        if isinstance(n, (ast.Module, ast.ClassDef, ast.FunctionDef, ast.AsyncFunctionDef)) and n.body:
            b0 = n.body[0]
            if isinstance(b0, ast.Expr) and isinstance(b0.value, ast.Constant) and isinstance(b0.value.value,
  +> str):
                n.body = n.body[1:] or [ast.Pass()]
    return hashlib.sha256(ast.dump(tree).encode()).hexdigest()


def contract_identity():
    cur = {p.relative_to(ROOT).as_posix(): sha(p) for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    changed = sorted(k for k, h in Q15.P14R_CONTRACT_SHA256.items() if cur.get(k) != h)
    added = sorted(set(cur) - set(Q15.P14R_CONTRACT_SHA256))
    dred = ROOT / "code" / "p5_entry" / "dred.py"
    dred_changed_vs_p17 = cur.get("code/p5_entry/dred.py") != QC.P17_DRED_SHA256
    dred_ast_equal_p17 = ast_sha(dred) == QC.P17_DRED_AST_SHA256
    # p22 disclosed revision: the registered additive p22 contract revision (contract_registry: dred.py = the
  +> p18 file
    # + one delimited guard block, remainder byte-identical to p18) is accepted in place of the AST identity
    if not dred_ast_equal_p17:
        import contract_registry as CR
        dred_ast_equal_p17 = CR.dred_revision(ROOT)["state"] == "p18 + p22 guard block"
    ok = bool(changed == sorted(QC.P14R_CONTRACT_FILES_CHANGED_IN_P18) and added == sorted(QC.NEW_CONTRACT_FIL
  +> ES_P18)
            and dred_changed_vs_p17 and dred_ast_equal_p17)
    return ok, {"p14r_contract_files": len(Q15.P14R_CONTRACT_SHA256), "changed_vs_p14r": changed, "added": add
  +> ed,
                "dred_changed_vs_p17": dred_changed_vs_p17,
                "dred_executable_code_identical_to_p17_ast_without_docstrings": dred_ast_equal_p17,
                "disclosed_revisions_p18": QC.DISCLOSED_CONTRACT_REVISIONS_P18,
                "evanescent_hh_counterterm_added": False,
                "why_no_evanescent_hh_counterterm": ("full row decoupled: p_m G_D11 = 0 exactly in d and p_m (
  +> G_D01 + "
                                                     "G_D03) = 0 exactly in d (certificates/P18_hh_slot.json)"
  +> )}


def p17_layer():
    cur = {k: sha(ROOT / k) if (ROOT / k).is_file() else None for k in QC.P17_CALC_SHA256}
    ident = all(cur[k] == h for k, h in QC.P17_CALC_SHA256.items())
    r = jl("certificates/P17_release.json")
```

```
    ok = bool(ident and r.get("publication") == "accepted" and r.get("release_claim_accepted") is True
              and r.get("hh_evanescent_decoupling_proven") is True and r.get("hard_stops_hold") is True)
    return ok, {"p17_layer_files_byte_identical": ident, "P17_release_accepted": r.get("publication") == "acce
  +> pted",
                "hh_evanescent_decoupling_proven_p17": r.get("hh_evanescent_decoupling_proven")}, r


def p17_independent():
    f = ROOT / ARCH_P17_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P17_INDEP, "present": False, "pins_ok": False}
    rep = json.loads(f.read_text())
    live = ROOT / QC.P17_INDEPENDENT_REPORT
    rec = {"report": ARCH_P17_INDEP, "present": True, "report_sha256": sha(f),
           "report_sha256_pinned": sha(f) == QC.P17_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "bound_to_source_tree_sha256": rep.get("source_tree_sha256"),
           "bound_to_source_zip_sha256": rep.get("zip_sha256"),
           "tree_matches_p17": rep.get("source_tree_sha256") == QC.P17_SOURCE_TREE_SHA256,
           "zip_matches_p17": rep.get("zip_sha256") == QC.P17_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])), "findings": [x.get("id") for x in rep.get("findings",
  +>  [])],
           "caveat": rep.get("same_box_caveat"),
           "live_report_identical_to_archived": (live.read_bytes() == f.read_bytes()) if live.is_file()
           else "live report not in this tree",
           "NOT_inherited_by_p18": "bound to the p17 tree/zip; the p18 tree has its own (pending) independent
  +> flag"}
    rec["archived_p17_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P17_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p17_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p17"] and rec["zip_matches_p17"] and rec["mismatches"] == 0
              and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec


def m2_check():
    c = jl(f"certificates/{QC.CERT_PREFIX}_D11_M2.json")
    muts = c.get("mutations", {})
    ok = bool(c.get("revision") == QC.REVISION and c.get("D11_matching_verified") is True
              and c.get("D11_matching_verified_scope") == "4D components"
              and c.get("evanescent_components_matching", {}).get("status") == "not_applicable_not_verified"
              and str(c.get("M3_status", {}).get("status", "")).startswith("unconditional on the 4D components
  +> ")
              and str(c.get("verdict", "")).startswith("DISCHARGED")
              and len(c.get("controls", {})) >= 8 and all(v is True for v in c.get("controls", {}).values())
              and sorted(muts) == sorted(M2_MUTATIONS) and all(v.get("accepted_as_rejection") is True for v in
  +>  muts.values())
              and c.get("route_B", {}).get("mismatches") == 0 and c.get("route_B", {}).get("pairs") == 100)
    return ok, c


def d11_check(m2_ok):
    c = jl(f"certificates/{QC.CERT_PREFIX}_D11.json")
    ok = bool(m2_ok and c.get("revision") == QC.REVISION and c.get("evaluated") is True and c.get("D11_evaluat
  +> ed") is True
              and len(c.get("controls", {})) >= 14 and all(v is True for v in c.get("controls", {}).values())
              and all(v.get("accepted_as_rejection") is True for v in c.get("mutations", {}).values())
              and len(c.get("mutations", {})) >= 4)
    return ok, c


def slot_check():
    c = jl(f"certificates/{QC.CERT_PREFIX}_hh_slot.json")
    muts = c.get("mutations", {})
    ok = bool(c.get("revision") == QC.REVISION and c.get("decoupling_proven_full_row") is True
              and c.get("surviving_evanescent_structures_in_slot") == [] and c.get("D11_reopens_evanescent_que
  +> stion") is False
              and len(c.get("controls", {})) >= 11 and all(v is True for v in c.get("controls", {}).values())
              and sorted(muts) == sorted(SLOT_MUTATIONS) and all(v.get("accepted_as_rejection") is True for v
  +> in muts.values())
              and len(c.get("slot_values", {}).get("D01+D03+D11", {})) == 7)
    return ok, c


def inventory_recheck(d11_ok):
    """independent of d11.py: the required inputs are EVERY graph of every surviving ch row of the frozen ledg
  +> er
    (three slots: hh, K.c, one-point); each flag and status is re-read from its own certificate; no Missing.""
  +> "
    rows = [r for r in jl("ledgers/P13_ch_dependency_rows.json").get("rows", [])
```

```
            if str(r.get("status", ""
  +> )).startswith("surviving")]
    required = {gid.split("_")[0]: r["id"] for r in rows for gid in r.get("graphs", [])}
    b = jl("certificates/P15_batchA.json").get("batchA_evaluated", {})
    claim = b.get("claim", "") if isinstance(b, dict) else ""
    c16 = jl("certificates/P16_batchC.json").get("graphs", {})
    d13 = jl("certificates/P15_D13.json").get("D13_evaluated", {})
    inv = {g: bool(isinstance(b, dict) and b.get("execution") is True and g in claim)
           for g in ("D02", "D12", "D14a", "D14b", "D14c")}
    inv["D01"] = c16.get("D01", {}).get("evaluated") is True
    inv["D03"] = c16.get("D03", {}).get("evaluated") is True
    inv["D13"] = isinstance(d13, dict) and d13.get("execution") is True
    inv["D11"] = bool(d11_ok)
    sa = {g: v.get("status") for g, v in jl("certificates/P15_batchA.json").get("graphs", {}).items()}
    status = {**sa, "D01": c16.get("D01", {}).get("status"), "D03": c16.get("D03", {}).get("status"),
              "D13": "evaluated" if inv["D13"] else None,
              "D11": jl(f"certificates/{QC.CERT_PREFIX}_D11.json").get("status") if d11_ok else None}
    lst = [{"graph": g, "slot_row": required.get(g), "evaluated": inv.get(g, False), "status": status.get(g),
            "certificate": QC.INPUT_EVALUATION_P18.get(g)} for g in sorted(set(required) | set(inv))]
    no_missing = all(x["status"] and "missing" not in str(x["status"]).lower()
                     and "unevaluated" not in str(x["status"]).lower() for x in lst)
    ok = bool(set(required) == set(inv) == set(QC.INVENTORY_P18) and len(set(required.values())) == 3
              and all(inv.values()) and no_missing)
    return ok, lst


def change_log():
    f = ROOT / QC.P17_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P17_ARCHIVED_RECORDS[QC.P17_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT) if r != SM.REFERENCE_FILE}
    contract = set(Q15.P14R_CONTRACT_SHA256) | {p.relative_to(ROOT).as_posix() for p in (ROOT / "code" / "p5_e
  +> ntry").glob("*.py")}
    added = sorted((set(new) - set(old)) | {SM.REFERENCE_FILE})
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])
    changed = sorted(added + removed + modified)
    return {"base": f"{QC.P17_SOURCE_LIST} (pinned; files of the p17 zip sha256 {QC.P17_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok,
            "reference_file": f"{SM.REFERENCE_FILE} listed as added by declaration (written after the author b
  +> uild)",
            "added": added, "removed": removed, "modified": modified,
            "contract_files_changed": [x for x in changed if x in contract],
            "non_contract_files_changed": [x for x in changed if x not in contract],
            "counts": {"added": len(added), "removed": len(removed), "modified": len(modified)}}


def milestones(p17r, ok):
    st = dict(p17r.get("milestones", {}).get("status", {}))
    st["P.5_calculation"] = ("quantum inputs evaluated (p15: Batch A D02/D12/D14a-c, Batch B D13; p16: E2 deco
  +> upled, "
                             "Batch C D01/D03; p17: D01/D03 evanescent poles decoupled as the sum; p18: M2 dis
  +> charged, "
                             "D11 evaluated, full hh row decoupled); the projected identity is NOT assembled"
                             if ok else "open (p18 D11 / full-slot analysis not accepted)")
    st["P.5_identity"] = "open"
    return {"definitions": p17r.get("milestones", {}).get("definitions"), "order": p17r.get("milestones", {}).
  +> get("order"),
            "status": st, "P5_entry_earned": p17r.get("P5_entry_earned"),
            "P3_P4_frozen_as_P5_inputs": p17r.get("P3_P4_frozen_as_P5_inputs"),
            "P4_is_NOT": "coupled one-loop restoration (projected_identity_verified / projected_restoration_ve
  +> rified not earned)"}


def tex(x):
    t = str(x).replace("**", "^").replace("\\", "/").replace("_", "\\_").replace("^", "\\^{}").replace("&", "\
  +> \&")
    return "\\texttt{" + t + "}"


def results_table(m2c, d11c, sc):
    rows = [f"{tex(k)} & {'yes' if v['evanescent'] else 'no'} & {tex(v['UV_pole'])} & {tex(v['nonlocal_coeffic
  +> ient_of_L'])} "
            f"& {tex(v['local_polynomial'])} & {tex(v['local_shift_from_d_dependence'])} \\\\"
            for k, v in d11c["result"].items()]
    fd = sc["four_D_poles"]
    crow = [f"{g} & " + " & ".join(tex(x) for x in fd["graphs"][g]["coordinates_p14r_basis"])
            + f" & {'yes' if fd['graphs'][g]['transverse'] else 'no'} \\\\" for g in ("D11", "D01+D03", "D01+D
  +> 03+D11")]
    srow = [f"{tex(n)} & {tex(sc['slot_values']['D01'][n])} & {tex(sc['slot_values']['D03'][n])} & "
            f"{tex(sc['slot_values']['D11'][n])} & {tex(sc['slot_values']['D01+D03+D11'][n])} \\\\"
            for n in sc["slot_values"]["D01+D03+D11"]]
```

```
    rb = m2c["route_B"]
    return ("% generated by code/p5_calc/release_p18.py -- do not edit\n"
            f"\\noindent M2: {tex(m2c['verdict'])}; route B: {rb['pairs']} component pairs, {rb['mismatches']}
  +>  "
            f"mismatches, constant {tex(rb['constant'])}; c\\_FT = {tex(m2c['FT_constant']['c_FT_from_DR_bubbl
  +> e'])} "
            f"(DR bubble) = {tex(m2c['FT_constant']['c_FT_from_minus_r'])} (-r).\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{3.0cm}p{0.6cm}p{1.5cm}p{1.5cm}p{3.6cm}p{2.4cm}}\\hl
  +> ine\n"
            "D11 structure & evan. & UV pole & coeff. of L & local (MS-bar) & shift (d=4 first) \\\\\\hline\n"
            + "\n".join(rows) + "\n\\hline\\end{tabular}\\end{center}\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{2.0cm}p{1.3cm}p{1.3cm}p{1.3cm}p{1.3cm}p{1.3cm}p{1.2
  +> cm}}\\hline\n"
            "4D pole & $z_1$ & $z_2$ & $z_3$ & $z_4$ & $z_5$ & transverse \\\\\\hline\n"
            + "\n".join(crow) + "\n\\hline\\end{tabular}\\end{center}\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{2.6cm}p{2.6cm}p{2.6cm}p{1.2cm}p{1.6cm}}\\hline\n"
            "assignment $(c,h)$ & $X_{D01}$ & $X_{D03}$ & $X_{D11}$ & sum \\\\\\hline\n"
            + "\n".join(srow) + "\n\\hline\\end{tabular}\\end{center}\n"
            f"\\noindent Full-row verdict: {tex(sc['verdict'])}.\n")


def _main():
    id_ok, ident = contract_identity()
    l17_ok, l17, p17r = p17_layer()
    p17i_ok, p17i = p17_independent()
    m2_ok, m2c = m2_check()
    d11_ok, d11c = d11_check(m2_ok)
    s_ok, sc = slot_check()
    inv_ok, inv = inventory_recheck(d11_ok)
    q_earned = bool(inv_ok and s_ok and sc.get("quantum_inputs_evaluated") == "earned")
    p14r, p15r, p16r = (jl(f"certificates/{n}_release.json") for n in ("P14", "P15", "P16"))
    hard = dict(QC.HARD_STOPS_P18)
    hard_ok = bool(all(hard[k] == v for k, v in QC.HARD_STOPS_P18.items())
                   and all(r.get("hard_stops_hold") is True for r in (p14r, p15r, p16r, p17r))
                   and not any(str(sc.get(k, "not_earned")) == "earned" for k in
                               ("projected_identity_verified", "projected_restoration_verified"))
                   and sc.get("bulk_symmetry_restored", False) is False)
    graphs = dict(p17r.get("graphs", {}))
    if d11_ok:
        graphs["D11"] = {"status": "evaluated", "certificate": f"certificates/{QC.CERT_PREFIX}_D11.json",
                         "sector_status": d11c.get("status"), "M2": m2c.get("verdict"),
                         "ch_slot": "decoupled by itself (p_m G_D11 = 0 exactly in d)"}
    else:
        graphs["D11"] = {"status": "unevaluated", "reason": "M2 not discharged or D11 certificate not accepted
  +> "}
    cl = change_log()
    execution_pass = bool(id_ok and l17_ok and m2_ok and d11_ok and s_ok and hard_ok and cl["base_pinned_ok"]
                          and p17i.get("pins_ok") is True)
    publish = execution_pass
    author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
    indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
    m = milestones(p17r, execution_pass)
    flags = dict(p17r.get("flags", {}))
    flags["D11_matching_M2"] = {"execution": m2_ok, "claim": m2c.get("verdict") if m2_ok else "not discharged"
  +> ,
                                "scope": m2c.get("scope")}
    flags["hh_slot_full_row_decoupled"] = {"execution": s_ok, "claim": sc.get("verdict") if s_ok else "not est
  +> ablished",
                                           "scope": ("row (S0,Gamma1^[2]) s2 K:K.c; D01 + D02 + D03 + D11 + D1
  +> 2; the "
                                                     "summed evanescent poles remain (transverse) UV poles of
  +> the hh "
                                                     "Green function outside this projection")}
    cert = {
        "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
  +> TE,
        "version_label": QC.VERSION_LABEL,
        "deliverable": "P.5 calculation, fourth step (M2 discharge, D11 evaluation, full hh slot; H1-H5)",
        "execution_pass": execution_pass,
        "claim_status": ("P.5_calculation: every graph of the inventory (D01, D02, D03, D11, D12, D13, D14a-c)
  +>  evaluated "
                         "under the one declared prescription; M2 discharged (4D components); the hh row decou
  +> pled from "
                         "the ch-projected slot (D11 by itself, D01 + D03 as the sum); P.5_identity open")
        if execution_pass else "P.5_calculation: not accepted",
        "scope": ("quantum inputs = the evaluated one-loop graphs of the three ch slots (hh, K.c, one-point) u
  +> nder DRED "
                  "R1-R6 / Feynman +i0 / MS-bar / S0, d opaque. NOT the projected defect/identity, NOT project
  +> ed "
                  "restoration, NOT bulk restoration, NOT the insertion. Evanescent poles of the hh (D01, D03,
  +>  D11) and "
```

```
              "K.c (D13 E2) Green functions are retained and nonzero; they decouple from the ch projection
+> , and "
              "renormalising those Green functions themselves would need evanescent counterterms (not clai
+> med)."),
      "release_claim_accepted": execution_pass,
      "release_claim_accepted_definition": ("p14r contract accepted AND p15, p16, p17 layers accepted and "
                                            "byte-identical AND contract identity (only dred.py changed vs p
+> 14r; vs "
                                            "p17 comments/docstrings only; no new contract file) AND M2 disc
+> harged "
                                            "AND D11 evaluated AND the full-row slot decoupling AND p18 hard
+>  stops"),
      "publication": "accepted" if publish else "REJECTED",
      "flags": flags,
      "D11_matching_verified": bool(m2_ok and publish),
      "D11_matching_verified_scope": m2c.get("D11_matching_verified_scope") if m2_ok else None,
      "D11_evanescent_components_matching": m2c.get("evanescent_components_matching", {}).get("status"),
      "M3_status": m2c.get("M3_status", {}).get("status") if m2_ok else "conditional on M2 (not discharged)"
+> ,
      "D11_evaluated": bool(d11_ok and publish),
      "quantum_inputs_evaluated": "earned" if (q_earned and publish) else "not_earned",
      "quantum_inputs_inventory": inv,
      "projected_defect_computed": "not_earned", "projected_identity_verified": "not_earned",
      "projected_restoration_verified": "not_earned",
      "batchA_evaluated": p17r.get("batchA_evaluated"), "D13_evaluated": p17r.get("D13_evaluated"),
      "batchC_evaluated": p17r.get("batchC_evaluated"), "library_certified": p17r.get("library_certified"),
      "E2_status": p17r.get("E2_status"), "E2_decoupling_proven": p17r.get("E2_decoupling_proven"),
      "hh_evanescent_status": p17r.get("hh_evanescent_status"),
      "hh_evanescent_decoupling_proven": p17r.get("hh_evanescent_decoupling_proven"),
      "hh_full_row_decoupling_proven": s_ok, "evanescent_hh_counterterm_added": False,
      "D01_evaluated": p17r.get("D01_evaluated"), "D03_evaluated": p17r.get("D03_evaluated"),
      "graphs": graphs, "D12_status": p17r.get("D12_status"), "D12_status_derivation": p17r.get("D12_status_
+> derivation"),
      "M2_summary": {k: m2c.get(k) for k in ("verdict", "D11_matching_verified_scope",
                                             "evanescent_components_matching", "M3_status", "statement", "r
+> oute_B", "FT_constant", "controls",
                                             "analytic_inputs_cited", "scope", "M3_consequence")},
      "D11_summary": {k: d11c.get(k) for k in ("status", "prescription", "prescription_usage", "UV_pole_in_p
+> 14r_hh_basis",
                                               "evanescent_pole_nonzero", "controls")},
      "hh_slot_summary": {k: sc.get(k) for k in ("verdict", "row", "graphs", "how_D11_decouples",
                                                 "surviving_evanescent_structures_in_slot", "consequence", "
+> controls")},
      "phase_convention_G6": QC.PHASE_CONVENTION_G6,
      "contract_identity": ident, "contract_revision_disclosed": True,
      "disclosed_test_revisions_p18": QC.DISCLOSED_TEST_REVISIONS_P18,
      "p17_layer": l17, "change_log_vs_p17": cl,
      "P5_entry_earned": p17r.get("P5_entry_earned"), "P5_entry_flags": p17r.get("P5_entry_flags"),
      "P3_P4_frozen_as_P5_inputs": p17r.get("P3_P4_frozen_as_P5_inputs"), "classical": p17r.get("classical")
+> ,
      "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": m,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p17": p17i_ok,
      "independent_rebuild_passed_p16": p17r.get("independent_rebuild_passed_p16"),
      "independent_rebuild_passed_p15": p17r.get("independent_rebuild_passed_p15"),
      "independent_rebuild_passed_p14r": p17r.get("independent_rebuild_passed_p14r"),
      "independent_rebuild_passed_p13": p17r.get("independent_rebuild_passed_p13"),
      "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p18
+> ) source "
                             "tree only (independent: pending until a reviewer report for this tree exists);
+>  "
                             f"independent_rebuild_passed_p17 refers to the p17 tree {QC.P17_SOURCE_TREE_SHA
+> 256} / zip "
                             f"{QC.P17_SOURCE_ZIP_SHA256} (same-box caveat) and is not inherited by p18; "
                             "_p16 / _p15 / _p14r / _p13 as recorded by the p17 layer"),
      "p17_independent_rebuild": p17i,
      "p16_independent_rebuild": p17r.get("p16_independent_rebuild"),
      "p15_independent_rebuild": p17r.get("p15_independent_rebuild"),
      "p14r_independent_rebuild": p17r.get("p14r_independent_rebuild"),
      "p13_independent_rebuild": p17r.get("p13_independent_rebuild"),
      "one_point_slot_kappa1": p17r.get("one_point_slot_kappa1"), "D13_summary": p17r.get("D13_summary"),
      "E2_summary": p17r.get("E2_summary"), "batchC_summary": p17r.get("batchC_summary"),
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p17_archive": ARCH_P17,
                          "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"},
      "XVI_P_registered": False, "XVI_M_sha256": p17r.get("XVI_M_sha256"), "XVI_M_pin_ok": p17r.get("XVI_M_p
+> in_ok"),
      "symmetry_restored": False, "bulk_symmetry_restored": False, "O1_closure": "NOT_CLOSED",
      "insertion_status": "Frozen_negative_obstruction_preserved", "XV_B_opened": False, "XIV_J_opened": Fal
```

```
  +> se,
        "Route_B_discharged": False, "u_R_c_delta_added": False,
        "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
            n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
            ("P14_release.json", "P15_release.json", "P16_release.json", "P17_release.json",
             f"{QC.CERT_PREFIX}_D11_M2.json", f"{QC.CERT_PREFIX}_D11.json", f"{QC.CERT_PREFIX}_hh_slot.json")
            if (ROOT / "certificates" / n).is_file()},
    }
    cdir = ROOT / "certificates"
    txt = certio.dumps(cert)
    if publish:
        (ROOT / "RELEASE_STATUS.json").write_text(txt)
        (cdir / "RELEASE_STATUS.json").write_text(txt)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
        certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
            "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
  +> tatus)",
            **m, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
            "independent_rebuild_passed_p17": p17i_ok, "hard_stops": hard,
            "flags": {"D11_matching_verified": cert["D11_matching_verified"], "D11_evaluated": cert["D11_evalu
  +> ated"],
                      "quantum_inputs_evaluated": cert["quantum_inputs_evaluated"],
                      "hh_full_row_decoupling_proven": s_ok}})
        (ROOT / "manuscript" / "body").mkdir(parents=True, exist_ok=True)
        (ROOT / "manuscript" / "body" / "P18-results-table.tex").write_text(results_table(m2c, d11c, sc))
    else:
        (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(certio.dumps({
            "landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", "contract_identity_ok": i
  +> d_ok,
            "contract_identity": ident, "p17_layer_ok": l17_ok, "M2_ok": m2_ok, "D11_ok": d11_ok, "slot_ok": s
  +> _ok,
            "p17_independent_pins_ok": p17i.get("pins_ok"), "hard_stops_hold": hard_ok,
            "change_log_base_ok": cl["base_pinned_ok"]}))
    print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
  +> ion"],
                      "contract_identity": id_ok, "changed_vs_p14r": ident["changed_vs_p14r"], "added": ident[
  +> "added"],
                      "p17_layer": l17_ok, "M2": m2_ok, "D11": d11_ok, "slot": s_ok, "inventory": inv_ok,
                      "quantum_inputs_evaluated": cert["quantum_inputs_evaluated"], "hard_ok": hard_ok,
                      "author": author_ok, "independent_p18": indep_ok, "independent_p17": p17i_ok,
                      "change_log_counts": cl["counts"]}, indent=1))
    return 0 if publish else 1


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        cdir = ROOT / "certificates"
        (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
            {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED",
             "error": f"{type(exc).__name__}: {exc}"}, indent=1) + "\n")
        print(f"release_p18 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

146 Source: tests/test_p18.py

[listing 113/133] tests/test_p18.py

```
"""XVI-P p18 tests: M2 discharge (route B recomputed covariantly here, independently of d11.py), D11 evaluatio
  +> n, the
full hh slot (seven external assignments incl. c hat / h tilde, H2), the transverse coordinates, the inventory
  +>  behind
quantum_inputs_evaluated, H1 extended static scan (planted violations), H3/H5 dred documentation revision (AST
unchanged), p18 status / hard stops / p17 independent record, change log, p18 gate mutations."""
from __future__ import annotations
import ast
import hashlib
import json
import shutil
import subprocess
```

```
import sys
import tempfile
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools"):
    sys.path.insert(0, str(ROOT / p))
import contract_registry as CR  # p22 (disclosed): registered contract revision
import p18_conventions as QC  # noqa: E402
import p17_conventions as Q17  # noqa: E402
import p16_conventions as Q16  # noqa: E402
import p15_conventions as Q15  # noqa: E402
import dred as DR  # noqa: E402

CERT = ROOT / "certificates"
PKG = ("code/p5_calc", "code/p5_entry")
I = sp.I


def cj(name):
    return json.loads((CERT / name).read_text())


def rs():
    # p19: the p18 LAYER record (the top-level RELEASE_STATUS.json is the p19 status)
    return json.loads((CERT / "P18_release.json").read_text())


def P(text):
    return DR.parse_d(text.replace("p.p", "pp"), {"I": sp.I})


def Z(text):
    """certificate coordinate string (no d) -> number"""
    return DR.parse_d(text, {"I": sp.I})


# ============================================================== H1: extended static scan (pattern check, no
  +> t a proof)
FORBIDDEN_NAMES = {"eval", "exec", "parse_expr", "lambdify", "ufuncify", "autowrap", "all_coeffs"}
PROBE_FUNCTIONS = {"f3_hardening"}         # release_p16 / release_p17 guard probes (deliberate hazards, must
  +> raise)
PARSERS = {"sympify", "S", "parse_expr"}
SYMBOL_MAKERS = {"Symbol", "symbols", "Dummy", "var"}
HAZARD_ATTRS = {"subs", "xreplace", "replace", "evalf", "n", "eval", "func", "as_poly", "all_coeffs", "doit",
                "rewrite", "__class__", "sympify"}
OPAQUE_D_VALUES = ("d", "dred.d", "DR.d")


def _name(f):
    return f.attr if isinstance(f, ast.Attribute) else f.id if isinstance(f, ast.Name) else None


def _src(src, node):
    return ast.get_source_segment(src, node) or ""


def scan(path, root=ROOT):
    """AST pattern scan for value paths the runtime guard cannot hook (p17 patterns + H1).  Returns (violation
  +> s,
    parse_sites).  violations: builtin eval/exec, parse_expr, lambdify/ufuncify/autowrap, any `.eval(` call,
    `.replace(` with a non-string first argument, `.func(` rebuilds; H1: a call OF a call to Poly/as_poly
    (Poly(e, d)(4), e.as_poly(d)(4)), all_coeffs (Horner evaluation), type(x)(..) and x.__class__(..) rebuilds
  +> ,
    getattr with a non-literal or hazardous attribute name, aliased import/assignment of sympify/S/parse_expr,
    sympify/S locals whose 'd' value is not the opaque d, plain Symbol('d')/symbols('d')/Dummy('d') creation.
    parse_sites: every sympify / S call without a 'd' local (must be on the reviewed list)."""
    src = path.read_text()
    tree = ast.parse(src)
    try:
        rel = path.relative_to(root).as_posix()
    except ValueError:
        rel = path.name
    bad, sites = [], []

    def flag(node, what):
        bad.append(f"{rel}:{node.lineno}:{what}")
```

```
    def visit(node, fn=None):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            fn = node.name
        probe = fn in PROBE_FUNCTIONS
        if isinstance(node, ast.ImportFrom) and not probe:
            for a in node.names:
                if a.name in PARSERS and a.asname and a.asname != a.name:
                    flag(node, f"aliased import {a.name} as {a.asname}")
        if isinstance(node, ast.Assign) and not probe:
            v = node.value
            if _name(v) in PARSERS and isinstance(v, (ast.Attribute, ast.Name)):
                flag(node, f"alias assignment of {_name(v)}")
        if isinstance(node, ast.Call) and not probe:
            f = node.func
            name = _name(f)
            if name in FORBIDDEN_NAMES:
                flag(node, name)
            if name == "replace" and isinstance(f, ast.Attribute) and node.args:
                a = node.args[0]
                if not (isinstance(a, ast.Constant) and isinstance(a.value, str)):
                    flag(node, "replace(non-literal)")
            if name == "func" and isinstance(f, ast.Attribute):
                flag(node, ".func(rebuild)")
            if isinstance(f, ast.Call) and _name(f.func) in ("Poly", "as_poly", "poly", "type"):
                flag(node, f"{_name(f.func)}(..)(call)")
            if name == "__class__" and isinstance(f, ast.Attribute):
                flag(node, "__class__(rebuild)")
            if name == "getattr":
                at = node.args[1] if len(node.args) > 1 else None
                if not (isinstance(at, ast.Constant) and isinstance(at.value, str)) or at.value in HAZARD_ATTR
  +> S:
                    flag(node, "getattr(indirection)")
            if name in SYMBOL_MAKERS and node.args:
                a = node.args[0]
                if isinstance(a, ast.Constant) and isinstance(a.value, str) and "d" in a.value.replace(",", "
  +> ").split():
                    flag(node, f"plain {name}('d')")
            if name in ("sympify", "S"):
                loc = {k.arg: k.value for k in node.keywords}.get("locals")
                if isinstance(loc, ast.Dict):
                    for k, v in zip(loc.keys, loc.values):
                        if isinstance(k, ast.Constant) and k.value == "d" and _src(src, v) not in OPAQUE_D_VAL
  +> UES:
                            flag(node, "sympify locals 'd' not the opaque d")
        if isinstance(node, ast.Call):
            name = _name(node.func)
            if name in ("sympify", "S"):
                loc = {k.arg: k.value for k in node.keywords}.get("locals")
                has_d = isinstance(loc, ast.Dict) and any(isinstance(k, ast.Constant) and k.value == "d" for k
  +>  in loc.keys)
                if not has_d:
                    sites.append(f"{rel}::{fn}::{_src(src, node)}")
        for ch in ast.iter_child_nodes(node):
            visit(ch, fn)
    visit(tree)
    return bad, sites


def _pkg_files():
    return [f for d in PKG for f in sorted((ROOT / d).glob("*.py"))]


def test_h1_static_scan_package_clean_and_sites_reviewed():
    bad, sites = [], []
    for f in _pkg_files():
        b, s = scan(*CR.scan_view(f))   # p22: p18 contract core of the registered dred revision
        bad += b
        sites += s
    assert not bad, bad
    assert not sorted(set(sites) - set(QC.PARSE_SITES_REVIEWED))                # every site reviewed
    assert not sorted(set(QC.PARSE_SITES_REVIEWED) - set(sites))                # no stale entry


PLANTS = {
    "poly_call": "import sympy as sp\ndef f(e, d):\n    return sp.Poly(e, d)(4)\n",
    "as_poly_call": "def f(e, d):\n    return e.as_poly(d)(4)\n",
    "horner_all_coeffs": "import sympy as sp\ndef f(e, d):\n    c = sp.Poly(e, d).all_coeffs()\n    return c\n
  +> ",
    "type_rebuild": "def f(e):\n    return type(e)(*e.args)\n",
    "class_rebuild": "def f(e):\n    return e.__class__(*e.args)\n",
    "getattr_dynamic": "def f(e, m):\n    return getattr(e, m)\n",
```

```
    "getattr_hazard": "def f(e, d):\n    return getattr(e, 'subs')(d, 4)\n",
    "aliased_import": "from sympy import sympify as sy\ndef f(t):\n    return t\n",
    "alias_assign": "import sympy as sp\nsy = sp.sympify\n",
    "locals_plain_d": "import sympy as sp\ndef f(t):\n    return sp.sympify(t, locals={'d': sp.Symbol('x')})\n
  +> ",
    "plain_symbol_d": "import sympy as sp\nD = sp.Symbol('d')\n",
    "plain_symbols_d": "import sympy as sp\nA, D = sp.symbols('a d')\n",
    "eval_srepr": "def f(s):\n    return eval(s)\n",
    "func_rebuild": "def f(e):\n    return e.func(*e.args)\n",
    "replace_named": "def g(x):\n    return x\ndef f(e):\n    return e.replace(g, g)\n",
}


@pytest.mark.parametrize("name", sorted(PLANTS))
def test_h1_scan_detects_planted_violation(tmp_path, name):
    f = tmp_path / f"{name}.py"
    f.write_text(PLANTS[name])
    bad, _ = scan(f, tmp_path)
    assert bad, name


def test_h1_scan_allows_the_opaque_d_and_literal_getattr(tmp_path):
    f = tmp_path / "ok.py"
    f.write_text("import sympy as sp\nimport dred\nd = dred.d\ndef f(t, o):\n    x = getattr(o, 'active', Fals
  +> e)\n"
                 "    return sp.sympify(t, locals={'d': dred.d}), sp.sympify(t, locals={'d': d}), x\n")
    bad, sites = scan(f, tmp_path)
    assert not bad and not sites


def test_h1_disclosure_wording():
    txt = (ROOT / "code" / "p5_entry" / "dred.py").read_text()
    assert "pattern check, not a proof" in txt and "tests/test_p18.py" in txt


# ============================================================== H3 / H5: dred documentation revision
def test_h3_h5_dred_revision_is_documentation_only():
    import release_p18 as R
    assert (R.ast_sha(ROOT / "code" / "p5_entry" / "dred.py") == QC.P17_DRED_AST_SHA256
            or CR.dred_revision(ROOT)["state"] == "p18 + p22 guard block")     # p22: registered additive revi
  +> sion
    assert hashlib.sha256((ROOT / "code" / "p5_entry" / "dred.py").read_bytes()).hexdigest() != QC.P17_DRED_SH
  +> A256
    txt = (ROOT / "code" / "p5_entry" / "dred.py").read_text()
    assert "tests/test_p16.py scans the package code for them" not in txt                    # H3 stale commen
  +> t gone
    assert "srepr" in txt and "RAISES PlainDimensionSymbol" in txt                           # H5 documented
    hdr = (ROOT / "code" / "p5_calc" / "p18_conventions.py").read_text()
    assert "STORED STRINGS" in hdr                                                          # H3 header correc
  +> ted


def test_h5_parse_d_srepr_raises():
    e = DR.d ** 2 + 1
    with pytest.raises(DR.PlainDimensionSymbol):
        DR.parse_d(sp.srepr(e))
    assert sp.cancel(DR.parse_d(str(e)) - e) == 0


def test_contract_and_layers_identical():
    cur = {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest()
           for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    assert sorted(k for k, h in Q15.P14R_CONTRACT_SHA256.items() if cur.get(k) != h) == ["code/p5_entry/dred.p
  +> y"]
    assert set(cur) == set(Q15.P14R_CONTRACT_SHA256)
    for k, h in {**Q16.P15_CALC_SHA256, **Q17.P16_CALC_SHA256, **QC.P17_CALC_SHA256}.items():
        assert hashlib.sha256((ROOT / k).read_bytes()).hexdigest() == h, k


def test_p14_to_p17_certificate_values_unchanged_vs_p17_reference():
    import certio
    ref = json.loads((ROOT / "archival" / "p17" / "reference_values_p17.json").read_text())["values"]
    ch = ref["certificate_content_sha256_ignoring_timings"]
    diff = [n for n, h in ch.items()
            if n.startswith(("P13_", "P14_", "P15_", "P16_", "P17_"))
            and n not in ("P15_release", "P16_release", "P17_release")
            and certio.file_content_sha(CERT / f"{n}.json", QC.RUN_SPECIFIC_KEYS) != h]
    assert not diff, diff


# ============================================================== priority 1: M2
```

```
def test_m2_certificate_verdict_controls_mutations():
    c = cj("P18_D11_M2.json")
    assert c["revision"] == "p18" and c["D11_matching_verified"] is True and c["verdict"].startswith("DISCHARG
  +> ED")
    assert all(v is True for v in c["controls"].values()) and len(c["controls"]) == 8
    assert c["route_B"]["pairs"] == 100 and c["route_B"]["mismatches"] == 0 and c["route_B"]["nonzero_pairs"]
  +> > 0
    assert sp.simplify(Z(c["route_B"]["constant"]) + I / 4) == 0
    assert c["FT_constant"]["c_FT_from_DR_bubble"] == "I*pi**2" == c["FT_constant"]["c_FT_from_minus_r"]
    assert len(c["mutations"]) == 6 and all(v["accepted_as_rejection"] and v["mismatches"] > 0
                                            for v in c["mutations"].values())
    assert c["route_A"]["M1"]["pass"] is True and c["route_A"]["XVI_O"]["ok"] is True
    assert "4D" in c["scope"] and len(c["evanescent_L_coefficients"]) == 8
    assert c["D11_matching_verified_scope"] == "4D components"
    assert c["evanescent_components_matching"]["status"] == "not_applicable_not_verified"
    assert c["M3_status"]["status"].startswith("unconditional on the 4D components")


def test_m2_covariant_recheck_independent_of_d11_code():
    """the nonlocal coefficient -A(4) of the 4D structures, read from P18_D11.json, against the hand expansion
  +>  of
    -(i/4)(Dhat2/60 + Dhat0/18), Phat_mn = p_m p_n - g_mn p^2 (one representative per orbit)."""
    res = cj("P18_D11.json")["result"]
    want = {"p_m p_n p_r p_s": -I / 60, "g4_mn p_r p_s (p.p)^1": I / 80, "g4_rs p_m p_n (p.p)^1": I / 80,
            "g4_mn g4_rs (p.p)^2": -I / 80, "g4_mr p_n p_s (p.p)^1": I / 480, "g4_mr g4_ns (p.p)^2": -I / 480}
    four = {k: v for k, v in res.items() if not v["evanescent"]}
    assert set(four) == set(want)
    for k, w in want.items():
        assert sp.simplify(-P(four[k]["nonlocal_coefficient_of_L"]) - w) == 0, k      # L-coeff = -A(4) = X


# ============================================================== D11 evaluation
def test_d11_evaluated_controls_mutations():
    c = cj("P18_D11.json")
    assert c["evaluated"] is True and c["D11_evaluated"] is True and c["D11_matching_verified"] is True
    assert len(c["controls"]) == 14 and all(v is True for v in c["controls"].values())
    assert c["controls"]["exactly_degree_four_every_structure"] and c["controls"]["N4_no_order_0_or_2_contact_
  +> arises"]
    assert "N4" in c["prescription_usage"]["N4_contact_exclusions"]
    assert sorted(c["mutations"]) == sorted(("premature_d4", "drop_evanescent_gt", "wrong_external_index_posit
  +> ion",
                                             "wrong_offdiagonal_weight"))
    assert all(v["accepted_as_rejection"] for v in c["mutations"].values())
    assert c["evanescent_pole_nonzero"] is True and c["longitudinal_projection_p_m_G"] == {}
    assert len(c["result"]) == 14


def test_d11_laurent_consistency():
    for k, v in cj("P18_D11.json")["result"].items():
        a = P(v["A_exact_d_times_MB"])
        a4 = DR.substitute_d(a, 4, "test_p18: A(4)")
        da = DR.substitute_d(sp.diff(a, DR.d), 4, "test_p18: A'(4)")
        assert sp.simplify(P(v["UV_pole"]) - a4) == 0, k
        L = sp.Symbol("L")
        assert sp.simplify(P(v["MSbar_finite"]) - (a4 * (2 - L) - 2 * da)) == 0, k


# ============================================================== full hh slot
def test_full_slot_verdict_and_seven_assignments():
    c = cj("P18_hh_slot.json")
    assert c["decoupling_proven_full_row"] is True and c["surviving_evanescent_structures_in_slot"] == []
    assert c["D11_reopens_evanescent_question"] is False
    assert all(v is True for v in c["controls"].values()) and len(c["controls"]) == 11
    sv = c["slot_values"]
    assert len(sv["D01+D03+D11"]) == 7 and "c hat, h tilde" in sv["D01+D03+D11"]                 # H2
    assert all(v == "0" for v in sv["D01+D03+D11"].values()) and all(v == "0" for v in sv["D11"].values())
    # H2: the reviewer's value X_D01 = p.p^2 a~.b~ c^.p/16 = -X_D03 (purely through the evanescent gt_rs p_n s
  +> tructure)
    assert sv["D01"]["c hat, h tilde"] == "a~.b~*c^.p*p.p**2/16" and sv["D03"]["c hat, h tilde"] == "-a~.b~*c^
  +> .p*p.p**2/16"
    assert any(v != "0" for v in sv["D11_under_trace_operator"].values())
    assert c["p_dot_G"]["D11"] == {} and c["p_dot_G"]["D01+D03+D11"] == {}
    assert all(v["accepted_as_rejection"] for v in c["mutations"].values())


def test_full_slot_transverse_coordinates():
    fd = cj("P18_hh_slot.json")["four_D_poles"]
    k1, k2 = [[sp.Rational(x) for x in v] for v in fd["transverse_basis_in_p14r_coordinates"]]
    z = {g: [Z(x) for x in fd["graphs"][g]["coordinates_p14r_basis"]] for g in ("D11", "D01+D03", "D01+D03+D11
  +> ")}
    a, b = sp.symbols("a b")
```

```
    sol = lambda w: sp.solve([a * u + b * v - x for u, v, x in zip(k1, k2, w)], [a, b], dict=True)  # noqa: E7
  +> 31
    assert sol(z["D11"]) == [{a: I / 240, b: -I / 120}]
    assert sol(z["D01+D03"]) == [{a: 61 * I / 120, b: -7 * I / 20}]
    assert sol(z["D01+D03+D11"]) == [{a: 41 * I / 80, b: -43 * I / 120}]
    assert all(fd["graphs"][g]["transverse"] for g in z)


def test_readback_detects_mutation_inside_wrapped_value():
    """the long full-slot verdict is wrapped in the listing; a mutation inside it must still fail the readback
  +> ."""
    import build as B
    rel = "certificates/P18_hh_slot.json"
    txt = B.wrap_lines(B.ascii_clean((ROOT / rel).read_text()))
    page = txt[: len(txt) // 2] + "\n" + QC.VERSION_LABEL + " | page 7\n" + txt[len(txt) // 2:]
    assert B.readback_values(page, ROOT, rel, page)["pass"] is True
    v = B.value_snippets(ROOT, rel)[0]
    assert v.startswith('"verdict"') and v not in page                      # wrapped: not verbatim
    bad = page.replace("D11 by itself", "D11 BY itself", 1)
    assert bad != page and B.readback_values(bad, ROOT, rel, bad)["pass"] is False


def test_inventory_and_quantum_inputs_flag():
    c = cj("P18_hh_slot.json")
    assert set(c["inventory"]) == set(QC.INVENTORY_P18) and all(v["evaluated"] for v in c["inventory"].values(
  +> ))
    # fresh-build ordering regression (presync #2): every input certificate, incl. this run's P18_D11.json, is
  +>  present
    assert all(v["present"] is True for v in c["inventory"].values())
    assert c["quantum_inputs_evaluated"] == "earned"
    led = json.loads((ROOT / "ledgers" / "P13_graph_inventory.json").read_text())["graphs"]
    assert {k.split("_")[0] for k in led} == set(QC.INVENTORY_P18)                # independent: the P13 ledg
  +> er
    rows = json.loads((ROOT / "ledgers" / "P13_ch_dependency_rows.json").read_text())["rows"]
    req = {g.split("_")[0] for r in rows if r["status"].startswith("surviving") for g in r["graphs"]}
    assert req == set(QC.INVENTORY_P18) and len({r["id"] for r in rows if r["status"].startswith("surviving")}
  +> ) == 3
    lst = rs()["quantum_inputs_inventory"]
    assert {x["graph"] for x in lst} == req and all(x["evaluated"] for x in lst)
    assert not [x for x in lst if not x["status"] or "missing" in x["status"].lower()
                or "unevaluated" in x["status"].lower()]


# ============================================================ status
def test_p18_release_status_and_flags():
    s = rs()
    assert s["revision"] == "p18" and s["publication"] == "accepted" and s["release_claim_accepted"] is True
    assert s["D11_matching_verified"] is True and s["D11_evaluated"] is True
    assert s["D11_matching_verified_scope"] == "4D components"
    assert s["D11_evanescent_components_matching"] == "not_applicable_not_verified"
    assert s["M3_status"].startswith("unconditional on the 4D components")
    assert s["quantum_inputs_evaluated"] == "earned" and s["hh_full_row_decoupling_proven"] is True
    assert s["projected_identity_verified"] == "not_earned" and s["projected_restoration_verified"] == "not_ea
  +> rned"
    assert s["projected_defect_computed"] == "not_earned"
    assert s["hard_stops"] == QC.HARD_STOPS_P18 and s["hard_stops_hold"] is True
    assert s["symmetry_restored"] is False and s["bulk_symmetry_restored"] is False and s["O1_closure"] == "NO
  +> T_CLOSED"
    assert s["XVI_P_registered"] is False and s["Route_B_discharged"] is False and s["u_R_c_delta_added"] is F
  +> alse
    assert s["XV_B_opened"] is False and s["XIV_J_opened"] is False and s["evanescent_hh_counterterm_added"] i
  +> s False
    assert s["insertion_status"] == "Frozen_negative_obstruction_preserved"
    assert s["XVI_M_sha256"].startswith("41aa1099") and s["XVI_M_pin_ok"] is True
    assert s["milestones"]["status"]["P.5_identity"] == "open"
    assert s["independent_rebuild_passed"] in ("pending", False)   # p19: the live p18 report is bound to the
  +> p18 tree
    assert s["contract_identity"]["added"] == [] and s["contract_identity"]["changed_vs_p14r"] == ["code/p5_en
  +> try/dred.py"]


def test_p17_independent_rebuild_recorded_and_not_inherited():
    s = rs()
    rec = s["p17_independent_rebuild"]
    assert s["independent_rebuild_passed_p17"] is True
    assert rec["report_sha256"] == QC.P17_INDEPENDENT_REPORT_SHA256 and rec["pins_ok"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P17_SOURCE_TREE_SHA256
    assert rec["bound_to_source_zip_sha256"] == QC.P17_SOURCE_ZIP_SHA256
    assert "SAME-BOX" in rec["caveat"] and rec["mismatches"] == 0
    import source_manifest as SM
    assert SM.tree_sha(ROOT) != QC.P17_SOURCE_TREE_SHA256
```

```
def test_change_log_vs_p17():
    s = rs()
    cl = s["change_log_vs_p17"]
    assert cl["base_pinned_ok"] is True
    base = json.loads((ROOT / QC.P17_SOURCE_LIST).read_text())["files"]
    import source_manifest as SM
    cur = {r: hashlib.sha256((ROOT / r).read_bytes()).hexdigest() for r in SM.source_files(ROOT) if r != SM.RE
  +> FERENCE_FILE}
    changed = sorted(set(cur) ^ set(base) | {k for k in set(cur) & set(base) if cur[k] != base[k]} | {SM.REFER
  +> ENCE_FILE})
    assert sorted(cl["added"] + cl["removed"] + cl["modified"]) == changed
    assert SM.REFERENCE_FILE in cl["added"] and cl["contract_files_changed"] == ["code/p5_entry/dred.py"]
    import p19_conventions as Q19                                  # p19: tests revised in p18 or (disclosed)
  +> in p19
    assert (set(x for x in cl["modified"] if x.startswith("tests/")   # p22 filter
                and not __import__("contract_registry").p22_revised(x))
            <= set(QC.DISCLOSED_TEST_REVISIONS_P18) | set(Q19.DISCLOSED_TEST_REVISIONS_P19))


def test_version_label_p18():
    import build as B
    assert QC.VERSION_LABEL == "XVI-P p18" and B.QC.VERSION_LABEL == f"XVI-P {B.QC.REVISION}"   # p19: current
  +>  label
    for f in ("certificates/P18_D11_M2.json", "certificates/P18_D11.json", "certificates/P18_hh_slot.json"):
        assert f in B.VALUE_BLOCKS


# ============================================================ p18 gate mutations
def _copy():
    d = Path(tempfile.mkdtemp(prefix="p18gate_"))
    for sub in ("code", "certificates", "ledgers", "exports", "tools", "archival", "reproducibility", "provena
  +> nce", "tests"):
        if (ROOT / sub).exists():
            shutil.copytree(ROOT / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__", "*.zip", "*.pdf"
  +> ))
    for f in ("README.md", "REBUILD_INSTRUCTIONS.md", "RELEASE_STATUS.json"):
        if (ROOT / f).exists():
            shutil.copy(ROOT / f, d / f)
    (d / "manuscript").mkdir()
    shutil.copy(ROOT / "manuscript" / "preamble.tex", d / "manuscript" / "preamble.tex")
    return d


def _edit_cert(d, name, fn):
    f = d / "certificates" / name
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c, indent=1))


def _m2_control(d):
    _edit_cert(d, "P18_D11_M2.json", lambda c: c["controls"].__setitem__("route_B_all_100_pairs_agree", False)
  +> )


def _m2_mutation_accepted(d):
    _edit_cert(d, "P18_D11_M2.json", lambda c: c["mutations"]["alpha_beta_swapped"].__setitem__(
        "accepted_as_rejection", False))


def _d11_not_evaluated(d):
    _edit_cert(d, "P18_D11.json", lambda c: c.__setitem__("evaluated", False))


def _surviving_structure(d):
    _edit_cert(d, "P18_hh_slot.json", lambda c: c.__setitem__("surviving_evanescent_structures_in_slot",
                                                              ["gt_rs p_m p_n (p.p)^1"]))


def _m2_scope_dropped(d):
    _edit_cert(d, "P18_D11_M2.json", lambda c: c.pop("D11_matching_verified_scope"))


def _inventory_hole(d):
    _edit_cert(d, "P15_D13.json", lambda c: c["D13_evaluated"].__setitem__("execution", False))


def _tamper_p17_layer(d):
    f = d / "code" / "p5_calc" / "hh_slot.py"
```

```
    f.write_text(f.read_text() + "\n# tampered\n")


def _dred_code_change(d):
    f = d / "code" / "p5_entry" / "dred.py"
    f.write_text(f.read_text() + "\n_TAMPER = 1\n")


def _new_contract_file(d):
    (d / "code" / "p5_entry" / "evanescent_hh_counterterm.py").write_text("# undisclosed contract file\n")


def _tamper_p17_report(d):
    f = d / "archival" / "p17" / "reproducibility" / "independent_rebuild_report_p17.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


def _projected_identity_claimed(d):
    _edit_cert(d, "P18_hh_slot.json", lambda c: c.__setitem__("projected_identity_verified", "earned"))


MUTS = [_m2_control, _m2_mutation_accepted, _m2_scope_dropped, _d11_not_evaluated, _surviving_structure, _inve
  +> ntory_hole,
        _tamper_p17_layer, _dred_code_change, _new_contract_file, _tamper_p17_report, _projected_identity_clai
  +> med]


@pytest.mark.parametrize("mut", MUTS, ids=lambda f: f.__name__)
def test_certificate_mutation_p18_gate_rejects_or_withholds(mut):
    d = _copy()
    try:
        mut(d)
        r = subprocess.run([sys.executable, "code/p5_calc/release_p18.py"], cwd=d, capture_output=True, text=T
  +> rue,
                           timeout=600)
        if mut is _inventory_hole:          # gate may publish, but must NOT earn quantum_inputs_evaluated
            if r.returncode == 0:
                assert json.loads((d / "RELEASE_STATUS.json").read_text())["quantum_inputs_evaluated"] == "not
  +> _earned"
            return
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d / "certificates" / "P18_release.json").exists()
        assert (d / "certificates" / "P18_release_rejected.json").exists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_p18_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = subprocess.run([sys.executable, "code/p5_calc/release_p18.py"], cwd=d, capture_output=True, text=T
  +> rue,
                           timeout=600)
        assert r.returncode == 0, r.stdout[-1500:]
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

147 Source: p19_conventions.py (p19 constants, version label, pins, disclosures)

[listing 114/133] code/p5_calc/p19_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p19 conventions: P.5 identity step (the projected one-loop kappa^2 ch-identity defect assembled from
  +> the
evaluated inputs; counterterm solve in the declared domain; D cap ker sigma_0); p18 reviewer findings I1-I3.

Layering.  p14r contract (code/p5_entry/*)  ->  p15 .. p17 layers  ->  p18 layer (code/p5_calc/{d11,p18_conven
  +> tions,
release_p18}.py; byte-identical, sha256 below)  ->  p19 layer (code/p5_calc/{defect,p19_conventions,release_p1
  +> 9}.py,
P19_*).  The p19 gate (release_p19.py) runs after the p14r, p15, p16, p17 and p18 gates and requires all five
  +> accepted.

No contract revision in p19: every p14r contract file (code/p5_entry/*) is byte-identical to p18 (release_p19
  +> checks).
"""
```

```
from __future__ import annotations

REVISION = "p19"
VERSION_LABEL = "XVI-P p19"          # single version variable (manuscript footer is checked against it)
CERT_PREFIX = "P19"
CONTRACT_REVISION = "p19 (p14r contract + disclosed p15/p16/p17/p18 revisions; none in p19)"
DATE = "2026-10-09"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p19.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p19.json"
P18_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p18.json"      # written by the reviewer;
  +>  never here
P18_INDEPENDENT_REPORT_SHA256 = "893f167ef7124d981c0569ca386035aed06e0dbb0c90a2362a6ceeafd1930cb2"
P18_SOURCE_TREE_SHA256 = "bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094"
P18_SOURCE_ZIP_SHA256 = "7c7d5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f"
P18_ARCHIVED_RECORDS = {
    "archival/p18/reference_values_p18.json": "8a867f9eb57af673d9c7d70a19910a6afe2e80630bf6c4a245bd2ca2c99a35b
  +> f",
    "archival/p18/reproducibility/author_rebuild_report_p18.json":
        "13da1ba2eaf9cab1e1c3990bb50c41e66996ed52147e303d6bf28d22cefa5fa5",
    "archival/p18/reproducibility/independent_rebuild_report_p18.json": P18_INDEPENDENT_REPORT_SHA256,
    # per-file sha256 list of the released p18 source-only zip (base of the p19 change log; works source-only)
    "archival/p18/p18_source_files.json": "9b5ebcb021597a7345cfd2c36d9486d42a21e38c77fd19bd8aec4d6a60c3a1f3",
}
P18_SOURCE_LIST = "archival/p18/p18_source_files.json"

# the ch dependency rows (ledgers/P13_ch_dependency_rows.json, surviving) and their inputs
ROW_HH = "(S0,Gamma1^[2]) s2 K:K.c"
ROW_KC = "(S0,Gamma1^[2]) s1 h:h^2"
ROW_1PT = "(S1,Gamma1^[1]) s2 K:K.c.h"
ROWS = {ROW_HH: ("D01", "D02", "D03", "D11", "D12"), ROW_KC: ("D13",), ROW_1PT: ("D14a", "D14b", "D14c")}
INVENTORY_P19 = ("D01", "D02", "D03", "D11", "D12", "D13", "D14a", "D14b", "D14c")
HH_EVALUATED = ("D01", "D03", "D11")               # D02 = D12 = 0 (P15_batchA.json)

HARD_STOPS_P19 = {
    "bulk_symmetry_restored": False,               # stays false regardless (the identity is PROJECTED onto ch
  +>  only)
    "symmetry_restored": False,
    "O1_closure": "NOT_CLOSED",
    "insertion_status": "Frozen_negative_obstruction_preserved",
    "XV_B_opened": False,
    "XIV_J_opened": False,
    "Route_B_discharged": False,
    "u_R_c_delta_added": False,
    "XVI_P_registered": False,
}
# flags that p19 may EARN, each only if its certificate says so (release_p19 reads them; never set by hand)
EARNABLE_P19 = ("projected_defect_computed", "projected_identity_verified", "projected_restoration_verified")

# p18 layer: byte-identical in p19 (sha256 from XVI-P-p18-source-only.zip; release_p19 recomputes them)
P18_CALC_SHA256 = {
    "code/p5_calc/d11.py": "432e1e91cf8d56001e49ffc5b673262ceb5438abb6a9d51b3477d613a7c7698d",
    "code/p5_calc/p18_conventions.py": "ea8ee2e148a873490050a09ea17d1a34297713f955c6bad134d9c3bc4b31f66e",
    "code/p5_calc/release_p18.py": "73a052ddf3ea2bfb2fcb4f1f6ee08c6675253058038576987 9a3556aafe0a0f5",
}
P18_DRED_SHA256 = "14c11a512c1609c9f9a7829dcb68e71a94c8ac3d38497e2dcac110a1b8d5e81c"     # dred.py as released
  +>  in p18
NEW_CONTRACT_FILES_P19 = ()
DISCLOSED_CONTRACT_REVISIONS_P19 = {}

DISCLOSED_TEST_REVISIONS_P19 = {    # test files changed vs the p18 zip (release_p19 change log)
    "tests/test_p14.py": "names p18 -> p19 (reference file, source zip, author report, status revision); "
                         "generated prefixes certificates/ledgers P19_, manuscript/body/P19-",
    "tests/test_p18.py": "rs() reads the p18 LAYER record certificates/P18_release.json; the live p18 independ
  +> ent "
                         "report is bound to the p18 tree (pending or False here); test-file change log accept
  +> s "
                         "p18 or disclosed p19 revisions; build label is the current revision",
}
RUN_SPECIFIC_KEYS = ("reproducibility", "author_rebuild_passed", "independent_rebuild_passed",
                     "live_report_identical_to_archived", "live_report_present", "archived_zip_sha256_ok",
                     "list_equals_archived_zip")

# I2 (p18 reviewer): the extended static scan (tests/test_p19.py; a PATTERN CHECK, NOT A PROOF) also lists eve
  +> ry
# package call of Poly.terms() / Poly.coeffs() (Horner-type evaluation route); each site is reviewed: the Poly
# generators are never d (loop momenta, polarisation symbols, Feynman parameter x, D1/D2, q, l.l/l.p, external
  +>  dot
# products); coefficients may carry d but terms()/coeffs() do not evaluate them.
COEFF_SITES_REVIEWED = {
    "code/p5_calc/batchA.py::route2::P.terms()": "generators: loop momentum components l0..l3",
    "code/p5_calc/batchC.py::count_quadratic_component_terms::sp.Poly(tot, *gts).monoms()":
```

```
        "generators: Gt_* evanescent metric symbols (degree count only)",
    "code/p5_calc/covariant.py::fp_term::sp.Poly(expr, *gens).terms()":
        "generators: loop/external momentum symbols, K2, KP, Feynman parameter X",
    "code/p5_calc/d13.py::ck_basis::sp.Poly(e, *PS).coeffs()": "generators: external momentum components PS",
    "code/p5_calc/d13.py::ccb_basis::sp.Poly(e, *PS).coeffs()": "generators: external momentum components PS",
    "code/p5_calc/defect.py::ghost_equation_relation::sp.Poly(e, *PS).coeffs()":
        "generators: external momentum components PS (K1 probe)",
    "code/p5_calc/defect.py::_eqs::sp.Poly(x, *syms).terms()":
        "generators: external dot products (d and the unknowns excluded explicitly)",
    "code/p5_calc/integrals.py::J_two_point::sp.Poly(num, D1, D2).terms()": "generators: propagator symbols D1
  +> , D2",
    "code/p5_calc/integrals.py::fp_two_point::P.terms()": "generators: shifted loop momenta k0..k3",
    "code/p5_calc/integrals.py::fp_two_point::Px.terms()": "generator: Feynman parameter x",
    "code/p5_entry/counterterm_domain.py::density::P.terms()": "generators: momentum components p0..p3",
    "code/p5_entry/counterterm_domain.py::two_point_vector::sp.Poly(rest, *p).monoms()":
        "generators: momentum components p0..p3 (exponents only)",
    "code/p5_entry/dred.py::_integral_functional::P.terms()": "generators: l.l, l.p (J_tag names; nothing eval
  +> uated)",
    "code/p5_entry/feynman.py::ge_vertex::P.terms()": "generators: graviton polarisation symbols (esym_pol)",
    "code/p5_entry/hessian_route.py::_ell_monomials::sp.Poly(e, *gens).terms()":
        "generators: free symbols of the classical 4D ell_k jets (field/momentum names); a d-like name would r
  +> aise "
        "(index parse of the name, rational coefficients cf.p/cf.q required): fails closed",
    "code/p5_entry/prescription_d11.py::vec::sp.Poly(v, *p).terms()": "generators: momentum components p0..p3"
  +> ,
    "code/p5_entry/prescription_d11.py::M3_local_space::sp.Poly(e, *p).terms()": "generators: momentum compone
  +> nts p0..p3",
    "code/p5_entry/shadow.py::expect::P.terms()": "generator: q",
}
POLY_GENERATOR_CALL_SITES_REVIEWED = {     # I2: Poly(.., *<call>) sites (generators built at run time)
    "code/p5_entry/feynman.py::ge_vertex::sp.Poly(v, *polsyms.keys())":
        "generators: graviton polarisation symbols esym_pol(leg) (dict keys); d never a key",
}
```

148 Source: defect.py (p19 layer, disclosed p20 revision: K1 orientation fix, full sigma-0 closure, restoration redefined)

[listing 115/133] code/p5_calc/defect.py

```
#!/usr/bin/env python3
"""XVI-P p19: the projected one-loop kappa^2 ch-identity DEFECT, assembled from the evaluated inputs, and the
counterterm solve in the declared domain (P.5 identity step).

Object.  The hbar^1 kappa^2 part of (S_Psi, Gamma_Psi) projected onto the ch pattern (one c, one h; P.3 frozen
projection, ledgers/P13_ch_dependency_rows.json).  Exactly three rows survive (the other three are derived zer
  +> o):
  R_hh   (S0, Gamma1^[2]) s2 K:K.c      X_hh = int (s0 h)_(mn)(p; c) G_hh^{(mn)(rs)}(p) h_(rs)(-p)
                                        inputs D01 + D02 + D03 + D11 + D12  (D02 = D12 = 0, P15_batchA)
  R_Kc   (S0, Gamma1^[2]) s1 h:h^2      X_Kc = int G_Kc_{(mn) nu}(p) c^nu(p) E^{(mn)(rs)}(p) h_(rs)(-p)
                                        input D13 (G_Kc = sum A_i(d) T_i + B_j(d) E_j, P15_D13)
  R_1pt  (S1, Gamma1^[1]) s2 K:K.c.h    one-point h tadpoles D14a-c (vanishing under the declared prescription
  +> )
  defect  Delta[c, h] = s_hh X_hh + s_Kc X_Kc + X_1pt   (s = +-1: antibracket summand orientation; reported
          independent of s when every row vanishes).
Every A(d) is kept exact in the opaque d; Laurent / MS-bar only at the end (MB = 1/eps + 2 - L):
  pole = X(4),  nonlocal (coefficient of L) = -X(4),  local finite = 2 X(4) - 2 X'(4).

Route C (covariant, exact in d): the frozen slot contractions (hh_slot.slot_value with s0 h; e2_evanescent.slo
  +> t with
  the h^2 operator E of S_Psi^(0)), seven external assignments (Q4S, QdS, tilde, mixed), evanescent parts sepa
  +> rately.
Route G (graded jets, 4D): sigma_0 = (S_Psi^(0), .) (frozen antibracket) applied to the 11 p14r domain densiti
  +> es D;
  ranks, D cap ker sigma_0 (never computed before p19), D cap Im sigma_0, the ch projection P_ch sigma_0, quot
  +> ient
  coordinates; the Gamma1 local data (pole, local finite) in the domain coordinates.
Counterterm solve: Delta + P_ch sigma_0 (x) = 0 for x in D, part by part (pole, nonlocal, local).  A nonlocal
  +> residual
  can never be cancelled by a local counterterm (obstruction); a local residual is cancellable iff it lies in
  +> the image.
Outcomes kept separate: projected_defect_computed / projected_identity_verified (regulated Gamma1: defect zero
  +>  or
  cancelled by allowed counterterms) / projected_restoration_verified (renormalised Gamma1^R = Gamma1 + MS-bar
  counterterms of the declared domain satisfies the projected identity; scope ch projection only) /
  bulk_symmetry_restored (stays false: nothing beyond the ch projection is claimed).
"""
from __future__ import annotations
```

```
import json
import sys
import time
from fractions import Fraction as Fr
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import dred  # noqa: E402
import covariant as C  # noqa: E402
import batchC as BC  # noqa: E402
import hh_slot as H  # noqa: E402
import e2_evanescent as E2M  # noqa: E402
import d13 as D13  # noqa: E402
import p19_conventions as QC  # noqa: E402

d = dred.d
I = sp.I
EXT = dict(H.EXT, **{"c hat, h tilde": ("c^", "a~", "b~")})      # the seven external assignments (p18 H2)
STRUCT = E2M.STRUCT
PPOW = E2M.PPOW
CERT = ROOT / "certificates"


def log(m):
    print(f"[defect {time.strftime('%H:%M:%S')}] {m}", flush=True)


def s(x):
    return str(sp.factor(x)) if x != 0 else "0"


def jl(name):
    return json.loads((CERT / name).read_text())


def P(text):
    return dred.parse_d(text, {"I": I})


# ================================================================ inputs (inventory; nothing Missing -> zero)
def inventory(drop=()):
    """every graph of every surviving ch row, re-read from its own certificate; returns (ok, rows)."""
    led = json.loads((ROOT / "ledgers" / "P13_ch_dependency_rows.json").read_text())["rows"]
    surv = {r["id"]: [g.split("_")[0] for g in r["graphs"]] for r in led if str(r["status"]).startswith("survi
  +> ving")}
    ba = jl("P15_batchA.json")
    bc = jl("P16_batchC.json")["graphs"]
    d13 = jl("P15_D13.json")
    d11 = jl("P18_D11.json")
    st = {g: v.get("status") for g, v in ba["graphs"].items()}
    claim = ba.get("batchA_evaluated", {}).get("claim", "")
    ev = {g: ba.get("batchA_evaluated", {}).get("execution") is True and g in claim
          for g in ("D02", "D12", "D14a", "D14b", "D14c")}
    st |= {"D01": bc["D01"]["status"], "D03": bc["D03"]["status"], "D13": d13.get("status"), "D11": d11.get("s
  +> tatus")}
    ev |= {"D01": bc["D01"]["evaluated"] is True, "D03": bc["D03"]["evaluated"] is True,
           "D13": d13.get("D13_evaluated", {}).get("execution") is True, "D11": d11.get("D11_evaluated") is Tr
  +> ue}
    rows = []
    for rid, gs in surv.items():
        for g in gs:
            present = g not in drop
            rows.append({"row": rid, "graph": g, "present": present, "evaluated": bool(present and ev.get(g)),
                         "status": st.get(g) if present else "MISSING"})
    ok = (set(surv) == set(QC.ROWS) and all(sorted(surv[r]) == sorted(QC.ROWS[r]) for r in QC.ROWS)
          and all(r["present"] and r["evaluated"] and r["status"] and "missing" not in str(r["status"]).lower(
  +> )
                  for r in rows)
          and jl("P18_hh_slot.json").get("quantum_inputs_evaluated") == "earned")
    return ok, rows


# ================================================================ Laurent
def laurent(x):
    """X(d) * MB -> pole, nonlocal (coefficient of L), local finite."""
```

```
    if x == 0:
        return {"pole": sp.Integer(0), "nonlocal_L": sp.Integer(0), "local": sp.Integer(0)}
    ex = D13.expand_coeff(x)
    return {"pole": sp.expand(ex["pole"]), "nonlocal_L": sp.expand(ex["nonlocal_L_coefficient"]),
            "local": sp.expand(ex["local"])}


# ============================================================== Route C rows
def hh_row(A, op, scale=None, drop_ev=()):
    terms = []
    for g, Ag in A.items():
        terms += H.g_terms(Ag, (scale or {}).get(g, 1), drop_evanescent=g in drop_ev)
    return {n: H.slot_value(op, terms, e) for n, e in EXT.items()}


def kc_row(A13, E):
    return {n: sp.factor(sum(A13[k] * E2M.slot(STRUCT[k], E, e, PPOW[k]) for k in STRUCT if A13.get(k, 0) != 0
  +> ))
            for n, e in EXT.items()}


def assemble(Xhh, Xkc, s_hh=1, s_kc=1):
    out = {}
    for n in EXT:
        tot = sp.factor(s_hh * Xhh[n] + s_kc * Xkc[n])
        out[n] = {"exact_d_times_MB": tot, **laurent(tot)}
    return out


def is_zero(defect, parts=("pole", "nonlocal_L", "local")):
    return all(sp.expand(v[p]) == 0 for v in defect.values() for p in parts)


# ============================================================== Route C counterterm images and solve
def ct_images_C(op, E, hhbasis, ckbasis):
    """ch images of the p14r h.h (5) and c.K (3) domain directions in the slot functionals (same conventions a
  +> s the
    rows: hh directions through s0 h, c.K directions through E).  c.cb and phi.phi have no ch image (Route G).
  +> """
    imgs = []
    for k, b in enumerate(hhbasis):
        v = hh_row({"b": b["coords"]}, op)
        imgs.append(("h.h#%d" % k, {n: sp.expand(dred.substitute_d(x, 4, "defect: 4D domain direction image (1
  +> ocal, "
                                                                    "d-free after contraction)")) if x != 0 e
  +> lse
                                  sp.Integer(0) for n, x in v.items()}))
    for k, b in enumerate(ckbasis):
        A = {nm: b["T_coordinates"][i] for i, nm in enumerate(("T1", "T2", "T3"))}
        v = kc_row(A, E)
        imgs.append(("c.K#%d" % k, {n: sp.expand(x) for n, x in v.items()}))
    return imgs


def _eqs(vals, unknowns=()):
    """{assignment: expression} -> list of coefficient expressions (in the external dot products)."""
    out = []
    for n, x in vals.items():
        x = sp.expand(x)
        if x == 0:
            continue
        syms = sorted(x.free_symbols - {d} - set(unknowns), key=str)
        if not syms:
            out.append(x)
            continue
        for _, cf in sp.Poly(x, *syms).terms():
            out.append(cf)
    return out


def solve_C(target, imgs):
    """find x with sum_k x_k img_k + target = 0 on all assignments (exact)."""
    xs = sp.symbols(f"x0:{len(imgs)}")
    comb = {n: sum(x * im[1][n] for x, im in zip(xs, imgs)) + target.get(n, 0) for n in EXT}
    eqs = _eqs(comb, xs)
    M = sp.Matrix([[sp.diff(e, x) for x in xs] for e in eqs]) if eqs else sp.zeros(0, len(xs))
    sol = sp.linsolve(eqs, *xs) if eqs else sp.FiniteSet(tuple(sp.Integer(0) for _ in xs))
    return {"rank": M.rank() if eqs else 0, "solvable": sol != sp.EmptySet,
            "solution": [s(v) for v in list(sol)[0]] if sol != sp.EmptySet else None}
```

```
def image_rank_C(imgs):
    xs = sp.symbols(f"x0:{len(imgs)}")
    comb = {n: sum(x * im[1][n] for x, im in zip(xs, imgs)) for n in EXT}
    eqs = _eqs(comb, xs)
    M = sp.Matrix([[sp.diff(e, x) for x in xs] for e in eqs])
    return M.rank(), [[s(v) for v in col] for col in M.nullspace()]


def classify(defect, imgs):
    """ZERO / CANCELLABLE (local finite residual in the image) / OBSTRUCTION (nonlocal or outside the image)."
  +> ""
    if is_zero(defect):
        return {"outcome": "ZERO", "residual": None}
    nl = {n: v["nonlocal_L"] for n, v in defect.items()}
    if any(sp.expand(x) != 0 for x in nl.values()):
        return {"outcome": "OBSTRUCTION (nonlocal residual: the L-coefficient of the defect is nonzero; no loc
  +> al "
                           "counterterm can cancel it)",
                "residual": {"nonlocal_L": {n: s(x) for n, x in nl.items()}}}
    pole = solve_C({n: v["pole"] for n, v in defect.items()}, imgs)
    loc = solve_C({n: v["local"] for n, v in defect.items()}, imgs)
    if pole["solvable"] and loc["solvable"]:
        return {"outcome": "NONZERO, CANCELLABLE by a counterterm in the declared domain", "pole_solution": po
  +> le,
                "local_solution": loc, "residual": {"local": {n: s(v["local"]) for n, v in defect.items()}}}
    return {"outcome": "OBSTRUCTION (local residual outside the image of the declared domain)",
            "residual": {"pole": {n: s(v["pole"]) for n, v in defect.items()},
                         "local": {n: s(v["local"]) for n, v in defect.items()}}}


# ============================================================== Route G (graded jets)
def relations(vecs):
    """sparse exact elimination: returns (rank, list of linear relations sum_i r_i vecs_i = 0 as dicts)."""
    basis, rels = [], []
    for i, v in enumerate(vecs):
        r = dict(v)
        comb = {i: Fr(1)}
        for piv, b, cb in basis:
            if piv in r:
                f = r[piv] / b[piv]
                for k, x in b.items():
                    nv = r.get(k, 0) - f * x
                    if nv:
                        r[k] = nv
                    else:
                        r.pop(k, None)
                for k, x in cb.items():
                    nc = comb.get(k, 0) - f * x
                    if nc:
                        comb[k] = nc
                    else:
                        comb.pop(k, None)
        if r:
            basis.append((min(r), r, comb))
        else:
            rels.append(comb)
    return len(basis), rels


class QI:
    """exact element of Q(i) as a pair of Fractions (Route G works over Q(i): the p14r densities carry explici
  +> t
    factors i for odd derivative counts, so a rational re/im split can count i-multiples as independent)."""
    __slots__ = ("re", "im")

    def __init__(self, re, im=Fr(0)):
        self.re, self.im = Fr(re), Fr(im)

    def __bool__(self):
        return bool(self.re or self.im)

    def __add__(self, o):
        return QI(self.re + o.re, self.im + o.im)

    def __sub__(self, o):
        return QI(self.re - o.re, self.im - o.im)

    def __mul__(self, o):
        return QI(self.re * o.re - self.im * o.im, self.re * o.im + self.im * o.re)

    def __truediv__(self, o):
```

```
        n = o.re * o.re + o.im * o.im
        return QI((self.re * o.re + self.im * o.im) / n, (self.im * o.re - self.re * o.im) / n)

    def sym(self):
        return sp.Rational(self.re.numerator, self.re.denominator) + I * sp.Rational(self.im.numerator,
                                                                                      self.im.denominator)


def complexify(v):
    """two_point_vector {(..., 're'|'im'): Fraction} -> {(...): QI}"""
    out = {}
    for k, x in v.items():
        kk = k[:-1]
        z = out.get(kk, QI(0))
        out[kk] = z + (QI(x) if k[-1] == "re" else QI(0, x))
    return {k: z for k, z in out.items() if z}


def relations_c(vecs):
    """exact elimination over Q(i): (rank, relations sum_i r_i vecs_i = 0 with r_i in Q(i))."""
    basis, rels = [], []
    for i, v in enumerate(vecs):
        r = dict(v)
        comb = {i: QI(1)}
        for piv, b, cb in basis:
            if piv in r:
                f = r[piv] / b[piv]
                for k, x in b.items():
                    nv = r.get(k, QI(0)) - f * x
                    if nv:
                        r[k] = nv
                    else:
                        r.pop(k, None)
                for k, x in cb.items():
                    nc = comb.get(k, QI(0)) - f * x
                    if nc:
                        comb[k] = nc
                    else:
                        comb.pop(k, None)
        if r:
            basis.append((min(r), r, comb))
        else:
            rels.append(comb)
    return len(basis), rels


def _c_vec(rel, n):
    return [rel[i].sym() if i in rel else sp.Integer(0) for i in range(n)]


def _frac_vec(rel, n):
    return [sp.Rational(rel.get(i, Fr(0)).numerator, rel.get(i, Fr(0)).denominator) for i in range(n)]


def route_g_vectors():
    """the 11 p14r domain densities, sigma_0 images (ghost +1 sectors), their ch parts, the domain vectors in
  +> the
    ghost-0 sectors and the Xi images (ghost -1, dim 5 bilinears; same enumeration as P14_counterterm_domain).
  +> """
    import counterterm_domain as CD
    from graded_jets import bracket
    lc, _ = CD.loop_capable_species()
    pairs = CD.admissible_pairs(2, lc)
    basis, labels = [], []
    for (X, Y), k in pairs:
        keep, _ = CD.independent(CD.sector_basis(X, Y, k), [(X, Y)])
        basis += keep
        labels += [f"{X}.{Y}#{i}" for i in range(len(keep))]
    S0 = CD.S0_density()
    s0chk = CD.S0_check(S0)
    O, SPC = CD.ORDER, CD.SPECIES
    gh1 = [(X, Y) for i, X in enumerate(O) for Y in O[i:]
           if SPC[X][1] + SPC[Y][1] == 1 and (SPC[X][2] + SPC[Y][2]) % 2 == 0 and 7 - SPC[X][0] - SPC[Y][0] >=
  +>  0]
    sig = [CD.two_point_vector(bracket(S0, b), gh1) for b in basis]
    ch = [{k: v for k, v in x.items() if (k[0], k[1]) == ("h", "c")} for x in sig]
    gh0 = [(X, Y) for i, X in enumerate(O) for Y in O[i:]
           if SPC[X][1] + SPC[Y][1] == 0 and (SPC[X][2] + SPC[Y][2]) % 2 == 0 and 6 - SPC[X][0] - SPC[Y][0] >=
  +>  0]
    D_full = [CD.two_point_vector(b, gh0) for b in basis]
    ims = []
```

```
    for i, X in enumerate(O):
        for Y in O[i:]:
            k = 5 - SPC[X][0] - SPC[Y][0]
            if SPC[X][1] + SPC[Y][1] == -1 and (SPC[X][2] + SPC[Y][2]) % 2 == 0 and k >= 0:
                ims += [CD.two_point_vector(bracket(S0, dn), gh0) for dn in CD.sector_basis(X, Y, k)]
    return {"labels": labels, "S0_check": s0chk, "gh1": gh1, "sig": sig, "ch": ch, "D_full": D_full, "ims": im
  +> s}


def route_g(V, field="Q(i)"):
    """ranks, D cap ker sigma_0, D cap ker P_ch sigma_0, D cap Im sigma_0, quotient -- over Q(i) (primary) or
  +> over Q
    with the re/im split of the p14r domain certificate (comparison only)."""
    cplx = field == "Q(i)"
    rel = relations_c if cplx else relations
    conv = (lambda vs: [complexify(v) for v in vs]) if cplx else (lambda vs: vs)
    vec = _c_vec if cplx else _frac_vec
    labels = V["labels"]
    n = len(labels)
    sig, ch = conv(V["sig"]), conv(V["ch"])
    r_sig, ker = rel(sig)
    r_ch, kerP = rel(ch)
    r_D, _ = rel(conv(V["D_full"]))
    r_im, _ = rel(conv(V["ims"]))
    _, rel_int = rel(conv(V["D_full"]) + conv(V["ims"]))
    cand = [vec({i: v for i, v in r.items() if i < n}, n) for r in rel_int]
    cand = [c for c in cand if any(x != 0 for x in c)]
    Wc = sp.Matrix.hstack(*[sp.Matrix(c) for c in cand]).columnspace() if cand else []
    kerV = [vec(r, n) for r in ker]
    kerPV = [vec(r, n) for r in kerP]
    K = sp.Matrix.hstack(*[sp.Matrix(v) for v in kerV]) if kerV else sp.zeros(n, 0)
    KP = sp.Matrix.hstack(*[sp.Matrix(v) for v in kerPV]) if kerPV else sp.zeros(n, 0)
    im_in_ker = all(sp.Matrix.hstack(K, w).rank() == K.rank() for w in Wc)
    im_in_kerP = all(sp.Matrix.hstack(KP, w).rank() == KP.rank() for w in Wc)
    comp, M = [], sp.Matrix.hstack(*Wc) if Wc else sp.zeros(n, 0)
    for i in range(n):
        e = sp.zeros(n, 1)
        e[i] = 1
        if sp.Matrix.hstack(M, e).rank() > M.rank():
            M = sp.Matrix.hstack(M, e)
            comp.append(i)
    Bq = sp.Matrix.hstack(*([sp.eye(n)[:, i] for i in comp] + list(Wc)))

    def quotient(v):
        c = Bq.LUsolve(sp.Matrix(v))
        return [sp.expand(c[j]) for j in range(len(comp))]

    def apply(vecs, coords):
        out = {}
        for c, v in zip(coords, vecs):
            if c == 0:
                continue
            for k, x in v.items():
                xv = x.sym() if cplx else sp.Rational(x.numerator, x.denominator)
                out[k] = sp.expand(out.get(k, 0) + c * xv)
        return {k: v for k, v in out.items() if v != 0}

    sector_rank = {}
    for sec in ("h.h", "c.cb", "c.K", "phi.phi"):
        idx = [i for i, l in enumerate(labels) if l.startswith(sec + "#")]
        sector_rank[sec] = {"dim": len(idx), "sigma0_rank": rel([sig[i] for i in idx])[0],
                            "P_ch_sigma0_rank": rel([ch[i] for i in idx])[0]}
    return {"field": field, "labels": labels, "n": n, "S0_check": V["S0_check"],
            "gh1_sectors": [f"{a}.{b}" for a, b in V["gh1"]], "dim_D": r_D, "image_rank_sigma0_on_Xi": r_im,
            "sigma0_rank_on_D": r_sig, "dim_D_cap_ker_sigma0": len(kerV), "D_cap_ker_sigma0_basis": kerV,
            "P_ch_sigma0_rank_on_D": r_ch, "dim_D_cap_ker_P_ch_sigma0": len(kerPV), "ker_P_ch_basis": kerPV,
            "dim_D_cap_Im_sigma0": len(Wc), "D_cap_Im_sigma0_basis": [list(w) for w in Wc],
            "Im_in_ker_sigma0": im_in_ker, "Im_in_ker_P_ch_sigma0": im_in_kerP,
            "cohomology_dim": len(kerV) - len(Wc), "quotient_complement": [labels[i] for i in comp],
            "quotient_dim": len(comp), "sector_ranks": sector_rank,
            "_quotient": quotient, "_apply": apply, "_sig": sig, "_ch": ch}


# ============================================================ Gamma1 local data in the domain coordinates
def hh_parts(A, graphs, basis):
    tot = {}
    for g in graphs:
        for k in BC.four_d_keys(A[g]):
            tot[k] = tot.get(k, 0) + A[g][k]
    parts = {"pole": {}, "nonlocal_L": {}, "local": {}}
    for k, v in tot.items():
```

```
        lt = laurent(v)
        for p in parts:
            parts[p][k] = lt[p]
    out = {}
    for p, vec in parts.items():
        z, ok = BC.pole_in_hh_basis(vec, basis)
        out[p] = {"z": z, "in_span": ok}
    return out


def ghost_equation_relation():
    """K1 (p19, informational): the ghost-equation operator 2 i D^rho (D13.deDonder_apply, the operator P15 ve
  +> rified
    on the one-loop functions: Sigma = 2 i D[G_Kc]) applied to the p14r c.K domain vertices V[K'(-p), c(p)],
    expressed in the p14r c.cb domain vertices V[cb(-p), c(p)] (same pw_vertex machinery and leg conventions a
  +> s the
    P15 fits).  Returns M with 2 i D V[c.K#j] = sum_m M[j][m] V[c.cb#m]."""
    import counterterm_domain as CD
    import feynman as F
    PS = D13.PS
    ck, _ = CD.independent(CD.sector_basis("c", "K", 3), [("c", "K")])
    cb, _ = CD.independent(CD.sector_basis("cb", "c", 4), [("cb", "c")])

    def vk(dn):
        out = {}
        for ap in range(10):
            for nu in range(4):
                v = F.pw_vertex(dn, [{"f": "K", "c": D13.HC[ap], "k": [-x for x in PS]}, {"f": "c", "c": nu, "
  +> k": list(PS)}])
                if v != 0:
                    out[(ap, nu)] = sp.expand(v)
        return out

    def vc(dn):
        out = {}
        for r in range(4):
            for nu in range(4):
                v = F.pw_vertex(dn, [{"f": "cb", "c": r, "k": [-x for x in PS]}, {"f": "c", "c": nu, "k": list
  +> (PS)}])
                if v != 0:
                    out[(r, nu)] = sp.expand(v)
        return out
    VC = [vc(dn) for dn in cb]
    M = []
    for dn in ck:
        g = D13.deDonder_apply(vk(dn))
        ms = sp.symbols(f"m0:{len(VC)}")
        eqs = []
        for k in set(g) | set().union(*VC):
            e = sp.expand(g.get(k, 0) - sum(m * V.get(k, 0) for m, V in zip(ms, VC)))
            if e != 0:
                eqs += sp.Poly(e, *PS).coeffs()
        sol = sp.solve(eqs, ms, dict=True)
        M.append([sol[0].get(m) for m in ms] if sol else None)
    return M


def orientation_ratio():
    """p20 K1 fix: route-G c.cb densities come from admissible_pairs in ORDER, i.e. the pair (c, cb); the P15
  +> Sigma
    coordinates (d13.ccb_basis) use sector_basis('cb', 'c').  pw_vertex takes ordered left derivatives, so the
  +>  two
    density families differ by a Grassmann sign.  Measured here (not assumed): r_k = V[routeG c.cb#k] / V[(cb,
  +> c)#k]
    on the same ordered legs [cb(-p), c(p)], required uniform over all components."""
    import counterterm_domain as CD
    import feynman as F
    PS = D13.PS
    g, _ = CD.independent(CD.sector_basis("c", "cb", 4), [("c", "cb")])
    q, _ = CD.independent(CD.sector_basis("cb", "c", 4), [("cb", "c")])
    out = []
    for dg, dq in zip(g, q):
        rs = set()
        for r in range(4):
            for nu in range(4):
                legs = [{"f": "cb", "c": r, "k": [-x for x in PS]}, {"f": "c", "c": nu, "k": list(PS)}]
                a, b = sp.expand(F.pw_vertex(dg, legs)), sp.expand(F.pw_vertex(dq, legs))
                if a == 0 and b == 0:
                    continue
                rs.add(sp.simplify(a / b) if b != 0 else None)
        out.append(rs.pop() if len(rs) == 1 and None not in rs else None)
```

```
    return out, len(g) == len(q) == 2


def tree_controls():
    """p20 tree-level controls on S0 itself: (i) the classical ghost equation V_S0[cb(-p), c(p)] = 2 i D V_S0[
  +> K(-p),
    c(p)] with the same operator P15 verified at one loop (D13.deDonder_apply); (ii) sigma_0(S0_Kc + S0_cbc) =
  +>  0 in
    every ghost +1 sector, S0_Kc / S0_cbc = the K.c and cb.c bilinear terms of the S0 density."""
    import counterterm_domain as CD
    import feynman as F
    from graded_jets import GE, bracket, _KEY
    PS = D13.PS
    S0 = CD.S0_density()
    vk, vc = {}, {}
    for ap in range(10):
        for nu in range(4):
            v = F.pw_vertex(S0, [{"f": "K", "c": D13.HC[ap], "k": [-x for x in PS]}, {"f": "c", "c": nu, "k":
  +> list(PS)}])
            if v != 0:
                vk[(ap, nu)] = sp.expand(v)
    for r in range(4):
        for nu in range(4):
            v = F.pw_vertex(S0, [{"f": "cb", "c": r, "k": [-x for x in PS]}, {"f": "c", "c": nu, "k": list(PS)
  +> }])
            if v != 0:
                vc[(r, nu)] = sp.expand(v)
    g = D13.deDonder_apply(vk)
    ghost_eq = bool(vk) and bool(vc) and all(sp.expand(g.get(k, 0) - vc.get(k, 0)) == 0 for k in set(g) | set(
  +> vc))

    def part(fields):
        out = GE()
        out.t = {k: v for k, v in S0.t.items() if len(k) == 2 and sorted(x[0] for x in k) == sorted(fields)
                 and not any(sym in _KEY for sym in v.free_symbols)}
        return out
    Skc, Scb = part(("K", "c")), part(("cb", "c"))
    O, SPC = CD.ORDER, CD.SPECIES
    gh1 = [(X, Y) for i, X in enumerate(O) for Y in O[i:]
           if SPC[X][1] + SPC[Y][1] == 1 and (SPC[X][2] + SPC[Y][2]) % 2 == 0 and 7 - SPC[X][0] - SPC[Y][0] >=
  +>  0]
    img_sum = CD.two_point_vector(bracket(S0, Skc + Scb), gh1)
    img_kc = CD.two_point_vector(bracket(S0, Skc), gh1)
    return {"ghost_equation_tree_V_cbc_eq_2iD_V_Kc": ghost_eq,
            "S0_Kc_terms": len(Skc.t), "S0_cbc_terms": len(Scb.t),
            "sigma0_S0Kc_plus_S0cbc_entries": len(img_sum),
            "sigma0_S0Kc_alone_entries (control, nonzero)": len(img_kc),
            "ok": bool(ghost_eq and Skc.t and Scb.t and not img_sum and img_kc)}


def main(argv=()):
    import certio
    t0 = time.time()
    ok_inv, inv_rows = inventory()
    ok_missing, rows_missing = inventory(drop=("D11",))          # control: Missing is refused, never zero
    if not ok_inv:
        log("inventory incomplete: the defect is NOT assembled (projected_defect_computed not earned)")
        certio.write(CERT / f"{QC.CERT_PREFIX}_defect.json",
                     {"landmark": "XVI-P", "revision": QC.REVISION, "projected_defect_computed": "not_earned",
                      "inventory": inv_rows, "reason": "inventory incomplete"})
        return 1
    log("inputs: D01, D03, D11 covariant vertices (batchC.evaluate_graph)")
    evs = {g: BC.evaluate_graph(g) for g in QC.HH_EVALUATED}
    A = {g: evs[g]["A"] for g in QC.HH_EVALUATED}
    d13c = jl("P15_D13.json")
    A13 = {k: P(d13c["result"][k]["A_exact_d_times_MB"]) for k in STRUCT}
    A13_naive = {k: P(v) for k, v in d13c["mutations"]["drop_evanescent_Gt"]["fit_if_accepted"].items()}
    log("operators: s0 h (K.c monomial of S0), E (h^2 monomials of S0), q^2 P (mutation)")
    op, opfit, _ = H.kc_operator()
    E, fitE = E2M.operator_from_polys(E2M.S0_hh_operator())
    Egf, _ = E2M.operator_from_polys(E2M.gauge_fixed_operator())
    # ---------------- Route C rows
    log("Route C: rows exact in d (7 external assignments)")
    Xhh = hh_row(A, op)
    Xkc = kc_row(A13, E)
    ev = {g: {k: v for k, v in A[g].items() if C.is_evanescent(k[0])} for g in A}
    Xhh_ev = hh_row(ev, op)
    Xkc_ev = kc_row({k: (A13[k] if k in ("E1", "E2") else 0) for k in STRUCT}, E)
    Xhh_g = {g: hh_row({g: A[g]}, op) for g in A}
    defect = assemble(Xhh, Xkc)
    signs = {f"s_hh={a},s_Kc={b}": is_zero(assemble(Xhh, Xkc, a, b)) for a in (1, -1) for b in (1, -1)}
```

```
  E2_alone = {n: E2M.slot(STRUCT["E2"], E, e, 1) for n, e in EXT.items()}
  T1_alone = {n: E2M.slot(STRUCT["T1"], E, e, 1) for n, e in EXT.items()}
  E1_alone = {n: E2M.slot(STRUCT["E1"], E, e, 1) for n, e in EXT.items()}
  # ---------------- domain directions in the slot functionals; solve
  log("domain bases (p14r h.h and c.K) and their ch images")
  hhb = BC.hh_basis()
  ckb = D13.ck_basis()
  imgs = ct_images_C(op, E, hhb, ckb)
  rankC, nullC = image_rank_C(imgs)
  outcome = classify(defect, imgs)
  # ---------------- Route G
  log("Route G: sigma_0 on the 11 domain densities, D cap ker, D cap Im, quotient")
  V = route_g_vectors()
  G = route_g(V, "Q(i)")
  Gq = route_g(V, "Q re/im split (p14r convention)")
  lab = G["labels"]
  n = G["n"]
  # Gamma1 local data in domain coordinates
  hp = hh_parts(A, QC.HH_EVALUATED, hhb)
  ckz = {"pole": [P(x) for x in d13c["UV_pole_in_p14r_cK_basis"]["coordinates"]],
         "local": [P(x) for x in d13c["local_polynomial_in_p14r_cK_basis"]],
         "nonlocal_L": [P(x) for x in d13c["nonlocal_in_p14r_cK_basis"]]}
  gse = d13c["ghost_self_energy_pole"]
  ccbz = {"pole": [P(x) for x in gse["pole_in_p14r_c_cb_coordinates"]],
          "local": [P(gse["local"][k]) for k in ("S1", "S2")]}
  data = {}
  for p in ("pole", "nonlocal_L", "local"):
      v = [sp.Integer(0)] * n
      for i, z in enumerate(hp[p]["z"]):
          v[lab.index(f"h.h#{i}")] = z
      for i, z in enumerate(ckz[p]):
          v[lab.index(f"c.K#{i}")] = z
      data[p] = v
  # Route G defect, sector by sector (relative sector normalisation not needed: each sector vanishes by itse
+> lf)
  g_def = {}
  for p, v in data.items():
      g_def[p] = {sec: G["_apply"]([G["_ch"][i] for i in range(n) if lab[i].startswith(sec)],
                                   [v[i] for i in range(n) if lab[i].startswith(sec)]) for sec in ("h.h", "c
+> .K")}
  g_zero = all(not x for per in g_def.values() for x in per.values())
  # quotient coordinates of the Gamma1 local data (ch sectors h.h and c.K; c.cb data recorded separately)
  quot = {p: [s(x) for x in G["_quotient"](v)] for p, v in data.items()}
  # MS-bar counterterm (pole) in ker P_ch sigma_0 and its full sigma_0 image by sector
  ct = [-x for x in data["pole"]]
  ct_ch = G["_apply"](G["_ch"], ct)
  ct_sig = G["_apply"](G["_sig"], ct)
  ct_sig_sectors = sorted({f"{k[0]}.{k[1]}" for k in ct_sig})
  # informational (beyond the ch projection): with the ghost self-energy pole (P15_D13, c.cb) added with a r
+> elative
  # factor lam, is the full MS-bar counterterm sigma_0-closed (all ghost +1 sectors, incl. c.b)?
  closure = {}
  for lam in (1, -1, I, -I):
      v = list(ct)
      for i, z in enumerate(ccbz["pole"]):
          v[lab.index(f"c.cb#{i}")] = -lam * z
      closure[str(lam)] = not G["_apply"](G["_sig"], v)
  # K1: is the relative factor -1 derived?  The verified ghost equation Sigma = 2 i D[G] maps the c.K coordi
+> nates z_K
  # to c.cb coordinates z_cb = M^T z_K (derived); sigma_0-closure of the full counterterm needs, per kernel
+> pair
  # c.cb#k + c.K#(k+1), the opposite relation.  Record both; the factor is NOT taken from any declared conve
+> ntion.
  log("K1: ghost-equation relation on the domain (derived, not selected)")
  Mg = ghost_equation_relation()
  Mg_ok = all(r is not None and all(x is not None for x in r) for r in Mg)
  zK = ckz["pole"]
  z_cb_derived = [sp.expand(sum(Mg[j][m] * zK[j] for j in range(len(zK)))) for m in range(len(Mg[0]))] if Mg
+> _ok else None
  derived_matches_p15 = bool(Mg_ok and all(sp.simplify(a - b) == 0 for a, b in zip(z_cb_derived, ccbz["pole"
+> ])))
  lam_derived = None
  for lam in (1, -1, I, -I):          # which factor makes the DERIVED c.cb data equal the P15 data used in
+> the probe
      if Mg_ok and all(sp.simplify(lam * a - b) == 0 for a, b in zip(z_cb_derived, ccbz["pole"])):
          lam_derived = lam
          break
  closure_with_derived = closure.get(str(lam_derived)) if lam_derived is not None else None
  # ---------------- p20 K1 FIX (disclosed erratum of p19): orientation.  The P15 Sigma coordinates are (cb,
+> c)-oriented;
  # the route-G slots are (c,cb).  Convert with the MEASURED orientation ratio, then test full sigma_0 closu
```

```
+> re of the
  # complete MS-bar counterterm (h.h + c.K + c.cb) with the DERIVED relative factor (+1) and Sigma held fixe
+> d.
  log("p20: orientation ratio, tree controls, full sigma_0 closure of the complete MS-bar counterterm")
  orient, orient_ok = orientation_ratio()
  orient_ok = bool(orient_ok and all(r is not None for r in orient) and len(set(orient)) == 1)
  sig_G = {pp: [z / r for z, r in zip(ccbz[pp], orient)] for pp in ccbz} if orient_ok else None
  tree = tree_controls()

  def full_image(zhh, zck, zcb, part="pole"):
      v = [sp.Integer(0)] * n
      for i, z in enumerate(zhh):
          v[lab.index(f"h.h#{i}")] = z
      for i, z in enumerate(zck):
          v[lab.index(f"c.K#{i}")] = z
      for i, z in enumerate(zcb):
          v[lab.index(f"c.cb#{i}")] = z
      img = G["_apply"](G["_sig"], [-x for x in v])
      return {"closed": not img, "entries": len(img), "sectors": sorted({f"{k[0]}.{k[1]}" for k in img})}
  zhh_full = hp["pole"]["z"]
  full_ct = full_image(zhh_full, ckz["pole"], sig_G["pole"]) if sig_G else {"closed": False, "entries": None
+> ,
                                                                              "sectors": ["orientation not est
+> ablished"]}
  full_ct_unconverted = full_image(zhh_full, ckz["pole"], ccbz["pole"])        # the p19 bookkeeping error
  full_ct_without_ccb = full_image(zhh_full, ckz["pole"], [0, 0])             # the p19 package S_ct (no c.c
+> b)
  # full-closure test as a MUTATION DETECTOR (Gamma1 data level; Sigma held fixed)

  def hh_z(graphs=("D01", "D03", "D11"), scale=None):
      AA = {g: ({k: (scale or {}).get(g, 1) * v for k, v in A[g].items()}) for g in graphs}
      return hh_parts(AA, list(graphs), hhb)["pole"]["z"]
  sg = sig_G["pole"] if sig_G else [0, 0]
  detector = {
      "drop_graph_D11": full_image(hh_z(("D01", "D03")), ckz["pole"], sg),
      "drop_graph_D13 (c.K data = 0, Sigma held fixed)": full_image(zhh_full, [0, 0, 0], sg),
      "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": full_image(zhh_full, [-x for x in ckz["pole"]], s
+> g),
      "drop_graph_D03": full_image(hh_z(("D01", "D11")), ckz["pole"], sg),
      "drop_graph_D01": full_image(hh_z(("D03", "D11")), ckz["pole"], sg),
      "flip_phase_D03": full_image(hh_z(scale={"D03": -1}), ckz["pole"], sg),
      "symmetry_factor_D03_x2": full_image(hh_z(scale={"D03": 2}), ckz["pole"], sg),
      "Sigma_orientation_unconverted (p19 error)": full_ct_unconverted}
  for k_, v_ in detector.items():
      v_["detected"] = not v_["closed"]
  # ---------------- mutations (each must change the outcome)
  log("mutations")
  muts = {}

  def mut(name, Xh, Xk, note):
      dm = assemble(Xh, Xk)
      o = classify(dm, imgs)
      muts[name] = {"outcome": o["outcome"], "changes_outcome": o["outcome"] != outcome["outcome"],
                    "residual": o.get("residual"), "note": note,
                    **({"finite_counterterm": o.get("local_solution")} if "local_solution" in o else {})}
  mut("drop_graph_D03", hh_row({g: A[g] for g in ("D01", "D11")}, op), Xkc, "hh row without the ghost loop D
+> 03")
  mut("drop_graph_D01", hh_row({g: A[g] for g in ("D03", "D11")}, op), Xkc, "hh row without the graviton bub
+> ble D01")
  mut("flip_phase_D03", hh_row(A, op, scale={"D03": -1}), Xkc, "relative sign of D03 flipped")
  mut("symmetry_factor_D03_x2", hh_row(A, op, scale={"D03": 2}), Xkc, "D03 doubled")
  mut("wrong_projection_trace_operator_hh", hh_row(A, H.trace_operator()), Xkc,
      "eta_mn p_v in place of s0 h (not the P.3 projection)")
  mut("wrong_projection_gauge_fixed_operator_Kc", Xhh, kc_row(A13, Egf),
      "de Donder q^2 P in place of the h^2-monomial operator E (not the P.3 projection)")
  mut("naive_4D_D03_evanescent_dropped", hh_row(A, op, drop_ev=("D03",)), Xkc, "D03 Gt parts dropped")
  mut("naive_4D_D13_Gt_dropped", Xhh, kc_row(A13_naive, E),
      "D13 evaluated with Gt dropped (P15_D13 mutation fit: T1 = -(d-4)/(16(d-1)))")
  insensitive = {}
  for name, (Xh, Xk, why) in {
          "drop_graph_D11": (hh_row({g: A[g] for g in ("D01", "D03")}, op), Xkc,
                             "D11 is transverse by itself (p_m G_D11 = 0 exactly in d): the slot cannot see
+> it; "
                             "covered by M2 (P18) and by the inventory refusal below"),
          "drop_graph_D13": (Xhh, {nn: sp.Integer(0) for nn in EXT},
                             "A_T1 = A_E1 = 0 and T2, T3, E2 are longitudinal: D13 reaches the slot with zer
+> o; "
                             "covered by the D13 certificate (two routes, mutations)"),
          "flip_row_sign_Kc": (Xhh, {nn: -x for nn, x in Xkc.items()}, "each row vanishes separately")}.item
+> s():
      o = classify(assemble(Xh, Xk), imgs)
```

```
      insensitive[name] = {"outcome": o["outcome"], "changes_outcome": o["outcome"] != outcome["outcome"],
                           "why_blind": why}
  missing_ctrl = {"D11_removed_from_inventory": {"inventory_ok": ok_missing,
                                                 "projected_defect_computed_if_removed": "not_earned" if not
                                                 ok_missing else "earned",
                                                 "changes_outcome": not ok_missing}}
  # ---------------- controls
  p14ct = jl("P14_counterterm_domain.json")["equivalences"]
  tr18 = jl("P18_hh_slot.json")["four_D_poles"]["transverse_basis_in_p14r_coordinates"]
  hh_idx = [lab.index(f"h.h#{i}") for i in range(5)]
  kerhh = [v for v in G["ker_P_ch_basis"] if all(v[i] == 0 for i in range(n) if i not in hh_idx)]
  span_eq = (sp.Matrix([[P(x) for x in r] for r in tr18]).rank() == 2 and
             sp.Matrix([[v[i] for i in hh_idx] for v in kerhh] + [[P(x) for x in r] for r in tr18]).rank() =
+> = 2
             and len(kerhh) == 2)
  controls = {
      "inventory_complete_no_missing": ok_inv,
      "missing_input_refused_not_zero": not ok_missing,
      "s0h_longitudinal": H.longitudinal_op(op),
      "E_transverse": E2M.transversality(E)["transverse"],
      "hh_row_zero_exact_in_d_all_7": all(x == 0 for x in Xhh.values()),
      "hh_row_evanescent_part_zero_all_7": all(x == 0 for x in Xhh_ev.values()),
      "hh_row_D01_D03_individually_nonzero": any(x != 0 for x in Xhh_g["D01"].values())
                                             and any(x != 0 for x in Xhh_g["D03"].values()),
      "Kc_row_zero_exact_in_d_all_7": all(x == 0 for x in Xkc.values()),
      "Kc_row_evanescent_part_zero_all_7": all(x == 0 for x in Xkc_ev.values()),
      "Kc_slot_map_nontrivial_T1_E1": any(x != 0 for x in T1_alone.values()) and any(x != 0 for x in E1_alon
+> e.values()),
      "E2_alone_zero_all_7": all(x == 0 for x in E2_alone.values()),
      "D13_A_T1_A_E1_zero_exact": A13["T1"] == 0 and A13["E1"] == 0,
      "onepoint_row_D14abc_vanishing": all(r["status"] == "vanishing_under_declared_prescription"
                                           for r in inv_rows if r["row"] == QC.ROW_1PT),
      "defect_zero_pole_nonlocal_local_all_7": is_zero(defect),
      "defect_independent_of_row_signs": all(signs.values()),
      "routeG_S0_check": G["S0_check"]["ok"],
      "routeG_defect_zero_by_sector": g_zero,
      "routeG_hh_local_data_in_span": all(hp[p]["in_span"] for p in hp),
      "routes_agree_on_ch_image_rank": rankC == G["P_ch_sigma0_rank_on_D"],
      "routeG_hh_kernel_equals_p18_transverse_subspace": span_eq,
      "D_cap_Im_inside_D_cap_ker_sigma0": G["Im_in_ker_sigma0"],
      "D_cap_Im_inside_ker_P_ch_sigma0": G["Im_in_ker_P_ch_sigma0"],
      "ghost_equation_relation_reproduces_P15_sigma_pole": derived_matches_p15,

      "MSbar_counterterm_in_ker_P_ch_sigma0": not ct_ch,
      "mutations_all_change_outcome": all(v["changes_outcome"] for v in muts.values()),
      "routeG_Q_split_reproduces_p14r_counts": (Gq["dim_D_cap_Im_sigma0"] == p14ct["domain_cap_image"]
                                                and Gq["quotient_dim"] == p14ct["quotient_dimension"]
                                                and Gq["image_rank_sigma0_on_Xi"] == p14ct["image_rank"]),
  }
  computed = all(controls[k] for k in ("inventory_complete_no_missing", "missing_input_refused_not_zero",
                                       "hh_row_D01_D03_individually_nonzero", "Kc_slot_map_nontrivial_T1_E1"
+> ,
                                       "routeG_S0_check", "routes_agree_on_ch_image_rank",
                                       "mutations_all_change_outcome"))
  identity = computed and all(controls.values()) and outcome["outcome"] == "ZERO"
  # projected_restoration_verified (distinct from identity): the RENORMALISED one-loop functional Gamma_R =
+> Gamma1 +
  # S_ct (S_ct = minus the MS-bar pole in the declared domain) satisfies the projected identity in every par
+> t, i.e.
  # P_ch sigma_0(S_ct) = 0 exactly, AND the linear solve for a finite symmetry-restoring counterterm in D re
+> turns
  # x = 0 (none required).  identity alone is a statement about the regulated Gamma1.
  p20_controls = {
      "orientation_ratio_uniform_minus_one": bool(orient_ok and orient[0] == -1),
      "tree_ghost_equation_V_cbc_eq_2iD_V_Kc": tree["ghost_equation_tree_V_cbc_eq_2iD_V_Kc"],
      "tree_sigma0_S0Kc_plus_S0cbc_zero": tree["ok"],
      "derived_ghost_relation_reproduces_P15_sigma": derived_matches_p15,
      "full_closure_detects_drop_D13_and_flip_Kc": bool(
          detector["drop_graph_D13 (c.K data = 0, Sigma held fixed)"]["detected"]
          and detector["flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)"]["detected"]),
      "full_closure_detects_p19_orientation_error": detector["Sigma_orientation_unconverted (p19 error)"]["d
+> etected"],
      "S_ct_without_ccb_not_closed (c.cb counterterm is needed)": not full_ct_without_ccb["closed"]}
  renormalised_defect_zero = identity and not ct_ch        # p19 criterion -> p20: route-consistency CONTROL
+>  only
  # p20 REDEFINITION (p19 reviewer: the p19 criterion follows from the identity + locality): full sigma_0 cl
+> osure
  # of the COMPLETE MS-bar counterterm S_ct (h.h + c.K + c.cb, Sigma held fixed, derived factor +1, orientat
+> ion
  # converted) in every ghost +1 sector (h.c, c.b, c.cbs).  Scope: the one-loop kappa^2 counterterm.
  restoration = bool(identity and full_ct["closed"] and lam_derived == 1 and all(p20_controls.values()))
```

```
  rec = {
      "landmark": "XVI-P", "revision": QC.REVISION,
      "deliverable": "P.5 identity: projected one-loop kappa^2 ch-identity defect, counterterm solve",
      "object": ("Delta[c,h] = ch projection (P.3, frozen) of the hbar kappa^2 part of (S_Psi, Gamma_Psi) =
+> "
                 "s_hh X_hh + s_Kc X_Kc + X_1pt (rows of ledgers/P13_ch_dependency_rows.json)"),
      "prescription": "declared (P14_prescription_D11_matching; DRED R1-R6; MS-bar; MB = 1/eps + 2 - L)",
      "inventory": inv_rows,
      "rows": {
          QC.ROW_HH: {"inputs": list(QC.ROWS[QC.ROW_HH]), "functional": "X_hh = int (s0 h)_(mn) G_hh^{(mn)(r
+> s)} h_(rs)",
                      "value_exact_d_times_MB": {k: s(v) for k, v in Xhh.items()},
                      "evanescent_part": {k: s(v) for k, v in Xhh_ev.items()},
                      "per_graph": {g: {k: s(v) for k, v in x.items()} for g, x in Xhh_g.items()}},
          QC.ROW_KC: {"inputs": ["D13"], "functional": "X_Kc = int G_Kc_{(mn) nu} c^nu E^{(mn)(rs)} h_(rs)",
                      "value_exact_d_times_MB": {k: s(v) for k, v in Xkc.items()},
                      "evanescent_part": {k: s(v) for k, v in Xkc_ev.items()},
                      "D13_coefficients": {k: s(v) for k, v in A13.items()}},
          QC.ROW_1PT: {"inputs": ["D14a", "D14b", "D14c"], "value": "0 (each one-point tadpole vanishes unde
+> r the "
                       "declared prescription, P15_batchA: evaluated, not Missing)"}},
      "external_assignments": {k: list(v) for k, v in EXT.items()},
      "defect": {k: {p: s(v[p]) for p in ("exact_d_times_MB", "pole", "nonlocal_L", "local")}
                 for k, v in defect.items()},
      "defect_zero_exact_in_d": is_zero(defect),
      "defect_row_sign_independence": signs,
      "evanescent_in_assembly": ("verified explicitly: the evanescent (gt) parts of the hh row (D01+D03+D11)
+>  and of the "
                                 "K.c row (D13 E1, E2) give ZERO for all seven external assignments, exactly
+>  in d; "
                                 "the evanescent poles themselves are nonzero (P18_hh_slot, P15_D13) -- they
+>  drop "
                                 "from the ch slots by the decoupling proofs, not by convention"),
      "route_G": {k: v for k, v in G.items() if not k.startswith("_")} | {
          "D_cap_ker_sigma0_basis": [[s(x) for x in v] for v in G["D_cap_ker_sigma0_basis"]],
          "ker_P_ch_basis": [[s(x) for x in v] for v in G["ker_P_ch_basis"]],
          "D_cap_Im_sigma0_basis": [[s(x) for x in v] for v in G["D_cap_Im_sigma0_basis"]]},
      "route_G_Q_split_p14r_convention": {k: v for k, v in Gq.items() if not k.startswith("_") and k not in
+> (
          "D_cap_ker_sigma0_basis", "ker_P_ch_basis", "D_cap_Im_sigma0_basis")},
      "field_note": ("Route G is computed over Q(i) (primary) and, for comparison, over Q with the re/im spl
+> it used "
                     "by the p14r domain certificate.  The p14r densities carry explicit factors i for odd d
+> erivative "
                     "counts (c.K: 3 derivatives), so over Q an i-multiple counts as independent: e.g. the c
+> h image "
                     "of c.K#0 is i times an h.h image.  The counts over Q(i) equal the real dimensions unde
+> r a "
                     "compatible real structure (one real coefficient per density, the i fixed by reality of
+>  the "
                     "action); ranks and solvability are quoted over Q(i)."),
      "gamma1_local_data_domain_coordinates": {
          "labels": lab, "convention": ("Ghat coordinates (G = kappa^2 N_eps Ghat MB) on the p14r densities;
+>  h.h "
                                        "from D01+D03+D11 (4D structures; D02 = D12 = 0), c.K from P15_D13;
+> c.cb "
                                        "(ghost self-energy, P15_D13) recorded separately: not a ch input "
                                        "(P_ch sigma_0 = 0 on c.cb); phi.phi not an input (sigma_0 = 0 on it
+> )"),
          **{p: [s(x) for x in v] for p, v in data.items()},
          "c_cb_ghost_self_energy (P15 (cb,c) orientation)": {p: [s(x) for x in v] for p, v in ccbz.items()}
+> ,
          "c_cb_ghost_self_energy (route-G (c,cb) orientation, p20)": (
              {p: [s(x) for x in v] for p, v in sig_G.items()} if sig_G else None)},
      "quotient_coordinates": {"complement_directions": G["quotient_complement"], **quot,
                               "reading": ("coordinates modulo D cap Im sigma_0 (field redefinitions / s-exa
+> ct); the "
                                           "h.h pole is transverse, i.e. in D cap Im sigma_0, so its quotien
+> t "
                                           "coordinates vanish")},
      "quotient_coordinates_including_Sigma": {pp: [s(x) for x in G["_quotient"](
          [v_ if not lab[i_].startswith("c.cb#") else sig_G[pp][int(lab[i_][-1])] for i_, v_ in enumerate(da
+> ta[pp])])]
          for pp in ("pole", "local")} if sig_G else None,
      "quotient_coordinates_Sigma_alone_routeG": {pp: [s(x) for x in G["_quotient"](
          [sig_G[pp][int(lab[i_][-1])] if lab[i_].startswith("c.cb#") else 0 for i_ in range(n)])]
          for pp in ("pole", "local")} if sig_G else None,
      "quotient_reading_p20": ("all in route-G (c,cb) labels: 'quotient_coordinates' is the class of the h.h
+>  + c.K "
                               "Gamma1 data WITHOUT the ghost self-energy, = -[Sigma] (c.cb entries (-3/8, 1
+> /4)); "
```

```
                                "Sigma itself has c.cb coordinates (+3/8, -1/4) (P15's (cb,c) numbers carry t
+> he "
                                "opposite sign); the complete data incl. Sigma have quotient 0, consistent wi
+> th "
                                "full closure and cohomology 0.  (The p19 reviewer read the p19 entries as Si
+> gma in "
                                "the wrong orientation; numerically they coincide with P15's (cb,c) Sigma.)")
+> ,
      "routeG_defect_by_sector": {p: {sec: len(v) for sec, v in per.items()} for p, per in g_def.items()},
      "counterterm_solve": {
          "system": "Delta + P_ch sigma_0(x) = 0, x in the declared domain D (11 directions), part by part",
          "rank_P_ch_sigma0_routeG": G["P_ch_sigma0_rank_on_D"], "rank_routeC": rankC,
          "routeC_image_relations": nullC,
          "outcome": outcome["outcome"], "residual": outcome.get("residual"),
          "solution": "x = 0 (no counterterm needed beyond the MS-bar poles); the identity-preserving finite
+>  "
                      "renormalisations are ker(P_ch sigma_0) cap D (dim %d), of which %d modulo D cap Im si
+> gma_0"
                      % (G["dim_D_cap_ker_P_ch_sigma0"], G["dim_D_cap_ker_P_ch_sigma0"] - G["dim_D_cap_Im_si
+> gma0"]),
          "MSbar_counterterm_ch_image_zero": not ct_ch,
          "MSbar_counterterm_full_sigma0_image_sectors": ct_sig_sectors,
          "full_closure_relative_factor_table_UNCONVERTED_p19_error": closure,
          "full_closure_reading": ("p20 (corrects p19): with the P15 Sigma coordinates converted from the (c
+> b,c) to the "
                                   "route-G (c,cb) orientation (measured ratio -1), the DERIVED factor +1 gi
+> ves full "
                                   "sigma_0 closure of the complete MS-bar counterterm (h.h + c.K + c.cb) in
+>  every "
                                   "ghost +1 sector; the p19 table above (factors 1, -1, i, -i on UNCONVERTE
+> D "
                                   "coordinates) is the bookkeeping error, kept for the erratum"),
          "K1_ghost_closure_convention": {
              "ghost_equation_operator": "2 i D^rho (d13.deDonder_apply), verified by P15: Sigma = 2 i D[G_K
+> c]",
              "M_2iD_cK_to_cbc": [[s(x) for x in r] if r else None for r in Mg],
              "z_cb_derived_from_cK_pole_cbc_orientation": [s(x) for x in z_cb_derived] if z_cb_derived else
+>  None,
              "z_cb_P15_sigma_fit_cbc_orientation": [s(x) for x in ccbz["pole"]],
              "derived_equals_P15": derived_matches_p15,
              "relative_factor_derived": s(lam_derived) if lam_derived is not None else None,
              "orientation_ratio_routeG_over_cbc": [s(x) for x in orient],
              "Sigma_pole_routeG_c_cb_coordinates": [s(x) for x in sig_G["pole"]] if sig_G else None,
              "Sigma_local_routeG_c_cb_coordinates": [s(x) for x in sig_G["local"]] if sig_G else None,
              "tree_controls": tree,
              "full_closure_complete_S_ct": full_ct,
              "full_closure_S_ct_without_ccb (p19 package S_ct)": full_ct_without_ccb,
              "full_closure_unconverted (p19 error)": full_ct_unconverted,
              "reading": ("ERRATUM to p19: the p19 statement 'closure fails with the derived factor, needs -
+> 1, "
                          "unresolved' was a Grassmann-orientation bookkeeping error (P15 (cb,c) coordinates
+>  placed "
                          "in route-G (c,cb) slots).  Corrected: derived +1, full closure holds (finding K1
+> of the "
                          "p19 independent rebuild).")},
          "note_full_sigma0": ("the h.h + c.K MS-bar counterterm ALONE has a nonzero (c, b) image (delta S0/
+> delta h "
                               "carries b); adding the c.cb ghost self-energy counterterm (correct orientati
+> on, "
                               "derived factor +1) closes it: see K1_ghost_closure_convention.full_closure_c
+> omplete_S_ct")},
      "D_cap_ker_sigma0": {"computed": True, "dim": G["dim_D_cap_ker_sigma0"], "dim_D_cap_Im_sigma0":
                           G["dim_D_cap_Im_sigma0"], "cohomology_dim": G["cohomology_dim"],
                           "sector_ranks": G["sector_ranks"]},
      "mutations": muts, "insensitive_variations": insensitive, "missing_control": missing_ctrl,
      "controls": controls,
      "why_zero": ("each surviving ch row vanishes separately in the projected slot; no counterterm cancella
+> tion and no "
                   "cancellation between rows is involved: (i) hh row (D01+D02+D03+D11+D12): full-row transv
+> erse "
                   "decoupling -- D01 and D03 are individually nonzero and cancel against each other in the
+> slot "
                   "(p17/p18 transverse sum), D11 is transverse by itself, D02 = D12 = 0; (ii) K.c row (D13)
+> : D13 "
                   "reaches the ch slot only through T1 and E1 (T2, T3, E2 are longitudinal, E transverse) a
+> nd "
                   "A_T1 = A_E1 = 0 exactly in d; (iii) one-point row: the tadpoles D14a-c vanish under the
+> declared "
                   "prescription.  The defect is therefore zero without any counterterm (solve x = 0)."),
      "rests_on": ["M2 / D11 matching verified on the 4D components only (D11_matching_verified_scope = '4D
+> "
```

```
                  "components'; evanescent components of the D11 matching: not applicable, not verified)",
                  "the evanescent decoupling proofs (P16 E2 decoupling, P17/P18 hh evanescent / full-row de
+> coupling)",
                  "the frozen P.3 projection and the P13 ch dependency ledger (three surviving rows)",
                  "the declared prescription (DRED R1-R6, MS-bar, P14_prescription_D11_matching)"],
      "flag_definitions": {
          "projected_defect_computed": ("Delta = P_ch[(S_Psi, Gamma_Psi)]_{hbar kappa^2} assembled from ever
+> y "
                                        "evaluated input of the three surviving rows (none Missing), exact i
+> n d, "
                                        "pole + nonlocal + MS-bar local parts, all seven external assignment
+> s"),
          "projected_identity_verified": ("Delta = 0 for the REGULATED one-loop functional Gamma1: pole, non
+> local "
                                          "and local parts vanish for all seven assignments, exactly in d, w
+> ith all "
                                          "controls true and every mutation changing the outcome"),
          "projected_restoration_verified": ("p20 definition (replaces p19's, which follows from the identit
+> y + "
                                             "locality): FULL sigma_0 closure of the COMPLETE one-loop kappa
+> ^2 MS-bar "
                                             "counterterm S_ct = -(pole) in the declared domain, h.h + c.K +
+>  c.cb "
                                             "(ghost self-energy Sigma held fixed at its P15 value, orientat
+> ion "
                                             "converted, derived relative factor +1), in every ghost +1 sect
+> or (h.c, "
                                             "c.b, c.cbs), together with projected_identity_verified.  Scope
+> : the "
                                             "one-loop kappa^2 counterterm only; NOT bulk symmetry, NOT O.1"
+> ),
          "bulk_symmetry_restored": "hard stop: False regardless (not claimed; O.1 NOT_CLOSED)"},
      "renormalised_defect_zero": renormalised_defect_zero,
      "route_consistency_control_p19_criterion": {
          "label": ("route-consistency control (the p19 restoration criterion, NOT a flag): P_ch sigma_0(S_c
+> t) = 0 for "
                    "the h.h + c.K MS-bar counterterm and the solve x = 0; implied by the identity + localit
+> y"),
          "holds": bool(renormalised_defect_zero and outcome["outcome"] == "ZERO")},
      "p20_controls": p20_controls,
      "full_closure_mutation_detector": {
          "test": "full sigma_0 closure (all ghost +1 sectors) of the complete MS-bar counterterm, Sigma hel
+> d fixed",
          "cases": detector,
          "reading": ("detects drop D13 and flip K.c (the p19 blind spots) through the c.b sector, and the h
+> h-row "
                      "mutations through h.c; drop D11 stays undetected (D11 transverse => its pole is sigma
+> _0-closed "
                      "by itself): covered by M2 (P18) and the p20 D11 normalisation pin (P20_D11_pin.json)"
+> )},
      "findings": {
          "J1_p14r_counterterm_domain_field": ("FINDING ABOUT THE p14r CONTRACT (contract NOT modified): the
+>  p14r "
                                               "counterterm-domain counts (Im sigma_0 3, quotient 8, D cap k
+> er 5, "
                                               "P_ch sigma_0 rank 4) are artefacts of a re/im split over Q (
+> the "
                                               "densities carry explicit i for odd derivative counts, so i-m
+> ultiples "
                                               "count as independent).  The counts over Q(i) -- sigma_0 rank
+>  5, D cap "
                                               "ker 6, D cap Im 6, cohomology 0, quotient 5, P_ch sigma_0 ra
+> nk 3 (= "
                                               "Route C) -- equal the real dimensions under a compatible rea
+> l "
                                               "structure.  The p14r certificate stays as released; both are
+>  reported"),
          "K1_ghost_closure_sign": ("p20 CORRECTION (erratum to p19): resolved -- an orientation bookkeeping
+>  error. "
                                    "With the P15 Sigma coordinates converted to the route-G (c,cb) orientat
+> ion "
                                    "(ratio -1), the derived factor +1 gives full sigma_0 closure of the com
+> plete "
                                    "MS-bar counterterm in h.c, c.b, c.cbs.  The p19 text 'closure fails / n
+> eeds -1 / "
                                    "unresolved' is retracted")},
      "projected_defect_computed": "earned" if computed else "not_earned",
      "projected_identity_verified": "earned" if identity else "not_earned",
      "projected_restoration_verified": "earned" if restoration else "not_earned",
      "bulk_symmetry_restored": False,
      "corrected_in_revision": "p20 (K1 orientation fix; restoration redefined; see the p20 erratum)",
```

```
      "scope": ("ch projection only (one c, one h; one loop; kappa^2; off-shell); the evanescent poles of th
  +> e hh and "
                "K.c Green functions remain regulated (not subtracted) and decouple from the ch slots; no st
  +> atement "
                "about higher orders or bulk symmetry.  projected_restoration_verified (p20 definition) addi
  +> tionally "
                "tests full sigma_0 closure of the complete one-loop kappa^2 MS-bar counterterm in all ghost
  +>  +1 "
                "sectors (h.c, c.b, c.cbs) -- a statement about that counterterm only, not bulk symmetry, no
  +> t O.1"),
      "authorised_d_substitutions": dict(sorted(dred.D_PROVENANCE.items())),
      "seconds": round(time.time() - t0, 1)}
  certio.write(CERT / f"{QC.CERT_PREFIX}_defect.json", rec)
  print(json.dumps({"outcome": outcome["outcome"], "computed": computed, "identity": identity,
                    "restoration": restoration, "controls": controls,
                    "mutations": {k: [v["outcome"][:40], v["changes_outcome"]] for k, v in muts.items()},
                    "kerD": G["dim_D_cap_ker_sigma0"], "ImD": G["dim_D_cap_Im_sigma0"], "quot_dim": G["quoti
  +> ent_dim"],
                    "Gq": {k: Gq[k] for k in ("dim_D_cap_ker_sigma0", "dim_D_cap_Im_sigma0", "quotient_dim",
                                              "P_ch_sigma0_rank_on_D", "image_rank_sigma0_on_Xi")},
                    "sector": G["sector_ranks"], "comp": G["quotient_complement"], "rankC": rankC,
                    "rankG": G["P_ch_sigma0_rank_on_D"], "quot": quot,
                    "K1": rec["counterterm_solve"]["K1_ghost_closure_convention"], "p20_controls": p20_contr
  +> ols,
                    "detector": {k: [v["closed"], v["sectors"]] for k, v in detector.items()},
                    "quot_incl_Sigma": rec["quotient_coordinates_including_Sigma"], "seconds": rec["seconds"
  +> ]},
                   indent=1, default=str))
  return 0 if identity else 1


if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1:]))
```

149 Source: release_p19.py (p19 layer gate, disclosed p20 revision)

[listing 116/133] code/p5_calc/release_p19.py

```
#!/usr/bin/env python3
"""XVI-P p19 release gate: P.5 identity step -- the projected one-loop kappa^2 ch-identity defect assembled fr
  +> om the
evaluated inputs (code/p5_calc/defect.py -> certificates/P19_defect.json), the counterterm solve in the declar
  +> ed
domain, D cap ker sigma_0; p18 reviewer findings I1-I3.

Runs AFTER the p14r, p15, p16, p17 and p18 gates (release_p18.py -> certificates/P18_release.json) and require
  +> s all
five accepted.  Recomputes:
  contract identity : p14r contract files byte-identical EXCEPT code/p5_entry/dred.py, which is byte-identical
  +>  to the
                      p18 release (no contract revision in p19; no new contract file);
  p18 layer         : d11.py / p18_conventions.py / release_p18.py byte-identical to the p18 zip; P18_release
  +> accepted
                      WITH quantum_inputs_evaluated == 'earned', D11 flags true (I1: the p18 gate itself did n
  +> ot gate
                      publication on quantum_inputs_evaluated; the p19 gate does, for the p18 layer and for it
  +> self);
  inventory         : re-read independently of defect.py (ledger rows; every input evaluated; no Missing);
  defect            : P19_defect.json -- every control true, every mutation changes the outcome, flags consist
  +> ent;
  earned flags      : projected_defect_computed, projected_identity_verified, projected_restoration_verified -
  +> - each
                      ONLY from the certificate and only if publication is accepted; bulk_symmetry_restored an
  +> d
                      symmetry_restored stay false regardless;
  hard stops        : QC.HARD_STOPS_P19.
Publication (I1) is gated on: execution_pass AND quantum inputs earned (p18 layer + recheck) AND
projected_defect_computed earned AND the flag implications (restoration => identity => computed).
Rebuild flags: author/independent for the CURRENT (p19) tree; independent_rebuild_passed_p18 from the archived
  +>  p18
independent report, bound to the p18 tree bfab3b4b... / zip 7c7d5451... (same-box caveat).
"""
from __future__ import annotations
import hashlib, json, sys
from pathlib import Path
```

```
HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p19_conventions as QC  # noqa: E402
import p15_conventions as Q15  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)

ARCH_P18 = "archival/p18"
ARCH_P18_INDEP = f"{ARCH_P18}/reproducibility/independent_rebuild_report_p18.json"
DEFECT_MUTATIONS = ("drop_graph_D03", "drop_graph_D01", "flip_phase_D03", "symmetry_factor_D03_x2",
                    "wrong_projection_trace_operator_hh", "wrong_projection_gauge_fixed_operator_Kc",
                    "naive_4D_D03_evanescent_dropped", "naive_4D_D13_Gt_dropped")
MIN_DEFECT_CONTROLS = 25
P5_CALC_STATUS = ("evaluated; 4D sector renormalized (MS-bar, declared domain); evanescent regulated (poles re
  +> tained, not "
                  "subtracted), decoupled from the ch slots")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def contract_identity():
    cur = {p.relative_to(ROOT).as_posix(): sha(p) for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    changed = sorted(k for k, h in Q15.P14R_CONTRACT_SHA256.items() if cur.get(k) != h)
    added = sorted(set(cur) - set(Q15.P14R_CONTRACT_SHA256))
    import contract_registry as CR          # p22 disclosed revision: registered additive p22 contract revisio
  +> n
    _dr = CR.dred_revision(ROOT)
    dred_same_as_p18 = cur.get("code/p5_entry/dred.py") == QC.P18_DRED_SHA256 or _dr["state"] == "p18 + p22 gu
  +> ard block"
    ok = bool(changed == ["code/p5_entry/dred.py"] and added == sorted(QC.NEW_CONTRACT_FILES_P19) and dred_sam
  +> e_as_p18)
    return ok, {"p14r_contract_files": len(Q15.P14R_CONTRACT_SHA256), "changed_vs_p14r": changed, "added": add
  +> ed,
                "dred_byte_identical_to_p18": dred_same_as_p18, "dred_registry_state_p22": _dr["state"], "cont
  +> ract_revision_in_p19": False,
                "disclosed_revisions_p19": QC.DISCLOSED_CONTRACT_REVISIONS_P19}


def p18_layer():
    cur = {k: sha(ROOT / k) if (ROOT / k).is_file() else None for k in QC.P18_CALC_SHA256}
    import contract_registry as CR          # p22 disclosed revision: registered p22 revision of release_p18.p
  +> y
    ident = all(CR.layer_file_ok(k, cur[k], h) for k, h in QC.P18_CALC_SHA256.items())
    r = jl("certificates/P18_release.json")
    q = r.get("quantum_inputs_evaluated") == "earned"
    ok = bool(ident and r.get("publication") == "accepted" and r.get("release_claim_accepted") is True
              and r.get("D11_matching_verified") is True and r.get("D11_evaluated") is True
              and r.get("hh_full_row_decoupling_proven") is True and r.get("hard_stops_hold") is True and q)
    return ok, {"p18_layer_files_byte_identical": ident, "P18_release_accepted": r.get("publication") == "acce
  +> pted",
                "P18_quantum_inputs_evaluated": r.get("quantum_inputs_evaluated"),
                "I1_gate": "the p19 gate requires the p18 layer record's quantum_inputs_evaluated == 'earned'"
  +> }, r


def p18_independent():
    f = ROOT / ARCH_P18_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P18_INDEP, "present": False, "pins_ok": False}
    rep = json.loads(f.read_text())
    live = ROOT / QC.P18_INDEPENDENT_REPORT
    rec = {"report": ARCH_P18_INDEP, "present": True, "report_sha256": sha(f),
           "report_sha256_pinned": sha(f) == QC.P18_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "bound_to_source_tree_sha256": rep.get("source_tree_sha256"),
           "bound_to_source_zip_sha256": rep.get("zip_sha256"),
           "tree_matches_p18": rep.get("source_tree_sha256") == QC.P18_SOURCE_TREE_SHA256,
           "zip_matches_p18": rep.get("zip_sha256") == QC.P18_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])), "findings": [x.get("id") for x in rep.get("findings",
  +>  [])],
           "caveat": rep.get("same_box_caveat"),
```

```
          "live_report_identical_to_archived": (live.read_bytes() == f.read_bytes()) if live.is_file()
          else "live report not in this tree",
          "NOT_inherited_by_p19": "bound to the p18 tree/zip; the p19 tree has its own (pending) independent
  +> flag"}
    rec["archived_p18_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P18_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p18_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p18"] and rec["zip_matches_p18"] and rec["mismatches"] == 0
              and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec


def inventory_recheck():
    """independent of defect.py: every graph of every surviving ch row; flags/statuses re-read; no Missing."""
    rows = [r for r in jl("ledgers/P13_ch_dependency_rows.json").get("rows", [])
            if str(r.get("status", "")).startswith("surviving")]
    required = {gid.split("_")[0]: r["id"] for r in rows for gid in r.get("graphs", [])}
    b = jl("certificates/P15_batchA.json").get("batchA_evaluated", {})
    claim = b.get("claim", "") if isinstance(b, dict) else ""
    c16 = jl("certificates/P16_batchC.json").get("graphs", {})
    d13 = jl("certificates/P15_D13.json").get("D13_evaluated", {})
    d11 = jl("certificates/P18_D11.json")
    inv = {g: bool(isinstance(b, dict) and b.get("execution") is True and g in claim)
           for g in ("D02", "D12", "D14a", "D14b", "D14c")}
    inv["D01"] = c16.get("D01", {}).get("evaluated") is True
    inv["D03"] = c16.get("D03", {}).get("evaluated") is True
    inv["D13"] = isinstance(d13, dict) and d13.get("execution") is True
    inv["D11"] = d11.get("D11_evaluated") is True
    sa = {g: v.get("status") for g, v in jl("certificates/P15_batchA.json").get("graphs", {}).items()}
    status = {**sa, "D01": c16.get("D01", {}).get("status"), "D03": c16.get("D03", {}).get("status"),
              "D13": "evaluated" if inv["D13"] else None, "D11": d11.get("status")}
    lst = [{"graph": g, "slot_row": required.get(g), "evaluated": inv.get(g, False), "status": status.get(g)}
           for g in sorted(set(required) | set(inv))]
    no_missing = all(x["status"] and "missing" not in str(x["status"]).lower()
                     and "unevaluated" not in str(x["status"]).lower() for x in lst)
    ok = bool(set(required) == set(inv) == set(QC.INVENTORY_P19) and len(set(required.values())) == 3
              and all(inv.values()) and no_missing)
    return ok, lst


def defect_check(inv_ok):
    c = jl(f"certificates/{QC.CERT_PREFIX}_defect.json")
    muts = c.get("mutations", {})
    ctrl = c.get("controls", {})
    flags = {k: c.get(k) for k in QC.EARNABLE_P19}
    cs = c.get("counterterm_solve", {})
    k1 = cs.get("K1_ghost_closure_convention", {})
    # the distinct definitions (flag_definitions in P19_defect.json), re-checked here from the recorded conten
  +> t
    content = {
        "projected_defect_computed": bool(len(c.get("defect", {})) == 7 and inv_ok),
        "projected_identity_verified": bool(c.get("defect_zero_exact_in_d") is True),
        # p20 definition: full sigma_0 closure of the complete MS-bar S_ct (incl. c.cb, Sigma held fixed)
        "projected_restoration_verified": bool(c.get("defect_zero_exact_in_d") is True
                                               and k1.get("full_closure_complete_S_ct", {}).get("closed") is T
  +> rue
                                               and str(k1.get("relative_factor_derived")) == "1"
                                               and k1.get("derived_equals_P15") is True
                                               and bool(c.get("p20_controls"))
                                               and all(v is True for v in c.get("p20_controls", {}).values()))
  +> }
    flags = {k: ("earned" if (v == "earned" and content[k]) else "not_earned") for k, v in flags.items()}
    implications = bool((flags["projected_restoration_verified"] != "earned"
                         or flags["projected_identity_verified"] == "earned")
                        and (flags["projected_identity_verified"] != "earned"
                             or flags["projected_defect_computed"] == "earned"))
    ok = bool(inv_ok and c.get("revision") == QC.REVISION and len(ctrl) >= MIN_DEFECT_CONTROLS
              and all(v is True for v in ctrl.values())
              and sorted(muts) == sorted(DEFECT_MUTATIONS) and all(v.get("changes_outcome") is True for v in m
  +> uts.values())
              and c.get("missing_control", {}).get("D11_removed_from_inventory", {}).get("changes_outcome") is
  +>  True
              and c.get("projected_defect_computed") == "earned" and implications
              and c.get("bulk_symmetry_restored") is False
              and c.get("D_cap_ker_sigma0", {}).get("computed") is True
              and len(c.get("defect", {})) == 7
              and set(c.get("flag_definitions", {})) >= set(QC.EARNABLE_P19) and bool(c.get("why_zero"))
              and "J1_p14r_counterterm_domain_field" in c.get("findings", {})
              and k1.get("derived_equals_P15") is True and k1.get("relative_factor_derived") is not None)
    return ok, c, flags, implications
```

```
def change_log():
    f = ROOT / QC.P18_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P18_ARCHIVED_RECORDS[QC.P18_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT) if r != SM.REFERENCE_FILE}
    contract = set(Q15.P14R_CONTRACT_SHA256) | {p.relative_to(ROOT).as_posix() for p in (ROOT / "code" / "p5_e
  +> ntry").glob("*.py")}
    added = sorted((set(new) - set(old)) | {SM.REFERENCE_FILE})
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])
    changed = sorted(added + removed + modified)
    return {"base": f"{QC.P18_SOURCE_LIST} (pinned; files of the p18 zip sha256 {QC.P18_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok,
            "reference_file": f"{SM.REFERENCE_FILE} listed as added by declaration (written after the author b
  +> uild)",
            "added": added, "removed": removed, "modified": modified,
            "contract_files_changed": [x for x in changed if x in contract],
            "non_contract_files_changed": [x for x in changed if x not in contract],
            "counts": {"added": len(added), "removed": len(removed), "modified": len(modified)}}


def milestones(p18r, calc_ok, dc, publish):
    st = dict(p18r.get("milestones", {}).get("status", {}))
    defs = dict(p18r.get("milestones", {}).get("definitions", {}))
    st["P.5_calculation"] = (P5_CALC_STATUS + " (p15-p18 inputs D01, D02, D03, D11, D12, D13, D14a-c)"
                             if calc_ok else "open (quantum inputs not accepted)")
    if publish and dc.get("projected_identity_verified") == "earned":
        st["P.5_identity"] = ("projected defect computed (zero exactly in d: pole, nonlocal and local parts, a
  +> ll seven "
                              "external assignments); projected identity verified; projected restoration verif
  +> ied for "
                              "the renormalised Gamma_R = Gamma1 + S_ct (ch projection, one loop, kappa^2 only
  +> ; "
                              "zero because each row vanishes separately, no counterterm cancellation); "
                              "projected restoration (p20 definition): full sigma_0 closure of the complete on
  +> e-loop "
                              "kappa^2 MS-bar counterterm incl. c.cb (K1 orientation corrected in p20; p19 'cl
  +> osure "
                              "fails' retracted); bulk symmetry NOT restored/claimed; 0.1 NOT_CLOSED")
    elif publish and dc.get("projected_defect_computed") == "earned":
        st["P.5_identity"] = "projected defect computed; identity NOT verified (see P19_defect.json)"
    else:
        st["P.5_identity"] = "open"
    return {"definitions": defs, "order": p18r.get("milestones", {}).get("order"), "status": st,
            "definition_vs_status_note": ("I3 (p18 reviewer): the P.5_calculation DEFINITION reads 'coefficien
  +> ts "
                                          "evaluated and renormalized'; the STATUS states exactly what holds:
  +> 4D "
                                          "sector renormalized, evanescent poles regulated and decoupled from
  +> the ch "
                                          "slots (not subtracted)"),
            "P5_entry_earned": p18r.get("P5_entry_earned"), "P3_P4_frozen_as_P5_inputs": p18r.get("P3_P4_froze
  +> n_as_P5_inputs"),
            "P4_is_NOT": "coupled one-loop restoration of the bulk symmetry (bulk_symmetry_restored false)"}


def tex(x):
    t = str(x).replace("**", "^").replace("\\", "/").replace("{", "\\{").replace("}", "\\}")
    t = t.replace("_", "\\_").replace("^", "\\^{}").replace("&", "\\&").replace("#", "\\#").replace("%", "\\%"
  +> )
    t = t.replace("$", "\\$").replace("~", "\\textasciitilde{}")
    return "\\texttt{" + t + "}"


def results_table(dc):
    drows = [f"{tex(n)} & {tex(v['pole'])} & {tex(v['nonlocal_L'])} & {tex(v['local'])} \\\\"
             for n, v in dc["defect"].items()]
    rG = dc["route_G"]
    q = dc["quotient_coordinates"]
    mrows = [f"{tex(k)} & {tex(v['outcome'][:44])} & {'yes' if v['changes_outcome'] else 'no'} \\\\"
             for k, v in dc["mutations"].items()]
    return ("% generated by code/p5_calc/release_p19.py -- do not edit\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{3.2cm}p{2.6cm}p{2.6cm}p{2.6cm}}\\hline\n"
            "assignment $(c,h)$ & defect pole & coeff. of L & local finite \\\\\\hline\n" + "\n".join(drows)
            + "\n\\hline\\end{tabular}\\end{center}\n"
            f"\\noindent Route G over {tex(rG['field'])}: rank $\\sigma_0|_D$ = {rG['sigma0_rank_on_D']}, "
            f"dim $D\\cap\\ker\\sigma_0$ = {rG['dim_D_cap_ker_sigma0']}, dim $D\\cap\\mathrm{{Im}}\\,\\sigma_0
  +> $ = "
            f"{rG['dim_D_cap_Im_sigma0']}, cohomology {rG['cohomology_dim']}, quotient {rG['quotient_dim']} "
```

```
            f"(complement {tex(', '.join(q['complement_directions']))}); rank $P_{{ch}}\\sigma_0$ = "
            f"{rG['P_ch_sigma0_rank_on_D']} (Route C: {dc['counterterm_solve']['rank_routeC']}).\n"
            f"Over Q with the re/im split (p14r convention): dim $D\\cap\\mathrm{{Im}}$ = "
            f"{dc['route_G_Q_split_p14r_convention']['dim_D_cap_Im_sigma0']}, quotient "
            f"{dc['route_G_Q_split_p14r_convention']['quotient_dim']}, dim $D\\cap\\ker$ = "
            f"{dc['route_G_Q_split_p14r_convention']['dim_D_cap_ker_sigma0']}.\n\n"
            f"\\noindent Quotient coordinates of the $\\Gamma_1$ local data: pole {tex(q['pole'])}, coeff. of
  +> L "
            f"{tex(q['nonlocal_L'])}, local {tex(q['local'])}.\n"
            "\\begin{center}\\scriptsize\\begin{tabular}{p{4.6cm}p{6.4cm}p{1.2cm}}\\hline\n"
            "mutation & outcome & changes \\\\\\hline\n" + "\n".join(mrows) + "\n\\hline\\end{tabular}\\end{ce
  +> nter}\n"
            f"\\noindent Counterterm solve: {tex(dc['counterterm_solve']['outcome'])}; "
            f"{tex(dc['counterterm_solve']['solution'])}.\n\n"
            "\\noindent\\textbf{Why zero.} " + tex(dc["why_zero"]) + "\n\n"
            "\\noindent\\textbf{Flag definitions.} " + " ".join(
                "\\textbf{" + tex(k) + "}: " + tex(v) + "." for k, v in dc["flag_definitions"].items()) + "\n\
  +> n"
            "\\noindent\\textbf{K1 (corrected in p20; erratum to p19).} derived relative factor "
            + tex(dc['counterterm_solve']['K1_ghost_closure_convention']["relative_factor_derived"])
            + "; orientation ratio (route-G (c,cb) over P15 (cb,c)) "
            + tex(dc['counterterm_solve']['K1_ghost_closure_convention']["orientation_ratio_routeG_over_cbc"])
            + "; Sigma pole in (c,cb) coordinates "
            + tex(dc['counterterm_solve']['K1_ghost_closure_convention']["Sigma_pole_routeG_c_cb_coordinates"]
  +> )
            + "; full sigma\\_0 image of the complete MS-bar counterterm: "
            + tex(dc['counterterm_solve']['K1_ghost_closure_convention']["full_closure_complete_S_ct"]) + ". "
            + tex(dc['counterterm_solve']['K1_ghost_closure_convention']["reading"]) + "\n\n"
            "\\noindent\\textbf{J1 (finding about the p14r contract).} "
            + tex(dc["findings"]["J1_p14r_counterterm_domain_field"]) + "\n")


def _main():
    id_ok, ident = contract_identity()
    l18_ok, l18, p18r = p18_layer()
    p18i_ok, p18i = p18_independent()
    inv_ok, inv = inventory_recheck()
    d_ok, dc, dflags, impl = defect_check(inv_ok)
    p14r, p15r, p16r, p17r = (jl(f"certificates/{n}_release.json") for n in ("P14", "P15", "P16", "P17"))
    hard = dict(QC.HARD_STOPS_P19)
    hard_ok = bool(all(hard[k] == v for k, v in QC.HARD_STOPS_P19.items())
                   and all(r.get("hard_stops_hold") is True for r in (p14r, p15r, p16r, p17r, p18r))
                   and dc.get("bulk_symmetry_restored", False) is False)
    cl = change_log()
    q_earned = bool(inv_ok and l18_ok and p18r.get("quantum_inputs_evaluated") == "earned")
    execution_pass = bool(id_ok and l18_ok and d_ok and hard_ok and cl["base_pinned_ok"] and p18i.get("pins_ok
  +> ") is True)
    publish = bool(execution_pass and q_earned and dflags.get("projected_defect_computed") == "earned" and imp
  +> l)   # I1
    author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
    indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
    m = milestones(p18r, q_earned, dc, publish)
    earned = {k: ("earned" if (publish and dflags.get(k) == "earned") else "not_earned") for k in QC.EARNABLE_
  +> P19}
    flags = dict(p18r.get("flags", {}))
    flags["projected_ch_identity"] = {"execution": d_ok, "claim": (dc.get("counterterm_solve", {}).get("outcom
  +> e")
                                                                   if d_ok else "not accepted"),
                                      "scope": dc.get("scope")}
    cert = {
        "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
  +> TE,
        "version_label": QC.VERSION_LABEL,
        "deliverable": "P.5 identity step (projected one-loop kappa^2 ch-identity defect; counterterm solve; I
  +> 1-I3)",
        "execution_pass": execution_pass,
        "claim_status": ("P.5_identity: projected defect assembled from every evaluated input (D01, D02, D03,
  +> D11, "
                         "D12, D13, D14a-c); outcome " + str(dc.get("counterterm_solve", {}).get("outcome")) +
                         ". WHY ZERO: " + str(dc.get("why_zero")) + " RESTS ON: " + "; ".join(dc.get("rests_on
  +> ", [])) +
                         ". FLAGS: " + ", ".join(f"{k}={v}" for k, v in earned.items()) +
                         " (definitions: flag_definitions). projected_restoration_verified (p20 definition) =
  +> full "
                         "sigma_0 closure of the COMPLETE one-loop kappa^2 MS-bar counterterm (h.h + c.K + c.c
  +> b; ghost "
                         "self-energy held fixed; derived relative factor +1; orientation converted) in every
  +> ghost +1 "
                         "sector (h.c, c.b, c.cbs): " + str(dc.get("counterterm_solve", {}).get(
                             "K1_ghost_closure_convention", {}).get("full_closure_complete_S_ct")) +
                         ". Scope: that counterterm only -- NOT bulk symmetry (bulk_symmetry_restored=False),
```

```
+> 0.1 "
                       "NOT_CLOSED. The p19 criterion (P_ch sigma_0(S_ct) = 0, x = 0) follows from the ident
+> ity + "
                       "locality and is kept only as a route-consistency control. ERRATUM (K1): the p19 text
+>  'full "
                       "closure fails with the derived factor, needs -1, unresolved' was a Grassmann-orienta
+> tion "
                       "bookkeeping error and is RETRACTED. J1: the p14r counterterm-domain counts are Q re/
+> im-split "
                       "artefacts; the Q(i) counts are the real dimensions under a compatible real structure
+>  "
                       "(finding about the p14r contract; contract not modified)")
      if publish else "P.5_identity: not accepted",
      "flag_definitions": dc.get("flag_definitions"),
      "projected_restoration_scope": ("p20 definition: full sigma_0 closure (h.c, c.b, c.cbs) of the complet
+> e one-loop "
                                      "kappa^2 MS-bar counterterm incl. the c.cb ghost self-energy counterte
+> rm (held "
                                      "fixed, derived factor +1); NOT bulk symmetry, NOT 0.1"),
      "route_consistency_control_p19_criterion": dc.get("route_consistency_control_p19_criterion"),
      "full_closure_mutation_detector": dc.get("full_closure_mutation_detector"),
      "p20_controls": dc.get("p20_controls"), "why_zero": dc.get("why_zero"), "rests_on": dc.get("rests_on")
+> ,
      "findings": dc.get("findings"),
      "K1_ghost_closure_convention": dc.get("counterterm_solve", {}).get("K1_ghost_closure_convention"),
      "scope": dc.get("scope"),
      "release_claim_accepted": publish,
      "release_claim_accepted_definition": ("p14r contract accepted AND p15..p18 layers accepted and byte-id
+> entical "
                                            "(p18 layer with quantum_inputs_evaluated earned) AND contract i
+> dentity "
                                            "(dred.py byte-identical to p18; no new contract file) AND the i
+> nventory "
                                            "recheck AND the defect certificate (all controls, all mutations
+>  change "
                                            "the outcome, flag implications) AND p19 hard stops"),
      "publication": "accepted" if publish else "REJECTED",
      "publication_gated_on": ["execution_pass", "quantum_inputs_evaluated (p18 layer + recheck)",
                               "projected_defect_computed", "flag implications restoration => identity => co
+> mputed"],
      "flags": flags,
      **earned,
      "projected_defect_summary": {k: dc.get(k) for k in ("defect_zero_exact_in_d", "evanescent_in_assembly"
+> ,
                                                          "quotient_coordinates", "D_cap_ker_sigma0", "field
+> _note")},
      "counterterm_solve": dc.get("counterterm_solve"),
      "defect_controls": dc.get("controls"), "defect_mutations": dc.get("mutations"),
      "defect_insensitive_variations": dc.get("insensitive_variations"),
      "quantum_inputs_evaluated": "earned" if (q_earned and publish) else "not_earned",
      "quantum_inputs_inventory": inv,
      "D11_matching_verified": p18r.get("D11_matching_verified"),
      "D11_matching_verified_scope": p18r.get("D11_matching_verified_scope"),
      "D11_evanescent_components_matching": p18r.get("D11_evanescent_components_matching"),
      "M3_status": p18r.get("M3_status"), "D11_evaluated": p18r.get("D11_evaluated"),
      "hh_full_row_decoupling_proven": p18r.get("hh_full_row_decoupling_proven"),
      "evanescent_hh_counterterm_added": False,
      "batchA_evaluated": p18r.get("batchA_evaluated"), "D13_evaluated": p18r.get("D13_evaluated"),
      "batchC_evaluated": p18r.get("batchC_evaluated"), "library_certified": p18r.get("library_certified"),
      "E2_status": p18r.get("E2_status"), "E2_decoupling_proven": p18r.get("E2_decoupling_proven"),
      "hh_evanescent_status": p18r.get("hh_evanescent_status"),
      "hh_evanescent_decoupling_proven": p18r.get("hh_evanescent_decoupling_proven"),
      "D01_evaluated": p18r.get("D01_evaluated"), "D03_evaluated": p18r.get("D03_evaluated"),
      "graphs": p18r.get("graphs"), "D12_status": p18r.get("D12_status"),
      "contract_identity": ident, "p18_layer": l18, "change_log_vs_p18": cl,
      "disclosed_test_revisions_p19": getattr(QC, "DISCLOSED_TEST_REVISIONS_P19", {}),
      "P5_entry_earned": p18r.get("P5_entry_earned"), "P5_entry_flags": p18r.get("P5_entry_flags"),
      "P3_P4_frozen_as_P5_inputs": p18r.get("P3_P4_frozen_as_P5_inputs"), "classical": p18r.get("classical")
+> ,
      "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": m,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p18": p18i_ok,
      "independent_rebuild_passed_p17": p18r.get("independent_rebuild_passed_p17"),
      "independent_rebuild_passed_p16": p18r.get("independent_rebuild_passed_p16"),
      "independent_rebuild_passed_p15": p18r.get("independent_rebuild_passed_p15"),
      "independent_rebuild_passed_p14r": p18r.get("independent_rebuild_passed_p14r"),
      "independent_rebuild_passed_p13": p18r.get("independent_rebuild_passed_p13"),
      "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p19
+> ) source "
                             "tree only (independent: pending until a reviewer report for this tree exists);
+>  "
```

```
                            f"independent_rebuild_passed_p18 refers to the p18 tree {QC.P18_SOURCE_TREE_SHA
  +> 256} / zip "
                            f"{QC.P18_SOURCE_ZIP_SHA256} (same-box caveat) and is not inherited by p19"),
        "p18_independent_rebuild": p18i,
        "p17_independent_rebuild": p18r.get("p17_independent_rebuild"),
        "p16_independent_rebuild": p18r.get("p16_independent_rebuild"),
        "p15_independent_rebuild": p18r.get("p15_independent_rebuild"),
        "p14r_independent_rebuild": p18r.get("p14r_independent_rebuild"),
        "p13_independent_rebuild": p18r.get("p13_independent_rebuild"),
        "M2_summary": p18r.get("M2_summary"), "D11_summary": p18r.get("D11_summary"),
        "hh_slot_summary": p18r.get("hh_slot_summary"), "D13_summary": p18r.get("D13_summary"),
        "E2_summary": p18r.get("E2_summary"), "batchC_summary": p18r.get("batchC_summary"),
        "one_point_slot_kappa1": p18r.get("one_point_slot_kappa1"),
        "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
  +> ource.py",
                            "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                            "author": author_rec, "independent": indep_rec, "p18_archive": ARCH_P18,
                            "kind": "package evidence (XVI-0 conv:package); not a mathematical certificate"},
        "XVI_P_registered": False, "XVI_M_sha256": p18r.get("XVI_M_sha256"), "XVI_M_pin_ok": p18r.get("XVI_M_p
  +> in_ok"),
        "symmetry_restored": False, "bulk_symmetry_restored": False, "O1_closure": "NOT_CLOSED",
        "insertion_status": "Frozen_negative_obstruction_preserved", "XV_B_opened": False, "XIV_J_opened": Fal
  +> se,
        "Route_B_discharged": False, "u_R_c_delta_added": False,
        "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
            n: certio.file_content_sha(ROOT / "certificates" / n, getattr(QC, "RUN_SPECIFIC_KEYS", ())) for n
  +> in
            ("P14_release.json", "P15_release.json", "P16_release.json", "P17_release.json", "P18_release.json
  +> ",
             f"{QC.CERT_PREFIX}_defect.json") if (ROOT / "certificates" / n).is_file()},
      }
      cdir = ROOT / "certificates"
      txt = certio.dumps(cert)
      if publish:
          (ROOT / "RELEASE_STATUS.json").write_text(txt)
          (cdir / "RELEASE_STATUS.json").write_text(txt)
          (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
          (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
          certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
              "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
  +> tatus)",
              **m, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
              "independent_rebuild_passed_p18": p18i_ok, "hard_stops": hard,
              "flags": {**earned, "quantum_inputs_evaluated": cert["quantum_inputs_evaluated"],
                        "bulk_symmetry_restored": False}})
          (ROOT / "manuscript" / "body").mkdir(parents=True, exist_ok=True)
          (ROOT / "manuscript" / "body" / "P19-results-table.tex").write_text(results_table(dc))
      else:
          (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
          for st in (ROOT / "RELEASE_STATUS.json", cdir / "RELEASE_STATUS.json"):     # p20: no stale status on
  +> reject
              st.unlink(missing_ok=True)
          (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(certio.dumps({
              "landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", "contract_identity_ok": i
  +> d_ok,
              "contract_identity": ident, "p18_layer_ok": l18_ok, "inventory_ok": inv_ok, "defect_ok": d_ok,
              "quantum_inputs_earned": q_earned, "flag_implications": impl,
              "p18_independent_pins_ok": p18i.get("pins_ok"), "hard_stops_hold": hard_ok,
              "change_log_base_ok": cl["base_pinned_ok"]}))
      print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
  +> ion"],
                        "contract_identity": id_ok, "changed_vs_p14r": ident["changed_vs_p14r"], "added": ident[
  +> "added"],
                        "p18_layer": l18_ok, "inventory": inv_ok, "defect": d_ok, "q": q_earned, **earned,
                        "hard_ok": hard_ok, "author": author_ok, "independent_p19": indep_ok,
                        "independent_p18": p18i_ok, "change_log_counts": cl["counts"]}, indent=1))
      return 0 if publish else 1


  def main():
      try:
          return _main()
      except Exception as exc:  # noqa: BLE001   fail closed
          cdir = ROOT / "certificates"
          (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
          for st in (ROOT / "RELEASE_STATUS.json", cdir / "RELEASE_STATUS.json"):     # p20: no stale status on
  +> reject
              st.unlink(missing_ok=True)
          (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
              {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED",
               "error": f"{type(exc).__name__}: {exc}"}, indent=1) + "\n")
```

```
        print(f"release_p19 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

150 Source: tests/test__p19.py

[listing 117/133] tests/test_p19.py

```
"""XVI-P p19 tests: the projected one-loop kappa^2 ch-identity defect (P19_defect: rows, seven assignments, ev
  +> anescent
decoupling in the assembly, Route C / Route G agreement, field convention J1, quotient coordinates re-derived
  +> here by
independent linear algebra, counterterm solve, controls / mutations / blind spots / missing control), I2 exten
  +> ded
static scan (pattern check, not a proof; planted violations; reviewed Poly.terms()/coeffs() sites), I1 fail-cl
  +> osed
gate (certificate mutations), I3 milestone wording, p19 status / hard stops / p18 independent record, change l
  +> og."""
from __future__ import annotations
import ast
import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools", "tests"):
    sys.path.insert(0, str(ROOT / p))
import contract_registry as CR  # p22 (disclosed): registered contract revision
import p19_conventions as QC  # noqa: E402
import p18_conventions as Q18  # noqa: E402
import p21_conventions as Q20  # noqa: E402   (p21: reviewed-site lists = p19 + p20 + p21; name kept)
import dred as DR  # noqa: E402

CERT = ROOT / "certificates"
PKG = ("code/p5_calc", "code/p5_entry")


def cj(name):
    return json.loads((CERT / name).read_text())


def rs():
    # p20 (disclosed): the p19 LAYER record (the top-level RELEASE_STATUS.json is the p20 status)
    return json.loads((ROOT / "certificates" / "P19_release.json").read_text())


def Z(text):
    return DR.parse_d(str(text), {"I": sp.I})


def V(lst):
    return sp.Matrix([Z(x) for x in lst])


# ============================================================== I2: extended static scan (pattern check, no
  +> t a proof)
ALIASABLE = {"eval", "exec", "compile", "__import__"}
REFLECT = {"vars", "globals", "locals"}
OPGET = {"attrgetter", "methodcaller"}
REBUILD = {"Add", "Mul", "Pow"}
SYMBOL_MAKERS = {"Symbol", "symbols", "Dummy", "var", "Wild"}
POLY_MAKERS = {"Poly", "poly", "as_poly"}
COEFF_ATTRS = {"terms", "coeffs", "monoms", "as_dict", "as_expr_dict"}


def _name(f):
    return f.attr if isinstance(f, ast.Attribute) else f.id if isinstance(f, ast.Name) else None
```

```
def _src(src, node):
    return ast.get_source_segment(src, node) or ""


def _d_like(node):
    """a Symbol name argument that is not a plain literal and could evaluate to 'd': any call (name built at r
  +> un
    time), a concatenation, or an f-string whose literal parts joined are '' or 'd' (literal characters always
    appear in the result, so f'dH{m}{n}_{r}' or f'{x}.{y}' can never be 'd')."""
    if isinstance(node, (ast.Call, ast.BinOp, ast.Name, ast.Subscript, ast.Attribute)):
        return isinstance(node, (ast.Call, ast.BinOp))
    if isinstance(node, ast.JoinedStr):
        lit = "".join(v.value for v in node.values if isinstance(v, ast.Constant) and isinstance(v.value, str)
  +> )
        return lit in ("", "d")
    return False


def scan2(path, root=ROOT):
    """I2 patterns (p18 reviewer gaps) on top of test_p18.scan: eval/exec aliases (assignment, import, builtin
  +> s
    attribute), vars()/globals()/locals() subscripts, operator.attrgetter/methodcaller, Add/Mul/Pow(*x.args)
    rebuilds, collect(..., evaluate=False), Poly/poly/as_poly with a non-literal generator expression that is
  +> a
    call (generator built at run time), Symbol/symbols/Dummy/Wild with a run-time name that could be 'd'
    (call, concatenation, f-string not starting with a non-d literal).  Also returns every Poly coefficient
    extraction site (.terms()/.coeffs()/.monoms()/.as_dict()) which must be on COEFF_SITES_REVIEWED."""
    src = path.read_text()
    tree = ast.parse(src)
    try:
        rel = path.relative_to(root).as_posix()
    except ValueError:
        rel = path.name
    bad, sites = [], []
    # names bound to a Poly/as_poly result anywhere in the module (I2: 'P = Poly(e, d); P(4)')
    poly_names = {t.id for n in ast.walk(tree) if isinstance(n, ast.Assign) and isinstance(n.value, ast.Call)
                  and _name(n.value.func) in POLY_MAKERS for t in n.targets if isinstance(t, ast.Name)}

    def flag(node, what):
        bad.append(f"{rel}:{node.lineno}:{what}")

    def visit(node, fn=None):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            fn = node.name
        if isinstance(node, ast.Call) and isinstance(node.func, ast.Name) and node.func.id in poly_names:
            flag(node, f"Poly held in {node.func.id} then called")
        if isinstance(node, ast.Assign) and _name(node.value) in ALIASABLE and isinstance(node.value, (ast.Nam
  +> e, ast.Attribute)):
            flag(node, f"alias of {_name(node.value)}")
        if isinstance(node, (ast.Import, ast.ImportFrom)):
            for a in node.names:
                if a.name.split(".")[-1] in ALIASABLE | OPGET:
                    flag(node, f"import {a.name}")
        if isinstance(node, ast.Attribute) and node.attr in ALIASABLE and _name(node.value) in ("builtins", "_
  +> _builtins__"):
            flag(node, f"builtins.{node.attr}")
        if isinstance(node, ast.Subscript) and isinstance(node.value, ast.Call) and _name(node.value.func) in
  +> REFLECT:
            flag(node, f"{_name(node.value.func)}()[..]")
        if isinstance(node, ast.Call):
            f = node.func
            name = _name(f)
            if name in OPGET:
                flag(node, name)
            if name == "get" and isinstance(f, ast.Attribute) and isinstance(f.value, ast.Call) \
                    and _name(f.value.func) in REFLECT:
                flag(node, f"{_name(f.value.func)}().get(..)")
            if name in REBUILD and any(isinstance(a, ast.Starred) and isinstance(a.value, ast.Attribute)
                                       and a.value.attr == "args" for a in node.args):
                flag(node, f"{name}(*x.args) rebuild")
            if name == "collect" and any(k.arg == "evaluate" and isinstance(k.value, ast.Constant)
                                         and k.value.value is False for k in node.keywords):
                flag(node, "collect(evaluate=False)")
            if name in POLY_MAKERS:
                gens = node.args[1:] if name != "as_poly" else node.args
                if any(isinstance(g, ast.Call) or (isinstance(g, ast.Starred) and isinstance(g.value, ast.Call
  +> ))
                       for g in gens) and f"{rel}::{fn}::{_src(src, node)}" not in QC.POLY_GENERATOR_CALL_SITE
  +> S_REVIEWED:
                    flag(node, f"{name}(generator built by a call)")
```

```
            sym_maker = name in SYMBOL_MAKERS and (name != "var" or _name(getattr(f, "value", None)) in (None,
  +>  "sp", "sympy"))
            if sym_maker and node.args and _d_like(node.args[0]):
                flag(node, f"{name}(run-time name)")
            if name in COEFF_ATTRS and isinstance(f, ast.Attribute) and not node.args:
                sites.append(f"{rel}::{fn}::{_src(src, node)}")
        for ch in ast.iter_child_nodes(node):
            visit(ch, fn)
    visit(tree)
    return bad, sites


def _pkg_files():
    return [f for d in PKG for f in sorted((ROOT / d).glob("*.py"))]


def test_i2_static_scan_package_clean_and_coeff_sites_reviewed():
    bad, sites = [], []
    for f in _pkg_files():
        b, s = scan2(f)
        bad += b
        sites += s
    assert not bad, bad
    rev = Q20.COEFF_SITES_REVIEWED
    assert not sorted(set(sites) - set(rev)), sorted(set(sites) - set(rev))
    assert not sorted(set(rev) - set(sites)), sorted(set(rev) - set(sites))
    assert all(v.startswith("generator") for v in rev.values())


def test_i2_p18_scan_still_clean_on_p19_package():
    import test_p18 as T18
    bad, sites = [], []
    for f in _pkg_files():
        b, s = T18.scan(*CR.scan_view(f))   # p22: p18 contract core
        bad += b
        sites += s
    assert not bad, bad
    assert set(sites) == set(Q18.PARSE_SITES_REVIEWED)


PLANTS2 = {
    "eval_alias_assign": "e = eval\ndef f(s):\n    return e(s)\n",
    "eval_builtins_attr": "import builtins\ndef f(s):\n    return builtins.eval(s)\n",
    "eval_import": "from builtins import eval as ev\n",
    "globals_subscript": "def f():\n    return globals()['d']\n",
    "locals_get": "def f(d):\n    return locals().get('d')\n",
    "sympy_var_call": "import sympy as sp\ndef f():\n    return sp.var(chr(100))\n",
    "symbol_fstring_only_var": "import sympy as sp\ndef f(k):\n    return sp.Symbol(f'{k}')\n",
    "vars_subscript": "import dred\ndef f():\n    return vars(dred)['d']\n",
    "attrgetter": "import operator\ndef f(e):\n    return operator.attrgetter('subs')(e)\n",
    "methodcaller": "from operator import methodcaller\ndef f(e, d):\n    return methodcaller('subs', d, 4)(e)
  +> \n",
    "poly_variable_then_called": "import sympy as sp\ndef f(e, d):\n    P = sp.Poly(e, d)\n    return P(4)\n",
    "as_poly_variable_then_called": "def f(e, d):\n    Q = e.as_poly(d)\n    return Q(4)\n",
    "vars_module_subscript": "import sympy as sp\ndef f(t):\n    return vars(sp)['sympify'](t)\n",
    "add_rebuild": "import sympy as sp\ndef f(e):\n    return sp.Add(*e.args[1:]), sp.Add(*e.args)\n",
    "mul_rebuild": "import sympy as sp\ndef f(e):\n    return sp.Mul(*e.args)\n",
    "collect_unevaluated": "import sympy as sp\ndef f(e, x):\n    return sp.collect(e, x, evaluate=False)\n",
    "poly_variable_call": "import sympy as sp\ndef f(e):\n    return sp.Poly(e, sp.Symbol(chr(100)))\n",
    "symbol_call_name": "import sympy as sp\ndef f():\n    return sp.Symbol(chr(100))\n",
    "symbol_fstring_d": "import sympy as sp\ndef f(k):\n    return sp.Symbol(f'd{k}')\n",
    "symbol_concat": "import sympy as sp\ndef f(k):\n    return sp.Symbol('d' + k)\n",
}


@pytest.mark.parametrize("name", sorted(PLANTS2))
def test_i2_scan_detects_planted_violation(tmp_path, name):
    f = tmp_path / f"{name}.py"
    f.write_text(PLANTS2[name])
    bad, _ = scan2(f, tmp_path)
    assert bad, name


def test_i2_scan_allows_benign_forms(tmp_path):
    f = tmp_path / "ok.py"
    f.write_text("import sympy as sp\ndef f(e, k, R):\n    a = sp.Symbol(f'dH{k}_0')\n    b = sp.symbols('p0:4
  +> ')\n"
                 "    c = sp.Symbol(f'{k}.{k}'), R.var(k), {k: v for k, v in locals().items()}\n"
                 "    P = sp.Poly(e, a, *b)\n    return sp.collect(e, a), sp.Add(*[a, b[0]]), P.terms()\n")
    bad, sites = scan2(f, tmp_path)
    assert not bad and sites == ["ok.py::f::P.terms()"]
```

```
def test_i2_unreviewed_coeff_site_is_reported(tmp_path):
    f = tmp_path / "new.py"
    f.write_text("import sympy as sp\ndef g(e, x):\n    return sp.Poly(e, x).coeffs()\n")
    _, sites = scan2(f, tmp_path)
    assert sites and not set(sites) <= set(Q20.COEFF_SITES_REVIEWED)


def test_i2_disclosure_wording():
    txt = (ROOT / "code" / "p5_calc" / "p19_conventions.py").read_text()
    assert "PATTERN CHECK, NOT A PROOF" in txt


# ============================================================== priority 1: the projected defect
def test_defect_inventory_complete():
    c = cj("P19_defect.json")
    inv = {x["graph"]: x for x in c["inventory"]}
    assert set(inv) == set(QC.INVENTORY_P19) and all(x["present"] and x["evaluated"] for x in inv.values())
    assert set(c["rows"]) == set(QC.ROWS) and len(QC.ROWS) == 3


def test_defect_zero_exact_in_d_all_seven_assignments_and_parts():
    c = cj("P19_defect.json")
    assert len(c["defect"]) == 7 and len(c["external_assignments"]) == 7
    assert "c hat, h tilde" in c["defect"]
    for a, v in c["defect"].items():
        for part in ("exact_d_times_MB", "pole", "nonlocal_L", "local"):
            assert Z(v[part]) == 0, (a, part)
    assert c["defect_zero_exact_in_d"] is True
    assert all(v == "true" or v is True for v in c["defect_row_sign_independence"].values())


def test_defect_evanescent_decoupling_in_assembly():
    c = cj("P19_defect.json")
    for row in (QC.ROW_HH, QC.ROW_KC):
        ev = c["rows"][row]["evanescent_part"]
        assert len(ev) == 7 and all(Z(x) == 0 for x in ev.values()), row
    assert c["evanescent_in_assembly"].startswith("verified explicitly")
    assert c["controls"]["hh_row_evanescent_part_zero_all_7"] is True
    assert c["controls"]["Kc_row_evanescent_part_zero_all_7"] is True


def test_defect_hh_row_nontrivial_cancellation():
    c = cj("P19_defect.json")
    pg = c["rows"][QC.ROW_HH]["per_graph"]
    assert any(str(x).strip() not in ("0", "") for x in pg["D01"].values())          # dot-product strings
    assert any(str(x).strip() not in ("0", "") for x in pg["D03"].values())
    assert c["controls"]["hh_row_D01_D03_individually_nonzero"] is True


def test_d13_slot_coefficients_from_p15():
    c = cj("P19_defect.json")
    co = c["rows"][QC.ROW_KC]["D13_coefficients"]
    assert Z(co["T1"]) == 0 and Z(co["E1"]) == 0
    d13 = cj("P15_D13.json")
    assert "D13_evaluated" in d13


def test_laurent_consistency_gamma1_local_data():
    g = cj("P19_defect.json")["gamma1_local_data_domain_coordinates"]
    assert len(g["pole"]) == len(g["nonlocal_L"]) == len(g["local"]) == 11
    assert all(Z(a) + Z(b) == 0 for a, b in zip(g["pole"], g["nonlocal_L"]))      # L-coefficient = -pole


def test_route_g_q_i_counts_and_cohomology():
    c = cj("P19_defect.json")
    g, q = c["route_G"], c["D_cap_ker_sigma0"]
    assert g["field"] == "Q(i)" and int(g["dim_D"]) == 11
    assert int(g["sigma0_rank_on_D"]) == 5 and int(g["dim_D_cap_ker_sigma0"]) == 6
    assert int(q["dim"]) == 6 and int(q["dim_D_cap_Im_sigma0"]) == 6 and int(q["cohomology_dim"]) == 0
    assert int(g["P_ch_sigma0_rank_on_D"]) == 3 and int(g["dim_D_cap_ker_P_ch_sigma0"]) == 8
    assert sum(int(s["dim"]) for s in q["sector_ranks"].values()) == 11
    # sector images overlap in the gh+1 space (c.cb#k + c.K#(k+1) in the kernel): per-sector ranks sum to more
    assert sum(int(s["sigma0_rank"]) for s in q["sector_ranks"].values()) >= int(g["sigma0_rank_on_D"])
    assert g["quotient_dim"] == 5 and g["Im_in_ker_sigma0"] is True


def test_field_convention_j1_q_split_reproduces_p14r():
    c = cj("P19_defect.json")
    s = c["route_G_Q_split_p14r_convention"]
```

```
    assert int(s["dim_D_cap_ker_sigma0"]) == 5 and int(s["dim_D_cap_Im_sigma0"]) == 3
    assert int(s["P_ch_sigma0_rank_on_D"]) == 4
    assert c["controls"]["routeG_Q_split_reproduces_p14r_counts"] is True
    # the mechanism: an i-multiple is dependent over Q(i) but independent after the re/im split over Q
    v = sp.Matrix([[1, sp.I], [sp.I, -1]])
    split = sp.Matrix([[sp.re(x) for x in v.row(r)] + [sp.im(x) for x in v.row(r)] for r in range(2)])
    assert v.rank() == 1 and split.rank() == 2


def test_d_cap_ker_basis_independent_and_contains_hh_pole():
    """independent linear algebra here: the certificate basis of D cap ker sigma_0 has rank 6 over Q(i), and t
  +> he hh
    part of the Gamma1 pole lies in its span (transverse => sigma_0-exact => zero quotient coordinates)."""
    c = cj("P19_defect.json")
    B = sp.Matrix.hstack(*[V(b) for b in c["route_G"]["D_cap_ker_sigma0_basis"]])
    assert B.shape == (11, 6) and B.rank() == 6
    pole = V(c["gamma1_local_data_domain_coordinates"]["pole"])
    hh = sp.Matrix(list(pole[:5]) + [0] * 6)
    assert sp.Matrix.hstack(B, hh).rank() == 6
    assert any(x != 0 for x in hh)


def test_quotient_coordinates_rederived():
    """reduce the Gamma1 data modulo D cap Im (= D cap ker here, cohomology 0) onto the complement
    (h.h#0, h.h#1, h.h#2, c.cb#0, c.cb#1): solve data = B y + C z for z, compare with the certificate."""
    c = cj("P19_defect.json")
    labels = c["gamma1_local_data_domain_coordinates"]["labels"]
    comp = c["quotient_coordinates"]["complement_directions"]
    assert comp == ["h.h#0", "h.h#1", "h.h#2", "c.cb#0", "c.cb#1"]
    B = sp.Matrix.hstack(*[V(b) for b in c["route_G"]["D_cap_ker_sigma0_basis"]])
    C = sp.Matrix.hstack(*[sp.Matrix([1 if l == k else 0 for l in labels]) for k in comp])
    M = sp.Matrix.hstack(B, C)
    assert M.rank() == 11
    for part in ("pole", "nonlocal_L", "local"):
        data = V(c["gamma1_local_data_domain_coordinates"][part])
        z = M.LUsolve(data)[6:, 0]
        want = [Z(x) for x in c["quotient_coordinates"][part]]
        assert [sp.nsimplify(sp.simplify(x)) for x in z] == want, part
    # the surviving quotient components are the ghost-sector (c.cb) ones; their pole equals minus the c.K pole
    q = [Z(x) for x in c["quotient_coordinates"]["pole"]]
    assert q[:3] == [0, 0, 0] and q[3:] == [sp.Rational(-3, 8), sp.Rational(1, 4)]


def test_counterterm_solve():
    s = cj("P19_defect.json")["counterterm_solve"]
    assert s["outcome"] == "ZERO" and s["residual"] is None
    assert int(s["rank_P_ch_sigma0_routeG"]) == int(s["rank_routeC"]) == 3
    assert s["MSbar_counterterm_ch_image_zero"] is True
    assert s["MSbar_counterterm_full_sigma0_image_sectors"] == ["c.b"]      # h.h + c.K only (without c.cb)
    # p20 (disclosed): the p19 factor table was computed on UNCONVERTED (cb,c) coordinates -- the K1 error
    cl = s["full_closure_relative_factor_table_UNCONVERTED_p19_error"]
    assert cl == {"1": False, "-1": True, "I": False, "-I": False}
    assert s["K1_ghost_closure_convention"]["full_closure_complete_S_ct"]["closed"] is True


def test_controls_mutations_blind_spots_missing_control():
    c = cj("P19_defect.json")
    assert len(c["controls"]) >= 25 and all(v is True for v in c["controls"].values())
    muts = c["mutations"]
    assert len(muts) == 8 and all(m["changes_outcome"] is True for m in muts.values())
    assert sum(m["outcome"].startswith("OBSTRUCTION") for m in muts.values()) == 7
    g = muts["naive_4D_D13_Gt_dropped"]
    assert g["outcome"].startswith("NONZERO, CANCELLABLE") and g["finite_counterterm"]["solvable"] is True
    assert int(g["finite_counterterm"]["rank"]) == 3
    blind = c["insensitive_variations"]
    assert set(blind) == {"drop_graph_D11", "drop_graph_D13", "flip_row_sign_Kc"}
    assert all(v["changes_outcome"] is False and v["why_blind"] for v in blind.values())
    m = c["missing_control"]["D11_removed_from_inventory"]
    assert m["inventory_ok"] is False and m["projected_defect_computed_if_removed"] == "not_earned"


def test_defect_flags_and_scope():
    c = cj("P19_defect.json")
    for k in QC.EARNABLE_P19:
        assert c[k] == "earned"
    assert c["bulk_symmetry_restored"] is False
    assert c["scope"].startswith("ch projection only")


# ============================================================ status (I1, I3)
def test_p19_release_status_and_flags():
```

```
    s = rs()
    assert s["revision"] == "p19" and s["publication"] == "accepted" and s["execution_pass"] is True
    for k in QC.EARNABLE_P19:
        assert s[k] == "earned", k
    assert s["quantum_inputs_evaluated"] == "earned"     # (p20: s is the p19 layer record)
    assert s["bulk_symmetry_restored"] is False and s["symmetry_restored"] is False
    assert s["O1_closure"] == "NOT_CLOSED" and s["XVI_P_registered"] is False and s["Route_B_discharged"] is F
  +> alse
    assert s["u_R_c_delta_added"] is False and s["XV_B_opened"] is False and s["XIV_J_opened"] is False
    assert s["evanescent_hh_counterterm_added"] is False
    assert s["insertion_status"] == "Frozen_negative_obstruction_preserved"
    assert s["XVI_M_sha256"].startswith("41aa1099") and s["XVI_M_pin_ok"] is True
    assert s["hard_stops"] == QC.HARD_STOPS_P19 and s["hard_stops_hold"] is True
    assert s["independent_rebuild_passed"] in ("pending", False)   # p20: the live p19 report is bound to the
  +> p19 tree
    assert s["contract_identity"]["added"] == [] and s["contract_identity"]["changed_vs_p14r"] == ["code/p5_en
  +> try/dred.py"]
    assert "quantum_inputs_evaluated (p18 layer + recheck)" in s["publication_gated_on"]


def test_i3_milestone_wording():
    st = rs()["milestones"]["status"]
    assert st["P.5_calculation"].startswith("evaluated; 4D sector renormalized (MS-bar, declared domain); evan
  +> escent "
                                     "regulated (poles retained, not subtracted), decoupled from the ch slots"
  +> )
    assert "renormalized" in st["P.5_calculation"] and "evanescent regulated" in st["P.5_calculation"]


def test_p18_independent_rebuild_recorded_and_not_inherited():
    s = rs()
    rec = s["p18_independent_rebuild"]
    assert s["independent_rebuild_passed_p18"] is True
    assert rec["report_sha256"] == QC.P18_INDEPENDENT_REPORT_SHA256 and rec["pins_ok"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P18_SOURCE_TREE_SHA256
    assert rec["bound_to_source_zip_sha256"] == QC.P18_SOURCE_ZIP_SHA256 and rec["mismatches"] == 0
    import source_manifest as SM
    assert SM.tree_sha(ROOT) != QC.P18_SOURCE_TREE_SHA256


def test_change_log_vs_p18():
    cl = rs()["change_log_vs_p18"]
    assert cl["base_pinned_ok"] is True
    base = json.loads((ROOT / QC.P18_SOURCE_LIST).read_text())["files"]
    import source_manifest as SM
    cur = {r: hashlib.sha256((ROOT / r).read_bytes()).hexdigest() for r in SM.source_files(ROOT) if r != SM.RE
  +> FERENCE_FILE}
    changed = sorted(set(cur) ^ set(base) | {k for k in set(cur) & set(base) if cur[k] != base[k]} | {SM.REFER
  +> ENCE_FILE})
    assert sorted(cl["added"] + cl["removed"] + cl["modified"]) == changed
    assert [x for x in cl["contract_files_changed"] if not CR.registered_contract_change(x)] == []   # p22: re
  +> gistered
    assert sorted(x for x in cl["modified"] if x.startswith("tests/") and not __import__("contract_registry").
  +> p22_revised(x)) == sorted(   # p22 filter
        x for x in QC.DISCLOSED_TEST_REVISIONS_P19 if not __import__("contract_registry").p22_revised(x))
    for f in ("code/p5_calc/defect.py", "code/p5_calc/release_p19.py", "code/p5_calc/p19_conventions.py",
              "tests/test_p19.py"):
        assert f in cl["added"], f


def test_p18_calc_layer_byte_identical():
    for k, h in QC.P18_CALC_SHA256.items():   # p22: or the registered p22 revision of that layer file
        assert CR.layer_file_ok(k, hashlib.sha256((ROOT / k).read_bytes()).hexdigest(), h), k


def test_version_label_p19():
    import build as B
    assert QC.VERSION_LABEL == "XVI-P p19" and B.QC.VERSION_LABEL == f"XVI-P {B.QC.REVISION}"   # p20: current
  +>  label
    assert "certificates/P19_defect.json" in B.VALUE_BLOCKS


# ============================================================== p19 gate mutations (I1: fail closed)
def _copy():
    d = Path(tempfile.mkdtemp(prefix="p19gate_"))
    for sub in ("code", "certificates", "ledgers", "exports", "tools", "archival", "reproducibility", "provena
  +> nce", "tests"):
        if (ROOT / sub).exists():
            shutil.copytree(ROOT / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__", "*.zip", "*.pdf"
  +> ))
    for f in ("README.md", "REBUILD_INSTRUCTIONS.md", "RELEASE_STATUS.json"):
```

```
        if (ROOT / f).exists():
            shutil.copy(ROOT / f, d / f)
    (d / "manuscript").mkdir()
    return d


def _edit_cert(d, name, fn):
    f = d / "certificates" / name
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c, indent=1))


def _p18_layer_q_not_earned(d):              # I1
    _edit_cert(d, "P18_release.json", lambda c: c.__setitem__("quantum_inputs_evaluated", "not_earned"))


def _inventory_hole_d13(d):                   # I1: in p18 this published with q not earned; p19 fails closed
    _edit_cert(d, "P15_D13.json", lambda c: c["D13_evaluated"].__setitem__("execution", False))


def _inventory_hole_d11(d):
    _edit_cert(d, "P18_D11.json", lambda c: c.__setitem__("D11_evaluated", False))


def _defect_not_computed(d):
    _edit_cert(d, "P19_defect.json", lambda c: c.__setitem__("projected_defect_computed", "not_earned"))


def _identity_without_computed(d):
    _edit_cert(d, "P19_defect.json", lambda c: c.__setitem__("projected_defect_computed", "not_earned"))
    _edit_cert(d, "P19_defect.json", lambda c: c.__setitem__("projected_identity_verified", "earned"))


def _defect_control_false(d):
    _edit_cert(d, "P19_defect.json", lambda c: c["controls"].__setitem__("E_transverse", False))


def _defect_mutation_silent(d):
    _edit_cert(d, "P19_defect.json", lambda c: c["mutations"]["flip_phase_D03"].__setitem__("changes_outcome",
  +>  False))


def _defect_mutation_removed(d):
    _edit_cert(d, "P19_defect.json", lambda c: c["mutations"].pop("wrong_projection_trace_operator_hh"))


def _missing_control_broken(d):
    _edit_cert(d, "P19_defect.json", lambda c: c["missing_control"]["D11_removed_from_inventory"].__setitem__(
        "changes_outcome", False))


def _bulk_claimed(d):
    _edit_cert(d, "P19_defect.json", lambda c: c.__setitem__("bulk_symmetry_restored", True))


def _assignment_dropped(d):
    _edit_cert(d, "P19_defect.json", lambda c: c["defect"].pop("c hat, h tilde"))


def _tamper_p18_layer(d):
    f = d / "code" / "p5_calc" / "d11.py"
    f.write_text(f.read_text() + "\n# tampered\n")


def _dred_code_change(d):
    f = d / "code" / "p5_entry" / "dred.py"
    f.write_text(f.read_text() + "\n_TAMPER = 1\n")


def _new_contract_file(d):
    (d / "code" / "p5_entry" / "evanescent_hh_counterterm.py").write_text("# undisclosed contract file\n")


def _tamper_p18_report(d):
    f = d / "archival" / "p18" / "reproducibility" / "independent_rebuild_report_p18.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


MUTS = [_p18_layer_q_not_earned, _inventory_hole_d13, _inventory_hole_d11, _defect_not_computed,
        _identity_without_computed, _defect_control_false, _defect_mutation_silent, _defect_mutation_removed,
```

```
        _missing_control_broken, _bulk_claimed, _assignment_dropped, _tamper_p18_layer, _dred_code_change,
        _new_contract_file, _tamper_p18_report]


def _run(d):
    return subprocess.run([sys.executable, "code/p5_calc/release_p19.py"], cwd=d, capture_output=True, text=Tr
  +> ue,
                          timeout=600)


@pytest.mark.parametrize("mut", MUTS, ids=lambda f: f.__name__)
def test_certificate_mutation_p19_gate_rejects(mut):
    d = _copy()
    try:
        mut(d)
        r = _run(d)
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d / "certificates" / "P19_release.json").exists()
        assert (d / "certificates" / "P19_release_rejected.json").exists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_p19_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = _run(d)
        assert r.returncode == 0, r.stdout[-1500:]
        assert json.loads((d / "RELEASE_STATUS.json").read_text())["quantum_inputs_evaluated"] == "earned"
    finally:
        shutil.rmtree(d, ignore_errors=True)


# ============================================================== honesty pass (user 07:12: a-d)
def test_a_why_zero_and_rests_on():
    c = cj("P19_defect.json")
    w = c["why_zero"]
    assert "each surviving ch row vanishes separately" in w and "no counterterm cancellation" in w
    assert "A_T1 = A_E1 = 0" in w and "tadpoles D14a-c vanish" in w and "transverse" in w
    r = " ".join(c["rests_on"])
    assert "4D components only" in r and "evanescent decoupling proofs" in r
    assert c["counterterm_solve"]["outcome"] == "ZERO"
    s = rs()
    assert "WHY ZERO" in s["claim_status"] and "RESTS ON" in s["claim_status"]


def test_b_flag_definitions_distinct_and_content_backed():
    c = cj("P19_defect.json")
    fd = c["flag_definitions"]
    assert set(fd) >= set(QC.EARNABLE_P19) | {"bulk_symmetry_restored"}
    # p20 (disclosed): restoration REDEFINED as full sigma_0 closure of the complete MS-bar S_ct (incl. c.cb)
    assert "REGULATED" in fd["projected_identity_verified"] and "COMPLETE" in fd["projected_restoration_verifi
  +> ed"]
    assert "NOT bulk symmetry" in fd["projected_restoration_verified"]
    assert fd["projected_identity_verified"] != fd["projected_restoration_verified"]
    assert c["renormalised_defect_zero"] is True and c["route_consistency_control_p19_criterion"]["holds"] is
  +> True
    assert c["counterterm_solve"]["MSbar_counterterm_ch_image_zero"] is True
    s = rs()
    assert s["flag_definitions"] == fd and s["bulk_symmetry_restored"] is False and s["O1_closure"] == "NOT_CL
  +> OSED"
    sc = s["projected_restoration_scope"]
    assert sc.startswith("p20 definition: full sigma_0 closure") and "NOT bulk symmetry, NOT 0.1" in sc
    assert "NOT bulk symmetry" in s["claim_status"] and "RETRACTED" in s["claim_status"]


def test_tex_escaping_of_results_table():
    import release_p19 as RP
    assert RP.tex("h.h#0 50% $x~{y}") == "\\texttt{h.h\\#0 50\\% \\$x\\textasciitilde{}\\{y\\}}"
    tab = (ROOT / "manuscript" / "body" / "P19-results-table.tex").read_text()
    import re
    assert not re.search(r"(?<!\\)#", tab)


def test_c_j1_disclosed_as_p14r_finding_contract_unchanged():
    c = cj("P19_defect.json")
    j = c["findings"]["J1_p14r_counterterm_domain_field"]
    assert j.startswith("FINDING ABOUT THE p14r CONTRACT (contract NOT modified)")
    p14 = cj("P14_counterterm_domain.json")                    # the released p14r certificate is untouched
    assert p14 and rs()["contract_identity"]["changed_vs_p14r"] == ["code/p5_entry/dred.py"]
    assert "J1" in rs()["claim_status"]
```

```
def test_d_k1_corrected_in_p20():
    """K1 (p20, disclosed revision; the p19 'closure fails / needs -1' conclusion is RETRACTED): the derived f
  +> actor +1
    closes the complete MS-bar counterterm once Sigma is converted to the route-G (c,cb) orientation (ratio -1
  +> )."""
    k = cj("P19_defect.json")["counterterm_solve"]["K1_ghost_closure_convention"]
    assert k["M_2iD_cK_to_cbc"] == [["0", "2"], ["-1", "0"], ["0", "-1"]]
    assert k["derived_equals_P15"] is True
    assert k["z_cb_derived_from_cK_pole_cbc_orientation"] == k["z_cb_P15_sigma_fit_cbc_orientation"] == ["-3/8
  +> ", "1/4"]
    assert k["relative_factor_derived"] == "1" and k["orientation_ratio_routeG_over_cbc"] == ["-1", "-1"]
    assert k["Sigma_pole_routeG_c_cb_coordinates"] == ["3/8", "-1/4"]
    assert k["full_closure_complete_S_ct"] == {"closed": True, "entries": 0, "sectors": []}
    assert k["full_closure_unconverted (p19 error)"]["sectors"] == ["c.b"]
    assert cj("P19_defect.json")["controls"]["ghost_equation_relation_reproduces_P15_sigma_pole"] is True
    cs = rs()["claim_status"]
    assert "RETRACTED" in cs and "selected by a scan" not in cs and "for which full closure fails" not in cs


def _restoration_without_content(d):        # p20: the content is full closure of the complete MS-bar S_ct
    _edit_cert(d, "P19_defect.json", lambda c: c["counterterm_solve"]["K1_ghost_closure_convention"][
        "full_closure_complete_S_ct"].__setitem__("closed", False))


def _k1_record_removed(d):
    _edit_cert(d, "P19_defect.json", lambda c: c["counterterm_solve"].pop("K1_ghost_closure_convention"))


def test_restoration_flag_withheld_without_content():
    d = _copy()
    try:
        _restoration_without_content(d)
        r = _run(d)
        if r.returncode == 0:      # may publish, but must NOT earn restoration
            assert json.loads((d / "RELEASE_STATUS.json").read_text())["projected_restoration_verified"] == "n
  +> ot_earned"
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_k1_record_required():
    d = _copy()
    try:
        _k1_record_removed(d)
        assert _run(d).returncode == 1
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

151 Source: p20_conventions.py (p20 constants, version label, pins, disclosures)

[listing 118/133] code/p5_calc/p20_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p20 conventions: CORRECTION release of p19 (no scope expansion), from the p19 independent-rebuild fin
  +> dings.

  K1  orientation fix: the P15 ghost self-energy Sigma is (cb,c)-oriented, the route-G c.cb slots are (c,cb);
  +> the
      Grassmann orientation ratio (-1) is measured and applied; the derived relative factor +1 then gives full
  +>  sigma_0
      closure of the complete one-loop kappa^2 MS-bar counterterm (p19 'closure fails / needs -1' RETRACTED, e
  +> rratum).
  R   projected_restoration_verified REDEFINED (p19 definition followed from identity + locality): full sigma_
  +> 0
      closure of the complete MS-bar S_ct incl. c.cb, Sigma held fixed; scope one-loop kappa^2 counterterm onl
  +> y.
  B   blind spots: full-closure mutation detector (drop D13, flip K.c); D11 normalisation pin (d11_pin.py ->
      P20_D11_pin.json: ratio pin derived; absolute pin OPEN).
  J1  wording; scan patterns extended (pattern check, not a proof); release reject leaves no stale RELEASE_STA
  +> TUS.

Layering.  p14r contract -> p15 .. p18 layers (byte-identical) -> p19 layer (code/p5_calc/{defect,p19_conventi
  +> ons,
release_p19}.py, P19_*; defect.py and release_p19.py are DISCLOSED p20 revisions, p19_conventions.py byte-iden
```

```
  +> tical)
-> p20 layer (code/p5_calc/{d11_pin,p20_conventions,release_p20}.py, P20_*).  No contract revision in p20.
"""
from __future__ import annotations
from p19_conventions import (ROW_HH, ROW_KC, ROW_1PT, ROWS, HH_EVALUATED, P18_CALC_SHA256,  # noqa: F401
                             P18_DRED_SHA256, RUN_SPECIFIC_KEYS)
from p19_conventions import COEFF_SITES_REVIEWED as _C19, POLY_GENERATOR_CALL_SITES_REVIEWED as _G19
from p19_conventions import HARD_STOPS_P19, EARNABLE_P19

REVISION = "p20"
VERSION_LABEL = "XVI-P p20"
CERT_PREFIX = "P20"
CONTRACT_REVISION = "p20 (p14r contract + disclosed p15/p16/p17/p18 revisions; none in p19 or p20)"
DATE = "2026-10-09"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p20.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p20.json"
P19_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p19.json"      # written by the reviewer;
  +>  never here
P19_INDEPENDENT_REPORT_SHA256 = "1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf008dd2811f45"
P19_SOURCE_TREE_SHA256 = "fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36"
P19_SOURCE_ZIP_SHA256 = "da9242752d2e3753d8b2ce7fbb21e7cdb28487cc7966c2ca52d1ef1376658875"
P19_INDEPENDENT_SAME_BOX = True     # disclosed: the p19 independent rebuild ran on the same box
P19_ARCHIVED_RECORDS = {
    "archival/p19/reference_values_p19.json": "08f10fa86305ccb03a57dbae65ec22b6991724931af7f954a8c4ca4aea9c44a
  +> f",
    "archival/p19/reproducibility/author_rebuild_report_p19.json":
        "5f0715f3b4ae756b261a0d757fd8247ede1bae1fc0c9ea8762635971b20ec1e4",
    "archival/p19/reproducibility/independent_rebuild_report_p19.json": P19_INDEPENDENT_REPORT_SHA256,
    "archival/p19/p19_source_files.json": "01b34cc820a3ceb42669ab2a534b82afb2fd6320634276093f8eae41914e37c6",
}
P19_SOURCE_LIST = "archival/p19/p19_source_files.json"
INVENTORY_P20 = ("D01", "D02", "D03", "D11", "D12", "D13", "D14a", "D14b", "D14c")
HARD_STOPS_P20 = dict(HARD_STOPS_P19)
EARNABLE_P20 = EARNABLE_P19 + ("D11_ratio_pin_verified",)
OPEN_PINS_P20 = ("D11_absolute_normalisation (Ghat -> a_2 map not derived)",)

# p19 layer in p20: p19_conventions.py byte-identical; defect.py, release_p19.py DISCLOSED revisions
P19_CONVENTIONS_SHA256 = "aa8560c22dd55fa6b4305368dad0cc95b9b5cf1ef6f50329d17ea2d03cd430af"
P19_RELEASED_SHA256 = {
    "code/p5_calc/defect.py": "4c154fb173b39aac7931234f78fa445b8039264f6ee07eb6e40785e349f7c8c5",
    "code/p5_calc/release_p19.py": "48dfe84616bab72b0c467642aec97998c26e9f24d51aeeef85f448fa00eb191b",
}
DISCLOSED_P19_LAYER_REVISIONS_P20 = {
    "code/p5_calc/defect.py": ("K1 orientation fix (measured ratio, Sigma converted to route-G (c,cb)); tree c
  +> ontrols; "
                               "full sigma_0 closure of the complete MS-bar S_ct; restoration REDEFINED; full-
  +> closure "
                               "mutation detector; p19 criterion relabelled route-consistency control; J1 word
  +> ing; "
                               "K1 erratum texts"),
    "code/p5_calc/release_p19.py": ("claim texts: K1 erratum (p19 'closure fails / needs -1' retracted); resto
  +> ration "
                                    "content checks on the p20 definition; reject removes stale RELEASE_STATUS
  +> .json"),
}
NEW_CONTRACT_FILES_P20 = ()
DISCLOSED_CONTRACT_REVISIONS_P20 = {}
DISCLOSED_TEST_REVISIONS_P20 = {
    "tests/test_p14.py": "names p19 -> p20; generated prefixes P20_",
    "tests/test_p19.py": ("K1 tests corrected (orientation, full closure with derived +1); restoration on the
  +> p20 "
                          "definition; J1 wording; p19 independent report bound to the p19 tree; scan sites =
  +> p19 + "
                          "p20 reviewed lists"),
}
COEFF_SITES_REVIEWED = dict(_C19)
COEFF_SITES_REVIEWED.update({
    "code/p5_calc/d11_pin.py::curvature_forms::sp.Poly(sp.expand(f - a * q2 - b * q0), *gens).coeffs()":
        "generators: metric perturbation symbols h_ab (no d anywhere in d11_pin)",
})
POLY_GENERATOR_CALL_SITES_REVIEWED = dict(_G19)
```

152 Source: d11_pin.py (p20 D11 pin; disclosed p21 revision: absolute pin derivation)

[listing 119/133] code/p5_calc/d11_pin.py

```
"""XVI-P p20 -- D11 normalisation pin (p19 independent-rebuild blind spot: 'drop D11' is invisible to the proj
  +> ected
identity and to full sigma_0 closure, because the D11 pole is transverse, hence sigma_0-closed by itself).

DERIVED here from package data (certificates/P18_D11.json, the released 4D UV pole of the D11 scalar loop):
  * the spin-2 / spin-0 decomposition of the 4D (non-evanescent) D11 pole with the Barnes-Rivers projectors, u
  +> sing
    the package's own structure convention (covariant.orbit over the symmetric pairs (m,n), (r,s));
  * transversality p^m T_mnrs = 0 (convention guard) and absence of spin-1 / spin-0w parts;
  * the quadratic forms of the curvature invariants on g = eta + h (linearised Riemann, built here):
    C^2 -> (1/2) p^4 h P2 h, R^2 -> 3 p^4 h P0s h, Gauss-Bonnet E -> 0.
COMPARED with an EXTERNAL literature value (not package-derived): the a_2 (Seeley-DeWitt) coefficient of a min
  +> imally
coupled real scalar, (1/120) C^2 + (1/2)(xi - 1/6)^2 R^2 (+ E, box R), xi = 0  =>  (1/120) C^2 + (1/72) R^2
[Birrell & Davies, Quantum Fields in Curved Space (1982), sec. 6.2; 't Hooft & Veltman, Ann. Inst. H. Poincare
  +>  A20
(1974) 69].  Prediction: spin-2 : spin-0 = (1/120)(1/2) : (1/72)(3) = 1/240 : 1/24 = 1 : 10.

PIN (ratio, derived + compared): c0 / c2 = 10 from the certificate.
ABSOLUTE PIN (p21, disclosed revision; p20 had it OPEN): absolute_pin() derives the sign and magnitude of the
  +> D11
pole from 1/2 Tr ln under the package's declared conventions (G = Gamma^(2) from amplitude_rules Gate 0 + G6;
  +> Wick
direction from N_eps = i/(16 pi^2) and the +1/eps bubble pole; g = eta + kappa h; eta = (+,-,-,-)) and compare
  +> s with
the heat-kernel a_2: exact equality (ratio 1) for both spins.  External inputs: mathematics only (a_2 values,
  +> the
heat-kernel pole formula); no external convention choice.
"""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(ROOT / "code" / "p5_entry"))
sys.path.insert(0, str(ROOT / "code" / "classical"))
import covariant as C  # noqa: E402
import dred  # noqa: E402   (guarded loader parse_d)
import p20_conventions as QC  # noqa: E402

CERT = ROOT / "certificates"
IDX = ("m", "n", "r", "s")
PAIRS = [("m", "n"), ("r", "s")]
LIT = {"C2": sp.Rational(1, 120), "R2_xi0": sp.Rational(1, 72),
       "source": ("Birrell & Davies, Quantum Fields in Curved Space (CUP 1982), sec. 6.2 (a_2 of a scalar fiel
  +> d); "
                  "'t Hooft & Veltman, Ann. Inst. H. Poincare A20 (1974) 69 -- EXTERNAL, not package-derived")
  +> }


def parse_key(key):
    """'g4_mr p_n p_s (p.p)^1' -> (factors, power of p.p); only 4D (g4, p) structures."""
    facs, e = [], 0
    for tok in key.split():
        if tok.startswith("(p.p)^"):
            e = int(tok[len("(p.p)^"):])
        elif tok.startswith("g4_"):
            facs.append(("g4", tok[3], tok[4]))
        elif tok.startswith("p_"):
            facs.append(("p", tok[2]))
        else:
            raise ValueError(f"not a 4D structure: {key}")
    return C.canon(facs), e


def tensor(coeffs, eta, p, scale=None):
    """T[m][n][r][s] from {key: coefficient} with the package orbit convention."""
    group = C.sym_group(PAIRS)
    pp = sum(eta[a] * p[a] ** 2 for a in range(4))
    T = {}
    for m in range(4):
        for n in range(4):
            for r in range(4):
                for s_ in range(4):
                    asg = dict(zip(IDX, (m, n, r, s_)))
```

```
                    v = 0
                    for key, c in coeffs.items():
                        facs, e = parse_key(key)
                        val = 0
                        for f in C.orbit(facs, group):
                            t = sp.Integer(1)
                            for x in f:
                                if x[0] == "g4":
                                    t *= eta[asg[x[1]]] if asg[x[1]] == asg[x[2]] else 0
                                else:
                                    t *= p[asg[x[1]]]
                            val += t
                        v += c * val * pp ** e
                    T[m, n, r, s_] = sp.expand(v)
    return T, pp


def projectors(eta, p, pp):
    def th(a, b):     # theta with both indices DOWN
        return (eta[a] if a == b else 0) - p[a] * p[b] / pp
    P2, P0, P1 = {}, {}, {}
    for m in range(4):
        for n in range(4):
            for r in range(4):
                for s_ in range(4):
                    S = (th(m, r) * th(n, s_) + th(m, s_) * th(n, r)) / 2
                    P0[m, n, r, s_] = th(m, n) * th(r, s_) / 3
                    P2[m, n, r, s_] = S - P0[m, n, r, s_]
    return P2, P0


def contract(A, B, eta):
    """A_mnrs B^mnrs (indices raised with eta, eta diagonal +-1)."""
    return sp.nsimplify(sp.expand(sum(A[k] * B[k] * eta[k[0]] * eta[k[1]] * eta[k[2]] * eta[k[3]] for k in A))
  +> )


def spin_parts(T, eta, p, pp):
    P2, P0 = projectors(eta, p, pp)
    n2, n0 = contract(P2, P2, eta), contract(P0, P0, eta)       # 5, 1
    c2 = sp.simplify(contract(T, P2, eta) / n2 / pp ** 2)
    c0 = sp.simplify(contract(T, P0, eta) / n0 / pp ** 2)
    resid = {k: sp.simplify(T[k] - pp ** 2 * (c2 * P2[k] + c0 * P0[k])) for k in T}
    trans = all(sp.simplify(sum(eta[m] * p[m] * T[m, n, r, s_] for m in range(4))) == 0
                for n in range(4) for r in range(4) for s_ in range(4))
    return {"c2": c2, "c0": c0, "n2": n2, "n0": n0, "transverse": trans,
            "only_spin2_spin0s": all(v == 0 for v in resid.values())}


def curvature_forms(p):
    """Euclidean quadratic forms (h_{-p} M h_p) of Riem^2, Ric^2, R^2 for g = delta + h, d -> i p; returned as
    coefficients on p^4 P2 and p^4 P0s (and the residual check)."""
    eta = (1, 1, 1, 1)
    pp = sum(x ** 2 for x in p)
    H = {}
    hs = {}
    for a in range(4):
        for b in range(a, 4):
            hs[a, b] = hs[b, a] = sp.Symbol(f"h{a}{b}")

    def riem(a, b, c, d_):    # linearised R_abcd = 1/2 (d_b d_c h_ad + d_a d_d h_bc - d_a d_c h_bd - d_b d_d
  +> h_ac); d -> i p
        return -sp.Rational(1, 2) * (p[b] * p[c] * hs[a, d_] + p[a] * p[d_] * hs[b, c] - p[a] * p[c] * hs[b, d
  +> _]
                                     - p[b] * p[d_] * hs[a, c])
    Rm = {(a, b, c, d_): riem(a, b, c, d_) for a in range(4) for b in range(4) for c in range(4) for d_ in ran
  +> ge(4)}
    Ric = {(b, d_): sum(Rm[a, b, a, d_] for a in range(4)) for b in range(4) for d_ in range(4)}
    R = sum(Ric[a, a] for a in range(4))
    # h_{-p} = h_p for real h and a quadratic form in p (even): the bilinear is the square
    forms = {"Riem2": sp.expand(sum(v ** 2 for v in Rm.values())), "Ric2": sp.expand(sum(v ** 2 for v in Ric.v
  +> alues())),
             "R2": sp.expand(R ** 2)}
    forms["C2"] = sp.expand(forms["Riem2"] - 2 * forms["Ric2"] + forms["R2"] / 3)
    forms["E_GaussBonnet"] = sp.expand(forms["Riem2"] - 4 * forms["Ric2"] + forms["R2"])
    P2, P0 = projectors(eta, p, pp)

    def hPh(P):
        return sp.expand(sum(hs[m, n] * P[m, n, r, s_] * hs[r, s_] for (m, n, r, s_) in P) * pp ** 2)
    q2, q0 = hPh(P2), hPh(P0)
    a, b = sp.symbols("a b")
```

```
    out = {}
    gens = sorted(set(hs.values()), key=str)
    for k, f in forms.items():
        sol = sp.solve(sp.Poly(sp.expand(f - a * q2 - b * q0), *gens).coeffs(), [a, b], dict=True)
        out[k] = {"P2": sp.nsimplify(sol[0][a]), "P0s": sp.nsimplify(sol[0][b])} if sol else None
    return out



# ------------------------------------------------------------------------------------------------ p21: ABSOLU
  +> TE pin
def curvature_forms_eta(p, eta):
    """quadratic forms (coefficients on (p.p)^2 h P2 h and (p.p)^2 h P0s h, all indices contracted with eta) o
  +> f
    Riem^2, Ric^2, R^2, C^2, E for g = eta + h at second order; d -> -i p (plane wave e^{-ip.x}); exact."""
    pp = sum(eta[a] * p[a] ** 2 for a in range(4))
    hs = {}
    for a in range(4):
        for b in range(a, 4):
            hs[a, b] = hs[b, a] = sp.Symbol(f"h{a}{b}")
    up = [sp.Integer(eta[a]) for a in range(4)]          # diagonal eta, eta^{-1} = eta

    def riem(a, b, c, d_):      # all lower; linearised; d_a d_b -> -p_a p_b
        return -sp.Rational(1, 2) * (p[b] * p[c] * hs[a, d_] + p[a] * p[d_] * hs[b, c] - p[a] * p[c] * hs[b, d
  +> _]
                                     - p[b] * p[d_] * hs[a, c])
    rng = range(4)
    Rm = {(a, b, c, d_): riem(a, b, c, d_) for a in rng for b in rng for c in rng for d_ in rng}
    Ric = {(b, d_): sum(up[a] * Rm[a, b, a, d_] for a in rng) for b in rng for d_ in rng}
    R = sum(up[a] * Ric[a, a] for a in rng)
    forms = {"Riem2": sp.expand(sum(v ** 2 * up[k[0]] * up[k[1]] * up[k[2]] * up[k[3]] for k, v in Rm.items())
  +> ),
             "Ric2": sp.expand(sum(v ** 2 * up[k[0]] * up[k[1]] for k, v in Ric.items())), "R2": sp.expand(R *
  +> * 2)}
    forms["C2"] = sp.expand(forms["Riem2"] - 2 * forms["Ric2"] + forms["R2"] / 3)
    forms["E_GaussBonnet"] = sp.expand(forms["Riem2"] - 4 * forms["Ric2"] + forms["R2"])
    P2, P0 = projectors(eta, p, pp)

    def hPh(P):                 # h_mn P^{mnrs} h_rs
        return sp.expand(sum(hs[m, n] * P[m, n, r, s_] * hs[r, s_] * up[m] * up[n] * up[r] * up[s_]
                             for (m, n, r, s_) in P) * pp ** 2)
    q2, q0 = hPh(P2), hPh(P0)
    a, b = sp.symbols("a b")
    gens = sorted(set(hs.values()), key=str)
    out = {}
    for k, f in forms.items():
        sol = sp.solve(sp.Poly(sp.expand(f - a * q2 - b * q0), *gens).coeffs(), [a, b], dict=True)
        out[k] = {"P2": sp.nsimplify(sol[0][a]), "P0s": sp.nsimplify(sol[0][b])} if sol else None
    return out


def absolute_pin(c2, c0, eta, p):
    """Exact derivation of the SIGN and MAGNITUDE of the D11 pole from 1/2 Tr ln (minimal scalar loop) under t
  +> he
    package's declared conventions, compared with the heat-kernel a_2.  Returns (record, pass)."""
    import integrals as IL
    import amplitude_rules as AR
    import p18_conventions as Q18
    import conventions as CC
    pi = sp.pi
    decl = {   # package declarations (checked verbatim; a change of any convention breaks the pin)
        "N_eps = i S_eps/(16 pi^2)": "N_eps = i S_eps/(16 pi^2)" in (IL.__doc__ or ""),
        "Gamma_1 = -i * graph sum": "Gamma_1 = -i * graph sum" in (AR.__doc__ or ""),
        "i Gamma_1 = int prefactor N/prod(q^2+i0)": "i Gamma_1^{(ext)} = int d^d l/(2pi)^d  prefactor * N(l,p)
  +> " in (AR.__doc__ or ""),
        "G6: G = kappa^2 N_eps Ghat, Ghat = -i prefactor/kappa^2 int N/(D1D2)": (
            "G = kappa^2 N_eps Ghat" in Q18.PHASE_CONVENTION_G6 and "Ghat = -i * prefactor/kappa^2" in Q18.PHA
  +> SE_CONVENTION_G6),
        "g = eta + kappa h": CC.METRIC_PARAMETRISATION.startswith("g_{mu nu} = eta_{mu nu} + kappa h_{mu nu}")
  +> ,
        "S_phi = (1/2) int sqrt(-g) g^{mu nu} d phi d phi (minimal, xi = 0)": CC.S_SCALAR.startswith(
            "S_phi = (1/2) int sqrt(-g) g^{mu nu} d_mu phi d_nu phi"),
        "eta = (+,-,-,-)": tuple(int(x) for x in eta) == (1, -1, -1, -1),
        "d = 4 - 2 eps; Ihat(1,1) pole = +1/eps": IL.master_series()[-1] == 1,
    }
    N = sp.I / (16 * pi ** 2)                          # N_eps at the pole (S_eps -> 1)
    # (1) G = Gamma_1^(2):  G = kappa^2 N_eps Ghat = -i pref int N/(D1D2) and i Gamma = pref int N/prod  =>  G
  +>  = Gamma
    G_over_Gamma = sp.simplify((-sp.I) * sp.I)        # = 1
    # (2) Wick direction from the package: int d^dl_M/(2pi)^d 1/((l^2+i0)((l+p)^2+i0)) = N_eps * (1/eps + ...)
    #     = +i x (positive Euclidean bubble 1/(16 pi^2 eps))  =>  d^dl_M = s_W i d^dl_E with s_W = +1 (l0 = i
```

```
  +> l4)
    s_W = sp.sign(sp.im(N)) * sp.sign(IL.master_series()[-1])
    # (3) dual rotation x0 = -i s_W x4: L_M = 1/2(phi_0^2 - grad^2) -> -L_E, d^4x_M = -i s_W d^4x_E  => i S_M
  +> = -s_W S_E
    f0, f4, fg = sp.symbols("phi_0 phi_4 phi_grad")
    LM = sp.Rational(1, 2) * (f0 ** 2 - fg ** 2)
    LM_rot = LM.subs(f0, sp.I * s_W * f4)                # d/dx0 = i s_W d/dx4
    LE = sp.Rational(1, 2) * (f4 ** 2 + fg ** 2)
    iSM_over_SE = sp.simplify(sp.I * (-sp.I * s_W) * LM_rot / LE)       # must be -1 (convergent e^{-S_E})
    # (4) e^{i Gamma_M} = e^{-Gamma_E}, Gamma_M = int d^4x_M L_M = -i s_W int d^4x_E L_M  =>  L_M^eff = -s_W L
  +> _E^eff
    LM_over_LE = -s_W
    # (5) heat kernel (EXTERNAL mathematics, Euclidean, positive operator): Tr ln Delta |_div = -(1/(16 pi^2 e
  +> ps)) int a_2,
    #     W_E = (1/2) Tr ln  =>  L_E,div = -a_2/(32 pi^2 eps); a_2 = C^2/120 + (1/2)(xi-1/6)^2 R^2 (+E, box R)
  +> , xi = 0
    LE_div_per_a2 = -1 / (32 * pi ** 2)
    LM_div_per_a2 = LM_over_LE * LE_div_per_a2         # coefficient of a_2/eps in L_M
    # (6) quadratic forms in Minkowski (eta of the package), g = eta + kappa h  -> kappa^2 h ... h
    cf = curvature_forms_eta(p, eta)
    # Gamma^(2) = 2 x (coefficient of the quadratic form)  (Gamma >= (1/2) h T h with T = Gamma^(2); W2 16-com
  +> ponent)
    pred2 = 2 * LM_div_per_a2 * (LIT["C2"] * cf["C2"]["P2"] + LIT["R2_xi0"] * cf["R2"]["P2"])
    pred0 = 2 * LM_div_per_a2 * (LIT["C2"] * cf["C2"]["P0s"] + LIT["R2_xi0"] * cf["R2"]["P0s"])
    # package: Gamma^(2)|_pole = G|_pole = kappa^2 N_eps Ghat_UV_pole / eps  (per kappa^2/eps)
    pkg2, pkg0 = sp.simplify(G_over_Gamma * N * c2), sp.simplify(G_over_Gamma * N * c0)
    r2 = sp.simplify(pkg2 / pred2) if pred2 != 0 else None
    r0 = sp.simplify(pkg0 / pred0) if pred0 != 0 else None
    ok = bool(all(decl.values()) and G_over_Gamma == 1 and s_W == 1 and iSM_over_SE == -1
              and cf["E_GaussBonnet"] == {"P2": 0, "P0s": 0} and r2 == 1 and r0 == 1)
    rec = {"package_declarations_checked": decl,
           "steps": {"1_G_equals_Gamma2": str(G_over_Gamma),
                     "2_wick_sign_s_W_from_N_eps_and_Ihat_pole": str(s_W),
                     "3_i_S_M_over_S_E (must be -1)": str(iSM_over_SE),
                     "4_L_M_eff_over_L_E_eff": str(LM_over_LE),
                     "5_L_M_div_per_a2_over_eps": str(LM_div_per_a2),
                     "6_minkowski_quadratic_forms": {k: ({kk: str(vv) for kk, vv in v.items()} if v else None)
                                                     for k, v in cf.items()}},
           "package_Gamma2_pole_per_kappa2_over_eps": {"spin2": str(pkg2), "spin0s": str(pkg0)},
           "heat_kernel_prediction_per_kappa2_over_eps": {"spin2": str(pred2), "spin0s": str(pred0)},
           "ratio_package_over_prediction": {"spin2": str(r2), "spin0s": str(r0)},
           "external_inputs": ["a_2 coefficients of a minimally coupled real scalar: C^2/120, (1/2)(xi-1/6)^2
  +> R^2 "
                               "(Birrell-Davies sec. 6.2; 't Hooft-Veltman 1974)",
                               "heat-kernel pole Tr ln Delta|_div = -(1/(16 pi^2 eps)) int a_2 (Euclidean, d =
  +>  4-2eps)"],
           "convention_choices_external": "none: every convention (G vs Gamma^(2), Minkowski i, Wick direction
  +> , "
                                          "sign of W_div, N_eps, kappa normalisation, signature) is fixed by t
  +> he "
                                          "package declarations checked above",
           "status": "PASS" if ok else "FAIL"}
    return rec, ok

def main(argv=None):
    t0 = time.time()
    cert = json.loads((CERT / "P18_D11.json").read_text())
    res = cert["result"]
    four = {k: dred.parse_d(str(v["UV_pole"]), {"I": sp.I}) for k, v in res.items() if not v["evanescent"]}
    eta = tuple(int(x) for x in C.ETA)
    p = (sp.Rational(3), sp.Rational(1), sp.Rational(2), sp.Rational(-5, 7))        # generic, p.p != 0
    T, pp = tensor(four, eta, p)
    sp_ = spin_parts(T, eta, p, pp)
    ratio = sp.simplify(sp_["c0"] / sp_["c2"]) if sp_["c2"] != 0 else None
    p_e = (sp.Rational(2), sp.Rational(-1), sp.Rational(3), sp.Rational(5, 3))
    cf = curvature_forms(p_e)
    lit2, lit0 = LIT["C2"] * cf["C2"]["P2"], LIT["R2_xi0"] * cf["R2"]["P0s"]
    lit_ratio = sp.simplify(lit0 / lit2)
    common = sp.simplify(sp_["c2"] / lit2) if lit2 else None
    common0 = sp.simplify(sp_["c0"] / lit0) if lit0 else None
    ratio_pin = bool(sp_["transverse"] and sp_["only_spin2_spin0s"] and ratio is not None and ratio == lit_rat
  +> io
                     and cf["C2"]["P0s"] == 0 and cf["E_GaussBonnet"] == {"P2": 0, "P0s": 0})

    absrec, abs_ok = absolute_pin(sp_["c2"], sp_["c0"], eta, p)

    def mutated(scale_all=None, drop=False, swap=False):
        co = {} if drop else {k: (scale_all or 1) * v for k, v in four.items()}
        if swap:     # exchange the two g4 g4 structures (a convention error)
            a_, b_ = "g4_mn g4_rs (p.p)^2", "g4_mr g4_ns (p.p)^2"
            co[a_], co[b_] = co[b_], co[a_]
```

```
      Tm, _ = tensor(co, eta, p)
      r = spin_parts(Tm, eta, p, pp)
      rr = sp.simplify(r["c0"] / r["c2"]) if r["c2"] != 0 else None
      ok = bool(r["transverse"] and r["only_spin2_spin0s"] and rr == lit_ratio)
      _, abs_m = absolute_pin(r["c2"], r["c0"], eta, p) if (r["c2"] != 0 and r["c0"] != 0) else (None, False
 +> )
      return {"c2": str(r["c2"]), "c0": str(r["c0"]), "ratio": str(rr), "ratio_pin_passes": ok,
              "detected_by_ratio_pin": not ok, "absolute_pin_passes": bool(abs_m),
              "detected_by_absolute_pin": not abs_m}
  muts = {"drop_graph_D11": mutated(drop=True), "D11_normalisation_x2": mutated(scale_all=2),
          "D11_sign_flip": mutated(scale_all=-1), "swap_g4g4_structures": mutated(swap=True)}
  rec = {"landmark": "XVI-P", "revision": QC.REVISION,
         "deliverable": "D11 normalisation pin via the trace-anomaly / C^2 coefficient (p19 reviewer blind s
 +> pot)",
         "source_certificate": "certificates/P18_D11.json", "graph": "D11 (hh scalar loop, minimal coupling
 +> h.phi^2)",
         "input_4D_UV_pole": {k: str(v) for k, v in four.items()},
         "orbit_convention": "covariant.orbit over the symmetric pairs (m,n), (r,s) (as in the D11 fit)",
         "spin_decomposition": {"c2_spin2": str(sp_["c2"]), "c0_spin0s": str(sp_["c0"]),
                                "projector_norms": [str(sp_["n2"]), str(sp_["n0"])],
                                "transverse": sp_["transverse"], "only_spin2_and_spin0s": sp_["only_spin2_sp
 +> in0s"],
                                "ratio_c0_over_c2": str(ratio)},
         "curvature_quadratic_forms_derived": {k: ({kk: str(vv) for kk, vv in v.items()} if v else None)
                                               for k, v in cf.items()},
         "literature_a2_minimal_scalar": {"C2": str(LIT["C2"]), "R2_xi0": str(LIT["R2_xi0"]), "source": LIT[
 +> "source"],
                                          "predicted_spin2": str(lit2), "predicted_spin0s": str(lit0),
                                          "predicted_ratio": str(lit_ratio)},
         "ratio_pin": {"status": "PASS" if ratio_pin else "FAIL",
                       "statement": ("spin-0 : spin-2 of the D11 4D UV pole = 10 : 1, derived from the certi
 +> ficate; "
                                     "equals the minimal-scalar a_2 prediction (1/72)(3) : (1/120)(1/2)")},
         "absolute_pin": {"status": "PASS" if abs_ok else "FAIL",
                          "observed_common_factor_c2_vs_bare_a2": str(common),
                          "observed_common_factor_c0_vs_bare_a2": str(common0),
                          "derivation_p21": absrec,
                          "statement": ("p21: SIGN and MAGNITUDE derived exactly from 1/2 Tr ln of the minim
 +> al scalar "
                                        "loop under the package's declared conventions (G = Gamma^(2); N_eps
 +>  = i/(16 "
                                        "pi^2) and the +1/eps bubble pole fix the Wick direction, i S_M = -S
 +> _E, L_M = "
                                        "-L_E; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^
 +> 2/(16 pi^2 "
                                        "eps) [p^4 P2/240 + p^4 P0s/24], equal to the package pole kappa^2 N
 +> _eps "
                                        "Ghat/eps exactly (ratio 1, both spins).  External inputs are mathem
 +> atics "
                                        "only (a_2 coefficients; heat-kernel pole formula); no external conv
 +> ention "
                                        "choice.  p20 had left this OPEN.")},
         "mutations": muts,
         "reading": ("drop D11 and a structure swap are detected by the ratio pin; an overall x2 or a sign f
 +> lip keeps "
                     "the ratio and is detected by the absolute pin (p21) and, for x2, by M2 (P18)"),
         "corrected_in_revision": "p21 (absolute pin derived; p20 had it OPEN)",
         "D11_absolute_pin_verified": abs_ok,
         "D11_ratio_pin_verified": ratio_pin,
         "c2_spin2": str(sp_["c2"]), "c0_spin0s": str(sp_["c0"]), "ratio_c0_over_c2": str(ratio),
         "ratio_pin_status": "PASS" if ratio_pin else "FAIL", "absolute_pin_status": "PASS" if abs_ok else "
 +> FAIL",
         "seconds": round(time.time() - t0, 1)}
  import certio
  certio.write(CERT / f"{QC.CERT_PREFIX}_D11_pin.json", rec)
  print(json.dumps({k: rec[k] for k in ("spin_decomposition", "ratio_pin", "absolute_pin", "mutations",
                                        "curvature_quadratic_forms_derived")}, indent=1, default=str))
  return 0 if (ratio_pin and abs_ok) else 1


if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1:]))
```

153 Source: release_p20.py (p20 layer gate, disclosed p21 revision)

[listing 120/133] code/p5_calc/release_p20.py

```
#!/usr/bin/env python3
"""XVI-P p20 release gate: CORRECTION release of p19 (no scope expansion), from the p19 independent-rebuild fi
  +> ndings.

Runs AFTER the p14r .. p19 gates (release_p19.py -> certificates/P19_release.json, the p19 LAYER record) and r
  +> equires:
  contract identity : as in p19 (p14r contract + dred.py byte-identical to p18; no new contract file);
  p18 layer         : byte-identical (pinned sha256);
  p19 layer         : p19_conventions.py byte-identical; defect.py and release_p19.py differ from the p19 zip
  +> ONLY as
                      the disclosed p20 revisions (QC.DISCLOSED_P19_LAYER_REVISIONS_P20); P19_release accepted
  +>  with
                      quantum_inputs_evaluated and projected_defect_computed earned;
  K1 correction     : P19_defect.json (p20 content): orientation ratio -1, Sigma in route-G (c,cb) coordinates
                      (+3/8, -1/4), derived factor +1, full sigma_0 closure of the complete MS-bar S_ct, tree
  +> controls,
                      every p20 control true, full-closure detector catches drop D13 and flip K.c;
  retraction        : the retracted p19 K1 / J1 phrases appear nowhere in README.md, the P19 note, P19_defect.
  +> json or
                      P19_release.json (they may appear only quoted inside the erratum);
  D11 pin           : P20_D11_pin.json ratio pin PASS (derived); absolute pin PASS (derived in p21 from 1/2 Tr
  +>  ln
                      under the declared conventions; p20 had it OPEN); the top-level *_pin_status keys (types
  +> et in
                      the PDF) must equal the nested statuses; drop D11 / x2 / sign flip detected;
  p19 independent   : archived report pinned (1b01f1d8...), PASS, bound to tree fcf90354... / zip da924275...
                      (same-box caveat) -> independent_rebuild_passed_p19; the p20 independent flag is pending
  +> ;
  hard stops        : QC.HARD_STOPS_P20 (bulk_symmetry_restored false regardless; 0.1 NOT_CLOSED).
p21 DISCLOSED REVISION (p20 independent-rebuild findings): (i) absolute pin PASS required, top-level / nested
  +> status
consistency checked; (ii) run standalone, the gate no longer trusts the P19 layer record alone: it re-runs the
  +>  p18
layer check (P18 quantum_inputs_evaluated == 'earned', p18 files byte-identical) and the graph inventory reche
  +> ck.
Publication fails closed (I1 style).  On reject: no P20_release.json and NO RELEASE_STATUS.json is left behind
  +> .
"""
from __future__ import annotations
import hashlib, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p20_conventions as QC  # noqa: E402
import p19_conventions as Q19  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)
import release_p19 as R19  # noqa: E402   (helpers only; main() not run)

ARCH_P19 = "archival/p19"
ARCH_P19_INDEP = f"{ARCH_P19}/reproducibility/independent_rebuild_report_p19.json"
# p19 texts retracted by the p20 erratum (K1 orientation error; J1 wording)
RETRACTED_PHRASES = (
    "full c.b / ghost-sector closure FAILS",
    "full c.b / ghost-sector closure of the MS-bar counterterm FAILS",
    "for which full closure fails",
    "with +1 full closure fails",
    "full closure \\emph{fails}",
    "This is unresolved",
    "This is not resolved in p19",
    "needs relative factor -1; the derived ghost-equation relation gives +1; not resolved",
    "factor +1 (K1) full c.b closure FAILS",
    "full c.b closure NOT established (K1)",
    "the physical field",
)
RETRACTION_SCOPE = ("README.md", "manuscript/notes/P19_defect.tex", "certificates/P19_defect.json",
                    "certificates/P19_release.json", "manuscript/body/P19-results-table.tex")
STATUS_FILES = ("RELEASE_STATUS.json", "certificates/RELEASE_STATUS.json")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}
```

```
def p19_layer():
    conv_ok = sha(ROOT / "code/p5_calc/p19_conventions.py") == QC.P19_CONVENTIONS_SHA256
    revs = {k: {"sha256": sha(ROOT / k), "differs_from_p19": sha(ROOT / k) != h,
                "disclosed": k in QC.DISCLOSED_P19_LAYER_REVISIONS_P20} for k, h in QC.P19_RELEASED_SHA256.ite
  +> ms()}
    r = jl("certificates/P19_release.json")
    l18_ok, l18 = R19.p18_layer()[:2]           # p21: recheck the lower layers (do not trust the P19 record a
  +> lone)
    inv_ok, _inv = R19.inventory_recheck()
    ok = bool(conv_ok and all(v["disclosed"] for v in revs.values()) and l18_ok and inv_ok
              and r.get("revision") == Q19.REVISION and r.get("publication") == "accepted"
              and r.get("release_claim_accepted") is True and r.get("quantum_inputs_evaluated") == "earned"
              and r.get("projected_defect_computed") == "earned" and r.get("hard_stops_hold") is True
              and r.get("bulk_symmetry_restored") is False)
    return ok, {"p19_conventions_byte_identical": conv_ok, "disclosed_p20_revisions": revs,
                "P19_release_accepted": r.get("publication") == "accepted",
                "lower_layers_rechecked_p21": {"p18_layer_ok": l18_ok,
                                               "P18_quantum_inputs_evaluated": l18.get("P18_quantum_inputs_eva
  +> luated"),
                                               "inventory_recheck_ok": inv_ok}}, r


def k1_check():
    c = jl("certificates/P19_defect.json")
    k1 = c.get("counterterm_solve", {}).get("K1_ghost_closure_convention", {})
    det = c.get("full_closure_mutation_detector", {}).get("cases", {})
    ctrl = c.get("p20_controls", {})
    chk = {
        "content_revision_p20": str(c.get("corrected_in_revision", "")).startswith("p20"),
        "orientation_ratio_minus_one": k1.get("orientation_ratio_routeG_over_cbc") == ["-1", "-1"],
        "Sigma_pole_routeG_c_cb": k1.get("Sigma_pole_routeG_c_cb_coordinates") == ["3/8", "-1/4"],
        "derived_factor_plus_one": str(k1.get("relative_factor_derived")) == "1",
        "derived_relation_reproduces_P15": k1.get("derived_equals_P15") is True,
        "full_closure_complete_S_ct": k1.get("full_closure_complete_S_ct", {}).get("closed") is True,
        "without_ccb_not_closed": k1.get("full_closure_S_ct_without_ccb (p19 package S_ct)", {}).get("closed")
  +>  is False,
        "tree_controls_ok": k1.get("tree_controls", {}).get("ok") is True,
        "p20_controls_all_true": bool(ctrl) and all(v is True for v in ctrl.values()),
        "detector_drop_D13": det.get("drop_graph_D13 (c.K data = 0, Sigma held fixed)", {}).get("detected") is
  +>  True,
        "detector_flip_Kc": det.get("flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)", {}).get("detected") i
  +> s True,
        "detector_drop_D11_recorded_blind": det.get("drop_graph_D11", {}).get("detected") is False,
        "route_consistency_control_holds": c.get("route_consistency_control_p19_criterion", {}).get("holds") i
  +> s True,
        "quotient_incl_Sigma_zero": all(x == "0" for v in (c.get("quotient_coordinates_including_Sigma") or {"
  +> x": ["?"]})
                                        .values() for x in v),
        "quotient_Sigma_alone_c_cb_plus_3_8_minus_1_4": (c.get("quotient_coordinates_Sigma_alone_routeG") or {
  +> })
        .get("pole", [None] * 5)[3:] == ["3/8", "-1/4"],
        "J1_wording": "compatible real structure" in str(c.get("findings", {}).get("J1_p14r_counterterm_domain
  +> _field")),
        "restoration_definition_p20": "COMPLETE" in str(c.get("flag_definitions", {})
                                                       .get("projected_restoration_verified", "")),
        "bulk_false": c.get("bulk_symmetry_restored") is False}
    return all(chk.values()), chk, c


def retraction_check():
    hits = {}
    for rel in RETRACTION_SCOPE:
        f = ROOT / rel
        if not f.is_file():
            hits[rel] = ["missing file"]
            continue
        t = f.read_text()
        found = [ph for ph in RETRACTED_PHRASES if ph in t]
        if found:
            hits[rel] = found
    return not hits, {"phrases": list(RETRACTED_PHRASES), "scope": list(RETRACTION_SCOPE), "hits": hits}


def d11_pin_check():
    c = jl(f"certificates/{QC.CERT_PREFIX}_D11_pin.json")
    m = c.get("mutations", {})
    chk = {"ratio_pin_PASS": c.get("ratio_pin", {}).get("status") == "PASS",
           "ratio_10": c.get("spin_decomposition", {}).get("ratio_c0_over_c2") == "10",
           "transverse": c.get("spin_decomposition", {}).get("transverse") is True,
```

```
          "C2_quadratic_form_half_P2": c.get("curvature_quadratic_forms_derived", {}).get("C2") == {"P2": "1/
  +> 2", "POs": "O"},
          "GaussBonnet_zero": c.get("curvature_quadratic_forms_derived", {}).get("E_GaussBonnet") == {"P2": "
  +> 0", "POs": "O"},
          "absolute_pin_PASS": c.get("absolute_pin", {}).get("status") == "PASS",
          "absolute_pin_flag": c.get("D11_absolute_pin_verified") is True,
          "top_level_absolute_status_consistent": c.get("absolute_pin_status") == c.get("absolute_pin", {}).g
  +> et("status"),
          "top_level_ratio_status_consistent": c.get("ratio_pin_status") == c.get("ratio_pin", {}).get("statu
  +> s"),
          "absolute_ratios_one": (c.get("absolute_pin", {}).get("derivation_p21", {})
                                  .get("ratio_package_over_prediction") == {"spin2": "1", "spin0s": "1"}),
          "no_external_convention_choice": str(c.get("absolute_pin", {}).get("derivation_p21", {})
                                               .get("convention_choices_external", "")).startswith("none"),
          "drop_D11_detected": m.get("drop_graph_D11", {}).get("detected_by_ratio_pin") is True,
          "x2_detected_by_absolute": m.get("D11_normalisation_x2", {}).get("detected_by_absolute_pin") is Tru
  +> e,
          "sign_flip_detected_by_absolute": m.get("D11_sign_flip", {}).get("detected_by_absolute_pin") is Tru
  +> e,
          "corrected_in_p21": str(c.get("corrected_in_revision", "")).startswith("p21"),
          "literature_source_stated": bool(c.get("literature_a2_minimal_scalar", {}).get("source")),
          "revision": c.get("revision") == QC.REVISION}
    return all(chk.values()), chk, c


def p19_independent():
    f = ROOT / ARCH_P19_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P19_INDEP, "present": False, "pins_ok": False}
    rep = json.loads(f.read_text())
    live = ROOT / QC.P19_INDEPENDENT_REPORT
    rec = {"report": ARCH_P19_INDEP, "present": True, "report_sha256": sha(f),
          "report_sha256_pinned": sha(f) == QC.P19_INDEPENDENT_REPORT_SHA256,
          "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
          "bound_to_source_tree_sha256": rep.get("source_tree_sha256"), "bound_to_source_zip_sha256": rep.get
  +> ("zip_sha256"),
          "tree_matches_p19": rep.get("source_tree_sha256") == QC.P19_SOURCE_TREE_SHA256,
          "zip_matches_p19": rep.get("zip_sha256") == QC.P19_SOURCE_ZIP_SHA256,
          "mismatches": len(rep.get("mismatches", [])), "findings": [x.get("id") for x in rep.get("findings",
  +>  [])
                                                                    if isinstance(x, dict)],
          "caveat": rep.get("same_box_caveat") or "same box (disclosed)",
          "live_report_identical_to_archived": (live.read_bytes() == f.read_bytes()) if live.is_file()
          else "live report not in this tree",
          "NOT_inherited_by_p20": "bound to the p19 tree/zip; the p20 tree has its own (pending) independent
  +> flag"}
    rec["archived_p19_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P19_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p19_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p19"] and rec["zip_matches_p19"] and rec["mismatches"] == 0
              and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec


def change_log():
    f = ROOT / QC.P19_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P19_ARCHIVED_RECORDS[QC.P19_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT) if r != SM.REFERENCE_FILE}
    added = sorted((set(new) - set(old)) | {SM.REFERENCE_FILE})
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])
    contract = [x for x in added + removed + modified if x.startswith("code/p5_entry/")
                and not __import__("contract_registry").registered_contract_change(x, ROOT)]   # p22 registry
    return {"base": f"{QC.P19_SOURCE_LIST} (pinned; files of the p19 zip sha256 {QC.P19_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok, "added": added, "removed": removed, "modified": modified,
            "contract_files_changed": contract, "counts": {"added": len(added), "removed": len(removed),
                                                           "modified": len(modified)}}


def _main():
    id_ok, ident = R19.contract_identity()
    import contract_registry as CR          # p22 disclosed revision: registered p22 revision of release_p18.p
  +> y
    l18_ok = all((ROOT / k).is_file() and CR.layer_file_ok(k, sha(ROOT / k), h) for k, h in QC.P18_CALC_SHA256
  +> .items())
    l19_ok, l19, r19 = p19_layer()
    k1_ok, k1, dc = k1_check()
    rt_ok, rt = retraction_check()
    pin_ok, pin, pc = d11_pin_check()
```

```
  p19i_ok, p19i = p19_independent()
  hard = dict(QC.HARD_STOPS_P20)
  hard_ok = bool(all(r19.get("hard_stops", {}).get(k) == v for k, v in hard.items())
                 and dc.get("bulk_symmetry_restored") is False)
  cl = change_log()
  execution_pass = bool(id_ok and l18_ok and l19_ok and k1_ok and rt_ok and pin_ok and hard_ok
                        and cl["base_pinned_ok"] and not cl["contract_files_changed"] and p19i.get("pins_ok"
+> ) is True)
  publish = execution_pass
  author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
  indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
  earned = {k: ("earned" if (publish and r19.get(k) == "earned") else "not_earned") for k in Q19.EARNABLE_P1
+> 9}
  earned["D11_ratio_pin_verified"] = "earned" if (publish and pc.get("D11_ratio_pin_verified") is True) else
+>  "not_earned"
  earned["D11_absolute_pin_verified"] = ("earned" if (publish and pc.get("D11_absolute_pin_verified") is Tru
+> e)
                                         else "not_earned")
  ms = json.loads(json.dumps(r19.get("milestones", {})))
  if publish:
      ms.setdefault("status", {})["P.5_identity"] = (
          "projected defect computed (zero exactly in d, all parts, all seven assignments; each row vanishes
+>  "
          "separately); projected identity verified; projected restoration verified (p20 definition: full si
+> gma_0 "
          "closure of the complete one-loop kappa^2 MS-bar counterterm incl. c.cb, h.c/c.b/c.cbs); K1 correc
+> ted in "
          "p20 (orientation erratum); D11 ratio pin verified; D11 absolute pin verified (p21); bulk symmetry
+>  NOT restored/claimed; "
          "O.1 NOT_CLOSED")
  cert = dict(r19)
  for k in ("change_log_vs_p18", "disclosed_test_revisions_p19"):
      cert[f"p19_layer_{k}"] = cert.pop(k, None)
  cert.update({
      "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
+> TE,
      "version_label": QC.VERSION_LABEL,
      "deliverable": "p20 CORRECTION of p19 (K1 orientation; restoration redefined; blind-spot pins; J1 word
+> ing; scan)",
      "execution_pass": execution_pass,
      "claim_status": (("p20 CORRECTION release (no scope expansion). " + str(r19.get("claim_status", "")) +
                       " D11: ratio pin verified (spin-0 : spin-2 = 10 : 1, derived from P18_D11 and equal
+> to the "
                       "minimal-scalar a_2 prediction, literature source stated); absolute pin (sign and ma
+> gnitude) derived in p21 from "
                       "1/2 Tr ln under the declared conventions and equal to the heat-kernel a_2 (p20: OPE
+> N).")
                      if publish else "p20: not accepted"),
      "erratum_p19": {
          "K1": ("p19 stated that full sigma_0 closure of the MS-bar counterterm fails with the derived ghos
+> t-equation "
                 "factor +1 and needs -1 (unresolved). RETRACTED: Grassmann-orientation bookkeeping error --
+>  the P15 "
                 "Sigma coordinates are (cb,c)-oriented, the route-G c.cb slots (c,cb), V[c.cb#k] = -V[(cb,c
+> )#k]. "
                 "Corrected: Sigma = (+3/8, -1/4) in (c,cb) coordinates, derived +1, full closure in h.c, c.
+> b, c.cbs."),
          "restoration": ("p19's projected_restoration_verified (P_ch sigma_0(S_ct) = 0, x = 0) followed fro
+> m the "
                          "identity + locality; REDEFINED in p20 as full sigma_0 closure of the complete MS-
+> bar S_ct "
                          "(incl. c.cb), kept as a labelled route-consistency control"),
          "J1": ("'over Q(i), the physical field' replaced by: the Q(i) counts are the real dimensions under
+>  a "
                 "compatible real structure"),
          "found_by": "p19 independent rebuild (report sha256 " + QC.P19_INDEPENDENT_REPORT_SHA256 + ")"},
      "K1_correction": k1, "retraction_check": rt, "D11_pin": {"check": pin, "ratio_pin": pc.get("ratio_pin"
+> ),
                                                               "absolute_pin": pc.get("absolute_pin"),
                                                               "spin_decomposition": pc.get("spin_decomposit
+> ion"),
                                                               "mutations": pc.get("mutations")},
      "open_pins": [x for x in QC.OPEN_PINS_P20
                    if not (x.startswith("D11_absolute") and pc.get("D11_absolute_pin_verified") is True)],
      "open_pins_closed_in_p21": [x for x in QC.OPEN_PINS_P20
                                  if x.startswith("D11_absolute") and pc.get("D11_absolute_pin_verified") is
+>  True],
      "release_claim_accepted": publish,
      "release_claim_accepted_definition": ("p19 layer accepted (with the disclosed p20 revisions of defect.
+> py and "
                                            "release_p19.py only) AND p18 layer + contract byte-identical AN
```

```
  +> D the K1 "
                                          "correction content AND the retraction check AND the D11 ratio p
  +> in AND "
                                          "the pinned p19 independent record AND the p20 hard stops"),
        "publication": "accepted" if publish else "REJECTED",
        **earned,
        "p19_layer": l19, "p18_layer_byte_identical": l18_ok, "contract_identity": ident,
        "change_log_vs_p19": cl,
        "disclosed_p19_layer_revisions_p20": QC.DISCLOSED_P19_LAYER_REVISIONS_P20,
        "disclosed_test_revisions_p20": QC.DISCLOSED_TEST_REVISIONS_P20,
        "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": ms,
        "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
        "independent_rebuild_passed_p19": p19i_ok, "p19_independent_rebuild": p19i,
        "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p20
  +> ) source "
                               "tree only (independent: pending until a reviewer report for this tree exists);
  +>  "
                               f"independent_rebuild_passed_p19 refers to the p19 tree {QC.P19_SOURCE_TREE_SHA
  +> 256} / zip "
                               f"{QC.P19_SOURCE_ZIP_SHA256} (same-box caveat) and is not inherited by p20"),
        "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
  +> ource.py",
                            "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                            "author": author_rec, "independent": indep_rec, "p19_archive": ARCH_P19,
                            "kind": "package evidence (XVI-0 conv:package); not a mathematical certificate"},
        "XVI_P_registered": False, "symmetry_restored": False, "bulk_symmetry_restored": False,
        "O1_closure": "NOT_CLOSED", "insertion_status": "Frozen_negative_obstruction_preserved",
        "XV_B_opened": False, "XIV_J_opened": False, "Route_B_discharged": False, "u_R_c_delta_added": False,
        "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
            n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
            ("P19_release.json", "P19_defect.json", f"{QC.CERT_PREFIX}_D11_pin.json")
            if (ROOT / "certificates" / n).is_file()},
    })
    cdir = ROOT / "certificates"
    if publish:
        txt = certio.dumps(cert)
        for st in STATUS_FILES:
            (ROOT / st).write_text(txt)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
        certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
            "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
  +> tatus)",
            **ms, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
            "independent_rebuild_passed_p19": p19i_ok, "hard_stops": hard,
            "flags": {**earned, "quantum_inputs_evaluated": cert.get("quantum_inputs_evaluated"),
                      "bulk_symmetry_restored": False}})
    else:
        _reject({"contract_identity_ok": id_ok, "p18_layer_ok": l18_ok, "p19_layer_ok": l19_ok, "k1_ok": k1_ok
  +> ,
                 "k1": k1, "retraction_ok": rt_ok, "retraction": rt, "d11_pin_ok": pin_ok, "d11_pin": pin,
                 "p19_independent_pins_ok": p19i.get("pins_ok"), "hard_stops_hold": hard_ok,
                 "change_log_base_ok": cl["base_pinned_ok"], "contract_files_changed": cl["contract_files_chan
  +> ged"]})
    print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
  +> ion"],
                      "contract": id_ok, "p18_layer": l18_ok, "p19_layer": l19_ok, "k1": k1_ok, "retraction":
  +> rt_ok,
                      "retraction_hits": rt["hits"], "d11_pin": pin_ok, "hard_ok": hard_ok, **earned,
                      "author": author_ok, "independent_p20": indep_ok, "independent_p19": p19i_ok,
                      "change_log_counts": cl["counts"]}, indent=1))
    return 0 if publish else 1


def _reject(detail):
    cdir = ROOT / "certificates"
    (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
    for st in STATUS_FILES:                      # p19 reviewer: a reject must not leave a stale RELEASE_STATU
  +> S.json
        (ROOT / st).unlink(missing_ok=True)
    (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
        {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", **detail}, indent=1, default
  +> =str) + "\n")


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        _reject({"error": f"{type(exc).__name__}: {exc}"})
        print(f"release_p20 error: {type(exc).__name__}: {exc}")
```

```
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

154 Source: tests/test_p20.py

[listing 121/133] tests/test_p20.py

```
"""XVI-P p20 tests: correction release of p19 (K1 orientation; restoration redefined; full-closure mutation de
  +> tector;
D11 ratio pin; J1 wording; extended static scan -- a PATTERN CHECK, NOT A PROOF; release gate fail-closed with
  +>  no
stale RELEASE_STATUS)."""
from __future__ import annotations
import ast
import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools", "tests"):
    sys.path.insert(0, str(ROOT / p))
import contract_registry as CR  # p22 (disclosed): registered contract revision
import p20_conventions as QC  # noqa: E402
import p19_conventions as Q19  # noqa: E402
import p21_conventions as Q21  # noqa: E402   (p21: reviewed-site lists)

CERT = ROOT / "certificates"


def cj(name):
    return json.loads((CERT / name).read_text())


def rs():
    # p21 (disclosed): the p20 LAYER record (the top-level RELEASE_STATUS.json is the p21 status)
    return json.loads((CERT / "P20_release.json").read_text())


# ============================================================== K1: orientation and full closure
def test_k1_live_orientation_ratio_minus_one():
    import defect as DF
    r, ok = DF.orientation_ratio()
    assert ok and [sp.sympify(x) for x in r] == [-1, -1]


def test_k1_live_tree_controls():
    """tree level: V_S0[cb,c] = 2iD V_S0[K,c]; sigma_0(S0_Kc + S0_cbc) = 0 while sigma_0(S0_Kc) alone is nonze
  +> ro."""
    import defect as DF
    t = DF.tree_controls()
    assert t["ghost_equation_tree_V_cbc_eq_2iD_V_Kc"] is True
    assert t["sigma0_S0Kc_plus_S0cbc_entries"] == 0 and t["sigma0_S0Kc_alone_entries (control, nonzero)"] > 0
    assert t["S0_Kc_terms"] > 0 and t["S0_cbc_terms"] > 0 and t["ok"] is True


def test_k1_full_closure_complete_counterterm():
    c = cj("P19_defect.json")
    k = c["counterterm_solve"]["K1_ghost_closure_convention"]
    assert k["relative_factor_derived"] == "1" and k["derived_equals_P15"] is True
    assert k["Sigma_pole_routeG_c_cb_coordinates"] == ["3/8", "-1/4"]
    assert k["Sigma_local_routeG_c_cb_coordinates"] == ["2/3", "-5/12"]
    assert k["full_closure_complete_S_ct"] == {"closed": True, "entries": 0, "sectors": []}
    assert k["full_closure_S_ct_without_ccb (p19 package S_ct)"]["sectors"] == ["c.b"]
    assert k["full_closure_S_ct_without_ccb (p19 package S_ct)"]["entries"] == 88
    assert k["full_closure_unconverted (p19 error)"]["entries"] == 88
    assert c["corrected_in_revision"].startswith("p20")
```

```
def test_k1_sigma_conversion_consistent_with_p15():
    """Sigma(c,cb) = orientation ratio x Sigma(cb,c) with the P15 fit."""
    k = cj("P19_defect.json")["counterterm_solve"]["K1_ghost_closure_convention"]
    cbc = [sp.Rational(x) for x in k["z_cb_P15_sigma_fit_cbc_orientation"]]
    r = [sp.Rational(x) for x in k["orientation_ratio_routeG_over_cbc"]]
    assert [a * b for a, b in zip(cbc, r)] == [sp.Rational(x) for x in k["Sigma_pole_routeG_c_cb_coordinates"]
  +> ]


def test_quotient_coordinates_with_and_without_sigma():
    c = cj("P19_defect.json")
    assert c["quotient_coordinates"]["pole"] == ["0", "0", "0", "-3/8", "1/4"]       # h.h + c.K only = -[Sigm
  +> a]
    assert c["quotient_coordinates_Sigma_alone_routeG"]["pole"] == ["0", "0", "0", "3/8", "-1/4"]
    assert all(x == "0" for p in c["quotient_coordinates_including_Sigma"].values() for x in p)
    assert "(c,cb)" in c["quotient_reading_p20"]


def test_restoration_redefined_and_earned_only_on_full_closure():
    c = cj("P19_defect.json")
    fd = c["flag_definitions"]["projected_restoration_verified"]
    assert "FULL sigma_0 closure" in fd and "COMPLETE" in fd and "c.cb" in fd and "NOT bulk symmetry" in fd
    assert c["projected_restoration_verified"] == "earned" and c["bulk_symmetry_restored"] is False
    rc = c["route_consistency_control_p19_criterion"]
    assert rc["holds"] is True and "NOT a flag" in rc["label"]
    assert all(v is True for v in c["p20_controls"].values()) and len(c["p20_controls"]) >= 7


def test_full_closure_mutation_detector():
    det = cj("P19_defect.json")["full_closure_mutation_detector"]["cases"]
    assert det["drop_graph_D13 (c.K data = 0, Sigma held fixed)"]["detected"] is True
    assert det["flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)"]["detected"] is True
    assert det["drop_graph_D13 (c.K data = 0, Sigma held fixed)"]["sectors"] == ["c.b"]
    for k in ("drop_graph_D01", "drop_graph_D03", "flip_phase_D03", "symmetry_factor_D03_x2"):
        assert det[k]["detected"] is True and det[k]["sectors"] == ["h.c"], k
    assert det["Sigma_orientation_unconverted (p19 error)"]["detected"] is True
    assert det["drop_graph_D11"]["detected"] is False          # recorded blind: covered by the D11 pin


# ============================================================== D11 pin
def test_d11_pin_certificate():
    c = cj("P20_D11_pin.json")
    sd = c["spin_decomposition"]
    assert sd["c2_spin2"] == "-I/240" and sd["c0_spin0s"] == "-I/24" and sd["ratio_c0_over_c2"] == "10"
    assert sd["transverse"] is True and sd["only_spin2_and_spin0s"] is True
    assert c["ratio_pin"]["status"] == "PASS" and c["D11_ratio_pin_verified"] is True
    # p21 (disclosed): absolute pin derived (sign and magnitude); p20 had it OPEN
    assert c["absolute_pin"]["status"] == "PASS" and c["absolute_pin_status"] == "PASS"
    assert c["D11_absolute_pin_verified"] is True and c["corrected_in_revision"].startswith("p21")
    assert "Birrell" in c["literature_a2_minimal_scalar"]["source"]
    assert "EXTERNAL" in c["literature_a2_minimal_scalar"]["source"]
    m = c["mutations"]
    assert m["drop_graph_D11"]["detected_by_ratio_pin"] is True
    assert m["D11_normalisation_x2"]["detected_by_ratio_pin"] is False
    assert m["D11_normalisation_x2"]["detected_by_absolute_pin"] is True
    assert m["D11_sign_flip"]["detected_by_ratio_pin"] is False and m["D11_sign_flip"]["detected_by_absolute_p
  +> in"] is True


def test_d11_pin_live_recompute():
    import d11_pin as P
    import covariant as C
    cert = cj("P18_D11.json")
    import dred
    four = {k: dred.parse_d(str(v["UV_pole"]), {"I": sp.I}) for k, v in cert["result"].items() if not v["evane
  +> scent"]}
    eta = tuple(int(x) for x in C.ETA)
    p = (sp.Integer(2), sp.Rational(1, 3), sp.Integer(-1), sp.Integer(4))     # a different generic momentum
    T, pp = P.tensor(four, eta, p)
    r = P.spin_parts(T, eta, p, pp)
    assert r["c2"] == -sp.I / 240 and r["c0"] == -sp.I / 24 and r["transverse"] and r["only_spin2_spin0s"]


def test_curvature_quadratic_forms():
    import d11_pin as P
    f = P.curvature_forms((sp.Integer(1), sp.Integer(2), sp.Integer(-3), sp.Rational(1, 2)))
    assert f["C2"] == {"P2": sp.Rational(1, 2), "P0s": 0} and f["R2"] == {"P2": 0, "P0s": 3}
    assert f["E_GaussBonnet"] == {"P2": 0, "P0s": 0}
    assert (P.LIT["R2_xi0"] * 3) / (P.LIT["C2"] * sp.Rational(1, 2)) == 10
```

```
# ============================================================ J1 wording, retraction
def test_j1_wording():
    j = cj("P19_defect.json")["findings"]["J1_p14r_counterterm_domain_field"]
    assert "compatible real structure" in j and "physical field" not in j
    assert "compatible real structure" in cj("P19_defect.json")["field_note"]


def test_retraction_check_clean():
    s = rs()
    assert s["retraction_check"]["hits"] == {}
    import release_p20 as R
    ok, rec = R.retraction_check()
    assert ok, rec["hits"]
    note = (ROOT / "manuscript" / "notes" / "P20_correction.tex").read_text()
    assert "Erratum K1" in note and "retracted" in note


# ============================================================ scan (p19 reviewer gaps; a PATTERN CHECK, N
  +> OT A PROOF)
ARGS_ITER_CALLS = {"Add", "Mul", "Pow", "reduce", "func"}
ALIAS_WATCH = {"Poly", "poly", "sympify", "parse_expr", "Symbol", "symbols", "S", "N", "lambdify"}


def _iter_over_args(node):
    return any(isinstance(g.iter, ast.Attribute) and g.iter.attr == "args"
               for n in ast.walk(node) if isinstance(n, (ast.ListComp, ast.GeneratorExp, ast.SetComp))
               for g in n.generators)


def scan3(path, root=ROOT):
    """p20 additions on top of test_p19.scan2: rebuilds from comprehensions over e.args (Add/Mul/Pow/func/any
    starred call, functools.reduce), aliased imports of Poly/sympify/Symbol..., any __call__ attribute,
    as_coefficients_dict, Symbol/symbols(name=<run-time>) keywords, explicit .func(*..args) constructor rebuil
  +> ds."""
    import test_p19 as T19
    bad, sites = T19.scan2(path, root)
    src = path.read_text()
    tree = ast.parse(src)
    try:
        rel = path.relative_to(root).as_posix()
    except ValueError:
        rel = path.name
    for node in ast.walk(tree):
        if isinstance(node, ast.ImportFrom):
            for a in node.names:
                if a.asname and a.name in ALIAS_WATCH:
                    bad.append(f"{rel}:{node.lineno}:aliased import {a.name} as {a.asname}")
        if isinstance(node, ast.Attribute) and node.attr == "__call__":
            bad.append(f"{rel}:{node.lineno}:__call__")
        if isinstance(node, ast.Call):
            name = T19._name(node.func)
            if name == "as_coefficients_dict":
                bad.append(f"{rel}:{node.lineno}:as_coefficients_dict")
            if any(isinstance(a, ast.Starred) and _iter_over_args(a.value) for a in node.args):
                bad.append(f"{rel}:{node.lineno}:{name}(*[.. for .. in x.args]) rebuild")
            if name == "reduce" and any(_iter_over_args(a) or (isinstance(a, ast.Attribute) and a.attr == "arg
  +> s")
                                        for a in node.args):
                bad.append(f"{rel}:{node.lineno}:reduce over x.args")
            if name in T19.SYMBOL_MAKERS and any(k.arg in ("name", "names") and T19._d_like(k.value)
                                                 for k in node.keywords):
                bad.append(f"{rel}:{node.lineno}:{name}(name=<run-time>)")
            if name == "func" or (isinstance(node.func, ast.Attribute) and node.func.attr == "func"):
                if any(isinstance(a, ast.Starred) for a in node.args):
                    bad.append(f"{rel}:{node.lineno}:explicit .func(*..) constructor rebuild")
    return bad, sites


def _wrap(code):      # the p19 reviewer's plant template
    return "import sympy as sp\nfrom dred import d\n" f"def g(e):\n    {code}\n"


PLANTS3 = {
    "p18_explicit_constructor_rebuild": "return sp.Add(*[4 if x == d else x for x in e.args])",
    "p19_add_listcomp_over_args": "return sp.Add(*[4 if x == d else x for x in e.args])",
    "p19_poly_alias_import": "from sympy import Poly as Q\n    return Q(e, d)(4)",
    "p19_poly_dunder_call": "return sp.Poly.__call__(sp.Poly(e, d), 4)",
    "p19_as_coefficients_dict": "return sum(c * (4 if k == d else k) for k, c in e.as_coefficients_dict().item
  +> s())",
    "p19_reduce_rebuild": "import functools, operator\n    return functools.reduce(operator.add, [4 if x == d
  +> else x for x in e.args])",
```

```
    "p19_symbol_kw_name": "return sp.Symbol(name=chr(100))",
    "func_starred_rebuild": "return e.func(*[4 if x == d else x for x in e.args])",
    "mul_genexp_over_args": "return sp.Mul(*(x for x in e.args if x != d))",
}


@pytest.mark.parametrize("name", sorted(PLANTS3))
def test_scan3_detects_reviewer_plants(tmp_path, name):
    f = tmp_path / f"{name}.py"
    f.write_text(_wrap(PLANTS3[name]))
    bad, _ = scan3(f, tmp_path)
    assert bad, name


def test_scan3_package_clean():
    bad = []
    for d in ("code/p5_calc", "code/p5_entry"):
        for f in sorted((ROOT / d).glob("*.py")):
            bad += scan3(*CR.scan_view(f))[0]   # p22: p18 contract core
    assert not bad, bad


def test_scan3_allows_benign_forms(tmp_path):
    f = tmp_path / "ok.py"
    f.write_text("import sympy as sp\nfrom sympy import Poly\ndef f(e, a, b):\n    s = sp.Symbol(name='x')\n"
                 "    return sp.Add(*[a, b]), sp.Add(*[t for t in (a, b)]), Poly(e, a).terms(), s\n")
    bad, _ = scan3(f, tmp_path)
    assert not bad, bad


def test_scan_disclosure_wording():
    assert "pattern check, not a proof" in (ROOT / "README.md").read_text()


# ============================================================ status
def test_p20_release_status_and_flags():
    s = rs()
    assert s["revision"] == "p20" and s["publication"] == "accepted" and s["execution_pass"] is True
    for k in QC.EARNABLE_P20:
        assert s[k] == "earned", k
    assert s["bulk_symmetry_restored"] is False and s["symmetry_restored"] is False and s["O1_closure"] == "NO
  +> T_CLOSED"
    assert s["hard_stops"] == QC.HARD_STOPS_P20 and s["hard_stops_hold"] is True
    assert s["XVI_P_registered"] is False and s["Route_B_discharged"] is False and s["u_R_c_delta_added"] is F
  +> alse
    assert s["independent_rebuild_passed"] is not True and s["independent_rebuild_passed_p19"] is True   # p21
  +>  tree
    assert s["open_pins"] == list(Q21.OPEN_PINS_P21) and s["D11_absolute_pin_verified"] == "earned"     # p21
    assert "RETRACTED" in s["erratum_p19"]["K1"] and "REDEFINED" in s["erratum_p19"]["restoration"]
    assert s["milestones"]["status"]["P.5_identity"].count("bulk symmetry NOT restored") == 1


def test_p19_independent_recorded_and_not_inherited():
    s = rs()
    rec = s["p19_independent_rebuild"]
    assert rec["report_sha256"] == QC.P19_INDEPENDENT_REPORT_SHA256 and rec["pins_ok"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P19_SOURCE_TREE_SHA256
    assert rec["bound_to_source_zip_sha256"] == QC.P19_SOURCE_ZIP_SHA256 and rec["mismatches"] == 0
    import source_manifest as SM
    assert SM.tree_sha(ROOT) != QC.P19_SOURCE_TREE_SHA256


def test_layers_and_disclosed_revisions():
    assert hashlib.sha256((ROOT / "code/p5_calc/p19_conventions.py").read_bytes()).hexdigest() == QC.P19_CONVE
  +> NTIONS_SHA256
    for k, h in QC.P18_CALC_SHA256.items():   # p22: or the registered p22 revision of that layer file
        assert CR.layer_file_ok(k, hashlib.sha256((ROOT / k).read_bytes()).hexdigest(), h), k
    for k, h in QC.P19_RELEASED_SHA256.items():
        assert hashlib.sha256((ROOT / k).read_bytes()).hexdigest() != h and k in QC.DISCLOSED_P19_LAYER_REVISI
  +> ONS_P20


def test_change_log_vs_p19():
    cl = rs()["change_log_vs_p19"]
    assert cl["base_pinned_ok"] is True and cl["contract_files_changed"] == []
    import source_manifest as SM
    base = json.loads((ROOT / QC.P19_SOURCE_LIST).read_text())["files"]
    cur = {r: hashlib.sha256((ROOT / r).read_bytes()).hexdigest() for r in SM.source_files(ROOT) if r != SM.RE
  +> FERENCE_FILE}
    changed = sorted(set(cur) ^ set(base) | {k for k in set(cur) & set(base) if cur[k] != base[k]} | {SM.REFER
  +> ENCE_FILE})
```

```
    assert sorted(cl["added"] + cl["removed"] + cl["modified"]) == changed
    assert sorted(x for x in cl["modified"] if x.startswith("tests/") and not __import__("contract_registry").
  +> p22_revised(x)) == sorted(   # p22 filter
        x for x in QC.DISCLOSED_TEST_REVISIONS_P20 if not __import__("contract_registry").p22_revised(x))
    mod_code = sorted(x for x in cl["modified"] if x.startswith("code/") and not __import__("contract_registry
  +> ").p22_revised(x))
    assert mod_code == sorted(x for x in QC.DISCLOSED_P19_LAYER_REVISIONS_P20
                              if not __import__("contract_registry").p22_revised(x)), mod_code
    for f in ("code/p5_calc/d11_pin.py", "code/p5_calc/release_p20.py", "code/p5_calc/p20_conventions.py",
              "tests/test_p20.py", "manuscript/notes/P20_correction.tex", "archival/p19/README.md"):
        assert f in cl["added"], f


def test_version_label_p20():
    import build as B
    assert QC.VERSION_LABEL == "XVI-P p20" and B.QC.VERSION_LABEL == f"XVI-P {B.QC.REVISION}"   # p21: current
  +>  label
    assert "certificates/P20_D11_pin.json" in B.VALUE_BLOCKS


# ============================================================== p20 gate mutations (fail closed; no stale s
  +> tatus)
def _copy():
    d = Path(tempfile.mkdtemp(prefix="p20gate_"))
    for sub in ("code", "certificates", "ledgers", "exports", "tools", "archival", "reproducibility", "provena
  +> nce",
                "tests"):
        if (ROOT / sub).exists():
            shutil.copytree(ROOT / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__", "*.zip", "*.pdf"
  +> ))
    for f in ("README.md", "REBUILD_INSTRUCTIONS.md", "RELEASE_STATUS.json"):
        if (ROOT / f).exists():
            shutil.copy(ROOT / f, d / f)
    (d / "manuscript" / "notes").mkdir(parents=True)
    (d / "manuscript" / "body").mkdir(parents=True)
    for f in ("manuscript/notes/P19_defect.tex", "manuscript/notes/P20_correction.tex",
              "manuscript/body/P19-results-table.tex", "manuscript/body/XVI-P-body.tex"):
        if (ROOT / f).exists():
            shutil.copy(ROOT / f, d / f)
    return d


def _edit(d, name, fn):
    f = d / "certificates" / name
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c, indent=1))


def _k1(c):
    return c["counterterm_solve"]["K1_ghost_closure_convention"]


def _closure_broken(d):
    _edit(d, "P19_defect.json", lambda c: _k1(c)["full_closure_complete_S_ct"].__setitem__("closed", False))


def _orientation_wrong(d):
    _edit(d, "P19_defect.json", lambda c: _k1(c).__setitem__("orientation_ratio_routeG_over_cbc", ["1", "1"]))


def _sigma_unconverted(d):
    _edit(d, "P19_defect.json", lambda c: _k1(c).__setitem__("Sigma_pole_routeG_c_cb_coordinates", ["-3/8", "1
  +> /4"]))


def _derived_factor_minus_one(d):
    _edit(d, "P19_defect.json", lambda c: _k1(c).__setitem__("relative_factor_derived", "-1"))


def _detector_blind_d13(d):
    _edit(d, "P19_defect.json", lambda c: c["full_closure_mutation_detector"]["cases"][
        "drop_graph_D13 (c.K data = 0, Sigma held fixed)"].__setitem__("detected", False))


def _p20_control_false(d):
    _edit(d, "P19_defect.json", lambda c: c["p20_controls"].__setitem__("tree_sigma0_S0Kc_plus_S0cbc_zero", Fa
  +> lse))


def _retracted_text_back(d):
```

```
    f = d / "README.md"
    f.write_text(f.read_text() + "\nunder the derived convention (+1, K1) full c.b / ghost-sector closure FAIL
  +> S\n")


def _j1_old_wording(d):
    f = d / "manuscript" / "notes" / "P19_defect.tex"
    f.write_text(f.read_text() + "\nOver Q(i), the physical field.\n")


def _d11_pin_fail(d):
    _edit(d, "P20_D11_pin.json", lambda c: c["ratio_pin"].__setitem__("status", "FAIL"))


def _d11_absolute_failed(d):          # p21: absolute pin required
    _edit(d, "P20_D11_pin.json", lambda c: c["absolute_pin"].__setitem__("status", "FAIL"))


def _d11_top_level_status_inconsistent(d):     # p20 reviewer: top-level key unchecked
    _edit(d, "P20_D11_pin.json", lambda c: c.__setitem__("absolute_pin_status", "OPEN"))


def _d11_sign_flip_undetected(d):
    _edit(d, "P20_D11_pin.json", lambda c: c["mutations"]["D11_sign_flip"].__setitem__("detected_by_absolute_p
  +> in", False))


def _p18_q_not_earned(d):                     # p20 reviewer: standalone p20 gate must recheck the lower layer
  +> s
    _edit(d, "P18_release.json", lambda c: c.__setitem__("quantum_inputs_evaluated", "not_earned"))


def _bulk_claimed(d):
    _edit(d, "P19_defect.json", lambda c: c.__setitem__("bulk_symmetry_restored", True))


def _tamper_p19_report(d):
    f = d / "archival" / "p19" / "reproducibility" / "independent_rebuild_report_p19.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


def _p19_conventions_changed(d):
    f = d / "code" / "p5_calc" / "p19_conventions.py"
    f.write_text(f.read_text() + "\n# tampered\n")


def _p19_layer_rejected(d):
    _edit(d, "P19_release.json", lambda c: c.__setitem__("publication", "REJECTED"))


MUTS = [_closure_broken, _orientation_wrong, _sigma_unconverted, _derived_factor_minus_one, _detector_blind_d1
  +> 3,
        _p20_control_false, _retracted_text_back, _j1_old_wording, _d11_pin_fail, _d11_absolute_failed, _bulk_
  +> claimed,
        _tamper_p19_report, _p19_conventions_changed, _p19_layer_rejected,
        _d11_top_level_status_inconsistent, _d11_sign_flip_undetected, _p18_q_not_earned]


def _run(d, script="code/p5_calc/release_p20.py"):
    return subprocess.run([sys.executable, script], cwd=d, capture_output=True, text=True, timeout=600)


@pytest.mark.parametrize("mut", MUTS, ids=lambda f: f.__name__)
def test_certificate_mutation_p20_gate_rejects_without_stale_status(mut):
    d = _copy()
    try:
        assert (d / "RELEASE_STATUS.json").exists()
        mut(d)
        r = _run(d)
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d / "certificates" / "P20_release.json").exists()
        assert (d / "certificates" / "P20_release_rejected.json").exists()
        assert not (d / "RELEASE_STATUS.json").exists() and not (d / "certificates" / "RELEASE_STATUS.json").e
  +> xists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_p19_gate_reject_leaves_no_stale_status():
    d = _copy()
    try:
```

```
        _edit(d, "P18_release.json", lambda c: c.__setitem__("quantum_inputs_evaluated", "not_earned"))
        r = _run(d, "code/p5_calc/release_p19.py")
        assert r.returncode == 1
        assert not (d / "RELEASE_STATUS.json").exists() and not (d / "certificates" / "P19_release.json").exis
  +> ts()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_p20_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = _run(d)
        assert r.returncode == 0, r.stdout[-1500:]
        s = json.loads((d / "RELEASE_STATUS.json").read_text())
        assert s["revision"] == "p20" and s["projected_restoration_verified"] == "earned"
        assert s["D11_ratio_pin_verified"] == "earned" and s["bulk_symmetry_restored"] is False
        assert s["D11_absolute_pin_verified"] == "earned"
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

155 Source: p21_conventions.py (p21 constants, version label, pins, disclosures, scan allow-list)

[listing 122/133] code/p5_calc/p21_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p21 conventions: small release from p20 (no scope expansion), from the p20 independent-rebuild findin
  +> gs.

  A  absolute D11 pin: sign and magnitude of the D11 pole derived from 1/2 Tr ln (minimal scalar loop) under t
  +> he
     package's declared conventions (G = Gamma^(2); N_eps = i S_eps/(16 pi^2) and the +1/eps bubble pole fix t
  +> he Wick
     direction; g = eta + kappa h; eta = (+,-,-,-)) and compared with the heat-kernel a_2 = C^2/120 + R^2/72:
  +> exact
     equality, both spins (d11_pin.absolute_pin; p20 had it OPEN).  Mutations x2 and sign flip detected.
  G  gate: top-level *_pin_status keys checked against the nested statuses; release_p20 run standalone re-chec
  +> ks the
     p18 layer (quantum_inputs_evaluated) and the graph inventory instead of trusting the P19 layer record.
  S  d-evaluation scan: the enumerated pattern list (test_p16 .. test_p20) is complemented by an ALLOW-LIST sc
  +> an
     (tests/test_p21.py): expression-tree decomposition, sympy callable aliases, and dynamic symbol names are
  +> refused
     everywhere except at reviewed sites listed below, plus a RUNTIME check that stored exact-d amplitudes re-
  +> parse
     with the opaque d and reproduce their poles only through substitute_d.  Still a check, not a proof.
  D  disclosure: the p20 edit of manuscript/notes/P19_defect.tex is listed (DISCLOSED_MANUSCRIPT_REVISIONS); t
  +> he
     reference key /p19/change_log_vs_p18 is renamed (it was recomputed on the current tree, not the p19 relea
  +> se log).

Layering.  ... -> p19 layer -> p20 layer (p20_conventions.py byte-identical; d11_pin.py and release_p20.py DIS
  +> CLOSED
p21 revisions) -> p21 layer (code/p5_calc/{p21_conventions,release_p21}.py, P21_*).  No contract revision.
"""
from __future__ import annotations
from p20_conventions import (ROW_HH, ROW_KC, ROW_1PT, ROWS, HH_EVALUATED, P18_CALC_SHA256,  # noqa: F401
                             P18_DRED_SHA256, RUN_SPECIFIC_KEYS, HARD_STOPS_P20, EARNABLE_P20, OPEN_PINS_P20)
from p20_conventions import COEFF_SITES_REVIEWED as _C20, POLY_GENERATOR_CALL_SITES_REVIEWED as _G20

REVISION = "p21"
VERSION_LABEL = "XVI-P p21"
CERT_PREFIX = "P21"
CONTRACT_REVISION = "p21 (p14r contract + disclosed p15/p16/p17/p18 revisions; none in p19, p20 or p21)"
DATE = "2026-10-09"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p21.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p21.json"
P20_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p20.json"      # written by the reviewer;
  +>  never here
P20_INDEPENDENT_REPORT_SHA256 = "e8f358f9c4f13c76693968211b49e04d546a0bb3a4c0ab4b1a0e485f62595389"
P20_SOURCE_TREE_SHA256 = "2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700"
P20_SOURCE_ZIP_SHA256 = "ca32ce018933f12d1a623217d6a98d647817360dd12211e2e9469cf850a63bb2"
P20_INDEPENDENT_SAME_BOX = True     # disclosed: the p20 independent rebuild ran on the same box
P20_ARCHIVED_RECORDS = {
    "archival/p20/reference_values_p20.json": "4877f5dedc84e6dc07fdc9a72077d3098308bb95c168cba36cafdda23d719e0
  +> 4",
```

```
    "archival/p20/reproducibility/author_rebuild_report_p20.json":
        "686e2c9c7835e3d4331cf8bcb0c3176661d1e7bfbb146a19bbca4d3135856ac0",
    "archival/p20/reproducibility/independent_rebuild_report_p20.json": P20_INDEPENDENT_REPORT_SHA256,
    "archival/p20/p20_source_files.json": "8031db7fb5fd4832affbc5721a104af554ad9404b665bd7634a11b441d6d64cc",
}
P20_SOURCE_LIST = "archival/p20/p20_source_files.json"
INVENTORY_P21 = ("D01", "D02", "D03", "D11", "D12", "D13", "D14a", "D14b", "D14c")
HARD_STOPS_P21 = dict(HARD_STOPS_P20)
EARNABLE_P21 = EARNABLE_P20 + ("D11_absolute_pin_verified",)
OPEN_PINS_P21 = tuple(x for x in OPEN_PINS_P20 if not x.startswith("D11_absolute"))

# p20 layer in p21: p20_conventions.py byte-identical; d11_pin.py, release_p20.py DISCLOSED revisions
P20_CONVENTIONS_SHA256 = "325a7b296d7fc50ea06dae2ebfd6735be14bbd7907f328d435420c1348bf901b"
P20_RELEASED_SHA256 = {
    "code/p5_calc/d11_pin.py": "a31b718a36da390ef8814d400c36f698711142302a86d6f7e852a4e5b7142ae8",
    "code/p5_calc/release_p20.py": "034a6eba47c216b34ac49e7fc9b67d9e07dc0dab0c592b3e335ef10e36eb9fd2",
}
DISCLOSED_P20_LAYER_REVISIONS_P21 = {
    "code/p5_calc/d11_pin.py": ("absolute_pin(): sign and magnitude derived from 1/2 Tr ln under the declared
  +> "
                                "conventions (Minkowski quadratic forms curvature_forms_eta); absolute pin PAS
  +> S; "
                                "mutations x2 and sign flip; top-level absolute_pin_status from the derivation
  +> "),
    "code/p5_calc/release_p20.py": ("absolute pin PASS required; top-level / nested status consistency; standa
  +> lone "
                                    "run re-checks the p18 layer (q earned) and the graph inventory; claim tex
  +> ts"),
}
# p20 reviewer finding P19_note_edit_disclosure: manuscript edits of earlier-layer notes are disclosed here
DISCLOSED_MANUSCRIPT_REVISIONS = {
    "manuscript/notes/P19_defect.tex": {
        "p19_sha256": "04b974b5ea742c6885f3d694ccad6bda7908105c0a23bf483e96bc113d90bbc8",
        "p20_sha256": "1c71a71a48ce9c1749bd2164a0f720a3589f2d679877cfd04309d69670e5dfa8",
        "revised_in": "p20",
        "change": ("K1 paragraph retracted/rewritten (Grassmann orientation erratum), restoration definition R
  +> EDEFINED, "
                   "J1 wording, blind-spot text; marked inline [corrected in p20] / [REDEFINED in p20]; unchan
  +> ged in "
                   "p21")},
}
NEW_CONTRACT_FILES_P21 = ()
DISCLOSED_CONTRACT_REVISIONS_P21 = {}
DISCLOSED_TEST_REVISIONS_P21 = {
    "tests/test_p14.py": "names p20 -> p21; generated prefixes P21_",
    "tests/test_p19.py": "reviewed coefficient sites read from p21_conventions (adds the d11_pin Minkowski for
  +> m site)",
    "tests/test_p20.py": ("D11 pin tests: absolute pin PASS (p21) instead of OPEN; gate mutations on the p21 c
  +> hecks; "
                          "reviewed sites from p21_conventions; status/version expectations p21"),
}
COEFF_SITES_REVIEWED = dict(_C20)
COEFF_SITES_REVIEWED.update({
    "code/p5_calc/d11_pin.py::curvature_forms_eta::sp.Poly(sp.expand(f - a * q2 - b * q0), *gens).coeffs()":
        "generators: metric perturbation symbols h_ab (no d anywhere in d11_pin)",
})
POLY_GENERATOR_CALL_SITES_REVIEWED = dict(_G20)

# ------------------------------------------------------------------------------ allow-list scan (S, p21)
# attributes / helpers that decompose a sympy expression tree (rebuild paths with no hook in the opaque d)
DECOMPOSE_ATTRS = ("args", "func", "make_args", "as_ordered_terms", "as_ordered_factors", "as_coefficients_dic
  +> t",
                   "as_coeff_add", "as_coeff_mul", "as_coeff_Add", "as_coeff_Mul", "as_independent", "as_two_t
  +> erms",
                   "as_numer_denom", "as_powers_dict", "as_terms", "as_content_primitive", "as_base_exp",
                   "as_coeff_exponent", "preorder_traversal", "postorder_traversal", "iter_args", "_args",
                   "as_expr_dict")
# reviewed sites  file::function::source-segment -> reason (count must match exactly)
DECOMPOSE_SITES_REVIEWED = {
    "code/p5_calc/batchA.py::evaluate::sp.Add.make_args":
        "iterates the mass-regulator pole terms to read their m2 degree (sp.degree); no expression is rebuilt"
  +> ,
    "code/p5_entry/counterterm_domain.py::two_point_vector::sp.Add.make_args":
        "terms of a d-free momentum polynomial -> monomial coordinates (read only)",
    "code/p5_entry/counterterm_domain.py::two_point_vector::t.as_independent":
        "split numeric coefficient / momentum monomial (read only; no d)",
    "code/p5_entry/feynman.py::from_sympy::sp.Add.make_args": "conversion into the frozen sparse polynomial ty
  +> pe",
    "code/p5_entry/feynman.py::from_sympy::t.as_independent": "conversion into the frozen sparse polynomial ty
  +> pe",
    "code/p5_entry/feynman.py::ell_terms::sp.Add.make_args": "loop-momentum term listing (frozen contract, p14
```

```
  +> r)",
    "code/p5_entry/feynman.py::ell_terms::t.as_coeff_Mul": "loop-momentum term listing (frozen contract, p14r)
  +> ",
    "code/p5_entry/quantum_object.py::patterns::sp.Add.make_args": "field-content pattern listing (no d)",
    "code/p5_entry/quantum_object.py::patterns::t.as_powers_dict": "field-content pattern listing (no d)",
    "code/p5_entry/quantum_object.py::b_elimination::sp.Add.make_args": "term counting / degree filter (read o
  +> nly)",
    "code/p5_entry/feynman.py::from_sympy::rest.as_powers_dict": "monomial exponents for the frozen sparse typ
  +> e",
    "code/p5_entry/feynman.py::ell_terms::rest.as_powers_dict": "monomial exponents (frozen contract, p14r)",
}
DECOMPOSE_SITE_COUNTS = {"code/p5_entry/quantum_object.py::b_elimination::sp.Add.make_args": 3}
# sympy attributes that may be bound to a module-level / local alias (constants and two special functions)
SYMPY_ALIAS_ALLOWED = ("I", "pi", "E", "oo", "EulerGamma", "gamma", "rf")
# from-imports out of sympy submodules that were reviewed (no alias)
SYMPY_FROM_IMPORT_REVIEWED = {
    "code/p5_calc/covariant.py::covariant_fit::from sympy.polys.matrices import DomainMatrix":
        "exact rational linear algebra for the covariant fit (numbers only; no d)",
}
# symbol constructors: name must be a literal; non-literal names only at reviewed sites
SYMBOL_MAKERS = ("Symbol", "symbols", "Dummy", "Wild", "var")
SYMBOL_SITES_REVIEWED = {
    "code/p5_calc/release_p17.py::f3_hardening::sp.Symbol(\"d\")": (
        "deliberate PLAIN d in the F3 hardening demonstration (shows that the plain symbol is refused by "
        "assert_opaque_d / parse_d); never enters a value path"),
    "code/p5_entry/dred.py::dot::sp.Symbol(f\"{x}.{y}\")": "contraction symbols 'x.y' (always contain '.', nev
  +> er d)",
    "code/p5_calc/batchA.py::_poly::sp.Symbol(v)": "v ranges over stored monomial names l0..l3, p0..p3",
    "code/p5_calc/batchC.py::count_quadratic_component_terms::sp.Symbol(v)": "v startswith 'p' (momentum compo
  +> nents)",
    "code/p5_calc/d13.py::route1::sp.Symbol(v)": "v startswith 'p' (momentum components)",
    "code/p5_entry/dred.py::lift_reduce_polynomial::sp.Symbol(v)": "v startswith 'p' (momentum components)",
}

# (d) p21 runtime finding: value calls that mention d are refused statically except at reviewed demonstration
  +> sites
VALUE_CALLS = ("subs", "evalf", "N", "n", "limit", "Lambda", "Subs", "doit", "lambdify", "series", "nsimplify"
  +> )
DVALUE_SITES_REVIEWED = {        # each is an EXPECT-TO-RAISE demonstration of the R5 guard, never a value pat
  +> h
    "code/p5_calc/batchC.py::mutations::sp.sympify(a0).subs(d, 4)": "M3 premature-d4 demo (rejected recorded)"
  +> ,
    "code/p5_calc/d11.py::d11_mutations::a0.subs(d, 4)": "premature-d4 demo (rejected recorded)",
    'code/p5_calc/d13.py::_opaque_d_refuses::sp.sympify(fit1["T2"]).subs(d, 4)': "R5 safeguard demo",
    "code/p5_calc/release_p16.py::f3_hardening::sp.lambdify(d, e)": "F3 guard demo",
    "code/p5_calc/release_p16.py::f3_hardening::e.subs(d, 4)": "F3 guard demo",
    "code/p5_calc/release_p16.py::f3_hardening::e.evalf(subs={d: 4})": "F3 guard demo",
    "code/p5_calc/release_p17.py::f3_hardening::sp.lambdify(d, e)": "F3/G2 guard demo",
    "code/p5_calc/release_p17.py::f3_hardening::e.subs(d, 4)": "F3/G2 guard demo",
    "code/p5_calc/release_p17.py::f3_hardening::dred.parse_d(str(e)).subs(d, 4)": "G2 loader demo",
}
DALIAS_SITES_REVIEWED = {
    'code/p5_entry/dred.py::parse_d::loc["d"] = d': "the guarded loader binds the OPAQUE d as the parse local"
  +> ,
}
DVALUE_SITE_COUNTS = {}
RUNTIME_GUARD_LIMITATION_P21 = (
    "dred.OpaqueDimension does not intercept expr.subs(d, v) when d occurs only as a power base (e.g. 1/(d**2
  +> - 1)): "
    "sympy's Pow substitution matches the base without calling the symbol's _subs.  Found by the p21 runtime c
  +> heck; "
    "contract (dred.py) NOT changed in p21 (no contract revision; out of scope).  Mitigation: static allow-lis
  +> t rule "
    "(d) refuses every subs/evalf/N/limit/Lambda/... call mentioning d and every rebinding of d outside review
  +> ed "
    "demonstration sites; released premature-d4 demonstrations (P16_batchC, P18_D11) used components that rais
  +> e.")
```

156 Source: release_p21.py (p21 layer gate, disclosed p22 revision)

[listing 123/133] code/p5_calc/release_p21.py

```
#!/usr/bin/env python3
"""XVI-P p21 release gate: small release from p20 (no scope expansion), from the p20 independent-rebuild findi
  +> ngs.
```

```
Runs AFTER the p14r .. p20 gates (release_p20.py -> certificates/P20_release.json, the p20 LAYER record) and r
  +> equires:
  contract identity : as in p19/p20 (no contract revision);
  lower layers      : RE-CHECKED live, not read from layer records: p18 layer (files byte-identical, P18
                      quantum_inputs_evaluated == 'earned'), graph inventory, p19 layer (release_p20.p19_layer
  +> ), K1
                      correction content (release_p20.k1_check), retraction check;
  p20 layer         : p20_conventions.py byte-identical; d11_pin.py and release_p20.py differ from the p20 zip
  +>  ONLY as
                      the disclosed p21 revisions; P20_release accepted with D11_ratio_pin_verified and
                      D11_absolute_pin_verified earned;
  absolute D11 pin  : P20_D11_pin.json absolute pin PASS (derived from 1/2 Tr ln under the declared convention
  +> s, no
                      external convention choice), top-level *_pin_status == nested status, x2 / sign flip / d
  +> rop D11
                      detected (release_p20.d11_pin_check, live);
  disclosure        : every file modified vs the p20 zip is a disclosed revision (code, tests, manuscript) or
  +> a
                      documentation / build file listed in DISCLOSED_OTHER; the p20 edit of P19_defect.tex is
  +> listed;
  p20 independent   : archived report pinned (e8f358f9...), PASS, bound to tree 2c0aa153... / zip ca32ce01...
                      (same-box caveat) -> independent_rebuild_passed_p20; the p21 independent flag is pending
  +> ;
  hard stops        : QC.HARD_STOPS_P21 (bulk_symmetry_restored false regardless; 0.1 NOT_CLOSED).
Publication fails closed.  On reject: no P21_release.json and NO RELEASE_STATUS.json is left behind.
"""
from __future__ import annotations
import hashlib, json, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p21_conventions as QC  # noqa: E402
import p20_conventions as Q20  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)
import release_p19 as R19  # noqa: E402   (helpers only)
import release_p20 as R20  # noqa: E402   (helpers only)

ARCH_P20 = "archival/p20"
ARCH_P20_INDEP = f"{ARCH_P20}/reproducibility/independent_rebuild_report_p20.json"
STATUS_FILES = ("RELEASE_STATUS.json", "certificates/RELEASE_STATUS.json")
# documentation / build / archive files that every release updates (not package logic)
DISCLOSED_OTHER_PREFIXES = ("README.md", "REBUILD_INSTRUCTIONS.md", "tools/", "archival/", "manuscript/",
                            "reproducibility/", "build/")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def lower_layers():
    id_ok, ident = R19.contract_identity()
    l18_ok, l18 = R19.p18_layer()[:2]
    inv_ok, _ = R19.inventory_recheck()
    l19_ok, l19, _r19 = R20.p19_layer()
    k1_ok, k1, _ = R20.k1_check()
    rt_ok, rt = R20.retraction_check()
    ok = bool(id_ok and l18_ok and inv_ok and l19_ok and k1_ok and rt_ok)
    return ok, {"contract_identity": id_ok, "p18_layer": l18_ok,
                "P18_quantum_inputs_evaluated": l18.get("P18_quantum_inputs_evaluated"), "inventory": inv_ok,
                "p19_layer": l19_ok, "k1_correction": k1_ok, "retraction": rt_ok, "retraction_hits": rt.get("h
  +> its"),
                "contract_detail": ident, "rechecked_live": True}


def p20_layer():
    conv_ok = sha(ROOT / "code/p5_calc/p20_conventions.py") == QC.P20_CONVENTIONS_SHA256
    revs = {k: {"sha256": sha(ROOT / k), "differs_from_p20": sha(ROOT / k) != h,
                "disclosed": k in QC.DISCLOSED_P20_LAYER_REVISIONS_P21} for k, h in QC.P20_RELEASED_SHA256.ite
  +> ms()}
    r = jl("certificates/P20_release.json")
    ok = bool(conv_ok and all(v["disclosed"] for v in revs.values())
```

```
            and r.get("revision") == Q20.REVISION and r.get("publication") == "accepted"
            and r.get("release_claim_accepted") is True and r.get("D11_ratio_pin_verified") == "earned"
            and r.get("D11_absolute_pin_verified") == "earned" and r.get("hard_stops_hold") is True
            and r.get("bulk_symmetry_restored") is False)
    return ok, {"p20_conventions_byte_identical": conv_ok, "disclosed_p21_revisions": revs,
                "P20_release_accepted": r.get("publication") == "accepted",
                "P20_D11_absolute_pin_verified": r.get("D11_absolute_pin_verified")}, r


def p20_independent():
    f = ROOT / ARCH_P20_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P20_INDEP, "present": False, "pins_ok": False}
    rep = json.loads(f.read_text())
    live = ROOT / QC.P20_INDEPENDENT_REPORT
    rec = {"report": ARCH_P20_INDEP, "present": True, "report_sha256": sha(f),
           "report_sha256_pinned": sha(f) == QC.P20_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "bound_to_source_tree_sha256": rep.get("source_tree_sha256"), "bound_to_source_zip_sha256": rep.get
  +> ("zip_sha256"),
           "tree_matches_p20": rep.get("source_tree_sha256") == QC.P20_SOURCE_TREE_SHA256,
           "zip_matches_p20": rep.get("zip_sha256") == QC.P20_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])),
           "findings": [x.get("id") for x in rep.get("findings", []) if isinstance(x, dict)],
           "caveat": rep.get("same_box_caveat") or "same box (disclosed)",
           "live_report_identical_to_archived": (live.read_bytes() == f.read_bytes()) if live.is_file()
           else "live report not in this tree",
           "NOT_inherited_by_p21": "bound to the p20 tree/zip; the p21 tree has its own (pending) independent
  +> flag"}
    rec["archived_p20_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P20_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p20_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p20"] and rec["zip_matches_p20"] and rec["mismatches"] == 0
              and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec


def change_log():
    f = ROOT / QC.P20_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P20_ARCHIVED_RECORDS[QC.P20_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT) if r != SM.REFERENCE_FILE}
    added = sorted((set(new) - set(old)) | {SM.REFERENCE_FILE})
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])
    contract = [x for x in added + removed + modified if x.startswith("code/p5_entry/")
                and not __import__("contract_registry").registered_contract_change(x, ROOT)]   # p22 registry
    disclosed = (set(QC.DISCLOSED_P20_LAYER_REVISIONS_P21) | set(QC.DISCLOSED_TEST_REVISIONS_P21)
                 | set(QC.DISCLOSED_MANUSCRIPT_REVISIONS)
                 | set(__import__("contract_registry").P22_REVISED_FILES)
                 | set(__import__("contract_registry").P23_REVISED_FILES))   # p22 disclosed revisions (pinned
  +>  by the p22 gate)
    undisclosed = [x for x in modified + removed if x not in disclosed and not x.startswith(DISCLOSED_OTHER_PR
  +> EFIXES)]
    return {"base": f"{QC.P20_SOURCE_LIST} (pinned; files of the p20 zip sha256 {QC.P20_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok, "added": added, "removed": removed, "modified": modified,
            "contract_files_changed": contract, "undisclosed_package_changes": undisclosed,
            "counts": {"added": len(added), "removed": len(removed), "modified": len(modified)}}


def manuscript_disclosure():
    rec = {}
    for k, v in QC.DISCLOSED_MANUSCRIPT_REVISIONS.items():
        cur = sha(ROOT / k) if (ROOT / k).is_file() else None
        rec[k] = {"current_equals_disclosed_p20": cur == v["p20_sha256"], "differs_from_p19": cur != v["p19_sh
  +> a256"],
                  "revised_in": v["revised_in"]}
    return all(x["current_equals_disclosed_p20"] for x in rec.values()), rec


def _main():
    low_ok, low = lower_layers()
    l20_ok, l20, r20 = p20_layer()
    pin_ok, pin, pc = R20.d11_pin_check()
    p20i_ok, p20i = p20_independent()
    md_ok, md = manuscript_disclosure()
    hard = dict(QC.HARD_STOPS_P21)
    hard_ok = bool(all(r20.get("hard_stops", {}).get(k) == v for k, v in hard.items())
                   and r20.get("bulk_symmetry_restored") is False)
    cl = change_log()
```

```
  execution_pass = bool(low_ok and l20_ok and pin_ok and hard_ok and md_ok and cl["base_pinned_ok"]
                        and not cl["contract_files_changed"] and not cl["undisclosed_package_changes"]
                        and p20i.get("pins_ok") is True)
  publish = execution_pass
  author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
  indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
  earned = {k: ("earned" if (publish and r20.get(k) == "earned") else "not_earned") for k in QC.EARNABLE_P21
+> }
  ms = json.loads(json.dumps(r20.get("milestones", {})))
  cert = dict(r20)
  for k in ("change_log_vs_p19", "disclosed_test_revisions_p20", "disclosed_p19_layer_revisions_p20",
            "p19_layer", "erratum_p19", "K1_correction", "retraction_check", "D11_pin", "open_pins",
            "open_pins_closed_in_p21", "p19_independent_rebuild", "independent_rebuild_passed_p19"):
      cert[f"p20_layer_{k}"] = cert.pop(k, None)
  # reviewer (p20): the P19 gate's change log is recomputed on the CURRENT tree; name it so
  if "p19_layer_change_log_vs_p18" in cert:
      cert["p19_gate_change_log_vs_p18_recomputed_on_current_tree"] = cert.pop("p19_layer_change_log_vs_p18"
+> )
  cert.update({
      "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
+> TE,
      "version_label": QC.VERSION_LABEL,
      "deliverable": "p21 small release (absolute D11 pin derived; gate, scan, disclosure fixes)",
      "execution_pass": execution_pass,
      "claim_status": (("p21 small release from p20 (no scope expansion). D11 absolute pin: the sign and mag
+> nitude "
                        "of the D11 UV pole are derived from 1/2 Tr ln of the minimal scalar loop under the
+> "
                        "package's declared conventions and equal the heat-kernel a_2 = C^2/120 + R^2/72 exa
+> ctly "
                        "(Gamma^(2)|_pole = kappa^2/(16 pi^2 eps) [p^4 P2/240 + p^4 P0s/24]); external input
+> s are "
                        "mathematics only (a_2 coefficients, heat-kernel pole formula), no external conventi
+> on "
                        "choice. Projected (ch-only) results as in p20; bulk symmetry NOT restored/claimed;
+> "
                        "O.1 NOT_CLOSED.") if publish else "p21: not accepted"),
      "absolute_pin": {"check": pin, "absolute_pin": pc.get("absolute_pin"),
                       "absolute_pin_status": pc.get("absolute_pin_status"), "mutations": pc.get("mutations"
+> )},
      "lower_layers": low, "p20_layer": l20, "manuscript_disclosure": md, "change_log_vs_p20": cl,
      "open_pins": list(QC.OPEN_PINS_P21),
      "release_claim_accepted": publish,
      "release_claim_accepted_definition": ("lower layers re-checked live (contract, p18 incl. q earned, inv
+> entory, "
                                            "p19 layer, K1, retraction) AND p20 layer accepted (disclosed p2
+> 1 "
                                            "revisions of d11_pin.py and release_p20.py only) AND the absolu
+> te D11 "
                                            "pin AND every package change disclosed AND the pinned p20 indep
+> endent "
                                            "record AND the p21 hard stops"),
      "publication": "accepted" if publish else "REJECTED",
      **earned,
      "disclosed_p20_layer_revisions_p21": QC.DISCLOSED_P20_LAYER_REVISIONS_P21,
      "disclosed_test_revisions_p21": QC.DISCLOSED_TEST_REVISIONS_P21,
      "disclosed_manuscript_revisions": QC.DISCLOSED_MANUSCRIPT_REVISIONS,
      "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": ms,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p20": p20i_ok, "p20_independent_rebuild": p20i,
      "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p21
+> ) source "
                             "tree only (independent: pending until a reviewer report for this tree exists);
+>  "
                             f"independent_rebuild_passed_p20 refers to the p20 tree {QC.P20_SOURCE_TREE_SHA
+> 256} / zip "
                             f"{QC.P20_SOURCE_ZIP_SHA256} (same-box caveat) and is not inherited by p21"),
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p20_archive": ARCH_P20,
                          "kind": "package evidence (XVI-0 conv:package); not a mathematical certificate"},
      "XVI_P_registered": False, "symmetry_restored": False, "bulk_symmetry_restored": False,
      "O1_closure": "NOT_CLOSED", "insertion_status": "Frozen_negative_obstruction_preserved",
      "XV_B_opened": False, "XIV_J_opened": False, "Route_B_discharged": False, "u_R_c_delta_added": False,
      "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
          n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
          ("P20_release.json", "P20_D11_pin.json") if (ROOT / "certificates" / n).is_file()},
  })
  if publish:
      ms.setdefault("status", {})["P.5_identity"] = str(ms.get("status", {}).get("P.5_identity", "")) + \
```

```
            " [p21: D11 absolute pin derived and verified]"
    cdir = ROOT / "certificates"
    if publish:
        txt = certio.dumps(cert)
        for st in STATUS_FILES:
            (ROOT / st).write_text(txt)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
        certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
            "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
  +> tatus)",
            **ms, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
            "independent_rebuild_passed_p20": p20i_ok, "hard_stops": hard,
            "flags": {**earned, "quantum_inputs_evaluated": cert.get("quantum_inputs_evaluated"),
                      "bulk_symmetry_restored": False}})
    else:
        _reject({"lower_layers_ok": low_ok, "lower_layers": low, "p20_layer_ok": l20_ok, "p20_layer": l20,
                 "absolute_pin_ok": pin_ok, "absolute_pin": pin, "manuscript_disclosure_ok": md_ok,
                 "p20_independent_pins_ok": p20i.get("pins_ok"), "hard_stops_hold": hard_ok,
                 "change_log_base_ok": cl["base_pinned_ok"], "contract_files_changed": cl["contract_files_chan
  +> ged"],
                 "undisclosed_package_changes": cl["undisclosed_package_changes"]})
    print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
  +> ion"],
                      "lower_layers": low_ok, "p20_layer": l20_ok, "absolute_pin": pin_ok, "manuscript": md_ok
  +> ,
                      "hard_ok": hard_ok, **earned, "author": author_ok, "independent_p21": indep_ok,
                      "independent_p20": p20i_ok, "undisclosed": cl["undisclosed_package_changes"],
                      "change_log_counts": cl["counts"]}, indent=1))
    return 0 if publish else 1


def _reject(detail):
    cdir = ROOT / "certificates"
    (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
    for st in STATUS_FILES:                      # a reject must not leave a stale RELEASE_STATUS.json
        (ROOT / st).unlink(missing_ok=True)
    (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
        {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", **detail}, indent=1, default
  +> =str) + "\n")


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        _reject({"error": f"{type(exc).__name__}: {exc}"})
        print(f"release_p21 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

157 Source: tests/test__p21.py

[listing 124/133] tests/test_p21.py

```
"""XVI-P p21 tests: absolute D11 pin (sign and magnitude derived from 1/2 Tr ln under the declared conventions
  +> );
gate fixes (top-level pin status, lower layers re-checked); ALLOW-LIST scan + runtime opaque-d check for the
d-evaluation contract (a check, NOT A PROOF); disclosure fixes; p21 gate fail-closed with no stale RELEASE_STA
  +> TUS."""
from __future__ import annotations
import ast
import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from collections import Counter
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
```

```
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools", "tests"):
    sys.path.insert(0, str(ROOT / p))
import p21_conventions as QC  # noqa: E402
import p23_conventions as QS  # noqa: E402   (p23: scan allow-lists incl. the reviewed p22/p23 guard sites)

CERT = ROOT / "certificates"
SCAN_DIRS = ("code/p5_calc", "code/p5_entry")


def cj(name):
    return json.loads((CERT / name).read_text())


def rs():
    return json.loads((CERT / "P21_release.json").read_text())      # p22: the p21 LAYER record


# ============================================================ A: absolute D11 pin
def test_absolute_pin_certificate():
    c = cj("P20_D11_pin.json")
    a = c["absolute_pin"]
    d = a["derivation_p21"]
    assert a["status"] == "PASS" and c["absolute_pin_status"] == a["status"] and c["D11_absolute_pin_verified"
  +> ] is True
    assert all(v is True for v in d["package_declarations_checked"].values()) and len(d["package_declarations_
  +> checked"]) == 8
    st = d["steps"]
    assert st["1_G_equals_Gamma2"] == "1" and st["2_wick_sign_s_W_from_N_eps_and_Ihat_pole"] == "1"
    assert st["3_i_S_M_over_S_E (must be -1)"] == "-1" and st["4_L_M_eff_over_L_E_eff"] == "-1"
    assert st["6_minkowski_quadratic_forms"]["C2"] == {"P2": "1/2", "P0s": "0"}
    assert st["6_minkowski_quadratic_forms"]["R2"] == {"P2": "0", "P0s": "3"}
    assert d["package_Gamma2_pole_per_kappa2_over_eps"] == {"spin2": "1/(3840*pi**2)", "spin0s": "1/(384*pi**2
  +> )"}
    assert d["heat_kernel_prediction_per_kappa2_over_eps"] == d["package_Gamma2_pole_per_kappa2_over_eps"]
    assert d["ratio_package_over_prediction"] == {"spin2": "1", "spin0s": "1"}
    assert d["convention_choices_external"].startswith("none")


def test_absolute_pin_live_and_mutations():
    import d11_pin as P
    import covariant as C
    eta = tuple(int(x) for x in C.ETA)
    p = (sp.Integer(2), sp.Rational(1, 3), sp.Integer(-1), sp.Integer(4))
    c2, c0 = -sp.I / 240, -sp.I / 24
    assert P.absolute_pin(c2, c0, eta, p)[1] is True
    assert P.absolute_pin(2 * c2, 2 * c0, eta, p)[1] is False          # x2
    assert P.absolute_pin(-c2, -c0, eta, p)[1] is False                # sign flip
    assert P.absolute_pin(sp.I * c2, sp.I * c0, eta, p)[1] is False    # phase error
    assert P.absolute_pin(c2, 2 * c0, eta, p)[1] is False              # spin-0 only
    # 2 x pole magnitude = kappa^2/(240 16 pi^2) (reviewer's magnitude): N_eps c2 = 1/(240 16 pi^2)
    assert sp.simplify(sp.I / (16 * sp.pi ** 2) * c2 - 1 / (240 * 16 * sp.pi ** 2)) == 0


def test_absolute_pin_minkowski_forms_match_euclidean():
    import d11_pin as P
    p = (sp.Integer(1), sp.Integer(2), sp.Integer(-3), sp.Rational(1, 2))
    fm = P.curvature_forms_eta(p, (1, -1, -1, -1))
    fe = P.curvature_forms(p)
    assert fm == fe


def test_absolute_pin_breaks_if_a_declared_convention_changes(monkeypatch):
    import d11_pin as P
    import conventions as CC
    import covariant as C
    eta = tuple(int(x) for x in C.ETA)
    p = (sp.Integer(2), sp.Rational(1, 3), sp.Integer(-1), sp.Integer(4))
    monkeypatch.setattr(CC, "METRIC_PARAMETRISATION", "g_{mu nu} = eta_{mu nu} + 2 kappa h_{mu nu}")
    assert P.absolute_pin(-sp.I / 240, -sp.I / 24, eta, p)[1] is False


# ============================================================ G: gate fixes
def test_p20_gate_checks_top_level_and_lower_layers():
    import release_p20 as R
    ok, chk, _ = R.d11_pin_check()
    assert ok and chk["top_level_absolute_status_consistent"] and chk["top_level_ratio_status_consistent"]
    ok19, l19, _ = R.p19_layer()
    assert ok19 and l19["lower_layers_rechecked_p21"]["P18_quantum_inputs_evaluated"] == "earned"


# ============================================================ S: allow-list scan (a check, NOT A PROOF)
```

```
def _fn_index(tree):
    fn = {}
    for n in ast.walk(tree):
        if isinstance(n, (ast.FunctionDef, ast.AsyncFunctionDef)):
            for c in ast.walk(n):
                fn.setdefault(id(c), n.name)
    return fn


def _mentions_d(node, bare=False):
    """node is / contains the dimension: Name 'd' or attribute '.d' (dred.d, D.d); bare: node itself only."""
    def is_d(x):
        return (isinstance(x, ast.Name) and x.id == "d") or (isinstance(x, ast.Attribute) and x.attr == "d")
    return is_d(node) if bare else any(is_d(x) for x in ast.walk(node))


def scan4(path, root=ROOT):
    """ALLOW-LIST scan.  Refuses, except at reviewed sites: (a) ANY access to an expression-tree decomposer
    (QC.DECOMPOSE_ATTRS: .args, make_args, as_ordered_terms, as_coefficients_dict, ...), whatever is done with
  +>  the
    result afterwards (variable, map, sum, starred rebuild); (b) binding any sympy attribute other than the al
  +> lowed
    constants to a name (Q = sp.Poly), and any 'import sympy as X' / 'from sympy import Y as Z' alias; (c) a s
  +> ympy
    symbol constructor whose name is not a literal (starred, **kwargs, non-literal keyword, computed name);
    (d) p21 runtime finding: a value call (QC.VALUE_CALLS: subs, evalf, N, limit, Lambda, ...) whose arguments
  +>  mention
    d, and any binding of d to another name (D = dred.d) -- the runtime guard does NOT intercept subs when d o
  +> ccurs
    only as a power base, so subs on d is refused statically except at reviewed demonstration sites.
    Returns a list of violations (site keys)."""
    src = Path(path).read_text()
    tree = ast.parse(src)
    fn = _fn_index(tree)
    rel = Path(path).resolve().relative_to(root.resolve()).as_posix() if Path(path).is_absolute() or True else
  +>  path
    sympy_names = {"sp", "sympy"}
    out = []

    def key(n):
        return f"{rel}::{fn.get(id(n), '<module>')}::{ast.get_source_segment(src, n)}"

    for n in ast.walk(tree):
        if isinstance(n, ast.Import):
            for a in n.names:
                if a.name.split(".")[0] == "sympy" and a.asname not in (None, "sp"):
                    out.append(f"ALIAS {key(n)}")
        if isinstance(n, ast.ImportFrom) and (n.module or "").split(".")[0] == "sympy":
            for a in n.names:
                if a.asname is not None or a.name not in QC.SYMPY_ALIAS_ALLOWED:
                    out.append(f"ALIAS {key(n)}")
        if isinstance(n, (ast.Assign, ast.AnnAssign, ast.NamedExpr)):
            v = n.value
            if isinstance(v, ast.Attribute) and isinstance(v.value, ast.Name) and v.value.id in sympy_names \
                    and v.attr not in QC.SYMPY_ALIAS_ALLOWED:
                out.append(f"ALIAS {key(n)}")
            if isinstance(v, ast.Name) and v.id in sympy_names:
                out.append(f"ALIAS {key(n)}")
        if isinstance(n, ast.Attribute) and n.attr in QC.DECOMPOSE_ATTRS:
            out.append(f"DECOMPOSE {key(n)}")
        if isinstance(n, ast.Name) and n.id in QC.DECOMPOSE_ATTRS and isinstance(n.ctx, ast.Load):
            out.append(f"DECOMPOSE {key(n)}")
        if isinstance(n, (ast.Assign, ast.AnnAssign, ast.NamedExpr)) and _mentions_d(n.value, bare=True):
            tg = getattr(n, "targets", None) or [n.target]
            if not all(isinstance(x, ast.Name) and x.id == "d" for x in tg):
                out.append(f"DALIAS {key(n)}")
        if isinstance(n, ast.Call):
            f = n.func
            fname = f.attr if isinstance(f, ast.Attribute) else (f.id if isinstance(f, ast.Name) else "")
            if fname in QC.VALUE_CALLS and any(_mentions_d(a) for a in list(n.args) + [k.value for k in n.keyw
  +> ords]):
                out.append(f"DVALUE {key(n)}")
            is_maker = ((isinstance(f, ast.Attribute) and isinstance(f.value, ast.Name) and f.value.id in symp
  +> y_names
                         and f.attr in QC.SYMBOL_MAKERS) or (isinstance(f, ast.Name) and f.id in QC.SYMBOL_MAK
  +> ERS))
            if is_maker:
                bad = any(isinstance(a, ast.Starred) for a in n.args) or any(k.arg is None for k in n.keywords
  +> )
                bad |= any(k.arg in ("name", "names") and not isinstance(k.value, ast.Constant) for k in n.key
  +> words)
```

```
                lit = [k.value for k in n.keywords if k.arg in ("name", "names")] + n.args[:1]
                for a in lit:
                    if isinstance(a, ast.Constant):
                        bad |= not isinstance(a.value, str) or "d" in a.value.replace(",", " ").split()
                    elif isinstance(a, ast.JoinedStr):
                        head = a.values[0] if a.values else None
                        bad |= not (isinstance(head, ast.Constant) and str(head.value) not in ("", "d"))
                    else:
                        bad = True
                if bad:
                    out.append(f"SYMBOL {key(n)}")
    return out


def _package_violations():
    v = []
    for d in SCAN_DIRS:
        for f in sorted((ROOT / d).glob("*.py")):
            v += scan4(f)
    return v


def _unreviewed(viol):
    reviewed = {**{f"DECOMPOSE {k}": c for k, c in ((k, QS.DECOMPOSE_SITE_COUNTS.get(k, 1))
                                                    for k in QS.DECOMPOSE_SITES_REVIEWED)},
                **{f"SYMBOL {k}": 1 for k in QS.SYMBOL_SITES_REVIEWED},
                **{f"ALIAS {k}": 1 for k in QS.SYMPY_FROM_IMPORT_REVIEWED},
                **{f"DVALUE {k}": QS.DVALUE_SITE_COUNTS.get(k, 1) for k in QS.DVALUE_SITES_REVIEWED},
                **{f"DALIAS {k}": 1 for k in QS.DALIAS_SITES_REVIEWED}}
    cnt = Counter(viol)
    return {k: c for k, c in cnt.items() if reviewed.get(k) != c} | \
        {k: 0 for k in reviewed if k not in cnt}


def test_scan4_package_clean_against_allow_list():
    assert _unreviewed(_package_violations()) == {}


REVIEWER_PLANTS_P20 = {   # the 8 misses of test_p18.scan + test_p20.scan3 reported by the p20 independent reb
  +> uild
    "assign_alias_Poly": "import sympy as sp\nQ = sp.Poly\ndef f(e, d):\n    return Q(e, d).eval(4)\n",
    "map_over_args_rebuild": ("import sympy as sp\ndef f(e, d):\n"
                              "    return sp.Add(*map(lambda t: t.xreplace({}), e.args))\n"),
    "args_via_variable": "import sympy as sp\ndef f(e):\n    a = e.args\n    return sp.Add(*[t for t in a])\n"
  +> ,
    "ordered_terms_rebuild": "import sympy as sp\ndef f(e):\n    return sp.Add(*e.as_ordered_terms())\n",
    "builtin_sum_over_args": "def f(e):\n    return sum(t for t in e.args)\n",
    "symbol_starred_literal": "import sympy as sp\nd = sp.Symbol(*['d'])\n",
    "symbol_kwargs_dict": "import sympy as sp\nd = sp.Symbol(**{'name': chr(100)})\n",
    "make_args_rebuild": "import sympy as sp\ndef f(e):\n    return sp.Add(*[t for t in sp.Add.make_args(e)])\
  +> n",
}
EXTRA_PLANTS = {
    "from_import_alias": "from sympy import Poly as Q\n",
    "module_alias": "import sympy as S2\n",
    "module_rebind": "import sympy as sp\nX = sp\n",
    "func_rebuild": "def f(e):\n    return e.func(*e.args)\n",
    "coeff_dict": "def f(e):\n    return e.as_coefficients_dict()\n",
    "preorder": "from sympy import preorder_traversal\n",
    "symbol_computed": "import sympy as sp\nd = sp.Symbol(chr(100))\n",
    "symbol_kw_computed": "import sympy as sp\nd = sp.Symbol(name=chr(100))\n",
    "symbols_literal_d": "import sympy as sp\na, d = sp.symbols('a d')\n",
    "symbol_fstring_d": "import sympy as sp\nd = sp.Symbol(f'{chr(100)}')\n",
    "subs_d_power_base": "import dred\ndef f(e):\n    return e.subs(dred.d, 4)\n",
    "subs_d_dict": "from dred import d\ndef f(e):\n    return e.subs({d: 4})\n",
    "d_rebound": "import dred\nD = dred.d\n",
    "evalf_subs_d": "from dred import d\ndef f(e):\n    return e.evalf(subs={d: 4})\n",
    "walrus_alias": "import sympy as sp\nif (P := sp.Poly):\n    pass\n",
}


@pytest.mark.parametrize("name", sorted(REVIEWER_PLANTS_P20) + sorted(EXTRA_PLANTS))
def test_scan4_detects_plants(name, tmp_path):
    src = {**REVIEWER_PLANTS_P20, **EXTRA_PLANTS}[name]
    f = tmp_path / "plant.py"
    f.write_text(src)
    assert scan4(f, root=tmp_path), name


def test_scan4_reviewer_plants_were_missed_by_pattern_scans(tmp_path):
    """honesty control: the reviewer's misses really pass the older pattern scans (so the allow-list adds cove
```

```
  +> rage)."""
    import test_p20 as T20
    missed = 0
    for name, src in REVIEWER_PLANTS_P20.items():
        f = tmp_path / f"{name}.py"
        f.write_text(src)
        try:
            hits = [x for part in T20.scan3(f, root=tmp_path) for x in part]   # scan3 -> (list, list)
        except Exception:      # noqa: BLE001
            hits = ["error"]
        missed += not hits
    assert missed == len(REVIEWER_PLANTS_P20) == 8


def test_scan4_allows_benign_forms(tmp_path):
    f = tmp_path / "ok.py"
    f.write_text("import sympy as sp\nI = sp.I\nx = sp.Symbol('x')\ny = sp.Symbol(f'p{3}')\n"
                 "a, b = sp.symbols('a b')\ndef g(e):\n    return sp.expand(e * x)\n")
    assert scan4(f, root=tmp_path) == []


def test_scan_wording_honest():
    txt = (ROOT / "README.md").read_text()
    assert "allow-list" in txt and "not a proof" in txt


# ------------------------------------------------------------------ runtime opaque-d check
def test_runtime_stored_exact_d_amplitudes_carry_opaque_d_and_reproduce_poles():
    """RUNTIME check: every stored exact-d D11 amplitude re-parses (parse_d) with the OPAQUE d (type check), c
  +> annot be
    evaluated at d = 4 by xreplace/replace/lambdify (subs: limitation recorded below), and reproduces its stor
  +> ed UV pole only via the authorised
    substitute_d path (integrals.laurent)."""
    import dred
    import integrals as IL
    res = cj("P18_D11.json")["result"]
    n = 0
    for k, v in res.items():
        A = dred.parse_d(str(v["A_exact_d_times_MB"]), {"I": sp.I})
        dsyms = [s for s in A.free_symbols if s.name == "d"]
        assert all(isinstance(s, dred.OpaqueDimension) for s in dsyms), k
        if dsyms:
            for bad in (lambda e: e.xreplace({dred.d: 4}),          # subs: see the recorded limitation below
                        lambda e: e.replace(dred.d, 4), lambda e: sp.lambdify(dred.d, e)(4)):
                with pytest.raises(dred.PrematureD4Substitution):
                    bad(A)
        pole = IL.laurent(A * IL.master_exact(), 0)[-1]
        assert sp.simplify(pole - dred.parse_d(str(v["UV_pole"]), {"I": sp.I})) == 0, k
        n += 1
    assert n >= 5


def test_runtime_guard_subs_limitation_recorded():
    """p21 finding (disclosed, contract unchanged): expr.subs(d, v) is NOT intercepted when d occurs only as a
  +>  power
    base (sympy's Pow substitution does not call the symbol's _subs).  The package has no such value path (sta
  +> tic
    rule (d) of scan4); the released premature-d4 demonstrations used components that DO raise."""
    import dred
    A = dred.parse_d("1/(d**2 - 1)")
    with pytest.raises(dred.PrematureD4Substitution):         # p22: the p21 limitation is CLOSED by the runti
  +> me guard
        A.subs(dred.d, 4)
    with pytest.raises(dred.PrematureD4Substitution):
        dred.parse_d("1/(d - 1)").subs(dred.d, 4)
    assert "power base" in QC.RUNTIME_GUARD_LIMITATION_P21
    for f, path in (("P18_D11.json", ("mutations", "premature_d4")), ("P16_batchC.json", ("mutations", "premat
  +> ure_d4"))):
        x = cj(f)
        for p in path:
            x = x[p]
        assert x["rejected"] is True, f


# ============================================================ D: disclosure
def test_p19_note_edit_disclosed():
    md = QC.DISCLOSED_MANUSCRIPT_REVISIONS["manuscript/notes/P19_defect.tex"]
    cur = hashlib.sha256((ROOT / "manuscript/notes/P19_defect.tex").read_bytes()).hexdigest()
    assert cur == md["p20_sha256"] != md["p19_sha256"] and md["revised_in"] == "p20"
    assert rs()["manuscript_disclosure"]["manuscript/notes/P19_defect.tex"]["current_equals_disclosed_p20"] is
  +>  True
```

```
def test_change_log_key_renamed():
    s = rs()
    assert "p19_layer_change_log_vs_p18" not in s and "p19_gate_change_log_vs_p18_recomputed_on_current_tree"
  +> in s
    src = (ROOT / "tools" / "rebuild_from_source.py").read_text()       # the reference-value generator
    assert '"change_log_vs_p18":' not in src and '"change_log_vs_p19":' not in src
    assert '"p19_gate_change_log_vs_p18_recomputed_on_current_tree"' in src
    assert '"p20_gate_change_log_vs_p19_recomputed_on_current_tree"' in src


# ============================================================ status
def test_p21_release_status_and_flags():
    s = rs()
    assert s["revision"] == "p21" and s["publication"] == "accepted" and s["execution_pass"] is True
    for k in QC.EARNABLE_P21:
        assert s[k] == "earned", k
    assert s["bulk_symmetry_restored"] is False and s["symmetry_restored"] is False and s["O1_closure"] == "NO
  +> T_CLOSED"
    assert s["hard_stops"] == QC.HARD_STOPS_P21 and s["hard_stops_hold"] is True
    assert s["independent_rebuild_passed"] is not True and s["independent_rebuild_passed_p20"] is True   # p22
  +>  tree
    assert s["open_pins"] == list(QC.OPEN_PINS_P21) and s["lower_layers"]["rechecked_live"] is True
    assert s["change_log_vs_p20"]["undisclosed_package_changes"] == []


def test_p20_independent_recorded_and_not_inherited():
    rec = rs()["p20_independent_rebuild"]
    assert rec["report_sha256"] == QC.P20_INDEPENDENT_REPORT_SHA256 and rec["pins_ok"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P20_SOURCE_TREE_SHA256
    assert rec["bound_to_source_zip_sha256"] == QC.P20_SOURCE_ZIP_SHA256 and rec["mismatches"] == 0
    import source_manifest as SM
    assert SM.tree_sha(ROOT) != QC.P20_SOURCE_TREE_SHA256


def test_layers_and_disclosed_revisions():
    assert hashlib.sha256((ROOT / "code/p5_calc/p20_conventions.py").read_bytes()).hexdigest() == QC.P20_CONVE
  +> NTIONS_SHA256
    for k, h in QC.P20_RELEASED_SHA256.items():
        assert hashlib.sha256((ROOT / k).read_bytes()).hexdigest() != h and k in QC.DISCLOSED_P20_LAYER_REVISI
  +> ONS_P21


def test_change_log_vs_p20():
    cl = rs()["change_log_vs_p20"]
    import contract_registry as CR                                  # p22: the p22 revised files are disclosed t
  +> here
    assert cl["base_pinned_ok"] is True and cl["contract_files_changed"] == []
    assert sorted(x for x in cl["modified"] if x.startswith("tests/") and x not in CR.P22_REVISED_FILES) == \
        sorted(x for x in QC.DISCLOSED_TEST_REVISIONS_P21 if x not in CR.P22_REVISED_FILES)
    import contract_registry as CR                                  # p22: the p22 revised files are disclosed t
  +> here
    assert sorted(x for x in cl["modified"] if x.startswith("code/") and x not in CR.P22_REVISED_FILES) == \
        sorted(x for x in QC.DISCLOSED_P20_LAYER_REVISIONS_P21 if x not in CR.P22_REVISED_FILES)
    for f in ("code/p5_calc/p21_conventions.py", "code/p5_calc/release_p21.py", "tests/test_p21.py"):
        assert f in cl["added"], f


def test_version_label_p21():
    import build as B
    assert QC.VERSION_LABEL == "XVI-P p21" and B.QC.VERSION_LABEL == f"XVI-P {B.QC.REVISION}"   # p22: current
  +>  label


# ============================================================ p21 gate mutations (fail closed; no stale s
  +> tatus)
def _copy():
    import test_p20 as T20
    d = T20._copy()
    import source_manifest as SM
    for rel in SM.source_files(ROOT):          # the p21 gate checks the change log of the whole source tree
        if not (d / rel).exists():
            (d / rel).parent.mkdir(parents=True, exist_ok=True)
            shutil.copy(ROOT / rel, d / rel)
    return d


def _edit(d, name, fn):
    f = d / "certificates" / name
    c = json.loads(f.read_text())
```

```
    fn(c)
    f.write_text(json.dumps(c, indent=1))


def _abs_fail(d):
    _edit(d, "P20_D11_pin.json", lambda c: c["absolute_pin"].__setitem__("status", "FAIL"))


def _abs_top_level_inconsistent(d):
    _edit(d, "P20_D11_pin.json", lambda c: c.__setitem__("absolute_pin_status", "OPEN"))


def _abs_x2_undetected(d):
    _edit(d, "P20_D11_pin.json", lambda c: c["mutations"]["D11_normalisation_x2"].__setitem__(
        "detected_by_absolute_pin", False))


def _p18_q_not_earned(d):
    _edit(d, "P18_release.json", lambda c: c.__setitem__("quantum_inputs_evaluated", "not_earned"))


def _p20_layer_rejected(d):
    _edit(d, "P20_release.json", lambda c: c.__setitem__("publication", "REJECTED"))


def _p20_conventions_changed(d):
    f = d / "code" / "p5_calc" / "p20_conventions.py"
    f.write_text(f.read_text() + "\n# tampered\n")


def _undisclosed_code_change(d):
    f = d / "code" / "p5_calc" / "hh_slot.py"
    f.write_text(f.read_text() + "\n# undisclosed\n")


def _tamper_p20_report(d):
    f = d / "archival" / "p20" / "reproducibility" / "independent_rebuild_report_p20.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


def _p19_note_changed(d):
    f = d / "manuscript" / "notes" / "P19_defect.tex"
    f.write_text(f.read_text() + "\n% edit\n")


def _k1_closure_broken(d):
    _edit(d, "P19_defect.json", lambda c: c["counterterm_solve"]["K1_ghost_closure_convention"][
        "full_closure_complete_S_ct"].__setitem__("closed", False))


MUTS = [_abs_fail, _abs_top_level_inconsistent, _abs_x2_undetected, _p18_q_not_earned, _p20_layer_rejected,
        _p20_conventions_changed, _undisclosed_code_change, _tamper_p20_report, _p19_note_changed, _k1_closure
  +> _broken]


def _run(d, script="code/p5_calc/release_p21.py"):
    return subprocess.run([sys.executable, script], cwd=d, capture_output=True, text=True, timeout=600)


@pytest.mark.parametrize("mut", MUTS, ids=lambda f: f.__name__)
def test_certificate_mutation_p21_gate_rejects_without_stale_status(mut):
    d = _copy()
    try:
        assert (d / "RELEASE_STATUS.json").exists()
        mut(d)
        r = _run(d)
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d / "certificates" / "P21_release.json").exists()
        assert (d / "certificates" / "P21_release_rejected.json").exists()
        assert not (d / "RELEASE_STATUS.json").exists() and not (d / "certificates" / "RELEASE_STATUS.json").e
  +> xists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_p21_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = _run(d)
        assert r.returncode == 0, r.stdout[-1500:]
        s = json.loads((d / "RELEASE_STATUS.json").read_text())
```

```
        assert s["revision"] == "p21" and s["D11_absolute_pin_verified"] == "earned"
        assert s["bulk_symmetry_restored"] is False
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

158 Source: p22_conventions.py (p22 constants, version label, disclosures, guard plants)

[listing 125/133] code/p5_calc/p22_conventions.py

```
#!/usr/bin/env python3
"""XVI-P p22 conventions: the 'bigger fronts' release after p21 (user-approved), WITH a disclosed contract rev
  +> ision.

  EV-M2  evanescent components of D11: the provable matching statement (local evanescent poles + exact decoupl
  +> ing of the
         renormalised evanescent remainder on QdS external legs); NOT matched to XVI-O (no counterpart; the g4
  +>  -> ghat and
         g4-kept continuations of the 4D kernel are both refuted).  D11_matching_verified_scope widened accord
  +> ingly.
  EV-CT  p22 CONTRACT REVISION (declared): evanescent counterterms -- h.h gt-operators (DO1 + DO3 + D11), K'.c
  +>  E2
         (D13), and the ghost-equation partner cb.c SE (Sigma) -- coefficient = -(UV pole).  Checked: sigma_0
  +> closure
         (h.c, c.b; c.cbs unreachable), projected identity unchanged, remnants (QdS: none; Q4S trace mode: a f
  +> inite
         O(eps) x pole term, disclosed as a scheme convention).  P.5 calculation status -> renormalized (one l
  +> oop,
         kappa^2, two-point functions, MS-bar DRED + p22 evanescent counterterms, QdS external legs).
  G      p22 CONTRACT REVISION: runtime d guard (code/p5_entry/dred.py, one delimited additive block; the p18
  +> text is
         byte-identical outside it -- contract_registry.dred_revision).  Entry-point guards on subs / xreplace
  +>  /
         replace / evalf(subs) / Poly value + coefficient accessors / coeff / as_independent / as_coeff_expone
  +> nt / series
         / collect / limit / Lambda(d, .) / d == d.  The static allow-list scan is kept as the SECONDARY barri
  +> er.
  K      gate: no prefix exemptions (tools/, manuscript/ included); every added, modified or removed file vs t
  +> he p21
         zip must be listed in DISCLOSED_REVISIONS_P22; earlier-layer notes and added code files are covered.
  T      science text: the p20 'Absolute pin: OPEN' passage is marked inline [superseded in p21].
"""
from __future__ import annotations
from p21_conventions import (ROW_HH, ROW_KC, ROW_1PT, ROWS, HH_EVALUATED, P18_CALC_SHA256,  # noqa: F401
                             P18_DRED_SHA256, RUN_SPECIFIC_KEYS, HARD_STOPS_P21, EARNABLE_P21, OPEN_PINS_P21,
                             INVENTORY_P21, DECOMPOSE_ATTRS, SYMPY_ALIAS_ALLOWED, SYMBOL_MAKERS)
import p21_conventions as _Q21

REVISION = "p22"
VERSION_LABEL = "XVI-P p22"
CERT_PREFIX = "P22"
CONTRACT_REVISION = ("p22 (p14r contract + disclosed p15/p16/p17/p18 revisions + the DISCLOSED p22 contract re
  +> vision: "
                     "evanescent counterterms (EV-CT) and the runtime d guard in code/p5_entry/dred.py)")
DATE = "2026-10-10"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p22.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p22.json"
P21_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p21.json"      # written by the reviewer;
  +>  never here
P21_INDEPENDENT_REPORT_SHA256 = "8a582b332cd242689c54b1f962dc88270b9592040c0ba432576a505f0e36a60d"
P21_SOURCE_TREE_SHA256 = "b835353ffe16a8ff159e2916fe3efdf2b9d89a4d512e80e9e6da587cecd0fe5e"
P21_SOURCE_ZIP_SHA256 = "a71d6b5203bc3914833db9fd13b9d186bef93d1ae3007b2e4c99fdfce11e0678"
P21_INDEPENDENT_SAME_BOX = True     # disclosed: the p21 independent rebuild ran on the same box
P21_SOURCE_LIST = "archival/p21/p21_source_files.json"
P21_ARCHIVED_RECORDS = {
    "archival/p21/reference_values_p21.json": "96ae946bb9d9c59e7128e3497669e3e9987b046e921f87e7f9a7e425d6aeeb4
  +> 6",
    "archival/p21/reproducibility/author_rebuild_report_p21.json":
        "a5f886ad6ac6b8cff9580236e7c3ddf2ab6d8d1f875173eb092ff01e4a8fb0cf",
    "archival/p21/reproducibility/independent_rebuild_report_p21.json": P21_INDEPENDENT_REPORT_SHA256,
    "archival/p21/p21_source_files.json": "b1f8704b51d1b2b223de6fe83c4956628f19d50682823ef8757af775405778b8",
}
INVENTORY_P22 = INVENTORY_P21
HARD_STOPS_P22 = dict(HARD_STOPS_P21)
EARNABLE_P22 = EARNABLE_P21 + ("evanescent_M2_verified", "evanescent_counterterms_verified", "d_runtime_guard_
  +> verified")
OPEN_PINS_P22 = tuple(OPEN_PINS_P21)
```

```
P5_STATUS_P22 = ("renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-bar in DRED with t
  +> he p22 "
                "evanescent counterterms; external legs in QdS)")

# ------------------------------------------------------------- p21 layer in p22
P21_CONVENTIONS_SHA256 = None        # filled at freeze (tools/p22_pins.py) from the p21 zip file list
# every file added / modified / removed vs the p21 zip, with its reason (K: no prefix exemptions)
DISCLOSED_REVISIONS_P22 = {
    # contract (p22 contract revision)
    "code/p5_entry/dred.py": "CONTRACT REVISION p22: runtime d guard (one delimited additive block; p18 text u
  +> nchanged)",
    # new p22 code
    "code/p5_calc/p22_conventions.py": "new: p22 conventions",
    "code/p5_calc/evanescent.py": "new: EV-M2 and EV-CT (priorities 1, 2)",
    "code/p5_calc/contract_registry.py": "new: registry of the post-p18 contract revision (consulted by layer
  +> gates)",
    "code/p5_calc/release_p22.py": "new: p22 release gate",
    # earlier-layer gates (disclosed p22 revisions; consult contract_registry)
    "code/p5_calc/release_p18.py": "p22: accept the registered dred.py revision in place of the p17 AST identi
  +> ty",
    "code/p5_calc/release_p19.py": ("p22: contract identity accepts the registered dred.py revision; p18-layer
  +>  pin "
                                    "accepts the registered release_p18.py revision"),
    "code/p5_calc/release_p20.py": ("p22: change log ignores the registered contract revision; p18-layer pin a
  +> ccepts "
                                    "the registered release_p18.py revision"),
    "code/p5_calc/release_p21.py": ("p22: change log ignores the registered contract revision and treats the p
  +> 22 "
                                    "revised files (contract_registry.P22_REVISED_FILES) as disclosed"),
    # tests
    "tests/test_p14.py": "names p21 -> p22",
    "tests/test_p16.py": "the replace(lambda x: x == d, ..) demonstration now RAISES (p22 runtime guard)",
    "tests/test_p17.py": ("g1 scan reviews the p18 contract core of the registered dred revision "
                          "(contract_registry.scan_view; the p22 block is covered by the runtime probe + scan4
  +> )"),
    "tests/test_p18.py": ("h1 scan via contract_registry.scan_view; h3/h5 accept the registered p18+p22-block
  +> dred state"),
    "tests/test_p19.py": ("i2 scan via contract_registry.scan_view; p18 calc layer accepts the registered p22
  +> revision "
                          "of release_p18.py (contract_registry.layer_file_ok)"),
    "tests/test_p20.py": ("status / version expectations p22; p21 tree no longer current; scan3 via scan_view;
  +>  p18 calc "
                          "layer via contract_registry.layer_file_ok"),
    "tests/test_p21.py": ("scan allow-lists read from p22_conventions; the power-base subs limitation test now
  +>  asserts the "
                          "p22 fix; status/version expectations p22; p21 layer independent flag not current"),
    "tests/test_p22.py": "new: p22 tests",
    # manuscript
    "manuscript/notes/P20_correction.tex": "inline [superseded in p21] marker on the 'Absolute pin: OPEN' item
  +> ",
    "manuscript/notes/P22_evanescent.tex": "new: p22 note",
    "manuscript/body/XVI-P-body.tex": "input of the p22 note",
    # tools / docs / reproducibility
    "tools/build.py": "p22 build: release_p22 stage, p22 listings, value blocks, generated files",
    "tools/source_manifest.py": "reference file / zip name p22; p21 archive records",
    "tools/rebuild_from_source.py": "p22 reference values, release criteria, archival pins",
    "README.md": "p22 section", "REBUILD_INSTRUCTIONS.md": "p22 instructions",
}
DISCLOSED_PREFIXES_ADDED_P22 = ("archival/p21/",)     # the p21 archive (records pinned in P21_ARCHIVED_RECORD
  +> S)
GENERATED_EXEMPT_P22 = ("reproducibility/reference_values_p22.json",
                        "reproducibility/reference_values_p21.json")   # generated p21 reference, archived und
  +> er archival/p21
DISCLOSED_MANUSCRIPT_REVISIONS_P22 = {
    "manuscript/notes/P20_correction.tex": {
        "p21_sha256": "138c858852906d4daf3ed86d2e48a8d5ee8ef48717514bde00d4ca7151d3e7fb", "revised_in": "p22",
        "change": "inline [superseded in p21] marker on the 'Absolute pin: OPEN' item (reviewer stale_OPEN_tex
  +> t)"},
}
P22_PINNED_SHA256 = {   # frozen: exact sha256 of every disclosed changed/added code/test/tool/note file (p22_
  +> conventions itself, README.md, REBUILD_INSTRUCTIONS.md: tree/zip sha only -- their manifest hashes must n
  +> ot appear in a PDF)
    "code/p5_calc/contract_registry.py": "70367b9df076c582214459f3afedf534aafd528b5fbae0659c588f2e9a1b2c22",
    "code/p5_calc/evanescent.py": "8e5755f435ed822ea2ba033dabc6ec6237a494337e429ef0793d38b9132da7a6",
    "code/p5_calc/release_p18.py": "f2e8743b4902b4304679c68b994dfc9c6df4a0b769827eb2693a72495a4e9c5b",
    "code/p5_calc/release_p19.py": "4d048345270a39a77cc30f71d3bdcbef17fffec6c44c1ac89487135ea3555970",
    "code/p5_calc/release_p20.py": "a1afba60c21c923886e682b960dda8b2619353712517f078c9967ad9c57e4d5f",
    "code/p5_calc/release_p21.py": "be1257870ad2944db009f1fe9f68f4c5cbadcb53f391d64ae881fc6b97330329",
    "code/p5_calc/release_p22.py": "8f6a0a2fd3d572f37a1dfe4bff26a56cbc7d4421fd8bc17572728077a1c2f269",
    "code/p5_entry/dred.py": "1f994759452b9d76eea3cc3a90e02bbb1ed237f9c0865a99d1bc5a2cbe0b08bc",
    "manuscript/body/XVI-P-body.tex": "fadeae094ce611234ef3a15c79826b3b00172fd9759697803d3d68758506629b",
```

```
    "manuscript/notes/P20_correction.tex": "b36516c84f6bb1acc5b80b7c384e16c54a0ec2a26af0f4a9a3dc81e98e634ddf",
    "manuscript/notes/P22_evanescent.tex": "49c8ff716c5f916dfcfd773613652cdb1190e226e68256d27ab2d68ddd97345c",
    "tests/test_p14.py": "0b8292eac118aaaa0ac56495f81ed75a92fb6f5bf86deb336343883e483c89a5",
    "tests/test_p16.py": "3abd2bca6d1cb29ef4a1a788e1ba1249daf31035e825bb10228b1aa2f1c29ed3",
    "tests/test_p17.py": "1e542ce116a5c5508bab74761fcd4b591f379840a814895ea7733c3c97ae2028",
    "tests/test_p18.py": "757676b25a71067d4150d0293b512d5d09da97c86686c16b97954ff8ac11e619",
    "tests/test_p19.py": "3578e13b4bfa9d429a551add3c3572672e569baec3517a69098aae158817a60c",
    "tests/test_p20.py": "274d9186979f00056f77ddf7a53e941f7e4e5e2d766a826ea291fe9555a4c668",
    "tests/test_p21.py": "067359ea2fa94e1228d5db1d3f7d7fc4f0af4e0468daa69b74638ccf23e5af0a",
    "tests/test_p22.py": "d8cbebcf34e960a59d718e2b8a3622dcc5f3941f012022cf3abc511d4801f663",
    "tools/build.py": "5e7ed48ee2f6dc5542a9da2579e736928660a947bb3a574da96db15d8e127248",
    "tools/rebuild_from_source.py": "56d7d205739e95574cd86ba820580ea4d39a3863bd008ed7e1ed827b800ab496",
    "tools/source_manifest.py": "46f1ae5f397a3f7e300b9e0679617b95b26a93d1a8e8da3f0eb106b43cc406c7",
}

# ------------------------------------------------------------ scan (secondary barrier): p21 allow-lists +
  +> p22 guard sites
DECOMPOSE_SITES_REVIEWED = dict(_Q21.DECOMPOSE_SITES_REVIEWED)
DECOMPOSE_SITES_REVIEWED.update({
    "code/p5_entry/dred.py::_pairs_of_subs::args": "the wrapper's *args (subs call arguments), not an expressi
  +> on tree",
    "code/p5_entry/dred.py::_install::args": "the wrappers' *args (call arguments), not an expression tree",
})
DECOMPOSE_SITE_COUNTS = dict(_Q21.DECOMPOSE_SITE_COUNTS)
DECOMPOSE_SITE_COUNTS.update({"code/p5_entry/dred.py::_pairs_of_subs::args": 5, "code/p5_entry/dred.py::_insta
  +> ll::args": 7})
SYMPY_FROM_IMPORT_REVIEWED = dict(_Q21.SYMPY_FROM_IMPORT_REVIEWED)
SYMPY_FROM_IMPORT_REVIEWED.update({
    "code/p5_entry/dred.py::<module>::from sympy.core.basic import Basic as _Basic": "p22 guard: patched entry
  +>  point",
    "code/p5_entry/dred.py::<module>::from sympy.core.expr import Expr as _Expr": "p22 guard: patched entry po
  +> int",
    "code/p5_entry/dred.py::<module>::from sympy.core.evalf import EvalfMixin as _EvalfMixin": "p22 guard",
    "code/p5_entry/dred.py::<module>::from sympy.polys.polytools import Poly as _Poly": "p22 guard",
    "code/p5_entry/dred.py::_install::from sympy.core.basic import Atom as _Atom": "p22 guard",
    "code/p5_entry/dred.py::_install::import sympy as _sp": "p22 guard",
    "code/p5_entry/dred.py::_install::from sympy.core.function import Lambda as _Lambda": "p22 guard",
})
SYMBOL_SITES_REVIEWED = dict(_Q21.SYMBOL_SITES_REVIEWED)
VALUE_CALLS = _Q21.VALUE_CALLS
DVALUE_SITES_REVIEWED = dict(_Q21.DVALUE_SITES_REVIEWED)
DVALUE_SITE_COUNTS = dict(_Q21.DVALUE_SITE_COUNTS)
DALIAS_SITES_REVIEWED = dict(_Q21.DALIAS_SITES_REVIEWED)
COEFF_SITES_REVIEWED = dict(_Q21.COEFF_SITES_REVIEWED)
POLY_GENERATOR_CALL_SITES_REVIEWED = dict(_Q21.POLY_GENERATOR_CALL_SITES_REVIEWED)

# ------------------------------------------------------------ runtime guard (G)
GUARD_PLANTS_P22 = {   # reviewer p21 probe (8 forms x 4 expressions) + p21 VALUE_CALLS + extra; every one mus
  +> t RAISE
    "subs_d": "e.subs(d, 4)", "xreplace_d": "e.xreplace({d: 4})", "list_unpack_alias_then_subs": "D, = [d]; e.
  +> subs(D, 4)",
    "free_symbols_pop_subs": "e.subs([y for y in e.free_symbols if y.name == 'd'][0], 4)",
    "lambda_wrapper_Poly": "(lambda *a: sp.Poly(*a))(sp.expand(e * (d**2 - 1)), d).eval(4)",
    "coeff_extract": "sum(sp.expand(e).coeff(d, k) * 4**k for k in range(5))",
    "dunder_getattribute_eq": "sp.Add(*[4 if y == d else y for y in e.__getattribute__('args')])  (d among arg
  +> s)",
    "subs_power": "e.subs(d**2, 16)", "subs_dict": "e.subs({d: 4})", "subs_list": "e.subs([(d, 4)])",
    "evalf_subs": "e.evalf(subs={d: 4})", "N_subs": "sp.N(e, subs={d: 4})", "Lambda": "sp.Lambda(d, e)(4)",
    "Subs_doit": "sp.Subs(e, d, 4).doit()", "limit": "sp.limit(e, d, 4)", "replace": "e.replace(d, 4)",
    "replace_fn": "e.replace(lambda y: y == d, lambda y: 4)", "as_poly_call": "P.as_poly(d)(4)",
    "all_coeffs": "sp.Poly(P, d).all_coeffs()", "as_expr_values": "sp.Poly(P, d).as_expr(4)",
    "collect": "sp.collect(e, d, evaluate=False)", "as_independent": "e.as_independent(d)",
    "series": "e.series(d, 4, 2)", "rename_then_value": "e.subs(d, sp.Dummy())", "lambdify": "sp.lambdify(d, e
  +> )(4)",
    "getattr_subs": "getattr(e, 'subs')(d, 4)", "methodcaller": "operator.methodcaller('subs', d, 4)(e)",
    "unbound_Basic_subs": "sp.Basic.subs(e, d, 4)",
    "Symbol_new_classmethod": "e.subs(sp.Symbol.__new__(type(d), 'd'), 4)  (reviewer p21: Symbol_new_classmeth
  +> od)",
    "Function_call_d": "sp.Function('f')(e).subs(d, 4)  (reviewer p21: Function_call_d)",
}
RUNTIME_GUARD_RESIDUE_P22 = (
    "NOT runtime-interceptable (code-review matters; the static allow-list scan is the secondary barrier): ide
  +> ntity / "
    "name / type tests in hand-written expression-tree rebuilds (`x is d`, x.name == 'd', isinstance), string
  +> "
    "round-trips (str(e).replace('d', '4')), low-level polynomial representations (Poly.rep, DMP, polyutils di
  +> ct "
    "builders), and code that disguises its module name as sympy.  Symbol.__new__(Symbol, 'd') yields a PLAIN
  +> d that "
    "does not match the opaque d (no value is produced); parse_d / assert_opaque_d refuse plain d in loaded da
  +> ta.")
```

159 Source: evanescent.py (p22 EV-M2 and evanescent counterterms)

[listing 126/133] code/p5_calc/evanescent.py

```
#!/usr/bin/env python3
"""XVI-P p22: evanescent components of the one-loop kappa^2 two-point functions (priorities 1 and 2).

  EV-M2  what the 8 evanescent (gt) D11 structures must satisfy.  XVI-O is strictly 4D, so "match XVI-O" has n
  +> o
         content for them.  Tested exactly (opaque d, covariant contraction):
           (i)   locality: every evanescent UV pole is a constant times a polynomial structure in p (local ope
  +> rator);
           (ii)  continuation hypotheses: are the evanescent components the d-dim continuation of the 4D kerne
  +> l?
                 g4 -> ghat (= g4 - gt) continuation and g4-kept continuation, tested on all 2^4 hat/tilde ext
  +> ernal
                 assignments -- both are REFUTED (recorded, not hidden);
           (iii) 4D decoupling: every evanescent structure (pole, finite part, nonlocal L part alike) vanishes
                 identically on hatted (4D) external legs, exactly in d.
         The provable matching statement is therefore EV-M2 = (i) + (iii): after the evanescent MS-bar subtrac
  +> tion the
         evanescent components are a pure local evanescent counterterm plus a finite evanescent remainder that
  +>  is
         identically zero on 4D external configurations; nothing is matched to XVI-O.
  EV-CT  p22 CONTRACT REVISION: evanescent counterterms (h.h gt-operators for DO1/DO3/D11; K'.c gt-operator E2
  +>  for D13;
         and its ghost-equation partner, the gt part SE of the ghost self-energy, c.cb) with coefficient = -(p
  +> ole).
         Checked: (a) sigma_O closure in every sector they can reach (h.c: p.T = 0, E.E2 = 0; c.b: calibrated
  +> on the
         closed 4D pair), (b) they leave the ch-projected identity unchanged (slot values zero, exact in d), (
  +> c) the
         finite evanescent remnants and O(eps) x pole effects at this order.
"""
from __future__ import annotations
import itertools
import json
import sys
import time
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p_ in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical"):
    sys.path.insert(0, str(p_))
import dred  # noqa: E402
import covariant as C  # noqa: E402
import e2_evanescent as E2M  # noqa: E402
import hh_slot as H  # noqa: E402
import batchC as BC  # noqa: E402
import p22_conventions as QC  # noqa: E402

I = sp.I
d = dred.d
PP = C.PP
CERT = ROOT / "certificates"
GRAPHS = ("DO1", "DO3", "D11")


def log(m):
    print(f"[evanescent {time.strftime('%H:%M:%S')}] {m}", flush=True)


def s(x):
    return str(sp.factor(x)) if x != 0 else "0"


def jl(name):
    return json.loads((CERT / name).read_text())


def zero(res):
    return all(sp.cancel(sp.together(v)) == 0 for v in res.values())


def pole(A):
```

```
    return dred.substitute_d(A, 4, "evanescent: UV pole A(4) of an evanescent structure (after exact-d algebra
  +> )")


# ------------------------------------------------------------------ tensor helpers (free indices m n r s)
def full_contract(gterms, ext):
    """T^{mnrs} x_m y_n z_r w_s, scalar, exact in d."""
    x, y, z, w = ext
    res = E2M.contract_all([(c, E2M.to_dred(f) + [("v", x, "m"), ("v", y, "n"), ("v", z, "r"), ("v", w, "s")])
                            for c, f in gterms])
    assert all(k == () for k in res)
    return sp.cancel(sp.together(res.get((), 0)))


EXT4 = [tuple(f"{n}{k}" for n, k in zip("abce", ks)) for ks in itertools.product("^~", repeat=4)]


def swap_g4_to_gh(gterms):
    out = []
    for c, f in gterms:
        out.append((c, [("gh",) + tuple(x[1:]) if x[0] == "g4" else x for x in f]))
    return out


def dhat(struct_terms, ppow):
    """de Donder operator (ghost equation, P15: Sigma = 2 i D[G]) without the i: D^m_v X = p_n X^{(mn)}_v
    - 1/2 p^m g4_{ab} X^{(ab)}_v  (free m, v)."""
    terms = []
    for t in struct_terms:
        ren = [tuple({"m": "a", "n": "b"}.get(i, i) if j else i for j, i in enumerate(f)) for f in t]
        terms.append((PP ** (ppow + 0), E2M.to_dred([tuple({"n": "b"}.get(i, i) if j else i for j, i in enumer
  +> ate(f))
                                                     for f in t]) + [("v", "p", "b")]))
        terms.append((-sp.Rational(1, 2) * PP ** ppow, E2M.to_dred(ren) + [("g", "4", "a", "b"), ("v", "p", "m
  +> ")]))
    return terms


def ytensor(name):
    """ghost self-energy structures (P15_D13 ghost_self_energy_pole), free m (= rho), v."""
    return {"S1": [(PP ** 2, E2M.to_dred([("g4", "m", "v")]))],
            "S2": [(PP, E2M.to_dred([("p", "m"), ("p", "v")]))],
            "SE": [(PP ** 2, E2M.to_dred([("gt", "m", "v")]))]}[name]


def close(terms):
    return E2M.contract_all(terms)


def main():
    import certio
    t0 = time.time()
    log("inputs: D01, D03, D11 exact-d amplitudes (batchC.evaluate_graph), P15_D13")
    A = {g: BC.evaluate_graph(g)["A"] for g in GRAPHS}
    ev = {g: {k: v for k, v in A[g].items() if C.is_evanescent(k[0])} for g in GRAPHS}
    four = {g: {k: v for k, v in A[g].items() if not C.is_evanescent(k[0])} for g in GRAPHS}
    d13 = jl("P15_D13.json")
    P = lambda t: dred.parse_d(str(t), {"I": I})  # noqa: E731
    A13 = {k: P(d13["result"][k]["A_exact_d_times_MB"]) for k in E2M.STRUCT}
    gse = d13["ghost_self_energy_pole"]
    sig_exact = {k: P(v) for k, v in gse["coefficients_exact_d"].items()}
    # ------------------------------------------------------------ coefficients (MS-bar: counterterm = -pole)
    evpole = {g: {C.label(*k): pole(v) for k, v in ev[g].items()} for g in GRAPHS}
    evpole_sum = {}
    for g in GRAPHS:
        for k, v in evpole[g].items():
            evpole_sum[k] = sp.nsimplify(evpole_sum.get(k, 0) + v)
    e2_pole, se_pole = pole(A13["E2"]), pole(sig_exact["SE"])
    stored_ok = (all(sp.simplify(pole(A13[k]) - P(d13["result"][k]["UV_pole"])) == 0 for k in E2M.STRUCT)
                 and all(sp.simplify(pole(sig_exact[k]) - P(gse["pole"][k])) == 0 for k in sig_exact))
    d11c = jl("P18_D11.json")["result"]
    d11_stored_ok = all(sp.simplify(evpole["D11"][k] - P(d11c[k]["UV_pole"])) == 0 for k in evpole["D11"])
    hhc = jl("P18_hh_slot.json")["evanescent_poles_D01_D03_D11_in_hh_green_function"]
    hh_stored_ok = set(hhc) == set(evpole_sum) and all(sp.simplify(evpole_sum[k] - P(hhc[k])) == 0 for k in hh
  +> c)
    pole_terms = {g: H.g_terms({k: pole(v) for k, v in ev[g].items()}) for g in GRAPHS}
    pole_sum_terms = sum((pole_terms[g] for g in GRAPHS), [])
    # ------------------------------------------------------------ EV-M2
    log("EV-M2: locality, continuation hypotheses, 4D decoupling")
    locality = all(not v.has(d) for g in GRAPHS for v in evpole[g].values())
    hat = tuple(f"{n}^" for n in "abce")
```

```
  dec4 = {}
  for g in GRAPHS:
      for k, v in ev[g].items():
          dec4[f"{g}: {C.label(*k)}"] = s(full_contract(H.g_terms({k: sp.Integer(1)}), hat))
  four_d_decoupled = all(v == "0" for v in dec4.values())
  cont = {}
  for g in ("D11",):
      full = H.g_terms(A[g])
      c_hat = swap_g4_to_gh(H.g_terms(four[g]))
      c_keep = H.g_terms(four[g])
      mis_hat = {"".join(e): s(full_contract(full, e) - full_contract(c_hat, e)) for e in EXT4}
      mis_keep = {"".join(e): s(full_contract(full, e) - full_contract(c_keep, e)) for e in EXT4}
      cont[g] = {"ghat_continuation_mismatches": {k: v for k, v in mis_hat.items() if v != "0"},
                 "g4_kept_continuation_mismatches": {k: v for k, v in mis_keep.items() if v != "0"},
                 "ghat_continuation_holds": all(v == "0" for v in mis_hat.values()),
                 "g4_kept_continuation_holds": all(v == "0" for v in mis_keep.values()),
                 "hatted_externals_agree_both": mis_hat["a^b^c^e^"] == "0" and mis_keep["a^b^c^e^"] == "0"}
  # ------------------------------------------------------------ EV-CT (a) closure
  log("EV-CT (a): sigma_0 closure, h.c and c.b")
  pdg = {g: {str(k): s(v) for k, v in H.p_dot_G(pole_terms[g]).items()} for g in GRAPHS}
  pdg_sum = {str(k): s(v) for k, v in H.p_dot_G(pole_sum_terms).items()}
  pdg_dd = {str(k): s(v) for k, v in H.p_dot_G(pole_terms["DO1"] + pole_terms["DO3"]).items()}
  E, fitE = E2M.operator_from_polys(E2M.SO_hh_operator())
  # E^{(mn)(rs)} E2_{(mn)v}: free r, s, v
  ee2 = close([(c * PP, E2M.to_dred(list(t)) + E2M.to_dred(f)) for t in E2M.STRUCT["E2"] for c, f in E])
  hc_closed = (not pdg_sum) and not ee2
  # c.b: image = alpha Dhat[X] + beta Y; calibrate beta/alpha on the closed 4D pair, then test the evanescen
+> t pair.
  X4 = {k: pole(A13[k]) for k in ("T1", "T2", "T3")}
  Y4 = {k: pole(sig_exact[k]) for k in ("S1", "S2")}
  rho = sp.Symbol("rho")
  dx4 = sum((dhat([tuple(t) for t in E2M.STRUCT[k]], E2M.PPOW[k]) for k in X4 if X4[k] != 0), [])
  img4 = close([(c * X4[k], f) for k in X4 if X4[k] != 0
                for c, f in dhat([tuple(t) for t in E2M.STRUCT[k]], E2M.PPOW[k])]
               + [(rho * c * Y4[k], f) for k in Y4 for c, f in ytensor(k)])
  sol = sp.solve([sp.expand(v) for v in img4.values()], rho, dict=True)
  rho_val = sol[0][rho] if len(sol) == 1 else None
  del dx4
  if rho_val is not None:
      img_ev = close([(c * e2_pole, f) for c, f in dhat([tuple(t) for t in E2M.STRUCT["E2"]], 1)]
                     + [(rho_val * c * se_pole, f) for c, f in ytensor("SE")])
      img_ev_noSE = close([(c * e2_pole, f) for c, f in dhat([tuple(t) for t in E2M.STRUCT["E2"]], 1)])
      img_ev_flip = close([(-c * e2_pole, f) for c, f in dhat([tuple(t) for t in E2M.STRUCT["E2"]], 1)]
                          + [(rho_val * c * se_pole, f) for c, f in ytensor("SE")])
      exact_ghost_eq = close([(c * A13["E2"], f) for c, f in dhat([tuple(t) for t in E2M.STRUCT["E2"]], 1)]
                             + [(rho_val * c * sig_exact["SE"], f) for c, f in ytensor("SE")])
  else:
      img_ev = img_ev_noSE = img_ev_flip = exact_ghost_eq = {"calibration": "failed"}
  cb_closed = rho_val is not None and not img_ev
  # ------------------------------------------------------------ EV-CT (b) decoupling from the projected ident
+> ity
  log("EV-CT (b): ch-slot values of the counterterms")
  op, opfit, _ = H.kc_operator()
  slot_hh = {n: s(H.slot_value(op, pole_sum_terms, e)) for n, e in H.EXT.items()}
  slot_hh_g = {g: {n: s(H.slot_value(op, pole_terms[g], e)) for n, e in H.EXT.items()} for g in GRAPHS}
  slot_kc = {n: s(e2_pole * E2M.slot(E2M.STRUCT["E2"], E, e, 1)) for n, e in E2M.EXT.items()}
  identity_unchanged = all(v == "0" for v in slot_hh.values()) and all(v == "0" for v in slot_kc.values())
  # ------------------------------------------------------------ EV-CT (c) remnants
  log("EV-CT (c): finite remnants and O(eps) x pole traces")
  traces = {}
  for g in GRAPHS:
      # g4^{mn} T_{mnrs} (a trace over one index pair; (4 - d) factors appear only through tr gt)
      r = close([(c, E2M.to_dred(f) + [("g", "4", "m", "n"), ("v", "a^", "r"), ("v", "b^", "s")])
                 for c, f in pole_terms[g]])
      traces[g] = s(sp.cancel(sp.together(r.get((), 0))))
  rsum = close([(c, E2M.to_dred(f) + [("g", "4", "m", "n"), ("v", "a^", "r"), ("v", "b^", "s")])
                for c, f in pole_sum_terms])
  trace_sum = sp.cancel(sp.together(rsum.get((), 0)))
  remnants = {
      "finite_evanescent_remainder": "MS-bar finite parts (local + L) x gt-structures: recorded per structur
+> e in "
                                     "P18_D11 / P16_batchC / P15_D13; NONZERO; identically zero on 4D (hatte
+> d) "
                                     "external legs, exactly in d (four_d_decoupling below)",
      "O_eps_times_pole_at_this_order": ("none with QdS (hatted) external legs: all four indices are externa
+> l, so "
                                         "tr gt = 4 - d never arises and the evanescent operators are ZERO o
+> n hatted "
                                         "legs at every d; see the Q4S trace entry for the one place it can
+> arise"),
      "trace_of_pole_tensor_g4^{mn}T_mn,rs (hatted r,s)": {"per_graph": traces, "sum": s(trace_sum)},
```

```
      "O_eps_times_pole_Q4S_trace_external": {
          "finite_term_if_external_trace_mode_uses_g4": s(sp.cancel(trace_sum / (d - 4)) * (-2)),
          "rule": "(d - 4) x (1/eps) -> -2 (d = 4 - 2 eps); times the residue above",
          "status": ("NONZERO: a finite local transverse term arises iff the external trace mode is defined
+> with the "
                     "quasi-4D g4 (it then carries eps-scalar components); with QdS (hatted) external legs i
+> t is "
                     "zero.  This is a scheme / external-state convention (DRED eps-scalar sector), disclose
+> d: the "
                     "p22 renormalised result is stated for QdS external legs")},
      "where_O_eps_x_pole_would_enter": ("insertions of the evanescent counterterms into loops (two loops) o
+> r into "
                                         "composite operators with traces: (4 - d) x (1/eps) = finite local
+> terms "
                                         "-- beyond this order; not evaluated, not claimed"),
  }
  # ------------------------------------------------------------ mutations
  muts = {
      "drop_D03_evanescent_ct (h.c)": bool(H.p_dot_G(pole_terms["D01"] + pole_terms["D11"])),
      "drop_D01_evanescent_ct (h.c)": bool(H.p_dot_G(pole_terms["D03"] + pole_terms["D11"])),
      "D03_evanescent_ct_x2 (h.c)": bool(H.p_dot_G(pole_terms["D01"] + pole_terms["D03"] + pole_terms["D03"]
                                                   + pole_terms["D11"])),
      "drop_SE_partner (c.b)": bool(img_ev_noSE),
      "flip_sign_E2_ct (c.b)": bool(img_ev_flip),
      "trace_operator_in_place_of_s0h (slot)": any(
          H.slot_value(H.trace_operator(), pole_sum_terms, e) != 0 for e in H.EXT.values()),
  }
  insensitive = {"D11_evanescent_ct_x2 (h.c)": ("p_m T_D11,ev = 0 by itself: the D11 evanescent counterterm
+> is "
                                                "closed alone; its normalisation is pinned by the D11 pins,
+> not "
                                                "by closure"),
                 "detected": bool(H.p_dot_G(pole_terms["D01"] + pole_terms["D03"] + pole_terms["D11"]
                                            + pole_terms["D11"]))}
  controls = {
      "poles_equal_stored_D13_and_Sigma": stored_ok, "D11_evanescent_poles_equal_stored": d11_stored_ok,
      "hh_evanescent_pole_sum_equals_P18_hh_slot": hh_stored_ok,
      "ev_poles_local": locality, "ev_four_D_decoupling_exact": four_d_decoupled,
      "ev_D11_hatted_externals_agree": cont["D11"]["hatted_externals_agree_both"],
      "hc_p_dot_T_ev_sum_zero": not pdg_sum, "hc_E_dot_E2_zero": not ee2,
      "hc_D01_D03_ev_sum_transverse": not pdg_dd, "hc_D01_alone_not_transverse": bool(pdg["D01"]),
      "cb_calibration_unique_on_4D_pair": rho_val is not None,
      "cb_ev_pair_closed": cb_closed, "ghost_equation_ev_exact_in_d": rho_val is not None and not exact_ghos
+> t_eq,
      "identity_unchanged_hh_slot_zero_all_ext": all(v == "0" for v in slot_hh.values()),
      "identity_unchanged_kc_slot_zero_all_ext": all(v == "0" for v in slot_kc.values()),
      "D01_ev_ct_alone_reaches_slot": any(v != "0" for v in slot_hh_g["D01"].values()),
      "mutations_all_detected": all(muts.values()),
  }
  ok = all(controls.values())
  ev_m2_proven = locality and four_d_decoupled
  rec = {
      "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION,
      "deliverable": "p22 priorities 1-2: evanescent matching statement (EV-M2) and evanescent counterterms
+> (EV-CT)",
      "EV_M2": {
          "question": "what must the 8 evanescent D11 structures satisfy, given XVI-O has no evanescent coun
+> terpart?",
          "candidates": {
              "match a d-dim continuation of the XVI-O kernel": (
                  "REFUTED for both natural continuations (g4 -> ghat; g4 kept): the exact D11 tensor differ
+> s from "
                  "either continuation of its own 4D part on tilde external legs; XVI-O defines no other "
                  "continuation -- not a usable statement"),
              "vanish as eps -> 0 in the renormalised result": (
                  "TRUE in the stronger form: after subtraction, every evanescent component is identically Z
+> ERO "
                  "on 4D external legs at every d (not merely O(eps))" if four_d_decoupled else "NOT ESTABLI
+> SHED"),
              "be pure local evanescent operators": (
                  "TRUE for the UV poles (constants x polynomial gt-structures); FALSE for the finite parts
+> "
                  "(they carry L = log(-p^2/mu^2)): the finite remainder is nonlocal but evanescent"
                  if locality else "NOT ESTABLISHED")},
          "statement": ("EV-M2: the evanescent D11 poles are local evanescent operators (removed by the p22
+> "
                        "evanescent counterterm), and the renormalised evanescent remainder vanishes identic
+> ally "
                        "on 4D external configurations, exactly in d.  No matching to XVI-O is asserted."),
          "verdict": "PROVEN (exact, opaque d)" if ev_m2_proven else "NOT ESTABLISHED",
          "locality": locality, "four_D_decoupling": dec4, "continuation_tests": cont},
```

```
        "D11_matching_verified_scope": (
            "4D components: M2 discharged (P18_D11_M2, unchanged); evanescent components: EV-M2 proven (local
  +> poles + "
            "exact 4D decoupling of the renormalised remainder); NOT matched to XVI-O (no counterpart; both na
  +> tural "
            "d-dim continuations refuted)" if ev_m2_proven else "4D components only (P18); evanescent: not ver
  +> ified"),
        "EV_CT": {
            "contract_revision": ("p22: evanescent counterterms admitted outside the p14r 4D domain -- h.h ope
  +> rators "
                                  "O_k = 1/2 h_mn S_k^{(mn)(rs)}(gt, g4, p) h_rs for the 8 gt-structures of th
  +> e hh "
                                  "two-point function (D01 + D03 + D11), the K'.c operator K'^{mn} E2_{(mn)v}
  +> c^v "
                                  "(D13) and its ghost-equation partner cb_m SE^m_v c^v (Sigma, gt p^4); "
                                  "coefficient = -(UV pole) (MS-bar, N_eps kappa^2 / eps normalisation of P14)
  +> "),
            "coefficients_hh_counterterm_per_graph": {g: {k: s(-v) for k, v in evpole[g].items()} for g in GRA
  +> PHS},
            "coefficients_hh_counterterm_sum": {k: s(-v) for k, v in sorted(evpole_sum.items())},
            "coefficient_Kc_E2": s(-e2_pole), "coefficient_ccb_SE": s(-se_pole),
            "a_closure": {"h.c": {"p_dot_T_ev_per_graph": pdg, "p_dot_T_ev_sum": pdg_sum, "E_dot_E2": len(ee2)
  +> ,
                                  "closed": hc_closed},
                          "c.b": {"calibration_rho_from_4D_pair": s(rho_val) if rho_val is not None else None,
                                  "image_ev_pair": {str(k): s(v) for k, v in img_ev.items()} if rho_val is not
  +>  None
                                  else img_ev, "closed": cb_closed,
                                  "ghost_equation_exact_in_d_gt_sector": rho_val is not None and not exact_gho
  +> st_eq},
                          "c.cbs": ("unreachable: S0's quadratic part is h.h, h.b, b.b, b.cbs (+ K.c, cb.c); s
  +> igma_0 "
                                    "of h.h, K'.c, cb.c bilinears lands in h.c and c.b only"),
                          "full_closure_with_4D_counterterm": ("closed: the 4D MS-bar counterterm is closed (P
  +> 20_defect, "
                                                               "unchanged) and sigma_0 is linear"
                                                               if hc_closed and cb_closed else "NOT closed")},
            "b_projected_identity": {"hh_slot_values_sum": slot_hh, "hh_slot_values_per_graph": slot_hh_g,
                                     "Kc_slot_values": slot_kc, "unchanged": identity_unchanged},
            "c_remnants": remnants},
        "mutations": muts, "insensitive_variations": insensitive, "controls": controls,
        "evanescent_counterterm_earned": ok,
        "EV_M2_status": "proven" if ev_m2_proven else "not proven",
        "EV_CT_status": "earned" if ok else "not earned",
        "P5_calculation_status": ("renormalized (one loop, kappa^2, two-point functions hh / K'.c / cb.c; MS-b
  +> ar in "
                                  "DRED with the p22 evanescent counterterms; external legs in QdS)" if ok els
  +> e
                                  "renormalized (4D) / regulated (evanescent poles nonzero, retained R6)"),
        "scope": ("one loop, kappa^2, the two-point functions entering the ch rows; NOT the one-point row beyo
  +> nd "
                  "kappa^1, not higher points, not two loops; not bulk symmetry, not O.1"),
        "authorised_d_substitutions": dict(dred.D_PROVENANCE),
        "seconds": round(time.time() - t0, 1),
    }
    certio.write(CERT / f"{QC.CERT_PREFIX}_evanescent.json", rec)
    log(f"EV-M2 {rec['EV_M2']['verdict']}; EV-CT earned={ok}; controls failing: "
        f"{[k for k, v in controls.items() if not v]}")
    return 0 if ok else 1


if __name__ == "__main__":
    sys.exit(main())
```

160 Source: contract_registry.py (p22 registry of the post-p18 contract revision)

[listing 127/133] code/p5_calc/contract_registry.py

```
#!/usr/bin/env python3
"""XVI-P p22: registry of DISCLOSED contract revisions after p18 (consulted by the p19, p20, p21 layer gates a
  +> nd the p22
gate).  The only post-p18 contract revision is p22's runtime d guard: code/p5_entry/dred.py = the p18 dred.py
  +> with ONE
delimited block inserted (markers below).  dred_revision() strips the block and requires the remainder to be b
  +> yte-identical
to the p18 release (P18_DRED_SHA256): the p18 contract text is untouched, the revision is purely additive.  Th
```

```
  +> e p22 gate
additionally pins the exact sha of the revised file (p22_conventions.P22_DRED_SHA256)."""
from __future__ import annotations
import hashlib
from pathlib import Path

ROOT = Path(__file__).resolve().parents[2]
DRED = "code/p5_entry/dred.py"
P18_DRED_SHA256 = "14c11a512c1609c9f9a7829dcb68e71a94c8ac3d38497e2dcac110a1b8d5e81c"
BEGIN = ("\n\n# ============================================================================================ "
         "p22 CONTRACT REVISION: runtime d guard\n")
END = "# ============================================================================================ end p22 runti
  +> me d guard\n"
BEGIN23 = ("\n\n# ============================================================================================ "
           "p23 CONTRACT REVISION: polynomial-domain refusal\n")
END23 = "# ============================================================================================ end p23 pol
  +> ynomial-domain refusal\n"
P22_BLOCK_SHA256 = "9d2bbe3e77030115d0c302da3895e1b4615fe8c0aa8ba9a5b5dbf02b640482ad"   # the p22 block as rel
  +> eased


def p22_block_unchanged(root=ROOT):
    t = (Path(root) / DRED).read_text()
    if t.count(BEGIN) != 1 or t.count(END) != 1:
        return False
    return hashlib.sha256(t[t.index(BEGIN):t.index(END) + len(END)].encode()).hexdigest() == P22_BLOCK_SHA256


def p23_registered(root=ROOT):
    """the p23 contract state: p18 core byte-identical, p22 block byte-identical to its release, p23 block pre
  +> sent."""
    r = dred_revision(root)
    return bool(r["ok"] and "p23 polynomial-domain refusal" in r.get("registered_blocks", []) and p22_block_un
  +> changed(root))


REVISED_CONTRACT_FILES = {DRED: "p22 contract revision: runtime d guard (additive block)"}


def dred_revision(root=ROOT):
    raw = (Path(root) / DRED).read_bytes()
    cur = hashlib.sha256(raw).hexdigest()
    if cur == P18_DRED_SHA256:
        return {"state": "p18", "ok": True, "sha256": cur}
    t = raw.decode()
    blocks = ["p22 runtime d guard"]
    if t.count(BEGIN23) == 1 and t.count(END23) == 1 and t.index(BEGIN23) < t.index(END23) and t.endswith(END2
  +> 3):
        t = t[:t.index(BEGIN23)]                # p23 block appended at the end of the file (after the p22 EN
  +> D)
        blocks.append("p23 polynomial-domain refusal")
    elif t.count(BEGIN23) or t.count(END23):
        return {"state": "unregistered", "ok": False, "sha256": cur}
    if t.count(BEGIN) != 1 or t.count(END) != 1 or t.index(BEGIN) > t.index(END):
        return {"state": "unregistered", "ok": False, "sha256": cur}
    core = t[:t.index(BEGIN)] + t[t.index(END) + len(END):]
    ok = hashlib.sha256(core.encode()).hexdigest() == P18_DRED_SHA256
    # compatibility key: "p18 + p22 guard block" = p18 core byte-identical + the p22 block + any LATER registe
  +> red
    # block listed in registered_blocks (p23: polynomial-domain refusal)
    return {"state": "p18 + p22 guard block" if ok else "unregistered", "ok": ok, "sha256": cur,
            "p18_core_byte_identical": ok, "registered_blocks": blocks if ok else []}


def scan_view(path, root=ROOT):
    """Older-layer static scans (p17..p20 tests) review the p18 CONTRACT text: for the registered revised dred
  +> .py they
    are handed the p18 core (block stripped, byte-identical to the p18 release) in a temporary mirror root.  T
  +> he p22
    guard block itself is covered by the p22 runtime probe and the p21 allow-list scan (scan4, p22 allow-lists
  +> )."""
    import tempfile
    path, root = Path(path), Path(root)
    try:
        rel = path.relative_to(root).as_posix()
    except ValueError:
        return path, root
    if rel != DRED or dred_revision(root)["state"] != "p18 + p22 guard block":
        return path, root
    t = path.read_text()
    if BEGIN23 in t:
```

```
        t = t[:t.index(BEGIN23)]
    core = t[:t.index(BEGIN)] + t[t.index(END) + len(END):]
    tmp = Path(tempfile.mkdtemp(prefix="p22scanview_"))
    (tmp / DRED).parent.mkdir(parents=True)
    (tmp / DRED).write_text(core)
    return tmp / DRED, tmp


def registered_contract_change(rel, root=ROOT):
    """True iff `rel` is a contract file whose change is a registered (disclosed) revision in its current stat
  +> e."""
    return rel == DRED and dred_revision(root)["state"] == "p18 + p22 guard block"


# p18-layer gate revised in p22 (pinned byte-identically by release_p19.p18_layer): exact sha of the revised f
  +> ile
LAYER_FILE_REVISIONS_P22 = {"code/p5_calc/release_p18.py": "f2e8743b4902b4304679c68b994dfc9c6df4a0b769827eb269
  +> 3a72495a4e9c5b"}


def layer_file_ok(rel, cur_sha, pinned_sha):
    return cur_sha == pinned_sha or LAYER_FILE_REVISIONS_P22.get(rel) == cur_sha


# files revised in p22 (disclosed; listed in p22_conventions.DISCLOSED_REVISIONS_P22 with reasons and pinned t
  +> here by sha);
# the earlier-layer change logs (release_p21 vs the p20 zip) treat exactly these as disclosed
P22_REVISED_FILES = (
    "code/p5_entry/dred.py", "code/p5_calc/release_p18.py", "code/p5_calc/release_p19.py",
    "code/p5_calc/release_p20.py", "code/p5_calc/release_p21.py",
    "tests/test_p14.py", "tests/test_p16.py", "tests/test_p17.py", "tests/test_p18.py", "tests/test_p19.py",
    "tests/test_p20.py", "tests/test_p21.py",
)


# p23 (disclosed in p23_conventions.DISCLOSED_REVISIONS_P23, exact shas pinned by the p23 gate): files changed
  +>  or added
# in p23 -- earlier layer gates/tests accept them as later registered revisions
P23_REVISED_FILES = (
    "code/p5_entry/dred.py", "code/p5_calc/contract_registry.py", "code/p5_calc/release_p21.py",
    "code/p5_calc/release_p22.py",
    "tests/test_p14.py", "tests/test_p21.py", "tests/test_p22.py",
    "tools/build.py", "tools/source_manifest.py", "tools/rebuild_from_source.py",
    "README.md", "REBUILD_INSTRUCTIONS.md", "manuscript/body/XVI-P-body.tex",
)
P23_ADDED_FILES = ("code/p5_calc/power_counting.py", "code/p5_calc/p23_conventions.py", "code/p5_calc/release_
  +> p23.py",
                   "tests/test_p23.py", "manuscript/notes/P23_01_scope.tex")
P23_ADDED_PREFIXES = ("archival/p22/", "provenance/XVI-0/")


P23_GENERATED = ("reproducibility/reference_values_p23.json",)   # generated reference (exempt in the p23 gate
  +> )


def p23_disclosed(rel):
    return (rel in P23_REVISED_FILES or rel in P23_ADDED_FILES or rel in P23_GENERATED
            or rel.startswith(P23_ADDED_PREFIXES))


def p22_revised(rel):
    """revised by a LATER registered release (p22, or p23)."""
    return rel in P22_REVISED_FILES or p23_disclosed(rel)
```

161 Source: release_p22.py (p22 layer gate, disclosed p23 revision)

[listing 128/133] code/p5_calc/release_p22.py

```
#!/usr/bin/env python3
"""XVI-P p22 release gate (bigger fronts after p21; WITH the disclosed p22 contract revision).

Runs AFTER the p14r .. p21 gates (release_p21.py -> certificates/P21_release.json, the p21 LAYER record) and r
  +> equires:
  contract          : the ONLY contract change is the registered p22 revision of dred.py (contract_registry: p
  +> 18 text
                      byte-identical outside one delimited guard block; exact sha pinned in P22_PINNED_SHA256)
```

```
  +> ;
  lower layers      : re-checked live (release_p21.lower_layers: contract identity under the registry, p18 lay
  +> er with q
                      earned, inventory, p19 layer, K1, retraction) + p20 layer + absolute D11 pin (live);
  p21 layer         : P21_release accepted with its earnables; p21_conventions.py byte-identical to the p21 zi
  +> p;
  EV-M2 / EV-CT     : certificates/P22_evanescent.json -- EV-M2 PROVEN, evanescent counterterms closed (h.c, c
  +> .b),
                      projected identity unchanged, all controls true, mutations detected;
  runtime d guard   : LIVE probe in a fresh interpreter -- every plant of QC.GUARD_PLANTS_P22 raises
                      PrematureD4Substitution on four expressions; ordinary algebra and substitute_d still wor
  +> k;
  disclosure (K)    : EVERY file added / modified / removed vs the pinned p21 zip list is in DISCLOSED_REVISIO
  +> NS_P22 (no
                      tools/ or manuscript/ exemption; added files included), except the p21 archive (pinned r
  +> ecords)
                      and the generated reference file; disclosed code revisions match their pinned sha;
  p21 independent   : archived report pinned (8a582b33...), PASS, bound to the p21 tree/zip (same-box caveat);
  hard stops        : QC.HARD_STOPS_P22 (bulk_symmetry_restored false; 0.1 NOT_CLOSED; insertion frozen; ...).
Publication fails closed.  On reject: no P22_release.json and NO RELEASE_STATUS.json is left behind.
"""
from __future__ import annotations
import hashlib, json, subprocess, sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p22_conventions as QC  # noqa: E402
import p21_conventions as Q21  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
import contract_registry as CR  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)
import release_p20 as R20  # noqa: E402   (helpers only)
import release_p21 as R21  # noqa: E402   (helpers only)

ARCH_P21 = "archival/p21"
ARCH_P21_INDEP = f"{ARCH_P21}/reproducibility/independent_rebuild_report_p21.json"
STATUS_FILES = ("RELEASE_STATUS.json", "certificates/RELEASE_STATUS.json")

GUARD_PROBE = r'''
import sys, json, operator
sys.path[:0] = [sys.argv[1] + "/code/p5_calc", sys.argv[1] + "/code/p5_entry", sys.argv[1] + "/code/classical"
  +> ]
import sympy as sp, dred
d = dred.d; x = sp.Symbol("x")
exprs = {"linear_and_pow": d * x + 1 / (d - 1), "power_base_only": 1 / (d ** 2 - 1), "d_alone": d, "mixed_pow"
  +> : x * d ** 2 + d}
D, = [d]
P = lambda e: sp.expand(e * (d ** 2 - 1)) if not e.has(1 / (d - 1)) else sp.expand(x * d ** 3 + d ** 2)
runs = {
 "subs_d": lambda e: e.subs(d, 4), "xreplace_d": lambda e: e.xreplace({d: 4}),
 "list_unpack_alias_then_subs": lambda e: e.subs(D, 4),
 "free_symbols_pop_subs": lambda e: e.subs([y for y in e.free_symbols if y.name == "d"][0], 4),
 "lambda_wrapper_Poly": lambda e: (lambda *a: sp.Poly(*a))(P(e), d).eval(4),
 "coeff_extract": lambda e: sum(sp.expand(e).coeff(d, k) * 4 ** k for k in range(5)),
 "dunder_getattribute_eq": lambda e: sp.Add(*[4 if y == d else y for y in (e.__getattribute__("args") + (d,))]
  +> ),
 "subs_power": lambda e: e.subs(d ** 2, 16) if e.has(d ** 2) else e.subs(d, 4),
 "subs_dict": lambda e: e.subs({d: 4}), "subs_list": lambda e: e.subs([(d, 4)]),
 "evalf_subs": lambda e: e.evalf(subs={d: 4}), "N_subs": lambda e: sp.N(e, subs={d: 4}),
 "Lambda": lambda e: sp.Lambda(d, e)(4), "Subs_doit": lambda e: sp.Subs(e, d, 4).doit(),
 "limit": lambda e: sp.limit(e, d, 4), "replace": lambda e: e.replace(d, 4),
 "replace_fn": lambda e: e.replace(lambda y: y == d, lambda y: 4),
 "as_poly_call": lambda e: P(e).as_poly(d)(4), "all_coeffs": lambda e: sp.Poly(P(e), d).all_coeffs(),
 "as_expr_values": lambda e: sp.Poly(P(e), d).as_expr(4),
 "collect": lambda e: sp.collect(sp.expand(e), d, evaluate=False), "as_independent": lambda e: sp.expand(e).as
  +> _independent(d),
 "series": lambda e: e.series(d, 4, 2), "rename_then_value": lambda e: e.subs(d, sp.Dummy()),
 "lambdify": lambda e: sp.lambdify(d, e)(4), "getattr_subs": lambda e: getattr(e, "subs")(d, 4),
 "methodcaller": lambda e: operator.methodcaller("subs", d, 4)(e), "unbound_Basic_subs": lambda e: sp.Basic.su
  +> bs(e, d, 4),
 "Symbol_new_classmethod": lambda e: e.subs(sp.Symbol.__new__(type(d), "d"), 4),
 "Function_call_d": lambda e: sp.Function("f")(e).subs(d, 4).args[0],
}
out = {}
for rn, f in runs.items():
    out[rn] = {}
    for en, e in exprs.items():
```

```
        try:
            v = f(e); out[rn][en] = "NOT_RAISED: " + str(v)[:60]
        except dred.PrematureD4Substitution:
            out[rn][en] = "RAISED"
        except Exception as ex:
            out[rn][en] = "OTHER " + type(ex).__name__
e = (d ** 2 - 2 * d - 2) / (32 * (d ** 2 - 1))
alg = {"cancel_together": str(sp.factor(sp.cancel(sp.together(e + 1 / (d + 1))))), "apart": str(sp.apart(e, d)
  +> ),
       "solve": str(sp.solve(sp.Symbol("r") * d - 1, sp.Symbol("r"))), "simplify": str(sp.simplify(e * (d - 1)
  +> )),
       "substitute_d": str(dred.substitute_d(e, 4, "release_p22 guard probe")),
       "authorised_d": None}
with dred.authorised_d("release_p22 guard probe: authorised coefficient read"):
    alg["authorised_d"] = str(sp.Poly(sp.cancel(e * 32 * (d ** 2 - 1)), d).all_coeffs())
print(json.dumps({"plants": out, "algebra": alg, "entry_points": dred.GUARDED_ENTRY_POINTS,
                  "revision": dred.GUARD_REVISION}))
'''


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def guard_probe():
    r = subprocess.run([sys.executable, "-c", GUARD_PROBE, str(ROOT)], capture_output=True, text=True, timeout
  +> =600)
    if r.returncode != 0:
        return False, {"error": r.stderr[-800:]}
    rec = json.loads(r.stdout.strip().splitlines()[-1])
    # an "OTHER" outcome is accepted only where sympy itself refuses the construction (no value produced)
    not_raised = {f"{k}/{e}": v for k, per in rec["plants"].items() for e, v in per.items() if v.startswith("N
  +> OT_RAISED")}
    other = {f"{k}/{e}": v for k, per in rec["plants"].items() for e, v in per.items() if v.startswith("OTHER"
  +> )}
    alg = rec["algebra"]
    alg_ok = (alg["substitute_d"] == "1/80" and alg["solve"] == "[1/d]" and alg["authorised_d"] == "[1, -2, -2
  +> ]"
              and alg["apart"] and alg["cancel_together"])
    rec["not_raised"], rec["other"], rec["algebra_ok"] = not_raised, other, alg_ok
    rec["n_plants"] = len(rec["plants"])
    rec["all_plants_covered"] = set(rec["plants"]) == set(QC.GUARD_PLANTS_P22)
    ok = bool(not not_raised and alg_ok and rec["all_plants_covered"] and rec["revision"] == "p22")
    return ok, rec


def contract_check():
    dr = CR.dred_revision(ROOT)
    cur = {p.relative_to(ROOT).as_posix(): sha(p) for p in sorted((ROOT / "code" / "p5_entry").glob("*.py"))}
    pinned = QC.P22_PINNED_SHA256.get(CR.DRED)
    ok = bool(dr["state"] == "p18 + p22 guard block" and (pinned is None or cur.get(CR.DRED) == pinned
                                                          or CR.p23_registered(ROOT)))   # p23: registered la
  +> ter block
    return ok, {"dred_registry": dr, "dred_sha256": cur.get(CR.DRED), "dred_pinned": pinned,
                "dred_pin_ok": pinned is not None and cur.get(CR.DRED) == pinned,
                "contract_revision_p22": QC.CONTRACT_REVISION}


def p21_layer():
    r = jl("certificates/P21_release.json")
    base = json.loads((ROOT / QC.P21_SOURCE_LIST).read_text())["files"]
    conv_ok = sha(ROOT / "code/p5_calc/p21_conventions.py") == base.get("code/p5_calc/p21_conventions.py")
    ok = bool(conv_ok and r.get("revision") == Q21.REVISION and r.get("publication") == "accepted"
              and r.get("release_claim_accepted") is True and r.get("hard_stops_hold") is True
              and r.get("bulk_symmetry_restored") is False
              and all(r.get(k) == "earned" for k in Q21.EARNABLE_P21))
    return ok, {"p21_conventions_byte_identical": conv_ok, "P21_release_accepted": r.get("publication") == "ac
  +> cepted",
                "P21_earned": {k: r.get(k) for k in Q21.EARNABLE_P21}}, r


def p21_independent():
    f = ROOT / ARCH_P21_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P21_INDEP, "present": False, "pins_ok": False}
    rep = json.loads(f.read_text())
```

```
    live = ROOT / QC.P21_INDEPENDENT_REPORT
    rec = {"report": ARCH_P21_INDEP, "present": True, "report_sha256": sha(f),
           "report_sha256_pinned": sha(f) == QC.P21_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "bound_to_source_tree_sha256": rep.get("source_tree_sha256"), "bound_to_source_zip_sha256": rep.get
  +> ("zip_sha256"),
           "tree_matches_p21": rep.get("source_tree_sha256") == QC.P21_SOURCE_TREE_SHA256,
           "zip_matches_p21": rep.get("zip_sha256") == QC.P21_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])),
           "findings": [x.get("id") for x in rep.get("findings", []) if isinstance(x, dict)],
           "caveat": "same box (disclosed)",
           "live_report_identical_to_archived": (live.read_bytes() == f.read_bytes()) if live.is_file()
           else "live report not in this tree",
           "NOT_inherited_by_p22": "bound to the p21 tree/zip; the p22 tree has its own (pending) independent
  +> flag"}
    rec["archived_p21_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P21_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p21_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p21"] and rec["zip_matches_p21"] and rec["mismatches"] == 0
              and rec["live_report_identical_to_archived"] in (True, "live report not in this tree"))
    return ok, rec


def change_log():
    f = ROOT / QC.P21_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P21_ARCHIVED_RECORDS[QC.P21_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT)}
    added = sorted(set(new) - set(old))
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])

    def disclosed(x):
        return (x in QC.DISCLOSED_REVISIONS_P22 or x in QC.GENERATED_EXEMPT_P22
                or (x in added and x.startswith(QC.DISCLOSED_PREFIXES_ADDED_P22))
                or (CR.p23_registered(ROOT) and CR.p23_disclosed(x)))          # p23: registered later revisio
  +> n
    undisclosed = [x for x in added + removed + modified if not disclosed(x)]
    pin_bad = {k: {"pinned": h, "current": new.get(k)} for k, h in QC.P22_PINNED_SHA256.items() if new.get(k)
  +> != h
               and not (CR.p23_registered(ROOT) and CR.p23_disclosed(k))}     # p23 revisions pinned by the p2
  +> 3 gate
    listed_unchanged = sorted(k for k in QC.DISCLOSED_REVISIONS_P22 if k not in added + modified + removed)
    contract = [x for x in added + removed + modified if x.startswith("code/p5_entry/")]
    return {"base": f"{QC.P21_SOURCE_LIST} (pinned; files of the p21 zip sha256 {QC.P21_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok, "added": added, "removed": removed, "modified": modified,
            "contract_files_changed": contract, "contract_changes_registered": all(x == CR.DRED for x in contr
  +> act),
            "undisclosed_changes": undisclosed, "pinned_sha_mismatches": pin_bad,
            "disclosed_but_unchanged": listed_unchanged,
            "exemptions": {"generated": list(QC.GENERATED_EXEMPT_P22),
                           "added_archive_prefixes": list(QC.DISCLOSED_PREFIXES_ADDED_P22)},
            "counts": {"added": len(added), "removed": len(removed), "modified": len(modified)}}


def manuscript_disclosure():
    rec = {}
    for k, v in QC.DISCLOSED_MANUSCRIPT_REVISIONS_P22.items():
        cur = sha(ROOT / k) if (ROOT / k).is_file() else None
        txt = (ROOT / k).read_text() if cur else ""
        rec[k] = {"differs_from_p21": cur != v["p21_sha256"], "superseded_marker_present": "[superseded in p21
  +> " in txt,
                  "revised_in": v["revised_in"]}
    return all(x["differs_from_p21"] and x["superseded_marker_present"] for x in rec.values()), rec


def evanescent_check():
    c = jl(f"certificates/{QC.CERT_PREFIX}_evanescent.json")
    ev = c.get("EV_CT", {})
    chk = {
        "EV_M2_proven": c.get("EV_M2", {}).get("verdict", "").startswith("PROVEN"),
        "EV_CT_earned": c.get("evanescent_counterterm_earned") is True,
        "controls_all_true": bool(c.get("controls")) and all(c["controls"].values()),
        "hc_closed": ev.get("a_closure", {}).get("h.c", {}).get("closed") is True,
        "cb_closed": ev.get("a_closure", {}).get("c.b", {}).get("closed") is True,
        "identity_unchanged": ev.get("b_projected_identity", {}).get("unchanged") is True,
        "mutations_detected": bool(c.get("mutations")) and all(c["mutations"].values()),
        "continuations_refuted_recorded": c.get("EV_M2", {}).get("continuation_tests", {}).get("D11", {})
        .get("ghat_continuation_holds") is False,
        "Q4S_trace_remnant_disclosed": "O_eps_times_pole_Q4S_trace_external" in ev.get("c_remnants", {}),
```

```
        "P5_status": c.get("P5_calculation_status") == QC.P5_STATUS_P22,
        "contract_revision_named": c.get("contract_revision") == QC.CONTRACT_REVISION,
    }
    return all(chk.values()), chk, c


def _main():
    low_ok, low = R21.lower_layers()
    l20_ok, l20, _r20 = R21.p20_layer()
    pin_ok, pin, _pc = R20.d11_pin_check()
    l21_ok, l21, r21 = p21_layer()
    con_ok, con = contract_check()
    ev_ok, ev, evc = evanescent_check()
    g_ok, g = guard_probe()
    p21i_ok, p21i = p21_independent()
    md_ok, md = manuscript_disclosure()
    hard = dict(QC.HARD_STOPS_P22)
    hard_ok = bool(all(r21.get("hard_stops", {}).get(k) == v for k, v in hard.items())
                   and r21.get("bulk_symmetry_restored") is False)
    cl = change_log()
    execution_pass = bool(low_ok and l20_ok and pin_ok and l21_ok and con_ok and ev_ok and g_ok and hard_ok an
  +> d md_ok
                          and cl["base_pinned_ok"] and cl["contract_changes_registered"]
                          and not cl["undisclosed_changes"] and not cl["pinned_sha_mismatches"]
                          and p21i.get("pins_ok") is True)
    publish = execution_pass
    author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
    indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
    earned = {k: ("earned" if (publish and r21.get(k) == "earned") else "not_earned") for k in Q21.EARNABLE_P2
  +> 1}
    earned.update({"evanescent_M2_verified": "earned" if publish and ev["EV_M2_proven"] else "not_earned",
                   "evanescent_counterterms_verified": "earned" if publish and ev_ok else "not_earned",
                   "d_runtime_guard_verified": "earned" if publish and g_ok else "not_earned"})
    ms = json.loads(json.dumps(r21.get("milestones", {})))
    cert = dict(r21)
    for k in ("absolute_pin", "lower_layers", "p20_layer", "manuscript_disclosure", "change_log_vs_p20", "open
  +> _pins",
              "disclosed_p20_layer_revisions_p21", "disclosed_test_revisions_p21", "disclosed_manuscript_revis
  +> ions",
              "independent_rebuild_passed_p20", "p20_independent_rebuild"):
        if k in cert:
            cert[f"p21_layer_{k}"] = cert.pop(k)
    scope = evc.get("D11_matching_verified_scope")
    cert.update({
        "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
  +> TE,
        "version_label": QC.VERSION_LABEL,
        "deliverable": ("p22: evanescent M2 statement, evanescent counterterms (p22 contract revision), runtim
  +> e d guard "
                        "(p22 contract revision), gate without prefix exemptions"),
        "execution_pass": execution_pass,
        "claim_status": (("p22. EV-M2: the 8 evanescent D11 structures have local UV poles and their renormali
  +> sed "
                          "remainder vanishes identically on QdS external legs (exact in d); they are NOT matc
  +> hed to XVI-O "
                          "(no counterpart; both natural d-dim continuations refuted). EV-CT: the declared eva
  +> nescent "
                          "counterterms (h.h gt-operators, K'.c E2, cb.c SE; coefficient = -pole) are sigma_O-
  +> closed "
                          "together with the 4D MS-bar counterterm and leave the projected identity unchanged;
  +>  P.5 "
                          "calculation status: " + QC.P5_STATUS_P22 + ". Runtime d guard at the sympy entry po
  +> ints "
                          "(contract revision); static scan secondary. Projected (ch-only) results as in p20/p
  +> 21; bulk "
                          "symmetry NOT restored/claimed; O.1 NOT_CLOSED.") if publish else "p22: not accepted
  +> "),
        "D11_matching_verified_scope": scope if publish else cert.get("D11_matching_verified_scope"),
        "evanescent_hh_counterterm_added": bool(publish and ev_ok),
        "evanescent": {"check": ev, "EV_M2": evc.get("EV_M2", {}).get("statement"),
                       "EV_M2_verdict": evc.get("EV_M2", {}).get("verdict"),
                       "coefficients_hh_sum": evc.get("EV_CT", {}).get("coefficients_hh_counterterm_sum"),
                       "coefficient_Kc_E2": evc.get("EV_CT", {}).get("coefficient_Kc_E2"),
                       "coefficient_ccb_SE": evc.get("EV_CT", {}).get("coefficient_ccb_SE"),
                       "remnants": evc.get("EV_CT", {}).get("c_remnants")},
        "P5_calculation_status": QC.P5_STATUS_P22 if publish else "renormalized (4D) / regulated (evanescent p
  +> oles)",
        "d_runtime_guard": {"ok": g_ok, "n_plants": g.get("n_plants"), "not_raised": g.get("not_raised"),
                            "other_outcomes": g.get("other"), "algebra_ok": g.get("algebra_ok"),
                            "entry_points": g.get("entry_points"), "residue_not_interceptable": QC.RUNTIME_GUA
  +> RD_RESIDUE_P22,
```

```
                        "static_scan": "secondary barrier (tests/test_p21.py scan4 allow-list, p22 sites r
+> eviewed)"},
      "contract": con, "lower_layers": low, "p20_layer": l20, "p21_layer": l21, "absolute_pin_check": pin,
      "manuscript_disclosure": md, "change_log_vs_p21": cl, "open_pins": list(QC.OPEN_PINS_P22),
      "release_claim_accepted": publish,
      "release_claim_accepted_definition": ("contract = registered p22 revision only (pinned) AND lower laye
+> rs re-checked "
                                            "live AND p20/p21 layers AND absolute pin AND EV-M2 + EV-CT cert
+> ificate AND "
                                            "live runtime guard probe AND every added/modified/removed file
+> disclosed "
                                            "(no prefix exemption) AND the pinned p21 independent record AND
+>  the p22 "
                                            "hard stops"),
      "publication": "accepted" if publish else "REJECTED",
      **earned,
      "disclosed_revisions_p22": QC.DISCLOSED_REVISIONS_P22,
      "disclosed_manuscript_revisions_p22": QC.DISCLOSED_MANUSCRIPT_REVISIONS_P22,
      "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": ms,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p21": p21i_ok, "p21_independent_rebuild": p21i,
      "rebuild_flag_scope": ("author_rebuild_passed and independent_rebuild_passed refer to the CURRENT (p22
+> ) source "
                             "tree only (independent: pending until a reviewer report for this tree exists);
+>  "
                             f"independent_rebuild_passed_p21 refers to the p21 tree {QC.P21_SOURCE_TREE_SHA
+> 256} / zip "
                             f"{QC.P21_SOURCE_ZIP_SHA256} (same-box caveat) and is not inherited by p22"),
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p21_archive": ARCH_P21,
                          "kind": "package evidence (XVI-0 conv:package); not a mathematical certificate"},
      "XVI_P_registered": False, "symmetry_restored": False, "bulk_symmetry_restored": False,
      "O1_closure": "NOT_CLOSED", "insertion_status": "Frozen_negative_obstruction_preserved",
      "XV_B_opened": False, "XIV_J_opened": False, "Route_B_discharged": False, "u_R_c_delta_added": False,
      "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
          n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
          ("P21_release.json", f"{QC.CERT_PREFIX}_evanescent.json") if (ROOT / "certificates" / n).is_file()
+> },
  })
  if publish:
      st = ms.setdefault("status", {})
      st["P.5_identity"] = str(st.get("P.5_identity", "")) + " [p22: evanescent counterterms closed; identit
+> y unchanged]"
      st["P.5_calculation"] = QC.P5_STATUS_P22
  cdir = ROOT / "certificates"
  if publish:
      txt = certio.dumps(cert)
      for s_ in STATUS_FILES:
          (ROOT / s_).write_text(txt)
      (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
      (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
      certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
          "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
+> tatus)",
          **ms, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
          "independent_rebuild_passed_p21": p21i_ok, "hard_stops": hard,
          "flags": {**earned, "quantum_inputs_evaluated": cert.get("quantum_inputs_evaluated"),
                    "bulk_symmetry_restored": False}})
  else:
      _reject({"lower_layers_ok": low_ok, "p20_layer_ok": l20_ok, "absolute_pin_ok": pin_ok, "p21_layer_ok":
+>  l21_ok,
               "p21_layer": l21, "contract_ok": con_ok, "contract": con, "evanescent_ok": ev_ok, "evanescent
+> ": ev,
               "guard_ok": g_ok, "guard_not_raised": g.get("not_raised"), "guard_error": g.get("error"),
               "manuscript_disclosure_ok": md_ok, "p21_independent_pins_ok": p21i.get("pins_ok"),
               "hard_stops_hold": hard_ok, "change_log_base_ok": cl["base_pinned_ok"],
               "undisclosed_changes": cl["undisclosed_changes"], "pinned_sha_mismatches": cl["pinned_sha_mis
+> matches"],
               "contract_changes_registered": cl["contract_changes_registered"]})
  print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
+> ion"],
                    "lower_layers": low_ok, "p20_layer": l20_ok, "p21_layer": l21_ok, "absolute_pin": pin_ok
+> ,
                    "contract": con_ok, "evanescent": ev_ok, "guard": g_ok, "manuscript": md_ok, "hard_ok":
+> hard_ok,
                    **earned, "author": author_ok, "independent_p22": indep_ok, "independent_p21": p21i_ok,
                    "undisclosed": cl["undisclosed_changes"], "pin_mismatch": list(cl["pinned_sha_mismatches
+> "]),
                    "change_log_counts": cl["counts"]}, indent=1))
```

```
    return 0 if publish else 1


def _reject(detail):
    cdir = ROOT / "certificates"
    (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
    for s_ in STATUS_FILES:                      # a reject must not leave a stale RELEASE_STATUS.json
        (ROOT / s_).unlink(missing_ok=True)
    (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
        {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", **detail}, indent=1, default
  +> =str) + "\n")


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        _reject({"error": f"{type(exc).__name__}: {exc}"})
        print(f"release_p22 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

162 Source: tests/test__p22.py

[listing 129/133] tests/test_p22.py

```
"""XVI-P p22 tests: EV-M2 (evanescent matching statement), EV-CT (evanescent counterterms: closure, identity u
  +> nchanged,
remnants), the p22 runtime d guard (contract revision; every reviewer plant raises; the residue is disclosed),
  +>  the
gate without prefix exemptions (tools/, earlier-layer notes, added code files), the superseded-text marker, st
  +> atus."""
from __future__ import annotations
import hashlib
import json
import operator
import shutil
import subprocess
import sys
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools", "tests"):
    sys.path.insert(0, str(ROOT / p))
import p22_conventions as QC  # noqa: E402
import dred  # noqa: E402
import contract_registry as CR  # noqa: E402

CERT = ROOT / "certificates"
d = dred.d


def cj(name):
    return json.loads((CERT / name).read_text())


def rs():
    # p23 (disclosed): the p22 LAYER record (the top-level RELEASE_STATUS.json is the p23 status)
    return json.loads((ROOT / "certificates" / "P22_release.json").read_text())


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


# ============================================================ EV-M2
def test_ev_m2_statement_and_scope():
    c = cj("P22_evanescent.json")
    m = c["EV_M2"]
    assert m["verdict"].startswith("PROVEN") and m["locality"] is True
    assert all(v == "0" for v in m["four_D_decoupling"].values()) and len(m["four_D_decoupling"]) >= 8 + 2
    ct = m["continuation_tests"]["D11"]
```

```
    assert ct["ghat_continuation_holds"] is False and ct["g4_kept_continuation_holds"] is False   # refuted, r
  +> ecorded
    assert ct["hatted_externals_agree_both"] is True
    assert "NOT matched to XVI-O" in c["D11_matching_verified_scope"]
    assert rs()["D11_matching_verified_scope"] == c["D11_matching_verified_scope"]
    assert "REFUTED" in m["candidates"]["match a d-dim continuation of the XVI-O kernel"]


def test_d11_evanescent_poles_are_the_stored_ones():
    c = cj("P22_evanescent.json")
    r = cj("P18_D11.json")["result"]
    ev = {k: v for k, v in r.items() if v["evanescent"]}
    assert len(ev) == 8
    co = c["EV_CT"]["coefficients_hh_counterterm_per_graph"]["D11"]
    for k, v in ev.items():
        assert sp.simplify(dred.parse_d(co[k], {"I": sp.I}) + dred.parse_d(v["UV_pole"], {"I": sp.I})) == 0, k


# ============================================================== EV-CT
def test_ev_ct_coefficients_closure_identity():
    c = cj("P22_evanescent.json")
    e = c["EV_CT"]
    assert e["coefficient_Kc_E2"] == "-1/16" and e["coefficient_ccb_SE"] == "1/8"
    assert e["coefficients_hh_counterterm_sum"]["g4_mn gt_rs (p.p)^2"] == "-7*I/120"
    a = e["a_closure"]
    assert a["h.c"]["closed"] is True and a["h.c"]["p_dot_T_ev_sum"] == {} and a["h.c"]["E_dot_E2"] == 0
    assert a["h.c"]["p_dot_T_ev_per_graph"]["D11"] == {} and a["h.c"]["p_dot_T_ev_per_graph"]["D01"] != {}
    assert a["c.b"]["closed"] is True and a["c.b"]["calibration_rho_from_4D_pair"] == "1/2"
    assert a["c.b"]["ghost_equation_exact_in_d_gt_sector"] is True and a["c.b"]["image_ev_pair"] == {}
    b = e["b_projected_identity"]
    assert b["unchanged"] is True and all(v == "0" for v in b["hh_slot_values_sum"].values())
    assert all(v == "0" for v in b["Kc_slot_values"].values())
    assert any(v != "0" for v in b["hh_slot_values_per_graph"]["D01"].values())     # only the SUM decouples
    assert all(c["mutations"].values()) and c["insensitive_variations"]["detected"] is False
    assert all(c["controls"].values()) and c["evanescent_counterterm_earned"] is True


def test_ev_ct_remnants_honest():
    r = cj("P22_evanescent.json")["EV_CT"]["c_remnants"]
    q = r["O_eps_times_pole_Q4S_trace_external"]
    assert q["finite_term_if_external_trace_mode_uses_g4"] == "7*I*p.p*(a^.b^*p.p - a^.p*b^.p)/60"
    assert "NONZERO" in q["status"] and "QdS" in q["status"]
    assert "not evaluated, not claimed" in r["where_O_eps_x_pole_would_enter"]


def test_ev_ct_c_b_closure_live():
    """live: the de Donder image of the E2 pole + rho x SE pole vanishes; rho from the 4D pair (T2, T3 vs S1,
  +> S2)."""
    import evanescent as EV
    import e2_evanescent as E2M
    X4 = {"T2": sp.Rational(3, 16), "T3": sp.Rational(-1, 4)}
    Y4 = {"S1": sp.Rational(-3, 8), "S2": sp.Rational(1, 4)}
    rho = sp.Symbol("rho")
    img = EV.close([(c * X4[k], f) for k in X4 for c, f in EV.dhat([tuple(t) for t in E2M.STRUCT[k]], E2M.PPOW
  +> [k])]
                   + [(rho * c * Y4[k], f) for k in Y4 for c, f in EV.ytensor(k)])
    sol = sp.solve(list(img.values()), rho, dict=True)
    assert sol == [{rho: sp.Rational(1, 2)}]
    ev = EV.close([(c * sp.Rational(1, 16), f) for c, f in EV.dhat([tuple(t) for t in E2M.STRUCT["E2"]], 1)]
                  + [(sp.Rational(1, 2) * c * sp.Rational(-1, 8), f) for c, f in EV.ytensor("SE")])
    assert ev == {}
    bad = EV.close([(c * sp.Rational(1, 16), f) for c, f in EV.dhat([tuple(t) for t in E2M.STRUCT["E2"]], 1)])
    assert bad != {}


def test_p5_status_renormalized_scoped():
    s = rs()
    assert s["P5_calculation_status"] == QC.P5_STATUS_P22 and "QdS" in QC.P5_STATUS_P22
    assert s["evanescent_hh_counterterm_added"] is True
    assert s["milestones"]["status"]["P.5_calculation"] == QC.P5_STATUS_P22


# ============================================================== runtime d guard (contract revision)
X = sp.Symbol("x")
EXPRS = {"linear_and_pow": d * X + 1 / (d - 1), "power_base_only": 1 / (d ** 2 - 1), "d_alone": d,
         "mixed_pow": X * d ** 2 + d}


def _poly_input(e):
    return sp.cancel(e * (d ** 2 - 1) * (d - 1))
```

```
PLANTS = {
    "subs_d": lambda e: e.subs(d, 4), "xreplace_d": lambda e: e.xreplace({d: 4}),
    "list_unpack_alias_then_subs": lambda e: (lambda D: e.subs(D, 4))(*[d]),
    "free_symbols_pop_subs": lambda e: e.subs([y for y in e.free_symbols if y.name == "d"][0], 4),
    "lambda_wrapper_Poly": lambda e: (lambda *a: sp.Poly(*a))(_poly_input(e), d).eval(4),
    "coeff_extract": lambda e: sum(sp.expand(e).coeff(d, k) * 4 ** k for k in range(5)),
    "dunder_getattribute_eq": lambda e: sp.Add(*[4 if y == d else y for y in e.__getattribute__("args") + (d,)
  +> ]),
    "subs_power": lambda e: e.subs(d ** 2, 16), "subs_dict": lambda e: e.subs({d: 4}),
    "subs_list": lambda e: e.subs([(d, 4)]), "evalf_subs": lambda e: e.evalf(subs={d: 4}),
    "N_subs": lambda e: sp.N(e, subs={d: 4}), "Lambda": lambda e: sp.Lambda(d, e)(4),
    "Subs_doit": lambda e: sp.Subs(e, d, 4).doit(), "limit": lambda e: sp.limit(e, d, 4),
    "replace": lambda e: e.replace(d, 4), "replace_fn": lambda e: e.replace(lambda y: y == d, lambda y: 4),
    "as_poly_call": lambda e: _poly_input(e).as_poly(d)(4), "all_coeffs": lambda e: sp.Poly(_poly_input(e), d)
  +> .all_coeffs(),
    "as_expr_values": lambda e: sp.Poly(_poly_input(e), d).as_expr(4),
    "collect": lambda e: sp.collect(sp.expand(e), d, evaluate=False),
    "as_independent": lambda e: sp.expand(e).as_independent(d), "series": lambda e: e.series(d, 4, 2),
    "rename_then_value": lambda e: e.subs(d, sp.Dummy()), "lambdify": lambda e: sp.lambdify(d, e)(4),
    "getattr_subs": lambda e: getattr(e, "subs")(d, 4),
    "methodcaller": lambda e: operator.methodcaller("subs", d, 4)(e),
    "unbound_Basic_subs": lambda e: sp.Basic.subs(e, d, 4),
    "Symbol_new_classmethod": lambda e: e.subs(sp.Symbol.__new__(type(d), "d"), 4),
    "Function_call_d": lambda e: sp.Function("f")(e).subs(d, 4).args[0],
}


def test_plant_list_matches_conventions():
    assert set(PLANTS) == set(QC.GUARD_PLANTS_P22)


@pytest.mark.parametrize("expr", sorted(EXPRS))
@pytest.mark.parametrize("plant", sorted(PLANTS))
def test_guard_every_plant_raises(plant, expr):
    with pytest.raises(dred.PrematureD4Substitution):
        PLANTS[plant](EXPRS[expr])


def test_guard_power_base_hole_closed():
    with pytest.raises(dred.PrematureD4Substitution):
        dred.parse_d("1/(d**2 - 1)").subs(dred.d, 4)            # p21 limitation, closed by the p22 contract r
  +> evision


def test_guard_ordinary_algebra_and_authorised_paths():
    e = (d ** 2 - 2 * d - 2) / (32 * (d ** 2 - 1))
    assert sp.factor(sp.cancel(sp.together(e + 1 / (d + 1)))) != 0
    assert sp.cancel(sp.apart(e, d) - (sp.Rational(1, 32) - 1 / (64 * (d + 1)) - 3 / (64 * (d - 1)))) == 0
    r = sp.Symbol("r")
    assert sp.solve(r * d - 1, r) == [1 / d]
    assert dred.substitute_d(e, 4, "test_p22") == sp.Rational(1, 80)
    with dred.authorised_d("test_p22: authorised coefficient read"):
        assert sp.Poly(sp.cancel(e * 32 * (d ** 2 - 1)), d).all_coeffs() == [1, -2, -2]
    assert "test_p22: authorised coefficient read" in dred.D_PROVENANCE
    with pytest.raises(dred.PrematureD4Substitution):                       # authorisation does not leak
        e.subs(d, 4)


def test_guard_symbol_new_and_plain_d_produce_no_value():
    plain = sp.Symbol.__new__(sp.Symbol, "d")
    assert plain != d and (d * X).subs(plain, 4) == d * X
    with pytest.raises(dred.PlainDimensionSymbol):
        dred.assert_opaque_d(plain + 1)


def test_guard_residue_disclosed_and_demonstrated():
    """the residue the guard cannot intercept is disclosed (and covered only by the secondary static scan)."""
    txt = QC.RUNTIME_GUARD_RESIDUE_P22
    for w in ("identity", "string round-trips", "Poly.rep", "static allow-list scan"):
        assert w in txt
    e = X * d ** 2 + d
    assert sp.Add(*[4 if y is d else y for y in e.args]).has(d)          # `is` test: d**2 survives ...
    assert sp.sympify(str(e).replace("d", "4")) == 16 * X + 4            # ... a string round-trip is NOT inte
  +> rcepted


def test_guard_entry_points_recorded_and_registry():
    for k in ("Basic.subs", "Basic.xreplace", "Atom.xreplace (bare d)", "Poly.eval", "Expr.coeff", "collect",
  +> "limit",
              "Lambda(d, ...)", "OpaqueDimension.__eq__ (d == d)", "evalf/n(subs=...)"):
```

```
        assert k in dred.GUARDED_ENTRY_POINTS, k
    r = CR.dred_revision(ROOT)
    assert r["state"] == "p18 + p22 guard block" and r["p18_core_byte_identical"] is True
    assert sha(ROOT / CR.DRED) == QC.P22_PINNED_SHA256[CR.DRED] or CR.p23_registered(ROOT)   # p23 registered
  +> block
    assert rs()["d_runtime_guard"]["ok"] is True and rs()["d_runtime_guard"]["not_raised"] == {}


def test_guard_not_in_block_is_unregistered(tmp_path):
    (tmp_path / "code/p5_entry").mkdir(parents=True)
    t = (ROOT / CR.DRED).read_text()
    (tmp_path / CR.DRED).write_text(t.replace('ETA = (1, -1, -1, -1)', 'ETA = (1, 1, 1, 1)'))
    assert CR.dred_revision(tmp_path)["state"] == "unregistered"
    t = t[:t.index(CR.BEGIN23)] if CR.BEGIN23 in t else t          # p23: strip the registered p23 block too
    (tmp_path / CR.DRED).write_text(t[:t.index(CR.BEGIN)] + t[t.index(CR.END) + len(CR.END):])
    assert CR.dred_revision(tmp_path)["state"] == "p18"


# ============================================================ T: science text
def test_p20_absolute_open_passage_marked_superseded():
    txt = (ROOT / "manuscript/notes/P20_correction.tex").read_text()
    i = txt.index("Absolute pin: OPEN")
    assert "[superseded in p21" in txt[i:i + 200]
    assert sha(ROOT / "manuscript/notes/P20_correction.tex") != \
        QC.DISCLOSED_MANUSCRIPT_REVISIONS_P22["manuscript/notes/P20_correction.tex"]["p21_sha256"]


# ============================================================ status
def test_p22_release_status_and_flags():
    s = rs()
    assert s["revision"] == "p22" and s["publication"] == "accepted" and s["execution_pass"] is True
    for k in QC.EARNABLE_P22:
        assert s[k] == "earned", k
    assert s["bulk_symmetry_restored"] is False and s["symmetry_restored"] is False and s["O1_closure"] == "NO
  +> T_CLOSED"
    assert s["insertion_status"] == "Frozen_negative_obstruction_preserved"
    assert s["hard_stops"] == QC.HARD_STOPS_P22 and s["hard_stops_hold"] is True
    # p23: the p22 LAYER gate evaluates the "current" independent flag against the p23 tree -> never True here
    assert s["independent_rebuild_passed"] is not True and s["independent_rebuild_passed_p21"] is True
    assert s["change_log_vs_p21"]["undisclosed_changes"] == [] and s["change_log_vs_p21"]["pinned_sha_mismatch
  +> es"] == {}
    assert s["contract_revision"] == QC.CONTRACT_REVISION and "DISCLOSED p22 contract revision" in QC.CONTRACT
  +> _REVISION


def test_p21_independent_recorded_and_not_inherited():
    rec = rs()["p21_independent_rebuild"]
    assert rec["report_sha256"] == QC.P21_INDEPENDENT_REPORT_SHA256 and rec["pins_ok"] is True
    assert rec["bound_to_source_tree_sha256"] == QC.P21_SOURCE_TREE_SHA256 and rec["tree_matches_p21"] is True
    assert rec["bound_to_source_zip_sha256"] == QC.P21_SOURCE_ZIP_SHA256 and rec["mismatches"] == 0
    import source_manifest as SM
    assert SM.tree_sha(ROOT) != QC.P21_SOURCE_TREE_SHA256


def test_disclosure_complete_and_exact():
    cl = rs()["change_log_vs_p21"]
    changed = set(cl["added"]) | set(cl["modified"]) | set(cl["removed"])
    listed = set(QC.DISCLOSED_REVISIONS_P22)
    assert cl["disclosed_but_unchanged"] == [] and listed <= changed
    for x in changed - listed:
        assert (x in QC.GENERATED_EXEMPT_P22 or (x in cl["added"] and x.startswith(QC.DISCLOSED_PREFIXES_ADDED
  +> _P22))
                or CR.p23_disclosed(x)), x                    # p23: registered later revisions (pinned by the
  +>  p23 gate)
    assert cl["contract_files_changed"] == ["code/p5_entry/dred.py"] and cl["contract_changes_registered"] is
  +> True
    assert set(CR.P22_REVISED_FILES) <= listed


def test_version_label_p22():
    import build as B
    assert QC.VERSION_LABEL == "XVI-P p22" and B.QC.VERSION_LABEL == f"XVI-P {B.QC.REVISION}"   # p23: current
  +>  label


# ============================================================ p22 gate mutations (fail closed; no stale s
  +> tatus)
def _copy():
    import test_p21 as T21
    return T21._copy()
```

```
def _edit(d_, name, fn):
    f = d_ / "certificates" / name
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c, indent=1))


def _tools_change(d_):                      # reviewer: tools/ was exempt in p21
    f = d_ / "tools" / "manifest.py"            # p23: a tools file NOT revised in p23 (revised ones are pinne
  +> d by the p23 gate)
    f.write_text(f.read_text() + "\n# undisclosed\n")


def _earlier_note_change(d_):               # reviewer: earlier-layer notes were exempt in p21
    f = d_ / "manuscript" / "notes" / "P18_D11.tex"
    f.write_text(f.read_text() + "\n% undisclosed\n")


def _added_code_file(d_):                   # reviewer: added files were not disclosure-checked in p21
    (d_ / "code" / "p5_calc" / "zz_extra.py").write_text("x = 1\n")


def _dred_guard_removed(d_):
    f = d_ / CR.DRED
    t = f.read_text()
    f.write_text(t[:t.index(CR.BEGIN)] + t[t.index(CR.END) + len(CR.END):])


def _dred_core_changed(d_):
    f = d_ / CR.DRED
    f.write_text(f.read_text().replace('ETA = (1, -1, -1, -1)', 'ETA = (1, -1, -1, -1)  # edit'))


def _guard_weakened(d_):                    # in-block edit: subs guard disabled -> sha pin AND live probe fai
  +> l
    f = d_ / CR.DRED
    src = f.read_text()
    old = '_check_rule(_pairs_of_subs(args, kwargs), "subs", 4)'
    assert old in src
    f.write_text(src.replace(old, 'None'))


def _ev_cb_open(d_):
    _edit(d_, "P22_evanescent.json", lambda c: c["EV_CT"]["a_closure"]["c.b"].__setitem__("closed", False))


def _ev_identity_changed(d_):
    _edit(d_, "P22_evanescent.json", lambda c: c["EV_CT"]["b_projected_identity"].__setitem__("unchanged", Fal
  +> se))


def _ev_m2_not_proven(d_):
    _edit(d_, "P22_evanescent.json", lambda c: c["EV_M2"].__setitem__("verdict", "NOT ESTABLISHED"))


def _superseded_marker_removed(d_):
    f = d_ / "manuscript" / "notes" / "P20_correction.tex"
    f.write_text(f.read_text().replace("[superseded in p21", "[note"))


def _p21_layer_rejected(d_):
    _edit(d_, "P21_release.json", lambda c: c.__setitem__("publication", "REJECTED"))


def _tamper_p21_report(d_):
    f = d_ / "archival" / "p21" / "reproducibility" / "independent_rebuild_report_p21.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


MUTS = [_tools_change, _earlier_note_change, _added_code_file, _dred_guard_removed, _dred_core_changed,
        _guard_weakened, _ev_cb_open, _ev_identity_changed, _ev_m2_not_proven, _superseded_marker_removed,
        _p21_layer_rejected, _tamper_p21_report]


def _run(d_):
    return subprocess.run([sys.executable, "code/p5_calc/release_p22.py"], cwd=d_, capture_output=True, text=T
  +> rue,
                          timeout=900)
```

```
@pytest.mark.parametrize("mut", MUTS, ids=lambda f: f.__name__)
def test_p22_gate_mutation_rejects_without_stale_status(mut):
    d_ = _copy()
    try:
        assert (d_ / "RELEASE_STATUS.json").exists()
        mut(d_)
        r = _run(d_)
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d_ / "certificates" / "P22_release.json").exists()
        assert (d_ / "certificates" / "P22_release_rejected.json").exists()
        assert not (d_ / "RELEASE_STATUS.json").exists() and not (d_ / "certificates" / "RELEASE_STATUS.json")
  +> .exists()
    finally:
        shutil.rmtree(d_, ignore_errors=True)


def test_p22_gate_accepts_unmodified_copy():
    d_ = _copy()
    try:
        r = _run(d_)
        assert r.returncode == 0, r.stdout[-1500:]
        s = json.loads((d_ / "RELEASE_STATUS.json").read_text())
        assert s["revision"] == "p22" and s["evanescent_counterterms_verified"] == "earned"
        assert s["d_runtime_guard_verified"] == "earned" and s["bulk_symmetry_restored"] is False
    finally:
        shutil.rmtree(d_, ignore_errors=True)
```

163 Source: p23_conventions.py (p23 constants, disclosures, pinned EV values, poly plants)

[listing 130/133] code/p5_calc/p23_conventions.py

```
"""XVI-P p23 conventions (stage release on the 0.1 direction).  Inherits p22; adds the 0.1 requirement record,
  +>  the one-loop
power counting / enlarge-rule check and the P.6 ENTRY inventory (kappa^3 three-point, topology level), the p23
  +>  contract
revision (polynomial-domain refusal in dred.py, one delimited block after the p22 block), the EV value check a
  +> nd the
'two-point / QdS' qualifier in the title line."""
from p22_conventions import *  # noqa: F401,F403
import p22_conventions as _Q22

REVISION = "p23"
VERSION_LABEL = "XVI-P p23"
CERT_PREFIX = "P23"
DATE = "2026-10-10"
AUTHOR_REPORT = "reproducibility/author_rebuild_report_p23.json"
INDEPENDENT_REPORT_CURRENT = "reproducibility/independent_rebuild_report_p23.json"
CONTRACT_REVISION = ("p23 (p14r contract + disclosed p15/p16/p17/p18 revisions + the p22 contract revision + t
  +> he DISCLOSED "
                     "p23 contract revision: polynomial-domain refusal for d and Dummy-only internal renames,
  +> one "
                     "delimited block appended to code/p5_entry/dred.py; p18 core and p22 block byte-identical
  +> )")

# ------------------------------------------------------------------ p22 layer in p23
P22_INDEPENDENT_REPORT = "reproducibility/independent_rebuild_report_p22.json"
P22_INDEPENDENT_REPORT_SHA256 = "023e5b83b1eba35d7c27b75841f446cb259580808410dbfb00253ea0122b15f3"
P22_SOURCE_TREE_SHA256 = "1bc6edff6ca0788bb0fee4e8bf259bf8a0c80791f6dd08be77f79e56b492a50c"
P22_SOURCE_ZIP_SHA256 = "ade2fa5c2dac6614c41ae38dc97d7c485975015a134650734d466ea6ffdef76f"
P22_SOURCE_LIST = "archival/p22/p22_source_files.json"
P22_ARCHIVED_RECORDS = {
    "archival/p22/p22_source_files.json": "75fb3bf4dfcbf6ddffe98ad6dae061a85cce8d4b070ca71ad826b544dc3c0111",
    "archival/p22/reference_values_p22.json": "0d55633950c993e905627774d3962285f3bbdc7372208c9975170300601eef7
  +> 2",
    "archival/p22/reproducibility/author_rebuild_report_p22.json":
        "ef5055428a0a550526857094dd0defe7ed00e76ff6ad5b55de4e6ad1451d1b34",
    "archival/p22/reproducibility/independent_rebuild_report_p22.json": P22_INDEPENDENT_REPORT_SHA256,
    "archival/p22/certificates/P22_evanescent.json": "91ea75b7da0d839513611f19665d1008c38fe0c90da326d49008099d
  +> 1ce4d463",
}
EARNABLE_P23 = _Q22.EARNABLE_P22 + ("O1_requirement_recorded", "one_loop_power_counting_verified",
                                    "P6_entry_inventory", "d_poly_domain_refusal_verified", "EV_values_verifie
  +> d")
HARD_STOPS_P23 = dict(_Q22.HARD_STOPS_P22)
OPEN_PINS_P23 = tuple(_Q22.OPEN_PINS_P22)
P5_STATUS_P23 = _Q22.P5_STATUS_P22
```

```
TITLE_QUALIFIER = "P.5 renormalized at one loop kappa^2 for the two-point functions with QdS external legs"

# ------------------------------------------------------------------ 0.1 (read-only source, verbatim copy pinned)
O1_LEDGER = "provenance/XVI-O/O_quantum_target.json"
O1_LEDGER_SHA256 = "3cd3d1ba491381ba0a73d1677bfed10d3743e0575f61c71c325e626e4a78a645"   # = XVI-O/ledgers/O_qu
  +> antum_target.json (o70-sync)
P6_ENTRY_E3_GRAPHS = 15
P6_ENTRY_E3_BY_EXTERNAL = {"K,c,h": 2, "c,c,cs": 1, "c,cb,h": 2, "c,phi,phis": 1, "h,h,h": 5, "h,phi,phi": 4}
ONE_POINT_E1_GRAPHS = 3

# ------------------------------------------------------------------ EV values checked by the gate (reviewer p22)
EV_EXPECTED = {
    "calibration_rho_from_4D_pair": "1/2", "coefficient_Kc_E2": "-1/16", "coefficient_ccb_SE": "1/8",
    "coefficients_hh_counterterm_sum": {
        "g4_mn gt_rs (p.p)^2": "-7*I/120", "g4_mr gt_ns (p.p)^2": "-I/160", "g4_rs gt_mn (p.p)^2": "-7*I/120",
        "gt_mn gt_rs (p.p)^2": "I/160", "gt_mn p_r p_s (p.p)^1": "7*I/120", "gt_mr gt_ns (p.p)^2": "I/160",
        "gt_mr p_n p_s (p.p)^1": "I/160", "gt_rs p_m p_n (p.p)^1": "7*I/120"},
}

# ------------------------------------------------------------------ poly-domain refusal probe (reviewer p22 form
  +> s)
POLY_PLANTS_P23 = {
    "rem": "sp.rem(numer(e), d - 4, d)", "resultant": "sp.resultant(numer(e), d - 4, d)",
    "compose": "sp.compose(numer(e), 4, d)", "div": "sp.div(numer(e), d - 4, d)",
    "prem": "sp.prem(numer(e), d - 4, d)", "Poly_shift_as_coeff_add": "Poly(numer(e), d).shift(-4).as_expr().a
  +> s_coeff_add()",
    "Poly_rem_method": "Poly(numer(e), d).rem(Poly(d - 4, d))", "Matrix_subs_rename": "Matrix([e]).subs(d, y).
  +> subs(y, 4)",
    "Matrix_xreplace_rename": "Matrix([e]).xreplace({d: y}).subs(y, 4)",
}

# ------------------------------------------------------------------ disclosure (every change vs the p22 zip)
DISCLOSED_REVISIONS_P23 = {
    "code/p5_entry/dred.py": "p23 contract revision: polynomial-domain refusal + Dummy-only internal renames (
  +> appended block)",
    "code/p5_calc/contract_registry.py": "p23 block recognised; p22 block pinned byte-identical; p23 revised/a
  +> dded lists",
    "code/p5_calc/release_p21.py": "accepts the p23 registered revisions (contract_registry.P23_REVISED_FILES)
  +> ",
    "code/p5_calc/release_p22.py": "accepts the registered p23 dred state and p23-disclosed files (pinned by t
  +> he p23 gate)",
    "code/p5_calc/power_counting.py": "new: O.1 requirement, one-loop power counting, enlarge rule, P.6 entry
  +> inventory",
    "code/p5_calc/p23_conventions.py": "new",
    "code/p5_calc/release_p23.py": "new: p23 gate",
    "tests/test_p14.py": "p23 names / status expectations",
    "tests/test_p21.py": "allow-list scan reads p23 sites; status expectations",
    "tests/test_p22.py": "p22 layer record read from certificates/P22_release.json; version label current",
    "tests/test_p23.py": "new",
    "tools/build.py": "p23 stages (power_counting, release_p23), listings, value blocks, archival p22",
    "tools/source_manifest.py": "p23 zip / reference; p22 archive records",
    "tools/rebuild_from_source.py": "p23 reference / certificates / ledgers",
    "README.md": "p23 section; title line with the two-point / QdS qualifier",
    "REBUILD_INSTRUCTIONS.md": "p23",
    "manuscript/body/XVI-P-body.tex": "input of the p23 note",
    "manuscript/notes/P23_O1_scope.tex": "new: p23 note",
    "provenance/XVI-O/O_quantum_target.json": "new: verbatim read-only copy of the XVI-O O.1 ledger (sha pinne
  +> d)",
}
GENERATED_EXEMPT_P23 = ("reproducibility/reference_values_p23.json", "reproducibility/reference_values_p22.jso
  +> n")
DISCLOSED_PREFIXES_ADDED_P23 = ("archival/p22/",)
P23_PINNED_SHA256 = {   # frozen: exact sha256 of every disclosed file (except p23_conventions itself, README.
  +> md, REBUILD_INSTRUCTIONS.md: tree/zip sha only -- their manifest hashes must not appear in a PDF)
    "code/p5_calc/contract_registry.py": "07f36b4f37c0c29e89e2fdc9093ab5b23e04dc32b80a9bd322e83c1af6be3669",
    "code/p5_calc/power_counting.py": "2d80c9d0da4b671ddba820bad820045fc1fe388ecc537e7a79725d82a9b24a75",
    "code/p5_calc/release_p21.py": "6bc5ce80f40ec311a662f078ac5be9e23d752f1fc37d4affefedb24a0d80a0cf",
    "code/p5_calc/release_p22.py": "6a9502c134e7fe55f70e7343e2515b4d66965eb167b606de6a37d8c748fe96a7",
    "code/p5_calc/release_p23.py": "505d5dc04b69e1ef54f3ca4aa0c8c67cf9c54872138c946eccf3a6c437275649",
    "code/p5_entry/dred.py": "24112d9a6584613474e96f538840a9ff6aecbf9ebd588cd890fd38d2e295de5a",
    "manuscript/body/XVI-P-body.tex": "2cf0dc365202b07590275721b152a2c9f17841fd45e05264eb052a0a92bde71b",
    "manuscript/notes/P23_O1_scope.tex": "68a691ebc8f5b3032f0fdae50c0e8c72f46d10e164e8bc44a11805c27e8cd424",
    "provenance/XVI-O/O_quantum_target.json": "3cd3d1ba491381ba0a73d1677bfed10d3743e0575f61c71c325e626e4a78a64
  +> 5",
    "tests/test_p14.py": "d5b2111a8efb60f1abc00c6910fa8ab0f8309bb35543d4fa25f2a4b41d7d4a34",
    "tests/test_p21.py": "4d7edc3f4b5e27c1d1aaa546f745666fc852979208425ee78c7f8d745e2879c8",
    "tests/test_p22.py": "764714b0f185356856811d96b01879933096a53d85c5dcc46e3efd3954341546",
    "tests/test_p23.py": "01486f56765e278a789afbe10273c2e751920bec214daaa592c77016ffc9c5f3",
    "tools/build.py": "8bf2de0e1b583eab4b3bfe46c09393d61c4ecdb5f7cecbd02e68ac73c48630c3",
    "tools/rebuild_from_source.py": "6770523164de112711014b616b0dc62564ce8e13f975705c2f50ba97820c2260",
    "tools/source_manifest.py": "a620ccb1a334c5d43557fe0ec91865b2a60d0d3d83519bd8f38d6444327957fd",
```

```
}

# ------------------------------------------------------------- scan allow-lists: reviewed p23 block sites (
  +> dred.py)
DECOMPOSE_SITES_REVIEWED = dict(_Q22.DECOMPOSE_SITES_REVIEWED)
DECOMPOSE_SITES_REVIEWED.update({
    "code/p5_entry/dred.py::_install_poly_refusal::args": "the wrappers' *args (call arguments), not an expres
  +> sion tree"})
DECOMPOSE_SITE_COUNTS = dict(_Q22.DECOMPOSE_SITE_COUNTS)
DECOMPOSE_SITE_COUNTS.update({"code/p5_entry/dred.py::_install_poly_refusal::args": 5})
SYMPY_FROM_IMPORT_REVIEWED = dict(_Q22.SYMPY_FROM_IMPORT_REVIEWED)
SYMPY_FROM_IMPORT_REVIEWED.update({
    "code/p5_entry/dred.py::_install_poly_refusal::import sympy as _sympy":
        "p23 guard installation: rebinds the guarded polytools functions in the sympy namespace",
    "code/p5_entry/dred.py::_install_poly_refusal::import sympy.polys.polytools as _pt":
        "p23 guard installation: wraps the polynomial functions / Poly methods that can evaluate d"})
```

164 Source: power_counting.py (p23 O.1 requirement, power counting, P.6 entry inventory)

[listing 131/133] code/p5_calc/power_counting.py

```
#!/usr/bin/env python3
"""XVI-P p23 (stage 1 of the O.1 direction): O.1 requirement record, exact one-loop power counting, the kappa^
  +> 2
identity decomposition (enlarge-rule check) and the P.6 ENTRY inventory (one-loop kappa^3 three-point graphs).

Exact, mechanical, from the frozen vertex inventory (feynman.vertices) -- no hand-written graph list:
  * every interaction vertex carries kappa^(legs-2) (sources/antifields count as legs)  =>  a one-loop 1PI gra
  +> ph with
    E external legs is exactly kappa^E (I = V at L = 1);  verified vertex by vertex AND graph by graph;
  * all one-loop 1PI graphs (cycles of interaction vertices, >= 3 legs, loop legs paired by the DECLARED propa
  +> gators
    h-h, phi-phi, c-cbar) are enumerated for E = 1, 2, 3 (topology level: species, vertices, loop lines; no sy
  +> mmetry
    factors, no amplitudes);
  * E = 2 must reproduce the frozen kappa^2 one-loop inventory exactly (D01..D06, D11, D12 + D13), E = 1 the
    one-point row;
  * the one-loop ST identity at kappa^2 is (S^(0), Gamma_1^(2)) + (S^(1), Gamma_1^(1)) (+ (S^(2), Gamma_1^(0))
  +>  = 0,
    Gamma_1^(0) is field independent): only E = 2 and E = 1 enter, so the O.1 enlarge rule ("the stated identi
  +> ty
    constrains n-point pieces at the same coupling order") is NOT triggered by one-loop three-point functions
  +> at
    kappa^2 -- the kappa^2 three-point one-loop sector is EMPTY.  Three-point first appears at kappa^3 (P.6 en
  +> try).
This is package evidence (XVI-O conv:package), not a certificate of O.1.
"""
from __future__ import annotations
import hashlib
import json
import sys
import time
from collections import Counter
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in ("code/p5_entry", "code/classical", "code/p5_calc"):
    sys.path.insert(0, str(ROOT / p))
import feynman as F  # noqa: E402
import certio  # noqa: E402

CERT = ROOT / "certificates"
O_LEDGER = ROOT / "provenance" / "XVI-O" / "O_quantum_target.json"   # verbatim copy of XVI-O/ledgers/O_quantu
  +> m_target.json (sha pinned in p23_conventions; XVI-O untouched)
PROPAGATING = {frozenset(("h",)), frozenset(("phi",)), frozenset(("c", "cb"))}   # declared propagators (P14 G
  +> ate 0)
FROZEN_E2 = {   # frozen kappa^2 one-loop inventory (diagram_inventory.json D01..D12 + P15_D13), species level
    "D01_hh_bubble_cubic": ("h,h", ("V3_grav", "V3_grav")), "D02_hh_tadpole_quartic": ("h,h", ("V4_grav",)),
    "D03_hh_ghost_loop": ("h,h", ("V_cb_c_h", "V_cb_c_h")), "D04_phiphi_bubble": ("phi,phi", ("V_h_phi_phi", "
  +> V_h_phi_phi")),
    "D05a_phiphi_grav_tadpole": ("phi,phi", ("V_hh_phi_phi",)), "D06_cc_ghost_two_point": ("c,cb", ("V_cb_c_h"
  +> , "V_cb_c_h")),
    "D11_hh_scalar_loop": ("h,h", ("V_h_phi_phi", "V_h_phi_phi")), "D12_hh_scalar_tadpole": ("h,h", ("V_hh_phi
  +> _phi",)),
    "D13_Kc_ghost_self_energy": ("K,c", ("V_K_c_h", "V_cb_c_h")),
```

```
}


def pair_ok(a, b):
    return frozenset((a, b)) in PROPAGATING


def interaction_vertices(V):
    return {k: v for k, v in V.items() if len(v.legs) >= 3}


def vertex_power_ok(V):
    return {k: (v.kappa_power == len(v.legs) - 2) for k, v in V.items()}


def _vstates(V):
    """(vertex, in_species, out_species, external multiset) for every choice of two distinct loop legs."""
    out = []
    for name, v in V.items():
        L = list(v.legs)
        seen = set()
        for i in range(len(L)):
            for j in range(len(L)):
                if i == j:
                    continue
                ext = tuple(sorted(L[k] for k in range(len(L)) if k not in (i, j)))
                key = (name, L[i], L[j], ext)
                if key not in seen:
                    seen.add(key)
                    out.append(key)
    return out


def _canon(cyc):
    n = len(cyc)
    forms = []
    for seq in (cyc, [(v, o, i, e) for (v, i, o, e) in reversed(cyc)]):
        for r in range(n):
            forms.append(tuple(seq[r:] + seq[:r]))
    return min(forms)


def one_loop_graphs(V, E):
    """all one-loop 1PI graphs with exactly E external legs (interaction vertices only)."""
    Vi = interaction_vertices(V)
    states = _vstates(Vi)
    found = {}

    def ext_of(s):
        return len(s[3])

    def rec(cyc, e):
        if e > E:
            return
        last = cyc[-1]
        if e == E and pair_ok(last[2], cyc[0][1]):
            c = _canon(list(cyc))
            found[c] = c
        if e == E:
            return
        for s in states:
            if ext_of(s) >= 1 and pair_ok(last[2], s[1]):
                rec(cyc + [s], e + ext_of(s))

    for s in states:
        if ext_of(s) >= 1 or E == 0:
            rec([s], ext_of(s))
    out = []
    for c in sorted(found):
        kp = sum(Vi[v].kappa_power for (v, _, _, _) in c)
        ext = sorted(x for (_, _, _, e) in c for x in e)
        out.append({"vertices": [v for (v, _, _, _) in c],
                    "loop_lines": [f"{c[k][2]}-{c[(k + 1) % len(c)][1]}" for k in range(len(c))],
                    "external": ",".join(ext), "kappa_power": kp, "V": len(c), "I": len(c), "L": 1,
                    "kappa_equals_E": kp == E})
    return out


def frozen_match(g2):
    have = Counter((g["external"], tuple(sorted(g["vertices"]))) for g in g2)
    want = Counter((ex, tuple(sorted(vs))) for ex, vs in FROZEN_E2.values())
```

```
    return {"enumerated": sum(have.values()), "frozen": sum(want.values()),
            "missing_from_enumeration": sorted(map(str, (want - have).elements())),
            "not_in_frozen_inventory": sorted(map(str, (have - want).elements())),
            "equal": have == want}


def run(V):
    g = {E: one_loop_graphs(V, E) for E in (1, 2, 3)}
    return g


def main():
    t0 = time.time()
    raw = O_LEDGER.read_bytes()
    oled = json.loads(raw)
    V = F.vertices()
    vp = vertex_power_ok(V)
    g = run(V)
    fm = frozen_match(g[2])
    e1 = sorted((x["external"], tuple(sorted(x["vertices"]))) for x in g[1])
    e3_species = Counter(x["external"] for x in g[3])
    # mutations (must be detected)
    muts = {}

    class _VT:
        def __init__(self, legs, kp):
            self.legs, self.kappa_power = legs, kp
    m1 = {k: _VT(v.legs, v.kappa_power) for k, v in V.items()}
    m1["V3_grav"] = _VT(V["V3_grav"].legs, 2)
    muts["V3_grav_kappa_power_+1"] = not all(vertex_power_ok(m1).values())
    m2 = {k: _VT(v.legs, v.kappa_power) for k, v in V.items() if k != "V_cb_c_h"}
    muts["drop_V_cb_c_h"] = not frozen_match(one_loop_graphs(m2, 2))["equal"]
    m3 = {k: _VT(v.legs, v.kappa_power) for k, v in V.items() if k != "V_hh_phi_phi"}
    muts["drop_V_hh_phi_phi"] = not frozen_match(one_loop_graphs(m3, 2))["equal"]
    PROPAGATING.add(frozenset(("c", "h")))
    muts["spurious_c_h_propagator"] = not frozen_match(one_loop_graphs(V, 2))["equal"]
    PROPAGATING.discard(frozenset(("c", "h")))
    in_scope = {
        "State retained quantum identity (O.6)": ("PARTLY in XVI-P", "projected ch identity at one loop kappa^
  +> 2 stated "
                                                  "and verified (P19/P20), incl. evanescent CTs (P22); the BUL
  +> K "
                                                  "retained identity (ST4b functional) is XVI-O's statement, n
  +> ot changed"),
        "List amplitudes constrained by identity (O.4)": ("IN XVI-P (this record)", "one-loop kappa^2: exactly
  +>  the E=2 "
                                                          "two-point and E=1 one-point graphs; enumeration rep
  +> roduces "
                                                          "the frozen inventory; completeness at topology leve
  +> l only"),
        "S_inv^{[2]} classical consistency (O.2)": ("OUT OF SCOPE", "classical kappa^2 master 2(S0,S2)+(S1,S1)
  +> : grav ell_4 "
                                                    "Incomplete in XVI-O/XVI-N; XVI-A..O untouched -> O.1 cann
  +> ot "
                                                    "close inside XVI-P"),
        "Evaluate inventory graphs (O.5)": ("IN XVI-P", "all E=2 kappa^2 graphs evaluated (P15..P18, D12 Missi
  +> ng never "
                                             "zero per P14 -> see open items)"),
        "Demonstrate identity remainder cancellation / CT solution (O.6/O.7)": (
            "PARTLY in XVI-P", "projected: closure + restoration earned (P20, P22); bulk ST4a/ST4b split: XVI-
  +> O"),
        "Enlarge target if identity requires more legs/orders (O.1)": (
            "IN XVI-P (this record)", "NOT triggered at kappa^2 at one loop (three-point one-loop sector empty
  +>  at "
            "kappa^2); next order kappa^3 three-point = P.6 entry"),
    }
    out = {
        "landmark": "XVI-P", "revision": "p23",
        "deliverable": "p23 stage 1: O.1 exact requirement, one-loop power counting, enlarge-rule check, P.6 e
  +> ntry inventory",
        "O1_source": {"file": O_LEDGER.relative_to(ROOT).as_posix(), "sha256": hashlib.sha256(raw).hexdigest()
  +> , "read_only": True,
                      "gate": oled.get("gate"), "target": oled.get("proposed_target"),
                      "closure_question": oled["closure_check"]["question"],
                      "outcome": oled["closure_check"]["outcome"],
                      "checklist": oled["closure_check"]["checklist"],
                      "enlarge_rules": oled["closure_check"]["enlarge_rules"]},
        "O1_requirement": ("O.1 closes iff the one-loop kappa^2 bulk two-point sector on S_inv closes under a
  +> VERIFIED "
                           "retained quantum identity: every checklist item (O.2 classical kappa^2 master Cons
  +> tructed incl. "
```

```
                          "grav ell_4; O.4 inventory; O.5 all graphs evaluated; O.6 identity verified; O.7 re
  +> mainder "
                          "cancellation / CT solution) earned, and the target enlarged first if the identity
  +> constrains "
                          "more legs/orders at kappa^2."),
        "O1_items_scope": {k: {"scope": a, "evidence": b} for k, (a, b) in in_scope.items()},
        "O1_closable_in_XVI_P": False, "O1_closure_status": "NOT_CLOSED",
        "P6_entry_n_graphs": len(g[3]), "one_point_n_graphs": len(g[1]),
        "O1_closable_reason": "O.2 (classical kappa^2 master, grav ell_4 Incomplete) and the bulk ST4b identit
  +> y are owned "
                              "by XVI-O/XVI-N, which stay untouched",
        "vertex_kappa_power_equals_legs_minus_2": vp, "vertex_rule_holds": all(vp.values()),
        "one_loop_kappa_equals_E": all(x["kappa_equals_E"] for E in g for x in g[E]),
        "identity_at_kappa2": {"terms": ["(S^(0), Gamma_1^(2))  [E=2]", "(S^(1), Gamma_1^(1))  [E=1]",
                                         "(S^(2), Gamma_1^(0)) = 0  [field independent]"],
                               "orders_entering": ["E=2 two-point (hh, phiphi, c.cb, K.c)", "E=1 one-point"],
                               "three_point_one_loop_at_kappa2": "EMPTY (kappa^E with E=3 is kappa^3)",
                               "enlarge_rule_triggered": False},
        "graphs_E1": [{"external": a, "vertices": list(b)} for a, b in e1],
        "graphs_E2": g[2], "frozen_E2_match": fm,
        "P6_entry_kappa3_three_point": {"n_graphs": len(g[3]), "by_external": dict(sorted(e3_species.items()))
  +> ,
                                        "graphs": g[3],
                                        "status": "inventory only (topology level). NOT evaluated: amplitude r
  +> ules, "
                                                  "counterterm domain and evaluation are the next release"},
        "mutations": muts, "mutations_detected": all(muts.values()),
        "seconds": round(time.time() - t0, 2),
    }
    out["pass"] = bool(out["vertex_rule_holds"] and out["one_loop_kappa_equals_E"] and fm["equal"]
                       and out["mutations_detected"] and not out["identity_at_kappa2"]["enlarge_rule_triggered
  +> "])
    certio.write_json(CERT / "P23_O1_scope.json", out) if hasattr(certio, "write_json") else \
        (CERT / "P23_O1_scope.json").write_text(json.dumps(out, indent=1) + "\n")
    print(json.dumps({"pass": out["pass"], "frozen_E2_match": fm, "E1": len(g[1]), "E2": len(g[2]), "E3": len(
  +> g[3]),
                      "E3_by_external": out["P6_entry_kappa3_three_point"]["by_external"], "mutations": muts},
  +>  indent=1))
    return 0 if out["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

165 Source: release_p23.py (p23 release gate)

[listing 132/133] code/p5_calc/release_p23.py

```
#!/usr/bin/env python3
"""XVI-P p23 release gate (stage release on the O.1 direction; WITH the disclosed p23 contract revision).
Runs AFTER the p14r .. p22 gates (release_p22.py -> certificates/P22_release.json, the p22 LAYER record) and r
  +> equires:
  contract        : dred.py = p18 core byte-identical + the p22 block byte-identical to its release + the regi
  +> stered p23
                    block (contract_registry.p23_registered); exact sha pinned in P23_PINNED_SHA256;
  p22 layer       : P22_release accepted with all p22 earnables; p22_conventions.py byte-identical to the p22
  +> zip; the p22
                    contract check, runtime guard probe and EV certificate check re-run LIVE (release_p22 help
  +> ers);
  EV values       : rho, the K'.c E2 and cb.c SE coefficients and the summed h.h counterterm coefficients in
                    certificates/P22_evanescent.json equal BOTH the pinned values (p23_conventions.EV_EXPECTED
  +> ) and the
                    archived p22 certificate (sha pinned)  -- reviewer p22: flags alone were checked;
  poly refusal    : LIVE probe in a fresh interpreter: every POLY_PLANTS_P23 form (reviewer p22: rem / resulta
  +> nt /
                    compose / div / prem / Poly.shift+as_coeff_add / Poly.rem / Matrix.subs and xreplace renam
  +> es) raises
                    PrematureD4Substitution on four expressions; rational-function algebra unchanged;
  O.1 scope       : certificates/P23_O1_scope.json -- the O.1 ledger copy matches its pinned sha; vertex rule
                    kappa^(legs-2); one-loop kappa^E; enumerated E=2 == frozen inventory; enlarge rule not tri
  +> ggered at
                    kappa^2; O1_closable_in_XVI_P false; P.6 entry counts as pinned; mutations detected;
  title qualifier : README title line carries the two-point / QdS qualifier (reviewer p22);
  disclosure      : every added/modified/removed file vs the pinned p22 zip list disclosed; disclosed files sh
  +> a-pinned;
  p22 independent : archived report pinned (023e5b83...), PASS, bound to the p22 tree/zip (same-box caveat);
  hard stops      : bulk_symmetry_restored false; O.1 NOT_CLOSED; insertion frozen; ...
Publication fails closed.  On reject: no P23_release.json and NO RELEASE_STATUS.json is left behind.
```

```
"""
from __future__ import annotations
import hashlib, json, subprocess, sys  # noqa: E401
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
for p in (HERE, ROOT / "code" / "p5_entry", ROOT / "code" / "classical", ROOT / "tools"):
    sys.path.insert(0, str(p))
import p23_conventions as QC  # noqa: E402
import p22_conventions as Q22  # noqa: E402
import certio  # noqa: E402
import source_manifest as SM  # noqa: E402
import contract_registry as CR  # noqa: E402
from release import rebuild_flag  # noqa: E402   (frozen p13 helper)
import release_p22 as R22  # noqa: E402   (helpers only)

ARCH_P22 = "archival/p22"
ARCH_P22_INDEP = f"{ARCH_P22}/reproducibility/independent_rebuild_report_p22.json"
STATUS_FILES = ("RELEASE_STATUS.json", "certificates/RELEASE_STATUS.json")
POLY_PROBE = r'''
import sys, json
sys.path[:0] = [sys.argv[1] + "/code/p5_entry"]
import sympy as sp, dred
d = dred.d; X = sp.Symbol("X"); y = sp.Symbol("y")
num = lambda e: sp.numer(sp.together(e))
EXPRS = {"linear_and_pow": d * X + 1 / (d - 1), "power_base_only": 1 / (d ** 2 - 1), "d_alone": d,
         "mixed_pow": (d ** 2 - 2 * d - 2) / (32 * (d ** 2 - 1))}
PLANTS = {
 "rem": lambda e: sp.rem(num(e), d - 4, d), "resultant": lambda e: sp.resultant(num(e), d - 4, d),
 "compose": lambda e: sp.compose(num(e), 4, d), "div": lambda e: sp.div(num(e), d - 4, d),
 "prem": lambda e: sp.prem(num(e), d - 4, d),
 "Poly_shift_as_coeff_add": lambda e: sp.Poly(num(e), d).shift(-4).as_expr().as_coeff_add(),
 "Poly_rem_method": lambda e: sp.Poly(num(e), d).rem(sp.Poly(d - 4, d)),
 "Matrix_subs_rename": lambda e: sp.Matrix([e]).subs(d, y).subs(y, 4),
 "Matrix_xreplace_rename": lambda e: sp.Matrix([e]).xreplace({d: y}).subs(y, 4),
}
out = {}
for k, f in PLANTS.items():
    out[k] = {}
    for n, e in EXPRS.items():
        try:
            f(e); out[k][n] = "NOT_RAISED"
        except dred.PrematureD4Substitution:
            out[k][n] = "RAISED"
        except Exception as x:
            out[k][n] = "OTHER:" + type(x).__name__
e = EXPRS["mixed_pow"]
alg = {"cancel_factor": str(sp.factor(sp.cancel(e + 1 / (d + 1)))), "apart": str(sp.apart(e, d)),
       "together": str(sp.together(e)), "authorised_value": str(dred.substitute_d(e, 4, "p23 probe: authorised
  +> "))}
print(json.dumps({"plants": out, "algebra": alg}))
'''
ALGEBRA_EXPECTED = {"cancel_factor": "(d**2 + 30*d - 34)/(32*(d - 1)*(d + 1))",
                    "apart": "1/32 - 1/(64*(d + 1)) - 3/(64*(d - 1))", "authorised_value": "1/80"}   # (16-8-2
  +> )/(32*15)


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def jl(rel):
    f = ROOT / rel
    return json.loads(f.read_text()) if f.is_file() else {}


def poly_probe():
    p = subprocess.run([sys.executable, "-c", POLY_PROBE, str(ROOT)], capture_output=True, text=True, timeout=
  +> 600)
    try:
        r = json.loads(p.stdout.strip().splitlines()[-1])
    except Exception:  # noqa: BLE001
        return False, {"error": (p.stderr or p.stdout)[-1500:]}
    not_raised = {k: [n for n, v in r["plants"][k].items() if v != "RAISED"] for k in r["plants"]}
    not_raised = {k: v for k, v in not_raised.items() if v}
    alg_ok = all(r["algebra"].get(k) == v for k, v in ALGEBRA_EXPECTED.items())
    rec = {"plants": r["plants"], "not_raised": not_raised, "algebra": r["algebra"], "algebra_ok": alg_ok,
           "n_plants": len(r["plants"]), "all_plants_covered": set(r["plants"]) == set(QC.POLY_PLANTS_P23)}
    return bool(not not_raised and alg_ok and rec["all_plants_covered"]), rec
```

```
def contract_check():
    dr = CR.dred_revision(ROOT)
    cur = sha(ROOT / CR.DRED)
    pinned = QC.P23_PINNED_SHA256.get(CR.DRED)
    rec = {"dred_registry": dr, "p22_block_unchanged": CR.p22_block_unchanged(ROOT), "p23_registered":
           CR.p23_registered(ROOT), "dred_sha256": cur, "dred_pinned": pinned, "contract_revision_p23": QC.CON
  +> TRACT_REVISION,
           "other_contract_files_changed": [p.relative_to(ROOT).as_posix() for p in (ROOT / "code" / "p5_entry
  +> ").glob("*.py")
                                           if p.name != "dred.py" and p.relative_to(ROOT).as_posix() in
                                           jl(QC.P22_SOURCE_LIST).get("files", {}) and
                                           sha(p) != jl(QC.P22_SOURCE_LIST)["files"][p.relative_to(ROOT).as_p
  +> osix()]]}
    ok = bool(rec["p23_registered"] and (pinned is None or cur == pinned) and not rec["other_contract_files_ch
  +> anged"])
    return ok, rec


def p22_layer():
    r22 = jl("certificates/P22_release.json")
    base = jl(QC.P22_SOURCE_LIST).get("files", {})
    conv_same = base.get("code/p5_calc/p22_conventions.py") == sha(ROOT / "code/p5_calc/p22_conventions.py")
    con_ok, con = R22.contract_check()
    g_ok, g = R22.guard_probe()
    ev_ok, ev, _ = R22.evanescent_check()
    earned = {k: r22.get(k) == "earned" for k in Q22.EARNABLE_P22}
    rec = {"publication": r22.get("publication"), "earned": earned, "p22_conventions_byte_identical": conv_sam
  +> e,
           "live_contract_check": con_ok, "live_guard_probe": g_ok, "guard_not_raised": g.get("not_raised"),
           "live_evanescent_check": ev_ok, "hard_stops_hold": r22.get("hard_stops_hold"),
           "bulk_symmetry_restored": r22.get("bulk_symmetry_restored"), "O1_closure": r22.get("O1_closure")}
    ok = bool(r22.get("publication") == "accepted" and all(earned.values()) and conv_same and con_ok and g_ok
  +> and ev_ok
              and r22.get("hard_stops_hold") is True and r22.get("bulk_symmetry_restored") is False
              and r22.get("O1_closure") == "NOT_CLOSED")
    return ok, rec, r22


def ev_values():
    cur = jl("certificates/P22_evanescent.json").get("EV_CT", {})
    arch_f = ROOT / ARCH_P22 / "certificates" / "P22_evanescent.json"
    arch_ok = arch_f.is_file() and sha(arch_f) == QC.P22_ARCHIVED_RECORDS[f"{ARCH_P22}/certificates/P22_evanes
  +> cent.json"]
    arch = json.loads(arch_f.read_text()).get("EV_CT", {}) if arch_ok else {}

    def get(c, k):
        if k == "calibration_rho_from_4D_pair":
            return c.get("a_closure", {}).get("c.b", {}).get(k)
        return c.get(k)
    rec = {}
    for k, v in QC.EV_EXPECTED.items():
        rec[k] = {"expected": v, "current": get(cur, k), "archived_p22": get(arch, k),
                  "ok": get(cur, k) == v and get(arch, k) == v}
    return bool(arch_ok and all(x["ok"] for x in rec.values())), {"archived_cert_pinned": arch_ok, "values": r
  +> ec}


def o1_scope():
    c = jl(f"certificates/{QC.CERT_PREFIX}_O1_scope.json")
    led = ROOT / QC.O1_LEDGER
    led_ok = led.is_file() and sha(led) == QC.O1_LEDGER_SHA256
    p6 = c.get("P6_entry_kappa3_three_point", {})
    rec = {"present": bool(c), "ledger_sha_pinned": led_ok,
           "cert_ledger_sha": c.get("O1_source", {}).get("sha256") == QC.O1_LEDGER_SHA256,
           "ledger_outcome_NOT_CLOSED": c.get("O1_source", {}).get("outcome") == "NOT_CLOSED",
           "vertex_rule_holds": c.get("vertex_rule_holds") is True,
           "one_loop_kappa_equals_E": c.get("one_loop_kappa_equals_E") is True,
           "frozen_E2_match": c.get("frozen_E2_match", {}).get("equal") is True,
           "enlarge_rule_not_triggered": c.get("identity_at_kappa2", {}).get("enlarge_rule_triggered") is Fals
  +> e,
           "O1_not_closable_in_XVI_P": c.get("O1_closable_in_XVI_P") is False,
           "E1_count": len(c.get("graphs_E1", [])) == QC.ONE_POINT_E1_GRAPHS,
           "E3_count": p6.get("n_graphs") == QC.P6_ENTRY_E3_GRAPHS,
           "E3_by_external": p6.get("by_external") == QC.P6_ENTRY_E3_BY_EXTERNAL,
           "mutations_detected": c.get("mutations_detected") is True, "pass": c.get("pass") is True}
    return bool(all(rec.values())), rec, c


def title_qualifier():
    first = (ROOT / "README.md").read_text().splitlines()[0]
```

```
    pdf_titles = (ROOT / "tools" / "build.py").read_text().count("for the two-point functions with QdS externa
  +> l legs")
    ok = "two-point" in first and "QdS" in first and pdf_titles >= 2          # README title + science/combine
  +> d PDF titles
    return ok, {"title_line": first, "pdf_title_lines_with_qualifier": pdf_titles, "required": QC.TITLE_QUALIF
  +> IER}


def p22_independent():
    f = ROOT / ARCH_P22_INDEP
    if not f.is_file():
        return False, {"report": ARCH_P22_INDEP, "present": False, "pins_ok": False}
    rep = json.loads(f.read_text())
    rec = {"report": ARCH_P22_INDEP, "report_sha256_pinned": sha(f) == QC.P22_INDEPENDENT_REPORT_SHA256,
           "pass": rep.get("pass"), "independent_rebuild_passed": rep.get("independent_rebuild_passed"),
           "tree_matches_p22": rep.get("source_tree_sha256") == QC.P22_SOURCE_TREE_SHA256,
           "zip_matches_p22": rep.get("zip_sha256") == QC.P22_SOURCE_ZIP_SHA256,
           "mismatches": len(rep.get("mismatches", [])),
           "findings": [x.get("id") for x in rep.get("findings", []) if isinstance(x, dict)],
           "caveat": "same box (disclosed)",
           "NOT_inherited_by_p23": "bound to the p22 tree/zip; the p23 tree has its own (pending) independent
  +> flag"}
    rec["archived_p22_records_pinned"] = {p: (ROOT / p).is_file() and sha(ROOT / p) == h
                                          for p, h in QC.P22_ARCHIVED_RECORDS.items()}
    rec["pins_ok"] = bool(rec["report_sha256_pinned"] and all(rec["archived_p22_records_pinned"].values()))
    ok = bool(rec["pins_ok"] and rep.get("pass") is True and rep.get("independent_rebuild_passed") is True
              and rec["tree_matches_p22"] and rec["zip_matches_p22"] and rec["mismatches"] == 0)
    return ok, rec


def change_log():
    f = ROOT / QC.P22_SOURCE_LIST
    base_ok = f.is_file() and sha(f) == QC.P22_ARCHIVED_RECORDS[QC.P22_SOURCE_LIST]
    old = json.loads(f.read_text())["files"] if base_ok else {}
    new = {r: sha(ROOT / r) for r in SM.source_files(ROOT)}
    added = sorted(set(new) - set(old))
    removed = sorted(set(old) - set(new))
    modified = sorted(k for k in set(new) & set(old) if new[k] != old[k])

    def disclosed(x):
        return (x in QC.DISCLOSED_REVISIONS_P23 or x in QC.GENERATED_EXEMPT_P23
                or (x in added and x.startswith(QC.DISCLOSED_PREFIXES_ADDED_P23)))
    undisclosed = [x for x in added + removed + modified if not disclosed(x)]
    pin_bad = {k: {"pinned": h, "current": new.get(k)} for k, h in QC.P23_PINNED_SHA256.items() if new.get(k)
  +> != h}
    unpinned = sorted(k for k in QC.DISCLOSED_REVISIONS_P23 if k not in QC.P23_PINNED_SHA256
                      and k not in ("code/p5_calc/p23_conventions.py", "README.md", "REBUILD_INSTRUCTIONS.md")
  +> )
    listed_unchanged = sorted(k for k in QC.DISCLOSED_REVISIONS_P23 if k not in added + modified + removed)
    registry_consistent = sorted(set(CR.P23_REVISED_FILES) | set(CR.P23_ADDED_FILES)) == sorted(
        k for k in QC.DISCLOSED_REVISIONS_P23 if not k.startswith(CR.P23_ADDED_PREFIXES))
    return {"base": f"{QC.P22_SOURCE_LIST} (pinned; files of the p22 zip sha256 {QC.P22_SOURCE_ZIP_SHA256})",
            "base_pinned_ok": base_ok, "added": added, "removed": removed, "modified": modified,
            "undisclosed_changes": undisclosed, "pinned_sha_mismatches": pin_bad, "disclosed_but_unpinned": un
  +> pinned,
            "disclosed_but_unchanged": listed_unchanged, "registry_lists_match_disclosure": registry_consisten
  +> t,
            "counts": {"added": len(added), "removed": len(removed), "modified": len(modified)}}


def _main():
    con_ok, con = contract_check()
    l22_ok, l22, r22 = p22_layer()
    ev_ok, ev = ev_values()
    pp_ok, pp = poly_probe()
    o1_ok, o1, o1c = o1_scope()
    tq_ok, tq = title_qualifier()
    p22i_ok, p22i = p22_independent()
    hard = dict(QC.HARD_STOPS_P23)
    hard_ok = bool(all(r22.get("hard_stops", {}).get(k) == v for k, v in hard.items())
                   and r22.get("bulk_symmetry_restored") is False)
    cl = change_log()
    execution_pass = bool(con_ok and l22_ok and ev_ok and pp_ok and o1_ok and tq_ok and hard_ok
                          and cl["base_pinned_ok"] and not cl["undisclosed_changes"] and not cl["pinned_sha_mi
  +> smatches"]
                          and not cl["disclosed_but_unpinned"] and not cl["disclosed_but_unchanged"]
                          and cl["registry_lists_match_disclosure"] and p22i.get("pins_ok") is True)
    publish = execution_pass
    author_ok, author_rec = rebuild_flag(QC.AUTHOR_REPORT, pending_if_absent=False)
    indep_ok, indep_rec = rebuild_flag(QC.INDEPENDENT_REPORT_CURRENT, pending_if_absent=True)
    earned = {k: ("earned" if (publish and r22.get(k) == "earned") else "not_earned") for k in Q22.EARNABLE_P2
```

```
+> 2}
  earned.update({"O1_requirement_recorded": "earned" if publish and o1["ledger_sha_pinned"] else "not_earned
+> ",
                 "one_loop_power_counting_verified": "earned" if publish and o1_ok else "not_earned",
                 "P6_entry_inventory": "earned" if publish and o1["E3_count"] and o1["E3_by_external"] else
+> "not_earned",
                 "d_poly_domain_refusal_verified": "earned" if publish and pp_ok else "not_earned",
                 "EV_values_verified": "earned" if publish and ev_ok else "not_earned"})
  ms = json.loads(json.dumps(r22.get("milestones", {})))
  cert = dict(r22)            # the p22 record carried forward (lower-layer keys), p23 keys added / overwrit
+> ten below
  p6 = o1c.get("P6_entry_kappa3_three_point", {})
  cert.update({
      "landmark": "XVI-P", "revision": QC.REVISION, "contract_revision": QC.CONTRACT_REVISION, "date": QC.DA
+> TE,
      "version_label": QC.VERSION_LABEL,
      "deliverable": ("p23 (stage release on the O.1 direction): O.1 exact requirement and scope; one-loop p
+> ower "
                      "counting and enlarge-rule check; P.6 ENTRY inventory (kappa^3 three-point, topology l
+> evel, NOT "
                      "evaluated); polynomial-domain refusal for d (p23 contract revision); EV value check;
+> title "
                      "qualifier"),
      "execution_pass": execution_pass,
      "claim_status": (("p23. O.1 requires the one-loop kappa^2 bulk two-point sector on S_inv to close unde
+> r a VERIFIED "
                        "retained identity with O.2 (classical kappa^2 master incl. grav ell_4), O.4, O.5, O
+> .6, O.7 all "
                        "earned. Exact power counting: every vertex is kappa^(legs-2), so one-loop graphs ar
+> e kappa^E; "
                        "the kappa^2 identity involves only E=2 and E=1, the enumeration reproduces the froz
+> en kappa^2 "
                        "inventory exactly, and the enlarge rule is NOT triggered (no one-loop three-point s
+> ector at "
                        "kappa^2). Three-point first appears at kappa^3: P.6 ENTRY inventory = 15 one-loop g
+> raphs, NOT "
                        "evaluated. O.1 is not closable inside XVI-P (O.2 and the bulk ST4b identity are XVI
+> -O/XVI-N "
                        "items, untouched). " + QC.TITLE_QUALIFIER + ". Bulk symmetry NOT restored/claimed;
+> O.1 "
                        "NOT_CLOSED.") if publish else "p23: not accepted"),
      "O1": {"requirement": o1c.get("O1_requirement"), "items_scope": o1c.get("O1_items_scope"),
             "closable_in_XVI_P": False, "reason": o1c.get("O1_closable_reason"), "ledger": QC.O1_LEDGER,
             "ledger_sha256": QC.O1_LEDGER_SHA256},
      "power_counting": {"vertex_rule": o1c.get("vertex_rule_holds"), "kappa_equals_E": o1c.get("one_loop_ka
+> ppa_equals_E"),
                         "identity_at_kappa2": o1c.get("identity_at_kappa2"), "frozen_E2_match": o1c.get("fr
+> ozen_E2_match")},
      "P6_entry": {"sector": "one loop, kappa^3, three-point (incl. antifield K,c,h and cs,c,c / phis,c,phi)
+> ",
                   "n_graphs": p6.get("n_graphs"), "by_external": p6.get("by_external"),
                   "status": "inventory only (topology level); amplitude rules, counterterm domain, evaluati
+> on, "
                             "defect and closure NOT done", "evaluated": False},
      "three_point_defect": "not computed (no graph evaluated)",
      "P5_calculation_status": QC.P5_STATUS_P23, "title_qualifier": tq,
      "d_poly_domain_refusal": {"ok": pp_ok, **{k: pp.get(k) for k in ("n_plants", "not_raised", "algebra",
+> "algebra_ok")}},
      "EV_values": ev, "contract": con, "p22_layer": l22, "o1_scope_check": o1, "change_log_vs_p22": cl,
      "open_pins": list(QC.OPEN_PINS_P23), "release_claim_accepted": publish,
      "publication": "accepted" if publish else "REJECTED",
      **earned,
      "disclosed_revisions_p23": QC.DISCLOSED_REVISIONS_P23,
      "hard_stops": hard, "hard_stops_hold": hard_ok, "milestones": ms,
      "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
      "independent_rebuild_passed_p22": p22i_ok, "p22_independent_rebuild": p22i,
      "rebuild_flag_scope": ("author/independent flags refer to the CURRENT (p23) tree only (independent: pe
+> nding until "
                             f"a reviewer report exists); independent_rebuild_passed_p22 refers to the p22 t
+> ree "
                             f"{QC.P22_SOURCE_TREE_SHA256} / zip {QC.P22_SOURCE_ZIP_SHA256} (same-box caveat
+> ), not "
                             "inherited"),
      "reproducibility": {"contract": "REBUILD_INSTRUCTIONS.md", "entrypoint": "python3 tools/rebuild_from_s
+> ource.py",
                          "source_zip": SM.ZIP_NAME, "reference_values": SM.REFERENCE_FILE,
                          "author": author_rec, "independent": indep_rec, "p22_archive": ARCH_P22,
                          "kind": "package evidence (XVI-O conv:package); not a mathematical certificate"},
      "XVI_P_registered": False, "symmetry_restored": False, "bulk_symmetry_restored": False,
      "O1_closure": "NOT_CLOSED", "insertion_status": "Frozen_negative_obstruction_preserved",
      "XV_B_opened": False, "XIV_J_opened": False, "Route_B_discharged": False, "u_R_c_delta_added": False,
```

```
        "inputs_content_sha256_ignoring_timings_and_run_specific_keys": {
            n: certio.file_content_sha(ROOT / "certificates" / n, QC.RUN_SPECIFIC_KEYS) for n in
            ("P22_release.json", f"{QC.CERT_PREFIX}_O1_scope.json") if (ROOT / "certificates" / n).is_file()},
    })
    if publish:
        st = ms.setdefault("status", {})
        st["P.6_entry"] = "inventory (one loop kappa^3 three-point, 15 graphs, topology level); not evaluated"
        st["O.1"] = "NOT_CLOSED (requirement recorded; not closable inside XVI-P)"
    cdir = ROOT / "certificates"
    if publish:
        txt = certio.dumps(cert)
        for s_ in STATUS_FILES:
            (ROOT / s_).write_text(txt)
        (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").unlink(missing_ok=True)
        (cdir / f"{QC.CERT_PREFIX}_release.json").write_text(txt)
        certio.write(ROOT / "ledgers" / f"{QC.CERT_PREFIX}_milestones.json", {
            "landmark": "XVI-P", "revision": QC.REVISION, "ledger": "milestones (authoritative definitions + s
  +> tatus)",
            **ms, "author_rebuild_passed": author_ok, "independent_rebuild_passed": indep_ok,
            "independent_rebuild_passed_p22": p22i_ok, "hard_stops": hard,
            "flags": {**earned, "quantum_inputs_evaluated": cert.get("quantum_inputs_evaluated"),
                      "bulk_symmetry_restored": False}})
    else:
        _reject({"contract_ok": con_ok, "contract": con, "p22_layer_ok": l22_ok, "p22_layer": l22, "EV_values_
  +> ok": ev_ok,
                 "EV_values": ev, "poly_probe_ok": pp_ok, "poly_not_raised": pp.get("not_raised"),
                 "poly_error": pp.get("error"), "O1_scope_ok": o1_ok, "O1_scope": o1, "title_qualifier_ok": tq
  +> _ok,
                 "p22_independent_pins_ok": p22i.get("pins_ok"), "hard_stops_hold": hard_ok,
                 "change_log_base_ok": cl["base_pinned_ok"], "undisclosed_changes": cl["undisclosed_changes"],
                 "pinned_sha_mismatches": cl["pinned_sha_mismatches"], "disclosed_but_unpinned": cl["disclosed
  +> _but_unpinned"],
                 "disclosed_but_unchanged": cl["disclosed_but_unchanged"],
                 "registry_lists_match_disclosure": cl["registry_lists_match_disclosure"]})
    print(json.dumps({"revision": QC.REVISION, "execution_pass": execution_pass, "publication": cert["publicat
  +> ion"],
                      "contract": con_ok, "p22_layer": l22_ok, "EV_values": ev_ok, "poly_probe": pp_ok, "O1_sc
  +> ope": o1_ok,
                      "title": tq_ok, "hard_ok": hard_ok, **earned, "author": author_ok, "independent_p23": in
  +> dep_ok,
                      "independent_p22": p22i_ok, "undisclosed": cl["undisclosed_changes"],
                      "pin_mismatch": list(cl["pinned_sha_mismatches"]), "unpinned": cl["disclosed_but_unpinne
  +> d"],
                      "unchanged": cl["disclosed_but_unchanged"], "registry_match": cl["registry_lists_match_d
  +> isclosure"],
                      "change_log_counts": cl["counts"]}, indent=1))
    return 0 if publish else 1


def _reject(detail):
    cdir = ROOT / "certificates"
    (cdir / f"{QC.CERT_PREFIX}_release.json").unlink(missing_ok=True)
    for s_ in STATUS_FILES:                      # a reject must not leave a stale RELEASE_STATUS.json
        (ROOT / s_).unlink(missing_ok=True)
    (cdir / f"{QC.CERT_PREFIX}_release_rejected.json").write_text(json.dumps(
        {"landmark": "XVI-P", "revision": QC.REVISION, "publication": "REJECTED", **detail}, indent=1, default
  +> =str) + "\n")


def main():
    try:
        return _main()
    except Exception as exc:  # noqa: BLE001   fail closed
        _reject({"error": f"{type(exc).__name__}: {exc}"})
        print(f"release_p23 error: {type(exc).__name__}: {exc}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

166 Source: tests/test__p23.py

[listing 133/133] tests/test_p23.py

```
"""XVI-P p23 tests: O.1 requirement record, exact one-loop power counting / enlarge rule, P.6 entry inventory,
polynomial-domain refusal (p23 contract revision), EV value check, title qualifier, p23 gate (fail closed)."""
from __future__ import annotations
```

```
import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

import pytest
import sympy as sp

ROOT = Path(__file__).resolve().parents[1]
for p in ("code/p5_calc", "code/p5_entry", "code/classical", "tools"):
    sys.path.insert(0, str(ROOT / p))
import p23_conventions as QC  # noqa: E402
import contract_registry as CR  # noqa: E402
import dred  # noqa: E402
import power_counting as PC  # noqa: E402

CERT = ROOT / "certificates"
d = dred.d
X, y = sp.Symbol("X"), sp.Symbol("y")


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def rs():
    return json.loads((ROOT / "RELEASE_STATUS.json").read_text())


def o1c():
    return json.loads((CERT / "P23_O1_scope.json").read_text())


# ============================================================== O.1 requirement
def test_o1_ledger_copy_pinned_and_read_only_source_recorded():
    assert sha(ROOT / QC.O1_LEDGER) == QC.O1_LEDGER_SHA256
    c = o1c()
    assert c["O1_source"]["sha256"] == QC.O1_LEDGER_SHA256 and c["O1_source"]["outcome"] == "NOT_CLOSED"
    assert c["O1_source"]["gate"] == "O.1" and len(c["O1_source"]["checklist"]) == 6


def test_o1_requirement_and_scope_honest():
    c = o1c()
    assert c["O1_closable_in_XVI_P"] is False
    sc = c["O1_items_scope"]
    assert sc["S_inv^{[2]} classical consistency (O.2)"]["scope"] == "OUT OF SCOPE"
    for w in ("VERIFIED", "O.2", "ell_4", "enlarged"):
        assert w in c["O1_requirement"]


# ============================================================== exact power counting (live recomputation)
def test_vertex_rule_and_kappa_equals_E_live():
    import feynman as F
    V = F.vertices()
    assert all(PC.vertex_power_ok(V).values())
    for E in (1, 2, 3):
        assert all(g["kappa_power"] == E for g in PC.one_loop_graphs(V, E))


def test_enumeration_reproduces_frozen_kappa2_inventory_live():
    import feynman as F
    fm = PC.frozen_match(PC.one_loop_graphs(F.vertices(), 2))
    assert fm["equal"] and fm["enumerated"] == fm["frozen"] == 9


def test_frozen_list_matches_inventory_records():
    f = CERT / "diagram_inventory.json"           # inherited (author tree); absent from the source-only tree
    if f.is_file():
        inv = json.loads(f.read_text())["diagrams"]
        loop = {x["id"] for x in inv if x.get("kappa_power") == 2 and x.get("hbar_power") == 1}
        assert loop == set(PC.FROZEN_E2) - {"D13_Kc_ghost_self_energy"}
    led = (ROOT / "ledgers" / "P13_graph_inventory.json").read_text()   # built in every tree
    for k in ("D01_hh_bubble_cubic", "D02_hh_tadpole_quartic", "D03_hh_ghost_loop", "D11_hh_scalar_loop",
              "D12_hh_scalar_tadpole"):
        assert k in PC.FROZEN_E2 and f'"{k}"' in led
    assert '"D13_Kc_ghost_graviton_loop"' in led
    assert sum(f'"D14{s}_h_onepoint_' in led for s in "abc") == QC.ONE_POINT_E1_GRAPHS
```

```
def test_enlarge_rule_not_triggered_and_P6_entry_counts():
    c = o1c()
    assert c["identity_at_kappa2"]["enlarge_rule_triggered"] is False
    assert len(c["graphs_E1"]) == QC.ONE_POINT_E1_GRAPHS
    p6 = c["P6_entry_kappa3_three_point"]
    assert p6["n_graphs"] == QC.P6_ENTRY_E3_GRAPHS and p6["by_external"] == QC.P6_ENTRY_E3_BY_EXTERNAL
    assert "NOT evaluated" in p6["status"]


def test_power_counting_mutations_detected():
    c = o1c()
    assert c["mutations_detected"] is True and all(c["mutations"].values()) and len(c["mutations"]) == 4


# ============================================================ polynomial-domain refusal (p23 contract rev
  +> ision)
num = lambda e: sp.numer(sp.together(e))  # noqa: E731
EXPRS = {"linear_and_pow": d * X + 1 / (d - 1), "power_base_only": 1 / (d ** 2 - 1), "d_alone": d,
         "mixed_pow": (d ** 2 - 2 * d - 2) / (32 * (d ** 2 - 1))}
PLANTS = {
    "rem": lambda e: sp.rem(num(e), d - 4, d), "resultant": lambda e: sp.resultant(num(e), d - 4, d),
    "compose": lambda e: sp.compose(num(e), 4, d), "div": lambda e: sp.div(num(e), d - 4, d),
    "prem": lambda e: sp.prem(num(e), d - 4, d),
    "Poly_shift_as_coeff_add": lambda e: sp.Poly(num(e), d).shift(-4).as_expr().as_coeff_add(),
    "Poly_rem_method": lambda e: sp.Poly(num(e), d).rem(sp.Poly(d - 4, d)),
    "Matrix_subs_rename": lambda e: sp.Matrix([e]).subs(d, y).subs(y, 4),
    "Matrix_xreplace_rename": lambda e: sp.Matrix([e]).xreplace({d: y}).subs(y, 4),
}


def test_poly_plant_list_matches_conventions():
    assert set(PLANTS) == set(QC.POLY_PLANTS_P23)


@pytest.mark.parametrize("expr", sorted(EXPRS))
@pytest.mark.parametrize("plant", sorted(PLANTS))
def test_poly_refusal_every_plant_raises(plant, expr):
    with pytest.raises(dred.PrematureD4Substitution):
        PLANTS[plant](EXPRS[expr])


def test_poly_refusal_keeps_rational_algebra_and_authorised_path():
    e = EXPRS["mixed_pow"]
    assert sp.cancel(sp.apart(e, d) - e) == 0
    assert sp.cancel(sp.factor(sp.cancel(e + 1 / (d + 1))) - (d ** 2 + 30 * d - 34) / (32 * (d - 1) * (d + 1))
  +> ) == 0
    assert dred.substitute_d(e, 4, "test_p23: authorised") == sp.Rational(1, 80)
    assert sp.rem(X ** 3 + 1, X - 1, X) == 2                    # d-free polynomial algebra unaffected


def test_contract_state_registered():
    r = CR.dred_revision(ROOT)
    assert r["ok"] and r["registered_blocks"] == ["p22 runtime d guard", "p23 polynomial-domain refusal"]
    assert CR.p22_block_unchanged(ROOT) and CR.p23_registered(ROOT)
    assert sha(ROOT / CR.DRED) == QC.P23_PINNED_SHA256.get(CR.DRED, sha(ROOT / CR.DRED))


# ============================================================ EV values, title qualifier, status
def test_ev_values_checked_against_pinned_and_archived():
    import release_p23 as R
    ok, rec = R.ev_values()
    assert ok and rec["archived_cert_pinned"]
    assert rec["values"]["calibration_rho_from_4D_pair"]["current"] == "1/2"
    assert rec["values"]["coefficient_Kc_E2"]["current"] == "-1/16"


def test_title_qualifier_readme_and_pdf_titles():
    first = (ROOT / "README.md").read_text().splitlines()[0]
    assert "two-point" in first and "QdS" in first
    assert (ROOT / "tools" / "build.py").read_text().count("for the two-point functions with QdS external legs
  +> ") >= 2


def test_p23_release_status_and_flags():
    s = rs()
    assert s["revision"] == "p23" and s["publication"] == "accepted" and s["execution_pass"] is True
    for k in QC.EARNABLE_P23:
        assert s[k] == "earned", k
    assert s["bulk_symmetry_restored"] is False and s["symmetry_restored"] is False and s["O1_closure"] == "NO
  +> T_CLOSED"
```

```
    assert s["P6_entry"]["evaluated"] is False and s["three_point_defect"].startswith("not computed")
    assert s["O1"]["closable_in_XVI_P"] is False
    assert s["independent_rebuild_passed"] is not True and s["independent_rebuild_passed_p22"] is True
    assert s["hard_stops"] == QC.HARD_STOPS_P23 and s["hard_stops_hold"] is True


def test_disclosure_complete_exact_and_pinned():
    cl = rs()["change_log_vs_p22"]
    assert cl["undisclosed_changes"] == [] and cl["pinned_sha_mismatches"] == {}
    assert cl["disclosed_but_unpinned"] == [] and cl["disclosed_but_unchanged"] == []
    assert cl["registry_lists_match_disclosure"] is True


def test_version_label_p23():
    import build as B
    assert QC.VERSION_LABEL == "XVI-P p23" and B.QC.VERSION_LABEL == QC.VERSION_LABEL
    assert "certificates/P23_O1_scope.json" in B.VALUE_BLOCKS


# ============================================================ p23 gate mutations (fail closed; no stale s
  +> tatus)
def _copy():
    d_ = Path(tempfile.mkdtemp(prefix="p23gate_"))
    shutil.copytree(ROOT, d_ / "XVI-P", ignore=shutil.ignore_patterns("__pycache__", "dist", "stale", "build")
  +> ,
                    symlinks=True)
    return d_ / "XVI-P"


def _edit_json(d_, rel, fn):
    f = d_ / rel
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c, indent=1) + "\n")


def _rho_edited(d_):
    _edit_json(d_, "certificates/P22_evanescent.json",
               lambda c: c["EV_CT"]["a_closure"]["c.b"].__setitem__("calibration_rho_from_4D_pair", "1"))


def _E2_sign_flipped(d_):
    _edit_json(d_, "certificates/P22_evanescent.json", lambda c: c["EV_CT"].__setitem__("coefficient_Kc_E2", "
  +> 1/16"))


def _poly_refusal_removed(d_):
    f = d_ / CR.DRED
    f.write_text(f.read_text().replace("\n_install_poly_refusal()\n", "\n"))


def _p22_block_edited(d_):
    f = d_ / CR.DRED
    f.write_text(f.read_text().replace('_deny(f"replace({query}, ...)")', '_deny(f"replace({query},...)")'))


def _o1_ledger_edited(d_):
    _edit_json(d_, QC.O1_LEDGER, lambda c: c["closure_check"].__setitem__("outcome", "CLOSED"))


def _p6_count_edited(d_):
    _edit_json(d_, "certificates/P23_O1_scope.json",
               lambda c: c["P6_entry_kappa3_three_point"].__setitem__("n_graphs", 14))


def _enlarge_flag_edited(d_):
    _edit_json(d_, "certificates/P23_O1_scope.json",
               lambda c: c["identity_at_kappa2"].__setitem__("enlarge_rule_triggered", True))


def _title_qualifier_removed(d_):
    f = d_ / "README.md"
    lines = f.read_text().splitlines(True)
    lines[0] = lines[0].replace("two-point", "2pt")
    f.write_text("".join(lines))


def _undisclosed_note(d_):
    f = d_ / "manuscript" / "notes" / "P18_D11.tex"
    f.write_text(f.read_text() + "\n% undisclosed\n")
```

```
def _pinned_tool_edited(d_):
    f = d_ / "tools" / "source_manifest.py"
    f.write_text(f.read_text() + "\n# undisclosed\n")


def _p22_independent_tampered(d_):
    f = d_ / "archival" / "p22" / "reproducibility" / "independent_rebuild_report_p22.json"
    f.write_text(f.read_text().replace('"pass": true', '"pass": false', 1))


MUTS = [_rho_edited, _E2_sign_flipped, _poly_refusal_removed, _p22_block_edited, _o1_ledger_edited, _p6_count_
  +> edited,
        _enlarge_flag_edited, _title_qualifier_removed, _undisclosed_note, _pinned_tool_edited,
        _p22_independent_tampered]


def _run(d_):
    return subprocess.run([sys.executable, "code/p5_calc/release_p23.py"], cwd=d_, capture_output=True, text=T
  +> rue,
                          timeout=1200)


@pytest.mark.parametrize("mut", MUTS, ids=lambda f: f.__name__)
def test_p23_gate_mutation_rejects_without_stale_status(mut):
    d_ = _copy()
    try:
        assert (d_ / "RELEASE_STATUS.json").exists()
        mut(d_)
        r = _run(d_)
        assert r.returncode == 1, r.stdout[-1500:]
        assert not (d_ / "certificates" / "P23_release.json").exists()
        assert (d_ / "certificates" / "P23_release_rejected.json").exists()
        assert not (d_ / "RELEASE_STATUS.json").exists() and not (d_ / "certificates" / "RELEASE_STATUS.json")
  +> .exists()
    finally:
        shutil.rmtree(d_.parent, ignore_errors=True)


def test_p23_gate_accepts_unmodified_copy():
    d_ = _copy()
    try:
        r = _run(d_)
        assert r.returncode == 0, r.stdout[-1500:]
        s = json.loads((d_ / "RELEASE_STATUS.json").read_text())
        assert s["revision"] == "p23" and s["O1_closure"] == "NOT_CLOSED" and s["bulk_symmetry_restored"] is F
  +> alse
    finally:
        shutil.rmtree(d_.parent, ignore_errors=True)
```

167 Archival material (p22): evanescent M2/CT and runtime d guard, superseded by p23

Archival.

The p22 release (`archival/p22/`) is superseded by p23. Its independent rebuild PASSED (same box; report 023e5b83…), bound to tree 1bc6edff… and zip ade2fa5c…; p23 records this as independent_rebuild_passed_p22 and does not inherit it. Findings (polynomial-algebra bypass of the d guard; EV values unchecked by the gate; title-line qualifier) are addressed in p23.

168 Archival material (p21): small release, superseded by p22

Archival.

The p21 release (`archival/p21/`) is superseded by p22. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p21 tree b835353f... and zip a71d6b52...; p22 records this as independent_rebuild_passed_p21 and does not inherit it. Its findings (gate prefix exemptions; eight scan misses and the power-base subs hole of the d guard; stale OPEN text in the p20 section) are addressed in p22.

```
# archival/p21 -- the p21 release (small release: D11 absolute pin, gate/scan/disclosure fixes; superseded by
  +> p22)

p21 was released on 2026-10-09. Source-only zip `XVI-P-p21-source-only.zip`: sha256
a71d6b5203bc3914833db9fd13b9d186bef93d1ae3007b2e4c99fdfce11e0678, 183 files. Source tree
b835353ffe16a8ff159e2916fe3efdf2b9d89a4d512e80e9e6da587cecd0fe5e. Reference values reference_values_p21.json
96ae946b.... Author rebuild passed (report a5f886ad...).

INDEPENDENT REBUILD OF p21: PASSED, with findings (no mismatches). Report
`reproducibility/independent_rebuild_report_p21.json` (archived copy byte-identical, sha256
8a582b332cd242689c54b1f962dc88270b9592040c0ba432576a505f0e36a60d), bound to tree b835353f... / zip a71d6b52...
  +> .
SAME-BOX caveat: it ran on the same machine as the author build (fresh directory, fresh venv, from the source-
  +> only
zip), NOT on a second machine. p22 records independent_rebuild_passed_p21 = true; the p22 tree does NOT inheri
  +> t it.

Reviewer findings, addressed in p22 (which also carries the user-approved bigger fronts and a DISCLOSED contra
  +> ct revision):
- p21 gate disclosure prefix exemptions (low): tools/ and manuscript/ edits and added code files escaped the
  undisclosed-change check -> p22 gate: every changed/added/removed file must be listed (no prefix exemption f
  +> or
  tools/, notes or code; only generated reference values and the listed archival/p21 records are exempt).
- Scan misses (low; 8 forms) and the d-guard power-base hole (low) -> p22 contract revision: a runtime d guard
  +>  in
  code/p5_entry/dred.py (one delimited additive block; the p18 core is byte-identical) that intercepts subs/xr
  +> eplace/
  replace/evalf/Poly/coeff/collect/limit/Lambda/series/d==d from outside sympy; a live probe (30 forms x 4 exp
  +> ressions,
  incl. all reviewer forms and the power-base hole) must raise. The static allow-list scan stays as secondary.
- Stale 'Absolute pin: OPEN' text in the P20 section (info/low) -> inline [superseded in p21] marker.
- p20 gate standalone scope, absolute-pin step-1 hard-coding, reviewer rule note (info): no action needed.
```

169 Archival material (p20): correction of p19, superseded by p21

Archival.

The p20 release (`archival/p20/`) is superseded by p21. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p20 tree 2c0aa153... and zip ca32ce01...; p21 records this as independent_rebuild_passed_p20 and does not inherit it. Its findings (absolute D11 pin derivable; top-level pin status unchecked; eight scan misses; P19 note edit not listed; misleading change-log key) are addressed in p21.

```
# archival/p20 -- the p20 release (CORRECTION of p19: K1 orientation erratum, restoration redefined, D11 ratio
  +>  pin; superseded by p21)

p20 was released on 2026-10-09. Source-only zip `XVI-P-p20-source-only.zip`: sha256
ca32ce018933f12d1a623217d6a98d647817360dd12211e2e9469cf850a63bb2, 174 files. Source tree
2c0aa1533b5a64d65f9169cc510ea3f0761c9471f9601cdec6973a2aa8d45700. Reference values reference_values_p20.json
4877f5de.... Author rebuild passed (422/422; report 686e2c9c...).

INDEPENDENT REBUILD OF p20: PASSED, with findings (no mismatches). Report
`reproducibility/independent_rebuild_report_p20.json` (archived copy byte-identical, sha256
e8f358f9c4f13c76693968211b49e04d546a0bb3a4c0ab4b1a0e485f62595389), bound to tree 2c0aa153... / zip ca32ce01...
  +> .
SAME-BOX caveat: it ran on the same machine as the author build (fresh directory, fresh venv, from the source-
  +> only
zip), NOT on a second machine. p21 records independent_rebuild_passed_p20 = true; the p21 tree does NOT inheri
```

```
  +> t it.

Reviewer findings, addressed in p21 (a small release, no scope expansion):
- D11 absolute pin left OPEN but derivable (low): p21 derives the sign AND magnitude of the D11 pole from 1/2
  +> Tr ln
  under the package's declared conventions (G = Gamma^(2) from Gate 0 + G6; Wick direction from N_eps = i/(16
  +> pi^2)
  and the +1/eps bubble pole; g = eta + kappa h; eta = (+,-,-,-)): Gamma^(2)|_pole = kappa^2/(16 pi^2 eps)
  [p^4 P2/240 + p^4 POs/24], equal to the heat-kernel a_2 = C^2/120 + R^2/72 exactly. No external convention c
  +> hoice;
  external inputs are mathematics only (a_2 coefficients, heat-kernel pole formula). x2 and sign flip detected
  +> .
- Scan misses (low): eight forms passed the pattern scans; p21 adds an allow-list scan and a runtime opaque-d
  +> check.
- Gate: top-level absolute_pin_status unchecked (low) -> checked against the nested status in p21.
- p20 gate trusted the P19 layer record (info) -> p21 re-checks the p18 layer (q earned) and the inventory liv
  +> e.
- P19 note edit not in DISCLOSED_* (info) -> listed in p21_conventions.DISCLOSED_MANUSCRIPT_REVISIONS.
- /p19/change_log_vs_p18 recomputed on the current tree (info/low) -> renamed in p21.
- p19 quotient finding withdrawn by the reviewer; restoration scope wording confirmed (info).
```

170 Archival material (p19): P.5 identity step, superseded by the p20 correction

Archival.

The p19 release (`archival/p19/`) is superseded by the p20 correction release. Its independent rebuild PASSED with findings (same box, fresh directory and venv; NOT a second machine), bound to the p19 tree fcf90354... and zip da924275...; p20 records this as independent_rebuild_passed_p19 and does not inherit it. Finding K1 (the p19 claim that full closure fails with the derived factor) was an orientation bookkeeping error and is retracted (erratum in the p20 note); all findings are addressed in p20.

```
# archival/p19 -- the p19 release (P.5 identity step: projected one-loop kappa^2 ch-identity defect; supersede
  +> d by the p20 correction)

p19 was released on 2026-10-09. Source-only zip `XVI-P-p19-source-only.zip`: sha256
da9242752d2e3753d8b2ce7fbb21e7cdb28487cc7966c2ca52d1ef1376658875, 839598 bytes, 164 files. Source tree
fcf90354d746e244fa2839048118bd8d64b1859ffd3825c0bfa3e1138ee88f36. Reference values reference_values_p19.json
08f10fa8.... Author rebuild passed (320/320; report 5f0715f3...).

INDEPENDENT REBUILD OF p19: PASSED, with findings. Report `reproducibility/independent_rebuild_report_p19.json
  +> `
(archived copy byte-identical, sha256 1b01f1d8e509afedcfd73d91685806205c56e7fe57a500b3a0bf008dd2811f45), bound
  +>  to
tree fcf90354... / zip da924275.... SAME-BOX caveat: it ran on the same machine as the author build (fresh dir
  +> ectory,
fresh venv, from the source-only zip), NOT on a second machine. p20 records independent_rebuild_passed_p19 = t
  +> rue; the
p20 tree does NOT inherit it.

Reviewer findings, all corrected in p20 (a correction release, no scope expansion):
- K1 (medium): the p19 claim "full sigma_0 closure of the MS-bar counterterm fails with the derived factor +1,
  +>  needs -1,
  unresolved" was WRONG. It was a Grassmann-orientation bookkeeping error: the P15 Sigma coordinates are (cb,c
  +> ), the
  route-G slots (c,cb). Corrected: the derived +1 closes the complete counterterm in h.c, c.b and c.cbs; Sigma
  +>  is
  (+3/8, -1/4) in (c,cb) coordinates. The p19 text is retracted (erratum in manuscript/notes/P20_correction.te
  +> x).
- Quotient c.cb orientation (low): clarified in p20; all coordinates are now stated in route-G labels.
- Restoration flag not independent (low/medium): REDEFINED in p20 as full sigma_0 closure of the complete MS-b
  +> ar
  S_ct, including c.cb. The p19 criterion is kept as a route-consistency control.
- Mutation criterion / blind spots (low): the p20 full-closure detector catches drop D13 and flip K.c. The p20
  +>  D11
  ratio pin (trace anomaly, spin-0 : spin-2 = 10 : 1) catches drop D11; the absolute pin stays OPEN.
- J1 wording (info): the Q(i) counts are the real dimensions under a compatible real structure.
- I2 scan misses (low): seven patterns added (still a pattern check, not a proof).
- Gate stale RELEASE_STATUS (low): a standalone reject now removes RELEASE_STATUS.json.

Contents:
```

```
- the p19 source-only zip;
- RELEASE_STATUS, RELEASE_MANIFEST, README and REBUILD_INSTRUCTIONS of p19, and the p19 science PDF;
- the p19 reference values and the p19 author and independent rebuild reports;
- the P19_* certificates as released, with the P19 milestones ledger;
- p19_source_files.json, the base of the p20 change log.

The archived P19_* certificates carry the retracted K1 text as released. The active p19 layer files in the p20
  +>  tree
are corrected.
```

171 Archival material (p18): M2 discharged, D11 evaluated, full hh slot, superseded by p19

Archival.

The p18 release (`archival/p18/`) is superseded as a release by p19; its calculation code (d11, release_p18) stays byte-identical and active as the p18 layer. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p18 tree bfab3b4b… and zip 7c7d5451…; p19 records this as independent_rebuild_passed_p18 and does not inherit it. Reviewer findings I1–I3 are addressed in p19 (I1 fail-closed gate, I2 extended scan – a pattern check, not a proof, I3 milestone wording); no contract revision.

```
# archival/p18 -- the p18 release (M2 discharged, D11 evaluated, full hh slot; superseded as a release by p19)

p18 was released on 2026-10-09. Source-only zip `XVI-P-p18-source-only.zip`: sha256
7c7d5451adb459092edc2606e1b3322909325000d47bab89c14b7f7a6d25c92f, 753894 bytes, 154 files. Source tree
bfab3b4b318fc37f2fcca209d097d27a24955e3b3d8469f5a8ab2e904ace7094 (52976da5... without reference). Reference va
  +> lues
reference_values_p18.json 8a867f9e.... Author rebuild passed (290/290; report 13da1ba2...).

INDEPENDENT REBUILD OF p18: PASSED. Report `reproducibility/independent_rebuild_report_p18.json` (archived cop
  +> y
byte-identical, sha256 893f167ef7124d981c0569ca386035aed06e0dbb0c90a2362a6ceeafd1930cb2), bound to tree bfab3b
  +> 4b... /
zip 7c7d5451.... SAME-BOX caveat: it ran on the same machine as the author build (fresh directory, fresh venv,
  +>  from
the source-only zip), NOT on a second machine. p19 records independent_rebuild_passed_p18 = true; the p19 tree
  +>  does
NOT inherit it.

Reviewer findings carried into p19:
- I1: release_p18.py published without gating on quantum_inputs_evaluated. Now release_p19.py and the build's
  publication gate fail closed: publication requires the p18 layer's quantum_inputs_evaluated == 'earned', the
  inventory recheck and projected_defect_computed. Gate mutations are in tests/test_p19.py.
- I2: static-scan pattern gaps. The tests/test_p19.py scan now flags: a Poly or as_poly result held in a varia
  +> ble
  and then called; collect(evaluate=False); Add/Mul/Pow(*x.args) rebuilds; eval/exec aliases; vars()/globals()
  +> /
  locals() subscripts; operator.attrgetter/methodcaller; Symbol/symbols/var with a run-time name that could be
  +>  'd'.
  Every Poly .terms()/.coeffs()/.monoms() site is on a reviewed list. This is still a pattern check, not a pro
  +> of.
- I3: milestone wording. P.5_calculation status now reads "evaluated; 4D sector renormalized (MS-bar, declared
  domain); evanescent regulated (poles retained, not subtracted), decoupled from the ch slots".
- I4-I7: informational; unchanged.

Bug fixes disclosed in p18 (d11.py inventory ordering; the stale rebuild criterion) are verified by the review
  +> er.

Contents:
- the p18 source-only zip;
- RELEASE_STATUS, RELEASE_MANIFEST, README and REBUILD_INSTRUCTIONS of p18;
- the p18 reference values;
- the p18 author and independent rebuild reports;
- the P18_* certificates and the P18 milestones ledger;
- the p18 per-file source list (p18_source_files.json, the base of the p19 change log);
- the p18 science PDF.
```

172 Archival material (p17): D01/D03 evanescent poles vs the ch slot, superseded by p18

Archival.

The p17 release (`archival/p17/`) is superseded as a release by p18; its calculation code (hh_slot, release_p17) stays byte-identical and active as the p17 layer. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p17 tree 2a5a6317… and zip 7f4f61df…; p18 records this as independent_rebuild_passed_p17 and does not inherit it. Reviewer findings H1–H5 are addressed in p18 (one disclosed dred.py revision, documentation only).

```
# archival/p17 -- the p17 release (D01/D03 evanescent poles vs the ch slot; superseded as a release by p18)

p17 was released on 2026-10-08: source-only zip `XVI-P-p17-source-only.zip`, sha256
7f4f61df9ce9c6483d76fa7b7b024591d6abfbae80ddc3dc0bee387245d25416, 678669 bytes, 144 files; source tree
2a5a6317f7322db910c26fb257e589b28aa017df740879194e0ac2eb505ddd92 (8a6658a8... without reference); reference va
  +> lues
reference_values_p17.json d4345c282aee89676ee7d0b077fd8fc94ebd8735cd78499550b2666a0b9103b1; author rebuild pas
  +> sed
(282/282; report bd7d8bae...).

INDEPENDENT REBUILD OF p17: PASSED, report `reproducibility/independent_rebuild_report_p17.json` (archived cop
  +> y
byte-identical, sha256 92542a2215c24b984e2220f69ac52698d6b09dbb6b860402bbc367d970c176eb), bound to tree 2a5a63
  +> 17... /
zip 7f4f61df.... SAME-BOX caveat: same machine as the author build (fresh directory, fresh venv, from the sour
  +> ce-only
zip), NOT a second machine.  p18 records independent_rebuild_passed_p17 = true; it is NOT inherited by the p18
  +>  tree.

Reviewer findings carried into p18: H1 (static-scan pattern gaps: Poly(e, d)(4), as_poly(d)(4), Horner via
all_coeffs, type(x)(*args) / x.__class__(*args), getattr indirection, aliased sympify import, sympify locals w
  +> ith a
plain Symbol('d') value, plain Symbol('d') creation; no package use) -> the tests/test_p18.py scan flags all o
  +> f them
(planted violations detected), still a pattern check, not a proof; H2 (assignment table omitted c hat / h tild
  +> e) ->
seventh assignment added (P18_hh_slot.json; the reviewer's value X_D01 = p.p^2 a~.b~ c^.p/16 = -X_D03 is repro
  +> duced);
H3 (stale comments: dred.py matches comment; PARSE_SITES_REVIEWED header) -> corrected (dred.py: documentation
  +> -only
revision, AST unchanged); H4 (curly apostrophes in typeset listings) -> straight quotes in verbatim listings
(preamble, upquote mechanism via textcomp); H5 (parse_d(srepr(e)) raises, undocumented) -> documented (fails
closed); H6 (scope: D11 unevaluated) -> p18 discharges M2 and evaluates D11.

Contents: the p17 source-only zip, RELEASE_STATUS / RELEASE_MANIFEST / README / REBUILD_INSTRUCTIONS of p17, t
  +> he
p17 reference values, the p17 author and independent rebuild reports, the P17_* certificates, the P17 mileston
  +> es
ledger, the p17 per-file source list (base of the p18 change log) and the science PDF.
```

173 Archival material (p16): E2 decoupling proof and Batch C (D01, D03), superseded by p17

Archival.

The p16 release (`archival/p16/`) is superseded as a release by p17; its calculation code (covariant route, Batch C, E2 proof, release_p16) stays byte-identical and active as the p16 layer. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p16 tree 90b4a665… and zip 56d5887d…; p17 records this as independent_rebuild_passed_p16 and does not inherit it. Reviewer findings G1–G4 and G6 are addressed in p17 (G2: one disclosed dred.py revision, the guarded loader parse_d).

```
# archival/p16 -- the p16 release (E2 evanescent decoupling proof; Batch C D01, D03; superseded as a release b
  +> y p17)
```

```
p16 was released on 2026-10-08: source-only zip `XVI-P-p16-source-only.zip`, sha256
56d5887daa7a96dcc7cf2f70c6735e802125c9b19a33ac17a3ef975e330fef57, 608444 bytes, 134 files; source tree
90b4a6659cddd341436a59d285a56eb14bdd575bb56e69faf4ba75be4d7b549e (d02d30a8... without reference); reference va
  +> lues
reference_values_p16.json a1fda1e67a4a0033014f40b8bdac57b4daacfa507588e0765bb7139587b053d9; author rebuild pas
  +> sed
(272/272; report b4f9bcf3...).

INDEPENDENT REBUILD OF p16: PASSED (272/272 entrypoint values equal; 107 tests in a separate copy; own E2
decoupling derivation; own Batch C spot-check; determinism across PYTHONHASHSEED), report
`reproducibility/independent_rebuild_report_p16.json` (archived copy byte-identical, sha256
1a8861964afe7ae4d4b38aaae5381230d8cda82610c51ef2b7919ebd6e326843), bound to tree 90b4a665... / zip 56d5887d...
  +> .
SAME-BOX caveat: same machine as the author build (fresh directory, fresh venv, from the source-only zip), NOT
  +>  a
second machine.  p17 records independent_rebuild_passed_p16 = true; it is NOT inherited by the p17 tree.

Reviewer findings carried into p17: G1 (static scan narrower than the disclosure: named-function replace, stri
  +> ng
round-trips, eval(srepr), manual tree rebuilds; no package use) -> scan extended, wording corrected; G2 (sympi
  +> fy of a
stored string gives a plain Symbol('d')) -> guarded loader dred.parse_d + assert_opaque_d (disclosed contract
revision); G3 (docstring 'expr.match(d)' wording; Wild-pattern match not refused) -> wording corrected; G4 (ch
  +> ange
log omitted the added reference file) -> fixed; G5 (disclosed test_p15 relaxation) -> unchanged; G6 (overall i
phase of D01/D03 vs real D13) -> convention documented; G7 (hh-basis coordinates not recomputed) -> p17 recomp
  +> utes
the coordinates explicitly (certificates/P17_hh_slot.json).

Contents: the p16 source-only zip, RELEASE_STATUS / RELEASE_MANIFEST / README / REBUILD_INSTRUCTIONS of p16, t
  +> he
p16 reference values, the p16 author and independent rebuild reports, the P16_* certificates, the P16 mileston
  +> es
ledger, the p16 per-file source list (base of the p17 change log) and the science PDF.
```

174 Archival material (p15): first P.5 calculation (Batch A, D13), superseded by p16

Archival.

The p15 release (`archival/p15/`) is superseded as a release by p16; its calculation code (integral library, Batch A, D13, release_p15) stays byte-identical and active as the p15 layer. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p15 tree 79f53752… and zip 54325c1f…; p16 records this as independent_rebuild_passed_p15 and does not inherit it. Reviewer findings F3–F5 are addressed in p16 (F3: one disclosed dred.py revision).

```
# archival/p15 -- the p15 release (first P.5 calculation; superseded as a release by p16)

p15 was released on 2026-10-08: source-only zip `XVI-P-p15-source-only.zip`, sha256
54325c1ff6c079e0389f96d9c0875524c73f6a706e079fd1734c1b1701c707d3, 535086 bytes, 122 files; source tree
79f5375272d6e7327721572ae7eeff59d6bf69d2eb13d353204d38ce9c1dc84f (fce59770... without reference); reference va
  +> lues
reference_values_p15.json 0866adfe0f9dc281053b84dcabbe751a8a7425f522d37839373b4d373415cda8; author rebuild pas
  +> sed
(report 1e729eba...); 75 tests.

INDEPENDENT REBUILD OF p15: PASSED (198/198 entrypoint values equal, 75 tests, own D13 spot-check, own Batch A
omega analysis, own p14r toys under the corrected dred), report `reproducibility/independent_rebuild_report_p1
  +> 5.json`
(archived copy byte-identical, sha256 6bc760c24f48d61fe066dde62cbe5b41daa7e4eafc347f8fd72e1a84fcab4713), bound
  +>  to
tree 79f53752... / zip 54325c1f.... SAME-BOX caveat: same machine as the author build (fresh directory, fresh
venv, from the source-only zip), NOT a second machine. p16 records independent_rebuild_passed_p15 = true; it i
  +> s NOT
inherited by the p16 tree.

Reviewer findings carried into p16: F3 (expr.replace / lambdify bypass of OpaqueDimension; no package use) ->
hardened in p16; F4 (stale footer 'XVI-P p14'; entrypoint PDF readback covered only the 6 classical records) -
  +> >
fixed in p16; F5 (non-contract file changes) -> p16 change log lists all; F6 (D13 spot-check started from the
```

```
exported integrand) -> scope note; F7 (E2 evanescent pole, no p14r evanescent counterterm) -> addressed in p16
  +> .

Contents: the p15 source-only zip, RELEASE_STATUS / RELEASE_MANIFEST / README / REBUILD_INSTRUCTIONS of p15, t
  +> he
p15 reference values, the p15 author and independent rebuild reports, the P15_* certificates, P14_dred_rules.j
  +> son
as released in p15 (with the p15 dred correction), the P15 milestones ledger, build records and the science PD
  +> F.
```

175 Archival material (p14r): the P.5-entry contract release (contract active in p15, p16)

Archival.

The p14r release (`archival/p14r/`) is superseded as a release by p15; its contract code stays active as the frozen contract on which p15 evaluates. Its independent rebuild PASSED (same box, fresh directory and venv; NOT a second machine), bound to the p14r tree a64c0ae3... and zip 19e8bf59.... ONE disclosed contract revision inside p15: the heuristic PrematureD4 guard of dred.py (finding F1: a false positive and a false negative) is replaced by a structural safeguard; released p14r values are unaffected (Pipeline.laurent was used only on toys).

```
# archival/p14r -- the p14r release (P.5-entry contract repair; superseded as a release by p15)

p14r was released on 2026-10-08 (source-only zip `XVI-P-p14r-source-only.zip`, sha256
19e8bf599daf102d8e23382544e6c6991ea307b2c3f31a0a97ad66d0b8a3dba3, 468541 bytes, 111 files; source tree
a64c0ae365980fa6b077f991df4316347527613075e87f17fecfe422db6e3894; author rebuild passed; 60 tests).

INDEPENDENT REBUILD OF p14r: PASSED (143/143 compared values equal, 60 tests, reviewer toy checks OK), report
`reproducibility/independent_rebuild_report_p14r.json` (archived copy, byte-identical to the reviewer's file,
  +> sha256
09cda7db0d1f8e1cf7409212e4ee88ef60f21975e1367472f2c3ed45f1f7004b), bound to the p14r tree a64c0ae3... and zip
19e8bf59... SAME-BOX caveat: the reviewer ran on the same machine as the author build (fresh directory, fresh
  +> venv,
from the source-only zip), NOT on a second machine. p15 records this as independent_rebuild_passed_p14r = true
  +> ;
it is NOT inherited by the p15 tree (p15 has its own, pending, independent flag).

Contents: the p14r source-only zip, RELEASE_STATUS / RELEASE_MANIFEST / README / REBUILD_INSTRUCTIONS of p14r,
  +>  the
p14r reference values, the p14r author and independent rebuild reports, the P14_* certificates and ledgers as
released in p14r (P14_dred_rules.json is the p14r version, before the p15 correction), build records and the
science PDF. The p14r contract (code/p5_entry/*) remains ACTIVE in p15 as the frozen contract on which p15
evaluates; this directory is provenance for the p14r release itself.

Why p14r is superseded as a release (p15 = first P.5 calculation):

1. p15 evaluates Batch A (D02, D12, D14a, D14b, D14c) and Batch B (D13) under the frozen p14r prescription.
2. ONE disclosed contract revision inside p15 (finding F1 of the p14r independent rebuild): the p14r
   PrematureD4Substitution guard in code/p5_entry/dred.py was a heuristic on the shape of the expression befor
  +> e the
   Laurent expansion. It gave a FALSE POSITIVE on eta^{rs} int l_r l_s/(l^2-m^2) = J (a correct d-free result)
  +>  and a
   FALSE NEGATIVE under the package's own premature_d4 mutation on the same integral (eta_trace_Gt reintroduce
  +> s d
   via tr gtilde = 4-d, giving d J/4 and a finite error -m^4/2). The R5 docstring claim that premature d = 4 "
  +> is
   detected" did not hold in general. p15 replaces it by a structural, provenance-based safeguard (opaque d;
   substitution only through dred.substitute_d; numeric traces of ghat/gtilde refused) and adds the reviewer's
   toys as regression tests. Rules R1-R6 are unchanged. Released p14r values are unaffected: in p14r
   Pipeline.laurent was applied only to the toy placeholders; no loop integral was evaluated.
3. Every other p14r contract file is byte-identical in p15 (checked by release_p15 against pinned sha256).
```

176 Archival material (p14): superseded by the p14r repair

Archival.

The p14 release (`archival/p14/`) is superseded by p14r, a narrow repair: p14 stated a DRED-type prescription without executable index rules and claimed that no evanescent remainder arises (withdrawn); its M3 labels conflated the scalar-extension and kernel scaling degrees; it called the counterterm quotient "essential" directions; its W5 regeneration did not cover D01, D02, D03, D12; and its gate-mutation test copies lacked frozen inputs. P.3/P.4 code is unchanged.

```
# archival/p14 -- the p14 release (superseded by the p14r repair)

p14 was released on 2026-10-08 (source-only zip `XVI-P-p14-source-only.zip`, sha256 7a43e8b0a374000d9ec565b64d
  +> d95d8deec5f3612512ed322307859e5e7a19ba,
source tree c8c6e044bc0a2a6cbf94276b6b125cd94c95d8496486be5221fc5b99d03158aa; author rebuild passed;
independent rebuild of p14: pending, never run). This directory keeps that release for provenance only. It is
  +> NOT
p14r evidence and is not read by the p14r pipeline.

Contents: the p14 source-only zip, RELEASE_STATUS / RELEASE_MANIFEST / README / REBUILD_INSTRUCTIONS of p14, t
  +> he p14
reference values (`reference_values_p14.json`) and author rebuild report (`reproducibility/author_rebuild_repo
  +> rt_p14.json`),
the P14_* certificates and ledgers, the p14 P.5-entry code, tests, tools, notes, build records and the science
  +>  PDF.

Why p14 is superseded (narrow p14r repair; P.3/P.4 classical code unchanged, no loop evaluation):

1. The prescription was "DRED-type" in words only: no executable index rules, and it claimed that no evanescen
  +> t
   remainder arises. p14r makes DRED operational (code/p5_entry/dred.py, certificates/P14_dred_rules.json). Th
  +> e
   rules are g4 = ghat + gtilde with tr ghat = d and tr gtilde = 4-d; l and p in QdS; exported l_r lifted as l
  +> .e_(r);
   ghat-projected Gram/PV reduction; all contractions before the Laurent expansion with d symbolic; evanescent
   parts retained. The "none arise" claim is withdrawn.
2. The D11 M3 labels conflated the scalar-extension scaling degree (4) with the kernel bound (8). p14r states:
   kernel scaling degree at most eight; degree of divergence at most four; contact derivative order at most fo
  +> ur;
   sd(d^alpha delta) = 4 + |alpha|. M3 is now conditional on M2. Exactly degree four is an explicit normalizat
  +> ion
   condition N4; the order-0 and order-2 contacts are enumerated.
3. The counterterm quotient was called "essential counterterm directions". It is now "eight candidate directio
  +> ns
   modulo the computed exact equivalences, before imposing the relevant identity constraints". D cap ker sigma
  +> _0 has
   not been computed.
4. W5 regenerated only D14a-c, D11 and D13. p14r adds W6, an independent Hessian/Tr-log regeneration of D01, D
  +> 02,
   D03 and D12. The on-shell check is renamed "Ward contraction with two free on-shell graviton legs and an of
  +> f-shell
   gauge leg".
5. The p14 gate-mutation test copies lacked the frozen tests/ inputs, so every copy was unfrozen and P.5 mutat
  +> ions
   were rejected for the wrong reason. p14r copies all frozen inputs and adds an unmutated baseline that must
  +> be
   accepted.
```

177 Archival material (p13): frozen classical P.3/P.4 inputs

Archival.

The p13 release (`archival/p13/`) is superseded by p14 as a release but its classical P.3/P.4 code is kept byte-identical and is the frozen P.5 input. The archive holds the p13 source-only zip (tree 2b1baf33…, zip 2f85a566…), its reference values, its author rebuild report and the independent rebuild report (passed; SAME box with a fresh venv and a source-only build, NOT a second machine). The p14 freeze check re-verifies these pins on every release.

```
# ARCHIVAL -- XVI-P revision p13 (superseded as a release by p14; its classical P.3/P.4 code is the FROZEN P.5
  +>  input)

Snapshot of the p13 release (2026-10-08). p14 keeps the classical P.3/P.4 code byte-identical to this snapshot
  +>  and
re-checks the freeze on every release (`code/p5_entry/freeze_p13.py`: F1 independent report, F2 frozen files,
F3 classical values).

Pins (also in `code/p5_entry/p14_conventions.py`):
* p13 source tree sha256 `2b1baf335468944aa800c05eaa8b0b4df67ee893491079fcc944eb8fb0e69a70`
* p13 source-only zip sha256 `2f85a566c49689dea329df9fe3ea213eb6495759d4e95546495e245e7fdf3266`
  (`XVI-P-source-only_p13.zip`, a byte copy of `/workspace/XVI-P-source-only.zip`; zips are never source, so t
  +> his
  copy is not in the p14 source-only zip)
* p13 reference values sha256 `d8bf3d97457913dbd1fa03a2e801630005aeca601b93ebb53307fb30e1bc4095`
  (`reference_values_p13.json`, a byte copy of the p13 `reproducibility/reference_values.json`)
* independent rebuild report sha256 `7ce496cc446f2ef1948de6aaf034b09e9dcab0b9878cd6ae6015cda63a6515e4`
  (`reproducibility/independent_rebuild_report_p13.json`, a byte copy of the reviewer's
  `reproducibility/independent_rebuild_report.json`)
* frozen file list `frozen_p13_inputs.json` (54 classical files with their p13 sha256)

Independent rebuild of p13: PASSED (112/112 reference values matched, 77 values recomputed independently, no
mismatches, hand spot check of the conformal boundary identity). CAVEAT: it ran on the SAME box (shared kernel
  +> ,
TeX Live, poppler) in a fresh venv from the source-only zip; it was NOT run on a second machine. The resulting
  +>  flag
`independent_rebuild_passed = true` is bound to the p13 tree above and is not inherited by p14 changes.

Contents:
* `README_p13.md`, `REBUILD_INSTRUCTIONS_p13.md`, `RELEASE_MANIFEST_p13.txt`, `RELEASE_STATUS_p13.json`;
* `XVI-P-source-only_p13.zip`, `reference_values_p13.json`, `frozen_p13_inputs.json`;
* `reproducibility/`: author and independent rebuild reports and the inherited data pins of p13;
* `build/`, `certificates/`, `ledgers/`, `code/`, `tools/`, `tests/`, `manuscript/`: the p13 build outputs, re
  +> lease
  code, tools, tests and notes as released (superseded by the p14 versions; not read by the p14 pipeline).

Only `reference_values_p13.json`, `frozen_p13_inputs.json` and the independent report are read by the p14 pipe
  +> line,
each verified against its pinned sha256 before use.
```

178 Archival material (p12): NOT p14 evidence

Archival.

The p12 release (`archival/p12/`) is superseded by p13, which adds no mathematics. p12 used "classical readiness" without the milestone labels fixed in p13 (P.3, P.4 = classical readiness, P.5 entry/calculation/identity), had no completion dossier, and had no source-only rebuild contract or rebuild flags.

```
# ARCHIVAL -- XVI-P revision p12 (superseded by p13; not p13 acceptance evidence)

Snapshot of the p12 active release (2026-10-08). It contains:
* code, tools and tests (`test_p12.py`);
* certificates `P12_*` and ledgers `P12_*`;
* the science note `manuscript/notes/P12_classical_closure.tex`, the generated manuscript bodies and the pream
  +> ble;
* `RELEASE_STATUS_p12.json`, `RELEASE_MANIFEST_p12.txt` and `README_p12.md`;
* `build/` (test results, PDF readback, PDF inputs, build log).

p12 was the evidence-reconciliation release (critique t103u). Its mathematics is unchanged in p13.

Why it was superseded (plan t104u):
* **Milestones were not defined.** p12 spoke of "classical readiness" without milestone labels. Older notes us
  +> ed
  "P-next.N", and a p08 table row in which "P4" meant coupled one-loop restoration. p13 fixes the definitions:
  P.3 = projection + dependencies + graph inventory; P.4 = classical readiness, NOT coupled one-loop restorati
  +> on;
  P.5 entry / calculation / identity.
* **No completion dossier.** p12 described its evidence as a release history. p13 adds one table in which ever
  +> y
  claim is linked to its file, function, certificate key and test, and the links are resolved mechanically
  (`certificates/P13_dossier.json`).
* **No reproducibility contract.** p12 had no source-only rebuild entrypoint, no reference values, no
  `author_rebuild_passed` / `independent_rebuild_passed` flags, and no source-only zip. Its builders read
```

```
  `/workspace/XVI-M.tex` and `/workspace/XVI-N-r10/code/prep/grav_density.py` by absolute path, and one certif
  +> icate
  recorded that absolute path. p13 resolves these through sha256 pins, with vendored copies.

The p12 certificates are internally consistent: every residual record is `[0,0,0,0]`/`true`, and the quartic
and master records agree. They are not p13 evidence because p13 regenerates everything from source.
```

179 Archival material (p11): NOT p14 evidence

Archival.

The p11 release (`archival/p11/`) is superseded. Its certificate FILES were internally consistent (all Noether residual counts zero), but its appendix typeset multi-line JSON arrays: a page break inside `Q_terms` placed the footer page number (12, 29) between array elements, so a text extraction read [0,0,120,0] and [290,0,0,0]. p12 prints scalar arrays on one line, prefixes page numbers, enforces the count/flag invariant before serialization and in the release gate, cross-checks the quartic and master records, and gates every pdf/zip/sync stage.

```
# ARCHIVAL -- XVI-P revision p11 (superseded by p12; not p12 acceptance evidence)

Snapshot of the p11 active release (2026-10-08): code, tools, tests (`test_p11.py`), certificates `P11_*`, led
  +> gers
`P11_*`, the science note `P11_classical_closure.tex`, the generated manuscript bodies, `RELEASE_STATUS_p11.js
  +> on`,
`RELEASE_MANIFEST_p11.txt` and `build/` (test results).

Why it was superseded (critique t103u) -- root cause of the "contradictory certificates":
* The certificate FILES were internally consistent and agreed with each other. Every gravitational Noether rec
  +> ord had
  `Q_terms = [0, 0, 0, 0]` and `Q_identically_zero = true` at kappa0, kappa1 and kappa2, in both
  `P11_quartic_verified.json` and `P11_master_kappa2.json`. Both records come from `grav_noether.noether`, whe
  +> re the
  count and the flag derive from the same normalised dicts.
* The p11 APPENDIX printed these arrays over several lines (`json.dumps(indent=2)`). In two places a page brea
  +> k fell
  inside an array:
  - quartic kappa0, after the 2nd element, footer "12";
  - master kappa1, right after "[", footer "29".
  A plain text extraction of the PDF then reads `0, 0, 12 0, 0` -> [0,0,120,0] and `29 0, 0, 0, 0` -> [290,0,0
  +> ,0].
  The listings were byte-identical to the files executed and written in that build (`check_listings_current`).
  The defect was a presentation one, not stale snapshots, a different code path, or zero-valued dict keys:
  `jetpoly` drops zero coefficients in every operation.
* Missing safeguards that p12 adds:
  - no count/flag invariant before serialization or in the release gate;
  - no cross-certificate check;
  - `--from-stage pdf|zip|manifest` bypassed `release.py`;
  - no readback of the typeset values;
  - bare page numbers.
```

180 Archival material (p10): NOT p14 evidence

Archival.

The p10 release (`archival/p10/`) is superseded and is not p14 evidence: its C3 was vacuous and stated the wrong orientation $+\partial_\mu B^\mu$; it cited samples as proof; its master certificate carried an unconditional pass; its multidegree lemma and polarization statement were imprecise. Its note is preserved verbatim in `archival/p10/notes/` and is not typeset here.

```
# ARCHIVAL -- XVI-P revision p10 (superseded by p11; not p11 acceptance evidence)

Snapshot of the p10 active release (2026-10-08): certificates `P10_*`, ledgers `P10_*`, code, tests, tools,
science note `P10_classical_closure.tex`, `RELEASE_STATUS_p10.json`, `RELEASE_MANIFEST_p10.txt`.

Superseded by p11 because (critique t102u):
* C3 (sqrt(-q)R - L_GG = d_mu B^mu on h00 = A(x0)) was VACUOUS (only Gamma^0_00 != 0; R = L_GG = B = 0) and it
```

```
  +> s
  stated orientation was WRONG: with the ell4_verify Ricci convention, sqrt(-g)R - L_GG = - d_mu B^mu.
* Several acceptance statements cited sampled evaluations (plane-wave points, 4 evaluator samples, one Hessian
  momentum) as proof; p11 replaces them by deterministic symbolic identities and keeps samples as regression c
  +> ontrols.
* master_kappa2 reported an unconditional "pass": true; the multidegree lemma was misstated (E_u P_d = P_{d-e_
  +> u} E_u);
  the V4 polarisation normalisation was stated inconsistently.
```

181 Archival material (p02–p09): NOT p14 evidence; superseded milestone wording

Archival.

Everything in this section is superseded material from revisions p02–p09, kept for provenance only (`archival/p02_p09/`). It is not read by the p14 pipeline and is not acceptance evidence. HISTORY ONLY: its milestone wording ("P-next.N", and the table row "P4 coupled one-loop restoration" in `P_scope.tex`) is superseded by the p13 definitions kept in p14 (P.4 = classical readiness, NOT coupled one-loop restoration; see the milestone section of the science volume). The archived `ell4_verify.py` and `ell4_generator.py` state the WRONG boundary orientation ($+\partial_\mu B^\mu$) and a vacuous C3; see `P13_boundary_identity.json`.

```
# ARCHIVAL -- XVI-P revisions p02-p09 (not active, not acceptance evidence for p10)

Moved here by the p10 release (2026-10-08). Nothing in this directory is read by the p10 pipeline
(`tools/build_p10.py`) or by the active tests (`tests/test_p10.py`, `tests/test_matter_loop_gates.py`,
`tests/test_inverse_metric_series.py`).

* `certificates/`, `ledgers/`, `notes/`, `code/`, `tests/` -- p02-p09 material (P1-P4, P9 series).
  The p09 `required_classical_BV_components_verified=false` / residual `complete_af_ghost_0_kappa2_on_D_h`
  is superseded by `certificates/P10_master_kappa2.json`.
* `tests/test_quantum_normalization.py` -- inherited XVI-O o70-sync package test; it pins
  `RELEASE_STATUS.revision == "o70-sync"` and already failed on the p09 tree (first check). The
  wrong-prescription and source-comparison controls it was cited for are exercised, and pass, in the
  active `tests/test_matter_loop_gates.py` (route-A/B prefactor mutations; Ww prescription-id mutations).
* `certs_pre_p09/`, `listings_p02_p07/`, `listing_src_snapshot_p09/`, `RELEASE_MANIFEST_p09.txt` -- earlier ar
  +> chives.
* `release_p09/` -- the p09 PDFs and zips formerly at the package root (superseded). The two p09 zips are kept
  +>  on
  disk here but are NOT nested into the p10 zips.
* `build_pre_p10/` -- pre-p10 build scratch (old listing copies, LaTeX logs, outputs, PDFs).
```

181.1 P.1 — Freeze of the D11 distributional kernel

Inherited source. XVI-O revision `o70-sync` at `/workspace/XVI-O/` (read-only; not edited in place). This section does not prove a new physics theorem. It freezes the conjunction already used as kernel acceptance.

181.1.1 Four prerequisites (conjunction)

1. `lem:analytic_FP_u_mu_R_o70`: $F(\zeta) = (Q - i0)^\zeta$ is a holomorphic family; then $\Box F = 4\zeta(\zeta+1)\phi_{-1+\zeta}$ yields holomorphy of $\phi_{-1+\zeta}$ near the origin; only then the second recurrence with Gate 0 gives residue $r = -i\pi^2$ and $u_\mu^R = \mathrm{FP}_{\zeta=0}[\mu^{2\zeta}(Q - i0)^{-2+\zeta}]$.

2. `lem:BV_rules_on_U`: common-prescription boundary-value rules $(D)/(Q)/(P)$ on U for the used powers; cleared 256-component rational reduction translates through that algebra. $\alpha = A^2/60$, $\beta = A^2/18$ derived; $\Pi^R_{\mathrm{tr}}|_U = (i\kappa^2/4)T$.

3. Active reference only `W_w_jet_pullback_via_Pi_tr_o68`: $\langle \Pi^R_w, f\rangle = \langle \Pi^R_{\mathrm{tr}}, W_w f\rangle$.

4. Contact: $\Pi^R_w + L = \Pi^R_{\mathrm{tr}}$ with L the explicit local contact supported at the origin. Scaling of Π^R_w is controlled by the finite local difference L from Π^R_{tr}. `kernel_two_point_Ward` holds; $s_0 C_L$ is not evaluated in P.1.

Theorem 6 (D11 kernel freeze)**.** *Under the stated massless scalar-loop prescription, the D11 two-point kernel has a transverse distributional extension. Its difference from the chosen reference is the explicitly defined local contact L.*

The analytic qualifications for items (1)–(2) are *displayed* in Section 181.2 (local integrability/holomorphy of $F(\zeta)$, and the common-prescription rules $(D)/(Q)/(P)$ on U), rather than by citation of an external definition alone.
Not earned here: functional ST4b; graviton-loop renormalization; bulk BV restoration; insertion closure.

181.1.2 Reproducibility

Clean rebuild without stale generated certificates; dependency order, environment versions, exit codes, negative prerequisite tests, and an external manifest are published under `certificates/P1_D11_freeze.json` and `RELEASE_MANIFEST_EXTERNAL.txt`.

181.2 XVI-P supplemental proofs for P.1 premises

Purpose. Displayed local arguments for the inherited identifiers `lem:analytic_FP_u_mu_R_o70` and `lem:BV_rules_on_U`. XVI-O is not edited in place. Revision `p04` repairs the origin angular bound and the common-regularization bridge.

181.2.1 Holomorphy of $F(\zeta) = (Q - i0)^\zeta$ near $\zeta = 0$, including the origin

Lemma 7 (Local integrability / holomorphic family)**.** *Let $Q(z) = \eta_{\mu\nu} z^\mu z^\nu$ with $\eta = \mathrm{diag}(+---)$, and let $(Q - i0)^\zeta$ be the boundary value of the principal branch from $\operatorname{Im} Q < 0$. Then $F(\zeta) := (Q - i0)^\zeta$ is a holomorphic family of distributions near $\zeta = 0$ in $\mathcal{D}'(\mathbb{R}^4)$, including tests that meet the origin.*

Proof. 1. **Away from $\{Q = 0\}$.** On $|Q| \geq \delta > 0$, $(Q - i0)^\zeta = \exp(\zeta \operatorname{Log}(Q - i0))$ is smooth in z and entire in ζ.

2. **Near a nonzero null point.** Locally Q is a submersion. In a transverse coordinate $t = Q$,

$$\int \left|(t - i0)^\zeta \chi(t, y)\right| dt\, dy \leq C \int_{-\varepsilon}^{\varepsilon} |t|^{\Re\zeta}\, dt < \infty \quad \text{for} \quad \Re\zeta > -1.$$

3. **At the origin (angular factor retained).** Write $z = r\omega$, $Q = r^2 q(\omega)$ with q smooth on S^3 and vanishing on the lightlike locus. The full origin integral is

$$\int_{|z|<\varepsilon} |Q|^{\Re\zeta}\, d^4 z = \int_0^{\varepsilon} r^{3+2\Re\zeta}\, dr \int_{S^3} |q(\omega)|^{\Re\zeta}\, d\omega. \tag{1}$$

The radial factor converges for $\Re\zeta > -2$. Near the lightlike locus of S^3, q has a simple zero transversely, so the angular integral behaves like $\int |s|^{\Re\zeta}\, ds$ and requires $\Re\zeta > -1$. The *combined* sufficient range is therefore

$$\Re\zeta > -1.$$

This still contains a neighbourhood of $\zeta = 0$. On a small disk $\Re\zeta > -\delta$ with $0 < \delta < 1$, the logarithmic ζ-derivatives are dominated by an integrable majorant of the same type, so the $\mathcal{D}'$-valued map $\zeta \mapsto F(\zeta)$ is holomorphic near 0.

4. **Gluing.** The three regimes agree on overlaps; analytic continuation in ζ is unique.

□

Corollary 8 (Recurrence and $\phi_{-1+\zeta}$)**.** *With F holomorphic for $\Re\zeta > -1$ near 0, const-coeff differentiation gives $\Box F(\zeta) = 4\zeta(\zeta + 1)\, \phi_{-1+\zeta}$. Since $F(0) = 1$, $\Box F(0) = 0$, so $\Box F(\zeta) = \zeta\, G(\zeta)$ with G holomorphic, and*

$$\phi_{-1+\zeta} = \frac{G(\zeta)}{4(\zeta + 1)}$$

is holomorphic near $\zeta = 0$. The residue $r = -i\pi^2$ for $u_\mu^R = \mathrm{FP}_{\zeta=0}[\mu^{2\zeta}(Q - i0)^{-2+\zeta}]$ is unchanged and may proceed only after this premise.

Non-claim. Does not reopen Gate 3; does not add $c\delta$ to u_μ^R; XVI-J `prop:extension` remains freedom of a narrowed square.

181.2.2 Common-prescription rules $(D)/(Q)/(P)$ on U

Let $U = \mathbb{R}^4 \setminus \{0\}$ and $\varphi_n := (Q - i0)^{-n}$.

Lemma 9 (Boundary-value calculus on U)**.** *Under the common $Q - i0$ prescription, in $\mathcal{D}'(U)$,*

$$\text{(D)} \qquad \partial_\mu \varphi_n = -2n\, z_\mu\, \varphi_{n+1}, \tag{2}$$

$$\text{(Q)} \qquad Q \cdot \varphi_n = \varphi_{n-1} \quad (n \geq 1), \tag{3}$$

$$\text{(P)} \qquad \varphi_a\, \varphi_b = \varphi_{a+b}. \tag{4}$$

Proof. Work in the chart $t = Q$ already used above (submersion on U). Define the one-dimensional boundary values

$$b_n(t) = \lim_{\varepsilon \downarrow 0} (t - i\varepsilon)^{-n} \quad \text{in } \mathcal{D}'(\mathbb{R}_t).$$

1. **Algebraic identity before the limit.**

$$t(t - i\varepsilon)^{-n} = (t - i\varepsilon)^{-(n-1)} + i\varepsilon\,(t - i\varepsilon)^{-n}.$$

2. **Vanishing of the remainder.** For a compact test $\chi(t, y)$ supported in a chart of U,

$$\big\langle i\varepsilon(t - i\varepsilon)^{-n},\, \chi \big\rangle \to 0 \quad (\varepsilon \downarrow 0)$$

   in $\mathcal{D}'(U)$: the family $\varepsilon(t - i\varepsilon)^{-n}$ is bounded in the standard boundary-value topology and tends to zero off $t = 0$, hence the pairing vanishes (no residual cone-supported contribution on U). Therefore $t\, b_n = b_{n-1}$ in $\mathcal{D}'(U)$, which is (Q) after transporting by the submersion Q. Differentiation under the same limit gives (D).

3. **Product under common regularization.** Both factors are taken with the *same* $\varepsilon \downarrow 0$ limit before any identification:

$$(t - i\varepsilon)^{-a}(t - i\varepsilon)^{-b} = (t - i\varepsilon)^{-(a+b)}.$$

   For compact tests on a chart of U, the family is bounded in the standard boundary-value topology, so the limit exists in $\mathcal{D}'(U)$ and equals b_{a+b}. This is a common-prescription product, not a uniqueness argument from the off-cone smooth algebra alone, and not a separate Hörmander existence claim without the shared regulator. Transporting by the submersion Q gives (P).

4. **256-component translation.** Off-cone, the cleared polynomial identity $\mathcal{K}u - T = 0$ holds. With (D)/(Q)/(P) as the common-prescription continuation, $\mathcal{K}u = T$ in $\mathcal{D}'(U)$ with no added cone-supported term. The 256-component symbolic check is unchanged.

□

Non-claim. Not Lorentz covariance of L; not Gate 3 spanning; wavefront existence alone is not equality without the common-limit identification.

181.3 Antibracket generator correspondence for s_0

Purpose. Positive calculation that $s_0 h = (S_0, h)$ from the inherited bracket, with the odd right-derivative sign displayed. Exclusion of the XIV-A relative plateau identity remains; it is not a substitute for this calculation.

181.3.1 Inherited bracket

$$(F,G)=\int\Big(\frac{\delta^R F}{\delta\Phi_A}\,\frac{\delta^L G}{\delta\Phi^{A\star}}-\frac{\delta^R F}{\delta\Phi^{A\star}}\,\frac{\delta^L G}{\delta\Phi_A}\Big)\,dx. \tag{5}$$

Free antifield coupling (XVI-N split, $a=\frac{1}{2}$):

$$S_0\supset\int h^\star_{\mu\nu}\,R_0(c)^{\mu\nu}\,dx,\qquad R_0(c)^{\mu\nu}:=a\,\Pi^{\mu\nu}{}_{\rho\sigma}\big(\partial^\rho c^\sigma+\partial^\sigma c^\rho\big), \tag{6}$$

where Π is the symmetric projector on two-tensors ($\Pi^{\mu\nu}{}_{\rho\sigma}=\frac{1}{2}(\delta^\mu_\rho\delta^\nu_\sigma+\delta^\mu_\sigma\delta^\nu_\rho)$).

181.3.2 Which summand contributes

For $G=h$ (no antifield dependence), $\delta^L G/\delta\Phi^\star=0$, so the *first* summand of (5) vanishes. The contributing term is

$$(S_0,h)=-\int\frac{\delta^R S_0}{\delta h^\star}\,\frac{\delta^L h}{\delta h}\,dx. \tag{7}$$

Because $h^\star$ and $R_0(c)$ are both odd, the right derivative on the monomial $h^\star R_0(c)$ supplies a minus:

$$\frac{\delta^R S_0}{\delta h^\star}=-R_0(c). \tag{8}$$

Hence

$$(S_0,h)=-\int\big(-R_0(c)\big)\,dx=R_0(c)=a\,\Pi(\partial c+\partial c), \tag{9}$$

i.e. $s_0h^{\mu\nu}=a(\partial^\mu c^\nu+\partial^\nu c^\mu)$ after applying Π. The executable check constructs $s0_from_bracket$ from (8) and compares it to the declared s_0h; it must not assign the same expression to both sides by fiat.

181.3.3 Index convention

$h^{\mu\nu}$ raised; kernel $K_{\mu\nu\rho\sigma}$ lowered; flat η of signature $(+---)$. Proposition 10 and Theorem 11 remain under this declared s_0h. Bulk ST4b unestablished; D12 Missing.

181.4 P.2 — D11 linearized functional identity

First new calculation. XVI-O left s_0C_L unevaluated. This section derives the linearized cancellation on the declared matter-loop projection.

181.4.1 Definitions

For compactly supported symmetric metric perturbations h,

$$\Gamma^{(2)}_{1,w}[h]=\tfrac{1}{2}\int h^{\mu\nu}(x)\,\Pi^R_{w,\mu\nu\rho\sigma}(x-y)\,h^{\rho\sigma}(y)\,dx\,dy, \tag{10}$$

$$C_L[h]=\tfrac{1}{2}\int h^{\mu\nu}(x)\,L_{\mu\nu\rho\sigma}(x-y)\,h^{\rho\sigma}(y)\,dx\,dy. \tag{11}$$

Free BRST on the metric (XVI-N antifield split; ghost normalization $a=\frac{1}{2}$, distinct from D11 (α,β)):

$$s_0h^{\mu\nu}=a\big(\partial^\mu c^\nu+\partial^\nu c^\mu\big). \tag{12}$$

181.4.2 General variation formula

Proposition 10 (Linearized variation of a quadratic kernel functional)**.** *Let K be a translation-invariant two-point kernel with pair symmetry $K_{\mu\nu\rho\sigma} = K_{\nu\mu\rho\sigma} = K_{\mu\nu\sigma\rho}$ and exchange symmetry $K_{\mu\nu\rho\sigma}(z) = K_{\rho\sigma\mu\nu}(-z)$. Then, on compactly supported symmetric h,*

$$\boxed{s_0 C_K = -2a \int c^\nu(x)\big[\partial^\mu K_{\mu\nu\rho\sigma}(x-y)\big] h^{\rho\sigma}(y)\, dx\, dy.} \tag{13}$$

Proof. Varying both external legs of (11) (with L replaced by K) and using exchange symmetry equates the two contributions and cancels the prefactor $\frac{1}{2}$, giving $\int (s_0 h^{\mu\nu}) K_{\mu\nu\rho\sigma} h^{\rho\sigma}$. Substitute (12). Pair symmetry equates the $\partial^\mu c^\nu$ and $\partial^\nu c^\mu$ terms, producing the factor $2a \int (\partial^\mu c^\nu) K_{\mu\nu\rho\sigma} h^{\rho\sigma}$. Distributional integration by parts (compact support of c) moves ∂^μ onto K and supplies a minus sign, yielding (13). Ghost number: $\mathrm{gh}(C_K) = 0$ and $\mathrm{gh}(c) = 1$ imply $\mathrm{gh}(s_0 C_K) = 1$. The sign is that of the graded derivation $s_0 = (S_0, \cdot\,)$ acting on the quadratic functional (Section 181.3); it is *not* imported from the XIV-A relative plateau identity. Index convention: $h^{\mu\nu}$ raised, $K_{\mu\nu\rho\sigma}$ lowered. □

181.4.3 Cancellation

From the frozen contact identity $\Pi^R_w + L = \Pi^R_{\mathrm{tr}}$ and transversality $\partial \Pi^R_{\mathrm{tr}} = 0$,

$$\partial^\mu L_{\mu\nu\rho\sigma} = -\partial^\mu \Pi^R_{w,\mu\nu\rho\sigma}. \tag{14}$$

Apply Proposition 10 to $K = L$ and $K = \Pi^R_w$:

Theorem 11 (D11 linearized functional cancellation)**.** *On compactly supported symmetric h,*

$$\boxed{s_0\big(\Gamma^{(2)}_{1,w} + C_L\big) = 0.} \tag{15}$$

Acceptance: `D11_linearized_functional_cancellation=verified` *on the declared matter-loop projection. Bulk functional ST4b is* not *established.*

181.4.4 One-point / D12 obligation

The quadratic identity (15) is distinct from the order-κ^2 matter identity. With coupling-stripped coefficients examine

$$s_0 \Gamma^{[2]}_{1,\mathrm{matter}} + s_1 \Gamma^{[1]}_{1,\mathrm{matter}}. \tag{16}$$

Shape guidance: the mixed Noether density identity of Gate 1 (`prop:noether`) — not $(S_{\mathrm{sc}}, S_{\mathrm{sc}}) = 0$ and not (15).

- D12 is **Missing**: the massless dimensional-regularization prescription is declared, not evaluated; D12 is never silently set to zero in (16).
- The one-point term is retained in (16); its dim-reg prescription is declared, not evaluated.

181.5 P-next.1 — $P_{\mathrm{in}}/P_{\mathrm{out}}$ and `P3_hh_identity`

Scope. Freeze the split projections and derive the three bracket dependency rows for the named output pattern *ch*. This release does *not* generate a full graph inventory, construct ℓ_4, invent s_2, open P.5, or claim symmetry restoration. The invariant antibracket is not mixed with an unspecified gauge-fixed action.

181.5.1 Split projections

Definition 12 (P_{in} and P_{out})**.**

- P_{in} *extracts* one-loop functional coefficients *in* Γ_1 *(starting input: the hh one-loop coefficient).*
- P_{out} *extracts* identity-defect coefficients *of the ghost-number-*1 *defect* A_1 *in* $(S, \Gamma_1) = A_1$.

These replace the mixed single P_{ret} *of p04. A ghost-number-*0 *kernel counterterm is not the ghost-number-*1 *defect.*

181.5.2 Named target `P3_hh_identity`

Field	Value
Output pattern	ch (gh= 1 field-only defect $\sim \int c^\nu W_\nu[h,\dots]$)
Input starting set	hh one-loop coefficient (D11 kernel projection); *not* a claim that hh alone completes ch
Coupling / loop	κ^2 total; one loop
Test domain	compact smooth smearings; off-shell
Baseline	S_{inv} (not S_D)
Excluded initially	$\phi\phi$ and ghost two-points (add only if a bracket row names them)
Equivalences	free-EOM Unresolved (unused); XIV-A relative plateau excluded as sign source

181.5.3 Three bracket rows (by functional differentiation)

With

$$I_1^{[2]} = (S_0, \Gamma_1^{[2]}) + (S_1, \Gamma_1^{[1]}) + (S_2, \Gamma_1^{[0]}),$$

project onto output ch using the classical antibracket and the odd right-derivative of p04. Dependencies are *not* inferred from diagram names.

Output	Bracket term	Required input	S
ch	$(S_0, \Gamma_1^{[2]})$	hh one-loop (field-only)	n
ch	$(S_1, \Gamma_1^{[1]})$	$\Gamma_1^{[1]}$ coeffs (incl. one-point where applicable); `AMP_S1` placeholder until a derivative produces it	n
ch	$(S_2, \Gamma_1^{[0]})$	free $\Gamma_1^{[0]}$	n

Antifield remark. One-loop antifield coefficients at κ^2 could feed the first summand of $(S_0, \Gamma_1^{[2]})$. Tree seeds D07–D09 are *not* those coefficients. Such channels remain on the unresolved list.

181.5.4 Statuses

Status	This release
`projection_defined`	earned
`dependency_inventory_complete`	earned (relative to declared catalogue; unresolved list explicit)
`classical_inputs_constructed`	not earned (P.4)
`quantum_inputs_evaluated`	not earned (P.5)
`projected_identity_verified`	not earned (P.5)

181.5.5 P.3.3 interface stub

Must later record: the invariant extended identity object; which propagators are gauge-fixed; which canonical transformation connects them. Not invented here; not supplied by the XVI-O kernel alone.

181.5.6 Non-claims

No full graph generation; no ℓ_4 construction; no s_2; no symmetry restoration; bulk ST4b unestablished; D12 Missing; O.1 `NOT_CLOSED`; insertion freeze unchanged. **p05 note.** The mixed P_{ret} account below is retained as history. The active cut is Section 181.5 (`P3_hh_identity`).

181.6 P.3 — Identity-driven domain (provisional)

Scope. Identity-driven dependency analysis only. No P.4 classical closure, no P.5 restoration, no invented s_2, no Constructed ℓ_4, no `symmetry_restored=true`.

181.6.1 Projection P_{ret} by retained monomials

Definition 13 (Retained projection)**.** P_{ret} *is defined by the retained* functional monomials *and* external-leg patterns *at one loop and total order* κ^2 *on* S_{inv}*:*

- *bulk two-point patterns hh, and conditionally* $\phi\phi$ */ ghost two-point only when those monomials are retained;*
- *matter-induced hh (D11 kernel; D12 Missing);*
- *gravitational hh candidates D01–D03;*
- *antifield content only through vertex-derived* $\mathrm{gh} = 0$ *tree seeds (D07–D09), not as one-loop antifield amplitudes.*

A field list alone does not decide whether a $\phi\phi$ *or ghost two-point enters: that requires the monomial to be retained. Ghost number of* A_1 *is* 1*; admitted counterterms have ghost number* 0*. Test domain: compact smearings; switches s-inert; off-shell BV.*

181.6.2 Order-κ^2 expansion

$$(S_0, \Gamma_1^{[2]}) + (S_1, \Gamma_1^{[1]}) + (S_2, \Gamma_1^{[0]}) = A_1^{[2]}. \tag{17}$$

Lower-coupling terms are not assumed to vanish without evidence.

On $(S_2, \Gamma_1^{[0]})$**.** If the normalized free one-loop coefficient $\Gamma_1^{[0]}$ is field-independent (a pure constant functional), then $(S_2, \Gamma_1^{[0]}) = 0$ without constructing ℓ_4. If it depends on fields, the term remains blocked by the unresolved ℓ_4/S_2 sector (no s_2 invented). The package records the field-independence check explicitly.

181.6.3 Repaired topology (vertices + propagator pairings)

- **D04:** mixed bubble — one graviton propagator and one scalar propagator from the cubic scalar–graviton vertex (XI-A). Not "graviton or scalar per placement."
- **D06:** catalogue vertices with *one graviton* plus *one ghost–antighost* propagator. Not $V_{hcc\bar{c}}/V_{ccc}$ as a frozen two-vertex substitute.
- **D03:** two ghost–antighost contractions of $V_{hc\bar{c}}$ (single mixed ghost line structure). Not separate $\Delta(c)$ and $\Delta(\bar{c})$.
- **`AMP_S1_Gamma1_1_hh`:** identity-dependency *placeholder*, not a generated amplitude and not an XVI-M selected image.

Classical tree seeds D07–D09 remain separate from one-loop amplitudes.

181.6.4 Compatibility and acceptance

Dependencies outside the constructed kernel, or unconstructed required inputs, trigger `domain_provisional`. Present status: provisional; target not closed. D12 Missing; Gate 3 spanning Partial; bulk `symmetry_restored=false`; O.1 `NOT_CLOSED`; insertion frozen.

181.7 P-next.2 — Gauge-fixed interface and named graphs

Scope. Keep P-next.1 (`P3_hh_identity`, $P_{\mathrm{in}}/P_{\mathrm{out}}$, three *ch* rows) intact. Record the P.3.3 interface honestly and generate graphs only for coefficients *named* by those rows. No s_2, no ℓ_4 Constructed claim, no P.5, no symmetry restoration.

181.7.1 P.3.3 — Gauge-fixed / minimal-BV interface

Object	Record
Invariant extended BV identity	S_{inv} (baseline; not S_D)
Gauge-fixed propagators / loops	$S_{\mathrm{gf}} = \int D_\nu D^\nu$; ghost kinetic from $s\Psi$; free $\Delta(h)$, $\Delta_F(\phi)$, $\Delta(c,\bar{c})$
Antifield / source	$V^{0,1}_{\mathrm{af},h}$, $V_{\mathrm{af},\phi}$, $V_{\mathrm{af},c}$, $V_{\mathrm{af},\bar{c}}$; one-loop antifield coeffs at κ^2 Unresolved
Nonminimal fields	b^μ, $\bar{c}^\star$ (recorded; do not remove XVI-N obstruction)
Connecting map	**Unresolved** — not invented; not supplied by the XVI-O kernel alone; do *not* silently mix the invariant antibracket with an unspecified gauge-fixed effective action

Interface status: `Partial`. $s\Psi$ through κ^2 remains Partial; no invented s_2.

181.7.2 P.3.4 — Graphs for named *ch*-row coefficients only

Generated-graph table

ID	Ext.	Vertices / pairings	Kind	Status
D11	hh	$V_{3,h\phi\phi}\times 2$; $\phi\phi\times 2$	bubble	Constructed (kernel)
D01	hh	$V_{3,\mathrm{grav}}\times 2$; $hh\times 2$	bubble	candidate, not evaluated
D03	hh	$V_{3,\bar{c}ch}\times 2$; two c–$\bar{c}$ contractions of $V_{hc\bar{c}}$	bubble	candidate, not evaluated
D12	hh	$V_{4,\mathrm{sg}}$ $(hh\phi\phi)$; $\phi\phi$ self-contraction	tadpole	**Missing** (vertex identified)
D02	hh	$V_{4,\mathrm{grav}}$; hh self-contraction	tadpole	**blocked by** ℓ_4 (not Constructed)

`AMP_S1_Gamma1_1_hh` remains an identity-dependency placeholder and stays *outside* the generated-graph table. $(S_2, \Gamma_1^{[0]})$ needs no graph under the field-independent free normalization.

Topology repairs not required by named *ch* rows

- **D04:** two $V_{3,h\phi\phi}$; one scalar + one graviton internal line (not "graviton or scalar per placement"). External $\phi\phi$ — excluded until a bracket row names $\phi\phi$.
- **D06:** vertices $V_{3,\bar{c}ch}\times 2$; one graviton + one ghost–antighost propagator (not $V_{hcc\bar{c}}/V_{ccc}$ substitute). External $c\bar{c}$ — excluded until a row names ghost two-point.
- **D05a:** not included (no named *ch* row requires it).

P_{in} is *not* enlarged: D01/D03/D02 are further graphs for the same hh one-loop coefficient.

181.7.3 Blocked for P.4

$\ell_4/V_{4,\mathrm{grav}}$ (blocks D02); s_2 (not invented); D12 evaluation (Missing); P.3.3 connecting map (Unresolved); concrete $\Gamma_1^{[1]}$ coefficients; one-loop antifield coeffs at κ^2.

181.7.4 Statuses

`projection_defined` and `dependency_inventory_complete` remain earned (p05). `classical_inputs_constructed` / `quantum_inputs_evaluated` / `projected_identity_verified` remain not earned.

181.8 P-next.3 — Classical prerequisites

Scope. Keep P-next.1/P-next.2 intact. Construct only classical ingredients named by the *ch*-row / graph inventory. No P.5, no `symmetry_restored=true`, no invented s_2.

181.8.1 P.4.1 Conventions

Metric $q = \eta + \kappa h$ (signature $+---$); $S_{\mathrm{EH}} = -(2/\kappa^2)\int \sqrt{-q}\, R$. Coupling-stripped $S = S_0 + \kappa S_1 + \kappa^2 S_2 + \cdots$. IBP: compact support; discard total derivatives. Antibracket with left/right derivatives and odd-field signs as in XVI-N. Ghost $a = \frac{1}{2}$ (not D11 (α, β)). Minimal/nonminimal fields recorded; P.3.3 connecting map remains Unresolved. Labels: *absent* / *vanishing* / *unconstructed* kept distinct. Status: **Constructed**.

181.8.2 Exported vertices

Vertex	Status	Note
$V_{4,\mathrm{sg}}$ $(hh\phi\phi)$	Constructed	Closed a_2, b_2 from $\mathrm{Lag}_q\|_{m=0}$; export `exports/V4_scalar_grav.json`. D12 evaluation still Missing.
$V_{4,\mathrm{grav}}$ $(hhhh)$	Partial	Algorithm $\ell_4 = -2[\varepsilon^4](\sqrt{-q}R)$ + IBP policy + conformal probe; full 10-component polynomial **Unresolved**. D02 still blocked.

181.8.3 ℓ_4 indexed generator

Steps: form $q = \eta + \varepsilon h$; invert; build Γ, R; expand $\sqrt{-q}\, R$; set $\ell_4 = -2[\varepsilon^4]$. Sparse/conformal probes supplement and do not replace the full polynomial. Exact blocker: no complete indexed generating polynomial in the ten $h_{\mu\nu}$ components exported.

181.8.4 P.4.3 BRST / antifield order

Through $O(\kappa)$: Constructed inheritance ($s = s_0 + \kappa s_1$). At κ^2: $V_{\mathrm{ghost},\kappa^2}$ is **absent** under inherited s through $O(\kappa)$ with linear D (Psi sector). The s_2 slot is **not invented**. Complete antifield/ghost $O(\kappa^2)$ on $D[h]$ remains Partial. Nonminimal + gauge-fixing recorded; connecting map Unresolved.

181.8.5 P.4.4 Master check (lite)

$(S_0, S_0) = 0$: Constructed. $(S_0, S_1) = 0$: Constructed inheritance. $2(S_0, S_2) + (S_1, S_1) = 0$: **Partial** (scalar Constructed via Gate 1; grav blocked by ℓ_4; ghost/antifield Partial). Not claimed complete.

181.8.6 P.4.5 Dependency rerun (lite)

D02 availability: still_blocked_by_ell4. D12: vertex Constructed; evaluation Missing (not defaulted to zero).

181.8.7 Statuses

`projection_defined` / `dependency_inventory_complete`: earned (preserved). `classical_inputs_constructed`: **not earned** (assessment Partial; missing: full ℓ_4, complete antifield κ^2 on $D[h]$, P.3.3 map). `quantum_inputs_evaluated` / `projected_identity_verified`: not earned.

181.9 P-next.4 — Full ℓ_4 and classical master

Scope. Construct the full indexed gravitational $\ell_4/V_{4,\mathrm{grav}}$ polynomial required by D02; verify the retained classical master on the declared domain. No P.5, no invented s_2, no `symmetry_restored`.

181.9.1 Full indexed ℓ_4

Definition: $\ell_4 = -2[\varepsilon^4]L_\Gamma$ with $L_\Gamma = \sqrt{-q}\, q^{\mu\nu}(\Gamma^\alpha{}_{\mu\beta}\Gamma^\beta{}_{\nu\alpha} - \Gamma^\alpha{}_{\mu\nu}\Gamma^\beta{}_{\alpha\beta})$, equal to $\sqrt{-q}R$ modulo a total derivative (IBP policy: compact support; discard d(stuff)). Metric $q = \eta + \varepsilon H$, $\eta = \mathrm{diag}(+---)$, $S_{\mathrm{EH}} = -(2/\kappa^2)\int \sqrt{-q}\, R$.

Generating polynomial: all ten independent $H_{\mu\nu}$ plus first-derivative jets $\partial_\rho H_{\mu\nu}$ (order-by-order series with correct inverse-metric controls). Status: **Constructed** (3360 terms). Exports: `exports/V4_grav.json`, `exports/ell4_generating_polynomial.srepr`. Sparse $\sqrt{-q}R$ probes remain supplemental. Cubic $[\varepsilon^3]$ recorded for XII-C ℓ_3 regression without claiming XII-C is the quartic.

181.9.2 D02 availability

D02 tadpole vertex unblocked (`unblocked_vertex_Constructed`). Pairing template: `exports/D02_pairing_template.json`. Quantum tadpole evaluation not performed.

181.9.3 Master table

Equation	Status	Note
$(S_0, S_0) = 0$	Constructed	free nilpotence
$(S_0, S_1) = 0$	Constructed inheritance	through $O(\kappa)$
$2(S_0, S_2) + (S_1, S_1) = 0$	grav+scalar Constructed; ghost/af Partial	grav uses Constructed ℓ_4 + EH structural invariance; scalar Gate 1; no s_2

181.9.4 Statuses

`classical_inputs_constructed`: **earned** (named classical vertices/action). `quantum_inputs_evaluated` / `projected_identity_verified`: not earned. P.3.3 connecting map: Unresolved. D12 evaluation: Missing.

181.10 p09 — Evidence-and-closure release

Scope. Earn classical readiness for the fixed *ch* identity **only** as the conjunction

`quartic_generator_verified` ∧ `ch_projection_defined` ∧ `ch_dependency_inventory_complete` ∧ `gauge_fixing_map_constructed` ∧ `required_classical_BV_components_verified`.

Do *not* inherit p08's `classical_inputs_constructed`. Separate `execution_pass` / `claim_status` / `scope`. No quantum eval; no symmetry restoration; O.1 stays `NOT_CLOSED`; insertion freeze; `D12_quantum_evaluated=false`.

181.10.1 Quartic verification

Builders: `code/classical/ell4_generator.py`, `ell4_verify.py`. Acceptance: full 10-component $q\, q^{-1} = \delta + O(\varepsilon^{N+1})$; independent det vs trace-log; $\sqrt{-q}R - L_{\Gamma\Gamma} = \partial B$ under actual curvature (divergence discarded *only* for integrated action under declared BC); negative mutations reject (wrong inverse sign, off-diag multiplicity, missing det coeff, swapped contractions, wrong overall norm). 3360 is a reproducibility count, not acceptance. V_4 polarization: 1/4!, i, κ factors recorded. Earn: `quartic_generator_verified`; `D02_vertex_available=true`; `D02_quantum_evaluated=false`.

181.10.2 Source-dependent *ch* audit

Exact output: one c, one h, zero $\phi/\bar{c}/b$/antifield; gh = 1; κ^2; 1-loop; off-shell. Defined by $\delta/\delta c\, \delta/\delta h$ at zero remaining — not $W[h, \ldots]$. Both summands expanded; all field/antifield pairs tabulated; "names TBD" must vanish. Unknown numerical values OK. Separate κ one-point vs κ^2 D12 hh tadpole. Graphs for every surviving coefficient. AMP_S1 placeholder until derivative produces it. Earn inventory complete with `unknown_coefficient_patterns=[]` and explicit `unevaluated_named_coefficients`.

181.10.3 Gauge-fixing map

Invariant extended action (nonminimal) vs transformed gauge-fixed with external antifields retained. From $\Psi = -i \int \bar{c}_\mu (2D^\mu[h] - b^\mu)$: field maps, antifield shifts, inverse, L/R signs vs XVI-N bracket — shifts *derived*, not imported. Verify: antibracket preservation; transformed source couplings; GF density; ghost kinetic; interacting ghost verts; Hessians = stated propagators. Transformed EA coeffs related to invariant identity — not identical. Rerun *ch* audit in resulting variables. Earn: `gauge_fixing_map_constructed`; `propagators_match_transformed_Hessians`; `source_identity_relation_defined`.

181.10.4 Classical BV

Expand $s = s_0 + \kappa s_1 + \kappa^2 s_2 + \cdots$ from frozen variables; *determine* whether s_2 present/vanishes (derived $s_2 = 0 \neq$ policy "no invent"). Build source/antifield action with ordering/signs; include ghost/nonminimal for Ψ. Check $2(S_0, S_2) + (S_1, S_1)$ by antifield number (0 = Noether; > 0 = algebra; nonminimal); report cross-term residuals. Structural OK only with explicit ID to exported coeffs. Earn `required_classical_BV_components_verified` and `required_master_residuals=[]` only if complete; else Partial. Ghost/af master Partial ⇒ classical readiness Partial/not earned.

181.10.5 Package

Archive p02–p07 listings/certs to `archival/`. Build order: conventions→quartic→transforms/source→gauge map→ *ch* audit→graphs→master→certs→tests→PDFs→external manifest LAST. Mutation tests invalidate acceptance. Sync `/workspace/XVI-P-*.pdf`; XVI-M pin unchanged; XVI-O untouched. **p08 scope.** P-next.4: full indexed $\ell_4/V_{4,\mathrm{grav}}$ Constructed; classical master on declared domain; D02 unblocked; cis earned. No P.5.

p07 scope. P-next.3: classical prerequisites named by inventory (conventions, $V_{4,\mathrm{sg}}$, ℓ_4 algorithm, BRST order, master lite). P-next.1/2 intact. No P.5; no invented s_2.

p06 scope. P-next.2: P.3.3 interface (map Unresolved); P.3.4 graphs for named *ch*-row coefficients only. P-next.1 intact. No s_2, no ℓ_4 Constructed, no P.5.

181.11 Scope, baseline, and four verdicts

Baseline. Invariant action S_{inv} (not S_D). Classical antibracket of XIV-A / XVI-N. Ghost normalization $a = \frac{1}{2}$ (not the D11 pair (α, β)). Coupling-stripped expansion $S = S_0 + \kappa S_1 + \kappa^2 S_2$ with no extra κ^j inside coefficients. En01 / Em01 / Em02-B are not premises. XV-B is not opened. XVI-M graph–action non-identification retained.

Verdict	Status	Note
P1 D11 distributional kernel	Constructed	Freeze of o70-sync conjunction
P2 D11 linearized functional identity	Constructed	$s_0(\Gamma^{(2)}_{1,w} + C_L) = 0$
P3 retained identity domain	provisional	identity-driven map; domain_provisional; not closed
P4 coupled one-loop restoration	not established	blocked by Partial master / empty matched set
general renormalization	not claimed	—
insertion completion	not claimed	XVI-N freeze unchanged

Bulk `symmetry_restored=false`. O.1 `NOT_CLOSED`. Insertion freeze unchanged. XVI-M pin unchanged. P.3–P.5 are out of scope for this release.